Ali Baba and the Forty Thieves

Reverse engineered from the Apple II version

Xekri Redmane

Contents

1 Ali Baba and the Forty Thieves **2**

1.1 Introduction 2

1.2 About this document 3

2 6502 programming techniques **5**

2.1 Zero page temporaries 5

2.2 Tail calls 5

2.3 Unconditional branches 5

2.4 Stretchy branches 6

2.5 Shared code 6

2.6 Macros 6

2.6.1 STOW, STOW2 7

2.6.2 MOVB, MOVW, STOB 8

2.6.3 PSHW, PULB, PULW 9

2.6.4 INCW 10

2.6.5 ADDA, ADDAC, ADDB, ADDB2, ADDW, ADDWC . . 11

2.6.6 SUBB, SUBB2, SUBW 14

2.6.7 ROLW, RORW 16

2.6.8 ROL2 16

2.6.9 LDWIY 17
2.6.10 Comparison macros 18
2.7 The APSTR postprocessor 22

3 Game Overview **23**

4 Game Data **25**
4.1 Character and location records 25
4.1.1 Character record pointer 25
4.2 Event records 31
4.2.1 Plot character 49
4.3 Undocumented data regions 50
4.3.1 Runtime data area 50
4.3.2 Small gap fills 51
4.3.3 Equipment and combat data 62
4.3.4 Duplicate item/combat strings 64
4.3.5 HRCG data and border bitmaps 66
4.4 Mob data records 68
4.5 Mob name strings 197

5 The Main Loop **205**

6 The Game Turn **228**

7 Input System **247**
7.0.1 Menu handler routines 265
7.0.2 Save/load slot routines 275
7.0.3 Save game slot menu 278

7.0.4 Scenario disk menu 280
7.0.5 Character selection menu 281
7.0.6 Alternative input table 285
7.1 Save/Load Disk I/O 287
7.1.1 Disk I/O data 287
7.1.2 Continue game handler 287
7.1.3 Shuffle entities 290
7.1.4 Save game handler 293
7.1.5 Scenario disk handlers 295
7.1.6 Game disk verification 297
7.1.7 Disk format and signature 298
7.1.8 IOB setup and game disk check 300
7.1.9 Write game data 302
7.1.10 Read game data and IOB helpers 304
7.1.11 Menu strings 310

8 Combat **312**

8.1 Encounter detection 312
8.2 Combat resolution 324
8.2.1 Attack direction handlers 329
8.2.2 Find attack target 330
8.2.3 Init attack target display 331
8.2.4 Someone else handler and attack eligibility 333
8.2.5 Combat willingness and attack resolution 336
8.2.6 Player attack resolution 341
8.2.7 Mob attack resolution 345

8.2.8 Damage application 348
8.2.9 Mob movement encounter 356
8.3 Target selection 365
8.3.1 Entity threat evaluation 365
8.4 Combat utilities 374

9 NPC AI **383**
9.1 Shops 401
9.1.1 Shop buy effects 409
9.1.2 XP award 411

10 Events and Scripting **415**
10.1 Between-round random events 415
10.2 Character event processing 424
10.3 Scene event dispatch 427
10.3.1 Flush pending XP 433
10.3.2 Drop gold system 434
10.3.3 Arithmetic helpers 442
10.4 Turn events 444
10.4.1 Scene event trigger 444
10.4.2 Treasure pickup 446
10.4.3 Increment strength / level 450
10.4.4 Clear treasure digit fields 451
10.4.5 Clear and display message 452
10.4.6 Event list update 453
10.4.7 Leave-area check 456

11 Entities **458**

11.1 Entity classification 458

11.2 Entity rendering 464

11.2.1 Render font character 465

11.2.2 Font position to text position 467

11.2.3 Find entity at position 468

11.2.4 Draw character at position 471

11.2.5 Draw blink alternate 472

11.2.6 Draw blink character 472

11.2.7 Set blink target 474

11.3 Entity management 476

12 Map and Scene **491**

12.0.1 Scene setup 491

12.0.2 Scene loop 496

12.0.3 Load scene data 529

12.0.4 Load map tiles 530

12.0.5 Setup scene display 531

12.0.6 Enter location 533

12.0.7 Check at boundary 536

12.0.8 Fill map 538

12.0.9 Clear map 539

12.0.10 Draw map column 540

12.0.11 Draw map entities 541

12.0.12 Draw map icon 544

12.0.13 RWTS command dispatch 546

12.1 Scene data encoding 547

12.2 Scene $00: Copyright notice 547

13 Mob Data **549**

14 Game Display **550**

14.1 Character stats display 553

15 Initialization **560**

15.0.1 Zeroed hi-res page 567

15.0.2 Border animation handler 583

16 Utility Routines **593**

16.1 Random number generation 593

16.2 Coordinate conversion 599

16.3 Timing and animation 603

16.4 Field utilities 608

16.4.1 Character selection browsing 610

17 Text and Display Primitives **621**

17.1 Game message strings 640

18 Sound **646**

18.1 Tune player 650

18.2 Game support routines 662

18.2.1 Tune stubs and keyboard dispatch 662

18.2.2 Read keyboard 663

18.2.3 Load scene data from disk 665

18.2.4 Tune player dispatch 669

18.2.5 Integrity verification 670

18.2.6 Cold restart error handler 674

18.2.7 Pointer advance helpers 675

19 The Attract Screen **685**

19.0.1 Sound and settings routines 685

20 Resident Routines **689**

21 The HRCG Text Engine **714**

22 Disk I/O **769**

23 The boot process **794**

23.1 The booting of a disk 795

23.1.1 Sector interleaving 795

23.2 Extracting the sections for analysis 796

24 BOOT1 **798**

25 EALDR: The Electronic Arts Loader **819**

25.1 Unused copy-protection VM ($A000–$A2FF) 819

25.1.1 Resident routines copy ($A300–$A5FF) 831

25.2 Entry point 832

25.2.1 Opcode table 835

25.3 VM architecture 844

25.3.1 Opcode dispatch 844

25.3.2 Bytecode encryption 845

25.3.3 VM interpreter 845

25.3.4 Dispatch table 845

25.3.5 Opcode handlers 848

25.3.6 Machine language subroutines 853

25.3.7 DXR decryption step 855

25.3.8 Copy protection (unused) 856
25.4 EA splash screen 860
25.4.1 Checksum source listing 870
25.5 Disk I/O routines 872

26 Startup **886**
26.1 Memory map at entry 886
26.1.1 Group 1 (conditional, skipped in this image) 886
26.1.2 Group 2: game data 886
26.1.3 Group 3: main program 887
26.1.4 DXR decryption 887
26.1.5 Final copy 887
26.1.6 Summary 887
26.2 Runtime memory map 888
26.3 Extracting `main.bin` 889

27 Dead Code **892**
27.1 Residual development string 892
27.2 Compressed cursor routine 893
27.3 Leftover DOS code 894
27.4 Residual assembler source text 895
27.5 Orphaned encounter handler 898
27.6 Apple 6502 Assembler/Editor residual 901

28 Apple II Hi-Res Graphics **906**

29 Assembly output **909**

30 Index **923**

Chapter 1

Ali Baba and the Forty Thieves

1.1 Introduction

Ali Baba and the Forty Thieves (Wikipedia link) is a graphical computer role-playing game (`CRPG`) written by Stuart Smith in 1981 and released in 1982

for the Apple II.

It was published by Quality Software. The game is a "hot-seat" multiplayer game, where players may take turns at the keyboard to control their character. The game can also be played single-player, with the player controlling all characters.

The goal is to rescue Princess Buddir and bring her back to Ali Baba's home. Forty thieves wander the dungeon, and there are other fixed enemies and wandering friendly `NPC`s who can join your party.

There is a good overview of the gameplay at CRPG Addict.

The purpose of this document is to reverse engineer the game. The disk image used is from the Internet Archive:

- Ali Baba and the Forty Thieves (4am and san inc crack)

Although originally published by Quality Software, the disk image shows that it was published by Electronic Arts. In 1986, Electronic Arts published Age of Adventure, a compilation of Ali Baba and Stuart Smith's The Return of Heracles (1983). This image is almost certainly from this compilation: 4am also cracked The Return of Heracles on the same day.

1.2 About this document

All files can be found on Github.

The source for this document, `main.nw`, is a literate programming document. This means the explanatory text is interspersed with source code. The assembly code and `LaTeX` file can be extracted from the document and compiled.

The goal is to provide all the source code necessary to reproduce a binary identical to the images extracted from the disk.

The code was reverse-engineered using Ghidra.

The assembly code can then be assembled with `dasm` using this command line:

```
dasm main.asm -omain.bin -f3 -v4
```

The document is written in LaTeX.

This document doesn't explain every last detail. It's assumed that the reader can find enough details on the 6502 processor and the Apple II series of computers to fill in the gaps.

Useful resources:

- 6502 Assembly Language Programming, by Lance A. Leventhal, 1979.
- Apple II Monitors Peeled, Apple, 1981.
- Beneath Apple DOS, by Don Worth and Pieter Lechner, 1982.
- Understanding the Apple II, by Jim Sather, 1983.
- Apple II Computer Graphics, by Ken Williams, Bob Kernaghan, and Lisa Kernaghan, 1983.
- Beagle Bros Apple Colors and ASCII Values, Beagle Bros Micro Software Inc, 1984.
- Hi-Res Graphics and Animation Using Assembly Language, The Guide for Apple II Programmers, by Leanard I. Malkin, 1985.
- Apple IIe Technical Reference Manual, Apple, 1985.
- Apple II ROM disassemblies, collected by Andy McFadden, 2019.
- I/O Memory, by Jon Relay.
- 6502 Instruction Set, by Norbert Landsteiner.

Chapter 2

6502 programming techniques

2.1 Zero page temporaries

Zero-page consists essentially of global variables. Sometimes we need local temporaries, and Apple II programs mostly don't use the stack for those. Rather, some "global" variables are reserved for temporaries. You might see multiple symbols equated to a single zero-page location. The names of such symbols are used to make sense within their context.

2.2 Tail calls

Rather than a `JSR` immediately followed by an `RTS`, instead a `JMP` can be used to save stack space, code space, and time. This is known as a tail call, because it is a call that happens at the tail of a function.

2.3 Unconditional branches

The 6502 doesn't have an unconditional short jump. However, if you can find a condition that is always true, this can serve as an unconditional short jump, which saves space and time.

2.4 Stretchy branches

6502 branches have a limit to how far they can jump. If they really need to jump farther than that, you have to put a `JMP` or an unconditional branch within reach.

2.5 Shared code

To save space, sometimes code at the end of one function is also useful to the next function, as long as it is within reach. This can save space, at the expense of functions being completely independent.

2.6 Macros

We use these macros to make our assembly language listings a little less verbose. However, a restriction is that you cannot use local labels as arguments to a macro, since macros implicitly create an anonymous local scope for themselves. Thus, they are not good candidates for use in self-modifying code.

Macro	Args	Description
Store / Move		
STOW	val, dst	Store 16-bit immediate to memory
STOW2	val, dst	Same as STOW, opposite instruction order
STOB	#val, dst	Store immediate byte to memory
MOVB	src, dst	Move byte between memory locations
MOVW	src, dst	Move 16-bit value between memory locations
LDWIY	ptr, #off, dst	Load 16-bit word via indirect Y-indexed pointer
Stack		
PSHW	addr	Push 16-bit value onto stack
PULB	addr	Pull byte from stack to memory
PULW	addr	Pull 16-bit value from stack to memory
Arithmetic		
INCW	addr	Increment 16-bit value in memory
ADDA	addr	Add A to 16-bit memory (with carry propagation)
ADDAC	addr	Add A + carry to 16-bit memory
ADDB	addr, val	Add 8-bit value to 16-bit memory
ADDB2	addr, val	Same as ADDB, swapped CLC/LDA order
ADDW	a, b, c	Add two 16-bit values, store to third
ADDWC	a, b, c	Same as ADDW but without CLC (uses existing carry)
SUBB	addr, val	Subtract 8-bit value from 16-bit memory
SUBB2	addr, val	Same as SUBB, swapped SEC/LDA order
SUBW	a, b, c	Subtract two 16-bit values, store to third
SUBWL	#val, b, c	Subtract 16-bit memory from 16-bit literal
Rotate		
ROLW	addr	Rotate 16-bit memory left
RORW	addr	Rotate 16-bit memory right
ROL2		Rotate accumulator left by two bits
Compare and branch		
BAEQ	val, lbl	Branch if A equals val
BANE	val, lbl	Branch if A does not equal val
BAPL	val, lbl	Branch if A minus val is positive
BAMI	val, lbl	Branch if A minus val is negative
BALT	val, lbl	Branch if A is less than val (unsigned)
BAGE	val, lbl	Branch if A is greater than or equal to val (unsigned)
BXEQ	val, lbl	Branch if X equals val
BXNE	val, lbl	Branch if X does not equal val
BYEQ	val, lbl	Branch if Y equals val
BYNE	val, lbl	Branch if Y does not equal val

2.6.1 STOW, STOW2

STOW stores a 16-bit literal value to a memory location.

For example, `STOW #$01FF, $0200` stores the 16-bit value `#$01FF` to memory location `$0200` (of course in little-endian order).

8a ⟨*Macros* 8a⟩≡ (909 921 922) 8b ▷

```
MACRO STOW
    LDA     #<{1}
    STA     {2}
    LDA     #>{1}
    STA     {2}+1
ENDM
```

Defines:
`STOW`, used in chunks 26, 219, 338, 375, 404, 419, 447, 459, 466, 527, 535, 550, 556, 570b, 592, 632, 681, 736, 801, and 853.

`STOW2` does the same, but in the opposite order, still retaining little-endianness.

8b ⟨*Macros* 8a⟩+≡ (909 921 922) ◁ 8a 8c ▷

```
MACRO STOW2
    LDA     #>{1}
    STA     {2}+1
    LDA     #<{1}
    STA     {2}
ENDM
```

Defines:
`STOW2`, used in chunks 259 and 537.

2.6.2 MOVB, MOVW, STOB

MOVB moves a byte from one memory location to another, while STOB stores a literal byte to a memory location. The implementation is identical, and the only difference is documentation.

For example, MOVB $01, $0200 moves the byte at memory location $01 to memory location $0200, while STOB #$01, $0200 stores the byte #$01 to memory location $0200.

8c ⟨*Macros* 8a⟩+≡ (909 921 922) ◁8b 9a▷

```
MACRO MOVB
    LDA     {1}
    STA     {2}
ENDM
MACRO STOB
    LDA     {1}
    STA     {2}
ENDM
```

Defines:
MOVB, used in chunks 225, 239, 264, 312, 322, 349, 354, 357, 360, 368, 370a, 385, 398, 404, 419, 420, 424, 462, 472a, 473b, 475, 487, 489a, 535, 538, 542, 545, 644, 738, 747a, 749, 751, 754, 757, 763, 764, 766, 809, 877, and 898.
STOB, used in chunks 49, 219, 220, 222, 232, 239, 241, 308, 312, 320, 338, 346, 349, 354, 357, 368, 370a, 377, 378, 385, 390, 398, 404, 419, 420, 422, 428–31, 452, 462, 473b, 487, 494, 527, 532c, 537, 538, 542, 545, 546, 550, 552, 564, 566, 568, 570b, 574, 576b, 590, 592, 617, 626–28, 632, 637, 639, 644, 653, 678, 679, 681, 738, 745, 747–49, 751, 753, 754, 756, 757, 759, 763, 801, 816, 833, 854b, 856, 883, and 898.

MOVW moves a 16-bit value from one memory location to the another.

For example, MOVW $01FF, $A000 moves the 16-bit value at memory location $01FF to memory location $A000.

9a ⟨*Macros* 8a⟩+≡ (909 921 922) ◁8c 9b▷

```
MACRO MOVW
    LDA     {1}
    STA     {2}
    LDA     {1}+1
    STA     {2}+1
ENDM
```

Defines:
MOVW, used in chunks 220, 222, 312, 315, 320, 342, 349, 352, 357, 372a, 382, 385, 420, 422, 426, 430, 432, 444, 447, 475, 478, 482, 488, 489a, 494, 619c, 620, 637, 679, 681, 704, 787, 848a, and 898.

2.6.3 PSHW, PULB, PULW

PSHW is a macro that pushes a 16-bit value in memory onto the stack.

For example, PSHW $01FF pushes the 16-bit value at memory location $01FF onto the stack.

9b ⟨*Macros* 8a⟩+≡ (909 921 922) ◁9a 9c▷

```
MACRO PSHW
    LDA    {1}
    PHA
    LDA    {1}+1
    PHA
ENDM
```

Defines:
PSHW, used in chunks 232, 255, 473b, 644, 790b, and 849.

PULB is a macro that pulls an 8-bit value from the stack to memory.

For example, PULB $01FF pulls an 8-bit value from the stack and stores it at memory location $01FF.

9c ⟨*Macros* 8a⟩+≡ (909 921 922) ◁9b 10a▷

```
MACRO PULB
    PLA
    STA    {1}
ENDM
```

Defines:
PULB, used in chunks 472a, 702, 708, 764, 783, 784, 846, and 847b.

PULW is a macro that pulls a 16-bit value from the stack to memory.

For example, PULW $01FF pulls a 16-bit value from the stack and stores it at memory location $01FF.

10a ⟨*Macros* 8a⟩+≡ (909 921 922) ◁9c 10b▷

```
MACRO PULW
    PLA
    STA    {1}+1
    PLA
    STA    {1}
ENDM
```

Defines:
PULW, used in chunks 232, 255, 473b, 644, 791a, and 849.

2.6.4 INCW

INCW is a macro that increments a 16-bit little-endian value in memory.

For example, INCW $01FF increments the 16-bit value at memory location $01FF.

10b ⟨*Macros* 8a⟩+≡ (909 921 922) ◁10a 11a▷

```
    MACRO INCW
        INC    {1}
        BNE    .continue
        INC    {1}+1
.continue
    ENDM
```

Defines:
INCW, used in chunk 849.

2.6.5 ADDA, ADDAC, ADDB, ADDB2, ADDW, ADDWC

ADDA is a macro that adds the A register to a 16-bit little-endian memory location.

For example, ADDA $01FF adds the contents of the A register to the 16-bit value at memory location $01FF.

11a ⟨*Macros* 8a⟩+≡ (909 921 922) ◁10b 11b▷

```
    MACRO ADDA
        CLC
        ADC    {1}
        STA    {1}
        BCC    .continue
        INC    {1}+1
.continue
    ENDM
```

Defines:
ADDA, never used.

ADDAC is a macro that adds the A register, and whatever the carry flag is set to, to a 16-bit memory location.

11b ⟨*Macros* 8a⟩+≡ (909 921 922) ◁11a 12a▷

```
    MACRO ADDAC
        ADC    {1}
        STA    {1}
        BCC    .continue
        INC    {1}+1
.continue
    ENDM
```

Defines:
ADDAC, never used.

ADDB is a macro that adds an 8-bit immediate value, or the 8-bit contents of memory, to a 16-bit memory location.

For example, ADDB $01FF, #$01 adds the immediate value #$01 to the 16-bit value at memory location $01FF, while ADDB $01FF, $0300 adds the 8-bit value at memory location $0300 to the 16-bit value at memory location $01FF.

12a ⟨*Macros* 8a⟩+≡ (909 921 922) ◁11b 12b▷

```
    MACRO ADDB
        LDA    {1}
        CLC
        ADC    {2}
        STA    {1}
        BCC    .continue
        INC    {1}+1
.continue
    ENDM
```

Defines:
ADDB, never used.

ADDB2 is the same as ADDB except that it swaps the initial CLC and LDA instructions.

12b ⟨*Macros* 8a⟩+≡ (909 921 922) ◁12a 13a▷

```
    MACRO ADDB2
        CLC
        LDA    {1}
        ADC    {2}
        STA    {1}
        BCC    .continue
        INC    {1}+1
.continue
    ENDM
```

Defines:
ADDB2, never used.

ADDW is a macro that adds two 16-bit values in memory and stores it to a third 16-bit memory location.

For example, ADDW $01FF, $0300, $0400 adds the 16-bit value at memory location $01FF to the 16-bit value at memory location $0300, and stores the result at memory location TEXT_PAGE1.

13a ⟨*Macros* 8a⟩+≡ (909 921 922) ◁12b 13b▷

```
MACRO ADDW
    CLC
    LDA     {1}
    ADC     {2}
    STA     {3}
    LDA     {1}+1
    ADC     {2}+1
    STA     {3}+1
ENDM
```

Defines:
ADDW, never used.

ADDWC is a macro that adds two 16-bit values in memory, plus the carry bit, and stores it to a third 16-bit memory location.

13b ⟨*Macros* 8a⟩+≡ (909 921 922) ◁13a 14a▷

```
MACRO ADDWC
    LDA     {1}
    ADC     {2}
    STA     {3}
    LDA     {1}+1
    ADC     {2}+1
    STA     {3}+1
ENDM
```

Defines:
ADDWC, never used.

2.6.6 SUBB, SUBB2, SUBW

SUBB is a macro that subtracts an 8-bit value from a 16-bit memory location. This is the same as SUBB in the original Infocom source code. The immediate value is the second argument.

For example, SUBB $01FF, #$01 subtracts the immediate value #$01 from the 16-bit value at memory location $01FF and stores it back.

14a ⟨*Macros* 8a⟩+≡ (909 921 922) ◁13b 14b▷

```
    MACRO SUBB
        LDA     {1}
        SEC
        SBC     {2}
        STA     {1}
        BCS     .continue
        DEC     {1}+1
.continue
    ENDM
```

Defines:
SUBB, never used.

SUBB2 is the same as SUBB except that it swaps the initial SEC and LDA instructions.

14b ⟨*Macros* 8a⟩+≡ (909 921 922) ◁14a 15a▷

```
    MACRO SUBB2
        SEC
        LDA     {1}
        SBC     {2}
        STA     {1}
        BCS     .continue
        DEC     {1}+1
.continue
    ENDM
```

Defines:
SUBB2, never used.

SUBW is a macro that subtracts the 16-bit memory value in the second argument from a 16-bit memory location in the first argument, and stores it in the 16-bit memory location in the third argument.

For example, SUBW $01FF, $0300, $0400 subtracts the 16-bit value at memory location $0300 from the 16-bit value at memory location $01FF, and stores the result at memory location TEXT_PAGE1.

15a ⟨*Macros* 8a⟩+≡ (909 921 922) ◁14b 15b▷

```
MACRO SUBW
    SEC
    LDA     {1}
    SBC     {2}
    STA     {3}
    LDA     {1}+1
    SBC     {2}+1
    STA     {3}+1
ENDM
```

Defines:
SUBW, used in chunk 632.

SUBWL is a macro that subtracts the 16-bit memory value in the second argument from the 16-bit literal in the first argument, and stores it in the 16-bit memory location in the third argument.

For example, SUBWL #$01FF, $0300, $0400 subtracts the 16-bit value at memory location $0300 from the 16-bit value #$01FF, and stores the result at memory location TEXT_PAGE1.

15b ⟨*Macros* 8a⟩+≡ (909 921 922) ◁15a 16a▷

```
MACRO SUBWL
    SEC
    LDA     <{1}
    SBC     {2}
    STA     {3}
    LDA     >{1}
    SBC     {2}+1
    STA     {3}+1
ENDM
```

Defines:
SUBWL, never used.

2.6.7 ROLW, RORW

ROLW rotates a 16-bit memory location left.

16a ⟨*Macros* 8a⟩+≡ (909 921 922) ◁15b 16b▷

```
MACRO ROLW
    ROL     {1}
    ROL     {1}+1
ENDM
```

Defines:
ROLW, never used.

RORW rotates a 16-bit memory location right.

16b ⟨*Macros* 8a⟩+≡ (909 921 922) ◁16a 16c▷

```
MACRO RORW
    ROR     {1}+1
    ROR     {1}
ENDM
```

Defines:
RORW, never used.

2.6.8 ROL2

ROL2 rotates the accumulator left by two bit positions.

16c ⟨*Macros* 8a⟩+≡ (909 921 922) ◁16b 17▷

```
MACRO ROL2
    ROL
    ROL
ENDM
```

Defines:
ROL2, used in chunks 360, 380, 390, 404, 531a, and 602.

2.6.9 LDWIY

LDWIY loads a 16-bit word from an indirect pointer indexed by Y into a destination address. The Y register is set to the offset, the low byte is loaded and stored, Y is incremented, and the high byte is loaded and stored.

For example, LDWIY CHAR_PTR,#$00,PRINT_STRING_ADDR loads bytes 0 and 1 from the record pointed to by CHAR_PTR into PRINT_STRING_ADDR/PRINT_STRING_ADDR+1.

17 ⟨*Macros* 8a⟩+≡ (909 921 922) ◁16c 18▷

```
MACRO LDWIY
    LDY    {2}
    LDA    ({1}),Y
    STA    {3}
    INY
    LDA    ({1}),Y
    STA    {3}+1
ENDM
```

Defines:
LDWIY, used in chunks 247, 253, 257, 260, 349, 398, 424, 447, 454, 483–86, and 539.

2.6.10 Comparison macros

Often there is a compare instruction (CMP, CPX, CPY) followed by a branch. These macros combine the two instructions as a one-liner.

18 ⟨*Macros* 8a⟩+≡ (909 921 922) ◁17 19▷

```
MACRO BAEQ
    CMP     {1}
    BEQ     {2}
ENDM

MACRO BANE
    CMP     {1}
    BNE     {2}
ENDM

MACRO BAPL
    CMP     {1}
    BPL     {2}
ENDM

MACRO BAMI
    CMP     {1}
    BMI     {2}
ENDM

MACRO BALT
    CMP     {1}
    BCC     {2}
ENDM

MACRO BAGE
    CMP     {1}
    BCS     {2}
ENDM

MACRO BXEQ
    CPX     {1}
    BEQ     {2}
ENDM

MACRO BXNE
    CPX     {1}
    BNE     {2}
ENDM

MACRO BYEQ
    CPY     {1}
    BEQ     {2}
ENDM
```

```
MACRO BYNE
    CPY     {1}
    BNE     {2}
ENDM
```

Defines:
BAEQ,, never used.
BAGE,, never used.
BALT,, never used.
BAMI,, never used.
BANE,, never used.
BAPL,, never used.
BXEQ,, never used.
BXNE,, never used.
BYEQ,, never used.
BYNE, used in chunks 259, 317, 469, 637, and 753.

Event list macros. Each location has a variable-length event list of 3-byte entries (see §4.1.1). Each event is triggered or placed at a particular map location. The following macros encode those entries, replacing raw HEX triples with readable forms. Map positions are given as separate row and column arguments; the macro packs them as $20 \times row + col$.

EVSTOP marks the end of the list. EVDATA encodes an empty slot with two raw data bytes.

19 ⟨*Macros* 8a⟩+≡ (909 921 922) ◁18 20a▷

```
; --- Event list end marker ---
MACRO EVSTOP
    HEX FF FF FF
ENDM

; --- Empty slot with raw data bytes ---
MACRO EVDATA          ; {1}=byte1, {2}=byte2
    DC.B $FE, {1}, {2}
ENDM
```

Uses EVDATA 22 and EVSTOP 22.

EVRAND encodes a random encounter ($00–$3F). The avoidance check is always the character's strength (bits 4–0 of field 5) out of 20. The id argument (0–$3F) is a *deferred encounter ID*: if the character avoids the encounter, HANDLE_ENCOUNTER sets bit 6 of byte 1 (ORA #$40), mutating the event in-place into a specific encounter with that ID. On the character's next visit, the event fires as a specific encounter rather than a random one. The spawn position (sr, sc) is where the mob appears when the encounter eventually triggers.

EVENCO encodes a specific encounter ($40–$BF). The mob index (0–$7F) is offset by $40 automatically.

20a ⟨*Macros* 8a⟩+≡ (909 921 922) ◁19 20b▷

```
; --- Random encounter: id=$00-$3F (deferred encounter ID), spawn at (sr,sc) ---
MACRO EVRAND           ; {1}=row, {2}=col, {3}=id, {4}=spawn row, {5}=spawn col
    DC.B 20*{1}+{2}, {3}, 20*{4}+{5}
ENDM

; --- Specific encounter: mob index 0-$7F, spawn at (sr,sc) ---
MACRO EVENCO           ; {1}=row, {2}=col, {3}=mob, {4}=spawn row, {5}=spawn col
    DC.B 20*{1}+{2}, {3}+$40, 20*{4}+{5}
ENDM
```

Uses EVENCO 22 and EVRAND 22.

EVGOLD encodes dropped treasure ($C0–$D1). The gold amount (0–4607) is split across bytes 1 and 2: byte 1 = (gold ≫ 8) + $C0, byte 2 = gold & $FF.

20b ⟨*Macros* 8a⟩+≡ (909 921 922) ◁20a 21▷

```
; --- Dropped treasure: gold amount 0-4607 ---
MACRO EVGOLD           ; {1}=row, {2}=col, {3}=gold
    DC.B 20*{1}+{2}, [[{3}]>>8]+$C0, [{3}]&$FF
ENDM
```

Uses EVGOLD 22.

EVPSCN encodes a persistent scene event ($D2)—the scene remains active after triggering. EV1SCN encodes a one-time scene ($D3)—the slot is cleared to $FE after the scene plays. EVISCN encodes a scene with item drops ($D4)—like EV1SCN but also spawns 1–4 bonus items.

21 ⟨*Macros* 8a⟩+≡ (909 921 922) ◁20b 22▷

```
; --- Persistent scene (stays active) ---
MACRO EVPSCN           ; {1}=row, {2}=col, {3}=scene ID
    DC.B 20*{1}+{2}, $D2, {3}
ENDM

; --- One-time scene (cleared after trigger) ---
MACRO EV1SCN           ; {1}=row, {2}=col, {3}=scene ID
    DC.B 20*{1}+{2}, $D3, {3}
ENDM

; --- Scene + item drops ---
MACRO EVISCN           ; {1}=row, {2}=col, {3}=scene ID
    DC.B 20*{1}+{2}, $D4, {3}
ENDM
```

Uses EV1SCN 22, EVISCN 22, and EVPSCN 22.

EVSHOP encodes a shop or map/area event ($D5–$DA). The type byte is passed directly ($D5 = weapons, $D6 = armor, etc.) and the mask is the 8-bit item availability bitmask.

EVSLOT places an interactable empty-slot object ($FE) at a map position. The data byte's top two bits select the interaction category (see HANDLE_EVENT_SLOT): $00 = trap/message, $40+ = treasure chest, $C0 = activated event.

22 ⟨*Macros* 8a⟩+≡ (909 921 922) ◁21

```
; --- Shop/map event: type=$D5-$DA, mask=item availability ---
MACRO EVSHOP           ; {1}=row, {2}=col, {3}=type, {4}=mask
    DC.B 20*{1}+{2}, {3}, {4}
ENDM

; --- Interactable empty slot at a map position ---
MACRO EVSLOT           ; {1}=row, {2}=col, {3}=data byte
    DC.B 20*{1}+{2}, $FE, {3}
ENDM
```

Defines:
EV1SCN, used in chunks 21 and 32.
EVDATA, used in chunks 19 and 32.
EVENCO, used in chunks 20a and 32.
EVGOLD, used in chunks 20b and 32.
EVISCN, used in chunks 21 and 32.
EVPSCN, used in chunks 21 and 32.
EVRAND, used in chunks 20a and 32.
EVSHOP, used in chunk 32.
EVSLOT, used in chunk 32.
EVSTOP, used in chunks 19 and 32.

2.7 The APSTR postprocessor

Ali Baba uses Pascal-formatted strings. This means that string data has one length byte, then that number of character bytes after. In addition, the strings are in "Apple ASCII", where the high bit of each character is set. Rather than just adding raw hex bytes to the assembly listing, I've opted to write this postprocessor into the weave program. It does a pass over the assembly source to find APSTR directives with strings and convert them to raw HEX data.

Chapter 3

Game Overview

Ali Baba and the Forty Thieves is a real-time role-playing game where the player controls a party of characters exploring interconnected locations. This chapter gives a high-level overview of how the game works, as context for the detailed assembly code in later chapters.

Locations and maps. The game world is divided into locations ("Ali Baba Home," "Dusty Road," "Sultan's Palace," etc.), each defined by a 9-byte record in `LOCATION_DATA`. Each location has a map, a bounding rectangle, an event list, and a linked list of mob groups.

The main loop. `MAIN_ENTRY` calls `GAME_INIT` once, then loops forever. Each pass iterates the character records: for each active character it calls `ENTER_LOCATION` (to redraw the map if the location changed), dispatches `CHECK_SCENE_EVENTS` for players 1 and 3, and calls `PROCESS_MOB_GROUPS` to run AI turns for all mob groups in the current location. After all characters are processed, `RANDOM_EVENTS` fires between-round events and the loop restarts.

Turns. Each mob group gets a turn via `GAME_TURN_LOOP`. Player-controlled characters read keyboard or joystick input; NPCs use the AI system to choose a target and direction. A turn consists of spending movement points to walk around the map, encountering events, picking up treasure, entering shops, or engaging in combat.

Combat. When a character enters a cell occupied by a hostile mob, combat begins. Strike probability is based on the attacker's dexterity, adjusted by

the victim's state (defending, resting, running) and co-location. Damage is computed from strength and weapon type.

Events. Each location has a variable-length event list of 3-byte entries. Events encode random encounters, specific encounters, treasure, scenes, shops, and interactable objects. Random encounters that are avoided mutate in-place into deferred specific encounters.

Scenes. Scenes are text sequences loaded from disk and rendered with the custom font. They display story text, hints, shop menus, and the attract-mode demo. `SCENE_LOOP` renders a scene until the player presses a key or a transition chain completes.

Entities. Characters and mobs share the same record structure, distinguished by bit 7 of the high pointer byte. Mob groups are linked lists walked by `PROCESS_MOB_GROUPS`; each group member has appearance, strength, HP, speed, and AI mode fields.

Chapter 4

Game Data

4.1 Character and location records

4.1.1 Character record pointer

GET_CHAR_RECORD computes a zero-page pointer to a character's 9-byte record in the character table, given a 0-based index in A. The character table begins at CHAR_TABLE_BASE ($4000), with records laid out sequentially at stride CHAR_RECORD_SIZE (9 bytes). Index 0 returns a pointer to the base/header record. The result is stored in CHAR_REC ($BE/$BF).

25a ⟨*zero page defines* 25a⟩≡ (909) 53c ▷

```
CHAR_REC                EQU     $BE     ; character record pointer from GET_CHAR_RECORD (2 bytes)
```

Defines:
CHAR_REC, used in chunks 26, 61a, 478, 488, and 494.
Uses GET_CHAR_RECORD 26.

25b ⟨*defines* 25b⟩≡ (909) 275a ▷

```
CHAR_TABLE_BASE         EQU     $4000   ; base address of character record array
CHAR_RECORD_SIZE        EQU     $09     ; bytes per character record
```

Defines:
CHAR_RECORD_SIZE, used in chunks 26 and 219.
CHAR_TABLE_BASE, used in chunk 26.

26 ⟨*get char record* 26⟩≡ (909)

```
    ORG     $1053
GET_CHAR_RECORD:
    SUBROUTINE GET_CHAR_RECORD
    ; Compute pointer to character record from index.
    ;
    ; Inputs:  A = 0-based character index
    ; Outputs: CHAR_REC/+1 = CHAR_TABLE_BASE + A * CHAR_RECORD_SIZE
    ; Clobbers: A, X

    TAX                                 ; X = index (loop counter)
    STOW    CHAR_TABLE_BASE,CHAR_REC    ; CHAR_REC = $4000 (base)
.loop:
    DEX
    BPL     .advance                    ; records to skip -> advance
    RTS                                 ; index exhausted: CHAR_REC ready
.advance:
    CLC                                 ; \
    LDA     CHAR_REC                    ;  | CHAR_REC += CHAR_RECORD_SIZE
    ADC     #CHAR_RECORD_SIZE           ;  | (9 bytes per record)
    STA     CHAR_REC                    ;  |
    LDA     CHAR_REC+1                  ;  |
    ADC     #$00                        ;  | propagate carry
    STA     CHAR_REC+1                  ; /
    BNE     .loop                       ; always taken: high byte >= $40
```

Defines:
GET_CHAR_RECORD, used in chunks 25a, 61a, 422, 423, 429, 478, 488, and 494.
Uses CHAR_REC 25a, CHAR_RECORD_SIZE 25b, CHAR_TABLE_BASE 25b, and STOW 8a.

Location data and character data share the same 9-byte record table at MOB_BASE_LO. GET_CHAR_RECORD indexes into this table (index × 9 + $4000). Record 0 at MOB_BASE_LO is a header; records 1–62 are the actual entries. LOCATION_DATA points to record 1 ($4009).

Each record is 9 bytes:

Offset	Name	Description
0–1	name_ptr	16-bit pointer to length-prefixed location name
2	mob_head	Head of mob linked list (character index, 0 = none)
3	min_pos	Top-left bounding box (row × 20 + col)
4	max_pos	Bottom-right bounding box (row × 20 + col)
5	map_id	Bits 0–6: map scene ID. Bit 7: visited flag (set at runtime)
6–7	event_ptr	16-bit pointer to event/object list (3-byte entries)
8	mob_count	Number of mobs (summed into TOTAL_MOB_COUNT)

The location name strings follow the data table. Each is a length-prefixed Apple II string (high-bit ASCII) created by the APSTR pseudo-op.

27 ⟨*Location names* 27⟩≡ (909)

```
    ORG     $4240
LOCATION_NAMES:
s_ALI_BABA_HOME:
    APSTR "ALI BABA HOME"
s_DUSTY_ROAD:
    APSTR "DUSTY ROAD"
s_SULTANS_PALACE:
    APSTR "SULTAN'S PALACE"
s_ROYAL_LIBRARY:
    APSTR "ROYAL LIBRARY"
s_ASTROLOGERS_LAB:
    APSTR "ASTROLOGER'S LAB"
s_BEAR_CAVE:
    APSTR "BEAR CAVE"
s_RATS_NEST:
    APSTR "RATS NEST"
s_DRAGON_LAIR:
    APSTR "DRAGON LAIR"
s_GOLD_FOR_THE_LUCKY:
    APSTR "GOLD FOR THE LUCKY"
s_SAVE_THE_WHALES:
    APSTR "SAVE THE WHALES"
s_SLIDE:
    APSTR "SLIDE"
s_DEEP_DARK_STAIRWAY:
    APSTR "DEEP DARK STAIRWAY"
s_MINOTAUR_LABYRINTH:
```

```
    APSTR "MINOTAUR LABYRINTH"
s_STINGER_STREWN_CELL:
    APSTR "STINGER STREWN CELL"
s_GUARD_ROOM:
    APSTR "GUARD ROOM"
s_LONG_WAY_FROM_HOME:
    APSTR "LONG WAY FROM HOME"
s_FOREST:
    APSTR "FOREST"
s_VIEW_FROM_A_TREE:
    APSTR "VIEW FROM A TREE"
s_FAILING_WALL:
    APSTR "FAILING WALL"
s_MINOTAUR_PLAYROOM:
    APSTR "MINOTAUR PLAYROOM"
s_THIEVES_CAVE:
    APSTR "THIEVES CAVE"
s_O:
    APSTR "O"
s_SPIRAL_STAIRCASE:
    APSTR "SPIRAL STAIRCASE"
s_Y:
    APSTR "Y"
s_MAGIC_TOURBILLION:
    APSTR "MAGIC TOURBILLION"
s_G:
    APSTR "G."
s_CRYSTAL_CAVE:
    APSTR "CRYSTAL CAVE"
s_B:
    APSTR "B"
s_THIEVES_CLEARING:
    APSTR "THIEVES CLEARING"
s_OLD_MINE_ROAD:
    APSTR "OLD MINE ROAD"
s_FOOTHILLS_CEMETERY:
    APSTR "FOOTHILLS CEMETERY"
s_I:
    APSTR "I"
s_UNDERGROUND_STREAM:
    APSTR "UNDERGROUND STREAM"
s_V:
    APSTR "V"
s_GEMINI:
    APSTR "GEMINI"
s_R:
    APSTR "R"
s_OLD_MINE_SHAFT:
    APSTR "OLD MINE SHAFT"
s_GILDED_PATHWAY:
```

```
    APSTR "GILDED PATHWAY"
s_MINOTAURS_DEN:
    APSTR "MINOTAUR'S DEN"
s_MINOTAURS_VAULT:
    APSTR "MINOTAUR'S VAULT"
s_INNER_STAIRCASE:
    APSTR "INNER STAIRCASE"
s_SECRET_PASSAGEWAY:
    APSTR "SECRET PASSAGEWAY"
s_VIRGO:
    APSTR "VIRGO"
s_LEO:
    APSTR "LEO"
s_LIBRA:
    APSTR "LIBRA"
s_SCORPIUS:
    APSTR "SCORPIUS"
s_SAGITTARIUS:
    APSTR "SAGITTARIUS"
s_CAPRICORNUS:
    APSTR "CAPRICORNUS"
s_AQUARIUS:
    APSTR "AQUARIUS"
s_PISCES:
    APSTR "PISCES"
s_HALLWAY:
    APSTR "HALLWAY"
s_ARIES:
    APSTR "ARIES"
s_TAURUS:
    APSTR "TAURUS"
s_GEMINI2:
    APSTR "GEMINI"
s_CANCER:
    APSTR "CANCER"
s_THE_MOUNTAIN:
    APSTR "THE MOUNTAIN"
s_SHAHRIAR_TREASURY:
    APSTR "SHAHRIAR TREASURY"
s_AIR_VENT:
    APSTR "AIR VENT"
s_DESERTED_TUNNEL:
    APSTR "DESERTED TUNNEL"
s_ROY_G_BIV:
    APSTR "ROY G. BIV"
s_JACKAL_ALTAR:
    APSTR "JACKAL ALTAR"
s_ASTROLOGERS_MAZE:
    APSTR "ASTROLOGER'S MAZE"
```

Defines:

`LOCATION_NAMES`, never used.

4.2 Event records

Each location has a variable-length event list pointed to by `EVENT_PTR` (location record fields 6–7). Entries are 3 bytes each, terminated by `$FF` followed by two `$FF` padding bytes. Each location's first entry is normally an empty slot (`$FE`) reserved for runtime use (e.g. dropped treasure after combat).

Byte 0: map position. A value 0–199 encodes a grid position (row × 20 + col). `$FE` marks an empty slot that can be filled at runtime. `$FF` marks the end of the list.

Byte 1: event type. The top two bits select the category:

Byte 1 range	Category	Details
$00–$3F	Random encounter	Avoidance = strength/20 (no per-event prob); if avoided, bit 6 set → deferred specific encounter
$40–$BF	Specific encounter	bits 5–0 = mob index → `ACTIVE_CHAR`
$C0–$D1	Dropped treasure	gold = ((`byte1` − $C0) × 256) + `byte2`
$D2	Persistent scene	`byte2` = scene ID; not cleared after trigger
$D3	One-time scene	`byte2` = scene ID; slot cleared to $FE
$D4	Scene + item drop	like $D3 but also spawns 1–4 bonus items
$D5–$DA	Map/area event	indexed from table at `SHOP_TABLE`
$DB–$FD	(unknown)	
$FE	Empty-slot data	`byte2` has sub-encoding

Byte 2: action data (meaning depends on byte 1). For encounters ($40–$BF), this is the map position where the mob appears. For treasure ($C0–$D1), the gold amount low byte. For scene events ($D2–$D4), a scene/event ID.

Key routines.

- $0B62 — searches a position for mobs first, then events; returns `ENTITY_REC` pointing to the found entry.
- `FUN_138D` ($138D) — finds the nearest non-activated event (byte 1 < $C0) using Manhattan distance; used for AI pathfinding.
- $1260 — fills empty `$FE` slots with combat loot, or accumulates value in $C0–$D0 entries (treasure drops). Entries saturate at byte 1 = $D1.

- HANDLE_ENCOUNTER — encounter handler: dispatches random vs. specific encounters based on byte 1 < $40.

32 ⟨*Location events* 32⟩≡ (909)

```
    ORG     $451E
LOCATION_EVENTS:
ev_ali_baba_home:                       ; 0: ALI BABA HOME
    EVDATA $00,$00
    EVENCO 6,11,$42,5,11
    EVSTOP

ev_dusty_road:                          ; 1: DUSTY ROAD
    EVDATA $00,$00
    EVRAND 5,7,$03,5,6
    EVRAND 3,9,$04,2,9
    EVENCO 6,11,$41,7,11
    EVRAND 3,15,$04,2,15
    EVISCN 5,15,$48
    EVENCO 6,18,$51,1,10
    EVENCO 4,19,$5F,3,2
    EVSTOP

ev_sultans_palace:                      ; 2: SULTAN'S PALACE
    EVDATA $00,$00
    EVENCO 1,7,$04,1,8
    EVGOLD 4,1,25
    EVISCN 5,1,$4C
    EVENCO 5,1,$39,4,12
    EVGOLD 6,1,45
    EV1SCN 5,5,$46
    EVENCO 5,7,$02,5,8
    EVSTOP

ev_royal_library:                       ; 3: ROYAL LIBRARY
    EVDATA $00,$00
    EVENCO 1,7,$03,1,6
    EVPSCN 1,9,$47
    EVRAND 3,9,$02,4,9
    EVPSCN 1,11,$51
    EVPSCN 1,13,$5E
    EVPSCN 1,15,$5C
    EVRAND 3,15,$02,4,15
    EVSTOP

ev_astrologers_lab:                     ; 4: ASTROLOGER'S LAB
    EVDATA $00,$00
    EVDATA $00,$00
    EVENCO 0,4,$35,7,2
    EVENCO 0,7,$34,7,6
    EVENCO 0,10,$33,7,10
```

```
    EVENCO 0,12,$32,7,13
    EVENCO 0,15,$31,7,17
    EVRAND 3,3,$36,2,15
    EVPSCN 3,10,$56
    EVENCO 3,16,$30,2,1
    EVENCO 6,3,$37,7,15
    EVENCO 6,16,$2F,7,1
    EVENCO 9,4,$2C,2,2
    EVRAND 9,7,$2B,2,6
    EVENCO 9,11,$2D,2,12
    EVENCO 9,15,$2E,2,16
    EVSTOP

ev_bear_cave:                           ; 5: BEAR CAVE
    EVDATA $00,$00
    EVSLOT 1,5,$1F
    EVSLOT 1,8,$DF
    EVENCO 2,0,$7A,2,15
    EVSLOT 2,5,$1F
    EVGOLD 2,6,150
    EVSLOT 2,8,$1F
    EVSLOT 3,6,$1F
    EVSLOT 3,7,$1F
    EVGOLD 1,12,100
    EVENCO 5,12,$07,6,12
    EVSTOP

ev_rats_nest:                           ; 6: RATS NEST
    EVDATA $00,$00
    EVENCO 5,12,$06,4,12
    EVENCO 6,0,$7A,6,15
    EVGOLD 7,4,50
    EVSTOP

ev_dragon_lair:                         ; 7: DRAGON LAIR
    EVDATA $00,$00
    EVRAND 4,5,$0A,3,5
    EVGOLD 8,5,300
    EVENCO 8,9,$49,8,10
    EVSTOP

ev_gold_for_the_lucky:                  ; 8: GOLD FOR THE LUCKY
    EVDATA $00,$00
    EVRAND 4,13,$0A,3,13
    EVSLOT 7,12,$1F
    EVSLOT 7,13,$1F
    EV1SCN 7,14,$59
    EVENCO 8,9,$48,8,8
    EVSLOT 8,11,$1F
    EVGOLD 8,13,50
```

```
    EVGOLD 8,13,500
    EVSLOT 8,14,$1F
    EVENCO 9,13,$6C,2,3
    EVSTOP

ev_save_the_whales:                     ; 9: SAVE THE WHALES
    EVDATA $00,$00
    EVENCO 0,5,$0E,8,5
    EVENCO 0,14,$10,8,14
    EVSLOT 1,9,$D6
    EVENCO 2,2,$69,1,15
    EVSLOT 2,8,$D6
    EVSLOT 2,10,$D6
    EVSLOT 3,9,$D6
    EVSLOT 2,9,$95
    EVISCN 2,9,$5A
    EVGOLD 2,9,2390
    EVRAND 4,5,$08,5,5
    EVSHOP 2,15,$D6,$E0
    EVRAND 4,13,$09,6,13
    EVSTOP

ev_slide:                               ; 10: SLIDE
    EV1SCN 2,2,$5D
    EVGOLD 5,2,100
    EVENCO 0,2,$7E,6,6
    EVENCO 3,2,$4B,4,2
    EVENCO 5,2,$4B,6,2
    EVENCO 7,2,$4B,8,2
    EVENCO 9,2,$4D,6,11
    EVSTOP

ev_deep_dark_stairway:                  ; 11: DEEP DARK STAIRWAY
    EVDATA $00,$00
    EVENCO 1,4,$62,7,4
    EVSLOT 2,6,$1F
    EVSLOT 2,7,$9F
    EVSLOT 2,8,$1F
    EVSLOT 3,7,$9F
    EVSLOT 3,8,$1F
    EVSLOT 4,4,$1F
    EVSLOT 4,8,$1F
    EVSLOT 5,4,$1F
    EVSLOT 5,5,$9F
    EVSLOT 6,4,$1F
    EVSLOT 6,5,$9F
    EVSLOT 6,6,$1F
    EVENCO 5,9,$4D,4,4
    EVENCO 6,9,$4D,5,4
    EVRAND 7,7,$13,2,1
```

```
    EVSTOP

ev_minotaur_labyrinth:                  ; 12: MINOTAUR LABYRINTH
    EVDATA $00,$00
    EVSLOT 1,5,$DF
    EVENCO 1,10,$4D,1,9
    EVGOLD 1,11,75
    EVENCO 1,15,$6A,2,1
    EVENCO 2,6,$4D,1,5
    EVSLOT 2,11,$1F
    EVSLOT 2,12,$DF
    EVSLOT 2,13,$1F
    EVSLOT 2,14,$1F
    EVSLOT 2,15,$1F
    EVSLOT 3,4,$DF
    EVSLOT 3,7,$1F
    EVSLOT 3,10,$1F
    EVENCO 4,3,$4C,5,8
    EV1SCN 4,5,$5F
    EVSLOT 4,5,$DF
    EVSLOT 4,8,$1F
    EVSLOT 4,9,$DF
    EVSLOT 4,14,$12
    EVENCO 4,16,$67,4,1
    EVENCO 5,3,$4C,6,8
    EVENCO 5,4,$4D,2,4
    EVSLOT 5,6,$1F
    EVENCO 5,9,$4D,5,8
    EVENCO 5,10,$4D,4,10
    EVSLOT 5,11,$1F
    EVSLOT 5,13,$C0
    EVSLOT 5,13,$DA
    EVSLOT 5,13,$1F
    EVENCO 6,5,$4D,6,11
    EVSLOT 6,6,$C0
    EVENCO 6,6,$59,8,11
    EVSLOT 6,8,$DF
    EVENCO 6,10,$4D,6,9
    EVSLOT 6,12,$1F
    EVSLOT 7,4,$04
    EVENCO 7,7,$4D,7,6
    EVSLOT 7,8,$1F
    EVSLOT 7,9,$1F
    EVSLOT 7,10,$9F
    EVSLOT 7,11,$1F
    EVSLOT 7,13,$9E
    EVSLOT 7,14,$1F
    EVGOLD 8,5,125
    EVENCO 8,7,$4D,7,7
    EVENCO 8,8,$61,1,8
```

```
    EVSLOT 8,9,$DB
    EVSLOT 8,14,$1E
    EVSTOP

ev_stinger_strewn_cell:                 ; 13: STINGER STREWN CELL
    EVDATA $00,$00
    EVRAND 5,6,$0F,4,6
    EVENCO 7,8,$19,7,10
    EVGOLD 8,4,228
    EVRAND 9,5,$0A,1,5
    EVSTOP

ev_guard_room:                          ; 14: GUARD ROOM
    EVDATA $00,$00
    EVRAND 0,9,$26,8,8
    EVENCO 5,6,$0E,6,6
    EVENCO 5,13,$10,6,13
    EVSTOP

ev_long_way_from_home:                  ; 15: LONG WAY FROM HOME
    EVDATA $00,$00
    EVRAND 5,13,$0F,4,13
    EVENCO 7,11,$19,7,12
    EVGOLD 8,15,136
    EVRAND 9,14,$0A,1,14
    EVSTOP

ev_forest:                              ; 16: FOREST
    EVDATA $00,$00
    EVENCO 0,10,$42,5,18
    EVGOLD 1,3,3
    EVSHOP 2,6,$D5,$90
    EVSHOP 5,3,$D6,$58
    EVSLOT 2,11,$1F
    EVSLOT 3,3,$1F
    EVSLOT 3,4,$1F
    EVSLOT 3,9,$1F
    EVSLOT 3,10,$1F
    EVSLOT 5,7,$1F
    EVSLOT 6,4,$1F
    EVSLOT 6,7,$1F
    EVSLOT 7,9,$1F
    EVGOLD 8,1,47
    EVSLOT 8,9,$1F
    EVENCO 6,13,$52,6,14
    EVENCO 8,13,$5E,2,1
    EVSTOP

ev_view_from_a_tree:                    ; 17: VIEW FROM A TREE
    EVDATA $00,$00
```

```
    EVENCO 1,16,$55,8,5
    EVSLOT 2,15,$1F
    EVSLOT 2,16,$DF
    EVSLOT 2,17,$1F
    EVPSCN 3,16,$4E
    EVENCO 6,13,$51,6,12
    EVENCO 7,16,$5E,2,3
    EVSTOP

ev_failing_wall:                        ; 18: FAILING WALL
    EVDATA $00,$00
    EVSLOT 2,9,$9E
    EVSLOT 3,8,$1E
    EVSLOT 4,7,$1E
    EVSLOT 5,6,$1E
    EVSLOT 6,6,$1E
    EVSLOT 7,5,$C0
    EVGOLD 7,5,2604
    EVSLOT 7,6,$1E
    EVSHOP 7,3,$D5,$46
    EVSLOT 5,8,$19
    EVENCO 5,14,$14,5,6
    EVSTOP

ev_minotaur_playroom:                   ; 19: MINOTAUR PLAYROOM
    EVDATA $00,$00
    EVSLOT 5,7,$19
    EVGOLD 7,11,200
    EVENCO 2,16,$54,2,16
    EVENCO 3,16,$54,3,16
    EVENCO 4,17,$54,4,17
    EVENCO 5,18,$54,5,18
    EVENCO 1,18,$27,8,3
    EVSTOP

ev_thieves_cave:                        ; 20: THIEVES CAVE
    EVGOLD 1,6,2479
    EVGOLD 1,9,30
    EVGOLD 1,12,60
    EVRAND 4,16,$1B,5,4
    EV1SCN 6,4,$50
    EVSLOT 6,5,$1F
    EVSLOT 6,6,$1F
    EVSLOT 7,6,$1F
    EVPSCN 8,5,$4F
    EVSLOT 8,6,$1F
    EVGOLD 8,9,10
    EVGOLD 8,12,2504
    EVGOLD 8,15,25
    EVENCO 9,5,$52,3,16
```

```
    EVSTOP

ev_o:                                   ; 21: O
    EVDATA $00,$00
    EV1SCN 4,12,$57
    EVENCO 6,10,$16,6,11
    EVGOLD 5,15,45
    EVSLOT 5,16,$89
    EVENCO 6,18,$18,2,2
    EVSTOP

ev_spiral_staircase:                    ; 22: SPIRAL STAIRCASE
    EVDATA $00,$00
    EVENCO 1,9,$3E,1,15
    EVSLOT 3,8,$1F
    EVSLOT 3,9,$1F
    EVSLOT 5,7,$1F
    EVSLOT 5,8,$1F
    EVRAND 7,7,$18,2,16
    EVSTOP

ev_y:                                   ; 23: Y
    EVDATA $00,$00
    EVENCO 1,16,$17,6,7
    EVSLOT 6,9,$1F
    EVSLOT 6,10,$1F
    EVSLOT 6,11,$1F
    EVSLOT 7,11,$DF
    EVENCO 7,9,$1A,2,9
    EVGOLD 7,10,2534
    EVSTOP

ev_magic_tourbillion:                   ; 24: MAGIC TOURBILLION
    EVDATA $00,$00
    EVGOLD 6,11,100
    EVENCO 7,11,$40,7,11
    EVSHOP 6,10,$D5,$92
    EVSHOP 6,12,$D6,$A8
    EVSTOP

ev_g:                                   ; 25: G.
    EVDATA $00,$00
    EVENCO 2,9,$62,7,4
    EVRAND 7,1,$1C,2,17
    EVGOLD 7,11,70
    EVSTOP

ev_crystal_cave:                        ; 26: CRYSTAL CAVE
    EVENCO 2,9,$7A,8,15
    EVSLOT 3,6,$C0
```

```
    EVGOLD 3,6,35
    EVSLOT 4,12,$C0
    EVGOLD 4,12,25
    EVRAND 5,3,$15,4,15
    EVGOLD 5,9,25
    EVSLOT 5,9,$C0
    EVSLOT 5,9,$C0
    EVGOLD 5,9,50
    EVSLOT 5,14,$C0
    EVSLOT 5,14,$C0
    EVGOLD 5,14,30
    EVENCO 5,15,$7E,4,4
    EVSLOT 7,4,$C0
    EVSLOT 7,4,$C0
    EVGOLD 7,4,50
    EV1SCN 7,8,$4D
    EVSLOT 7,8,$C0
    EVGOLD 7,8,35
    EVRAND 8,8,$24,2,5
    EVSLOT 6,12,$C0
    EVSLOT 6,12,$C0
    EVSLOT 6,12,$C0
    EVGOLD 6,12,70
    EVSTOP

ev_b:                                   ; 27: B
    EVRAND 2,1,$20,7,17
    EVDATA $00,$00
    EVENCO 2,11,$5C,2,17
    EVENCO 3,3,$5C,2,17
    EVENCO 4,9,$5C,2,17
    EVENCO 5,9,$00,8,12
    EVSHOP 7,3,$D6,$58
    EVENCO 5,10,$5C,2,17
    EVSTOP

ev_thieves_clearing:                    ; 28: THIEVES CLEARING
    EVDATA $00,$00
    EVRAND 4,8,$1E,3,8
    EVGOLD 8,6,75
    EVGOLD 7,11,25
    EVSTOP

ev_old_mine_road:                       ; 29: OLD MINE ROAD
    EVDATA $00,$00
    EVENCO 1,3,$52,6,16
    EVENCO 1,14,$78,8,12
    EVSHOP 3,15,$D6,$A8
    EVENCO 2,0,$51,8,12
    EVSLOT 3,7,$1F
```

```
    EVSLOT 3,8,$DF
    EVENCO 4,8,$1D,5,8
    EVSLOT 3,9,$1F
    EV1SCN 2,18,$4B
    EVGOLD 2,18,20
    EVENCO 2,18,$25,4,5
    EVSTOP

ev_foothills_cemetery:                  ; 30: FOOTHILLS CEMETERY
    EVDATA $00,$00
    EVENCO 3,1,$42,4,18
    EVSLOT 3,9,$5F
    EVSLOT 4,8,$5F
    EVSLOT 4,9,$9F
    EVGOLD 4,9,100
    EVSLOT 4,10,$5F
    EVSLOT 5,9,$5F
    EVSLOT 6,9,$5F
    EVGOLD 6,6,5
    EVENCO 7,18,$78,6,3
    EVSTOP

ev_i:                                   ; 31: I
    EVDATA $00,$00
    EVRAND 1,4,$22,7,4
    EVSLOT 2,9,$1F
    EVSLOT 3,9,$1F
    EVSLOT 4,9,$1F
    EVSLOT 5,9,$1F
    EVSLOT 6,9,$9E
    EVGOLD 7,4,125
    EVSLOT 7,9,$1F
    EVSTOP

ev_underground_stream:                  ; 32: UNDERGROUND STREAM
    EV1SCN 1,9,$5B
    EVSLOT 1,9,$D6
    EVGOLD 1,9,4352
    EVGOLD 1,9,50
    EVSLOT 1,9,$D6
    EVGOLD 1,9,50
    EVSLOT 1,9,$D6
    EVDATA $00,$00
    EVSHOP 2,12,$D6,$50
    EVENCO 0,13,$00,8,10
    EVENCO 3,13,$0D,8,8
    EVSTOP

ev_v:                                   ; 33: V
    EVDATA $00,$00
```

```
    EVGOLD 2,6,65
    EVRAND 2,18,$24,7,2
    EVENCO 8,4,$4C,2,4
    EVSTOP

ev_gemini:                          ; 34: GEMINI
    EVDATA $00,$00
    EVENCO 2,16,$05,3,4
    EVENCO 4,5,$26,1,8
    EVSTOP

ev_r:                               ; 35: R
    EVDATA $00,$00
    EVRAND 1,5,$1B,7,8
    EVSLOT 4,5,$1F
    EVSHOP 4,3,$D5,$82
    EVSLOT 5,5,$1F
    EVSLOT 6,5,$1F
    EVSLOT 7,5,$1F
    EVGOLD 7,4,100
    EVSLOT 7,6,$C0
    EVGOLD 7,6,65
    EVENCO 6,10,$16,6,11
    EVSTOP

ev_old_mine_shaft:                  ; 36: OLD MINE SHAFT
    EVDATA $00,$00
    EVENCO 4,4,$5E,2,18
    EV1SCN 4,9,$49
    EVSLOT 4,9,$1F
    EVENCO 4,14,$75,2,1
    EVSTOP

ev_gilded_pathway:                  ; 37: GILDED PATHWAY
    EVDATA $00,$00
    EVENCO 0,8,$63,3,5
    EVENCO 2,9,$40,7,11
    EVENCO 3,7,$40,8,10
    EVSLOT 3,8,$DF
    EVGOLD 4,9,2304
    EVGOLD 5,7,2304
    EVENCO 7,7,$4F,4,9
    EVSLOT 7,9,$C9
    EVENCO 9,8,$0F,1,9
    EVSTOP

ev_minotaurs_den:                   ; 38: MINOTAUR'S DEN
    EVDATA $00,$00
    EVDATA $00,$00
    EVENCO 4,0,$4D,4,15
```

```
    EVRAND 8,7,$28,8,8
    EVSTOP

ev_minotaurs_vault:                     ; 39: MINOTAUR'S VAULT
    EVDATA $00,$00
    EVRAND 6,12,$29,5,12
    EVRAND 7,7,$27,7,6
    EVGOLD 8,10,2604
    EVGOLD 8,14,2504
    EVSTOP

ev_inner_staircase:                     ; 40: INNER STAIRCASE
    EVDATA $00,$00
    EVRAND 1,16,$0A,2,3
    EVSLOT 1,12,$1F
    EVSLOT 1,13,$9F
    EVSLOT 2,12,$1F
    EVSLOT 3,12,$C0
    EVENCO 3,12,$6A,2,10
    EVSLOT 4,15,$1F
    EVSLOT 5,14,$5F
    EVSLOT 5,15,$1F
    EVRAND 6,12,$28,7,12
    EVSTOP

ev_secret_passageway:                   ; 41: SECRET PASSAGEWAY
    EVDATA $00,$00
    EVRAND 2,0,$0D,1,15
    EVRAND 2,11,$29,3,12
    EVSTOP

ev_virgo:                               ; 42: VIRGO
    EVDATA $00,$00
    EVENCO 1,6,$05,8,7
    EVENCO 6,9,$2D,6,10
    EVSTOP

ev_leo:                                 ; 43: LEO
    EVDATA $00,$00
    EVENCO 1,2,$05,8,4
    EVGOLD 7,2,250
    EVRAND 6,4,$2B,6,5
    EVSTOP

ev_libra:                               ; 44: LIBRA
    EVENCO 1,12,$05,8,11
    EVENCO 6,14,$2E,6,15
    EVSLOT 5,11,$B7
    EVGOLD 7,12,2404
    EVSTOP
```

```
ev_scorpius:                            ; 45: SCORPIUS
    EVENCO 1,16,$05,8,15
    EVDATA $00,$00
    EVENCO 6,19,$2F,8,12
    EVGOLD 7,16,150
    EVSTOP

ev_sagittarius:                         ; 46: SAGITTARIUS
    EVDATA $00,$00
    EVENCO 4,12,$30,3,12
    EVSLOT 6,6,$19
    EV1SCN 6,12,$53
    EVENCO 7,0,$05,6,15
    EVSTOP

ev_capricornus:                         ; 47: CAPRICORNUS
    EVDATA $00,$00
    EVENCO 0,12,$31,3,18
    EVSLOT 1,10,$1F
    EVENCO 2,0,$05,3,15
    EVSLOT 2,5,$0D
    EVISCN 2,9,$55
    EVSLOT 2,10,$1F
    EVGOLD 2,15,2554
    EVSLOT 3,5,$15
    EVSHOP 2,12,$D6,$F8
    EVSLOT 3,10,$1F
    EVSTOP

ev_aquarius:                            ; 48: AQUARIUS
    EVDATA $00,$00
    EVGOLD 2,17,4352
    EVENCO 3,15,$32,3,14
    EVSLOT 4,17,$16
    EVENCO 8,17,$05,1,15
    EVSTOP

ev_pisces:                              ; 49: PISCES
    EVDATA $00,$00
    EVSLOT 4,13,$95
    EVISCN 5,13,$52
    EVSLOT 6,13,$55
    EVENCO 8,13,$05,1,12
    EVSTOP

ev_hallway:                             ; 50: HALLWAY
    EVDATA $00,$00
    EVENCO 1,9,$7E,8,9
    EVENCO 1,10,$7E,8,10
```

```
    EVENCO 8,10,$05,1,10
    EVSTOP

ev_aries:                               ; 51: ARIES
    EVDATA $00,$00
    EVENCO 3,4,$35,3,3
    EVGOLD 2,6,100
    EVENCO 8,6,$05,1,7
    EVSTOP

ev_taurus:                              ; 52: TAURUS
    EVDATA $00,$00
    EVENCO 2,0,$65,4,13
    EVRAND 3,0,$36,1,5
    EVENCO 4,1,$75,4,2
    EVENCO 8,2,$05,1,4
    EVSTOP

ev_gemini2:                             ; 53: GEMINI (south)
    EVDATA $00,$00
    EVSLOT 1,8,$01
    EVSLOT 3,8,$01
    EVGOLD 2,5,2404
    EVENCO 2,16,$05,3,4
    EVENCO 4,5,$37,6,5
    EVSTOP

ev_cancer:                              ; 54: CANCER
    EVDATA $00,$00
    EVGOLD 7,3,50
    EVENCO 7,16,$05,6,4
    EV1SCN 6,12,$60
    EVENCO 9,5,$2C,6,1
    EVSTOP

ev_the_mountain:                        ; 55: THE MOUNTAIN
    EVDATA $00,$00
    EVISCN 1,3,$4A
    EVSLOT 1,4,$1F
    EVENCO 1,15,$3A,1,15
    EVSLOT 3,10,$1F
    EVSLOT 4,9,$1F
    EVSLOT 5,8,$1F
    EVSLOT 6,7,$1F
    EVSHOP 8,15,$D5,$44
    EVSLOT 2,3,$C0
    EVENCO 2,3,$52,2,18
    EVSLOT 2,4,$1F
    EVSLOT 3,4,$1F
    EVSLOT 3,14,$1F
```

```
    EVSLOT 3,15,$1F
    EVSLOT 4,14,$1F
    EVGOLD 4,15,50
    EVSLOT 5,14,$1F
    EVENCO 6,2,$5F,7,17
    EVGOLD 6,10,5
    EVSLOT 6,14,$1F
    EVISCN 8,4,$58
    EVENCO 9,12,$5E,2,14
    EVSTOP

ev_shahriar_treasury:                   ; 56: SHAHRIAR TREASURY
    EVDATA $00,$00
    EVGOLD 3,7,2554
    EVGOLD 4,6,50
    EVGOLD 4,7,100
    EVGOLD 4,8,50
    EVGOLD 5,7,50
    EVENCO 4,13,$43,5,2
    EVSTOP

ev_air_vent:                            ; 57: AIR VENT
    EVENCO 0,15,$78,1,15
    EVENCO 2,16,$46,2,1
    EVDATA $00,$00
    EVENCO 4,14,$7D,4,13
    EVENCO 6,16,$47,6,1
    EVENCO 8,14,$7B,8,13
    EVENCO 9,15,$5B,3,9
    EVSTOP

ev_deserted_tunnel:                     ; 58: DESERTED TUNNEL
    EVDATA $00,$00
    EVGOLD 7,7,45
    EV1SCN 7,8,$54
    EVSLOT 7,9,$1F
    EVSLOT 8,7,$1F
    EVSLOT 8,8,$C0
    EVENCO 8,8,$78,5,15
    EVENCO 8,14,$7A,8,15
    EVSTOP

ev_roy_g_biv:                           ; 59: ROY G. BIV
    EVENCO 4,2,$7D,4,3
    EVSLOT 5,1,$C1
    EVGOLD 5,1,100
    EVSLOT 7,1,$C1
    EVENCO 7,1,$7C,6,1
    EVSLOT 8,1,$C0
    EVENCO 8,1,$23,2,3
```

```
    EVSTOP

ev_jackal_altar:                        ; 60: JACKAL ALTAR
    EVDATA $00,$00
    EVGOLD 1,3,55
    EVSLOT 1,12,$1F
    EVSLOT 2,8,$1F
    EVSLOT 2,9,$1F
    EVSLOT 2,10,$1F
    EVSLOT 2,11,$1F
    EVSLOT 2,12,$1F
    EVSLOT 3,5,$1F
    EVSLOT 3,6,$1F
    EVSLOT 4,7,$1F
    EVSLOT 4,8,$1F
    EVSLOT 4,9,$1F
    EVENCO 4,2,$7C,4,1
    EVENCO 4,14,$7A,4,15
    EVSTOP

ev_astrologers_maze:                    ; 61: ASTROLOGER'S MAZE
    EVDATA $00,$00
    EVSLOT 1,10,$1F
    EVENCO 1,15,$57,2,9
    EVSLOT 2,6,$1F
    EVSLOT 2,11,$1F
    EVSLOT 2,13,$1F
    EVSLOT 2,14,$1F
    EVSLOT 2,15,$1F
    EVSLOT 3,7,$1F
    EVSLOT 3,10,$1F
    EVENCO 4,3,$5B,5,14
    EVSLOT 4,8,$1F
    EVSLOT 4,14,$1F
    EVSLOT 5,6,$1F
    EVSLOT 5,9,$1F
    EVSLOT 5,10,$1F
    EVSLOT 5,11,$1F
    EVSLOT 5,13,$1F
    EVSLOT 6,5,$1F
    EVSLOT 6,6,$C0
    EVENCO 6,6,$4B,1,2
    EVGOLD 6,11,2424
    EVSLOT 6,12,$1F
    EVSLOT 7,4,$1F
    EVSLOT 7,7,$1F
    EVSLOT 7,8,$1F
    EVSLOT 7,9,$1F
    EVSLOT 7,10,$1F
    EVSLOT 7,11,$1F
```

```
EVSLOT 7,14,$1F
EVGOLD 8,4,50
EVSLOT 8,7,$1F
EVSLOT 8,14,$1F
EVGOLD 8,15,10
EVENCO 9,9,$74,2,7
EVENCO 9,10,$73,2,10
EVSHOP 1,11,$D5,$C6
EVSTOP
```

Defines:
LOCATION_EVENTS, never used.
Uses EV1SCN 22, EVDATA 22, EVENCO 22, EVGOLD 22, EVISCN 22, EVPSCN 22, EVRAND 22, EVSHOP 22, EVSLOT 22, and EVSTOP 22.

The attract screen call flow:

- MAIN_ENTRY (MAIN_ENTRY): calls GAME_INIT, then ATTRACT_SCREEN if TOTAL_MOB_COUNT is zero.
- ATTRACT_SCREEN (ATTRACT_SCREEN): sets scene index 1, calls GET_MOB_DATA, tail-calls SCENE_LOOP with scene $26.
- SCENE_LOOP (SCENE_LOOP): saves ZP pointers, steps PRNG, conditionally loads scene data from disk. In a loop: calls LOAD_SCENE_FROM_DISK, displays status bar, renders scene text via TEXT_RENDERER, runs TUNE_PLAYER with tune #6. Exits on disk error or when scene sequence ends.

Key subsystems:

- **Text renderer** (TEXT_RENDERER): Reads byte streams with position commands, pause/reset control codes, and character data for the HRCG font.
- **Tune player** (TUNE_PLAYER): Plays sound effects and music from TUNE_TABLE. Supports loops, gosub, high/low octave selection, and PRNG-based note randomization.
- **Resident interpreter** (RESET_VECTOR–$07FF): A secondary VM copied from EALDR, using context-swapped zero-page state and XOR-encrypted addresses.
- **PRNG** (STEP_PRNG): Linear congruential generator with formula $\texttt{val}_{n+1} = 69 \cdot \texttt{val}_n + \$53 \pmod{256}$.

The start_splash_screen routine first loads up the initial splash screen, sets indirect_routine to note_played, and begins the sound routine by calling start_sound.

note_played plots a mob, then sets indirect_routine to splash_routine_2 and jumps to next_table_entry.

splash_routine_2 calls a routine first with $5E1F then with $5E3A, then jumps to splash_screen_step.

splash_screen_step then sets indirect_routine to splash_routine_1 then splash_routine_0 and calls set_splash_routine within note_played. After that, splash_screen_step presumably moves mobs around.

When the user ends the splash screen, the start_splash_screen routine replaces the indirect jump to indirect_routine in start_sound to an absolute jump to next_table_entry, since there is no further need to animate while playing sound.

Immediately after the event lists, the region $4C80--$5AFF holds the mob data table and associated name strings.

- $4C80--$547F: 128 mob data records, 16 bytes each. Bytes 0–1 are a pointer to the mob's name string; the remaining 14 bytes encode combat stats, group linkage, position, appearance, and level. GET_MOB_DATA indexes into this table via the base pointer at $4000/$4001.
- $5480--$59C8: Length-prefixed high-bit ASCII name strings for all 128 mobs ("ALI BABA", "SCHEHERAZADE", "MINOTAUR", etc.).
- $59C9--$5AFF: Residual debug/diagnostic text (unused at runtime).

The 128 mob data records and name strings are documented in Chapter 13.

s_PRINT_FONT_CHAR is the 10-byte self-modifying HRCG control string that RENDER_FONT_CHAR patches with character-set and glyph codes before passing to PRINT_STRING. It is a 9-byte self-modifying string: the routine patches CHAR_CHARSET, CHAR_UPPER_LEFT, CHAR_UPPER_RIGHT, CHAR_LOWER_LEFT, and CHAR_LOWER_RIGHT in place before each call. The HRCG control codes embedded in the string switch charset (ctrl-A), enter block mode (ctrl-B), move to the lower half (ctrl-C), and finish the block (ctrl-D), so that a single PRINT_STRING call renders an entire 2×2 font character.

4.2.1 Plot character

PLOT_CHAR renders a single custom font glyph at a given text coordinate. It stores the character number, column, and row, forces the custom font charset, and delegates to PRINT_FONTCHAR_AT_TEXT_POS.

49 ⟨*plot char* 49⟩≡ (909)

```
      ORG     $5DB0
  PLOT_CHAR:
      SUBROUTINE PLOT_CHAR
      ; Inputs:  A = character number, X = text column, Y = text row
      ; Outputs: glyph rendered to hi-res page
      ; Clobbers: A, X, Y

      STA     FONT_CHARNUM            ; save glyph index
      STX     TEXT_COL                ; save target column
      STY     TEXT_ROW                ; save target row
      STOB    #$01,FONT_CHARSET       ; force custom font charset
      JMP     PRINT_FONTCHAR_AT_TEXT_POS ; render and return
```

Defines:
PLOT_CHAR, used in chunk 580.
Uses FONT_CHARNUM 575, FONT_CHARSET 576a, PRINT_FONTCHAR_AT_TEXT_POS 466, STOB 8c, TEXT_COL 572a, and TEXT_ROW 572a.

4.3 Undocumented data regions

4.3.1 Runtime data area

The region $A8F2–$B2FF is a 2,574-byte runtime data area, initialized to mostly zeros in the disk image. The first 14 bytes ($A8F2–$A8FF) contain non-zero initialization data; the remaining 2,560 bytes ($A900–$B2FF) are zeroed. During gameplay, this area is used extensively as working storage with 77 absolute-addressed references from across the program, including:

- Entity/mob runtime state ($A919, $A960, $A964, $A96C, $A974)
- Subroutine at $AB45 (called from game data at $4A64)
- Display buffer at $B120 (indexed load from $1980)
- Comparison data at $ABD0 (tested from $59FE, $5A65, $5AE3)
- Disk I/O state ($ADBB, $AFBA)
- Jump vector at $AD06 (from $0601)

Full decomposition of individual fields is deferred.

50 ⟨*runtime data area* 50⟩≡ (909)

```
    ORG     $A8F2
RUNTIME_DATA_INIT:
    ; Non-zero initialization data ($A8F2-$A8FF)
    HEX     AA 00 40 55 A4 55 01 00 CA 02 01 00 80 80

RUNTIME_DATA_AREA:
    DS      2560,$00            ; zeroed runtime working storage ($A900-$B2FF)
```

Defines:
RUNTIME_DATA_AREA, never used.
RUNTIME_DATA_INIT, never used.

4.3.2 Small gap fills

The following HEX chunks fill byte-level gaps between documented routines. They represent undocumented code and data awaiting detailed analysis.

IDLE_ANIM_HANDLER runs idle processing during the keyboard poll loop. In keyboard mode (INPUT_MODE=0), it calls CALL_ROM_WAIT to burn time and decrements WAIT_LOOP_COUNT. When the wait count reaches zero, it checks the animation frame via ANIM_CHECK_FRAME, ticks the counter, and loops. In joystick mode, it runs a fixed 12-iteration countdown via DEC_WAIT_IF_NONZERO.

51 ⟨*var wait loop count* 51⟩≡ (909)

```
    ORG     $5A8C
WAIT_LOOP_COUNT:
    DC.B    $D9                     ; initial value (runtime: inner wait loop counter)
```

Defines:
WAIT_LOOP_COUNT, used in chunks 52, 354, 532c, 544a, 545, 604b, 606a, 683, and 898.

52 ⟨*idle anim handler* 52⟩≡ (909)

```
    ORG     $0D5E
IDLE_ANIM_HANDLER:
    SUBROUTINE IDLE_ANIM_HANDLER
    ; Idle animation handler called during keyboard polling.
    ;
    ; Behavior:
    ;   Keyboard mode: burns time via CALL_ROM_WAIT, decrements
    ;   WAIT_LOOP_COUNT.  On expiry, checks frame and loops.
    ;   Joystick mode: runs 12 iterations of DEC_WAIT_IF_NONZERO.
    ;
    ; Clobbers: A, X

    LDA     WAIT_LOOP_COUNT         ; anything to wait for?
    BEQ     .expired                ; zero -> frame check
    LDA     INPUT_MODE
    BNE     .joystick               ; nonzero -> joystick mode
    ; --- keyboard mode ---
    JSR     CALL_ROM_WAIT           ; burn time
    DEC     WAIT_LOOP_COUNT         ; count down
    RTS
.expired:
    JSR     ANIM_CHECK_FRAME        ; check/advance animation frame
    JSR     TICK_ANIM_COUNTER       ; tick counter, reload wait
    JMP     IDLE_ANIM_HANDLER       ; loop
.joystick:
    LDX     #$0C                    ; 12 iterations
.joy_loop:
    JSR     DEC_WAIT_IF_NONZERO     ; decrement if nonzero
    DEX
    BNE     .joy_loop
    RTS

DEC_WAIT_IF_NONZERO:
    LDA     WAIT_LOOP_COUNT
    BEQ     .done
    DEC     WAIT_LOOP_COUNT
.done:
    RTS
```

Defines:
DEC_WAIT_IF_NONZERO, never used.
IDLE_ANIM_HANDLER, used in chunk 663.
Uses ANIM_CHECK_FRAME 605b, CALL_ROM_WAIT 607a, INPUT_MODE 589c, TICK_ANIM_COUNTER 604b, and WAIT_LOOP_COUNT 51.

Character record table header: bytes 0–1 are the mob data base pointer ($4C80), bytes 2–8 are the first 9-byte record (group 0 header).

53a ⟨*char record header* 53a⟩≡ (909)

```
    ORG     $4000
CHAR_RECORD_HEADER:
MOB_BASE_LO:
    DC.B    <MOB_DATA_TABLE         ; $4000: mob data base pointer low
MOB_BASE_HI:
    DC.B    >MOB_DATA_TABLE         ; $4001: mob data base pointer high
CHAR_LIST_HEAD:
    DC.B    $05                     ; $4002: head of character linked list (mob index)
    DC.B    $9F                     ; $4003: position
    DC.B    $A0                     ; $4004: appearance
MSG_TABLE_PTR:
    DC.B    $A6                     ; $4005: message table base (2 bytes, $4005/$4006)
    DC.B    $9A                     ; $4006: HP
    DC.B    $C9                     ; $4007: attack
    DC.B    $B0                     ; $4008: XP/group
```

Defines:
CHAR_LIST_HEAD, used in chunk 612.
CHAR_RECORD_HEADER, never used.
MOB_BASE_HI, used in chunk 602.
MOB_BASE_LO, used in chunks 291, 305, 419, 420, and 602.
MSG_TABLE_PTR, used in chunk 637.
Uses MOB_DATA_TABLE 196.

Input callback jump vector, used by JMP (INPUT_JMP_VEC) at $5F5B.

53b ⟨*input jump vector* 53b⟩≡ (909)

```
    ORG     $5F5E
INPUT_JMP_VEC:
    DC.W    $A3E4                   ; input callback address (runtime-modified)
```

Defines:
INPUT_JMP_VEC, used in chunks 247 and 257.

STORE_SELECTION_TO_RECORD saves CURRENT_SELECTION into byte 1 of the menu descriptor record at MENU_DESC_PTR. Called by menu adjustment handlers (speed, danger, slot, character browse) to persist the current selection index back into the table.

53c ⟨*zero page defines* 25a⟩+≡ (909) ◁25a 353a▷

```
MENU_DESC_PTR           EQU     $C0     ; menu descriptor table pointer (2 bytes, $C0/$C1)
MENU_OPTION_PTR         EQU     $C2     ; current menu option record pointer (2 bytes, $C2/$C3)
```

Defines:
MENU_DESC_PTR, used in chunk 54b.
MENU_OPTION_PTR, never used.

54a ⟨*var current selection* 54a⟩≡ (909)

```
    ORG     $5AAB
CURRENT_SELECTION:
    DC.B    $A0                     ; initial value (runtime: current menu selection index)
```

Defines:
CURRENT_SELECTION, used in chunks 54b, 247, and 255.

54b ⟨*store selection to record* 54b⟩≡ (909)

```
    ORG     $6071
STORE_SELECTION_TO_RECORD:
    ; Store CURRENT_SELECTION into byte 1 of the menu descriptor record.
    ;
    ; Inputs:
    ;   MENU_DESC_PTR | pointer to menu descriptor table
    ;   CURRENT_SELECTION | current selection index
    ;
    ; Behavior:
    ;   Saves Y, writes CURRENT_SELECTION into (MENU_DESC_PTR)+1, restores Y.
    ;
    ; Clobbers: A

    TYA                         ; \ save Y
    PHA                         ; /
    LDY     #$01                ; byte 1 of descriptor
    LDA     CURRENT_SELECTION
    STA     (MENU_DESC_PTR),Y   ; write selection to record
    PLA                         ; \ restore Y
    TAY                         ; /
    RTS
```

Defines:
STORE_SELECTION_TO_RECORD, used in chunks 271, 273, 277, and 283.
Uses CURRENT_SELECTION 54a and MENU_DESC_PTR 53c.

SHOP_BUY_HANDLER is the buy action handler referenced from the shop record table at $6442. It compares the entity's gold (MFIELD_GOLD) against SHOP_PRICE; if insufficient, displays message $31 ("can't afford") and returns to the shop menu. Otherwise it copies the price into NUM_VALUE, calls the shop's deduct routine, displays message $26 ("bought"), then jumps through the item's effect handler pointer in the shop record.

54c ⟨*var shop price* 54c⟩≡ (909)

```
    ORG     $5A81
SHOP_PRICE:
    DC.B    $C4,$C1                 ; initial value (runtime: shop: accumulated price (2 bytes))
```

Defines:
SHOP_PRICE, used in chunks 56, 404, and 408.

55 ⟨*var shop item offset* 55⟩≡ (909)

```
    ORG     $5A85
SHOP_ITEM_OFFSET:
    DC.B    $C1                     ; initial value (runtime: offset into shop record (5 per item sl
```

Defines:
SHOP_ITEM_OFFSET, used in chunks 56 and 404.

56 ⟨*shop buy handler* 56⟩≡ (909)

```
    ORG     $71AC
SHOP_BUY_HANDLER:
    SUBROUTINE SHOP_BUY_HANDLER
    ; Handle the "buy" action in the shop menu.
    ;
    ; Inputs:
    ;   ENTITY_PTR | buyer's entity record
    ;   SHOP_PRICE/+1 | item price
    ;   SHOP_DATA_PTR | pointer to current shop record
    ;   SHOP_ITEM_OFFSET | offset to current item's handler
    ;
    ; Behavior:
    ;   Compares gold (bytes $0D/$0E) against SHOP_PRICE.
    ;   If gold < price, displays "can't afford" and returns.
    ;   Otherwise deducts price and jumps to item effect handler.
    ;
    ; Clobbers: A, X, Y

    ; --- Compare gold against price ---
    LDY     #MFIELD_GOLD+1          ; byte $0E: gold high
    LDA     (ENTITY_PTR),Y
    CMP     SHOP_PRICE+1            ; compare high byte
    BCC     .cant_afford            ; gold < price
    BNE     .can_afford             ; gold > price
    DEY                             ; byte $0D: gold low
    LDA     (ENTITY_PTR),Y
    CMP     SHOP_PRICE              ; compare low byte
    BCS     .can_afford
.cant_afford:
    JSR     SET_CURSOR_ROW21
    LDA     #$31                    ; message: "can't afford"
    JSR     DISPLAY_MESSAGE
    JMP BROWSE_HANDLER                  ; return to shop menu
.can_afford:
    ; --- Deduct price and execute purchase ---
    LDA     SHOP_PRICE              ; \ NUM_VALUE = price
    STA     NUM_VALUE               ;  |
    LDA     SHOP_PRICE+1            ;  |
    STA     NUM_VALUE+1             ; /
    LDY     #$00
    JSR MODIFY_CURRENT_STATS                ; deduct price from gold
    JSR     SET_CURSOR_ROW21
    LDA     #$26                    ; message: "bought"
    JSR     DISPLAY_MESSAGE
    LDY     SHOP_ITEM_OFFSET        ; \ load item effect handler
    LDA     (SHOP_DATA_PTR),Y       ;  | from shop record
    STA     NUM_VALUE               ;  |
    INY                             ;  |
    LDA     (SHOP_DATA_PTR),Y       ;  |
```

```
    STA     NUM_VALUE+1             ; /
    JMP     (NUM_VALUE)             ; execute item effect
```

Defines:
SHOP_BUY_HANDLER, used in chunk 285.
Uses BROWSE_HANDLER 404, DISPLAY_MESSAGE 637, ENTITY_PTR 561, MFIELD_GOLD 561, MODIFY_CURRENT_STATS 382, NUM_VALUE 630a, SET_CURSOR_ROW21 627, SHOP_DATA_PTR 401, SHOP_ITEM_OFFSET 55, and SHOP_PRICE 54c.

CHECK_INPUT polls the keyboard and joystick for input.

Keyboard mode. Reads KBD_DATA; if a key is pressed, compares it against three 5-byte direction key tables (uppercase IJKM+space, lowercase ijkm+CR, and control variants). If matched, returns the direction index (0–4) in A with the N flag clear. Unmatched keys clear the strobe and return with N set.

Joystick mode. If INPUT_MODE is nonzero, first checks the paddle button (PADDL0); if pressed, returns 0 (fire). Otherwise reads both paddles via the ROM PREAD routine (ROM_PREAD), converts each axis to a zone (0=low, 1=center, 2=high) using the threshold tables at JOY_THRESH_LO/JOY_THRESH_HI, and maps the two zones to a direction index (1=up, 2=down, 3=left, 4=right). If both axes are centered, re-checks the paddle button.

Paddle settle delay, used by CHECK_INPUT.

57 ⟨*joy settle delay* 57⟩≡ (909)

```
    ORG     $7AAE
JOY_SETTLE_DELAY:
    DC.B    $C8                     ; paddle settle delay (200)
```

Defines:
JOY_SETTLE_DELAY, used in chunk 58.

58 ⟨*check input* 58⟩≡ (909)

```
    ORG     $755E
CHECK_INPUT:
    SUBROUTINE CHECK_INPUT
    ; Poll keyboard/joystick for directional or fire input.
    ;
    ; Outputs:
    ;   A | direction index (0=fire, 1=up, 2=down, 3=left, 4=right)
    ;       or raw key code for non-direction keys
    ;   N flag | clear if input available, set if none
    ; Clobbers: A, X, Y

    ; --- Check keyboard ---
    LDA     $C000               ; read keyboard register
    BPL     .no_key             ; no key pressed
    TAX                         ; save key code
    LDA     INPUT_MODE
    BNE     .joy_read           ; joystick mode -> read paddles
    TXA                         ; restore key code
    ; --- Match against direction key tables ---
    LDX     #$04                ; 5 entries (index 4..0)
.check_keys:
    CMP     DIR_KEYS_UPPER,X
    BEQ     .found_dir
    CMP     DIR_KEYS_LOWER,X
    BEQ     .found_dir
    CMP     DIR_KEYS_CTRL,X
    BEQ     .found_dir
    DEX
    BPL     .check_keys
    BMI     .no_match           ; no direction key matched
.found_dir:
    TXA                         ; direction index -> A (N clear)
    RTS

    ; --- Direction key lookup tables ---
    ; Index: 0=fire, 1=up, 2=down, 3=left, 4=right
DIR_KEYS_UPPER:
    DC.B    $A0,$C9,$CD,$CA,$CB ; space, I, M, J, K
DIR_KEYS_LOWER:
    DC.B    $8D,$E9,$ED,$EA,$EB ; CR, i, m, j, k
DIR_KEYS_CTRL:
    DC.B    $9B,$8B,$8A,$88,$95 ; ESC, ctrl-K, ctrl-J, ctrl-H, ctrl-U

    ; --- No key ---
.no_key:
    LDA     INPUT_MODE
    BNE     .joy_read           ; joystick mode -> read paddles
.no_match:
    LDA     #$FF                ; \ clear keyboard strobe
```

```
    STA     $C010               ; / (A=$FF -> N set = no input)
    RTS
.check_button:
    LDA     $C061               ; read paddle button
    BPL     .no_match           ; not pressed -> no input
    LDA     #$00                ; button = fire (index 0, N clear)
    RTS

    ; --- Joystick axis reading ---
.joy_read:
    JSR     JOY_DELAY           ; wait for paddle settle
    LDX     #$00                ; paddle 0 (horizontal)
    JSR     $FB1E               ; PREAD: Y = paddle 0 value
    LDA     JOY_AXIS_X          ; \ map X-axis value to zone
    JSR     JOY_TO_ZONE         ;  |
    STA     JOY_AXIS_X          ; / store zone (0/1/2)
    JSR     JOY_DELAY
    LDX     #$01                ; paddle 1 (vertical)
    JSR     $FB1E               ; PREAD: Y = paddle 1 value
    LDA     JOY_AXIS_Y          ; \ map Y-axis value to zone
    JSR     JOY_TO_ZONE         ;  |
    STA     JOY_AXIS_Y          ; / store zone (0/1/2)
    ; --- Map zones to direction ---
    CMP     #$01
    BEQ     .y_center           ; Y center -> check X
    BMI     .y_low              ; Y low -> up candidate
    ; Y high (zone 2) -> check X for disambiguation
    LDA     JOY_AXIS_X
    CMP     #$01
    BNE     .no_dir             ; X not center -> ambiguous
    LDX     #$01                ; X center + Y high = up
.joy_dir:
    JMP     .found_dir
.y_low:
    LDA     JOY_AXIS_X
    CMP     #$01
    BNE     .no_dir             ; X not center -> ambiguous
    LDX     #$02                ; X center + Y low = down
    BNE     .joy_dir            ; always -> relay to .found_dir
.y_center:
    LDA     JOY_AXIS_X
    CMP     #$01
    BEQ     .no_dir             ; both center -> no direction
    BPL     .x_high
    LDX     #$04                ; X low + Y center = right
    BNE     .joy_dir
.x_high:
    LDX     #$03                ; X high + Y center = left
    BNE     .joy_dir
.no_dir:
```

```
    JMP     .check_button      ; no direction -> check fire button

    ; --- Joystick delay ---
JOY_DELAY:
    LDA     JOY_SETTLE_DELAY   ; paddle settle delay
    JSR     ROM_WAIT
    RTS

    ; --- Joystick axis threshold data ---
JOY_AXIS_X:
    DC.B    $01                ; X-axis zone (runtime storage)
JOY_AXIS_Y:
    DC.B    $01                ; Y-axis zone (runtime storage)
JOY_THRESH_LO:
    DC.B    $1A,$1D,$1D        ; low thresholds (horiz, vert, vert)
JOY_THRESH_HI:
    DC.B    $03,$03,$06        ; high thresholds (horiz, vert, vert)

    ; --- Map paddle value to zone (0=low, 1=center, 2=high) ---
JOY_TO_ZONE:
    TAX                        ; save old zone
    TYA                        ; A = paddle value (from PREAD)
    LSR                        ; \ divide by 8
    LSR                        ;  |
    LSR                        ; /
    CMP     JOY_THRESH_LO,X   ; above low threshold?
    BCS     .zone_hi_check     ; yes -> check high
    CMP     JOY_THRESH_HI,X   ; above high threshold?
    BCS     .zone_center       ; yes -> center
    LDA     #$02               ; below both = high zone
    RTS
.zone_center:
    LDA     #$01               ; between thresholds = center
    RTS
.zone_hi_check:
    LDA     #$00               ; above low threshold = low zone
    RTS
```

Defines:
CHECK_INPUT, used in chunk 663.
DIR_KEYS_CTRL, never used.
DIR_KEYS_LOWER, never used.
DIR_KEYS_UPPER, never used.
JOY_AXIS_X, never used.
JOY_AXIS_Y, never used.
JOY_DELAY, never used.
JOY_THRESH_HI, never used.
JOY_THRESH_LO, never used.
JOY_TO_ZONE, never used.
Uses INPUT_MODE 589c, JOY_SETTLE_DELAY 57, and ROM_WAIT 606b.

Dead code: loops through character records starting from CURRENT_PLAYER, clearing bit 7 of byte 5 (MFIELD_STRENGTH) on each record until the end sentinel is reached. Zero callers.

61a ⟨*clear strength flags* 61a⟩≡ (909)

```
        ORG     $793F
    CLEAR_STRENGTH_FLAGS:
        ; Dead code: clear bit 7 of MFIELD_STRENGTH on all characters.
    .loop:
        LDA     CURRENT_PLAYER
        JSR     GET_CHAR_RECORD         ; CHAR_REC = record for index A
        LDY     #$01
        LDA     (CHAR_REC),Y            ; byte 1: type
        CMP     #$FF                    ; end sentinel?
        BNE     .not_end
        RTS
    .not_end:
        LDY     #MFIELD_STRENGTH
        LDA     (CHAR_REC),Y            ; byte 5: strength
        AND     #$7F                    ; clear bit 7
        STA     (CHAR_REC),Y
        INC     CURRENT_PLAYER
        BNE     .loop                   ; always taken
```

Defines:
CLEAR_STRENGTH_FLAGS, never used.
Uses CHAR_REC 25a, CURRENT_PLAYER 562a, GET_CHAR_RECORD 26, and MFIELD_STRENGTH 561.

61b ⟨*tune bytecode data tail* 61b⟩≡ (909)

```
        ORG     $80E5
        HEX     80 02 23 C1 04 00 24 C1 04 00 24 09 A4 32 A4 37
        HEX     A4 FC A3 3C A4 24 1F 26 23 C1 05 22 50 06 3F 08
        HEX     25 23 60 10 00 24 25 1F 22 3F 05 50 04 80 02 23
        HEX     C1 04 00 24 81 C8 E7 6D 9D 81 28 EB 6D A8 81 64
        HEX     EF 6D BB 81 08 FF ED ED EF E6 FD ED EF
```

Dead code: original HRCG entry point, bypassed by INIT_HRCG patches.

61c ⟨*hrcg original entry* 61c⟩≡ (909)

```
        ORG     $92A5
    HRCG_ORIGINAL_ENTRY:
        JMP     HRCG_PRINT_VERSION      ; dead: original entry before patch
```

Defines:
HRCG_ORIGINAL_ENTRY, never used.
Uses HRCG_PRINT_VERSION 736.

4.3.3 Equipment and combat data

This region contains interleaved lookup tables and string data used by the shop, combat, and entity display systems.

Shop table ($7EDD–$7F2E). Two SHOP_RECORD_SIZE-byte ($29 = 41 bytes) shop records, selected by DISPLAY_SHOP based on the entity's type byte (byte 1 of ENTITY_REC, minus $D5). Record 0 is the weapon shop; record 1 is the armor shop.

Each record contains five 5-byte item slots at offsets $00, $05, $0A, $0F, and $14. Each slot holds a 2-byte buy-effect handler address, a 2-byte item name string pointer, and a 1-byte price-per-unit value. The entity's bitmask (byte 2, stored in SHOP_VALID_MASK) determines which slots are displayed; unused slots contain display format data. Bytes $19–$27 hold additional items or display format strings. Byte $28 is the shop scene number (passed to SCENE_LOOP when entering the shop).

Item name strings ($7F2F–$7FBD). Ten length-prefixed high-ASCII item name strings (5 weapons, 5 armor), interleaved with the SHOP_EFFECT_TABLE (5 bytes at $7F6D) and the XP_AWARD_TABLE (5 bytes at XP_AWARD_TABLE). The substring "CHAIN MAIL" at $7FA9 is referenced directly by 6 callers in CLASSIFY_ENTITY_TYPE ($1178–$11ED).

Map object names ($7FC3–$8039). Eight length-prefixed strings for map tile descriptions: A WALL, A STATUE, A TREASURE CHEST, EMPTY SPACE, AN ILLUSION, A DOOR, AN INVISIBLE WALL!, SOME RUNE WRITING, A TRADING OUTPOST. The string at $8003 ("AN INVISIBLE WALL!") is referenced from $A3A7.

Attack verb strings ($803A–$80A3). Fourteen length-prefixed combat verb strings, indexed by damage level (low to high): JABS, POKES, HITS, CLOUTS, POUNDS, SMACKS, SMITES, WHOPS, WHACKS, BASHES, WHOMPS, WALLOPS, SMASHES, CLOBBERS, LAMBASTES. The table base at $807D is referenced from the attack display routine at $7E2E.

62 ⟨*item name strings* 62⟩≡ (909)

```
    ORG     $7F2F
S_IRON_SWORD:
    APSTR   "AN IRON SWORD"
S_STEEL_SWORD:
    APSTR   "A STEEL SWORD"
S_OAK_CLUB:
```

```
    APSTR   "AN OAK CLUB"
S_STEEL_DAGGER:
    APSTR   "A STEEL DAGGER"
S_SHIV:
    APSTR   "A SHIV"
SHOP_EFFECT_TABLE:
    ; Base effect values by item type (5 bytes)
    HEX     07 88 84 83 06
S_PLATE_ARMOR:
    APSTR   "PLATE ARMOR"
S_RITHMIL_PLATE:
    APSTR   "RITHMIL PLATE ARMOR"
S_CHAIN_MAIL:
    APSTR   "CHAIN MAIL"
S_RITHMIL_CHAIN:
    APSTR   "RITHMIL CHAIN MAIL"
S_LEATHER_ARMOR:
    APSTR   "LEATHER ARMOR"
XP_AWARD_TABLE:
    ; XP by action type (5 bytes)
    HEX     75 35 53 23 21
```

Defines:
SHOP_EFFECT_TABLE, used in chunk 409.
S_CHAIN_MAIL, used in chunk 402.
S_IRON_SWORD, used in chunk 402.
S_LEATHER_ARMOR, used in chunk 402.
S_OAK_CLUB, used in chunk 402.
S_PLATE_ARMOR, used in chunk 402.
S_RITHMIL_CHAIN, used in chunk 402.
S_RITHMIL_PLATE, used in chunk 402.
S_SHIV, used in chunk 402.
S_STEEL_DAGGER, used in chunk 402.
S_STEEL_SWORD, used in chunk 402.
XP_AWARD_TABLE, used in chunk 412.

63 ⟨*map object names* 63⟩≡ (909)

```
    ORG     $7FC3
MAP_OBJECT_NAMES:
    APSTR   "A WALL"
    APSTR   "A STATUE"
    APSTR   "A TREASURE CHEST"
    APSTR   "EMPTY SPACE"
    APSTR   "AN ILLUSION"
    APSTR   "A DOOR"
    APSTR   "AN INVISIBLE WALL!"
    APSTR   "SOME RUNE WRITING"
    APSTR   "A TRADING OUTPOST"
```

Defines:
MAP_OBJECT_NAMES, never used.

4.3.4 Duplicate item/combat strings

The region $8133–$82B4 is a second copy of the equipment and combat string data, used by the "See Your Lot" display and entity classification routines.

HRCG control prefix ($8133–$8138). 6 bytes of HRCG font rendering control data preceding the string table.

Duplicate string table ($8139–$82AE). Byte-for-byte copy of $7F2E–$80A3 (weapon names, shop effect table, armor names, XP award table, map object names, attack verb strings). The address $8154 is written from $4681 (entity setup), and $82A5 is tested via BIT from $A29E. The attack verb subtable at $82AD is indexed from $7162 and $71CE (combat display).

HRCG control tail ($82AF–$82B4). 6 bytes of HRCG control string continuation.

64 ⟨*hrcg control prefix* 64⟩≡ (909)

```
      ORG     $8133
  HRCG_CTRL_PREFIX:
      HEX     F7 FD ED E6 ED FD
```

Defines:
HRCG_CTRL_PREFIX, never used.

65 ⟨*duplicate item strings* 65⟩≡ (909)

```
    ORG     $8139
ITEM_STRINGS_COPY:
    ; Duplicate of $7F2E-$80A3 (374 bytes)
    ; Weapon names
    DC.B    $33
    APSTR   "AN IRON SWORD"
    APSTR   "A STEEL SWORD"
    APSTR   "AN OAK CLUB"
    APSTR   "A STEEL DAGGER"
    APSTR   "A SHIV"
    ; Shop effect table (5 bytes)
    HEX     07 88 84 83 06
    ; Armor names
    APSTR   "PLATE ARMOR"
    APSTR   "RITHMIL PLATE ARMOR"
    APSTR   "CHAIN MAIL"
    APSTR   "RITHMIL CHAIN MAIL"
    APSTR   "LEATHER ARMOR"
    ; XP award table (5 bytes)
    HEX     75 35 53 23 21
    ; Map object names
    APSTR   "A WALL"
    APSTR   "A STATUE"
    APSTR   "A TREASURE CHEST"
    APSTR   "EMPTY SPACE"
    APSTR   "AN ILLUSION"
    APSTR   "A DOOR"
    APSTR   "AN INVISIBLE WALL!"
    APSTR   "SOME RUNE WRITING"
    APSTR   "A TRADING OUTPOST"
    ; Attack verb strings
    APSTR   "JABS"
    APSTR   "POKES"
    APSTR   "HITS"
    APSTR   "CLOUTS"
    APSTR   "POUNDS"
    APSTR   "SMACKS"
    APSTR   "SMITES"
    APSTR   "WHOPS"
    APSTR   "WHACKS"
    APSTR   "BASHES"
    APSTR   "WHOMPS"
    APSTR   "WALLOPS"
    APSTR   "SMASHES"
    APSTR   "CLOBBERS"
    APSTR   "LAMBASTES"
```

Defines:
ITEM_STRINGS_COPY, never used.

66a ⟨*hrcg control tail* 66a⟩≡ (909)

```
    ORG     $82AF
HRCG_CTRL_TAIL:
    HEX     09 81 B1 82 DB FF
```

Defines:
HRCG_CTRL_TAIL, never used.

4.3.5 HRCG data and border bitmaps

HRCG control continuation ($82B5–$82BE). 10 bytes of HRCG mode/address control data.

Tune pointer table copy ($82BF–$82D0). Duplicate of the tune pointer table at TUNE_TABLE, mapping tune indices to bytecode addresses in the $A3xx–$A4xx range.

Tune bytecodes copy ($82D1–$82EF, $82F0–$8305). Two copies of tune scripts 1–2 (combat hit / combat damage), followed by a second tune pointer fragment.

Border/icon bitmap data ($831A–$83A4). 139 bytes of hi-res bitmap patterns for border decorations and map icons. The address $838D is indexed from $7101 (border rendering).

66b ⟨*hrcg control continuation* 66b⟩≡ (909)

```
    ORG     $82B5
HRCG_CTRL_CONT:
    HEX     83 ED ED 84 D1 82 D2 82 E2 82
```

Defines:
HRCG_CTRL_CONT, never used.

66c ⟨*tune table copy* 66c⟩≡ (909)

```
    ORG     $82BF
TUNE_TABLE_COPY:
    ; Duplicate tune pointer table (9 entries)
    HEX     00 A3 C3 A3 CE A3 DE A3 09 A4 32 A4 37 A4 FC A3
    HEX     3C A4
```

Defines:
TUNE_TABLE_COPY, never used.

67a ⟨*tune bytecodes copy* 67a⟩≡ (909)

```
        ORG     $82D1
    TUNE_BYTECODES_COPY:
        ; Scripts 1-2 (combat hit, combat damage) | copy 1
        HEX     24 1F 26 23 C1 05 22 50 06 3F 08 25 23 60 10 00
        HEX     24 25 1F 22 3F 05 50 04 80 02 23 C1 04 00 24
        ; Tune pointer fragment + scripts 1-2 | copy 2
        HEX     A3 09 A4 32 A4 37 A4 FC A3 3C A4
        HEX     24 1F 26 23 C1 05 22 50 06 3F 08 25 23 60 10 00
        HEX     24 25 1F 22 3F 05 50 04 80 02 23 C1 04 00 24
```

Defines:
TUNE_BYTECODES_COPY, never used.

67b ⟨*border bitmap data* 67b⟩≡ (909)

```
        ORG     $831A
    BORDER_BITMAP_DATA:
        ; Hi-res bitmap patterns for border decorations and map icons
        HEX     2A 2A 2A 2A 2A 50 54 44 44 54 50 40 40 02 02 08
        HEX     08 02 02 00 00 41 55 40 40 54 04 04 05 20 2A 00
        HEX     00 0A 08 08 28 40 40 50 50 50 54 54 54 00 00 02
        HEX     02 02 0A 0A 0A AA A2 A2 AA 88 A8 A0 00 95 91 91
        HEX     95 84 85 81 00 00 84 84 85 95 95 85 84 00 00 00
        HEX     00 00 05 05 05 84 7F C4 C4 00 40 60 00 0F 1F 3F
        HEX     1F 0F 19 30 00 00 00 00 00 00 00 D0 D0 00 00 00
        HEX     00 00 00 00 00 A0 A0 A8 A0 A0 D0 88 00 00 00 81
        HEX     80 00 00 81 00 2A 2A 2A 2A 2A 2A
```

Defines:
BORDER_BITMAP_DATA, never used.

4.4 Mob data records

Each record is 16 bytes. GET_MOB_DATA indexes into the table via the base pointer at MOB_BASE_LO/MOB_BASE_HI.

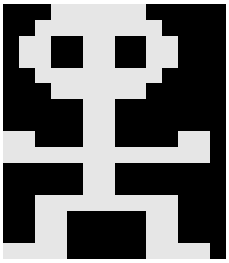
Offset	Field	Description
$00–$01	MFIELD_NAME	16-bit pointer to name string (DC.W)
$02	Next link	Index of next mob in group chain
$03	Position	Map position (row × 20 + col)
$04	Appearance	Font character index (÷ 21 = group)
$05	Strength	Bits 4–0 = strength / defense
$06	MFIELD_HP	Bits 5–0 = hit points, bits 7–6 = flags
$07	Attack	Attack rating
$08	XP / group	Bit 7 = status, 6–4 = pending XP, 3–0 = group count
$09	Defense	Defense rating
$0A	(unused)	
$0B	Level	Bits 7–5 = faction, 4–0 = level (0–31)
$0C	Level encoding	Hi nibble = XP multiplier, lo = base
$0D–$0E	MFIELD_GOLD	16-bit gold counter, ducats (DC.W)
$0F	Flags	Status flags (bit 2 = combat engaged)

Appearance groups. APPEARANCE_TO_FONTCHAR divides the appearance index by 21 to determine the font group. Groups 0–2 and 7 are friendly; groups 3–6 are hostile.

68 ⟨*mob data records* 68⟩≡ (909) 69 ▷

```
        ORG     $4C80
MOB_DATA_TABLE:
```

Uses MOB_DATA_TABLE 196.

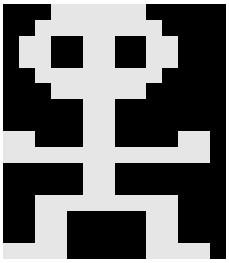


ALI BABA (font character 1, group 0 — friendly)

69 ⟨*mob data records* 68⟩+≡ (909) ◁68 70▷

```
; --- mob 0: ALI BABA ($4C80) ---
DC.W    s_ALI_BABA              ; +MFIELD_NAME
DC.B    $00               ; +MFIELD_NEXT (0)
DC.B    $AB               ; +MFIELD_POSITION (row 8, col 11)
DC.B    $00               ; +MFIELD_APPEARANCE (fontchar=1, group=0 friendly)
DC.B    $0C               ; +MFIELD_STRENGTH (12)
DC.B    $08               ; +MFIELD_HP (8)
DC.B    $01               ; +MFIELD_ATTACK (1)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $00               ; +MFIELD_DEFENSE (0)
DC.B    $00               ; +$0A
DC.B    $10               ; +MFIELD_LEVEL (faction=0 level=16)
DC.B    $08               ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_ALI_BABA 197.

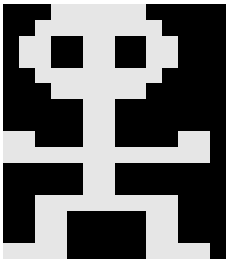


SCHEHERAZADE (font character 1, group 0)

70 ⟨*mob data records* 68⟩+≡ (909) ◁69 71▷

```
; --- mob 1: SCHEHERAZADE ($4C90) ---
DC.W    s_SCHEHERAZADE              ; +MFIELD_NAME
DC.B    $08                ; +MFIELD_NEXT (8)
DC.B    $00                ; +MFIELD_POSITION (unplaced)
DC.B    $00                ; +MFIELD_APPEARANCE (fontchar=1, group=0 friendly)
DC.B    $08                ; +MFIELD_STRENGTH (8)
DC.B    $08                ; +MFIELD_HP (8)
DC.B    $05                ; +MFIELD_ATTACK (5)
DC.B    $00                ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $04                ; +MFIELD_DEFENSE (4)
DC.B    $00                ; +$0A
DC.B    $14                ; +MFIELD_LEVEL (faction=0 level=20)
DC.B    $09                ; +MFIELD_LEVEL_ENC (xpmult=0, base=9)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_SCHEHERAZADE 197.

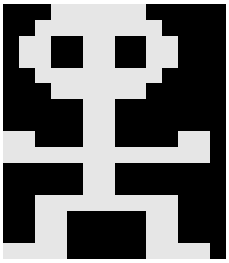


HAROUD EL-LARGE (font character 1, group 0)

71 ⟨*mob data records* 68⟩+≡ (909) ◁70 72▷

```
; --- mob 2: HAROUD EL-LARGE ($4CA0) ---
DC.W    s_HAROUD_EL_LARGE              ; +MFIELD_NAME
DC.B    $07                ; +MFIELD_NEXT (7)
DC.B    $00                ; +MFIELD_POSITION (unplaced)
DC.B    $00                ; +MFIELD_APPEARANCE (fontchar=1, group=0 friendly)
DC.B    $12                ; +MFIELD_STRENGTH (18)
DC.B    $12                ; +MFIELD_HP (18)
DC.B    $09                ; +MFIELD_ATTACK (9)
DC.B    $00                ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $06                ; +MFIELD_DEFENSE (6)
DC.B    $00                ; +$0A
DC.B    $0A                ; +MFIELD_LEVEL (faction=0 level=10)
DC.B    $07                ; +MFIELD_LEVEL_ENC (xpmult=0, base=7)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_HAROUD_EL_LARGE 197.

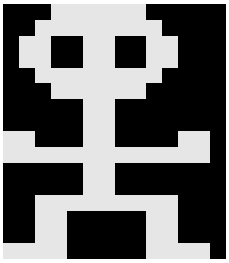


BAIRAM MEDIO (font character 1, group 0)

72 ⟨*mob data records* 68⟩+≡ (909) ◁71 73▷

```
; --- mob 3: BAIRAM MEDIO ($4CB0) ---
DC.W    s_BAIRAM_MEDIO              ; +MFIELD_NAME
DC.B    $09                 ; +MFIELD_NEXT (9)
DC.B    $00                 ; +MFIELD_POSITION (unplaced)
DC.B    $00                 ; +MFIELD_APPEARANCE (fontchar=1, group=0 friendly)
DC.B    $0E                 ; +MFIELD_STRENGTH (14)
DC.B    $0E                 ; +MFIELD_HP (14)
DC.B    $06                 ; +MFIELD_ATTACK (6)
DC.B    $00                 ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $05                 ; +MFIELD_DEFENSE (5)
DC.B    $00                 ; +$0A
DC.B    $0E                 ; +MFIELD_LEVEL (faction=0 level=14)
DC.B    $08                 ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0000                 ; +MFIELD_GOLD
DC.B    $00                 ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_BAIRAM_MEDIO 197.

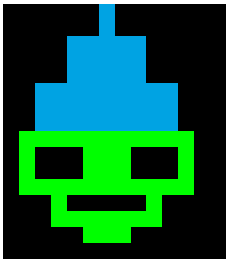


ABOU HASSAN (font character 1, group 0)

73 ⟨*mob data records* 68⟩+≡ (909) ◁72 74▷

```
; --- mob 4: ABOU HASSAN ($4CC0) ---
DC.W    s_ABOU_HASSAN              ; +MFIELD_NAME
DC.B    $06                ; +MFIELD_NEXT (6)
DC.B    $00                ; +MFIELD_POSITION (unplaced)
DC.B    $00                ; +MFIELD_APPEARANCE (fontchar=1, group=0 friendly)
DC.B    $0D                ; +MFIELD_STRENGTH (13)
DC.B    $0D                ; +MFIELD_HP (13)
DC.B    $07                ; +MFIELD_ATTACK (7)
DC.B    $32                ; +MFIELD_XP (st=0 xp=3 grp=2)
DC.B    $04                ; +MFIELD_DEFENSE (4)
DC.B    $00                ; +$0A
DC.B    $0B                ; +MFIELD_LEVEL (faction=0 level=11)
DC.B    $06                ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_ABOU_HASSAN 197.

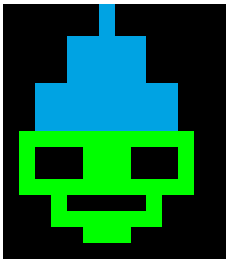


LUTHIEN (font character 2, group 0 — friendly)

74 ⟨*mob data records* 68⟩+≡ (909) ◁73 75▷

```
; --- mob 5: LUTHIEN ($4CD0) ---
DC.W    s_LUTHIEN               ; +MFIELD_NAME
DC.B    $0C               ; +MFIELD_NEXT (12)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $01               ; +MFIELD_APPEARANCE (fontchar=2, group=0 friendly)
DC.B    $06               ; +MFIELD_STRENGTH (6)
DC.B    $09               ; +MFIELD_HP (9)
DC.B    $04               ; +MFIELD_ATTACK (4)
DC.B    $01               ; +MFIELD_XP (st=0 xp=0 grp=1)
DC.B    $05               ; +MFIELD_DEFENSE (5)
DC.B    $00               ; +$0A
DC.B    $54               ; +MFIELD_LEVEL (faction=2 level=20)
DC.B    $0B               ; +MFIELD_LEVEL_ENC (xpmult=0, base=11)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_LUTHIEN 197.

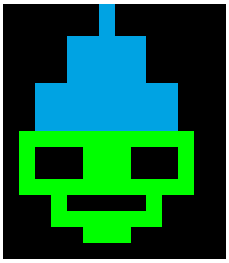


HUAN (font character 2, group 0)

75 ⟨*mob data records* 68⟩+≡ (909) ◁74 76▷

```
; --- mob 6: HUAN ($4CE0) ---
DC.W    s_HUAN              ; +MFIELD_NAME
DC.B    $0D               ; +MFIELD_NEXT (13)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $01               ; +MFIELD_APPEARANCE (fontchar=2, group=0 friendly)
DC.B    $10               ; +MFIELD_STRENGTH (16)
DC.B    $10               ; +MFIELD_HP (16)
DC.B    $07               ; +MFIELD_ATTACK (7)
DC.B    $01               ; +MFIELD_XP (st=0 xp=0 grp=1)
DC.B    $08               ; +MFIELD_DEFENSE (8)
DC.B    $00               ; +$0A
DC.B    $4A               ; +MFIELD_LEVEL (faction=2 level=10)
DC.B    $09               ; +MFIELD_LEVEL_ENC (xpmult=0, base=9)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_HUAN 197.

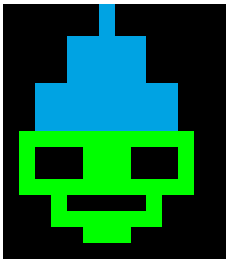


CELEGORM (font character 2, group 0)

76 ⟨*mob data records* 68⟩+≡ (909) ◁75 77▷

```
; --- mob 7: CELEGORM ($4CF0) ---
DC.W    s_CELEGORM              ; +MFIELD_NAME
DC.B    $0B             ; +MFIELD_NEXT (11)
DC.B    $00             ; +MFIELD_POSITION (unplaced)
DC.B    $01             ; +MFIELD_APPEARANCE (fontchar=2, group=0 friendly)
DC.B    $0D             ; +MFIELD_STRENGTH (13)
DC.B    $0D             ; +MFIELD_HP (13)
DC.B    $06             ; +MFIELD_ATTACK (6)
DC.B    $01             ; +MFIELD_XP (st=0 xp=0 grp=1)
DC.B    $07             ; +MFIELD_DEFENSE (7)
DC.B    $00             ; +$0A
DC.B    $4D             ; +MFIELD_LEVEL (faction=2 level=13)
DC.B    $0A             ; +MFIELD_LEVEL_ENC (xpmult=0, base=10)
DC.W    $0000             ; +MFIELD_GOLD
DC.B    $00             ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_CELEGORM 197.

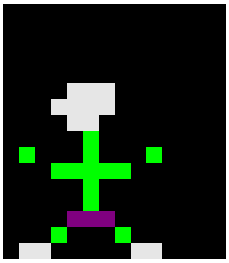


CURUFIN (font character 2, group 0)

77 ⟨*mob data records* 68⟩+≡ (909) ◁76 78▷

```
; --- mob 8: CURUFIN ($4D00) ---
DC.W    s_CURUFIN               ; +MFIELD_NAME
DC.B    $0A               ; +MFIELD_NEXT (10)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $01               ; +MFIELD_APPEARANCE (fontchar=2, group=0 friendly)
DC.B    $0C               ; +MFIELD_STRENGTH (12)
DC.B    $0C               ; +MFIELD_HP (12)
DC.B    $05               ; +MFIELD_ATTACK (5)
DC.B    $23               ; +MFIELD_XP (st=0 xp=2 grp=3)
DC.B    $06               ; +MFIELD_DEFENSE (6)
DC.B    $00               ; +$0A
DC.B    $4B               ; +MFIELD_LEVEL (faction=2 level=11)
DC.B    $08               ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_CURUFIN 197.

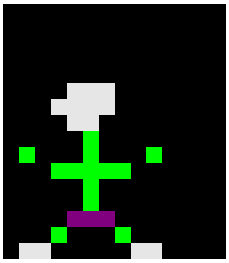


ELDO (font character 4, group 0 — friendly)

78 ⟨*mob data records* 68⟩+≡ (909) ◁77 79▷

```
; --- mob 9: ELDO ($4D10) ---
DC.W    s_ELDO              ; +MFIELD_NAME
DC.B    $11               ; +MFIELD_NEXT (17)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $03               ; +MFIELD_APPEARANCE (fontchar=4, group=0 friendly)
DC.B    $06               ; +MFIELD_STRENGTH (6)
DC.B    $16               ; +MFIELD_HP (22)
DC.B    $03               ; +MFIELD_ATTACK (3)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $03               ; +MFIELD_DEFENSE (3)
DC.B    $00               ; +$0A
DC.B    $54               ; +MFIELD_LEVEL (faction=2 level=20)
DC.B    $08               ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_ELDO 197.

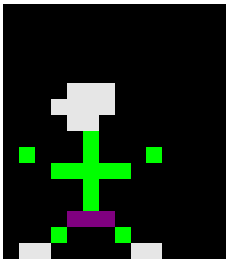


MERRYWATER (font character 4, group 0)

79 ⟨*mob data records* 68⟩+≡ (909) ◁78 80▷

```
; --- mob 10: MERRYWATER ($4D20) ---
DC.W    s_MERRYWATER              ; +MFIELD_NAME
DC.B    $0E               ; +MFIELD_NEXT (14)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $03               ; +MFIELD_APPEARANCE (fontchar=4, group=0 friendly)
DC.B    $08               ; +MFIELD_STRENGTH (8)
DC.B    $18               ; +MFIELD_HP (24)
DC.B    $04               ; +MFIELD_ATTACK (4)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $04               ; +MFIELD_DEFENSE (4)
DC.B    $00               ; +$0A
DC.B    $52               ; +MFIELD_LEVEL (faction=2 level=18)
DC.B    $08               ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_MERRYWATER 197.

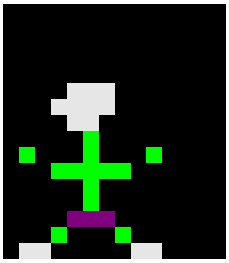


STILBO (font character 4, group 0)

80 ⟨*mob data records* 68⟩+≡ (909) ◁79 81▷

```
; --- mob 11: STILBO ($4D30) ---
DC.W    s_STILBO              ; +MFIELD_NAME
DC.B    $10               ; +MFIELD_NEXT (16)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $03               ; +MFIELD_APPEARANCE (fontchar=4, group=0 friendly)
DC.B    $0A               ; +MFIELD_STRENGTH (10)
DC.B    $1A               ; +MFIELD_HP (26)
DC.B    $05               ; +MFIELD_ATTACK (5)
DC.B    $21               ; +MFIELD_XP (st=0 xp=2 grp=1)
DC.B    $05               ; +MFIELD_DEFENSE (5)
DC.B    $00               ; +$0A
DC.B    $4F               ; +MFIELD_LEVEL (faction=2 level=15)
DC.B    $07               ; +MFIELD_LEVEL_ENC (xpmult=0, base=7)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_STILBO 197.

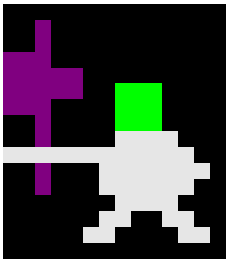


CINDER (font character 4, group 0)

81 ⟨*mob data records* 68⟩+≡ (909) ◁80 82▷

```
; --- mob 12: CINDER ($4D40) ---
DC.W    s_CINDER              ; +MFIELD_NAME
DC.B    $0F              ; +MFIELD_NEXT (15)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $03              ; +MFIELD_APPEARANCE (fontchar=4, group=0 friendly)
DC.B    $09              ; +MFIELD_STRENGTH (9)
DC.B    $19              ; +MFIELD_HP (25)
DC.B    $05              ; +MFIELD_ATTACK (5)
DC.B    $32              ; +MFIELD_XP (st=0 xp=3 grp=2)
DC.B    $05              ; +MFIELD_DEFENSE (5)
DC.B    $00              ; +$0A
DC.B    $4F              ; +MFIELD_LEVEL (faction=2 level=15)
DC.B    $06              ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $00              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_CINDER 197.

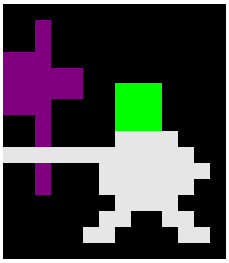


JATTE (font character 3, group 0 — friendly)

82 ⟨*mob data records* 68⟩+≡ (909) ◁81 83▷

```
; --- mob 13: JATTE ($4D50) ---
DC.W    s_JATTE              ; +MFIELD_NAME
DC.B    $04              ; +MFIELD_NEXT (4)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $02              ; +MFIELD_APPEARANCE (fontchar=3, group=0 friendly)
DC.B    $14              ; +MFIELD_STRENGTH (20)
DC.B    $1A              ; +MFIELD_HP (26)
DC.B    $0A              ; +MFIELD_ATTACK (10)
DC.B    $00              ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $05              ; +MFIELD_DEFENSE (5)
DC.B    $00              ; +$0A
DC.B    $08              ; +MFIELD_LEVEL (faction=0 level=8)
DC.B    $05              ; +MFIELD_LEVEL_ENC (xpmult=0, base=5)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $00              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_JATTE 197.

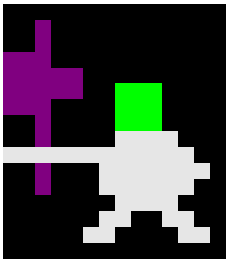


THORA (font character 3, group 0)

83 ⟨*mob data records* 68⟩+≡ (909) ◁82 84▷

```
; --- mob 14: THORA ($4D60) ---
DC.W    s_THORA              ; +MFIELD_NAME
DC.B    $1D              ; +MFIELD_NEXT (29)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $02              ; +MFIELD_APPEARANCE (fontchar=3, group=0 friendly)
DC.B    $11              ; +MFIELD_STRENGTH (17)
DC.B    $15              ; +MFIELD_HP (21)
DC.B    $09              ; +MFIELD_ATTACK (9)
DC.B    $21              ; +MFIELD_XP (st=0 xp=2 grp=1)
DC.B    $04              ; +MFIELD_DEFENSE (4)
DC.B    $00              ; +$0A
DC.B    $0A              ; +MFIELD_LEVEL (faction=0 level=10)
DC.B    $06              ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $00              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_THORA 197.

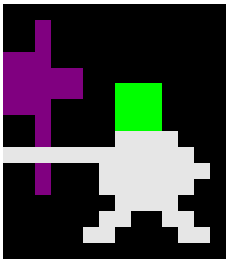


OG (font character 3, group 0)

84 ⟨*mob data records* 68⟩+≡ (909) ◁83 85▷

```
; --- mob 15: OG ($4D70) ---
DC.W    s_OG                ; +MFIELD_NAME
DC.B    $02               ; +MFIELD_NEXT (2)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $02               ; +MFIELD_APPEARANCE (fontchar=3, group=0 friendly)
DC.B    $0D               ; +MFIELD_STRENGTH (13)
DC.B    $12               ; +MFIELD_HP (18)
DC.B    $07               ; +MFIELD_ATTACK (7)
DC.B    $32               ; +MFIELD_XP (st=0 xp=3 grp=2)
DC.B    $03               ; +MFIELD_DEFENSE (3)
DC.B    $00               ; +$0A
DC.B    $0D               ; +MFIELD_LEVEL (faction=0 level=13)
DC.B    $05               ; +MFIELD_LEVEL_ENC (xpmult=0, base=5)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_OG 197.

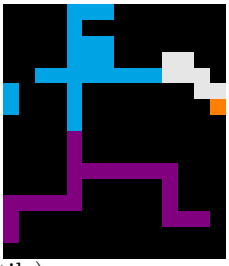


ULVA (font character 3, group 0)

85 ⟨*mob data records* 68⟩+≡ (909) ◁84 86▷

```
; --- mob 16: ULVA ($4D80) ---
DC.W    s_ULVA              ; +MFIELD_NAME
DC.B    $03               ; +MFIELD_NEXT (3)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $02               ; +MFIELD_APPEARANCE (fontchar=3, group=0 friendly)
DC.B    $10               ; +MFIELD_STRENGTH (16)
DC.B    $14               ; +MFIELD_HP (20)
DC.B    $08               ; +MFIELD_ATTACK (8)
DC.B    $43               ; +MFIELD_XP (st=0 xp=4 grp=3)
DC.B    $04               ; +MFIELD_DEFENSE (4)
DC.B    $00               ; +$0A
DC.B    $0A               ; +MFIELD_LEVEL (faction=0 level=10)
DC.B    $04               ; +MFIELD_LEVEL_ENC (xpmult=0, base=4)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_ULVA 197.

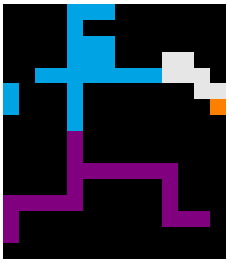


COGIA HOUSSAIN (font character 5, group 3 — hostile)

86 ⟨*mob data records* 68⟩+≡ (909) ◁85 87▷

```
; --- mob 17: COGIA HOUSSAIN ($4D90) ---
DC.W    s_COGIA_HOUSSAIN              ; +MFIELD_NAME
DC.B    $00               ; +MFIELD_NEXT (0)
DC.B    $84               ; +MFIELD_POSITION (row 6, col 12)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $10               ; +MFIELD_STRENGTH (16)
DC.B    $10               ; +MFIELD_HP (16)
DC.B    $09               ; +MFIELD_ATTACK (9)
DC.B    $03               ; +MFIELD_XP (st=0 xp=0 grp=3)
DC.B    $07               ; +MFIELD_DEFENSE (7)
DC.B    $00               ; +$0A
DC.B    $4E               ; +MFIELD_LEVEL (faction=2 level=14)
DC.B    $00               ; +MFIELD_LEVEL_ENC (xpmult=0, base=0)
DC.W    $00C8               ; +MFIELD_GOLD (200 ducats)
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_COGIA_HOUSSAIN 197.

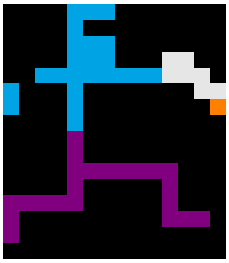


OMAR (font character 5, group 3)

87 ⟨*mob data records* 68⟩+≡ (909) ◁86 88▷

```
; --- mob 18: OMAR ($4DA0) ---
DC.W    s_OMAR              ; +MFIELD_NAME
DC.B    $00               ; +MFIELD_NEXT (0)
DC.B    $70               ; +MFIELD_POSITION (row 5, col 12)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $10               ; +MFIELD_STRENGTH (16)
DC.B    $10               ; +MFIELD_HP (16)
DC.B    $08               ; +MFIELD_ATTACK (8)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $09               ; +MFIELD_DEFENSE (9)
DC.B    $00               ; +$0A
DC.B    $0A               ; +MFIELD_LEVEL (faction=0 level=10)
DC.B    $07               ; +MFIELD_LEVEL_ENC (xpmult=0, base=7)
DC.W    $000F               ; +MFIELD_GOLD (15 ducats)
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_OMAR 197.

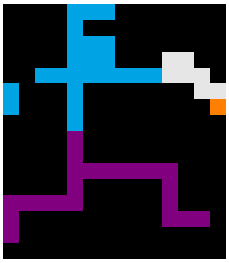


MAHMUD (font character 5, group 3)

88 ⟨*mob data records* 68⟩+≡ (909) ◁87 89▷

```
; --- mob 19: MAHMUD ($4DB0) ---
DC.W    s_MAHMUD              ; +MFIELD_NAME
DC.B    $13              ; +MFIELD_NEXT (19)
DC.B    $A8              ; +MFIELD_POSITION (row 8, col 8)
DC.B    $43              ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $0A              ; +MFIELD_STRENGTH (10)
DC.B    $0A              ; +MFIELD_HP (10)
DC.B    $05              ; +MFIELD_ATTACK (5)
DC.B    $00              ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $06              ; +MFIELD_DEFENSE (6)
DC.B    $00              ; +$0A
DC.B    $10              ; +MFIELD_LEVEL (faction=0 level=16)
DC.B    $07              ; +MFIELD_LEVEL_ENC (xpmult=0, base=7)
DC.W    $000A              ; +MFIELD_GOLD (10 ducats)
DC.B    $00              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_MAHMUD 197.

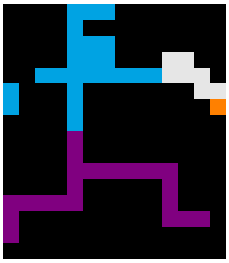


ABUL-FATH (font character 5, group 3)

89 ⟨*mob data records* 68⟩+≡ (909) ◁88 90▷

```
; --- mob 20: ABUL-FATH ($4DC0) ---
DC.W    s_ABUL_FATH              ; +MFIELD_NAME
DC.B    $24              ; +MFIELD_NEXT (36)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $43              ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $08              ; +MFIELD_STRENGTH (8)
DC.B    $08              ; +MFIELD_HP (8)
DC.B    $04              ; +MFIELD_ATTACK (4)
DC.B    $01              ; +MFIELD_XP (st=0 xp=0 grp=1)
DC.B    $02              ; +MFIELD_DEFENSE (2)
DC.B    $00              ; +$0A
DC.B    $10              ; +MFIELD_LEVEL (faction=0 level=16)
DC.B    $06              ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $A0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_ABUL_FATH 197.

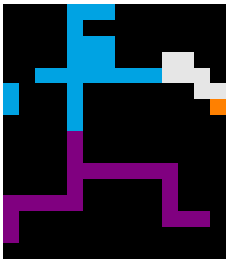


IBRAHIM (font character 5, group 3)

90 ⟨*mob data records* 68⟩+≡ (909) ◁89 91▷

```
; --- mob 21: IBRAHIM ($4DD0) ---
DC.W    s_IBRAHIM               ; +MFIELD_NAME
DC.B    $1C               ; +MFIELD_NEXT (28)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $0A               ; +MFIELD_STRENGTH (10)
DC.B    $0A               ; +MFIELD_HP (10)
DC.B    $05               ; +MFIELD_ATTACK (5)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $06               ; +MFIELD_DEFENSE (6)
DC.B    $00               ; +$0A
DC.B    $0A               ; +MFIELD_LEVEL (faction=0 level=10)
DC.B    $08               ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0005               ; +MFIELD_GOLD (5 ducats)
DC.B    $80               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_IBRAHIM 197.

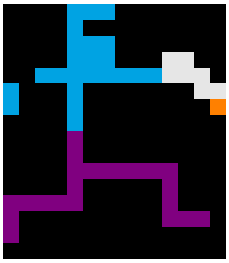


JALAL-UD-DIN (font character 5, group 4)

91 ⟨*mob data records* 68⟩+≡ (909) ◁90 92▷

```
; --- mob 22: JALAL-UD-DIN ($4DE0) ---
DC.W    s_JALAL_UD_DIN              ; +MFIELD_NAME
DC.B    $1E                ; +MFIELD_NEXT (30)
DC.B    $00                ; +MFIELD_POSITION (unplaced)
DC.B    $58                ; +MFIELD_APPEARANCE (fontchar=5, group=4 hostile)
DC.B    $0A                ; +MFIELD_STRENGTH (10)
DC.B    $0A                ; +MFIELD_HP (10)
DC.B    $05                ; +MFIELD_ATTACK (5)
DC.B    $02                ; +MFIELD_XP (st=0 xp=0 grp=2)
DC.B    $06                ; +MFIELD_DEFENSE (6)
DC.B    $00                ; +$0A
DC.B    $06                ; +MFIELD_LEVEL (faction=0 level=6)
DC.B    $06                ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0005                ; +MFIELD_GOLD (5 ducats)
DC.B    $80                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_JALAL_UD_DIN 197.

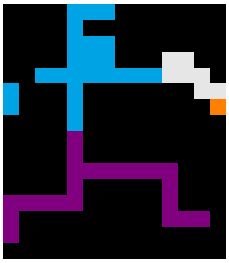


SALAMAN (font character 5, group 3)

92 ⟨*mob data records* 68⟩+≡ (909) ◁91 93▷

```
; --- mob 23: SALAMAN ($4DF0) ---
DC.W    s_SALAMAN               ; +MFIELD_NAME
DC.B    $6E               ; +MFIELD_NEXT (110)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $06               ; +MFIELD_STRENGTH (6)
DC.B    $06               ; +MFIELD_HP (6)
DC.B    $03               ; +MFIELD_ATTACK (3)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $04               ; +MFIELD_DEFENSE (4)
DC.B    $00               ; +$0A
DC.B    $0E               ; +MFIELD_LEVEL (faction=0 level=14)
DC.B    $08               ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $80               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_SALAMAN 197.

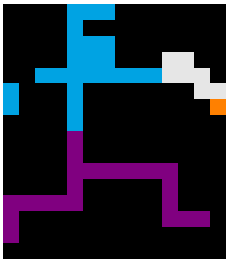


ABSAL (font character 5, group 3)

93 ⟨*mob data records* 68⟩+≡ (909) ◁92 94▷

```
; --- mob 24: ABSAL ($4E00) ---
DC.W    s_ABSAL               ; +MFIELD_NAME
DC.B    $3B               ; +MFIELD_NEXT (59)
DC.B    $A5               ; +MFIELD_POSITION (row 8, col 5)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $0A               ; +MFIELD_STRENGTH (10)
DC.B    $0A               ; +MFIELD_HP (10)
DC.B    $03               ; +MFIELD_ATTACK (3)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $04               ; +MFIELD_DEFENSE (4)
DC.B    $00               ; +$0A
DC.B    $0E               ; +MFIELD_LEVEL (faction=0 level=14)
DC.B    $08               ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $000A               ; +MFIELD_GOLD (10 ducats)
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_ABSAL 197.

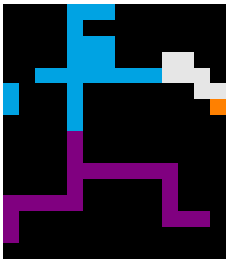


JAMI (font character 5, group 3)

94 ⟨*mob data records* 68⟩+≡ (909) ◁93 95▷

```
; --- mob 25: JAMI ($4E10) ---
DC.W    s_JAMI              ; +MFIELD_NAME
DC.B    $00               ; +MFIELD_NEXT (0)
DC.B    $A7               ; +MFIELD_POSITION (row 8, col 7)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $0C               ; +MFIELD_STRENGTH (12)
DC.B    $0C               ; +MFIELD_HP (12)
DC.B    $04               ; +MFIELD_ATTACK (4)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $03               ; +MFIELD_DEFENSE (3)
DC.B    $00               ; +$0A
DC.B    $0C               ; +MFIELD_LEVEL (faction=0 level=12)
DC.B    $07               ; +MFIELD_LEVEL_ENC (xpmult=0, base=7)
DC.W    $000A               ; +MFIELD_GOLD (10 ducats)
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_JAMI 197.

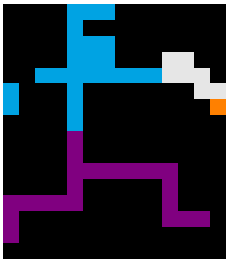


NIZAM UL MULK (font character 5, group 3)

95 ⟨*mob data records* 68⟩+≡ (909) ◁94 96▷

```
; --- mob 26: NIZAM UL MULK ($4E20) ---
DC.W    s_NIZAM_UL_MULK              ; +MFIELD_NAME
DC.B    $19                ; +MFIELD_NEXT (25)
DC.B    $92                ; +MFIELD_POSITION (row 7, col 6)
DC.B    $43                ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $0E                ; +MFIELD_STRENGTH (14)
DC.B    $0E                ; +MFIELD_HP (14)
DC.B    $05                ; +MFIELD_ATTACK (5)
DC.B    $01                ; +MFIELD_XP (st=0 xp=0 grp=1)
DC.B    $05                ; +MFIELD_DEFENSE (5)
DC.B    $00                ; +$0A
DC.B    $0E                ; +MFIELD_LEVEL (faction=0 level=14)
DC.B    $06                ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0014                ; +MFIELD_GOLD (20 ducats)
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_NIZAM_UL_MULK 197.

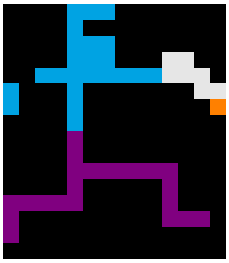


ALP ARSLAN (font character 5, group 3)

96 ⟨*mob data records* 68⟩+≡ (909) ◁95 97▷

```
; --- mob 27: ALP ARSLAN ($4E30) ---
DC.W    s_ALP_ARSLAN              ; +MFIELD_NAME
DC.B    $17               ; +MFIELD_NEXT (23)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $06               ; +MFIELD_STRENGTH (6)
DC.B    $06               ; +MFIELD_HP (6)
DC.B    $03               ; +MFIELD_ATTACK (3)
DC.B    $02               ; +MFIELD_XP (st=0 xp=0 grp=2)
DC.B    $04               ; +MFIELD_DEFENSE (4)
DC.B    $00               ; +$0A
DC.B    $0A               ; +MFIELD_LEVEL (faction=0 level=10)
DC.B    $06               ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $80               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_ALP_ARSLAN 197.

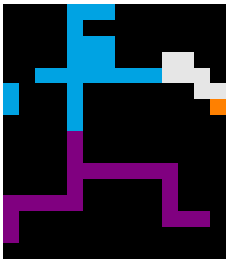


MALIK (font character 5, group 3)

97 ⟨*mob data records* 68⟩+≡ (909) ◁96 98▷

```
; --- mob 28: MALIK ($4E40) ---
DC.W    s_MALIK              ; +MFIELD_NAME
DC.B    $18              ; +MFIELD_NEXT (24)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $43              ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $08              ; +MFIELD_STRENGTH (8)
DC.B    $08              ; +MFIELD_HP (8)
DC.B    $04              ; +MFIELD_ATTACK (4)
DC.B    $01              ; +MFIELD_XP (st=0 xp=0 grp=1)
DC.B    $05              ; +MFIELD_DEFENSE (5)
DC.B    $00              ; +$0A
DC.B    $0A              ; +MFIELD_LEVEL (faction=0 level=10)
DC.B    $07              ; +MFIELD_LEVEL_ENC (xpmult=0, base=7)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $80              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_MALIK 197.

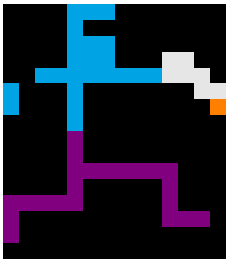


TOGHRUL BEG (font character 5, group 4)

98 ⟨*mob data records* 68⟩+≡ (909) ◁97 99▷

```
; --- mob 29: TOGHRUL BEG ($4E50) ---
DC.W    s_TOGHRUL_BEG               ; +MFIELD_NAME
DC.B    $6F               ; +MFIELD_NEXT (111)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $58               ; +MFIELD_APPEARANCE (fontchar=5, group=4 hostile)
DC.B    $08               ; +MFIELD_STRENGTH (8)
DC.B    $08               ; +MFIELD_HP (8)
DC.B    $04               ; +MFIELD_ATTACK (4)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $05               ; +MFIELD_DEFENSE (5)
DC.B    $00               ; +$0A
DC.B    $0C               ; +MFIELD_LEVEL (faction=0 level=12)
DC.B    $08               ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $80               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_TOGHRUL_BEG 197.

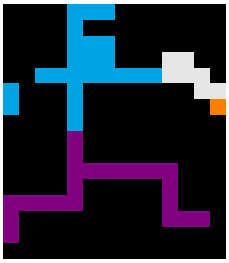


MOWAFFAK (font character 5, group 3)

99 ⟨*mob data records* 68⟩+≡ (909) ◁98 100▷

```
; --- mob 30: MOWAFFAK ($4E60) ---
DC.W    s_MOWAFFAK              ; +MFIELD_NAME
DC.B    $21                ; +MFIELD_NEXT (33)
DC.B    $00                ; +MFIELD_POSITION (unplaced)
DC.B    $43                ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $10                ; +MFIELD_STRENGTH (16)
DC.B    $10                ; +MFIELD_HP (16)
DC.B    $08                ; +MFIELD_ATTACK (8)
DC.B    $00                ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $09                ; +MFIELD_DEFENSE (9)
DC.B    $00                ; +$0A
DC.B    $0A                ; +MFIELD_LEVEL (faction=0 level=10)
DC.B    $07                ; +MFIELD_LEVEL_ENC (xpmult=0, base=7)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $A0                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_MOWAFFAK 197.

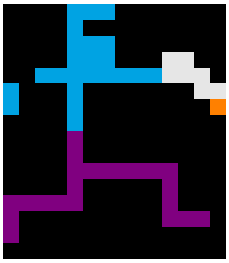


ABD-US-SAMAD (font character 5, group 3)

100 ⟨*mob data records* 68⟩+≡ (909) ◁99 101▷

```
; --- mob 31: ABD-US-SAMAD ($4E70) ---
DC.W    s_ABD_US_SAMAD              ; +MFIELD_NAME
DC.B    $26                ; +MFIELD_NEXT (38)
DC.B    $00                ; +MFIELD_POSITION (unplaced)
DC.B    $43                ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $0F                ; +MFIELD_STRENGTH (15)
DC.B    $0F                ; +MFIELD_HP (15)
DC.B    $07                ; +MFIELD_ATTACK (7)
DC.B    $01                ; +MFIELD_XP (st=0 xp=0 grp=1)
DC.B    $08                ; +MFIELD_DEFENSE (8)
DC.B    $00                ; +$0A
DC.B    $09                ; +MFIELD_LEVEL (faction=0 level=9)
DC.B    $07                ; +MFIELD_LEVEL_ENC (xpmult=0, base=7)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $A0                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_ABD_US_SAMAD 197.

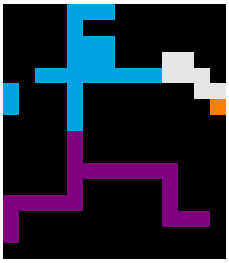


HAKIM (font character 5, group 3)

101 ⟨*mob data records* 68⟩+≡ (909) ◁100 102▷

```
; --- mob 32: HAKIM ($4E80) ---
DC.W    s_HAKIM              ; +MFIELD_NAME
DC.B    $25              ; +MFIELD_NEXT (37)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $43              ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $0E              ; +MFIELD_STRENGTH (14)
DC.B    $0E              ; +MFIELD_HP (14)
DC.B    $07              ; +MFIELD_ATTACK (7)
DC.B    $02              ; +MFIELD_XP (st=0 xp=0 grp=2)
DC.B    $07              ; +MFIELD_DEFENSE (7)
DC.B    $00              ; +$0A
DC.B    $08              ; +MFIELD_LEVEL (faction=0 level=8)
DC.B    $06              ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $A0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_HAKIM 197.

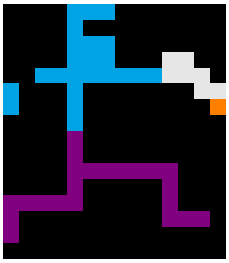


HASAN BEN SABBAH (font character 5, group 3)

102 ⟨*mob data records* 68⟩+≡ (909) ◁101 103▷

```
; --- mob 33: HASAN BEN SABBAH ($4E90) ---
DC.W    s_HASAN_BEN_SABBAH              ; +MFIELD_NAME
DC.B    $00                ; +MFIELD_NEXT (0)
DC.B    $56                ; +MFIELD_POSITION (row 4, col 6)
DC.B    $43                ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $0D                ; +MFIELD_STRENGTH (13)
DC.B    $0D                ; +MFIELD_HP (13)
DC.B    $06                ; +MFIELD_ATTACK (6)
DC.B    $03                ; +MFIELD_XP (st=0 xp=0 grp=3)
DC.B    $06                ; +MFIELD_DEFENSE (6)
DC.B    $00                ; +$0A
DC.B    $09                ; +MFIELD_LEVEL (faction=0 level=9)
DC.B    $04                ; +MFIELD_LEVEL_ENC (xpmult=0, base=4)
DC.W    $0019                ; +MFIELD_GOLD (25 ducats)
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_HASAN_BEN_SABBAH 197.

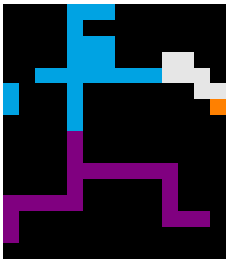


ALI (font character 5, group 3)

103 ⟨*mob data records* 68⟩+≡ (909) ◁102 104▷

```
; --- mob 34: ALI ($4EA0) ---
DC.W    s_ALI               ; +MFIELD_NAME
DC.B    $7B               ; +MFIELD_NEXT (123)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $14               ; +MFIELD_STRENGTH (20)
DC.B    $14               ; +MFIELD_HP (20)
DC.B    $08               ; +MFIELD_ATTACK (8)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $09               ; +MFIELD_DEFENSE (9)
DC.B    $00               ; +$0A
DC.B    $0E               ; +MFIELD_LEVEL (faction=0 level=14)
DC.B    $08               ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0064               ; +MFIELD_GOLD (100 ducats)
DC.B    $E0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_ALI 197.

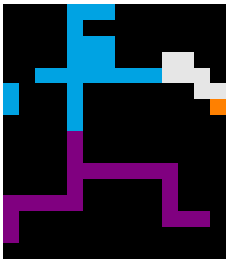


KHORASSAN (font character 5, group 3)

104 ⟨*mob data records* 68⟩+≡ (909) ◁103 105▷

```
; --- mob 35: KHORASSAN ($4EB0) ---
DC.W    s_KHORASSAN              ; +MFIELD_NAME
DC.B    $73              ; +MFIELD_NEXT (115)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $43              ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $0D              ; +MFIELD_STRENGTH (13)
DC.B    $0D              ; +MFIELD_HP (13)
DC.B    $06              ; +MFIELD_ATTACK (6)
DC.B    $00              ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $07              ; +MFIELD_DEFENSE (7)
DC.B    $00              ; +$0A
DC.B    $0D              ; +MFIELD_LEVEL (faction=0 level=13)
DC.B    $08              ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $A0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_KHORASSAN 197.

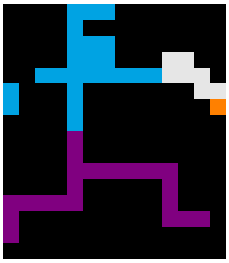


KHWAJAH NIZAMI (font character 5, group 4)

105 ⟨*mob data records* 68⟩+≡ (909) ◁104 106▷

```
; --- mob 36: KHWAJAH NIZAMI ($4EC0) ---
DC.W    s_KHWAJAH_NIZAMI              ; +MFIELD_NAME
DC.B    $70               ; +MFIELD_NEXT (112)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $58               ; +MFIELD_APPEARANCE (fontchar=5, group=4 hostile)
DC.B    $0C               ; +MFIELD_STRENGTH (12)
DC.B    $0C               ; +MFIELD_HP (12)
DC.B    $06               ; +MFIELD_ATTACK (6)
DC.B    $01               ; +MFIELD_XP (st=0 xp=0 grp=1)
DC.B    $06               ; +MFIELD_DEFENSE (6)
DC.B    $00               ; +$0A
DC.B    $0C               ; +MFIELD_LEVEL (faction=0 level=12)
DC.B    $07               ; +MFIELD_LEVEL_ENC (xpmult=0, base=7)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $A0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_KHWAJAH_NIZAMI 197.

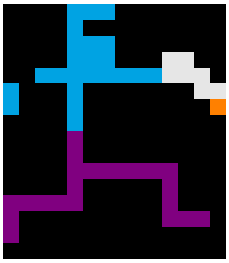


HAFIZ (font character 5, group 3)

106 ⟨*mob data records* 68⟩+≡ (909) ◁105 107▷

```
; --- mob 37: HAFIZ ($4ED0) ---
DC.W    s_HAFIZ              ; +MFIELD_NAME
DC.B    $72              ; +MFIELD_NEXT (114)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $43              ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $0B              ; +MFIELD_STRENGTH (11)
DC.B    $0B              ; +MFIELD_HP (11)
DC.B    $05              ; +MFIELD_ATTACK (5)
DC.B    $02              ; +MFIELD_XP (st=0 xp=0 grp=2)
DC.B    $06              ; +MFIELD_DEFENSE (6)
DC.B    $00              ; +$0A
DC.B    $0B              ; +MFIELD_LEVEL (faction=0 level=11)
DC.B    $06              ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $000A              ; +MFIELD_GOLD (10 ducats)
DC.B    $A0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_HAFIZ 197.

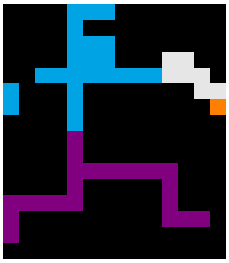


JELALUDDIN (font character 5, group 3)

107 ⟨*mob data records* 68⟩+≡ (909) ◁106 108▷

```
; --- mob 38: JELALUDDIN ($4EE0) ---
DC.W    s_JELALUDDIN              ; +MFIELD_NAME
DC.B    $71               ; +MFIELD_NEXT (113)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $0A               ; +MFIELD_STRENGTH (10)
DC.B    $0A               ; +MFIELD_HP (10)
DC.B    $06               ; +MFIELD_ATTACK (6)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $07               ; +MFIELD_DEFENSE (7)
DC.B    $00               ; +$0A
DC.B    $10               ; +MFIELD_LEVEL (faction=0 level=16)
DC.B    $08               ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $000A               ; +MFIELD_GOLD (10 ducats)
DC.B    $A0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_JELALUDDIN 197.

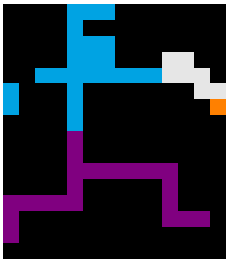


KWADRACH (font character 5, group 3)

108 ⟨*mob data records* 68⟩+≡ (909) ◁107 109▷

```
; --- mob 39: KWADRACH ($4EF0) ---
DC.W    s_KWADRACH              ; +MFIELD_NAME
DC.B    $74               ; +MFIELD_NEXT (116)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $09               ; +MFIELD_STRENGTH (9)
DC.B    $09               ; +MFIELD_HP (9)
DC.B    $05               ; +MFIELD_ATTACK (5)
DC.B    $01               ; +MFIELD_XP (st=0 xp=0 grp=1)
DC.B    $06               ; +MFIELD_DEFENSE (6)
DC.B    $00               ; +$0A
DC.B    $0F               ; +MFIELD_LEVEL (faction=0 level=15)
DC.B    $07               ; +MFIELD_LEVEL_ENC (xpmult=0, base=7)
DC.W    $000A               ; +MFIELD_GOLD (10 ducats)
DC.B    $A0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_KWADRACH 197.

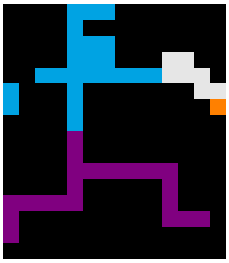


ATTAR (font character 5, group 4)

109 ⟨*mob data records* 68⟩+≡ (909) ◁108 110▷

```
; --- mob 40: ATTAR ($4F00) ---
DC.W    s_ATTAR              ; +MFIELD_NAME
DC.B    $7C              ; +MFIELD_NEXT (124)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $58              ; +MFIELD_APPEARANCE (fontchar=5, group=4 hostile)
DC.B    $10              ; +MFIELD_STRENGTH (16)
DC.B    $10              ; +MFIELD_HP (16)
DC.B    $08              ; +MFIELD_ATTACK (8)
DC.B    $01              ; +MFIELD_XP (st=0 xp=0 grp=1)
DC.B    $09              ; +MFIELD_DEFENSE (9)
DC.B    $00              ; +$0A
DC.B    $10              ; +MFIELD_LEVEL (faction=0 level=16)
DC.B    $07              ; +MFIELD_LEVEL_ENC (xpmult=0, base=7)
DC.W    $0014              ; +MFIELD_GOLD (20 ducats)
DC.B    $E0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_ATTAR 197.

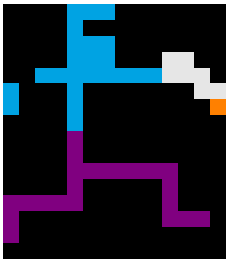


PEHLEVI (font character 5, group 3)

110 ⟨*mob data records* 68⟩+≡ (909) ◁109 111▷

```
; --- mob 41: PEHLEVI ($4F10) ---
DC.W    s_PEHLEVI               ; +MFIELD_NAME
DC.B    $7F               ; +MFIELD_NEXT (127)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $0E               ; +MFIELD_STRENGTH (14)
DC.B    $0E               ; +MFIELD_HP (14)
DC.B    $07               ; +MFIELD_ATTACK (7)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $08               ; +MFIELD_DEFENSE (8)
DC.B    $00               ; +$0A
DC.B    $14               ; +MFIELD_LEVEL (faction=0 level=20)
DC.B    $08               ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0014               ; +MFIELD_GOLD (20 ducats)
DC.B    $E0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_PEHLEVI 197.

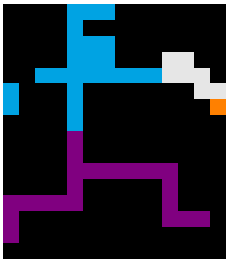


JAMSHYD (font character 5, group 3)

111 ⟨*mob data records* 68⟩+≡ (909) ◁110 112▷

```
; --- mob 42: JAMSHYD ($4F20) ---
DC.W    s_JAMSHYD               ; +MFIELD_NAME
DC.B    $7A               ; +MFIELD_NEXT (122)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $14               ; +MFIELD_STRENGTH (20)
DC.B    $14               ; +MFIELD_HP (20)
DC.B    $09               ; +MFIELD_ATTACK (9)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $0A               ; +MFIELD_DEFENSE (10)
DC.B    $00               ; +$0A
DC.B    $0A               ; +MFIELD_LEVEL (faction=0 level=10)
DC.B    $08               ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $001E               ; +MFIELD_GOLD (30 ducats)
DC.B    $C0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_JAMSHYD 197.

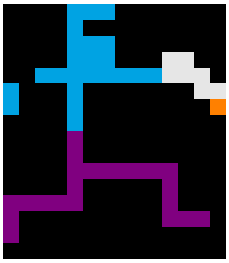


IRAM (font character 5, group 3)

112 ⟨*mob data records* 68⟩+≡ (909) ◁111 113▷

```
; --- mob 43: IRAM ($4F30) ---
DC.W    s_IRAM              ; +MFIELD_NAME
DC.B    $33               ; +MFIELD_NEXT (51)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $10               ; +MFIELD_STRENGTH (16)
DC.B    $10               ; +MFIELD_HP (16)
DC.B    $08               ; +MFIELD_ATTACK (8)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $08               ; +MFIELD_DEFENSE (8)
DC.B    $00               ; +$0A
DC.B    $0E               ; +MFIELD_LEVEL (faction=0 level=14)
DC.B    $08               ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $000A               ; +MFIELD_GOLD (10 ducats)
DC.B    $C0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_IRAM 197.

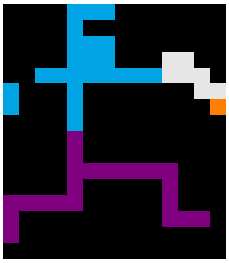


KAIKOBAD (font character 5, group 3)

113 ⟨*mob data records* 68⟩+≡ (909) ◁112 114▷

```
; --- mob 44: KAIKOBAD ($4F40) ---
DC.W    s_KAIKOBAD              ; +MFIELD_NAME
DC.B    $76                ; +MFIELD_NEXT (118)
DC.B    $00                ; +MFIELD_POSITION (unplaced)
DC.B    $43                ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $0C                ; +MFIELD_STRENGTH (12)
DC.B    $0C                ; +MFIELD_HP (12)
DC.B    $06                ; +MFIELD_ATTACK (6)
DC.B    $00                ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $07                ; +MFIELD_DEFENSE (7)
DC.B    $00                ; +$0A
DC.B    $12                ; +MFIELD_LEVEL (faction=0 level=18)
DC.B    $08                ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $000A                ; +MFIELD_GOLD (10 ducats)
DC.B    $C0                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_KAIKOBAD 197.

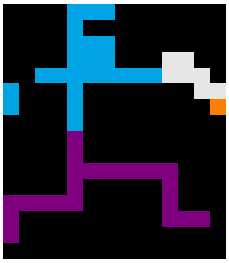


KAIKHOSRU (font character 5, group 4)

114 ⟨*mob data records* 68⟩+≡ (909) ◁113 115▷

```
; --- mob 45: KAIKHOSRU ($4F50) ---
DC.W    s_KAIKHOSRU              ; +MFIELD_NAME
DC.B    $31              ; +MFIELD_NEXT (49)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $58              ; +MFIELD_APPEARANCE (fontchar=5, group=4 hostile)
DC.B    $0A              ; +MFIELD_STRENGTH (10)
DC.B    $0A              ; +MFIELD_HP (10)
DC.B    $06              ; +MFIELD_ATTACK (6)
DC.B    $00              ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $06              ; +MFIELD_DEFENSE (6)
DC.B    $00              ; +$0A
DC.B    $14              ; +MFIELD_LEVEL (faction=0 level=20)
DC.B    $08              ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $003C              ; +MFIELD_GOLD (60 ducats)
DC.B    $C0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_KAIKHOSRU 197.

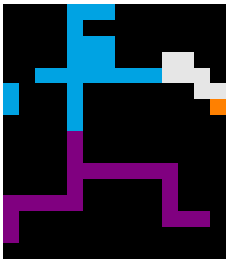


RUSTUM (font character 5, group 3)

115 ⟨*mob data records* 68⟩+≡ (909) ◁114 116▷

```
; --- mob 46: RUSTUM ($4F60) ---
DC.W    s_RUSTUM              ; +MFIELD_NAME
DC.B    $78               ; +MFIELD_NEXT (120)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $12               ; +MFIELD_STRENGTH (18)
DC.B    $12               ; +MFIELD_HP (18)
DC.B    $09               ; +MFIELD_ATTACK (9)
DC.B    $01               ; +MFIELD_XP (st=0 xp=0 grp=1)
DC.B    $09               ; +MFIELD_DEFENSE (9)
DC.B    $00               ; +$0A
DC.B    $0A               ; +MFIELD_LEVEL (faction=0 level=10)
DC.B    $07               ; +MFIELD_LEVEL_ENC (xpmult=0, base=7)
DC.W    $0032               ; +MFIELD_GOLD (50 ducats)
DC.B    $C0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_RUSTUM 197.

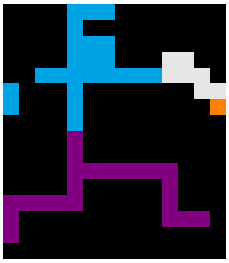


HATIM TAI (font character 5, group 3)

116 ⟨*mob data records* 68⟩+≡ (909) ◁115 117▷

```
; --- mob 47: HATIM TAI ($4F70) ---
DC.W    s_HATIM_TAI              ; +MFIELD_NAME
DC.B    $2D              ; +MFIELD_NEXT (45)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $43              ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $0C              ; +MFIELD_STRENGTH (12)
DC.B    $0C              ; +MFIELD_HP (12)
DC.B    $06              ; +MFIELD_ATTACK (6)
DC.B    $02              ; +MFIELD_XP (st=0 xp=0 grp=2)
DC.B    $07              ; +MFIELD_DEFENSE (7)
DC.B    $00              ; +$0A
DC.B    $0E              ; +MFIELD_LEVEL (faction=0 level=14)
DC.B    $06              ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0028              ; +MFIELD_GOLD (40 ducats)
DC.B    $C0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_HATIM_TAI 197.

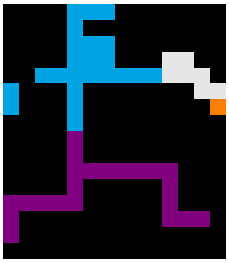


BAHRAM (font character 5, group 3)

117 ⟨*mob data records* 68⟩+≡ (909) ◁116 118▷

```
; --- mob 48: BAHRAM ($4F80) ---
DC.W    s_BAHRAM              ; +MFIELD_NAME
DC.B    $80               ; +MFIELD_NEXT (128)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $0D               ; +MFIELD_STRENGTH (13)
DC.B    $0D               ; +MFIELD_HP (13)
DC.B    $06               ; +MFIELD_ATTACK (6)
DC.B    $03               ; +MFIELD_XP (st=0 xp=0 grp=3)
DC.B    $07               ; +MFIELD_DEFENSE (7)
DC.B    $00               ; +$0A
DC.B    $0F               ; +MFIELD_LEVEL (faction=0 level=15)
DC.B    $05               ; +MFIELD_LEVEL_ENC (xpmult=0, base=5)
DC.W    $003C               ; +MFIELD_GOLD (60 ducats)
DC.B    $E0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_BAHRAM 197.

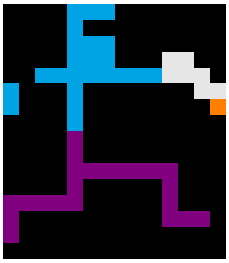


DINARZADE (font character 5, group 3)

118 ⟨*mob data records* 68⟩+≡ (909) ◁117 119▷

```
; --- mob 49: DINARZADE ($4F90) ---
DC.W    s_DINARZADE              ; +MFIELD_NAME
DC.B    $77              ; +MFIELD_NEXT (119)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $43              ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $0A              ; +MFIELD_STRENGTH (10)
DC.B    $0A              ; +MFIELD_HP (10)
DC.B    $05              ; +MFIELD_ATTACK (5)
DC.B    $03              ; +MFIELD_XP (st=0 xp=0 grp=3)
DC.B    $06              ; +MFIELD_DEFENSE (6)
DC.B    $00              ; +$0A
DC.B    $0E              ; +MFIELD_LEVEL (faction=0 level=14)
DC.B    $05              ; +MFIELD_LEVEL_ENC (xpmult=0, base=5)
DC.W    $0019              ; +MFIELD_GOLD (25 ducats)
DC.B    $C0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_DINARZADE 197.

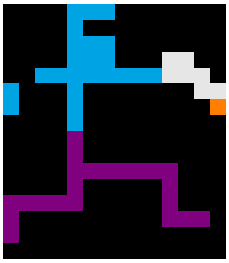


SHAHZENAN (font character 5, group 4)

119 ⟨*mob data records* 68⟩+≡ (909) ◁118 120▷

```
; --- mob 50: SHAHZENAN ($4FA0) ---
DC.W    s_SHAHZENAN              ; +MFIELD_NAME
DC.B    $75              ; +MFIELD_NEXT (117)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $58              ; +MFIELD_APPEARANCE (fontchar=5, group=4 hostile)
DC.B    $0A              ; +MFIELD_STRENGTH (10)
DC.B    $0A              ; +MFIELD_HP (10)
DC.B    $05              ; +MFIELD_ATTACK (5)
DC.B    $04              ; +MFIELD_XP (st=0 xp=0 grp=4)
DC.B    $04              ; +MFIELD_DEFENSE (4)
DC.B    $00              ; +$0A
DC.B    $0C              ; +MFIELD_LEVEL (faction=0 level=12)
DC.B    $04              ; +MFIELD_LEVEL_ENC (xpmult=0, base=4)
DC.W    $0019              ; +MFIELD_GOLD (25 ducats)
DC.B    $C0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_SHAHZENAN 197.

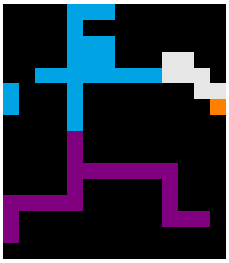


DOUBAN (font character 5, group 3)

120 ⟨*mob data records* 68⟩+≡ (909) ◁119 121▷

```
; --- mob 51: DOUBAN ($4FB0) ---
DC.W    s_DOUBAN              ; +MFIELD_NAME
DC.B    $32               ; +MFIELD_NEXT (50)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $0F               ; +MFIELD_STRENGTH (15)
DC.B    $0F               ; +MFIELD_HP (15)
DC.B    $07               ; +MFIELD_ATTACK (7)
DC.B    $01               ; +MFIELD_XP (st=0 xp=0 grp=1)
DC.B    $08               ; +MFIELD_DEFENSE (8)
DC.B    $00               ; +$0A
DC.B    $0D               ; +MFIELD_LEVEL (faction=0 level=13)
DC.B    $07               ; +MFIELD_LEVEL_ENC (xpmult=0, base=7)
DC.W    $0019               ; +MFIELD_GOLD (25 ducats)
DC.B    $C0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_DOUBAN 197.

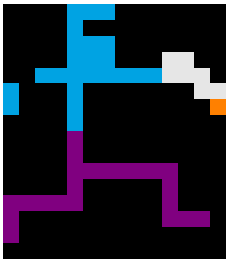


FEROZE (font character 5, group 3)

121 ⟨*mob data records* 68⟩+≡ (909) ◁120 122▷

```
; --- mob 52: FEROZE ($4FC0) ---
DC.W    s_FEROZE              ; +MFIELD_NAME
DC.B    $79               ; +MFIELD_NEXT (121)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $10               ; +MFIELD_STRENGTH (16)
DC.B    $10               ; +MFIELD_HP (16)
DC.B    $08               ; +MFIELD_ATTACK (8)
DC.B    $02               ; +MFIELD_XP (st=0 xp=0 grp=2)
DC.B    $08               ; +MFIELD_DEFENSE (8)
DC.B    $00               ; +$0A
DC.B    $0A               ; +MFIELD_LEVEL (faction=0 level=10)
DC.B    $06               ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0019               ; +MFIELD_GOLD (25 ducats)
DC.B    $C0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_FEROZE 197.

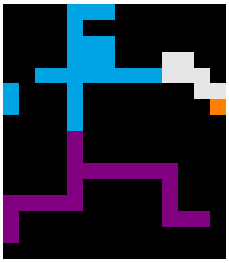


AHMED (font character 5, group 3)

122 ⟨*mob data records* 68⟩+≡ (909) ◁121 123▷

```
; --- mob 53: AHMED ($4FD0) ---
DC.W    s_AHMED               ; +MFIELD_NAME
DC.B    $27               ; +MFIELD_NEXT (39)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $43               ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $08               ; +MFIELD_STRENGTH (8)
DC.B    $08               ; +MFIELD_HP (8)
DC.B    $04               ; +MFIELD_ATTACK (4)
DC.B    $02               ; +MFIELD_XP (st=0 xp=0 grp=2)
DC.B    $05               ; +MFIELD_DEFENSE (5)
DC.B    $00               ; +$0A
DC.B    $0E               ; +MFIELD_LEVEL (faction=0 level=14)
DC.B    $06               ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0005               ; +MFIELD_GOLD (5 ducats)
DC.B    $A0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_AHMED 197.

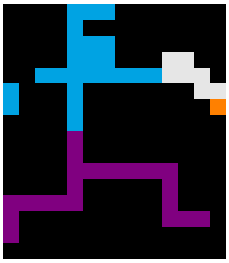


PERIE BANOU (font character 5, group 3)

123 ⟨*mob data records* 68⟩+≡ (909) ◁122 124▷

```
; --- mob 54: PERIE BANOU ($4FE0) ---
DC.W    s_PERIE_BANOU              ; +MFIELD_NAME
DC.B    $35              ; +MFIELD_NEXT (53)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $43              ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $07              ; +MFIELD_STRENGTH (7)
DC.B    $07              ; +MFIELD_HP (7)
DC.B    $04              ; +MFIELD_ATTACK (4)
DC.B    $03              ; +MFIELD_XP (st=0 xp=0 grp=3)
DC.B    $04              ; +MFIELD_DEFENSE (4)
DC.B    $00              ; +$0A
DC.B    $0D              ; +MFIELD_LEVEL (faction=0 level=13)
DC.B    $05              ; +MFIELD_LEVEL_ENC (xpmult=0, base=5)
DC.W    $0005              ; +MFIELD_GOLD (5 ducats)
DC.B    $A0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_PERIE_BANOU 197.

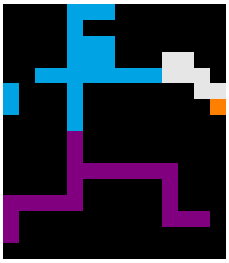


HAROUN AL-RASCHID (font character 5, group 3)

124 ⟨*mob data records* 68⟩+≡ (909) ◁123 125▷

```
; --- mob 55: HAROUN AL-RASCHID ($4FF0) ---
DC.W    s_HAROUN_AL_RASCHID              ; +MFIELD_NAME
DC.B    $28              ; +MFIELD_NEXT (40)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $43              ; +MFIELD_APPEARANCE (fontchar=5, group=3 hostile)
DC.B    $06              ; +MFIELD_STRENGTH (6)
DC.B    $06              ; +MFIELD_HP (6)
DC.B    $03              ; +MFIELD_ATTACK (3)
DC.B    $04              ; +MFIELD_XP (st=0 xp=0 grp=4)
DC.B    $04              ; +MFIELD_DEFENSE (4)
DC.B    $00              ; +$0A
DC.B    $0C              ; +MFIELD_LEVEL (faction=0 level=12)
DC.B    $04              ; +MFIELD_LEVEL_ENC (xpmult=0, base=4)
DC.W    $0005              ; +MFIELD_GOLD (5 ducats)
DC.B    $A0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_HAROUN_AL_RASCHID 197.

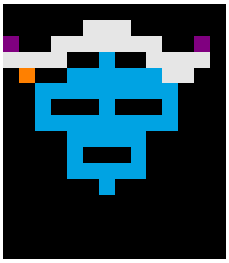


CODADAD (font character 5, group 4)

125 ⟨*mob data records* 68⟩+≡ (909) ◁124 126▷

```
; --- mob 56: CODADAD ($5000) ---
DC.W    s_CODADAD               ; +MFIELD_NAME
DC.B    $36               ; +MFIELD_NEXT (54)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $58               ; +MFIELD_APPEARANCE (fontchar=5, group=4 hostile)
DC.B    $0C               ; +MFIELD_STRENGTH (12)
DC.B    $0C               ; +MFIELD_HP (12)
DC.B    $06               ; +MFIELD_ATTACK (6)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $07               ; +MFIELD_DEFENSE (7)
DC.B    $00               ; +$0A
DC.B    $0E               ; +MFIELD_LEVEL (faction=0 level=14)
DC.B    $08               ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0005               ; +MFIELD_GOLD (5 ducats)
DC.B    $A0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_CODADAD 197.

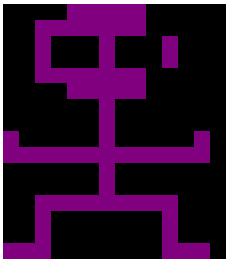


MORGIANA (font character 15, group 1 — friendly)

126 ⟨*mob data records* 68⟩+≡ (909) ◁125 127▷

```
; --- mob 57: MORGIANA ($5010) ---
DC.W    s_MORGIANA              ; +MFIELD_NAME
DC.B    $22                 ; +MFIELD_NEXT (34)
DC.B    $6B                 ; +MFIELD_POSITION (row 5, col 7)
DC.B    $23                 ; +MFIELD_APPEARANCE (fontchar=15, group=1 friendly)
DC.B    $0A                 ; +MFIELD_STRENGTH (10)
DC.B    $0A                 ; +MFIELD_HP (10)
DC.B    $05                 ; +MFIELD_ATTACK (5)
DC.B    $00                 ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $07                 ; +MFIELD_DEFENSE (7)
DC.B    $00                 ; +$0A
DC.B    $58                 ; +MFIELD_LEVEL (faction=2 level=24)
DC.B    $08                 ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0000                 ; +MFIELD_GOLD
DC.B    $00                 ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_MORGIANA 197.

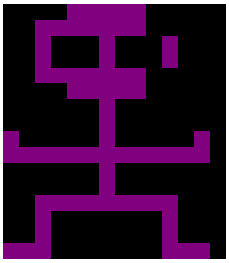


ABDALLA (font character 33, group 1 — friendly)

127 ⟨*mob data records* 68⟩+≡ (909) ◁126 128▷

```
; --- mob 58: ABDALLA ($5020) ---
DC.W    s_ABDALLA               ; +MFIELD_NAME
DC.B    $1A               ; +MFIELD_NEXT (26)
DC.B    $AC               ; +MFIELD_POSITION (row 8, col 12)
DC.B    $15               ; +MFIELD_APPEARANCE (fontchar=33, group=1 friendly)
DC.B    $0A               ; +MFIELD_STRENGTH (10)
DC.B    $0A               ; +MFIELD_HP (10)
DC.B    $04               ; +MFIELD_ATTACK (4)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $02               ; +MFIELD_DEFENSE (2)
DC.B    $00               ; +$0A
DC.B    $0C               ; +MFIELD_LEVEL (faction=0 level=12)
DC.B    $08               ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_ABDALLA 197.

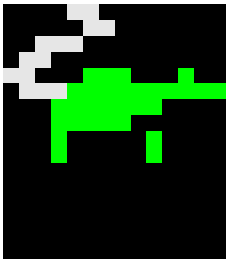


ENCHANTED STATUE (font character 33, group 3)

128 ⟨*mob data records* 68⟩+≡ (909) ◁127 129▷

```
; --- mob 59: ENCHANTED STATUE ($5030) ---
DC.W    s_ENCHANTED_STATUE              ; +MFIELD_NAME
DC.B    $3D                ; +MFIELD_NEXT (61)
DC.B    $57                ; +MFIELD_POSITION (row 4, col 7)
DC.B    $3F                ; +MFIELD_APPEARANCE (fontchar=33, group=3 hostile)
DC.B    $08                ; +MFIELD_STRENGTH (8)
DC.B    $10                ; +MFIELD_HP (16)
DC.B    $06                ; +MFIELD_ATTACK (6)
DC.B    $00                ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $05                ; +MFIELD_DEFENSE (5)
DC.B    $00                ; +$0A
DC.B    $8C                ; +MFIELD_LEVEL (faction=4 level=12)
DC.B    $00                ; +MFIELD_LEVEL_ENC (xpmult=0, base=0)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_ENCHANTED_STATUE 197.

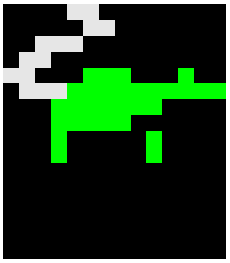


TAN RAT (font character 7, group 3 — hostile)

129 ⟨*mob data records* 68⟩+≡ (909) ◁128 130▷

```
; --- mob 60: TAN RAT ($5040) ---
DC.W    s_TAN_RAT               ; +MFIELD_NAME
DC.B    $3E               ; +MFIELD_NEXT (62)
DC.B    $91               ; +MFIELD_POSITION (row 7, col 5)
DC.B    $45               ; +MFIELD_APPEARANCE (fontchar=7, group=3 hostile)
DC.B    $03               ; +MFIELD_STRENGTH (3)
DC.B    $03               ; +MFIELD_HP (3)
DC.B    $02               ; +MFIELD_ATTACK (2)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $02               ; +MFIELD_DEFENSE (2)
DC.B    $00               ; +$0A
DC.B    $4A               ; +MFIELD_LEVEL (faction=2 level=10)
DC.B    $06               ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_TAN_RAT 197.

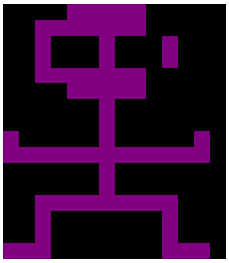


GRAY RAT (font character 7, group 3)

130 ⟨*mob data records* 68⟩+≡ (909) ◁129 131▷

```
; --- mob 61: GRAY RAT ($5050) ---
DC.W    s_GRAY_RAT              ; +MFIELD_NAME
DC.B    $00                ; +MFIELD_NEXT (0)
DC.B    $92                ; +MFIELD_POSITION (row 7, col 6)
DC.B    $45                ; +MFIELD_APPEARANCE (fontchar=7, group=3 hostile)
DC.B    $03                ; +MFIELD_STRENGTH (3)
DC.B    $03                ; +MFIELD_HP (3)
DC.B    $02                ; +MFIELD_ATTACK (2)
DC.B    $00                ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $02                ; +MFIELD_DEFENSE (2)
DC.B    $00                ; +$0A
DC.B    $48                ; +MFIELD_LEVEL (faction=2 level=8)
DC.B    $06                ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $80                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_GRAY_RAT 197.

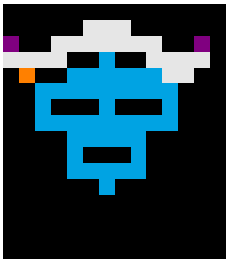


SULTAN SHAHRIAR (font character 33, group 3)

131 ⟨*mob data records* 68⟩+≡ (909) ◁130 132▷

```
; --- mob 62: SULTAN SHAHRIAR ($5060) ---
DC.W    s_SULTAN_SHAHRIAR              ; +MFIELD_NAME
DC.B    $41              ; +MFIELD_NEXT (65)
DC.B    $65              ; +MFIELD_POSITION (row 5, col 1)
DC.B    $3F              ; +MFIELD_APPEARANCE (fontchar=33, group=3 hostile)
DC.B    $10              ; +MFIELD_STRENGTH (16)
DC.B    $10              ; +MFIELD_HP (16)
DC.B    $08              ; +MFIELD_ATTACK (8)
DC.B    $02              ; +MFIELD_XP (st=0 xp=0 grp=2)
DC.B    $06              ; +MFIELD_DEFENSE (6)
DC.B    $00              ; +$0A
DC.B    $10              ; +MFIELD_LEVEL (faction=0 level=16)
DC.B    $00              ; +MFIELD_LEVEL_ENC (xpmult=0, base=0)
DC.W    $0032              ; +MFIELD_GOLD (50 ducats)
DC.B    $00              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_SULTAN_SHAHRIAR 197.

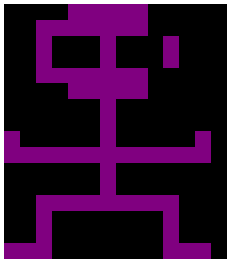


PRINCESS BUDDIR (font character 15, group 1)

132 ⟨*mob data records* 68⟩+≡ (909) ◁131 133▷

```
; --- mob 63: PRINCESS BUDDIR ($5070) ---
DC.W    s_PRINCESS_BUDDIR              ; +MFIELD_NAME
DC.B    $00                ; +MFIELD_NEXT (0)
DC.B    $2D                ; +MFIELD_POSITION (row 2, col 5)
DC.B    $23                ; +MFIELD_APPEARANCE (fontchar=15, group=1 friendly)
DC.B    $08                ; +MFIELD_STRENGTH (8)
DC.B    $08                ; +MFIELD_HP (8)
DC.B    $00                ; +MFIELD_ATTACK (0)
DC.B    $00                ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $00                ; +MFIELD_DEFENSE (0)
DC.B    $00                ; +$0A
DC.B    $0F                ; +MFIELD_LEVEL (faction=0 level=15)
DC.B    $08                ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $000A                ; +MFIELD_GOLD (10 ducats)
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_PRINCESS_BUDDIR 197.

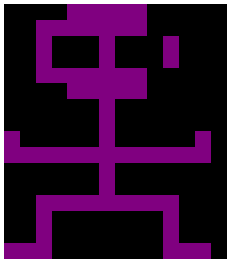


SULTANS RIGHT GUARD (font character 33, group 3)

133 ⟨*mob data records* 68⟩+≡ (909) ◁132 134▷

```
; --- mob 64: SULTANS RIGHT GUARD ($5080) ---
DC.W    s_SULTANS_RIGHT_GUARD              ; +MFIELD_NAME
DC.B    $42                ; +MFIELD_NEXT (66)
DC.B    $7A                ; +MFIELD_POSITION (row 6, col 2)
DC.B    $3F                ; +MFIELD_APPEARANCE (fontchar=33, group=3 hostile)
DC.B    $08                ; +MFIELD_STRENGTH (8)
DC.B    $08                ; +MFIELD_HP (8)
DC.B    $04                ; +MFIELD_ATTACK (4)
DC.B    $00                ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $02                ; +MFIELD_DEFENSE (2)
DC.B    $00                ; +$0A
DC.B    $CB                ; +MFIELD_LEVEL (faction=6 level=11)
DC.B    $00                ; +MFIELD_LEVEL_ENC (xpmult=0, base=0)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_SULTANS_RIGHT_GUARD 197.

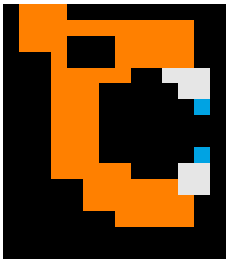


SULTANS LEFT GUARD (font character 33, group 3)

134 ⟨*mob data records* 68⟩+≡ (909) ◁133 135▷

```
; --- mob 65: SULTANS LEFT GUARD ($5090) ---
DC.W    s_SULTANS_LEFT_GUARD              ; +MFIELD_NAME
DC.B    $00               ; +MFIELD_NEXT (0)
DC.B    $52               ; +MFIELD_POSITION (row 4, col 2)
DC.B    $3F               ; +MFIELD_APPEARANCE (fontchar=33, group=3 hostile)
DC.B    $08               ; +MFIELD_STRENGTH (8)
DC.B    $08               ; +MFIELD_HP (8)
DC.B    $04               ; +MFIELD_ATTACK (4)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $03               ; +MFIELD_DEFENSE (3)
DC.B    $00               ; +$0A
DC.B    $CA               ; +MFIELD_LEVEL (faction=6 level=10)
DC.B    $00               ; +MFIELD_LEVEL_ENC (xpmult=0, base=0)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_SULTANS_LEFT_GUARD 197.

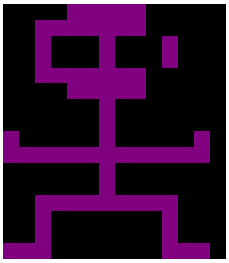


TIGER (font character 16, group 3 — hostile)

135 ⟨*mob data records* 68⟩+≡ (909) ◁134 136▷

```
; --- mob 66: TIGER ($50A0) ---
DC.W    s_TIGER              ; +MFIELD_NAME
DC.B    $00              ; +MFIELD_NEXT (0)
DC.B    $7A              ; +MFIELD_POSITION (row 6, col 2)
DC.B    $4E              ; +MFIELD_APPEARANCE (fontchar=16, group=3 hostile)
DC.B    $10              ; +MFIELD_STRENGTH (16)
DC.B    $0C              ; +MFIELD_HP (12)
DC.B    $04              ; +MFIELD_ATTACK (4)
DC.B    $81              ; +MFIELD_XP (st=1 xp=0 grp=1)
DC.B    $05              ; +MFIELD_DEFENSE (5)
DC.B    $00              ; +$0A
DC.B    $52              ; +MFIELD_LEVEL (faction=2 level=18)
DC.B    $0A              ; +MFIELD_LEVEL_ENC (xpmult=0, base=10)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $00              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_TIGER 197.

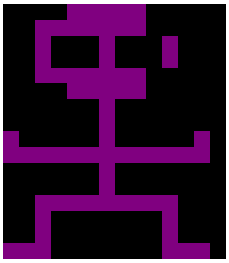


WITLESS ZOMBIE (font character 33, group 4)

136 ⟨*mob data records* 68⟩+≡ (909) ◁135 137▷

```
; --- mob 67: WITLESS ZOMBIE ($50B0) ---
DC.W    s_WITLESS_ZOMBIE              ; +MFIELD_NAME
DC.B    $00               ; +MFIELD_NEXT (0)
DC.B    $73               ; +MFIELD_POSITION (row 5, col 15)
DC.B    $54               ; +MFIELD_APPEARANCE (fontchar=33, group=4 hostile)
DC.B    $0B               ; +MFIELD_STRENGTH (11)
DC.B    $0B               ; +MFIELD_HP (11)
DC.B    $03               ; +MFIELD_ATTACK (3)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $05               ; +MFIELD_DEFENSE (5)
DC.B    $00               ; +$0A
DC.B    $86               ; +MFIELD_LEVEL (faction=4 level=6)
DC.B    $03               ; +MFIELD_LEVEL_ENC (xpmult=0, base=3)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $80               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_WITLESS_ZOMBIE 197.

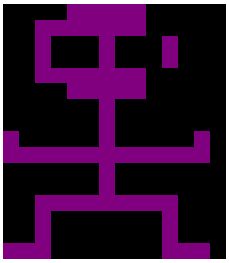


SMALL ZOMBIE (font character 33, group 3)

137 ⟨*mob data records* 68⟩+≡ (909) ◁136 138▷

```
; --- mob 68: SMALL ZOMBIE ($50C0) ---
DC.W    s_SMALL_ZOMBIE              ; +MFIELD_NAME
DC.B    $46              ; +MFIELD_NEXT (70)
DC.B    $88              ; +MFIELD_POSITION (row 6, col 16)
DC.B    $3F              ; +MFIELD_APPEARANCE (fontchar=33, group=3 hostile)
DC.B    $08              ; +MFIELD_STRENGTH (8)
DC.B    $08              ; +MFIELD_HP (8)
DC.B    $02              ; +MFIELD_ATTACK (2)
DC.B    $00              ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $04              ; +MFIELD_DEFENSE (4)
DC.B    $00              ; +$0A
DC.B    $89              ; +MFIELD_LEVEL (faction=4 level=9)
DC.B    $03              ; +MFIELD_LEVEL_ENC (xpmult=0, base=3)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $00              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_SMALL_ZOMBIE 197.

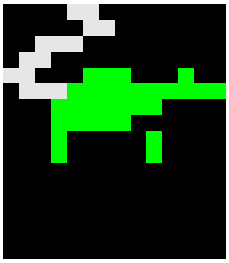


BIG ZOMBIE (font character 33, group 3)

138 ⟨*mob data records* 68⟩+≡ (909) ◁137 139▷

```
; --- mob 69: BIG ZOMBIE ($50D0) ---
DC.W    s_BIG_ZOMBIE              ; +MFIELD_NAME
DC.B    $44               ; +MFIELD_NEXT (68)
DC.B    $9A               ; +MFIELD_POSITION (row 7, col 14)
DC.B    $3F               ; +MFIELD_APPEARANCE (fontchar=33, group=3 hostile)
DC.B    $0D               ; +MFIELD_STRENGTH (13)
DC.B    $0D               ; +MFIELD_HP (13)
DC.B    $04               ; +MFIELD_ATTACK (4)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $05               ; +MFIELD_DEFENSE (5)
DC.B    $00               ; +$0A
DC.B    $87               ; +MFIELD_LEVEL (faction=4 level=7)
DC.B    $03               ; +MFIELD_LEVEL_ENC (xpmult=0, base=3)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_BIG_ZOMBIE 197.

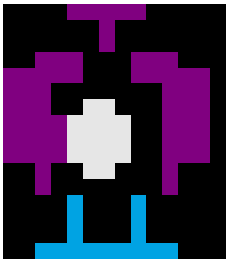


CALICO RAT (font character 7, group 3)

139 ⟨*mob data records* 68⟩+≡ (909) ◁138 140▷

```
; --- mob 70: CALICO RAT ($50E0) ---
DC.W    s_CALICO_RAT              ; +MFIELD_NAME
DC.B    $48               ; +MFIELD_NEXT (72)
DC.B    $4D               ; +MFIELD_POSITION (row 3, col 17)
DC.B    $45               ; +MFIELD_APPEARANCE (fontchar=7, group=3 hostile)
DC.B    $03               ; +MFIELD_STRENGTH (3)
DC.B    $03               ; +MFIELD_HP (3)
DC.B    $02               ; +MFIELD_ATTACK (2)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $03               ; +MFIELD_DEFENSE (3)
DC.B    $00               ; +$0A
DC.B    $4D               ; +MFIELD_LEVEL (faction=2 level=13)
DC.B    $06               ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $84               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_CALICO_RAT 197.

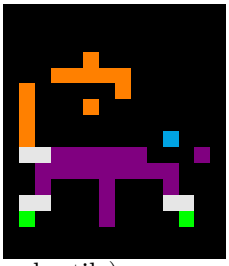


HOOT OWL (font character 19, group 2 — friendly)

140 ⟨*mob data records* 68⟩+≡ (909) ◁139 141▷

```
; --- mob 71: HOOT OWL ($50F0) ---
DC.W    s_HOOT_OWL              ; +MFIELD_NAME
DC.B    $00                ; +MFIELD_NEXT (0)
DC.B    $47                ; +MFIELD_POSITION (row 3, col 11)
DC.B    $3C                ; +MFIELD_APPEARANCE (fontchar=19, group=2 friendly)
DC.B    $05                ; +MFIELD_STRENGTH (5)
DC.B    $05                ; +MFIELD_HP (5)
DC.B    $01                ; +MFIELD_ATTACK (1)
DC.B    $00                ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $00                ; +MFIELD_DEFENSE (0)
DC.B    $00                ; +$0A
DC.B    $4C                ; +MFIELD_LEVEL (faction=2 level=12)
DC.B    $09                ; +MFIELD_LEVEL_ENC (xpmult=0, base=9)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_HOOT_OWL 197.

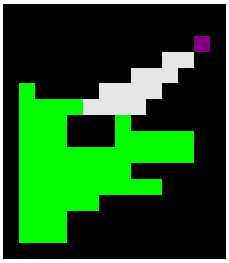


MOUNTAIN SCORPION (font character 14, group 3 — hostile)

141 ⟨*mob data records* 68⟩+≡ (909) ◁140 142▷

```
; --- mob 72: MOUNTAIN SCORPION ($5100) ---
DC.W    s_MOUNTAIN_SCORPION              ; +MFIELD_NAME
DC.B    $00              ; +MFIELD_NEXT (0)
DC.B    $5F              ; +MFIELD_POSITION (row 4, col 15)
DC.B    $4C              ; +MFIELD_APPEARANCE (fontchar=14, group=3 hostile)
DC.B    $08              ; +MFIELD_STRENGTH (8)
DC.B    $06              ; +MFIELD_HP (6)
DC.B    $06              ; +MFIELD_ATTACK (6)
DC.B    $81              ; +MFIELD_XP (st=1 xp=0 grp=1)
DC.B    $08              ; +MFIELD_DEFENSE (8)
DC.B    $00              ; +$0A
DC.B    $09              ; +MFIELD_LEVEL (faction=0 level=9)
DC.B    $05              ; +MFIELD_LEVEL_ENC (xpmult=0, base=5)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $00              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_MOUNTAIN_SCORPION 197.

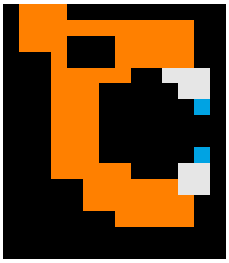


UNICORN (font character 20, group 7 — friendly)

142 ⟨*mob data records* 68⟩+≡ (909) ◁141 143▷

```
; --- mob 73: UNICORN ($5110) ---
DC.W    s_UNICORN               ; +MFIELD_NAME
DC.B    $00               ; +MFIELD_NEXT (0)
DC.B    $81               ; +MFIELD_POSITION (row 6, col 9)
DC.B    $A6               ; +MFIELD_APPEARANCE (fontchar=20, group=7 friendly)
DC.B    $12               ; +MFIELD_STRENGTH (18)
DC.B    $12               ; +MFIELD_HP (18)
DC.B    $07               ; +MFIELD_ATTACK (7)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $04               ; +MFIELD_DEFENSE (4)
DC.B    $00               ; +$0A
DC.B    $58               ; +MFIELD_LEVEL (faction=2 level=24)
DC.B    $0C               ; +MFIELD_LEVEL_ENC (xpmult=0, base=12)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $80               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_UNICORN 197.

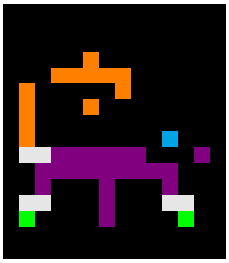


LEO (font character 16, group 3)

143 ⟨*mob data records* 68⟩+≡ (909) ◁142 144▷

```
; --- mob 74: LEO ($5120) ---
DC.W    sm_LEO              ; +MFIELD_NAME
DC.B    $00               ; +MFIELD_NEXT (0)
DC.B    $66               ; +MFIELD_POSITION (row 5, col 2)
DC.B    $4E               ; +MFIELD_APPEARANCE (fontchar=16, group=3 hostile)
DC.B    $0E               ; +MFIELD_STRENGTH (14)
DC.B    $0E               ; +MFIELD_HP (14)
DC.B    $07               ; +MFIELD_ATTACK (7)
DC.B    $81               ; +MFIELD_XP (st=1 xp=0 grp=1)
DC.B    $09               ; +MFIELD_DEFENSE (9)
DC.B    $00               ; +$0A
DC.B    $0E               ; +MFIELD_LEVEL (faction=0 level=14)
DC.B    $0A               ; +MFIELD_LEVEL_ENC (xpmult=0, base=10)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and sm_LEO 197.

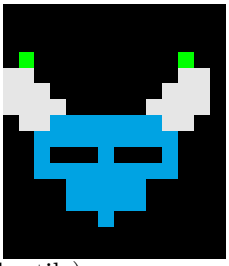


SCORPIUS (font character 14, group 3)

144 ⟨*mob data records* 68⟩+≡ (909) ◁143 145▷

```
; --- mob 75: SCORPIUS ($5130) ---
DC.W    sm_SCORPIUS              ; +MFIELD_NAME
DC.B    $00              ; +MFIELD_NEXT (0)
DC.B    $75              ; +MFIELD_POSITION (row 5, col 17)
DC.B    $4C              ; +MFIELD_APPEARANCE (fontchar=14, group=3 hostile)
DC.B    $0C              ; +MFIELD_STRENGTH (12)
DC.B    $0F              ; +MFIELD_HP (15)
DC.B    $07              ; +MFIELD_ATTACK (7)
DC.B    $53              ; +MFIELD_XP (st=0 xp=5 grp=3)
DC.B    $07              ; +MFIELD_DEFENSE (7)
DC.B    $00              ; +$0A
DC.B    $0D              ; +MFIELD_LEVEL (faction=0 level=13)
DC.B    $04              ; +MFIELD_LEVEL_ENC (xpmult=0, base=4)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $00              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and sm_SCORPIUS 197.

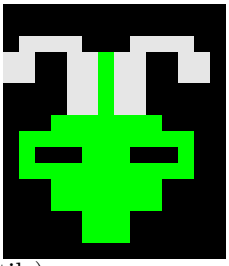


STATUE OF TAURUS (font character 18, group 3 — hostile)

145 ⟨*mob data records* 68⟩+≡ (909) ◁144 146▷

```
; --- mob 76: STATUE OF TAURUS ($5140) ---
DC.W    s_STATUE_OF_TAURUS              ; +MFIELD_NAME
DC.B    $00                ; +MFIELD_NEXT (0)
DC.B    $52                ; +MFIELD_POSITION (row 4, col 2)
DC.B    $50                ; +MFIELD_APPEARANCE (fontchar=18, group=3 hostile)
DC.B    $14                ; +MFIELD_STRENGTH (20)
DC.B    $14                ; +MFIELD_HP (20)
DC.B    $00                ; +MFIELD_ATTACK (0)
DC.B    $81                ; +MFIELD_XP (st=1 xp=0 grp=1)
DC.B    $08                ; +MFIELD_DEFENSE (8)
DC.B    $00                ; +$0A
DC.B    $14                ; +MFIELD_LEVEL (faction=0 level=20)
DC.B    $00                ; +MFIELD_LEVEL_ENC (xpmult=0, base=0)
DC.W    $0032                ; +MFIELD_GOLD (50 ducats)
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_STATUE_OF_TAURUS 197.

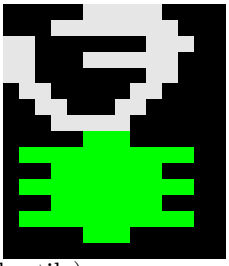


ARIES, THE RAM (font character 13, group 3 — hostile)

146 ⟨*mob data records* 68⟩+≡ (909) ◁145 147▷

```
; --- mob 77: ARIES, THE RAM ($5150) ---
DC.W    s_ARIES_THE_RAM              ; +MFIELD_NAME
DC.B    $00                ; +MFIELD_NEXT (0)
DC.B    $56                ; +MFIELD_POSITION (row 4, col 6)
DC.B    $4B                ; +MFIELD_APPEARANCE (fontchar=13, group=3 hostile)
DC.B    $10                ; +MFIELD_STRENGTH (16)
DC.B    $10                ; +MFIELD_HP (16)
DC.B    $07                ; +MFIELD_ATTACK (7)
DC.B    $00                ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $05                ; +MFIELD_DEFENSE (5)
DC.B    $00                ; +$0A
DC.B    $0F                ; +MFIELD_LEVEL (faction=0 level=15)
DC.B    $08                ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_ARIES_THE_RAM 197.

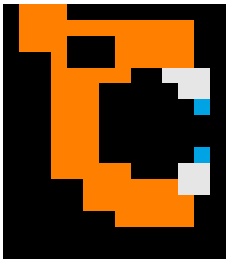


CANCER, THE CRAB (font character 17, group 3 — hostile)

147 ⟨*mob data records* 68⟩+≡ (909) ◁146 148▷

```
; --- mob 78: CANCER, THE CRAB ($5160) ---
DC.W    s_CANCER_THE_CRAB              ; +MFIELD_NAME
DC.B    $00                ; +MFIELD_NEXT (0)
DC.B    $94                ; +MFIELD_POSITION (row 7, col 8)
DC.B    $4F                ; +MFIELD_APPEARANCE (fontchar=17, group=3 hostile)
DC.B    $0A                ; +MFIELD_STRENGTH (10)
DC.B    $0A                ; +MFIELD_HP (10)
DC.B    $05                ; +MFIELD_ATTACK (5)
DC.B    $04                ; +MFIELD_XP (st=0 xp=0 grp=4)
DC.B    $05                ; +MFIELD_DEFENSE (5)
DC.B    $00                ; +$0A
DC.B    $CA                ; +MFIELD_LEVEL (faction=6 level=10)
DC.B    $03                ; +MFIELD_LEVEL_ENC (xpmult=0, base=3)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_CANCER_THE_CRAB 197.

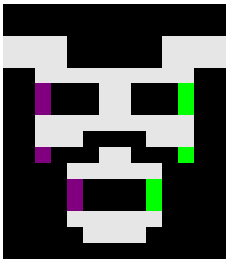


WEREJACKAL (font character 16, group 3)

148 ⟨*mob data records* 68⟩+≡ (909) ◁147 149▷

```
; --- mob 79: WEREJACKAL ($5170) ---
DC.W    s_WEREJACKAL              ; +MFIELD_NAME
DC.B    $00               ; +MFIELD_NEXT (0)
DC.B    $3F               ; +MFIELD_POSITION (row 3, col 3)
DC.B    $4E               ; +MFIELD_APPEARANCE (fontchar=16, group=3 hostile)
DC.B    $0A               ; +MFIELD_STRENGTH (10)
DC.B    $0A               ; +MFIELD_HP (10)
DC.B    $06               ; +MFIELD_ATTACK (6)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $07               ; +MFIELD_DEFENSE (7)
DC.B    $00               ; +$0A
DC.B    $4D               ; +MFIELD_LEVEL (faction=2 level=13)
DC.B    $09               ; +MFIELD_LEVEL_ENC (xpmult=0, base=9)
DC.W    $0032               ; +MFIELD_GOLD (50 ducats)
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_WEREJACKAL 197.

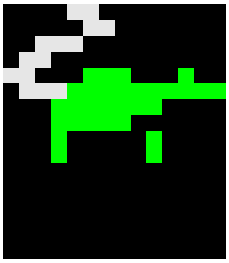


CAVEBEAR (font character 6, group 3 — hostile)

149 ⟨*mob data records* 68⟩+≡ (909) ◁148 150▷

```
; --- mob 80: CAVEBEAR ($5180) ---
DC.W    s_CAVEBEAR              ; +MFIELD_NAME
DC.B    $00                ; +MFIELD_NEXT (0)
DC.B    $2E                ; +MFIELD_POSITION (row 2, col 6)
DC.B    $44                ; +MFIELD_APPEARANCE (fontchar=6, group=3 hostile)
DC.B    $12                ; +MFIELD_STRENGTH (18)
DC.B    $18                ; +MFIELD_HP (24)
DC.B    $06                ; +MFIELD_ATTACK (6)
DC.B    $00                ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $08                ; +MFIELD_DEFENSE (8)
DC.B    $00                ; +$0A
DC.B    $86                ; +MFIELD_LEVEL (faction=4 level=6)
DC.B    $04                ; +MFIELD_LEVEL_ENC (xpmult=0, base=4)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_CAVEBEAR 197.

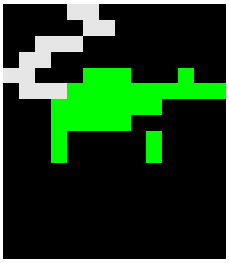


CAVE RAT (font character 7, group 3)

150 ⟨*mob data records* 68⟩+≡ (909) ◁149 151▷

```
; --- mob 81: CAVE RAT ($5190) ---
DC.W    s_CAVE_RAT              ; +MFIELD_NAME
DC.B    $53             ; +MFIELD_NEXT (83)
DC.B    $98             ; +MFIELD_POSITION (row 7, col 12)
DC.B    $45             ; +MFIELD_APPEARANCE (fontchar=7, group=3 hostile)
DC.B    $06             ; +MFIELD_STRENGTH (6)
DC.B    $06             ; +MFIELD_HP (6)
DC.B    $03             ; +MFIELD_ATTACK (3)
DC.B    $81             ; +MFIELD_XP (st=1 xp=0 grp=1)
DC.B    $04             ; +MFIELD_DEFENSE (4)
DC.B    $00             ; +$0A
DC.B    $4F             ; +MFIELD_LEVEL (faction=2 level=15)
DC.B    $06             ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000             ; +MFIELD_GOLD
DC.B    $00             ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_CAVE_RAT 197.

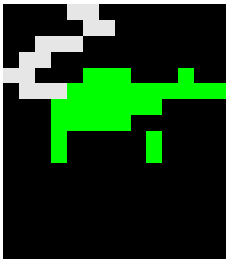


BROWN RAT (font character 7, group 3)

151 ⟨*mob data records* 68⟩+≡ (909) ◁150 152▷

```
; --- mob 82: BROWN RAT ($51A0) ---
DC.W    s_BROWN_RAT              ; +MFIELD_NAME
DC.B    $54              ; +MFIELD_NEXT (84)
DC.B    $AB              ; +MFIELD_POSITION (row 8, col 11)
DC.B    $45              ; +MFIELD_APPEARANCE (fontchar=7, group=3 hostile)
DC.B    $03              ; +MFIELD_STRENGTH (3)
DC.B    $03              ; +MFIELD_HP (3)
DC.B    $02              ; +MFIELD_ATTACK (2)
DC.B    $00              ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $02              ; +MFIELD_DEFENSE (2)
DC.B    $00              ; +$0A
DC.B    $4A              ; +MFIELD_LEVEL (faction=2 level=10)
DC.B    $06              ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $A0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_BROWN_RAT 197.

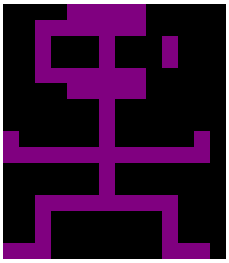


BLACK RAT (font character 7, group 3)

152 ⟨*mob data records* 68⟩+≡ (909) ◁151 153▷

```
; --- mob 83: BLACK RAT ($51B0) ---
DC.W    s_BLACK_RAT              ; +MFIELD_NAME
DC.B    $00              ; +MFIELD_NEXT (0)
DC.B    $AD              ; +MFIELD_POSITION (row 8, col 13)
DC.B    $45              ; +MFIELD_APPEARANCE (fontchar=7, group=3 hostile)
DC.B    $03              ; +MFIELD_STRENGTH (3)
DC.B    $03              ; +MFIELD_HP (3)
DC.B    $02              ; +MFIELD_ATTACK (2)
DC.B    $00              ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $02              ; +MFIELD_DEFENSE (2)
DC.B    $00              ; +$0A
DC.B    $47              ; +MFIELD_LEVEL (faction=2 level=7)
DC.B    $06              ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $A0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_BLACK_RAT 197.

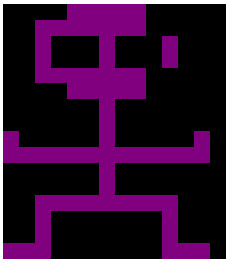


GEMINI I (font character 33, group 7)

153 ⟨*mob data records* 68⟩+≡ (909) ◁152 154▷

```
; --- mob 84: GEMINI I ($51C0) ---
DC.W    s_GEMINI_I              ; +MFIELD_NAME
DC.B    $56               ; +MFIELD_NEXT (86)
DC.B    $1C               ; +MFIELD_POSITION (row 1, col 8)
DC.B    $93               ; +MFIELD_APPEARANCE (fontchar=33, group=7 friendly)
DC.B    $0A               ; +MFIELD_STRENGTH (10)
DC.B    $0F               ; +MFIELD_HP (15)
DC.B    $0F               ; +MFIELD_ATTACK (15)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $0F               ; +MFIELD_DEFENSE (15)
DC.B    $00               ; +$0A
DC.B    $10               ; +MFIELD_LEVEL (faction=0 level=16)
DC.B    $00               ; +MFIELD_LEVEL_ENC (xpmult=0, base=0)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_GEMINI_I 197.

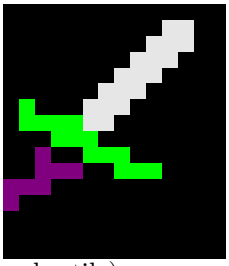


GEMINI II (font character 33, group 7)

154 ⟨*mob data records* 68⟩+≡ (909) ◁153 155▷

```
; --- mob 85: GEMINI II ($51D0) ---
DC.W    s_GEMINI_II              ; +MFIELD_NAME
DC.B    $40              ; +MFIELD_NEXT (64)
DC.B    $44              ; +MFIELD_POSITION (row 3, col 8)
DC.B    $93              ; +MFIELD_APPEARANCE (fontchar=33, group=7 friendly)
DC.B    $0A              ; +MFIELD_STRENGTH (10)
DC.B    $0F              ; +MFIELD_HP (15)
DC.B    $0F              ; +MFIELD_ATTACK (15)
DC.B    $00              ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $0F              ; +MFIELD_DEFENSE (15)
DC.B    $00              ; +$0A
DC.B    $10              ; +MFIELD_LEVEL (faction=0 level=16)
DC.B    $00              ; +MFIELD_LEVEL_ENC (xpmult=0, base=0)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $00              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_GEMINI_II 197.

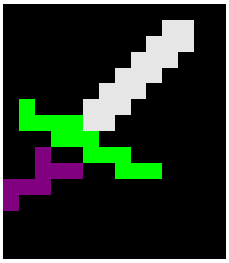


SWORD OF DAMOCLES (font character 12, group 3 — hostile)

155 ⟨*mob data records* 68⟩+≡ (909) ◁154 156▷

```
; --- mob 86: SWORD OF DAMOCLES ($51E0) ---
DC.W    s_SWORD_OF_DAMOCLES              ; +MFIELD_NAME
DC.B    $59              ; +MFIELD_NEXT (89)
DC.B    $1C              ; +MFIELD_POSITION (row 1, col 8)
DC.B    $4A              ; +MFIELD_APPEARANCE (fontchar=12, group=3 hostile)
DC.B    $0F              ; +MFIELD_STRENGTH (15)
DC.B    $0F              ; +MFIELD_HP (15)
DC.B    $05              ; +MFIELD_ATTACK (5)
DC.B    $00              ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $00              ; +MFIELD_DEFENSE (0)
DC.B    $00              ; +$0A
DC.B    $0E              ; +MFIELD_LEVEL (faction=0 level=14)
DC.B    $00              ; +MFIELD_LEVEL_ENC (xpmult=0, base=0)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $00              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_SWORD_OF_DAMOCLES 197.

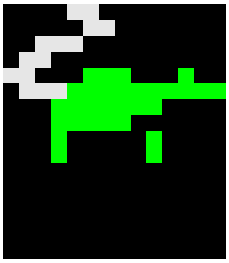


TWO EDGED SWORD (font character 12, group 3)

156 ⟨*mob data records* 68⟩+≡ (909) ◁155 157▷

```
; --- mob 87: TWO EDGED SWORD ($51F0) ---
DC.W    s_TWO_EDGED_SWORD              ; +MFIELD_NAME
DC.B    $00                ; +MFIELD_NEXT (0)
DC.B    $1E                ; +MFIELD_POSITION (row 1, col 10)
DC.B    $4A                ; +MFIELD_APPEARANCE (fontchar=12, group=3 hostile)
DC.B    $0F                ; +MFIELD_STRENGTH (15)
DC.B    $0F                ; +MFIELD_HP (15)
DC.B    $08                ; +MFIELD_ATTACK (8)
DC.B    $00                ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $00                ; +MFIELD_DEFENSE (0)
DC.B    $00                ; +$0A
DC.B    $07                ; +MFIELD_LEVEL (faction=0 level=7)
DC.B    $00                ; +MFIELD_LEVEL_ENC (xpmult=0, base=0)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_TWO_EDGED_SWORD 197.

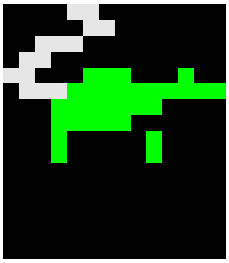


WERERAT (font character 7, group 3)

157 ⟨*mob data records* 68⟩+≡ (909) ◁156 158▷

```
; --- mob 88: WERERAT ($5200) ---
DC.W    s_WERERAT               ; +MFIELD_NAME
DC.B    $5B               ; +MFIELD_NEXT (91)
DC.B    $55               ; +MFIELD_POSITION (row 4, col 5)
DC.B    $45               ; +MFIELD_APPEARANCE (fontchar=7, group=3 hostile)
DC.B    $08               ; +MFIELD_STRENGTH (8)
DC.B    $08               ; +MFIELD_HP (8)
DC.B    $04               ; +MFIELD_ATTACK (4)
DC.B    $81               ; +MFIELD_XP (st=1 xp=0 grp=1)
DC.B    $04               ; +MFIELD_DEFENSE (4)
DC.B    $00               ; +$0A
DC.B    $4C               ; +MFIELD_LEVEL (faction=2 level=12)
DC.B    $06               ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $C0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_WERERAT 197.

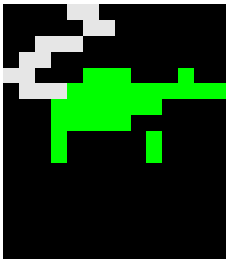


SEWER RAT (font character 7, group 3)

158 ⟨*mob data records* 68⟩+≡ (909) ◁157 159▷

```
; --- mob 89: SEWER RAT ($5210) ---
DC.W    s_SEWER_RAT              ; +MFIELD_NAME
DC.B    $58              ; +MFIELD_NEXT (88)
DC.B    $5A              ; +MFIELD_POSITION (row 4, col 10)
DC.B    $45              ; +MFIELD_APPEARANCE (fontchar=7, group=3 hostile)
DC.B    $0A              ; +MFIELD_STRENGTH (10)
DC.B    $0A              ; +MFIELD_HP (10)
DC.B    $05              ; +MFIELD_ATTACK (5)
DC.B    $82              ; +MFIELD_XP (st=1 xp=0 grp=2)
DC.B    $06              ; +MFIELD_DEFENSE (6)
DC.B    $00              ; +$0A
DC.B    $48              ; +MFIELD_LEVEL (faction=2 level=8)
DC.B    $06              ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $C0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_SEWER_RAT 197.

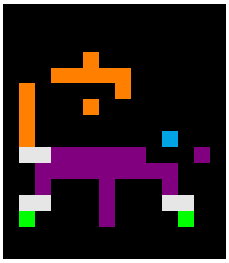


JUNGLE RAT (font character 7, group 3)

159 ⟨*mob data records* 68⟩+≡ (909) ◁158 160▷

```
; --- mob 90: JUNGLE RAT ($5220) ---
DC.W    s_JUNGLE_RAT              ; +MFIELD_NAME
DC.B    $5A               ; +MFIELD_NEXT (90)
DC.B    $5E               ; +MFIELD_POSITION (row 4, col 14)
DC.B    $45               ; +MFIELD_APPEARANCE (fontchar=7, group=3 hostile)
DC.B    $09               ; +MFIELD_STRENGTH (9)
DC.B    $09               ; +MFIELD_HP (9)
DC.B    $04               ; +MFIELD_ATTACK (4)
DC.B    $81               ; +MFIELD_XP (st=1 xp=0 grp=1)
DC.B    $04               ; +MFIELD_DEFENSE (4)
DC.B    $00               ; +$0A
DC.B    $4A               ; +MFIELD_LEVEL (faction=2 level=10)
DC.B    $06               ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $C0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_JUNGLE_RAT 197.

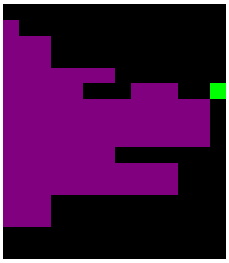


GIANT SCORPION (font character 14, group 4)

160 ⟨*mob data records* 68⟩+≡ (909) ◁159 161▷

```
; --- mob 91: GIANT SCORPION ($5230) ---
DC.W    s_GIANT_SCORPION              ; +MFIELD_NAME
DC.B    $00               ; +MFIELD_NEXT (0)
DC.B    $7C               ; +MFIELD_POSITION (row 6, col 4)
DC.B    $61               ; +MFIELD_APPEARANCE (fontchar=14, group=4 hostile)
DC.B    $0C               ; +MFIELD_STRENGTH (12)
DC.B    $0C               ; +MFIELD_HP (12)
DC.B    $08               ; +MFIELD_ATTACK (8)
DC.B    $82               ; +MFIELD_XP (st=1 xp=0 grp=2)
DC.B    $08               ; +MFIELD_DEFENSE (8)
DC.B    $00               ; +$0A
DC.B    $0B               ; +MFIELD_LEVEL (faction=0 level=11)
DC.B    $06               ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_GIANT_SCORPION 197.

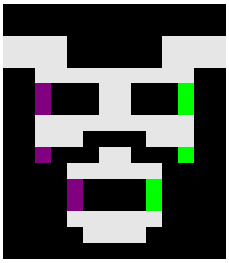


WEREWOLF (font character 8, group 3 — hostile)

161 ⟨*mob data records* 68⟩+≡ (909) ◁160 162▷

```
; --- mob 92: WEREWOLF ($5240) ---
DC.W    s_WEREWOLF              ; +MFIELD_NAME
DC.B    $00                ; +MFIELD_NEXT (0)
DC.B    $87                ; +MFIELD_POSITION (row 6, col 15)
DC.B    $46                ; +MFIELD_APPEARANCE (fontchar=8, group=3 hostile)
DC.B    $0E                ; +MFIELD_STRENGTH (14)
DC.B    $0E                ; +MFIELD_HP (14)
DC.B    $06                ; +MFIELD_ATTACK (6)
DC.B    $81                ; +MFIELD_XP (st=1 xp=0 grp=1)
DC.B    $08                ; +MFIELD_DEFENSE (8)
DC.B    $00                ; +$0A
DC.B    $0C                ; +MFIELD_LEVEL (faction=0 level=12)
DC.B    $0A                ; +MFIELD_LEVEL_ENC (xpmult=0, base=10)
DC.W    $0064                ; +MFIELD_GOLD (100 ducats)
DC.B    $C0                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_WEREWOLF 197.

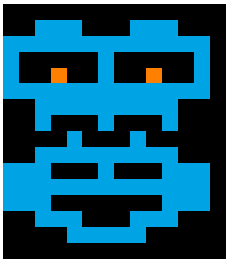


GRIZZLY BEAR (font character 6, group 3)

162 ⟨*mob data records* 68⟩+≡ (909) ◁161 163▷

```
; --- mob 93: GRIZZLY BEAR ($5250) ---
DC.W    s_GRIZZLY_BEAR              ; +MFIELD_NAME
DC.B    $5D                ; +MFIELD_NEXT (93)
DC.B    $99                ; +MFIELD_POSITION (row 7, col 13)
DC.B    $44                ; +MFIELD_APPEARANCE (fontchar=6, group=3 hostile)
DC.B    $14                ; +MFIELD_STRENGTH (20)
DC.B    $1E                ; +MFIELD_HP (30)
DC.B    $09                ; +MFIELD_ATTACK (9)
DC.B    $82                ; +MFIELD_XP (st=1 xp=0 grp=2)
DC.B    $0B                ; +MFIELD_DEFENSE (11)
DC.B    $00                ; +$0A
DC.B    $C5                ; +MFIELD_LEVEL (faction=6 level=5)
DC.B    $04                ; +MFIELD_LEVEL_ENC (xpmult=0, base=4)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $C0                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_GRIZZLY_BEAR 197.

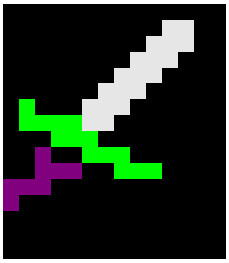


GREEN DRAGON (font character 9, group 3 — hostile)

163 ⟨*mob data records* 68⟩+≡ (909) ◁162 164▷

```
; --- mob 94: GREEN DRAGON ($5260) ---
DC.W    s_GREEN_DRAGON              ; +MFIELD_NAME
DC.B    $00                ; +MFIELD_NEXT (0)
DC.B    $67                ; +MFIELD_POSITION (row 5, col 3)
DC.B    $47                ; +MFIELD_APPEARANCE (fontchar=9, group=3 hostile)
DC.B    $1F                ; +MFIELD_STRENGTH (31)
DC.B    $30                ; +MFIELD_HP (48)
DC.B    $0C                ; +MFIELD_ATTACK (12)
DC.B    $85                ; +MFIELD_XP (st=1 xp=0 grp=5)
DC.B    $08                ; +MFIELD_DEFENSE (8)
DC.B    $00                ; +$0A
DC.B    $89                ; +MFIELD_LEVEL (faction=4 level=9)
DC.B    $1C                ; +MFIELD_LEVEL_ENC (xpmult=1, base=12)
DC.W    $012C                ; +MFIELD_GOLD (300 ducats)
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_GREEN_DRAGON 197.

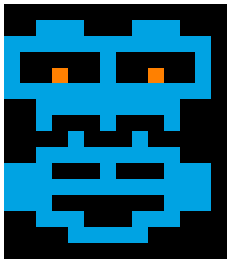


MECHANICAL SWORD (font character 12, group 4)

164 ⟨*mob data records* 68⟩+≡ (909) ◁163 165▷

```
; --- mob 95: MECHANICAL SWORD ($5270) ---
DC.W    s_MECHANICAL_SWORD              ; +MFIELD_NAME
DC.B    $00               ; +MFIELD_NEXT (0)
DC.B    $71               ; +MFIELD_POSITION (row 5, col 13)
DC.B    $5F               ; +MFIELD_APPEARANCE (fontchar=12, group=4 hostile)
DC.B    $05               ; +MFIELD_STRENGTH (5)
DC.B    $05               ; +MFIELD_HP (5)
DC.B    $08               ; +MFIELD_ATTACK (8)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $01               ; +MFIELD_DEFENSE (1)
DC.B    $00               ; +$0A
DC.B    $22               ; +MFIELD_LEVEL (faction=1 level=2)
DC.B    $00               ; +MFIELD_LEVEL_ENC (xpmult=0, base=0)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_MECHANICAL_SWORD 197.

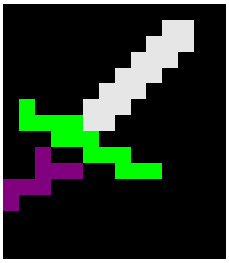


VERY YOUNG DRAGON (font character 9, group 3)

165 ⟨*mob data records* 68⟩+≡ (909) ◁164 166▷

```
; --- mob 96: VERY YOUNG DRAGON ($5280) ---
DC.W    s_VERY_YOUNG_DRAGON              ; +MFIELD_NAME
DC.B    $00              ; +MFIELD_NEXT (0)
DC.B    $90              ; +MFIELD_POSITION (row 7, col 4)
DC.B    $47              ; +MFIELD_APPEARANCE (fontchar=9, group=3 hostile)
DC.B    $10              ; +MFIELD_STRENGTH (16)
DC.B    $10              ; +MFIELD_HP (16)
DC.B    $07              ; +MFIELD_ATTACK (7)
DC.B    $81              ; +MFIELD_XP (st=1 xp=0 grp=1)
DC.B    $04              ; +MFIELD_DEFENSE (4)
DC.B    $00              ; +$0A
DC.B    $CA              ; +MFIELD_LEVEL (faction=6 level=10)
DC.B    $0A              ; +MFIELD_LEVEL_ENC (xpmult=0, base=10)
DC.W    $0064              ; +MFIELD_GOLD (100 ducats)
DC.B    $00              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_VERY_YOUNG_DRAGON 197.

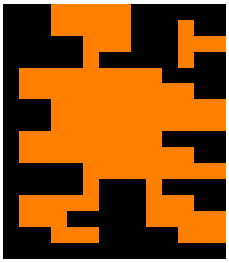


ENCHANTED SWORD (font character 12, group 3)

166 ⟨*mob data records* 68⟩+≡ (909) ◁165 167▷

```
; --- mob 97: ENCHANTED SWORD ($5290) ---
DC.W    s_ENCHANTED_SWORD              ; +MFIELD_NAME
DC.B    $00                ; +MFIELD_NEXT (0)
DC.B    $87                ; +MFIELD_POSITION (row 6, col 15)
DC.B    $4A                ; +MFIELD_APPEARANCE (fontchar=12, group=3 hostile)
DC.B    $01                ; +MFIELD_STRENGTH (1)
DC.B    $0A                ; +MFIELD_HP (10)
DC.B    $07                ; +MFIELD_ATTACK (7)
DC.B    $00                ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $02                ; +MFIELD_DEFENSE (2)
DC.B    $00                ; +$0A
DC.B    $0D                ; +MFIELD_LEVEL (faction=0 level=13)
DC.B    $00                ; +MFIELD_LEVEL_ENC (xpmult=0, base=0)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_ENCHANTED_SWORD 197.

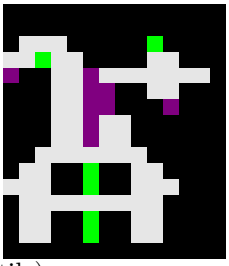


THE BLOB (font character 10, group 3 — hostile)

167 ⟨*mob data records* 68⟩+≡ (909) ◁166 168▷

```
; --- mob 98: THE BLOB ($52A0) ---
DC.W    s_THE_BLOB              ; +MFIELD_NAME
DC.B    $00               ; +MFIELD_NEXT (0)
DC.B    $87               ; +MFIELD_POSITION (row 6, col 15)
DC.B    $48               ; +MFIELD_APPEARANCE (fontchar=10, group=3 hostile)
DC.B    $16               ; +MFIELD_STRENGTH (22)
DC.B    $18               ; +MFIELD_HP (24)
DC.B    $03               ; +MFIELD_ATTACK (3)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $08               ; +MFIELD_DEFENSE (8)
DC.B    $00               ; +$0A
DC.B    $07               ; +MFIELD_LEVEL (faction=0 level=7)
DC.B    $03               ; +MFIELD_LEVEL_ENC (xpmult=0, base=3)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_THE_BLOB 197.

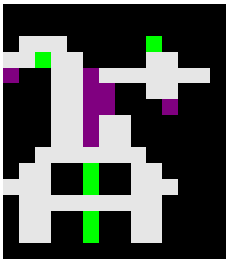


SUN WANDERER (font character 11, group 4 — hostile)

168 ⟨*mob data records* 68⟩+≡ (909) ◁167 169▷

```
; --- mob 99: SUN WANDERER ($52B0) ---
DC.W    s_SUN_WANDERER              ; +MFIELD_NAME
DC.B    $66                 ; +MFIELD_NEXT (102)
DC.B    $3F                 ; +MFIELD_POSITION (row 3, col 3)
DC.B    $5E                 ; +MFIELD_APPEARANCE (fontchar=11, group=4 hostile)
DC.B    $05                 ; +MFIELD_STRENGTH (5)
DC.B    $0A                 ; +MFIELD_HP (10)
DC.B    $03                 ; +MFIELD_ATTACK (3)
DC.B    $04                 ; +MFIELD_XP (st=0 xp=0 grp=4)
DC.B    $03                 ; +MFIELD_DEFENSE (3)
DC.B    $00                 ; +$0A
DC.B    $11                 ; +MFIELD_LEVEL (faction=0 level=17)
DC.B    $13                 ; +MFIELD_LEVEL_ENC (xpmult=1, base=3)
DC.W    $0028                 ; +MFIELD_GOLD (40 ducats)
DC.B    $A0                 ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_SUN_WANDERER 197.

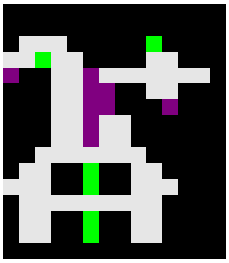


EARTH WANDERER (font character 11, group 4)

169 ⟨*mob data records* 68⟩+≡ (909) ◁168 170▷

```
; --- mob 100: EARTH WANDERER ($52C0) ---
DC.W    s_EARTH_WANDERER              ; +MFIELD_NAME
DC.B    $67               ; +MFIELD_NEXT (103)
DC.B    $5C               ; +MFIELD_POSITION (row 4, col 12)
DC.B    $5E               ; +MFIELD_APPEARANCE (fontchar=11, group=4 hostile)
DC.B    $05               ; +MFIELD_STRENGTH (5)
DC.B    $0A               ; +MFIELD_HP (10)
DC.B    $03               ; +MFIELD_ATTACK (3)
DC.B    $04               ; +MFIELD_XP (st=0 xp=0 grp=4)
DC.B    $03               ; +MFIELD_DEFENSE (3)
DC.B    $00               ; +$0A
DC.B    $11               ; +MFIELD_LEVEL (faction=0 level=17)
DC.B    $05               ; +MFIELD_LEVEL_ENC (xpmult=0, base=5)
DC.W    $0019               ; +MFIELD_GOLD (25 ducats)
DC.B    $A0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_EARTH_WANDERER 197.

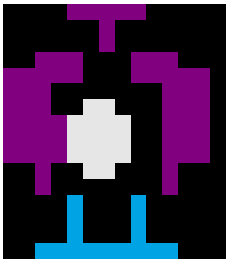


SKY WANDERER (font character 11, group 4)

170 ⟨*mob data records* 68⟩+≡ (909) ◁169 171▷

```
; --- mob 101: SKY WANDERER ($52D0) ---
DC.W    s_SKY_WANDERER              ; +MFIELD_NAME
DC.B    $65                 ; +MFIELD_NEXT (101)
DC.B    $7F                 ; +MFIELD_POSITION (row 6, col 7)
DC.B    $5E                 ; +MFIELD_APPEARANCE (fontchar=11, group=4 hostile)
DC.B    $05                 ; +MFIELD_STRENGTH (5)
DC.B    $0A                 ; +MFIELD_HP (10)
DC.B    $03                 ; +MFIELD_ATTACK (3)
DC.B    $04                 ; +MFIELD_XP (st=0 xp=0 grp=4)
DC.B    $03                 ; +MFIELD_DEFENSE (3)
DC.B    $00                 ; +$0A
DC.B    $11                 ; +MFIELD_LEVEL (faction=0 level=17)
DC.B    $07                 ; +MFIELD_LEVEL_ENC (xpmult=0, base=7)
DC.W    $0023                 ; +MFIELD_GOLD (35 ducats)
DC.B    $A0                 ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_SKY_WANDERER 197.

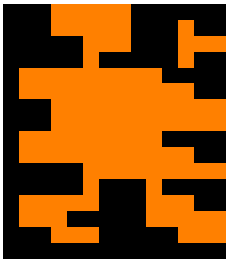


LOST BARN OWL (font character 19, group 2)

171 ⟨*mob data records* 68⟩+≡ (909) ◁170 172▷

```
; --- mob 102: LOST BARN OWL ($52E0) ---
DC.W    s_LOST_BARN_OWL              ; +MFIELD_NAME
DC.B    $00                ; +MFIELD_NEXT (0)
DC.B    $88                ; +MFIELD_POSITION (row 6, col 16)
DC.B    $3C                ; +MFIELD_APPEARANCE (fontchar=19, group=2 friendly)
DC.B    $05                ; +MFIELD_STRENGTH (5)
DC.B    $06                ; +MFIELD_HP (6)
DC.B    $01                ; +MFIELD_ATTACK (1)
DC.B    $00                ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $01                ; +MFIELD_DEFENSE (1)
DC.B    $00                ; +$0A
DC.B    $0E                ; +MFIELD_LEVEL (faction=0 level=14)
DC.B    $09                ; +MFIELD_LEVEL_ENC (xpmult=0, base=9)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_LOST_BARN_OWL 197.

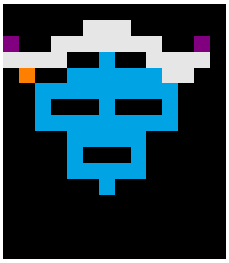


SLIME (font character 10, group 3)

172 ⟨*mob data records* 68⟩+≡ (909) ◁171 173▷

```
; --- mob 103: SLIME ($52F0) ---
DC.W    s_SLIME              ; +MFIELD_NAME
DC.B    $00              ; +MFIELD_NEXT (0)
DC.B    $4C              ; +MFIELD_POSITION (row 3, col 16)
DC.B    $48              ; +MFIELD_APPEARANCE (fontchar=10, group=3 hostile)
DC.B    $10              ; +MFIELD_STRENGTH (16)
DC.B    $14              ; +MFIELD_HP (20)
DC.B    $05              ; +MFIELD_ATTACK (5)
DC.B    $00              ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $06              ; +MFIELD_DEFENSE (6)
DC.B    $00              ; +$0A
DC.B    $0D              ; +MFIELD_LEVEL (faction=0 level=13)
DC.B    $02              ; +MFIELD_LEVEL_ENC (xpmult=0, base=2)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $00              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_SLIME 197.

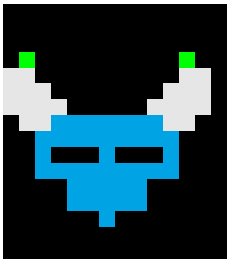


SUCCUBUS (font character 15, group 3)

173 ⟨*mob data records* 68⟩+≡ (909) ◁172 174▷

```
; --- mob 104: SUCCUBUS ($5300) ---
DC.W    s_SUCCUBUS              ; +MFIELD_NAME
DC.B    $00                 ; +MFIELD_NEXT (0)
DC.B    $58                 ; +MFIELD_POSITION (row 4, col 8)
DC.B    $4D                 ; +MFIELD_APPEARANCE (fontchar=15, group=3 hostile)
DC.B    $0A                 ; +MFIELD_STRENGTH (10)
DC.B    $0A                 ; +MFIELD_HP (10)
DC.B    $01                 ; +MFIELD_ATTACK (1)
DC.B    $00                 ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $09                 ; +MFIELD_DEFENSE (9)
DC.B    $00                 ; +$0A
DC.B    $0E                 ; +MFIELD_LEVEL (faction=0 level=14)
DC.B    $15                 ; +MFIELD_LEVEL_ENC (xpmult=1, base=5)
DC.W    $0064                 ; +MFIELD_GOLD (100 ducats)
DC.B    $00                 ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_SUCCUBUS 197.

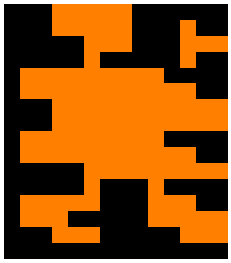


STATUE OF MINOTAUR (font character 18, group 3)

174 ⟨*mob data records* 68⟩+≡ (909) ◁173 175▷

```
; --- mob 105: STATUE OF MINOTAUR ($5310) ---
DC.W    s_STATUE_OF_MINOTAUR              ; +MFIELD_NAME
DC.B    $00               ; +MFIELD_NEXT (0)
DC.B    $5E               ; +MFIELD_POSITION (row 4, col 14)
DC.B    $50               ; +MFIELD_APPEARANCE (fontchar=18, group=3 hostile)
DC.B    $14               ; +MFIELD_STRENGTH (20)
DC.B    $1E               ; +MFIELD_HP (30)
DC.B    $0A               ; +MFIELD_ATTACK (10)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $04               ; +MFIELD_DEFENSE (4)
DC.B    $00               ; +$0A
DC.B    $8B               ; +MFIELD_LEVEL (faction=4 level=11)
DC.B    $00               ; +MFIELD_LEVEL_ENC (xpmult=0, base=0)
DC.W    $00FA               ; +MFIELD_GOLD (250 ducats)
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_STATUE_OF_MINOTAUR 197.

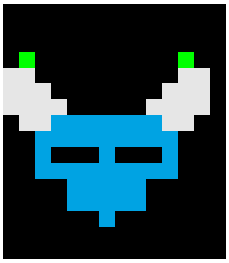


OOZING STENCHBEAST (font character 10, group 3)

175 ⟨*mob data records* 68⟩+≡ (909) ◁174 176▷

```
; --- mob 106: OOZING STENCHBEAST ($5320) ---
DC.W    s_OOZING_STENCHBEAST              ; +MFIELD_NAME
DC.B    $00               ; +MFIELD_NEXT (0)
DC.B    $70               ; +MFIELD_POSITION (row 5, col 12)
DC.B    $48               ; +MFIELD_APPEARANCE (fontchar=10, group=3 hostile)
DC.B    $1F               ; +MFIELD_STRENGTH (31)
DC.B    $10               ; +MFIELD_HP (16)
DC.B    $02               ; +MFIELD_ATTACK (2)
DC.B    $83               ; +MFIELD_XP (st=1 xp=0 grp=3)
DC.B    $08               ; +MFIELD_DEFENSE (8)
DC.B    $00               ; +$0A
DC.B    $CB               ; +MFIELD_LEVEL (faction=6 level=11)
DC.B    $04               ; +MFIELD_LEVEL_ENC (xpmult=0, base=4)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $00               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_OOZING_STENCHBEAST 197.

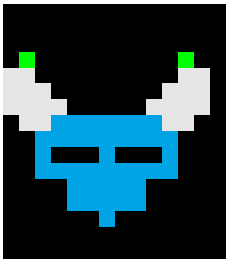


SON OF MINOTAUR (font character 18, group 3)

176 ⟨*mob data records* 68⟩+≡ (909) ◁175 177▷

```
; --- mob 107: SON OF MINOTAUR ($5330) ---
DC.W    s_SON_OF_MINOTAUR              ; +MFIELD_NAME
DC.B    $00                ; +MFIELD_NEXT (0)
DC.B    $62                ; +MFIELD_POSITION (row 4, col 18)
DC.B    $50                ; +MFIELD_APPEARANCE (fontchar=18, group=3 hostile)
DC.B    $14                ; +MFIELD_STRENGTH (20)
DC.B    $14                ; +MFIELD_HP (20)
DC.B    $08                ; +MFIELD_ATTACK (8)
DC.B    $82                ; +MFIELD_XP (st=1 xp=0 grp=2)
DC.B    $0A                ; +MFIELD_DEFENSE (10)
DC.B    $00                ; +$0A
DC.B    $0C                ; +MFIELD_LEVEL (faction=0 level=12)
DC.B    $19                ; +MFIELD_LEVEL_ENC (xpmult=1, base=9)
DC.W    $012C                ; +MFIELD_GOLD (300 ducats)
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_SON_OF_MINOTAUR 197.

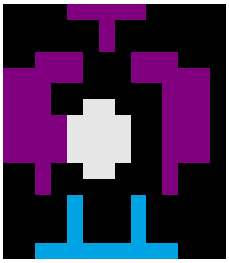


MINOTAUR (font character 18, group 3)

177 ⟨*mob data records* 68⟩+≡ (909) ◁176 178▷

```
; --- mob 108: MINOTAUR ($5340) ---
DC.W    s_MINOTAUR              ; +MFIELD_NAME
DC.B    $00                ; +MFIELD_NEXT (0)
DC.B    $7B                ; +MFIELD_POSITION (row 6, col 3)
DC.B    $50                ; +MFIELD_APPEARANCE (fontchar=18, group=3 hostile)
DC.B    $18                ; +MFIELD_STRENGTH (24)
DC.B    $18                ; +MFIELD_HP (24)
DC.B    $09                ; +MFIELD_ATTACK (9)
DC.B    $83                ; +MFIELD_XP (st=1 xp=0 grp=3)
DC.B    $0B                ; +MFIELD_DEFENSE (11)
DC.B    $00                ; +$0A
DC.B    $0E                ; +MFIELD_LEVEL (faction=0 level=14)
DC.B    $37                ; +MFIELD_LEVEL_ENC (xpmult=3, base=7)
DC.W    $02EE                ; +MFIELD_GOLD (750 ducats)
DC.B    $00                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_MINOTAUR 197.

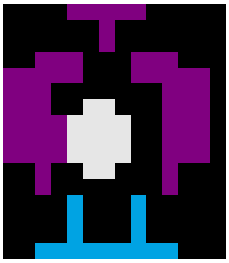


BARN OWL (font character 19, group 2)

178 ⟨*mob data records* 68⟩+≡ (909) ◁177 179▷

```
; --- mob 109: BARN OWL ($5350) ---
DC.W    s_BARN_OWL              ; +MFIELD_NAME
DC.B    $16               ; +MFIELD_NEXT (22)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $3C               ; +MFIELD_APPEARANCE (fontchar=19, group=2 friendly)
DC.B    $08               ; +MFIELD_STRENGTH (8)
DC.B    $08               ; +MFIELD_HP (8)
DC.B    $01               ; +MFIELD_ATTACK (1)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $01               ; +MFIELD_DEFENSE (1)
DC.B    $00               ; +$0A
DC.B    $0E               ; +MFIELD_LEVEL (faction=0 level=14)
DC.B    $0C               ; +MFIELD_LEVEL_ENC (xpmult=0, base=12)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $80               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_BARN_OWL 197.

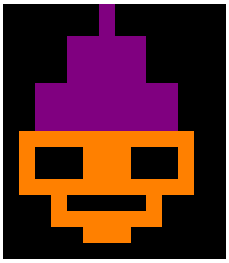


BABY OWL (font character 19, group 2)

179 ⟨*mob data records* 68⟩+≡ (909) ◁178 180▷

```
; --- mob 110: BABY OWL ($5360) ---
DC.W    s_BABY_OWL              ; +MFIELD_NAME
DC.B    $1F              ; +MFIELD_NEXT (31)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $3C              ; +MFIELD_APPEARANCE (fontchar=19, group=2 friendly)
DC.B    $04              ; +MFIELD_STRENGTH (4)
DC.B    $04              ; +MFIELD_HP (4)
DC.B    $00              ; +MFIELD_ATTACK (0)
DC.B    $00              ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $00              ; +MFIELD_DEFENSE (0)
DC.B    $00              ; +$0A
DC.B    $07              ; +MFIELD_LEVEL (faction=0 level=7)
DC.B    $08              ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $80              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_BABY_OWL 197.

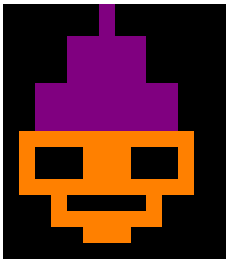


GANADRINIL (font character 34, group 7 — friendly)

180 ⟨*mob data records* 68⟩+≡ (909) ◁179 181▷

```
; --- mob 111: GANADRINIL ($5370) ---
DC.W    s_GANADRINIL              ; +MFIELD_NAME
DC.B    $20               ; +MFIELD_NEXT (32)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $94               ; +MFIELD_APPEARANCE (fontchar=34, group=7 friendly)
DC.B    $0A               ; +MFIELD_STRENGTH (10)
DC.B    $0A               ; +MFIELD_HP (10)
DC.B    $05               ; +MFIELD_ATTACK (5)
DC.B    $01               ; +MFIELD_XP (st=0 xp=0 grp=1)
DC.B    $06               ; +MFIELD_DEFENSE (6)
DC.B    $00               ; +$0A
DC.B    $50               ; +MFIELD_LEVEL (faction=2 level=16)
DC.B    $09               ; +MFIELD_LEVEL_ENC (xpmult=0, base=9)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $A0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_GANADRINIL 197.

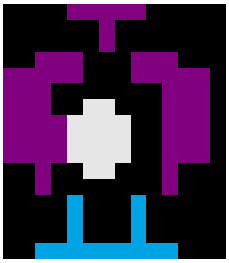


LISSOM (font character 34, group 7)

181 ⟨*mob data records* 68⟩+≡ (909) ◁180 182▷

```
; --- mob 112: LISSOM ($5380) ---
DC.W    s_LISSOM              ; +MFIELD_NAME
DC.B    $38               ; +MFIELD_NEXT (56)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $94               ; +MFIELD_APPEARANCE (fontchar=34, group=7 friendly)
DC.B    $08               ; +MFIELD_STRENGTH (8)
DC.B    $08               ; +MFIELD_HP (8)
DC.B    $04               ; +MFIELD_ATTACK (4)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $06               ; +MFIELD_DEFENSE (6)
DC.B    $00               ; +$0A
DC.B    $54               ; +MFIELD_LEVEL (faction=2 level=20)
DC.B    $0B               ; +MFIELD_LEVEL_ENC (xpmult=0, base=11)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $A0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_LISSOM 197.

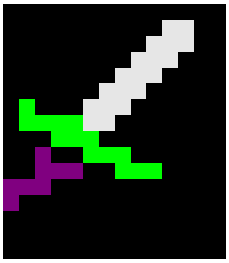


DOCTOR WHO (font character 19, group 2)

182 ⟨*mob data records* 68⟩+≡ (909) ◁181 183▷

```
; --- mob 113: DOCTOR WHO ($5390) ---
DC.W    s_DOCTOR_WHO              ; +MFIELD_NAME
DC.B    $15               ; +MFIELD_NEXT (21)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $3C               ; +MFIELD_APPEARANCE (fontchar=19, group=2 friendly)
DC.B    $06               ; +MFIELD_STRENGTH (6)
DC.B    $06               ; +MFIELD_HP (6)
DC.B    $02               ; +MFIELD_ATTACK (2)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $02               ; +MFIELD_DEFENSE (2)
DC.B    $00               ; +$0A
DC.B    $0F               ; +MFIELD_LEVEL (faction=0 level=15)
DC.B    $0C               ; +MFIELD_LEVEL_ENC (xpmult=0, base=12)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $A0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_DOCTOR_WHO 197.

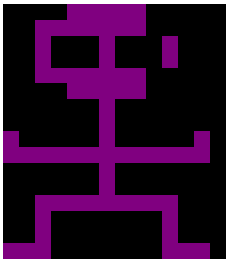


BANE OF THIEVES (font character 12, group 7)

183 ⟨*mob data records* 68⟩+≡ (909) ◁182 184▷

```
; --- mob 114: BANE OF THIEVES ($53A0) ---
DC.W    s_BANE_OF_THIEVES              ; +MFIELD_NAME
DC.B    $39                ; +MFIELD_NEXT (57)
DC.B    $00                ; +MFIELD_POSITION (unplaced)
DC.B    $9E                ; +MFIELD_APPEARANCE (fontchar=12, group=7 friendly)
DC.B    $08                ; +MFIELD_STRENGTH (8)
DC.B    $0C                ; +MFIELD_HP (12)
DC.B    $06                ; +MFIELD_ATTACK (6)
DC.B    $00                ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $04                ; +MFIELD_DEFENSE (4)
DC.B    $00                ; +$0A
DC.B    $4C                ; +MFIELD_LEVEL (faction=2 level=12)
DC.B    $05                ; +MFIELD_LEVEL_ENC (xpmult=0, base=5)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $A0                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_BANE_OF_THIEVES 197.

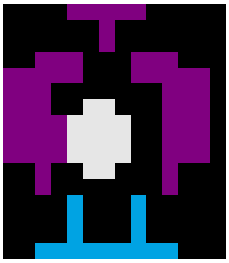


ALADDIN (font character 33, group 7)

184 ⟨*mob data records* 68⟩+≡ (909) ◁183 185▷

```
; --- mob 115: ALADDIN ($53B0) ---
DC.W    s_ALADDIN               ; +MFIELD_NAME
DC.B    $37               ; +MFIELD_NEXT (55)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $93               ; +MFIELD_APPEARANCE (fontchar=33, group=7 friendly)
DC.B    $0E               ; +MFIELD_STRENGTH (14)
DC.B    $0E               ; +MFIELD_HP (14)
DC.B    $09               ; +MFIELD_ATTACK (9)
DC.B    $03               ; +MFIELD_XP (st=0 xp=0 grp=3)
DC.B    $06               ; +MFIELD_DEFENSE (6)
DC.B    $00               ; +$0A
DC.B    $0B               ; +MFIELD_LEVEL (faction=0 level=11)
DC.B    $06               ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $A0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_ALADDIN 197.

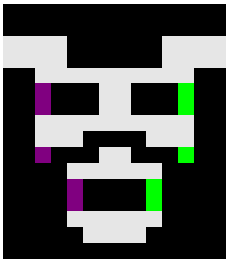


RING EYED OWL (font character 19, group 2)

185 ⟨*mob data records* 68⟩+≡ (909) ◁184 186▷

```
; --- mob 116: RING EYED OWL ($53C0) ---
DC.W    s_RING_EYED_OWL              ; +MFIELD_NAME
DC.B    $34              ; +MFIELD_NEXT (52)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $3C              ; +MFIELD_APPEARANCE (fontchar=19, group=2 friendly)
DC.B    $07              ; +MFIELD_STRENGTH (7)
DC.B    $07              ; +MFIELD_HP (7)
DC.B    $02              ; +MFIELD_ATTACK (2)
DC.B    $00              ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $03              ; +MFIELD_DEFENSE (3)
DC.B    $00              ; +$0A
DC.B    $0E              ; +MFIELD_LEVEL (faction=0 level=14)
DC.B    $0C              ; +MFIELD_LEVEL_ENC (xpmult=0, base=12)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $C0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_RING_EYED_OWL 197.

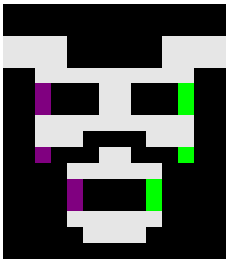


BLACK BEAR (font character 6, group 3)

186 ⟨*mob data records* 68⟩+≡ (909) ◁185 187▷

```
; --- mob 117: BLACK BEAR ($53D0) ---
DC.W    s_BLACK_BEAR              ; +MFIELD_NAME
DC.B    $2B               ; +MFIELD_NEXT (43)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $44               ; +MFIELD_APPEARANCE (fontchar=6, group=3 hostile)
DC.B    $14               ; +MFIELD_STRENGTH (20)
DC.B    $14               ; +MFIELD_HP (20)
DC.B    $06               ; +MFIELD_ATTACK (6)
DC.B    $83               ; +MFIELD_XP (st=1 xp=0 grp=3)
DC.B    $08               ; +MFIELD_DEFENSE (8)
DC.B    $00               ; +$0A
DC.B    $CA               ; +MFIELD_LEVEL (faction=6 level=10)
DC.B    $04               ; +MFIELD_LEVEL_ENC (xpmult=0, base=4)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $C0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_BLACK_BEAR 197.

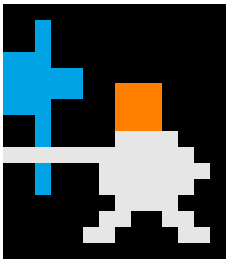


BROWN BEAR (font character 6, group 4)

187 ⟨*mob data records* 68⟩+≡ (909) ◁186 188▷

```
; --- mob 118: BROWN BEAR ($53E0) ---
DC.W    s_BROWN_BEAR              ; +MFIELD_NAME
DC.B    $2F               ; +MFIELD_NEXT (47)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $59               ; +MFIELD_APPEARANCE (fontchar=6, group=4 hostile)
DC.B    $10               ; +MFIELD_STRENGTH (16)
DC.B    $10               ; +MFIELD_HP (16)
DC.B    $04               ; +MFIELD_ATTACK (4)
DC.B    $82               ; +MFIELD_XP (st=1 xp=0 grp=2)
DC.B    $06               ; +MFIELD_DEFENSE (6)
DC.B    $00               ; +$0A
DC.B    $CA               ; +MFIELD_LEVEL (faction=6 level=10)
DC.B    $06               ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $C0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_BROWN_BEAR 197.

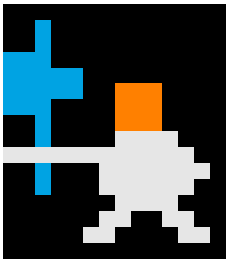


WAUKA (font character 35, group 3 — hostile)

188 ⟨*mob data records* 68⟩+≡ (909) ◁187 189▷

```
; --- mob 119: WAUKA ($53F0) ---
DC.W    s_WAUKA              ; +MFIELD_NAME
DC.B    $2E              ; +MFIELD_NEXT (46)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $41              ; +MFIELD_APPEARANCE (fontchar=35, group=3 hostile)
DC.B    $12              ; +MFIELD_STRENGTH (18)
DC.B    $12              ; +MFIELD_HP (18)
DC.B    $09              ; +MFIELD_ATTACK (9)
DC.B    $00              ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $04              ; +MFIELD_DEFENSE (4)
DC.B    $00              ; +$0A
DC.B    $0C              ; +MFIELD_LEVEL (faction=0 level=12)
DC.B    $06              ; +MFIELD_LEVEL_ENC (xpmult=0, base=6)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $C0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_WAUKA 197.

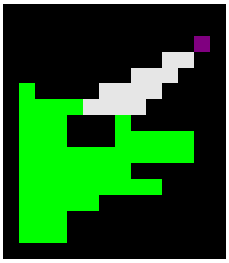


TROG (font character 35, group 3)

189 ⟨*mob data records* 68⟩+≡ (909) ◁188 190▷

```
; --- mob 120: TROG ($5400) ---
DC.W    s_TROG              ; +MFIELD_NAME
DC.B    $30               ; +MFIELD_NEXT (48)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $41               ; +MFIELD_APPEARANCE (fontchar=35, group=3 hostile)
DC.B    $12               ; +MFIELD_STRENGTH (18)
DC.B    $12               ; +MFIELD_HP (18)
DC.B    $09               ; +MFIELD_ATTACK (9)
DC.B    $01               ; +MFIELD_XP (st=0 xp=0 grp=1)
DC.B    $04               ; +MFIELD_DEFENSE (4)
DC.B    $00               ; +$0A
DC.B    $0A               ; +MFIELD_LEVEL (faction=0 level=10)
DC.B    $05               ; +MFIELD_LEVEL_ENC (xpmult=0, base=5)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $C0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_TROG 197.

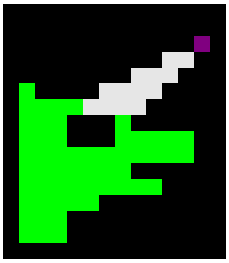


NELL (font character 20, group 7)

190 ⟨*mob data records* 68⟩+≡ (909) ◁189 191▷

```
; --- mob 121: NELL ($5410) ---
DC.W    s_NELL              ; +MFIELD_NAME
DC.B    $2C               ; +MFIELD_NEXT (44)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $A6               ; +MFIELD_APPEARANCE (fontchar=20, group=7 friendly)
DC.B    $12               ; +MFIELD_STRENGTH (18)
DC.B    $12               ; +MFIELD_HP (18)
DC.B    $07               ; +MFIELD_ATTACK (7)
DC.B    $81               ; +MFIELD_XP (st=1 xp=0 grp=1)
DC.B    $05               ; +MFIELD_DEFENSE (5)
DC.B    $00               ; +$0A
DC.B    $54               ; +MFIELD_LEVEL (faction=2 level=20)
DC.B    $0A               ; +MFIELD_LEVEL_ENC (xpmult=0, base=10)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $C0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_NELL 197.

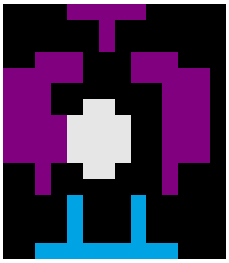


EQUUS (font character 20, group 7)

191 ⟨*mob data records* 68⟩+≡ (909) ◁190 192▷

```
; --- mob 122: EQUUS ($5420) ---
DC.W    s_EQUUS              ; +MFIELD_NAME
DC.B    $7E              ; +MFIELD_NEXT (126)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $A6              ; +MFIELD_APPEARANCE (fontchar=20, group=7 friendly)
DC.B    $14              ; +MFIELD_STRENGTH (20)
DC.B    $14              ; +MFIELD_HP (20)
DC.B    $08              ; +MFIELD_ATTACK (8)
DC.B    $82              ; +MFIELD_XP (st=1 xp=0 grp=2)
DC.B    $05              ; +MFIELD_DEFENSE (5)
DC.B    $00              ; +$0A
DC.B    $5A              ; +MFIELD_LEVEL (faction=2 level=26)
DC.B    $09              ; +MFIELD_LEVEL_ENC (xpmult=0, base=9)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $E0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_EQUUS 197.

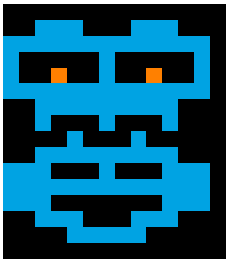


WHITE OWL (font character 19, group 2)

192 ⟨*mob data records* 68⟩+≡ (909) ◁191 193▷

```
; --- mob 123: WHITE OWL ($5430) ---
DC.W    s_WHITE_OWL              ; +MFIELD_NAME
DC.B    $2A              ; +MFIELD_NEXT (42)
DC.B    $00              ; +MFIELD_POSITION (unplaced)
DC.B    $3C              ; +MFIELD_APPEARANCE (fontchar=19, group=2 friendly)
DC.B    $04              ; +MFIELD_STRENGTH (4)
DC.B    $04              ; +MFIELD_HP (4)
DC.B    $01              ; +MFIELD_ATTACK (1)
DC.B    $00              ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $01              ; +MFIELD_DEFENSE (1)
DC.B    $00              ; +$0A
DC.B    $0F              ; +MFIELD_LEVEL (faction=0 level=15)
DC.B    $08              ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0000              ; +MFIELD_GOLD
DC.B    $E0              ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_WHITE_OWL 197.

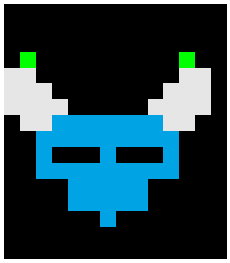


WHITE DRAGON (font character 9, group 7)

193 ⟨*mob data records* 68⟩+≡ (909) ◁192 194▷

```
; --- mob 124: WHITE DRAGON ($5440) ---
DC.W    s_WHITE_DRAGON              ; +MFIELD_NAME
DC.B    $29                ; +MFIELD_NEXT (41)
DC.B    $00                ; +MFIELD_POSITION (unplaced)
DC.B    $9B                ; +MFIELD_APPEARANCE (fontchar=9, group=7 friendly)
DC.B    $1F                ; +MFIELD_STRENGTH (31)
DC.B    $30                ; +MFIELD_HP (48)
DC.B    $0C                ; +MFIELD_ATTACK (12)
DC.B    $83                ; +MFIELD_XP (st=1 xp=0 grp=3)
DC.B    $08                ; +MFIELD_DEFENSE (8)
DC.B    $00                ; +$0A
DC.B    $CD                ; +MFIELD_LEVEL (faction=6 level=13)
DC.B    $0D                ; +MFIELD_LEVEL_ENC (xpmult=0, base=13)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $E0                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_WHITE_DRAGON 197.

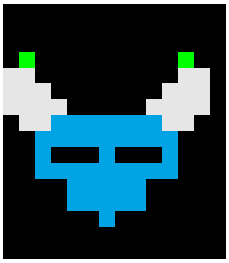


SONNY BOY MINOTAUR (font character 18, group 4)

194 ⟨*mob data records* 68⟩+≡ (909) ◁193 195▷

```
; --- mob 125: SONNY BOY MINOTAUR ($5450) ---
DC.W    s_SONNY_BOY_MINOTAUR              ; +MFIELD_NAME
DC.B    $00               ; +MFIELD_NEXT (0)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $65               ; +MFIELD_APPEARANCE (fontchar=18, group=4 hostile)
DC.B    $12               ; +MFIELD_STRENGTH (18)
DC.B    $16               ; +MFIELD_HP (22)
DC.B    $06               ; +MFIELD_ATTACK (6)
DC.B    $81               ; +MFIELD_XP (st=1 xp=0 grp=1)
DC.B    $08               ; +MFIELD_DEFENSE (8)
DC.B    $00               ; +$0A
DC.B    $0B               ; +MFIELD_LEVEL (faction=0 level=11)
DC.B    $08               ; +MFIELD_LEVEL_ENC (xpmult=0, base=8)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $E0               ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_SONNY_BOY_MINOTAUR 197.

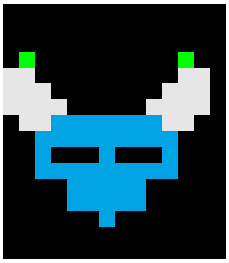


DAUNTLESS MINNIE (font character 18, group 3)

195 ⟨*mob data records* 68⟩+≡ (909) ◁194 196▷

```
; --- mob 126: DAUNTLESS MINNIE ($5460) ---
DC.W    s_DAUNTLESS_MINNIE              ; +MFIELD_NAME
DC.B    $23                ; +MFIELD_NEXT (35)
DC.B    $00                ; +MFIELD_POSITION (unplaced)
DC.B    $50                ; +MFIELD_APPEARANCE (fontchar=18, group=3 hostile)
DC.B    $10                ; +MFIELD_STRENGTH (16)
DC.B    $15                ; +MFIELD_HP (21)
DC.B    $08                ; +MFIELD_ATTACK (8)
DC.B    $83                ; +MFIELD_XP (st=1 xp=0 grp=3)
DC.B    $09                ; +MFIELD_DEFENSE (9)
DC.B    $00                ; +$0A
DC.B    $0D                ; +MFIELD_LEVEL (faction=0 level=13)
DC.B    $0A                ; +MFIELD_LEVEL_ENC (xpmult=0, base=10)
DC.W    $0000                ; +MFIELD_GOLD
DC.B    $E0                ; +MFIELD_FLAGS
```

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_DAUNTLESS_MINNIE 197.



BULL (font character 18, group 3)

196 ⟨*mob data records* 68⟩+≡ (909) ◁195

```
; --- mob 127: BULL ($5470) ---
DC.W    s_BULL              ; +MFIELD_NAME
DC.B    $7D               ; +MFIELD_NEXT (125)
DC.B    $00               ; +MFIELD_POSITION (unplaced)
DC.B    $50               ; +MFIELD_APPEARANCE (fontchar=18, group=3 hostile)
DC.B    $10               ; +MFIELD_STRENGTH (16)
DC.B    $14               ; +MFIELD_HP (20)
DC.B    $05               ; +MFIELD_ATTACK (5)
DC.B    $00               ; +MFIELD_XP (st=0 xp=0 grp=0)
DC.B    $04               ; +MFIELD_DEFENSE (4)
DC.B    $00               ; +$0A
DC.B    $CA               ; +MFIELD_LEVEL (faction=6 level=10)
DC.B    $07               ; +MFIELD_LEVEL_ENC (xpmult=0, base=7)
DC.W    $0000               ; +MFIELD_GOLD
DC.B    $E0               ; +MFIELD_FLAGS
```

Defines:
MOB_DATA_TABLE, used in chunks 53a and 68.

Uses MFIELD_APPEARANCE 561, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_NAME 561, MFIELD_NEXT 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MFIELD_XP 561, and s_BULL 197.

4.5 Mob name strings

128 length-prefixed high-ASCII name strings, one per mob record. The name pointer in bytes 0–1 of each mob record points into this table.

197 ⟨*mob name strings* 197⟩≡ (909)

```
    ORG     $5480
s_ALI_BABA:
    APSTR "ALI BABA"
s_SCHEHERAZADE:
    APSTR "SCHEHERAZADE"
s_HAROUD_EL_LARGE:
    APSTR "HAROUD EL-LARGE"
s_BAIRAM_MEDIO:
    APSTR "BAIRAM MEDIO"
s_ABOU_HASSAN:
    APSTR "ABOU HASSAN"
s_LUTHIEN:
    APSTR "LUTHIEN"
s_HUAN:
    APSTR "HUAN"
s_CELEGORM:
    APSTR "CELEGORM"
s_CURUFIN:
    APSTR "CURUFIN"
s_ELDO:
    APSTR "ELDO"
s_MERRYWATER:
    APSTR "MERRYWATER"
s_STILBO:
    APSTR "STILBO"
s_CINDER:
    APSTR "CINDER"
s_JATTE:
    APSTR "JATTE"
s_THORA:
    APSTR "THORA"
s_OG:
    APSTR "OG"
s_ULVA:
    APSTR "ULVA"
s_COGIA_HOUSSAIN:
    APSTR "COGIA HOUSSAIN"
s_OMAR:
    APSTR "OMAR"
s_MAHMUD:
    APSTR "MAHMUD"
s_ABUL_FATH:
    APSTR "ABUL-FATH"
s_IBRAHIM:
```

```
    APSTR "IBRAHIM"
s_JALAL_UD_DIN:
    APSTR "JALAL-UD-DIN"
s_SALAMAN:
    APSTR "SALAMAN"
s_ABSAL:
    APSTR "ABSAL"
s_JAMI:
    APSTR "JAMI"
s_NIZAM_UL_MULK:
    APSTR "NIZAM UL MULK"
s_ALP_ARSLAN:
    APSTR "ALP ARSLAN"
s_MALIK:
    APSTR "MALIK"
s_TOGHRUL_BEG:
    APSTR "TOGHRUL BEG"
s_MOWAFFAK:
    APSTR "MOWAFFAK"
s_ABD_US_SAMAD:
    APSTR "ABD-US-SAMAD"
s_HAKIM:
    APSTR "HAKIM"
s_HASAN_BEN_SABBAH:
    APSTR "HASAN BEN SABBAH"
s_ALI:
    APSTR "ALI"
s_KHORASSAN:
    APSTR "KHORASSAN"
s_KHWAJAH_NIZAMI:
    APSTR "KHWAJAH NIZAMI"
s_HAFIZ:
    APSTR "HAFIZ"
s_JELALUDDIN:
    APSTR "JELALUDDIN"
s_KWADRACH:
    APSTR "KWADRACH"
s_ATTAR:
    APSTR "ATTAR"
s_PEHLEVI:
    APSTR "PEHLEVI"
s_JAMSHYD:
    APSTR "JAMSHYD"
s_IRAM:
    APSTR "IRAM"
s_KAIKOBAD:
    APSTR "KAIKOBAD"
s_KAIKHOSRU:
    APSTR "KAIKHOSRU"
s_RUSTUM:
```

```
    APSTR "RUSTUM"
s_HATIM_TAI:
    APSTR "HATIM TAI"
s_BAHRAM:
    APSTR "BAHRAM"
s_DINARZADE:
    APSTR "DINARZADE"
s_SHAHZENAN:
    APSTR "SHAHZENAN"
s_DOUBAN:
    APSTR "DOUBAN"
s_FEROZE:
    APSTR "FEROZE"
s_AHMED:
    APSTR "AHMED"
s_PERIE_BANOU:
    APSTR "PERIE BANOU"
s_HAROUN_AL_RASCHID:
    APSTR "HAROUN AL-RASCHID"
s_CODADAD:
    APSTR "CODADAD"
s_MORGIANA:
    APSTR "MORGIANA"
s_ABDALLA:
    APSTR "ABDALLA"
s_ENCHANTED_STATUE:
    APSTR "ENCHANTED STATUE"
s_TAN_RAT:
    APSTR "TAN RAT"
s_GRAY_RAT:
    APSTR "GRAY RAT"
s_SULTAN_SHAHRIAR:
    APSTR "SULTAN SHAHRIAR"
s_PRINCESS_BUDDIR:
    APSTR "PRINCESS BUDDIR"
s_SULTANS_RIGHT_GUARD:
    APSTR "SULTANS RIGHT GUARD"
s_SULTANS_LEFT_GUARD:
    APSTR "SULTANS LEFT GUARD"
s_TIGER:
    APSTR "TIGER"
s_WITLESS_ZOMBIE:
    APSTR "WITLESS ZOMBIE"
s_SMALL_ZOMBIE:
    APSTR "SMALL ZOMBIE"
s_BIG_ZOMBIE:
    APSTR "BIG ZOMBIE"
s_CALICO_RAT:
    APSTR "CALICO RAT"
s_HOOT_OWL:
```

```
    APSTR "HOOT OWL"
s_MOUNTAIN_SCORPION:
    APSTR "MOUNTAIN SCORPION"
s_UNICORN:
    APSTR "UNICORN"
sm_LEO:
    APSTR "LEO"
sm_SCORPIUS:
    APSTR "SCORPIUS"
s_STATUE_OF_TAURUS:
    APSTR "STATUE OF TAURUS"
s_ARIES_THE_RAM:
    APSTR "ARIES, THE RAM"
s_CANCER_THE_CRAB:
    APSTR "CANCER, THE CRAB"
s_WEREJACKAL:
    APSTR "WEREJACKAL"
s_CAVEBEAR:
    APSTR "CAVEBEAR"
s_CAVE_RAT:
    APSTR "CAVE RAT"
s_BROWN_RAT:
    APSTR "BROWN RAT"
s_BLACK_RAT:
    APSTR "BLACK RAT"
s_GEMINI_I:
    APSTR "GEMINI I"
s_GEMINI_II:
    APSTR "GEMINI II"
s_SWORD_OF_DAMOCLES:
    APSTR "SWORD OF DAMOCLES"
s_TWO_EDGED_SWORD:
    APSTR "TWO EDGED SWORD"
s_WERERAT:
    APSTR "WERERAT"
s_SEWER_RAT:
    APSTR "SEWER RAT"
s_JUNGLE_RAT:
    APSTR "JUNGLE RAT"
s_GIANT_SCORPION:
    APSTR "GIANT SCORPION"
s_WEREWOLF:
    APSTR "WEREWOLF"
s_GRIZZLY_BEAR:
    APSTR "GRIZZLY BEAR"
s_GREEN_DRAGON:
    APSTR "GREEN DRAGON"
s_MECHANICAL_SWORD:
    APSTR "MECHANICAL SWORD"
s_VERY_YOUNG_DRAGON:
```

```
    APSTR "VERY YOUNG DRAGON"
s_ENCHANTED_SWORD:
    APSTR "ENCHANTED SWORD"
s_THE_BLOB:
    APSTR "THE BLOB"
s_SUN_WANDERER:
    APSTR "SUN WANDERER"
s_EARTH_WANDERER:
    APSTR "EARTH WANDERER"
s_SKY_WANDERER:
    APSTR "SKY WANDERER"
s_LOST_BARN_OWL:
    APSTR "LOST BARN OWL"
s_SLIME:
    APSTR "SLIME"
s_SUCCUBUS:
    APSTR "SUCCUBUS"
s_STATUE_OF_MINOTAUR:
    APSTR "STATUE OF MINOTAUR"
s_OOZING_STENCHBEAST:
    APSTR "OOZING STENCHBEAST"
s_SON_OF_MINOTAUR:
    APSTR "SON OF MINOTAUR"
s_MINOTAUR:
    APSTR "MINOTAUR"
s_BARN_OWL:
    APSTR "BARN OWL"
s_BABY_OWL:
    APSTR "BABY OWL"
s_GANADRINIL:
    APSTR "GANADRINIL"
s_LISSOM:
    APSTR "LISSOM"
s_DOCTOR_WHO:
    APSTR "DOCTOR WHO"
s_BANE_OF_THIEVES:
    APSTR "BANE OF THIEVES"
s_ALADDIN:
    APSTR "ALADDIN"
s_RING_EYED_OWL:
    APSTR "RING EYED OWL"
s_BLACK_BEAR:
    APSTR "BLACK BEAR"
s_BROWN_BEAR:
    APSTR "BROWN BEAR"
s_WAUKA:
    APSTR "WAUKA"
s_TROG:
    APSTR "TROG"
s_NELL:
```

```
    APSTR "NELL"
s_EQUUS:
    APSTR "EQUUS"
s_WHITE_OWL:
    APSTR "WHITE OWL"
s_WHITE_DRAGON:
    APSTR "WHITE DRAGON"
s_SONNY_BOY_MINOTAUR:
    APSTR "SONNY BOY MINOTAUR"
s_DAUNTLESS_MINNIE:
    APSTR "DAUNTLESS MINNIE"
s_BULL:
    APSTR "BULL"
```

Defines:
s_ABDALLA, used in chunk 127.
s_ABD_US_SAMAD, used in chunk 100.
s_ABOU_HASSAN, used in chunk 73.
s_ABSAL, used in chunk 93.
s_ABUL_FATH, used in chunk 89.
s_AHMED, used in chunk 122.
s_ALADDIN, used in chunk 184.
s_ALI, used in chunk 103.
s_ALI_BABA, used in chunk 69.
s_ALP_ARSLAN, used in chunk 96.
s_ARIES_THE_RAM, used in chunk 146.
s_ATTAR, used in chunk 109.
s_BABY_OWL, used in chunk 179.
s_BAHRAM, used in chunk 117.
s_BAIRAM_MEDIO, used in chunk 72.
s_BANE_OF_THIEVES, used in chunk 183.
s_BARN_OWL, used in chunk 178.
s_BIG_ZOMBIE, used in chunk 138.
s_BLACK_BEAR, used in chunk 186.
s_BLACK_RAT, used in chunk 152.
s_BROWN_BEAR, used in chunk 187.
s_BROWN_RAT, used in chunk 151.
s_BULL, used in chunk 196.
s_CALICO_RAT, used in chunk 139.
s_CANCER_THE_CRAB, used in chunk 147.
s_CAVEBEAR, used in chunk 149.
s_CAVE_RAT, used in chunk 150.
s_CELEGORM, used in chunk 76.
s_CINDER, used in chunk 81.
s_CODADAD, used in chunk 125.
s_COGIA_HOUSSAIN, used in chunk 86.
s_CURUFIN, used in chunk 77.
s_DAUNTLESS_MINNIE, used in chunk 195.
s_DINARZADE, used in chunk 118.
s_DOCTOR_WHO, used in chunk 182.
s_DOUBAN, used in chunk 120.
s_EARTH_WANDERER, used in chunk 169.
s_ELDO, used in chunk 78.
s_ENCHANTED_STATUE, used in chunk 128.
s_ENCHANTED_SWORD, used in chunk 166.
s_EQUUS, used in chunk 191.

s_FEROZE, used in chunk 121.
s_GANADRINIL, used in chunk 180.
s_GEMINI_I, used in chunk 153.
s_GEMINI_II, used in chunk 154.
s_GIANT_SCORPION, used in chunk 160.
s_GRAY_RAT, used in chunk 130.
s_GREEN_DRAGON, used in chunk 163.
s_GRIZZLY_BEAR, used in chunk 162.
s_HAFIZ, used in chunk 106.
s_HAKIM, used in chunk 101.
s_HAROUD_EL_LARGE, used in chunk 71.
s_HAROUN_AL_RASCHID, used in chunk 124.
s_HASAN_BEN_SABBAH, used in chunk 102.
s_HATIM_TAI, used in chunk 116.
s_HOOT_OWL, used in chunk 140.
s_HUAN, used in chunk 75.
s_IBRAHIM, used in chunk 90.
s_IRAM, used in chunk 112.
s_JALAL_UD_DIN, used in chunk 91.
s_JAMI, used in chunk 94.
s_JAMSHYD, used in chunk 111.
s_JATTE, used in chunk 82.
s_JELALUDDIN, used in chunk 107.
s_JUNGLE_RAT, used in chunk 159.
s_KAIKHOSRU, used in chunk 114.
s_KAIKOBAD, used in chunk 113.
s_KHORASSAN, used in chunk 104.
s_KHWAJAH_NIZAMI, used in chunk 105.
s_KWADRACH, used in chunk 108.
s_LISSOM, used in chunk 181.
s_LOST_BARN_OWL, used in chunk 171.
s_LUTHIEN, used in chunk 74.
s_MAHMUD, used in chunk 88.
s_MALIK, used in chunk 97.
s_MECHANICAL_SWORD, used in chunk 164.
s_MERRYWATER, used in chunk 79.
s_MINOTAUR, used in chunk 177.
s_MORGIANA, used in chunk 126.
s_MOUNTAIN_SCORPION, used in chunk 141.
s_MOWAFFAK, used in chunk 99.
s_NELL, used in chunk 190.
s_NIZAM_UL_MULK, used in chunk 95.
s_OG, used in chunk 84.
s_OMAR, used in chunk 87.
s_OOZING_STENCHBEAST, used in chunk 175.
s_PEHLEVI, used in chunk 110.
s_PERIE_BANOU, used in chunk 123.
s_PRINCESS_BUDDIR, used in chunk 132.
s_RING_EYED_OWL, used in chunk 185.
s_RUSTUM, used in chunk 115.
s_SALAMAN, used in chunk 92.
s_SCHEHERAZADE, used in chunk 70.
s_SEWER_RAT, used in chunk 158.
s_SHAHZENAN, used in chunk 119.
s_SKY_WANDERER, used in chunk 170.
s_SLIME, used in chunk 172.
s_SMALL_ZOMBIE, used in chunk 137.

s_SONNY_BOY_MINOTAUR, used in chunk 194.
s_SON_OF_MINOTAUR, used in chunk 176.
s_STATUE_OF_MINOTAUR, used in chunk 174.
s_STATUE_OF_TAURUS, used in chunk 145.
s_STILBO, used in chunk 80.
s_SUCCUBUS, used in chunk 173.
s_SULTANS_LEFT_GUARD, used in chunk 134.
s_SULTANS_RIGHT_GUARD, used in chunk 133.
s_SULTAN_SHAHRIAR, used in chunk 131.
s_SUN_WANDERER, used in chunk 168.
s_SWORD_OF_DAMOCLES, used in chunk 155.
s_TAN_RAT, used in chunk 129.
s_THE_BLOB, used in chunk 167.
s_THORA, used in chunk 83.
s_TIGER, used in chunk 135.
s_TOGHRUL_BEG, used in chunk 98.
s_TROG, used in chunk 189.
s_TWO_EDGED_SWORD, used in chunk 156.
s_ULVA, used in chunk 85.
s_UNICORN, used in chunk 142.
s_VERY_YOUNG_DRAGON, used in chunk 165.
s_WAUKA, used in chunk 188.
s_WEREJACKAL, used in chunk 148.
s_WERERAT, used in chunk 157.
s_WEREWOLF, used in chunk 161.
s_WHITE_DRAGON, used in chunk 193.
s_WHITE_OWL, used in chunk 192.
s_WITLESS_ZOMBIE, used in chunk 136.
sm_LEO, used in chunk 143.
sm_SCORPIUS, used in chunk 144.

Chapter 5

The Main Loop

MAIN_ENTRY is where the EALDR transfers control after loading and decrypting the game data. It initializes hardware via GAME_INIT, then runs the top-level game loop indefinitely.

Startup. On first run (TOTAL_MOB_COUNT = 0), the attract screen plays until the player presses a key. CHAR_PTR is set to LOCATION_DATA, the base of the 9-byte character record array.

Main loop. Each pass iterates the character records, advancing CHAR_PTR by CHAR_RECORD_SIZE (9) per step:

- Skip inactive characters (MFIELD_XP byte = 0).
- Call ENTER_LOCATION to redraw the map if the location changed.
- Dispatch CHECK_SCENE_EVENTS for players 1 and 3.
- Call PROCESS_MOB_GROUPS to run AI turns for all mob groups.

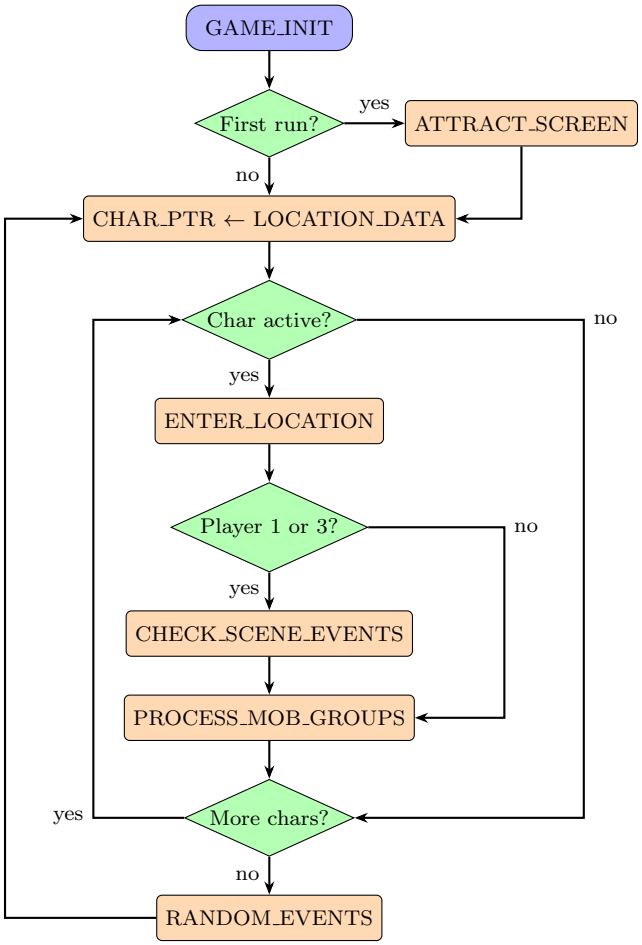
When byte 1 of the next record equals $FF (end sentinel), the pass calls RANDOM_EVENTS and restarts.

The event list at EVENT_PTR consists of 3-byte entries (see §4.1.1 for the full format). Byte 0 is a map position ($FE = empty slot, $FF = end-of-list); byte 1 encodes the event type and status; byte 2 is type-dependent action data.

The map scene ID (field 5, bits 0–6) is compared with LOCATION_FLAG2 ($5A56); when it changes, FILL_MAP is called to redraw the map. Bit 7 is set

on first entry via `ORA \#$80` and tested with `BMI` to skip first-visit logic on subsequent entries.

The sentinel `FFFF` in the name pointer marks the end of the table.



Each location record is 9 bytes (`CHAR_RECORD_SIZE`):

Byte	Field	Description
0–1	Name pointer	Address of the location's display name string
2	Mob head	Head of the mob linked list (0 = no mobs)
3	Min position	Top-left boundary (packed $20 \times row + col$)
4	Max position	Bottom-right boundary (same encoding)
5	Map ID	Scene/map number (bit 7 = visited flag)
6–7	Event pointer	Address of the location's event list
8	XP/status	Bit 7 = status, bits 6–4 = pending XP, bits 3–0 = group count

The mob head (byte 2) is the index of the first mob group in this location. Each mob's data record contains a next-link field (`MFIELD_NEXT`, byte 2), forming a singly linked list of mob groups. `PROCESS_MOB_GROUPS` walks this list from the head until it reaches a zero link (end of chain). A head value of $00 means the location has no mobs.

207 ⟨*Location data* 207⟩≡ (909)

```
    ORG     $4009
LOCATION_DATA:
    ; --- 0: ALI BABA HOME ---
    DC.W s_ALI_BABA_HOME
    DC.B $01                ; mob head $01
    DC.B 20*6+9             ; min pos (6,9)
    DC.B 20*9+13            ; max pos (9,13)
    DC.B $01                ; map 1
    DC.W ev_ali_baba_home
    DC.B $01

    ; --- 1: DUSTY ROAD ---
    DC.W s_DUSTY_ROAD
    DC.B $00                ; no mobs
    DC.B 20*3+7             ; min pos (3,7)
    DC.B 20*6+19            ; max pos (6,19)
    DC.B $01                ; map 1
    DC.W ev_dusty_road
    DC.B $00

    ; --- 2: SULTAN'S PALACE ---
    DC.W s_SULTANS_PALACE
    DC.B $3F                ; mob head $3F
    DC.B 20*0+0             ; min pos (0,0)
    DC.B 20*9+7             ; max pos (9,7)
    DC.B $01                ; map 1
    DC.W ev_sultans_palace
    DC.B $00

    ; --- 3: ROYAL LIBRARY ---
    DC.W s_ROYAL_LIBRARY
    DC.B $00                ; no mobs
```

```
DC.B 20*0+7             ; min pos (0,7)
DC.B 20*3+16            ; max pos (3,16)
DC.B $01                ; map 1
DC.W ev_royal_library
DC.B $00

; --- 4: ASTROLOGER'S LAB ---
DC.W s_ASTROLOGERS_LAB
DC.B $4A                ; mob head $4A
DC.B 20*0+3             ; min pos (0,3)
DC.B 20*9+16            ; max pos (9,16)
DC.B $25                ; map 37
DC.W ev_astrologers_lab
DC.B $00

; --- 5: BEAR CAVE ---
DC.W s_BEAR_CAVE
DC.B $51                ; mob head $51
DC.B 20*0+0             ; min pos (0,0)
DC.B 20*5+14            ; max pos (5,14)
DC.B $29                ; map 41
DC.W ev_bear_cave
DC.B $00

; --- 6: RATS NEST ---
DC.W s_RATS_NEST
DC.B $52                ; mob head $52
DC.B 20*5+0             ; min pos (5,0)
DC.B 20*9+14            ; max pos (9,14)
DC.B $29                ; map 41
DC.W ev_rats_nest
DC.B $00

; --- 7: DRAGON LAIR ---
DC.W s_DRAGON_LAIR
DC.B $5F                ; mob head $5F
DC.B 20*4+2             ; min pos (4,2)
DC.B 20*9+9             ; max pos (9,9)
DC.B $49                ; map 73
DC.W ev_dragon_lair
DC.B $00

; --- 8: GOLD FOR THE LUCKY ---
DC.W s_GOLD_FOR_THE_LUCKY
DC.B $60                ; mob head $60
DC.B 20*4+9             ; min pos (4,9)
DC.B 20*9+16            ; max pos (9,16)
DC.B $49                ; map 73
DC.W ev_gold_for_the_lucky
DC.B $00
```

```
; --- 9: SAVE THE WHALES ---
DC.W s_SAVE_THE_WHALES
DC.B $00                ; no mobs
DC.B 20*0+2             ; min pos (0,2)
DC.B 20*4+16            ; max pos (4,16)
DC.B $49                ; map 73
DC.W ev_save_the_whales
DC.B $00

; --- 10: SLIDE ---
DC.W s_SLIDE
DC.B $00                ; no mobs
DC.B 20*0+1             ; min pos (0,1)
DC.B 20*9+3             ; max pos (9,3)
DC.B $60                ; map 96
DC.W ev_slide
DC.B $00

; --- 11: DEEP DARK STAIRWAY ---
DC.W s_DEEP_DARK_STAIRWAY
DC.B $00                ; no mobs
DC.B 20*1+3             ; min pos (1,3)
DC.B 20*7+9             ; max pos (7,9)
DC.B $61                ; map 97
DC.W ev_deep_dark_stairway
DC.B $00

; --- 12: MINOTAUR LABYRINTH ---
DC.W s_MINOTAUR_LABYRINTH
DC.B $6A                ; mob head $6A
DC.B 20*0+3             ; min pos (0,3)
DC.B 20*9+16            ; max pos (9,16)
DC.B $62                ; map 98
DC.W ev_minotaur_labyrinth
DC.B $00

; --- 13: STINGER STREWN CELL ---
DC.W s_STINGER_STREWN_CELL
DC.B $5C                ; mob head $5C
DC.B 20*5+3             ; min pos (5,3)
DC.B 20*9+8             ; max pos (9,8)
DC.B $48                ; map 72
DC.W ev_stinger_strewn_cell
DC.B $00

; --- 14: GUARD ROOM ---
DC.W s_GUARD_ROOM
DC.B $57                ; mob head $57
DC.B 20*0+3             ; min pos (0,3)
```

```
DC.B 20*5+16            ; max pos (5,16)
DC.B $48                ; map 72
DC.W ev_guard_room
DC.B $00

; --- 15: LONG WAY FROM HOME ---
DC.W s_LONG_WAY_FROM_HOME
DC.B $5E                ; mob head $5E
DC.B 20*5+11            ; min pos (5,11)
DC.B 20*9+16            ; max pos (9,16)
DC.B $48                ; map 72
DC.W ev_long_way_from_home
DC.B $00

; --- 16: FOREST ---
DC.W s_FOREST
DC.B $43                ; mob head $43
DC.B 20*0+0             ; min pos (0,0)
DC.B 20*9+13            ; max pos (9,13)
DC.B $04                ; map 4
DC.W ev_forest
DC.B $00

; --- 17: VIEW FROM A TREE ---
DC.W s_VIEW_FROM_A_TREE
DC.B $00                ; no mobs
DC.B 20*1+13            ; min pos (1,13)
DC.B 20*7+19            ; max pos (7,19)
DC.B $04                ; map 4
DC.W ev_view_from_a_tree
DC.B $00

; --- 18: FAILING WALL ---
DC.W s_FAILING_WALL
DC.B $6B                ; mob head $6B
DC.B 20*1+0             ; min pos (1,0)
DC.B 20*8+14            ; max pos (8,14)
DC.B $64                ; map 100
DC.W ev_failing_wall
DC.B $00

; --- 19: MINOTAUR PLAYROOM ---
DC.W s_MINOTAUR_PLAYROOM
DC.B $6C                ; mob head $6C
DC.B 20*1+5             ; min pos (1,5)
DC.B 20*8+19            ; max pos (8,19)
DC.B $65                ; map 101
DC.W ev_minotaur_playroom
DC.B $00
```

```
; --- 20: THIEVES CAVE ---
DC.W s_THIEVES_CAVE
DC.B $00                ; no mobs
DC.B 20*0+3             ; min pos (0,3)
DC.B 20*9+16            ; max pos (9,16)
DC.B $21                ; map 33
DC.W ev_thieves_cave
DC.B $00

; --- 21: O ---
DC.W s_O
DC.B $62                ; mob head $62
DC.B 20*1+10            ; min pos (1,10)
DC.B 20*8+18            ; max pos (8,18)
DC.B $40                ; map 64
DC.W ev_o
DC.B $00

; --- 22: SPIRAL STAIRCASE ---
DC.W s_SPIRAL_STAIRCASE
DC.B $00                ; no mobs
DC.B 20*1+6             ; min pos (1,6)
DC.B 20*7+10            ; max pos (7,10)
DC.B $46                ; map 70
DC.W ev_spiral_staircase
DC.B $00

; --- 23: Y ---
DC.W s_Y
DC.B $63                ; mob head $63
DC.B 20*1+1             ; min pos (1,1)
DC.B 20*8+18            ; max pos (8,18)
DC.B $41                ; map 65
DC.W ev_y
DC.B $00

; --- 24: MAGIC TOURBILLION ---
DC.W s_MAGIC_TOURBILLION
DC.B $00                ; no mobs
DC.B 20*5+9             ; min pos (5,9)
DC.B 20*9+13            ; max pos (9,13)
DC.B $60                ; map 96
DC.W ev_magic_tourbillion
DC.B $00

; --- 25: G. ---
DC.W s_G
DC.B $64                ; mob head $64
DC.B 20*1+1             ; min pos (1,1)
DC.B 20*8+18            ; max pos (8,18)
```

```
DC.B $42                ; map 66
DC.W ev_g
DC.B $00

; --- 26: CRYSTAL CAVE ---
DC.W s_CRYSTAL_CAVE
DC.B $00                ; no mobs
DC.B 20*2+3             ; min pos (2,3)
DC.B 20*8+15            ; max pos (8,15)
DC.B $22                ; map 34
DC.W ev_crystal_cave
DC.B $00

; --- 27: B ---
DC.W s_B
DC.B $00                ; no mobs
DC.B 20*1+1             ; min pos (1,1)
DC.B 20*8+18            ; max pos (8,18)
DC.B $43                ; map 67
DC.W ev_b
DC.B $00

; --- 28: THIEVES CLEARING ---
DC.W s_THIEVES_CLEARING
DC.B $1B                ; mob head $1B
DC.B 20*4+4             ; min pos (4,4)
DC.B 20*9+13            ; max pos (9,13)
DC.B $05                ; map 5
DC.W ev_thieves_clearing
DC.B $00

; --- 29: OLD MINE ROAD ---
DC.W s_OLD_MINE_ROAD
DC.B $47                ; mob head $47
DC.B 20*1+0             ; min pos (1,0)
DC.B 20*4+19            ; max pos (4,19)
DC.B $05                ; map 5
DC.W ev_old_mine_road
DC.B $00

; --- 30: FOOTHILLS CEMETERY ---
DC.W s_FOOTHILLS_CEMETERY
DC.B $45                ; mob head $45
DC.B 20*1+1             ; min pos (1,1)
DC.B 20*8+18            ; max pos (8,18)
DC.B $02                ; map 2
DC.W ev_foothills_cemetery
DC.B $00

; --- 31: I ---
```

```
DC.W s_I
DC.B $68                ; mob head $68
DC.B 20*1+1             ; min pos (1,1)
DC.B 20*8+18            ; max pos (8,18)
DC.B $44                ; map 68
DC.W ev_i
DC.B $00

; --- 32: UNDERGROUND STREAM ---
DC.W s_UNDERGROUND_STREAM
DC.B $00                ; no mobs
DC.B 20*0+7             ; min pos (0,7)
DC.B 20*3+14            ; max pos (3,14)
DC.B $60                ; map 96
DC.W ev_underground_stream
DC.B $00

; --- 33: V ---
DC.W s_V
DC.B $69                ; mob head $69
DC.B 20*1+1             ; min pos (1,1)
DC.B 20*8+18            ; max pos (8,18)
DC.B $45                ; map 69
DC.W ev_v
DC.B $00

; --- 34: GEMINI ---
DC.W s_GEMINI
DC.B $55                ; mob head $55
DC.B 20*0+2             ; min pos (0,2)
DC.B 20*4+16            ; max pos (4,16)
DC.B $20                ; map 32
DC.W ev_gemini
DC.B $00

; --- 35: R ---
DC.W s_R
DC.B $61                ; mob head $61
DC.B 20*1+1             ; min pos (1,1)
DC.B 20*8+10            ; max pos (8,10)
DC.B $40                ; map 64
DC.W ev_r
DC.B $00

; --- 36: OLD MINE SHAFT ---
DC.W s_OLD_MINE_SHAFT
DC.B $00                ; no mobs
DC.B 20*3+4             ; min pos (3,4)
DC.B 20*5+14            ; max pos (5,14)
DC.B $06                ; map 6
```

```
DC.W ev_old_mine_shaft
DC.B $00

; --- 37: GILDED PATHWAY ---
DC.W s_GILDED_PATHWAY
DC.B $00                ; no mobs
DC.B 20*0+6             ; min pos (0,6)
DC.B 20*9+10            ; max pos (9,10)
DC.B $47                ; map 71
DC.W ev_gilded_pathway
DC.B $00

; --- 38: MINOTAUR'S DEN ---
DC.W s_MINOTAURS_DEN
DC.B $6D                ; mob head $6D
DC.B 20*3+0             ; min pos (3,0)
DC.B 20*9+7             ; max pos (9,7)
DC.B $63                ; map 99
DC.W ev_minotaurs_den
DC.B $00

; --- 39: MINOTAUR'S VAULT ---
DC.W s_MINOTAURS_VAULT
DC.B $00                ; no mobs
DC.B 20*6+7             ; min pos (6,7)
DC.B 20*9+16            ; max pos (9,16)
DC.B $63                ; map 99
DC.W ev_minotaurs_vault
DC.B $00

; --- 40: INNER STAIRCASE ---
DC.W s_INNER_STAIRCASE
DC.B $00                ; no mobs
DC.B 20*0+11            ; min pos (0,11)
DC.B 20*6+16            ; max pos (6,16)
DC.B $63                ; map 99
DC.W ev_inner_staircase
DC.B $00

; --- 41: SECRET PASSAGEWAY ---
DC.W s_SECRET_PASSAGEWAY
DC.B $00                ; no mobs
DC.B 20*1+0             ; min pos (1,0)
DC.B 20*3+11            ; max pos (3,11)
DC.B $63                ; map 99
DC.W ev_secret_passageway
DC.B $00

; --- 42: VIRGO ---
DC.W s_VIRGO
```

```
DC.B $3A                ; mob head $3A
DC.B 20*1+4             ; min pos (1,4)
DC.B 20*8+9             ; max pos (8,9)
DC.B $26                ; map 38
DC.W ev_virgo
DC.B $00

; --- 43: LEO ---
DC.W s_LEO
DC.B $4B                ; mob head $4B
DC.B 20*1+0             ; min pos (1,0)
DC.B 20*8+4             ; max pos (8,4)
DC.B $26                ; map 38
DC.W ev_leo
DC.B $00

; --- 44: LIBRA ---
DC.W s_LIBRA
DC.B $00                ; no mobs
DC.B 20*1+9             ; min pos (1,9)
DC.B 20*8+14            ; max pos (8,14)
DC.B $26                ; map 38
DC.W ev_libra
DC.B $00

; --- 45: SCORPIUS ---
DC.W s_SCORPIUS
DC.B $4C                ; mob head $4C
DC.B 20*1+14            ; min pos (1,14)
DC.B 20*8+19            ; max pos (8,19)
DC.B $26                ; map 38
DC.W ev_scorpius
DC.B $00

; --- 46: SAGITTARIUS ---
DC.W s_SAGITTARIUS
DC.B $12                ; mob head $12
DC.B 20*4+0             ; min pos (4,0)
DC.B 20*9+16            ; max pos (9,16)
DC.B $27                ; map 39
DC.W ev_sagittarius
DC.B $00

; --- 47: CAPRICORNUS ---
DC.W s_CAPRICORNUS
DC.B $00                ; no mobs
DC.B 20*0+0             ; min pos (0,0)
DC.B 20*4+16            ; max pos (4,16)
DC.B $27                ; map 39
DC.W ev_capricornus
```

```
DC.B $00

; --- 48: AQUARIUS ---
DC.W s_AQUARIUS
DC.B $00                ; no mobs
DC.B 20*1+15            ; min pos (1,15)
DC.B 20*8+19            ; max pos (8,19)
DC.B $24                ; map 36
DC.W ev_aquarius
DC.B $00

; --- 49: PISCES ---
DC.W s_PISCES
DC.B $00                ; no mobs
DC.B 20*1+11            ; min pos (1,11)
DC.B 20*8+15            ; max pos (8,15)
DC.B $24                ; map 36
DC.W ev_pisces
DC.B $00

; --- 50: HALLWAY ---
DC.W s_HALLWAY
DC.B $00                ; no mobs
DC.B 20*1+8             ; min pos (1,8)
DC.B 20*8+11            ; max pos (8,11)
DC.B $24                ; map 36
DC.W ev_hallway
DC.B $00

; --- 51: ARIES ---
DC.W s_ARIES
DC.B $4E                ; mob head $4E
DC.B 20*1+4             ; min pos (1,4)
DC.B 20*8+8             ; max pos (8,8)
DC.B $24                ; map 36
DC.W ev_aries
DC.B $00

; --- 52: TAURUS ---
DC.W s_TAURUS
DC.B $4D                ; mob head $4D
DC.B 20*1+0             ; min pos (1,0)
DC.B 20*8+4             ; max pos (8,4)
DC.B $24                ; map 36
DC.W ev_taurus
DC.B $00

; --- 53: GEMINI (south) ---
DC.W s_GEMINI2
DC.B $00                ; no mobs
```

```
DC.B 20*0+2             ; min pos (0,2)
DC.B 20*4+16            ; max pos (4,16)
DC.B $20                ; map 32
DC.W ev_gemini2
DC.B $00

; --- 54: CANCER ---
DC.W s_CANCER
DC.B $4F                ; mob head $4F
DC.B 20*5+2             ; min pos (5,2)
DC.B 20*9+16            ; max pos (9,16)
DC.B $20                ; map 32
DC.W ev_cancer
DC.B $00

; --- 55: THE MOUNTAIN ---
DC.W s_THE_MOUNTAIN
DC.B $49                ; mob head $49
DC.B 20*0+2             ; min pos (0,2)
DC.B 20*9+16            ; max pos (9,16)
DC.B $07                ; map 7
DC.W ev_the_mountain
DC.B $00

; --- 56: SHAHRIAR TREASURY ---
DC.W s_SHAHRIAR_TREASURY
DC.B $3C                ; mob head $3C
DC.B 20*1+4             ; min pos (1,4)
DC.B 20*8+13            ; max pos (8,13)
DC.B $03                ; map 3
DC.W ev_shahriar_treasury
DC.B $00

; --- 57: AIR VENT ---
DC.W s_AIR_VENT
DC.B $00                ; no mobs
DC.B 20*0+14            ; min pos (0,14)
DC.B 20*9+16            ; max pos (9,16)
DC.B $28                ; map 40
DC.W ev_air_vent
DC.B $00

; --- 58: DESERTED TUNNEL ---
DC.W s_DESERTED_TUNNEL
DC.B $00                ; no mobs
DC.B 20*6+6             ; min pos (6,6)
DC.B 20*9+14            ; max pos (9,14)
DC.B $28                ; map 40
DC.W ev_deserted_tunnel
DC.B $00
```

```
    ; --- 59: ROY G. BIV ---
    DC.W s_ROY_G_BIV
    DC.B $00                ; no mobs
    DC.B 20*3+0             ; min pos (3,0)
    DC.B 20*9+2             ; max pos (9,2)
    DC.B $28                ; map 40
    DC.W ev_roy_g_biv
    DC.B $00

    ; --- 60: JACKAL ALTAR ---
    DC.W s_JACKAL_ALTAR
    DC.B $50                ; mob head $50
    DC.B 20*0+2             ; min pos (0,2)
    DC.B 20*6+14            ; max pos (6,14)
    DC.B $28                ; map 40
    DC.W ev_jackal_altar
    DC.B $00

    ; --- 61: ASTROLOGER'S MAZE ---
    DC.W s_ASTROLOGERS_MAZE
    DC.B $14                ; mob head $14
    DC.B 20*0+3             ; min pos (0,3)
    DC.B 20*9+16            ; max pos (9,16)
    DC.B $23                ; map 35
    DC.W ev_astrologers_maze
    DC.B $00

    DC.W $FFFF              ; end sentinel
EVENT_THRESHOLD:
    DC.B    $12               ; $4239: random event probability threshold (~7%)
SCENE_COMBAT_FLAG:
    DC.B    $01               ; $423A: nonzero = combat pending
SCENE_P3_DONE:
    DC.B    $00               ; $423B: nonzero = player 3 scene processed
SCENE_P1_DONE:
    DC.B    $00               ; $423C: nonzero = player 1 scene processed
SCENE_HOSTILE_DONE:
    DC.B    $00               ; $423D: nonzero = hostile encounter processed
SCENE_EVENT_RESULT:
    DC.B    $00               ; $423E: nonzero = event fully handled
    DC.B    $A0               ; $423F: padding
```

Defines:
EVENT_THRESHOLD, used in chunks 415, 685, and 686.
LOCATION_DATA, used in chunk 219.
SCENE_COMBAT_FLAG, used in chunks 346 and 428.
SCENE_EVENT_RESULT, used in chunks 220, 428, and 617.
SCENE_HOSTILE_DONE, used in chunk 428.
SCENE_P1_DONE, used in chunk 428.
SCENE_P3_DONE, used in chunk 428.

219 ⟨*main entry* 219⟩≡ (909)

```
    ORG     $0800
MAIN_ENTRY:
    SUBROUTINE MAIN_ENTRY
    ; Main program entry point and top-level game loop.
    ;
    ; Inputs:
    ;   TOTAL_MOB_COUNT  | 0 on first run, triggers attract screen
    ;   LOCATION_DATA    | ($4009) base of character record array
    ;
    ; Behavior:
    ;   Calls GAME_INIT, then sets CHAR_PTR to LOCATION_DATA.  If
    ;   TOTAL_MOB_COUNT is zero (first run), calls ATTRACT_SCREEN.
    ;   Main loop: sets CURRENT_PLAYER=1, iterates 9-byte character
    ;   records.  For each active record (MFIELD_XP != 0), calls
    ;   ENTER_LOCATION, dispatches scene events for players 1 and 3,
    ;   and calls PROCESS_MOB_GROUPS.  Terminates the pass when byte 1
    ;   of the next record equals $FF, then calls RANDOM_EVENTS and
    ;   restarts unconditionally.
    ;
    ; Modifies:
    ;   CHAR_PTR, CURRENT_PLAYER, and numerous others via subcalls
    ; Clobbers: A, X, Y
    JSR     GAME_INIT
.restart:
    STOW    LOCATION_DATA,CHAR_PTR ;  $FA/$FB = LOCATION_DATA ($4009)
    LDA     TOTAL_MOB_COUNT    ; zero on first run (no mobs yet)
    BNE     .has_mobs          ; nonzero = game in progress
    JSR     ATTRACT_SCREEN
.has_mobs:
    STOB    #$01,CURRENT_PLAYER ; start with player 1
.char_loop:
    LDY     #MFIELD_XP
    LDA     (CHAR_PTR),Y       ; char record byte 8 (status/group)
    BEQ     .next_char         ; zero = inactive, skip
    JSR     ENTER_LOCATION     ; draw map if location changed
    LDA     CURRENT_PLAYER
    BAEQ    #$01,.run_scene    ; player 1?
    BANE    #$03,.process_mobs ; player 3?
.run_scene:
    JSR     CHECK_SCENE_EVENTS ; scene event dispatch (players 1 & 3)
.process_mobs:
    JSR     PROCESS_MOB_GROUPS ; walk mob list, run turns
.next_char:
    INC     CURRENT_PLAYER
    CLC
    LDA     #CHAR_RECORD_SIZE  ; advance to next 9-byte record
    ADC     CHAR_PTR
    STA     CHAR_PTR
    BCC     .no_carry
```

```
    INC     CHAR_PTR+1
.no_carry:
    LDY     #$01
    LDA     (CHAR_PTR),Y        ; byte 1 of next record
    BANE    #$FF,.char_loop     ; more characters -> loop
    JSR     RANDOM_EVENTS       ; between-round random event dispatch
    JMP     .restart            ; restart the game loop
```

Defines:
MAIN_ENTRY, used in chunks 428 and 675.
Uses ATTRACT_SCREEN 220, CHAR_PTR 591a, CHAR_RECORD_SIZE 25b, CHECK_SCENE_EVENTS 428, CURRENT_PLAYER 562a, ENTER_LOCATION 535, GAME_INIT 564, LOCATION_DATA 207, MFIELD_XP 561, PROCESS_MOB_GROUPS 222, RANDOM_EVENTS 415, STOB 8c, STOW 8a, and TOTAL_MOB_COUNT 591b.

ATTRACT_SCREEN initializes the attract-mode demo. Called from MAIN_ENTRY on first run (TOTAL_MOB_COUNT = 0), it sets ACTIVE_CHAR to 1, obtains the mob data pointer for scene 1 via GET_MOB_DATA, runs SCENE_SETUP, clears LOCATION_POS, and enters SCENE_LOOP with scene $26 (the attract demo).

220 ⟨*attract screen* 220⟩≡ (909)

```
    ORG     $09F4
ATTRACT_SCREEN:
    SUBROUTINE ATTRACT_SCREEN
    ; Initialize attract-mode demo screen.
    ;
    ; Behavior:
    ;   Sets ACTIVE_CHAR to 1, initializes mob data pointer for scene 1,
    ;   runs scene setup, clears location flag, and enters SCENE_LOOP
    ;   with scene $26 (attract demo).
    ;
    ; Modifies: ACTIVE_CHAR, SCENE_EVENT_RESULT, MOB_PTR, LOCATION_POS
    ; Clobbers: A, X, Y

    STOB    #$01,ACTIVE_CHAR            ; scene/character index = 1
    STA     SCENE_EVENT_RESULT          ; mark event as handled
    JSR     GET_MOB_DATA                ; scene 1 -> MOB_DATA_PTR
    MOVW    MOB_DATA_PTR,MOB_PTR        ; save pointer
    JSR     SCENE_SETUP
    STOB    #$00,LOCATION_POS           ; clear location-change flag
    JSR     SET_CURSOR_ROW21
    LDA     #$26                        ; scene $26 (attract demo)
    JMP     SCENE_LOOP                  ; tail-call
```

Defines:
ATTRACT_SCREEN, used in chunk 219.
Uses ACTIVE_CHAR 492c, GET_MOB_DATA 602, LOCATION_POS 562b, MOB_DATA_PTR 600b, MOB_PTR 608, MOVW 9a, SCENE_EVENT_RESULT 207, SCENE_LOOP 527, SCENE_SETUP 494, SET_CURSOR_ROW21 627, and STOB 8c.

RES_VM_LOOP is the entry point and main loop for the resident bytecode interpreter (the copy-protection verifier). Despite its former name "ATTRACT_LOOP," it has nothing to do with the attract screen (ATTRACT_SCREEN).

CONTEXT_SWAP swaps zero-page $50–$5F with the shadow area at RES_ZP_SHADOW, giving the VM its own private workspace (including VM_IP, VM_OPERAND, VM_ACC). RESIDENT_DISPATCH then runs the *entire* bytecode program from RESIDENT_BYTECODES ($0572) to completion. When the VM exits via VM_EXIT_OK (which calls CONTEXT_SWAP again to restore the main ZP state), the loop restarts.

221 ⟨*res vm loop* 221⟩≡ (909)

```
      ORG     $0569
  RES_VM_LOOP:
      SUBROUTINE RES_VM_LOOP
      ; Entry point for the resident bytecode interpreter.
      ;
      ; Behavior:
      ;   Swaps ZP context ($50-$5F) to restore VM state, then runs the
      ;   ENTIRE bytecode program from $0572 (RESIDENT_BYTECODES) to
      ;   completion.  Handlers chain via JMP RESIDENT_DISPATCH_FETCH
      ;   and never return here between opcodes.  VM_EXIT_OK swaps the
      ;   ZP context back and returns; then the loop restarts.
      ;
      ; Clobbers: A, X, Y

      JSR     CONTEXT_SWAP              ; swap main ZP <-> VM ZP
      JSR     RESIDENT_DISPATCH         ; run bytecode program to completion
      JMP     RES_VM_LOOP               ; restart
```

Defines:
RES_VM_LOOP, used in chunks 689 and 702.
Uses CONTEXT_SWAP 700, RESIDENT_BYTECODES 696, RESIDENT_DISPATCH 702, and VM_EXIT_OK 696.

PROCESS_MOB_GROUPS walks the mob linked list for the current character, processing each group according to its type.

Group classification. Each group's hit points (MFIELD_HP, bits 5–0 of mob data byte 6) determine how it is handled:

- HP < 3 (*inactive*): the mob goes idle and heals. SHOW_IDLE_STATUS prints a status message; AUTO_HEAL gradually restores hit points.
- HP ≥ 3 (*active*): START_MOB_TURN sets up the sprite and enters GAME_TURN_LOOP if the group needs a turn.

Disk checkpoint. A 1-in-24 PRNG roll (AND $17) triggers a periodic disk checkpoint at VERIFY_DISK_IO.

222 ⟨*process mob groups* 222⟩≡ (909)

```
    ORG     $0884
PROCESS_MOB_GROUPS:
    SUBROUTINE PROCESS_MOB_GROUPS
    ; Walk the mob linked list for the current character and process each mob.
    ;
    ; Inputs:
    ;   CHAR_PTR  | pointer to current character record
    ;
    ; Behavior:
    ;   Reads CFIELD_NEXT from (CHAR_PTR) as the initial link and saves it
    ;   in CURRENT_ROOM.  Rolls a 1-in-24 PRNG chance to call the disk
    ;   checkpoint at [[VERIFY_DISK_IO]].  Then iterates the linked list: for each mob,
    ;   resolves via GET_MOB_DATA, copies to ENTITY_PTR, reads the next link.
    ;   HP < 3 (inactive): SHOW_IDLE_STATUS + AUTO_HEAL.
    ;   HP >= 3 (active): START_MOB_TURN, then reset BLINK_COL to MAP_WIDTH.
    ;   Loop continues until the link value is 0.
    ;
    ; Modifies:
    ;   CURRENT_ROOM, ENTITY_PTR, MOB_DATA_PTR, BLINK_COL
    ; Clobbers: A, X, Y
    ; Walk the mob linked list for the current character.
    ; For each group: inactive (type < 3) -> display message + auto-heal;
    ; active (type >= 3) -> set up scene and enter GAME_TURN_LOOP.
    LDY     #CFIELD_NEXT
    LDA     (CHAR_PTR),Y     ; char record byte 2 = first group link
    STA     CURRENT_ROOM     ; save as current link
    JSR     CLEAR_MAP
    JSR     STEP_PRNG
    LDA     PRNG_OUTPUT
    AND     #$17             ; 1-in-24 chance (bits 0-2,4)
    BNE     .walk_list
```

```
    JSR VERIFY_DISK_IO            ; periodic disk checkpoint
.walk_list:
    LDA     CURRENT_ROOM      ; current link in chain
    BNE     .has_mob
    RTS                       ; zero = end of list
.has_mob:
    JSR     GET_MOB_DATA      ; resolve link -> MOB_DATA_PTR/MOB_DATA_PTR+1
    MOVW    MOB_DATA_PTR,ENTITY_PTR ;  ENTITY_PTR/ENTITY_PTR+1 = mob data pointer
    LDY     #MFIELD_NEXT
    LDA     (ENTITY_PTR),Y    ; mob data byte 2 = next link
    STA     CURRENT_ROOM      ; save for next iteration
    LDY     #MFIELD_HP
    LDA     (ENTITY_PTR),Y    ; mob data byte 6
    AND     #HP_MASK    ; bits 5-0 = HP
    BAPL    #$03,.active_mob ; type >= 3?
    JSR     SHOW_IDLE_STATUS ; inactive: print name + message
    JSR     AUTO_HEAL         ; heal party members
    JMP     .walk_list
.active_mob:
    JSR     START_MOB_TURN    ; active: set up sprite, enter turn loop
    STOB    #$14,BLINK_COL ; reset blink column to 20
    JMP     .walk_list
```

Defines:
PROCESS_MOB_GROUPS, used in chunk 219.

Uses AUTO_HEAL 377, BLINK_COL 563b, CFIELD_NEXT 561, CHAR_PTR 591a, CLEAR_MAP 539, CURRENT_ROOM 476, ENTITY_PTR 561, GAME_TURN_LOOP 232, GET_MOB_DATA 602, MAP_WIDTH 388a, MFIELD_HP 561, MFIELD_NEXT 561, MOB_DATA_PTR 600b, MOVW 9a, PRNG_OUTPUT 593a, SHOW_IDLE_STATUS 224, START_MOB_TURN 225, STEP_PRNG 595, STOB 8c, and VERIFY_DISK_IO 671.

When an inactive mob group is encountered during the list walk, SHOW_IDLE_STATUS prints a quick status line. It scrolls the status area, prints the mob's name, scrolls again, and displays message $20—the idle status text.

224 ⟨*show idle status* 224⟩≡ (909)

```
    ORG     $08D1
SHOW_IDLE_STATUS:
    SUBROUTINE SHOW_IDLE_STATUS
    ; Display a brief status message for an inactive mob group.
    ;
    ; Inputs:
    ;   ENTITY_PTR  | pointer to current mob record (bytes 0-1 = name)
    ;
    ; Behavior:
    ;   Scrolls the status area, prints the mob's name via PRINT_MOB_NAME,
    ;   scrolls again, then displays MSG_REST_STRONGER ("resting, growing
    ;   stronger") via DISPLAY_MESSAGE.
    ; Clobbers: A, X, Y
    JSR     SCROLL_STATUS_LINE
    JSR     PRINT_MOB_NAME
    JSR     SCROLL_STATUS_LINE
    LDA     #MSG_REST_STRONGER                  ; message $20: idle status text
    JMP     DISPLAY_MESSAGE
```

Defines:
SHOW_IDLE_STATUS, used in chunk 222.
Uses DISPLAY_MESSAGE 637, ENTITY_PTR 561, MSG_REST_STRONGER 634, PRINT_MOB_NAME 484, and SCROLL_STATUS_LINE 639.

START_MOB_TURN prepares the display for an active mob group (type ≥ 3) and enters the turn loop.

AI mode derivation. The mob's MFIELD_APPEARANCE (byte 4) is converted to a font character via APPEARANCE_TO_FONTCHAR. The Y register after conversion holds the font group, which doubles as the AI behavior mode stored in CHAR_AI_MODE: 0 = player-controlled, higher values select different NPC AI strategies.

Turn gate. CHECK_MOB_NEEDS_TURN decides whether the group actually needs a turn. If carry is clear, the mob is skipped. Otherwise the scene display is set up and control enters GAME_TURN_LOOP.

225 ⟨*start mob turn* 225⟩≡ (909)

```
    ORG     $08DF
START_MOB_TURN:
    SUBROUTINE START_MOB_TURN
    ; Set up the sprite and enter the turn loop for an active mob.
    ;
    ; Inputs:
    ;   ENTITY_PTR  | pointer to current mob record
    ;
    ; Behavior:
    ;   Reads MFIELD_APPEARANCE (byte 4) from the mob record and converts
    ;   it to a font character via APPEARANCE_TO_FONTCHAR.  The font group
    ;   (in Y) doubles as the AI behavior mode, stored in CHAR_AI_MODE.
    ;   Calls CHECK_MOB_NEEDS_TURN; if carry clear, returns immediately.
    ;   Otherwise positions the cursor, sets up the scene display, and
    ;   jumps to GAME_TURN_LOOP.
    ;
    ; Modifies:
    ;   CHAR_SPRITE, CHAR_AI_MODE
    ; Clobbers: A, X, Y
    LDY     #MFIELD_APPEARANCE
    LDA     (ENTITY_PTR),Y       ; mob data byte 4 = appearance
    JSR     APPEARANCE_TO_FONTCHAR
    STA     CHAR_SPRITE          ; save font character
    STY     CHAR_AI_MODE         ; Y = font group -> AI behavior mode
    JSR     CHECK_MOB_NEEDS_TURN ; carry set = needs a turn
    BCS     .enter_turn
    RTS                          ; carry clear = skip this mob
.enter_turn:
    JSR     SET_CURSOR_ROW21
    JSR     SETUP_SCENE_DISPLAY
    MOVB    CHAR_AI_MODE,$5AA6 ; save AI mode (unused in main.bin)
    JMP     GAME_TURN_LOOP
```

Defines:

START_MOB_TURN, used in chunk 222.
Uses APPEARANCE_TO_FONTCHAR 461, CHAR_AI_MODE 531b, CHAR_SPRITE 486b, CHECK_MOB_NEEDS_TURN 227, ENTITY_PTR 561, GAME_TURN_LOOP 232, MFIELD_APPEARANCE 561, MOVB 8c, SETUP_SCENE_DISPLAY 532c, and SET_CURSOR_ROW21 627.

Not every mob group needs to act every round. CHECK_MOB_NEEDS_TURN applies three filters, returning with carry set if the mob should take a turn and carry clear if it can be skipped.

Player-controlled characters (AI mode 0) always get a turn. For AI-controlled mobs, the routine checks mob data byte 12's low nibble: if nonzero, the mob has a pending action queued from a previous round. Failing that, it checks the mob's current position for encounters (via CHECK_ENCOUNTER) and adjacent threats (via CHECK_ADJACENT_THREATS). If neither is found, the mob has nothing to react to, so CLEAR_RESTRICTED is called and the turn is skipped. This keeps the game moving briskly—mobs in empty corridors don't waste time.

226a ⟨*var encounter result* 226a⟩≡ (909)

```
    ORG     $5A1F
ENCOUNTER_RESULT:
    DC.B    $C3                     ; initial value (runtime: 0/1/2 from CHECK_ENCOUNTER)
```

Defines:
ENCOUNTER_RESULT, used in chunks 227, 232, 244, 308, 312, 327, 357, 368, and 898.
Uses CHECK_ENCOUNTER 312.

226b ⟨*var adjacent threat* 226b⟩≡ (909)

```
    ORG     $5A26
ADJACENT_THREAT:
    DC.B    $8D                     ; initial value (runtime: accumulated adjacent threat level)
```

Defines:
ADJACENT_THREAT, used in chunks 227, 308, 320, 322, 357, and 898.

227 ⟨*check mob needs turn* 227⟩≡ (909)

```
      ORG     $0958
  CHECK_MOB_NEEDS_TURN:
      SUBROUTINE CHECK_MOB_NEEDS_TURN
      ; Determine whether a mob group needs to take a game turn.
      ;
      ; Inputs:
      ;   CHAR_AI_MODE  | 0 = player (always gets turn), nonzero = AI
      ;   ENTITY_PTR    | pointer to current mob record
      ;
      ; Behavior:
      ;   Player-controlled characters always get a turn.  For AI mobs,
      ;   checks three conditions: (1) MFIELD_LEVEL_ENC low nibble nonzero
      ;   (pending action), (2) encounter at current MFIELD_POSITION, or
      ;   (3) adjacent threats.  If none apply, clears the restricted flag
      ;   and skips the turn.
      ;
      ; Outputs:
      ;   C flag  | set = needs turn, clear = skip
      ; Clobbers: A, Y
      LDA     CHAR_AI_MODE
      BEQ     .needs_turn      ; player-controlled always gets a turn
      LDY     #MFIELD_LEVEL_ENC
      LDA     (ENTITY_PTR),Y   ; mob data byte 12
      AND     #LOW_NIBBLE_MASK ; low nibble = pending action
      BNE     .needs_turn      ; nonzero = pending action -> turn
      LDY     #MFIELD_POSITION
      LDA     (ENTITY_PTR),Y   ; mob data byte 3 = position
      JSR     CHECK_ENCOUNTER  ; check for encounter at this position
      LDA     ENCOUNTER_RESULT
      BNE     .needs_turn      ; encounter found -> turn
      LDY     #MFIELD_POSITION
      LDA     (ENTITY_PTR),Y   ; position again
      JSR     CHECK_ADJACENT_THREATS
      LDA     ADJACENT_THREAT
      BNE     .needs_turn      ; threats nearby -> turn
      JSR     CLEAR_RESTRICTED ; no action needed -> clear restricted flag
      CLC                      ; carry clear = skip
      RTS
  .needs_turn:
      SEC                      ; carry set = needs turn
      RTS
```

Defines:
CHECK_MOB_NEEDS_TURN, used in chunk 225.

Uses ADJACENT_THREAT 226b, CHAR_AI_MODE 531b, CHECK_ADJACENT_THREATS 320, CHECK_ENCOUNTER 312, CLEAR_RESTRICTED 245, ENCOUNTER_RESULT 226a, ENTITY_PTR 561, LOW_NIBBLE_MASK 553, and MFIELD_POSITION 561.

Chapter 6

The Game Turn

According to the original manual:

Strike probability

Whenever any character tries to strike another character, the game will compute his chance of success. Any character's base chance of striking his victim is 3.12511 has a 34.375adjusted by several factors:

If the victim is:

particularly difficult to hit

(halflings and elves have this ability to dodge) . . .. subtract 12.5particularly easy to hit. . . . . add 12.5extremely easy to hit .... add 25down (unconscious, in hand to hand combat, or resting) .. . . add 18.75currently resting (this is in addition to the adjustment for being down) . . . . add 9.375 defending . .. . subtract 9.375attacking .... no adjustment running . . .. subtract 3.125 on the same space with attacker, the stronger character is is given an advantage .. . .. . add 1.5625subtract 1.5625

The chance of striking the victim is then adjusted, if necessary, so that it is never less than 3.125

The probabilities are in increments of 1/32. In the code, 1/32 is represented by a single count in 6 bits, so the maximum count is 31 (96.875

DATA[0,1] = name DATA[4] = creature type: 0x00: Human 0x01: Elf 0x02: Dwarf 0x03: Halfling 0x15: Human 0x23: Human (Morgiana, Princess Buddir)

0x3C: Owl 0x3F: Statue, Sultan, Right guard, Left guard, small zombie, big zombie 0x41: Dwarf 0x43: Thief 0x44: Bear 0x45: Rat 0x46: Wolf 0x47: Dragon 0x48: Greasy 0x4A: Sword 0x4B: Ram 0x4C: Scorpion 0x4D: Succubus 0x4E: Tiger, Leo, Jackal 0x4F: Crab 0x50: Bull 0x54: Zombie (Witless) 0x58: Thief 0x59: Brown Bear 0x5E: Wanderer 0x5F: Mechanical Sword 0x61: Scorpion (Giant) 0x65: Sonny Boy Minotaur 0x93: Statue (Gemini I, Gemini II) 0x93: Aladdin 0x94: Elf 0x9B: White Dragon 0x9E: Bane of Thieves 0xA6: Unicorn

DATA[5] = STR DATA[6] = CON/HP 0-3F DATA[7] = MEL-1 DATA[9] = HTH-1 DATA[B] = 7: Extremely easy to hit 6: Particularly difficult to hit 5: ? Incredibly easy to hit? 4: DEX-1 3: DEX-1 2: DEX-1 1: DEX-1 0: DEX-1

DATA[C] = SPD DATA[F] = 7: 6: 5: 4: 3: 2: Unconscious/Down 1: State 0: State

State: 0: 1: 2: 3: Defending

Handles one character's complete turn—movement, input, encounters, and event dispatch. Called once per character per game tick.

Setup. The character's current position (field 3 of the record at `ENTITY_PTR`) is split into column and row. Movement flags are cleared, and the number of movement points is read from field 12 (low nibble). If the character's constitution (field 6, bits 0–5) is below 3, or if `CHECK_ENCOUNTER` returns 2 (hostile territory), a "restricted" flag is set (bit 2 of field 15).

Input. For the player (`CHAR_AI_MODE` = 0), keyboard input is read via the keyboard input routine at `READ_KEYBOARD`. For NPCs, `AI_CHOOSE_TARGET` selects a destination and `FUN_6742` computes a direction. The direction value encodes both movement and commands:

Value	Action
1	North (row−)
2	South (row+)
3	West (col−)
4	East (col+)
5	Pass (skip turn)
6	Action 6 → $14CD
7	Action 7 → $14B7
8	Action 8 → set state 1
9	Fight/attack → $143F

Movement. After applying the direction, the new position is converted via `COLROW_TO_POS` and checked with `FIND_ENTITY_AT_POS`. If the cell is empty, the

move is committed (position updated, sprite redrawn) and adjacent threats are re-checked. Finding no threats ends the turn early.

Encounter dispatch. If something occupies the destination cell, the result from FIND_ENTITY_AT_POS determines the handler:

- $BF ≥ $80 — mob encounter → $1E5A
- Event byte 1 bits 7–6 ≠ $C0 — HANDLE_ENCOUNTER
- $C0–$D1 — PICKUP_TREASURE
- $D2–$D4 — TRIGGER_SCENE_EVENT
- $D5–$DA — map/area event → $70DF
- $FE — empty-slot interaction → $1D4D

TURN_START_COL and TURN_START_ROW record the character's position at the start of the turn (for restoring the old cell after a move). MOVE_POINTS is the movement budget; STEPS_TAKEN counts steps consumed so far. INPUT_DIR holds the direction or command code (1–9) from the player or AI. TURN_ACTIVE is set to 1 while the turn loop is running. All six variables are initialized by GAME_TURN_LOOP at the start of each character's turn.

230a ⟨*var turn start col* 230a⟩≡ (909)

```
    ORG     $5A61
TURN_START_COL:
    DC.B    $C1                     ; initial value (runtime: column at start of turn)
```

Defines:
TURN_START_COL, used in chunks 232, 239, 354, 357, 360, and 898.

230b ⟨*var turn start row* 230b⟩≡ (909)

```
    ORG     $5A62
TURN_START_ROW:
    DC.B    $A0                     ; initial value (runtime: row at start of turn)
```

Defines:
TURN_START_ROW, used in chunks 232, 239, 354, 357, 360, and 898.

230c ⟨*var move points* 230c⟩≡ (909)

```
    ORG     $5A65
MOVE_POINTS:
    DC.B    $CD                     ; initial value (runtime: movement points remaining)
```

Defines:
MOVE_POINTS, used in chunks 232, 261, 264, and 360.

231a ⟨*var steps taken* 231a⟩≡ (909)

```
    ORG     $5A66
STEPS_TAKEN:
    DC.B    $D0                     ; initial value (runtime: steps taken this turn)
```

Defines:
STEPS_TAKEN, used in chunks 232, 241, 308, and 368.

231b ⟨*var input dir* 231b⟩≡ (909)

```
    ORG     $5A67
INPUT_DIR:
    DC.B    $AB                     ; initial value (runtime: direction/command from input (1-9))
```

Defines:
INPUT_DIR, used in chunk 232.

232 ⟨*game turn loop* 232⟩≡ (909)

```
        ORG     $19AD
    GAME_TURN_LOOP:
        SUBROUTINE GAME_TURN_LOOP
        ; Execute one character's complete movement turn.
        ;
        ; Inputs:
        ;   ENTITY_PTR    | pointer to current entity record
        ;   CHAR_AI_MODE  | 0 = player (keyboard input), nonzero = AI
        ;
        ; Behavior:
        ;   Saves starting position, clears flags.  Checks HP (field 6 low
        ;   6 bits < 3 = restricted) and encounter type (2 = restricted).
        ;   For AI, calls AI_CHOOSE_TARGET.  Computes movement points from
        ;   field 12 low nibble, optionally reduced by threat count.
        ;   Main loop: reads direction (player keyboard via $A44C, or
        ;   NPC_MOVE_AI), applies to col/row, checks destination cell.
        ;   Empty = commit move + re-check threats.  Occupied = dispatch
        ;   by entity type: mob encounter, treasure, scene event, shop,
        ;   or empty-slot handler.  Exits when points exhausted or combat.
        ;
        ; Modifies:
        ;   TURN_ACTIVE, CURRENT_COL, CURRENT_ROW, TURN_START_COL,
        ;   TURN_START_ROW, STEPS_TAKEN, MOVE_POINTS, INPUT_DIR,
        ;   ENCOUNTER_RESULT
        ; Clobbers: A, X, Y
        ; --- setup: save position, clear flags ---
        STOB    #$01,TURN_ACTIVE
        LDY     #MFIELD_POSITION
        LDA     (ENTITY_PTR),Y          ; char field 3 = packed position
        JSR     POS_TO_COLROW
        STA     CURRENT_COL
        STA     TURN_START_COL
        STY     CURRENT_ROW
        STY     TURN_START_ROW
        STOB    #$00,STEPS_TAKEN
        JSR     CLEAR_RESTRICTED        ; clear restricted flag (bit 2 of field 15)

        ; --- check constitution and encounters ---
        LDY     #MFIELD_HP
        LDA     (ENTITY_PTR),Y          ; char field 6 (constitution/HP)
        AND     #HP_MASK
        BAPL    #$03,.con_ok
        JSR     SET_RESTRICTED          ; set restricted (too weak)
    .con_ok:
        LDY     #MFIELD_POSITION
        LDA     (ENTITY_PTR),Y
        JSR     CHECK_ENCOUNTER
        LDA     ENCOUNTER_RESULT
        BANE    #$02,.enc_ok
```

```
    JSR     SET_RESTRICTED          ; set restricted (hostile territory)
.enc_ok:
    LDA     CHAR_AI_MODE
    BEQ     .get_speed
    JSR     AI_CHOOSE_TARGET        ; NPC: pick a destination

    ; --- determine movement points ---
.get_speed:
    LDY     #MFIELD_LEVEL_ENC
    LDA     (ENTITY_PTR),Y          ; char field 12
    AND     #LOW_NIBBLE_MASK        ; low nibble = speed
    STA     MOVE_POINTS

    ; --- adjust points based on threats ---
    LDA     ENCOUNTER_RESULT
    BAEQ    #$02,.chk_limit         ; hostile -> check if enough to limit
    JSR     CHECK_THREATS_HERE      ; check adjacent threats
    BAEQ    #$01,.move_loop         ; exactly 1 threat -> keep full
    LDA     MOVE_POINTS
    BALT    #$02,.move_loop ; < 2 points -> keep as is
    JSR     RANDOM_IN_RANGE         ; random(move_points)
    BAEQ    #$00,.do_limit          ; random == 0 -> force limit
    STA     MOVE_POINTS             ; reduce randomly
    BNE     .move_loop
.chk_limit:
    LDA     MOVE_POINTS
    BALT    #$02,.move_loop ; < 2 -> use as is
.do_limit:
    LDA     #$01
    JSR     RANDOM_IN_RANGE         ; random(1) -> 0 or 1
    CLC
    ADC     #$01                    ; 1 or 2
    STA     MOVE_POINTS
    BNE     .move_loop              ; always

    ; === MOVEMENT LOOP ===
MOVE_LOOP:                      ; $1A31 - external entry point
.move_loop:
    LDA     #$00
    BANE    MOVE_POINTS,.has_points
    BANE    STEPS_TAKEN,.end_turn   ; out of points, took steps -> done
    BAEQ    CHAR_AI_MODE,.has_points ; player with 0 points, 0 steps -> still prompt
.end_turn:
    JMP     END_TURN                ; end turn

.has_points:
    BAEQ    CHAR_AI_MODE,.player_input
    JSR     NPC_MOVE_AI             ; NPC movement AI
    JMP     .dispatch_cmd
```

```
    ; --- player input ---
.player_input:
    JSR     CONFIGURE_INPUT          ; set up input UI
    LDA     GAME_INPUT_TABLE+4       ; check if movement enabled
    BNE     .have_input
    JMP     TEARDOWN_INPUT           ; no input -> exit
.have_input:
    JSR     GET_INPUT_TABLE_PTR
    JSR     SETUP_INPUT
    JSR READ_KEYBOARD                    ; read keyboard
    STA     INPUT_DIR
    BANE    #$00,.do_move            ; player goes straight to movement
    JMP     TEARDOWN_INPUT           ; cancelled

    ; --- NPC command dispatch (player skips this) ---
.dispatch_cmd:
    STA     INPUT_DIR
    BALT    #$05,.do_move ; 1-4: movement direction
    BNE     .not_pass
    JMP     .move_loop               ; 5: pass
.not_pass:
    BANE    #$09,.not_fight
    JSR     SET_RESTRICTED           ; 9: fight -> set restricted
    LDA     #$00
    JSR     SET_CHAR_STATE
    JMP     AUTO_HEAL
.not_fight:
    PHA
    JSR     CHECK_AND_RESTRICT       ; re-check encounter
    PLA
    BANE    #$07,.not_cmd7
    JMP     ENTER_COMBAT_STATE       ; 7: action
.not_cmd7:
    BCS     .cmd8
    JMP     START_COMBAT             ; 6: action
.cmd8:
    JMP     SET_STATE_FIGHT          ; 8: set state 1

    ; --- apply movement direction ---
.do_move:
    LDA     ENCOUNTER_RESULT
    BAMI    #$02,.move_ok
    JSR     CHECK_CAN_LEAVE          ; can we leave hostile zone?
    BAEQ    #$01,.move_ok
    JMP     END_TURN                 ; blocked -> end turn
.move_ok:
    STOB    #$00,ENCOUNTER_RESULT
    LDA     INPUT_DIR
    DEC     MOVE_POINTS
    INC     STEPS_TAKEN
```

```
    BAPL    #$02,.not_north
    DEC     CURRENT_ROW             ; 1: north
    JMP     .check_dest
.not_north:
    BNE     .not_south
    INC     CURRENT_ROW             ; 2: south
    JMP     .check_dest
.not_south:
    BANE    #$03,.go_east
    DEC     CURRENT_COL             ; 3: west
    JMP     .check_dest
.go_east:
    INC     CURRENT_COL             ; 4: east

    ; --- check what's at the destination ---
.check_dest:
    LDA     CURRENT_COL
    LDY     CURRENT_ROW
    JSR     COLROW_TO_POS
    JSR     FIND_ENTITY_AT_POS      ; -> ENTITY_REC/ENTITY_REC+1
    LDA     #$00
    BANE    ENTITY_REC+1,.something ; ENTITY_REC+1 != 0 -> occupied
    BAEQ    ENTITY_REC,.empty       ; ENTITY_REC == 0 -> empty cell
    LDX     #$0E                    ; tune 7: entity interaction
    JSR     TUNE_PLAYER
    JMP     SHOW_LOCKED_MSG

.empty:
    JSR     COMMIT_MOVE             ; commit move (update pos, draw sprite)
    JSR     CHECK_THREATS_HERE      ; check adjacent threats
    BANE    #$00,.keep_moving       ; threats remain -> continue
    STA     MOVE_POINTS             ; no threats -> stop
    STA     TURN_ACTIVE
.keep_moving:
    JMP     .move_loop

    ; --- destination occupied: dispatch by type ---
.something:
    LDA     ENTITY_REC+1
    BAMI    #MOB_PTR_FLAG,.event    ; ENTITY_REC+1 < $80 -> event entry
    JMP     HANDLE_MOB_ENCOUNTER    ; ENTITY_REC+1 >= $80 -> mob encounter

.event:
    LDY     #EVFIELD_TYPE
    LDA     (ENTITY_REC),Y          ; event byte 1
    TAX
    AND     #EVENT_ACTIVATED
    BAEQ    #EVENT_ACTIVATED,.activated
    JMP     HANDLE_ENCOUNTER        ; non-activated -> encounter
```

```
.activated:
    TXA
    BAPL    #$D2,.not_treasure
    JMP     PICKUP_TREASURE          ; $C0-$D1: treasure

.not_treasure:
    BAPL    #$D5,.not_scene
    JMP     TRIGGER_SCENE_EVENT      ; $D2-$D4: scene event

.not_scene:
    BAPL    #$DB,.not_map_event
    PSHW    ENTITY_REC               ; $D5-$DA: save ENTITY_REC/ENTITY_REC+1
    JSR     COMMIT_MOVE              ; commit move first
    PULW    ENTITY_REC               ; restore ENTITY_REC/ENTITY_REC+1
    JMP     DISPLAY_SHOP

.not_map_event:
    BAEQ    #EVENT_EMPTY,.empty_slot
    JMP     TEARDOWN_INPUT           ; $DB-$FD: unknown

.empty_slot:
    JMP     HANDLE_EVENT_SLOT        ; EVENT_EMPTY ($FE): empty slot interaction
```

Defines:
GAME_TURN_LOOP, used in chunks 222 and 225.
MOVE_LOOP, used in chunks 273, 278, 281, 283, 293, 295, 357, 360, 433, 440, 617, 644, and 898.

Uses AI_CHOOSE_TARGET 385, AUTO_HEAL 377, CHAR_AI_MODE 531b, CHECK_AND_RESTRICT 244, CHECK_CAN_LEAVE 456, CHECK_ENCOUNTER 312, CHECK_THREATS_HERE 323, CLEAR_RESTRICTED 245, COLROW_TO_POS 600a, COMMIT_MOVE 239, CONFIGURE_INPUT 264, CURRENT_COL 472b, CURRENT_ROW 473a, DISPLAY_SHOP 404, ENCOUNTER_RESULT 226a, END_TURN 241, ENTER_COMBAT_STATE 327, ENTITY_PTR 561, ENTITY_REC 541b, EVENT_ACTIVATED 397a, EVENT_EMPTY 397a, EVFIELD_TYPE 397a, FIND_ENTITY_AT_POS 469, GAME_INPUT_TABLE 261, GET_INPUT_TABLE_PTR 263, HANDLE_ENCOUNTER 354, HANDLE_EVENT_SLOT 360, HANDLE_MOB_ENCOUNTER 357, INPUT_DIR 231b, LOW_NIBBLE_MASK 553, MFIELD_HP 561, MFIELD_POSITION 561, MOB_PTR_FLAG 468, MOVE_POINTS 230c, NPC_MOVE_AI 390, PICKUP_TREASURE 447, POS_TO_COLROW 599, PSHW 9b, PULW 10a, RANDOM_IN_RANGE 597, READ_KEYBOARD 663, SETUP_INPUT 249a, SET_CHAR_STATE 242, SET_RESTRICTED 246, SET_STATE_FIGHT 243, SHOW_LOCKED_MSG 363, START_COMBAT 328, STEPS_TAKEN 231a, STOB 8c, TEARDOWN_INPUT 262, TRIGGER_SCENE_EVENT 444, TUNE_PLAYER 682, TURN_ACTIVE 570b, TURN_START_COL 230a, and TURN_START_ROW 230b.

TURN_DISPATCH is the decision point after a character finishes moving. Player-controlled characters (CHAR_AI_MODE = 0) jump to TEARDOWN_INPUT to present the action menu. AI characters call SELECT_COMBAT_TARGET, which returns a status code:

- 2 — hostile at same position → ENTER_COMBAT_STATE
- Negative — AI commits to attack → START_COMBAT
- Other — area safe → AUTO_HEAL

237 ⟨*turn dispatch* 237⟩≡ (909)

```
    ORG     $14A3
TURN_DISPATCH:
    SUBROUTINE TURN_DISPATCH
    ; Post-movement decision point.
    ;
    ; Inputs:
    ;   CHAR_AI_MODE  | 0 = player, nonzero = AI
    ;
    ; Behavior:
    ;   If player-controlled, jumps to TEARDOWN_INPUT to present the
    ;   action menu.  For AI, calls SELECT_COMBAT_TARGET: result 2
    ;   enters combat state, negative commits an attack via START_COMBAT,
    ;   otherwise attempts AUTO_HEAL.
    ; Clobbers: A
    LDA     CHAR_AI_MODE
    BEQ     .no_combat
    JSR     SELECT_COMBAT_TARGET
    BAEQ    #$02,ENTER_COMBAT_STATE
    BMI     START_COMBAT
    JMP     AUTO_HEAL
.no_combat:
    JMP     TEARDOWN_INPUT
```

Defines:
TURN_DISPATCH, used in chunks 241 and 262.
Uses AUTO_HEAL 377, CHAR_AI_MODE 531b, ENTER_COMBAT_STATE 327, SELECT_COMBAT_TARGET 368, START_COMBAT 328, and TEARDOWN_INPUT 262.

COMMIT_MOVE finalizes a character's movement on the map:

1. Writes CURRENT_COL and CURRENT_ROW into the font and blink-cursor parameters.
2. Packs them into a single byte with COLROW_TO_POS.
3. Stores the result in the entity's position field (byte 3 of ENTITY_PTR).
4. Draws the character's sprite at the new location via RENDER_FONT_CHAR.
5. Sets TURN_TUNE_GUARD to $16.

Execution falls through into RESET_TURN_POS, which:

1. Converts the turn's starting position back into a packed byte.
2. Calls DRAW_CHAR_AT_POS to restore whatever was underneath the character's old location.
3. Updates the starting position to the new current position.

If TURN_TUNE_GUARD is exactly $16—meaning this is the first call since COMMIT_MOVE set it—the guard is decremented and the TUNE_PLAYER is invoked to play the movement sound effect.

238 ⟨*var turn tune guard* 238⟩≡ (909)

```
    ORG     $5AA9
TURN_TUNE_GUARD:
    DC.B    $D9                     ; initial value (runtime: turn reset: guards one-shot script exe
```

Defines:
TURN_TUNE_GUARD, used in chunk 239.

239 ⟨*commit move* 239⟩≡ (909)

```
    ORG     $1B52
COMMIT_MOVE:
    SUBROUTINE COMMIT_MOVE
    ; Finalize character movement on the map.
    ;
    ; Inputs:
    ;   CURRENT_COL, CURRENT_ROW  | new position
    ;   ENTITY_PTR                | pointer to entity record
    ;   CHAR_SPRITE               | font glyph for this character
    ;   TURN_START_COL/ROW        | position at start of turn
    ;
    ; Behavior:
    ;   Copies CURRENT_COL/ROW into font and blink cursor vars, packs
    ;   them via COLROW_TO_POS, stores in entity MFIELD_POSITION.
    ;   Draws the sprite via RENDER_FONT_CHAR.  Sets TURN_TUNE_GUARD
    ;   to $16 to arm the one-shot movement sound.  Falls through to
    ;   RESET_TURN_POS, which redraws the old cell and optionally
    ;   plays the movement tune.
    ;
    ; Modifies:
    ;   FONT_COL, FONT_ROW, FONT_CHARNUM, FONT_CHARSET,
    ;   BLINK_COL, BLINK_ROW, BLINK_CHAR, TURN_TUNE_GUARD,
    ;   TURN_START_COL, TURN_START_ROW
    ; Clobbers: A, X, Y
    MOVB    CURRENT_COL,FONT_COL
    STA     BLINK_COL
    LDY     CURRENT_ROW
    STY     FONT_ROW
    STY     BLINK_ROW
    JSR     COLROW_TO_POS
    LDY     #MFIELD_POSITION
    STA     (ENTITY_PTR),Y   ; update char packed position
    MOVB    CHAR_SPRITE,FONT_CHARNUM
    STA     BLINK_CHAR
    STOB    #$01,FONT_CHARSET
    JSR     RENDER_FONT_CHAR ; draw sprite
    LDX     #$16
    STX     TURN_TUNE_GUARD
    ; falls through to RESET_TURN_POS
RESET_TURN_POS:
    ; Reset to turn start: convert start col/row to packed pos,
    ; call $1156, update current->start, conditionally play tune
    LDA     TURN_START_COL
    LDY     TURN_START_ROW
    JSR     COLROW_TO_POS
    JSR     DRAW_CHAR_AT_POS
    MOVB    CURRENT_COL,TURN_START_COL
    MOVB    CURRENT_ROW,TURN_START_ROW
    LDX     TURN_TUNE_GUARD ; X=$16 = tune 11: movement effect
```

```
    BXEQ    #$16,.run_script
    RTS
.run_script:
    DEC     TURN_TUNE_GUARD
    JMP     TUNE_PLAYER     ; X still $16 from LDX above
```

Defines:
COMMIT_MOVE, used in chunks 232, 354, 357, 360, 444, 447, and 898.
RESET_TURN_POS, used in chunks 354 and 360.

Uses BLINK_CHAR 544c, BLINK_COL 563b, BLINK_ROW 544b, CHAR_SPRITE 486b, COLROW_TO_POS 600a, CURRENT_COL 472b, CURRENT_ROW 473a, DRAW_CHAR_AT_POS 471, ENTITY_PTR 561, FONT_CHARNUM 575, FONT_CHARSET 576a, FONT_COL 492a, FONT_ROW 534c, MFIELD_POSITION 561, MOVB 8c, RENDER_FONT_CHAR 466, STOB 8c, TUNE_PLAYER 682, TURN_START_COL 230a, TURN_START_ROW 230b, and TURN_TUNE_GUARD 238.

END_TURN wraps up a character's turn and checks for bonus movement.

Bonus turn. After clearing TURN_ACTIVE and updating the restricted flag via CHECK_AND_RESTRICT:

- If STEPS_TAKEN is zero, loops back to TURN_DISPATCH.
- Otherwise, extracts the speed tier from MFIELD_LEVEL_ENC (bits 2–0 after ROR). If the speed tier exceeds STEPS_TAKEN, the character gets another turn via TURN_DISPATCH.

241 ⟨*end turn* 241⟩≡ (909)

```
    ORG     $1BA7
END_TURN:
    SUBROUTINE END_TURN
    ; Wrap up the character's turn and check for bonus movement.
    ;
    ; Inputs:
    ;   ENTITY_PTR   | pointer to current entity record
    ;   STEPS_TAKEN  | steps consumed this turn
    ;
    ; Behavior:
    ;   Clears TURN_ACTIVE, calls CHECK_AND_RESTRICT to update the
    ;   restricted flag at the final position.  If no steps taken, loops
    ;   back to TURN_DISPATCH.  Otherwise extracts speed tier from
    ;   MFIELD_LEVEL_ENC bits 2-0 (after ROR) and compares against
    ;   STEPS_TAKEN; if level exceeds steps, grants another turn.
    ;
    ; Modifies:
    ;   TURN_ACTIVE
    ; Clobbers: A, Y
    ; End turn: clear TURN_ACTIVE, check encounter, compare level vs STEPS_TAKEN
    STOB    #$00,TURN_ACTIVE
    JSR     CHECK_AND_RESTRICT
    LDA     STEPS_TAKEN
    BNE     .check_level
    JMP     TURN_DISPATCH
.check_level:
    LDY     #MFIELD_LEVEL_ENC
    LDA     (ENTITY_PTR),Y
    ROR
    AND     #$07
    BAMI    STEPS_TAKEN,.too_low
    JMP     TURN_DISPATCH
.too_low:
```

Defines:
END_TURN, used in chunks 232, 357, and 898.
Uses CHECK_AND_RESTRICT 244, ENTITY_PTR 561, STEPS_TAKEN 231a, STOB 8c, TURN_ACTIVE 570b, and TURN_DISPATCH 237.

SET_CHAR_STATE writes a new value into the low two bits of byte 15 of the current entity record, preserving the upper six bits. The two-bit field encodes the character's turn phase: 0 for idle, 1 for fighting, 2 for entering combat, and 3 for actively attacking. Multiple routines call this to advance or reset the character through the turn state machine.

242 ⟨*set char state* 242⟩≡ (909)

```
      ORG     $1BE0
  SET_CHAR_STATE:
      SUBROUTINE SET_CHAR_STATE
      ; Write a new turn-phase state into entity field 15.
      ;
      ; Inputs:
      ;   A           | new state value (0-3)
      ;   ENTITY_PTR  | pointer to current entity record
      ;
      ; Behavior:
      ;   Writes A into the low 2 bits of MFIELD_FLAGS (byte 15),
      ;   preserving the upper 6 bits.  States: 0=idle, 1=fighting,
      ;   2=entering combat, 3=attacking.
      ;
      ; Modifies:
      ;   Entity byte MFIELD_FLAGS (low 2 bits)
      ; Clobbers: A, Y
      ; Set char field 15 low 2 bits to A
      STA     $5A60
      LDY     #MFIELD_FLAGS
      LDA     (ENTITY_PTR),Y
      AND     #$FC              ; ~FLAGS_STATE_MASK: clear low 2 bits
      ORA     $5A60             ; set to new value
      STA     (ENTITY_PTR),Y
      RTS
```

Defines:
SET_CHAR_STATE, used in chunks 232, 243, 327, 328, 354, 360, 363, 444, and 447.
Uses ENTITY_PTR 561 and FLAGS_STATE_MASK 336.

SET_STATE_FIGHT sets the character's turn state to 1 (fighting) by loading $01 and tail-calling SET_CHAR_STATE. This state indicates the character is engaged in combat but has not yet committed to an attack.

243 ⟨*set state fight* 243⟩≡ (909)

```
      ORG     $1BC6
  SET_STATE_FIGHT:
      SUBROUTINE SET_STATE_FIGHT
      ; Set the character's turn state to 1 (fighting).
      ;
      ; Behavior:
      ;   Loads A=1 and tail-calls SET_CHAR_STATE.  State 1 means the
      ;   character is engaged in combat but has not committed to an attack.
      ; Clobbers: A, Y
      LDA     #$01
      JMP     SET_CHAR_STATE
```

Defines:
SET_STATE_FIGHT, used in chunk 232.
Uses SET_CHAR_STATE 242.

CHECK_AND_RESTRICT evaluates the encounter at the character's current position and updates the restricted flag accordingly. It reads byte 3 of ENTITY_PTR (the packed position), calls CHECK_ENCOUNTER, clears the restricted flag, and then re-sets it only if the encounter result is 2 (hostile present). This ensures characters at hostile positions cannot freely move away without the flee mechanic.

244 ⟨*check and restrict* 244⟩≡ (909)

```
    ORG     $1BCB
CHECK_AND_RESTRICT:
    SUBROUTINE CHECK_AND_RESTRICT
    ; Evaluate the encounter at the entity's position and update restriction.
    ;
    ; Inputs:
    ;   ENTITY_PTR  | pointer to current entity record
    ;
    ; Behavior:
    ;   Reads MFIELD_POSITION from the entity record, calls CHECK_ENCOUNTER,
    ;   clears the restricted flag, then re-sets it only if ENCOUNTER_RESULT
    ;   is 2 (hostile present).
    ;
    ; Modifies:
    ;   Entity byte MFIELD_FLAGS (bit 2), ENCOUNTER_RESULT
    ; Clobbers: A, Y
    ; Check encounter at char position, clear restricted flag,
    ; set it if encounter type is 2.
    LDY     #MFIELD_POSITION
    LDA     (ENTITY_PTR),Y
    JSR     CHECK_ENCOUNTER
    JSR     CLEAR_RESTRICTED
    LDA     ENCOUNTER_RESULT
    BANE    #$02,.done
    JSR     SET_RESTRICTED
.done:
    RTS
```

Defines:
CHECK_AND_RESTRICT, used in chunks 232 and 241.
Uses CHECK_ENCOUNTER 312, CLEAR_RESTRICTED 245, ENCOUNTER_RESULT 226a, ENTITY_PTR 561, MFIELD_POSITION 561, and SET_RESTRICTED 246.

CLEAR_RESTRICTED clears bit 2 of the entity's field 15, removing the restricted-movement flag. This is typically called at the start of encounter evaluation before conditionally re-setting the flag.

245 ⟨*clear restricted* 245⟩≡ (909)

```
      ORG     $1BEF
  CLEAR_RESTRICTED:
      SUBROUTINE CLEAR_RESTRICTED
      ; Clear the restricted-movement flag (bit 2 of MFIELD_FLAGS).
      ;
      ; Inputs:
      ;   ENTITY_PTR  | pointer to current entity record
      ;
      ; Modifies:
      ;   Entity byte MFIELD_FLAGS (clears FLAGS_ENGAGED_BIT)
      ; Clobbers: A, Y
      LDY     #MFIELD_FLAGS
      LDA     (ENTITY_PTR),Y
      AND     #$FB              ; ~FLAGS_ENGAGED_BIT
      STA     (ENTITY_PTR),Y
      RTS
```

Defines:
CLEAR_RESTRICTED, used in chunks 227, 232, 244, and 327.
Uses ENTITY_PTR 561 and FLAGS_ENGAGED_BIT 336.

SET_RESTRICTED sets bit 2 of the entity's field 15, marking the character as movement-restricted. Restricted characters cannot freely walk away from their current position—they must either fight or attempt a flee check.

246 ⟨*set restricted* 246⟩≡ (909)

```
      ORG     $1BF8
  SET_RESTRICTED:
      SUBROUTINE SET_RESTRICTED
      ; Set the restricted-movement flag (bit 2 of MFIELD_FLAGS).
      ;
      ; Inputs:
      ;   ENTITY_PTR  | pointer to current entity record
      ;
      ; Behavior:
      ;   Sets FLAGS_ENGAGED_BIT in MFIELD_FLAGS.  Restricted characters
      ;   cannot freely leave their cell | they must fight or flee.
      ;
      ; Modifies:
      ;   Entity byte MFIELD_FLAGS (sets FLAGS_ENGAGED_BIT)
      ; Clobbers: A, Y
      ; Set restricted flag (bit 2 of char field 15)
      LDY     #MFIELD_FLAGS
      LDA     (ENTITY_PTR),Y
      ORA     #FLAGS_ENGAGED_BIT
      STA     (ENTITY_PTR),Y
      RTS
```

Defines:
SET_RESTRICTED, used in chunks 232, 244, 327, and 360.
Uses ENTITY_PTR 561 and FLAGS_ENGAGED_BIT 336.

Chapter 7

Input System

The game's menu-driven input system uses a table-based architecture. Each menu is defined by a descriptor block pointed to by MENU_DESC_PTR: byte 0 is the number of options, byte 1 is the default selection, bytes 2–3 hold a pointer to an initialization routine, and each option record contains display data and an exit handler. INPUT_LOOP drives the cycle: it calls SETUP_INPUT to initialize the menu, then enters a loop that calculates the record pointer for the current selection, sets up the cursor highlight, and waits for keyboard input. If the player presses return (indicated by a zero return from READ_KEYBOARD), the exit handler address is loaded from bytes 8–9 of the option record and executed via an indirect jump. Any other keypress is validated by VALIDATE_INPUT; if it changes the selection, the cursor is repositioned and the loop continues; if the selection stays the same, the tune player is called with tune $12 to play the in-place action.

INPUT_JMP_VEC is a 2-byte indirect jump vector used by INPUT_LOOP to dispatch exit handlers and initialization callbacks. CURRENT_SELECTION tracks which menu option is currently highlighted. INPUT_LOOP writes the exit handler address from the selected record into INPUT_JMP_VEC and executes it via JMP (INPUT_JMP_VEC).

READ_KEYBOARD is the keyboard input routine. INPUT_LOOP calls it to wait for a keypress; a zero return indicates the player pressed return (select), while any other value is passed to VALIDATE_INPUT.

247 ⟨*input loop* 247⟩≡ (909)

```
      ORG     $5F19
  INPUT_LOOP:
      SUBROUTINE INPUT_LOOP
      ; Menu driver loop: highlight selection, read input, dispatch.
      ;
```

```
    ; Inputs:
    ;   X  | high byte of menu descriptor pointer
    ;   Y  | low byte of menu descriptor pointer
    ;
    ; Behavior:
    ;   Calls SETUP_INPUT to initialize the menu.  Reads the default
    ;   selection from descriptor byte 1.  Loops: calculates the record
    ;   pointer, highlights via SETUP_CURSOR_EXT, waits for keyboard
    ;   input.  On return key (0), loads the exit handler address from
    ;   record bytes 8-9 into INPUT_JMP_VEC and jumps through it.
    ;   Other keys are validated; if the selection changes, repositions
    ;   the cursor; if unchanged, plays the in-place tune.
    ;
    ; Modifies:
    ;   CURRENT_SELECTION, INPUT_JMP_VEC
    ; Clobbers: A, X, Y
    JSR     SETUP_INPUT
    LDY     #$01
    LDA     ($C0),Y
    STA     CURRENT_SELECTION
.loop:
    LDA     CURRENT_SELECTION
    JSR     CALC_RECORD_PTR
    JSR     SETUP_CURSOR_EXT
    JSR     READ_KEYBOARD
    BEQ     .exit
    JSR     VALIDATE_INPUT
    BAEQ    CURRENT_SELECTION,.run_script
    BAEQ    #$00,.run_script
    STA     CURRENT_SELECTION
    JSR     SETUP_CURSOR
    JMP     .loop
.run_script:
    LDX     #$12                ; tune 9: menu selection effect
    JSR     TUNE_PLAYER
    JMP     .loop
.exit:
    LDWIY    $C2,#$08,INPUT_JMP_VEC
    JMP     (INPUT_JMP_VEC)
```

Defines:
INPUT_LOOP, used in chunks 262, 266, 267b, 270a, 272, 273, 279, 280b, 283, 286, and 588a.
Uses CALC_RECORD_PTR 251, CURRENT_SELECTION 54a, INPUT_JMP_VEC 53b, LDWIY 17, READ_KEYBOARD 663, SETUP_CURSOR 253, SETUP_CURSOR_EXT 252, SETUP_INPUT 249a, TUNE_PLAYER 682, and VALIDATE_INPUT 255.

SETUP_INPUT initializes a menu interaction. It stores the X and Y register values as the high and low bytes of the menu descriptor pointer ($C1 and $C0), calls the menu's initialization routine through CALL_INPUT_INIT, clears the 120-byte input display buffer, draws all menu options with DRAW_INPUT_OPTIONS, and renders the buffer to the screen.

249a ⟨*setup input* 249a⟩≡ (909)

```
      ORG     $5F60
  SETUP_INPUT:
      SUBROUTINE SETUP_INPUT
      ; Initialize a menu: store descriptor, call init, populate display.
      ;
      ; Inputs:
      ;   X  | high byte of menu descriptor pointer
      ;   Y  | low byte of menu descriptor pointer
      ;
      ; Behavior:
      ;   Stores X/Y into $C1/$C0 as the menu descriptor pointer, calls
      ;   the menu's init routine via CALL_INPUT_INIT, clears the 120-byte
      ;   input buffer, draws all options with DRAW_INPUT_OPTIONS, and
      ;   renders the buffer to screen.
      ; Clobbers: A, X, Y
      ; Set up input: store X/Y in $C1/$C0, call setup routines.
      STX     $C1
      STY     $C0
      JSR     CALL_INPUT_INIT
      JSR     CLEAR_INPUT_BUFFER
      JSR     DRAW_INPUT_OPTIONS
      JMP     RENDER_INPUT_BUFFER
```

Defines:
SETUP_INPUT, used in chunks 232 and 247.
Uses CALL_INPUT_INIT 257, CLEAR_INPUT_BUFFER 258, DRAW_INPUT_OPTIONS 250, and RENDER_INPUT_BUFFER 259.

DRAW_INPUT_OPTIONS iterates through all menu options in reverse order (from the count stored in byte 0 of the menu descriptor down to 1). For each option, it calls CALC_RECORD_PTR to locate the option's data record, checks whether byte 0 of the record is non-zero (enabled), and if so copies the record's display data into the input buffer with COPY_RECORD_DATA.

249b ⟨*var input option count* 249b⟩≡ (909)

```
      ORG     $5AAA
  INPUT_OPTION_COUNT:
      DC.B    $8D                     ; initial value (runtime: number of input options to draw)
```

Defines:
INPUT_OPTION_COUNT, used in chunk 250.

250 ⟨*draw input options* 250⟩≡ (909)

```
      ORG     $5F70
  DRAW_INPUT_OPTIONS:
      SUBROUTINE DRAW_INPUT_OPTIONS
      ; Populate the input display buffer with all enabled menu options.
      ;
      ; Inputs:
      ;   $C0/$C1  | pointer to menu descriptor (byte 0 = option count)
      ;
      ; Behavior:
      ;   Iterates options in reverse order (count down to 1).  For each
      ;   option, calls CALC_RECORD_PTR, checks byte 0 of the record
      ;   (enable flag); if non-zero, copies the option text into
      ;   INPUT_BUFFER via COPY_RECORD_DATA.
      ;
      ; Modifies:
      ;   INPUT_OPTION_COUNT, INPUT_BUFFER
      ; Clobbers: A, X, Y
      LDY     #$00
      LDA     ($C0),Y
      STA     INPUT_OPTION_COUNT
  .loop:
      LDA     INPUT_OPTION_COUNT
      JSR     CALC_RECORD_PTR
      LDY     #$00
      LDA     ($C2),Y
      BEQ     .skip
      JSR     COPY_RECORD_DATA
  .skip:
      DEC     INPUT_OPTION_COUNT
      BNE     .loop
      RTS
```

Defines:
DRAW_INPUT_OPTIONS, used in chunk 249a.
Uses CALC_RECORD_PTR 251, COPY_RECORD_DATA 260, INPUT_BUFFER 611, and INPUT_OPTION_COUNT 249b.

CALC_RECORD_PTR translates a 1-based option index in A to a pointer to that option's 10-byte record. The formula is MENU_OPTION_PTR = MENU_DESC_PTR + A×10 − 6, which accounts for the 4-byte table header (so option 1 maps to the first record after the header). The resulting pointer in MENU_OPTION_PTR is used by SETUP_CURSOR, VALIDATE_INPUT, and other input routines to access the selected option's data.

251 ⟨*calc record ptr* 251⟩≡ (909)

```
      ORG     $5F8C
      SUBROUTINE CALC_RECORD_PTR
      ; Compute pointer to a menu option's 10-byte record.
      ;
      ; Inputs:
      ;   A       | 1-based option index
      ;   $C0/$C1 | pointer to menu descriptor base
      ;
      ; Outputs:
      ;   $C2/$C3 | pointer to the selected option's record
      ;             ($C0/$C1 + A*10 - 6)
      ; Clobbers: A
  ; $C2/$C3 = $C0/$C1 + A*10 - 6
  CALC_RECORD_PTR:
      ASL
      STA     $BA
      ASL
      ASL
      CLC
      ADC     $BA
      ADC     #$FA
      CLC
      ADC     $C0
      STA     $C2
      LDA     $C1
      ADC     #$00
      STA     $C3
      RTS
```

Defines:
CALC_RECORD_PTR, used in chunks 247, 250, and 255.

SETUP_CURSOR_EXT extends SETUP_CURSOR by first calling SET_INVERSE_VIDEO to enable inverse-video mode, then falling through into SETUP_CURSOR. This highlights the currently selected menu option.

252 ⟨*setup cursor ext* 252⟩≡ (909)

```
      ORG     $5FA2
      SUBROUTINE SETUP_CURSOR_EXT
      ; Enable inverse video, then fall through to SETUP_CURSOR.
      ;
      ; Behavior:
      ;   Calls SET_INVERSE_VIDEO to highlight the current selection,
      ;   then falls through into SETUP_CURSOR to position and print.
      ; Clobbers: A
  ; JSR SET_INVERSE_VIDEO then falls through to SETUP_CURSOR
  SETUP_CURSOR_EXT:
      JSR     SET_INVERSE_VIDEO
```

Defines:
SETUP_CURSOR_EXT, used in chunk 247.
Uses SETUP_CURSOR 253 and SET_INVERSE_VIDEO 624.

SETUP_CURSOR positions the cursor at the location specified by the current option record, sends HRCG control codes A/B via PRINT_CTRL_AB to enter character output mode, loads the option's display string pointer from bytes 6–7 of the record at MENU_OPTION_PTR, prints it with PRINT_STRING, and finishes with ctrl-N to restore normal video.

253 ⟨*setup cursor* 253⟩≡ (909)

```
      ORG     $5FA5
      SUBROUTINE SETUP_CURSOR
      ; Position cursor at option, print its string, restore normal video.
      ;
      ; Inputs:
      ;   $C2/$C3 | pointer to current option record
      ;
      ; Behavior:
      ;   Calls SET_CURSOR_POS to position from record byte 1, sends
      ;   HRCG ctrl-AB, loads the display string pointer from record
      ;   bytes 6-7, prints it, then sends ctrl-N to restore normal video.
      ;
      ; Modifies:
      ;   PRINT_STRING_ADDR, TEXT_COL, TEXT_ROW
      ; Clobbers: A, Y

  SETUP_CURSOR:
      JSR     SET_CURSOR_POS
      JSR     PRINT_CTRL_AB
      LDWIY    $C2,#$06,PRINT_STRING_ADDR
      JSR     PRINT_STRING
      JMP     PRINT_CTRL_N
```

Defines:
SETUP_CURSOR, used in chunks 247 and 252.
Uses LDWIY 17, PRINT_CTRL_AB 623a, PRINT_CTRL_N 623b, PRINT_STRING 622, PRINT_STRING_ADDR 576c, SET_CURSOR_POS 254, TEXT_COL 572a, and TEXT_ROW 572a.

SET_CURSOR_POS converts the linear position stored in byte 1 of the option record at MENU_OPTION_PTR into screen coordinates. Positions 0–39 map to row 21, 40–79 to row 22, and 80+ to row 23. The column within each row is the remainder after subtracting the row offset. The result is written to the Apple II cursor position bytes $24 (HTAB) and $25 (VTAB).

254 ⟨*set cursor pos* 254⟩≡ (909)

```
      ORG     $5FBC
      SUBROUTINE SET_CURSOR_POS
      ; Convert linear menu position to screen row and column.
      ;
      ; Inputs:
      ;   $C2/$C3 | pointer to option record; byte 1 = linear position
      ;
      ; Behavior:
      ;   Reads the linear position from record byte 1.  Maps 0-39 to
      ;   row 21, 40-79 to row 22, 80+ to row 23.  The remainder within
      ;   each row becomes the column.  Stores in TEXT_COL and TEXT_ROW.
      ;
      ; Modifies:
      ;   TEXT_COL, TEXT_ROW
      ; Clobbers: A, Y
  ; Maps linear position at ($C2)+1 to TEXT_ROW ($25) and TEXT_COL ($24)
  ; Positions 0-39 -> row 21, 40-79 -> row 22, 80+ -> row 23
  SET_CURSOR_POS:
      LDY     #$01
      LDA     ($C2),Y
      BAGE    #$28,.row2
      LDY     #$15 ; row 21
      BNE     .set_pos
  .row2:
      SEC
      SBC     #$28
      BAGE    #$28,.row3
      LDY     #$16 ; row 22
      BNE     .set_pos
  .row3:
      LDY     #$17 ; row 23
      SEC
      SBC     #$28
  .set_pos:
      STY     TEXT_ROW
      STA     TEXT_COL
      RTS
```

Defines:
SET_CURSOR_POS, used in chunk 253.
Uses TEXT_COL 572a and TEXT_ROW 572a.

VALIDATE_INPUT saves the current record pointer and repeatedly calls READ_INPUT_CHAR until a valid (enabled) option is found. For each candidate, it calls CALC_RECORD_PTR and checks byte 0 of the record: if non-zero, the option is valid and its index is returned in A. Zero or the current selection are returned directly (indicating "no change" or "same selection"). The saved MENU_OPTION_PTR pointer is restored before returning so the caller's record context is preserved.

INPUT_SCAN_OFFSET is a scratch byte used by VALIDATE_INPUT and READ_INPUT_CHAR as an index into the current input record. It saves the starting offset for the scan so READ_INPUT_CHAR can return the key code one position beyond it.

255 ⟨*validate input* 255⟩≡ (909)

```
    ORG     $5FDD
    SUBROUTINE VALIDATE_INPUT
    ; Scan for the next valid (enabled) menu option matching a key.
    ;
    ; Inputs:
    ;   A               | initial key code to validate
    ;   CURRENT_SELECTION | current selection index
    ;   $C2/$C3         | pointer to current option record
    ;
    ; Behavior:
    ;   Saves $C2/$C3 and the key code.  Loops calling READ_INPUT_CHAR
    ;   and CALC_RECORD_PTR until finding a record whose byte 0 (enable
    ;   flag) is non-zero.  Returns 0 for no match, CURRENT_SELECTION
    ;   for same selection, or the new index.  Restores $C2/$C3.
    ;
    ; Outputs:
    ;   A  | validated option index (0 = no match)
    ; Clobbers: A, Y
; Validates input: saves $C2/$C3, loops reading input until valid record found
VALIDATE_INPUT:
    STA     INPUT_SCAN_OFFSET
    PSHW    $C2 ; save $C2/$C3
.retry:
    JSR     READ_INPUT_CHAR
    BAEQ    #$00,.done
    BAEQ    CURRENT_SELECTION,.done
    PHA
    JSR     CALC_RECORD_PTR
    LDY     #$00
    LDA     ($C2),Y
    BNE     .found
    PLA
    JMP     .retry
.found:
    PLA
.done:
    TAY
    PULW    $C2 ; restore $C2/$C3
```

```
    TYA
    RTS
```

Defines:
VALIDATE_INPUT, used in chunk 247.
Uses CALC_RECORD_PTR 251, CURRENT_SELECTION 54a, INPUT_SCAN_OFFSET 570b, PSHW 9b, PULW 10a, and READ_INPUT_CHAR 256.

READ_INPUT_CHAR reads a character code from the option record by indexing into the record at MENU_OPTION_PTR using the offset stored at $5B24 plus 1. This returns the key code that would select the option one position after the stored offset, used by VALIDATE_INPUT to scan through valid inputs.

256 ⟨*read input char* 256⟩≡ (909)

```
    ORG     $600A
    SUBROUTINE READ_INPUT_CHAR
    ; Read the next key code from the option record.
    ;
    ; Inputs:
    ;   INPUT_SCAN_OFFSET | current scan position
    ;   $C2/$C3           | pointer to option record
    ;
    ; Outputs:
    ;   A  | key code at ($C2)[INPUT_SCAN_OFFSET + 1]
    ; Clobbers: A, Y
; Returns character at ($C2) + INPUT_SCAN_OFFSET + 1
READ_INPUT_CHAR:
    CLC
    LDA     INPUT_SCAN_OFFSET
    ADC     #$01
    TAY
    LDA     ($C2),Y
    RTS
```

Defines:
READ_INPUT_CHAR, used in chunk 255.
Uses INPUT_SCAN_OFFSET 570b.

CALL_INPUT_INIT reads the initialization routine address from bytes 2–3 of the menu descriptor at MENU_DESC_PTR and executes it via an indirect jump through INPUT_JMP_VEC. This allows each menu to run custom setup code—for example, enabling or disabling options based on the current game state—before the input loop begins.

257 ⟨*call input init* 257⟩≡ (909)

```
      ORG     $6014
  CALL_INPUT_INIT:
      SUBROUTINE CALL_INPUT_INIT
      ; Execute the menu's initialization routine.
      ;
      ; Inputs:
      ;   $C0/$C1 | pointer to menu descriptor; bytes 2-3 = init routine address
      ;
      ; Behavior:
      ;   Reads the init routine address from descriptor bytes 2-3 into
      ;   INPUT_JMP_VEC and jumps through it.
      ; Clobbers: A, Y
      LDWIY    $C0,#$02,INPUT_JMP_VEC
      JMP     (INPUT_JMP_VEC)
```

Defines:
CALL_INPUT_INIT, used in chunk 249a.
Uses INPUT_JMP_VEC 53b and LDWIY 17.

CLEAR_INPUT_BUFFER fills the 120-byte input display buffer (INPUT_BUFFER through INPUT_BUFFER+$77) with high-bit spaces ($A0). This provides a clean slate before menu options are drawn into the buffer.

258 ⟨*clear input buffer* 258⟩≡ (909)

```
      ORG     $6024
  CLEAR_INPUT_BUFFER:
      SUBROUTINE CLEAR_INPUT_BUFFER
      ; Fill the 120-byte input display buffer with spaces.
      ;
      ; Behavior:
      ;   Writes $A0 (high-bit space) to INPUT_BUFFER[0..$77].
      ;
      ; Modifies:
      ;   INPUT_BUFFER (120 bytes)
      ; Clobbers: A, Y
      LDY     #$77
      LDA     #$A0
  .loop:
      STA     INPUT_BUFFER,Y
      DEY
      BPL     .loop
      RTS
```

Defines:
CLEAR_INPUT_BUFFER, used in chunk 249a.
Uses INPUT_BUFFER 611.

RENDER_INPUT_BUFFER copies the 120-byte input display buffer to the screen. It positions the cursor at row $15 (21), column 0, issues a ctrl-A/B sequence via PRINT_CTRL_AB to configure HRCG display mode, then outputs each byte of the buffer through ROM_COUT1. The Y register serves as the loop index and is saved/restored around each COUT call (since COUT may clobber it).

259 ⟨*render input buffer* 259⟩≡ (909)

```
      ORG     $602F
  RENDER_INPUT_BUFFER:
      SUBROUTINE RENDER_INPUT_BUFFER
      ; Output the 120-byte input display buffer to the screen.
      ;
      ; Behavior:
      ;   Positions the cursor at row 21, column 0.  Sends HRCG ctrl-AB,
      ;   then outputs each byte of INPUT_BUFFER through ROM_COUT1.
      ;   Y is saved in $BA across each COUT call.
      ;
      ; Modifies:
      ;   TEXT_COL, TEXT_ROW
      ; Clobbers: A, X, Y
      STOW2    $1500,$24
      JSR     PRINT_CTRL_AB
      LDY     #$00
  .loop:
      LDA     INPUT_BUFFER,Y
      STY     $BA
      JSR     ROM_COUT1
      LDY     $BA
      INY
      BYNE    #$78,.loop
      RTS
```

Defines:
RENDER_INPUT_BUFFER, used in chunk 249a.
Uses BYNE 18, INPUT_BUFFER 611, PRINT_CTRL_AB 623a, ROM_COUT1 571a, STOW2 8b, TEXT_COL 572a, and TEXT_ROW 572a.

COPY_RECORD_DATA copies an option's display string into the input display buffer. It loads the string pointer from bytes 6–7 of the record at MENU_OPTION_PTR, reads the string length from byte 0 of the string, uses byte 1 of the record as a column offset, and copies the string's characters into INPUT_BUFFER at the computed position. The copy runs backwards (from last character to first) to avoid needing a separate length counter.

260 ⟨*copy record data* 260⟩≡ (909)

```
      ORG     $604C
      SUBROUTINE COPY_RECORD_DATA
      ; Copy an option's display string into the input buffer.
      ;
      ; Inputs:
      ;   $C2/$C3 | pointer to option record; bytes 6-7 = string pointer,
      ;             byte 1 = column offset in INPUT_BUFFER
      ;
      ; Behavior:
      ;   Loads the string pointer from record bytes 6-7, reads the
      ;   string length from byte 0 of the string, computes the buffer
      ;   destination from the column offset, and copies characters into
      ;   INPUT_BUFFER backwards (last to first).
      ;
      ; Modifies:
      ;   PRINT_STRING_ADDR, INPUT_BUFFER (partial)
      ; Clobbers: A, X, Y
  ; Copies data from pointer at ($C2)+6/7 into INPUT_BUFFER
  COPY_RECORD_DATA:
      LDWIY    $C2,#$06,PRINT_STRING_ADDR
      LDY     #$01
      LDA     ($C2),Y
      STA     $BA
      DEY
      LDA     (PRINT_STRING_ADDR),Y
      TAY
      DEC     $BA
      CLC
      ADC     $BA
      TAX
  .copy_loop:
      LDA     (PRINT_STRING_ADDR),Y
      STA     INPUT_BUFFER,X
      DEX
      DEY
      BNE     .copy_loop
      RTS
```

Defines:
COPY_RECORD_DATA, used in chunk 250.
Uses INPUT_BUFFER 611, LDWIY 17, and PRINT_STRING_ADDR 576c.

GAME_INPUT_TABLE is the 74-byte data block at $607D that defines the player's main action menu. It has a 4-byte header (option count, max movement range, and a pointer to CONFIGURE_INPUT as the initialization routine) followed by seven 10-byte option records. Each record contains an enable flag (byte 0, patched at runtime by CONFIGURE_INPUT), a column offset (byte 1), a key code (byte 2), a display string pointer (bytes 6–7), and an exit handler address (bytes 8–9). The seven options correspond to movement, two combat choices, flee, and three additional actions (one of which is rest).

261 ⟨*game input table* 261⟩≡ (909)

```
      ORG     $607D
  ; Game input option table (7 records x 10 bytes + 4-byte header).
  ; Record format: [enable, col_offset, key, ?, ?, ?, str_ptr_lo, str_ptr_hi, ?, ?]
  ; Byte 0 of records 1-6 is patched by CONFIGURE_INPUT based on combat state.
  ; Header: option count, max_range (patched), init_ptr lo/hi
  GAME_INPUT_TABLE:
  .header:
      HEX 07   ; 7 options
  .max_range = *
      HEX 03   ; max movement range (patched: UNDER_LEVEL*3+3)
      HEX D260 ; init function pointer ($60D2 = CONFIGURE_INPUT)
  .rec1:                  ; record 1 ($6081) - movement
  .rec1_enable = *
      HEX 01   ; enable (patched: MOVE_POINTS or 0)
      HEX 07040207
      HEX 05537D311A
  .rec2:                  ; record 2 ($608B) - combat option
  .rec2_enable = *
      HEX 01   ; enable (patched: ~UNDER_LEVEL)
      HEX 2F0103
      HEX 05063E7D0761
  .rec3:                  ; record 3 ($6095) - combat option
  .rec3_enable = *
      HEX 01   ; enable (patched: ~UNDER_LEVEL)
      HEX 570204
      HEX 0604787BB714
  .rec4:                  ; record 4 ($609F) - flee option
  .rec4_enable = *
      HEX 01   ; enable (patched: UNDER_LEVEL)
      HEX 570301
      HEX 03077F7BC61B
  .rec5:                  ; record 5 ($60A9)
      HEX 01140706
      HEX 0102457D8C61
  .rec6:                  ; record 6 ($60B3) - rest option
  .rec6_enable = *
      HEX 01   ; enable (patched: SAFE_TO_REST)
      HEX 3C0507
      HEX 0203737B3F14
  .rec7:                  ; record 7 ($60BD)
```

```
    HEX 01640605
    HEX 0401667B0161
```

Defines:
GAME_INPUT_TABLE, used in chunks 232 and 262–64.
Uses CONFIGURE_INPUT 264, MOVE_POINTS 230c, SAFE_TO_REST 570b, and UNDER_LEVEL 570b.

TEARDOWN_INPUT bridges the turn system and the input system: it calls GET_INPUT_TABLE_PTR to load the default game input table pointer, then jumps to INPUT_LOOP to present the player with the standard action menu. This is the entry point used by TURN_DISPATCH when the current character is under player control.

262 ⟨*teardown input* 262⟩≡ (909)

```
    ORG     $60C7
TEARDOWN_INPUT:
    SUBROUTINE TEARDOWN_INPUT
    ; Present the default game action menu to the player.
    ;
    ; Behavior:
    ;   Loads the GAME_INPUT_TABLE pointer via GET_INPUT_TABLE_PTR,
    ;   then jumps to INPUT_LOOP to run the standard action menu.
    ;   Entry point from TURN_DISPATCH for player-controlled turns.
    ; Clobbers: A, X, Y
    ; Teardown input and jump to $5F19.
    JSR     GET_INPUT_TABLE_PTR
    JMP     INPUT_LOOP
```

Defines:
TEARDOWN_INPUT, used in chunks 232, 237, and 265.
Uses GAME_INPUT_TABLE 261, GET_INPUT_TABLE_PTR 263, INPUT_LOOP 247, and TURN_DISPATCH 237.

GET_INPUT_TABLE_PTR simply returns the address of the game's main input table in X (high byte $60) and Y (low byte $7D). Callers pass these registers to SETUP_INPUT to initialize the standard action menu.

263 ⟨*get input table ptr* 263⟩≡ (909)

```
      ORG     $60CD
  GET_INPUT_TABLE_PTR:
      SUBROUTINE GET_INPUT_TABLE_PTR
      ; Return a pointer to GAME_INPUT_TABLE.
      ;
      ; Outputs:
      ;   X | high byte of GAME_INPUT_TABLE
      ;   Y | low byte of GAME_INPUT_TABLE
      ;
      ; Clobbers: X, Y

      ; Return the address of GAME_INPUT_TABLE in X (hi) and Y (lo).
      ;
      ; Outputs:
      ;   X  | high byte of GAME_INPUT_TABLE
      ;   Y  | low byte of GAME_INPUT_TABLE
      ; Return X=high byte, Y=low byte (pointer to GAME_INPUT_TABLE).
      LDX     #>GAME_INPUT_TABLE
      LDY     #<GAME_INPUT_TABLE
      RTS
```

Defines:
GET_INPUT_TABLE_PTR, used in chunks 232 and 262.
Uses GAME_INPUT_TABLE 261.

CONFIGURE_INPUT patches GAME_INPUT_TABLE to enable or disable menu options based on the current turn state.

Option patching. After calling CHECK_COMBAT_ELIGIBILITY, the enable bytes are set as follows:

- Records 2–3 (combat): enabled when UNDER_LEVEL is clear (character is in an encounter).
- Record 4 (flee): enabled when UNDER_LEVEL is set.
- Record 6 (rest): enabled when SAFE_TO_REST is set.
- Record 1 (movement): set to MOVE_POINTS, but cleared to 0 if TURN_ACTIVE is zero.
- Max movement range: UNDER_LEVEL + 3 (3 normally, 4 in encounters).

264 ⟨*configure input* 264⟩≡ (909)

```
    ORG     $60D2
CONFIGURE_INPUT:
    SUBROUTINE CONFIGURE_INPUT
    ; Patch GAME_INPUT_TABLE enable flags from the current turn state.
    ;
    ; Behavior:
    ;   Calls CHECK_COMBAT_ELIGIBILITY, then patches the enable bytes:
    ;   rec6 (rest) from SAFE_TO_REST, rec4 (flee) from UNDER_LEVEL,
    ;   rec2/3 (combat) from ~UNDER_LEVEL, rec1 (movement) from
    ;   MOVE_POINTS (cleared to 0 if TURN_ACTIVE is 0).  Sets max
    ;   movement range to UNDER_LEVEL + 3.
    ;
    ; Modifies:
    ;   GAME_INPUT_TABLE (enable flags and max range)
    ; Clobbers: A
    ; Configure input handler: patch game input table enable flags
    ; from UNDER_LEVEL/SAFE_TO_REST/MOVE_POINTS/TURN_ACTIVE.
    JSR     CHECK_COMBAT_ELIGIBILITY
    MOVB    SAFE_TO_REST,GAME_INPUT_TABLE+54 ; rec6 byte 0 (rest option)
    MOVB    UNDER_LEVEL,GAME_INPUT_TABLE+34 ; rec4 byte 0 (flee option)
    EOR     #$01
    STA     GAME_INPUT_TABLE+14 ; rec2 byte 0 (combat option)
    STA     GAME_INPUT_TABLE+24 ; rec3 byte 0 (combat option)
    EOR     #$01
    CLC
    ADC     #$03
    STA     GAME_INPUT_TABLE+1  ; max movement range
    MOVB    MOVE_POINTS,GAME_INPUT_TABLE+4 ; rec1 byte 0 (movement)
    LDA     TURN_ACTIVE
    BEQ     .clear
```

```
    RTS
.clear:
    STA     GAME_INPUT_TABLE+4  ; rec1 byte 0 (movement)
    RTS
```

Defines:
CONFIGURE_INPUT, used in chunks 232 and 261.
Uses CHECK_COMBAT_ELIGIBILITY 308, GAME_INPUT_TABLE 261, MOVB 8c, MOVE_POINTS 230c, SAFE_TO_REST 570b, TURN_ACTIVE 570b, and UNDER_LEVEL 570b.

7.0.1 Menu handler routines

The main game menu at GAME_INPUT_TABLE dispatches to sub-menus and actions throughout the $6101–$62E4 region. Each sub-menu has a descriptor table (same format as GAME_INPUT_TABLE: 4-byte header followed by 10-byte records) and a small code entry point that loads the table address into X/Y and jumps to INPUT_LOOP.

Table	Entry	Menu	Options
$607D	$60C7	Main actions	Move, Attack, Defend, Stop, Other, Rest, See Your Lot
$610F	$6145	Attack direction	Defend, Attack Left/Above/Right/Below
$614C	$618C	Other options	Move, Drop Gold/Armor, Retire, Add Someone, more...
$6193	$61F6	Game settings	Move, Game Speed, Sound, Danger, Save Game
$623F	$6261	Attack target	Attack [target], Someone Else, Cancel
$626C	$62A2	Drop gold	Ducats to Drop, Drop Gold, Keep All Gold
$62B9	—	Save/load	Begin New Game, Continue Game Number

"See Your Lot" ($6101) shows the character stats screen, then returns to the main menu. "Attack" ($6107) checks eligibility via CHECK_ATTACK_ELIGIBLE, then either enters the attack direction menu or exits.

265 ⟨*menu handlers* 265⟩≡ (909)

```
    ORG     $6101
MENU_SEE_YOUR_LOT:
    JSR     DISPLAY_CHAR_STATS ; show character stats
    JMP     TEARDOWN_INPUT     ; return to main menu

MENU_CHECK_ATTACK:
    JSR     CHECK_ATTACK_ELIGIBLE ; check for hostile at same position
    BCC     ENTER_ATTACK_MENU  ; no hostile here -> choose direction
    JMP     FIND_ATTACK_TARGET ; hostile here -> target directly
```

Defines:
MENU_CHECK_ATTACK, used in chunk 266.
MENU_SEE_YOUR_LOT, never used.
Uses CHECK_ATTACK_ELIGIBLE 334, DISPLAY_CHAR_STATS 556, ENTER_ATTACK_MENU 266, FIND_ATTACK_TARGET 330, and TEARDOWN_INPUT 262.

The attack direction menu at ATTACK_DIR_TABLE.

266 ⟨*attack direction table* 266⟩≡ (909)

```
      ORG     $610F
  ATTACK_DIR_TABLE:
      ; 5 options, max_range=1, init=$626B
      HEX     05 01 6B 62
      ; rec0: DEFEND
      HEX     01 39 03 05 02 04
      DC.W    $7B78              ; "DEFEND"
      DC.W    ENTER_COMBAT_STATE
      ; rec1: LEFT
      HEX     01 29 03 05 04 01
      DC.W    $7C6A              ; left arrow
      DC.W    ATTACK_LEFT
      ; rec2: ABOVE
      HEX     01 0C 05 01 02 04
      DC.W    $7C59              ; up arrow
      DC.W    ATTACK_UP
      ; rec3: RIGHT
      HEX     01 44 03 05 01 02
      DC.W    $7C76              ; right arrow
      DC.W    ATTACK_RIGHT
      ; rec4: BELOW
      HEX     01 5C 01 03 02 04
      DC.W    $7C83              ; down arrow
      DC.W    ATTACK_DOWN

      ; --- enter attack (target of BCC from MENU_CHECK_ATTACK) ---
  ENTER_ATTACK_MENU:
      LDX     #>ATTACK_DIR_TABLE ; X = high byte
      LDY     #<ATTACK_DIR_TABLE ; Y = low byte
      JMP     INPUT_LOOP
```

Defines:
ATTACK_DIR_TABLE, never used.
ENTER_ATTACK_MENU, used in chunk 265.

Uses ATTACK_DOWN 329, ATTACK_LEFT 329, ATTACK_RIGHT 329, ATTACK_UP 329, ENTER_COMBAT_STATE 327, INPUT_LOOP 247, and MENU_CHECK_ATTACK 265.

The "Other Options" sub-menu at OTHER_OPTIONS_TABLE. Record 5 ("Other Options") chains to the game settings menu via MENU_GAME_SETTINGS.

267a ⟨*other options table* 267a⟩≡ (909)

```
      ORG     $614C
  OTHER_OPTIONS_TABLE:
      ; 6 options, max_range=1, init=$626B
      HEX     06 01 6B 62
      ; rec0: MOVE       rec1: DROP GOLD   rec2: DROP ARMOR
      ; rec3: OTHER OPT  rec4: RETIRE      rec5: ADD SOMEONE
      HEX     01 05 03 02 06 04 53 7D 31 1A
      HEX     01 2D 01 03 04 05 FA 7A 17 6F
      HEX     01 55 02 01 05 06 DC 7A E9 6E
      HEX     01 15 06 05 01 02 45 7D F1 61
      HEX     01 3D 04 06 02 03 E7 7A E7 09
      ; rec5: ADD SOMEONE
      HEX     01 65 05 04 03 01
      DC.W    $7AEE              ; "ADD SOMEONE"
      DC.W    CHAR_SELECT_INIT
```

Defines:
OTHER_OPTIONS_TABLE, used in chunk 267b.
Uses CHAR_SELECT_INIT 283.

267b ⟨*other options entry* 267b⟩≡ (909)

```
      ORG     $618C
  MENU_OTHER_OPTIONS:
      LDX     #>OTHER_OPTIONS_TABLE
      LDY     #<OTHER_OPTIONS_TABLE
      JMP     INPUT_LOOP
```

Defines:
MENU_OTHER_OPTIONS, used in chunk 436.
Uses INPUT_LOOP 247 and OTHER_OPTIONS_TABLE 267a.

RETIRE_CHARACTER is the handler for the "RETIRE" option in the Other Options menu (rec 4 of OTHER_OPTIONS_TABLE). It clears ACTIVE_CHAR, reorders the character list, and re-enters SCENE_LOOP with scene SCENE_RETIRE ($23).

268 ⟨*retire character* 268⟩≡ (909)

```
      ORG     $09E7
  RETIRE_CHARACTER:
      ; Retire the active character from the party.
      ;
      ; Behavior:
      ;   Clears ACTIVE_CHAR, calls REORDER_CURRENT_CHAR to update
      ;   the character list, then enters SCENE_LOOP with SCENE_RETIRE.
      ;
      ; Modifies: ACTIVE_CHAR
      ; Clobbers: A, X, Y

      LDA     #$00
      STA     ACTIVE_CHAR                ; clear active character
      JSR     REORDER_CURRENT_CHAR
      LDA     #SCENE_RETIRE              ; retirement scene
      JMP     SCENE_LOOP
```

Defines:
RETIRE_CHARACTER, never used.
Uses ACTIVE_CHAR 492c, REORDER_CURRENT_CHAR 489a, SCENE_LOOP 527, and SCENE_RETIRE 685.

The game settings menu at GAME_SETTINGS_TABLE. This includes game speed adjustment (records 1–3 with arrow keys), sound toggle, danger level (records 5–7), and save game. The "game settings" entry at MENU_GAME_SETTINGS is an indirect jump through $A963, a resident vector. MENU_GAME_SETTINGS_ENTER is the direct entry that loads the settings table and enters INPUT_LOOP. MENU_SETTINGS_INIT initializes the settings display: it calls the splash routine at SETTINGS_DISPLAY_INIT (SETTINGS_DISPLAY_INIT), then formats the current game speed (ANIM_INDEX) as a digit into the "GAME SPEED" label string at $7D66.

269 ⟨*game settings table* 269⟩≡ (909)

```
      ORG     $6193
  GAME_SETTINGS_TABLE:
      ; 9 options, max_range=1, init=$61FD
      ; header: 9 options, max_range=1, init=MENU_SETTINGS_INIT
      DC.B    $09,$01
      DC.W    MENU_SETTINGS_INIT
      ; rec0: MOVE             rec1: GAME SPEED      rec2: # (speed up)
      ; rec3: @ (speed down)   rec4: SOUND IS ON      rec5: DANGER :
      ; rec6: # (danger up)    rec7: @ (danger down)  rec8: SAVE GAME
      HEX     01 03 05 09 08 03 53 7D 31 1A
      HEX     01 13 00 00 00 00 58 7D 6B 62
      HEX     01 23 07 07 01 04 60 7B 09 62
      HEX     01 25 08 08 03 09 62 7B 20 62
      HEX     01 53 09 01 09 07 04 7B 84 09
      HEX     01 66 00 00 00 00 1B 7B 6B 62
      HEX     01 73 03 03 05 08 60 7B 2D 62
      HEX     01 75 04 04 07 01 62 7B 36 62
      ; rec8: SAVE GAME
      HEX     01 2B 01 05 04 05
      DC.W    $7B11              ; "SAVE GAME"
      DC.W    ENTER_SAVE_GAME_SLOT
```

Defines:
GAME_SETTINGS_TABLE, used in chunk 270a.
Uses ENTER_SAVE_GAME_SLOT 279 and MENU_SETTINGS_INIT 270a.

270a ⟨*game settings entry* 270a⟩≡ (909)

```
      ORG     $61F1
  MENU_GAME_SETTINGS:
      SUBROUTINE MENU_GAME_SETTINGS
      ; Enter the game settings menu.
      ;
      ; Behavior:
      ;   Patches max_range to 1 in GAME_SETTINGS_TABLE, then enters
      ;   INPUT_LOOP with the settings table pointer.
      ;
      ; Clobbers: A, X, Y

      LDA     #$01
      STA     GAME_SETTINGS_TABLE+1 ; patch max_range = 1
  MENU_GAME_SETTINGS_ENTER:
      LDX     #>GAME_SETTINGS_TABLE
      LDY     #<GAME_SETTINGS_TABLE
      JMP     INPUT_LOOP

  MENU_SETTINGS_INIT:
      JSR SETTINGS_DISPLAY_INIT              ; splash screen setup
      LDA     ANIM_INDEX          ; game speed value
      ORA     #$B0                ; convert to ASCII digit
      STA     $7D66               ; patch into "GAME SPEED : X" string
      RTS
```

Defines:
MENU_GAME_SETTINGS, used in chunk 686.
MENU_SETTINGS_INIT, used in chunk 269.
Uses ANIM_INDEX 583a, GAME_SETTINGS_TABLE 269, INPUT_LOOP 247, and SETTINGS_DISPLAY_INIT 686.

The settings adjustment handlers increment or decrement the game speed (ANIM_INDEX) and danger threshold, then re-enter the settings menu. MAX_GAME_SPEED is the maximum game speed.

Maximum game speed index, used by SETTINGS_HANDLERS.

270b ⟨*max game speed* 270b⟩≡ (909)

```
      ORG     $7ABC
  MAX_GAME_SPEED:
      DC.B    $05                     ; maximum game speed index
```

Defines:
MAX_GAME_SPEED, used in chunk 271.

271 ⟨*settings handlers* 271⟩≡ (909)

```
    ORG     $6209
SETTINGS_SPEED_UP:
    SUBROUTINE SETTINGS_SPEED_UP
    ; Increase game speed by one step (clamped to max).
    ;
    ; Behavior:
    ;   Increments ANIM_INDEX (game speed), clamps to MAX_GAME_SPEED,
    ;   then re-enters the settings menu. SETTINGS_SPEED_DOWN decrements
    ;   (clamped to 1).
    ;
    ; Modifies: ANIM_INDEX
    ; Clobbers: A, X

    JSR     STORE_SELECTION_TO_RECORD
    INC     ANIM_INDEX          ; speed++
    LDA     ANIM_INDEX
    CMP     MAX_GAME_SPEED      ; at max?
    BCC     .done
    LDA     MAX_GAME_SPEED      ; clamp to max
    STA     ANIM_INDEX
.done:
    JMP     MENU_GAME_SETTINGS_ENTER ; re-enter settings menu

SETTINGS_SPEED_DOWN:
    JSR     STORE_SELECTION_TO_RECORD
    DEC     ANIM_INDEX          ; speed--
    BNE     .done               ; nonzero -> ok
    INC     ANIM_INDEX          ; clamp to 1
    BNE     .done               ; (always taken)

SETTINGS_DANGER_UP:
    JSR     STORE_SELECTION_TO_RECORD
    JSR INC_THRESHOLD_WRAP              ; increment danger threshold
    JMP     MENU_GAME_SETTINGS_ENTER

SETTINGS_DANGER_DOWN:
    JSR     STORE_SELECTION_TO_RECORD
    JSR DEC_THRESHOLD_WRAP              ; decrement danger threshold
    JMP     MENU_GAME_SETTINGS_ENTER
```

Defines:
SETTINGS_DANGER_DOWN, never used.
SETTINGS_DANGER_UP, never used.
SETTINGS_SPEED_DOWN, never used.
SETTINGS_SPEED_UP, never used.

Uses ANIM_INDEX 583a, DEC_THRESHOLD_WRAP 686, INC_THRESHOLD_WRAP 686, MAX_GAME_SPEED 270b, and STORE_SELECTION_TO_RECORD 54b.

The attack target menu at ATTACK_TARGET_TABLE.

272 ⟨*attack target table* 272⟩≡ (909)

```
      ORG     $623F
  ATTACK_TARGET_TABLE:
      ; 3 options, max_range=2, init=MENU_ATTACK_INIT
      HEX     03 02
      DC.W    MENU_ATTACK_INIT
      ; rec0: ATTACK [target]
      HEX     01 06 03 02 00 00
      DC.W    $7C94              ; "ATTACK" + name buffer
      DC.W    RESOLVE_ATTACK
      ; rec1: SOMEONE ELSE
      HEX     01 2E 01 03 00 00
      DC.W    $7CB0              ; "SOMEONE ELSE"
      DC.W    SELECT_SOMEONE_ELSE
      ; rec2: CANCEL
      HEX     01 56 02 01 00 00
      DC.W    $7CC4              ; "CANCEL"
      DC.W    $626B              ; no-op (RTS)

  MENU_ATTACK_TARGET:
      LDX     #>ATTACK_TARGET_TABLE
      LDY     #<ATTACK_TARGET_TABLE
      JMP     INPUT_LOOP

  MENU_ATTACK_INIT:
      JSR     INIT_ATTACK_DISPLAY ; copy target name to display buffer
      RTS
```

Defines:
ATTACK_TARGET_TABLE, never used.
MENU_ATTACK_INIT, never used.
MENU_ATTACK_TARGET, used in chunks 330 and 334.
Uses INIT_ATTACK_DISPLAY 332, INPUT_LOOP 247, RESOLVE_ATTACK 338, and SELECT_SOMEONE_ELSE 334.

The drop gold menu at DROP_GOLD_TABLE.

273 ⟨*drop gold table* 273⟩≡ (909)

```
        ORG     $626C
    DROP_GOLD_TABLE:
        ; 5 options, max_range=2, init=MENU_DROP_GOLD_INIT
        HEX     05 02
        DC.W    MENU_DROP_GOLD_INIT
        ; rec0: DUCATS   rec1: # (inc)  rec2: @ (dec)
        ; rec3: DROP     rec4: KEEP ALL
        ; rec0: DUCATS (display only)
        HEX     01 2A 00 00 00 00
        DC.W    $7CD9               ; "DUCATS"
        DC.W    MENU_DROP_GOLD      ; no-op (re-enter menu)
        ; rec1: # (increase drop amount)
        HEX     01 43 04 04 04 03
        DC.W    $7B60               ; "#"
        DC.W    MENU_DROP_DEC
        ; rec2: @ (decrease drop amount)
        HEX     01 45 04 04 02 04
        DC.W    $7B62               ; "@"
        DC.W    MENU_DROP_INC
        ; rec3: DROP GOLD
        HEX     01 52 02 03 03 05
        DC.W    $7AFA               ; "DROP GOLD"
        DC.W    DROP_GOLD_EXEC
        ; rec4: KEEP ALL GOLD
        HEX     01 69 02 03 04 04
        DC.W    $7CF1               ; "KEEP ALL GOLD"
        DC.W    MOVE_LOOP

    MENU_DROP_GOLD:
        LDX     #>DROP_GOLD_TABLE
        LDY     #<DROP_GOLD_TABLE
        JMP     INPUT_LOOP

    MENU_DROP_GOLD_INIT:
        JSR     INIT_DROP_DISPLAY   ; convert drop amount to decimal
        RTS

    MENU_DROP_INC:
        JSR     STORE_SELECTION_TO_RECORD
        JMP     ADJ_DROP_INC        ; binary search: subtract step

    MENU_DROP_DEC:
        JSR     STORE_SELECTION_TO_RECORD
        JMP     ADJ_DROP_DEC        ; binary search: add step
```

Defines:
DROP_GOLD_TABLE, never used.
MENU_DROP_DEC, never used.
MENU_DROP_GOLD, used in chunks 436 and 438.

MENU_DROP_GOLD_INIT, never used.
MENU_DROP_INC, never used.
Uses ADJ_DROP_DEC 438, ADJ_DROP_INC 438, DROP_GOLD_EXEC 440, INIT_DROP_DISPLAY 437, INPUT_LOOP 247, MOVE_LOOP 232, and STORE_SELECTION_TO_RECORD 54b.

The save/load game menu at WORLD_INIT_DATA.

274 ⟨*save load table* 274⟩≡ (909)

```
    ORG     $62B9
WORLD_INIT_DATA:
SAVE_LOAD_TABLE:
    ; 4 options, max_range=1, init=SAVE_LOAD_INIT
    DC.B    $04,$01
    DC.W    SAVE_LOAD_INIT
    ; rec0: NEW GAME
    HEX     01 05 02 02 00 00
    DC.W    $7BF3               ; "BEGIN NEW GAME"
    DC.W    $626B               ; no-op init
    ; rec1: CONTINUE GAME NUMBER NN
    HEX     01 55 01 01 04 03
    DC.W    $7BDB               ; "CONTINUE GAME NUMBER NN"
    DC.W    CONTINUE_GAME       ; continue game handler
    ; rec2: # (increment slot)
    HEX     01 6F 01 01 02 04
    DC.W    $7B60               ; "#"
    DC.W    SLOT_INC_ENTER
    ; rec3: @ (decrement slot)
    HEX     01 71 01 01 03 02
    DC.W    $7B62               ; "@"
    DC.W    SLOT_DEC_ENTER
```

Defines:
SAVE_LOAD_TABLE, never used.
WORLD_INIT_DATA, used in chunk 588a.
Uses CONTINUE_GAME 289, SAVE_LOAD_INIT 275c, SLOT_DEC_ENTER 277, and SLOT_INC_ENTER 277.

7.0.2 Save/load slot routines

SAVE_LOAD_INIT is the initialization callback for the save/load menu (SAVE_LOAD_TABLE). It calls FORMAT_SLOT_NUMBER to display the current slot number, then returns.

275a ⟨*defines* 25b⟩+≡ (909) ◁25b 331▷

```
S_CONTINUE_TENS     EQU     $7BF1   ; tens digit in "CONTINUE GAME NUMBER NN"
S_CONTINUE_ONES     EQU     $7BF2   ; ones digit in "CONTINUE GAME NUMBER NN"
S_SAVE_SLOT_TENS    EQU     $7D81   ; tens digit in "SAVE GAME IN SLOT NUMBER 00"
S_SAVE_SLOT_ONES    EQU     $7D82   ; ones digit in "SAVE GAME IN SLOT NUMBER 00"
```

Defines:
S_CONTINUE_ONES, used in chunk 277.
S_CONTINUE_TENS, used in chunk 277.
S_SAVE_SLOT_ONES, used in chunk 277.
S_SAVE_SLOT_TENS, used in chunk 277.

Current save/load slot number, used by SAVE_LOAD_INIT.

275b ⟨*slot number* 275b⟩≡ (909)

```
    ORG     $7AC0
SLOT_NUMBER:
    DC.B    $01                     ; current save/load slot number
```

Defines:
SLOT_NUMBER, used in chunks 277, 289, 293, and 305.

275c ⟨*save load init* 275c⟩≡ (909)

```
    ORG     $62EC
SAVE_LOAD_INIT:
    JSR     FORMAT_SLOT_NUMBER ; format current slot number into strings
    RTS
```

Defines:
SAVE_LOAD_INIT, used in chunk 274.
Uses FORMAT_SLOT_NUMBER 277.

SLOT_INC increments SLOT_NUMBER. If the result exceeds MAX_SLOT_NUMBER, it clamps back. Either way, it patches the menu record via STORE_SELECTION_TO_RECORD and reformats the display.

SLOT_DEC decrements SLOT_NUMBER. If it reaches zero, it wraps back to 1.

SLOT_UPDATE patches the current menu record via STORE_SELECTION_TO_RECORD, then jumps to FORMAT_SLOT_NUMBER to redisplay the number.

SLOT_INC_ENTER and SLOT_DEC_ENTER are the menu handlers for the # and @ arrow keys in the save/load menu. They adjust the slot number and re-enter INIT_WORLD (which presents the save/load menu).

FORMAT_SLOT_NUMBER converts SLOT_NUMBER (1–19) into two high-ASCII decimal digits and writes them into both the "CONTINUE GAME NUMBER" and "SAVE GAME IN SLOT NUMBER" display strings. Values below 10 get a leading space; values 10–19 get a "1" tens digit.

Maximum slot number, used by SLOT_NUMBER_ROUTINES.

276 ⟨*max slot number* 276⟩≡ (909)

```
    ORG     $7AC6
MAX_SLOT_NUMBER:
    DC.B    $11                     ; maximum slot number (17)
```

Defines:
MAX_SLOT_NUMBER, used in chunk 277.

277 ⟨*slot number routines* 277⟩≡ (909)

```
    ORG     $62F0
SLOT_INC:
    INC     SLOT_NUMBER
    LDA     SLOT_NUMBER
    CMP     MAX_SLOT_NUMBER
    BCC     SLOT_UPDATE         ; still below max -> update display
    DEC     SLOT_NUMBER         ; undo: clamp to max
    JMP     SLOT_UPDATE

SLOT_DEC:
    DEC     SLOT_NUMBER
    BNE     SLOT_UPDATE         ; nonzero -> update display
    INC     SLOT_NUMBER         ; undo: wrap 0 -> 1

SLOT_UPDATE:
    JSR     STORE_SELECTION_TO_RECORD
    JMP     FORMAT_SLOT_NUMBER

SLOT_INC_ENTER:
    JSR     SLOT_INC
    JMP     INIT_WORLD

SLOT_DEC_ENTER:
    JSR     SLOT_DEC
    JMP     INIT_WORLD

FORMAT_SLOT_NUMBER:
    LDA     #$A0                ; space (high ASCII)
    STA     S_CONTINUE_TENS     ; default tens digit to space
    STA     S_SAVE_SLOT_TENS
    LDA     SLOT_NUMBER
    CMP     #$0A                ; >= 10?
    BCC     .ones               ; no -> skip tens
    SEC
    SBC     #$0A                ; remainder
    PHA
    LDA     #$B1                ; "1" (high ASCII)
    STA     S_CONTINUE_TENS
    STA     S_SAVE_SLOT_TENS
    PLA
.ones:
    ORA     #$B0                ; convert 0-9 to high ASCII digit
    STA     S_CONTINUE_ONES
    STA     S_SAVE_SLOT_ONES
    RTS
```

Defines:
FORMAT_SLOT_NUMBER, used in chunks 275c and 278.
SLOT_DEC, used in chunk 279.
SLOT_DEC_ENTER, used in chunk 274.

SLOT_INC, used in chunk 279.
SLOT_INC_ENTER, used in chunk 274.
SLOT_UPDATE, never used.
Uses MAX_SLOT_NUMBER 276, SLOT_NUMBER 275b, STORE_SELECTION_TO_RECORD 54b, S_CONTINUE_ONES 275a, S_CONTINUE_TENS 275a, S_SAVE_SLOT_ONES 275a, and S_SAVE_SLOT_TENS 275a.

7.0.3 Save game slot menu

The save game slot menu is entered from the "SAVE GAME" option in the game settings menu. It presents four options:

- MOVE — return to the game (MOVE_LOOP)
- SAVE GAME IN SLOT NUMBER NN — save to the current slot (SAVE_GAME)
- # — increment slot number (SLOT_INC_REENTER)
- @ — decrement slot number (SLOT_DEC_REENTER)

278 ⟨*save game slot table* 278⟩≡ (909)

```
    ORG     $6340
SAVE_GAME_SLOT_TABLE:
    ; 4 options, max_range=2, init=FORMAT_SLOT_NUMBER
    DC.B    $04,$02
    DC.W    FORMAT_SLOT_NUMBER
    ; rec0: MOVE
    HEX     01 04 02 02 02 02
    DC.W    $7D53               ; "MOVE"
    DC.W    MOVE_LOOP
    ; rec1: SAVE GAME IN SLOT NUMBER NN
    HEX     01 2C 01 01 04 03
    DC.W    $7D67               ; "SAVE GAME IN SLOT NUMBER 00"
    DC.W    SAVE_GAME           ; save game handler
    ; rec2: # (increment)
    HEX     01 49 01 01 02 04
    DC.W    $7B60               ; "#"
    DC.W    SLOT_INC_REENTER
    ; rec3: @ (decrement)
    HEX     01 4B 01 01 03 02
    DC.W    $7B62               ; "@"
    DC.W    SLOT_DEC_REENTER
```

Defines:
SAVE_GAME_SLOT_TABLE, used in chunk 279.
Uses FORMAT_SLOT_NUMBER 277, MOVE_LOOP 232, SAVE_GAME 293, SLOT_DEC_REENTER 279, and SLOT_INC_REENTER 279.

ENTER_SAVE_GAME_SLOT is the entry point for the save game slot menu, called from the "SAVE GAME" handler in the game settings menu. SLOT_DEC_REENTER and SLOT_INC_REENTER adjust the slot number and re-enter the save game slot menu.

279 ⟨*save game slot entry* 279⟩≡ (909)

```
    ORG     $636C
ENTER_SAVE_GAME_SLOT:
    LDX     #>SAVE_GAME_SLOT_TABLE
    LDY     #<SAVE_GAME_SLOT_TABLE
    JMP     INPUT_LOOP

SLOT_DEC_REENTER:
    JSR     SLOT_DEC
    JMP     ENTER_SAVE_GAME_SLOT

SLOT_INC_REENTER:
    JSR     SLOT_INC
    JMP     ENTER_SAVE_GAME_SLOT
```

Defines:
ENTER_SAVE_GAME_SLOT, used in chunk 269.
SLOT_DEC_REENTER, used in chunk 278.
SLOT_INC_REENTER, used in chunk 278.
Uses INPUT_LOOP 247, SAVE_GAME_SLOT_TABLE 278, SLOT_DEC 277, and SLOT_INC 277.

7.0.4 Scenario disk menu

The scenario disk menu presents three options for managing save game disks. It is invoked from event script 59 (ROY G. BIV):

- USE PREVIOUSLY FORMATTED SCENARIO DISK (USE_SCENARIO_DISK)
- FORMAT AND USE NEW SCENARIO DISK (FORMAT_SCENARIO_DISK)
- CANCEL REQUEST (CANCEL_SCENARIO)

280a ⟨*scenario disk table* 280a⟩≡ (909)

```
    ORG     $637F
SCENARIO_DISK_TABLE:
    ; 3 options, max_range=1, init=$626B (RTS = no-op)
    DC.B    $03,$01
    DC.W    $626B
    ; rec0: USE PREVIOUSLY FORMATTED SCENARIO DISK
    HEX     01 00 03 02 00 00
    DC.W    $7C02               ; "USE PREVIOUSLY FORMATTED SCENARIO DISK"
    DC.W    USE_SCENARIO_DISK   ; use existing disk handler
    ; rec1: FORMAT AND USE NEW SCENARIO DISK
    HEX     01 28 01 03 00 00
    DC.W    $7C29               ; "FORMAT AND USE NEW SCENARIO DISK"
    DC.W    FORMAT_SCENARIO_DISK ; format new disk handler
    ; rec2: CANCEL REQUEST
    HEX     01 50 02 01 00 00
    DC.W    $7C4A               ; "CANCEL REQUEST"
    DC.W    CANCEL_SCENARIO     ; cancel handler
```

Defines:
SCENARIO_DISK_TABLE, used in chunk 280b.
Uses CANCEL_SCENARIO 295, FORMAT_SCENARIO_DISK 295, and USE_SCENARIO_DISK 295.

ENTER_SCENARIO_DISK is the entry point called from event scripts.

280b ⟨*scenario disk entry* 280b⟩≡ (909)

```
    ORG     $63A1
ENTER_SCENARIO_DISK:
    LDX     #>SCENARIO_DISK_TABLE
    LDY     #<SCENARIO_DISK_TABLE
    JMP     INPUT_LOOP
```

Defines:
ENTER_SCENARIO_DISK, used in chunks 293 and 295.
Uses INPUT_LOOP 247 and SCENARIO_DISK_TABLE 280a.

7.0.5 Character selection menu

The character selection menu is used when the player is asked to choose a character—for example, when an NPC offers to join the party ("ADD SOMEONE") or when the game needs a specific character selected. Six options are available:

- MOVE — return to the game (MOVE_LOOP)
- ADD [name] — add the currently displayed character (INIT_CHAR_SELECT)
- # — browse to the next PC (CHAR_SELECT_NEXT)
- @ — browse to the previous PC (CHAR_SELECT_PREV)
- TO THIS ROOM — transport character here (CHAR_SELECT_HERE)
- TO ALI BABA'S HOME — send character home (CHAR_SELECT_HOME)

Records 4 and 5 are dynamically enabled by CHAR_SELECT_INIT based on whether CURRENT_PLAYER equals 9. The table's max_range is also patched at runtime.

281 ⟨*char select table* 281⟩≡ (909)

```
    ORG     $63A8
CHAR_SELECT_TABLE:
    ; 6 options, max_range=2, init=$626B (patched at runtime)
    DC.B    $06,$02
    DC.W    $626B
    ; rec0: MOVE
    HEX     01 05 06 02 06 02
    DC.W    $7D53               ; "MOVE"
    DC.W    MOVE_LOOP
    ; rec1: ADD [character name]
    HEX     01 2D 01 05 01 03
    DC.W    $7B47               ; "ADD" + CHAR_SELECT_NAME buffer
    DC.W    INIT_CHAR_SELECT    ; add character handler
    ; rec2: # (next PC)
    HEX     01 49 01 05 02 04
    DC.W    $7B60               ; "#"
    DC.W    CHAR_SELECT_NEXT
    ; rec3: @ (previous PC)
    HEX     01 4B 01 05 03 05
    DC.W    $7B62               ; "@"
    DC.W    CHAR_SELECT_PREV
    ; rec4: TO THIS ROOM (dynamically enabled)
    HEX     01 55 02 06 04 06
    DC.W    $7B27               ; "TO THIS ROOM"
    DC.W    CHAR_SELECT_HERE
```

```
; rec5: TO ALI BABA'S HOME (dynamically enabled)
HEX     00 55 05 01 05 01
DC.W    $7B34               ; "TO ALI BABA'S HOME"
DC.W    CHAR_SELECT_HOME
```

Defines:
CHAR_SELECT_TABLE, used in chunk 283.
Uses CHAR_SELECT_HERE 283, CHAR_SELECT_HOME 283, CHAR_SELECT_NAME 610a, CHAR_SELECT_NEXT 283, CHAR_SELECT_PREV 283, INIT_CHAR_SELECT 617, and MOVE_LOOP 232.

ENTER_CHAR_SELECT loads the character selection table and enters INPUT_LOOP. This is the jump target from INIT_CHAR_SELECT at $0B4F.

CHAR_SELECT_INIT is called when the "Add Someone" option is selected from the "Other Options" menu. It calls FIND_FIRST_PC to locate a player character. If none found (carry set), it displays message $35 ("YOU SAVED ME FROM MY IMPRISONMENT,") and returns to MOVE_LOOP.

If a PC is found, it checks CURRENT_PLAYER. If equal to 9, records 4 and 5 (TO THIS ROOM / TO ALI BABA'S HOME) are both enabled, and ACTIVE_CHAR is set to 1. Otherwise, record 4 is enabled (with CURRENT_PLAYER stored in ACTIVE_CHAR), record 5 is disabled, and the table's max_range is set to 2. Control then jumps to ENTER_CHAR_SELECT.

CHAR_SELECT_NEXT and CHAR_SELECT_PREV browse through player characters. They patch the current menu record via STORE_SELECTION_TO_RECORD, advance or retreat in the PC list, then re-enter the character selection menu.

CHAR_SELECT_HERE and CHAR_SELECT_HOME are the handlers for the "TO THIS ROOM" and "TO ALI BABA'S HOME" options. They patch the table's enable flags based on CURRENT_PLAYER and re-enter the menu.

283 ⟨*char select handlers* 283⟩≡ (909)

```
    ORG     $63E8
ENTER_CHAR_SELECT:
    LDX     #>CHAR_SELECT_TABLE
    LDY     #<CHAR_SELECT_TABLE
    JMP     INPUT_LOOP

CHAR_SELECT_INIT:
    SUBROUTINE CHAR_SELECT_INIT
    ; Initialize the character selection menu.
    ;
    ; Behavior:
    ;   Finds the first player character via FIND_FIRST_PC. If none
    ;   found, displays a message and returns to the move loop.
    ;   Otherwise configures the menu table based on whether
    ;   CURRENT_PLAYER is Ali Baba and enters INPUT_LOOP.
    ;
    ; Clobbers: A, X, Y

    JSR     FIND_FIRST_PC       ; find first player character
    BCC     CHAR_SELECT_HOME    ; carry clear -> found, enter init path
    JSR     SET_CURSOR_ROW21    ; no PCs: position cursor
    LDA     #MSG_SAVED_ME               ; message $35: "YOU SAVED ME..."
    JSR     DISPLAY_MESSAGE
    JMP     MOVE_LOOP           ; back to main loop

CHAR_SELECT_NEXT:
    JSR     STORE_SELECTION_TO_RECORD
```

```
    JSR     FIND_NEXT_PC_WRAP  ; advance to next PC
    JMP     ENTER_CHAR_SELECT

CHAR_SELECT_PREV:
    JSR     STORE_SELECTION_TO_RECORD
    JSR     FIND_PREV_PC       ; retreat to previous PC
    JMP     ENTER_CHAR_SELECT

; "TO THIS ROOM" handler: set ACTIVE_CHAR=1, disable rec4, -> .patch
CHAR_SELECT_HERE:
    LDA     #$01
    STA     ACTIVE_CHAR
    LDX     #$00
    BEQ     .patch             ; always taken

; "TO ALI BABA'S HOME" handler AND .found target from CHAR_SELECT_INIT.
; If CURRENT_PLAYER==9 -> CHAR_SELECT_HERE (enable rec5, disable rec4).
; Otherwise store CURRENT_PLAYER in ACTIVE_CHAR, disable rec5, enable rec4.
CHAR_SELECT_HOME:
    LDA     CURRENT_PLAYER
    CMP     #$09
    BEQ     CHAR_SELECT_HERE
    STA     ACTIVE_CHAR
    LDA     #$00               ; disable rec5
    LDX     #$01               ; enable rec4

.patch:
    STA     CHAR_SELECT_TABLE+54 ; rec5 byte 0 (enable/disable)
    STX     CHAR_SELECT_TABLE+44 ; rec4 byte 0 (enable/disable)
    LDY     #$02
    STY     CHAR_SELECT_TABLE+1  ; max_range = 2
    JMP     ENTER_CHAR_SELECT
```

Defines:
CHAR_SELECT_HERE, used in chunk 281.
CHAR_SELECT_HOME, used in chunk 281.
CHAR_SELECT_INIT, used in chunk 267a.
CHAR_SELECT_NEXT, used in chunk 281.
CHAR_SELECT_PREV, used in chunk 281.
ENTER_CHAR_SELECT, used in chunk 617.

Uses ACTIVE_CHAR 492c, CHAR_SELECT_TABLE 281, CURRENT_PLAYER 562a, DISPLAY_MESSAGE 637, FIND_FIRST_PC 612, FIND_NEXT_PC_WRAP 614, FIND_PREV_PC 616, INPUT_LOOP 247, MOVE_LOOP 232, MSG_SAVED_ME 634, SET_CURSOR_ROW21 627, and STORE_SELECTION_TO_RECORD 54b.

7.0.6 Alternative input table

The alternative input table is used by START_GAME_INPUT for shop interactions and special scenes. It has three options: "PAY [amount] FOR [item]" (handler at SHOP_BUY_HANDLER), "BUY SOMETHING ELSE" (handler at $7194), and "NO SALE" (return to no-op init at $626B).

285 ⟨*alt input table* 285⟩≡ (909)

```
    ORG     $6436
ALT_INPUT_TABLE:
    ; 3 options, max_range=2, init=$626B (RTS = no-op)
    DC.B    $03,$02
    DC.W    $626B
    ; rec0: PAY [amount] FOR [item]
    HEX     01 05 03 02 00 00
    DC.W    S_PAY_FOR          ; "PAY 000000 FOR ..."
    DC.W    SHOP_BUY_HANDLER   ; buy handler
    ; rec1: BUY SOMETHING ELSE
    HEX     01 2D 01 03 00 00
    DC.W    $7D23              ; "BUY SOMETHING ELSE"
    DC.W    BROWSE_HANDLER     ; browse: advance to next item
    ; rec2: NO SALE
    HEX     01 55 02 01 00 00
    DC.W    $7D36              ; "NO SALE"
    DC.W    $626B              ; no-op (return)
```

Defines:
ALT_INPUT_TABLE, used in chunk 286.
Uses BROWSE_HANDLER 404, SHOP_BUY_HANDLER 56, and S_PAY_FOR 310.

START_GAME_INPUT enters the input loop with a pointer to the alternative input table at ALT_INPUT_TABLE (X=$64, Y=$36). This is used when the game needs to present a different set of options than the standard action menu—for example, during shop interactions or special scenes.

286 ⟨*start game input* 286⟩≡ (909)

```
      ORG     $6458
  START_GAME_INPUT:
      SUBROUTINE START_GAME_INPUT
      ; Enter the alternative input loop (shops/special scenes).
      ;
      ; Behavior:
      ;   Loads ALT_INPUT_TABLE pointer into X/Y and tail-calls INPUT_LOOP.
      ;
      ; Clobbers: A, X, Y

      LDX     #>ALT_INPUT_TABLE
      LDY     #<ALT_INPUT_TABLE
      JMP     INPUT_LOOP
```

Defines:
START_GAME_INPUT, used in chunk 404.
Uses ALT_INPUT_TABLE 285 and INPUT_LOOP 247.

7.1 Save/Load Disk I/O

7.1.1 Disk I/O data

The save/load system uses a block of configuration data in the menu option strings area. GAME_DISK_SLOTS determines how many save slots fit on the game disk itself (just 1); higher-numbered slots require a separate scenario disk. SAVE_TRACK_BASE is a two-byte table of base track numbers indexed by disk (0=game disk at track $13, 1=scenario disk at track $01). Each slot occupies two tracks (26 sectors at 13 sectors per track).

SAVE_SECTOR_COUNT is the number of sectors per save game (26). GAME_SIG_TRACK and GAME_SIG_SECTOR specify where the game disk signature lives (track $11, sector $00). DISK_SIGNATURE is the 3-byte "ALI" signature (high ASCII $C1,$CC,$C9) used to identify valid scenario disks. SHUFFLE_COUNT controls how many iterations the entity-shuffle routine performs (100).

7.1.2 Continue game handler

CONTINUE_GAME is the handler for the "CONTINUE" option in the save/load menu (SAVE_LOAD_TABLE).

- If SLOT_NUMBER is below GAME_DISK_SLOTS (i.e. slot 0 on the game disk), it calls ENSURE_GAME_DISK to verify the game disk, then jumps to READ_GAME_DATA to load the save.
- Otherwise the save is on a scenario disk. It displays message $33 (insert scenario disk prompt), waits for a keypress, then calls VERIFY_DISK_SIGNATURE to check the disk has the "ALI" signature. If valid, it reads the save and writes it back to the game disk via ENSURE_GAME_DISK. If invalid, it displays message $16 (error) and returns to INIT_WORLD.

Number of save slots on the game disk, used by CONTINUE_GAME.

287 ⟨*game disk slots* 287⟩≡ (909)

```
    ORG     $7AC5
GAME_DISK_SLOTS:
    DC.B    $01                     ; save slots on game disk
```

Defines:
GAME_DISK_SLOTS, used in chunks 289, 293, and 305.

Disk signature bytes (high-ASCII "ALI"), used by CONTINUE_GAME.

288 ⟨*disk signature* 288⟩≡ (909)

```
    ORG     $7ACC
DISK_SIGNATURE:
    HEX     C1 CC C9                ; "ALI" (high ASCII)
```

Defines:
DISK_SIGNATURE, used in chunk 298.

289 ⟨*continue game* 289⟩≡ (909)

```
      ORG     $69C8
  CONTINUE_GAME:
      SUBROUTINE CONTINUE_GAME
      ; Load a saved game from the appropriate disk.
      ;
      ; Inputs:
      ;   SLOT_NUMBER | save slot index
      ;
      ; Behavior:
      ;   If slot is on game disk, verifies disk and reads save data.
      ;   If slot is on scenario disk, prompts for insert, verifies
      ;   signature, reads data, then re-verifies game disk.
      ;
      ; Clobbers: A, X, Y

      LDA     SLOT_NUMBER
      CMP     GAME_DISK_SLOTS
      BCS     .scenario          ; slot >= 1 -> scenario disk needed
      JSR     ENSURE_GAME_DISK   ; verify game disk
      JMP     READ_GAME_DATA     ; load save from game disk

  .scenario:
      JSR     SET_CURSOR_ROW21
      LDA     #MSG_INSERT_SCENARIO               ; "INSERT SCENARIO DISK"
      JSR     DISPLAY_MESSAGE
      JSR     WAIT_KEYPRESS      ; wait for user
      JSR     VERIFY_DISK_SIGNATURE ; check "ALI" signature
      BCS     .bad_disk          ; carry set -> signature mismatch
      JSR     READ_GAME_DATA     ; load from scenario disk
      JSR     PROMPT_INSERT_DISK ; "INSERT GAME DISK"
      JMP     ENSURE_GAME_DISK   ; re-verify game disk

  .bad_disk:
      JSR     SET_CURSOR_ROW21
      LDA     #MSG_TOUGH_HIDE               ; error message
      JSR     DISPLAY_MESSAGE
      JSR     ENSURE_GAME_DISK
      JMP     INIT_WORLD         ; back to save/load menu
```

Defines:
CONTINUE_GAME, used in chunk 274.

Uses DISPLAY_MESSAGE 637, ENSURE_GAME_DISK 297, GAME_DISK_SLOTS 287, MSG_INSERT_SCENARIO 634, MSG_TOUGH_HIDE 634, PROMPT_INSERT_DISK 297, READ_GAME_DATA 305, SET_CURSOR_ROW21 627, SLOT_NUMBER 275b, VERIFY_DISK_SIGNATURE 298, and WAIT_KEYPRESS 297.

7.1.3 Shuffle entities

SHUFFLE_ENTITIES randomly reorders the entity linked list by performing SHUFFLE_COUNT (100) iterations of a pick-and-relink algorithm. Each iteration:

- Picks a random position (0–63) in the list via RANDOM_IN_RANGE.
- Walks that many links forward from the list head at $4000.
- Unlinks the reached node and re-appends it at the end of the chain.

This effectively shuffles entity ordering so that saved games do not preserve a deterministic entity sequence.

Entity shuffle iteration count, used by SHUFFLE_ENTITIES.

290 ⟨*shuffle count* 290⟩≡ (909)

```
    ORG     $7ACF
SHUFFLE_COUNT:
    DC.B    $64                     ; entity shuffle iterations (100)
```

Defines:
SHUFFLE_COUNT, used in chunk 291.

291 ⟨*shuffle entities* 291⟩≡ (909)

```
    ORG     $69FD
SHUFFLE_ENTITIES:
    SUBROUTINE SHUFFLE_ENTITIES
    ; Randomize entity order by repeated unlink-and-reappend.
    ;
    ; Behavior:
    ;   Each iteration walks a random distance (0-63) into the linked
    ;   list at [[MOB_BASE_LO]], unlinks the reached node, then re-appends it
    ;   at the end. Recurses until SHUFFLE_COUNT < 0.
    ;
    ; Modifies: entity linked list (byte 2 pointers)
    ; Clobbers: A, X, Y

    DEC     SHUFFLE_COUNT
    BPL     .go
    RTS

.go:
    LDA     #$3F                ; random 0-63
    JSR     RANDOM_IN_RANGE
    STA     $5AA5               ; temp: walk count
    LDA     #$00                ; \ $BA/$BB = $4000 (list head)
    STA     $BA                 ;  |
    LDA     #$40                ;  |
    STA     $BB                 ; /

.walk:
    LDA     $BA                 ; \ copy $BA/$BB -> $BE/$BF
    STA     $BE                 ;  | (previous pointer)
    LDA     $BB                 ;  |
    STA     $BF                 ; /
    LDY     #$02
    LDA     ($BE),Y             ; get next-pointer (byte 2)
    STA     CURRENT_ROOM        ; temp: save next-pointer
    JSR     GET_MOB_DATA        ; advance $BA/$BB to next record
    DEC     $5AA5               ; decrement walk count
    BPL     .walk               ; loop until count exhausted

    ; Unlink: splice current node out of chain
    LDY     #$02
    LDA     ($BA),Y             ; source's next-pointer
    STA     ($BE),Y             ; patch previous to skip source
    LDA     #$00
    STA     ($BA),Y             ; clear source's next-pointer

    ; Re-append: walk to end of chain, link source there
    LDA     ($BE),Y             ; follow chain from previous
.find_end:
    JSR     GET_MOB_DATA        ; advance
```

```
    LDY     #$02
    LDA     ($BA),Y
    BNE     .find_end          ; loop until next = 0 (end of list)

    LDA     CURRENT_ROOM       ; restore saved next-pointer
    STA     ($BA),Y            ; link source at end
    JMP     SHUFFLE_ENTITIES   ; next iteration
```

Defines:
SHUFFLE_ENTITIES, never used.
Uses CURRENT_ROOM 476, GET_MOB_DATA 602, MOB_BASE_LO 53a, RANDOM_IN_RANGE 597, and SHUFFLE_COUNT 290.

7.1.4 Save game handler

SAVE_GAME is the handler for the "SAVE GAME IN SLOT NUMBER NN" option in the save game slot menu.

- If SLOT_NUMBER is below GAME_DISK_SLOTS, it saves directly to the game disk via WRITE_GAME_DATA, then returns to MOVE_LOOP.
- Otherwise, it checks the self-modifying flag at SCENARIO_INIT_FLAG (SCENARIO_INIT_FLAG). On first use, it calls ENSURE_GAME_DISK and enters scene $42 (the scenario disk prompt scene), then sets the flag. Subsequent calls jump directly to ENTER_SCENARIO_DISK.

293 ⟨*save game* 293⟩≡ (909)

```
    ORG     $6A47
SAVE_GAME:
    SUBROUTINE SAVE_GAME
    ; Save current game state to game or scenario disk.
    ;
    ; Inputs:
    ;   SLOT_NUMBER | save slot index
    ;
    ; Behavior:
    ;   If slot is on game disk, verifies disk and writes save data.
    ;   If slot is on scenario disk, shows intro scene on first use,
    ;   sets SCENARIO_INIT_FLAG, then enters scenario disk handler.
    ;
    ; Clobbers: A, X, Y

    LDA     SLOT_NUMBER
    CMP     GAME_DISK_SLOTS
    BCS     .scenario           ; slot >= 1 -> scenario disk
    JSR     ENSURE_GAME_DISK
    JSR     WRITE_GAME_DATA
    JMP     MOVE_LOOP

SCENARIO_INIT_FLAG:
    DC.B    $00                 ; $6A58: self-modifying flag (0=first, 1=initialized)

.scenario:
    LDA     SCENARIO_INIT_FLAG
    BNE     .already_init       ; flag set -> skip scene
    JSR     ENSURE_GAME_DISK
    LDA     #$42                ; scene $42: scenario disk prompt
    JSR     SCENE_LOOP
    LDA     #$01
    STA     SCENARIO_INIT_FLAG ; set flag (self-modifying)
.already_init:
    JMP     ENTER_SCENARIO_DISK
```

Defines:
SAVE_GAME, used in chunk 278.
SCENARIO_INIT_FLAG, never used.
Uses ENSURE_GAME_DISK 297, ENTER_SCENARIO_DISK 280b, GAME_DISK_SLOTS 287, MOVE_LOOP 232, SCENE_LOOP 527, SLOT_NUMBER 275b, and WRITE_GAME_DATA 303.

7.1.5 Scenario disk handlers

CANCEL_SCENARIO cancels the scenario disk menu and returns to MOVE_LOOP after verifying the game disk.

SCENARIO_INSERT_PROMPT displays message $33 (insert scenario disk) and re-enters the scenario disk menu.

FORMAT_SCENARIO_DISK formats a new scenario disk:

- Calls VERIFY_GAME_DISK to confirm the inserted disk. If invalid, prompts and re-enters the scenario menu.
- Verifies the "ALI" signature via VERIFY_DISK_SIGNATURE. If the disk already has saved data, prompts with FORMAT_DISK_TRACK.
- If the signature check indicates a blank disk, writes the signature via WRITE_DISK_SIGNATURE, then falls through to write game data.
- On success, displays message $1B (confirmation) and returns to MOVE_LOOP.

USE_SCENARIO_DISK uses a previously formatted scenario disk: verifies the signature, and if valid, writes game data. If invalid, displays message $16 (error).

DISPLAY_INSERT_MSG displays message $1B ("INSERT SCENARIO DISK" or similar confirmation).

295 ⟨*scenario disk handlers* 295⟩≡ (909)

```
    ORG     $6A6E
CANCEL_SCENARIO:
    JSR     ENSURE_GAME_DISK
    JMP     MOVE_LOOP

SCENARIO_INSERT_PROMPT:
    JSR     SET_CURSOR_ROW21
    LDA     #MSG_INSERT_SCENARIO              ; "INSERT SCENARIO DISK"
    JSR     DISPLAY_MESSAGE
    JMP     ENTER_SCENARIO_DISK

FORMAT_SCENARIO_DISK:
    SUBROUTINE FORMAT_SCENARIO_DISK
    ; Format and initialize a blank scenario disk for saving.
    ;
    ; Behavior:
    ;   Checks for existing signature, formats track if needed,
    ;   writes signature and game data. Re-prompts for game disk
    ;   and returns to the move loop on completion.
    ;
    ; Clobbers: A, X, Y
```

```
    JSR     VERIFY_GAME_DISK    ; check game disk inserted
    BCC     SCENARIO_INSERT_PROMPT ; not game disk -> prompt
    JSR     VERIFY_DISK_SIGNATURE ; check "ALI" signature
    BCC     .write              ; no signature -> blank disk, write
    JSR     FORMAT_DISK_TRACK   ; format the track
    BCC     .check_sig          ; format ok -> check signature
.show_msg:
    JSR     DISPLAY_INSERT_MSG ; display confirmation
.cleanup:
    JSR     PROMPT_INSERT_DISK ; "INSERT GAME DISK"
    JSR     ENSURE_GAME_DISK
    JMP     MOVE_LOOP

.check_sig:
    JSR     WRITE_DISK_SIGNATURE ; write "ALI" to disk
    BCS     .show_msg           ; carry set -> show insert msg, then cleanup
    BCC     .write              ; carry clear -> write data

USE_SCENARIO_DISK:
    JSR     VERIFY_DISK_SIGNATURE ; check "ALI" signature
    BCS     .sig_error          ; no match -> error
.write:
    JSR     WRITE_GAME_DATA     ; save game data to disk
    JMP     .cleanup

.sig_error:
    JSR     SET_CURSOR_ROW21
    LDA     #MSG_TOUGH_HIDE                ; error message
    JSR     DISPLAY_MESSAGE
    JMP     .cleanup

DISPLAY_INSERT_MSG:
    JSR     SET_CURSOR_ROW21
    LDA     #MSG_BREAKS_FREE                ; "INSERT" confirmation message
    JMP     DISPLAY_MESSAGE
```

Defines:
CANCEL_SCENARIO, used in chunk 280a.
DISPLAY_INSERT_MSG, never used.
FORMAT_SCENARIO_DISK, used in chunk 280a.
SCENARIO_INSERT_PROMPT, never used.
USE_SCENARIO_DISK, used in chunk 280a.

Uses DISPLAY_MESSAGE 637, ENSURE_GAME_DISK 297, ENTER_SCENARIO_DISK 280b, FORMAT_DISK_TRACK 298, MOVE_LOOP 232, MSG_BREAKS_FREE 634, MSG_INSERT_SCENARIO 634, MSG_TOUGH_HIDE 634, PROMPT_INSERT_DISK 297, SET_CURSOR_ROW21 627, VERIFY_DISK_SIGNATURE 298, VERIFY_GAME_DISK 301, WRITE_DISK_SIGNATURE 298, and WRITE_GAME_DATA 303.

7.1.6 Game disk verification

ENSURE_GAME_DISK loops until the game disk is detected. It calls VERIFY_GAME_DISK and if carry is set (wrong disk), displays the insert prompt and retries.

PROMPT_INSERT_DISK displays messages $03 and $30, then waits for a keypress.

WAIT_KEYPRESS displays message $30 ("PRESS ANY KEY"), clears the keyboard strobe, and polls until a key is pressed.

297 ⟨*ensure game disk* 297⟩≡ (909)

```
    ORG     $6ABF
ENSURE_GAME_DISK:
    JSR     VERIFY_GAME_DISK
    BCS     .wrong_disk
    RTS
.wrong_disk:
    JSR     PROMPT_INSERT_DISK
    JMP     ENSURE_GAME_DISK

PROMPT_INSERT_DISK:
    JSR     SET_CURSOR_ROW21
    LDA     #MSG_INSERT_GAME                ; "INSERT GAME DISK"
    JSR     DISPLAY_MESSAGE

WAIT_KEYPRESS:
    LDA     #MSG_PRESS_KEY                ; "PRESS ANY KEY"
    JSR     DISPLAY_MESSAGE
    STA     $C010              ; clear keyboard strobe
.wait:
    LDA     $C000              ; read keyboard
    BPL     .wait              ; loop until key pressed
    RTS
```

Defines:
ENSURE_GAME_DISK, used in chunks 289, 293, and 295.
PROMPT_INSERT_DISK, used in chunks 289 and 295.
WAIT_KEYPRESS, used in chunk 289.

Uses DISPLAY_MESSAGE 637, MSG_INSERT_GAME 634, MSG_PRESS_KEY 634, SET_CURSOR_ROW21 627, and VERIFY_GAME_DISK 301.

7.1.7 Disk format and signature

FORMAT_DISK_TRACK issues an RWTS format command (command 4) for track $04 with 12 sectors.

WRITE_DISK_SIGNATURE writes the 3-byte "ALI" signature (DISK_SIGNATURE) to the disk via SETUP_DATA_IOB. It copies the signature to SCENE_TEXT_DATA, sets up the IOB, then calls DISK_WRITE with up to 3 retries. Returns carry clear on success.

VERIFY_DISK_SIGNATURE reads the disk and compares the first 3 bytes against DISK_SIGNATURE. Returns carry clear if the signature matches.

298 ⟨*format and signature* 298⟩≡ (909)

```
    ORG     $6AE1
FORMAT_DISK_TRACK:
    LDY     #$0C                ; \ RWTS format command
    LDA     #$04                ;  | command = 4 (format track)
    JMP     RWTS_DISPATCH       ; /

WRITE_DISK_SIGNATURE:
    SUBROUTINE WRITE_DISK_SIGNATURE
    ; Write the 3-byte "ALI" signature to the scenario disk.
    ;
    ; Behavior:
    ;   Copies DISK_SIGNATURE into SCENE_TEXT_DATA, configures IOB,
    ;   then attempts up to 3 disk writes.
    ;
    ; Outputs:
    ;   C | clear on success, set on failure
    ; Clobbers: A, X, Y

    LDA     #$02                ; retry counter
    STA     $BF
    LDX     #$02
.copy:
    LDA     DISK_SIGNATURE,X    ; copy "ALI" to buffer
    STA     SCENE_TEXT_DATA,X
    DEX
    BPL     .copy
    JSR     SETUP_DATA_IOB      ; set IOB buffer = $5C00
.retry:
    JSR     DISK_WRITE          ; write to disk
    BCS     .error
    RTS                         ; success (carry clear)
.error:
    DEC     $BF
    BPL     .retry
    SEC                         ; all retries failed
    RTS
```

```
VERIFY_DISK_SIGNATURE:
    SUBROUTINE VERIFY_DISK_SIGNATURE
    ; Read and verify the "ALI" disk signature.
    ;
    ; Behavior:
    ;   Configures IOB, reads the signature sector with up to 3
    ;   retries, and compares 3 bytes against DISK_SIGNATURE.
    ;
    ; Outputs:
    ;   C | clear if signature matches, set on mismatch or error
    ; Clobbers: A, X, Y

    JSR     SETUP_DATA_IOB      ; set IOB buffer = $5C00
    LDA     #$02                ; retry counter
    STA     $BF
.retry:
    JSR     DISK_READ           ; read from disk
    BCC     .check              ; success -> verify bytes
    DEC     $BF
    BPL     .retry
.fail:
    SEC                         ; all retries failed / mismatch
    RTS
.check:
    LDX     #$02
.cmp:
    LDA     DISK_SIGNATURE,X    ; expected "ALI"
    CMP     SCENE_TEXT_DATA,X   ; actual
    BNE     .fail               ; mismatch -> fail immediately
    DEX
    BPL     .cmp
    CLC                         ; signature match
    RTS
```

Defines:
FORMAT_DISK_TRACK, used in chunk 295.
VERIFY_DISK_SIGNATURE, used in chunks 289 and 295.
WRITE_DISK_SIGNATURE, used in chunk 295.

Uses DISK_READ 546, DISK_SIGNATURE 288, DISK_WRITE 546, RWTS_DISPATCH 546, SCENE_TEXT_DATA 526a, and SETUP_DATA_IOB 301.

7.1.8 IOB setup and game disk check

SETUP_DATA_IOB configures the RWTS IOB for a single-sector operation with the data buffer at SCENE_TEXT_DATA. It clears the track/sector fields and sets the buffer pointer.

VERIFY_GAME_DISK reads the game disk at GAME_SIG_TRACK/ GAME_SIG_SECTOR and checks for the magic signature bytes $03,$0F,$10. Returns carry clear if the game disk is present.

Game disk signature track and sector, used by SETUP_DATA_IOB.

300 ⟨*game sig params* 300⟩≡ (909)

```
    ORG     $7ACA
GAME_SIG_TRACK:
    DC.B    $11                     ; game disk signature track
GAME_SIG_SECTOR:
    DC.B    $00                     ; game disk signature sector
```

Defines:
GAME_SIG_SECTOR, used in chunks 301 and 666.
GAME_SIG_TRACK, used in chunks 301 and 666.

301 ⟨*setup data iob* 301⟩≡ (909)

```
        ORG     $6B27
    SETUP_DATA_IOB:
        LDA     #$00
        LDY     #$04
        STA     (RWTS_IOB_PTR),Y   ; IOB+4 = track 0
        INY
        STA     (RWTS_IOB_PTR),Y   ; IOB+5 = sector 0
        LDA     #$00
        LDY     #$08
        STA     (RWTS_IOB_PTR),Y   ; IOB+8 = buffer lo ($00)
        INY
        LDA     #$5C
        STA     (RWTS_IOB_PTR),Y   ; IOB+9 = buffer hi ($5C -> $5C00)
        RTS

    VERIFY_GAME_DISK:
        SUBROUTINE VERIFY_GAME_DISK
        ; Verify that the inserted disk is the game disk.
        ;
        ; Behavior:
        ;   Reads track $11 sector $00 via RWTS with up to 3 retries.
        ;   Checks that the first three bytes are $03, $0F, $10 (VTOC).
        ;
        ; Outputs:
        ;   C | clear if game disk confirmed, set on mismatch
        ; Clobbers: A, X, Y

        JSR     SETUP_DATA_IOB
        LDA     GAME_SIG_TRACK     ; track $11
        LDY     #$04
        STA     (RWTS_IOB_PTR),Y   ; IOB+4 = track
        LDA     GAME_SIG_SECTOR    ; sector $00
        INY
        STA     (RWTS_IOB_PTR),Y   ; IOB+5 = sector
        LDA     #$02               ; retry counter
        STA     $BE
    .retry:
        JSR     DISK_READ
        BCC     .check
        DEC     $BE
        BNE     .retry
    .fail:
        SEC                        ; all retries failed / mismatch
        RTS
    .check:
        LDA     SCENE_TEXT_DATA    ; byte 0
        CMP     #$03
        BNE     .fail
        LDA     SCENE_TEXT_DATA+1  ; byte 1
```

```
    CMP     #$0F
    BNE     .fail
    LDA     SCENE_TEXT_DATA+2  ; byte 2
    CMP     #$10
    BNE     .fail
    CLC                        ; game disk confirmed
    RTS
```

Defines:
SETUP_DATA_IOB, used in chunk 298.
VERIFY_GAME_DISK, used in chunks 295 and 297.
Uses DISK_READ 546, GAME_SIG_SECTOR 300, GAME_SIG_TRACK 300, RWTS_IOB_PTR 589a, and SCENE_TEXT_DATA 526a.

7.1.9 Write game data

WRITE_GAME_DATA writes SAVE_SECTOR_COUNT (26) sectors of game state from the $4000 buffer to disk. It calls SETUP_SLOT_IOB to calculate the starting track/sector from SLOT_NUMBER, then writes each sector via DISK_WRITE, calling ADVANCE_SECTOR to step through consecutive sectors. Returns A=0 on success, A=1 on failure.

Sectors per save game, used by WRITE_GAME_DATA.

302 ⟨*save sector count* 302⟩≡ (909)

```
    ORG     $7AC9
SAVE_SECTOR_COUNT:
    DC.B    $1A                    ; sectors per save game (26)
```

Defines:
SAVE_SECTOR_COUNT, used in chunks 303 and 305.

303 ⟨*write game data* 303⟩≡ (909)

```
    ORG     $6B72
WRITE_GAME_DATA:
    SUBROUTINE WRITE_GAME_DATA
    ; Write 26 sectors of game data to disk from $4000+.
    ;
    ; Behavior:
    ;   Configures IOB for the current save slot, then writes
    ;   SAVE_SECTOR_COUNT consecutive sectors, advancing buffer
    ;   and sector after each. Retries once on error.
    ;
    ; Outputs:
    ;   A | 0 on success, 1 on failure
    ; Clobbers: A, X, Y

    LDA     #$02                ; outer retry counter
    STA     $BF
    JSR     SETUP_SLOT_IOB      ; calculate track from slot
    LDA     SAVE_SECTOR_COUNT   ; 26 sectors
    STA     $BE
.sector:
    JSR     DISK_WRITE
    BCS     .error
    JSR     ADVANCE_SECTOR
    DEC     $BE
    BNE     .sector
    LDA     #$00                ; success
    RTS
.error:
    DEC     $BF
    BNE     .sector             ; retry remaining sectors
    LDA     #$01                ; failure
    RTS
```

Defines:
WRITE_GAME_DATA, used in chunks 293 and 295.
Uses ADVANCE_SECTOR 305, DISK_WRITE 546, SAVE_SECTOR_COUNT 302, and SETUP_SLOT_IOB 305.

7.1.10 Read game data and IOB helpers

READ_GAME_DATA reads SAVE_SECTOR_COUNT (26) sectors of save data from disk into the $4000 buffer. On persistent failure, it displays message $3B and jumps to COLD_RESTART.

SETUP_SLOT_IOB calculates the disk track for a given slot:

- If SLOT_NUMBER ¡ GAME_DISK_SLOTS, uses disk 0 (game disk) with base track from SAVE_TRACK_BASE[0].
- Otherwise, subtracts GAME_DISK_SLOTS and uses disk 1 (scenario disk) with base track from SAVE_TRACK_BASE[1].
- Track = base + slot × 2 (each slot spans 2 tracks).

Sets the IOB buffer to $4000 and clears the byte-count field.

ADVANCE_SECTOR increments the buffer page (IOB+9) and sector (IOB+5). When the sector reaches 13 ($0D), it wraps to 0 and increments the track (IOB+4), starting the next track.

Base track numbers for save data, used by READ_GAME_DATA.

304 ⟨*save track base* 304⟩≡ (909)

```
    ORG     $7AC7
SAVE_TRACK_BASE:
    HEX     13 01                   ; base track: [0]=game ($13), [1]=scenario ($01)
```

Defines:
SAVE_TRACK_BASE, used in chunk 305.

305 ⟨*read game data* 305⟩≡ (909)

```
    ORG     $6B95
READ_GAME_DATA:
    SUBROUTINE READ_GAME_DATA
    ; Read 26 sectors of game data from disk into $4000+.
    ;
    ; Behavior:
    ;   Configures IOB for the current save slot, then reads
    ;   SAVE_SECTOR_COUNT consecutive sectors. Retries once on
    ;   error; on persistent failure, cold-restarts the system.
    ;
    ; Clobbers: A, X, Y

    LDA     #$02                ; outer retry counter
    STA     $BF
    JSR     SETUP_SLOT_IOB
    LDA     SAVE_SECTOR_COUNT   ; 26 sectors
    STA     $BE
.sector:
    JSR     DISK_READ
    BCS     .error
    JSR     ADVANCE_SECTOR
    DEC     $BE
    BNE     .sector
    LDA     #$00                ; success
    RTS
.error:
    DEC     $BF
    BNE     .sector
    JSR     SET_CURSOR_ROW21
    LDA     #MSG_READ_ERROR                ; read error message
    JSR     DISPLAY_MESSAGE
    JMP     COLD_RESTART        ; fatal: reboot from disk

SETUP_SLOT_IOB:
    SUBROUTINE SETUP_SLOT_IOB
    ; Configure the RWTS IOB for a save-slot disk operation.
    ;
    ; Inputs:
    ;   SLOT_NUMBER | save slot index
    ;
    ; Behavior:
    ;   Determines game vs scenario disk, computes starting track
    ;   as slot*2 + base, sets IOB fields: track, sector 0,
    ;   buffer at [[MOB_BASE_LO]].
    ;
    ; Clobbers: A, X, Y

    LDX     #$00                ; disk index 0 (game disk)
    LDA     SLOT_NUMBER
```

```
    CMP     GAME_DISK_SLOTS
    BCC     .set_track          ; slot < threshold -> game disk
    SEC
    SBC     GAME_DISK_SLOTS     ; adjust for scenario disk
    INX                         ; disk index 1 (scenario disk)
.set_track:
    ASL                         ; slot * 2 (2 tracks per slot)
    CLC
    ADC     SAVE_TRACK_BASE,X   ; add base track for this disk
    LDY     #$04
    STA     (RWTS_IOB_PTR),Y    ; IOB+4 = track
    INY
    LDA     #$00
    STA     (RWTS_IOB_PTR),Y    ; IOB+5 = sector 0
    LDA     #$00
    LDY     #$08
    STA     (RWTS_IOB_PTR),Y    ; IOB+8 = buffer lo ($00)
    LDA     #$40
    INY
    STA     (RWTS_IOB_PTR),Y    ; IOB+9 = buffer hi ($40 -> $4000)
    LDA     #$00
    LDY     #$0B
    STA     (RWTS_IOB_PTR),Y    ; IOB+$0B = byte count (0 = 256)
    RTS

ADVANCE_SECTOR:
    SUBROUTINE ADVANCE_SECTOR
    ; Advance IOB to the next sector and buffer page.
    ;
    ; Behavior:
    ;   Increments the buffer page (IOB+9) and sector (IOB+5).
    ;   If sector reaches 13, wraps to 0 and increments track.
    ;
    ; Clobbers: A, X, Y

    LDY     #$09
    LDA     (RWTS_IOB_PTR),Y    ; buffer page
    TAX
    INX                         ; next page
    TXA
    STA     (RWTS_IOB_PTR),Y
    LDY     #$05
    LDA     (RWTS_IOB_PTR),Y    ; sector number
    TAX
    INX
    CPX     #$0D                ; 13 sectors per track
    BCS     .next_track
    TXA
    STA     (RWTS_IOB_PTR),Y    ; store incremented sector
    RTS
```

```
.next_track:
    LDA     #$00                ; sector = 0
    STA     (RWTS_IOB_PTR),Y
    DEY                         ; Y = 4 (track)
    LDA     (RWTS_IOB_PTR),Y
    TAX
    INX                         ; next track
    TXA
    STA     (RWTS_IOB_PTR),Y
    RTS
```

Defines:
ADVANCE_SECTOR, used in chunk 303.
READ_GAME_DATA, used in chunk 289.
SETUP_SLOT_IOB, used in chunk 303.

Uses COLD_RESTART 675, DISK_READ 546, DISPLAY_MESSAGE 637, GAME_DISK_SLOTS 287, MOB_BASE_LO 53a, MSG_READ_ERROR 634, RWTS_IOB_PTR 589a, SAVE_SECTOR_COUNT 302, SAVE_TRACK_BASE 304, SET_CURSOR_ROW21 627, and SLOT_NUMBER 275b.

CHECK_COMBAT_ELIGIBILITY determines which turn options are available to the current entity. It clears SAFE_TO_REST and UNDER_LEVEL, then checks the entity's level (field $0C, bits 2–0 after ROR) against STEPS_TAKEN. If the level is below the step count, UNDER_LEVEL is set, which enables the flee menu option and disables combat options. If STEPS_TAKEN is zero, the routine performs a deeper check: it compares offense (field 5) against defense (field 6), and if offense exceeds defense and there are no adjacent threats or hostile encounters at the entity's position, it sets SAFE_TO_REST to enable the rest option.

308 ⟨*check combat eligibility* 308⟩≡ (909)

```
      ORG     $750C
  CHECK_COMBAT_ELIGIBILITY:
      SUBROUTINE CHECK_COMBAT_ELIGIBILITY
      ; Check whether the current entity is eligible to enter combat.
      ;
      ; Inputs:
      ;   ENTITY_PTR | pointer to the entity record
      ;
      ; Behavior:
      ;   Extracts level threshold from byte 12; if STEPS_TAKEN is below
      ;   it, sets UNDER_LEVEL. Otherwise checks strength, adjacent
      ;   threats, and encounter state. Sets SAFE_TO_REST=1 if no threats.
      ;
      ; Outputs: SAFE_TO_REST, UNDER_LEVEL
      ; Clobbers: A, X, Y

      STOB    #$00,SAFE_TO_REST
      STA     UNDER_LEVEL
      LDY     #$0C
      LDA     (ENTITY_PTR),Y
      ROR
      AND     #$07
      BAMI    STEPS_TAKEN,.under_level
      LDA     STEPS_TAKEN
      BNE     .done
      LDY     #$05
      LDA     (ENTITY_PTR),Y
      AND     #$1F
      STA     $BA
      INY
      LDA     (ENTITY_PTR),Y
      AND     #$3F
      BAPL    $BA,.done
      LDY     #$03
      LDA     (ENTITY_PTR),Y
      JSR     CHECK_ADJACENT_THREATS
      LDA     ADJACENT_THREAT
      BNE     .done
      LDY     #$03
      LDA     (ENTITY_PTR),Y
```

```
    JSR     CHECK_ENCOUNTER
    LDA     ENCOUNTER_RESULT
    BAEQ    #$02,.done
    STOB    #$01,SAFE_TO_REST
    JMP     .done
.under_level:
    STOB    #$01,UNDER_LEVEL
.done:
    RTS
```

Defines:
CHECK_COMBAT_ELIGIBILITY, used in chunk 264.

Uses ADJACENT_THREAT 226b, CHECK_ADJACENT_THREATS 320, CHECK_ENCOUNTER 312, ENCOUNTER_RESULT 226a, ENTITY_PTR 561, SAFE_TO_REST 570b, STEPS_TAKEN 231a, STOB 8c, and UNDER_LEVEL 570b.

7.1.11 Menu strings

Menu strings ($7ADC–$7D82). Length-prefixed high-ASCII strings for all menu options: action menus (MOVE, ATTACK, REST, DEFEND), combat (direction labels, target names), shop (PAY/BUY/NO SALE), settings (SOUND, SPEED, DANGER), character management (ADD SOMEONE, RETIRE, DROP GOLD/ARMOR), save/load (CONTINUE GAME, BEGIN NEW GAME, scenario disk options), and treasure chest messages. Several strings contain mutable fields: digit placeholders in "CONTINUE GAME NUMBER NN" (S_CONTINUE_TENS/ S_CONTINUE_ONES), "SAVE GAME IN SLOT NUMBER 00", and "PAY 000000 FOR" (price display), plus the 20-byte name buffer at ATTACK_NAME_BUF and drop amount display at DROP_DISPLAY_BUF.

310 ⟨*menu strings* 310⟩≡ (909)

```
    ORG     $7ADC
MENU_OPTION_STRINGS:
    ; Menu option strings ($7ADC-$7D82) | length-prefixed high ASCII
    APSTR   "DROP ARMOR"
    APSTR   "RETIRE"
    APSTR   "ADD SOMEONE"
    APSTR   "DROP GOLD"
    APSTR   "SOUND IS ON "
    APSTR   "SAVE GAME"
    APSTR   "DANGER :   "
    APSTR   "TO THIS ROOM"
    APSTR   "TO ALI BABA'S HOME"
    ; "ADD" + 20-byte name buffer (overwritten at runtime)
    APSTR   "ADD 1  .  .5 FL~ XA 5 rB"
    APSTR   "#"
    APSTR   "@"
    APSTR   " "
    APSTR   "SEE YOUR LOT"
    APSTR   "REST"
    APSTR   "DEFEND"
    APSTR   "STOP"
S_CHEST_OPENS:
    APSTR   "THE TREASURE CHEST OPENS UP"
    ; Treasure display (digit fields overwritten at runtime)
S_REVEAL_GOLD:
    APSTR   "TO REVEAL   L kO GOLD DUCATS."
S_NOW_HAVE_GOLD:
    APSTR   "YOU NOW HAVE  dOR O DUCATS."
    DC.B    $00                 ; separator
    APSTR   "CONTINUE GAME NUMBER NN"
    APSTR   "BEGIN NEW GAME"
    APSTR   "USE PREVIOUSLY FORMATTED SCENARIO DISK"
    APSTR   "FORMAT AND USE NEW SCENARIO DISK"
    APSTR   "CANCEL REQUEST"
    APSTR   "ATTACK ABOVE YOU"
```

```
    APSTR   "ATTACK LEFT"
    APSTR   "ATTACK RIGHT"
    APSTR   "ATTACK BELOW YOU"
    ; "ATTACK" + 20-byte name buffer (overwritten at runtime)
    APSTR   "ATTACK 5i<% SIBCM  T1     I"
    APSTR   "ATTACK SOMEONE ELSE"
    APSTR   "CANCEL ATTACK"
S_MISSES:
    APSTR   "MISSES"
    ; "DUCATS TO DROP" + display buffer (overwritten at runtime)
    APSTR   "DUCATS TO DROP :  4L :0"
    APSTR   "KEEP ALL GOLD"
S_PAY_FOR:
    ; Price digits and item name are overwritten at runtime by DISPLAY_SHOP.
    ; The 20 bytes after "FOR " are development leftovers.
    APSTR   "PAY 000000 FOR  _    Y `   B7 1E Au"
    APSTR   "BUY SOMETHING ELSE"
    APSTR   "NO SALE"
    APSTR   "ATTACK"
    APSTR   "OTHER OPTIONS"
    APSTR   "MOVE"
    APSTR   "GAME SPEED : 0"
    APSTR   "SAVE GAME IN SLOT NUMBER 00"
```

Defines:
MENU_OPTION_STRINGS, never used.
S_CHEST_OPENS, used in chunk 447.
S_MISSES, used in chunk 338.
S_NOW_HAVE_GOLD, used in chunk 451a.
S_PAY_FOR, used in chunks 285 and 407a.
S_REVEAL_GOLD, used in chunk 451a.
Uses DISPLAY_SHOP 404.

Chapter 8

Combat

8.1 Encounter detection

Checks whether a given character (by char index in A) is co-located with a mob from an opposing faction. Calls FIND_ENTITY_AT_POS to resolve the character to a mob pointer, then walks the group chain via FIND_MOB_AT_SAME_POS looking for another mob at the same map position. Candidate mobs must have byte 6 bits 5–0 $\geq$ 3 (active mob type) and be from a different faction (checked via CHECK_HOSTILE).

Result stored in ENCOUNTER_RESULT:

- 0 — no mob found at same position (or character is not a mob)
- 1 — found co-located mob but same faction (wrapped search)
- 2 — found co-located mob from opposing faction (enemy encounter)

Callers use the result to set or clear bit 2 of byte $0F of the character's record, which appears to be a "combat engaged" flag.

The routine below implements the encounter check.

312 ⟨*check encounter* 312⟩≡ (909)

```
      ORG     $0E25
  CHECK_ENCOUNTER:
      SUBROUTINE CHECK_ENCOUNTER
      ; Check for hostile encounter at entity's position.
      ;
      ; Input:  A = position (passed to FIND_ENTITY_AT_POS)
```

```
    ; Output: ENCOUNTER_RESULT = 0 (none), 1 (friendly), 2 (hostile)
    ;         MOB_PTR = encountered mob (if found)
    ; Clobbers: A, X, Y

    JSR     FIND_ENTITY_AT_POS      ; -> ENTITY_REC
    LDA     ENTITY_REC+1
    BAEQ    #$00,.none              ; no entity found
    BAMI    #$80,.none              ; bit 7 clear = character, skip
    AND     #$7F                    ; strip mob flag
    STA     MOB_PTR+1               ; \ set up mob pointer
    MOVB    ENTITY_REC,MOB_PTR      ; /
.search:
    MOVW    MOB_PTR,TARGET_REC      ; save current mob as reference
    JSR     FIND_MOB_AT_SAME_POS    ; find next mob at same position
    BAMI    #$01,.none              ; 0 = none found
    BEQ     .friendly               ; 1 = wrapped (same faction)
    LDY     #MFIELD_HP        ; result >= 2: found ahead
    LDA     (MOB_PTR),Y
    AND     #HP_MASK          ; bits 5-0 = HP
    BALT    #$03,.search            ; type < 3 = inactive, skip
    JSR     CHECK_HOSTILE
    BAEQ    #$00,.search            ; same faction -> skip
    STOB    #$02,ENCOUNTER_RESULT   ; hostile encounter
    RTS
.none:
    LDA     #$00
    BEQ     .store
.friendly:
    LDA     #$01
.store:
    STA     ENCOUNTER_RESULT
    RTS
```

Defines:
CHECK_ENCOUNTER, used in chunks 226a, 227, 232, 244, 308, 327, 357, 368, and 898.

Uses CHECK_HOSTILE 317, ENCOUNTER_RESULT 226a, ENTITY_REC 541b, FIND_ENTITY_AT_POS 469, FIND_MOB_AT_SAME_POS 315, MFIELD_HP 561, MOB_PTR 608, MOVB 8c, MOVW 9a, STOB 8c, and TARGET_REC 618b.

Walks the group chain starting from the current mob (MOB_PTR), looking for another mob at the same map position (byte 3 of mob data). If the current mob is the last in the chain (byte 2 = 0), the search wraps around to the first mob via the group head (CHAR_PTR).

Returns in A:

- 0 — no other mob at same position (full circle)
- 1 — found co-located mob (search wrapped past end of chain)
- 2 — found co-located mob ahead in the chain

On success, MOB_PTR is updated to point to the found mob.

COLOCATE_POS saves the current mob's position (byte 3) as the comparison target. COLOCATE_LINK saves the starting mob's next-link (byte 2) as a stop condition to detect a full circle. COLOCATE_WRAPPED is set to 2 initially (found ahead in chain) and changed to 1 if the search wraps past the end. FIND_MOB_AT_SAME_POS returns this value on success.

314a ⟨*var colocate link* 314a⟩≡ (909)

```
    ORG     $5A20
COLOCATE_LINK:
    DC.B    $D4                     ; initial value (runtime: saved next-link (stop condition))
```

Defines:
COLOCATE_LINK, used in chunk 315.

314b ⟨*var colocate pos* 314b⟩≡ (909)

```
    ORG     $5A21
COLOCATE_POS:
    DC.B    $CF                     ; initial value (runtime: saved position for co-location search)
```

Defines:
COLOCATE_POS, used in chunk 315.

314c ⟨*var colocate wrapped* 314c⟩≡ (909)

```
    ORG     $5A22
COLOCATE_WRAPPED:
    DC.B    $D2                     ; initial value (runtime: 1 if wrapped, 2 if ahead)
```

Defines:
COLOCATE_WRAPPED, used in chunk 315.

The routine below implements the co-location search.

315 ⟨*find mob at same pos* 315⟩≡ (909)

```
    ORG     $0E6C
FIND_MOB_AT_SAME_POS:
    SUBROUTINE FIND_MOB_AT_SAME_POS
    ; Walk mob linked list for next mob at same map position.
    ;
    ; Input:  MOB_PTR | starting mob, CHAR_PTR | group head
    ; Output: A=0 none found, A=1 found (wrapped), A=2 found (ahead)
    ;         MOB_PTR updated to found mob
    ; Modifies: COLOCATE_POS, COLOCATE_LINK, COLOCATE_WRAPPED
    ; Clobbers: A, Y

    LDY     #MFIELD_POSITION        ; save starting mob's position
    LDA     (MOB_PTR),Y
    STA     COLOCATE_POS
    DEY                             ; Y = MFIELD_NEXT
    LDA     (MOB_PTR),Y             ; save next-link as stop condition
    STA     COLOCATE_LINK
    STY     COLOCATE_WRAPPED        ; init = 2 (found ahead)
.check_link:
    BANE    #$00,.resolve           ; valid link -> follow
    LDY     #$01                    ; \ wrap to first mob
    STY     COLOCATE_WRAPPED        ;  | mark as wrapped (1)
    INY                             ;  | Y = CFIELD_NEXT
    LDA     (CHAR_PTR),Y            ; / first mob from group head
.resolve:
    JSR     GET_MOB_DATA            ; A -> MOB_DATA_PTR
    LDY     #MFIELD_NEXT
    LDA     (MOB_DATA_PTR),Y        ; this mob's next-link
    BANE    COLOCATE_LINK,.not_self ; differs from start -> not self
    LDA     #$00                    ; full circle -> not found
    RTS
.not_self:
    INY                             ; Y = MFIELD_POSITION
    LDA     (MOB_DATA_PTR),Y
    BAEQ    COLOCATE_POS,.found     ; same position -> found
    DEY                             ; Y = MFIELD_NEXT
    LDA     (MOB_DATA_PTR),Y        ; follow next link
    JMP     .check_link
.found:
    MOVW    MOB_DATA_PTR,MOB_PTR    ; update MOB_PTR to found mob
    LDA     COLOCATE_WRAPPED        ; return 1 (wrapped) or 2 (ahead)
    RTS
```

Defines:
FIND_MOB_AT_SAME_POS, used in chunks 312, 322, 330, and 334.

Uses CFIELD_NEXT 561, CHAR_PTR 591a, COLOCATE_LINK 314a, COLOCATE_POS 314b, COLOCATE_WRAPPED 314c, GET_MOB_DATA 602, MFIELD_NEXT 561, MFIELD_POSITION 561, MOB_DATA_PTR 600b, MOB_PTR 608, and MOVW 9a.

Determines whether two mobs are hostile to each other by comparing their faction classifications. Reads byte 4 (appearance index) from the mob at MOB_PTR and the mob at \TARGET_REC/\TARGET_REC+1, converts each via APPEARANCE_TO_FONTCHAR to get the font group (quotient of dividing by 21), then classifies via GET_HOSTILITY: groups 0–2 and 7 are "friendly" (Y=0), groups 3–6 are "hostile" (Y=1).

Returns A=1 if the two mobs are on different sides, A=0 if same. When both are hostile (Y=1), they are still hostile to each other if their font-char remainders differ (different hostile sub-factions).

HOSTILE_APP saves the appearance byte of the second mob so CHECK_HOSTILE can classify both mobs without losing state. HOSTILE_A and HOSTILE_Y save the return values (remainder and faction flag) from the first GET_HOSTILITY call, allowing the routine to compare both factions after the second call completes.

316a ⟨*var hostile app* 316a⟩≡ (909)

```
    ORG     $5A23
HOSTILE_APP:
    DC.B    $A9                     ; initial value (runtime: saved appearance for hostility check)
```

Defines:
HOSTILE_APP, used in chunk 317.

316b ⟨*var hostile a* 316b⟩≡ (909)

```
    ORG     $5A24
HOSTILE_A:
    DC.B    $AC                     ; initial value (runtime: saved A from first GET_HOSTILITY)
```

Defines:
HOSTILE_A, used in chunk 317.
Uses GET_HOSTILITY 318.

316c ⟨*var hostile y* 316c⟩≡ (909)

```
    ORG     $5A25
HOSTILE_Y:
    DC.B    $D9                     ; initial value (runtime: saved Y from first GET_HOSTILITY)
```

Defines:
HOSTILE_Y, used in chunk 317.
Uses GET_HOSTILITY 318.

The routine below implements the hostility comparison.

317 ⟨*check hostile* 317⟩≡ (909)

```
      ORG     $0EB1
  CHECK_HOSTILE:
      SUBROUTINE CHECK_HOSTILE
      ; Compare faction hostility of two entities.
      ;
      ; Input:  MOB_PTR | first entity, TARGET_REC | second entity
      ; Output: A=1 hostile to each other, A=0 same faction
      ; Clobbers: A, Y

      LDY     #MFIELD_APPEARANCE     ; byte 4: appearance
      LDA     (MOB_PTR),Y            ; first entity appearance
      STA     HOSTILE_APP
      LDA     (TARGET_REC),Y         ; second entity appearance
      JSR     APPEARANCE_TO_FONTCHAR ; -> Y = font group
      JSR     GET_HOSTILITY          ; -> Y = hostility flag
      STA     HOSTILE_A
      STY     HOSTILE_Y
      LDA     HOSTILE_APP            ; first entity appearance
      JSR     APPEARANCE_TO_FONTCHAR
      JSR     GET_HOSTILITY
      BYNE    HOSTILE_Y,.different   ; different factions -> hostile
      BYEQ    #$00,.same             ; both friendly -> same side
      BANE    HOSTILE_A,.different   ; both hostile, different sub -> hostile
  .same:
      LDA     #$00
      RTS
  .different:
      LDA     #$01
      RTS
```

Defines:
CHECK_HOSTILE, used in chunks 312, 322, 334, 365, 385, and 430.

Uses APPEARANCE_TO_FONTCHAR 461, BYNE 18, GET_HOSTILITY 318, HOSTILE_A 316b, HOSTILE_APP 316a, HOSTILE_Y 316c, MFIELD_APPEARANCE 561, MOB_PTR 608, and TARGET_REC 618b.

Classifies a font group number (in Y, from APPEARANCE_TO_FONTCHAR) as friendly or hostile. Groups 0–2 and 7 are friendly (Y←0), groups 3–6 are hostile (Y←1). The value in A is preserved.

318 ⟨*get hostility* 318⟩≡ (909)

```
    ORG     $0EE3
GET_HOSTILITY:
    SUBROUTINE GET_HOSTILITY
    ; Classify font group as friendly or hostile.
    ;
    ; Input:  Y = font group (appearance / 21)
    ; Output: Y=0 friendly (groups 0-2, 7), Y=1 hostile (groups 3-6)
    ;         A preserved

    CPY     #$03                    ; groups 0-2 = friendly
    BCC     .friendly
    BYEQ    #$07,.friendly          ; group 7 = friendly
    LDY     #$01                    ; groups 3-6 = hostile
    RTS
.friendly:
    LDY     #$00
    RTS
```

Defines:
GET_HOSTILITY, used in chunks 316 and 317.

Checks the four cardinal neighbors of a map position for hostile mobs and accumulates their combined threat level. Input: A = position (0–199), ENTITY_PTR = character record pointer, \TARGET_REC/\TARGET_REC+1 = character's mob data pointer.

For each adjacent position (left, right, up, down), calls SUM_HOSTILE_AT_POS to find hostile mobs not already in combat and sum their strength (byte 5 bits 4–0) into ADJACENT_THREAT. After checking all neighbors, compares the accumulated threat against the character's defense value (byte 5 bits 4–1 of the ENTITY_PTR record).

Returns A=1 if defense $\geq$ threat (safe), A=0 if overwhelmed. The threat total remains in ADJACENT_THREAT for callers that check it directly.

Boundary checks prevent wrapping: position 0 has no left neighbor, position 199 no right, positions 0–19 no up, positions 180–199 no down.

ADJACENT_POS saves the target map position on entry to CHECK_ADJACENT_THREATS. The routine adds and subtracts offsets (1 for left/right, 20 for up/down) from this saved position to compute the four cardinal neighbors, passing each to SUM_HOSTILE_AT_POS.

319 ⟨*var adjacent pos* 319⟩≡ (909)

```
    ORG     $5A27
ADJACENT_POS:
    DC.B    $A0                     ; initial value (runtime: saved position for adjacency check)
```

Defines:
ADJACENT_POS, used in chunk 320.

The routine below implements the four-direction threat scan.

320 ⟨*check adjacent threats* 320⟩≡ (909)

```
    ORG     $0EF1
CHECK_ADJACENT_THREATS:
    SUBROUTINE CHECK_ADJACENT_THREATS
    ; Sum hostile strength in 4 cardinal directions, compare
    ; against entity's defense.
    ;
    ; Input:  A = map position, ENTITY_PTR = character
    ; Output: A=1 safe (defense >= threat), A=0 overwhelmed
    ;         ADJACENT_THREAT = total hostile strength
    ; Clobbers: A, X, Y

    STA     ADJACENT_POS            ; save center position
    STOB    #$00,ADJACENT_THREAT    ; clear threat accumulator
    MOVW    ENTITY_PTR,TARGET_REC

    ; --- check left ---
    LDA     ADJACENT_POS
    BEQ     .skip_left              ; pos 0 -> no left
    SEC
    SBC     #$01                    ; pos - 1
    JSR     SUM_HOSTILE_AT_POS
.skip_left:
    ; --- check right ---
    LDA     ADJACENT_POS
    BAEQ    #$C7,.skip_right        ; pos 199 -> no right
    CLC
    ADC     #$01                    ; pos + 1
    JSR     SUM_HOSTILE_AT_POS
.skip_right:
    ; --- check up ---
    LDA     ADJACENT_POS
    BMI     .do_up                  ; >= 128 -> has up
    BAMI    #MAP_WIDTH,.skip_up     ; row 0 -> no up
.do_up:
    SEC
    SBC     #MAP_WIDTH              ; pos - 20
    JSR     SUM_HOSTILE_AT_POS
.skip_up:
    ; --- check down ---
    LDA     ADJACENT_POS
    BPL     .do_down                ; < 128 -> has down
    BAPL    #$B4,.skip_down         ; row 9 -> no down
.do_down:
    CLC
    ADC     #MAP_WIDTH              ; pos + 20
    JSR     SUM_HOSTILE_AT_POS
.skip_down:
```

```
    ; --- compare defense vs threat ---
    LDY     #MFIELD_STRENGTH       ; byte 5: defense in bits 4-1
    LDA     (ENTITY_PTR),Y
    ROR                            ; shift bits 4-1 to 3-0
    AND     #DEFENSE_HALF_MASK
    BAPL    ADJACENT_THREAT,.safe  ; defense >= threat -> safe
    LDA     #$00                   ; overwhelmed
    RTS
.safe:
    LDA     #$01                   ; safe
    RTS
```

Defines:
CHECK_ADJACENT_THREATS, used in chunks 227, 308, and 323.

Uses ADJACENT_POS 319, ADJACENT_THREAT 226b, DEFENSE_HALF_MASK 336, ENTITY_PTR 561, MAP_WIDTH 388a, MFIELD_STRENGTH 561, MOVW 9a, STOB 8c, SUM_HOSTILE_AT_POS 322, and TARGET_REC 618b.

Finds all hostile mobs at a given map position and adds their strength to the threat accumulator ADJACENT_THREAT. Input: A = position, \TARGET_REC/\TARGET_REC+1 = reference mob for faction comparison.

For each mob found at the position, checks hostility via CHECK_HOSTILE and skips mobs that are already combat-engaged (bit 2 of byte $0F set). Hostile mobs contribute their strength (byte 5 bits 4–0) to the running total, clamped at 127.

322 ⟨*sum hostile at pos* 322⟩≡ (909)

```
    ORG     $0F49
SUM_HOSTILE_AT_POS:
    SUBROUTINE SUM_HOSTILE_AT_POS
    ; Sum hostile mob strength at a position into ADJACENT_THREAT.
    ;
    ; Input:  A = map position, TARGET_REC = reference entity
    ;         ADJACENT_THREAT = running total
    ; Output: ADJACENT_THREAT updated (capped at $7F)
    ; Clobbers: A, X, Y

    JSR     FIND_ENTITY_AT_POS      ; -> ENTITY_REC
    LDA     ENTITY_REC+1
    BEQ     .done                   ; no entity
    BAMI    #$80,.done              ; bit 7 clear = character, skip
    AND     #$7F                    ; strip mob flag
    STA     MOB_PTR+1               ; \ set up mob pointer
    MOVB    ENTITY_REC,MOB_PTR      ; /
.check_mob:
    JSR     CHECK_HOSTILE
    BAEQ    #$00,.next              ; not hostile -> skip
    LDY     #MFIELD_FLAGS           ; byte 15: status flags
    LDA     (MOB_PTR),Y
    AND     #FLAGS_ENGAGED_BIT      ; bit 2: already in combat?
    BNE     .next                   ; yes -> skip
    LDY     #MFIELD_STRENGTH        ; byte 5: strength
    LDA     (MOB_PTR),Y
    AND     #STRENGTH_MASK          ; bits 4-0
    CLC
    ADC     ADJACENT_THREAT         ; accumulate
    BPL     .store
    LDA     #$7F                    ; cap at 127
.store:
    STA     ADJACENT_THREAT
.next:
    JSR     FIND_MOB_AT_SAME_POS    ; next mob at same position
    BAEQ    #$02,.check_mob         ; found -> check it
.done:
    RTS
```

Defines:
SUM_HOSTILE_AT_POS, used in chunks 320, 357, and 898.

Uses ADJACENT_THREAT 226b, CHECK_HOSTILE 317, ENTITY_REC 541b, FIND_ENTITY_AT_POS 469, FIND_MOB_AT_SAME_POS 315, FLAGS_ENGAGED_BIT 336, MFIELD_STRENGTH 561, MOB_PTR 608, MOVB 8c, and TARGET_REC 618b.

CHECK_THREATS_HERE is a convenience wrapper that packs CURRENT_COL and CURRENT_ROW into a single position byte via COLROW_TO_POS and tail-calls CHECK_ADJACENT_THREATS to evaluate whether hostile entities are present at that position.

323 ⟨*check threats here* 323⟩≡ (909)

```
    ORG     $645F
CHECK_THREATS_HERE:
    SUBROUTINE CHECK_THREATS_HERE
    ; Check adjacent threats at current cursor position.
    ;
    ; Input:  CURRENT_COL/ROW, ENTITY_PTR
    ; Output: A=1 safe, A=0 overwhelmed (see CHECK_ADJACENT_THREATS)
    ; Clobbers: A, X, Y

    LDA     CURRENT_COL
    LDY     CURRENT_ROW
    JSR     COLROW_TO_POS          ; pack to position byte
    JMP     CHECK_ADJACENT_THREATS ; tail-call
```

Defines:
CHECK_THREATS_HERE, used in chunks 232, 357, 360, 368, and 898.
Uses CHECK_ADJACENT_THREATS 320, COLROW_TO_POS 600a, CURRENT_COL 472b, CURRENT_ROW 473a, and ENTITY_PTR 561.

8.2 Combat resolution

CALC_COMBAT_STRENGTH computes the effective attack power for the current combatant. If the attacker is the player or is at a different position from the target, field 7 (ranged attack) is used; if attacker and target share a position, field 9 (melee attack) is used instead. The chosen field's low nibble becomes the base: two calls to RANDOM_IN_RANGE produce a value between 0 and twice the base (with a carry add), stored in COMBAT_STRENGTH. For player-controlled characters (AI mode 0) when it is not the player's turn, there is an additional critical-hit/fumble check: depending on the high bit of the chosen field, a random roll against either $0D or $50 determines whether a special event triggers. On a roll of zero, the field is marked $81, scene 1 is displayed, and a bonus of 0–9 is added to the combat strength (capped at $1F).

324 ⟨*var combat strength* 324⟩≡ (909)

```
    ORG     $5A78
COMBAT_STRENGTH:
    DC.B    $C4                     ; initial value (runtime: result of CALC_COMBAT_STRENGTH)
```

Defines:
COMBAT_STRENGTH, used in chunks 325, 342, and 346.
Uses CALC_COMBAT_STRENGTH 325.

325 ⟨*calc combat strength* 325⟩≡ (909)

```
    ORG     $16E0
CALC_COMBAT_STRENGTH:
    SUBROUTINE CALC_COMBAT_STRENGTH
    ; Compute effective attack power for the current combatant.
    ;
    ; Input:  ENTITY_PTR | attacker, MOB_PTR | target
    ;         IS_PLAYER_TURN, CHAR_AI_MODE
    ; Output: COMBAT_STRENGTH (0-$1F)
    ;
    ; Behavior:
    ;   Same position -> melee (MFIELD_DEFENSE), else ranged
    ;   (MFIELD_ATTACK).  Low nibble rolled twice and summed.
    ;   Player-controlled on mob's turn: fumble check (roll
    ;   against $0D or $50; zero = fumble with +0..9 bonus).
    ;
    ; Modifies: COMBAT_STRENGTH, may mark stat field $81
    ; Clobbers: A, X, Y

    ; --- select melee or ranged stat ---
    LDA     IS_PLAYER_TURN
    BAEQ    #$01,.use_ranged       ; not player turn -> ranged
    LDY     #MFIELD_POSITION
    LDA     (MOB_PTR),Y
    CMP     (ENTITY_PTR),Y
    BNE     .use_ranged            ; different pos -> ranged
    LDY     #MFIELD_DEFENSE        ; same pos -> melee
    BNE     .got_field
.use_ranged:
    LDY     #MFIELD_ATTACK         ; ranged stat
.got_field:

    ; --- roll base combat strength ---
    TYA                            ; save field index
    PHA
    LDA     (ENTITY_PTR),Y
    AND     #DEFENSE_HALF_MASK     ; low nibble = base
    TAX
    JSR     RANDOM_IN_RANGE        ; first roll
    STA     COMBAT_STRENGTH
    TXA
    JSR     RANDOM_IN_RANGE        ; second roll
    SEC                            ; +1 via carry
    ADC     COMBAT_STRENGTH
    STA     COMBAT_STRENGTH

    ; --- fumble check (player-controlled, mob's turn) ---
    LDA     CHAR_AI_MODE
    BEQ     .check_turn
.exit_pla:
```

```
    PLA
    RTS
.check_turn:
    LDA     IS_PLAYER_TURN
    BNE     .exit_pla               ; player's turn -> no fumble
    PLA
    TAY                             ; Y = field index
    TAX                             ; X = saved for later
    LDA     (ENTITY_PTR),Y
    AND     #TARGET_BIT7_MASK       ; test stat high bit
    BNE     .use_50
    LDA     #$0D                    ; bit 7 clear: 1-in-13
    BNE     .do_random
.use_50:
    LDA     #$50                    ; bit 7 set: 1-in-80
.do_random:
    JSR     RANDOM_IN_RANGE
    BAEQ    #$00,.fumbled           ; zero = fumble
    RTS

    ; --- fumble: mark field, show scene, add bonus ---
.fumbled:
    TXA
    TAY
    LDA     #$81                    ; fumble marker
    STA     (ENTITY_PTR),Y
    JSR     SET_CURSOR_ROW21
    LDA     #$01                    ; scene 1: fumble
    JSR     SCENE_LOOP
    LDA     #$0A                    ; bonus range 0-9
    JSR     RANDOM_IN_RANGE
    SEC
    ADC     COMBAT_STRENGTH
    BALT    #$1F,.cap_ok            ; under cap?
    LDA     #$1F                    ; clamp to $1F
.cap_ok:
    STA     COMBAT_STRENGTH
    RTS
```

Defines:
CALC_COMBAT_STRENGTH, used in chunks 324, 342, and 346.

Uses CHAR_AI_MODE 531b, COMBAT_STRENGTH 324, DEFENSE_HALF_MASK 336, ENTITY_PTR 561, IS_PLAYER_TURN 474, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_POSITION 561, MOB_PTR 608, RANDOM_IN_RANGE 597, SCENE_LOOP 527, SET_CURSOR_ROW21 627, and TARGET_BIT7_MASK 336.

ENTER_COMBAT_STATE transitions the character to combat readiness (state 2). It first calls CHECK_ENCOUNTER to evaluate what is at the character's position, then clears the restricted flag. If the encounter result is not 2 (meaning the character is not co-located with a hostile), the restricted flag is re-set to limit the character's options. Finally, SET_CHAR_STATE is called with 2 to record the new state.

327 ⟨*enter combat state* 327⟩≡ (909)

```
      ORG     $14B7
  ; ----------------------------------------------------------------
  ; ENTER_COMBAT_STATE | transition character to combat-ready
  ;
  ; Evaluates the encounter at the character's position.  If no
  ; hostile is co-located (result != 2), the restricted flag is
  ; set to limit the character's options.  State is set to 2
  ; (combat ready).
  ;
  ; In:  ENTITY_PTR = current character record
  ; Out: char state = 2; restricted flag set if no hostile present
  ; ----------------------------------------------------------------
  ENTER_COMBAT_STATE:
      SUBROUTINE ENTER_COMBAT_STATE
      ; Transition the current entity to combat-ready state (state 2).
      ;
      ; Behavior:
      ;   Evaluates the encounter at the entity's position, clears the
      ;   restricted flag, then re-sets it if no hostile is co-located.
      ;   Sets the character state to 2 (combat ready).
      ;
      ; Clobbers: A, Y

      ; Transition entity to combat-ready state (state 2).
      ; Checks encounter, clears/sets restricted flag.
      ; Clobbers: A, Y
      JSR     CHECK_ENCOUNTER         ; evaluate encounter at position
      JSR     CLEAR_RESTRICTED        ; start with restriction cleared
      LDA     #$02
      BANE    ENCOUNTER_RESULT,.skip  ; hostile co-located? skip
      JSR     SET_RESTRICTED          ; no hostile -> restrict options
  .skip:
      LDA     #$02                    ; state 2 = combat ready
      JSR     SET_CHAR_STATE
      RTS
```

Defines:
ENTER_COMBAT_STATE, used in chunks 232, 237, and 266.
Uses CHECK_ENCOUNTER 312, CLEAR_RESTRICTED 245, ENCOUNTER_RESULT 226a, ENTITY_PTR 561, SET_CHAR_STATE 242, and SET_RESTRICTED 246.

START_COMBAT advances the character to active attack state (state 3), loads the combat target pointer from TARGET_PTR/TARGET_PTR+1 via LOAD_TARGET_PTR, and jumps to RESOLVE_ATTACK where the actual willingness check and damage calculation take place.

328 ⟨*start combat* 328⟩≡ (909)

```
      ORG     $14CD
  ; ---------------------------------------------------------------
  ; START_COMBAT | begin active attack
  ;
  ; Sets char state to 3 (attacking), loads the pre-selected
  ; combat target into TARGET_REC, and jumps to RESOLVE_ATTACK
  ; for the willingness check and damage calculation.
  ;
  ; In:  ENTITY_PTR = current character record
  ;      TARGET_PTR/HI = pre-selected target address
  ; Out: char state = 3; control transfers to RESOLVE_ATTACK
  ; ---------------------------------------------------------------
  START_COMBAT:
      SUBROUTINE START_COMBAT
      ; Enter active attack (state 3) and resolve attack.
      ; Input: TARGET_PTR/HI = pre-selected target
      ; Clobbers: A, X, Y
      LDA     #$03                    ; state 3 = attacking
      JSR     SET_CHAR_STATE
      JSR     LOAD_TARGET_PTR         ; TARGET_REC <- target address
      JMP     RESOLVE_ATTACK          ; perform attack resolution
```

Defines:
START_COMBAT, used in chunks 232 and 237.

Uses ENTITY_PTR 561, LOAD_TARGET_PTR 370a, RESOLVE_ATTACK 338, SET_CHAR_STATE 242, TARGET_PTR 366, and TARGET_REC 618b.

8.2.1 Attack direction handlers

The attack direction menu (ATTACK_DIR_TABLE) has four direction handlers that adjust CURRENT_COL or CURRENT_ROW by ±1, then fall through to FIND_ATTACK_TARGET. Each handler undoes the adjustment if it causes the value to go negative (underflow past zero wraps to $FF, which is >$7F).

- ATTACK_RIGHT: increment column
- ATTACK_LEFT: decrement column
- ATTACK_UP: decrement row
- ATTACK_DOWN: increment row

329 ⟨*attack direction handlers* 329⟩≡ (909)

```
    ORG     $14D8
ATTACK_RIGHT:
    INC     CURRENT_COL
    BPL     FIND_ATTACK_TARGET ; positive -> valid, else fall through undoes it

ATTACK_LEFT:
    DEC     CURRENT_COL
    BPL     FIND_ATTACK_TARGET ; positive -> valid, else fall through undoes it

ATTACK_UP:
    DEC     CURRENT_ROW
    BPL     FIND_ATTACK_TARGET ; positive -> valid, else fall through undoes it

ATTACK_DOWN:
    INC     CURRENT_ROW        ; fall through to FIND_ATTACK_TARGET
```

Defines:
ATTACK_DOWN, used in chunk 266.
ATTACK_LEFT, used in chunk 266.
ATTACK_RIGHT, used in chunk 266.
ATTACK_UP, used in chunk 266.
Uses CURRENT_COL 472b, CURRENT_ROW 473a, and FIND_ATTACK_TARGET 330.

8.2.2 Find attack target

FIND_ATTACK_TARGET converts CURRENT_COL/CURRENT_ROW to a map position via COLROW_TO_POS, then calls FIND_ENTITY_AT_POS to see what entity occupies that square.

- If no entity is found ($BF = 0) or a player character is found (bit 7 of $BF clear), it sets IS_PLAYER_TURN = 1, stores the entity pointer into MOB_PTR, and jumps to RESOLVE_ATTACK.
- If a mob is found (bit 7 of $BF set), it sets IS_PLAYER_TURN = 0, strips bit 7, stores the pointer into MOB_PTR, and calls FIND_MOB_AT_SAME_POS to check if there are multiple mobs at the position. If there are (A≠0), it calls NEXT_COLOCATED_MOB twice to cycle through candidates, then enters the attack target menu at MENU_ATTACK_TARGET. If only one mob, it jumps directly to RESOLVE_ATTACK.

330 ⟨*find attack target* 330⟩≡ (909)

```
    ORG     $14EA
FIND_ATTACK_TARGET:
    SUBROUTINE FIND_ATTACK_TARGET
    ; Convert direction col/row to position, find entity, dispatch.
    ; Input: CURRENT_COL/ROW = target square
    ; Output: IS_PLAYER_TURN set, MOB_PTR loaded, jumps to RESOLVE_ATTACK
    ; Clobbers: A, X, Y
    LDA     CURRENT_COL
    LDY     CURRENT_ROW
    JSR     COLROW_TO_POS      ; A = map position
    JSR     FIND_ENTITY_AT_POS ; $BE/$BF = entity pointer
    LDA     $BF
    BEQ     .player            ; $BF = 0 -> no entity / player
    CMP     #$80
    BPL     .mob               ; bit 7 set -> mob

.player:
    LDA     #$01
    STA     IS_PLAYER_TURN
    LDA     $BE
    STA     MOB_PTR
    LDA     $BF
    STA     MOB_PTR+1
    JMP     RESOLVE_ATTACK

.mob:
    LDA     #$00
    STA     IS_PLAYER_TURN
    LDA     $BE
    STA     MOB_PTR
    LDA     $BF
```

```
    AND     #$7F                ; strip bit 7 (mob indicator)
    STA     MOB_PTR+1
    JSR     FIND_MOB_AT_SAME_POS ; check for multiple mobs
    CMP     #$00
    BNE     .multiple           ; nonzero -> multiple mobs here
    JMP     RESOLVE_ATTACK      ; only one -> attack directly

.multiple:
    JSR     NEXT_COLOCATED_MOB ; cycle to first candidate
    JSR     NEXT_COLOCATED_MOB ; cycle to second candidate
    CMP     #$00
    BNE     .choose
    JMP     RESOLVE_ATTACK      ; no valid candidate -> attack directly

.choose:
    JMP     MENU_ATTACK_TARGET ; present target selection menu
```

Defines:
FIND_ATTACK_TARGET, used in chunks 265 and 329.
Uses COLROW_TO_POS 600a, CURRENT_COL 472b, CURRENT_ROW 473a, FIND_ENTITY_AT_POS 469, FIND_MOB_AT_SAME_POS 315, IS_PLAYER_TURN 474, MENU_ATTACK_TARGET 272, MOB_PTR 608, NEXT_COLOCATED_MOB 334, and RESOLVE_ATTACK 338.

8.2.3 Init attack target display

INIT_ATTACK_DISPLAY is the initialization callback for the attack target menu (ATTACK_TARGET_TABLE). It reads the target's name from the entity record at MOB_PTR and copies it into the display buffer at ATTACK_NAME_BUF. Byte 0 of the record gives the name length; bytes 1–20 are the name characters. Positions beyond the name length are filled with spaces ($20).

331 ⟨*defines* 25b⟩+≡ (909) ◁275a 336▷

```
ATTACK_NAME_BUF     EQU     $7C9B   ; 20-byte buffer for attack target name
```

Defines:
ATTACK_NAME_BUF, used in chunk 332.

332 ⟨*init attack display* 332⟩≡ (909)

```
    ORG     $1537
INIT_ATTACK_DISPLAY:
    SUBROUTINE INIT_ATTACK_DISPLAY
    ; Copy target name to ATTACK_NAME_BUF for menu display.
    ; Input: MOB_PTR = target entity
    ; Clobbers: A, X, Y
    LDY     #$00
    LDA     (MOB_PTR),Y        ; byte 0 = record pointer lo
    STA     $BC
    INY
    LDA     (MOB_PTR),Y        ; byte 1 = record pointer hi
    STA     $BD
    LDY     #$00
    LDA     ($BC),Y            ; name length
    TAX
    LDA     #<ATTACK_NAME_BUF
    STA     $BA
    LDA     #>ATTACK_NAME_BUF
    STA     $BB
.copy:
    INY
    LDA     ($BC),Y            ; name character
    DEX
    BPL     .store             ; X >= 0 -> still within name
    LDA     #$20               ; past end -> space
.store:
    STA     ($BA),Y
    CPY     #$14               ; 20 characters
    BNE     .copy
    RTS
```

Defines:
INIT_ATTACK_DISPLAY, used in chunk 272.
Uses ATTACK_NAME_BUF 331 and MOB_PTR 608.

8.2.4 Someone else handler and attack eligibility

SELECT_SOMEONE_ELSE is the handler for the "SOMEONE ELSE" option in the attack target menu. It calls NEXT_COLOCATED_MOB to advance to the next mob at the same position, then re-enters the attack target menu.

CHECK_ATTACK_ELIGIBLE checks whether a hostile mob is co-located with the current entity. It copies ENTITY_PTR to TARGET_REC and MOB_PTR, then repeatedly calls FIND_MOB_AT_SAME_POS and CHECK_HOSTILE to scan co-located mobs. Returns carry set if a hostile mob is at the same position (attack it directly), carry clear if none found (player must choose a direction).

NEXT_COLOCATED_MOB advances MOB_PTR to the next mob at the same map position. It saves byte 2 (next link) of the current mob, calls FIND_MOB_AT_SAME_POS, and checks if the position actually changed. If the position is unchanged, it returns A=0 (no new mob). If byte 4 (appearance) is ≥$15, the mob is a valid attackable target and it returns A=1. Otherwise it retries up to the loop limit.

333 ⟨*var saved mob link* 333⟩≡ (909)

```
    ORG     $5A75
SAVED_MOB_LINK:
    DC.B    $8D                     ; initial value (runtime: temp: saved byte 2 of mob record)
```

Defines:
SAVED_MOB_LINK, used in chunk 334.

334 ⟨*attack eligibility* 334⟩≡ (909)

```
        ORG     $155E
    SELECT_SOMEONE_ELSE:
        ; "SOMEONE ELSE" menu handler: advance to next colocated mob.
        ; Input:  MOB_PTR | current mob
        ; Output: jumps to MENU_ATTACK_TARGET
        ; Clobbers: A, X, Y

        JSR     NEXT_COLOCATED_MOB      ; advance to next mob
        JMP     MENU_ATTACK_TARGET      ; re-enter target menu

    CHECK_ATTACK_ELIGIBLE:
        SUBROUTINE CHECK_ATTACK_ELIGIBLE
        ; Check if a hostile mob is at the entity's position.
        ;
        ; Input:  ENTITY_PTR | entity to check
        ; Output: C=1 hostile found (MOB_PTR set to it)
        ;         C=0 no hostile at position
        ; Modifies: MOB_PTR, TARGET_REC
        ; Clobbers: A, X, Y

        LDA     ENTITY_PTR              ; \ copy ENTITY_PTR
        STA     TARGET_REC              ;  | to TARGET_REC
        STA     MOB_PTR                 ;  | and MOB_PTR
        LDA     ENTITY_PTR+1            ;  |
        STA     TARGET_REC+1            ;  |
        STA     MOB_PTR+1               ; /
    .scan:
        JSR     FIND_MOB_AT_SAME_POS
        LDA     ENTITY_PTR              ; \ check if pointer changed
        CMP     MOB_PTR                 ;  |
        BNE     .found                  ;  |
        LDA     ENTITY_PTR+1            ;  |
        CMP     MOB_PTR+1               ;  |
        BEQ     .none                   ; / same pointer -> none found
    .found:
        JSR     CHECK_HOSTILE           ; A=0 if friendly
        CMP     #$00
        BEQ     .scan                   ; not hostile -> keep scanning
        SEC                             ; hostile found
        RTS
    .none:
        CLC                             ; no hostile found
        RTS

    NEXT_COLOCATED_MOB:
        SUBROUTINE NEXT_COLOCATED_MOB
        ; Advance MOB_PTR to the next valid NPC at the same position.
        ;
        ; Input:  MOB_PTR | current mob
```

```
    ; Output: A=1 valid target found, A=0 no new mob
    ;         MOB_PTR updated if found
    ; Modifies: MOB_PTR, SAVED_MOB_LINK
    ; Clobbers: A, X, Y

    LDY     #MFIELD_NEXT            ; byte 2: next link
    LDA     (MOB_PTR),Y
    STA     SAVED_MOB_LINK          ; save current link
.retry:
    JSR     FIND_MOB_AT_SAME_POS
    LDY     #MFIELD_NEXT            ; byte 2: new mob's link
    LDA     (MOB_PTR),Y
    CMP     SAVED_MOB_LINK          ; same as saved?
    BNE     .check                  ; different -> check validity
    LDA     #$00                    ; wrapped around, none found
    RTS
.check:
    LDY     #MFIELD_APPEARANCE      ; byte 4: appearance
    LDA     (MOB_PTR),Y
    CMP     #NPC_APPEARANCE_MIN     ; >= $15 = valid NPC target?
    BCC     .retry                  ; below threshold -> skip
    LDA     #$01                    ; valid target found
    RTS
```

Defines:
CHECK_ATTACK_ELIGIBLE, used in chunk 265.
NEXT_COLOCATED_MOB, used in chunk 330.
SELECT_SOMEONE_ELSE, used in chunk 272.

Uses CHECK_HOSTILE 317, ENTITY_PTR 561, FIND_MOB_AT_SAME_POS 315, MENU_ATTACK_TARGET 272, MFIELD_APPEARANCE 561, MFIELD_NEXT 561, MOB_PTR 608, SAVED_MOB_LINK 333, and TARGET_REC 618b.

8.2.5 Combat willingness and attack resolution

RESOLVE_ATTACK determines whether an entity commits to an attack or hesitates. Player characters always attack; AI entities compute a *willingness score* that is tested against a random roll.

Willingness score. For players, willingness is fixed at 28. For AI, it starts at the entity's level (byte 11, bits 4–0) and accumulates modifiers:

Condition	Source	Delta
Attacker rage flag (bit 5 of byte 11)	attacker	+24
Target cautious flag (bit 6 of byte 11)	target	−4
Target aggressive flag (bit 7 of byte 11)	target	+8
Target combat-engaged (bit 2 of byte 15)	target	+6
Target idle (state 0, bits 1–0 of byte 15)	target	+3
Target guarding (state 1)	target	−1
Target retreating (state 3)	target	−3
Target critically wounded (HP < 3)	target	+24
Defense differential (same position only)	both	attacker − target

The final score is clamped to 0–30.

Attack roll. A random number 0–30 is generated. If the roll is at or below the willingness score, the attack proceeds to PLAYER_ATTACK or MOB_ATTACK depending on IS_PLAYER_TURN. Otherwise the attacker hesitates and "MISSES" is displayed.

Rage consumption. When the rage flag (bit 5) is set, the routine grants the +24 bonus and then clears bit 4 of byte 11, consuming the rage for future turns.

336 ⟨*defines* 25b⟩+≡ (909) ◁331 388a▷

```
    ; --- Byte 11 (MFIELD_LEVEL) flag bit masks ---
LEVEL_BIT5_MASK     EQU     $20     ; byte 11 bit 5: rage/berserk flag
LEVEL_CLEAR_BIT4    EQU     $EF     ; byte 11: clear bit 4 (consume rage)
TARGET_BIT6_MASK    EQU     $40     ; byte 11 bit 6: cautious flag
TARGET_BIT7_MASK    EQU     $80     ; byte 11 bit 7: aggressive flag
    ; --- Byte 15 (MFIELD_FLAGS) bit masks ---
FLAGS_ENGAGED_BIT   EQU     $04     ; byte 15 bit 2: combat engaged
FLAGS_STATE_MASK    EQU     $03     ; byte 15 bits 1-0: combat state (0-3)
    ; --- Byte 5 defense mask ---
DEFENSE_HALF_MASK   EQU     $0F     ; byte 5: defense (bits 4-1 shifted down via ROR)
    ; --- Scratch ---
```

Defines:
DEFENSE_HALF_MASK, used in chunks 320, 325, 338, and 346.
FLAGS_ENGAGED_BIT, used in chunks 245, 246, 322, 338, and 349.
FLAGS_STATE_MASK, used in chunks 242 and 338.
LEVEL_BIT5_MASK, used in chunk 338.
LEVEL_CLEAR_BIT4, used in chunk 338.
TARGET_BIT6_MASK, used in chunk 338.
TARGET_BIT7_MASK, used in chunks 325 and 338.
Uses MFIELD_LEVEL 561.

337a ⟨*var combat willingness* 337a⟩≡ (909)

```
    ORG     $5A76
COMBAT_WILLINGNESS:
    DC.B    $A0                     ; initial value (runtime: accumulated willingness-to-fight score
```

Defines:
COMBAT_WILLINGNESS, used in chunks 338 and 372b.

337b ⟨*var target defense* 337b⟩≡ (909)

```
    ORG     $5A77
TARGET_DEFENSE:
    DC.B    $CC                     ; initial value (runtime: scratch: target's defense value)
```

Defines:
TARGET_DEFENSE, used in chunk 338.

338 ⟨*resolve attack* 338⟩≡ (909)

```
    ORG     $15AB
RESOLVE_ATTACK:
    SUBROUTINE RESOLVE_ATTACK
    ; Compute combat willingness and resolve attack or hesitation.
    ;
    ; Input:  ENTITY_PTR     | attacker record
    ;         MOB_PTR        | target record
    ;         IS_PLAYER_TURN | 1 = player, 0 = AI
    ;
    ; Output: branches to PLAYER_ATTACK or MOB_ATTACK on hit;
    ;         prints "MISSES" and returns on hesitation.
    ;
    ; Behavior:
    ;   Player willingness is fixed at 28.  AI willingness starts
    ;   at the entity's level (byte 11 bits 4-0) and accumulates
    ;   modifiers from flag bits, combat state, HP, and defense.
    ;   Clamped to 0-30, then random roll 0-30 determines hit.
    ;
    ; Modifies: COMBAT_WILLINGNESS, TARGET_DEFENSE, entity byte 11
    ; Clobbers: A, X, Y

    ; --- set blink cursor to target ---
    JSR     SET_BLINK_TARGET

    ; --- player: fixed willingness ---
    LDA     IS_PLAYER_TURN
    BANE    #$01,.not_player
    STOB    #$1C,COMBAT_WILLINGNESS ; player = 28 (always willing)
    JMP     .clamp_done

    ; --- AI: base level ---
.not_player:
    LDY     #MFIELD_LEVEL           ; byte 11: flags | level
    LDA     (ENTITY_PTR),Y
    TAX
    AND     #LEVEL_MASK             ; level (bits 4-0) = base willingness
    STA     COMBAT_WILLINGNESS

    ; --- AI: attacker flag bits (byte 11) ---
    TXA
    AND     #LEVEL_BIT5_MASK        ; bit 5: rage flag
    BEQ     .no_bit5
    TXA
    AND     #LEVEL_CLEAR_BIT4       ; consume rage: clear bit 4
    STA     (ENTITY_PTR),Y
    LDA     #$18                    ; +24 (rage bonus)
    JSR     ADD_WILLINGNESS
.no_bit5:
```

```
    ; --- AI: target flag bits (byte 11) ---
    LDA     (MOB_PTR),Y           ; target byte 11
    TAY
    AND     #TARGET_BIT6_MASK      ; bit 6: cautious
    BEQ     .no_bit6
    LDA     #$FC                   ; -4 (target cautious)
    JSR     ADD_WILLINGNESS
.no_bit6:
    TYA
    AND     #TARGET_BIT7_MASK      ; bit 7: aggressive
    BEQ     .no_bit7
    LDA     #$08                   ; +8 (target aggressive)
    JSR     ADD_WILLINGNESS

    ; --- AI: target combat state (byte 15) ---
.no_bit7:
    LDY     #MFIELD_FLAGS          ; byte 15: status flags
    LDA     (MOB_PTR),Y
    TAX
    AND     #FLAGS_ENGAGED_BIT     ; bit 2: combat engaged
    BEQ     .no_restrict
    LDA     #$06                   ; +6 (already fighting)
    JSR     ADD_WILLINGNESS
.no_restrict:
    TXA
    AND     #FLAGS_STATE_MASK      ; bits 1-0: combat state
    BAPL    #$01,.ge1
    LDA     #$03                   ; +3 (state 0: idle)
    JSR     ADD_WILLINGNESS
    JMP     .state_done
.ge1:
    BNE     .ne1
    LDA     #$FF                   ; -1 (state 1: guarding)
    JSR     ADD_WILLINGNESS
    JMP     .state_done
.ne1:
    BANE    #$03,.state_done
    LDA     #$FD                   ; -3 (state 3: retreating)
    JSR     ADD_WILLINGNESS

    ; --- AI: target HP check ---
.state_done:
    LDY     #MFIELD_HP       ; byte 6: HP in bits 5-0
    LDA     (MOB_PTR),Y
    AND     #HP_MASK
    BAGE    #$03,.hp_ok            ; HP >= 3 -> not critical
    LDA     #$18                   ; +24 (target critically wounded)
    JSR     ADD_WILLINGNESS

    ; --- AI: defense differential (same position only) ---
```

```
.hp_ok:
    LDY     #MFIELD_POSITION       ; byte 3: map position
    LDA     (ENTITY_PTR),Y
    CMP     (MOB_PTR),Y
    BNE     .diff_pos              ; not co-located -> skip
    LDY     #MFIELD_STRENGTH       ; byte 5: defense in bits 4-1
    LDA     (MOB_PTR),Y           ; \ target defense
    ROR                            ;  | shift bits 4-1 to 3-0
    AND     #DEFENSE_HALF_MASK     ;  |
    STA     TARGET_DEFENSE         ; /
    LDA     (ENTITY_PTR),Y        ; \ attacker defense
    ROR                            ;  | shift bits 4-1 to 3-0
    AND     #DEFENSE_HALF_MASK     ; /
    SEC
    SBC     TARGET_DEFENSE         ; delta = attacker - target
    JSR     ADD_WILLINGNESS

    ; --- clamp willingness to 0-30 ---
.diff_pos:
    LDA     COMBAT_WILLINGNESS
    BPL     .not_neg
    STOB    #$00,COMBAT_WILLINGNESS ; floor at 0
    JMP     .clamp_done
.not_neg:
    BAMI    #$1F,.clamp_done
    STOB    #$1E,COMBAT_WILLINGNESS ; ceiling at 30

    ; --- random roll: attack or hesitate ---
.clamp_done:
    LDA     #$1F                   ; range 0-30
    JSR     RANDOM_IN_RANGE
    BAMI    COMBAT_WILLINGNESS,.attack ; roll < willingness -> hit
    BEQ     .attack                ; roll == willingness -> hit

    ; --- hesitate: print "[name] MISSES" ---
    JSR     SET_CURSOR_ROW21
    JSR     PRINT_MOB_NAME
    JSR     SCROLL_STATUS_LINE
    STOW    S_MISSES,TARGET_REC
    JSR     PRINT_BOTTOM_CENTERED
    JSR     SCROLL_STATUS_LINE
    LDA     #$01
    BAEQ    IS_PLAYER_TURN,.is_player
    JSR     PRINT_MEMBER_NAME
.is_player:
    JSR     DELAY_WITH_ANIMATION
    RTS

    ; --- dispatch attack ---
.attack:
```

```
    LDA     IS_PLAYER_TURN
    BEQ     .mob_attack
    JMP     PLAYER_ATTACK
.mob_attack:
    JMP     MOB_ATTACK
```

Defines:
RESOLVE_ATTACK, used in chunks 272, 328, and 330.
Uses ADD_WILLINGNESS 372b, COMBAT_WILLINGNESS 337a, DEFENSE_HALF_MASK 336, DELAY_WITH_ANIMATION 603b, ENTITY_PTR 561, FLAGS_ENGAGED_BIT 336, FLAGS_STATE_MASK 336, IS_PLAYER_TURN 474, LEVEL_BIT5_MASK 336, LEVEL_CLEAR_BIT4 336, LEVEL_MASK 411a, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MOB_ATTACK 346, MOB_PTR 608, PLAYER_ATTACK 342, PRINT_BOTTOM_CENTERED 628, PRINT_MEMBER_NAME 485, PRINT_MOB_NAME 484, RANDOM_IN_RANGE 597, SCROLL_STATUS_LINE 639, SET_BLINK_TARGET 475, SET_CURSOR_ROW21 627, STOB 8c, STOW 8a, S_MISSES 310, TARGET_BIT6_MASK 336, TARGET_BIT7_MASK 336, TARGET_DEFENSE 337b, and TARGET_REC 618b.

8.2.6 Player attack resolution

PLAYER_ATTACK resolves an attack initiated by the player character. The routine computes combat strength, classifies the target, and then branches into one of several resolution paths depending on the target's type and state.

Setup. The routine computes the attacker's combat strength via CALC_COMBAT_STRENGTH, displays the attacker's name followed by a strength-based descriptor from MAP_VALUE_TO_STRING, copies MOB_PTR to ENTITY_REC, and plays tune 2 with the combat strength as a parameter.

Target classification. The target entity is classified by examining its record fields:

- If byte 1 is $FE (dead/empty), the routine falls through to event-slot processing.
- If byte 2 has high bits equal to $C0, the target is an activated event slot. Small sub-type values trigger a scene; larger values clear the slot.
- Otherwise the target is a normal entity and the routine proceeds to attack resolution.

Locked containers. If the target has a type flag indicating a locked container and the player's strength is below the required threshold, the attack is blocked with message $10.

Hit roll. For normal targets, a random number 0–12 is compared against the combat strength:

- Roll ≥ strength: attack blocked ("immune").
- Roll < strength: hit confirmed — bit 6 of byte 1 is set (damage taken), the entity is redrawn, tune $0C plays, and a hit confirmation is displayed.

342 ⟨*player attack* 342⟩≡ (909)

```
    ORG     $1751
PLAYER_ATTACK:
    SUBROUTINE PLAYER_ATTACK
    ; Resolve player-initiated attack on target entity.
    ;
    ; Input:  MOB_PTR | target, ENTITY_PTR | attacker
    ; Behavior:
    ;   Computes combat strength, classifies target (dead/event/
    ;   normal), rolls hit (0-12 vs strength), handles locked
    ;   containers and event slot processing.
    ; Clobbers: A, X, Y

    ; --- setup: strength, name, tune ---
    JSR     CALC_COMBAT_STRENGTH
    JSR     SET_CURSOR_ROW21
    JSR     PRINT_MOB_NAME
    JSR     SCROLL_STATUS_LINE
    LDA     COMBAT_STRENGTH
    JSR     MAP_VALUE_TO_STRING    ; strength descriptor
    JSR     SCROLL_STATUS_LINE
    MOVW    MOB_PTR,ENTITY_REC     ; ENTITY_REC = target
    JSR     CLASSIFY_ENTITY_TYPE
    JSR     PRINT_BOTTOM_CENTERED
    LDY     COMBAT_STRENGTH
    LDX     #$02                   ; tune 1: combat hit
    JSR     TUNE_PLAYER

    ; --- target classification ---
    LDY     #MFIELD_TYPE
    LDA     (MOB_PTR),Y           ; target type byte
    BAEQ    #EVENT_EMPTY,.check_event ; $FE = dead/empty
    BALT    #$40,.roll_hit         ; < $40 = normal entity
.immune:
    JSR     DISPATCH_ON_MODE
    JSR     SCROLL_STATUS_LINE
    LDA     #MSG_TOUCHED           ; "as soon as it is touched"
    JMP     DISPLAY_MESSAGE

    ; --- hit roll ---
.roll_hit:
    LDA     #$0D                   ; range 0-12
```

```
    JSR     RANDOM_IN_RANGE
    BAGE    COMBAT_STRENGTH,.immune ; roll >= strength: miss
    JSR     DISPATCH_ON_MODE
    LDY     #MFIELD_TYPE            ; \ set damage-taken flag (bit 6)
    LDA     (ENTITY_REC),Y          ;  |
    ORA     #$40                    ;  |
    STA     (ENTITY_REC),Y          ; /
    DEY                             ; Y = MFIELD_CHAR (byte 0)
    LDA     (ENTITY_REC),Y
    JSR     POS_TO_COLROW
    STY     FONT_ROW
    STA     FONT_COL
    JSR     GET_ENTITY_FONTCHAR
    JSR     RENDER_FONT_CHAR        ; redraw entity
    LDX     #$0C                    ; tune 6: hit confirmed
    JSR     TUNE_PLAYER
    JSR     SET_CURSOR_ROW21
    LDA     #MSG_INSERT_DISK        ; hit confirmation
    JMP     DISPLAY_MESSAGE

    ; --- event slot processing ---
.check_event:
    LDY     #MFIELD_NEXT            ; byte 2: event data
    LDA     (MOB_PTR),Y
    TAX
    AND     #FLAGS_CLASS_MASK       ; isolate activation bits
    BANE    #FLAGS_CLASS_MASK,.check_type ; $C0 = activated event
    TXA
    AND     #STRENGTH_MASK          ; sub-type (bits 4-0)
    BAPL    #$1D,.immune            ; >= $1D: immune
    JSR     PROCESS_ENTITY_SCRIPT
    JSR     SET_CURSOR_ROW21
    LDA     #$02                    ; scene 2
    JMP     SCENE_LOOP

    ; --- container / locked object ---
.check_type:
    BAEQ    #$00,.immune            ; no activation: immune
    LDA     COMBAT_STRENGTH
    BAPL    #$0A,.strong_enough     ; strength >= 10: can open
    TXA
    BAPL    #$80,.blocked           ; bit 7: locked container
    LDA     COMBAT_STRENGTH
    BAMI    #$04,.blocked           ; strength < 4: too weak
.strong_enough:
    TXA
    AND     #STRENGTH_MASK          ; sub-type (bits 4-0)
    BAMI    #STRENGTH_MASK,.take_item ; sub-type = max: take item
    STX     $BA                     ; \ decrement counter
    DEC     $BA                     ;  |
```

```
    LDA     $BA                     ;  |
    LDY     #MFIELD_NEXT            ;  |
    STA     (MOB_PTR),Y             ; / store decremented event byte
    LDX     #$0E                    ; tune 7: interaction
    JSR     TUNE_PLAYER
    JSR     DISPATCH_ON_MODE
    LDY     #MFIELD_CHAR            ; byte 0: display char
    LDA     (MOB_PTR),Y
    JSR     DRAW_CHAR_AT_POS        ; redraw at position
    JSR     SET_CURSOR_ROW21
.show_found:
    LDA     #MSG_TOPPLES            ; "the sculpture topples over"
    JSR     DISPLAY_MESSAGE
    JMP     EVENT_PRNG_CHECK
.take_item:
    JSR     PROCESS_ENTITY_SCRIPT
    JSR     SET_CURSOR_ROW21
    LDY     #MFIELD_NEXT            ; byte 2: event data
    LDA     (MOB_PTR),Y
    AND     #STRENGTH_MASK          ; sub-type
    BAPL    #$1B,.show_found        ; >= $1B: show found
    LDA     #$03                    ; scene 3
    JSR     SCENE_LOOP
    JMP     EVENT_PRNG_CHECK
.blocked:
    JSR     DISPATCH_ON_MODE
    JSR     SET_CURSOR_ROW21
    LDA     #MSG_HEAVY_ROCK         ; "a heavy piece of rock"
    JMP     DISPLAY_MESSAGE
```

Defines:
PLAYER_ATTACK, used in chunk 338.

Uses CALC_COMBAT_STRENGTH 325, CLASSIFY_ENTITY_TYPE 459, COMBAT_STRENGTH 324, DISPATCH_ON_MODE 607b, DISPLAY_MESSAGE 637, DRAW_CHAR_AT_POS 471, ENTITY_PTR 561, ENTITY_REC 541b, EVENT_EMPTY 397a, EVENT_PRNG_CHECK 360, FONT_COL 492a, FONT_ROW 534c, GET_ENTITY_FONTCHAR 462, MAP_VALUE_TO_STRING 375, MFIELD_NEXT 561, MOB_PTR 608, MOVW 9a, MSG_HEAVY_ROCK 634, MSG_INSERT_DISK 634, MSG_TOPPLES 634, MSG_TOUCHED 634, POS_TO_COLROW 599, PRINT_BOTTOM_CENTERED 628, PRINT_MOB_NAME 484, PROCESS_ENTITY_SCRIPT 371, RANDOM_IN_RANGE 597, RENDER_FONT_CHAR 466, SCENE_LOOP 527, SCROLL_STATUS_LINE 639, SET_CURSOR_ROW21 627, and TUNE_PLAYER 682.

8.2.7 Mob attack resolution

MOB_ATTACK resolves an attack from a mob against the current character. The overall structure mirrors PLAYER_ATTACK: compute combat strength, display names, play tune 2, and call DISPATCH_ON_MODE.

Defense calculation. The key difference from PLAYER_ATTACK is in defense handling. The routine reads byte 8 of the target (via MOB_PTR) and uses its low 4 bits as a defense value:

- Defense = 0: full combat strength applied via APPLY_DAMAGE_TO_MOB.
- Defense > 0: subtracted from strength. A zero or negative result means the attack is absorbed, with message $19/$1A ("blocked") or $34 for critical blocks when HP < 3.
- Positive remainder becomes effective damage, displayed with message $17 or $18 depending on byte 8's sign bit.

Player character special case. For player characters (appearance below NPC_APPEARANCE_MIN), SCENE_COMBAT_FLAG is cleared to signal that the player is under attack.

345 ⟨*var mob defense* 345⟩≡ (909)

```
    ORG     $5A79
MOB_DEFENSE:
    DC.B    $D9                     ; initial value (runtime: mob defense value (low 4 bits of mob b
```

Defines:
MOB_DEFENSE, used in chunk 346.

346 ⟨*mob attack* 346⟩≡ (909)

```
    ORG     $1860
MOB_ATTACK:
    SUBROUTINE MOB_ATTACK
    ; Resolve a mob's attack against the current character.
    ;
    ; Input:  ENTITY_PTR | target, MOB_PTR | attacker
    ; Behavior:
    ;   Computes strength, compares vs defense (byte 8 low 4 bits).
    ;   Defense=0: full damage. Strength>defense: partial hit.
    ;   Strength<=defense: absorbed (or critical if HP<3).
    ; Modifies: COMBAT_STRENGTH, MOB_DEFENSE, SCENE_COMBAT_FLAG
    ; Clobbers: A, X, Y

    ; --- setup: strength, names, tune ---
    JSR     CALC_COMBAT_STRENGTH
    JSR     SET_CURSOR_ROW21
    JSR     PRINT_MOB_NAME
    JSR     SCROLL_STATUS_LINE
    LDA     COMBAT_STRENGTH
    JSR     MAP_VALUE_TO_STRING    ; strength descriptor
    JSR     SCROLL_STATUS_LINE
    JSR     PRINT_MEMBER_NAME      ; defender name
    LDY     COMBAT_STRENGTH
    LDX     #$02                   ; tune 1: combat hit
    JSR     TUNE_PLAYER

    ; --- player character special case ---
    LDY     #MFIELD_APPEARANCE
    LDA     (ENTITY_PTR),Y
    BAGE    #NPC_APPEARANCE_MIN,.npc ; NPC -> skip
    STOB    #$00,SCENE_COMBAT_FLAG  ; player under attack
.npc:
    JSR     DISPATCH_ON_MODE

    ; --- read defense from byte 8 ---
    LDY     #MFIELD_XP             ; byte 8: status/XP/count
    LDA     (MOB_PTR),Y
    STA     MOB_DEFENSE            ; save full byte
    AND     #DEFENSE_HALF_MASK     ; low 4 bits = defense
    BANE    #$00,.has_defense
    LDA     COMBAT_STRENGTH        ; defense=0: full damage
    JMP     APPLY_DAMAGE_TO_MOB

    ; --- compare strength vs defense ---
.has_defense:
    JSR     SCROLL_STATUS_LINE
    LDX     #$17                   ; msg $17 (sign bit set)
    LDA     MOB_DEFENSE
    BMI     .negative
```

```
    LDX     #$18                    ; msg $18 (sign bit clear)
.negative:
    LDA     MOB_DEFENSE
    AND     #DEFENSE_HALF_MASK      ; isolate defense
    STA     MOB_DEFENSE
    LDA     COMBAT_STRENGTH
    SEC
    SBC     MOB_DEFENSE             ; strength - defense
    BEQ     .absorbed
    BPL     .partial                ; positive -> partial hit

    ; --- absorbed: attack blocked ---
.absorbed:
    INX                             ; $17->$19 or $18->$1A
    INX
    LDY     #MFIELD_HP        ; byte 6: HP
    LDA     (MOB_PTR),Y
    AND     #HP_MASK
    BAMI    #$03,.critical          ; HP < 3 -> critical block
    TXA
    JMP     DISPLAY_MESSAGE         ; "blocked" message
.critical:
    LDA     #MSG_ENOUGH             ; "ENOUGH IS ENOUGH!"
    JMP     DISPLAY_MESSAGE

    ; --- partial hit: remainder = effective damage ---
.partial:
    STA     COMBAT_STRENGTH         ; save remainder
    TXA
    JSR     DISPLAY_MESSAGE         ; "hit" message
    LDA     COMBAT_STRENGTH
    JMP     APPLY_DAMAGE_TO_MOB  ; apply partial damage
```

Defines:
MOB_ATTACK, used in chunk 338.

Uses APPLY_DAMAGE_TO_MOB 372a, CALC_COMBAT_STRENGTH 325, COMBAT_STRENGTH 324, DEFENSE_HALF_MASK 336, DISPATCH_ON_MODE 607b, DISPLAY_MESSAGE 637, ENTITY_PTR 561, MAP_VALUE_TO_STRING 375, MFIELD_APPEARANCE 561, MFIELD_HP 561, MFIELD_XP 561, MOB_DEFENSE 345, MOB_PTR 608, MSG_ENOUGH 634, PRINT_MEMBER_NAME 485, PRINT_MOB_NAME 484, SCENE_COMBAT_FLAG 207, SCROLL_STATUS_LINE 639, SET_CURSOR_ROW21 627, STOB 8c, and TUNE_PLAYER 682.

8.2.8 Damage application

APPLY_DAMAGE is the central damage-resolution routine. It takes a damage amount in A and the target record pointer in EFFECT_REC.

Damage description. The routine computes a message index by counting the leading zero bits of the damage value (via repeated ROR). Larger damage maps to lower indices, producing increasingly alarming descriptions. Tune 4 (damage sound) is played.

HP reduction. Field 6 of the target record holds HP in bits 5–0 (with flags in bits 7–6). The damage is subtracted. If HP drops to zero or below, execution falls into the death handler.

Death handler. When a character dies:

- Death flag set in field 15.
- Character re-sorted out of group via REORDER_CHAR.
- Gold zeroed via MODIFY_CHAR_STATS.
- UPDATE_EVENT_LIST processes death-triggered events.
- Map position redrawn.

The death message is chosen randomly: 1-in-3 chance of one variant, otherwise a coin flip between two alternatives.

Survival status. If the character survives, a status message is displayed based on remaining HP:

- Below 3: "badly hurt" — the restricted flag is set in field 15, limiting actions on subsequent turns.
- Below 8: "wounded."
- 8 or above: "scratched."

348 ⟨*var damage amount* 348⟩≡ (909)

```
    ORG     $5A10
DAMAGE_AMOUNT:
    DC.B    $CC                     ; initial value (runtime: damage to apply / new HP after subtrac
```

Defines:
DAMAGE_AMOUNT, used in chunk 349.

349 ⟨*apply damage* 349⟩≡ (909)

```
        ORG     $106F
    APPLY_DAMAGE:
        SUBROUTINE APPLY_DAMAGE
        ; Apply damage to entity at (EFFECT_REC).
        ;
        ; Input:  A = raw damage, EFFECT_REC = target record
        ; Behavior:
        ;   Masks damage to 5 bits, displays severity message,
        ;   subtracts from HP (byte 6 bits 5-0).  If HP <= 0:
        ;   death handler.  If survives: status message by HP tier.
        ; Modifies: DAMAGE_AMOUNT, entity bytes 6/15, ACTIVE_CHAR
        ; Clobbers: A, X, Y

        ; --- damage message selection ---
        AND     #STRENGTH_MASK          ; mask to 5-bit damage
        STA     DAMAGE_AMOUNT
        JSR     SET_CURSOR_ROW21
        LDA     DAMAGE_AMOUNT
        LDX     #$03                    ; base message index
    .ror_loop:
        INX                             ; next severity tier
        CLC
        ROR                             ; count leading zeros
        BNE     .ror_loop
        TXA
        JSR     DISPLAY_MESSAGE         ; show damage descriptor

        ; --- HP reduction ---
        LDY     DAMAGE_AMOUNT
        LDX     #$04                    ; tune 2: damage sound
        JSR     TUNE_PLAYER
        LDY     #MFIELD_HP        ; byte 6: HP + flags
        LDA     (EFFECT_REC),Y
        TAX                             ; save original byte
        AND     #HP_MASK                ; isolate HP (bits 5-0)
        SEC
        SBC     DAMAGE_AMOUNT           ; HP - damage
        BCC     .dead                   ; underflow -> dead
        BEQ     .dead                   ; zero -> dead
        STA     DAMAGE_AMOUNT           ; save remaining HP
        TXA
        AND     #FLAGS_CLASS_MASK       ; preserve flags (bits 7-6)
        ORA     DAMAGE_AMOUNT
        STA     (EFFECT_REC),Y          ; store updated byte 6

        ; --- survival status ---
        JSR     SET_CURSOR_ROW21
        LDA     DAMAGE_AMOUNT
        BAPL    #$03,.check8            ; HP >= 3: check next tier
```

```
    LDY     #MFIELD_FLAGS           ; \ set restricted flag
    LDA     (EFFECT_REC),Y          ;  |
    AND     #$F8                    ;  | clear low 3 bits
    ORA     #FLAGS_ENGAGED_BIT      ;  | set bit 2
    STA     (EFFECT_REC),Y          ; /
    JSR     PRINT_ENTITY_NAME
    LDA     #MSG_FEELING_WEAK       ; "is feeling rather weak"
    JSR     DISPLAY_MESSAGE
    RTS
.check8:
    BAPL    #$08,.ge8               ; HP >= 8: minor
    JSR     PRINT_ENTITY_NAME
    LDA     #MSG_LOTS_OF_FIGHT      ; "still has lots of fight"
    JMP     DISPLAY_MESSAGE
.ge8:
    JSR     PRINT_ENTITY_NAME
    LDA     #MSG_HIT_WALL           ; "just a scratch"
    JMP     DISPLAY_MESSAGE

    ; --- death handler ---
.dead:
    LDY     #MFIELD_APPEARANCE      ; byte 4: appearance
    LDA     (EFFECT_REC),Y
    LDX     #$08                    ; tune 4: player death
    BALT    #NPC_APPEARANCE_MIN,.skip_flag ; < $15: player
    LDY     #MFIELD_FLAGS           ; \ set death flag (bit 4)
    LDA     (EFFECT_REC),Y          ;  |
    ORA     #$10                    ;  |
    STA     (EFFECT_REC),Y          ; /
    LDX     #$0A                    ; tune 5: NPC death
.skip_flag:
    TXA
    PHA                             ; save tune index
    JSR     PRINT_ENTITY_NAME

    ; --- random death message ---
    LDA     #$03
    JSR     RANDOM_IN_RANGE         ; random(3)
    BAEQ    #$00,.rand0
    LDA     #$0E                    ; death variant A
    BNE     .show_death
.rand0:
    LDA     #$02
    JSR     RANDOM_IN_RANGE         ; random(2)
    CLC
    ADC     #$27                    ; message $27 or $28
.show_death:
    JSR     DISPLAY_MESSAGE         ; death message
    PLA
    TAX
```

```
JSR     TUNE_PLAYER             ; death tune

; --- death cleanup ---
STOB    #$00,ACTIVE_CHAR
MOVB    CURRENT_PLAYER,SOURCE_CHAR
MOVW    EFFECT_REC,TARGET_REC
JSR     REORDER_CHAR            ; remove from group
LDWIY    EFFECT_REC,#MFIELD_GOLD,TARGET_REC
LDY     #$00                    ; Y=0: subtract mode
JSR     MODIFY_CHAR_STATS       ; zero out gold
LDY     #MFIELD_POSITION
LDA     (EFFECT_REC),Y
JSR     UPDATE_EVENT_LIST       ; process death events
LDY     #MFIELD_POSITION
LDA     (EFFECT_REC),Y
JMP     DRAW_CHAR_AT_POS        ; redraw and return
```

Defines:
APPLY_DAMAGE, used in chunks 352 and 372a.

Uses ACTIVE_CHAR 492c, CURRENT_PLAYER 562a, DAMAGE_AMOUNT 348, DISPLAY_MESSAGE 637, DRAW_CHAR_AT_POS 471, EFFECT_REC 483a, FLAGS_ENGAGED_BIT 336, LDWIY 17, MFIELD_APPEARANCE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_POSITION 561, MODIFY_CHAR_STATS 380, MOVB 8c, MOVW 9a, MSG_FEELING_WEAK 634, MSG_HIT_WALL 634, MSG_LOTS_OF_FIGHT 634, PRINT_ENTITY_NAME 483b, RANDOM_IN_RANGE 597, REORDER_CHAR 478, SET_CURSOR_ROW21 627, SOURCE_CHAR 492b, STOB 8c, TARGET_REC 618b, TUNE_PLAYER 682, and UPDATE_EVENT_LIST 454.

APPLY_DAMAGE_TO_CURRENT is a convenience wrapper that copies ENTITY_PTR into EFFECT_REC and tail-calls APPLY_DAMAGE with the damage amount in A. It saves and restores A across the pointer setup using the X register as a temporary.

352 ⟨*apply damage to current* 352⟩≡ (909)

```
      ORG     $1425
  ; ---------------------------------------------------------------
  ; APPLY_DAMAGE_TO_CURRENT | apply damage to current entity
  ;
  ; Convenience wrapper: copies ENTITY_PTR into EFFECT_REC and
  ; tail-calls APPLY_DAMAGE.  A is preserved across the pointer
  ; copy via X as a temporary.
  ;
  ; In:  A = damage amount
  ;      ENTITY_PTR = target entity record
  ; Out: EFFECT_REC = ENTITY_PTR; falls through to APPLY_DAMAGE
  ; ---------------------------------------------------------------
  APPLY_DAMAGE_TO_CURRENT:
      SUBROUTINE APPLY_DAMAGE_TO_CURRENT
      ; Apply damage A to entity at ENTITY_PTR.
      ; Input: A = damage amount
      ; Output: copies ENTITY_PTR to EFFECT_REC, jumps to APPLY_DAMAGE
      ; Clobbers: X
      TAX                             ; save damage amount in X
      MOVW    ENTITY_PTR,EFFECT_REC   ; target <- current entity
      TXA                             ; restore damage amount
      JMP     APPLY_DAMAGE            ; apply damage to EFFECT_REC
```

Defines:
APPLY_DAMAGE_TO_CURRENT, used in chunks 354, 360, 444, and 447.
Uses APPLY_DAMAGE 349, EFFECT_REC 483a, ENTITY_PTR 561, and MOVW 9a.

When the player steps onto an occupied cell, HANDLE_ENCOUNTER determines what happens. The event type (byte 1 of the event entry) splits the logic into two paths.

For random encounters (type below $40), the character's avoidance stat (low 5 bits of field 5) is compared against a random roll out of 20. If the stat is high enough, the encounter is avoided: byte 1 is ORed with $40, *mutating the event in-place into a specific encounter* with the original byte 1 value as the encounter ID. On the character's next visit the event will fire as a specific encounter rather than a random one. The character is returned to its starting position. Otherwise a second PRNG roll decides whether this is a normal encounter (tune $14) or a rare special encounter (scene 9, which deals 1 damage).

For specific encounters (type $40 and above), the routine extracts a 6-bit encounter ID and a destination position from the event entry, commits the move, reorders the character into the appropriate group, and updates the character's position. If the encounter ID is zero, it is derived from the PRNG.

EVENT_DEST_POS holds the destination position from event byte 2 during specific encounters (type $40 and above). HANDLE_ENCOUNTER writes the character's updated position from this variable after committing the move and reordering the character.

EVENT_PTR is a 2-byte zero-page pointer at ENTITY_REC that points to the current event list entry. HANDLE_ENCOUNTER reads the event type from byte 1 and the destination from byte 2 through this pointer.

353a ⟨*zero page defines* 25a⟩+≡ (909) ◁53c 370b▷

```
EVENT_PTR               EQU     $BE     ; event list entry pointer (2 bytes)
```

Defines:
EVENT_PTR, used in chunks 354, 360, 398, 424, 444, 447, 454, and 898.

Initial wait loop count, used by HANDLE_ENCOUNTER.

353b ⟨*wait loop init* 353b⟩≡ (909)

```
    ORG     $7ABE
WAIT_LOOP_INIT:
    DC.B    $20                     ; initial wait loop count
```

Defines:
WAIT_LOOP_INIT, used in chunk 354.

353c ⟨*var event dest pos* 353c⟩≡ (909)

```
    ORG     $5A5F
EVENT_DEST_POS:
    DC.B    $D3                     ; initial value (runtime: event handler: destination position)
```

Defines:
EVENT_DEST_POS, used in chunk 354.

354 ⟨*handle encounter* 354⟩≡ (909)

```
    ORG     $1C01
HANDLE_ENCOUNTER:
    SUBROUTINE HANDLE_ENCOUNTER
    ; Process encounter when stepping on an event cell.
    ;
    ; Input:  EVENT_PTR | event list entry (3 bytes)
    ;         ENTITY_PTR | character stepping on event
    ; Behavior:
    ;   Type < $40: random encounter with avoidance check.
    ;   Type >= $40: specific encounter with position update.
    ; Clobbers: A, X, Y

    ; --- random encounter path (event type $00-$3F) ---
    LDA     #$00
    JSR     SET_CHAR_STATE              ; state = idle
    LDY     #EVFIELD_TYPE
    LDA     (EVENT_PTR),Y               ; event type byte
    BAPL    #$40,.specific              ; >= $40 -> specific encounter
    LDA     #$14                        ; range = 20
    JSR     RANDOM_IN_RANGE
    STA     PRNG_OUTPUT
    LDY     #MFIELD_STRENGTH
    LDA     (ENTITY_PTR),Y             ; char strength field
    AND     #STRENGTH_MASK              ; bits 4-0 = avoidance stat
    BAMI    PRNG_OUTPUT,.roll_type      ; stat < random -> encounter
    JMP     .avoided                    ; stat >= random -> avoid
.roll_type:
    JSR     STEP_PRNG
    LDA     #$74
    BAPL    PRNG_OUTPUT,.normal ; $74 >= random -> normal encounter
    JMP     .special                    ; rare: special encounter
.normal:
    JSR     SET_CURSOR_ROW21
    LDA     #MSG_DOOR_OPENS
    JSR     DISPLAY_MESSAGE             ; "encounter" message
    LDX     #$14               ; tune 10: encounter effect
    JMP     TUNE_PLAYER

    ; --- specific encounter path (type >= $40) ---
.specific:
    LDY     #EVFIELD_POSITION
    LDA     (EVENT_PTR),Y               ; event position byte
    STA     EVENT_POS
    LDA     #$01                        ; state = engaged
    JSR     SET_CHAR_STATE
    LDY     #EVFIELD_TYPE
    LDA     (EVENT_PTR),Y
    AND     #FLAGS_STATUS_MASK          ; low 6 bits = encounter ID
    BNE     .have_id
```

```
    JSR     STEP_PRNG                   ; ID 0 -> derive from PRNG
    LDA     PRNG_OUTPUT
    ROR
    ROR
    AND     #$1C
    ORA     #$01
.have_id:
    STA     ACTIVE_CHAR                 ; store encounter ID
    ROR
    BCC     .no_clear
    LDY     #MFIELD_APPEARANCE
    LDA     (ENTITY_PTR),Y              ; char appearance
    BMI     .no_clear
    BAMI    #$2A,.no_clear
    BAPL    #$3F,.no_clear
    STOB    #$00,ACTIVE_CHAR ; clear ID if char in range $2A-$3E
.no_clear:
    LDY     #$02
    LDA     (EVENT_PTR),Y               ; event byte 2 = destination pos
    STA     EVENT_DEST_POS
    JSR     COMMIT_MOVE                 ; commit move
    MOVB    WAIT_LOOP_INIT,WAIT_LOOP_COUNT
    JSR     REORDER_CURRENT_CHAR
    JSR     TIMED_WAIT
    LDY     #MFIELD_POSITION
    LDA     EVENT_DEST_POS
    STA     (ENTITY_PTR),Y              ; update char packed position
    LDA     ACTIVE_CHAR
    BAEQ    CURRENT_PLAYER,.same_scene ; compare encounter ID with current scene
    JMP     RESET_TURN_POS              ; different -> reset position
.same_scene:
    LDA     EVENT_DEST_POS
    JSR     POS_TO_COLROW
    STA     CURRENT_COL
    STY     CURRENT_ROW
    JMP     COMMIT_MOVE                 ; commit to new position

    ; --- encounter avoided ---
.avoided:
    LDY     #$01
    LDA     (EVENT_PTR),Y
    ORA     #$40                        ; mark event as activated
    STA     (EVENT_PTR),Y
    MOVB    CURRENT_COL,TURN_START_COL
    MOVB    CURRENT_ROW,TURN_START_ROW
    JSR     RESET_TURN_POS              ; reset to turn start
    MOVB    TURN_START_COL,BLINK_COL
    MOVB    TURN_START_ROW,BLINK_ROW
    MOVB    FONT_CHARNUM,BLINK_CHAR
    LDX     #$0C              ; tune 6: combat avoided effect
```

```
    JSR     TUNE_PLAYER
    JSR     SET_CURSOR_ROW21
    LDA     #MSG_INSERT_DISK
    JMP     DISPLAY_MESSAGE             ; "avoided encounter" message

    ; --- special encounter (rare) ---
.special:
    JSR     SET_CURSOR_ROW21
    LDA     #$09
    JSR     SCENE_LOOP                  ; trigger scene 9
    LDA     #$01
    JMP     APPLY_DAMAGE_TO_CURRENT
```

Defines:
HANDLE_ENCOUNTER, used in chunk 232.

Uses ACTIVE_CHAR 492c, APPLY_DAMAGE_TO_CURRENT 352, BLINK_CHAR 544c, BLINK_COL 563b, BLINK_ROW 544b, COMMIT_MOVE 239, CURRENT_COL 472b, CURRENT_PLAYER 562a, CURRENT_ROW 473a, DISPLAY_MESSAGE 637, ENTITY_PTR 561, EVENT_DEST_POS 353c, EVENT_POS 532b, EVENT_PTR 353a, EVFIELD_POSITION 397a, EVFIELD_TYPE 397a, FONT_CHARNUM 575, MFIELD_APPEARANCE 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MOVB 8c, MSG_DOOR_OPENS 634, MSG_INSERT_DISK 634, POS_TO_COLROW 599, PRNG_OUTPUT 593a, RANDOM_IN_RANGE 597, REORDER_CURRENT_CHAR 489a, RESET_TURN_POS 239, SCENE_LOOP 527, SET_CHAR_STATE 242, SET_CURSOR_ROW21 627, STEP_PRNG 595, STOB 8c, TIMED_WAIT 606a, TUNE_PLAYER 682, TURN_START_COL 230a, TURN_START_ROW 230b, WAIT_LOOP_COUNT 51, and WAIT_LOOP_INIT 353b.

8.2.9 Mob movement encounter

HANDLE_MOB_ENCOUNTER resolves what happens when a character's movement brings it into contact with another entity.

Flee check. The routine first sums the hostile strength at the destination via SUM_HOSTILE_AT_POS. If hostiles are present, a flee threshold is computed:

- Base courage = strength (byte 5 bits 4–0) + $10.
- Subtract total adjacent threat; clamp result to 4–$1C.
- Threshold = (clamped value × 4) OR $03.

If the PRNG output exceeds this threshold, the character flees: its position resets to the turn's starting location and END_TURN is called.

Move commit. If the character holds ground (or no threat was present), COMMIT_MOVE updates the position.

Encounter resolution. CHECK_ENCOUNTER classifies the destination:

- Result < 2 (no hostile): CHECK_THREATS_HERE decides whether to continue moving (MOVE_LOOP) or end the turn.
- Result ≥ 2 (hostile present): the routine walks the target group's linked list, sets the restricted flag on every co-located member, displays a combat message, and ends the turn.

357 ⟨*handle mob encounter* 357⟩≡ (909)

```
    ORG     $1E5A
HANDLE_MOB_ENCOUNTER:
    SUBROUTINE HANDLE_MOB_ENCOUNTER
    ; Handle mob-on-mob movement encounter with flee mechanics.
    ;
    ; Input:  ENTITY_PTR | moving character
    ;         CURRENT_COL/ROW | destination
    ; Behavior:
    ;   Evaluates threat at destination. If hostiles present,
    ;   computes flee threshold from strength vs threat.
    ;   If character doesn't flee, commits move and checks
    ;   for combat encounters.
    ; Clobbers: A, X, Y

    ; --- evaluate threat at destination ---
    STOB    #$00,ADJACENT_THREAT
    MOVW    ENTITY_PTR,TARGET_REC
    LDA     CURRENT_COL
    LDY     CURRENT_ROW
    JSR     COLROW_TO_POS
    JSR     SUM_HOSTILE_AT_POS
    LDA     ADJACENT_THREAT
    BEQ     .no_flee                ; no hostiles -> skip flee

    ; --- compute flee threshold ---
    LDY     #MFIELD_STRENGTH
    LDA     (ENTITY_PTR),Y
    AND     #STRENGTH_MASK          ; strength (bits 4-0)
    CLC
    ADC     #$10                    ; courage = strength + $10
    SEC
    SBC     ADJACENT_THREAT         ; minus adjacent threat
    BMI     .clamp_low              ; negative -> clamp
    BAMI    #$1D,.in_range          ; < $1D -> use as-is
    LDA     #$1C                    ; clamp high to $1C
    BNE     .in_range
.clamp_low:
    LDA     #$04                    ; clamp low to $04
.in_range:
    ASL                             ; \ threshold *= 4
```

```
    ASL                             ; /
    ORA     #$03                    ; set low 2 bits
    TAX

    ; --- flee roll ---
    JSR     STEP_PRNG
    TXA
    BAPL    PRNG_OUTPUT,.no_flee    ; threshold >= random -> hold
    JSR     SET_CURSOR_ROW21
    JSR     PRINT_MOB_NAME
    JSR     SCROLL_STATUS_LINE
    LDA     #MSG_FLYING_TACKLE      ; "leaps into a flying tackle"
    JSR     DISPLAY_MESSAGE
    MOVB    TURN_START_COL,CURRENT_COL ; reset to start
    MOVB    TURN_START_ROW,CURRENT_ROW
    JMP     END_TURN                ; fled

    ; --- no flee: commit move and check encounter ---
.no_flee:
    JSR     COMMIT_MOVE
    LDY     #MFIELD_POSITION
    LDA     (ENTITY_PTR),Y
    JSR     CHECK_ENCOUNTER
    LDA     ENCOUNTER_RESULT
    BAPL    #$02,.hostile           ; result >= 2 -> hostile
    JSR     CHECK_THREATS_HERE
    BAEQ    #$01,.one_threat        ; exactly 1 -> continue
    JMP     END_TURN
.one_threat:
    JMP     MOVE_LOOP

    ; --- hostile: restrict co-located group members ---
.hostile:
    JSR     FIRST_GROUP_MEMBER
.walk_group:
    LDA     MOB_PTR+1
    BEQ     .group_done             ; null -> end of list
    LDY     #MFIELD_POSITION
    LDA     (ENTITY_PTR),Y
    CMP     (MOB_PTR),Y             ; same position?
    BNE     .next_member
    JSR     SET_TARGET_RESTRICTED   ; restrict member
.next_member:
    JSR     NEXT_GROUP_MEMBER
    JMP     .walk_group
.group_done:
    JSR     SET_CURSOR_ROW21
    JSR     PRINT_MOB_NAME
    JSR     SCROLL_STATUS_LINE
    LDA     #MSG_UNCONSCIOUS        ; "is unconscious"
```

```
    JSR     DISPLAY_MESSAGE
    JMP     END_TURN
```

Defines:
HANDLE_MOB_ENCOUNTER, used in chunk 232.
Uses ADJACENT_THREAT 226b, CHECK_ENCOUNTER 312, CHECK_THREATS_HERE 323, COLROW_TO_POS 600a, COMMIT_MOVE 239, CURRENT_COL 472b, CURRENT_ROW 473a, DISPLAY_MESSAGE 637, ENCOUNTER_RESULT 226a, END_TURN 241, ENTITY_PTR 561, FIRST_GROUP_MEMBER 482, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MOB_PTR 608, MOVB 8c, MOVE_LOOP 232, MOVW 9a, MSG_FLYING_TACKLE 634, MSG_UNCONSCIOUS 634, NEXT_GROUP_MEMBER 482, PRINT_MOB_NAME 484, PRNG_OUTPUT 593a, SCROLL_STATUS_LINE 639, SET_CURSOR_ROW21 627, SET_TARGET_RESTRICTED 364, STEP_PRNG 595, STOB 8c, SUM_HOSTILE_AT_POS 322, TARGET_REC 618b, TURN_START_COL 230a, and TURN_START_ROW 230b.

HANDLE_EVENT_SLOT is the main interaction handler when a character steps onto an event tile. It resets the character to idle state, then reads byte 2 of the event entry and branches on its top two bits. If the high bits are $C0 (activated event), the low 5 bits give a sub-type: values below $1D clear the event slot and optionally trigger scene $0A, then the character commits the move, checks for adjacent threats, and continues the movement loop. If the event is not activated, the routine plays tune $0E, then dispatches on the event's high-nibble type. Type $00 (no special flag) with a small sub-value displays a message and deals 1 damage; sub-values at $1B or above trigger SHOW_LOCKED_MSG. Type $40 or above represents a treasure chest: a strength check (field 5 vs. a random threshold) determines whether the character can open it. Failure displays a "can't open" message and converts the event to type $40 for the next attempt, followed by a trap-damage check at EVENT_PRNG_CHECK. Success with a small sub-value triggers scene $13; larger sub-values show message $12 ("found treasure") and either clear the event slot or decrement a counter for multi-use containers. EVENT_PRNG_CHECK itself rolls the PRNG and, if the result is below $14, applies 1–2 points of trap damage with the restricted flag set.

359a ⟨*var event slot byte* 359a⟩≡ (909)

```
    ORG     $5A59
EVENT_SLOT_BYTE:
    DC.B    $CC                     ; initial value (runtime: saved byte 2 of event slot (type+value
```

Defines:
EVENT_SLOT_BYTE, used in chunks 360 and 898.

359b ⟨*var event sub value* 359b⟩≡ (909)

```
    ORG     $5A5A
EVENT_SUB_VALUE:
    DC.B    $C6                     ; initial value (runtime: lower 5 bits of event slot byte 2)
```

Defines:
EVENT_SUB_VALUE, used in chunks 360 and 898.

360 ⟨*handle event slot* 360⟩≡ (909)

```
    ORG     $1D4D
HANDLE_EVENT_SLOT:
    SUBROUTINE HANDLE_EVENT_SLOT
    ; Process event tile interaction.
    ; Input: EVENT_PTR = event entry, ENTITY_PTR = character
    ; Behavior: branches on event byte 2 top bits for
    ;   activated events, locked objects, and treasure.
    ; Clobbers: A, X, Y
    ; Handle empty slot interaction: treasure pickup, traps,
    ; and event activation.
    LDA     #$00
    JSR     SET_CHAR_STATE
    LDY     #$02
    LDA     (EVENT_PTR),Y
    STA     EVENT_SLOT_BYTE
    AND     #$C0
    BANE    #$C0,.not_active
    LDA     (EVENT_PTR),Y
    AND     #$1F
    TAX
    BAPL    #$1D,.no_remove
    LDY     #$00
    LDA     #$FE
    STA     (EVENT_PTR),Y
    TXA
    BAEQ    #$00,.no_remove
    LDA     #$0A
    JSR     SCENE_LOOP
.no_remove:
    JSR     COMMIT_MOVE
    JSR     CHECK_THREATS_HERE
    BANE    #$00,.has_threat
    STA     MOVE_POINTS
.has_threat:
    JMP     MOVE_LOOP
.not_active:
    PHA
    LDX     #$0E              ; tune 7: entity interaction
    JSR     TUNE_PLAYER
    PLA
    BANE    #$00,.has_type
    LDA     EVENT_SLOT_BYTE
    AND     #$1F
    BAMI    #$1B,.small_val
    JMP     SHOW_LOCKED_MSG
.small_val:
    JSR     SET_CURSOR_ROW21
    JSR     PRINT_MOB_NAME
    JSR     SCROLL_STATUS_LINE
```

```
    LDA     #MSG_DEATH
    JSR     DISPLAY_MESSAGE
    LDA     #$01
    JMP     APPLY_DAMAGE_TO_CURRENT
.has_type:
    BAEQ    #$40,.type40
    LDA     #$19
    JSR     RANDOM_IN_RANGE
    STA     PRNG_OUTPUT
    LDY     #$05
    LDA     (ENTITY_PTR),Y
    AND     #$1F
    BAPL    PRNG_OUTPUT,.type40
    JSR     SET_CURSOR_ROW21
    LDA     #MSG_HEAVY_ROCK
    JSR     DISPLAY_MESSAGE
    LDY     #$02
    LDA     EVENT_SLOT_BYTE
    AND     #$3F
    ORA     #$40
    STA     (EVENT_PTR),Y
EVENT_PRNG_CHECK:               ; $1DDC - external entry point
    JSR     STEP_PRNG
    LDA     PRNG_OUTPUT
    BAMI    #$14,.trap_damage
    RTS
.trap_damage:
    JSR     SET_CURSOR_ROW21
    LDA     #MSG_DUST_CLOUD
    JSR     DISPLAY_MESSAGE
    JSR     SET_RESTRICTED
    JSR     PRINT_MOB_NAME
    JSR     DELAY_WITH_ANIMATION
    JSR     STEP_PRNG
    LDA     PRNG_OUTPUT
    ROL2
    LDA     #$01
    ADC     #$00
    JMP     APPLY_DAMAGE_TO_CURRENT
.type40:
    LDA     EVENT_SLOT_BYTE
    AND     #$1F
    STA     EVENT_SUB_VALUE
    BAPL    #$1B,.big_treasure
    JSR     SET_CURSOR_ROW21
    LDA     #$13
    JSR     SCENE_LOOP
    JMP     .clear_slot
.big_treasure:
    JSR     SET_CURSOR_ROW21
```

```
    LDA     #MSG_TOPPLES
    JSR     DISPLAY_MESSAGE
    LDA     EVENT_SUB_VALUE
    BAMI    #$1F,.clear_slot
    DEC     EVENT_SUB_VALUE
    LDA     EVENT_SLOT_BYTE
    AND     #$E0
    ORA     EVENT_SUB_VALUE
    LDY     #$02
    STA     (EVENT_PTR),Y
    JMP     .update_pos
.clear_slot:
    LDY     #$00
    LDA     #$FE
    STA     (EVENT_PTR),Y
.update_pos:
    MOVB    CURRENT_COL,TURN_START_COL
    MOVB    CURRENT_ROW,TURN_START_ROW
    JSR     RESET_TURN_POS
    JSR     ANIM_TICK_AND_WAIT
    JMP     EVENT_PRNG_CHECK
```

Defines:
EVENT_PRNG_CHECK, used in chunk 342.
HANDLE_EVENT_SLOT, used in chunk 232.

Uses ANIM_TICK_AND_WAIT 605b, APPLY_DAMAGE_TO_CURRENT 352, CHECK_THREATS_HERE 323, COMMIT_MOVE 239, CURRENT_COL 472b, CURRENT_ROW 473a, DELAY_WITH_ANIMATION 603b, DISPLAY_MESSAGE 637, ENTITY_PTR 561, EVENT_PTR 353a, EVENT_SLOT_BYTE 359a, EVENT_SUB_VALUE 359b, MOVB 8c, MOVE_LOOP 232, MOVE_POINTS 230c, MSG_DEATH 634, MSG_DUST_CLOUD 634, MSG_HEAVY_ROCK 634, MSG_TOPPLES 634, PRINT_MOB_NAME 484, PRNG_OUTPUT 593a, RANDOM_IN_RANGE 597, RESET_TURN_POS 239, ROL2 16c, SCENE_LOOP 527, SCROLL_STATUS_LINE 639, SET_CHAR_STATE 242, SET_CURSOR_ROW21 627, SET_RESTRICTED 246, SHOW_LOCKED_MSG 363, STEP_PRNG 595, TUNE_PLAYER 682, TURN_START_COL 230a, and TURN_START_ROW 230b.

SHOW_LOCKED_MSG displays message $0C (a "locked" or "blocked" notification), prints the mob's name for context, and resets the character's turn state to 0 (idle). This is called when a character attempts to interact with an event slot that requires a higher strength or special item.

363 ⟨*show locked msg* 363⟩≡ (909)

```
    ORG     $1CF1
; SHOW_LOCKED_MSG | display locked/blocked message, reset to idle
SHOW_LOCKED_MSG:
    SUBROUTINE SHOW_LOCKED_MSG
    ; Display the locked/statue message and reset entity to idle.
    ;
    ; Behavior:
    ;   Positions cursor at row 21, prints the mob name, scrolls the
    ;   status line, displays the statue message, and sets state to 0.
    ;
    ; Clobbers: A, Y

    ; Display "locked container" message and reset state.
    ; Clobbers: A, Y
    JSR     SET_CURSOR_ROW21        ; position cursor
    JSR     PRINT_MOB_NAME          ; print blocking entity name
    JSR     SCROLL_STATUS_LINE
    LDA     #MSG_HIT_STATUE         ; "CHARGES INTO A STONE STATUE."
    JSR     DISPLAY_MESSAGE
    LDA     #$00                    ; state 0 = idle
    JMP     SET_CHAR_STATE
```

Defines:
SHOW_LOCKED_MSG, used in chunks 232 and 360.

Uses DISPLAY_MESSAGE 637, MSG_HIT_STATUE 634, PRINT_MOB_NAME 484, SCROLL_STATUS_LINE 639, SET_CHAR_STATE 242, and SET_CURSOR_ROW21 627.

SET_TARGET_RESTRICTED sets bit 2 of field 15 on the mob record at MOB_PTR, marking that mob as movement-restricted. This is used by HANDLE_MOB_ENCOUNTER to pin down every hostile at the encounter position so they cannot simply walk away.

364 ⟨*set target restricted* 364⟩≡ (909)

```
    ORG     $1F03
; SET_TARGET_RESTRICTED | pin mob at current position
SET_TARGET_RESTRICTED:
    SUBROUTINE SET_TARGET_RESTRICTED
    ; Set the combat-engaged flag (bit 2) on the mob at MOB_PTR.
    ;
    ; Inputs:
    ;   MOB_PTR | pointer to the mob record
    ;
    ; Behavior:
    ;   ORs bit 2 into byte 15 of the mob record, marking it as
    ;   pinned in combat.
    ;
    ; Modifies: mob record byte 15
    ; Clobbers: A, Y

    ; Set restricted flag (bit 2) on entity at MOB_PTR byte 15.
    ; Clobbers: A, Y
    LDY     #MFIELD_FLAGS           ; byte 15: status flags
    LDA     (MOB_PTR),Y
    ORA     #$04                    ; set bit 2 = combat engaged
    STA     (MOB_PTR),Y
    RTS
```

Defines:
SET_TARGET_RESTRICTED, used in chunk 357.
Uses MOB_PTR 608.

8.3 Target selection

SELECT_COMBAT_TARGET evaluates the battlefield to decide what the current entity should fight. It dispatches on CHAR_AI_MODE into three paths. For modes below 3 ("path low"), the routine checks STEPS_TAKEN, the encounter result, and adjacent threats; if any of these indicate combat, it returns 2, otherwise it returns 4 (safe). For modes 3–4 ("path mid"), the routine walks the group's linked list looking for the hostile mob with the highest threat score (via GET_ENTITY_THREAT), saving its pointer in TARGET_PTR/TARGET_PTR+1; if a target is found, it returns 1 (attack ready). For mode 5 and above ("path high"), it performs the same walk but ranks targets by field 5 strength instead of threat score, again returning 1 if a valid target is found. If no hostile is found in the mid or high paths, execution falls through to the low path for a final check.

BEST_THREAT accumulates the highest threat or strength score seen so far while SELECT_COMBAT_TARGET iterates through group members.

TARGET_PTR and TARGET_PTR+1 record the pointer to the mob with the best score. When SELECT_COMBAT_TARGET finishes and returns 1 (attack ready), the caller uses these to load the target.

8.3.1 Entity threat evaluation

GET_ENTITY_THREAT evaluates the threat level of the mob at MOB_PTR relative to the current entity at ENTITY_PTR.

Threat levels.

- 0 — not hostile, or hostile but distant (distance > 1)
- 1 — hostile and adjacent (distance = 1)
- 2 — hostile and co-located (distance = 0)

The routine copies ENTITY_PTR to TARGET_REC for CHECK_HOSTILE, then measures MANHATTAN_DISTANCE from the mob's position to the entity.

365 ⟨*get entity threat* 365⟩≡ (909)

```
    ORG     $1979
GET_ENTITY_THREAT:
    SUBROUTINE GET_ENTITY_THREAT
    ; Evaluate threat level of mob at MOB_PTR vs entity at ENTITY_PTR.
    ;
```

```
    ; Input:  ENTITY_PTR | reference entity
    ;         MOB_PTR    | mob to evaluate
    ; Output: A = 0 (no threat), 1 (adjacent), 2 (co-located)
    ; Modifies: TARGET_REC
    ; Clobbers: A, Y

    ; --- check hostility ---
    LDA     ENTITY_PTR              ; \ copy ENTITY_PTR to TARGET_REC
    STA     TARGET_REC              ;  |
    LDA     ENTITY_PTR+1            ;  |
    STA     TARGET_REC+1            ; /
    JSR     CHECK_HOSTILE
    CMP     #$00
    BNE     .hostile                ; hostile -> measure distance
    RTS                             ; not hostile -> return 0

    ; --- measure distance ---
.hostile:
    LDY     #MFIELD_POSITION        ; mob's map position
    LDA     (MOB_PTR),Y
    JSR     MANHATTAN_DISTANCE      ; A = distance to entity
    CMP     #$01
    BNE     .not_adjacent
    RTS                             ; distance = 1 -> return 1 (adjacent)
.not_adjacent:
    BPL     .far                    ; distance > 1 -> too far
    LDA     #$02                    ; distance = 0 -> return 2 (co-located)
    RTS
.far:
    LDA     #$00                    ; return 0 (too far)
    RTS
```

Defines:
GET_ENTITY_THREAT, used in chunk 368.
Uses CHECK_HOSTILE 317, ENTITY_PTR 561, MANHATTAN_DISTANCE 400, MFIELD_POSITION 561, MOB_PTR 608, and TARGET_REC 618b.

366 ⟨*var target ptr* 366⟩≡ (909)

```
    ORG     $5A86
TARGET_PTR:
    DC.B    $C3,$D4                 ; initial value (runtime: selected combat target pointer (2 byte
```

Defines:
TARGET_PTR, used in chunks 328, 368, and 370a.

367 ⟨*var best threat* 367⟩≡ (909)

```
    ORG     $5A88
BEST_THREAT:
    DC.B    $CF                     ; initial value (runtime: best threat level during target select
```

Defines:
BEST_THREAT, used in chunk 368.

368 ⟨*select combat target* 368⟩≡ (909)

```
        ORG     $18E4
    SELECT_COMBAT_TARGET:
        SUBROUTINE SELECT_COMBAT_TARGET
        ; Select a combat target for the current AI entity.
        ;
        ; Input:  CHAR_AI_MODE  | AI behavior mode (0-5+)
        ;         ENTITY_PTR    | current entity record
        ;         STEPS_TAKEN   | steps taken this turn
        ;
        ; Output: A = 1: target found (TARGET_PTR/HI set)
        ;         A = 2: engage combat (no specific target)
        ;         A = 4: safe (no threats)
        ;
        ; Modifies: BEST_THREAT, TARGET_PTR, TARGET_PTR+1
        ; Clobbers: A, X, Y

        ; --- dispatch on AI mode ---
        LDA     CHAR_AI_MODE
        BAMI    #$03,.path_low         ; mode < 3 -> check encounter/threats
        BAMI    #$05,.path_mid         ; mode 3-4 -> pick highest-threat mob

        ; --- path high (mode >= 5): pick strongest mob ---
        JSR     FIRST_GROUP_MEMBER
        STOB    #$00,BEST_THREAT
    .loop_high:
        JSR     GET_ENTITY_THREAT
        BAEQ    #$00,.skip_high        ; not hostile -> skip
        LDY     #MFIELD_STRENGTH
        LDA     (MOB_PTR),Y
        AND     #STRENGTH_MASK         ; strength (bits 4-0)
        BAMI    BEST_THREAT,.skip_high ; not stronger -> skip
        STA     BEST_THREAT            ; new best
        MOVB    MOB_PTR,TARGET_PTR
        MOVB    MOB_PTR+1,TARGET_PTR+1
    .skip_high:
        JSR     NEXT_GROUP_MEMBER
        LDA     #$00
        BANE    MOB_PTR+1,.loop_high   ; more members -> continue
        BAEQ    BEST_THREAT,.path_low  ; no target -> fall through
        LDA     #$01                   ; target found
        RTS

        ; --- path low (mode < 3): check encounter and threats ---
    .path_low:
        LDA     #$00
        BANE    STEPS_TAKEN,.ret_2     ; already moved -> engage
        LDY     #MFIELD_POSITION
        LDA     (ENTITY_PTR),Y        ; entity's map position
        JSR     CHECK_ENCOUNTER
```

```
    LDA     #$02
    BAEQ    ENCOUNTER_RESULT,.ret_2 ; encounter found -> engage
    JSR     CHECK_THREATS_HERE
    BANE    #$01,.ret_2             ; threats present -> engage
    LDA     #$04                    ; safe
    RTS
.ret_2:
    LDA     #$02                    ; engage combat
    RTS

    ; --- path mid (mode 3-4): pick highest-threat mob ---
.path_mid:
    STOB    #$00,BEST_THREAT
    JSR     FIRST_GROUP_MEMBER
.loop_mid:
    JSR     GET_ENTITY_THREAT
    BAEQ    BEST_THREAT,.skip_mid   ; threat == 0 -> skip
    BMI     .skip_mid               ; negative threat -> skip
    STA     BEST_THREAT             ; new best threat
    MOVB    MOB_PTR,TARGET_PTR
    MOVB    MOB_PTR+1,TARGET_PTR+1
.skip_mid:
    JSR     NEXT_GROUP_MEMBER
    LDA     #$00
    BANE    MOB_PTR+1,.loop_mid     ; more members -> continue
    BAEQ    BEST_THREAT,.path_low   ; no target -> fall through
    LDA     #$01                    ; target found
    RTS
```

Defines:
SELECT_COMBAT_TARGET, used in chunks 237 and 370a.

Uses BEST_THREAT 367, CHAR_AI_MODE 531b, CHECK_ENCOUNTER 312, CHECK_THREATS_HERE 323, ENCOUNTER_RESULT 226a, ENTITY_PTR 561, FIRST_GROUP_MEMBER 482, GET_ENTITY_THREAT 365, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MOB_PTR 608, MOVB 8c, NEXT_GROUP_MEMBER 482, STEPS_TAKEN 231a, STOB 8c, and TARGET_PTR 366.

LOAD_TARGET_PTR copies the pre-selected combat target address from TARGET_PTR/TARGET_PTR+1 into MOB_PTR and clears IS_PLAYER_TURN to zero, indicating that the next attack sequence is from the AI's perspective. START_COMBAT calls this to set up MOB_PTR before entering RESOLVE_ATTACK.

370a ⟨*load target ptr* 370a⟩≡ (909)

```
      ORG     $199D
  LOAD_TARGET_PTR:
      SUBROUTINE LOAD_TARGET_PTR
      ; Load the pre-selected combat target into MOB_PTR.
      ;
      ; Input:  TARGET_PTR/HI | saved target from SELECT_COMBAT_TARGET
      ; Output: MOB_PTR = target pointer, IS_PLAYER_TURN = 0
      ; Clobbers: A

      MOVB    TARGET_PTR,MOB_PTR
      MOVB    TARGET_PTR+1,MOB_PTR+1
      STOB    #$00,IS_PLAYER_TURN    ; AI's turn (not player)
      RTS
```

Defines:
LOAD_TARGET_PTR, used in chunk 328.
Uses IS_PLAYER_TURN 474, MOB_PTR 608, MOVB 8c, SELECT_COMBAT_TARGET 368, STOB 8c, and TARGET_PTR 366.

PROCESS_ENTITY_SCRIPT temporarily removes an entity from the map, runs a tune, and restores it. The routine saves the entity's byte 0 (position character) from MOB_PTR, overwrites it with $FE (marking the slot empty), plays tune $0E via TUNE_PLAYER, calls DISPATCH_ON_MODE for timing/animation, and then redraws the original character at the entity's position with DRAW_CHAR_AT_POS. This is used during event interactions where the entity needs to be temporarily invisible—for example, when opening a chest.

370b ⟨*zero page defines* 25a⟩+≡ (909) ◁353a 379a▷

```
  SAVED_CHAR              EQU     $BA     ; scratch: saved entity display character
```

Defines:
SAVED_CHAR, used in chunk 371.

371 ⟨*process entity script* 371⟩≡ (909)

```
        ORG     $1849
    PROCESS_ENTITY_SCRIPT:
        SUBROUTINE PROCESS_ENTITY_SCRIPT
        ; Temporarily hide an entity, play a tune, then restore it.
        ;
        ; Input:  MOB_PTR | entity record to process
        ; Behavior:
        ;   Saves display char (byte 0), marks slot empty ($FE),
        ;   plays interaction tune, dispatches animation, then
        ;   restores and redraws the entity.
        ; Modifies: SAVED_CHAR, MOB_PTR byte 0
        ; Clobbers: A, X, Y

        LDY     #MFIELD_CHAR            ; \ save display character
        LDA     (MOB_PTR),Y             ;  |
        STA     SAVED_CHAR              ; /
        LDA     #EVENT_EMPTY            ; mark slot empty ($FE)
        STA     (MOB_PTR),Y
        LDX     #$0E                    ; tune index: entity interaction
        JSR     TUNE_PLAYER
        JSR     DISPATCH_ON_MODE
        LDA     SAVED_CHAR              ; restore saved character
        JMP     DRAW_CHAR_AT_POS        ; redraw at entity position
```

Defines:
PROCESS_ENTITY_SCRIPT, used in chunk 342.

Uses DISPATCH_ON_MODE 607b, DRAW_CHAR_AT_POS 471, EVENT_EMPTY 397a, MOB_PTR 608, SAVED_CHAR 370b, and TUNE_PLAYER 682.

APPLY_DAMAGE_TO_MOB copies MOB_PTR into EFFECT_REC and tail-calls APPLY_DAMAGE with the damage amount in A. This is the counterpart to APPLY_DAMAGE_TO_CURRENT: where that routine applies damage to the entity at ENTITY_PTR, this one applies damage to the mob at MOB_PTR—used during mob-on-player combat.

372a ⟨*apply damage to mob* 372a⟩≡ (909)

```
      ORG     $1432
  APPLY_DAMAGE_TO_MOB:
      SUBROUTINE APPLY_DAMAGE_TO_MOB
      ; Apply damage to the mob at MOB_PTR.
      ;
      ; Input:  A = damage amount, MOB_PTR = target mob
      ; Output: EFFECT_REC = MOB_PTR, jumps to APPLY_DAMAGE
      ; Clobbers: X

      TAX                             ; save damage value
      MOVW    MOB_PTR,EFFECT_REC      ; copy target pointer
      TXA                             ; restore damage value
      JMP     APPLY_DAMAGE
```

Defines:
APPLY_DAMAGE_TO_MOB, used in chunk 346.
Uses APPLY_DAMAGE 349, EFFECT_REC 483a, MOB_PTR 608, and MOVW 9a.

ADD_WILLINGNESS is a tiny helper that adds A (signed) to the COMBAT_WILLINGNESS accumulator. It exists as a separate routine because RESOLVE_ATTACK calls it from many points during the willingness calculation and inlining each addition would waste significant code space.

372b ⟨*add willingness* 372b⟩≡ (909)

```
      ORG     $169B
  ADD_WILLINGNESS:
      SUBROUTINE ADD_WILLINGNESS
      ; Add A to the combat willingness accumulator.
      ;
      ; Input:  A = willingness delta (signed)
      ; Output: COMBAT_WILLINGNESS += A
      ; Clobbers: A

      CLC
      ADC     COMBAT_WILLINGNESS
      STA     COMBAT_WILLINGNESS
      RTS
```

Defines:
ADD_WILLINGNESS, used in chunk 338.
Uses COMBAT_WILLINGNESS 337a.

PICK_RANDOM_MEMBER selects a random eligible member from the current character's group. It reads the group size from byte 8 of the record at $FA, calls RANDOM_IN_RANGE to pick a random index, and then walks the group's linked list with FIRST_GROUP_MEMBER/NEXT_GROUP_MEMBER, counting down to the Nth member whose field 4 falls in the valid range ($00–$14). Members with negative field 4 values or values at $15 or above are skipped. If the random roll returns zero, the routine retries. On success, MOB_PTR points to the selected member's record; if the group is empty, both bytes of MOB_PTR are zeroed as a "not found" sentinel.

373 ⟨*pick random member* 373⟩≡ (909)

```
    ORG     $12D0
PICK_RANDOM_MEMBER:
    SUBROUTINE PICK_RANDOM_MEMBER
    ; Pick a random player character from the current group.
    ;
    ; Input:  CHAR_PTR | group leader's character record
    ; Output: MOB_PTR  | selected member (zeroed if empty group)
    ;
    ; Behavior:
    ;   Reads group size from CFIELD_XP (byte 8 bits 3-0), picks
    ;   a random index, then walks the linked list skipping dead
    ;   (bit 7 set) and NPC (appearance >= $15) members until the
    ;   Nth eligible member is found.  Retries if random index = 0.
    ;
    ; Modifies: MOB_PTR
    ; Clobbers: A, X, Y

.loop:
    LDY     #CFIELD_XP              ; byte 8: group count in bits 3-0
    LDA     (CHAR_PTR),Y
    BEQ     .zero                   ; empty group
    JSR     RANDOM_IN_RANGE         ; random(size)
    BAEQ    #$00,.loop              ; 0 -> retry
    TAX                             ; X = count down to Nth
    JSR     FIRST_GROUP_MEMBER
.check:
    LDY     #MFIELD_APPEARANCE      ; byte 4: appearance index
    LDA     (MOB_PTR),Y
    BMI     .next                   ; bit 7 set -> dead/invalid, skip
    BAPL    #NPC_APPEARANCE_MIN,.next             ; >= $15 -> NPC, skip
    DEX
    BNE     .next                   ; not Nth yet
    RTS                             ; found
.next:
    JSR     NEXT_GROUP_MEMBER
    JMP     .check
.zero:
    STA     MOB_PTR                 ; \ clear pointer (A=0 from BEQ)
    STA     MOB_PTR+1               ; /
```

```
        RTS
```

Defines:
PICK_RANDOM_MEMBER, used in chunk 385.
Uses CFIELD_XP 561, CHAR_PTR 591a, FIRST_GROUP_MEMBER 482, MFIELD_APPEARANCE 561, MOB_PTR 608, NEXT_GROUP_MEMBER 482, and RANDOM_IN_RANGE 597.

8.4 Combat utilities

MAP_VALUE_TO_STRING converts a numeric value in A into a descriptive string for display. Values 0–7 map directly to string indices; values 8–15 are compressed (ROR, AND $03, ORA $08) into indices 8–11; and values 16+ are further compressed into indices 12–15. The routine then walks a linked list of length-prefixed strings starting at $803A, advancing by each string's length byte until the target index is reached, and tail-calls PRINT_BOTTOM_CENTERED to display it. This provides combat messages that scale with the magnitude of the value—for example, mapping low damage to "scratches" and high damage to "devastating blow."

374 ⟨*attack verb strings* 374⟩≡ (909)

```
        ORG     $803A
    ATTACK_VERB_STRINGS:
        APSTR   "JABS"
        APSTR   "POKES"
        APSTR   "HITS"
        APSTR   "CLOUTS"
        APSTR   "POUNDS"
        APSTR   "SMACKS"
        APSTR   "SMITES"
        APSTR   "WHOPS"
        APSTR   "WHACKS"
        APSTR   "BASHES"
        APSTR   "WHOMPS"
        APSTR   "WALLOPS"
        APSTR   "SMASHES"
        APSTR   "CLOBBERS"
        APSTR   "LAMBASTES"
```

Defines:
ATTACK_VERB_STRINGS, used in chunk 375.

375 ⟨*map value to string* 375⟩≡ (909)

```
    ORG     $11FC
MAP_VALUE_TO_STRING:
    SUBROUTINE MAP_VALUE_TO_STRING
    ; Map a damage/effect value to a verb string and display it.
    ;
    ; Input:  A = numeric value (damage level)
    ; Output: displays the corresponding verb string
    ;
    ; Behavior:
    ;   Values 0-7 map directly to string indices 0-7.
    ;   Values 8-15 compress to indices 8-11.
    ;   Values 16+ compress to indices 12-15.
    ;   Walks the ATTACK_VERB_STRINGS list to find the
    ;   Nth entry and prints it via PRINT_BOTTOM_CENTERED.
    ;
    ; Modifies: PRINT_STRING_ADDR
    ; Clobbers: A, X, Y

    ; --- map value to string index ---
    BAMI    #$08,.use_as_is        ; < 8 -> use directly
    BAPL    #$10,.high_range       ; < 16 -> mid range
    ROR                            ; \ compress bits 2-1
    AND     #$03                   ;  | to 2-bit index
    ORA     #$08                   ; / -> index 8-11
    BNE     .use_as_is
.high_range:
    ROR                            ; \ compress bits 4-3
    ROR                            ;  | to 2-bit index
    AND     #$03                   ;  |
    ORA     #$0C                   ; / -> index 12-15
.use_as_is:
    TAX                            ; X = string index

    ; --- walk string list to Xth entry ---
    STOW    ATTACK_VERB_STRINGS,PRINT_STRING_ADDR
    LDY     #$00
.loop:
    DEX
    BNE     .next                  ; not there yet
    JMP     PRINT_BOTTOM_CENTERED  ; print and return
.next:
    SEC                            ; \ advance ptr by length + 1
    LDA     (PRINT_STRING_ADDR),Y  ;  | (length byte)
    ADC     PRINT_STRING_ADDR      ;  |
    STA     PRINT_STRING_ADDR      ;  |
    LDA     #$00                   ;  |
    ADC     PRINT_STRING_ADDR+1    ;  |
    STA     PRINT_STRING_ADDR+1    ; /
    BNE     .loop
```

Defines:
MAP_VALUE_TO_STRING, used in chunks 342 and 346.
Uses ATTACK_VERB_STRINGS 374, PRINT_BOTTOM_CENTERED 628, PRINT_STRING_ADDR 576c, and STOW 8a.

AUTO_HEAL provides passive healing for characters between combat turns. If the entity's HP (field 6, low 6 bits) is below $28 (40 decimal), there is a random chance of healing: RANDOM_IN_RANGE with argument 1 gives a 50% chance of initiating the process. Once started, the routine enters a loop that increments HP via INCREMENT_HP and continues as long as the PRNG output is at or above $50—so healing stops probabilistically, with each additional point becoming less likely. For player characters (field 4 below $15), the routine displays a "healed" message the first time, controlled by the HEAL_NOTIFIED flag to avoid repeating the notification on subsequent HP increments.

376 ⟨*var heal notified* 376⟩≡ (909)

```
    ORG     $5A16
HEAL_NOTIFIED:
    DC.B    $B1                     ; initial value (runtime: auto heal: nonzero = already printed h
```

Defines:
HEAL_NOTIFIED, used in chunks 377 and 378.

377 ⟨*auto heal* 377⟩≡ (909)

```
    ORG     $143F
AUTO_HEAL:
    SUBROUTINE AUTO_HEAL
    ; Randomly restore HP for injured entities.
    ;
    ; Input:  ENTITY_PTR | entity record
    ; Behavior:
    ;   If HP (byte 6, bits 5-0) < 40, 50% chance to start
    ;   healing.  Loops incrementing HP while PRNG >= $50.
    ;   For player characters (appearance < $15), displays
    ;   MSG_REGAIN_CONSC the first time.
    ; Modifies: HEAL_NOTIFIED, entity byte 6
    ; Clobbers: A, X, Y

    LDY     #MFIELD_HP       ; byte 6: HP in bits 5-0
    LDA     (ENTITY_PTR),Y
    AND     #HP_MASK               ; isolate HP (bits 5-0)
    BAMI    #$28,.do_heal          ; HP < 40 -> try healing
    RTS
.do_heal:
    LDA     #$01
    JSR     RANDOM_IN_RANGE        ; 50% chance
    BANE    #$00,.start_heal       ; nonzero -> proceed
    RTS
.start_heal:
    STOB    #$00,HEAL_NOTIFIED     ; clear notification flag
.heal_loop:
    JSR     INCREMENT_HP           ; +1 HP
    JSR     STEP_PRNG
    LDA     PRNG_OUTPUT
    BAPL    #$50,.heal_loop        ; keep healing while PRNG < $50
    LDY     #MFIELD_APPEARANCE     ; check if player character
    LDA     (ENTITY_PTR),Y
    BALT    #NPC_APPEARANCE_MIN,.player           ; appearance < $15 -> player
.done:
    RTS
.player:
    LDA     HEAL_NOTIFIED          ; already notified?
    BNE     .done
    JSR     SET_CURSOR_ROW21
    JSR     PRINT_MOB_NAME
    JSR     SCROLL_STATUS_LINE
    LDA     #MSG_REGAIN_CONSC      ; "struggles back to consciousness"
    JMP     DISPLAY_MESSAGE
```

Defines:
AUTO_HEAL, used in chunks 222, 232, and 237.

Uses DISPLAY_MESSAGE 637, ENTITY_PTR 561, HEAL_NOTIFIED 376, INCREMENT_HP 378, MFIELD_APPEARANCE 561, MFIELD_HP 561, MSG_REGAIN_CONSC 634, PRINT_MOB_NAME 484, PRNG_OUTPUT 593a, RANDOM_IN_RANGE 597, SCROLL_STATUS_LINE 639, SET_CURSOR_ROW21 627,

STEP_PRNG 595, and STOB 8c.

INCREMENT_HP adds 1 to the entity's field 6 (which holds HP in its low 6 bits along with flags in the upper 2). Since the HP value is less than 63, a simple ADC $01 on the full byte works without disturbing the flag bits. When the masked HP reaches exactly 3—the threshold at which a character is no longer "badly hurt"—the routine notifies the player by printing the mob's name and displaying message $22, and sets HEAL_NOTIFIED to prevent AUTO_HEAL from showing its own separate notification.

378 ⟨*increment hp* 378⟩≡ (909)

```
    ORG     $1482
INCREMENT_HP:
    SUBROUTINE INCREMENT_HP
    ; Add 1 to entity's HP; notify when HP reaches 3.
    ;
    ; Input:  ENTITY_PTR | entity record
    ; Behavior:
    ;   Increments byte 6 (HP in bits 5-0).  When HP
    ;   reaches exactly 3 (no longer "badly hurt"), prints
    ;   the entity name and sets HEAL_NOTIFIED = $22
    ;   (which also serves as the message number for
    ;   DISPLAY_MESSAGE via fall-through).
    ; Modifies: entity byte 6, HEAL_NOTIFIED
    ; Clobbers: A, Y

    LDY     #MFIELD_HP       ; byte 6: HP in bits 5-0
    LDA     (ENTITY_PTR),Y
    CLC
    ADC     #$01                    ; HP += 1
    STA     (ENTITY_PTR),Y
    AND     #HP_MASK                ; isolate HP (bits 5-0)
    BAEQ    #$03,.notify            ; HP == 3 -> threshold crossed
    RTS
.notify:
    JSR     SET_CURSOR_ROW21
    JSR     PRINT_MOB_NAME
    JSR     SCROLL_STATUS_LINE
    STOB    #$22,HEAL_NOTIFIED      ; mark notified (A=$22 = message)
    JMP     DISPLAY_MESSAGE
```

Defines:
INCREMENT_HP, used in chunk 377.
Uses DISPLAY_MESSAGE 637, ENTITY_PTR 561, HEAL_NOTIFIED 376, MFIELD_HP 561, PRINT_MOB_NAME 484, SCROLL_STATUS_LINE 639, SET_CURSOR_ROW21 627, and STOB 8c.

MODIFY_CHAR_STATS is the low-level gold modification routine. It operates on the character record at EFFECT_REC, adding or subtracting the 16-bit value in NUM_VALUE/NUM_VALUE+1 to fields 13–14 (the gold counter). The direction is controlled by Y on entry: Y=0 subtracts, Y≠0 adds. After modifying gold, the routine recomputes field $0C, which encodes a derived level/XP value. The recomputation uses field 5 (shifted left 3 bits) as a scaling factor and iterates through a level progression, comparing accumulated thresholds against the character's gold (fields 13–14) to determine the appropriate level encoding.

379a ⟨*zero page defines* 25a⟩+≡ (909) ◁370b 383▷

```
  LEVEL_THRESHOLD     EQU     $BA     ; 16-bit level threshold accumulator (2 bytes)
```

Defines:
LEVEL_THRESHOLD, used in chunk 380.

379b ⟨*var stat scale* 379b⟩≡ (909)

```
      ORG     $5A7E
  STAT_SCALE:
      DC.B    $8D                     ; initial value (runtime: strength-derived level threshold step)
```

Defines:
STAT_SCALE, used in chunk 380.

379c ⟨*var stat level bits* 379c⟩≡ (909)

```
      ORG     $5A7F
  STAT_LEVEL_BITS:
      DC.B    $A0                     ; initial value (runtime: level bits during stat modification)
```

Defines:
STAT_LEVEL_BITS, used in chunk 380.

380 ⟨*modify char stats* 380⟩≡ (909)

```
    ORG     $704D
MODIFY_CHAR_STATS:
    SUBROUTINE MODIFY_CHAR_STATS
    ; Modify gold and recompute level encoding.
    ;
    ; Input:  EFFECT_REC     | pointer to entity record
    ;         NUM_VALUE+0/+1 | 16-bit gold amount
    ;         Y              | 0 = subtract, nonzero = add
    ;
    ; Output: bytes $0D/$0E (gold) updated,
    ;         byte $0C (level encoding) recomputed
    ;
    ; Modifies: STAT_SCALE, STAT_LEVEL_BITS, LEVEL_THRESHOLD/HI
    ; Clobbers: A, X, Y

    ; --- add or subtract gold ---
    BYEQ    #$00,.subtract
    LDY     #MFIELD_GOLD        ; \ add NUM_VALUE to gold
    LDA     (EFFECT_REC),Y         ;  |
    CLC                            ;  |
    ADC     NUM_VALUE              ;  |
    STA     (EFFECT_REC),Y         ;  |
    INY                            ;  | Y = MFIELD_GOLD+1
    LDA     (EFFECT_REC),Y         ;  |
    ADC     NUM_VALUE+1            ;  |
    STA     (EFFECT_REC),Y         ; /
    JMP     .aftermath
.subtract:
    LDY     #MFIELD_GOLD        ; \ subtract NUM_VALUE from gold
    SEC                            ;  |
    LDA     (EFFECT_REC),Y         ;  |
    SBC     NUM_VALUE              ;  |
    STA     (EFFECT_REC),Y         ;  |
    INY                            ;  | Y = MFIELD_GOLD+1
    LDA     (EFFECT_REC),Y         ;  |
    SBC     NUM_VALUE+1            ;  |
    STA     (EFFECT_REC),Y         ; /

    ; --- recompute level encoding from gold and strength ---
.aftermath:
    LDY     #MFIELD_STRENGTH       ; \ compute threshold step
    LDA     (EFFECT_REC),Y         ;  | from strength (bits 4-0)
    ROL2                           ;  | shift left 3 bits
    ROL                            ;  |
    AND     #$F8                   ;  |
    STA     STAT_SCALE             ; /
    LDY     #MFIELD_LEVEL_ENC      ; \ unpack current level encoding
    LDA     (EFFECT_REC),Y         ;  |
    PHA                            ;  |
```

```
    AND     #$0F                    ;  | X = low nibble (base level)
    TAX                             ;  |
    PLA                             ;  |
    ROR                             ;  | \ extract high nibble
    ROR                             ;  |  | (XP multiplier)
    ROR                             ;  |  |
    ROR                             ;  | /
    AND     #$0F                    ;  |
    TAY                             ; / Y = high nibble
.decy:
    DEY                             ; \ add high nibble to base level
    BMI     .done_cnt               ;  |
    INX                             ;  |
    BPL     .decy                   ; /
.done_cnt:
    INY                             ; Y = 0
    STY     STAT_LEVEL_BITS         ; init level bits accumulator
    STY     LEVEL_THRESHOLD         ; \ init 16-bit threshold
    STY     LEVEL_THRESHOLD+1      ; / accumulator to 0
.outer_loop:
    LDY     #MFIELD_GOLD+1
    BXEQ    #$00,.store_final       ; no levels remain -> store
    CLC                             ; \ add STAT_SCALE to threshold
    LDA     LEVEL_THRESHOLD         ;  |
    ADC     STAT_SCALE              ;  |
    STA     LEVEL_THRESHOLD         ;  |
    BCC     .no_carry               ;  |
    INC     LEVEL_THRESHOLD+1      ; /
.no_carry:
    LDA     (EFFECT_REC),Y          ; gold high byte
    BALT    LEVEL_THRESHOLD+1,.store_final ; gold_hi < thresh_hi -> done
    BNE     .next_iter              ; gold_hi > thresh_hi -> next
    DEY                             ; Y = MFIELD_GOLD
    LDA     (EFFECT_REC),Y          ; gold low byte
    BALT    LEVEL_THRESHOLD,.store_final ; gold_lo < thresh_lo -> done
.next_iter:
    DEX                             ; one fewer level
    CLC                             ; \ advance level bits by $10
    LDA     STAT_LEVEL_BITS         ;  | (one level in high nibble)
    ADC     #$10                    ;  |
    STA     STAT_LEVEL_BITS         ; /
    BNE     .outer_loop
.store_final:
    TXA                             ; remaining levels = low nibble
    ORA     STAT_LEVEL_BITS         ; merge with level bits
    LDY     #MFIELD_LEVEL_ENC
    STA     (EFFECT_REC),Y          ; store recomputed encoding
    RTS
```

Defines:
MODIFY_CHAR_STATS, used in chunks 349 and 382.

Uses EFFECT_REC 483a, LEVEL_THRESHOLD 379a, MFIELD_GOLD 561, MFIELD_STRENGTH 561, NUM_VALUE 630a, ROL2 16c, STAT_LEVEL_BITS 379c, and STAT_SCALE 379b.

MODIFY_CURRENT_STATS copies ENTITY_PTR into EFFECT_REC and tail-calls MODIFY_CHAR_STATS, which performs a treasure table lookup and applies the stat modification to the target character. This wrapper allows callers to apply treasure effects to the current entity without manually setting up the effect record pointer.

382 ⟨*modify current stats* 382⟩≡ (909)

```
      ORG     $70D4
  MODIFY_CURRENT_STATS:
      SUBROUTINE MODIFY_CURRENT_STATS
      ; Wrapper: modify stats for the current entity.
      ;
      ; Input:  ENTITY_PTR | current entity
      ;         NUM_VALUE  | amount, Y | direction (see MODIFY_CHAR_STATS)
      ; Clobbers: A, X, Y

      MOVW    ENTITY_PTR,EFFECT_REC  ; copy entity ptr to effect rec
      JMP     MODIFY_CHAR_STATS      ; tail-call
```

Defines:
MODIFY_CURRENT_STATS, used in chunks 56, 440, and 447.
Uses EFFECT_REC 483a, ENTITY_PTR 561, MODIFY_CHAR_STATS 380, MOVW 9a, and NUM_VALUE 630a.

Chapter 9

NPC AI

AI_CHOOSE_TARGET is the decision-making hub for NPC and mob turns. It selects a map position for the entity to move toward, based on the AI mode stored in CHAR_AI_MODE. Six modes are supported. Modes 0 and 1 check whether an NPC should join the player's party: PICK_RANDOM_MEMBER selects a candidate, and if one is found, the game displays a "joins party" message, merges the NPC's status flags into the party record, increments the group count, and switches the character to player control (mode 0). Mode 2 compares the entity's offense and defense stats; if offense dominates, it calls FIND_NEAREST_EVENT to locate the closest event on the map and targets that position. Mode 3 searches for the nearest hostile mob by iterating through group members with CHECK_HOSTILE and MANHATTAN_DISTANCE, keeping track of the closest one found; if none are hostile, it falls back to a default position. Mode 4 produces no target at all—the entity stays put. Mode 5 and above seek the *strongest* hostile instead of the nearest, iterating the group and tracking the highest strength value in AI_BEST_STR. After target selection, all modes converge at a common exit that unpacks the target position into column and row via POS_TO_COLROW.

AI_PREV_DIR stores the last direction the NPC moved, used by NPC_MOVE_AI as a bias to encourage straight-line travel. AI_TARGET_POS is the packed map position that AI_CHOOSE_TARGET selects as the destination, and AI_TARGET_COL holds the column component after unpacking.

AI_BEST_POS and AI_BEST_STR are zero-page temporaries at $BE and $BF that AI_CHOOSE_TARGET uses in mode 5 to track the strongest hostile mob found during the group iteration.

383 ⟨*zero page defines* 25a⟩+≡ (909) ◁379a 388b▷

```
AI_SCRATCH          EQU     $BA     ; scratch byte in AI_CHOOSE_TARGET
AI_BEST_POS         EQU     $BE     ; best hostile position in ai_choose_target
AI_BEST_STR         EQU     $BF     ; best hostile strength in ai_choose_target
```

Defines:
AI_BEST_POS, used in chunk 385.
AI_BEST_STR, used in chunk 385.
AI_SCRATCH, used in chunk 385.
Uses AI_CHOOSE_TARGET 385.

384a ⟨*var ai prev dir* 384a⟩≡ (909)

```
    ORG     $5A5D
AI_PREV_DIR:
    DC.B    $8D                     ; initial value (runtime: NPC AI: previous move direction (bias
```

Defines:
AI_PREV_DIR, used in chunks 385 and 390.

384b ⟨*var ai target pos* 384b⟩≡ (909)

```
    ORG     $5A71
AI_TARGET_POS:
    DC.B    $CF                     ; initial value (runtime: AI target position (set by AI_CHOOSE_T
```

Defines:
AI_TARGET_POS, used in chunks 385, 390, and 394.
Uses AI_CHOOSE_TARGET 385.

384c ⟨*var ai target col* 384c⟩≡ (909)

```
    ORG     $5A72
AI_TARGET_COL:
    DC.B    $D7                     ; initial value (runtime: AI target column / min distance temp)
```

Defines:
AI_TARGET_COL, used in chunk 385.

384d ⟨*var ai target row* 384d⟩≡ (909)

```
    ORG     $5A73
AI_TARGET_ROW:
    DC.B    $CE                     ; initial value (runtime: AI target row (from POS_TO_COLROW))
```

Defines:
AI_TARGET_ROW, used in chunk 385.
Uses POS_TO_COLROW 599.

385 ⟨*ai choose target* 385⟩≡ (909)

```
      ORG     $68AB
  AI_CHOOSE_TARGET:
      SUBROUTINE AI_CHOOSE_TARGET
      ; Select a movement target for an NPC based on its AI mode.
      ;
      ; Input:  ENTITY_PTR    | NPC entity record
      ;         CHAR_PTR      | character index record
      ;         MOB_PTR       | mob data pointer (set by group iteration)
      ;         CHAR_AI_MODE  | AI behavior mode (0-5+)
      ;
      ; Output: AI_TARGET_POS | packed position of chosen target
      ;         AI_TARGET_COL | target column
      ;         AI_TARGET_ROW | target row
      ;         AI_PREV_DIR   | cleared to 0
      ;
      ; Behavior:
      ;   Mode 0/1: attempt to join player's party; if no match,
      ;     fall through to mode 2.
      ;   Mode 2: if strength > HP, target nearest event;
      ;     otherwise no target.
      ;   Mode 3: scan group for nearest hostile mob.
      ;   Mode 4: no target (forced movement by NPC_MOVE_AI).
      ;   Mode 5+: scan group for strongest hostile mob.
      ;
      ; Modifies: AI_TARGET_POS, AI_TARGET_COL, AI_TARGET_ROW,
      ;           AI_PREV_DIR, AI_BEST_POS, AI_BEST_STR, TARGET_REC
      ; Clobbers: A, X, Y

      ; --- dispatch by AI mode ---
      STOB    #$00,AI_PREV_DIR
      LDA     CHAR_AI_MODE
      BAPL    #$02,.mode_ge2

      ; --- mode 0/1: check if NPC should join party ---
      JSR     PICK_RANDOM_MEMBER
      LDA     MOB_PTR+1
      BAEQ    #$00,.simple_target    ; no match -> simple target
      JSR     SET_CURSOR_ROW21
      LDA     #MSG_JOINS_PARTY                  ; "joins party" message
      JSR     DISPLAY_MESSAGE
      JSR     PRINT_MEMBER_NAME
      JSR     SCROLL_STATUS_LINE
      JSR     DELAY_WITH_ANIMATION
      LDA     #MSG_THANK_YOU                  ; confirmation message
      JSR     DISPLAY_MESSAGE
      STOB    #$00,CHAR_AI_MODE      ; switch to player control
      LDY     #MFIELD_APPEARANCE     ; \ adjust appearance (remove NPC flag)
      LDA     (ENTITY_PTR),Y         ;  |
      SEC                            ;  |
```

```
    SBC     #NPC_APPEARANCE_MIN                 ;  |
    STA     (ENTITY_PTR),Y          ; /
    LDY     #MFIELD_FLAGS           ; \ copy class bits (7-6) from source
    LDA     (MOB_PTR),Y             ;  | to entity's flags field
    AND     #FLAGS_CLASS_MASK       ;  |
    STA     AI_SCRATCH              ;  |
    LDA     (ENTITY_PTR),Y          ;  |
    AND     #FLAGS_STATUS_MASK      ;  | keep status bits (5-0)
    ORA     AI_SCRATCH              ;  | merge class bits from source
    STA     (ENTITY_PTR),Y          ; /
    LDY     #CFIELD_XP              ; \ increment party size in group record
    LDA     (CHAR_PTR),Y            ;  |
    CLC                             ;  |
    ADC     #$01                    ;  |
    STA     (CHAR_PTR),Y            ; /
    INC     TOTAL_MOB_COUNT         ; increment global count
    RTS

    ; --- mode 2: compare strength vs HP for target ---
.mode_ge2:
    BNE     .mode_ge3
.simple_target:
    LDY     #MFIELD_STRENGTH        ; strength (bits 4-0)
    LDA     (ENTITY_PTR),Y
    AND     #STRENGTH_MASK
    STA     AI_SCRATCH
    INY                             ; Y = MFIELD_HP
    LDA     (ENTITY_PTR),Y         ; HP (bits 5-0)
    AND     #HP_MASK
    BAPL    AI_SCRATCH,.use_encounter ; HP >= strength -> seek encounter
    STOB    #$00,AI_TARGET_POS      ; no target
    JMP     .done
.use_encounter:
    JSR     FIND_NEAREST_EVENT
    STA     AI_TARGET_POS
    JMP     .done

    ; --- mode 3: find nearest hostile ---
.mode_ge3:
    BAPL    #$04,.mode_ge4
    MOVW    ENTITY_PTR,TARGET_REC
    STOB    #$3C,AI_TARGET_COL      ; initial min distance = 60
    JSR     FIRST_GROUP_MEMBER
.loop3:
    JSR     CHECK_HOSTILE
    BAEQ    #$00,.skip3             ; not hostile -> skip
    LDY     #MFIELD_POSITION
    LDA     (MOB_PTR),Y
    JSR     MANHATTAN_DISTANCE
    BAGE    AI_TARGET_COL,.skip3    ; not closer -> skip
```

```
    STA     AI_TARGET_COL          ; new minimum distance
    LDY     #MFIELD_POSITION
    LDA     (MOB_PTR),Y
    STA     AI_TARGET_POS          ; target = this mob's position
.skip3:
    JSR     NEXT_GROUP_MEMBER
    LDA     MOB_PTR+1
    BNE     .loop3
    LDA     AI_TARGET_COL
    BANE    #$3C,.found3           ; found one (distance changed from $3C)
    MOVB    EVENT_POS,AI_TARGET_POS ; no hostile -> fallback to event
.found3:
    JMP     .done

    ; --- mode 4: no target (forced movement) ---
.mode_ge4:
    BNE     .mode_gt4
    STOB    #$00,AI_TARGET_POS
    JMP     .done

    ; --- mode 5+: find strongest hostile ---
.mode_gt4:
    STOB    #$00,AI_BEST_STR       ; max strength seen = 0
    MOVW    ENTITY_PTR,TARGET_REC
    JSR     FIRST_GROUP_MEMBER
.loop5:
    JSR     CHECK_HOSTILE
    BAEQ    #$00,.skip5            ; not hostile -> skip
    LDY     #MFIELD_STRENGTH
    LDA     (MOB_PTR),Y
    AND     #STRENGTH_MASK         ; strength (bits 4-0)
    BAMI    AI_BEST_STR,.skip5     ; not stronger -> skip
    STA     AI_BEST_STR            ; new max strength
    LDY     #MFIELD_POSITION
    LDA     (MOB_PTR),Y
    STA     AI_BEST_POS            ; record position
.skip5:
    JSR     NEXT_GROUP_MEMBER
    LDA     MOB_PTR+1
    BNE     .loop5
    LDA     AI_BEST_STR
    BEQ     .no_target5            ; none found
    MOVB    AI_BEST_POS,AI_TARGET_POS
    JMP     .done
.no_target5:
    MOVB    EVENT_POS,AI_TARGET_POS ; fallback to event
    JMP     .done

    ; --- convert target to col/row ---
.done:
```

```
        LDA     AI_TARGET_POS
        JSR     POS_TO_COLROW
        STA     AI_TARGET_COL
        STY     AI_TARGET_ROW
        RTS
```

Defines:
AI_CHOOSE_TARGET, used in chunks 232, 383, and 384b.
Uses AI_BEST_POS 383, AI_BEST_STR 383, AI_PREV_DIR 384a, AI_SCRATCH 383, AI_TARGET_COL 384c, AI_TARGET_POS 384b, AI_TARGET_ROW 384d, CFIELD_XP 561, CHAR_AI_MODE 531b, CHAR_PTR 591a, CHECK_HOSTILE 317, DELAY_WITH_ANIMATION 603b, DISPLAY_MESSAGE 637, ENTITY_PTR 561, EVENT_POS 532b, FIND_NEAREST_EVENT 398, FIRST_GROUP_MEMBER 482, MANHATTAN_DISTANCE 400, MFIELD_APPEARANCE 561, MFIELD_HP 561, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MOB_PTR 608, MOVB 8c, MOVW 9a, MSG_JOINS_PARTY 634, MSG_THANK_YOU 634, NEXT_GROUP_MEMBER 482, NPC_MOVE_AI 390, PICK_RANDOM_MEMBER 373, POS_TO_COLROW 599, PRINT_MEMBER_NAME 485, SCROLL_STATUS_LINE 639, SET_CURSOR_ROW21 627, STOB 8c, TARGET_REC 618b, and TOTAL_MOB_COUNT 591b.

NPC_MOVE_AI implements the movement decision for AI-controlled characters. It evaluates five options—north, south, west, east, and stay—by assigning each a score and selecting the lowest. The base score starts at $0A (or $13 if the AI has a non-zero target), and the previous direction gets a bonus of +3 to encourage straight-line movement. Each candidate direction is tested by EVALUATE_DIRECTION, which computes a score based on map tile traversability, distance to the target, and threat assessment. A randomization step multiplies each score by 4 (ROL twice) and adds a random value (range 6 normally, range $0F in AI mode 4) to break ties unpredictably. The direction with the lowest final score wins. The chosen direction index is XOR'd with 3 or 7 (depending on whether it is below 3) to convert from the internal direction numbering to the movement encoding expected by the caller.

DIR_SCORES is a 5-byte array holding the score for each direction (indices 0–4: stay, north, south, west, east). AI_DIRECTION is the index currently being evaluated, and AI_TEST_POS holds the candidate position for that direction. NPC_MOVE_AI fills the array, randomizes the scores, and picks the direction with the lowest value.

388a ⟨*defines* 25b⟩+≡ (909) ◁336 397a▷

```
MAP_WIDTH           EQU     $14     ; map grid width (20 columns)
AI_RANDOM_NORMAL    EQU     $06     ; randomization range for normal AI
AI_RANDOM_PHASE4    EQU     $0F     ; randomization range for phase 4 AI
```

Defines:
AI_RANDOM_NORMAL, used in chunk 390.
AI_RANDOM_PHASE4, used in chunk 390.
MAP_WIDTH, used in chunks 222, 320, 390, 473b, 527, 540, 545, 552, 599, 600a, and 644.

388b ⟨*zero page defines* 25a⟩+≡ (909) ◁383 399▷

```
AI_MIN_SCORE        EQU     $BA     ; best (lowest) score during direction selection
AI_BEST_DIR         EQU     $BB     ; direction index with best score
```

Defines:
AI_BEST_DIR, used in chunk 390.
AI_MIN_SCORE, used in chunk 390.

389a ⟨*var ai best dist* 389a⟩≡ (909)

```
    ORG     $5A68
AI_BEST_DIST:
    DC.B    $B1                     ; initial value (runtime: current best distance to AI target)
```

Defines:
AI_BEST_DIST, used in chunks 390 and 394.

389b ⟨*var dir scores* 389b⟩≡ (909)

```
    ORG     $5A6A
DIR_SCORES:
    HEX     A0 CA D3 D2 A0
    ; initial values (runtime: array of 5 direction scores (indexed 0-4))
```

Defines:
DIR_SCORES, used in chunks 390, 394, and 396.

389c ⟨*var ai direction* 389c⟩≡ (909)

```
    ORG     $5A6F
AI_DIRECTION:
    DC.B    $D3                     ; initial value (runtime: current direction being evaluated / ra
```

Defines:
AI_DIRECTION, used in chunks 390, 394, and 396.

389d ⟨*var ai test pos* 389d⟩≡ (909)

```
    ORG     $5A70
AI_TEST_POS:
    DC.B    $C8                     ; initial value (runtime: position being tested for direction ev
```

Defines:
AI_TEST_POS, used in chunks 390 and 394.

390 ⟨*npc move ai* 390⟩≡ (909)

```
    ORG     $6742
NPC_MOVE_AI:
    SUBROUTINE NPC_MOVE_AI
    ; Choose a movement direction for an NPC entity.
    ;
    ; Input:  ENTITY_PTR     | entity to move
    ;         CHAR_AI_MODE   | AI mode ($04 = forced movement)
    ;         AI_TARGET_POS  | desired target position (0 = none)
    ;         AI_PREV_DIR    | previous direction (penalized to avoid loops)
    ;
    ; Output: A = chosen direction index (0-4)
    ;         AI_PREV_DIR updated
    ;
    ; Behavior:
    ;   Initializes DIR_SCORES for 5 options (stay + 4 directions).
    ;   Evaluates each direction via EVALUATE_DIRECTION, which penalizes
    ;   walls, entities, and non-improving moves.  Scores are scaled,
    ;   randomized, and the lowest-scoring direction is chosen.  The
    ;   result is XOR-encoded to produce a direction value.
    ;
    ; Modifies: DIR_SCORES, AI_DIRECTION, AI_TEST_POS, AI_BEST_DIST,
    ;           AI_SAVED_POS, AI_MIN_SCORE, AI_BEST_DIR, AI_PREV_DIR
    ; Clobbers: A, X, Y

    JSR     ANIM_TICK_AND_WAIT

    ; --- check if AI should run ---
    LDA     CHAR_AI_MODE
    BAEQ    #$04,.do_ai             ; mode 4 = forced movement
    LDA     AI_TARGET_POS
    BANE    #$00,.do_ai             ; has target -> run AI
    RTS                             ; no target, not forced -> skip

    ; --- initialize direction scores ---
.do_ai:
    STOB    #$00,DIR_SCORES+1       ; \ clear scores for 4 directions
    STA     DIR_SCORES+2            ;  |
    STA     DIR_SCORES+3            ;  |
    STA     DIR_SCORES+4            ; /
    STOB    #$0A,DIR_SCORES         ; stay score = 10 (mild penalty)
    LDA     AI_TARGET_POS
    JSR     MANHATTAN_DISTANCE
    STA     AI_BEST_DIST            ; current distance to target
    BAEQ    #$00,.no_target
    STOB    #$13,DIR_SCORES         ; stay score = 19 (strong penalty if has target)
.no_target:
    LDX     AI_PREV_DIR             ; \ penalize previous direction (+3)
    BEQ     .no_prev                ;  | to avoid back-and-forth loops
    INC     DIR_SCORES,X            ;  |
```

```
    INC     DIR_SCORES,X            ;  |
    INC     DIR_SCORES,X            ; /
.no_prev:

    ; --- evaluate 4 directions ---
    STOB    #$01,AI_DIRECTION       ; direction 1: up
    LDY     #MFIELD_POSITION
    LDA     (ENTITY_PTR),Y
    SEC
    SBC     #MAP_WIDTH              ; position - 20 = one row up
    STA     AI_TEST_POS
    JSR     EVALUATE_DIRECTION

    INC     AI_DIRECTION            ; direction 2: down
    LDA     AI_TEST_POS
    CLC
    ADC     #MAP_WIDTH*2            ; position + 20 = one row down (from up)
    STA     AI_TEST_POS
    JSR     EVALUATE_DIRECTION

    INC     AI_DIRECTION            ; direction 3: left
    LDA     AI_TEST_POS
    SEC
    SBC     #MAP_WIDTH+1            ; back to center, then -1 = one col left
    STA     AI_TEST_POS
    JSR     EVALUATE_DIRECTION

    INC     AI_DIRECTION            ; direction 4: right
    INC     AI_TEST_POS             ; \ position + 2 = one col right (from left)
    INC     AI_TEST_POS             ; /
    JSR     EVALUATE_DIRECTION

    ; --- scale scores and add randomness ---
    STOB    #AI_RANDOM_NORMAL,AI_DIRECTION ; reuse AI_DIRECTION as random range
    LDA     CHAR_AI_MODE
    BANE    #$04,.not_phase4
    STOB    #AI_RANDOM_PHASE4,AI_DIRECTION ; wider randomness in phase 4
.not_phase4:
    LDX     #$04
    CLC
.score_loop:
    LDA     DIR_SCORES,X            ; \ scale score x4
    ROL2                            ; /
    STA     DIR_SCORES,X
    LDA     AI_DIRECTION            ; random range
    JSR     RANDOM_IN_RANGE
    CLC
    ADC     DIR_SCORES,X            ; add random offset
    STA     DIR_SCORES,X
    DEX
```

```
    BPL     .score_loop

    ; --- find direction with lowest score ---
    STOB    #$7F,AI_MIN_SCORE      ; initial min = 127
    LDX     #$04
.find_min:
    LDA     DIR_SCORES,X
    BAGE    AI_MIN_SCORE,.not_lower
    STA     AI_MIN_SCORE           ; new minimum
    STX     AI_BEST_DIR            ; record direction
.not_lower:
    DEX
    BPL     .find_min

    ; --- encode direction index to movement value ---
    LDA     AI_BEST_DIR
    BAPL    #$03,.ge3
    EOR     #$03                   ; directions 0-2: XOR 3
    BPL     .store_dir
.ge3:
    EOR     #$07                   ; directions 3-4: XOR 7
.store_dir:
    STA     AI_PREV_DIR            ; save for next turn's bias
    LDA     AI_BEST_DIR            ; return direction index
    RTS
```

Defines:
NPC_MOVE_AI, used in chunks 232 and 385.

Uses AI_BEST_DIR 388b, AI_BEST_DIST 389a, AI_DIRECTION 389c, AI_MIN_SCORE 388b, AI_PREV_DIR 384a, AI_RANDOM_NORMAL 388a, AI_RANDOM_PHASE4 388a, AI_SAVED_POS 393, AI_TARGET_POS 384b, AI_TEST_POS 389d, ANIM_TICK_AND_WAIT 605b, CHAR_AI_MODE 531b, DIR_SCORES 389b, ENTITY_PTR 561, EVALUATE_DIRECTION 394, MANHATTAN_DISTANCE 400, MAP_WIDTH 388a, MFIELD_POSITION 561, RANDOM_IN_RANGE 597, ROL2 16c, and STOB 8c.

EVALUATE_DIRECTION scores a single candidate direction for the NPC movement AI. It temporarily moves the entity to the test position (AI_TEST_POS) and evaluates the result. If the test position matches the AI's target, a large penalty is added (the entity is already there). The routine computes the Manhattan distance to the target from the test position; if the distance is not closer than the current distance, a smaller penalty is added. It then calls FIND_ENTITY_AT_POS to check what occupies the candidate square: empty squares return cleanly, walls add a large penalty, and entities are scored based on their type byte—friendly mobs, hostile mobs of various strength ranges, and locked/safe events each contribute different penalty amounts via INC_MOB_COUNTERS. The entity's position is restored before returning.

393 ⟨*var ai saved pos* 393⟩≡ (909)

```
    ORG     $5A69
AI_SAVED_POS:
    DC.B    $8D                     ; initial value (runtime: saved entity position during direction
```

Defines:
AI_SAVED_POS, used in chunks 390 and 394.

394 ⟨*evaluate direction* 394⟩≡ (909)

```
    ORG     $6815
EVALUATE_DIRECTION:
    SUBROUTINE EVALUATE_DIRECTION
    ; Score a candidate direction for NPC movement AI.
    ;
    ; Input:  AI_TEST_POS     | candidate position to evaluate
    ;         AI_TARGET_POS   | AI's desired target position
    ;         AI_DIRECTION    | index into DIR_SCORES for this direction
    ;         AI_BEST_DIST    | current best distance to target
    ;         ENTITY_PTR      | entity being moved
    ;
    ; Output: DIR_SCORES[AI_DIRECTION] updated with penalty points
    ;
    ; Behavior:
    ;   Penalizes the direction based on: already at target (+9),
    ;   not closer than current best (+2), wall (+7), and entity
    ;   type at the candidate square (friendly +3, hostile +1..+6,
    ;   locked event +7, safe event +1).
    ;
    ; Modifies: DIR_SCORES, AI_SAVED_POS
    ; Clobbers: A, X, Y

    ; --- check if already at target ---
    LDA     AI_TARGET_POS
    BAEQ    AI_TEST_POS,.skip_target
    LDY     #$09                    ; heavy penalty: already there
    JSR     INC_MOB_COUNTERS

    ; --- check distance improvement ---
.skip_target:
    LDY     #MFIELD_POSITION        ; \ save entity's current position
    LDA     (ENTITY_PTR),Y          ;  |
    STA     AI_SAVED_POS            ; /
    LDA     AI_TEST_POS             ; \ temporarily set entity to test pos
    STA     (ENTITY_PTR),Y          ; / (for MANHATTAN_DISTANCE)
    LDA     AI_TARGET_POS
    BEQ     .no_distance            ; no target -> skip distance check
    JSR     MANHATTAN_DISTANCE
    BALT    AI_BEST_DIST,.no_distance ; closer -> no penalty
    LDY     #$02                    ; penalty: not closer
    JSR     INC_MOB_COUNTERS

    ; --- restore position and check occupant ---
.no_distance:
    LDY     #MFIELD_POSITION        ; \ restore original position
    LDA     AI_SAVED_POS            ;  |
    STA     (ENTITY_PTR),Y          ; /
    LDA     AI_TEST_POS
    JSR     FIND_ENTITY_AT_POS      ; what's at the candidate square?
```

```
    LDA     ENTITY_REC+1
    BNE     .has_entity             ; nonzero hi byte -> entity found
    LDA     ENTITY_REC
    BNE     .wall                   ; nonzero lo byte -> wall/obstacle
    RTS                             ; empty square -> no penalty

.wall:
    LDY     #$07                    ; heavy penalty: wall
    JMP     INC_MOB_COUNTERS

    ; --- classify entity by type byte ---
.has_entity:
    BALT    #$80,.mob               ; bit 7 clear -> mob
    RTS                             ; bit 7 set -> player char, no penalty

.mob:
    LDY     #MFIELD_TYPE
    LDA     (ENTITY_REC),Y        ; entity type/class byte
    BAGE    #$80,.ge80
    LDY     #$03                    ; type < $80: friendly mob
    JMP     INC_MOB_COUNTERS
.ge80:
    BAGE    #$C0,.geC0
    LDY     #$06                    ; type $80-$BF: hostile (strong)
    JMP     INC_MOB_COUNTERS
.geC0:
    BAGE    #$D2,.geD2
    LDY     #$01                    ; type $C0-$D1: hostile (weak)
    JMP     INC_MOB_COUNTERS
.geD2:
    BAGE    #$D5,.geD5
    LDY     #$02                    ; type $D2-$D4: hostile (medium)
    JMP     INC_MOB_COUNTERS
.geD5:
    BAGE    #$D8,.geD8
    LDY     #$03                    ; type $D5-$D7: shop/special
    JMP     INC_MOB_COUNTERS

    ; --- event entity: check lock status ---
.geD8:
    INY                             ; Y = MFIELD_NEXT (byte 2: event data for type >= $D8)
    LDA     (ENTITY_REC),Y
    BAGE    #$40,.safe
    LDY     #$07                    ; locked event: heavy penalty
    JMP     INC_MOB_COUNTERS
.safe:
    LDY     #$01                    ; safe event: light penalty
; falls through to INC_MOB_COUNTERS
```

Defines:
EVALUATE_DIRECTION, used in chunk 390.

Uses AI_BEST_DIST 389a, AI_DIRECTION 389c, AI_SAVED_POS 393, AI_TARGET_POS 384b, AI_TEST_POS 389d, DIR_SCORES 389b, ENTITY_PTR 561, ENTITY_REC 541b, FIND_ENTITY_AT_POS 469, INC_MOB_COUNTERS 396, MANHATTAN_DISTANCE 400, MFIELD_NEXT 561, and MFIELD_POSITION 561.

INC_MOB_COUNTERS adds Y to the direction score for the current AI direction being evaluated. The current direction index is in AI_DIRECTION, and the scores are in the DIR_SCORES array indexed by X. The routine decrements Y in a loop, incrementing DIR_SCORES,X each time, returning when Y reaches zero. This is used by EVALUATE_DIRECTION to penalize directions based on what occupies the candidate square.

396 ⟨*inc mob counters* 396⟩≡ (909)

```
    ORG     $689F
INC_MOB_COUNTERS:
    SUBROUTINE INC_MOB_COUNTERS
    ; Add Y to the direction penalty score for the current AI direction.
    ;
    ; Input:  Y             | penalty count (decremented to zero)
    ;         AI_DIRECTION  | index into DIR_SCORES array
    ;
    ; Output: DIR_SCORES[AI_DIRECTION] incremented by Y
    ; Clobbers: X, Y, flags

    LDX     AI_DIRECTION            ; current direction index
    DEY                             ; pre-decrement (Y=0 -> done)
    BPL     .do_inc
    RTS
.do_inc:
    INC     DIR_SCORES,X            ; add one penalty point
    BPL     INC_MOB_COUNTERS        ; loop until Y < 0
```

Defines:
INC_MOB_COUNTERS, used in chunk 394.
Uses AI_DIRECTION 389c and DIR_SCORES 389b.

FIND_NEAREST_EVENT scans the character's event list for the closest non-activated event, measured by Manhattan distance from the entity's current position. It loads the event list pointer from bytes 6–7 of the character record at CHAR_PTR and walks the 3-byte entries: entries with byte 1 at $C0 or above are already activated and are skipped, entries marked $FE (empty slot) are skipped, and $FF marks the end of the list. For each eligible entry, the routine calls MANHATTAN_DISTANCE and keeps track of the nearest one found. The winning position is returned in A and stored at $5A13 for the AI to use as a movement target.

397a ⟨*defines* 25b⟩+≡ (909) ◁388a 407a▷

```
    ; --- Event record field offsets (3-byte entries) ---
EVFIELD_POSITION    EQU     $00     ; byte 0: packed map position
EVFIELD_TYPE        EQU     $01     ; byte 1: event type (>= $C0 = activated)
EVFIELD_VALUE       EQU     $02     ; byte 2: event value (low byte)
EVENT_RECORD_SIZE   EQU     $03     ; size of one event list entry (3 bytes)
EVENT_ACTIVATED     EQU     $C0     ; event type threshold: >= $C0 = already activated
EVENT_ACCUM_MAX     EQU     $D1     ; event types < this are accumulable ($C0-$D0)
EVENT_VALUE_HI_MASK EQU     $07     ; mask for value high bits stored in type byte
EVENT_EMPTY         EQU     $FE     ; event list: empty slot marker
EVENT_END           EQU     $FF     ; event list: end-of-list marker
```

Defines:
EVENT_ACCUM_MAX, used in chunk 454.
EVENT_ACTIVATED, used in chunks 232, 398, 454, and 542.
EVENT_EMPTY, used in chunks 232, 342, 371, 398, 454, 459, 462, and 542.
EVENT_END, used in chunks 398, 454, and 469.
EVENT_RECORD_SIZE, used in chunks 398, 454, and 469.
EVENT_VALUE_HI_MASK, used in chunk 454.
EVFIELD_POSITION, used in chunks 354, 398, and 454.
EVFIELD_TYPE, used in chunks 232, 354, 398, 454, and 542.
EVFIELD_VALUE, used in chunk 454.

397b ⟨*var nearest pos* 397b⟩≡ (909)

```
    ORG     $5A13
NEAREST_POS:
    DC.B    $A0                     ; initial value (runtime: position of nearest event found)
```

Defines:
NEAREST_POS, used in chunk 398.

397c ⟨*var nearest dist* 397c⟩≡ (909)

```
    ORG     $5A14
NEAREST_DIST:
    DC.B    $A3                     ; initial value (runtime: current minimum distance during event
```

Defines:
NEAREST_DIST, used in chunk 398.

398 ⟨*find nearest event* 398⟩≡ (909)

```
    ORG     $138D
FIND_NEAREST_EVENT:
    SUBROUTINE FIND_NEAREST_EVENT
    ; Find the nearest non-activated event to the current entity.
    ;
    ; Input:  ENTITY_PTR  | entity whose position to measure from
    ;         CHAR_PTR    | bytes 6-7 of record hold event list pointer
    ;
    ; Output: A = packed position of nearest event
    ;
    ; Behavior:
    ;   Walks the 3-byte event list (position, type, data).  Skips
    ;   activated events (type >= $C0) and empty slots ($FE).  For
    ;   each candidate, computes Manhattan distance and keeps the
    ;   closest.
    ;
    ; Modifies: NEAREST_DIST, NEAREST_POS, CANDIDATE_POS, EVENT_PTR
    ; Clobbers: A, X, Y

    STOB    #$7F,NEAREST_DIST      ; initial min = 127
    LDWIY    CHAR_PTR,#CFIELD_EVENTS,EVENT_PTR ; load event list pointer
    JMP     .check_end

    ; --- evaluate one event ---
.eval:
    LDY     #EVFIELD_TYPE
    LDA     (EVENT_PTR),Y          ; event type byte
    BAGE    #EVENT_ACTIVATED,.advance ; activated -> skip
    LDY     #EVFIELD_POSITION
    LDA     (EVENT_PTR),Y          ; event position byte
    STA     CANDIDATE_POS
    JSR     MANHATTAN_DISTANCE
    BAPL    NEAREST_DIST,.advance  ; not closer -> skip
    STA     NEAREST_DIST           ; new minimum distance
    MOVB    CANDIDATE_POS,NEAREST_POS ; record its position

    ; --- advance to next entry ---
.advance:
    LDA     EVENT_PTR
    CLC
    ADC     #EVENT_RECORD_SIZE     ; next 3-byte entry
    STA     EVENT_PTR
    BCC     .check_end
    INC     EVENT_PTR+1

    ; --- check list termination ---
.check_end:
    LDY     #EVFIELD_POSITION
    LDA     (EVENT_PTR),Y
```

```
    BAEQ    #EVENT_EMPTY,.advance  ; empty slot -> skip
    BANE    #EVENT_END,.eval       ; not end -> evaluate
    LDA     NEAREST_POS            ; return nearest position
    RTS
```

Defines:
FIND_NEAREST_EVENT, used in chunks 385, 532b, and 535.
Uses CANDIDATE_POS 534d, CFIELD_EVENTS 561, CHAR_PTR 591a, ENTITY_PTR 561, EVENT_ACTIVATED 397a, EVENT_EMPTY 397a, EVENT_END 397a, EVENT_PTR 353a, EVENT_RECORD_SIZE 397a, EVFIELD_POSITION 397a, EVFIELD_TYPE 397a, LDWIY 17, MANHATTAN_DISTANCE 400, MOVB 8c, NEAREST_DIST 397c, NEAREST_POS 397b, and STOB 8c.

MANHATTAN_DISTANCE computes the taxicab distance between a given packed position (in A) and the current entity's position at offset 3 of ENTITY_PTR. It unpacks both positions into column/row pairs with POS_TO_COLROW, takes the absolute value of each axis difference using the standard 6502 negate-if-negative idiom (EOR $FF, ADC $01), and sums the results. The function temporarily borrows FONT_COL and FONT_ROW as scratch storage—these are safe to clobber here since no rendering is in progress.

399 ⟨*zero page defines* 25a⟩+≡ (909) ◁388b 401▷

```
DIST_COL                EQU     $BA     ; |col difference| scratch in MANHATTAN_DISTANCE
```

Defines:
DIST_COL, used in chunk 400.
Uses MANHATTAN_DISTANCE 400.

400 ⟨*manhattan distance* 400⟩≡ (909)

```
    ORG     $13DB
MANHATTAN_DISTANCE:
    SUBROUTINE MANHATTAN_DISTANCE
    ; Compute Manhattan distance between a position and the current entity.
    ;
    ; Input:  A = packed map position to measure from
    ; Output: A = |col_diff| + |row_diff|
    ;
    ; Uses ENTITY_PTR byte MFIELD_POSITION as the entity's position.
    ; FONT_COL/FONT_ROW and DIST_COL used as scratch.
    ; Clobbers: A, X, Y

    ; --- unpack target position ---
    JSR     POS_TO_COLROW          ; A -> col in A, row in Y
    STA     FONT_COL               ; save target col
    STY     FONT_ROW               ; save target row

    ; --- unpack entity position ---
    LDY     #MFIELD_POSITION
    LDA     (ENTITY_PTR),Y         ; entity's packed position
    JSR     POS_TO_COLROW

    ; --- |col difference| ---
    SEC
    SBC     FONT_COL               ; entity col - target col
    BCS     .pos1
    EOR     #$FF                   ; \ negate if negative
    ADC     #$01                   ; / (two's complement abs)
.pos1:
    STA     DIST_COL               ; save |delta col|

    ; --- |row difference| ---
    TYA
    SEC
    SBC     FONT_ROW               ; entity row - target row
    BCS     .pos2
    EOR     #$FF                   ; \ negate if negative
    ADC     #$01                   ; /
.pos2:
    CLC
    ADC     DIST_COL               ; A = |delta col| + |delta row|
    RTS
```

Defines:
MANHATTAN_DISTANCE, used in chunks 365, 385, 390, 394, 398, and 399.
Uses DIST_COL 399, ENTITY_PTR 561, FONT_COL 492a, FONT_ROW 534c, MFIELD_POSITION 561, and POS_TO_COLROW 599.

9.1 Shops

DISPLAY_SHOP presents the shop interface when a character enters a store tile. It only runs for player-controlled characters (AI mode 0). The routine loads the shop data pointer and advances it past earlier shop entries based on the event type (byte 1 minus $D5 gives the shop index). It then iterates through the 8-bit item availability mask in byte 2: for each set bit, it computes the item's price by summing price components based on the location style (1–4 components), formats the price with NUM_TO_DECIMAL, copies the item name string into the display buffer, and enters the game's alternative input loop via START_GAME_INPUT so the player can choose to buy. The item mask is rotated right to advance through items, wrapping around when all bits have been checked.

401 ⟨*zero page defines* 25a⟩+≡ (909) ◁399 411b▷

```
SHOP_DATA_PTR          EQU     $E6     ; ZP pointer to current shop item record (2 bytes)
```

Defines:
SHOP_DATA_PTR, used in chunks 56, 404, 407b, and 408.

402 ⟨*shop table* 402⟩≡ (909)

```
    ORG     $7EDD
SHOP_TABLE:
    ; === Weapon shop record (41 bytes) ===
    ; slot 0 [+$00]: AN IRON SWORD (price $0A)
    DC.W    SHOP_BUY_EFFECT_0
    DC.W    S_IRON_SWORD
    DC.B    $0A
    ; slot 1 [+$05]: A STEEL SWORD (price $32)
    DC.W    SHOP_BUY_EFFECT_1
    DC.W    S_STEEL_SWORD
    DC.B    $32
    ; slot 2 [+$0A]: display format (masked out)
    HEX     B7 C5 A0 AE A0
    ; slot 3 [+$0F]: AN OAK CLUB (price $03)
    DC.W    SHOP_BUY_EFFECT_2
    DC.W    S_OAK_CLUB
    DC.B    $03
    ; slot 4 [+$14]: display format (masked out)
    HEX     B2 AE A0 A0 AE
    ; [+$19]: A STEEL DAGGER (price $0F)
    DC.W    SHOP_BUY_EFFECT_3
    DC.W    S_STEEL_DAGGER
    DC.B    $0F
    ; [+$1E]: A SHIV (price $0A)
    DC.W    SHOP_BUY_EFFECT_4
    DC.W    S_SHIV
    DC.B    $0A
    ; [+$23]: display format
    HEX     B6 A0 B4 A0 C6
    ; [+$28]: scene number
    DC.B    $32                ; scene $32 (weapon shop)

    ; === Armor shop record (41 bytes) ===
    ; slot 0 [+$00]: PLATE ARMOR (price $50)
    DC.W    XP_AWARD_0
    DC.W    S_PLATE_ARMOR
    DC.B    $50
    ; slot 1 [+$05]: RITHMIL PLATE ARMOR (price $C8)
    DC.W    XP_AWARD_1
    DC.W    S_RITHMIL_PLATE
    DC.B    $C8
    ; slot 2 [+$0A]: CHAIN MAIL (price $28)
    DC.W    XP_AWARD_2
    DC.W    S_CHAIN_MAIL
    DC.B    $28
    ; slot 3 [+$0F]: RITHMIL CHAIN MAIL (price $64)
    DC.W    XP_AWARD_3
    DC.W    S_RITHMIL_CHAIN
    DC.B    $64
```

```
    ; slot 4 [+$14]: LEATHER ARMOR (price $08)
    DC.W    XP_AWARD_4
    DC.W    S_LEATHER_ARMOR
    DC.B    $08
    ; [+$19]: display format
    HEX     A0 B5 CF AE B5 A0 CB CC B1 CD D0 AF B1 C1 B9
    ; [+$28]: scene number
    DC.B    $33                ; scene $33 (armor shop)
```

Defines:
SHOP_TABLE, used in chunk 404.
Uses SHOP_BUY_EFFECT_0 409, SHOP_BUY_EFFECT_1 409, SHOP_BUY_EFFECT_2 409, SHOP_BUY_EFFECT_3 409, SHOP_BUY_EFFECT_4 409, S_CHAIN_MAIL 62, S_IRON_SWORD 62, S_LEATHER_ARMOR 62, S_OAK_CLUB 62, S_PLATE_ARMOR 62, S_RITHMIL_CHAIN 62, S_RITHMIL_PLATE 62, S_SHIV 62, S_STEEL_DAGGER 62, S_STEEL_SWORD 62, XP_AWARD_0 412, XP_AWARD_1 412, XP_AWARD_2 412, XP_AWARD_3 412, and XP_AWARD_4 412.

403a ⟨*var shop item mask* 403a⟩≡ (909)

```
    ORG     $5A80
SHOP_ITEM_MASK:
    DC.B    $CC                    ; initial value (runtime: shop: bitmask for current item ($80 ro
```

Defines:
SHOP_ITEM_MASK, used in chunk 404.

403b ⟨*var shop valid mask* 403b⟩≡ (909)

```
    ORG     $5A83
SHOP_VALID_MASK:
    DC.B    $A0                    ; initial value (runtime: copy of item validity bitmask during s
```

Defines:
SHOP_VALID_MASK, used in chunk 404.

403c ⟨*var location style bits* 403c⟩≡ (909)

```
    ORG     $5A84
LOCATION_STYLE_BITS:
    DC.B    $A8                    ; initial value (runtime: location style (0-3), used as modifier
```

Defines:
LOCATION_STYLE_BITS, used in chunks 404, 409, and 412.

404 ⟨*display shop* 404⟩≡ (909)

```
    ORG     $70DF
DISPLAY_SHOP:
    SUBROUTINE DISPLAY_SHOP
    ; Display the shop menu for a trading post entity.
    ;
    ; Inputs:
    ;   CHAR_AI_MODE   | if nonzero, returns immediately (AI can't shop)
    ;   ENTITY_REC     | pointer to shop entity record:
    ;                     +SHOP_TYPE_FIELD: shop type ($D5=weapons, $D6=armor)
    ;                     +SHOP_AVAIL_FIELD: item validity bitmask
    ;   LOCATION_STYLE | bits 7-6 encode location tier (0-3), scales prices
    ;
    ; Behavior:
    ;   Selects the shop record from SHOP_TABLE, displays the shop scene,
    ;   then iterates 8 item slots.  For each valid slot (per bitmask),
    ;   computes price = base_price * (tier + 1), formats the
    ;   "PAY nnnnnn FOR [item]" string, and enters START_GAME_INPUT.
    ;
    ; Modifies:
    ;   SHOP_DATA_PTR, SHOP_VALID_MASK, SHOP_ITEM_MASK, SHOP_ITEM_OFFSET,
    ;   SHOP_PRICE, SHOP_PRICE+1, LOCATION_STYLE_BITS,
    ;   SHOP_PRICE_DIGITS, SHOP_ITEM_NAME, PRINT_STRING_ADDR
    ; Clobbers: A, X, Y

    LDA     CHAR_AI_MODE
    BEQ     .do_shop
    RTS
.do_shop:
    STOW    SHOP_TABLE,SHOP_DATA_PTR
    LDY     #SHOP_TYPE_FIELD        ; shop type ($D5=weapons, $D6=armor)
    LDA     (ENTITY_REC),Y
    SEC
    SBC     #$D5                    ; record index (0 or 1)
    TAX
.count_loop:
    DEX
    BMI     .counted
    JSR     ADVANCE_SHOP_PTR        ; skip to selected record
    JMP     .count_loop
.counted:
    LDY     #SHOP_AVAIL_FIELD       ; item validity bitmask
    LDA     (ENTITY_REC),Y
    STA     SHOP_VALID_MASK
    JSR     SET_CURSOR_ROW21
    LDY     #SHOP_SCENE_OFFSET      ; display shop scene
    LDA     (SHOP_DATA_PTR),Y
    JSR     SCENE_LOOP
    JSR     LOAD_CHAR_NAME_PTR
    JSR     SETUP_CHAR_SPRITE
```

```
    ; --- iterate through item slots ---
.item_start:
    STOB    #$80,SHOP_ITEM_MASK    ; start at slot 0 (bit 7)
    STOB    #$00,SHOP_ITEM_OFFSET  ; start at record offset 0
.item_loop:
    LDA     SHOP_VALID_MASK        ; check if current slot is valid
    AND     SHOP_ITEM_MASK
    BNE     .show_item
    JMP     .next_item

    ; --- display one item ---
.show_item:
    STOB    #$00,SHOP_PRICE        ; clear price accumulator
    STA     SHOP_PRICE+1
    LDY     SHOP_ITEM_OFFSET       ; Y = offset + 4 (price byte)
    INY
    INY
    INY
    INY
    LDA     LOCATION_STYLE         ; \ extract bits 7-6: location tier (0-3)
    ROL2                           ;  | rotate down to bits 1-0
    ROL2                           ;  |
    AND     #$03                   ; /
    TAX                            ; X = tier (price loop count)
    STA     LOCATION_STYLE_BITS
.price_loop:                       ; price = base_price * (tier + 1)
    JSR     ADD_PRICE_COMPONENT
    DEX
    BPL     .price_loop
    LDA     #$20                   ; \ clear price and item name fields
    LDX     #$04                   ;  | in "PAY 000000 FOR" string
.clear1:                           ;  |
    STA     SHOP_PRICE_DIGITS,X    ;  |
    DEX                            ;  |
    BPL     .clear1                ;  |
    LDX     #$13                   ;  |
.clear2:                           ;  |
    STA     SHOP_ITEM_NAME,X       ;  |
    DEX                            ;  |
    BPL     .clear2                ; /
    MOVB    SHOP_PRICE,PRINT_STRING_ADDR
    MOVB    SHOP_PRICE+1,PRINT_STRING_ADDR+1
    STOW    SHOP_PRICE_DIGITS,$BA
    JSR     NUM_TO_DECIMAL         ; write price digits
    LDY     SHOP_ITEM_OFFSET       ; \ copy item name string address
    INY                            ;  | from shop record (+2/+3)
    INY                            ;  | into PRINT_STRING_ADDR
    LDA     (SHOP_DATA_PTR),Y      ;  |
    STA     PRINT_STRING_ADDR      ;  |
```

```
    INY                             ;  |
    LDA     (SHOP_DATA_PTR),Y       ;  |
    STA     PRINT_STRING_ADDR+1     ; /
    LDY     #$00                    ; \ copy item name into display string
    LDA     (PRINT_STRING_ADDR),Y   ;  | Y = string length
    TAY                             ;  |
    TAX                             ;  |
    DEX                             ;  | X = length - 1 (dest index)
.copy_name:                         ;  |
    LDA     (PRINT_STRING_ADDR),Y   ;  |
    STA     SHOP_ITEM_NAME,X        ;  |
    DEY                             ;  |
    DEX                             ;  |
    BPL     .copy_name              ; /
    JMP     START_GAME_INPUT        ; enter buy/browse/no-sale menu

    ; --- advance to next slot ---
BROWSE_HANDLER:                         ; entry from "BUY SOMETHING ELSE" menu
.next_item:
    CLC
    LDA     SHOP_ITEM_MASK          ; rotate mask right
    ROR
    BCC     .more_items             ; didn't carry out -> more slots
    JMP     .item_start             ; carried out -> restart
.more_items:
    STA     SHOP_ITEM_MASK
    LDA     SHOP_ITEM_OFFSET        ; advance offset by 5
    ADC     #$05
    STA     SHOP_ITEM_OFFSET
    JMP     .item_loop
```

Defines:
BROWSE_HANDLER, used in chunks 56, 285, 409, and 412.
DISPLAY_SHOP, used in chunks 232 and 310.

Uses ADD_PRICE_COMPONENT 408, ADVANCE_SHOP_PTR 407b, CHAR_AI_MODE 531b, ENTITY_REC 541b, LOAD_CHAR_NAME_PTR 486a, LOCATION_STYLE 563a, LOCATION_STYLE_BITS 403c, MOVB 8c, NUM_TO_DECIMAL 632, PRINT_STRING_ADDR 576c, ROL2 16c, SCENE_LOOP 527, SETUP_CHAR_SPRITE 487, SET_CURSOR_ROW21 627, SHOP_AVAIL_FIELD 407a, SHOP_DATA_PTR 401, SHOP_ITEM_MASK 403a, SHOP_ITEM_NAME 407a, SHOP_ITEM_OFFSET 55, SHOP_PRICE 54c, SHOP_PRICE_DIGITS 407a, SHOP_SCENE_OFFSET 407a, SHOP_TABLE 402, SHOP_TYPE_FIELD 407a, SHOP_VALID_MASK 403b, START_GAME_INPUT 286, STOB 8c, and STOW 8a.

ADVANCE_SHOP_PTR adds SHOP_RECORD_SIZE (41 bytes) to SHOP_DATA_PTR, advancing it to the next shop entry in the data table.

407a ⟨*defines* 25b⟩+≡ (909) ◁397a 411a▷

```
SHOP_RECORD_SIZE        EQU     $29     ; size of one shop record (41 bytes)
SHOP_SCENE_OFFSET       EQU     $28     ; offset of scene number within shop record
SHOP_TYPE_FIELD         EQU     $01     ; entity record byte 1: shop type ($D5=weapons, $D6=armor)
SHOP_AVAIL_FIELD        EQU     $02     ; entity record byte 2: item validity bitmask
SHOP_PRICE_DIGITS       EQU     S_PAY_FOR+5  ; 6-byte price field in "PAY 000000 FOR" string
SHOP_ITEM_NAME          EQU     S_PAY_FOR+16 ; 20-byte item name field after "FOR " in pay string
```

Defines:
SHOP_AVAIL_FIELD, used in chunk 404.
SHOP_ITEM_NAME, used in chunk 404.
SHOP_PRICE_DIGITS, used in chunk 404.
SHOP_RECORD_SIZE, used in chunk 407b.
SHOP_SCENE_OFFSET, used in chunk 404.
SHOP_TYPE_FIELD, used in chunk 404.
Uses S_PAY_FOR 310.

407b ⟨*advance shop ptr* 407b⟩≡ (909)

```
    ORG     $71F0
ADVANCE_SHOP_PTR:
    SUBROUTINE ADVANCE_SHOP_PTR
    ; Advance SHOP_DATA_PTR to the next shop record.
    ;
    ; Input:  SHOP_DATA_PTR | current record
    ; Output: SHOP_DATA_PTR += SHOP_RECORD_SIZE (41 bytes)
    ; Clobbers: A

    CLC
    LDA     SHOP_DATA_PTR
    ADC     #SHOP_RECORD_SIZE
    STA     SHOP_DATA_PTR
    BCC     .done
    INC     SHOP_DATA_PTR+1
.done:
    RTS
```

Defines:
ADVANCE_SHOP_PTR, used in chunk 404.
Uses SHOP_DATA_PTR 401 and SHOP_RECORD_SIZE 407a.

ADD_PRICE_COMPONENT adds one byte from the shop data (indexed by Y) to the running 16-bit price total in SHOP_PRICE/SHOP_PRICE+1. It is called in a loop, once for each price component determined by the location style.

408 ⟨*add price component* 408⟩≡ (909)

```
      ORG     $71FC
  ADD_PRICE_COMPONENT:
      SUBROUTINE ADD_PRICE_COMPONENT
      ; Add one price byte from shop record to SHOP_PRICE.
      ;
      ; Input:  Y = offset into shop record, SHOP_DATA_PTR
      ; Output: SHOP_PRICE += (SHOP_DATA_PTR),Y
      ; Clobbers: A

      CLC
      LDA     (SHOP_DATA_PTR),Y
      ADC     SHOP_PRICE
      STA     SHOP_PRICE
      BCC     .done
      INC     SHOP_PRICE+1
  .done:
      RTS
```

Defines:
ADD_PRICE_COMPONENT, used in chunk 404.
Uses SHOP_DATA_PTR 401 and SHOP_PRICE 54c.

9.1.1 Shop buy effects

When a player buys an item, the buy handler at SHOP_BUY_HANDLER dispatches through a function pointer in the shop data. Five entry points (indexed 0–4) apply the purchased item's effect to the entity record at ENTITY_PTR. Each entry sets X (item type 0–4) and Y (target field offset: 7 or 9), then falls through to the common handler.

The common handler loads a base value from the effect table at SHOP_EFFECT_TABLE[X], adds LOCATION_STYLE_BITS (LOCATION_STYLE_BITS), and stores the result at byte Y of the entity record. It then extracts bits 2–1 of byte 5 (defense field) and adds that bonus to the same field. Finally it jumps to BROWSE_HANDLER (the browse handler, to continue shopping).

409 ⟨*shop buy effects* 409⟩≡ (909)

```
    ORG     $720B
SHOP_BUY_EFFECT_0:
    LDX     #$00
    LDY     #MFIELD_ATTACK     ; attack stat (swords, clubs)
    BNE     APPLY_SHOP_EFFECT

SHOP_BUY_EFFECT_1:
    LDX     #$01
    LDY     #MFIELD_ATTACK
    BNE     APPLY_SHOP_EFFECT

SHOP_BUY_EFFECT_2:
    LDX     #$02
    LDY     #MFIELD_ATTACK
    BNE     APPLY_SHOP_EFFECT

SHOP_BUY_EFFECT_3:
    LDX     #$03
    LDY     #MFIELD_DEFENSE    ; defense stat (armor, daggers, shivs)
    BNE     APPLY_SHOP_EFFECT

SHOP_BUY_EFFECT_4:
    LDX     #$04
    LDY     #MFIELD_DEFENSE
    BNE     APPLY_SHOP_EFFECT

APPLY_SHOP_EFFECT:
    LDA     SHOP_EFFECT_TABLE,X ; base effect value
    CLC
    ADC     LOCATION_STYLE_BITS ; + location tier modifier
    STA     (ENTITY_PTR),Y     ; store to weapon/armor field
    TYA                        ; \ save target field offset
    TAX                        ; / in X
    LDY     #MFIELD_STRENGTH
```

```
LDA     (ENTITY_PTR),Y      ; strength/defense byte
ROR                         ; \ extract bits 2-1
ROR                         ;  | as bits 1-0
ROR                         ; /
AND     #$03                ; mask to 2 bits (defense bonus)
PHA
TXA                         ; restore target field offset
TAY
PLA
CLC
ADC     (ENTITY_PTR),Y      ; add defense bonus
STA     (ENTITY_PTR),Y
JMP     BROWSE_HANDLER      ; return to browse handler
```

Defines:
APPLY_SHOP_EFFECT, never used.
SHOP_BUY_EFFECT_0, used in chunk 402.
SHOP_BUY_EFFECT_1, used in chunk 402.
SHOP_BUY_EFFECT_2, used in chunk 402.
SHOP_BUY_EFFECT_3, used in chunk 402.
SHOP_BUY_EFFECT_4, used in chunk 402.

Uses BROWSE_HANDLER 404, ENTITY_PTR 561, LOCATION_STYLE_BITS 403c, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_STRENGTH 561, and SHOP_EFFECT_TABLE 62.

9.1.2 XP award

Five entry points (indexed 0–4) award experience points after combat actions. Each sets X to the action type and falls through to the common handler.

The common handler:

- Extracts the current level from byte 11 (bits 4–0) and saves the upper bits (7–5).
- Extracts pending XP from byte 8 via EXTRACT_PENDING_XP (bits 6–4 → 0–7) and adds it to the saved level.
- Loads the XP award from XP_AWARD_TABLE[X], adds LOCATION_STYLE_BITS, and stores to byte 8.
- Extracts the new pending XP, and if it exceeds the accumulated level, subtracts 16 ($10) from byte 8 repeatedly until it fits.
- Computes the remaining level (old level − new pending XP) and merges it with the saved upper bits back into byte 11.
- Jumps to BROWSE_HANDLER (browse handler).

411a ⟨*defines* 25b⟩+≡ (909) ◁407a 425▷

```
      ; (ENTITY_LEVEL_FIELD and ENTITY_XP_FIELD removed; use MFIELD_LEVEL and MFIELD_XP)
  STRENGTH_MASK           EQU     $1F     ; byte 5: mask for strength bits 4-0
  HP_MASK                 EQU     $3F     ; byte 6: mask for HP bits 5-0
  LEVEL_MASK              EQU     $1F     ; byte 11: mask for level bits 4-0
  LEVEL_FLAG_MASK         EQU     $E0     ; byte 11: mask for flag bits 7-5
  PENDING_XP_UNIT         EQU     $10     ; byte 8: one pending XP unit (bit 4)
  FLAGS_CLASS_MASK        EQU     $C0     ; byte 15: mask for class bits 7-6
  FLAGS_STATUS_MASK       EQU     $3F     ; byte 15: mask for status bits 5-0
  HP_MASK             EQU     $3F     ; byte 6: mask for HP bits 5-0
  NPC_APPEARANCE_MIN      EQU     $15     ; appearance >= $15 = NPC, < $15 = player character
```

Defines:
LEVEL_FLAG_MASK, used in chunk 412.
LEVEL_MASK, used in chunks 338, 412, 447, 450, 478, and 556.
PENDING_XP_UNIT, used in chunk 412.
Uses MFIELD_LEVEL 561 and MFIELD_XP 561.

411b ⟨*zero page defines* 25a⟩+≡ (909) ◁401 418▷

```
  XP_ACCUM                EQU     $BA     ; accumulated level during XP award (scratch)
```

Defines:
XP_ACCUM, used in chunk 412.

412 ⟨*xp award* 412⟩≡ (909)

```
    ORG     $7249
XP_AWARD_0:
    LDX     #$00
    BEQ     AWARD_XP

XP_AWARD_1:
    LDX     #$01
    BNE     AWARD_XP

XP_AWARD_2:
    LDX     #$02
    BNE     AWARD_XP

XP_AWARD_3:
    LDX     #$03
    BNE     AWARD_XP

XP_AWARD_4:
    LDX     #$04
    BNE     AWARD_XP

AWARD_XP:
    SUBROUTINE AWARD_XP
    ; Award experience points after a shop buy action.
    ;
    ; Inputs:
    ;   X              | action type index (0-4), set by entry points
    ;   ENTITY_PTR     | pointer to entity receiving the XP
    ;   XP_AWARD_TABLE | base XP values indexed by action type
    ;   LOCATION_STYLE_BITS | location tier modifier (0-3)
    ;
    ; Behavior:
    ;   Extracts current level and pending XP from entity record.
    ;   Adds the award value (table[X] + tier) to byte 8, then
    ;   clamps so pending XP does not exceed the accumulated level.
    ;   Writes the updated level back to byte 11 and jumps to the
    ;   browse handler.
    ;
    ; Modifies:
    ;   ENTITY_PTR bytes 8 and 11, XP_ACCUM
    ; Clobbers: A, X, Y

    ; --- extract current level and pending XP ---
    LDY     #MFIELD_LEVEL    ; byte 11: flags|level
    LDA     (ENTITY_PTR),Y
    PHA
    AND     #LEVEL_MASK            ; level (bits 4-0)
    STA     XP_ACCUM
    PLA
```

```
    AND     #LEVEL_FLAG_MASK        ; preserve flag bits (7-5)
    STA     (ENTITY_PTR),Y          ; clear level in record
    LDY     #MFIELD_XP        ; byte 8: status|pending_xp|count
    LDA     (ENTITY_PTR),Y
    JSR     EXTRACT_PENDING_XP      ; A = pending XP (0-7)
    CLC
    ADC     XP_ACCUM                ; accumulated = level + pending
    STA     XP_ACCUM

    ; --- add XP award and clamp ---
    LDA     XP_AWARD_TABLE,X        ; base XP for action type
    CLC
    ADC     LOCATION_STYLE_BITS     ; + location tier modifier
.clamp:
    STA     (ENTITY_PTR),Y          ; write to byte 8
    JSR     EXTRACT_PENDING_XP      ; extract new pending XP
    CMP     XP_ACCUM                ; exceeds accumulated level?
    BCC     .ok                     ; no -> done
    SEC                             ; \ yes -> reduce by one XP unit
    LDA     (ENTITY_PTR),Y          ;  |
    SBC     #PENDING_XP_UNIT        ;  |
    JMP     .clamp                  ; / retry

    ; --- write updated level back ---
.ok:
    TAX                             ; X = new pending XP
    LDA     XP_ACCUM
    STX     XP_ACCUM
    SEC
    SBC     XP_ACCUM                ; remaining level = accumulated - pending
    LDY     #MFIELD_LEVEL     ; merge with saved flag bits
    ORA     (ENTITY_PTR),Y
    STA     (ENTITY_PTR),Y
    JMP     BROWSE_HANDLER

EXTRACT_PENDING_XP:
    ; Extract the pending XP field from a packed entity byte.
    ;
    ; Input:  A = entity byte 8 (status|pending_xp|count)
    ; Output: A = pending XP value (0-7), from bits 6-4
    ; Clobbers: flags
    ROR                             ; \ extract bits 6-4 of A
    ROR                             ;  | shift right 4 positions
    ROR                             ;  |
    ROR                             ; /
    AND     #$07                    ; mask to 3 bits (0-7)
    RTS
```

Defines:
AWARD_XP, never used.
EXTRACT_PENDING_XP, never used.

XP_AWARD_0, used in chunk 402.
XP_AWARD_1, used in chunk 402.
XP_AWARD_2, used in chunk 402.
XP_AWARD_3, used in chunk 402.
XP_AWARD_4, used in chunk 402.

Uses BROWSE_HANDLER 404, ENTITY_PTR 561, LEVEL_FLAG_MASK 411a, LEVEL_MASK 411a, LOCATION_STYLE_BITS 403c, MFIELD_LEVEL 561, MFIELD_XP 561, PENDING_XP_UNIT 411a, XP_ACCUM 411b, and XP_AWARD_TABLE 62.

Chapter 10

Events and Scripting

10.1 Between-round random events

After every character has taken a turn, RANDOM_EVENTS runs a cascading chain of random event checks. Each step rolls the PRNG and compares the result against EVENT_THRESHOLD (the byte at EVENT_THRESHOLD, approximately a 7% chance per roll).

The structure is deliberately cascading: each successive stage is only reached if the previous roll *failed*, meaning that on a lucky sequence of rolls, multiple events can fire in a single round. If all five individual checks pass and the sixth roll also succeeds, a *burst* fires: four mob spawns in rapid succession plus a special faction event.

Most rounds see zero or one random events, but occasionally a flurry of activity hits at once—new mobs spawn, faction skirmishes trigger, and the game world feels unpredictably alive.

EVENT_THRESHOLD is the probability threshold for random events. RANDOM_EVENTS compares each PRNG roll against this value; a roll below the threshold triggers the corresponding event handler.

415 ⟨*random events* 415⟩≡ (909)

```
      ORG     $65A9
  RANDOM_EVENTS:
      SUBROUTINE RANDOM_EVENTS
      ; ---- RANDOM_EVENTS ----
      ; Cascading random event dispatcher.
      ; Called between rounds to generate world activity.
      ;
```

```
    ; Input:  EVENT_THRESHOLD ($4239) = probability cutoff (~7%)
    ; Output: fires 0-10 events depending on PRNG rolls
    ; Clobbers: A, X, Y, CHAR_PTR, MOB_PTR, CHAR_INDEX, FACTION_MASK
    ;
    ; Each step rolls PRNG; if roll < threshold, the event fires
    ; AND the next step is tested.  Step 6 inverts the test: if
    ; roll >= threshold, a burst of 4 spawns + faction scene fires.
    JSR     STEP_PRNG               ; step 1: mob event
    LDA     PRNG_OUTPUT
    BAPL    EVENT_THRESHOLD,.ev2
    JSR     EVENT_PICK_MOB
.ev2:
    JSR     STEP_PRNG               ; step 2: char event
    LDA     PRNG_OUTPUT
    BAPL    EVENT_THRESHOLD,.ev3
    JSR     EVENT_PICK_CHAR
.ev3:
    JSR     STEP_PRNG               ; step 3: faction scene
    LDA     PRNG_OUTPUT
    BAPL    EVENT_THRESHOLD,.ev4
    JSR     EVENT_FACTION_SCENE
.ev4:
    JSR     STEP_PRNG               ; step 4: spawn mob
    LDA     PRNG_OUTPUT
    BAPL    EVENT_THRESHOLD,.ev5
    JSR     EVENT_SPAWN_MOB
.ev5:
    JSR     STEP_PRNG               ; step 5: spawn mob
    LDA     PRNG_OUTPUT
    BAPL    EVENT_THRESHOLD,.ev6
    JSR     EVENT_SPAWN_MOB
.ev6:
    JSR     STEP_PRNG               ; step 6: burst gate (inverted)
    LDA     PRNG_OUTPUT
    BAMI    EVENT_THRESHOLD,.burst
    RTS                             ; no burst this round
.burst:
    JSR     EVENT_SPAWN_MOB         ; burst: 4 spawns + faction scene
    JSR     EVENT_SPAWN_MOB
    JSR     EVENT_SPAWN_MOB
    JSR     EVENT_SPAWN_MOB
    JSR     PICK_RANDOM_CHAR
    JMP     EVENT_FACTION_SCENE_CONT
```

Defines:
RANDOM_EVENTS, used in chunks 219 and 419.

Uses CHAR_INDEX 418, CHAR_PTR 591a, EVENT_FACTION_SCENE 419, EVENT_FACTION_SCENE_CONT 419, EVENT_PICK_CHAR 417b, EVENT_PICK_MOB 417a, EVENT_SPAWN_MOB 420, EVENT_THRESHOLD 207, FACTION_MASK 418, MOB_PTR 608, PICK_RANDOM_CHAR 423, PRNG_OUTPUT 593a, and STEP_PRNG 595.

The individual event handlers are short routines that select a random target and dispatch to the appropriate processing function. EVENT_PICK_MOB selects a random mob group (weighted by group size) via PICK_RANDOM_MOB and passes it to PROCESS_CHAR_EVENTS, which walks the selected character's event list looking for triggerable entries.

417a ⟨*event pick mob* 417a⟩≡ (909)

```
    ORG     $660D
EVENT_PICK_MOB:
    SUBROUTINE EVENT_PICK_CHAR
    ; ---- EVENT_PICK_MOB ----
    ; Select a random mob group (weighted by size) and process its events.
    ;
    ; Input:  TOTAL_MOB_COUNT = total mob population
    ; Output: CHAR_PTR -> selected mob's char record
    ; Clobbers: A, X, Y, CHAR_INDEX, MOB_PTR, DATA_PTR
    JSR     PICK_RANDOM_MOB         ; weighted mob selection -> CHAR_PTR
    JMP     PROCESS_CHAR_EVENTS
```

Defines:
EVENT_PICK_MOB, used in chunk 415.
Uses CHAR_INDEX 418, CHAR_PTR 591a, DATA_PTR 618a, EVENT_PICK_CHAR 417b, MOB_PTR 608, PICK_RANDOM_MOB 422, PROCESS_CHAR_EVENTS 424, and TOTAL_MOB_COUNT 591b.

EVENT_PICK_CHAR works the same way, but selects a random character from the active player range (players 2 through CURRENT_PLAYER−1) instead of a mob group. The distinction is that EVENT_PICK_MOB weights its selection by mob group count, while EVENT_PICK_CHAR selects uniformly among characters.

417b ⟨*event pick char* 417b⟩≡ (909)

```
    ORG     $6613
EVENT_PICK_CHAR:
    SUBROUTINE EVENT_FACTION_SCENE
    ; ---- EVENT_PICK_CHAR ----
    ; Select a random character (uniform) and process its events.
    ;
    ; Input:  CURRENT_PLAYER = upper bound of player range
    ; Output: CHAR_PTR -> selected char record
    ; Clobbers: A, X, Y, CHAR_INDEX, DATA_PTR
    JSR     PICK_RANDOM_CHAR        ; uniform char selection -> CHAR_PTR
    JMP     PROCESS_CHAR_EVENTS
```

Defines:
EVENT_PICK_CHAR, used in chunks 415 and 417a.
Uses CHAR_INDEX 418, CHAR_PTR 591a, CURRENT_PLAYER 562a, DATA_PTR 618a, EVENT_FACTION_SCENE 419, PICK_RANDOM_CHAR 423, and PROCESS_CHAR_EVENTS 424.

EVENT_FACTION_SCENE picks a random mob and then searches the mob list for one whose faction matches. Factions are encoded in bits 6–5 of character record byte 5. If a matching mob is found, SCENE_SETUP is called to trigger an interaction between them. If no match exists, the routine falls back to EVENT_SPAWN_MOB instead.

The alternate entry EVENT_FACTION_SCENE_CONT is used by the burst path in RANDOM_EVENTS, which skips the initial mob selection and reuses the character already chosen by PICK_RANDOM_CHAR.

418 ⟨*zero page defines* 25a⟩+≡ (909) ◁411b 453▷

```
CHAR_INDEX          EQU     $F6     ; selected character/player index
FACTION_MASK        EQU     $F7     ; faction match value for FIND_MOB_BY_FACTION
```

Defines:
CHAR_INDEX, used in chunks 415, 417, 419, 420, and 422–24.
FACTION_MASK, used in chunks 415, 419, 420, and 426.
Uses FIND_MOB_BY_FACTION 426.

419 ⟨*event faction scene* 419⟩≡ (909)

```
    ORG     $6619
EVENT_FACTION_SCENE:
    SUBROUTINE EVENT_FACTION_SCENE_CONT
    ; ---- EVENT_FACTION_SCENE ----
    ; Pick a random mob, find another mob with matching faction,
    ; and trigger a scene between them.
    ;
    ; Input:  TOTAL_MOB_COUNT, char records at [[MOB_BASE_LO]]+
    ; Output: SCENE_SETUP called if faction match found,
    ;         else falls through to EVENT_SPAWN_MOB
    ; Clobbers: A, X, Y, CHAR_PTR, MOB_PTR, CHAR_INDEX,
    ;           FACTION_MASK, SOURCE_CHAR, ACTIVE_CHAR
    ;
    ; Alternate entry EVENT_FACTION_SCENE_CONT skips mob pick,
    ; reuses CHAR_PTR set by caller (burst path).
    JSR     PICK_RANDOM_MOB
EVENT_FACTION_SCENE_CONT:               ; alternate entry from RANDOM_EVENTS burst
    LDA     CHAR_PTR                    ; check low byte of CHAR_PTR
    BANE    #$09,.has_char              ; still at base ($4009)?
    RTS                                 ; no character selected -> skip
.has_char:
    LDY     #CFIELD_STYLE               ; byte 5: faction in bits 6-5
    LDA     (CHAR_PTR),Y
    AND     #CFIELD_FACTION_MASK        ; isolate faction bits
    STA     FACTION_MASK                ; save for FIND_MOB_BY_FACTION
    STOW    $4000,CHAR_PTR              ; reset CHAR_PTR to record base
    JSR     FIND_MOB_BY_FACTION         ; search for matching faction
    BANE    #$00,.found
    JMP     EVENT_SPAWN_MOB             ; no match -> try spawning
.found:
    STOB    #$00,SOURCE_CHAR            ; clear source char
    MOVB    CHAR_INDEX,ACTIVE_CHAR      ; set active char = found index
    JSR     SCENE_SETUP
    JSR     CHECK_LOCATION_MATCH
    RTS
```

Defines:
EVENT_FACTION_SCENE, used in chunks 415 and 417b.
EVENT_FACTION_SCENE_CONT, used in chunk 415.

Uses ACTIVE_CHAR 492c, CFIELD_FACTION_MASK 561, CFIELD_STYLE 561, CHAR_INDEX 418, CHAR_PTR 591a, CHECK_LOCATION_MATCH 429, EVENT_SPAWN_MOB 420, FACTION_MASK 418, FIND_MOB_BY_FACTION 426, MOB_BASE_LO 53a, MOB_PTR 608, MOVB 8c, PICK_RANDOM_MOB 422, RANDOM_EVENTS 415, SCENE_SETUP 494, SOURCE_CHAR 492b, STOB 8c, STOW 8a, and TOTAL_MOB_COUNT 591b.

EVENT_SPAWN_MOB attempts to activate a dormant character. It picks a random character and checks byte 8: if nonzero, the character is already active, so it tries once more. If both picks are active, it gives up. For an inactive character, the routine extracts its faction mask, searches for a matching mob, and if found, calls REORDER_CHAR to bring the character into the mob's group, followed by CLAMP_CHAR_FIELD to normalize its stats. This is the primary mechanism by which new entities enter the game world during play.

420 ⟨*event spawn mob* 420⟩≡ (909)

```
    ORG     $664E
EVENT_SPAWN_MOB:
    SUBROUTINE EVENT_SPAWN_MOB
    ; ---- EVENT_SPAWN_MOB ----
    ; Activate a dormant character by placing it into a matching
    ; mob's group.  Picks a random char; if already active (XP
    ; field nonzero), retries once.  On success, reorders the
    ; char into the mob group and clamps its stats.
    ;
    ; Input:  CURRENT_PLAYER, char records at [[MOB_BASE_LO]]+
    ; Output: REORDER_CHAR + CLAMP_CHAR_FIELD called if spawned
    ; Clobbers: A, X, Y, CHAR_PTR, MOB_PTR, CHAR_INDEX,
    ;           FACTION_MASK, SOURCE_CHAR, ACTIVE_CHAR, TARGET_REC
    JSR     PICK_RANDOM_CHAR
    LDY     #CFIELD_XP                  ; byte 8: status/group count
    LDA     (CHAR_PTR),Y
    BEQ     .inactive                   ; zero = dormant slot
    JSR     PICK_RANDOM_CHAR            ; retry once
    LDY     #CFIELD_XP
    LDA     (CHAR_PTR),Y
    BEQ     .inactive
    RTS                                 ; both active -> give up
.inactive:
    LDY     #CFIELD_STYLE               ; byte 5: faction in bits 6-5
    LDA     (CHAR_PTR),Y
    AND     #CFIELD_FACTION_MASK        ; isolate faction bits
    STA     FACTION_MASK
    JSR     FIND_MOB_BY_FACTION
    BANE    #$00,.found
    RTS                                 ; no matching mob -> skip
.found:
    MOVB    CHAR_INDEX,SOURCE_CHAR
    STOB    #$00,ACTIVE_CHAR
    MOVW    MOB_PTR,TARGET_REC          ; pass mob ptr to REORDER_CHAR
    JSR     REORDER_CHAR
    JMP     CLAMP_CHAR_FIELD
```

Defines:
EVENT_SPAWN_MOB, used in chunks 415 and 419.

Uses ACTIVE_CHAR 492c, CFIELD_FACTION_MASK 561, CFIELD_STYLE 561, CFIELD_XP 561, CHAR_INDEX 418, CHAR_PTR 591a, CLAMP_CHAR_FIELD 609, CURRENT_PLAYER 562a, FACTION_MASK 418, FIND_MOB_BY_FACTION 426, MOB_BASE_LO 53a, MOB_PTR 608, MOVB 8c,

MOVW 9a, PICK_RANDOM_CHAR 423, REORDER_CHAR 478, SOURCE_CHAR 492b, STOB 8c, and TARGET_REC 618b.

PICK_RANDOM_MOB selects a random mob group using weighted sampling. It generates a random number in the range 0..TOTAL_MOB_COUNT-1, then iterates through the character records, subtracting each character's group count (byte 8) from the running total. When the total reaches zero or goes negative, that character's record is selected. This weighting means characters with larger groups are proportionally more likely to be chosen.

If TOTAL_MOB_COUNT is zero (no mobs in the world), the routine falls through directly to PICK_RANDOM_CHAR. The shared tail COPY_BE_TO_FA copies the result from DATA_PTR to CHAR_PTR.

422 ⟨*pick random mob* 422⟩≡ (909)

```
    ORG     $6689
PICK_RANDOM_MOB:
    SUBROUTINE PICK_RANDOM_MOB
    ; ---- PICK_RANDOM_MOB ----
    ; Weighted random mob selection.  Generates a random number in
    ; [1..TOTAL_MOB_COUNT], then walks char records subtracting
    ; each group count (CFIELD_XP byte) until the accumulator
    ; hits zero or goes negative.
    ;
    ; Input:  TOTAL_MOB_COUNT = total mob population
    ; Output: CHAR_PTR -> selected char record
    ;         CHAR_INDEX = selected player index
    ; Clobbers: A, X, Y, MOB_PTR, DATA_PTR
    ;
    ; Falls through to PICK_RANDOM_CHAR if no mobs exist.
    LDA     TOTAL_MOB_COUNT
    BEQ     PICK_RANDOM_CHAR            ; no mobs -> pick char instead
    JSR     RANDOM_IN_RANGE             ; A = random 0..count-1
    BAEQ    #$00,PICK_RANDOM_MOB        ; zero = retry (need 1+)
    STA     MOB_PTR                     ; remaining count
    STOB    #$01,CHAR_INDEX             ; start at player 1
.next_char:
    LDA     CHAR_INDEX
    JSR     GET_CHAR_RECORD             ; DATA_PTR -> char record
    LDY     #CFIELD_XP                  ; byte 8: group count in bits 3-0
    LDA     MOB_PTR                     ; remaining
    SEC
    SBC     (DATA_PTR),Y                ; subtract this char's group count
    STA     MOB_PTR
    BEQ     .found                      ; exact match
    BMI     .found                      ; overshot -> select this one
    INC     CHAR_INDEX
    BNE     .next_char                  ; always branches (index < 256)
.found:
COPY_BE_TO_FA:
    MOVW    DATA_PTR,CHAR_PTR           ; copy result to CHAR_PTR
    RTS
```

Defines:

COPY_BE_TO_FA, used in chunk 423.
PICK_RANDOM_MOB, used in chunks 417a and 419.
Uses CFIELD_XP 561, CHAR_INDEX 418, CHAR_PTR 591a, DATA_PTR 618a, GET_CHAR_RECORD 26, MOB_PTR 608, MOVW 9a, PICK_RANDOM_CHAR 423, RANDOM_IN_RANGE 597, STOB 8c, and TOTAL_MOB_COUNT 591b.

PICK_RANDOM_CHAR selects uniformly from the active player range: players 2 through CURRENT_PLAYER−1. It computes the range, calls RANDOM_IN_RANGE, adds the offset to player 2, and loads the selected character's record via GET_CHAR_RECORD.

423 ⟨*pick random char* 423⟩≡ (909)

```
    ORG     $66BA
PICK_RANDOM_CHAR:
    SUBROUTINE PICK_RANDOM_CHAR
    ; ---- PICK_RANDOM_CHAR ----
    ; Uniform random character selection from players 2..CURRENT_PLAYER-1.
    ; Computes range = CURRENT_PLAYER - 3, picks random offset,
    ; adds base index 2.
    ;
    ; Input:  CURRENT_PLAYER = upper bound + 1 of player range
    ; Output: CHAR_PTR -> selected char record
    ;         CHAR_INDEX = selected player index
    ; Clobbers: A, X, Y, DATA_PTR
    LDA     CURRENT_PLAYER
    SEC
    SBC     #$03                        ; range = CURRENT_PLAYER - 3
    JSR     RANDOM_IN_RANGE             ; A = random 0..range-1
    CLC
    ADC     #$02                        ; offset to player 2
    STA     CHAR_INDEX                  ; save selected index
    JSR     GET_CHAR_RECORD             ; DATA_PTR -> char record
    JMP     COPY_BE_TO_FA               ; copy DATA_PTR -> CHAR_PTR
```

Defines:
PICK_RANDOM_CHAR, used in chunks 415, 417b, 420, and 422.
Uses CHAR_INDEX 418, CHAR_PTR 591a, COPY_BE_TO_FA 422, CURRENT_PLAYER 562a, DATA_PTR 618a, GET_CHAR_RECORD 26, and RANDOM_IN_RANGE 597.

10.2 Character event processing

PROCESS_CHAR_EVENTS walks a character's event list—a sequence of 3-byte entries pointed to by bytes 6–7 of the character record. For each entry whose second byte falls in the range $40--$7F, the routine subtracts $40 (normalizing to a 0-based index), stores the result back, and triggers the event via CHECK_LOCATION_MATCH. Entries with byte 1 outside this range are skipped, and $FF in byte 0 marks the end of the list.

424 ⟨*process char events* 424⟩≡ (909)

```
    ORG     $66CE
PROCESS_CHAR_EVENTS:
    SUBROUTINE PROCESS_CHAR_EVENTS
    ; Walk the character's 3-byte event list and trigger pending events.
    ;
    ; Input:  CHAR_PTR       -- character record (bytes 6-7 = event list ptr)
    ;         CHAR_INDEX     -- current player index
    ; Output: triggers CHECK_LOCATION_MATCH for each pending event
    ; Behavior:
    ;   - Loads event list pointer from CFIELD_EVENTS
    ;   - For each 3-byte entry:
    ;     $FF in byte 0 = end of list -> RTS
    ;     byte 1 bit 7 set -> skip (already activated)
    ;     byte 1 < $40 -> skip (not pending)
    ;     byte 1 in $40-$7F -> subtract $40, write back,
    ;       copy CHAR_INDEX to ACTIVE_CHAR, JMP CHECK_LOCATION_MATCH
    ;   - Advances EVENT_PTR by 3 for each skipped entry
    ; Clobbers: A, Y
    LDWIY    CHAR_PTR,#CFIELD_EVENTS,EVENT_PTR ; char record bytes 6-7 = event list ptr
.scan:
    LDY     #$00
    LDA     (EVENT_PTR),Y ; event byte 0
    BANE    #$FF,.check   ; $FF = end of list
    RTS
.check:
    INY
    LDA     (EVENT_PTR),Y ; event byte 1
    BMI     .advance      ; bit 7 set -> skip
    BAMI    #$40,.advance ; < $40?
    SEC
    SBC     #$40          ; normalize to 0-based
    STA     (EVENT_PTR),Y ; write back
    MOVB    CHAR_INDEX,ACTIVE_CHAR ; save triggering player index
    JMP     CHECK_LOCATION_MATCH
.advance:
    CLC
    LDA     EVENT_PTR
    ADC     #$03          ; next 3-byte entry
    STA     EVENT_PTR
```

```
    BCC     .no_carry
    INC     EVENT_PTR+1
.no_carry:
    JMP     .scan
```

Defines:
PROCESS_CHAR_EVENTS, used in chunk 417.
Uses ACTIVE_CHAR 492c, CFIELD_EVENTS 561, CHAR_INDEX 418, CHAR_PTR 591a, CHECK_LOCATION_MATCH 429, EVENT_PTR 353a, LDWIY 17, and MOVB 8c.

FIND_MOB_BY_FACTION searches the mob linked list starting from the current character (via CHAR_PTR byte 2) for a mob whose byte $0F has bit 7 set and whose bits 6–4 match FACTION_MASK. It returns A=1 if a match is found (with MOB_PTR pointing to it), or A=0 if the end of the list is reached without a match.

425 ⟨*defines* 25b⟩+≡ (909) ◁411a 434a▷

```
FLAGS_FACTION_MASK  EQU     $70     ; byte 15 bits 6-4: faction identifier
```

Defines:
FLAGS_FACTION_MASK, used in chunk 426.

426 ⟨*find mob by faction* 426⟩≡ (909)

```
    ORG     $6706
FIND_MOB_BY_FACTION:
    SUBROUTINE FIND_MOB_BY_FACTION
    ; Search the mob linked list for a faction match.
    ;
    ; Input:  CHAR_PTR       -- character record (byte 2 = first mob link)
    ;         FACTION_MASK   -- expected faction value (bits 6-4 of MFIELD_FLAGS)
    ; Output: A = 1 if found (MOB_PTR -> matching mob record)
    ;         A = 0 if end of list reached (no match)
    ; Behavior:
    ;   - Follows linked list via CFIELD_NEXT / MFIELD_NEXT
    ;   - For each mob, checks MFIELD_FLAGS bit 7 (active)
    ;     and bits 6-4 (faction) against FACTION_MASK
    ;   - Stops at first match or zero link
    ; Clobbers: A, Y, MOB_DATA_PTR
    LDY     #CFIELD_NEXT
    LDA     (CHAR_PTR),Y       ; char record byte 2 = first link
.check_link:
    BNE     .resolve
    RTS                        ; zero link -> return A=0
.resolve:
    JSR     GET_MOB_DATA       ; resolve link -> MOB_DATA_PTR
    MOVW    MOB_DATA_PTR,MOB_PTR ; MOB_PTR = mob data pointer
    LDY     #MFIELD_FLAGS
    LDA     (MOB_PTR),Y        ; mob data byte $0F
    BPL     .next              ; bit 7 clear -> skip
    AND     #FLAGS_FACTION_MASK ; bits 6-4
    BANE    FACTION_MASK,.next ; match faction?
    LDA     #$01               ; found -> return 1
    RTS
.next:
    LDY     #MFIELD_NEXT
    LDA     (MOB_PTR),Y        ; next link
    JMP     .check_link
```

Defines:
FIND_MOB_BY_FACTION, used in chunks 418–20.

Uses CFIELD_NEXT 561, CHAR_PTR 591a, FACTION_MASK 418, FLAGS_FACTION_MASK 425, GET_MOB_DATA 602, MFIELD_NEXT 561, MOB_DATA_PTR 600b, MOB_PTR 608, and MOVW 9a.

10.3 Scene event dispatch

CHECK_SCENE_EVENTS is the scene event dispatcher called from the main loop for players 1 and 3. It orchestrates the encounter system by scanning the mob list for hostile groups, then dispatching to SCENE_LOOP with different mode numbers depending on the situation:

- Mode 4: player 1 non-hostile scene
- Mode 5: other player scene
- Mode 6: hostile encounter introduction
- Mode 7: post-encounter resolution
- Mode 8: combat resolution

Several flag bytes in the $4230 region track which phases have already been processed this round, preventing duplicate dispatches.

SCENE_FOUND_FLAG and SCENE_HOSTILE_FLAG record the results of SCAN_FOR_HOSTILE. SCENE_GROUP_IDX is the starting group index for the scan. SCENE_P1_DONE, SCENE_P3_DONE, SCENE_HOSTILE_DONE, and SCENE_EVENT_RESULT are one-shot flags in the $4230 region that CHECK_SCENE_EVENTS tests and sets to prevent the same phase from firing twice in one round.

427a ⟨*var scene found flag* 427a⟩≡ (909)

```
    ORG     $5A89
SCENE_FOUND_FLAG:
    DC.B    $D2                     ; initial value (runtime: scan result: 1 = matching group found)
```

Defines:
SCENE_FOUND_FLAG, used in chunks 428, 430, and 432.

427b ⟨*var scene hostile flag* 427b⟩≡ (909)

```
    ORG     $5A8A
SCENE_HOSTILE_FLAG:
    DC.B    $A9                     ; initial value (runtime: scan result: nonzero = hostile encount
```

Defines:
SCENE_HOSTILE_FLAG, used in chunks 428 and 430.

427c ⟨*var scene group idx* 427c⟩≡ (909)

```
    ORG     $5A8B
SCENE_GROUP_IDX:
    DC.B    $AC                     ; initial value (runtime: current group index for scanning)
```

Defines:
SCENE_GROUP_IDX, used in chunks 428 and 430–32.

428 ⟨*check scene events* 428⟩≡ (909)

```
    ORG     $6DFA
SCENE_LOADER_END:
CHECK_SCENE_EVENTS:
    SUBROUTINE CHECK_SCENE_EVENTS
    ; Scene event dispatcher for players 1 and 3.
    ;
    ; Behavior:
    ;   Scans mob groups for hostile encounters via SCAN_FOR_HOSTILE.
    ;   Dispatches to SCENE_LOOP with mode 4-8 depending on player
    ;   number and encounter state. Processes mob groups on hostile
    ;   encounters.
    ;
    ; Clobbers: A, X, Y

    ; Scene event dispatcher for players 1 & 3.  Checks encounter
    ; state and dispatches to SCENE_LOOP with mode 4-8 based on
    ; game state flags.  Called from MAIN_ENTRY's character loop.
    LDA     #$00
    BANE    SCENE_HOSTILE_DONE,.done     ; yes -> skip
    STOB    #$40,SCENE_GROUP_IDX ; start scanning from group $40
    JSR     SCAN_FOR_HOSTILE             ; scan for hostile encounter
    LDA     #$01
    BANE    SCENE_FOUND_FLAG,.done       ; no -> skip
    BANE    CURRENT_PLAYER,.other_player ; is this player 1?
    BANE    SCENE_HOSTILE_FLAG,.hostile  ; hostile encounter found?
    BAEQ    SCENE_P1_DONE,.done          ; yes -> skip
    STA     SCENE_P1_DONE                ; mark as done
    LDA     #$04
    JMP     SCENE_LOOP                   ; mode 4: player 1 non-hostile scene
.done:
    RTS
.other_player:
    BAEQ    SCENE_P3_DONE,.done          ; yes -> skip
    STA     SCENE_P3_DONE                ; mark as done
    JSR     SET_CURSOR_ROW21
    LDA     #$05
    JMP     SCENE_LOOP                   ; mode 5: other player scene
.hostile:
    STA     SCENE_HOSTILE_DONE           ; mark hostile as processed
    LDA     #$06
    JSR     SCENE_LOOP                   ; mode 6: hostile encounter intro
    JSR     PROCESS_SCENE_GROUPS         ; process mob groups
    LDA     SCENE_EVENT_RESULT
    BNE     .done                        ; nonzero = event handled, stop
    LDA     #$07
    JSR     SCENE_LOOP                   ; mode 7: post-encounter
    LDA     SCENE_COMBAT_FLAG
    BEQ     .done                        ; no combat -> done
    JSR     SCROLL_STATUS_LINE
```

```
    LDA     #$08
    JMP     SCENE_LOOP                  ; mode 8: combat resolution
```

Defines:
CHECK_SCENE_EVENTS, used in chunk 219.
SCENE_LOADER_END, used in chunk 671.
Uses CURRENT_PLAYER 562a, MAIN_ENTRY 219, PROCESS_SCENE_GROUPS 431, SCAN_FOR_HOSTILE 430, SCENE_COMBAT_FLAG 207, SCENE_EVENT_RESULT 207, SCENE_FOUND_FLAG 427a, SCENE_GROUP_IDX 427c, SCENE_HOSTILE_DONE 207, SCENE_HOSTILE_FLAG 427b, SCENE_LOOP 527, SCENE_P1_DONE 207, SCENE_P3_DONE 207, SCROLL_STATUS_LINE 639, SET_CURSOR_ROW21 627, and STOB 8c.

CHECK_LOCATION_MATCH loads the character record for ACTIVE_CHAR and compares its position with LOCATION_POS. If they match, it clears LOCATION_POS to force a map redraw on the next ENTER_LOCATION call. This ensures the display updates when an event changes the scene.

429 ⟨*check location match* 429⟩≡ (909)

```
    ORG     $672E
CHECK_LOCATION_MATCH:
    SUBROUTINE CHECK_LOCATION_MATCH
    ; Check if the active character's position matches LOCATION_POS.
    ;
    ; Behavior:
    ;   Loads the character record for ACTIVE_CHAR, compares position
    ;   with LOCATION_POS. If they match, clears LOCATION_POS to force
    ;   a map redraw on the next ENTER_LOCATION.
    ;
    ; Modifies: LOCATION_POS (cleared on match)
    ; Clobbers: A, X, Y

    ; Load char record for ACTIVE_CHAR ($5A29), check if DATA_PTR matches
    ; LOCATION_POS.  If so, clear LOCATION_POS to force a redraw.
    LDA     ACTIVE_CHAR
    JSR     GET_CHAR_RECORD
    LDA     DATA_PTR
    BAEQ    LOCATION_POS,.match
    RTS
.match:
    STOB    #$00,LOCATION_POS ; clear to trigger redraw
    RTS
```

Defines:
CHECK_LOCATION_MATCH, used in chunks 419 and 424.
Uses ACTIVE_CHAR 492c, DATA_PTR 618a, ENTER_LOCATION 535, GET_CHAR_RECORD 26, LOCATION_POS 562b, and STOB 8c.

SCAN_FOR_HOSTILE walks the mob group chain starting from SCENE_GROUP_IDX, checking each group for hostility via CHECK_HOSTILE. It sets SCENE_FOUND_FLAG to 1 when the target group is encountered in the chain, and SCENE_HOSTILE_FLAG to nonzero if any hostile group is found. The scan continues through the entire chain even after finding a match, so it catches all hostiles.

430 ⟨*scan for hostile* 430⟩≡ (909)

```
    ORG     $6E5A
SCAN_FOR_HOSTILE:
    SUBROUTINE SCAN_FOR_HOSTILE
    ; Scan the mob group chain for hostile encounters.
    ;
    ; Behavior:
    ;   Walks the linked list of mob groups via byte 2 (next pointer).
    ;   Sets SCENE_FOUND_FLAG=1 when the target group is encountered.
    ;   Calls CHECK_HOSTILE on each group; sets SCENE_HOSTILE_FLAG
    ;   nonzero if any hostile is found.
    ;
    ; Outputs: SCENE_FOUND_FLAG, SCENE_HOSTILE_FLAG
    ; Clobbers: A, X, Y

    ; Scan mob groups starting from SCENE_GROUP_IDX for hostile encounters.
    ; Sets SCENE_FOUND_FLAG=1 if the group matches, SCENE_HOSTILE_FLAG
    ; nonzero if a hostile is found.
    STOB    #$00,SCENE_FOUND_FLAG
    STA     SCENE_HOSTILE_FLAG
    LDA     SCENE_GROUP_IDX
    JSR     GET_MOB_DATA                    ; resolve group -> MOB_DATA_PTR/MOB_DATA_PTR+1
    MOVW    MOB_DATA_PTR,MOB_PTR ;  MOB_PTR/MOB_PTR+1 = mob data pointer
    MOVW    CHAR_PTR,MOB_DATA_PTR ;  save $FA/$FB into MOB_DATA_PTR/MOB_DATA_PTR+1
.walk:
    LDY     #$02
    LDA     (MOB_DATA_PTR),Y                ; mob data byte 2 = next link
    BNE     .has_link
    RTS                                     ; end of chain
.has_link:
    BANE    SCENE_GROUP_IDX,.check_hostile ; matches the target group?
    LDX     #$01
    STX     SCENE_FOUND_FLAG                ; mark as found
.check_hostile:
    JSR     GET_MOB_DATA                    ; resolve link -> MOB_DATA_PTR/MOB_DATA_PTR+1
    MOVW    MOB_DATA_PTR,TARGET_REC ;  TARGET_REC/TARGET_REC+1 = resolved mob data
    JSR     CHECK_HOSTILE                   ; check if hostile
    BEQ     .walk                           ; not hostile -> continue
    STA     SCENE_HOSTILE_FLAG              ; hostile found
    BNE     .walk                           ; always branches, continue scan
```

Defines:
SCAN_FOR_HOSTILE, used in chunks 428 and 432.

Uses CHAR_PTR 591a, CHECK_HOSTILE 317, GET_MOB_DATA 602, MOB_DATA_PTR 600b, MOB_PTR 608, MOVW 9a, SCENE_FOUND_FLAG 427a, SCENE_GROUP_IDX 427c, SCENE_HOSTILE_FLAG 427b,

STOB 8c, and TARGET_REC 618b.

PROCESS_SCENE_GROUPS handles the hostile encounter phase by processing mob groups $3F, $41, and $42 with SCAN_AND_REORDER. Group $40 is skipped because it is the player's own group. The routine bookends the processing with LOAD_CHAR_TO_FA calls to save and restore the player 3 record pointer.

431 ⟨*process scene groups* 431⟩≡ (909)

```
      ORG     $6E9E
  PROCESS_SCENE_GROUPS:
      SUBROUTINE PROCESS_SCENE_GROUPS
      ; Process mob groups $3F, $41, $42 for hostile reordering.
      ;
      ; Behavior:
      ;   Saves the player 3 record pointer, then calls SCAN_AND_REORDER
      ;   for groups $3F, $41, and $42 (skipping $40). Restores the
      ;   current player's record pointer on exit.
      ;
      ; Clobbers: A, X, Y

      ; Process mob groups $3F, $41, $42 with SCAN_AND_REORDER,
      ; bookended by LOAD_CHAR_TO_FA for save/restore.
      STOB    #$03,SOURCE_CHAR
      JSR     LOAD_CHAR_TO_FA  ; save player 3's record to $FA/$FB
      STOB    #$00,ACTIVE_CHAR
      STOB    #$3F,SCENE_GROUP_IDX
      JSR     SCAN_AND_REORDER ; group $3F
      INC     SCENE_GROUP_IDX  ; $40 (skip -- that's the player's group)
      INC     SCENE_GROUP_IDX  ; $41
      JSR     SCAN_AND_REORDER ; group $41
      INC     SCENE_GROUP_IDX  ; $42
      JSR     SCAN_AND_REORDER ; group $42
      LDA     CURRENT_PLAYER
      JSR     LOAD_CHAR_TO_FA  ; restore current player's record
      RTS
```

Defines:
PROCESS_SCENE_GROUPS, used in chunk 428.
Uses ACTIVE_CHAR 492c, CURRENT_PLAYER 562a, LOAD_CHAR_TO_FA 488, SCAN_AND_REORDER 432, SCENE_GROUP_IDX 427c, SOURCE_CHAR 492b, and STOB 8c.

SCAN_AND_REORDER combines a hostile scan with a group reorder. It calls SCAN_FOR_HOSTILE and, if the target group is found, copies the mob pointer into TARGET_REC and calls REORDER_CHAR to move the character into the matching group.

432 ⟨*scan and reorder* 432⟩≡ (909)

```
      ORG     $6ED5
  SCAN_AND_REORDER:
      SUBROUTINE SCAN_AND_REORDER
      ; Scan for hostile in the current group and reorder if found.
      ;
      ; Inputs:
      ;   SCENE_GROUP_IDX | group index to scan
      ;
      ; Behavior:
      ;   Calls SCAN_FOR_HOSTILE; if SCENE_FOUND_FLAG is set, copies
      ;   MOB_PTR into TARGET_REC and calls REORDER_CHAR.
      ;
      ; Clobbers: A, X, Y

      ; Call SCAN_FOR_HOSTILE for current SCENE_GROUP_IDX.
      ; If a match is found, copy MOB_PTR/MOB_PTR+1 -> TARGET_REC/TARGET_REC+1 and reorder.
      JSR     SCAN_FOR_HOSTILE
      LDA     SCENE_FOUND_FLAG
      BNE     .found
      RTS                   ; nothing found
  .found:
      MOVW    MOB_PTR,TARGET_REC ;  TARGET_REC/TARGET_REC+1 = mob data pointer
      JMP     REORDER_CHAR
```

Defines:
SCAN_AND_REORDER, used in chunk 431.
Uses MOB_PTR 608, MOVW 9a, REORDER_CHAR 478, SCAN_FOR_HOSTILE 430, SCENE_FOUND_FLAG 427a, SCENE_GROUP_IDX 427c, and TARGET_REC 618b.

10.3.1 Flush pending XP

FLUSH_PENDING_XP extracts the pending XP field from byte 8 of the entity record at ENTITY_PTR and adds it to the level in byte 11. Byte 8 is a packed field:

- bits 6–4: pending XP (0–7)
- bit 7: status flag (preserved)
- bits 3–0: group count (cleared by this routine)

The routine clears the lower 7 bits of byte 8 (keeping only bit 7), extracts bits 6–4 as a 3-bit value (0–7), and adds it to byte 11 (the level field). If there was XP to flush, it displays message $2B. If the lower 7 bits were zero (no pending XP and no group members), it displays message $37 and returns to MOVE_LOOP.

433 ⟨*flush pending xp* 433⟩≡ (909)

```
    ORG     $6EE9
FLUSH_PENDING_XP:
    SUBROUTINE FLUSH_PENDING_XP
    ; Flush pending XP from entity byte 8 into the level in byte 11.
    ;
    ; Inputs:
    ;   ENTITY_PTR | pointer to the entity record
    ;
    ; Behavior:
    ;   Extracts bits 6-4 of byte 8 as pending XP (0-7), clears the
    ;   pending bits, and adds the XP to byte 11 (level). Displays
    ;   the XP gained message, or exits to MOVE_LOOP if none pending.
    ;
    ; Modifies: entity bytes 8 and 11
    ; Clobbers: A, X, Y

    LDY     #$08
    LDA     (ENTITY_PTR),Y      ; byte 8: status|pending_xp|group_count
    TAX                         ; save original
    AND     #$80                ; keep only status flag (bit 7)
    STA     (ENTITY_PTR),Y      ; clear pending XP and group count
    TXA
    AND     #$7F                ; strip status flag
    BEQ     .no_xp              ; bits 6-0 all zero -> nothing to flush
    ROR                         ; \ extract bits 6-4 (pending XP)
    ROR                         ;  | shift right 4 positions
    ROR                         ;  |
    ROR                         ; /
    AND     #$07                ; mask to 3 bits (0-7)
    CLC
    LDY     #$0B
    ADC     (ENTITY_PTR),Y      ; add pending XP to level
```

```
    STA     (ENTITY_PTR),Y     ; store updated level
    JSR     SET_CURSOR_ROW21
    LDA     #MSG_XP_GAINED               ; XP gained message
    JMP     DISPLAY_MESSAGE

.no_xp:
    JSR     SET_CURSOR_ROW21
    LDA     #MSG_NO_XP               ; no XP message
    JSR     DISPLAY_MESSAGE
    JMP     MOVE_LOOP
```

Defines:
FLUSH_PENDING_XP, never used.
Uses DISPLAY_MESSAGE 637, ENTITY_PTR 561, MOVE_LOOP 232, MSG_NO_XP 634, MSG_XP_GAINED 634, and SET_CURSOR_ROW21 627.

10.3.2 Drop gold system

The drop gold system uses a binary search algorithm to let the player choose how much gold to drop. Four variables track the state:

- DROP_AMOUNT_LO/DROP_AMOUNT_HI: the current drop amount
- DROP_STEP_LO/DROP_STEP_HI: the current binary search step

INIT_DROP_GOLD initializes the binary search. It calls FIND_MSB on the entity's gold (bytes $0D/$0E) to find the highest set bit, sets the drop amount to that power of 2, copies it to the step, halves the step, and enters the drop gold menu. If the entity has no gold, it displays message $2E and returns to the other options menu.

434a ⟨*defines* 25b⟩+≡ (909) ◁425 451a▷

```
DROP_DISPLAY_BUF    EQU     $7CEB    ; 5-byte decimal display buffer
```

Defines:
DROP_DISPLAY_BUF, used in chunk 437.

434b ⟨*var drop amount* 434b⟩≡ (909)

```
    ORG     $5A7A
DROP_AMOUNT:
    DC.B    $A0,$A3                  ; initial value (runtime: drop gold amount (2 bytes))
```

Defines:
DROP_AMOUNT, used in chunks 436–38, 440, and 442.

435 ⟨*var drop step* 435⟩≡ (909)

```
    ORG     $5A7C
DROP_STEP:
    DC.B    $B1,$B3                 ; initial value (runtime: binary search step (2 bytes))
```

Defines:
DROP_STEP, used in chunks 436, 438, 440, and 442.

436 ⟨*init drop gold* 436⟩≡ (909)

```
      ORG     $6F17
  INIT_DROP_GOLD:
      SUBROUTINE INIT_DROP_GOLD
      ; Initialize the drop gold binary search interface.
      ;
      ; Inputs:
      ;   ENTITY_PTR | pointer to the entity record
      ;
      ; Behavior:
      ;   Calls FIND_MSB on the entity's gold to set DROP_AMOUNT to the
      ;   highest power-of-2. Copies to DROP_STEP and halves it, then
      ;   enters the drop gold menu. If no gold, displays a message.
      ;
      ; Modifies: DROP_AMOUNT, DROP_STEP
      ; Clobbers: A, X, Y

      LDA     #$00
      STA     DROP_AMOUNT
      LDY     #$0E
      LDA     (ENTITY_PTR),Y     ; gold high byte
      JSR     FIND_MSB           ; X = highest set bit mask
      STX     DROP_AMOUNT+1
      CPX     #$00
      BNE     .has_gold
      DEY                        ; Y = $0D
      LDA     (ENTITY_PTR),Y     ; gold low byte
      JSR     FIND_MSB
      STX     DROP_AMOUNT
      BNE     .has_gold
      JSR     SET_CURSOR_ROW21
      LDA     #MSG_NO_GOLD               ; "no gold" message
      JSR     DISPLAY_MESSAGE
      JMP     MENU_OTHER_OPTIONS

  .has_gold:
      LDA     DROP_AMOUNT     ; \ copy amount to step
      STA     DROP_STEP       ;  |
      LDA     DROP_AMOUNT+1     ;  |
      STA     DROP_STEP+1       ; /
      JSR     HALVE_STEP         ; step = amount / 2
      JMP     MENU_DROP_GOLD     ; enter drop gold menu
```

Defines:
INIT_DROP_GOLD, used in chunk 438.

Uses DISPLAY_MESSAGE 637, DROP_AMOUNT 434b, DROP_STEP 435, ENTITY_PTR 561, FIND_MSB 442, HALVE_STEP 442, MENU_DROP_GOLD 273, MENU_OTHER_OPTIONS 267b, MSG_NO_GOLD 634, and SET_CURSOR_ROW21 627.

INIT_DROP_DISPLAY is the initialization callback for the drop gold menu. It clears the 5-byte display buffer at DROP_DISPLAY_BUF and converts DROP_AMOUNT_LO/DROP_AMOUNT_HI to a decimal string via NUM_TO_DECIMAL.

437 ⟨*init drop display* 437⟩≡ (909)

```
    ORG     $6F52
INIT_DROP_DISPLAY:
    SUBROUTINE INIT_DROP_DISPLAY
    ; Format the drop gold display buffer with the current amount.
    ;
    ; Behavior:
    ;   Clears DROP_DISPLAY_BUF with spaces, copies DROP_AMOUNT into
    ;   NUM_VALUE, sets DECIMAL_OUT_PTR to DROP_DISPLAY_BUF, and calls
    ;   NUM_TO_DECIMAL.
    ;
    ; Modifies: DROP_DISPLAY_BUF, NUM_VALUE, DECIMAL_OUT_PTR
    ; Clobbers: A, X

    LDX     #$04
    LDA     #$20                ; space
.clear:
    STA     DROP_DISPLAY_BUF,X
    DEX
    BPL     .clear
    LDA     DROP_AMOUNT
    STA     $BC
    LDA     DROP_AMOUNT+1
    STA     $BD
    LDA     #<DROP_DISPLAY_BUF
    STA     $BA
    LDA     #>DROP_DISPLAY_BUF
    STA     $BB
    JSR     NUM_TO_DECIMAL
    RTS
```

Defines:
INIT_DROP_DISPLAY, used in chunk 273.
Uses DECIMAL_OUT_PTR 630a, DROP_AMOUNT 434b, DROP_DISPLAY_BUF 434a, NUM_TO_DECIMAL 632, and NUM_VALUE 630a.

ADJ_DROP_INC and ADJ_DROP_DEC are the handlers for the # and keys in the drop gold menu. They implement a binary search:

- # (X=3): subtract the step from the drop amount, then halve the step.
- @ (X=4): add the step to the drop amount, halve the step, then check if the amount exceeds the entity's actual gold. If so, undo by double-subtracting the step and switch to decrement mode.

If the step reaches zero, the search reinitializes via INIT_DROP_GOLD.

438 ⟨*adj drop amount* 438⟩≡ (909)

```
    ORG     $6F72
ADJ_DROP_INC:
    LDX     #$03                ; # pressed
    BNE     .entry

ADJ_DROP_DEC:
    LDX     #$04                ; @ pressed

.entry:
    LDA     #$00
    CMP     DROP_STEP
    BNE     .nonzero
    CMP     DROP_STEP+1
    BEQ     INIT_DROP_GOLD      ; step = 0 -> reinitialize

.nonzero:
    TXA
    CMP     #$03
    BEQ     .dec_amount         ; X=3 (#) -> subtract step

    ; X=4 (@): add step, halve, check bounds
    JSR     ADD_DROP
    JSR     HALVE_STEP
    LDY     #$0E
    LDA     (ENTITY_PTR),Y      ; actual gold hi
    CMP     DROP_AMOUNT+1
    BCC     .over_limit         ; drop > gold -> too much
    BNE     .done               ; drop < gold -> ok
    DEY
    LDA     (ENTITY_PTR),Y      ; actual gold lo
    CMP     DROP_AMOUNT
    BCS     .done               ; drop <= gold -> ok

.over_limit:
    JSR     SUB_DROP            ; undo: amount -= step
    JSR     SUB_DROP            ; undo again (net: amount -= step)
    LDX     #$04                ; switch to @ mode
```

```
    BNE     .entry              ; retry with reduced step

.dec_amount:
    JSR     SUB_DROP            ; amount -= step
    JSR     HALVE_STEP          ; step /= 2

.done:
    JMP     MENU_DROP_GOLD      ; re-enter drop gold menu
```

Defines:
ADJ_DROP_DEC, used in chunk 273.
ADJ_DROP_INC, used in chunk 273.
Uses ADD_DROP 442, DROP_AMOUNT 434b, DROP_STEP 435, ENTITY_PTR 561, HALVE_STEP 442, INIT_DROP_GOLD 436, MENU_DROP_GOLD 273, and SUB_DROP 442.

DROP_GOLD_EXEC is the handler for the "DROP GOLD" option in the drop gold menu. It subtracts the drop amount from the entity's gold via MODIFY_CURRENT_STATS, then generates a random multiplier (1–25) and subtracts it from the amount. If gold remains, it displays message $2D and creates a gold drop event at the entity's position via UPDATE_EVENT_LIST. If the amount is zero or negative, it displays message $2C (gold lost/stolen) instead.

440 ⟨*drop gold exec* 440⟩≡ (909)

```
    ORG     $6FB5
DROP_GOLD_EXEC:
    SUBROUTINE DROP_GOLD_EXEC
    ; Execute the gold drop transaction.
    ;
    ; Inputs:
    ;   ENTITY_PTR | pointer to the entity record
    ;   DROP_AMOUNT | amount of gold to drop
    ;
    ; Behavior:
    ;   Subtracts DROP_AMOUNT from entity's gold, then subtracts a
    ;   random loss (1-25). If gold remains, creates a gold event on
    ;   the map. Exits via JMP MOVE_LOOP.
    ;
    ; Modifies: entity gold, event list
    ; Clobbers: A, X, Y

    LDA     DROP_AMOUNT
    STA     $BC
    LDA     DROP_AMOUNT+1
    STA     $BD
    LDY     #$00
    STY     DROP_STEP+1
    JSR     MODIFY_CURRENT_STATS ; subtract gold from entity
    LDA     #$19
    JSR     RANDOM_IN_RANGE    ; random 0-24
    STA     DROP_STEP
    INC     DROP_STEP       ; step = 1-25
    JSR     SET_CURSOR_ROW21
    JSR     SUB_DROP           ; amount -= random step (gold lost)
    LDA     DROP_AMOUNT+1
    BMI     .lost              ; negative -> all lost
    BNE     .has_gold
    LDA     DROP_AMOUNT
    BEQ     .lost              ; zero -> all lost

.has_gold:
    LDA     #MSG_GOLD_DROPPED               ; gold dropped message
    JSR     DISPLAY_MESSAGE
    LDA     DROP_AMOUNT
    STA     $BC
    LDA     DROP_AMOUNT+1
```

```
    STA     $BD
    LDY     #$03
    LDA     (ENTITY_PTR),Y      ; entity position
    JSR     UPDATE_EVENT_LIST   ; create gold event on map
    JMP     MOVE_LOOP

.lost:
    LDA     #MSG_GOLD_LOST                ; gold lost message
    JSR     DISPLAY_MESSAGE
    JMP     MOVE_LOOP
```

Defines:
DROP_GOLD_EXEC, used in chunk 273.

Uses DISPLAY_MESSAGE 637, DROP_AMOUNT 434b, DROP_STEP 435, ENTITY_PTR 561, MODIFY_CURRENT_STATS 382, MOVE_LOOP 232, MSG_GOLD_DROPPED 634, MSG_GOLD_LOST 634, RANDOM_IN_RANGE 597, SET_CURSOR_ROW21 627, SUB_DROP 442, and UPDATE_EVENT_LIST 454.

10.3.3 Arithmetic helpers

FIND_MSB returns in X the mask of the most significant set bit in A. It starts with X=$80 (bit 7) and shifts A left through carry; if the carry is set, the current bit position is the MSB. Otherwise it shifts X right and continues.

ADD_DROP and SUB_DROP perform 16-bit addition and subtraction on the drop amount (DROP_AMOUNT_LO/DROP_AMOUNT_HI) using the step (DROP_STEP_LO/DROP_STEP_HI).

HALVE_STEP divides the step by 2 via a 16-bit right shift.

442 ⟨*arithmetic helpers* 442⟩≡ (909)

```
    ORG     $7005
FIND_MSB:
    SUBROUTINE FIND_MSB
    ; Find the most significant set bit in A.
    ;
    ; Inputs:
    ;   A | value to scan
    ;
    ; Behavior:
    ;   Starting from bit 7, rotates A left and checks each bit.
    ;   Returns the mask of the first set bit found.
    ;
    ; Outputs:
    ;   X | single-bit mask of the MSB (e.g., $40 for bit 6)
    ; Clobbers: A, X

    SEC
    LDX     #$80                ; start with bit 7 mask
.test:
    ROL                         ; shift A left through carry
    BCC     .next               ; bit was 0 -> try next
    RTS                         ; bit was 1 -> X = mask
.next:
    PHA
    TXA
    LSR                         ; shift mask right
    TAX
    PLA
    CLC
    JMP     .test

ADD_DROP:
    CLC
    LDA     DROP_AMOUNT
    ADC     DROP_STEP
    STA     DROP_AMOUNT
    LDA     DROP_AMOUNT+1
    ADC     DROP_STEP+1
```

```
    STA     DROP_AMOUNT+1
    RTS

SUB_DROP:
    SEC
    LDA     DROP_AMOUNT
    SBC     DROP_STEP
    STA     DROP_AMOUNT
    LDA     DROP_AMOUNT+1
    SBC     DROP_STEP+1
    STA     DROP_AMOUNT+1
    RTS

HALVE_STEP:
    CLC
    LDA     DROP_STEP+1
    ROR
    STA     DROP_STEP+1
    LDA     DROP_STEP
    ROR
    STA     DROP_STEP
    RTS
```

Defines:
ADD_DROP, used in chunk 438.
FIND_MSB, used in chunk 436.
HALVE_STEP, used in chunks 436 and 438.
SUB_DROP, used in chunks 438 and 440.
Uses DROP_AMOUNT 434b and DROP_STEP 435.

10.4 Turn events

10.4.1 Scene event trigger

TRIGGER_SCENE_EVENT handles events in the $D2–$D4 range of the encounter dispatch table. It clears the character's turn state, commits the pending move, then triggers a scripted scene transition based on the event type byte.

Event types.

- **$D2** — Trigger the scene specified by event byte 2. The event entry is left intact for future encounters.
- **$D3** — Trigger the scene, then mark the event slot as empty ($FE) so it cannot fire again.
- **$D4** — Trigger the scene, mark the slot empty, then trigger a bonus scene $0F and apply 1–3 HP of damage to the current character.

444 ⟨*trigger scene event* 444⟩≡ (909)

```
    ORG     $1D04
TRIGGER_SCENE_EVENT:
    SUBROUTINE TRIGGER_SCENE_EVENT
    ; Trigger a scripted scene from event data.
    ;
    ; Input:  EVENT_PTR  -- 3-byte event entry
    ;         ENTITY_PTR -- current character
    ; Output: none (side-effects only)
    ; Behavior:
    ;   - Clears char turn state, commits move
    ;   - Event byte 2 = scene index
    ;   - Event byte 1 = type:
    ;     $D2: trigger scene only
    ;     $D3: trigger scene, mark empty
    ;     $D4: scene + bonus $0F + 1-3 dmg
    ; Clobbers: A, X, Y

    LDA     #$00
    JSR     SET_CHAR_STATE          ; clear char turn state
    MOVW    EVENT_PTR,$5A5B         ; save EVENT_PTR across call
    JSR     COMMIT_MOVE             ; commit move
    MOVW    $5A5B,EVENT_PTR         ; restore EVENT_PTR
    LDY     #$02
    LDA     (EVENT_PTR),Y           ; event byte 2 = scene index
    JSR     SCENE_LOOP              ; trigger scene
```

```
    LDY     #$01
    LDA     (EVENT_PTR),Y           ; event byte 1 = type
    BAPL    #$D3,.d3_or_above
    RTS                             ; $D2: trigger scene only
.d3_or_above:
    BNE     .d4
    LDA     #$FE                    ; $D3: mark event empty
    DEY
    STA     (EVENT_PTR),Y
    RTS
.d4:
    LDA     #$FE                    ; $D4: mark event empty
    DEY
    STA     (EVENT_PTR),Y
    LDA     #$0F
    JSR     SCENE_LOOP              ; trigger bonus scene $0F
    LDA     #$03
    JSR     RANDOM_IN_RANGE         ; random(3)
    CLC
    ADC     #$01                    ; 1-3
    JMP     APPLY_DAMAGE_TO_CURRENT ; apply damage to char
```

Defines:
TRIGGER_SCENE_EVENT, used in chunk 232.

Uses APPLY_DAMAGE_TO_CURRENT 352, COMMIT_MOVE 239, ENTITY_PTR 561, EVENT_PTR 353a, MOVW 9a, RANDOM_IN_RANGE 597, SCENE_LOOP 527, and SET_CHAR_STATE 242.

10.4.2 Treasure pickup

PICKUP_TREASURE handles the interaction when a character steps onto a treasure event. It clears the character's turn state, copies the event pointer to MOB_PTR, commits the pending move, and then dispatches on byte 1 of the event entry.

Normal treasure (byte 1 < $C9). The routine subtracts $C0 from the type byte to obtain a treasure table index (0–8), reads the quantity from byte 2, and calls MODIFY_CURRENT_STATS to apply the item. If the character's inventory is not full (appearance < $15), it formats a pickup message using NUM_TO_DECIMAL and CLEAR_AND_DISPLAY, then marks the event slot as empty ($FE).

Trapped chest (byte 1 ≥ $C9, ≠ $D1). A random roll (mod 4) selects one of four trap types:

Trap	Effect	Scene	Detail
0	HP loss	$36	Lose 1–5 HP
1	HP loss	$37	Lose 2–9 HP
2	Counter decrement	$38	Decrement field 11 low 5 bits
3	Gold zeroed	$39	Set gold low and high to 0

After applying the trap, the routine checks whether the character is in the current room. If not, it subtracts 9 from the type byte, converting the chest into a different treasure type for re-encounter. If the character is in the current room, the type is converted to a normal treasure index (type − $C9) and processing continues at the normal treasure path.

Special chest (byte 1 = $D1). This type triggers scene $0B, increments the 5-bit counters in both field 5 and field 11 (via INC_STRENGTH and INC_LEVEL), and then rolls a 1-in-3 chance for a bonus scene $0C (a lucky find). On the bonus path, the event slot is marked empty.

446 ⟨*var trap type* 446⟩≡ (909)

```
    ORG     $5A58
TRAP_TYPE:
    DC.B    $D3                     ; initial value (runtime: treasure trap: random trap type (0-3))
```

Defines:
TRAP_TYPE, used in chunk 447.

447 ⟨*pickup treasure* 447⟩≡ (909)

```
    ORG     $646B
PICKUP_TREASURE:
    SUBROUTINE PICKUP_TREASURE
    ; Handle treasure chest pickup from event data.
    ;
    ; Input:  EVENT_PTR  -- 3-byte event entry
    ;         ENTITY_PTR -- current character
    ; Output: event slot marked empty ($FE)
    ; Behavior:
    ;   - Clears char turn state, commits move
    ;   - Byte 1 < $C9: normal treasure
    ;     (subtract $C0 for index, byte 2 =
    ;     quantity, add to inventory if room)
    ;   - Byte 1 >= $C9, != $D1: trapped chest
    ;     (4 trap types: HP loss, counter dec,
    ;     gold zeroed; scene $36-$39)
    ;   - Byte 1 = $D1: special chest (scene
    ;     $0B, inc fields 5+11, 1/3 bonus)
    ; Clobbers: A, X, Y

    LDA     #$00
    JSR     SET_CHAR_STATE          ; clear char turn state
    MOVW    EVENT_PTR,MOB_PTR       ; copy EVENT_PTR -> MOB_PTR
    JSR     COMMIT_MOVE             ; commit move
    LDY     #$01
    LDA     (MOB_PTR),Y            ; event byte 1 = treasure type
    BAPL    #$C9,.trapped           ; >= $C9 -> trapped or special
    SEC
    SBC     #$C0                    ; $C0-$C8: treasure index 0-8
    STA     NUM_VALUE+1
.rejoin:
    LDY     #$02
    LDA     (MOB_PTR),Y            ; event byte 2 = quantity
    STA     NUM_VALUE
    LDY     #$01
    JSR     MODIFY_CURRENT_STATS   ; apply treasure to stats
    LDY     #MFIELD_APPEARANCE
    LDA     (ENTITY_PTR),Y         ; char field 4 = appearance
    BAGE    #$15,.mark_used        ; inventory full -> skip
    JSR     CLEAR_TREASURE_DIGITS  ; clear display digit fields
    STOW    S_REVEAL_DIGITS,DECIMAL_OUT_PTR ; output -> quantity digits
    JSR     NUM_TO_DECIMAL         ; format quantity
    LDWIY    ENTITY_PTR,#MFIELD_GOLD,NUM_VALUE ; gold low
    STOW    S_NOW_HAVE_DIGITS,DECIMAL_OUT_PTR ; output -> gold digits
    JSR     NUM_TO_DECIMAL         ; format gold amount
    JSR     SET_CURSOR_ROW21
    STOW    S_CHEST_OPENS,NUM_VALUE ; message pointer
    JSR     CLEAR_AND_DISPLAY      ; display pickup message
.mark_used:
```

```
    LDY     #$00
    LDA     #$FE                    ; mark event slot empty
    STA     (MOB_PTR),Y
    RTS

    ; --- trapped chest (type >= $C9) ---
.trapped:
    BANE    #$D1,.not_d1
    JMP     .handle_d1              ; $D1: special chest
.not_d1:
    LDA     #$06
    JSR     RANDOM_IN_RANGE         ; random(6)
    AND     #$03                    ; trap type 0-3
    STA     TRAP_TYPE
    LDY     #MFIELD_APPEARANCE
    LDA     (ENTITY_PTR),Y         ; char field 4 = appearance
    BMI     .apply_trap
    BAPL    #$15,.apply_trap
    JSR     SET_CURSOR_ROW21
    LDA     #$36
    CLC
    ADC     TRAP_TYPE               ; scene $36 + trap type
    JSR     SCENE_LOOP              ; show trap scene
.apply_trap:
    LDA     TRAP_TYPE
    BAMI    #$01,.trap0             ; trap 0: lose 1-5 HP
    BEQ     .trap1                  ; trap 1: lose 2-9 HP
    BAEQ    #$02,.trap2             ; trap 2: decrement level
    BNE     .trap3                  ; trap 3: zero gold
.trap0:
    LDA     #$05
    JSR     RANDOM_IN_RANGE
    CLC
    ADC     #$01                    ; 1-5 damage
    JSR     APPLY_DAMAGE_TO_CURRENT
    JMP     .after_trap
.trap1:
    LDA     #$08
    JSR     RANDOM_IN_RANGE
    CLC
    ADC     #$02                    ; 2-9 damage
    JSR     APPLY_DAMAGE_TO_CURRENT
    JMP     .after_trap
.trap2:
    LDY     #MFIELD_LEVEL
    LDA     (ENTITY_PTR),Y         ; char field 11 = level
    AND     #LEVEL_MASK             ; isolate level bits 4-0
    BEQ     .after_trap             ; already zero
    LDA     (ENTITY_PTR),Y
    SEC
```

```
    SBC     #$01                    ; decrement low 5 bits
    STA     (ENTITY_PTR),Y
    JMP     .after_trap
.trap3:
    LDY     #MFIELD_GOLD
    LDA     #$00
    STA     (ENTITY_PTR),Y          ; zero gold low
    INY
    STA     (ENTITY_PTR),Y          ; zero gold high
.after_trap:
    LDY     #MFIELD_NEXT
    LDA     (ENTITY_PTR),Y          ; char field 2 = room
    BAEQ    CURRENT_ROOM,.same_room ; current room?
    DEY
    LDA     (MOB_PTR),Y             ; event byte 1
    SEC
    SBC     #$09                    ; step back 9 in type
    STA     (MOB_PTR),Y
    RTS
.same_room:
    DEY
    LDA     (MOB_PTR),Y             ; event byte 1
    SEC
    SBC     #$C9                    ; convert to treasure index
    STA     NUM_VALUE+1
    JMP     .rejoin                 ; process as normal treasure

    ; --- special chest $D1 ---
.handle_d1:
    LDA     #$0B
    JSR     SCENE_LOOP              ; show scene $0B
    JSR     INC_STRENGTH      ; inc field 5 counter
    JSR     INC_LEVEL       ; inc field 11 counter
    LDA     #$03
    JSR     RANDOM_IN_RANGE         ; random(3)
    BAEQ    #$00,.lucky
    RTS                             ; 2/3 chance: nothing
.lucky:
    LDA     #$0C
    JSR     SCENE_LOOP              ; show bonus scene $0C
    JMP     .mark_used              ; mark event used
```

Defines:
PICKUP_TREASURE, used in chunk 232.

Uses APPLY_DAMAGE_TO_CURRENT 352, CLEAR_AND_DISPLAY 452, CLEAR_TREASURE_DIGITS 451b, COMMIT_MOVE 239, CURRENT_ROOM 476, DECIMAL_OUT_PTR 630a, ENTITY_PTR 561, EVENT_PTR 353a, INC_LEVEL 450, INC_STRENGTH 450, LDWIY 17, LEVEL_MASK 411a, MFIELD_APPEARANCE 561, MFIELD_GOLD 561, MFIELD_LEVEL 561, MFIELD_NEXT 561, MOB_PTR 608, MODIFY_CURRENT_STATS 382, MOVW 9a, NUM_TO_DECIMAL 632, NUM_VALUE 630a, RANDOM_IN_RANGE 597, SCENE_LOOP 527, SET_CHAR_STATE 242, SET_CURSOR_ROW21 627, STOW 8a, S_CHEST_OPENS 310, S_NOW_HAVE_DIGITS 451a, S_REVEAL_DIGITS 451a, and TRAP_TYPE 446.

10.4.3 Increment strength / level

INC_STRENGTH and INC_LEVEL increment a 5-bit counter packed into the low bits of a character record field, capped at 31.

- INC_STRENGTH increments MFIELD_STRENGTH (byte 5, bits 4–0).
- INC_LEVEL increments MFIELD_LEVEL (byte 11, bits 4–0).

The upper bits of the byte are preserved because the addition does not overflow past the 5-bit boundary (the cap at 31 prevents it).

450 ⟨*inc strength* 450⟩≡ (909)

```
    ORG     $657B
INC_STRENGTH:
    SUBROUTINE INC_STRENGTH
    ; Increment a 5-bit counter in a character record field.
    ;
    ; Entry points:
    ;   INC_STRENGTH | increments MFIELD_STRENGTH (byte 5)
    ;   INC_LEVEL    | increments MFIELD_LEVEL (byte 11)
    ;
    ; Inputs:
    ;   ENTITY_PTR | pointer to entity record
    ;
    ; Behavior:
    ;   Isolates bits 4-0, checks if already at max (31),
    ;   and if not, adds 1 to the full byte (preserving upper bits).
    ;
    ; Clobbers: A, X, Y

    LDY     #MFIELD_STRENGTH        ; byte 5: strength
    JMP     .body
INC_LEVEL:
    LDY     #MFIELD_LEVEL           ; byte 11: level
.body:
    LDA     (ENTITY_PTR),Y
    TAX                             ; save original byte
    AND     #LEVEL_MASK             ; isolate low 5 bits
    BANE    #LEVEL_MASK,.inc        ; already at max (31)?
    RTS
.inc:
    CLC
    TXA                             ; restore original byte
    ADC     #$01                    ; increment (preserves upper bits)
    STA     (ENTITY_PTR),Y
    RTS
```

Defines:

INC_LEVEL, used in chunk 447.
INC_STRENGTH, used in chunk 447.
Uses ENTITY_PTR 561, LEVEL_MASK 411a, MFIELD_LEVEL 561, and MFIELD_STRENGTH 561.

10.4.4 Clear treasure digit fields

CLEAR_TREASURE_DIGITS clears the two 5-byte digit fields in the treasure pickup message strings: the quantity area in S_REVEAL_GOLD and the gold amount area in S_NOW_HAVE_GOLD. Called before NUM_TO_DECIMAL formats new values into these buffers.

451a ⟨*defines* 25b⟩+≡ (909) ◁434a 464▷

```
S_REVEAL_DIGITS     EQU     S_REVEAL_GOLD+11   ; 5-byte quantity digit field
S_NOW_HAVE_DIGITS   EQU     S_NOW_HAVE_GOLD+14 ; 5-byte gold digit field
```

Defines:
S_NOW_HAVE_DIGITS, used in chunks 447 and 451b.
S_REVEAL_DIGITS, used in chunks 447 and 451b.
Uses S_NOW_HAVE_GOLD 310 and S_REVEAL_GOLD 310.

451b ⟨*clear treasure digits* 451b⟩≡ (909)

```
    ORG     $6593
CLEAR_TREASURE_DIGITS:
    SUBROUTINE CLEAR_TREASURE_DIGITS
    ; Clear the digit fields in the treasure pickup message strings.
    ;
    ; Behavior:
    ;   Fills S_REVEAL_DIGITS (5 bytes) and S_NOW_HAVE_DIGITS (5 bytes)
    ;   with spaces to erase stale digits before formatting new values.
    ;
    ; Modifies: S_REVEAL_DIGITS, S_NOW_HAVE_DIGITS
    ; Clobbers: A, X

    LDA     #$20                    ; space character
    LDX     #$04                    ; 5 bytes (index 4..0)
.loop:
    STA     S_REVEAL_DIGITS,X       ; quantity digit field
    STA     S_NOW_HAVE_DIGITS,X     ; gold digit field
    DEX
    BPL     .loop
    RTS
```

Defines:
CLEAR_TREASURE_DIGITS, used in chunk 447.
Uses S_NOW_HAVE_DIGITS 451a and S_REVEAL_DIGITS 451a.

10.4.5 Clear and display message

CLEAR_AND_DISPLAY zeroes the message line counter at $5A57 and tail-calls DISPLAY_MESSAGE_LOOP. Clearing the counter ensures the message loop starts from scratch rather than continuing a previous multi-part display sequence.

452 ⟨*clear and display* 452⟩≡ (909)

```
      ORG     $65A1
  CLEAR_AND_DISPLAY:
      SUBROUTINE CLEAR_AND_DISPLAY
      ; Clear message state and display message.
      ;
      ; Input:  NUM_VALUE -- message address
      ; Output: none (displays message)
      ; Behavior:
      ;   - Zeroes MSG_LINE_COUNT ($5A57)
      ;   - Tail-calls DISPLAY_MESSAGE_LOOP
      ; Clobbers: A, X, Y

      STOB    #$00,MSG_LINE_COUNT    ; reset line counter
      JMP     DISPLAY_MESSAGE_LOOP   ; display message
```

Defines:
CLEAR_AND_DISPLAY, used in chunk 447.
Uses DISPLAY_MESSAGE_LOOP 637, MSG_LINE_COUNT 636, NUM_VALUE 630a, and STOB 8c.

10.4.6 Event list update

UPDATE_EVENT_LIST inserts or updates a 3-byte event entry in the character's event list. It takes a map position in A and a value in NUM_VALUE/NUM_VALUE+1. If the value is zero, it returns immediately.

Scan behavior. The routine walks the 3-byte event entries starting from the pointer at bytes 6–7 of the character record (CFIELD_EVENTS). Each entry consists of a position byte, a type byte, and a value byte.

Slot handling. The scan processes each slot as follows:

- **$FF** (end-of-list) — No empty slot was found. Displays message $2A ("event list full").
- **$FE** (empty slot) — Inserts a new entry: byte 0 gets the position, byte 1 gets the value high bits (masked to 3 bits) OR'd with $C0 to mark it as activated, and byte 2 gets the value low byte.
- **Matching position with type $C0–$D0** — Adds the new value to the existing entry's value bytes rather than creating a duplicate.
- **Other** — Advances to the next 3-byte entry and continues scanning.

453 ⟨*zero page defines* 25a⟩+≡ (909) ◁418 458▷

```
EVENT_SAVED_POS          EQU     $BA     ; saved map position for event insert (aliased)
```

Defines:
EVENT_SAVED_POS, used in chunk 454.

454 ⟨*update event list* 454⟩≡ (909)

```
    ORG     $1260
UPDATE_EVENT_LIST:
    SUBROUTINE UPDATE_EVENT_LIST
    ; Insert or update a 3-byte event entry.
    ;
    ; Input:  A          -- map position
    ;         NUM_VALUE   -- value low byte
    ;         NUM_VALUE+1 -- value high byte
    ;         CHAR_PTR   -- char record (fields
    ;                       6-7 = event list ptr)
    ; Output: event list updated in place
    ; Behavior:
    ;   - If value = 0, returns immediately
    ;   - Scans 3-byte entries via CFIELD_EVENTS:
    ;     $FF = end -> msg $2A (list full)
    ;     $FE = empty -> insert new entry
    ;     same pos + type $C0-$D0 -> add value
    ;     else -> advance to next entry
    ; Clobbers: A, Y

    STA     EVENT_SAVED_POS        ; save position
    LDA     #$00
    BANE    NUM_VALUE,.has_value
    BANE    NUM_VALUE+1,.has_value
    RTS
.has_value:
    LDWIY    CHAR_PTR,#CFIELD_EVENTS,EVENT_PTR ; event list ptr
.scan:
    LDY     #EVFIELD_POSITION
    LDA     (EVENT_PTR),Y          ; byte 0 = position
    BANE    #EVENT_END,.not_end    ; $FF = end of list
    JSR     SET_CURSOR_ROW21
    LDA     #MSG_ARMOR_WORTHLESS
    JMP     DISPLAY_MESSAGE        ; list full message
.not_end:
    BANE    #EVENT_EMPTY,.check_match ; $FE = empty slot
    ; --- Insert new entry into empty slot ---
    LDA     EVENT_SAVED_POS
    STA     (EVENT_PTR),Y          ; byte 0 = position
    INY                            ; Y = EVFIELD_TYPE
    LDA     NUM_VALUE+1
    AND     #EVENT_VALUE_HI_MASK   ; keep high 3 bits of value
    ORA     #EVENT_ACTIVATED       ; mark as activated ($C0+)
    STA     (EVENT_PTR),Y
    INY                            ; Y = EVFIELD_VALUE
    LDA     NUM_VALUE
    STA     (EVENT_PTR),Y          ; byte 2 = value low
    RTS
.check_match:
```

```
    BANE    EVENT_SAVED_POS,.next  ; same position?
    INY                            ; Y = EVFIELD_TYPE
    LDA     (EVENT_PTR),Y          ; byte 1 = type
    BPL     .next                  ; not activated (< $80)
    BAMI    #EVENT_ACTIVATED,.next ; below $C0
    BAPL    #EVENT_ACCUM_MAX,.next ; above $D0 -> not accumulable
    ; --- Add value to existing entry ---
    CLC
    LDA     NUM_VALUE
    INY                            ; Y = EVFIELD_VALUE
    ADC     (EVENT_PTR),Y          ; byte 2 += value low
    STA     (EVENT_PTR),Y
    DEY                            ; Y = EVFIELD_TYPE
    LDA     NUM_VALUE+1
    AND     #EVENT_VALUE_HI_MASK   ; high 3 bits of value
    ADC     (EVENT_PTR),Y          ; byte 1 += value high
    STA     (EVENT_PTR),Y
    RTS
.next:
    CLC
    LDA     EVENT_PTR
    ADC     #EVENT_RECORD_SIZE     ; next 3-byte entry
    STA     EVENT_PTR
    BCC     .no_carry
    INC     EVENT_PTR+1
.no_carry:
    JMP     .scan
```

Defines:
UPDATE_EVENT_LIST, used in chunks 349 and 440.

Uses CFIELD_EVENTS 561, CHAR_PTR 591a, DISPLAY_MESSAGE 637, EVENT_ACCUM_MAX 397a, EVENT_ACTIVATED 397a, EVENT_EMPTY 397a, EVENT_END 397a, EVENT_PTR 353a, EVENT_RECORD_SIZE 397a, EVENT_SAVED_POS 453, EVENT_VALUE_HI_MASK 397a, EVFIELD_POSITION 397a, EVFIELD_TYPE 397a, EVFIELD_VALUE 397a, LDWIY 17, MSG_ARMOR_WORTHLESS 634, NUM_VALUE 630a, and SET_CURSOR_ROW21 627.

10.4.7 Leave-area check

CHECK_CAN_LEAVE uses the PRNG to determine whether a character can leave an area.

Mechanic. The routine steps the PRNG and compares the output against a threshold of $40:

- **Pass** (output ≥ $40, roughly 75% chance) — Prints the mob's name followed by message $1C, returns A = 1.
- **Fail** (output < $40, roughly 25% chance) — Prints the mob's name followed by message $1D, returns A = 0.

456 ⟨*check can leave* 456⟩≡ (909)

```
    ORG     $1231
CHECK_CAN_LEAVE:
    SUBROUTINE CHECK_CAN_LEAVE
    ; PRNG check whether entity can leave area.
    ;
    ; Input:  MOB_PTR -- mob whose name to print
    ; Output: A = 1 (can leave), A = 0 (blocked)
    ; Behavior:
    ;   - Steps PRNG; if output >= $40 (~75%):
    ;     prints mob name + msg $1C, returns 1
    ;   - Otherwise: msg $1D, returns 0
    ; Clobbers: A, X, Y

    JSR     STEP_PRNG
    LDA     PRNG_OUTPUT
    BAPL    #$40,.pass
    JMP     .fail
.pass:
    JSR     SET_CURSOR_ROW21
    JSR     PRINT_MOB_NAME
    JSR     SCROLL_STATUS_LINE
    LDA     #MSG_ESCAPE_FAIL       ; "fails to escape"
    JSR     DISPLAY_MESSAGE
    LDA     #$01                   ; success
    RTS
.fail:
    JSR     SET_CURSOR_ROW21
    JSR     PRINT_MOB_NAME
    JSR     SCROLL_STATUS_LINE
    LDA     #MSG_TACKLE_FAIL       ; "fails to tackle"
    JSR     DISPLAY_MESSAGE
    LDA     #$00                   ; failure
    RTS
```

Defines:
CHECK_CAN_LEAVE, used in chunk 232.
Uses DISPLAY_MESSAGE 637, MOB_PTR 608, MSG_ESCAPE_FAIL 634, MSG_TACKLE_FAIL 634, PRINT_MOB_NAME 484, PRNG_OUTPUT 593a, SCROLL_STATUS_LINE 639, SET_CURSOR_ROW21 627, and STEP_PRNG 595.

Chapter 11

Entities

11.1 Entity classification

CLASSIFY_ENTITY_TYPE examines the entity record at ENTITY_REC and sets HANDLER_PTR to the address of an appropriate handler routine. The classification tree checks ENTITY_REC+1 first: a zero high byte means the entity is in zero page, and the low byte further distinguishes two handler addresses. For entities in regular memory, byte 1 of the record determines the category: values below $80 get one handler, values in $80–$BF get another, $C0–$D1 map to a third, $D2–$D4 to a fourth, and $D5–$FD to a fifth. The special value $FE triggers a deeper check on byte 2: its low 5 bits and upper 2 bits select among three more handler addresses, with sub-values below $1B, at $1D, and above $1D each routed differently. Nine distinct handler addresses are used in total, covering the full range of entity types from empty slots to treasure, mobs, shops, and map features.

HANDLER_PTR is a 2-byte zero-page pointer at PRINT_STRING_ADDR that CLASSIFY_ENTITY_TYPE sets to the address of the handler routine matching the entity's type. Callers use this pointer to dispatch entity-specific behavior after classification.

458 ⟨*zero page defines* 25a⟩+≡ (909) ◁453 483a▷

```
HANDLER_PTR          EQU     $BC     ; entity type handler address (2 bytes)
```

Defines:
HANDLER_PTR, used in chunk 459.

459 ⟨*classify entity type* 459⟩≡ (909)

```
    ORG     $116B
CLASSIFY_ENTITY_TYPE:
    SUBROUTINE CLASSIFY_ENTITY_TYPE
    ; Classify entity at ENTITY_REC and set HANDLER_PTR.
    ;
    ; Input:  ENTITY_REC | pointer to entity record
    ; Output: HANDLER_PTR | handler address for entity type
    ;
    ; Behavior:
    ;   Decision tree on pointer value and byte 1/2 fields.
    ;   9 distinct handler addresses for empty slots, mobs,
    ;   shops, map features, treasure, and event types.
    ; Clobbers: A, X, Y

    LDA     #$00
    BANE    ENTITY_REC+1,.not_zero_page ; high byte nonzero -> entity
    BANE    ENTITY_REC,.type_C3    ; low byte nonzero -> ZP pointer
.type_E4:
    STOW    $7FE4,HANDLER_PTR      ; null pointer handler
    RTS
.type_C3:
    STOW    $7FC3,HANDLER_PTR      ; ZP pointer handler
    RTS
.not_zero_page:
    LDY     #MFIELD_TYPE           ; byte 1: entity type
    LDA     (ENTITY_REC),Y
    BMI     .check_ranges          ; bit 7 set -> check ranges
.type_FC:
    STOW    $7FFC,HANDLER_PTR      ; positive type handler
    RTS
.check_ranges:
    BAMI    #$C0,.type_E4          ; < $C0 -> null handler
    BAPL    #$D2,.check_d5         ; >= $D2 -> check next
    STOW    $7FD3,HANDLER_PTR      ; $C0-$D1 handler
    RTS
.check_d5:
    BAPL    #$D5,.check_fe         ; >= $D5 -> check event
    STOW    $8016,HANDLER_PTR      ; $D2-$D4 handler
    RTS
.check_fe:
    BAEQ    #EVENT_EMPTY,.type_fe_entry ; $FE -> event classification
    STOW    $8028,HANDLER_PTR      ; $D5-$FD handler
    RTS
.type_fe_entry:
    LDY     #MFIELD_NEXT           ; byte 2: event data
    LDA     (ENTITY_REC),Y
    TAX
    AND     #STRENGTH_MASK         ; bits 4-0
    BNE     .check_c0_bits         ; nonzero -> check class bits
```

```
    STOW    $8003,HANDLER_PTR      ; strength = 0 handler
    RTS
.check_c0_bits:
    TXA
    AND     #FLAGS_CLASS_MASK      ; bits 7-6
    BANE    #FLAGS_CLASS_MASK,.check_1b ; both set -> check further
.type_F0:
    STOW    $7FF0,HANDLER_PTR      ; class != $C0 handler
    RTS
.check_1b:
    TXA
    AND     #STRENGTH_MASK
    BAPL    #$1B,.check_1d         ; >= $1B -> check $1D
    STOW    $7FCA,HANDLER_PTR      ; < $1B handler
    RTS
.check_1d:
    BAMI    #$1D,.type_FC
    BEQ     .type_F0
    JMP     .type_C3
```

Defines:
CLASSIFY_ENTITY_TYPE, used in chunks 342, 542, and 561.
Uses ENTITY_REC 541b, EVENT_EMPTY 397a, HANDLER_PTR 458, MFIELD_NEXT 561, and STOW 8a.

Converts a mob appearance index (byte 4 of mob data) into a font character number. The custom font has 120 characters in groups of 32, but only 21 mob glyphs occupy each group (characters 1–21 and 33–36). Characters 5–21 repeat cyclically for higher appearance values.

The routine divides the appearance value by 21 (via repeated subtraction of 21), producing a quotient in `Y` and a 1-based remainder in `A`. If the remainder is ≥ 5, it is returned directly. If < 5 and the quotient is nonzero, 32 is added to index into the next font group.

461 ⟨*appearance to fontchar* 461⟩≡ (909)

```
    ORG     $0CF7
APPEARANCE_TO_FONTCHAR:
    SUBROUTINE APPEARANCE_TO_FONTCHAR
    ; Convert appearance index to font character number.
    ;
    ; Input:  A = appearance index
    ; Output: A = font character number, Y = font group (appearance / 21)
    ;         A preserved on carry-through paths

    LDY      #$00                  ; Y = font group counter
.loop:
    CLC
    ADC      #$EB                  ; subtract 21 (256-21)
    BCC      .done                 ; underflow -> done
    INY                            ; next group
    BCS      .loop                 ; always taken
.done:
    ADC      #$16                  ; restore remainder+1 (carry clear)
    BALT    #$05,.low              ; remainder >= 5 -> return as-is
    RTS
.low:
    BYEQ    #$00,.ret              ; group 0 -> return (chars 1-4)
    CLC
    ADC      #$20                  ; add 32 -> next font range
.ret:
    RTS
```

Defines:
`APPEARANCE_TO_FONTCHAR`, used in chunks 225, 317, 462, 475, and 542.

Determines the font character number for an entity whose record pointer is in \ENTITY_REC/\ENTITY_REC+1. Sets FONT_CHARSET to 1 (custom font) and stores the result in FONT_CHARNUM.

Three cases based on the pointer value:

- ENTITY_REC = $0000: empty cell — uses BLINK_ALT_CHAR
- Bit 7 of ENTITY_REC+1 set: mob — reads byte 4 (appearance) from the mob record and calls APPEARANCE_TO_FONTCHAR
- Otherwise: character record — decodes the top 2 bits of byte 1 to select from characters $18--$1D, with further subdivision based on the full byte 1 value

The routine below implements the font-character lookup.

462 ⟨*get entity fontchar* 462⟩≡ (909)

```
    ORG     $0C7E
GET_ENTITY_FONTCHAR:
    SUBROUTINE GET_ENTITY_FONTCHAR
    ; Determine font character for entity at ENTITY_REC.
    ;
    ; Input:  ENTITY_REC | pointer to entity record
    ; Output: FONT_CHARNUM set, FONT_CHARSET = 1
    ;
    ; Behavior:
    ;   Null pointer -> BLINK_ALT_CHAR.  Mob (bit 7) -> appearance
    ;   lookup.  Character -> top 2 bits of byte 1 select glyph.
    ; Clobbers: A, Y

    STOB    #$01,FONT_CHARSET      ; select custom font
    LDA     #$00
    BANE    ENTITY_REC+1,.has_entity
    BANE    ENTITY_REC,.low_only
.empty:
    MOVB    BLINK_ALT_CHAR,FONT_CHARNUM ; empty cell
    RTS
.low_only:
    MOVB    DEFAULT_CHAR,FONT_CHARNUM ; ZP-only pointer
    RTS
.has_entity:
    LDA     ENTITY_REC+1
    BAPL    #$80,.mob              ; bit 7 set -> mob record

    ; --- character record ---
    LDY     #MFIELD_TYPE           ; byte 1: type/class
    LDA     (ENTITY_REC),Y
    AND     #FLAGS_CLASS_MASK      ; top 2 bits
```

```
    BAEQ    #FLAGS_CLASS_MASK,.both_bits
    BAEQ    #$40,.bit6              ; bit 6 only -> $1C
    BPL     .empty                  ; neither -> empty
    LDA     #$1B                    ; bit 7 only -> $1B
    BNE     .store
.bit6:
    LDA     #$1C
.store:
    STA     FONT_CHARNUM
    RTS
.both_bits:
    LDA     (ENTITY_REC),Y         ; reload type byte
    BAEQ    #EVENT_EMPTY,.check_byte2 ; $FE -> check byte 2
    BAMI    #$D5,.lt_d5
    LDA     #$1D                    ; >= $D5 -> $1D
    JMP     .store
.lt_d5:
    BAMI    #$D2,.lt_d2
    LDA     #$18                    ; $D2-$D4 -> $18
    JMP     .store
.lt_d2:
    LDA     #$1A                    ; < $D2 -> $1A
    JMP     .store
.check_byte2:
    LDY     #MFIELD_NEXT            ; byte 2: event data
    LDA     (ENTITY_REC),Y
    AND     #STRENGTH_MASK          ; low 5 bits
    BEQ     .empty                  ; zero -> empty
    JMP     .store                  ; nonzero -> use as charnum

    ; --- mob record ---
.mob:
    MOVB    ENTITY_REC,MOB_DATA_PTR ; \ set up mob pointer
    LDA     ENTITY_REC+1             ;  | (clear bit 7)
    AND     #$7F                     ;  |
    STA     MOB_DATA_PTR+1           ; /
    LDY     #MFIELD_APPEARANCE      ; byte 4: appearance
    LDA     (MOB_DATA_PTR),Y
    JSR     APPEARANCE_TO_FONTCHAR
    JMP     .store
```

Defines:
GET_ENTITY_FONTCHAR, used in chunks 342, 471, 475, 540, and 542.

Uses APPEARANCE_TO_FONTCHAR 461, BLINK_ALT_CHAR 530b, DEFAULT_CHAR 530b, ENTITY_REC 541b, EVENT_EMPTY 397a, FONT_CHARNUM 575, FONT_CHARSET 576a, MFIELD_APPEARANCE 561, MFIELD_NEXT 561, MOB_DATA_PTR 600b, MOVB 8c, and STOB 8c.

11.2 Entity rendering

HRCG_CHARSET1_SEL is the HRCG ctrl-A argument that selects custom-font group 1: ASCII '1' with the high bit set ($B1). CHARS_PER_FONT_PAGE is the number of custom-font characters per HRCG charset page ($96 \div 4 = 24$). GLYPH_BASE ($20) is the glyph code for the first printable position in any HRCG charset (ASCII space), so the four glyph codes for font character n are $n \times 4 + \$20$ through $n \times 4 + \$23$.

464 ⟨*defines* 25b⟩+≡ (909) ◁451a 468▷

```
HRCG_CHARSET1_SEL   EQU     $B1     ; HRCG charset-select arg for custom font group 1
CHARS_PER_FONT_PAGE EQU     $18     ; custom font chars per HRCG charset page (96/4)
GLYPH_BASE          EQU     $20     ; first glyph code in any HRCG charset (ASCII space)
```

Defines:
CHARS_PER_FONT_PAGE, used in chunk 466.
GLYPH_BASE, used in chunk 466.
HRCG_CHARSET1_SEL, used in chunk 466.

11.2.1 Render font character

RENDER_FONT_CHAR draws one 2×2 custom-font character on the hi-res screen.

Charset mapping. The custom font spans multiple HRCG charsets. Group 1 starts at HRCG charset 1 (HRCG_CHARSET1_SEL = $B1); group 2 starts at HRCG charset 3 ($B3), skipping charset 2 which is reserved for border characters. If FONT_CHARNUM exceeds CHARS_PER_FONT_PAGE (24), the routine advances the HRCG charset selector one step at a time until the remainder fits.

Glyph computation. The four glyph codes are *charnum* $\times 4 +$ GLYPH_BASE through +3. These are patched into s_PRINT_FONT_CHAR, which is then passed to PRINT_STRING for HRCG to render.

PRINT_FONTCHAR_AT_TEXT_POS is an alternate entry point that omits the position conversion, for callers that have already set TEXT_COL and TEXT_ROW.

465 ⟨*print font char data* 465⟩≡ (909)

```
    ORG     $80A4
; Self-modifying HRCG control string for 2x2 block character rendering.
; RENDER_FONT_CHAR patches CHAR_CHARSET and the four glyph codes before each call.
s_PRINT_FONT_CHAR:
    HEX     09              ; string length: 9 bytes follow
    HEX     81              ; ctrl-A: select charset (next byte = selector)
CHAR_CHARSET:
    HEX     B1              ; $80A6: self-modified HRCG charset selector
    HEX     82              ; ctrl-B: enter block-display mode
CHAR_UPPER_LEFT:
    HEX     A0              ; $80A8: self-modified upper-left glyph
CHAR_UPPER_RIGHT:
    HEX     A0              ; $80A9: self-modified upper-right glyph
    HEX     83              ; ctrl-C: advance to lower-left of block
CHAR_LOWER_LEFT:
    HEX     A0              ; $80AB: self-modified lower-left glyph
CHAR_LOWER_RIGHT:
    HEX     A0              ; $80AC: self-modified lower-right glyph
    HEX     84              ; ctrl-D: complete block and emit it
```

Defines:
CHAR_CHARSET, used in chunks 466 and 570b.
CHAR_LOWER_LEFT, used in chunk 466.
CHAR_LOWER_RIGHT, used in chunk 466.
CHAR_UPPER_LEFT, used in chunk 466.
CHAR_UPPER_RIGHT, used in chunk 466.
s_PRINT_FONT_CHAR, used in chunk 466.
Uses RENDER_FONT_CHAR 466.

466 ⟨*render font char* 466⟩≡ (909)

```
    ORG     $7489
RENDER_FONT_CHAR:
    SUBROUTINE RENDER_FONT_CHAR
    ; Draw a 2x2 custom font character at the current font position.
    ;
    ; Inputs:
    ;   FONT_COL, FONT_ROW   | font grid position
    ;   FONT_CHARNUM          | character index within font group
    ;   FONT_CHARSET          | font group (1 = primary, 2 = border)
    ;
    ; Behavior:
    ;   Converts font position to HRCG text cursor, maps charnum to
    ;   four consecutive glyph codes across HRCG charset pages, patches
    ;   s_PRINT_FONT_CHAR, and calls PRINT_STRING to render.
    ;
    ; Modifies: s_PRINT_FONT_CHAR (CHAR_CHARSET, glyph codes),
    ;           PRINT_STRING_ADDR
    ; Clobbers: A, X, Y

    JSR     FONT_POS_TO_TEXT_POS        ; convert font pos to HRCG cursor
PRINT_FONTCHAR_AT_TEXT_POS:
    ; --- Map font group to initial HRCG charset selector ---
    LDX     #HRCG_CHARSET1_SEL          ; default: HRCG charset 1
    LDA     FONT_CHARSET
    BANE    #$02,.charset_ok            ; group 1 -> use charset 1
    INX                                 ; group 2: skip charset 2 (border)
    INX                                 ;          -> HRCG charset 3
.charset_ok:
    STX     CHAR_CHARSET                ; patch selector in control string
    ; --- Walk charset pages until charnum fits ---
RENDER_FONT_CHAR_DIRECT:                ; entry: render from FONT_CHARNUM directly
    LDA     FONT_CHARNUM
.check_range:
    BALT    #CHARS_PER_FONT_PAGE,.compute_glyphs ; < 24? fits in current page
    INX                                 ; advance to next HRCG charset
    STX     CHAR_CHARSET
    SEC
    SBC     #CHARS_PER_FONT_PAGE        ; charnum -= 24
    JMP     .check_range
    ; --- Compute four consecutive glyph codes ---
.compute_glyphs:
    ASL                                 ; charnum * 2
    ASL                                 ; charnum * 4
    CLC
    ADC     #GLYPH_BASE                 ; + $20 -> upper-left glyph code
    TAY
    STY     CHAR_UPPER_LEFT             ; glyph 0: upper-left
    INY
    STY     CHAR_UPPER_RIGHT            ; glyph 1: upper-right
```

```
    INY
    STY     CHAR_LOWER_LEFT             ; glyph 2: lower-left
    INY
    STY     CHAR_LOWER_RIGHT            ; glyph 3: lower-right
    ; --- Render via HRCG ---
    STOW    s_PRINT_FONT_CHAR,PRINT_STRING_ADDR ; aim string ptr at control string
    JMP     PRINT_STRING                ; HRCG renders the 2x2 block
```

Defines:
PRINT_FONTCHAR_AT_TEXT_POS, used in chunks 49 and 576b.
RENDER_FONT_CHAR, used in chunks 239, 342, 465, 471, 473b, 487, 538, 540, 542, 545, and 644.
RENDER_FONT_CHAR_DIRECT, used in chunk 584.
Uses CHARS_PER_FONT_PAGE 464, CHAR_CHARSET 465, CHAR_LOWER_LEFT 465, CHAR_LOWER_RIGHT 465, CHAR_UPPER_LEFT 465, CHAR_UPPER_RIGHT 465, FONT_CHARNUM 575, FONT_CHARSET 576a, FONT_COL 492a, FONT_POS_TO_TEXT_POS 467, FONT_ROW 534c, GLYPH_BASE 464, HRCG_CHARSET1_SEL 464, PRINT_STRING 622, PRINT_STRING_ADDR 576c, STOW 8a, and s_PRINT_FONT_CHAR 465.

11.2.2 Font position to text position

FONT_POS_TO_TEXT_POS converts a logical font position (FONT_COL, FONT_ROW) to HRCG text cursor coordinates. Each custom font character occupies a 2 × 2 block of HRCG cells, so both coordinates are doubled and stored into the Apple II cursor registers TEXT_COL ($24) and TEXT_ROW ($25).

467 ⟨*font pos* 467⟩≡ (909)

```
    ORG     $7723
FONT_POS_TO_TEXT_POS:
    SUBROUTINE FONT_POS_TO_TEXT_POS
    ; Convert font grid position to HRCG text cursor.
    ; Each font cell = 2x2 HRCG cells, so double both coordinates.
    ;
    ; Inputs:  FONT_COL, FONT_ROW
    ; Outputs: TEXT_COL ($24), TEXT_ROW ($25)
    ; Clobbers: A

    LDA     FONT_COL            ; font column (0-based)
    ASL                         ; x2: one font col = two text cols
    STA     TEXT_COL             ; store HRCG cursor column
    LDA     FONT_ROW            ; font row (0-based)
    ASL                         ; x2: one font row = two text rows
    STA     TEXT_ROW             ; store HRCG cursor row
    RTS
```

Defines:
FONT_POS_TO_TEXT_POS, used in chunks 466 and 538.
Uses FONT_COL 492a, FONT_ROW 534c, TEXT_COL 572a, and TEXT_ROW 572a.

11.2.3 Find entity at position

FIND_ENTITY_AT_POS searches for the entity (mob or player character) occupying a given map position.

- **Input:** A = target map position (0–199); CHAR_PTR ($FA/$FB) = base address of the character record.
- **Output:** ENTITY_REC/ENTITY_REC+1 = pointer to the entity record. Bit 7 of ENTITY_REC+1 is set (MOB_PTR_FLAG) if the entity is a mob; clear if it is a player character.

Search algorithm. The routine uses self-modifying code: absolute operands in .load1–.load4 are patched at runtime to redirect indexed loads to the appropriate record addresses. The search proceeds in two phases:

1. **Mob linked-list search** (entered first). CHAR_PTR+MFIELD_NEXT (byte 2) holds the first mob index. Zero means the list is exhausted. For each nonzero index, GET_MOB_DATA resolves the mob record, and MFIELD_POSITION (byte 3) is compared with the target. On a match, ENTITY_REC is set with MOB_PTR_FLAG ($80) ORed into the high byte.
2. **Player event-table search.** Bytes CFIELD_EVENTS–CFIELD_EVENTS+1 (bytes 6–7) of the base record point to a 3-byte-entry table whose byte 0 is the character's map position. EVENT_END ($FF) signals end-of-table; on a match, ENTITY_REC is set to the table entry address (bit 7 clear = character).

ENTITY_SEARCH_POS at LOCATION_ID temporarily holds the target map position during FIND_ENTITY_AT_POS.

MOB_PTR_FLAG ($80) is ORed into the high byte of ENTITY_REC+1 to distinguish mob records (bit 7 set) from character records (bit 7 clear).

468 ⟨*defines* 25b⟩+≡ (909) ◁464 491▷

```
MOB_PTR_FLAG        EQU     $80     ; bit 7 flag in ENTITY_REC+1: set = mob
```

Defines:
MOB_PTR_FLAG, used in chunks 232 and 469.
Uses ENTITY_REC 541b.

469 ⟨*find entity at pos* 469⟩≡ (909)

```
    ORG     $0B62
FIND_ENTITY_AT_POS:
    SUBROUTINE FIND_ENTITY_AT_POS
    ; Find the entity (mob or character) at a given map position.
    ;
    ; Inputs:
    ;   A               | target map position (0-199)
    ;   CHAR_PTR        | base address of character record
    ;
    ; Behavior:
    ;   Walks the mob linked list first; if no mob matches, searches
    ;   the player character event table.  Uses self-modifying code
    ;   to patch absolute operands for indexed loads.
    ;
    ; Outputs:
    ;   ENTITY_REC/+1   | pointer to matching entity record
    ;                      (bit 7 of high byte = MOB_PTR_FLAG if mob)
    ; Clobbers: A, X, Y

    STA     ENTITY_SEARCH_POS          ; save target position
    LDA     CHAR_PTR+1                 ; \ patch base address into
    STA     .load3+2                   ;  | player search (high byte)
    STA     .load2+2                   ;  | mob search (high byte)
    LDA     CHAR_PTR                   ;  |
    STA     .load3+1                   ;  | player search (low byte)
    STA     .load2+1                   ; / mob search (low byte)
    JMP     .mob_search

    ; --- Player character event-table search ---
.player_search:
    LDX     #CFIELD_EVENTS             ; \ copy event list pointer (bytes 6-7)
.load3:                                ;  |
    LDA     $0B7B,X                    ;  | (self-modified: base = CHAR_PTR)
    STA     .load4-5,X                 ;  | patch .load4 operand
    INX                                ;  |
    BXNE    #CFIELD_EVENTS+2,.load3    ; / loop for both bytes
.load4:
    LDY     $0B86                      ; (self-modified: read position from event table)
    BYNE    #EVENT_END,.check_player   ; $FF = end of table
    JMP     CHECK_AT_BOUNDARY
.check_player:
    BYEQ    ENTITY_SEARCH_POS,.found_player ; position matches target?
    INC     .load4+1                   ; \
    BNE     .no_carry1                 ;  |
    INC     .load4+2                   ;  | advance pointer by 3
.no_carry1:                            ;  | (EVENT_RECORD_SIZE = 3)
    INC     .load4+1                   ;  |
    BNE     .no_carry2                 ;  |
    INC     .load4+2                   ;  |
```

```
.no_carry2:                         ;  |
    INC     .load4+1                ;  |
    BNE     .load4                  ;  |
    INC     .load4+2                ; /
    JMP     .load4
.found_player:
    LDA     .load4+1                ; \ ENTITY_REC = event table entry address
    STA     ENTITY_REC              ;  | (bit 7 clear = character)
    LDA     .load4+2                ;  |
    STA     ENTITY_REC+1            ; /
    RTS

    ; --- Mob linked-list search ---
.mob_search:
    LDX     #MFIELD_NEXT            ; byte 2 = mob link index
.load2:
    LDA     $0BBD,X                 ; (self-modified: base = CHAR_PTR)
    BAEQ    #$00,.player_search     ; zero -> no more mobs, try players
    JSR     .get_mob                ; resolve mob data -> patch .load1
    LDX     #MFIELD_POSITION        ; byte 3 = map position
.load1:
    LDY     $0BC9,X                 ; (self-modified: read from mob record)
    BYEQ    ENTITY_SEARCH_POS,.found_mob ; position matches target?
    LDA     .load1+1                ; \ copy mob link address back
    STA     .load2+1                ;  | to .load2 for next iteration
    LDA     .load1+2                ;  |
    STA     .load2+2                ; /
    JMP     .mob_search
.found_mob:
    LDA     #MOB_PTR_FLAG           ; \ set bit 7 = mob pointer flag
    ORA     .load1+2                ;  |
    STA     ENTITY_REC+1            ;  |
    LDA     .load1+1                ;  |
    STA     ENTITY_REC              ; / ENTITY_REC = mob ptr | MOB_PTR_FLAG
    RTS

    ; --- Helper: load mob record and patch .load1 ---
.get_mob:
    JSR     GET_MOB_DATA            ; A = mob index -> MOB_DATA_PTR
    LDA     MOB_DATA_PTR            ; \
    STA     .load1+1                ;  | patch .load1 with mob record base
    LDA     MOB_DATA_PTR+1          ;  |
    STA     .load1+2                ; /
    RTS
```

Defines:
FIND_ENTITY_AT_POS, used in chunks 232, 312, 322, 330, 394, 471, and 540–42.
Uses BYNE 18, CFIELD_EVENTS 561, CHAR_PTR 591a, CHECK_AT_BOUNDARY 537, ENTITY_REC 541b, ENTITY_SEARCH_POS 536, EVENT_END 397a, EVENT_RECORD_SIZE 397a, GET_MOB_DATA 602, MFIELD_NEXT 561, MFIELD_POSITION 561, MOB_DATA_PTR 600b, and MOB_PTR_FLAG 468.

11.2.4 Draw character at position

DRAW_CHAR_AT_POS converts a map position to a font cursor location, resolves the occupying entity, and renders its font character.

471 ⟨*draw char at pos* 471⟩≡ (909)

```
      ORG     $1156
  DRAW_CHAR_AT_POS:
      SUBROUTINE DRAW_CHAR_AT_POS
      ; Draw the entity sprite at a map position.
      ;
      ; Inputs:  A = map position (0-199)
      ; Behavior:
      ;   Converts position to font col/row, finds the entity there,
      ;   determines its font character, and renders it.
      ; Clobbers: A, X, Y

      JSR     POS_TO_COLROW       ; A=pos -> A=col, Y=row
      STA     FONT_COL            ; set font column
      STY     FONT_ROW            ; set font row
      JSR     COLROW_TO_POS       ; re-linearize (A = col + row*20)
      JSR     FIND_ENTITY_AT_POS  ; resolve entity at pos -> ENTITY_REC
      JSR     GET_ENTITY_FONTCHAR ; entity data -> font char number
      JMP     RENDER_FONT_CHAR    ; tail-call: render and return
```

Defines:
DRAW_CHAR_AT_POS, used in chunks 239, 342, 349, 371, and 494.
Uses COLROW_TO_POS 600a, ENTITY_REC 541b, FIND_ENTITY_AT_POS 469, FONT_COL 492a, FONT_ROW 534c, GET_ENTITY_FONTCHAR 462, POS_TO_COLROW 599, and RENDER_FONT_CHAR 466.

11.2.5 Draw blink alternate

DRAW_BLINK_ALT temporarily swaps BLINK_CHAR for the alternate character BLINK_ALT_CHAR, draws it via DRAW_BLINK_CHAR, then restores the original. The animation caller alternates between DRAW_BLINK_CHAR (entity sprite) and DRAW_BLINK_ALT (location tile) to produce the blinking effect.

472a ⟨*draw blink alt* 472a⟩≡ (909)

```
      ORG     $74CB
  DRAW_BLINK_ALT:
      SUBROUTINE DRAW_BLINK_ALT
      ; Draw blink position with the alternate (tile) character.
      ;
      ; Behavior:
      ;   Saves BLINK_CHAR, swaps in BLINK_ALT_CHAR, draws, restores.
      ; Clobbers: A, X, Y

      LDA     BLINK_CHAR              ; save current blink char (entity sprite)
      PHA
      MOVB    BLINK_ALT_CHAR,BLINK_CHAR ; swap in alternate (location tile)
      JSR     DRAW_BLINK_CHAR         ; draw alternate at blink position
      PULB    BLINK_CHAR              ; restore sprite char
      RTS
```

Defines:
DRAW_BLINK_ALT, used in chunks 545 and 604b.
Uses BLINK_ALT_CHAR 530b, BLINK_CHAR 544c, DRAW_BLINK_CHAR 473b, MOVB 8c, and PULB 9c.

11.2.6 Draw blink character

DRAW_BLINK_NORMAL sets normal video mode via PRINT_CTRL_N, then falls through into DRAW_BLINK_CHAR. Called by ANIM_TICK_AND_WAIT when the animation frame expires.

DRAW_BLINK_CHAR copies the blink state (BLINK_COL, BLINK_ROW, BLINK_CHAR) into the font variables and calls RENDER_FONT_CHAR. If BLINK_COL $\geq$ MAP_WIDTH (20), blinking is disabled and the routine returns immediately. Saves and restores PRINT_STRING_ADDR across the call since RENDER_FONT_CHAR overwrites it.

472b ⟨*var current col* 472b⟩≡ (909)

```
      ORG     $5A63
  CURRENT_COL:
      DC.B    $D4                     ; initial value (runtime: current column during turn)
```

Defines:
CURRENT_COL, used in chunks 232, 239, 323, 329, 330, 354, 357, 360, 475, and 898.

473a ⟨*var current row* 473a⟩≡ (909)

```
    ORG     $5A64
CURRENT_ROW:
    DC.B    $C5                     ; initial value (runtime: current row during turn)
```

Defines:
CURRENT_ROW, used in chunks 232, 239, 323, 329, 330, 354, 357, 360, 475, and 898.

473b ⟨*draw blink char* 473b⟩≡ (909)

```
    ORG     $74DD
DRAW_BLINK_NORMAL:
    JSR     PRINT_CTRL_N                ; Ctrl-N: restore normal video

DRAW_BLINK_CHAR:
    SUBROUTINE DRAW_BLINK_CHAR
    ; Render the blink character at the stored blink position.
    ;
    ; Behavior:
    ;   If BLINK_COL >= MAP_WIDTH (disabled), returns immediately.
    ;   Otherwise copies blink state to font vars, selects charset 1,
    ;   and calls RENDER_FONT_CHAR.
    ;
    ; Modifies: FONT_COL, FONT_ROW, FONT_CHARNUM, FONT_CHARSET
    ; Clobbers: A, X, Y

    LDA     BLINK_COL               ; load blink column
    BALT    #MAP_WIDTH,.draw        ; < 20? -> active, draw it
    RTS                             ; blink disabled -- do nothing

.draw:
    STA     FONT_COL                ; set font column from blink state
    MOVB    BLINK_ROW,FONT_ROW      ; set font row
    MOVB    BLINK_CHAR,FONT_CHARNUM ; set character number
    STOB    #$01,FONT_CHARSET       ; charset 1 (custom font)
    PSHW    PRINT_STRING_ADDR       ; save (RENDER_FONT_CHAR overwrites)
    JSR     RENDER_FONT_CHAR        ; render the blink character
    PULW    PRINT_STRING_ADDR       ; restore caller's string pointer
    RTS
```

Defines:
DRAW_BLINK_CHAR, used in chunk 472a.
DRAW_BLINK_NORMAL, used in chunks 545 and 605b.
Uses BLINK_CHAR 544c, BLINK_COL 563b, BLINK_ROW 544b, FONT_CHARNUM 575, FONT_CHARSET 576a, FONT_COL 492a, FONT_ROW 534c, MAP_WIDTH 388a, MOVB 8c, PRINT_CTRL_N 623b, PRINT_STRING_ADDR 576c, PSHW 9b, PULW 10a, RENDER_FONT_CHAR 466, and STOB 8c.

11.2.7 Set blink target

SET_BLINK_TARGET loads BLINK_COL, BLINK_ROW, and BLINK_CHAR from the current combatant so the blink cursor tracks the active entity. IS_PLAYER_TURN selects one of two paths:

- **Mob's turn** (IS_PLAYER_TURN = 0): the mob's packed position (MFIELD_POSITION, byte 3 via MOB_PTR) is split into column and row by POS_TO_COLROW, and the appearance index (MFIELD_APPEARANCE, byte 4) is converted to a font character by APPEARANCE_TO_FONTCHAR.
- **Player's turn** (IS_PLAYER_TURN = 1): column and row come from CURRENT_COL/CURRENT_ROW; MOB_PTR is copied to ENTITY_REC so GET_ENTITY_FONTCHAR can look up the player sprite.

474 ⟨*var is player turn* 474⟩≡ (909)

```
    ORG     $5A74
IS_PLAYER_TURN:
    DC.B    $CD                     ; initial value (runtime: 0 = mob's turn, 1 = player's turn)
```

Defines:
IS_PLAYER_TURN, used in chunks 325, 330, 338, 370a, and 475.

475 ⟨*set blink target* 475⟩≡ (909)

```
      ORG     $16A3
  SET_BLINK_TARGET:
      SUBROUTINE SET_BLINK_TARGET
      ; Set blink cursor position and character from current combatant.
      ;
      ; Inputs:
      ;   IS_PLAYER_TURN  | 0 = mob, 1 = player
      ;   MOB_PTR         | pointer to current entity's record
      ;
      ; Behavior:
      ;   On mob's turn, unpacks position from mob record and converts
      ;   appearance to font char.  On player's turn, reads position
      ;   from CURRENT_COL/ROW and looks up font char via entity record.
      ;
      ; Outputs: BLINK_COL, BLINK_ROW, BLINK_CHAR
      ; Clobbers: A, X, Y

      LDA     IS_PLAYER_TURN              ; 0 = mob, 1 = player
      BAEQ    #$01,.player

      ; --- mob path: unpack position and look up appearance ---
      LDY     #MFIELD_POSITION            ; byte 3: packed map position
      LDA     (MOB_PTR),Y
      JSR     POS_TO_COLROW               ; -> A = col, Y = row
      STA     BLINK_COL
      STY     BLINK_ROW
      LDY     #MFIELD_APPEARANCE          ; byte 4: appearance index
      LDA     (MOB_PTR),Y
      JSR     APPEARANCE_TO_FONTCHAR      ; -> A = font char number
      STA     BLINK_CHAR
      RTS

      ; --- player path: position already split ---
  .player:
      MOVB    CURRENT_COL,BLINK_COL       ; copy player column
      MOVB    CURRENT_ROW,BLINK_ROW       ; copy player row
      MOVW    MOB_PTR,ENTITY_REC          ; point ENTITY_REC at player record
      JSR     GET_ENTITY_FONTCHAR         ; look up player's font char
      MOVB    FONT_CHARNUM,BLINK_CHAR     ; store result
      RTS
```

Defines:
SET_BLINK_TARGET, used in chunk 338.

Uses APPEARANCE_TO_FONTCHAR 461, BLINK_CHAR 544c, BLINK_COL 563b, BLINK_ROW 544b, CURRENT_COL 472b, CURRENT_ROW 473a, ENTITY_REC 541b, FONT_CHARNUM 575, GET_ENTITY_FONTCHAR 462, IS_PLAYER_TURN 474, MFIELD_APPEARANCE 561, MFIELD_POSITION 561, MOB_PTR 608, MOVB 8c, MOVW 9a, and POS_TO_COLROW 599.

11.3 Entity management

Performs one step of an insertion-sort on a linked list of characters, ordered by byte 11 (level, masked to 5 bits). It unlinks the character at index SOURCE_CHAR from the list and re-inserts it in sorted order relative to index ACTIVE_CHAR.

The linked-list "next" pointer is byte 2 of each character record. GET_MOB_DATA resolves a byte-2 value into a record pointer at \MOB_DATA_PTR/\MOB_DATA_PTR+1.

Phase 1 (removal):

1. Walks the chain from SOURCE_CHAR's record, following byte-2 links via GET_MOB_DATA until \MOB_DATA_PTR/\MOB_DATA_PTR+1 equals \TARGET_REC/\TARGET_REC+1 (found the predecessor).
2. Patches the predecessor's byte 2 to skip the removed node.
3. Decrements the source group's member count via DECR_GROUP_COUNT.

Phase 2 (insertion):

1. Walks the chain from ACTIVE_CHAR's record, comparing byte 11 (level) values.
2. Inserts the removed character before the first node with a lower level, or at the end if none found.
3. Increments the destination group's member count via INCR_GROUP_COUNT.
4. Adjusts TOTAL_MOB_COUNT to track list membership changes.

When ACTIVE_CHAR is zero, a random decision (via STEP_PRNG) determines the insertion point instead of the level comparison.

REORDER_SORT_KEY temporarily holds byte 11 (level) of the node being compared during the insertion phase. REORDER_LINK saves the removed node's next-link so it can be spliced back in. CURRENT_ROOM records which room is being processed. REORDER_CHAR uses all three during the two-phase unlink-and-reinsert operation.

476 ⟨*var current room* 476⟩≡ (909)

```
    ORG     $5A03
CURRENT_ROOM:
    DC.B    $A0                     ; initial value (runtime: current room/location being processed)
```

Defines:
CURRENT_ROOM, used in chunks 222, 291, 447, and 478.

477a ⟨*var reorder link* 477a⟩≡ (909)

```
    ORG     $5A2A
REORDER_LINK:
    DC.B    $C4                     ; initial value (runtime: temp: saved next-link during reorder)
```

Defines:
REORDER_LINK, used in chunk 478.

477b ⟨*var reorder sort key* 477b⟩≡ (909)

```
    ORG     $5A2B
REORDER_SORT_KEY:
    DC.B    $A0                     ; initial value (runtime: temp: byte 11 of node for sorted inser
```

Defines:
REORDER_SORT_KEY, used in chunk 478.

478 ⟨*reorder char* 478⟩≡ (909)

```
        ORG     $0F84
    REORDER_CHAR:
        SUBROUTINE REORDER_CHAR
        ; Move a character from one group's chain into another, sorted by level.
        ;
        ; Inputs:
        ;   SOURCE_CHAR     | index of the character to remove from its current group
        ;   ACTIVE_CHAR     | destination group head index (0 = random insertion)
        ;   TARGET_REC      | record pointer for the character being moved
        ;
        ; Behavior:
        ;   Phase 1 walks SOURCE_CHAR's group chain to find the predecessor node,
        ;   patches it to skip over the removed node (unlinking it), and decrements
        ;   the source group's member count.
        ;   Phase 2 walks the destination group chain and inserts the removed node
        ;   at the correct position: in ascending level order when ACTIVE_CHAR != 0,
        ;   or at a random position when ACTIVE_CHAR = 0 (coin-flip via STEP_PRNG).
        ;   After insertion, adjusts TOTAL_MOB_COUNT if the moved character is an
        ;   NPC (appearance < NPC_APPEARANCE_MIN): decrements if source was group 0,
        ;   increments if destination is group 0.
        ;
        ; Modifies:
        ;   REORDER_LINK, REORDER_SORT_KEY, CURRENT_ROOM, TOTAL_MOB_COUNT,
        ;   CHAR_REC, MOB_DATA_PTR
        ; Clobbers: A, X, Y

    ; --- Phase 1: unlink character SOURCE_CHAR from its current chain ---
        LDA     SOURCE_CHAR             ; source character index
        JSR     GET_CHAR_RECORD         ; CHAR_REC = record pointer
        JSR     DECR_GROUP_COUNT        ; source group lost a member
    .walk1:
        LDY     #CFIELD_NEXT            ; \
        LDA     (CHAR_REC),Y            ;  | REORDER_LINK = byte 2 of current record
        STA     REORDER_LINK            ; /  (the "next" link we're removing)
        JSR     GET_MOB_DATA            ; MOB_DATA_PTR/MOB_DATA_PTR+1 = resolve link; TARGET_REC/TARGET_R
        LDA     MOB_DATA_PTR            ; \
        BANE    TARGET_REC,.advance1    ; if different, keep walking
        LDA     MOB_DATA_PTR+1          ;  |
        BANE    TARGET_REC+1,.advance1
        ; Found predecessor: TARGET_REC/TARGET_REC+1 points to the node being removed
        LDY     #CFIELD_NEXT
        LDA     REORDER_LINK            ; \  if removed node's link == CURRENT_ROOM,
        BANE    CURRENT_ROOM,.patch1
        LDA     (TARGET_REC),Y          ; \
        STA     CURRENT_ROOM            ; /  CURRENT_ROOM = (TARGET_REC)[CFIELD_NEXT] (successor's link)
    .patch1:
        LDA     (TARGET_REC),Y          ; \  predecessor.byte2 = removed.byte2
        STA     (CHAR_REC),Y            ; /  (skip over removed node)
        JMP     .phase2
```

```
.advance1:
    MOVW    MOB_DATA_PTR,CHAR_REC ; CHAR_REC = MOB_DATA_PTR/MOB_DATA_PTR+1
    JMP     .walk1

; --- Phase 2: re-insert at correct sorted position in ACTIVE_CHAR's chain ---
.phase2:
    LDY     #MFIELD_LEVEL          ; \
    LDA     (TARGET_REC),Y         ;  | REORDER_SORT_KEY = byte 11 of removed node
    AND     #LEVEL_MASK            ;  |   (level, 5-bit)
    STA     REORDER_SORT_KEY       ; /
    LDA     ACTIVE_CHAR            ; destination character index
    JSR     GET_CHAR_RECORD        ; CHAR_REC = dest record pointer
    JSR     INCR_GROUP_COUNT       ; dest group gained a member
.walk2:
    LDY     #MFIELD_NEXT           ; \
    LDA     (CHAR_REC),Y           ;  | check byte 2 of current record
    BAEQ    #$00,.insert
    JSR     GET_MOB_DATA           ; resolve link
    LDA     ACTIVE_CHAR            ; \
    BEQ     .random                ; /  if dest index = 0, use random decision
    LDY     #MFIELD_LEVEL          ; \
    LDA     (MOB_DATA_PTR),Y       ;  | compare candidate's level
    AND     #LEVEL_MASK            ;  | with removed node's level
    BAPL    REORDER_SORT_KEY,.advance2 ; if >= , keep walking (insert later)
.insert:
    LDY     #MFIELD_NEXT           ; \
    LDA     (CHAR_REC),Y           ;  | (TARGET_REC)[CFIELD_NEXT] = (CHAR_REC)[MFIELD_NEXT]
    STA     (TARGET_REC),Y         ;  |
    LDA     REORDER_LINK           ;  | (CHAR_REC)[MFIELD_NEXT] = saved link (insert node)
    STA     (CHAR_REC),Y           ; /
    LDY     #CFIELD_APPEARANCE     ; \
    LDA     (TARGET_REC),Y         ;  | if byte 4 of ($BC) >= $15, done
    BALT    #$15,.adjust
    RTS                            ; /
.adjust:
    LDA     #$00                   ; \
    BANE    ACTIVE_CHAR,.chk28     ; decrement TOTAL_MOB_COUNT
    DEC     TOTAL_MOB_COUNT        ; /
.chk28:
    BANE    SOURCE_CHAR,.done      ; increment TOTAL_MOB_COUNT
    INC     TOTAL_MOB_COUNT        ; /
.done:
    RTS
.advance2:
    MOVW    MOB_DATA_PTR,CHAR_REC ; CHAR_REC = MOB_DATA_PTR/MOB_DATA_PTR+1
    JMP     .walk2
.random:
    JSR     STEP_PRNG              ; \  random decision
    LDA     PRNG_OUTPUT            ;  | if PRNG < 1 (i.e., == 0) -> insert
    BAMI    #$01,.insert ;  else keep walking
```

```
    BPL     .advance2               ; /
```

Defines:
REORDER_CHAR, used in chunks 349, 420, 432, 489a, and 494.

Uses ACTIVE_CHAR 492c, CFIELD_APPEARANCE 561, CFIELD_NEXT 561, CHAR_REC 25a, CURRENT_ROOM 476, DECR_GROUP_COUNT 481, GET_CHAR_RECORD 26, GET_MOB_DATA 602, INCR_GROUP_COUNT 481, LEVEL_MASK 411a, MFIELD_LEVEL 561, MFIELD_NEXT 561, MOB_DATA_PTR 600b, MOVW 9a, PRNG_OUTPUT 593a, REORDER_LINK 477a, REORDER_SORT_KEY 477b, SOURCE_CHAR 492b, STEP_PRNG 595, TARGET_REC 618b, and TOTAL_MOB_COUNT 591b.

The attract screen starts with scene index 1 (the first story character setup) and begins playing scene $26.

Characters are organized into groups via linked lists threaded through byte 2 of each 16-byte mob data record (accessed via GET_MOB_DATA). Byte 8 of the *group head*'s mob data record is a packed field:

- bits 3–0: **group member count** — how many characters are in this group's chain (0–4 in initial data)
- bits 6–4: **pending XP** — experience points accumulated but not yet applied to the character's level (0–7). The XP award routine at $7248 writes new values here; the flush routine at $6EE9 adds pending XP to byte 11's level field and clears it.
- bit 7: **status flag** — always zero in initial data; set during gameplay (controls message selection at $18AB). Preserved across XP flush.

These two entry points adjust the group member count (low nibble of byte 8) on the head record pointed to by \CHAR_REC/\CHAR_REC+1:

- DECR_GROUP_COUNT (DECR_GROUP_COUNT): decrements by 1 (source group lost a member)
- INCR_GROUP_COUNT (INCR_GROUP_COUNT): increments by 1 (destination group gained a member)

Guards: bails out if \CHAR_REC is zero (null pointer) or if byte 4 of the record at TARGET_REC is $\geq$ $15 (inactive/dead character). Called exclusively from REORDER_CHAR.

GROUP_COUNT_DELTA holds +1 or −1, set by the two entry points INCR_GROUP_COUNT and DECR_GROUP_COUNT respectively. The shared body adds this delta to the low nibble of byte 8 of the group head record.

480 ⟨*var group count delta* 480⟩≡ (909)

```
    ORG     $5A2C
GROUP_COUNT_DELTA:
    DC.B    $A3                     ; initial value (runtime: +1 or -1 delta for group count adjustm
```

Defines:
GROUP_COUNT_DELTA, used in chunk 481.

481 ⟨*adjust char byte8* 481⟩≡ (909)

```
    ORG     $102F
DECR_GROUP_COUNT:
    SUBROUTINE DECR_GROUP_COUNT
    ; Decrement or increment the group count byte of a mob data record.
    ;
    ; Inputs:
    ;   DATA_PTR    | pointer to mob data record ($BE/$BF); null = no-op
    ;   TARGET_REC  | pointer to character record ($BC/$BD); byte 4 (appearance)
    ;                 must be < NPC_APPEARANCE_MIN or call is a no-op
    ;
    ; Behavior:
    ;   Stores -1 or +1 in GROUP_COUNT_DELTA, then bails if DATA_PTR is null
    ;   or if the character's appearance index indicates an inactive NPC.
    ;   Otherwise adds the delta to byte 8 of the mob data record at DATA_PTR,
    ;   which encodes group count (bits 3-0), pending XP (bits 6-4), and status
    ;   (bit 7) in a packed byte.
    ;
    ; Modifies:
    ;   GROUP_COUNT_DELTA, mob data byte 8 at (DATA_PTR + MFIELD_XP)
    ; Clobbers: A, Y

    LDA     #$FF              ; delta = -1
    BNE     .apply            ; always taken
INCR_GROUP_COUNT:
    LDA     #$01              ; delta = +1
.apply:
    STA     GROUP_COUNT_DELTA ; save delta for later
    LDA     DATA_PTR          ; \
    BANE    #$00,.not_null
    RTS                       ; /
.not_null:
    LDY     #CFIELD_APPEARANCE ; \
    LDA     (TARGET_REC),Y    ;  | bail if appearance >= $15 (inactive/dead)
    BALT    #NPC_APPEARANCE_MIN,.ok
    RTS                       ; /
.ok:
    LDY     #MFIELD_XP        ; \
    LDA     (DATA_PTR),Y      ;  | group count (low nibble) += delta
    CLC                       ;  |
    ADC     GROUP_COUNT_DELTA ;  |
    STA     (DATA_PTR),Y      ; /
    RTS
```

Defines:
DECR_GROUP_COUNT, used in chunk 478.
INCR_GROUP_COUNT, used in chunk 478.

Uses CFIELD_APPEARANCE 561, DATA_PTR 618a, GROUP_COUNT_DELTA 480, MFIELD_XP 561, and TARGET_REC 618b.

Group iteration uses two zero-page pointer pairs: CHAR_PTR points to the group head (or the member currently being processed), and MOB_PTR is the iteration cursor that advances through the chain. Both routines follow the "next" link in byte 2 of a mob data record, resolve it via GET_MOB_DATA, and store the result in MOB_PTR. If the link is zero (end of chain), MOB_PTR is set to null.

- FIRST_GROUP_MEMBER (FIRST_GROUP_MEMBER): begins iteration by following byte 2 of the group head at CHAR_PTR. If that link is zero, falls through to check MOB_PTR as a fallback.
- NEXT_GROUP_MEMBER (NEXT_GROUP_MEMBER): continues iteration by following byte 2 of the current cursor at MOB_PTR.

482 ⟨*advance group link* 482⟩≡ (909)

```
    ORG     $12FA
FIRST_GROUP_MEMBER:
    SUBROUTINE FIRST_GROUP_MEMBER
    ; Advance group-member cursor to the first or next mob in a linked chain.
    ;
    ; Inputs:
    ;   CHAR_PTR  | points to the group-head character record ($FA/$FB);
    ;               byte 2 (CFIELD_NEXT) holds the first mob link index
    ;   MOB_PTR   | points to the current mob record ($F4/$F5);
    ;               byte 2 (MFIELD_NEXT) holds the next mob link index
    ;
    ; Behavior:
    ;   FIRST_GROUP_MEMBER reads the next-link from byte 2 of the group head
    ;   (CHAR_PTR).  NEXT_GROUP_MEMBER reads it from byte 2 of the current
    ;   mob cursor (MOB_PTR).  If the link index is non-zero, GET_MOB_DATA
    ;   resolves it to a 16-byte record pointer and advances MOB_PTR.  If the
    ;   link index is zero the chain is exhausted and MOB_PTR is set to null.
    ;
    ; Outputs:
    ;   MOB_PTR  | updated to point to the next mob record, or $0000 at
    ;              end of chain
    ;
    ; Modifies:
    ;   MOB_PTR, MOB_DATA_PTR
    ; Clobbers: A, Y

    LDY     #CFIELD_NEXT    ; \
    LDA     (CHAR_PTR),Y    ;  | check byte 2 of group head ($FA)
    BNE     .resolve        ; /  if nonzero, follow it
NEXT_GROUP_MEMBER:
    LDY     #MFIELD_NEXT    ; \
    LDA     (MOB_PTR),Y     ;  | check byte 2 of cursor ($F4)
    BNE     .resolve        ; /  if nonzero, follow it
    STA     MOB_PTR         ; \  end of chain:
```

```
    STA     MOB_PTR+1       ; /  null out MOB_PTR/MOB_PTR+1
    RTS
.resolve:
    JSR     GET_MOB_DATA    ; resolve link -> MOB_DATA_PTR/MOB_DATA_PTR+1
    MOVW    MOB_DATA_PTR,MOB_PTR ;  MOB_PTR/MOB_PTR+1 = MOB_DATA_PTR/MOB_DATA_PTR+1
    RTS
```

Defines:
FIRST_GROUP_MEMBER, used in chunks 357, 368, 373, 385, 542, and 898.
NEXT_GROUP_MEMBER, used in chunks 357, 368, 373, 385, 542, and 898.
Uses CFIELD_NEXT 561, CHAR_PTR 591a, GET_MOB_DATA 602, MFIELD_NEXT 561, MOB_DATA_PTR 600b, MOB_PTR 608, and MOVW 9a.

PRINT_ENTITY_NAME positions the cursor at row 21, loads the name string pointer from bytes 0–1 of the record at EFFECT_REC, prints it centered at the bottom of the screen via PRINT_BOTTOM_CENTERED, and resets the text window. This is the standard way to identify which character is being affected in damage, death, and status messages.

483a ⟨*zero page defines* 25a⟩+≡ (909) ◁458 541b▷

```
EFFECT_REC              EQU     $F6     ; target entity record for damage/stats (2 bytes)
```

Defines:
EFFECT_REC, used in chunks 349, 352, 372a, 380, 382, and 483b.

483b ⟨*print entity name* 483b⟩≡ (909)

```
    ORG     $1142
PRINT_ENTITY_NAME:
    SUBROUTINE PRINT_ENTITY_NAME
    ; Print the target entity's name centered on the status line.
    ;
    ; Inputs:
    ;   EFFECT_REC  | pointer to target entity record; bytes 0-1 hold name string pointer
    ;
    ; Behavior:
    ;   Positions the cursor on row 21, loads the name string pointer from
    ;   bytes 0-1 of the EFFECT_REC record into PRINT_STRING_ADDR, prints
    ;   the name centered on the bottom status line, then resets the text
    ;   window via SCROLL_STATUS_LINE.
    ;
    ; Modifies:
    ;   PRINT_STRING_ADDR
    ; Clobbers: A, Y

    JSR     SET_CURSOR_ROW21
    LDWIY    EFFECT_REC,#$00,PRINT_STRING_ADDR ; name pointer low
    JSR     PRINT_BOTTOM_CENTERED
    JMP     SCROLL_STATUS_LINE
```

Defines:
PRINT_ENTITY_NAME, used in chunk 349.
Uses EFFECT_REC 483a, LDWIY 17, PRINT_BOTTOM_CENTERED 628, PRINT_STRING_ADDR 576c, SCROLL_STATUS_LINE 639, and SET_CURSOR_ROW21 627.

Loads the name pointer from bytes 0–1 of the mob record at ENTITY_PTR into \PRINT_STRING_ADDR/\PRINT_STRING_ADDR+1, then tail-calls the text centering routine at $7705, which centers and prints the name on screen row 23. Used by 13 callers throughout the combat and interaction code.

484 ⟨*print mob name* 484⟩≡ (909)

```
    ORG     $0D8A
PRINT_MOB_NAME:
    SUBROUTINE PRINT_MOB_NAME
    ; Print the current mob or character's name centered on the status line.
    ;
    ; Inputs:
    ;   ENTITY_PTR  | pointer to mob/character record; bytes 0-1 hold name string pointer
    ;
    ; Behavior:
    ;   Loads the name string pointer from bytes 0-1 of the ENTITY_PTR record
    ;   into PRINT_STRING_ADDR, then jumps to PRINT_BOTTOM_CENTERED to print
    ;   the name centered on row 23.
    ;
    ; Modifies:
    ;   PRINT_STRING_ADDR
    ; Clobbers: A, Y

    LDWIY    ENTITY_PTR,#$00,PRINT_STRING_ADDR ;  PRINT_STRING_ADDR/PRINT_STRING_ADDR+1 = bytes 0-1
    JMP     PRINT_BOTTOM_CENTERED ; center and print on row 23
```

Defines:
PRINT_MOB_NAME, used in chunks 224, 338, 342, 346, 357, 360, 363, 377, 378, 456, and 898.
Uses ENTITY_PTR 561, LDWIY 17, PRINT_BOTTOM_CENTERED 628, and PRINT_STRING_ADDR 576c.

PRINT_MEMBER_NAME is a thin wrapper that copies the first two bytes of the record at MOB_PTR—which hold a pointer to the member's name string—into PRINT_STRING_ADDR, then tail-calls PRINT_BOTTOM_CENTERED to display the name centered on the status line.

485 ⟨*print member name* 485⟩≡ (909)

```
      ORG     $1406
  PRINT_MEMBER_NAME:
      SUBROUTINE PRINT_MEMBER_NAME
      ; Print the current group member's name centered on the status line.
      ;
      ; Inputs:
      ;   MOB_PTR  | pointer to group member record; bytes 0-1 hold name string pointer
      ;
      ; Behavior:
      ;   Loads the name string pointer from bytes 0-1 of the MOB_PTR record
      ;   into PRINT_STRING_ADDR, then jumps to PRINT_BOTTOM_CENTERED to print
      ;   the name centered on row 23.
      ;
      ; Modifies:
      ;   PRINT_STRING_ADDR
      ; Clobbers: A, Y

      LDWIY    MOB_PTR,#$00,PRINT_STRING_ADDR
      JMP     PRINT_BOTTOM_CENTERED
```

Defines:
PRINT_MEMBER_NAME, used in chunks 338, 346, 385, and 542.
Uses LDWIY 17, MOB_PTR 608, PRINT_BOTTOM_CENTERED 628, and PRINT_STRING_ADDR 576c.

LOAD_CHAR_NAME_PTR loads the name string pointer from the first two bytes of the entity record at ENTITY_PTR into PRINT_STRING_ADDR, then tail-calls SETUP_TEXT_POS_COL20 to position the text cursor in column 20 (the right-side status panel). This prepares the display for printing the current character's name during the status display.

486a ⟨*load char name ptr* 486a⟩≡ (909)

```
      ORG     $0925
  LOAD_CHAR_NAME_PTR:
      SUBROUTINE LOAD_CHAR_NAME_PTR
      ; Load entity name pointer and position cursor at the right status panel.
      ;
      ; Inputs:
      ;   ENTITY_PTR  | pointer to current entity record (bytes 0-1 = name string address)
      ;
      ; Behavior:
      ;   Reads the two-byte name pointer from bytes 0-1 of the entity record
      ;   and stores it in PRINT_STRING_ADDR.  Jumps to SETUP_TEXT_POS_COL20
      ;   to position the text cursor at column 20 (right status panel).
      ;
      ; Modifies:
      ;   PRINT_STRING_ADDR
      LDWIY    ENTITY_PTR,#$00,PRINT_STRING_ADDR
      JMP     SETUP_TEXT_POS_COL20
```

Defines:
LOAD_CHAR_NAME_PTR, used in chunks 404 and 532c.
Uses ENTITY_PTR 561, LDWIY 17, PRINT_STRING_ADDR 576c, and SETUP_TEXT_POS_COL20 552.

SETUP_CHAR_SPRITE reads the current entity's packed position from byte 3 of ENTITY_PTR, unpacks it to column and row, and stores the coordinates in both the font parameters (FONT_COL, FONT_ROW) and the blink-cursor parameters (BLINK_COL, BLINK_ROW). It then loads the character's sprite index from CHAR_SPRITE, sets FONT_CHARSET to 1 (custom font), and tail-calls RENDER_FONT_CHAR to draw the sprite at the entity's position. This is the standard way to draw or redraw a character's map icon.

486b ⟨*var char sprite* 486b⟩≡ (909)

```
      ORG     $5A04
  CHAR_SPRITE:
      DC.B    $CA                     ; initial value (runtime: character's font char number for displ
```

Defines:
CHAR_SPRITE, used in chunks 225, 239, and 487.

487 ⟨*setup char sprite* 487⟩≡ (909)

```
      ORG     $0933
  SETUP_CHAR_SPRITE:
      SUBROUTINE SETUP_CHAR_SPRITE
      ; Unpack entity position and render its sprite glyph with blink cursor.
      ;
      ; Inputs:
      ;   ENTITY_PTR   | pointer to current entity record (byte 3 = packed position)
      ;   CHAR_SPRITE  | glyph index to render for this entity
      ;
      ; Behavior:
      ;   Reads the packed map position from entity byte MFIELD_POSITION and
      ;   splits it into column and row via POS_TO_COLROW.  Stores in both
      ;   font cursor and blink cursor variables.  Loads CHAR_SPRITE as the
      ;   glyph, selects custom font charset 1, then tail-calls RENDER_FONT_CHAR.
      ;
      ; Modifies:
      ;   FONT_COL, FONT_ROW, FONT_CHARNUM, FONT_CHARSET,
      ;   BLINK_COL, BLINK_ROW, BLINK_CHAR
      ; Clobbers: A, X, Y
      LDY     #MFIELD_POSITION
      LDA     (ENTITY_PTR),Y
      JSR     POS_TO_COLROW
      STA     FONT_COL
      STY     FONT_ROW
      STA     BLINK_COL
      STY     BLINK_ROW
      TAX
      MOVB    CHAR_SPRITE,FONT_CHARNUM
      STA     BLINK_CHAR
      STOB    #$01,FONT_CHARSET
      JMP     RENDER_FONT_CHAR
```

Defines:
SETUP_CHAR_SPRITE, used in chunks 404 and 532c.

Uses BLINK_CHAR 544c, BLINK_COL 563b, BLINK_ROW 544b, CHAR_SPRITE 486b, ENTITY_PTR 561, FONT_CHARNUM 575, FONT_CHARSET 576a, FONT_COL 492a, FONT_ROW 534c, MFIELD_POSITION 561, MOVB 8c, POS_TO_COLROW 599, RENDER_FONT_CHAR 466, and STOB 8c.

LOAD_CHAR_TO_FA is a thin wrapper: it calls GET_CHAR_RECORD and copies the result from DATA_PTR to CHAR_PTR (CHAR_PTR). Several callers use this as a shortcut to both look up a character and set the main iteration pointer.

488 ⟨*load char to fa* 488⟩≡ (909)

```
      ORG     $6EC9
  LOAD_CHAR_TO_FA:
      SUBROUTINE LOAD_CHAR_TO_FA
      ; Load a character record and expose its pointer via CHAR_PTR.
      ;
      ; Inputs:
      ;   A  | character index
      ;
      ; Behavior:
      ;   Calls GET_CHAR_RECORD to resolve the index into a record pointer
      ;   in DATA_PTR, then copies it to CHAR_PTR so callers can iterate
      ;   the record via the main-loop zero-page pointer.
      ;
      ; Modifies:
      ;   CHAR_PTR
      ; Clobbers: A, X, Y
      ; Wrapper: GET_CHAR_RECORD then copy CHAR_REC -> $FA/$FB.
      JSR     GET_CHAR_RECORD
      MOVW    DATA_PTR,CHAR_PTR
      RTS
```

Defines:
LOAD_CHAR_TO_FA, used in chunk 431.
Uses CHAR_PTR 591a, CHAR_REC 25a, DATA_PTR 618a, GET_CHAR_RECORD 26, and MOVW 9a.

REORDER_CURRENT_CHAR is a convenience wrapper that copies ENTITY_PTR into TARGET_REC and CURRENT_PLAYER into SOURCE_CHAR, then tail-calls REORDER_CHAR to re-sort the current character within its group's linked list. This is used after stat changes that might affect the character's position in the sorted order.

489a ⟨*reorder current char* 489a⟩≡ (909)

```
      ORG     $1414
  REORDER_CURRENT_CHAR:
      SUBROUTINE REORDER_CURRENT_CHAR
      ; Re-sort the active character after a stat change.
      ;
      ; Inputs:
      ;   ENTITY_PTR      | pointer to the current character record
      ;   CURRENT_PLAYER  | index of the current player character
      ;
      ; Behavior:
      ;   Copies ENTITY_PTR into TARGET_REC and CURRENT_PLAYER into
      ;   SOURCE_CHAR, then tail-calls REORDER_CHAR to re-insert the
      ;   character into sorted order.
      ;
      ; Modifies:
      ;   TARGET_REC, SOURCE_CHAR
      ; Clobbers: A
      MOVW    ENTITY_PTR,TARGET_REC
      MOVB    CURRENT_PLAYER,SOURCE_CHAR
      JMP     REORDER_CHAR
```

Defines:
REORDER_CURRENT_CHAR, used in chunks 268 and 354.
Uses CURRENT_PLAYER 562a, ENTITY_PTR 561, MOVB 8c, MOVW 9a, REORDER_CHAR 478, SOURCE_CHAR 492b, and TARGET_REC 618b.

489b ⟨*press button string* 489b⟩≡ (909)

```
      ORG     $7EBA
  S_PRESS_BUTTON:
      ; Length $22 (34): CR + "  PRESS THE BUTTON TO CONTINUE  " + CLR
      DC.B    $22,$8C
      HEX     A0 A0 D0 D2 C5 D3 D3 A0 D4 C8 C5 A0 C2 D5 D4 D4
      HEX     CF CE A0 D4 CF A0 C3 CF CE D4 C9 CE D5 C5 A0 A0
      DC.B    $8B
```

Defines:
S_PRESS_BUTTON, used in chunk 550.

Byte	Meaning
$7D	Reset to full-screen mode (col=0, row=0, width=20)
$7E	Pause: run tune player and status bar, then resume
$7F	End of stream
$80–$FE	Set row (byte & $7F), next byte = column
Other	Character code for HRCG font

The text renderer data at STATUS_BORDER_DATA draws a decorative border for the status bar. Characters $33–$41 are box-drawing glyphs in the HRCG font.

490 ⟨*status border data* 490⟩≡ (909)

```
    ORG     $7E2F
STATUS_BORDER_DATA:
    HEX     7D                      ; $7D: reset to full-screen
    HEX     80 00                   ; row 0, col 0
    HEX     33 3C                   ; top-left corner, left tee
    HEX     34 34 34 34 34 34 34 34 ; horizontal bar (x16)
    HEX     34 34 34 34 34 34 34 34
    HEX     3D 35                   ; right tee, top-right corner
    HEX     3A 3F                   ; decorative
    HEX     81 12 3E 36 3A 3F       ; row 1, col 18, right edge
    HEX     82 12 3E 36 3A 3F       ; row 2
    HEX     83 12 3E 36 3A 3F       ; row 3
    HEX     84 12 3E 36 3A 3F       ; row 4
    HEX     85 12 3E 36 3A 3F       ; row 5
    HEX     86 12 3E 36 3A 3F       ; row 6
    HEX     87 12 3E 36 3A 3F       ; row 7
    HEX     88 12 3E 36 3A 3F       ; row 8
    HEX     89 12 3E 36 3A 3F       ; row 9
    HEX     8A 12 3E 36             ; row 10
    HEX     39 40                   ; bottom-left corner, left tee
    HEX     38 38 38 38 38 38 38 38 ; horizontal bar (x16)
    HEX     38 38 38 38 38 38 38 38
    HEX     41 37                   ; right tee, bottom-right corner
    HEX     7F                      ; end of stream
```

Defines:
STATUS_BORDER_DATA, used in chunks 550 and 555b.

Chapter 12

Map and Scene

12.0.1 Scene setup

SCENE_SETUP places a mob at a random position within the location's bounding box, updates the linked list, and draws the character if it is the current player.

1. Loads the record for ACTIVE_CHAR via GET_CHAR_RECORD.
2. Reads bytes 3–4 (LFIELD_MIN_POS, LFIELD_MAX_POS) and decodes each to column/row via POS_TO_COLROW.
3. Increments minimums by one (exclusive border).
4. Calls RANDOM_IN_RANGE twice to choose random column and row.
5. Stores the position in MFIELD_POSITION of the mob record via MOB_PTR, clears gold (MFIELD_GOLD/MFIELD_GOLD+1), and reorders the mob in the linked list.
6. If ACTIVE_CHAR equals CURRENT_PLAYER, draws the character.

LFIELD_MIN_POS and LFIELD_MAX_POS are location-record aliases for bytes 3 and 4. For character records these same offsets hold position and appearance; for location records they hold the bounding box corners.

491 ⟨*defines* 25b⟩+≡ (909) ◁468 526a▷

```
LFIELD_MIN_POS      EQU     $03     ; location record byte 3: top-left bounding position
LFIELD_MAX_POS      EQU     $04     ; location record byte 4: bottom-right bounding position
```

Defines:
LFIELD_MAX_POS, used in chunks 494 and 535.
LFIELD_MIN_POS, used in chunks 494 and 535.

492a ⟨*var font col* 492a⟩≡ (909)

```
    ORG     $5A0C
FONT_COL:
    DC.B    $CE                     ; initial value (runtime: font cursor column coordinate)
```

Defines:
FONT_COL, used in chunks 239, 342, 400, 466, 467, 471, 473b, 487, 494, 535, 538, 540, 542, 545, and 644.

492b ⟨*var source char* 492b⟩≡ (909)

```
    ORG     $5A28
SOURCE_CHAR:
    DC.B    $C1                     ; initial value (runtime: source character for reorder)
```

Defines:
SOURCE_CHAR, used in chunks 349, 419, 420, 431, 478, 489a, and 494.

492c ⟨*var active char* 492c⟩≡ (909)

```
    ORG     $5A29
ACTIVE_CHAR:
    DC.B    $CE                     ; initial value (runtime: character index being processed)
```

Defines:
ACTIVE_CHAR, used in chunks 220, 268, 283, 349, 354, 419, 420, 424, 429, 431, 478, and 494.

492d ⟨*var bounds min col* 492d⟩≡ (909)

```
    ORG     $5A51
BOUNDS_MIN_COL:
    DC.B    $A0                     ; initial value (runtime: scene setup: left column boundary)
```

Defines:
BOUNDS_MIN_COL, used in chunk 494.

492e ⟨*var bounds max col* 492e⟩≡ (909)

```
    ORG     $5A52
BOUNDS_MAX_COL:
    DC.B    $CC                     ; initial value (runtime: scene setup: right column boundary)
```

Defines:
BOUNDS_MAX_COL, used in chunk 494.

492f ⟨*var bounds min row* 492f⟩≡ (909)

```
    ORG     $5A53
BOUNDS_MIN_ROW:
    DC.B    $C4                     ; initial value (runtime: scene setup: top row boundary)
```

Defines:
BOUNDS_MIN_ROW, used in chunk 494.

493 ⟨*var bounds max row* 493⟩≡ (909)

```
    ORG     $5A54
BOUNDS_MAX_ROW:
    DC.B    $C1                     ; initial value (runtime: scene setup: bottom row boundary)
```

Defines:
BOUNDS_MAX_ROW, used in chunk 494.

494 ⟨*scene setup* 494⟩≡ (909)

```
        ORG     $0A17
    SCENE_SETUP:
        SUBROUTINE SCENE_SETUP
        ; Place a mob at a random position within the location's bounding box.
        ;
        ; Inputs:  ACTIVE_CHAR | character/scene index, MOB_PTR | mob record
        ; Behavior:
        ;   Reads bounding box from location record, picks random position,
        ;   stores into mob record, reorders linked list, clears gold.
        ;   If active char is current player, draws the character.
        ; Clobbers: A, X, Y

        LDA     ACTIVE_CHAR                 ; scene/character index
        JSR     GET_CHAR_RECORD             ; -> CHAR_REC = 9-byte record
        LDY     #LFIELD_MIN_POS             ; byte 3: top-left bounding position
        LDA     (CHAR_REC),Y
        JSR     POS_TO_COLROW               ; -> A = min_col, Y = min_row
        STA     BOUNDS_MIN_COL
        STY     BOUNDS_MIN_ROW
        LDY     #LFIELD_MAX_POS             ; byte 4: bottom-right bounding position
        LDA     (CHAR_REC),Y
        JSR     POS_TO_COLROW               ; -> A = max_col, Y = max_row
        STA     BOUNDS_MAX_COL
        STY     BOUNDS_MAX_ROW
        INC     BOUNDS_MIN_COL              ; exclusive border: min_col++
        INC     BOUNDS_MIN_ROW              ; exclusive border: min_row++
        CLC
        LDA     BOUNDS_MAX_COL              ; \
        SBC     BOUNDS_MIN_COL              ;  | col_range = max - min
        JSR     RANDOM_IN_RANGE             ;  | random offset in [0, range)
        CLC                                 ;  |
        ADC     BOUNDS_MIN_COL              ;  | col = min + offset
        STA     FONT_COL                    ; / save for COLROW_TO_POS
        CLC
        LDA     BOUNDS_MAX_ROW              ; \
        SBC     BOUNDS_MIN_ROW              ;  | row_range = max - min
        JSR     RANDOM_IN_RANGE             ;  | random offset
        CLC                                 ;  |
        ADC     BOUNDS_MIN_ROW              ; / row = min + offset
        TAY                                 ; Y = row
        LDA     FONT_COL                    ; A = col
        JSR     COLROW_TO_POS               ; -> A = linear map position
        LDY     #MFIELD_POSITION            ; byte 3: mob map position
        STA     (MOB_PTR),Y                 ; store new position
        MOVW    MOB_PTR,TARGET_REC          ; TARGET_REC = MOB_PTR
        STOB    #$00,SOURCE_CHAR            ; no source character
        JSR     REORDER_CHAR                ; splice into linked list
        LDY     #MFIELD_GOLD              ; byte 13: gold low byte
        LDA     #$00                        ; \
```

```
    STA     (MOB_PTR),Y             ;  | clear gold counter
    INY                             ;  | Y = MFIELD_GOLD+1
    STA     (MOB_PTR),Y             ; /
    JSR     CLAMP_CHAR_FIELD        ; enforce byte 6 >= byte 5
    LDA     CURRENT_PLAYER          ; current player index
    BANE    ACTIVE_CHAR,.done       ; not current player -> skip draw
    LDY     #MFIELD_POSITION        ; byte 3: mob map position
    LDA     (MOB_PTR),Y             ; reload position
    JSR     DRAW_CHAR_AT_POS        ; render character at position
.done:
    RTS
```

Defines:
SCENE_SETUP, used in chunks 220, 419, and 617.

Uses ACTIVE_CHAR 492c, BOUNDS_MAX_COL 492e, BOUNDS_MAX_ROW 493, BOUNDS_MIN_COL 492d, BOUNDS_MIN_ROW 492f, CHAR_REC 25a, CLAMP_CHAR_FIELD 609, COLROW_TO_POS 600a, CURRENT_PLAYER 562a, DRAW_CHAR_AT_POS 471, FONT_COL 492a, GET_CHAR_RECORD 26, LFIELD_MAX_POS 491, LFIELD_MIN_POS 491, MFIELD_GOLD 561, MFIELD_POSITION 561, MOB_PTR 608, MOVW 9a, POS_TO_COLROW 599, RANDOM_IN_RANGE 597, REORDER_CHAR 478, SOURCE_CHAR 492b, STOB 8c, and TARGET_REC 618b.

12.0.2 Scene loop

SCENE_LOOP is the main scene display loop, called with the scene number in A. It loops until the user presses a key or all scene transitions complete.

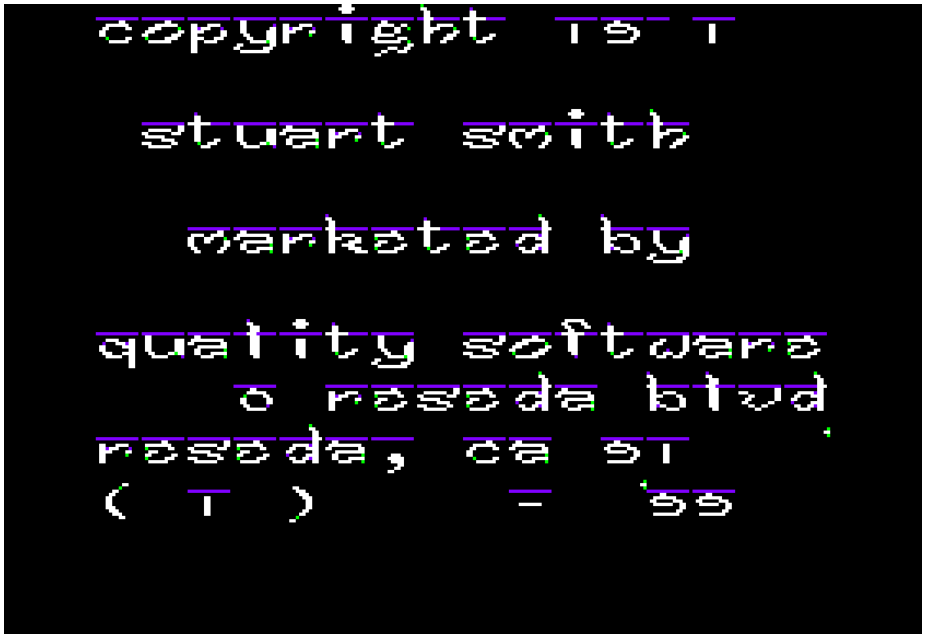
1. Disables the blink cursor and saves zero-page pointers.
2. Tests PRNG_OUTPUT bits 1 and 3; if any set (∼75%), skips LOAD_SCENE_DATA.
3. Calls LOAD_SCENE_FROM_DISK to load scene text from disk; carry set means the disk read failed.
4. Renders the status bar and scene text via TEXT_RENDERER.
5. Plays the attract tune (TUNE_ATTRACT, index 6).
6. If SCENE_TRANSITION is nonzero, uses it as a new scene number and loops back.
7. On exit, clears LOCATION_POS/LOCATION_STYLE, calls ENTER_LOCATION, and restores zero-page pointers.

Scene numbers are indices into on-disk text sectors loaded by LOAD_SCENE_FROM_DISK. The following table lists all known scene numbers and their contexts:

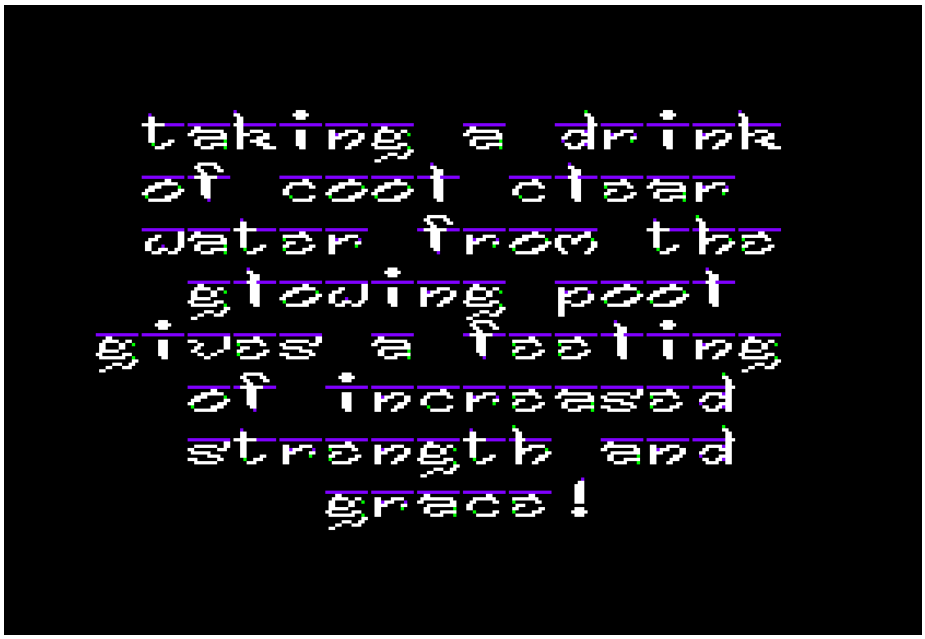
Scene	Hex	Text / Context
0	$00	"COPYRIGHT 1981 STUART SMITH ..."
1	$01	(empty)
2	$02	"TAKING A DRINK OF COOL CLEAR WATER ..."
3	$03	"THE OAK DOOR VIBRATES WITH A HOLLOW BOOM ..."
4	$04	"AND TO THINK THAT YOU HAVE DONE IT ALL ALONE ..."
5	$05	"THE SULTAN RUSHES OVER TO YOUR SIDE ..."
6	$06	"YOU BASH THE STATUE INTO SMITHEREENS ..."
7	$07	"YOUR WEAPON IS IMPALED IN YOUR VICTIM ..."
8	$08	(empty)
9	$09	"THE MAGIC POOL OF HEALING SPIRITS DRIES UP"
10	$0A	"AHA! AN ILLUSION! ...DISSOLVES INTO MIST"
11	$0B	"YOU HAVE NEVER HARMED A LIVING THING ..."
12	$0C	"PRAISE BE TO ALLAH! YOU HAVE RESCUED ..."
16	$10	"HE ALSO AWARDS YOU WITH GREAT WEALTH ..."
18	$12	"THE SULTAN SCURRIES BACK TO HIS THRONE"
25	$19	"THE STATUE TOPPLES OVER AND ..."
29	$1D	"THANK YOU FOR FIGHTING TO MAKE THIS ..."
35	$23	"I GRIEVE OVER YOUR UNTIMELY DEMISE ..." (retirement)
41	$29	"A RETIRED ADVENTURER IS SELLING ..."
43	$2B	"THE ASTROLOGER OF SHAHRIAR ..." (hint text)
48	$30	"A BLACKSMITH OFFERS YOU SOME FINE ..."
50	$32	"ERROR 47 FORTY-SEVEN" (weapon shop)
56	$38	"STRONG FIGHTERS SUCH AS DWARVES ..." (hint)
57	$39	"ABSORBED IN THE TREASURE CHEST ..." (trap: theft)
58	$3A	"A DARK GREEN VAPOR RISES FROM THE ..." (trap: gas)
63	$3F	"DWARVES ARE STRONG, SLOW AND CLUMSY ..." (hint)
64	$40	"DWARVES AND HUMANS GENERALLY DO MORE ..." (hint)
67	$43	"PICKING UP A BEAUTIFUL RUBY RING ..."
68	$44	"A LANDSLIDE BEHIND YOU OBLITERATES ..."
69	$45	"AN ORACULAR RUNE REVEALS ..." (hint)

Scenes 1, 8, 13–15, 19, 24, 26, 33–34, 36–38, and others are empty (all zeros) and serve as placeholders or are loaded conditionally. Scene data is stored on disk starting at track $11, sector 0; scene N maps to track $\$11 + N/13$, sector N mod 13. The byte encoding uses $00=space, $01–$1A=A–Z, 1B$–$24=0–9, with control codes for positioning ($80+) and flow ($7E=pause, 7F$=end).

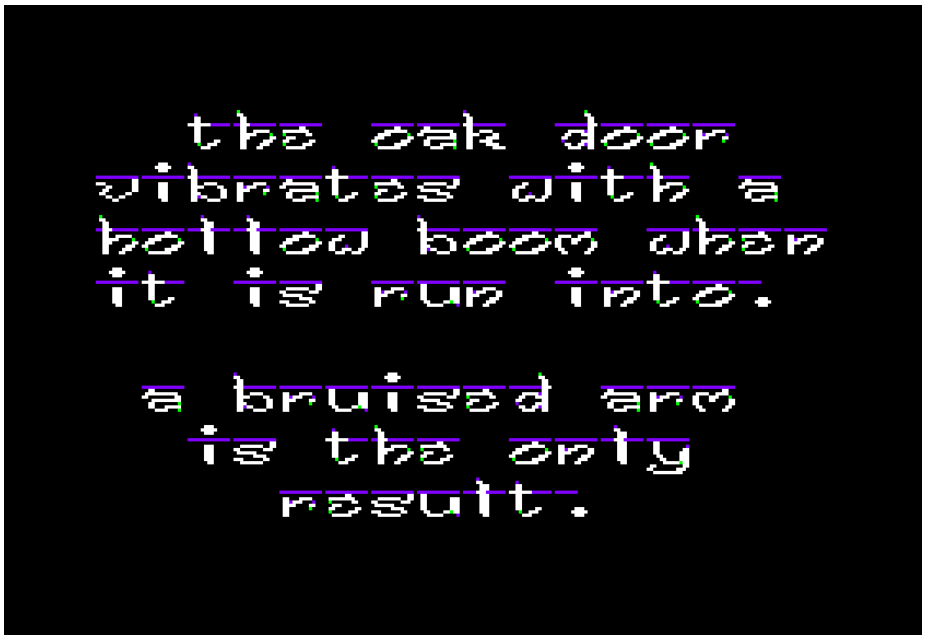
Scene $00 (0): COPYRIGHT 1981 STUART SMITH



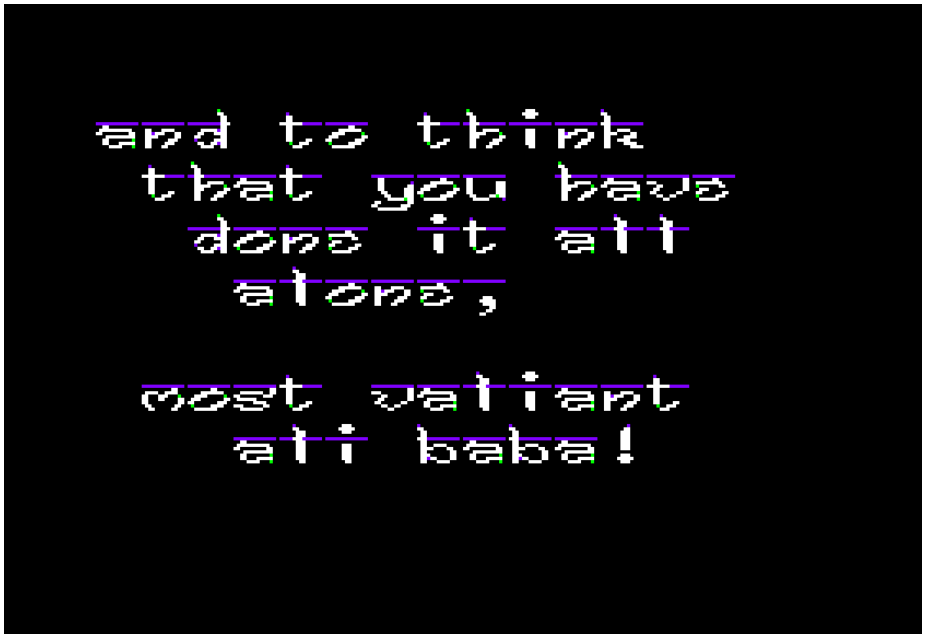
Scene $02 (2): TAKING A DRINK OF COOL CLEAR WATER . . .

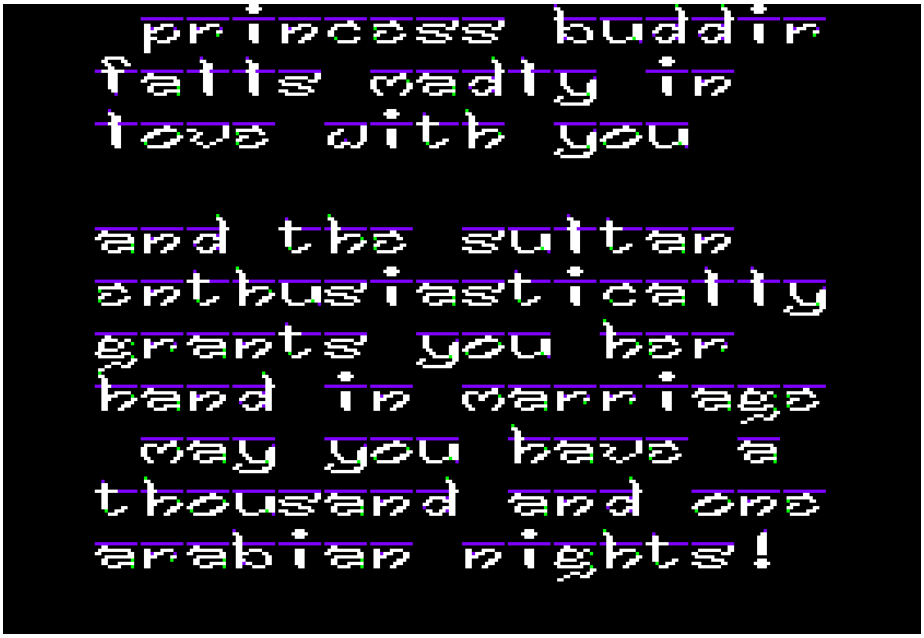


Scene $03 (3): THE OAK DOOR VIBRATES WITH A HOLLOW BOOM . . .

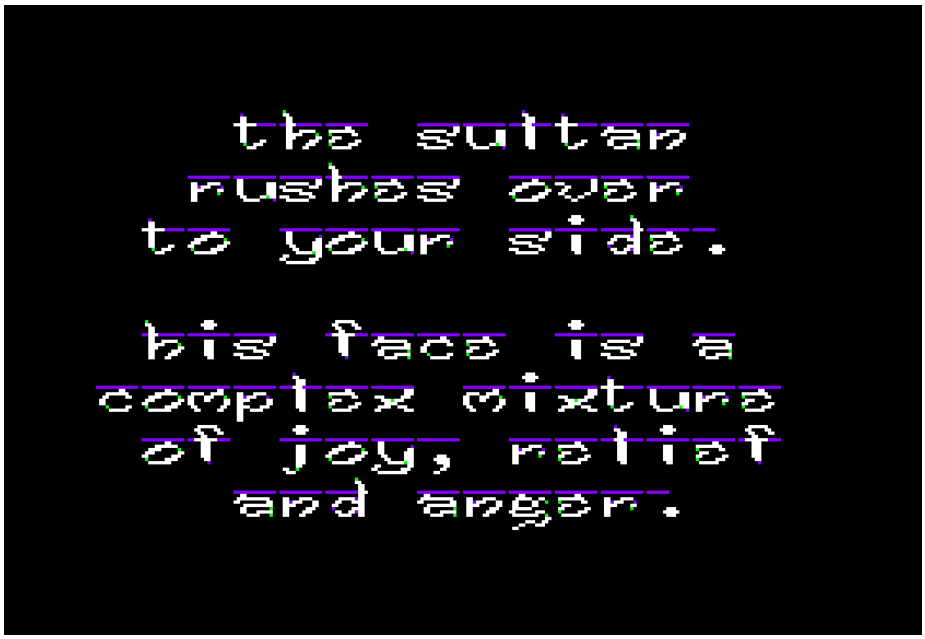


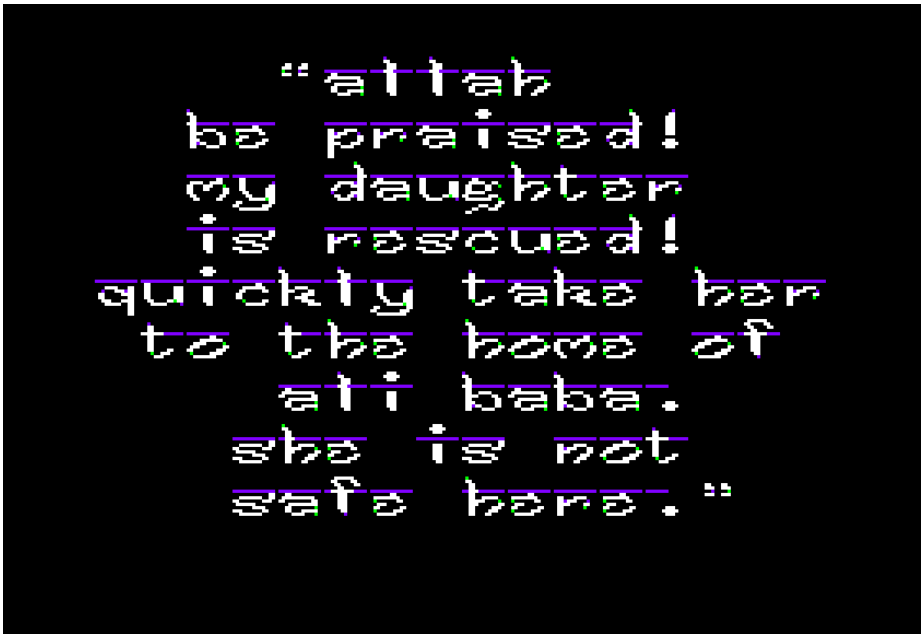
Scene $04 (4): AND TO THINK THAT YOU HAVE DONE IT ALL ALONE . . .



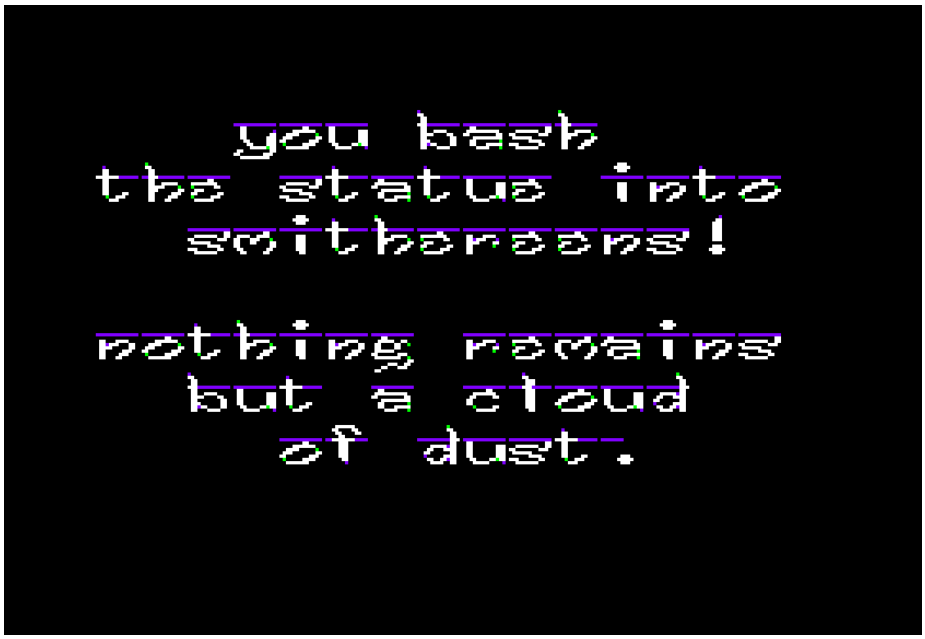


Scene $05 (5): THE SULTAN RUSHES OVER TO YOUR SIDE . . .

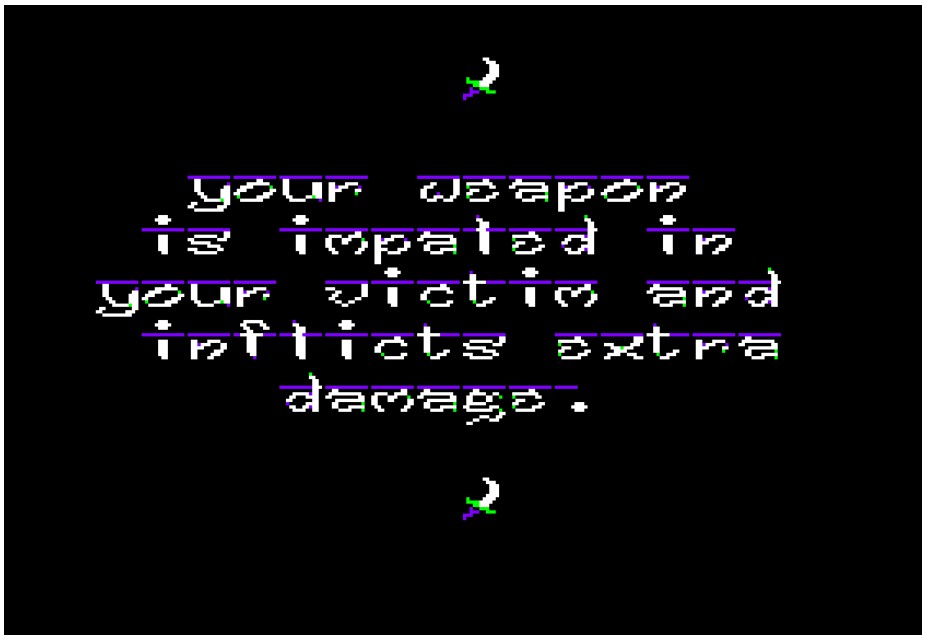


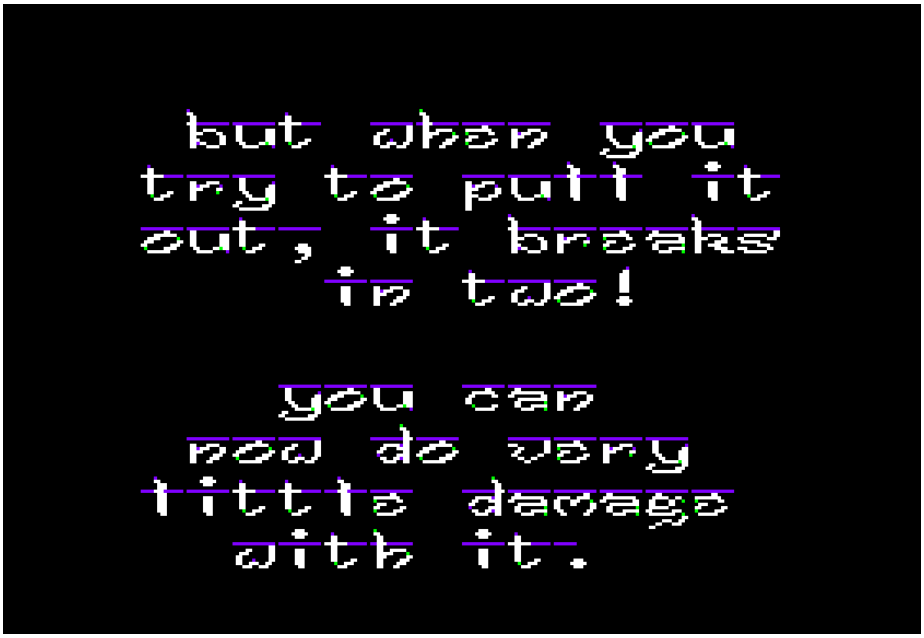


Scene $06 (6): YOU BASH THE STATUE INTO SMITHEREENS
...

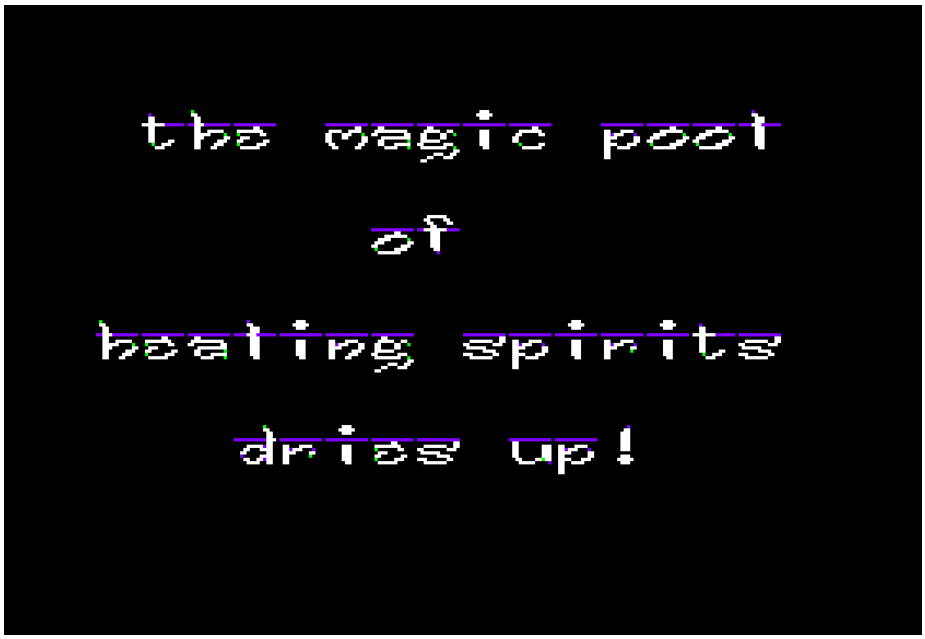


Scene $07 (7): YOUR WEAPON IS IMPALED IN YOUR VICTIM . . .

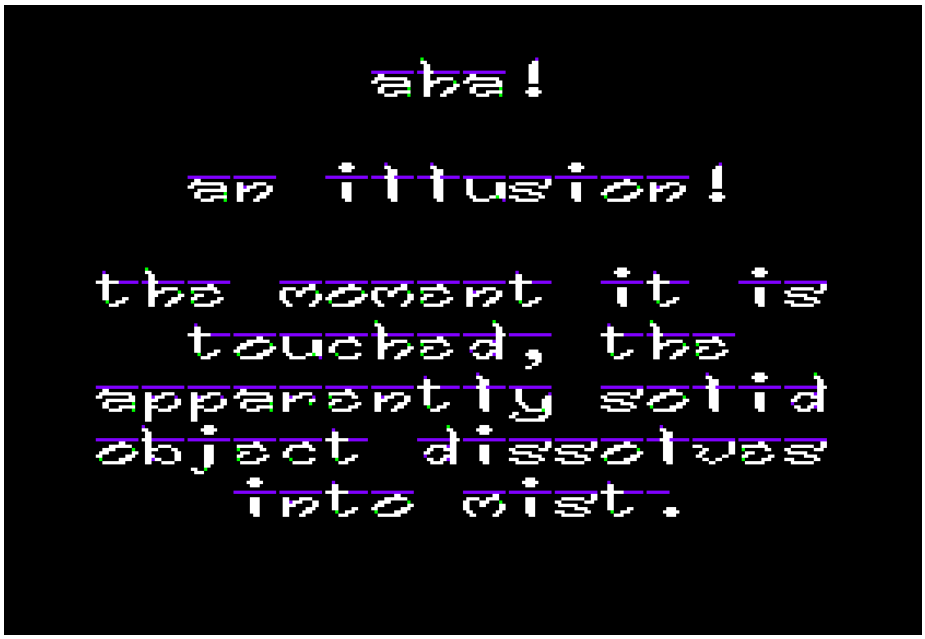




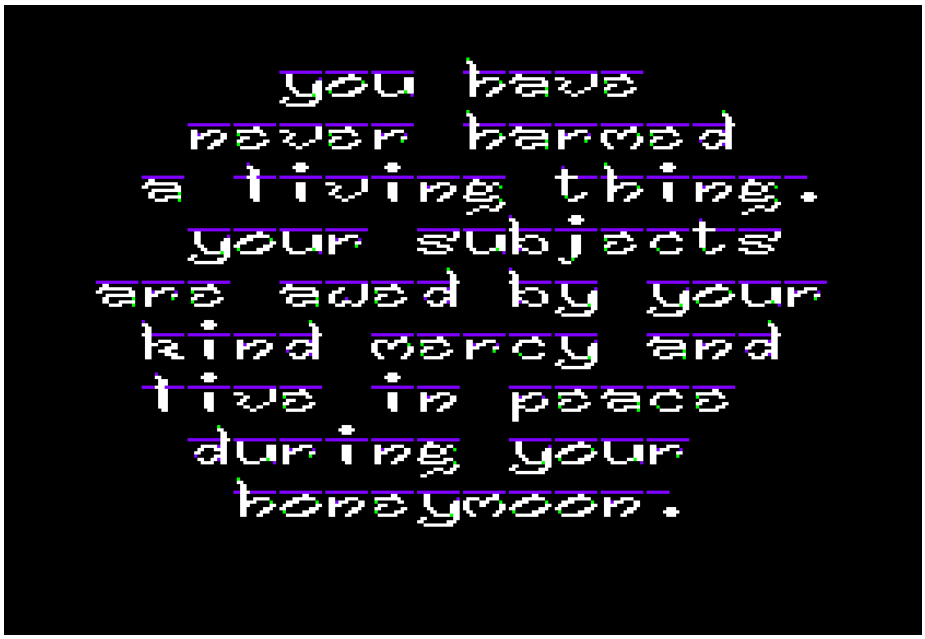
Scene $09 (9): THE MAGIC POOL OF HEALING SPIRITS DRIES UP

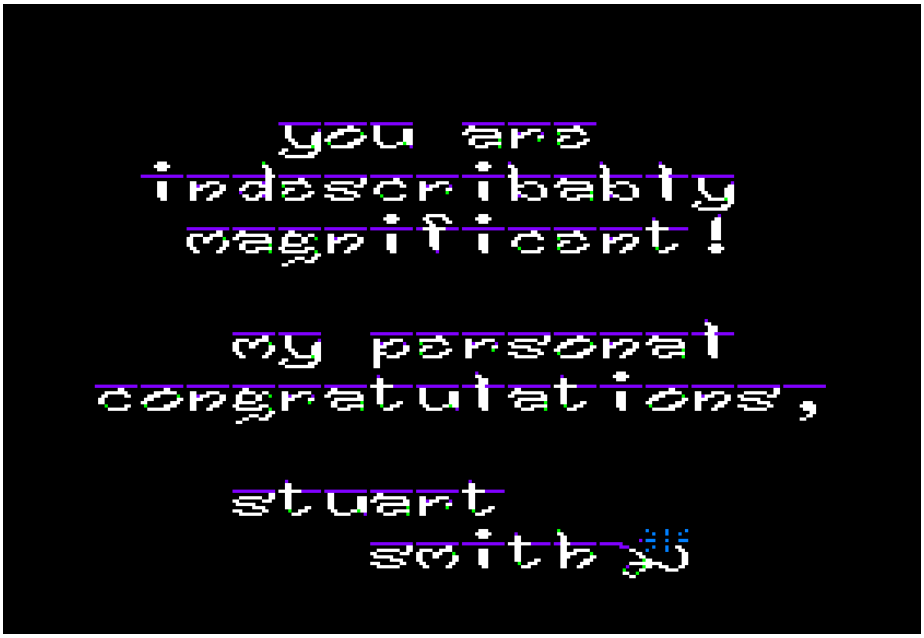


Scene $0A (10): AHA! AN ILLUSION! ... DISSOLVES INTO MIST

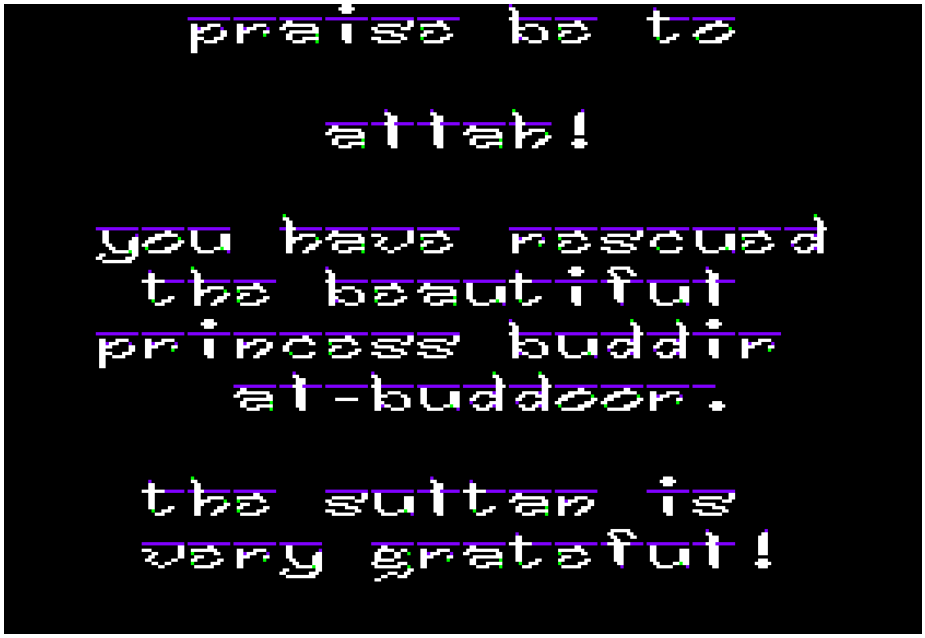


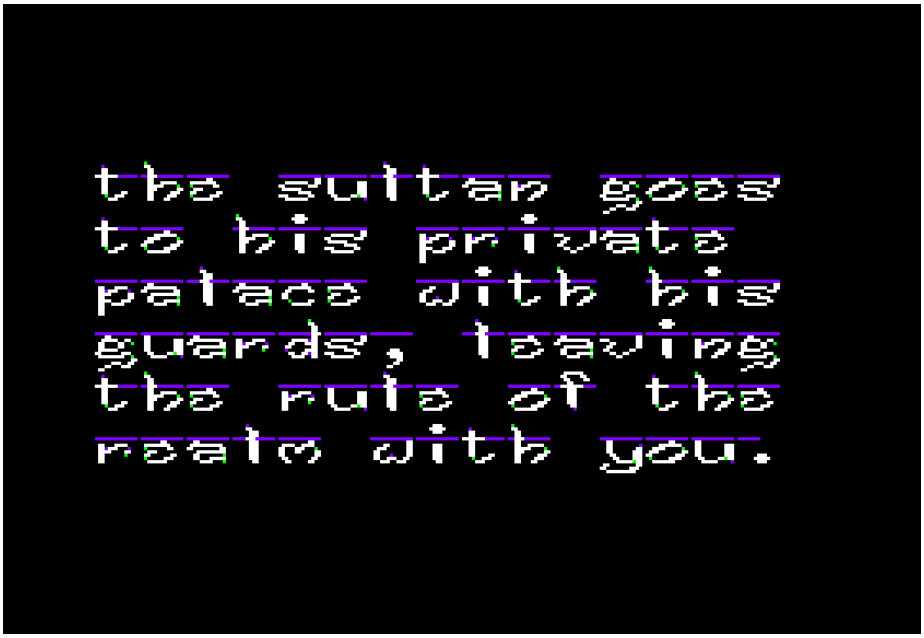
Scene $0B (11): YOU HAVE NEVER HARMED A LIVING THING ...



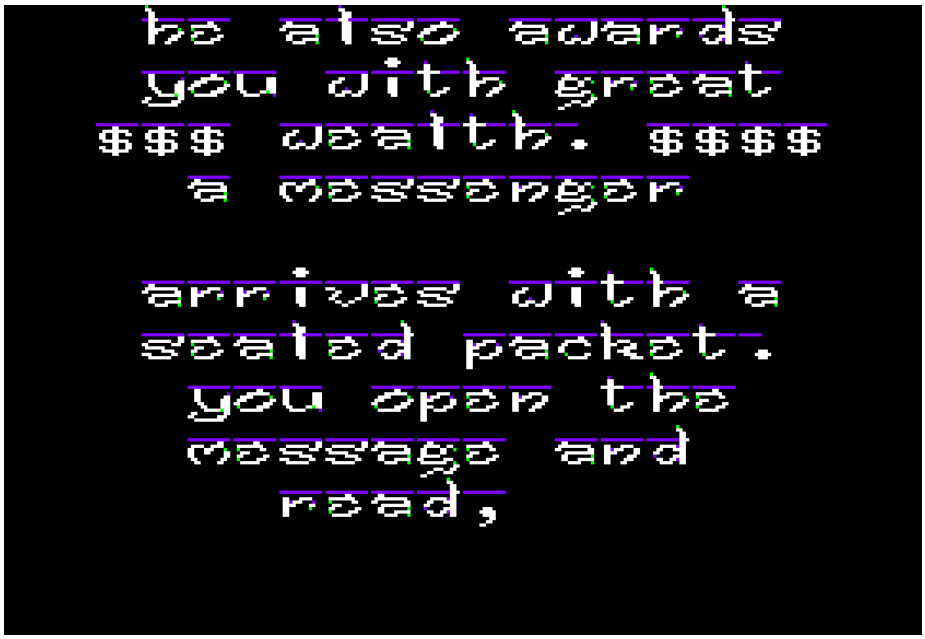


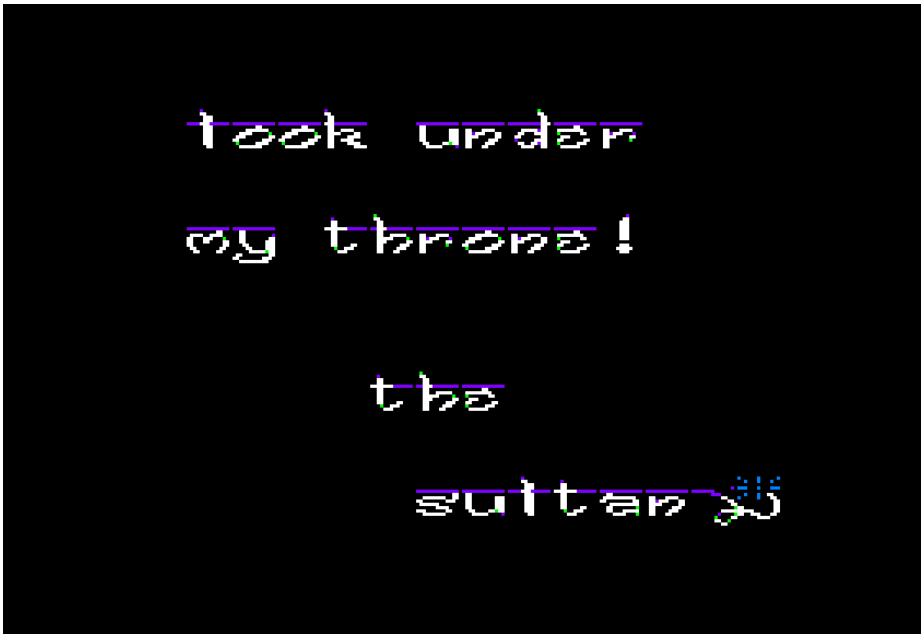
Scene $0C (12): PRAISE BE TO ALLAH! YOU HAVE RESCUED ...



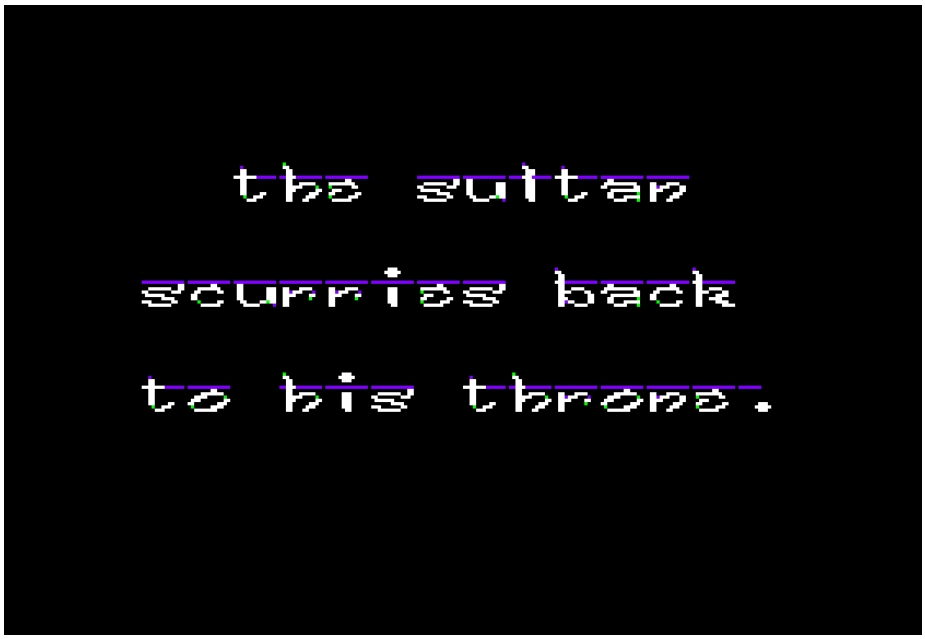


Scene $10 (16): HE ALSO AWARDS YOU WITH GREAT WEALTH . . .

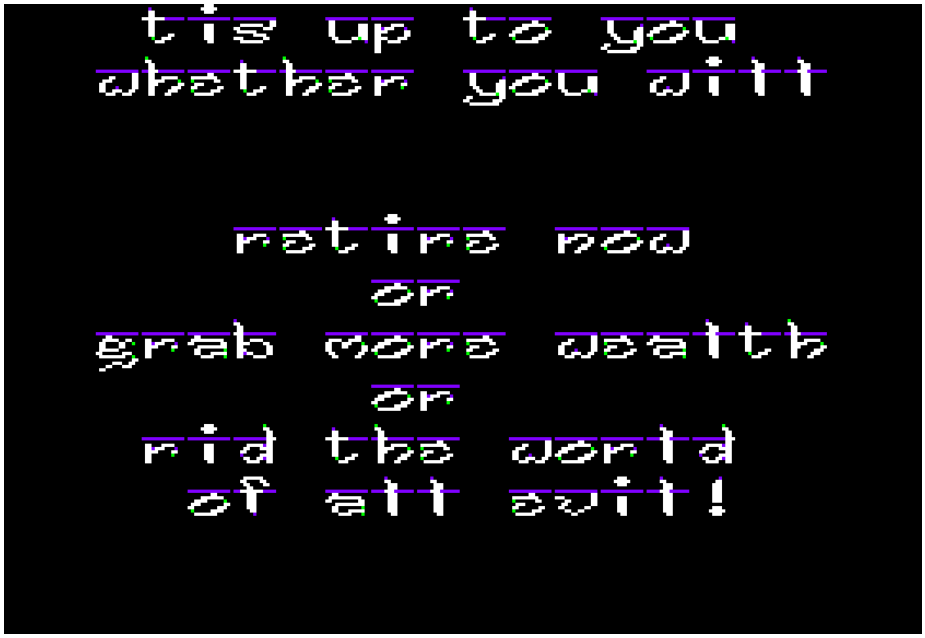




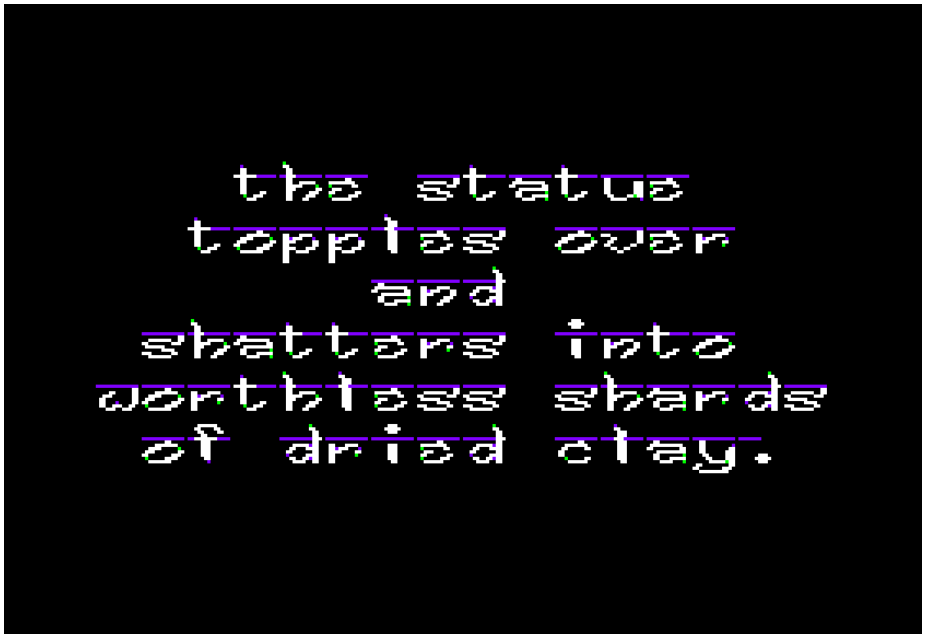
Scene $12 (18): THE SULTAN SCURRIES BACK TO HIS THRONE



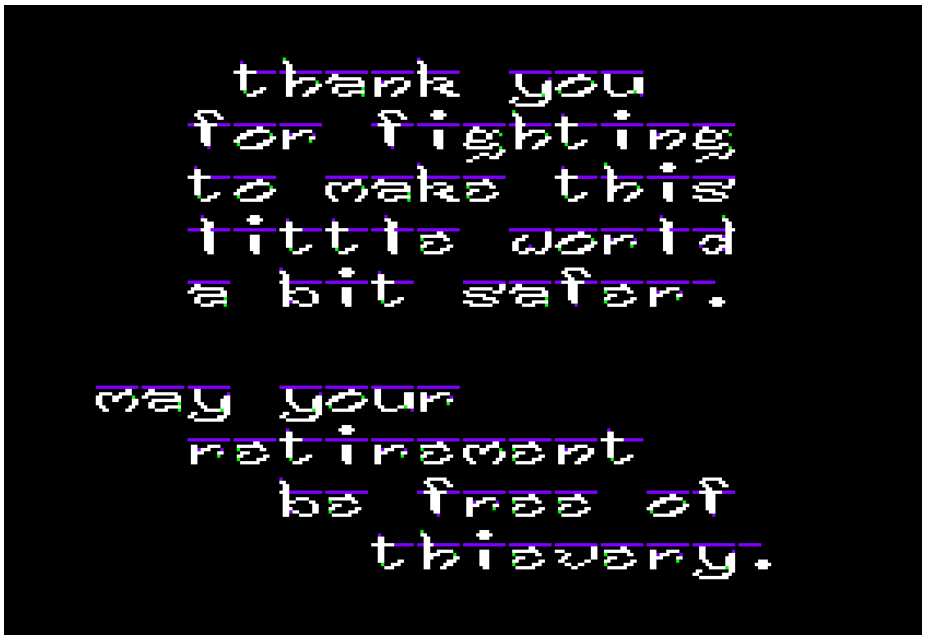
Scene $17 (23): TIS UP TO YOU WHETHER YOU WILL RETIRE NOW . . .



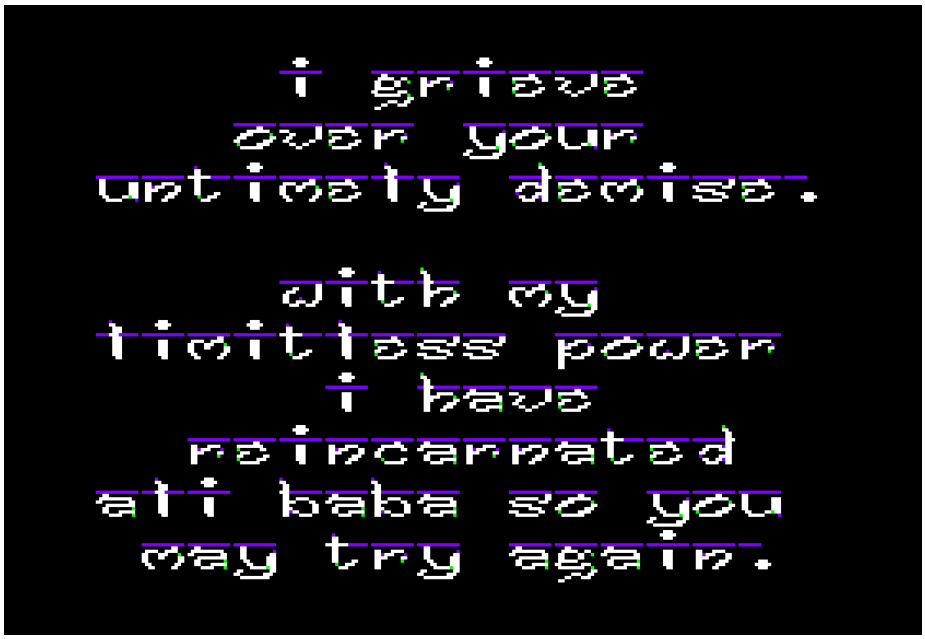
Scene $19 (25): THE STATUE TOPPLES OVER AND . . .



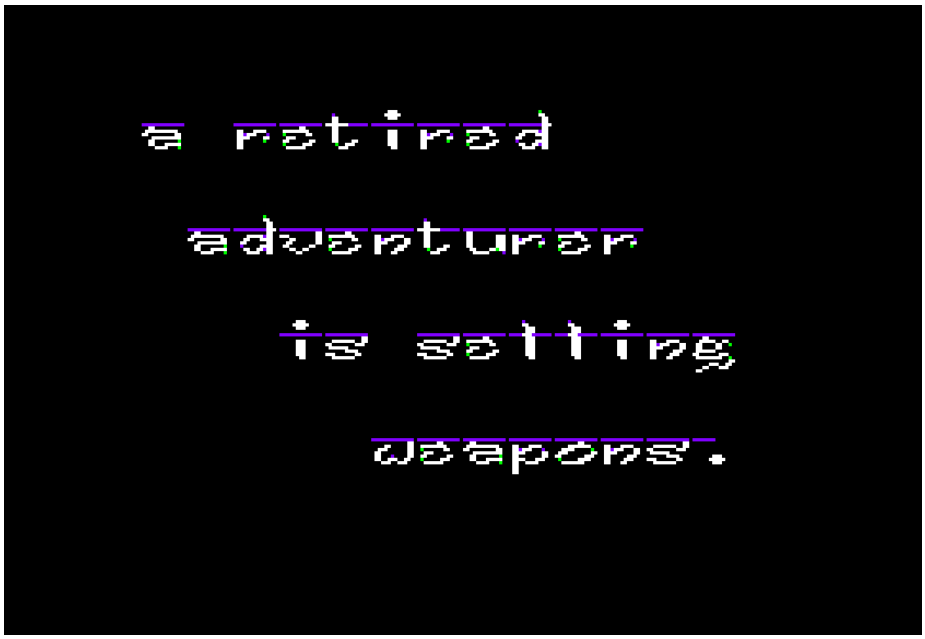
Scene $1D (29): THANK YOU FOR FIGHTING TO MAKE THIS . . .



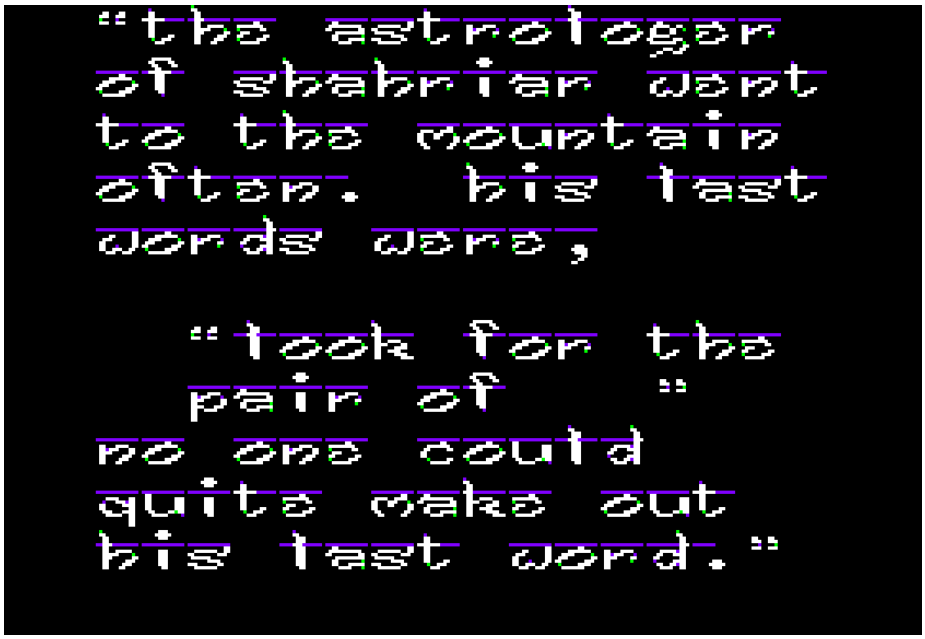
Scene $23 (35): I GRIEVE OVER YOUR UNTIMELY DEMISE . . . (retirement)



Scene $29 (41): A RETIRED ADVENTURER IS SELLING . . .



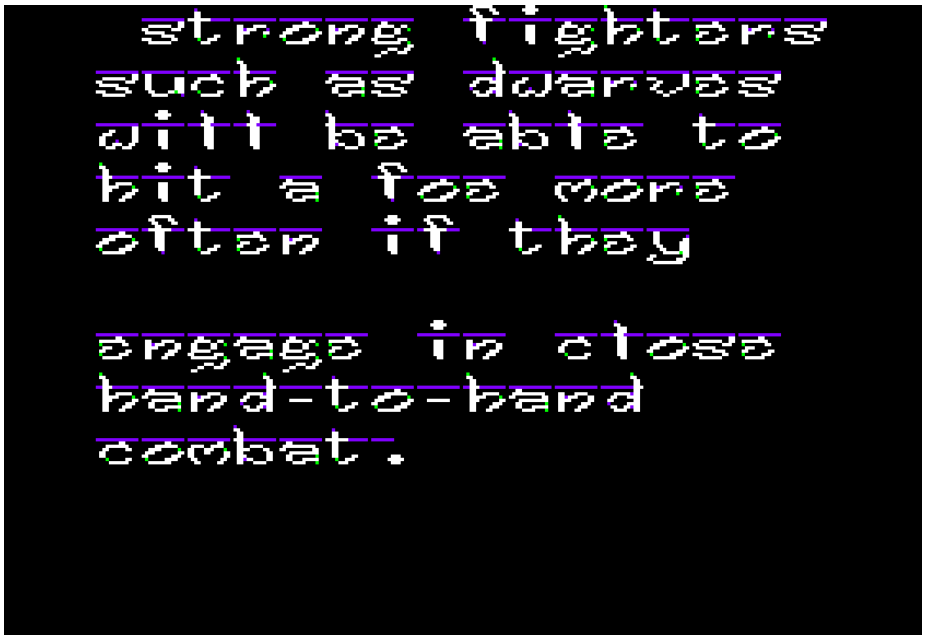
Scene $2B (43): THE ASTROLOGER OF SHAHRIAR ... (hint text)



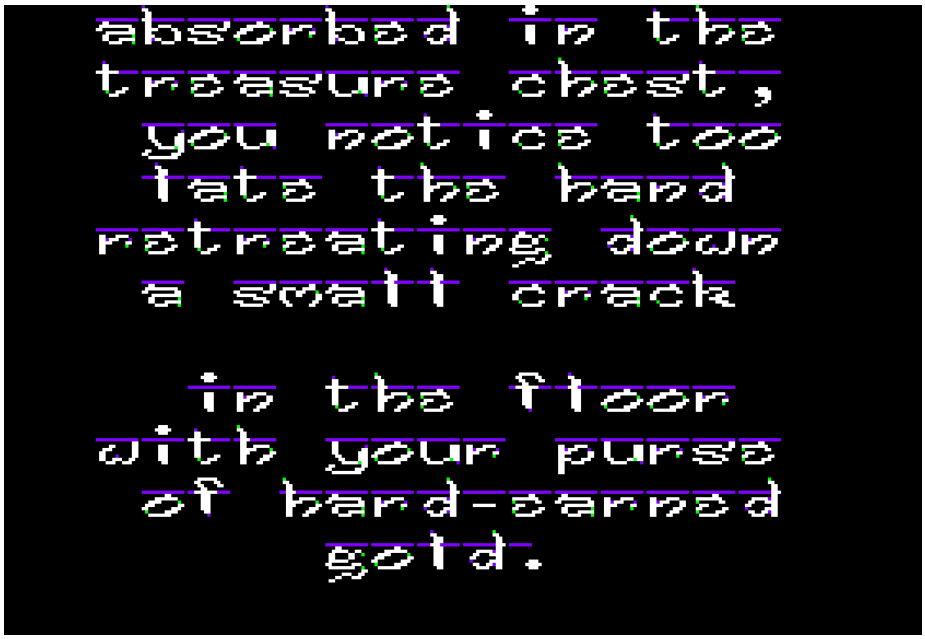
Scene $30 (48): A BLACKSMITH OFFERS YOU SOME FINE . . .

Scene $32 (50): ERROR 47 FORTY-SEVEN (weapon shop)

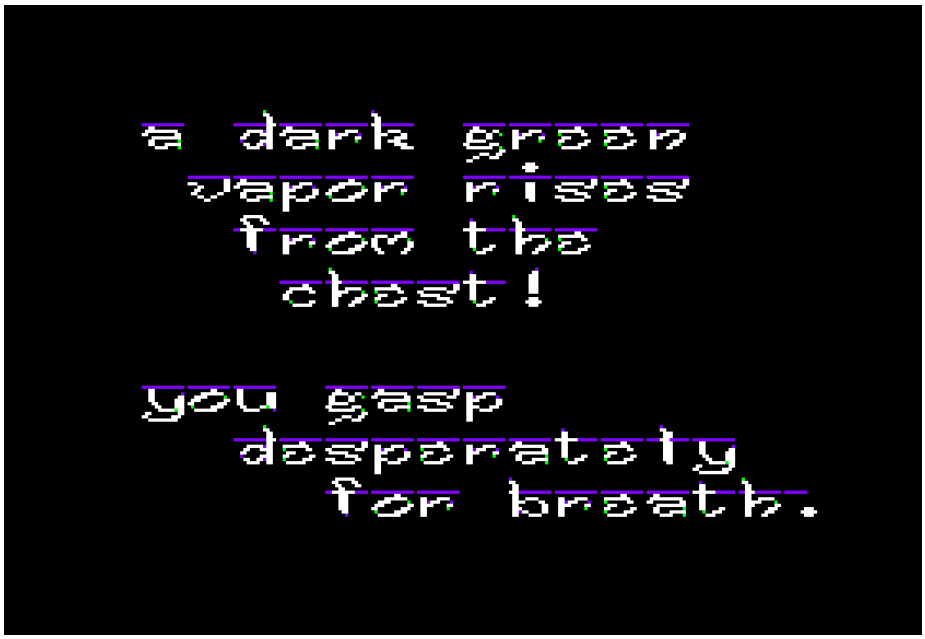
Scene $38 (56): STRONG FIGHTERS SUCH AS DWARVES . . . (hint)



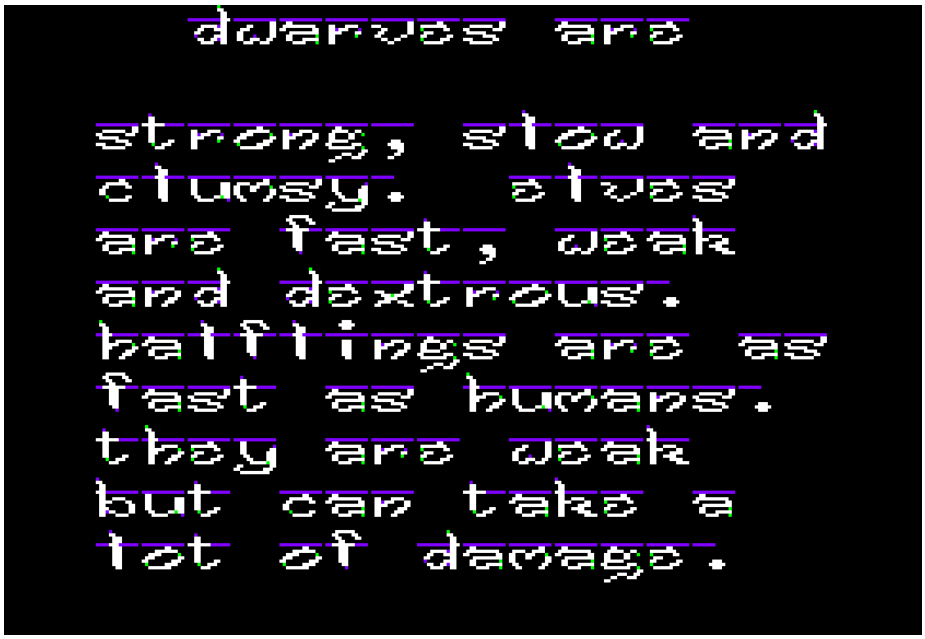
Scene $39 (57): ABSORBED IN THE TREASURE CHEST . . . (trap: theft)



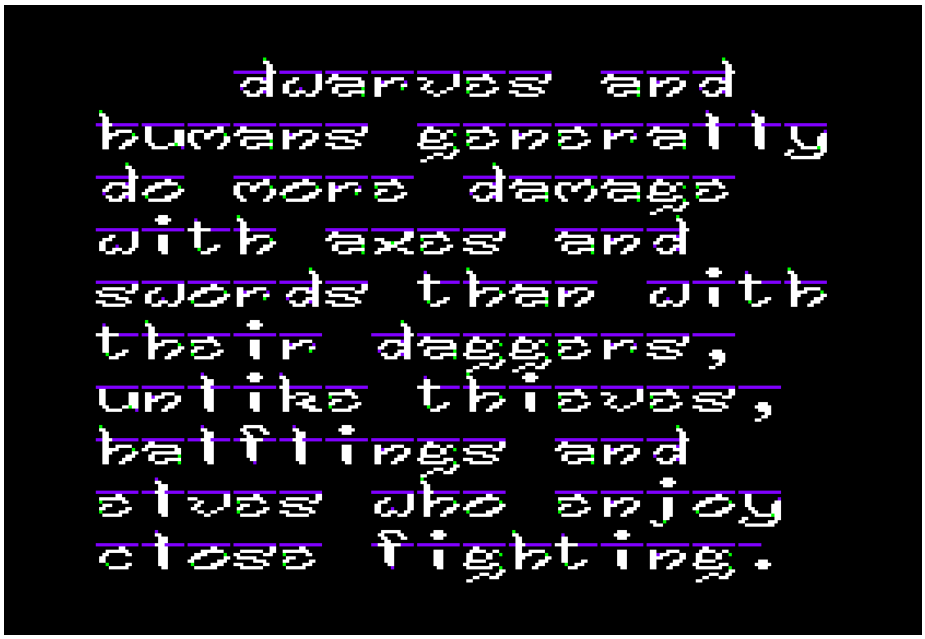
Scene $3A (58): A DARK GREEN VAPOR RISES FROM THE . . . (trap: gas)



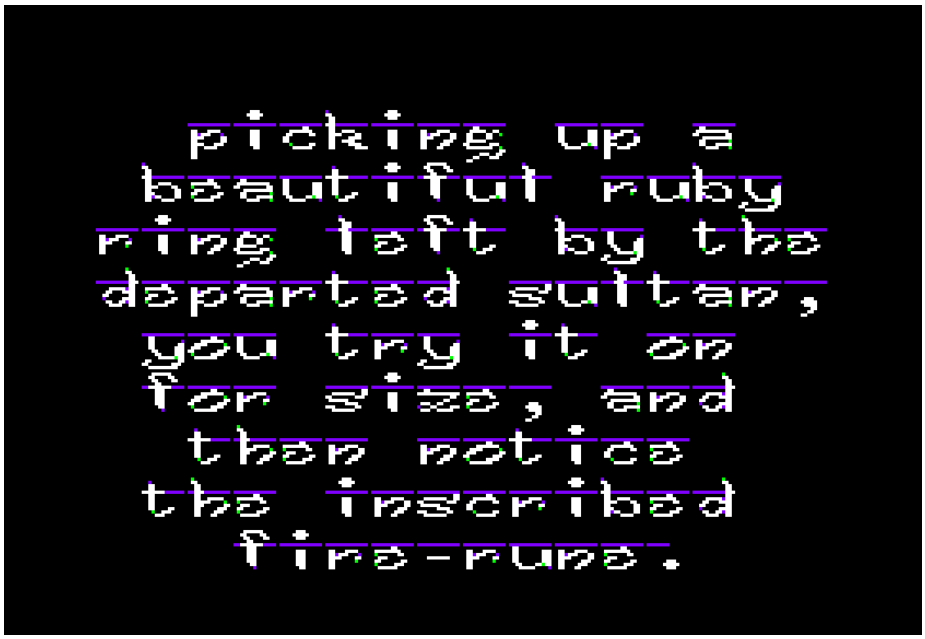
Scene $3F (63): DWARVES ARE STRONG, SLOW AND CLUMSY . . . (hint)



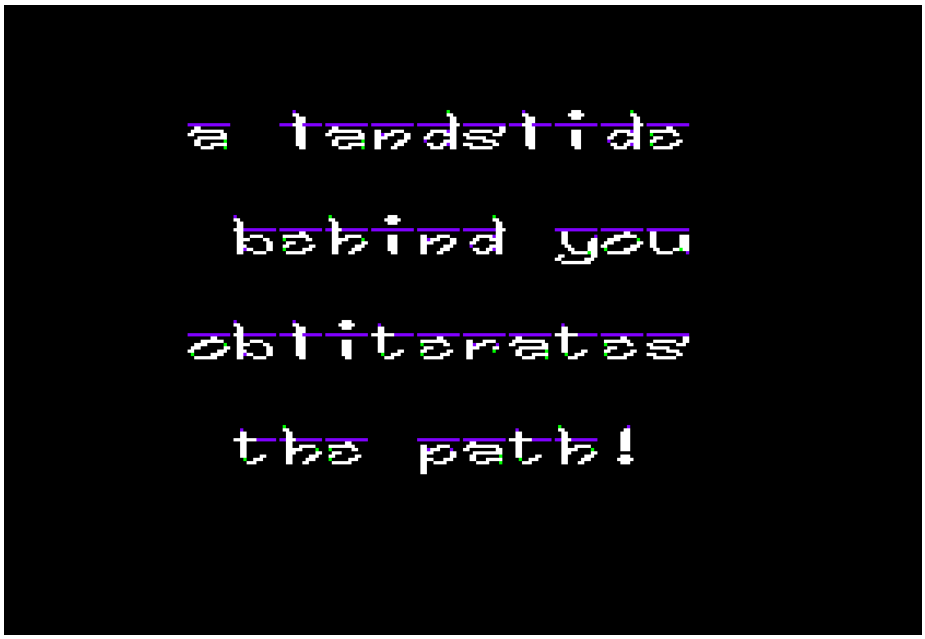
Scene $40 (64): DWARVES AND HUMANS GENERALLY DO MORE . . . (hint)

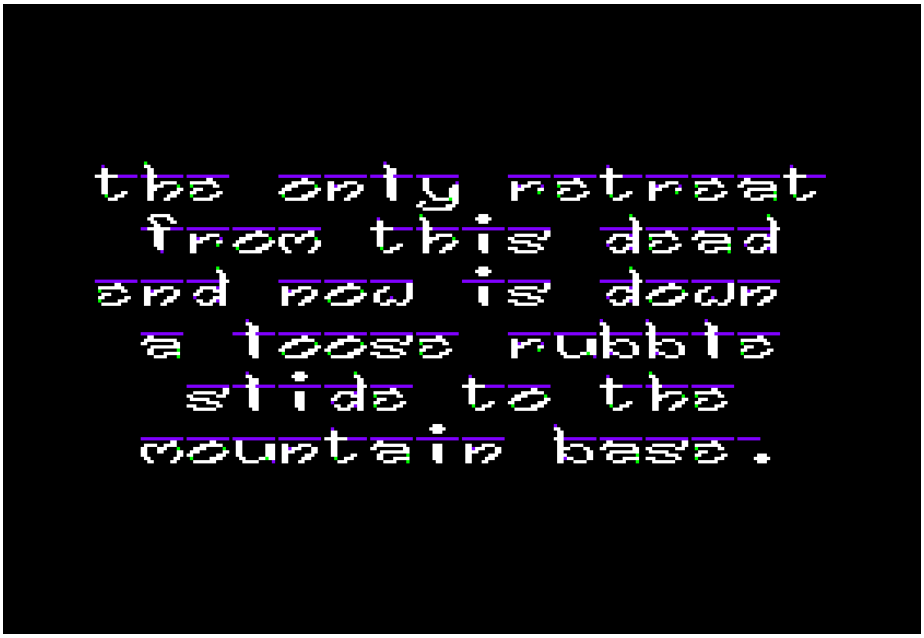


Scene $43 (67): PICKING UP A BEAUTIFUL RUBY RING ...

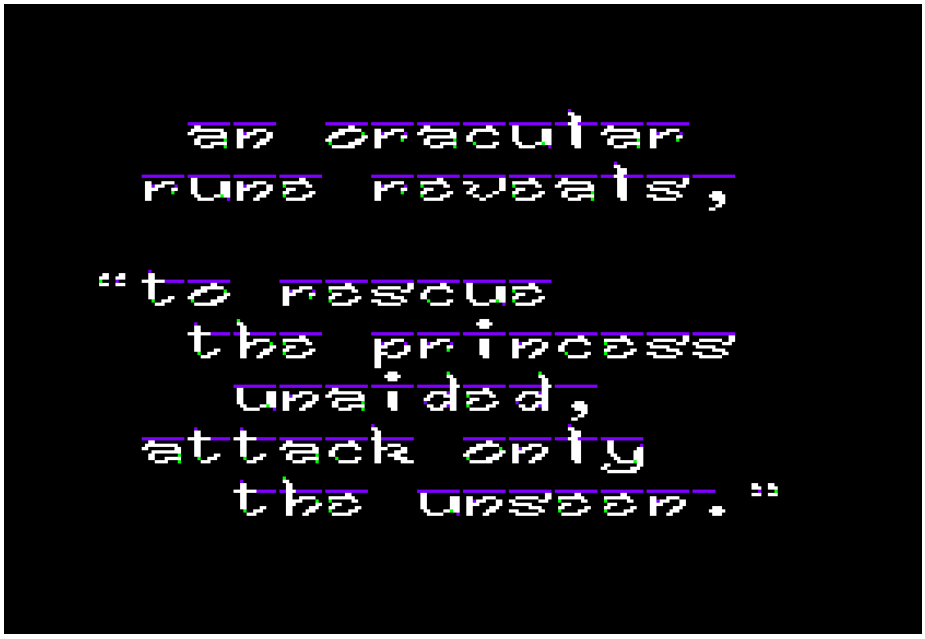


Scene $44 (68): A LANDSLIDE BEHIND YOU OBLITERATES . . .





Scene $45 (69): AN ORACULAR RUNE REVEALS . . . (hint)



526a ⟨*defines* 25b⟩+≡ (909) ◁491 530a▷

```
SCENE_TRANSITION     EQU     $5CFF    ; scene transition flag (nonzero = new scene number)
SCENE_TEXT_DATA      EQU     $5C00    ; scene text buffer (overwrites border code at [[SCENE_TEXT_DATA
PRNG_SCENE_SKIP_MASK EQU     $0A      ; bits 1+3: skip disk load ~75% of iterations
TUNE_ATTRACT         EQU     $06      ; tune player index for attract/opening music
```

Defines:
PRNG_SCENE_SKIP_MASK, used in chunk 527.
SCENE_TEXT_DATA, used in chunks 298, 301, 527, 584, and 666.
SCENE_TRANSITION, used in chunk 527.
TUNE_ATTRACT, used in chunks 527 and 644.

526b ⟨*var scene number* 526b⟩≡ (909)

```
    ORG     $5A99
SCENE_NUMBER:
    DC.B    $CE                      ; initial value (runtime: current scene number)
```

Defines:
SCENE_NUMBER, used in chunks 527 and 666.

527 ⟨*scene loop* 527⟩≡ (909)

```
        ORG     $6C37
    SCENE_LOOP:
        SUBROUTINE SCENE_LOOP
        ; Main scene display loop.
        ;
        ; Inputs:  A = scene number
        ; Behavior:
        ;   Displays scene text, plays attract tune, loops until keypress
        ;   or all transitions complete.  Redraws map on exit.
        ; Clobbers: A, X, Y

        STA     SCENE_NUMBER                ; store incoming scene number
        STOB    #MAP_WIDTH,BLINK_COL        ; disable blink cursor (sentinel = 20)

        JSR     SAVE_ZP_CONTEXT            ; save caller's ZP context

        JSR     STEP_PRNG                   ; advance PRNG
        LDA     PRNG_OUTPUT                 ; fresh random byte
        AND     #PRNG_SCENE_SKIP_MASK       ; test bits 1 and 3
        BNE     .display                    ; ~75%: skip disk reload
        JSR     LOAD_SCENE_DATA             ; reload scene text from disk

    .display:
        JSR     LOAD_SCENE_FROM_DISK        ; load scene text from disk
        BCS     .exit                       ; carry set = disk error

        JSR     DISPLAY_STATUS_BAR          ; refresh status bar
        STOW    SCENE_TEXT_DATA,TEXT_STREAM_PTR ; point at scene data
        JSR     TEXT_RENDERER               ; render scene text

        LDX     #TUNE_ATTRACT               ; tune 6: attract/opening music
        JSR     TUNE_PLAYER                 ; advance tune playback
        LDA     SCENE_TRANSITION            ; scene transition flag
        BEQ     .done                       ; zero = no transition
        STA     SCENE_NUMBER                ; nonzero = new scene number
        JMP     .display                    ; restart with new scene

    .done:
        JSR     SET_TEXT_WINDOW_BOTTOM      ; restore text window
        LDA     #$00
        STA     LOCATION_POS                ; clear location flags
        STA     LOCATION_STYLE
        JSR     ENTER_LOCATION              ; redraw map if location changed
        JMP     RESTORE_ZP_CONTEXT         ; restore ZP and return

    .exit:
        JSR     SET_TEXT_WINDOW_BOTTOM
        LDA     #MSG_BREAKS_FREE
        JSR     DISPLAY_MESSAGE             ; "breaks free" message
```

```
        JMP     .done
```

Defines:
SCENE_LOOP, used in chunks 220, 268, 293, 325, 342, 354, 360, 404, 428, 444, and 447.

Uses BLINK_COL 563b, DISPLAY_MESSAGE 637, DISPLAY_STATUS_BAR 550, ENTER_LOCATION 535, LOAD_SCENE_DATA 529, LOAD_SCENE_FROM_DISK 666, LOCATION_POS 562b, LOCATION_STYLE 563a, MAP_WIDTH 388a, MSG_BREAKS_FREE 634, PRNG_OUTPUT 593a, PRNG_SCENE_SKIP_MASK 526a, RESTORE_ZP_CONTEXT 620, SAVE_ZP_CONTEXT 619c, SCENE_NUMBER 526b, SCENE_TEXT_DATA 526a, SCENE_TRANSITION 526a, SET_TEXT_WINDOW_BOTTOM 627, STEP_PRNG 595, STOB 8c, STOW 8a, TEXT_RENDERER 644, TEXT_STREAM_PTR 571c, TUNE_ATTRACT 526a, and TUNE_PLAYER 682.

12.0.3 Load scene data

LOAD_SCENE_DATA reads scene data from disk into the $5C00 region. It asserts the disk motor on, sets the IOB track field, dispatches RWTS, then calls DISK_READ_FIELD. On failure, it writes a PRNG byte into the message string area as a graceful degradation.

529 ⟨*load scene data* 529⟩≡ (909)

```
      ORG     $6DD8
  SCENE_LOADER_START:
  LOAD_SCENE_DATA:
      SUBROUTINE LOAD_SCENE_DATA
      ; Load scene text data from disk.
      ;
      ; Inputs:  $08/$09 = IOB pointer
      ; Behavior:
      ;   Sets IOB track to 5, runs RWTS + DISK_READ_FIELD.
      ;   On error, writes PRNG byte into message area.
      ; Clobbers: A, Y

      BIT     $C0E9                   ; assert disk motor on
      LDA     #$05                    ; \
      LDY     #$04                    ;  | IOB byte 4 = track 5
      STA     ($08),Y                 ; /
      LDA     #$00
      JSR     RWTS_DISPATCH           ; execute RWTS command
      CLC
      JSR     DISK_READ_FIELD         ; read sector data
      BCS     .error
      RTS                             ; success

  INTEGRITY_FAIL_ADDR:
  .error:
      LDY     PRNG_OUTPUT             ; use PRNG as index
      JSR     STEP_PRNG               ; advance PRNG
      LDA     PRNG_OUTPUT             ; load new PRNG byte
      STA     $9AA6,Y                 ; fill into message area
      RTS
```

Defines:
INTEGRITY_FAIL_ADDR, used in chunk 671.
LOAD_SCENE_DATA, used in chunk 527.
SCENE_LOADER_START, used in chunk 671.
Uses DISK_READ_FIELD 771, PRNG_OUTPUT 593a, RWTS_DISPATCH 546, and STEP_PRNG 595.

12.0.4 Load map tiles

LOAD_MAP_TILES extracts the 2-bit style index from bits 7–6 of LOCATION_STYLE and uses it to look up three tile characters from parallel tables: TILE_BLINK_TAB, TILE_FILL_TAB, and TILE_DEFAULT_TAB. The results are stored in BLINK_ALT_CHAR, MAP_FILL_CHAR, and DEFAULT_CHAR.

530a ⟨*defines* 25b⟩+≡ (909) ◁526a 533a▷

```
STYLE_INDEX_MASK        EQU     $03     ; mask for 2-bit style index
```

Defines:
STYLE_INDEX_MASK, used in chunk 531a.

Default character codes for map display, used by LOAD_MAP_TILES.

530b ⟨*map char defaults* 530b⟩≡ (909)

```
    ORG     $7AC1
MAP_FILL_CHAR:
    DC.B    $25                     ; map grid fill character
DEFAULT_CHAR:
    DC.B    $2C                     ; default entity character
BLINK_ALT_CHAR:
    DC.B    $00                     ; alternate blink character
```

Defines:
BLINK_ALT_CHAR, used in chunks 462, 472a, and 531a.
DEFAULT_CHAR, used in chunks 462 and 531a.
MAP_FILL_CHAR, used in chunks 531a and 538.

Tile style lookup tables (4 entries each), used by LOAD_MAP_TILES.

530c ⟨*tile style tables* 530c⟩≡ (909)

```
    ORG     $7AD0
TILE_DEFAULT_TAB:
    HEX     2C 2C 2C 2C             ; default char per style
TILE_FILL_TAB:
    HEX     25 2A 20 29             ; fill char per style
TILE_BLINK_TAB:
    HEX     00 00 00 00             ; blink alt char per style
```

Defines:
TILE_BLINK_TAB, used in chunk 531a.
TILE_DEFAULT_TAB, used in chunk 531a.
TILE_FILL_TAB, used in chunk 531a.

531a ⟨*load map tiles* 531a⟩≡ (909)

```
    ORG     $746C
LOAD_MAP_TILES:
    SUBROUTINE LOAD_MAP_TILES
    ; Select tile characters for the current location style.
    ;
    ; Inputs:  LOCATION_STYLE (bits 7-6 = style index)
    ; Outputs: BLINK_ALT_CHAR, MAP_FILL_CHAR, DEFAULT_CHAR
    ; Clobbers: A, X

    LDA     LOCATION_STYLE          ; location style byte
    ROL2                            ; rotate bits 7-6 down to 1-0
    ROL2
    AND     #STYLE_INDEX_MASK       ; mask to 2-bit style index
    TAX
    LDA     TILE_BLINK_TAB,X        ; blink alternate char for this style
    STA     BLINK_ALT_CHAR
    LDA     TILE_FILL_TAB,X         ; map fill char for this style
    STA     MAP_FILL_CHAR
    LDA     TILE_DEFAULT_TAB,X      ; default char for this style
    STA     DEFAULT_CHAR
    RTS
```

Defines:
LOAD_MAP_TILES, used in chunk 535.
Uses BLINK_ALT_CHAR 530b, DEFAULT_CHAR 530b, LOCATION_STYLE 563a, MAP_FILL_CHAR 530b, ROL2 16c, STYLE_INDEX_MASK 530a, TILE_BLINK_TAB 530c, TILE_DEFAULT_TAB 530c, and TILE_FILL_TAB 530c.

12.0.5 Setup scene display

SETUP_SCENE_DISPLAY prepares the display for a character's scene turn. If CHAR_AI_MODE is nonzero (AI-controlled), sets AI_WAIT_FLAG to $0A (10 ticks) and plays the AI turn tune; otherwise clears the flag. After drawing the character sprite and loading its name pointer, shifts AI_WAIT_FLAG left by one (ROL) into WAIT_LOOP_COUNT and tail-calls TIMED_WAIT.

531b ⟨*var char ai mode* 531b⟩≡ (909)

```
    ORG     $5A05
CHAR_AI_MODE:
    DC.B    $D3                     ; initial value (runtime: 0 = player, 2-5 = AI behavior modes)
```

Defines:
CHAR_AI_MODE, used in chunks 225, 227, 232, 237, 325, 368, 385, 390, 404, and 532c.

532a ⟨*var ai wait flag* 532a⟩≡ (909)

```
      ORG     $5A06
  AI_WAIT_FLAG:
      DC.B    $D2                     ; initial value (runtime: wait delay for AI moves (0 or $0A))
```

Defines:
AI_WAIT_FLAG, used in chunk 532c.

532b ⟨*var event pos* 532b⟩≡ (909)

```
      ORG     $5A5E
  EVENT_POS:
      DC.B    $A0                     ; initial value (runtime: event position (from FIND_NEAREST_EVEN
```

Defines:
EVENT_POS, used in chunks 354, 385, and 535.
Uses FIND_NEAREST_EVENT 398.

532c ⟨*setup scene display* 532c⟩≡ (909)

```
      ORG     $0901
  SETUP_SCENE_DISPLAY:
      SUBROUTINE SETUP_SCENE_DISPLAY
      ; Initialize display for a character's scene turn.
      ;
      ; Inputs:  CHAR_AI_MODE | 0 = player, nonzero = AI
      ; Behavior:
      ;   AI turns get a 10-tick wait and the AI tune.
      ;   Sets up sprite and name, then waits proportionally.
      ; Clobbers: A, X

      STOB    #$00,AI_WAIT_FLAG       ; assume player: no wait
      BANE    CHAR_AI_MODE,.not_ai    ; branch if not AI
      STOB    #$0A,AI_WAIT_FLAG       ; AI: set 10-tick delay
      LDX     #$10                    ; tune 8: AI turn effect
      JSR     TUNE_PLAYER
  .not_ai:
      JSR     SETUP_CHAR_SPRITE       ; draw character sprite
      JSR     LOAD_CHAR_NAME_PTR      ; load character name pointer
      LDA     AI_WAIT_FLAG            ; 0 or 10
      ROL                             ; shift left: 0->0, 10->20
      STA     WAIT_LOOP_COUNT         ; store final wait count
      JMP     TIMED_WAIT              ; wait and return
```

Defines:
SETUP_SCENE_DISPLAY, used in chunk 225.
Uses AI_WAIT_FLAG 532a, CHAR_AI_MODE 531b, LOAD_CHAR_NAME_PTR 486a, SETUP_CHAR_SPRITE 487, STOB 8c, TIMED_WAIT 606a, TUNE_PLAYER 682, and WAIT_LOOP_COUNT 51.

12.0.6 Enter location

ENTER_LOCATION sets up the viewport when the player enters a new location. If CHAR_PTR low byte matches LOCATION_POS, the location is unchanged and the routine returns immediately. Otherwise:

1. Stores the new position and calls ANIM_TICK_AND_WAIT, SET_TEXT_WINDOW_BOTTOM, and CLEAR_MAP.
2. Points ENTITY_PTR at CANDIDATE_POS and calls FIND_NEAREST_EVENT to compute EVENT_POS.
3. Reads CFIELD_STYLE (byte 5); strips the visited flag with STYLE_ID_MASK. If the style changed, updates LOCATION_STYLE, reloads tiles, and fills the map.
4. Reads LFIELD_MIN_POS (byte 3) and LFIELD_MAX_POS (byte 4) to set the viewport bounds.
5. Draws all cells in a nested row/column loop via DRAW_MAP_COL.
6. If the visited flag (bit 7 of CFIELD_STYLE) is clear, calls DRAW_MAP_ENTITIES to overlay entity icons.

533a ⟨*defines* 25b⟩+≡ (909) ◁530a 541a▷

```
STYLE_ID_MASK       EQU     $7F     ; strip visited flag (bit 7) from CFIELD_STYLE
```

Defines:
STYLE_ID_MASK, used in chunk 535.
Uses CFIELD_STYLE 561.

533b ⟨*var view max col* 533b⟩≡ (909)

```
        ORG     $5A08
VIEW_MAX_COL:
        DC.B    $D3                     ; initial value (runtime: viewport max column)
```

Defines:
VIEW_MAX_COL, used in chunks 535, 537, and 542.

533c ⟨*var view max row* 533c⟩≡ (909)

```
        ORG     $5A09
VIEW_MAX_ROW:
        DC.B    $C8                     ; initial value (runtime: viewport max row)
```

Defines:
VIEW_MAX_ROW, used in chunks 535, 537, and 542.

534a ⟨*var view min col* 534a⟩≡ (909)

```
    ORG     $5A0A
VIEW_MIN_COL:
    DC.B    $CF                     ; initial value (runtime: viewport min column)
```

Defines:
VIEW_MIN_COL, used in chunks 535, 537, and 542.

534b ⟨*var view min row* 534b⟩≡ (909)

```
    ORG     $5A0B
VIEW_MIN_ROW:
    DC.B    $D7                     ; initial value (runtime: viewport min row)
```

Defines:
VIEW_MIN_ROW, used in chunks 535, 537, and 542.

534c ⟨*var font row* 534c⟩≡ (909)

```
    ORG     $5A0D
FONT_ROW:
    DC.B    $CD                     ; initial value (runtime: font cursor row coordinate)
```

Defines:
FONT_ROW, used in chunks 239, 342, 400, 466, 467, 471, 473b, 487, 535, 538, 540, 542, 545, and 644.

534d ⟨*var candidate pos* 534d⟩≡ (909)

```
    ORG     $5A15
CANDIDATE_POS:
    DC.B    $B1                     ; initial value (runtime: position of event being evaluated)
```

Defines:
CANDIDATE_POS, used in chunks 398 and 535.

535 ⟨*enter location* 535⟩≡ (909)

```
    ORG     $72A7
ENTER_LOCATION:
    SUBROUTINE ENTER_LOCATION
    ; Set up viewport when entering a new location.
    ;
    ; Inputs:  CHAR_PTR | current character record pointer
    ; Behavior:
    ;   If location unchanged, returns immediately.  Otherwise clears
    ;   map, finds nearest event, updates style/tiles if needed, and
    ;   redraws all cells in a nested row/column loop.
    ; Clobbers: A, X, Y

    LDA     CHAR_PTR                ; current location (low byte)
    BANE    LOCATION_POS,.changed   ; changed since last render?
    RTS
.changed:
    STA     LOCATION_POS            ; record new position
    JSR     ANIM_TICK_AND_WAIT      ; advance animation frame
    JSR     SET_TEXT_WINDOW_BOTTOM  ; configure text window
    JSR     CLEAR_MAP               ; clear map display
    STOW    CANDIDATE_POS,ENTITY_PTR ; ENTITY_PTR -> $5A15
    JSR     FIND_NEAREST_EVENT      ; find closest event
    STA     EVENT_POS               ; save event position
    LDY     #CFIELD_STYLE           ; byte 5: style | visited flag
    LDA     (CHAR_PTR),Y
    AND     #STYLE_ID_MASK          ; strip visited flag (bit 7)
    BAEQ    LOCATION_STYLE,.same_style ; style unchanged?
    STA     LOCATION_STYLE          ; update current style
    JSR     LOAD_MAP_TILES          ; reload tile set
    JSR     FILL_MAP                ; fill map with new tiles
.same_style:
    LDY     #LFIELD_MIN_POS         ; byte 3: min viewport position
    LDA     (CHAR_PTR),Y
    JSR     POS_TO_COLROW           ; A = col, Y = row
    STY     VIEW_MIN_ROW            ; set top edge
    STY     FONT_ROW                ; row iterator starts at top
    STA     VIEW_MIN_COL            ; set left edge
    LDY     #LFIELD_MAX_POS         ; byte 4: max viewport position
    LDA     (CHAR_PTR),Y
    JSR     POS_TO_COLROW           ; A = col, Y = row
    STY     VIEW_MAX_ROW            ; set bottom edge
    STA     VIEW_MAX_COL            ; set right edge
.row_loop:
    MOVB    VIEW_MIN_COL,FONT_COL   ; reset column to left edge
.col_loop:
    JSR     DRAW_MAP_COL            ; draw tile at (FONT_COL, FONT_ROW)
    INC     FONT_COL
    LDA     VIEW_MAX_COL
    BAPL    FONT_COL,.col_loop      ; next column
```

```
    INC     FONT_ROW
    LDA     VIEW_MAX_ROW
    BAPL    FONT_ROW,.row_loop      ; next row
    LDY     #CFIELD_STYLE           ; byte 5: check visited flag
    LDA     (CHAR_PTR),Y
    BMI     .skip_entities          ; bit 7 set = visited, skip
    JSR     DRAW_MAP_ENTITIES       ; overlay entity icons
.skip_entities:
    RTS
```

Defines:
ENTER_LOCATION, used in chunks 219, 429, and 527.
Uses ANIM_TICK_AND_WAIT 605b, CANDIDATE_POS 534d, CFIELD_STYLE 561, CHAR_PTR 591a, CLEAR_MAP 539, DRAW_MAP_COL 540, DRAW_MAP_ENTITIES 542, ENTITY_PTR 561, EVENT_POS 532b, FILL_MAP 538, FIND_NEAREST_EVENT 398, FONT_COL 492a, FONT_ROW 534c, LFIELD_MAX_POS 491, LFIELD_MIN_POS 491, LOAD_MAP_TILES 531a, LOCATION_POS 562b, LOCATION_STYLE 563a, MOVB 8c, POS_TO_COLROW 599, SET_TEXT_WINDOW_BOTTOM 627, STOW 8a, STYLE_ID_MASK 533a, VIEW_MAX_COL 533b, VIEW_MAX_ROW 533c, VIEW_MIN_COL 534a, and VIEW_MIN_ROW 534b.

12.0.7 Check at boundary

CHECK_AT_BOUNDARY tests whether LOCATION_ID lies on any viewport edge. Converts the position to column/row, compares against all four edges, and stores $0001 in DATA_PTR if at edge, $0000 if interior.

536 ⟨*var location id* 536⟩≡ (909)

```
    ORG     $5A17
LOCATION_ID:
ENTITY_SEARCH_POS:
    DC.B    $8D                     ; initial value (runtime: current location/position index)
```

Defines:
ENTITY_SEARCH_POS, used in chunk 469.
LOCATION_ID, used in chunk 537.

537 ⟨*check at boundary* 537⟩≡ (909)

```
      ORG     $0BFA
  CHECK_AT_BOUNDARY:
      SUBROUTINE CHECK_AT_BOUNDARY
      ; Test if LOCATION_ID is on any viewport edge.
      ;
      ; Inputs:  LOCATION_ID, VIEW_MIN/MAX_COL/ROW
      ; Outputs: DATA_PTR = 1 (at edge) or 0 (interior)
      ; Clobbers: A, Y

      LDA     LOCATION_ID            ; current map position
      JSR     POS_TO_COLROW          ; A = col, Y = row
      BAEQ    VIEW_MIN_COL,.at_edge  ; on left edge?
      BAEQ    VIEW_MAX_COL,.at_edge  ; on right edge?
      TYA                            ; row -> A
      BAEQ    VIEW_MIN_ROW,.at_edge  ; on top edge?
      BAEQ    VIEW_MAX_ROW,.at_edge  ; on bottom edge?
      STOB    #$00,DATA_PTR          ; interior: result = 0
      STA     DATA_PTR+1
      RTS
  .at_edge:
      STOW2    $0001,DATA_PTR        ; boundary: result = 1
      RTS
```

Defines:
CHECK_AT_BOUNDARY, used in chunk 469.
Uses DATA_PTR 618a, LOCATION_ID 536, POS_TO_COLROW 599, STOB 8c, STOW2 8b, VIEW_MAX_COL 533b, VIEW_MAX_ROW 533c, VIEW_MIN_COL 534a, and VIEW_MIN_ROW 534b.

12.0.8 Fill map

FILL_MAP fills the entire 20×10 map grid with MAP_FILL_CHAR. It renders the glyph once via RENDER_FONT_CHAR to build the HRCG control string, then loops over all 200 positions from bottom-right to top-left, re-printing the same pre-built string at each position.

538 ⟨*fill map* 538⟩≡ (909)

```
      ORG     $743B
  FILL_MAP:
      SUBROUTINE FILL_MAP
      ; Fill entire 20x10 map grid with MAP_FILL_CHAR.
      ;
      ; Inputs:  MAP_FILL_CHAR | tile character to fill with
      ; Behavior:
      ;   Renders glyph once, then stamps it at all 200 positions
      ;   by decrementing col (wrapping at 0) and row.
      ; Clobbers: A, X, Y

      STOB    #$13,FONT_COL           ; start at column 19 (rightmost)
      STOB    #$09,FONT_ROW           ; start at row 9 (bottom)
      MOVB    MAP_FILL_CHAR,FONT_CHARNUM ; fill character
      STOB    #$01,FONT_CHARSET       ; charset 1 = custom font
      JSR     RENDER_FONT_CHAR        ; build HRCG control string once
  .loop:
      DEC     FONT_COL                ; move one column left
      BPL     .draw                   ; col >= 0 -> draw
      STOB    #$13,FONT_COL           ; wrap to column 19
      DEC     FONT_ROW                ; move one row up
      BPL     .draw                   ; row >= 0 -> draw
      RTS                             ; all 200 positions filled
  .draw:
      JSR     FONT_POS_TO_TEXT_POS    ; set HRCG cursor
      JSR     PRINT_STRING            ; stamp pre-built glyph
      JMP     .loop
```

Defines:
FILL_MAP, used in chunks 535 and 564.

Uses FONT_CHARNUM 575, FONT_CHARSET 576a, FONT_COL 492a, FONT_POS_TO_TEXT_POS 467, FONT_ROW 534c, MAP_FILL_CHAR 530b, MOVB 8c, PRINT_STRING 622, RENDER_FONT_CHAR 466, and STOB 8c.

12.0.9 Clear map

CLEAR_MAP loads bytes 0–1 of the record at CHAR_PTR (the name string pointer) into PRINT_STRING_ADDR and tail-calls SETUP_TEXT_POS to configure the text window.

539 ⟨*clear map* 539⟩≡ (909)

```
      ORG     $76B0
  CLEAR_MAP:

      LDWIY    CHAR_PTR,#$00,PRINT_STRING_ADDR ; bytes 0-1: name ptr -> PRINT_STRING_ADDR
      JMP     SETUP_TEXT_POS
```

Defines:
CLEAR_MAP, used in chunks 222, 535, and 556.
Uses CHAR_PTR 591a, LDWIY 17, PRINT_STRING_ADDR 576c, and SETUP_TEXT_POS 552.

12.0.10 Draw map column

DRAW_MAP_COL draws a single map cell at FONT_COL/FONT_ROW. Computes the linear position inline (col + row × MAP_WIDTH) rather than calling COLROW_TO_POS, then resolves the entity and renders its font character.

540 ⟨*draw map col* 540⟩≡ (909)

```
      ORG     $7323
  DRAW_MAP_COL:
      SUBROUTINE DRAW_MAP_COL
      ; Draw one map cell at FONT_COL/FONT_ROW.
      ;
      ; Inputs:  FONT_COL, FONT_ROW
      ; Behavior:
      ;   Computes linear position, finds entity, draws glyph.
      ; Clobbers: A, X, Y

      LDA     FONT_COL                ; A = column
      CLC
      LDY     FONT_ROW                ; Y = row counter
  .mult_loop:
      DEY                             ; row--
      BMI     .mult_done              ; Y < 0 -> done
      ADC     #MAP_WIDTH              ; A += 20 (one row)
      BCC     .mult_loop
  .mult_done:
      JSR     FIND_ENTITY_AT_POS      ; find entity at position
      JSR     GET_ENTITY_FONTCHAR     ; resolve font character
      JSR     RENDER_FONT_CHAR        ; draw it
      RTS
```

Defines:
DRAW_MAP_COL, used in chunk 535.
Uses FIND_ENTITY_AT_POS 469, FONT_COL 492a, FONT_ROW 534c, GET_ENTITY_FONTCHAR 462, MAP_WIDTH 388a, and RENDER_FONT_CHAR 466.

12.0.11 Draw map entities

DRAW_MAP_ENTITIES overlays entity icons on the map. Three phases:

1. Sets CFIELD_DRAWN_FLAG (bit 7) in CFIELD_STYLE to mark entities drawn. If MAP_REDRAW_FLAG is set, jumps directly to mob drawing.
2. Checks STYLE_SCROLL_MASK (bits 6–5) of LOCATION_STYLE. If set, displays MSG_ROCK_FALLS, sets MAP_REDRAW_FLAG, and falls through to mob drawing.
3. Otherwise, scans the view grid cell by cell. Byte 2 (CFIELD_NEXT) is saved and zeroed during the scan. For each entity whose EVFIELD_TYPE is not EVENT_EMPTY and falls in the drawable range, draws its icon via DRAW_MAP_ICON_B.

After the grid scan, iterates the mob group via FIRST_GROUP_MEMBER/NEXT_GROUP_MEMBER. Each mob's MFIELD_POSITION and MFIELD_APPEARANCE set up the font cursor. If the low nibble of MFIELD_LEVEL_ENC is zero, the mob gets only a static render; otherwise the full blink sequence plays.

541a ⟨*defines* 25b⟩+≡ (909) ◁533a 553▷

```
CFIELD_DRAWN_FLAG       EQU     $80     ; bit 7 of CFIELD_STYLE: entities drawn
STYLE_SCROLL_MASK       EQU     $60     ; bits 6-5 of LOCATION_STYLE: scroll trigger
```

Defines:
CFIELD_DRAWN_FLAG, used in chunk 542.
STYLE_SCROLL_MASK, used in chunk 542.
Uses CFIELD_STYLE 561 and LOCATION_STYLE 563a.

541b ⟨*zero page defines* 25a⟩+≡ (909) ◁483a 565▷

```
ENTITY_REC              EQU     $BE     ; entity lookup result from FIND_ENTITY_AT_POS (2 bytes)
```

Defines:
ENTITY_REC, used in chunks 232, 312, 322, 342, 394, 404, 459, 462, 468, 469, 471, 475, and 542.
Uses FIND_ENTITY_AT_POS 469.

Map redraw flag, used by DRAW_MAP_ENTITIES.

541c ⟨*map redraw flag* 541c⟩≡ (909)

```
        ORG     $7AC4
MAP_REDRAW_FLAG:
        DC.B    $00                     ; non-zero = full map redraw needed
```

Defines:
MAP_REDRAW_FLAG, used in chunk 542.

542 ⟨*draw map entities* 542⟩≡ (909)

```
    ORG     $733B
DRAW_MAP_ENTITIES:
    SUBROUTINE DRAW_MAP_ENTITIES
    ; Draw entity icons on the current map viewport.
    ;
    ; Inputs:  CHAR_PTR, VIEW_MIN/MAX_COL/ROW
    ; Behavior:
    ;   Marks entities drawn, optionally triggers rock-fall scroll,
    ;   scans grid for drawable entities, then iterates mob group.
    ; Clobbers: A, X, Y

    LDY     #CFIELD_STYLE           ; byte 5: style + visited flag
    LDA     (CHAR_PTR),Y
    ORA     #CFIELD_DRAWN_FLAG      ; set bit 7 = entities drawn
    STA     (CHAR_PTR),Y
    LDA     MAP_REDRAW_FLAG         ; full redraw pending?
    BEQ     .no_redraw
    JMP     .draw_mobs              ; skip grid scan
.no_redraw:
    LDA     LOCATION_STYLE
    AND     #STYLE_SCROLL_MASK      ; bits 6-5: scroll trigger
    BEQ     .scan_grid              ; 0 = normal scan
    JSR     SET_CURSOR_ROW21
    LDA     #MSG_ROCK_FALLS
    JSR     DISPLAY_MESSAGE
    STOB    #$FF,MAP_REDRAW_FLAG    ; schedule full redraw
    BNE     .draw_mobs              ; always taken
.scan_grid:
    MOVB    VIEW_MIN_COL,FONT_COL   ; start at top-left of view
    MOVB    VIEW_MIN_ROW,FONT_ROW
    LDY     #CFIELD_NEXT            ; byte 2: mob head link
    LDA     (CHAR_PTR),Y
    PHA                             ; save mob head
    LDA     #$00
    STA     (CHAR_PTR),Y            ; clear during scan
.cell_check:
    LDA     FONT_COL
    LDY     FONT_ROW
    JSR     COLROW_TO_POS           ; A = packed position
    JSR     FIND_ENTITY_AT_POS      ; -> ENTITY_REC
    LDA     ENTITY_REC+1            ; high byte of entity pointer
    BEQ     .next_col               ; 0 = no entity here
    JSR     GET_ENTITY_FONTCHAR     ; resolve font character
    LDY     #EVFIELD_TYPE           ; byte 1: event type
    LDA     (ENTITY_REC),Y
    BAEQ    #EVENT_EMPTY,.next_col ; $FE = empty slot, skip
    BALT    #$80,.draw_it           ; $00-$7F = drawable
    BALT    #EVENT_ACTIVATED,.next_col ; $80-$BF = not activated, skip
.draw_it:                           ; $C0-$FF = activated, draw
```

```
    JSR     SET_CURSOR_ROW21
    JSR     CLASSIFY_ENTITY_TYPE
    JSR     PRINT_BOTTOM_CENTERED
    JSR     DRAW_MAP_ICON_B
.next_col:
    INC     FONT_COL
    LDA     VIEW_MAX_COL
    BAPL    FONT_COL,.cell_check   ; next column
    MOVB    VIEW_MIN_COL,FONT_COL  ; wrap column, advance row
    INC     FONT_ROW
    LDA     VIEW_MAX_ROW
    BAPL    FONT_ROW,.cell_check   ; next row
    LDY     #CFIELD_NEXT           ; restore mob head link
    PLA
    STA     (CHAR_PTR),Y
    BNE     .draw_mobs             ; non-zero -> draw mobs too
    RTS
.draw_mobs:
    JSR     FIRST_GROUP_MEMBER     ; MOB_PTR -> first mob
.mob_loop:
    LDY     #MFIELD_POSITION       ; byte 3: packed map position
    LDA     (MOB_PTR),Y
    JSR     POS_TO_COLROW          ; A = col, Y = row
    STY     FONT_ROW
    STA     FONT_COL
    STOB    #$01,FONT_CHARSET      ; custom font
    LDY     #MFIELD_APPEARANCE     ; byte 4: appearance index
    LDA     (MOB_PTR),Y
    JSR     APPEARANCE_TO_FONTCHAR ; -> font char number
    STA     FONT_CHARNUM
    BYEQ    #$00,.mob_blink        ; char 0 -> always blink
    LDY     #MFIELD_LEVEL_ENC      ; byte 12: level encoding
    LDA     (MOB_PTR),Y
    AND     #LOW_NIBBLE_MASK       ; low nibble = base level
    BEQ     .mob_anim              ; 0 -> static render only
.mob_blink:
    JSR     SET_CURSOR_ROW21
    JSR     PRINT_MEMBER_NAME
    JSR     DRAW_MAP_ICON_B        ; draw with blink
    JMP     .mob_done
.mob_anim:
    JSR     RENDER_FONT_CHAR       ; draw without blink
.mob_done:
    JSR     NEXT_GROUP_MEMBER      ; advance to next mob
    LDA     MOB_PTR+1              ; 0 = end of group
    BNE     .mob_loop
    RTS
```

Defines:
DRAW_MAP_ENTITIES, used in chunk 535.
Uses APPEARANCE_TO_FONTCHAR 461, CFIELD_DRAWN_FLAG 541a, CFIELD_NEXT 561,

CFIELD_STYLE 561, CHAR_PTR 591a, CLASSIFY_ENTITY_TYPE 459, COLROW_TO_POS 600a, DISPLAY_MESSAGE 637, DRAW_MAP_ICON_B 545, ENTITY_REC 541b, EVENT_ACTIVATED 397a, EVENT_EMPTY 397a, EVFIELD_TYPE 397a, FIND_ENTITY_AT_POS 469, FIRST_GROUP_MEMBER 482, FONT_CHARNUM 575, FONT_CHARSET 576a, FONT_COL 492a, FONT_ROW 534c, GET_ENTITY_FONTCHAR 462, LOCATION_STYLE 563a, LOW_NIBBLE_MASK 553, MAP_REDRAW_FLAG 541c, MFIELD_APPEARANCE 561, MFIELD_POSITION 561, MOB_PTR 608, MOVB 8c, MSG_ROCK_FALLS 634, NEXT_GROUP_MEMBER 482, POS_TO_COLROW 599, PRINT_BOTTOM_CENTERED 628, PRINT_MEMBER_NAME 485, RENDER_FONT_CHAR 466, SET_CURSOR_ROW21 627, STOB 8c, STYLE_SCROLL_MASK 541a, VIEW_MAX_COL 533b, VIEW_MAX_ROW 533c, VIEW_MIN_COL 534a, and VIEW_MIN_ROW 534b.

12.0.12 Draw map icon

DRAW_MAP_ICON_B renders an entity icon, flashes it briefly with the blink alternate frame, waits for BLINK_DELAY ticks, restores the normal frame, and disables the blink cursor.

Blink delay parameter, used by DRAW_MAP_ICON_B.

544a ⟨*blink delay* 544a⟩≡ (909)

```
    ORG     $7AAC
BLINK_DELAY:
    DC.B    $2D                     ; blink delay (loaded into WAIT_LOOP_COUNT)
```

Defines:
BLINK_DELAY, used in chunk 545.
Uses WAIT_LOOP_COUNT 51.

544b ⟨*var blink row* 544b⟩≡ (909)

```
    ORG     $5AA3
BLINK_ROW:
    DC.B    $C3                     ; initial value (runtime: font row)
```

Defines:
BLINK_ROW, used in chunks 239, 354, 473b, 475, 487, and 545.

544c ⟨*var blink char* 544c⟩≡ (909)

```
    ORG     $5AA4
BLINK_CHAR:
    DC.B    $D4                     ; initial value (runtime: font character number)
```

Defines:
BLINK_CHAR, used in chunks 239, 354, 472a, 473b, 475, 487, and 545.

545 ⟨*draw map icon b* 545⟩≡ (909)

```
      ORG     $740B
  DRAW_MAP_ICON_B:
      SUBROUTINE DRAW_MAP_ICON_B
      ; Draw entity icon with blink flash effect.
      ;
      ; Inputs:  FONT_COL, FONT_ROW, FONT_CHARNUM
      ; Behavior:
      ;   Renders glyph, copies to blink cursor, flashes alternate
      ;   frame, waits, restores normal, disables blink.
      ; Clobbers: A, X, Y

      JSR     SCROLL_STATUS_LINE     ; update status line
      JSR     RENDER_FONT_CHAR       ; draw glyph
      MOVB    FONT_COL,BLINK_COL     ; arm blink cursor: column
      MOVB    FONT_ROW,BLINK_ROW     ;                   row
      MOVB    FONT_CHARNUM,BLINK_CHAR;                   character
      JSR     DRAW_BLINK_ALT         ; draw alternate blink frame
      MOVB    BLINK_DELAY,WAIT_LOOP_COUNT ; load blink duration
      JSR     TIMED_WAIT             ; pause for blink interval
      JSR     DRAW_BLINK_NORMAL      ; restore normal frame
      JSR     ANIM_TICK_AND_WAIT     ; advance animation clock
      STOB    #MAP_WIDTH,BLINK_COL   ; disable blink cursor (sentinel = 20)
      RTS
```

Defines:
DRAW_MAP_ICON_B, used in chunk 542.

Uses ANIM_TICK_AND_WAIT 605b, BLINK_CHAR 544c, BLINK_COL 563b, BLINK_DELAY 544a, BLINK_ROW 544b, DRAW_BLINK_ALT 472a, DRAW_BLINK_NORMAL 473b, FONT_CHARNUM 575, FONT_COL 492a, FONT_ROW 534c, MAP_WIDTH 388a, MOVB 8c, RENDER_FONT_CHAR 466, SCROLL_STATUS_LINE 639, STOB 8c, TIMED_WAIT 606a, and WAIT_LOOP_COUNT 51.

12.0.13 RWTS command dispatch

DISK_READ and DISK_WRITE are thin wrappers that set the RWTS command code (IOB+$0C: 1=read, 2=write) and fall through to RWTS_DISPATCH.

RWTS_DISPATCH stores the command at IOB+$0C, clears the volume field at IOB+$03, then calls RWTS_ENTRY with the IOB pointer from $08/$09. Carry reflects the RWTS result (clear=success).

546 ⟨*disk read write* 546⟩≡ (909)

```
      ORG     $6C12
  DISK_READ:
      LDA     #$01                ; command = read
      BNE     RWTS_DISPATCH       ; always taken

  DISK_WRITE:
      LDA     #$02                ; command = write
                                  ; fall through
  RWTS_DISPATCH:
      SUBROUTINE RWTS_DISPATCH
      ; Dispatch an RWTS disk command.
      ;
      ; Inputs:
      ;   A | RWTS command ($01=read, $02=write)
      ;
      ; Behavior:
      ;   Stores the command at IOB+$0C, clears the volume field
      ;   at IOB+$03 (accept any volume), then calls RWTS_ENTRY.
      ;
      ; Outputs:
      ;   C | clear=success, set=error (from RWTS)
      ; Clobbers: A, Y

      LDY     #$0C
      STA     ($08),Y             ; IOB+$0C = command (A)
      LDA     #$00
      LDY     #$03
      STA     ($08),Y             ; IOB+$03 = volume (0 = any)
      LDA     $09                 ; \ A/Y = IOB pointer
      LDY     $08                 ; /
      JSR     RWTS_ENTRY          ; execute RWTS command
      STOB    #$00,$48            ; clear disk status
      RTS
```

Defines:
DISK_READ, used in chunks 298, 301, 305, and 662.
DISK_WRITE, used in chunks 298 and 303.
RWTS_DISPATCH, used in chunks 298 and 529.
Uses RWTS_ENTRY 770 and STOB 8c.

12.1 Scene data encoding

Scene text is stored on disk as 256-byte sectors, one per scene. `LOAD_SCENE_FROM_DISK` reads a scene into `SCENE_TEXT_DATA` ($5C00), and `TEXT_RENDERER` interprets the byte stream to display text on the hi-res screen using the custom 14×16 font.

Each scene is a sequence of character codes and control bytes:

- $00 = space.
- $01–$1A = letters A–Z.
- $1B–$24 = digits 0–9.
- $25 = comma, $26 = colon, $27 = hyphen, $28 = apostrophe, $2C = left paren, $2D = right paren, $2F = divider.
- $7D = reset to full-screen layout.
- $7E = page break (pause, play tune, refresh status bar).
- $7F = end of stream.
- $80–$FE = position command: bits 6–0 = row, next byte = column.

Scene sectors start at track $11, sector 0. Scene N maps to track $\$11 + \lfloor N/13 \rfloor$, sector $N \bmod 13$.

12.2 Scene $00: Copyright notice

Scene 0 is the copyright and publisher information screen, shown at startup. It is 106 bytes long (offsets $00–$69), terminated by $7F.

```
COPYRIGHT 1981          col 2

 STUART SMITH           col 3

  MARKETED BY           col 4

QUALITY SOFTWARE        col 2 (wraps at margin 18)
6660 RESEDA BLVD        col 2 (auto-wrap)
RESEDA, CA 91335        col 2 (auto-wrap)
(213) 344-6599          col 2 (auto-wrap)
```

Raw byte stream:

Offset	Hex	Meaning
$00	03 0F 10 19 12 09 07 08 14 00 1C 24 23 1C	"COPYRIGHT 1981"
$0E	82 03	Position: row 2, col 3
$10	13 14 15 01 12 14 00 13 0D 09 14 08	"STUART SMITH"
$1C	84 04	Position: row 4, col 4
$1E	0D 01 12 0B 05 14 05 04 00 02 19	"MARKETED BY"
$29	86 02	Position: row 6, col 2
$2B	11 15 01 0C 09 14 19 00 13 0F 06 14	"QUALITY SOFT"
$37	17 01 12 05 21 21 21 1B 00 12 05 13	"WARE6660 RES"
$43	05 04 01 00 02 0C 16 04 12 05 13 05	"EDA BLVDRESE"
$4F	04 01 25 00 03 01 00 24 1C 1E 1E 20	"DA, CA 91335"
$5B	2C 1D 1C 1E 2D 00 1E 1F 1F 27 21 20	"(213) 344-65"
$67	24 24	"99"
$69	7F	End of stream

The text wraps automatically between the default margins (columns 2 and 18). The position commands place each section at a specific row, centering the text visually. Note that Quality Software was Stuart Smith's publisher, located in Reseda, California.

Chapter 13

Mob Data

The region $4C80–$5AFF holds the mob data table (128 records × 16 bytes), associated name strings, and residual diagnostic text. This data is loaded from tracks $07–$09, DXR-decrypted during boot, and remains at these addresses throughout the game.

Chapter 14

Game Display

Draws the "PRESS SPACE BAR TO CONTINUE" (or "PRESS THE BUTTON TO CONTINUE") message in the text window at the bottom of the screen.

550 ⟨*display status bar* 550⟩≡ (909)

```
      ORG     $6D1F
  DISPLAY_STATUS_BAR:
      SUBROUTINE DISPLAY_STATUS_BAR
      ; Draw the status bar border and input-prompt string.
      ;
      ; Inputs:
      ;   INPUT_MODE          | 0 = keyboard, nonzero = joystick
      ;   STATUS_BORDER_DATA  | address of border-box text data
      ;
      ; Behavior:
      ;   Sends ctrl-P ($90) to HRCG to toggle inverse video, then draws
      ;   the border box via TEXT_RENDERER.  Sets text row to 23, column
      ;   to 4, and prints S_PRESS_SPACE ("PRESS SPACE BAR") when
      ;   INPUT_MODE is 0, or S_PRESS_BUTTON ("PRESS THE BUTTON")
      ;   otherwise.
      ;
      ; Modifies:
      ;   TEXT_STREAM_PTR, PRINT_STRING_ADDR, TEXT_ROW, TEXT_COL
      ; Clobbers: A
      LDA     #$90                              ; ctrl-P with high bit (toggle inverse)
      JSR     ROM_COUT1
      STOW    STATUS_BORDER_DATA,TEXT_STREAM_PTR ; status bar border data -> $06/$07
      JSR     TEXT_RENDERER                     ; draw border box
      JSR     PRINT_CTRL_AB                     ; Ctrl-A, '0'
      STOB    #$17,TEXT_ROW                     ; WNDBTM = 23

      LDA     INPUT_MODE                        ; 0 = keyboard, nonzero = joystick
```

```
    BNE     .joystick

    ; Keyboard path:
    STOB    #<S_PRESS_SPACE,PRINT_STRING_ADDR
    LDA     #>S_PRESS_SPACE
.store_hi_and_print:
    STA     PRINT_STRING_ADDR+1
.set_margin:
    STOB    #$04,TEXT_COL ; WNDLFT = 4
    JMP     PRINT_STRING                        ; print string, return

.joystick:
    STOB    #<S_PRESS_BUTTON,PRINT_STRING_ADDR
    LDA     #>S_PRESS_BUTTON
    JMP     .store_hi_and_print                 ; reuse STA PRINT_STRING_ADDR+1 from keyboard path
```

Defines:
DISPLAY_STATUS_BAR, used in chunks 527 and 644.

Uses INPUT_MODE 589c, PRINT_CTRL_AB 623a, PRINT_STRING 622, PRINT_STRING_ADDR 576c, ROM_COUT1 571a, STATUS_BORDER_DATA 490, STOB 8c, STOW 8a, S_PRESS_BUTTON 489b, S_PRESS_SPACE 572c, TEXT_COL 572a, TEXT_RENDERER 644, TEXT_ROW 572a, and TEXT_STREAM_PTR 571c.

SETUP_TEXT_POS and SETUP_TEXT_POS_COL20 configure the HRCG text window for the status panel. The first entry point sets column 0 (left side); the second sets column 20 (right side). Both set the cursor to row 20, send HRCG control codes A/B, output a carriage return ($8C) to position the cursor, fill 20 columns with spaces, then configure the text window width via SET_TEXT_WINDOW and output ctrl-K ($8B) to finalize.

552 ⟨*setup text pos* 552⟩≡ (909)

```
      ORG     $76D2
      SUBROUTINE SETUP_TEXT_POS
      ; Prepare an HRCG text window at row 20 for a status panel.
      ;
      ; Behavior:
      ;   SETUP_TEXT_POS sets the starting column to 0 (left panel);
      ;   SETUP_TEXT_POS_COL20 sets it to MAP_WIDTH (right panel).
      ;   Both entry points set TEXT_ROW to 20, send HRCG ctrl-AB and a
      ;   carriage return, fill 20 columns with hi-bit spaces, compute
      ;   TEXT_COL = 10 - TEXT_COL as a centering offset, call
      ;   PRINT_CENTERED_PLUS_COL, and finalize with ctrl-K.
      ;
      ; Modifies:
      ;   TEXT_COL, TEXT_ROW
      ; Clobbers: A, X
  SETUP_TEXT_POS:
      STOB    #$00,TEXT_COL
      BEQ     .set_row
  SETUP_TEXT_POS_COL20:          ; $76D8 - original SETUP_TEXT_POS_COL20 entry
      STOB    #MAP_WIDTH,TEXT_COL
  .set_row:
      STOB    #MAP_WIDTH,TEXT_ROW
      JSR     PRINT_CTRL_AB
      LDA     #$8C
      JSR     ROM_COUT1
      LDX     #MAP_WIDTH
  .space_loop:
      LDA     #$A0
      JSR     ROM_COUT1
      DEX
      BNE     .space_loop
      STOB    #MAP_WIDTH,TEXT_ROW
      LDA     #$0A
      SEC
      SBC     TEXT_COL
      STA     TEXT_COL
      JSR     PRINT_CENTERED_PLUS_COL
      LDA     #$8B
      JMP     ROM_COUT1
```

Defines:
SETUP_TEXT_POS, used in chunk 539.
SETUP_TEXT_POS_COL20, used in chunk 486a.

Uses MAP_WIDTH 388a, PRINT_CENTERED_PLUS_COL 629, PRINT_CTRL_AB 623a, ROM_COUT1 571a, STOB 8c, TEXT_COL 572a, and TEXT_ROW 572a.

14.1 Character stats display

DISPLAY_CHAR_STATS formats and displays the statistics of the character whose record is pointed to by ENTITY_PTR (ENTITY_PTR). It reads eight fields from the entity record, converts each to decimal via NUM_TO_DECIMAL, and writes the digits into a pre-formatted string template at STATS_TEMPLATE. After displaying the stats, it waits for a keypress (or joystick button if INPUT_MODE is nonzero), then exits via CLEAR_MAP.

- Field 6, bits 5–0: life force (max 63)
- Field 8, bits 3–0: group member count
- Field 7, bits 3–0: plus 1 = weapons
- Field 9, bits 3–0: plus 1 = ducats (gold)
- Field 5, bits 4–0: strength (max 31)
- Field 12, bits 3–0: sword rating
- Field 11, bits 4–0: level/armor
- Fields 13–14: 16-bit dagger/experience value

The routine first clears the digit positions in the template string by writing spaces ($A0), then fills them via NUM_TO_DECIMAL. After a cosmetic fix (suppressing a leading "0" in the experience field), it scrolls the status line, prints the formatted stats, and enters an input wait loop.

553 ⟨*defines* 25b⟩+≡ (909) ◁541a 555a▷

```
LOW_NIBBLE_MASK     EQU     $0F     ; bits 3-0: generic low nibble extraction
```

Defines:
LOW_NIBBLE_MASK, used in chunks 227, 232, 542, 556, and 602.

554 ⟨*stats template data* 554⟩≡ (909) 555b ▷

```
    ORG     $7D83
STATS_TEMPLATE:
    ; Pascal-format string: length $7D (125 bytes), body $7D84-$7E00.
    ; NUM_TO_DECIMAL writes 5 digit positions starting from each
    ; STATS_*_OUT address.  $FF bytes are line separators.
    DC.B    $7D                 ; length = 125 bytes
    ;   " LIFE FORCE 00"
    HEX     A0 CC C9 C6 C5 A0 C6 CF D2 C3 C5 A0
STATS_LIFE:                     ; $7D90
    HEX     B0 B0
    HEX     FF                  ; --- separator ---
    ;   CR " STRENGTH  00"
    HEX     8C A0 D3 D4 D2 C5 CE C7 D4 C8 A0 A0
STATS_STR:                      ; $7D9F
    HEX     B0 B0
    HEX     FF                  ; --- separator ---
    ;   VT "   WEAPONS  DUCATS  00000"
    HEX     8B A0 A0 A0 D7 C5 C1 D0 CF CE D3
    HEX     A0 A0 C4 D5 C3 C1 D4 D3 A0 A0
STATS_DUCATS:                   ; $7DB7 (16-bit, 5 digits)
    HEX     B0 B0 B0 B0 B0
    HEX     FF                  ; --- separator ---
    ;   CR " SPEED     00" VT
    HEX     8C A0 D3 D0 C5 C5 C4 A0 A0 A0 A0 A0
STATS_SPEED:                    ; $7DC9
    HEX     B0 B0
    HEX     8B
    HEX     FF                  ; --- separator ---
    ;   " SWORD  00  ARMOR       0"
    HEX     A0 D3 D7 CF D2 C4 A0 A0
STATS_SWORD:                    ; $7DD5
    HEX     B0 B0
    HEX     A0 A0 C1 D2 CD CF D2 A0 A0 A0
STATS_WEAP:                     ; $7DE1 (group count)
    HEX     A0 A0 A0 A0
STATS_ARMOR:                    ; $7DE5
    HEX     B0
    HEX     FF                  ; --- separator ---
    ;   CR " DEXTERITY 00" VT
    HEX     8C A0 C4 C5 D8 D4 C5 D2 C9 D4 D9 A0
STATS_DEXT:                     ; $7DF3
    HEX     B0 B0
    HEX     8B
    HEX     FF                  ; --- separator ---
    ;   " DAGGER 00"
    HEX     A0 C4 C1 C7 C7 C5 D2 A0
STATS_DAGGR:                    ; $7DFF
    HEX     B0 B0
```

Defines:

STATS_ARMOR, never used.
STATS_DAGGR, used in chunks 555a and 556.
STATS_DEXT, used in chunks 555a and 556.
STATS_DUCATS, used in chunks 555a and 556.
STATS_LIFE, used in chunks 555a and 556.
STATS_SPEED, used in chunks 555a and 556.
STATS_STR, used in chunks 555a and 556.
STATS_SWORD, used in chunks 555a and 556.
STATS_TEMPLATE, used in chunk 556.
STATS_WEAP, used in chunk 555a.
Uses NUM_TO_DECIMAL 632.

Since NUM_TO_DECIMAL writes five digit positions starting from DECIMAL_OUT_PTR (ten-thousands, thousands, hundreds, tens, ones), these EQUs mark where each stat's ten-thousands position falls in the template, so that the rightmost digits land on the placeholder "00"s.

555a ⟨*defines* 25b⟩+≡ (909) ◁553 561▷

```
STATS_LIFE_OUT      EQU     STATS_LIFE-3     ; $7D8D: life force output (2 digits at +3,+4)
STATS_STR_OUT       EQU     STATS_STR-3      ; $7D9C: strength output
STATS_DUCATS_OUT    EQU     STATS_DUCATS-1   ; $7DB6: ducats/experience output (5 digits at +0..+4)
STATS_SPEED_OUT     EQU     STATS_SPEED-3    ; $7DC6: speed output
STATS_SWORD_OUT     EQU     STATS_SWORD-3    ; $7DD2: sword output
STATS_WEAP_OUT      EQU     STATS_WEAP       ; $7DE1: weapons/group count output
STATS_DEXT_OUT      EQU     STATS_DEXT-3     ; $7DF0: dexterity output
STATS_DAGGR_OUT     EQU     STATS_DAGGR-3    ; $7DFC: dagger output
```

Defines:
STATS_DAGGR_OUT, used in chunk 556.
STATS_DEXT_OUT, used in chunk 556.
STATS_DUCATS_OUT, used in chunk 556.
STATS_LIFE_OUT, used in chunk 556.
STATS_SPEED_OUT, used in chunk 556.
STATS_STR_OUT, used in chunk 556.
STATS_SWORD_OUT, used in chunk 556.
STATS_WEAP_OUT, used in chunk 556.
Uses STATS_DAGGR 554, STATS_DEXT 554, STATS_DUCATS 554, STATS_LIFE 554, STATS_SPEED 554, STATS_STR 554, STATS_SWORD 554, and STATS_WEAP 554.

The input prompt strings at STATS_PROMPT_KEYBOARD and STATS_PROMPT_JOYSTICK are separate strings displayed after the stats.

555b ⟨*stats template data* 554⟩+≡ (909) ◁554

```
    ORG     $7E01
STATS_PROMPT_KEYBOARD:        ; $7E01
    APSTR "\x09PRESS M TO CONTINUE \x0E" ; \x09=tab, \x0E=ctrl-E (HRCG clear to EOL)
STATS_PROMPT_JOYSTICK:        ; $7E18
    APSTR "\x09PUSH BUTTON TO GO ON\x0E" ; \x09=tab, \x0E=ctrl-E (HRCG clear to EOL)
    ; (STATUS_BORDER_DATA follows immediately at [[STATUS_BORDER_DATA]])
```

Defines:
STATS_PROMPT_JOYSTICK, used in chunk 556.
STATS_PROMPT_KEYBOARD, used in chunk 556.
Uses STATUS_BORDER_DATA 490.

556 ⟨*display char stats* 556⟩≡ (909)

```
    ORG     $7780
DISPLAY_CHAR_STATS:
    SUBROUTINE DISPLAY_CHAR_STATS
    ; Format and display entity stats.
    ;
    ; Input:  ENTITY_PTR     -- mob/entity record
    ;         INPUT_MODE     -- 0 = keyboard, nonzero = joystick
    ; Output: screen updated with stats; waits for keypress
    ; Behavior:
    ;   - Reads 8 fields from entity record, converts to decimal
    ;   - Writes digits into STATS_TEMPLATE, displays it
    ;   - Shows mode-appropriate input prompt
    ;   - Polls keyboard/joystick until input received
    ;   - Exits via CLEAR_MAP
    ; Clobbers: A, X, Y

    ; --- Clear digit positions in the stats template ---
    LDA     #$A0              ; space (high-bit ASCII)
    STA     STATS_LIFE        ; life force digits
    STA     STATS_DUCATS      ; \
    STA     STATS_DUCATS+1    ;  | ducats digits
    STA     STATS_DUCATS+2    ; /
    STA     STATS_SWORD       ; sword digits
    STA     STATS_DAGGR       ; dagger digits
    STA     STATS_STR         ; strength digits
    STA     STATS_SPEED       ; speed digits
    STA     STATS_DEXT        ; dexterity digits

    ; --- Format life force: field 6, bits 5-0 ---
    LDA     #$00
    STA     NUM_VALUE+1                ; high byte = 0
    LDY     #MFIELD_HP
    LDA     (ENTITY_PTR),Y            ; field 6
    AND     #HP_MASK                   ; bits 5-0 = life force
    STA     NUM_VALUE
    STOW    STATS_LIFE_OUT,DECIMAL_OUT_PTR
    JSR     NUM_TO_DECIMAL

    ; --- Format group count: field 8, bits 3-0 ---
    LDY     #MFIELD_XP
    LDA     (ENTITY_PTR),Y            ; field 8
    AND     #LOW_NIBBLE_MASK           ; bits 3-0 = group count
    STA     NUM_VALUE
    STOW    STATS_WEAP_OUT,DECIMAL_OUT_PTR
    JSR     NUM_TO_DECIMAL

    ; --- Format weapons: field 7, bits 3-0, plus 1 ---
    LDY     #MFIELD_ATTACK
    LDA     (ENTITY_PTR),Y            ; field 7
```

```
AND     #LOW_NIBBLE_MASK           ; bits 3-0
CLC
ADC     #$01                       ; +1
STA     NUM_VALUE
STOW    STATS_SWORD_OUT,DECIMAL_OUT_PTR
JSR     NUM_TO_DECIMAL

; --- Format ducats: field 9, bits 3-0, plus 1 ---
LDY     #MFIELD_DEFENSE
LDA     (ENTITY_PTR),Y            ; field 9
AND     #LOW_NIBBLE_MASK           ; bits 3-0
CLC
ADC     #$01                       ; +1
STA     NUM_VALUE
STOW    STATS_DAGGR_OUT,DECIMAL_OUT_PTR
JSR     NUM_TO_DECIMAL

; --- Format strength: field 5, bits 4-0 ---
LDY     #MFIELD_STRENGTH
LDA     (ENTITY_PTR),Y            ; field 5
AND     #STRENGTH_MASK             ; bits 4-0 = strength
STA     NUM_VALUE
STOW    STATS_STR_OUT,DECIMAL_OUT_PTR
JSR     NUM_TO_DECIMAL

; --- Format armor: field 12, bits 3-0 ---
LDY     #MFIELD_LEVEL_ENC
LDA     (ENTITY_PTR),Y            ; field 12
AND     #LOW_NIBBLE_MASK           ; bits 3-0
STA     NUM_VALUE
STOW    STATS_SPEED_OUT,DECIMAL_OUT_PTR
JSR     NUM_TO_DECIMAL

; --- Format dexterity: field 11, bits 4-0 ---
LDY     #MFIELD_LEVEL
LDA     (ENTITY_PTR),Y            ; field 11
AND     #LEVEL_MASK                ; bits 4-0 = dexterity
STA     NUM_VALUE
STOW    STATS_DEXT_OUT,DECIMAL_OUT_PTR
JSR     NUM_TO_DECIMAL

; --- Format experience: fields 13-14 (16-bit) ---
LDY     #MFIELD_GOLD
LDA     (ENTITY_PTR),Y            ; field 13 (low byte)
STA     NUM_VALUE
INY
LDA     (ENTITY_PTR),Y            ; field 14 (high byte)
STA     NUM_VALUE+1
STOW    STATS_DUCATS_OUT,DECIMAL_OUT_PTR
JSR     NUM_TO_DECIMAL
```

```
    ; --- Cosmetic: if hundreds digit is blank and tens is "0", blank it ---
    LDA     STATS_DUCATS+2    ; hundreds digit
    CMP     #$A0              ; blank (leading-zero suppressed)?
    BNE     .show             ; no -> 3+ digits, show as-is
    LDA     STATS_DUCATS+3    ; tens digit
    CMP     #$B0              ; "0"?
    BNE     .show             ; no -> show as-is
    LDA     #$A0              ; replace "0" with blank
    STA     STATS_DUCATS+3

    ; --- Display the stats ---
.show:
    JSR     SCROLL_STATUS_LINE
    JSR     PRINT_CTRL_AB              ; save cursor state
    LDA     #$15                       ; \  cursor to column 21,
    STA     $25                        ;  | row 0 (right half of
    LDA     #$00                       ;  | screen)
    STA     $24                        ; /
    LDA     #<STATS_TEMPLATE           ; \ print the stats template
    STA     PRINT_STRING_ADDR          ;  | string at [[STATS_TEMPLATE]]
    LDA     #>STATS_TEMPLATE           ;  |
    STA     PRINT_STRING_ADDR+1        ; /
    JSR     PRINT_STRING               ; display stats
    LDA     #$14                       ; \  cursor to column 20,
    STA     $25                        ;  | row 0
    LDA     #$00                       ;  |
    STA     $24                        ; /

    ; --- Show input prompt based on mode ---
    LDA     INPUT_MODE
    BEQ     .keyboard                  ; 0 = keyboard mode
    LDX     #$18                       ; \ joystick: "PUSH BUTTON
    LDY     #$7E                       ; / TO GO ON" at [[STATS_PROMPT_JOYSTICK]]
    BNE     .print_prompt
.keyboard:
    LDX     #$01                       ; \ keyboard: "PRESS M TO
    LDY     #$7E                       ; / CONTINUE" at [[STATS_PROMPT_KEYBOARD]]
.print_prompt:
    JSR     PRINT_STRING_AT_ADDR       ; print prompt
    JSR     ANIM_TICK_AND_WAIT         ; tick animation + wait

    ; --- Wait for input ---
    STA     $C010                      ; clear keyboard strobe
.wait_key:
    LDA     $C000                      ; read keyboard
    BMI     .done                      ; key pressed -> exit
    LDA     INPUT_MODE
    BEQ     .wait_key                  ; keyboard mode -> loop
    LDX     #$00
```

```
    LDA     $C061,X                     ; read joystick button 0
    BMI     .done                       ; pressed -> exit
    INX
    LDA     $C061,X                     ; read joystick button 1
    BPL     .wait_key                   ; not pressed -> loop
.done:
    JMP     CLEAR_MAP                   ; exit via CLEAR_MAP
```

Defines:
DISPLAY_CHAR_STATS, used in chunk 265.

Uses ANIM_TICK_AND_WAIT 605b, CLEAR_MAP 539, DECIMAL_OUT_PTR 630a, ENTITY_PTR 561, INPUT_MODE 589c, LEVEL_MASK 411a, LOW_NIBBLE_MASK 553, MFIELD_ATTACK 561, MFIELD_DEFENSE 561, MFIELD_GOLD 561, MFIELD_HP 561, MFIELD_LEVEL 561, MFIELD_STRENGTH 561, MFIELD_XP 561, NUM_TO_DECIMAL 632, NUM_VALUE 630a, PRINT_CTRL_AB 623a, PRINT_STRING 622, PRINT_STRING_ADDR 576c, PRINT_STRING_AT_ADDR 621, SCROLL_STATUS_LINE 639, STATS_DAGGR 554, STATS_DAGGR_OUT 555a, STATS_DEXT 554, STATS_DEXT_OUT 555a, STATS_DUCATS 554, STATS_DUCATS_OUT 555a, STATS_LIFE 554, STATS_LIFE_OUT 555a, STATS_PROMPT_JOYSTICK 555b, STATS_PROMPT_KEYBOARD 555b, STATS_SPEED 554, STATS_SPEED_OUT 555a, STATS_STR 554, STATS_STR_OUT 555a, STATS_SWORD 554, STATS_SWORD_OUT 555a, STATS_TEMPLATE 554, STATS_WEAP_OUT 555a, and STOW 8a.

Chapter 15

Initialization

GAME_INIT sets up the Apple II for the game. It runs once at startup, before the main loop or attract screen. The steps, in order:

1. Set the reset vector to COLD_RESTART via the ROM routine at ROM_SETRES.
2. Disable the blink cursor by setting BLINK_COL to 20.
3. Call RELOCATE_HIRES_STAGING to copy the HRCG library and disk I/O routines from the staging area at $2000--$3FFF to their runtime addresses ($9600+ and $B300+), then clear the staging area to free the hi-res page for graphics.
4. Switch to hi-res graphics mode by setting the soft switches at TXTCLR (graphics), MIXCLR (full screen), PAGE1 (page 1), and HIRES (hi-res).
5. Call INIT_DISK_IOB to set up the disk I/O parameter block.
6. Call INIT_HRCG to patch and initialize the HRCG text engine.
7. Call SET_TEXT_WINDOW_WRAP to enable text window wrap mode.
8. Call INIT_STUB (a single RTS—likely a disabled hook point).
9. Call INIT_GAME_STATE to draw the game display (borders, labels, icons) and run the opening tune.
10. Call FILL_MAP to fill the map grid with default tiles.
11. Call SET_TEXT_WINDOW_BOTTOM to confine text output to the bottom three rows (the status/input area).
12. Call INIT_STUB2 (another no-op hook).

13. Clear state variables to zero:
    - LOCATION_POS—the last-drawn location identifier. The main loop compares this against the current character's location; when they differ, it triggers a full map redraw. Zeroing it here forces the first ENTER_LOCATION call to redraw unconditionally.
    - LOCATION_STYLE—the current map tile style (which tile set to use when drawing the map). Zeroing ensures it will be reloaded from the location record on the first scene entry.
    - ENTITY_PTR+1—the high byte of the entity record pointer (ENTITY_PTR). Zeroing signals "no entity selected," preventing stale pointer references before the first character is processed.
14. Call SELECT_INPUT_MODE to detect keyboard or joystick.
15. Call INIT_WORLD to initialize the game world from its data table.
16. Call INIT_MOB_COUNT to count all active mob groups.
17. Set CURRENT_PLAYER to 1. The main loop walks the 62-entry record table once per round, using CURRENT_PLAYER as a 1-based index. Initializing it to 1 starts the iteration at the first record.

The following state variables are referenced by GAME_INIT. BLINK_COL is the cursor blink column; storing $14 disables the blink. LOCATION_POS, LOCATION_STYLE, and CURRENT_PLAYER are the three variables cleared to zero during step 13/17 (described above). ENTITY_PTR (ENTITY_PTR) is the 2-byte pointer to the entity record currently being processed.

561 ⟨*defines* 25b⟩+≡ (909) ◁555a 571a▷

```
ENTITY_PTR          EQU     $F8     ; entity record pointer (2 bytes, $F8/$F9)

    ; --- Character index record field offsets (9-byte, via CHAR_PTR) ---
CFIELD_NEXT         EQU     $02     ; byte 2: next link pointer
CFIELD_POSITION     EQU     $03     ; byte 3: map position (packed col/row)
CFIELD_APPEARANCE   EQU     $04     ; byte 4: appearance index
CFIELD_STYLE        EQU     $05     ; byte 5: location style (bits 6-0)
CFIELD_EVENTS       EQU     $06     ; bytes 6-7: pointer to event list (2 bytes)
CFIELD_XP           EQU     $08     ; byte 8: status (7) | pending XP (6-4) | group count (3-0)
CFIELD_FACTION_MASK EQU     $60     ; bits 6-5 of byte 5: faction identifier

    ; --- Mob data record field offsets (16-byte, via ENTITY_PTR/MOB_PTR) ---
MFIELD_TYPE         EQU     $01     ; byte 1: entity type/class (for CLASSIFY_ENTITY_TYPE)
MFIELD_NEXT         EQU     $02     ; byte 2: next link pointer
MFIELD_POSITION     EQU     $03     ; byte 3: map position (packed col/row)
MFIELD_APPEARANCE   EQU     $04     ; byte 4: appearance index
MFIELD_STRENGTH     EQU     $05     ; byte 5: bits 4-0 = strength / bits 4-1 = defense
MFIELD_HP           EQU     $06     ; byte 6: bits 5-0 = hit points, bits 7-6 = flags
MFIELD_ATTACK       EQU     $07     ; byte 7: attack stat
```

```
MFIELD_XP           EQU     $08     ; byte 8: status (7) | pending XP (6-4) | group count (3-0)
MFIELD_DEFENSE      EQU     $09     ; byte 9: defense stat
MFIELD_LEVEL        EQU     $0B     ; byte 11: class/faction (7-5) | level (4-0)
MFIELD_LEVEL_ENC    EQU     $0C     ; byte 12: level encoding (hi nib = XP mult, lo nib = base)
MFIELD_GOLD         EQU     $0D     ; bytes 13-14: gold counter (16-bit, lo/hi)
MFIELD_CHAR         EQU     $00     ; byte 0: display character
MFIELD_NAME         EQU     $00     ; bytes 0-1: name pointer (16-bit, lo/hi)
MFIELD_FLAGS        EQU     $0F     ; byte 15: status flags (bit 2 = combat engaged)
HP_FLAGS_MASK       EQU     $C0     ; byte 6: mask for flags bits 7-6 (above HP)
```

Defines:
CFIELD_APPEARANCE, used in chunks 478 and 481.
CFIELD_EVENTS, used in chunks 398, 424, 454, and 469.
CFIELD_FACTION_MASK, used in chunks 419 and 420.
CFIELD_NEXT, used in chunks 222, 315, 426, 478, 482, and 542.
CFIELD_POSITION, never used.
CFIELD_STYLE, used in chunks 419, 420, 533a, 535, 541a, and 542.
CFIELD_XP, used in chunks 373, 385, 420, and 422.
ENTITY_PTR, used in chunks 56, 222, 224, 225, 227, 232, 239, 241, 242, 244–46, 308, 320, 323, 325, 327, 328, 334, 338, 342, 346, 352, 354, 357, 360, 365, 368, 377, 378, 382, 385, 390, 394, 398, 400, 409, 412, 433, 436, 438, 440, 444, 447, 450, 484, 486a, 487, 489a, 535, 556, 564, 609, 618–20, and 898.
HP_FLAGS_MASK, used in chunk 609.
MFIELD_APPEARANCE, used in chunks 69–196, 225, 317, 334, 346, 349, 354, 373, 377, 385, 447, 462, 475, 542, 612, and 615.
MFIELD_ATTACK, used in chunks 69–196, 325, 409, and 556.
MFIELD_DEFENSE, used in chunks 69–196, 325, 409, and 556.
MFIELD_GOLD, used in chunks 56, 69–196, 349, 380, 447, 494, and 556.
MFIELD_HP, used in chunks 69–196, 222, 232, 312, 338, 346, 349, 377, 378, 385, 556, and 609.
MFIELD_LEVEL, used in chunks 69–196, 336, 338, 411a, 412, 447, 450, 478, and 556.
MFIELD_NAME, used in chunks 69–196 and 613.
MFIELD_NEXT, used in chunks 69–196, 222, 315, 334, 342, 394, 426, 447, 459, 462, 469, 478, 482, 612, and 615.
MFIELD_POSITION, used in chunks 69–196, 227, 232, 239, 244, 315, 325, 338, 349, 354, 357, 365, 368, 385, 390, 394, 400, 469, 475, 487, 494, 542, and 898.
MFIELD_STRENGTH, used in chunks 61a, 69–196, 320, 322, 338, 354, 357, 368, 380, 385, 409, 450, 556, 609, and 898.
MFIELD_XP, used in chunks 69–196, 219, 346, 411a, 412, 481, and 556.
Uses CHAR_PTR 591a, CLASSIFY_ENTITY_TYPE 459, and MOB_PTR 608.

562a ⟨*var current player* 562a⟩≡ (909)

```
    ORG     $5A02
CURRENT_PLAYER:
    DC.B    $8D                     ; initial value (runtime: current player index (1-based))
```

Defines:
CURRENT_PLAYER, used in chunks 61a, 219, 283, 349, 354, 417b, 420, 423, 428, 431, 489a, 494, and 564.

562b ⟨*var location pos* 562b⟩≡ (909)

```
    ORG     $5A55
LOCATION_POS:
    DC.B    $A0                     ; initial value (runtime: current location position (triggers re
```

Defines:
LOCATION_POS, used in chunks 220, 429, 527, 535, and 564.

563a ⟨*var location style* 563a⟩≡ (909)

```
    ORG     $5A56
LOCATION_STYLE:
    DC.B    $A3                     ; initial value (runtime: current map style (from record field 5
```

Defines:
LOCATION_STYLE, used in chunks 404, 527, 531a, 535, 541a, 542, and 564.

563b ⟨*var blink col* 563b⟩≡ (909)

```
    ORG     $5AA2
BLINK_COL:
    DC.B    $C1                     ; initial value (runtime: font column (0-19; $14 = disabled))
```

Defines:
BLINK_COL, used in chunks 222, 239, 354, 473b, 475, 487, 527, 545, and 564.

564 ⟨*game init* 564⟩≡ (909)

```
      ORG     $78C7
  GAME_INIT:
      SUBROUTINE GAME_INIT
      ; Initialize the game from cold start.
      ;
      ; Behavior:
      ;   1. Sets reset vector to $A56F and disables blink cursor.
      ;   2. Relocates hi-res staging data to runtime addresses ($9600+, $B300+).
      ;   3. Switches to hi-res graphics mode and initializes HRCG, disk IOB,
      ;      text window, game display, map, and input mode.
      ;   4. Clears location/entity state, inits world and mob count.
      ;
      ; Modifies: reset vector ($03F2), BLINK_COL, LOCATION_POS, LOCATION_STYLE,
      ;   ENTITY_PTR+1, CURRENT_PLAYER, DISPATCH_VEC, display softswitches
      ; Clobbers: A, X, Y

      LDX     #$6F                   ; \
      LDY     #$A5                   ;  | reset vector = $A56F
      STX     $03F2                  ;  |
      STY     $03F3                  ; /
      JSR     $FB6F                  ; ROM: set reset vector
      STOB    #$14,BLINK_COL ; disable blink cursor
      JSR     RELOCATE_HIRES_STAGING ; copy $2000+ to $9600+/$B300+
      LDA     $C050                  ; graphics mode
      LDA     $C052                  ; full screen (no text split)
      LDA     $C054                  ; page 1
      LDA     $C057                  ; hi-res
      JSR     INIT_DISK_IOB
      JSR     INIT_HRCG
      JSR     SET_TEXT_WINDOW_WRAP
      JSR     INIT_STUB
      JSR     INIT_GAME_STATE
      JSR     FILL_MAP
      JSR     SET_TEXT_WINDOW_BOTTOM
      JSR     INIT_STUB2
      STOB    #$00,LOCATION_POS ;  clear state variables
      STA     LOCATION_STYLE         ;  |
      STA     ENTITY_PTR+1           ; /
      JSR     SELECT_INPUT_MODE
      JSR     INIT_WORLD
      JSR     INIT_MOB_COUNT
      STOB    #$01,CURRENT_PLAYER
      RTS
```

Defines:
GAME_INIT, used in chunk 219.

Uses BLINK_COL 563b, CURRENT_PLAYER 562a, DISPATCH_VEC 569b, ENTITY_PTR 561, FILL_MAP 538, INIT_DISK_IOB 589b, INIT_GAME_STATE 570b, INIT_HRCG 568, INIT_MOB_COUNT 592, INIT_STUB 569a, INIT_STUB2 588b, LOCATION_POS 562b, LOCATION_STYLE 563a, RELOCATE_HIRES_STAGING 566, SELECT_INPUT_MODE 590,

SET_TEXT_WINDOW_BOTTOM 627, SET_TEXT_WINDOW_WRAP 625, and STOB 8c.

During startup, tracks $0F and $10 are loaded into the hi-res page area $2000--$3FFF as a staging area. RELOCATE_HIRES_STAGING, called from GAME_INIT, copies the staged data to its runtime addresses and clears the hi-res page:

Source	Destination	Size
$2000–$32FF	$9600–$A8FF	19 pages (HRCG code + standard font)
$3300–$3FFF	$B300–$BFFF	13 pages (disk I/O routines)

The copy loop at .copy also clears each source byte, leaving the hi-res page zeroed for use as a graphics buffer.

RELOCATE_HIRES_STAGING uses two zero-page pointer pairs for its block copy: COPY_SRC (PRINT_STRING_ADDR) points to the source address in the staging area, and COPY_DEST (MOB_DATA_PTR) points to the runtime destination. The loop copies one byte at a time, zeroing the source as it goes.

565 ⟨*zero page defines* 25a⟩+≡ (909) ◁541b 569b▷

```
COPY_SRC        EQU     $BC     ; block copy source pointer (2 bytes)
COPY_DEST       EQU     $BA     ; block copy destination pointer (2 bytes)
```

Defines:
COPY_DEST, used in chunk 566.
COPY_SRC, used in chunk 566.

566 ⟨*relocate hires staging* 566⟩≡ (909)

```
    ORG     $6DA5
RELOCATE_HIRES_STAGING:
    SUBROUTINE RELOCATE_HIRES_STAGING
    ; Block-copy staged data from hi-res page to runtime addresses.
    ;
    ; Behavior:
    ;   1. Copies $2000-$32FF to $9600-$A8FF (19 pages: HRCG + font).
    ;   2. Copies $3300-$3FFF to $B300-$BFFF (13 pages: disk I/O).
    ;   3. Zeroes each source byte after copying, clearing the hi-res page.
    ;
    ; Modifies: COPY_SRC, COPY_DEST, $2000-$3FFF (zeroed), $9600-$A8FF, $B300-$BFFF
    ; Clobbers: A, X, Y

    STOB    #$00,COPY_DEST ; dest = $9600
    STA     COPY_SRC       ; \  src  = $2000
    STOB    #$20,COPY_SRC+1 ; src hi
    STOB    #$96,COPY_DEST+1 ; dest hi
    LDY     #$00
    LDX     #$13           ; 19 pages: $2000-$32FF -> $9600-$A8FF
    JSR     .copy
    LDX     #$0D           ; 13 pages: $3300-$3FFF -> $B300-$BFFF
    STOB    #$B3,COPY_DEST+1 ; dest hi = $B3
    INC     COPY_SRC+1     ; src hi = $33 (was $32 after first pass)

.copy:
    LDA     (COPY_SRC),Y   ; load from source
    STA     (COPY_DEST),Y  ; store to dest
    LDA     #$00
    STA     (COPY_SRC),Y   ; clear source byte
    DEY
    BNE     .copy          ; inner loop (256 bytes)
    DEX
    BNE     .next_page     ; more pages?
    RTS

.next_page:
    INC     COPY_SRC+1     ; next source page
    INC     COPY_DEST+1    ; next dest page
    JMP     .copy
```

Defines:
RELOCATE_HIRES_STAGING, used in chunk 564.
Uses COPY_DEST 565, COPY_SRC 565, and STOB 8c.

15.0.1 Zeroed hi-res page

After RELOCATE_HIRES_STAGING copies $2000–$32FF to $9600–$A8FF and $3300–$3FFF to $B300–$BFFF, it clears each source byte to zero. The result is 8192 bytes of zeros at $2000–$3FFF, making the hi-res page available for graphics.

567 ⟨*zeroed hires page* 567⟩≡ (909)

```
      ORG     $2000
  HIRES_PAGE:
      DS      $2000,$00          ; 8192 bytes zeroed by game_init
```

Defines:
HIRES_PAGE, never used.

The plain text screen (using the standard Apple text characters) is 40 columns by 24 rows. The TEXT_COL and TEXT_ROW locations keep track of the cursor position when ROM routines are used for printing text.

We can set the text window to start either from row 0 using SET_TEXT_WINDOW_UPPER_LEFT_ALL, or from row 21 using SET_TEXT_WINDOW_UPPER_LEFT_LOW. Using the bottom of the screen allows us to use it as a status and input window while the map is displayed above.

Initializes the Hi-Res Character Generator (HRCG) by patching two locations in the HRCG code and then jumping to its entry point at HRCG_INIT. The patches modify the HRCG initialization to work within the game's environment:

- $94B6 ← RTS ($60): causes HRCG init to return early, skipping the remainder of its setup.
- $92D5--$92D7 ← three NOPs ($EA): patches out a JSR $03EA that would reconnect DOS, which the game does not want.

After patching, the routine tail-calls the HRCG init via JMP HRCG_INIT, which returns to GAME_INIT via the patched RTS.

INIT_HRCG patches two locations in the HRCG library before calling its initialization entry point. First, it writes an RTS opcode at $94B6, causing the HRCG's internal initialization to return early—skipping setup steps that the game handles differently. Second, it writes three NOP opcodes over the JSR at $92D5, disabling the HRCG's attempt to reconnect DOS (which would conflict with the game's custom disk routines). With these patches in place, the routine jumps to HRCG_INIT at $92A8 to complete the HRCG startup.

568 ⟨*init hrcg* 568⟩≡ (909)

```
      ORG     $791A
  INIT_HRCG:
      SUBROUTINE INIT_HRCG
      ; Patch and initialize the HRCG character generator library.
      ;
      ; Behavior:
      ;   1. Patches $94B6 to RTS and $92D5-$92D7 to NOPs.
      ;   2. Tail-calls HRCG_INIT to complete initialization.
      ;
      ; Modifies: $94B6, $92D5-$92D7
      ; Clobbers: A

      STOB    #$60,$94B6 ; RTS opcode
      STOB    #$EA,$92D5 ; NOP opcode
      STA     $92D6     ;  | (NOP out DOS reconnect)
      STA     $92D7     ; /
      JMP     HRCG_INIT ; tail call: enter HRCG initialization
```

Defines:
INIT_HRCG, used in chunk 564.
Uses HRCG_INIT 736 and STOB 8c.

A single RTS—a no-op stub called from GAME_INIT. Likely a hook point or a disabled feature from development.

569a ⟨*init stub* 569a⟩≡ (909)

```
    ORG     $7919
INIT_STUB:
    RTS
```

Defines:
INIT_STUB, used in chunk 564.

Called once from GAME_INIT to set up the game display and run the opening tune. Steps:

1. Calls INIT_GAME_DISPLAY to initialize the game display (draws borders, sets text windows).
2. Sets DISPATCH_VEC to point to the game action handler at GAME_ACTION_HANDLER, used by the tune player's indirect dispatch at TUNE_ACTION_INDIRECT.
3. Clears INPUT_MODE (game state flag).
4. Patches CHAR_CHARSET to $B1 (LDA (zp),Y opcode), restoring the character renderer's indirect load.
5. Runs the opening tune (index 6) via TUNE_PLAYER.
6. Patches $7A49--$7A4B from JMP (DISPATCH_VEC) to JMP TUNE_ADVANCE, disabling the indirect action handler now that init is complete.

INIT_GAME_STATE stores GAME_ACTION_HANDLER into the zero-page vector DISPATCH_VEC (EFFECT_REC). The tune player uses this vector for indirect action dispatch; after the opening tune finishes, INIT_GAME_STATE patches the dispatch site to bypass the vector entirely.

GAME_ACTION_HANDLER is defined in <<border animation>> at GAME_ACTION_HANDLER.

569b ⟨*zero page defines* 25a⟩+≡ (909) ◁565 571c▷

```
DISPATCH_VEC            EQU     $F6     ; tune action dispatch vector (2 bytes, $F6/$F7)
```

Defines:
DISPATCH_VEC, used in chunks 564, 570b, 584, and 682.

Copyright notice string, first used by INIT_GAME_STATE.

570a ⟨*copyright string* 570a⟩≡ (909)

```
    ORG     $5EC7
S_COPYRIGHT:
    APSTR "COPYRIGHT 1982 STUART SMITH"
```

Defines:
S_COPYRIGHT, used in chunk 574.

570b ⟨*init game state* 570b⟩≡ (909)

```
    ORG     $5B00
INIT_GAME_STATE:
    SUBROUTINE INIT_GAME_STATE
    ; Set up game display, dispatch vector, and play opening tune.
    ;
    ; Behavior:
    ;   1. Calls INIT_GAME_DISPLAY to draw the main screen layout.
    ;   2. Sets DISPATCH_VEC to GAME_ACTION_HANDLER.
    ;   3. Patches character renderer at [[CHAR_CHARSET]] and jump at $7A49-$7A4B.
    ;   4. Plays tune 6 (attract/opening music).
    ;
    ; Modifies: DISPATCH_VEC, INPUT_MODE, $80A6, $7A49-$7A4B
    ; Clobbers: A, X, Y

    JSR     INIT_GAME_DISPLAY     ; set up game display / UI
    STOW    GAME_ACTION_HANDLER,DISPATCH_VEC ; $F6/DISPATCH_VEC+1 = game action handler
    STOB    #$00,$5A00 ; clear game state flag
    LDX     #$B1                  ; LDA (zp),Y opcode
    STX     $80A6                 ; patch character renderer
    LDX     #$06                  ; tune 3: attract/opening
    JSR     TUNE_PLAYER
    STOW    $4F4C,$7A49 ; patch $7A49 to JMP TUNE_ADVANCE
INPUT_SCAN_OFFSET:                      ; $5B24: also scan position index (runtime storage)
    LDA     #$7A                   ; (also UNDER_LEVEL at [[UNDER_LEVEL]])
SAFE_TO_REST:                           ; $5B26: also 1 if no adjacent threats (runtime storage)
    STA     $7A4B                  ; (also TURN_ACTIVE at [[TURN_ACTIVE]])
    RTS

UNDER_LEVEL     EQU INPUT_SCAN_OFFSET+1 ; $5B25: 1 if char level < steps taken (runtime storage)
TURN_ACTIVE     EQU SAFE_TO_REST+1      ; $5B27: 1 while turn is in progress (runtime storage)
```

Defines:
INIT_GAME_STATE, used in chunk 564.
INPUT_SCAN_OFFSET, used in chunks 255 and 256.
SAFE_TO_REST, used in chunks 261, 264, and 308.
TURN_ACTIVE, used in chunks 232, 241, and 264.
UNDER_LEVEL, used in chunks 261, 264, and 308.
Uses CHAR_CHARSET 465, DISPATCH_VEC 569b, GAME_ACTION_HANDLER 584, INIT_GAME_DISPLAY 574, INPUT_MODE 589c, STOB 8c, STOW 8a, TUNE_ADVANCE 683, and TUNE_PLAYER 682.

INIT_GAME_DISPLAY draws the game's main screen layout. It renders the background from the text stream at UI_TEXT_STREAM, draws 2×2 border characters, places text labels via DRAW_BORDER_LABELS, adds corner decorations via DRAW_BORDER_ICONS, and finishes with the copyright notice (S_COPYRIGHT) and "press space bar" prompt (S_PRESS_SPACE_BOOT). The decorative strings S_DECOR_ROW1 through S_DECOR_ROW3 are drawn on the right panel.

UI_TEXT_STREAM is the label at UI_TEXT_STREAM defined in <<ui text stream>>.

All character output in the game passes through ROM_COUT1, the Apple II ROM's character output routine at ROM_COUT1. When the HRCG is active, ROM_COUT1 is redirected through the CSW vector to HRCG_COUT, which renders characters in hi-res graphics mode.

571a ⟨*defines* 25b⟩+≡ (909) ◁561 571b▷

```
ROM_COUT1           EQU     $FDED   ; Apple II ROM COUT1 (character output)
ROM_PREAD           EQU     $FB1E   ; Apple II ROM PREAD (read paddle)
ROM_SETRES          EQU     $FB6F   ; Apple II ROM SETRES (set reset vector)
```

Defines:
ROM_COUT1, used in chunks 259, 550, 552, 574, 578, 622–27, 639, 736, 802, and 803.
ROM_PREAD, never used.
ROM_SETRES, never used.

571b ⟨*defines* 25b⟩+≡ (909) ◁571a 576d▷

```
KBD_DATA            EQU     $C000   ; keyboard data register (bit 7 = key available)
KBD_STROBE          EQU     $C010   ; keyboard strobe (clear bit 7)
SPEAKER             EQU     $C030   ; speaker toggle
PADDL0              EQU     $C061   ; paddle button 0
TXTCLR              EQU     $C050   ; display soft-switch: graphics mode
MIXCLR              EQU     $C052   ; display soft-switch: full screen (no text window)
PAGE1               EQU     $C054   ; display soft-switch: page 1
HIRES               EQU     $C057   ; display soft-switch: hi-res mode
```

Defines:
HIRES, used in chunk 837.
KBD_DATA, never used.
KBD_STROBE, never used.
MIXCLR, used in chunk 837.
PADDL0, never used.
PAGE1, never used.
SPEAKER, never used.
TXTCLR, used in chunk 837.

571c ⟨*zero page defines* 25a⟩+≡ (909) ◁569b 572a▷

```
TEXT_STREAM_PTR         EQU     $06     ; text stream pointer for TEXT_RENDERER (2 bytes)
```

Defines:
TEXT_STREAM_PTR, used in chunks 527, 550, 574, and 644.
Uses TEXT_RENDERER 644.

572a ⟨*zero page defines* 25a⟩+≡ (909) ◁571c 576c▷

```
TEXT_ROW                EQU     $25
TEXT_COL                EQU     $24     ; Cursor position
```

Defines:
TEXT_COL, used in chunks 49, 253, 254, 259, 467, 550, 552, 574, 576b, 578, 584, 626–29, 639, and 893b.
TEXT_ROW, used in chunks 49, 253, 254, 259, 467, 550, 552, 574, 576b, 578, 584, 626–28, 639, and 893b.

Boot prompt and decorative border font strings, printed by INIT_GAME_DISPLAY along the right panel.

572b ⟨*boot display strings* 572b⟩≡ (909)

```
    ORG     $5EE3
S_PRESS_SPACE_BOOT:
    APSTR "PRESS SPACE BAR TO CONTINUE"
S_DECOR_ROW1:
    DC.B    $07
    HEX     E8 E9 EA EB EC ED EE
S_DECOR_ROW2:
    DC.B    $08
    HEX     EF F0 F1 F2 F3 F4 F5 F6
S_DECOR_ROW3:
    DC.B    $08
    HEX     F7 F8 F9 FA FB FC FD FE
```

Defines:
S_DECOR_ROW1, used in chunk 574.
S_DECOR_ROW2, used in chunk 574.
S_DECOR_ROW3, used in chunk 574.
S_PRESS_SPACE_BOOT, used in chunk 574.

The "press space bar" prompt displayed by INIT_GAME_DISPLAY in the status window.

572c ⟨*press space string* 572c⟩≡ (909)

```
    ORG     $7E97
S_PRESS_SPACE:
    ; Length $22 (34): CR + "  PRESS SPACE BAR TO CONTINUE   " + CLR
    DC.B    $22,$8C
    HEX     A0 A0 D0 D2 C5 D3 D3 A0 D3 D0 C1 C3 C5 A0 C2 C1
    HEX     D2 A0 D4 CF A0 C3 CF CE D4 C9 CE D5 C5 A0 A0 A0
    DC.B    $8B
```

Defines:
S_PRESS_SPACE, used in chunk 550.

The UI text stream is a concatenated display string parsed by TEXT_RENDERER during INIT_GAME_DISPLAY. It contains HRCG control codes, border label position tables (used by DRAW_BORDER_LABELS), the frame pattern, copyright text, and decorative font characters.

The UI text stream begins with HRCG control codes and viewport configuration, parsed by TEXT_RENDERER when INIT_GAME_DISPLAY sets TEXT_STREAM_PTR to UI_TEXT_STREAM.

573 ⟨*ui text stream* 573⟩≡ (909)

```
    ORG     $5E55
UI_TEXT_STREAM:
    DC.B    $7D                 ; HRCG reset to full screen
    DC.B    $80,$02             ; mode control
    ; Map viewport bounds (2 sets)
    DC.B    $2F,$00,$00,$01,$0C,$09,$00,$00,$02,$01,$02,$01,$00,$00
    DC.B    $2F,$83,$01,$14,$08,$05,$00,$06,$0F,$12,$14,$19,$00,$14
    DC.B    $08,$09,$05,$16,$05,$13
    DC.B    $7F
```

Defines:
UI_TEXT_STREAM, used in chunk 574.

574 ⟨*init game display* 574⟩≡ (909)

```
      ORG     $5C6D
  INIT_GAME_DISPLAY:
      SUBROUTINE INIT_GAME_DISPLAY
      ; Draw the game's main screen layout with borders and decorations.
      ;
      ; Behavior:
      ;   1. Clears the hi-res screen, renders UI background from UI_TEXT_STREAM.
      ;   2. Draws 2x2 border characters at fixed positions and border labels/icons.
      ;   3. Prints copyright string (row 21) and press-space prompt (row 23).
      ;   4. Switches to charset 5 and draws decorative strings on the right panel.
      ;
      ; Modifies: TEXT_ROW, TEXT_COL, TEXT_STREAM_PTR, FONT_CHARNUM, FONT_CHARSET
      ; Clobbers: A, X, Y

      LDA     #$90                          ; HRCG control: clear hi-res screen
      JSR     ROM_COUT1
      stow    UI_TEXT_STREAM,TEXT_STREAM_PTR ; point text stream at UI data
      JSR     TEXT_RENDERER                 ; render game UI background

      ; Draw three 2x2 border characters across top (charset 2, row 3)
      LDA     #$01                          ; char 1
      LDX     #$10                          ; col 16
      JSR     DRAW_BORDER_CHAR
      LDA     #$0E                          ; char 14
      LDX     #$12                          ; col 18
      JSR     DRAW_BORDER_CHAR
      LDA     #$04                          ; char 4
      LDX     #$14                          ; col 20
      JSR     DRAW_BORDER_CHAR

      JSR     DRAW_BORDER_LABELS            ; draw border text labels
      JSR     DRAW_BORDER_ICONS             ; draw corner decorations

      JSR     PRINT_CTRL_AB                 ; reset HRCG block mode

      ; Print copyright at row 21, col 0
      STOB    #$15,TEXT_ROW
      STOB    #$00,TEXT_COL
      LDX     #<S_COPYRIGHT
      LDY     #>S_COPYRIGHT
      JSR     PRINT_STRING_AT_ADDR

      ; Print "PRESS SPACE BAR TO CONTINUE" at row 23, col 0
      STOB    #$17,TEXT_ROW
      STOB    #$00,TEXT_COL
      LDX     #<S_PRESS_SPACE_BOOT
      LDY     #>S_PRESS_SPACE_BOOT
      JSR     PRINT_STRING_AT_ADDR
```

```
; Switch to HRCG charset 5 for decorative chars
LDA     #$81                            ; ctrl-A (select charset)
JSR     ROM_COUT1
LDA     #$B5                            ; '5' = charset 5
JSR     ROM_COUT1

; Print decorative strings at col 30, rows 21-23
STOB    #$15,TEXT_ROW
STOB    #$1E,TEXT_COL
LDX     #<S_DECOR_ROW1
LDY     #>S_DECOR_ROW1
JSR     PRINT_STRING_AT_ADDR

INC     TEXT_ROW
STOB    #$1E,TEXT_COL
LDX     #<S_DECOR_ROW2
LDY     #>S_DECOR_ROW2
JSR     PRINT_STRING_AT_ADDR

INC     TEXT_ROW
STOB    #$1E,TEXT_COL
LDX     #<S_DECOR_ROW3
LDY     #>S_DECOR_ROW3
JMP     PRINT_STRING_AT_ADDR
```

Defines:
INIT_GAME_DISPLAY, used in chunk 570b.
Uses DRAW_BORDER_CHAR 576b, DRAW_BORDER_ICONS 580, DRAW_BORDER_LABELS 578, FONT_CHARNUM 575, FONT_CHARSET 576a, PRINT_CTRL_AB 623a, PRINT_STRING_AT_ADDR 621, ROM_COUT1 571a, STOB 8c, S_COPYRIGHT 570a, S_DECOR_ROW1 572b, S_DECOR_ROW2 572b, S_DECOR_ROW3 572b, S_PRESS_SPACE_BOOT 572b, TEXT_COL 572a, TEXT_RENDERER 644, TEXT_ROW 572a, TEXT_STREAM_PTR 571c, and UI_TEXT_STREAM 573.

DRAW_BORDER_CHAR is a helper for INIT_GAME_DISPLAY. It draws a 2×2 font character from charset 2 at column X, row 3. The character number is passed in A. It stores the character number into FONT_CHARNUM and the charset into FONT_CHARSET, then calls PRINT_FONTCHAR_AT_TEXT_POS to render it.

575 ⟨*var font charnum* 575⟩≡ (909)

```
    ORG     $5A0E
FONT_CHARNUM:
    DC.B    $8D                     ; initial value (runtime: font character number to render)
```

Defines:
FONT_CHARNUM, used in chunks 49, 239, 354, 462, 466, 473b, 475, 487, 538, 542, 545, 574, 576b, 584, and 644.

576a ⟨*var font charset* 576a⟩≡ (909)

```
    ORG     $5A0F
FONT_CHARSET:
    DC.B    $A0                     ; initial value (runtime: font charset index (1-5))
```

Defines:
FONT_CHARSET, used in chunks 49, 239, 462, 466, 473b, 487, 538, 542, 574, 576b, and 644.

576b ⟨*draw border char* 576b⟩≡ (909)

```
    ORG     $5CEC
DRAW_BORDER_CHAR:

    STX     TEXT_COL
    STA     FONT_CHARNUM
    LDY     #$03
    STY     TEXT_ROW
    STOB    #$02,FONT_CHARSET
    JMP     PRINT_FONTCHAR_AT_TEXT_POS
```

Defines:
DRAW_BORDER_CHAR, used in chunk 574.
Uses FONT_CHARNUM 575, FONT_CHARSET 576a, PRINT_FONTCHAR_AT_TEXT_POS 466, STOB 8c, TEXT_COL 572a, and TEXT_ROW 572a.

DRAW_BORDER_LABELS draws text labels around the game border. Three loops iterate over tables of (column, row) pairs, printing a border pattern string at each position. It selects HRCG charset 2 before drawing. The position tables (BORDER_POS_HORIZ, BORDER_POS_VERT_L, BORDER_POS_VERT_R) each contain pairs of column/row bytes. The pattern strings (BORDER_STR_HORIZ, BORDER_STR_VERT_L, BORDER_STR_VERT_R) are printed at each position via PRINT_STRING_ADDR (PRINT_STRING_ADDR). BORDER_LOOP_CTR is a RAM counter used to track the loop iteration.

576c ⟨*zero page defines* 25a⟩+≡ (909) ◁572a 589a▷

```
PRINT_STRING_ADDR       EQU     $BC     ; print string source pointer (2 bytes)
```

Defines:
PRINT_STRING_ADDR, used in chunks 253, 260, 375, 404, 466, 473b, 483–86, 539, 550, 556, 578, 621, 622, 628, 629, and 637.

576d ⟨*defines* 25b⟩+≡ (909) ◁571b 583b▷

```
BORDER_LOOP_CTR         EQU     $5DC3   ; loop counter for border label drawing
```

Defines:
BORDER_LOOP_CTR, used in chunks 578 and 584.

Border label position tables, used by DRAW_BORDER_LABELS to place text labels around the game map border.

577a ⟨*border label positions* 577a⟩≡ (909)

```
    ORG     $5E7B
BORDER_POS_HORIZ:                       ; 6 col/row pairs for horizontal labels
    DC.B    $01,$09,$01,$0E,$01,$13
    DC.B    $16,$09,$16,$0E,$16,$13
BORDER_POS_VERT_L:                      ; 10 col/row pairs for vertical labels (left)
    DC.B    $01,$0A,$01,$0B,$01,$0C,$01,$0D
    DC.B    $01,$0F,$01,$12
    DC.B    $16,$0A,$16,$0D,$16,$0F,$16,$12
BORDER_POS_VERT_R:                      ; 10 col/row pairs for vertical labels (right)
    DC.B    $10,$0A,$10,$0D,$10,$0F,$10,$12
    DC.B    $25,$0A,$25,$0D,$25,$0F
    DC.B    $25,$10,$25,$11,$25,$12
```

Defines:
BORDER_POS_HORIZ, used in chunk 578.
BORDER_POS_VERT_L, used in chunk 578.
BORDER_POS_VERT_R, used in chunk 578.

The horizontal border frame pattern string, printed repeatedly by DRAW_BORDER_LABELS along the top and bottom edges of the map.

577b ⟨*border frame pattern* 577b⟩≡ (909)

```
    ORG     $5EAF
BORDER_STR_HORIZ:
    DC.B    $11                 ; length = 17
    HEX     F0 F1 F0 F1 F0 F1 F0 F1
    HEX     F0 F1 F0 F1 F0 F1 F0 F1
    DC.B    $F0                 ; last byte of horiz pattern
BORDER_STR_VERT_L:
    DC.B    $02,$F0,$F1         ; vertical border pattern (left): len=2
BORDER_STR_VERT_R:
    DC.B    $02,$F1,$F0         ; vertical border pattern (right): len=2
```

Defines:
BORDER_STR_HORIZ, used in chunk 578.
BORDER_STR_VERT_L, used in chunk 578.
BORDER_STR_VERT_R, used in chunk 578.

578 ⟨*draw border labels* 578⟩≡ (909)

```
        ORG     $5CFD
    DRAW_BORDER_LABELS:
        SUBROUTINE DRAW_BORDER_LABELS
        ; Draw border label strings at predefined positions in three loops.
        ;
        ; Behavior:
        ;   1. Selects HRCG charset 2.
        ;   2. Loop 1: draws horizontal border strings at 6 positions.
        ;   3. Loop 2: draws left vertical border strings at 10 positions.
        ;   4. Loop 3: draws right vertical border strings at 10 positions.
        ;
        ; Modifies: TEXT_COL, TEXT_ROW, PRINT_STRING_ADDR, BORDER_LOOP_CTR
        ; Clobbers: A, X, Y

        ; Select HRCG charset 2
        LDA     #$81 ; ctrl-A
        JSR     ROM_COUT1
        LDA     #$B2 ; '2' = charset 2
        JSR     ROM_COUT1

        ; Loop 1: horizontal borders (6 positions from BORDER_POS_HORIZ)
        LDX     #$0A ; index 10 (5 pairs, counting down by 2)
        STX     BORDER_LOOP_CTR
        stow    BORDER_STR_HORIZ,PRINT_STRING_ADDR
    .loop1:
        LDA     BORDER_POS_HORIZ,X
        STA     TEXT_COL
        INX
        LDA     BORDER_POS_HORIZ,X
        STA     TEXT_ROW
        JSR     PRINT_STRING
        DEC     BORDER_LOOP_CTR
        DEC     BORDER_LOOP_CTR
        LDX     BORDER_LOOP_CTR
        BPL     .loop1

        ; Loop 2: vertical borders left (10 positions from BORDER_POS_VERT_L)
        LDX     #$12 ; index 18 (9 pairs, counting down by 2)
        STX     BORDER_LOOP_CTR
        stow    BORDER_STR_VERT_L,PRINT_STRING_ADDR
    .loop2:
        LDA     BORDER_POS_VERT_L,X
        STA     TEXT_COL
        INX
        LDA     BORDER_POS_VERT_L,X
        STA     TEXT_ROW
        JSR     PRINT_STRING
        DEC     BORDER_LOOP_CTR
        DEC     BORDER_LOOP_CTR
```

```
    LDX     BORDER_LOOP_CTR
    BPL     .loop2

    ; Loop 3: vertical borders right (10 positions from BORDER_POS_VERT_R)
    LDX     #$12
    STX     BORDER_LOOP_CTR
    stow    BORDER_STR_VERT_R,PRINT_STRING_ADDR
.loop3:
    LDA     BORDER_POS_VERT_R,X
    STA     TEXT_COL
    INX
    LDA     BORDER_POS_VERT_R,X
    STA     TEXT_ROW
    JSR     PRINT_STRING
    DEC     BORDER_LOOP_CTR
    DEC     BORDER_LOOP_CTR
    LDX     BORDER_LOOP_CTR
    BPL     .loop3
    RTS
```

Defines:
DRAW_BORDER_LABELS, used in chunks 574 and 584.

Uses BORDER_LOOP_CTR 576d, BORDER_POS_HORIZ 577a, BORDER_POS_VERT_L 577a, BORDER_POS_VERT_R 577a, BORDER_STR_HORIZ 577b, BORDER_STR_VERT_L 577b, BORDER_STR_VERT_R 577b, PRINT_STRING 622, PRINT_STRING_ADDR 576c, ROM_COUT1 571a, TEXT_COL 572a, and TEXT_ROW 572a.

DRAW_BORDER_ICONS plots six decorative 2×2 font characters at fixed positions around the game border. Characters 1–4 are corner ornaments; character 28 ($1C) appears at two midpoints.

580 ⟨*draw border icons* 580⟩≡ (909)

```
      ORG     $5D7A
  DRAW_BORDER_ICONS:
      ; Draw six decorative 2x2 font icons at fixed border positions.
      ;
      ; Behavior:
      ;   Calls PLOT_CHAR six times with hardcoded (char, col, row)
      ;   triples: four corner ornaments (chars 1-4) and two midpoint
      ;   icons (char $1C).
      ;
      ; Clobbers: A, X, Y

      LDA     #$02                ; \ char 2: left col 4, row 11
      LDX     #$04                ;  |
      LDY     #$0B                ;  |
      JSR     PLOT_CHAR           ; /

      LDA     #$01                ; \ char 1: left col 4, row 16
      LDX     #$04                ;  |
      LDY     #$10                ;  |
      JSR     PLOT_CHAR           ; /

      LDA     #$03                ; \ char 3: right col 34, row 16
      LDX     #$22                ;  |
      LDY     #$10                ;  |
      JSR     PLOT_CHAR           ; /

      LDA     #$04                ; \ char 4: right col 34, row 11
      LDX     #$22                ;  |
      LDY     #$0B                ;  |
      JSR     PLOT_CHAR           ; /

      LDA     #$1C                ; \ char 28: mid col 16, row 16
      LDX     #$10                ;  |
      LDY     #$10                ;  |
      JSR     PLOT_CHAR           ; /

      LDA     #$1C                ; \ char 28: mid col 22, row 11
      LDX     #$16                ;  |
      LDY     #$0B                ;  |
      JMP     PLOT_CHAR           ; / (tail-call)
```

Defines:
DRAW_BORDER_ICONS, used in chunk 574.
Uses PLOT_CHAR 49.

Border display data at BORDER_INITIAL_CHAR–$5F18. The border animation cycles through positions in BORDER_POS_TABLE, drawing icons at each map border cell. The 3-byte records are (icon, column, row): icon=$00 draws the default border glyph; nonzero icons draw special decorations (e.g. doors, markers).

Border animation state and position data, used by <<border animation>>.

581 ⟨*border animation data* 581⟩≡ (909)

```
    ORG     $5DBF
BORDER_INITIAL_CHAR:
    DC.B    $A0                 ; initial border character (space)
BORDER_ANIM_INDEX:
    DC.B    $06                 ; current animation index (cycles $05-$15)
BORDER_ANIM_SAVE:
    DC.B    $A0                 ; saved character at animated position
BORDER_ICON_CHARS:
    DC.B    $C4,$D3,$E9         ; icon character codes (D, S, i)
BORDER_HANDLER_TABLE:
    DC.W    BORDER_HANDLER_A    ; handler pointer 0
    DC.W    BORDER_HANDLER_B    ; handler pointer 1
    DC.W    BORDER_HANDLER_C    ; handler pointer 2

    ; --- Border position table: 46 entries of (icon, col, row) ---
    ; Bottom border (row $13, columns $09-$12)
BORDER_POS_TABLE:
    DC.B    $00,$09,$13         ; \
    DC.B    $00,$0A,$13         ;  |
    DC.B    $00,$0B,$13         ;  |
    DC.B    $00,$0C,$13         ;  | bottom border
    DC.B    $00,$0D,$13         ;  |
    DC.B    $0B,$0E,$13         ;  | ($0B = special icon)
    DC.B    $00,$0F,$13         ;  |
    DC.B    $00,$10,$13         ;  |
    DC.B    $00,$11,$13         ;  |
    DC.B    $00,$12,$13         ; /
    ; Right border (column $10, rows $06-$0E top, $18-$20 bottom)
    DC.B    $00,$10,$06         ; \
    DC.B    $00,$10,$07         ;  |
    DC.B    $00,$10,$08         ;  |
    DC.B    $00,$10,$09         ;  |
    DC.B    $00,$10,$0A         ;  | right border (top)
    DC.B    $00,$10,$0B         ;  |
    DC.B    $0F,$10,$0C         ;  | ($0F = special icon)
    DC.B    $00,$10,$0D         ;  |
    DC.B    $00,$10,$0E         ; /
    DC.B    $00,$10,$20         ; \
    DC.B    $00,$10,$1F         ;  |
    DC.B    $00,$10,$1E         ;  |
    DC.B    $00,$10,$1D         ;  |
    DC.B    $00,$10,$1C         ;  | right border (bottom)
```

```
DC.B    $05,$10,$1B       ;  | ($05 = special icon)
DC.B    $00,$10,$1A       ;  |
DC.B    $00,$10,$19       ;  |
DC.B    $00,$10,$18       ; /
; Left border (column $0B, rows $06-$0E top, $18-$20 bottom)
DC.B    $00,$0B,$06       ; \
DC.B    $00,$0B,$07       ;  |
DC.B    $00,$0B,$08       ;  |
DC.B    $00,$0B,$09       ;  | left border (top)
DC.B    $14,$0B,$0A       ;  | ($14 = special icon)
DC.B    $00,$0B,$0B       ;  |
DC.B    $00,$0B,$0C       ;  |
DC.B    $00,$0B,$0D       ;  |
DC.B    $00,$0B,$0E       ; /
DC.B    $00,$0B,$20       ; \
DC.B    $00,$0B,$1F       ;  |
DC.B    $00,$0B,$1E       ;  |
DC.B    $00,$0B,$1D       ;  |
DC.B    $0E,$0B,$1C       ;  | left border (bottom)
DC.B    $00,$0B,$1B       ;  | ($0E = special icon)
DC.B    $00,$0B,$1A       ;  |
DC.B    $00,$0B,$19       ;  |
DC.B    $00,$0B,$18       ; /
```

Defines:
BORDER_ANIM_INDEX, used in chunk 584.
BORDER_ANIM_SAVE, used in chunk 584.
BORDER_HANDLER_TABLE, used in chunk 584.
BORDER_ICON_CHARS, used in chunk 583b.
BORDER_INITIAL_CHAR, used in chunk 584.
BORDER_POS_TABLE, used in chunk 584.
Uses BORDER_HANDLER_A 584, BORDER_HANDLER_B 584, and BORDER_HANDLER_C 584.

15.0.2 Border animation handler

GAME_ACTION_HANDLER is called via DISPATCH_VEC between tune notes. It animates the game border by cycling through the BORDER_POS_TABLE entries, drawing icons at each border cell.

The handler runs 3 passes per call (controlled by BORDER_ANIM_SAVE). On pass 1, it calls ANIM_BORDER_ICONS_A (ANIM_BORDER_ICONS_A) which randomly places icons from $5E1F along the border. On pass 0, it calls ANIM_BORDER_ICONS_B (ANIM_BORDER_ICONS_B) which uses icons from $5DE9. On other passes, it iterates through the handler pointer table at BORDER_HANDLER_TABLE.

TICK_BORDER_ANIM advances the animation index through the border positions, wrapping from $15 back to $05.

Current game speed / animation index, used by BORDER_ANIMATION.

583a ⟨*anim index* 583a⟩≡ (909)

```
    ORG     $7ABD
ANIM_INDEX:
    DC.B    $02                     ; current game speed / animation index
```

Defines:
ANIM_INDEX, used in chunks 270a, 271, 604b, and 605b.

Runtime aliases for bytes within BORDER_ICON_CHARS (BORDER_ICON_OFFSET): the border animation reuses these three bytes as loop variables during icon drawing.

583b ⟨*defines* 25b⟩+≡ (909) ◁576d 601▷

```
BORDER_ICON_OFFSET      EQU     $5DC2   ; runtime: icon table offset (aliases BORDER_ICON_CHARS)
BORDER_SWAP_FLAG        EQU     $5DC4   ; runtime: swap flag (aliases BORDER_ICON_CHARS+2)
ICON_TABLE_PTR          EQU     $BE     ; pointer to border icon table (aliases DATA_PTR)
```

Defines:
BORDER_ICON_OFFSET, used in chunk 584.
BORDER_SWAP_FLAG, used in chunk 584.
ICON_TABLE_PTR, used in chunk 584.
Uses BORDER_ICON_CHARS 581 and DATA_PTR 618a.

584 ⟨*border animation* 584⟩≡ (909)

```
    ORG     $5B2A
GAME_ACTION_HANDLER:
    SUBROUTINE GAME_ACTION_HANDLER
    ; Border animation handler invoked via DISPATCH_VEC during idle loop.
    ;
    ; NOTE: Code from $5C00 onward (TICK_BORDER_ANIM tail through
    ; DRAW_BORDER_LABELS) is overwritten when LOAD_SCENE_FROM_DISK
    ; loads scene text into SCENE_TEXT_DATA ($5C00).  This permanently
    ; disables the idle border animation after the first scene load.
    ; See the LOAD_SCENE_FROM_DISK documentation for the full analysis.
    ;
    ; Behavior:
    ;   1. Runs a 3-pass loop: each pass dispatches three border handlers
    ;      (A, B, C) that swap/draw 2x2 icons along the border.
    ;   2. Between passes, randomly places icons via ANIM_BORDER_ICONS_A/B.
    ;   3. Each pass advances the animation index via TICK_BORDER_ANIM
    ;      and plays tune notes via TUNE_ADVANCE between handler calls.
    ;
    ; Modifies: BORDER_ANIM_SAVE, BORDER_ANIM_INDEX, FONT_CHARNUM,
    ;   DISPATCH_VEC, border position tables
    ; Clobbers: A, X, Y

    ; --- Main entry: 3-pass border animation loop ---
.restart:
    LDA     #$03
    STA     BORDER_ANIM_SAVE  ; 3 passes
    JSR     TICK_BORDER_ANIM  ; advance one position
.new_pass:
    LDX     #$04              ; start at handler pair 2
    STX     BORDER_LOOP_CTR   ; handler index
.handler_loop:
    LDX     BORDER_LOOP_CTR
    LDA     BORDER_HANDLER_TABLE,X ; \ load handler pointer
    STA     DISPATCH_VEC      ;  | into DISPATCH_VEC
    INX                       ;  |
    LDA     BORDER_HANDLER_TABLE,X ;  |
    STA     DISPATCH_VEC+1    ; /
    DEC     BORDER_LOOP_CTR   ; \ next handler pair
    DEC     BORDER_LOOP_CTR   ; /
    JMP     TUNE_ADVANCE      ; play next note, returns via DISPATCH_VEC
.check_pass:
    LDA     BORDER_LOOP_CTR
    BPL     .handler_loop     ; more handlers -> continue
    DEC     BORDER_ANIM_SAVE  ; next pass
    BMI     .restart          ; all passes done -> restart from top
    LDA     BORDER_ANIM_SAVE
    CMP     #$01
    BEQ     .pass_0
    BCS     .new_pass         ; pass 2 -> normal handlers
```

```
    JSR     ANIM_BORDER_ICONS_A ; pass 1 -> random icon placement
    JMP     .new_pass
.pass_0:
    JSR     ANIM_BORDER_ICONS_B ; pass 0 -> alternate icons
    JMP     .new_pass

    ; --- Handler A ($5B6D): draw from $5DE9/$5E04 tables ---
BORDER_HANDLER_A:
    LDA     #$E9
    LDX     #$5D
    JSR     DRAW_BORDER_ICON_LIST
    LDA     #$04
    LDX     #$5E
    JSR     DRAW_BORDER_ICON_LIST
    JMP     .check_pass

    ; --- Handler C ($5B7E): draw from $5E1F/$5E3A tables ---
BORDER_HANDLER_C:
    LDA     #$1F
    LDX     #$5E
    JSR     DRAW_BORDER_ICON_LIST
    LDA     #$3A
    LDX     #$5E
    JSR     DRAW_BORDER_ICON_LIST
    JMP     .check_pass

    ; --- Handler B ($5B8F): draw from $5DCB table (Y=$1B start) ---
BORDER_HANDLER_B:
    LDA     #$CB
    LDX     #$5D
    LDY     #$1B
    JSR     DRAW_BORDER_ICON_ENTRY
    JMP     .check_pass

    ; --- Draw a list of border icons from (A/X) pointer ---
DRAW_BORDER_ICON_LIST:
    LDY     #$18              ; default: start at offset $18
DRAW_BORDER_ICON_ENTRY:
    STA     ICON_TABLE_PTR    ; \ set pointer
    STX     ICON_TABLE_PTR+1  ; /
    STY     BORDER_ICON_OFFSET ; save offset
    LDA     #$00
    STA     BORDER_SWAP_FLAG  ; clear swap flag
.icon_loop:
    LDY     BORDER_ICON_OFFSET ; current offset
    LDA     BORDER_SWAP_FLAG  ; swap flag
    BNE     .swap_restore     ; nonzero -> restore saved char
    LDA     (ICON_TABLE_PTR),Y ; read icon from table
    STA     BORDER_SWAP_FLAG  ; save it
    BEQ     .skip             ; zero -> skip (empty)
```

```
    LDA     #$00
    STA     (ICON_TABLE_PTR),Y ; clear table entry
    JSR     DRAW_BORDER_CHAR_AT ; draw the icon
    JMP     .skip
.swap_restore:
    STA     (ICON_TABLE_PTR),Y ; restore saved icon
    JSR     DRAW_BORDER_CHAR_AT ; draw it
    LDA     #$00
    STA     BORDER_SWAP_FLAG  ; clear swap flag
.skip:
    DEC     BORDER_ICON_OFFSET ; \ next entry (3 bytes per record)
    DEC     BORDER_ICON_OFFSET ;  |
    DEC     BORDER_ICON_OFFSET ; /
    BPL     .icon_loop
    RTS

    ; --- Draw a single border character at position from table entry ---
DRAW_BORDER_CHAR_AT:
    STA     FONT_CHARNUM      ; set character to draw
    INY
    LDA     (ICON_TABLE_PTR),Y ; column
    STA     TEXT_ROW          ; -> cursor column
    INY
    LDA     (ICON_TABLE_PTR),Y ; row
    STA     TEXT_COL          ; -> cursor row
    JMP     RENDER_FONT_CHAR_DIRECT ; render font char

    ; --- Tick border animation: advance index, draw next position ---
TICK_BORDER_ANIM:
    LDA     BORDER_ANIM_INDEX ; current position index
    STA     BORDER_POS_TABLE+27 ; save to restore later ($5DE6)
    LDY     BORDER_POS_TABLE+28 ; \ get col/row from next position
    STY     TEXT_ROW          ;  |
    LDY     BORDER_POS_TABLE+29 ;  |
    STY     TEXT_COL          ; /
    STA     FONT_CHARNUM      ; draw border char
    JSR     RENDER_FONT_CHAR_DIRECT
    INC     BORDER_ANIM_INDEX ; advance index
    LDA     BORDER_ANIM_INDEX
    CMP     #$16              ; past end?
    BCC     .tick_done
    LDA     #$05              ; wrap to start
    STA     BORDER_ANIM_INDEX
.tick_done:
    RTS

    ; --- Random icon placement A (from $5E1F/$5E3A tables) ---
ANIM_BORDER_ICONS_A:
    LDA     #$02              ; \ random(2): pick table A or B
    PHA                       ; /
```

```
    LDX     #$06
    LDA     #$1F
    LDY     #$5E
    JMP     .anim_shared

    ; --- Random icon placement B (from $5DE9/$5DCB tables) ---
ANIM_BORDER_ICONS_B:
    LDA     #$04              ; \ random(4): pick position
    PHA                       ; /
    LDX     #$15
    LDA     #$E9
    LDY     #$5D
.anim_shared:
    STX     BORDER_INITIAL_CHAR ; save max index
    STA     ICON_TABLE_PTR    ; \ set pointer
    STY     ICON_TABLE_PTR+1  ; /
    PLA
    JSR     RANDOM_IN_RANGE
    CMP     #$01
    BEQ     .random_ok
    BCC     .random_small
    RTS                       ; > 1 -> skip (rare)
.random_ok:
    LDY     #$18
    BNE     .do_place
.random_small:
    LDY     #$33
.do_place:
    LDX     BORDER_INITIAL_CHAR
    LDA     BORDER_POS_TABLE,X ; read icon from border table
    STA     FONT_CHARNUM
    STA     (ICON_TABLE_PTR),Y ; store in target table
    LDA     #$00
    STA     BORDER_POS_TABLE,X ; clear source entry
    INY
    LDA     (ICON_TABLE_PTR),Y ; column
    STA     TEXT_ROW
    INY
    LDA     (ICON_TABLE_PTR),Y ; row
    STA     TEXT_COL
    JSR     RENDER_FONT_CHAR_DIRECT ; render
    LDA     #$00
    STA     FONT_CHARNUM
    LDX     BORDER_INITIAL_CHAR
    LDA     BORDER_POS_TABLE+1,X ; \ restore cursor from
    STA     TEXT_ROW          ;  | adjacent table entry
    LDA     BORDER_POS_TABLE+2,X ;  |
    STA     TEXT_COL          ; /
    JMP     RENDER_FONT_CHAR_DIRECT ; render and return
```

Defines:

ANIM_BORDER_ICONS_A, never used.
ANIM_BORDER_ICONS_B, never used.
BORDER_HANDLER_A, used in chunk 581.
BORDER_HANDLER_B, used in chunk 581.
BORDER_HANDLER_C, used in chunk 581.
DRAW_BORDER_CHAR_AT, never used.
DRAW_BORDER_ICON_ENTRY, never used.
DRAW_BORDER_ICON_LIST, never used.
GAME_ACTION_HANDLER, used in chunk 570b.
TICK_BORDER_ANIM, never used.

Uses BORDER_ANIM_INDEX 581, BORDER_ANIM_SAVE 581, BORDER_HANDLER_TABLE 581, BORDER_ICON_OFFSET 583b, BORDER_INITIAL_CHAR 581, BORDER_LOOP_CTR 576d, BORDER_POS_TABLE 581, BORDER_SWAP_FLAG 583b, DISPATCH_VEC 569b, DRAW_BORDER_LABELS 578, FONT_CHARNUM 575, ICON_TABLE_PTR 583b, LOAD_SCENE_FROM_DISK 666, RANDOM_IN_RANGE 597, RENDER_FONT_CHAR_DIRECT 466, SCENE_TEXT_DATA 526a, TEXT_COL 572a, TEXT_ROW 572a, and TUNE_ADVANCE 683.

INIT_WORLD is a thin wrapper that passes a pointer to the world initialization data table (WORLD_INIT_DATA) to the game setup routine INPUT_LOOP. Called once from GAME_INIT to initialize locations and scene state.

588a ⟨*init world* 588a⟩≡ (909)

```
    ORG     $62E5
INIT_WORLD:
    SUBROUTINE INIT_WORLD
    ; Pass world initialization data to the game setup loop.
    ;
    ; Behavior:
    ;   Loads X/Y with the address of WORLD_INIT_DATA and tail-calls INPUT_LOOP.
    ;
    ; Clobbers: X, Y

    LDX     #>WORLD_INIT_DATA ; \ X/Y = pointer to init data
    LDY     #<WORLD_INIT_DATA ; /
    JMP     INPUT_LOOP        ; initialize world from data table
```

Uses INPUT_LOOP 247 and WORLD_INIT_DATA 274.

A single RTS—another no-op stub called from GAME_INIT, like INIT_STUB.

588b ⟨*init stub2* 588b⟩≡ (909)

```
    ORG     $6B94
INIT_STUB2:
    RTS
```

Defines:
INIT_STUB2, used in chunk 564.

INIT_DISK_IOB sets the zero-page pointer RWTS_IOB_PTR ($08/$09) to the address of the RWTS Input/Output Block (RWTS_IOB). This single-assignment call during GAME_INIT enables all subsequent disk operations to access the IOB fields through indirect-indexed addressing (e.g. LDA ($08),Y with Y selecting the field).

589a ⟨*zero page defines* 25a⟩+≡ (909) ◁576c 591a▷

```
RWTS_IOB_PTR            EQU     $08     ; ZP pointer to RWTS IOB ($B7E8)
```

Defines:
RWTS_IOB_PTR, used in chunks 301, 305, 589b, 666, and 669.

589b ⟨*init disk iob* 589b⟩≡ (909)

```
    ORG     $6C2E
INIT_DISK_IOB:
    SUBROUTINE INIT_DISK_IOB
    ; Set the zero-page RWTS IOB pointer to the IOB address.
    ;
    ; Behavior:
    ;   Stores the address of RWTS_IOB ($B7E8) into RWTS_IOB_PTR ($08/$09).
    ;
    ; Modifies: RWTS_IOB_PTR
    ; Clobbers: A, Y

    LDA     #>RWTS_IOB     ; \ set RWTS IOB pointer
    LDY     #<RWTS_IOB     ;  | $08/$09 = $B7E8
    STA     RWTS_IOB_PTR+1 ;  |
    STY     RWTS_IOB_PTR   ; /
    RTS
```

Defines:
INIT_DISK_IOB, used in chunk 564.
Uses RWTS_IOB 770 and RWTS_IOB_PTR 589a.

Prompts the player to choose keyboard or joystick input. Displays message $38 (the prompt), then polls the keyboard and paddle button 1 in a tight loop. If the player presses the space bar, INPUT_MODE is set to 0 (keyboard). If the paddle button state changes (pressed or released), INPUT_MODE is set to 1 (joystick), the cursor is repositioned, and message $2F (joystick confirmation) is displayed.

589c ⟨*var input mode* 589c⟩≡ (909)

```
    ORG     $5A00
INPUT_MODE:
    DC.B    $AB                     ; initial value (runtime: 0 = keyboard, 1 = joystick)
```

Defines:
INPUT_MODE, used in chunks 52, 58, 550, 556, 570b, 590, and 683.

590 ⟨*select input mode* 590⟩≡ (909)

```
      ORG     $084E
  SELECT_INPUT_MODE:
      SUBROUTINE SELECT_INPUT_MODE
      ; Prompt the player to choose keyboard or joystick input.
      ;
      ; Behavior:
      ;   1. Displays the input mode prompt message.
      ;   2. Polls keyboard and paddle button 1 in a tight loop.
      ;   3. Space bar selects keyboard (INPUT_MODE=0); paddle button
      ;      state change selects joystick (INPUT_MODE=1).
      ;
      ; Outputs:
      ;   INPUT_MODE | 0 for keyboard, 1 for joystick
      ; Modifies: INPUT_MODE
      ; Clobbers: A, X, Y

      LDA     #MSG_INPUT_PROMPT           ; message $38: input mode prompt
      JSR     DISPLAY_MESSAGE
      LDA     $C061          ; read paddle button 1
      AND     #$80           ; isolate bit 7 (pressed state)
      STA     $BA            ; save initial button state
  .clear:
      STA     $C010          ; clear keyboard strobe
  .poll:
      LDX     $C000          ; read keyboard
      BMI     .key           ; key pressed -> check it
      LDA     $C061          ; re-read paddle button
      EOR     $BA            ; compare with initial state
      BMI     .joystick      ; button state changed -> joystick
      BPL     .poll          ; keep polling
  .key:
      BXEQ    #$A0,.keyboard ; yes -> keyboard mode
      BNE     .clear         ; other key -> ignore, clear strobe
  .keyboard:
      STOB    #$00,INPUT_MODE ; 0 = keyboard
      RTS
  .joystick:
      STOB    #$01,INPUT_MODE ; 1 = joystick
      JSR     SET_CURSOR_ROW21
      LDA     #MSG_JOYSTICK_MODE           ; message $2F: joystick confirmation
      JMP     DISPLAY_MESSAGE
```

Defines:
SELECT_INPUT_MODE, used in chunk 564.
Uses DISPLAY_MESSAGE 637, INPUT_MODE 589c, MSG_INPUT_PROMPT 634, MSG_JOYSTICK_MODE 634, SET_CURSOR_ROW21 627, and STOB 8c.

INIT_MOB_COUNT is called once during GAME_INIT to initialize TOTAL_MOB_COUNT. It walks all 9-byte character records using CHAR_PTR (CHAR_PTR) starting from index 1 (skipping the group-0 header at MOB_BASE_LO), summing byte 8 (group member count) of each record until the end sentinel (byte 1 = $FF) is reached.

The result in TOTAL_MOB_COUNT is the total number of mobs assigned to groups 1 and above. At startup, byte 8 contains only the group count in bits 3–0 (the upper bits—pending XP and status flag—are zero), so the sum is simply the total mob count.

591a ⟨*zero page defines* 25a⟩+≡ (909) ◁589a 600b▷

```
CHAR_PTR                EQU     $FA     ; character record pointer (2 bytes, $FA/$FB)
```

Defines:
CHAR_PTR, used in chunks 219, 222, 315, 373, 385, 398, 415, 417, 419, 420, 422–24, 426, 430, 454, 469, 482, 488, 535, 539, 542, 561, 592, and 612.

591b ⟨*var total mob count* 591b⟩≡ (909)

```
    ORG     $5A01
TOTAL_MOB_COUNT:
    DC.B    $B1                     ; initial value (runtime: total mobs across all groups (excl. gr
```

Defines:
TOTAL_MOB_COUNT, used in chunks 219, 385, 417a, 419, 422, 478, and 592.

592 ⟨*init mob count* 592⟩≡ (909)

```
    ORG     $0C51
INIT_MOB_COUNT:
    SUBROUTINE INIT_MOB_COUNT
    ; Sum mob counts from all character group records into TOTAL_MOB_COUNT.
    ;
    ; Behavior:
    ;   Starts at character record 1, walks 9-byte records summing
    ;   byte 8 (group member count) until end sentinel (byte 1 = $FF).
    ;
    ; Outputs:
    ;   TOTAL_MOB_COUNT | total mobs across groups 1+
    ; Modifies: TOTAL_MOB_COUNT, CHAR_PTR
    ; Clobbers: A, Y

    STOB    #$00,TOTAL_MOB_COUNT ; clear total
    STOW    $4009,CHAR_PTR ; $FA/$FB = $4009 (record 1)
.loop:
    LDY     #$01
    LDA     (CHAR_PTR),Y    ; byte 1 of record
    BANE    #$FF,.count     ; $FF = end sentinel
    RTS
.count:
    LDY     #$08
    LDA     (CHAR_PTR),Y    ; byte 8 = group member count
    CLC
    ADC     TOTAL_MOB_COUNT ; accumulate
    STA     TOTAL_MOB_COUNT
    LDA     #$09            ; \ advance $FA by 9 (next record)
    ADC     CHAR_PTR        ;  |
    STA     CHAR_PTR        ;  |
    BCC     .loop           ;  |
    INC     CHAR_PTR+1      ;  |
    BCS     .loop           ; / always taken
```

Defines:
INIT_MOB_COUNT, used in chunk 564.
Uses CHAR_PTR 591a, STOB 8c, STOW 8a, and TOTAL_MOB_COUNT 591b.

Chapter 16

Utility Routines

16.1 Random number generation

The pseudo-random number generator computes:

$$\texttt{val}_{n+1} = 69 \cdot \texttt{val}_n + \$53 \pmod{256}$$

and stores both the full result and a shifted copy.

STEP_PRNG maintains three state variables: PRNG_STATE holds the full LCG state, PRNG_TEMP is a scratch byte used during the 69× multiplication, and PRNG_OUTPUT receives the state shifted right by one bit—this is the value most callers test for random decisions.

593a ⟨*var prng output* 593a⟩≡ (909)

```
      ORG     $5A18
  PRNG_OUTPUT:
      DC.B    $A0                    ; initial value (runtime: PRNG output (state >> 1), used for ran
```

Defines:
PRNG_OUTPUT, used in chunks 222, 354, 357, 360, 377, 415, 456, 478, 527, 529, 595, 597, 663, 682, and 898.

593b ⟨*var prng state* 593b⟩≡ (909)

```
      ORG     $5A19
  PRNG_STATE:
      DC.B    $CC                    ; initial value (runtime: PRNG full state: val = 69*val + $53 (m
```

Defines:
PRNG_STATE, used in chunk 595.

594 ⟨*var prng temp* 594⟩≡ (909)

```
    ORG     $5A1A
PRNG_TEMP:
    DC.B    $C4                     ; initial value (runtime: PRNG temporary for multiplication)
```

Defines:
PRNG_TEMP, used in chunk 595.

595 ⟨*step prng* 595⟩≡ (909)

```
    ORG     $78A8
; ---------------------------------------------------------------
; STEP_PRNG -- advance the linear congruential PRNG
;
; Computes val = 69 * val + $53 (mod 256) and stores
; both the full result and a shifted copy.
;
; In:  PRNG_STATE = current LCG state
; Out: PRNG_STATE = updated state
;      PRNG_OUTPUT = state >> 1 (0..127)
;      A = PRNG_OUTPUT
; Clobbers: A
; ---------------------------------------------------------------
STEP_PRNG:
    SUBROUTINE STEP_PRNG
    ; Advance the PRNG state using LCG formula: val = 69*val + $53.
    ;
    ; Behavior:
    ;   Multiplies PRNG_STATE by 69 via shift-and-add, adds $53,
    ;   stores result in both PRNG_STATE and PRNG_OUTPUT.
    ;
    ; Outputs:
    ;   A | new PRNG value
    ;   PRNG_OUTPUT | same value
    ; Modifies: PRNG_STATE, PRNG_OUTPUT, PRNG_TEMP
    ; Clobbers: A

    ; Advance the PRNG: state = 69*state + $53, output = state >> 1.
    ;
    ; Output: PRNG_STATE updated, PRNG_OUTPUT = state >> 1, A = PRNG_OUTPUT
    ; Clobbers: A

    LDA     PRNG_STATE                  ; val
    ASL                                 ; val*2
    ASL                                 ; val*4
    STA     PRNG_TEMP                   ; temp = val*4
    ASL                                 ; val*8
    ASL                                 ; val*16
    ASL                                 ; val*32
    ASL                                 ; val*64
    CLC
    ADC     PRNG_TEMP                   ; val*64 + val*4 = val*68
    CLC
    ADC     PRNG_STATE                  ; val*68 + val = val*69
    CLC
    ADC     #$53                        ; + $53
    STA     PRNG_STATE                  ; store full result
    LSR                                 ; val >> 1
    STA     PRNG_OUTPUT                 ; shifted copy for random tests
```

```
    RTS
```

Defines:
STEP_PRNG, used in chunks 222, 354, 357, 360, 377, 415, 456, 478, 527, 529, 597, 663, 682, and 898.
Uses PRNG_OUTPUT 593a, PRNG_STATE 593b, and PRNG_TEMP 594.

Two shifts produce `val*4`, saved to `PRNG_TEMP`. Four more shifts give `val*64`. Adding `val*4` gives `val*68`, then adding `val` gives `val*69`. Adding `$53` completes the LCG step. This is a linear congruential generator with multiplier 69, increment $53, and modulus 256.

Returns a uniform random integer in `0..A$-$1` using rejection sampling. The routine finds the smallest bitmask that covers the range, then repeatedly calls `STEP_PRNG` and masks `PRNG_OUTPUT` until the result is less than or equal to the shifted limit. This avoids modulo bias.

Input: A = range limit (1–127). If A = 0, returns 0 immediately. Output: A = random value in `0..limit$-$1`.

`RANDOM_IN_RANGE` saves the limit in `RNG_LIMIT` and computes a bitmask in `RNG_MASK` that covers the range. It then loops, calling `STEP_PRNG` and masking `PRNG_OUTPUT` with `RNG_MASK`, rejecting values that exceed the limit.

The implementation below uses bit-mask tables for each power-of-two range bracket to minimize the number of PRNG calls.

596a ⟨*var rng limit* 596a⟩≡ (909)

```
    ORG     $5A11
RNG_LIMIT:
    DC.B    $C4                     ; initial value (runtime: random_in_range temporary (limit/shift
```

Defines:
RNG_LIMIT, used in chunk 597.

596b ⟨*var rng mask* 596b⟩≡ (909)

```
    ORG     $5A12
RNG_MASK:
    DC.B    $D9                     ; initial value (runtime: random_in_range bit mask)
```

Defines:
RNG_MASK, used in chunk 597.

597 ⟨*random in range* 597⟩≡ (909)

```
    ORG     $1317
; ---------------------------------------------------------------
; RANDOM_IN_RANGE -- uniform random integer via rejection sampling
;
; Finds the smallest power-of-two bitmask covering the range,
; then loops calling STEP_PRNG until the masked output falls
; within the limit.  Avoids modulo bias.
;
; In:  A = range limit (1..127); if A=0, returns 0
; Out: A = random value in 0..limit-1
; Clobbers: A, Y
; ---------------------------------------------------------------
RANDOM_IN_RANGE:
    SUBROUTINE RANDOM_IN_RANGE
    ; Return a random number in 0..A-1 via rejection sampling.
    ;
    ; Input:  A = range limit (1-127; 0 returns 0)
    ; Output: A = random value in 0..limit-1
    ; Clobbers: A, Y

    STA     RNG_LIMIT                   ; save range limit
    BANE    #$00,.nonzero
    RTS                                 ; range=0 -> return 0

.nonzero:
    BAPL    #$02,.ge2
    LDY     #$06                        ;  | range 1: mask=$40, shift=6
    LDA     #$40                        ;  |
    BNE     .have_mask                  ; /
.ge2:
    BAPL    #$04,.ge4
    LDY     #$05                        ;  | range 2-3: mask=$60, shift=5
    LDA     #$60                        ;  |
    BNE     .have_mask                  ; /
.ge4:
    BAPL    #$08,.ge8
    LDY     #$04                        ;  | range 4-7: mask=$70, shift=4
    LDA     #$70                        ;  |
    BNE     .have_mask                  ; /
.ge8:
    BAPL    #$10,.ge16
    LDY     #$03                        ;  | range 8-15: mask=$78, shift=3
    LDA     #$78                        ;  |
    BNE     .have_mask                  ; /
.ge16:
    BAPL    #$20,.ge32
    LDY     #$02                        ;  | range 16-31: mask=$7C, shift=2
    LDA     #$7C                        ;  |
    BNE     .have_mask                  ; /
```

```
.ge32:
    BAPL    #$40,.ge64
    LDY     #$01                        ;  | range 32-63: mask=$7E, shift=1
    LDA     #$7E                        ;  |
    BNE     .have_mask                  ; /
.ge64:
    LDY     #$00                        ; range 64-127: mask=$7F, shift=0
    LDA     #$7F

.have_mask:
    STA     RNG_MASK                    ; save bit mask
    LDA     RNG_LIMIT                   ; reload range limit
    STY     RNG_LIMIT                   ; save shift count in RNG_LIMIT
.shift_loop:
    DEY                                 ; \
    BMI     .shifted                    ;  | shift limit left by (shift count)
    CLC                                 ;  | positions so it aligns with the
    ROL                                 ;  | mask bits
    BNE     .shift_loop                 ; /
.shifted:
    LDY     RNG_LIMIT                   ; Y = shift count
    STA     RNG_LIMIT                   ; save shifted limit

.sample:
    JSR     STEP_PRNG                   ; get next random byte
    LDA     PRNG_OUTPUT                 ; load masked PRNG output
    AND     RNG_MASK                    ; mask to relevant bits
    BAEQ    RNG_LIMIT,.accept           ; equal -> accept
    BPL     .sample                     ; greater -> reject, retry

.accept:
    DEY                                 ; \
    BPL     .shift_right                ;  | shift result right by (shift count)
    RTS                                 ;  | to recover the final value
.shift_right:
    CLC                                 ;  |
    ROR                                 ;  |
    BCC     .accept                     ; / (always taken: masked bit is 0)
```

Defines:
RANDOM_IN_RANGE, used in chunks 232, 291, 325, 338, 342, 349, 354, 360, 373, 377, 390, 422, 423, 440, 444, 447, 494, and 584.
Uses PRNG_OUTPUT 593a, RNG_LIMIT 596a, RNG_MASK 596b, and STEP_PRNG 595.

16.2 Coordinate conversion

Converts a packed map position (0–199) into separate column and row values by dividing by 20. The map is 20 columns wide, so `column = pos mod 20` and `row = pos / 20`.

The loop repeatedly subtracts 20 (via `ADC \#$EC` with carry clear, since `$EC = 256 $-$ 20`), counting iterations in `Y`. When the subtraction underflows (carry clear on exit), it adds 20 back to recover the remainder.

599 ⟨*pos to colrow* 599⟩≡ (909)

```
      ORG     $0C25
  ; ------------------------------------------------
  ; POS_TO_COLROW | convert linear position to (col, row)
  ;
  ; Divides a packed map position by MAP_WIDTH (20)
  ; using repeated subtraction.
  ;
  ; Input:   A = map position (0-199)
  ; Output:  A = column (0-19), Y = row (0-9)
  ; Clobbers: flags
  ; ------------------------------------------------
  POS_TO_COLROW:
      SUBROUTINE POS_TO_COLROW
      ; Unpack a map position (0-199) into column and row.
      ;
      ; Input:  A = packed position
      ; Output: A = column (0-19), Y = row (0-9)
      ; Clobbers: flags

      LDY     #$00                        ; row counter = 0
  .loop:
      CLC
      ADC     #$EC                        ; A -= MAP_WIDTH ($EC = -20)
      BCC     .done                       ; underflow -> went past 0
      INY                                 ; row++
      BCS     .loop                       ; always taken (C=1)
  .done:
      ADC     #MAP_WIDTH                  ; add 20 back -> remainder
      RTS
```

Defines:
POS_TO_COLROW, used in chunks 232, 342, 354, 384d, 385, 400, 471, 475, 487, 494, 535, 537, and 542.
Uses MAP_WIDTH 388a.

The inverse of POS_TO_COLROW: converts a column in A and row in Y to a linear map position `A = col + row * 20`. The carry is cleared once on entry; thereafter the loop adds 20 per row and exits early if carry occurs (overflow past 255).

600a ⟨*colrow to pos* 600a⟩≡ (909)

```
    ORG     $0D98
; ------------------------------------------------
; COLROW_TO_POS | convert (col, row) to linear position
;
; Computes pos = col + row * MAP_WIDTH by adding
; MAP_WIDTH (20) once per row.
;
; Input:   A = column (0-19), Y = row (0-9)
; Output:  A = map position (0-199)
; Clobbers: Y, flags
; ------------------------------------------------
COLROW_TO_POS:
    SUBROUTINE COLROW_TO_POS
    ; Pack column and row into a map position.
    ;
    ; Input:  A = column (0-19), Y = row (0-9)
    ; Output: A = packed position (0-199)
    ; Clobbers: Y, flags

    CLC
.loop:
    DEY                                 ; row--
    BPL     .add                        ; row >= 0 -> add another 20
    RTS                                 ; Y exhausted, A = result
.add:
    ADC     #MAP_WIDTH                  ; A += 20
    BCC     .loop                       ; no overflow -> continue
                                        ; C=1 -> overflow, fall through
```

Defines:
COLROW_TO_POS, used in chunks 232, 239, 323, 330, 357, 471, 494, 542, and 898.
Uses MAP_WIDTH 388a.

GET_MOB_DATA computes a pointer into the game data from a 1-based index in A. Each record is 16 bytes. The result MOB_DATA_PTR (MOB_DATA_PTR) = MOB_BASE_LO/MOB_BASE_HI base + (index−1)×16.

600b ⟨*zero page defines* 25a⟩+≡ (909) ◁591a 608▷

```
MOB_DATA_PTR            EQU     $BA     ; mob data record pointer (2 bytes, $BA/$BB)
```

Defines:
MOB_DATA_PTR, used in chunks 220, 222, 315, 426, 430, 462, 469, 478, 482, 602, 609, 612, 613, and 615–17.

The mob data base address is stored in the first two bytes of the character table header record at MOB_BASE_LO. CHAR_LIST_HEAD is the head of the character linked list.

601 ⟨*defines* 25b⟩+≡ (909) ◁583b 606b▷

```
HIGH_NIBBLE_MASK    EQU     $F0     ; bits 7-4: generic high nibble extraction
NAME_BUF_LEN        EQU     $14     ; character name display buffer length (20 bytes)
```

Defines:
HIGH_NIBBLE_MASK, used in chunk 602.
NAME_BUF_LEN, used in chunk 613.

602 ⟨*get mob data* 602⟩≡ (909)

```
    ORG     $0C32
; ----------------------------------------------
; GET_MOB_DATA | compute pointer to mob data record
;
; Each mob record is 16 bytes.  Given a 1-based
; index, computes MOB_DATA_PTR = base + (index-1)*16
; where base = MOB_BASE_LO/MOB_BASE_HI.
;
; Input:   A = mob index (1-based)
; Output:  MOB_DATA_PTR ($BA/$BB) = record address
; Clobbers: A, Y, flags
; ----------------------------------------------
GET_MOB_DATA:
    SUBROUTINE GET_MOB_DATA
    ; Look up mob data record by 1-based index.
    ;
    ; Input:  A = mob index (1-based)
    ; Output: MOB_DATA_PTR ($BA/$BB) = pointer to mob record
    ; Clobbers: A, Y

    CLC
    ADC     #$FF                        ; A = index - 1
    ROL2                                ; rotate left 4 times
    ROL2                                ;   = multiply by 16 (with carry)
    TAY                                 ; save intermediate
    ROL                                 ; one more rotate
    AND     #LOW_NIBBLE_MASK            ; high nybble -> page offset
    CLC
    ADC     MOB_BASE_HI                 ; add base page
    STA     MOB_DATA_PTR+1              ; store high byte
    TYA
    AND     #HIGH_NIBBLE_MASK           ; low nybble -> byte offset
    CLC
    ADC     MOB_BASE_LO                 ; add base offset
    STA     MOB_DATA_PTR                ; store low byte
    BCC     .done
    INC     MOB_DATA_PTR+1              ; propagate carry
.done:
    RTS
```

Defines:
GET_MOB_DATA, used in chunks 220, 222, 291, 315, 426, 430, 469, 478, 482, 612, 615, and 616.
Uses HIGH_NIBBLE_MASK 601, LOW_NIBBLE_MASK 553, MOB_BASE_HI 53a, MOB_BASE_LO 53a, MOB_DATA_PTR 600b, and ROL2 16c.

16.3 Timing and animation

DELAY_WITH_ANIMATION delays for a configurable duration while keeping the status window animation running. It loads an iteration count from DELAY_COUNT and calls ANIM_TICK_AND_WAIT that many times.

Delay iteration count, used by DELAY_WITH_ANIMATION.

603a ⟨*delay count* 603a⟩≡ (909)

```
    ORG     $7ABF
DELAY_COUNT:
    DC.B    $04                     ; delay iterations
```

Defines:
DELAY_COUNT, used in chunk 603b.

603b ⟨*delay with animation* 603b⟩≡ (909)

```
    ORG     $0D10
DELAY_WITH_ANIMATION:
    SUBROUTINE DELAY_WITH_ANIMATION
    ; Delay for DELAY_COUNT iterations with animation.
    ;
    ; Behavior: calls ANIM_TICK_AND_WAIT in a loop
    ; Clobbers: A, X, Y

    LDA     DELAY_COUNT         ; load iteration count
.loop:
    PHA                         ; save counter
    JSR     ANIM_TICK_AND_WAIT  ; tick animation + timed wait
    PLA                         ; restore counter
    TAY
    DEY                         ; decrement
    TYA
    BNE     .loop               ; loop until zero
    RTS
```

Defines:
DELAY_WITH_ANIMATION, used in chunks 338, 360, 385, 607b, and 637.
Uses ANIM_TICK_AND_WAIT 605b and DELAY_COUNT 603a.

TICK_ANIM_COUNTER decrements ANIM_FRAME_CTR. When it reaches zero, it calls DRAW_BLINK_ALT to advance the animation frame, then reloads the wait loop count from ANIM_WAIT_TABLE (indexed by ANIM_INDEX) into WAIT_LOOP_COUNT.

Animation frame countdown and per-frame wait counts, used by TICK_ANIM_COUNTER.

604a ⟨*anim frame ctr* 604a⟩≡ (909)

```
    ORG     $7AAF
ANIM_FRAME_CTR:
    DC.B    $03                     ; animation frame countdown
ANIM_WAIT_TABLE:
    HEX     00 32 23 17 0C 05      ; per-frame wait counts (6 entries)
```

Defines:
ANIM_FRAME_CTR, used in chunks 604b and 605b.
ANIM_WAIT_TABLE, used in chunk 604b.

604b ⟨*tick anim counter* 604b⟩≡ (909)

```
    ORG     $0D1E
TICK_ANIM_COUNTER:
    SUBROUTINE TICK_ANIM_COUNTER
    ; Decrement animation frame counter; advance frame at zero.
    ;
    ; Behavior: decrements ANIM_FRAME_CTR, calls DRAW_BLINK_ALT
    ;   on expiry, reloads WAIT_LOOP_COUNT from ANIM_WAIT_TABLE
    ; Clobbers: A, X

    DEC     ANIM_FRAME_CTR    ; tick frame counter
    BNE     .reload           ; not zero -> just reload wait count
    JSR     DRAW_BLINK_ALT    ; counter hit zero -> draw alt char
.reload:
    LDX     ANIM_INDEX        ; current animation index
    LDA     ANIM_WAIT_TABLE,X ; look up wait count for this frame
    STA     WAIT_LOOP_COUNT   ; store for timed_wait to use
    RTS
```

Defines:
TICK_ANIM_COUNTER, used in chunks 52, 605b, and 663.
Uses ANIM_FRAME_CTR 604a, ANIM_INDEX 583a, ANIM_WAIT_TABLE 604a, DRAW_BLINK_ALT 472a, and WAIT_LOOP_COUNT 51.

ANIM_TICK_AND_WAIT combines an animation tick and a timed wait. It calls TICK_ANIM_COUNTER, then TIMED_WAIT to burn time. If the frame counter has reached zero, it calls DRAW_BLINK_NORMAL to restore the display and reloads ANIM_FRAME_CTR from the frame duration table ANIM_FRAME_TABLE (indexed by ANIM_INDEX).

Per-frame frame counts, used by ANIM_TICK_AND_WAIT.

605a ⟨*anim frame table* 605a⟩≡ (909)

```
    ORG     $7AB6
ANIM_FRAME_TABLE:
    HEX     00 02 02 04 09 13      ; per-frame frame counts (6 entries)
```

Defines:
ANIM_FRAME_TABLE, used in chunk 605b.

605b ⟨*anim tick and wait* 605b⟩≡ (909)

```
    ORG     $0D30
ANIM_TICK_AND_WAIT:
    SUBROUTINE ANIM_TICK_AND_WAIT
    ; Tick animation and burn time.
    ;
    ; Behavior: calls TICK_ANIM_COUNTER + TIMED_WAIT, then
    ;   advances frame (DRAW_BLINK_NORMAL + reload) on expiry
    ; Clobbers: A, X, Y

    JSR     TICK_ANIM_COUNTER  ; tick counter, reload wait count
    JSR     TIMED_WAIT         ; burn time
ANIM_CHECK_FRAME:
    LDA     ANIM_FRAME_CTR     ; did frame counter expire?
    BEQ     .advance           ; yes -> advance animation
    RTS                        ; no -> return

.advance:
    JSR     DRAW_BLINK_NORMAL  ; redraw blink char in normal video
    LDX     ANIM_INDEX         ; reload animation index
    LDA     ANIM_FRAME_TABLE,X ; look up frame duration
    STA     ANIM_FRAME_CTR     ; reset frame counter
    RTS
```

Defines:
ANIM_CHECK_FRAME, used in chunks 52 and 663.
ANIM_TICK_AND_WAIT, used in chunks 360, 390, 535, 545, 556, 603b, 607b, and 663.
Uses ANIM_FRAME_CTR 604a, ANIM_FRAME_TABLE 605a, ANIM_INDEX 583a, DRAW_BLINK_NORMAL 473b, TICK_ANIM_COUNTER 604b, and TIMED_WAIT 606a.

TIMED_WAIT calls the ROM WAIT wrapper CALL_ROM_WAIT repeatedly, decrementing WAIT_LOOP_COUNT until it reaches zero. If the count is already zero on entry, it returns immediately.

606a ⟨*timed wait* 606a⟩≡ (909)

```
    ORG     $0D49
TIMED_WAIT:
    SUBROUTINE TIMED_WAIT
    ; Wait for WAIT_LOOP_COUNT iterations via ROM WAIT.
    ;
    ; Input:  WAIT_LOOP_COUNT = iteration count (0 = immediate return)
    ; Clobbers: A

    LDA     WAIT_LOOP_COUNT ; anything to wait for?
    BNE     .wait           ; yes -> enter wait loop
    RTS                     ; no -> return immediately

.wait:
    JSR     CALL_ROM_WAIT   ; delay one unit
    DEC     WAIT_LOOP_COUNT ; decrement counter
    BNE     .wait           ; loop until zero
    RTS
```

Defines:
TIMED_WAIT, used in chunks 354, 532c, 545, and 605b.
Uses CALL_ROM_WAIT 607a and WAIT_LOOP_COUNT 51.

CALL_ROM_WAIT loads the wait duration from WAIT_DURATION and tail-calls the Apple II ROM WAIT routine (ROM_WAIT), which delays for approximately $A \times 2.5 + 13.5$ microseconds.

606b ⟨*defines* 25b⟩+≡ (909) ◁601 610a▷

```
ROM_WAIT                EQU     $FCA8   ; Apple II ROM WAIT routine
```

Defines:
ROM_WAIT, used in chunks 58, 607a, 696, 775, and 822.

ROM WAIT duration parameter, used by CALL_ROM_WAIT.

606c ⟨*wait duration* 606c⟩≡ (909)

```
    ORG     $7AAD
WAIT_DURATION:
    DC.B    $50                     ; ROM WAIT duration parameter
```

Defines:
WAIT_DURATION, used in chunk 607a.

607a ⟨*call rom wait* 607a⟩≡ (909)

```
      ORG     $0D58
  CALL_ROM_WAIT:
      SUBROUTINE CALL_ROM_WAIT
      ; Call the Apple II ROM WAIT routine.
      ;
      ; Inputs:
      ;   A | delay parameter (passed to ROM WAIT at [[ROM_WAIT]])
      ;
      ; Behavior:
      ;   Tail-calls the ROM WAIT routine. Delay is approximately
      ;   (26 + 27*A + 5*A*A) / 2 microseconds.
      ;
      ; Clobbers: A

      ; Load WAIT_DURATION and tail-call ROM WAIT ($FCA8).
      ;
      ; Clobbers: A

      LDA     WAIT_DURATION ; load duration parameter
      JMP     ROM_WAIT      ; tail-call ROM WAIT
```

Defines:
CALL_ROM_WAIT, used in chunks 52 and 606a.
Uses ROM_WAIT 606b and WAIT_DURATION 606c.

DISPATCH_ON_MODE provides a timing delay appropriate to the current display mode. It checks PLAY_HIGH_TONE: if the byte is $60 (an RTS opcode, used as a "sound disabled" sentinel), it calls DELAY_WITH_ANIMATION for a longer pause with blinking; otherwise it calls ANIM_TICK_AND_WAIT for a shorter animation step. This keeps the game's pacing consistent whether sound is enabled or not.

607b ⟨*dispatch on mode* 607b⟩≡ (909)

```
      ORG     $1F0C
      SUBROUTINE DISPATCH_ON_MODE
      ; Dispatch timing delay based on sound mode.
      ;
      ; Behavior: if sound disabled ($60 = RTS sentinel),
      ;   DELAY_WITH_ANIMATION; else ANIM_TICK_AND_WAIT
      ; Clobbers: A, X, Y
  ; Dispatches on PLAY_HIGH_TONE: if $60 (RTS) then DELAY_WITH_ANIMATION else ANIM_TICK_AND_WAIT
  DISPATCH_ON_MODE:
      LDA     PLAY_HIGH_TONE
      BAEQ    #$60,.mode_60
      JMP     ANIM_TICK_AND_WAIT
  .mode_60:
      JMP     DELAY_WITH_ANIMATION
```

Defines:
DISPATCH_ON_MODE, used in chunks 342, 346, and 371.
Uses ANIM_TICK_AND_WAIT 605b, DELAY_WITH_ANIMATION 603b, and PLAY_HIGH_TONE 647a.

16.4 Field utilities

CLAMP_CHAR_FIELD ensures that byte 6 of the record at MOB_PTR (MOB_PTR) has its lower 6 bits at least as large as byte 5's lower 5 bits. If byte 6 is smaller, its low bits are replaced with byte 5's value (preserving byte 6's top 2 bits). This clamps a stat floor— likely ensuring current HP does not fall below a minimum derived from another field.

608 ⟨*zero page defines* 25a⟩+≡ (909) ◁600b 618a▷

```
MOB_PTR                 EQU     $F4     ; mob data pointer (2 bytes, $F4/$F5)
```

Defines:
MOB_PTR, used in chunks 220, 312, 315, 317, 322, 325, 330, 332, 334, 338, 342, 346, 357, 364, 365, 368, 370–73, 385, 415, 417a, 419, 420, 422, 426, 430, 432, 447, 456, 475, 482, 485, 494, 542, 561, 609, 612–17, 619, 620, and 898.

609 ⟨*clamp char field* 609⟩≡ (909)

```
    ORG     $0A93
; ----------------------------------------------
; CLAMP_CHAR_FIELD -- Clamp HP floor to strength
;
; Ensures byte 6 (HP) low 6 bits >= byte 5
; (strength) low 5 bits.  If HP is below the
; strength floor, replaces HP bits with the
; strength value while preserving byte 6 flags.
;
; Input:   MOB_PTR -> 16-byte mob data record
; Output:  byte 6 HP clamped if needed
; Clobbers: A, Y, MOB_DATA_PTR (scratch)
; ----------------------------------------------
CLAMP_CHAR_FIELD:
    SUBROUTINE CLAMP_CHAR_FIELD
    ; Clamp entity strength and HP fields to valid ranges.
    ;
    ; Input:  ENTITY_PTR = entity record
    ; Behavior: caps strength (byte 5) at 31, HP (byte 6) at 63
    ; Clobbers: A, Y

    LDY     #MFIELD_STRENGTH            ; byte 5: strength
    LDA     (MOB_PTR),Y                 ; load strength byte
    AND     #STRENGTH_MASK              ; bits 4-0 = minimum HP
    STA     MOB_DATA_PTR                ; save as floor value
    INY                                 ; Y = MFIELD_HP (byte 6)
    LDA     (MOB_PTR),Y                 ; load HP byte
    AND     #HP_MASK                    ; bits 5-0 = current HP
    BALT    MOB_DATA_PTR,.clamp         ; HP < floor -> clamp
    RTS                                 ; HP >= floor -> done
.clamp:
    LDA     (MOB_PTR),Y                 ; reload HP byte
    AND     #HP_FLAGS_MASK              ; preserve flag bits 7-6
    ORA     MOB_DATA_PTR                ; set HP to floor value
    STA     (MOB_PTR),Y                 ; store clamped HP
    RTS
```

Defines:
CLAMP_CHAR_FIELD, used in chunks 420 and 494.

Uses ENTITY_PTR 561, HP_FLAGS_MASK 561, MFIELD_HP 561, MFIELD_STRENGTH 561, MOB_DATA_PTR 600b, and MOB_PTR 608.

16.4.1 Character selection browsing

These routines implement character selection browsing for the scene event menu. They walk the mob linked list (starting from the head pointer at CHAR_LIST_HEAD) looking for player characters (byte 4 of the mob data record < $15).

- FIND_FIRST_PC walks the list from the head, returning the first player character found.
- COPY_PC_NAME_TO_BUF copies the selected character's name string into the 20-byte display buffer at CHAR_SELECT_NAME.
- FIND_NEXT_PC follows the link from the current MOB_PTR to find the next player character, wrapping to the head if the chain ends.
- FIND_PREV_PC walks the entire list forward until wrapping back to the current character, yielding the predecessor.
- INIT_CHAR_SELECT is a scene event entry point: it sets up the scene, finds the first player character, and jumps to the character selection menu or back to MOVE_LOOP if none found.

BROWSE_CHAR_INDEX holds the mob index of the currently browsed character during selection. CHAR_SELECT_NAME is a 20-byte buffer inside the menu option strings area where the selected character's name is displayed.

610a ⟨*defines* 25b⟩+≡ (909) ◁606b 634▷

```
CHAR_SELECT_NAME    EQU     $7B4B   ; 20-byte name display buffer in menu
```

Defines:
CHAR_SELECT_NAME, used in chunks 281, 612, 613, 615, and 616.

FIND_FIRST_PC scans the mob linked list from the head pointer at CHAR_LIST_HEAD, following byte-2 links. For each mob whose byte 4 (appearance) is < $15 and bit 7 clear, it is a player character. On success, copies the name into CHAR_SELECT_NAME and returns with carry clear. If the list is exhausted (byte-2 link = 0), returns with carry set.

610b ⟨*var browse char index* 610b⟩≡ (909)

```
    ORG     $5A07
BROWSE_CHAR_INDEX:
    DC.B    $A0                     ; initial value (runtime: mob index of browsed character)
```

Defines:
BROWSE_CHAR_INDEX, used in chunks 612, 615, and 616.

611 ⟨*var input buffer* 611⟩≡ (909)

```
    ORG     $5AAC
INPUT_BUFFER:
    HEX     D3 D4 C1 A0 C1 D2 C7 AB B1 8D A0 CC C4 C1 A0 A3
    HEX     BE D3 CC C6 C4 D5 C3 8D A0 D3 D4 C1 A0 D4 C5 CD
    HEX     D0 8D A0 CC C4 C1 A0 A3 BC D3 CC C6 C4 D5 C3 8D
    HEX     A0 D3 D4 C1 A0 D4 C5 CD D0 AB B1 8D A0 CA D3 D2
    HEX     A0 D3 C8 CF D7 CE CD 8D A0 CC C4 C1 A0 D3 CC C6
    HEX     C4 D5 C3 AB
    ; runtime: 84-byte input text display buffer
```

Defines:
INPUT_BUFFER, used in chunks 250, 258–60, and 616.

612 ⟨*find first pc* 612⟩≡ (909) 613▷

```
    ORG     $0AAE
; ----------------------------------------------
; FIND_FIRST_PC -- Find first player character
;
; Scans mob linked list from CHAR_LIST_HEAD.
; A PC has appearance < NPC_APPEARANCE_MIN and
; bit 7 clear.
;
; Input:   none
; Output:  C=0: found, BROWSE_CHAR_INDEX set,
;               name copied to CHAR_SELECT_NAME
;          C=1: no PCs in list
; Clobbers: A, X, Y, MOB_DATA_PTR, MOB_PTR,
;           TARGET_REC
; ----------------------------------------------
FIND_FIRST_PC:
    SUBROUTINE FIND_FIRST_PC
    ; Find the first player character in the group list.
    ;
    ; Input:  CHAR_PTR = group head
    ; Output: MOB_PTR = first PC (appearance < NPC_APPEARANCE_MIN), carry clear
    ;         carry set if none found
    ; Clobbers: A, Y

    LDA     CHAR_LIST_HEAD              ; head of character linked list
.check:
    CMP     #$00                        ; end of list?
    BNE     .have_entry                 ; no -> examine this entry
    SEC                                 ; yes -> not found
    RTS
.have_entry:
    STA     BROWSE_CHAR_INDEX           ; save current mob index
    JSR     GET_MOB_DATA                ; -> MOB_DATA_PTR = mob record
    LDY     #MFIELD_APPEARANCE          ; byte 4: appearance
    LDA     (MOB_DATA_PTR),Y            ; load appearance
    BMI     .skip                       ; bit 7 set -> not a PC
    CMP     #NPC_APPEARANCE_MIN         ; < $15?
    BMI     COPY_PC_NAME_TO_BUF         ; yes -> player character
.skip:
    LDY     #MFIELD_NEXT                ; byte 2: next link
    LDA     (MOB_DATA_PTR),Y            ; follow chain
    JMP     .check                      ; check next entry
```

Defines:
FIND_FIRST_PC, used in chunks 283 and 615–17.

Uses BROWSE_CHAR_INDEX 610b, CHAR_LIST_HEAD 53a, CHAR_PTR 591a, CHAR_SELECT_NAME 610a, COPY_PC_NAME_TO_BUF 613, GET_MOB_DATA 602, MFIELD_APPEARANCE 561, MFIELD_NEXT 561, MOB_DATA_PTR 600b, MOB_PTR 608, and TARGET_REC 618b.

COPY_PC_NAME_TO_BUF copies the name of the mob pointed to by MOB_DATA_PTR into the CHAR_SELECT_NAME display buffer. It first copies MOB_DATA_PTR to MOB_PTR, reads bytes 0–1 of the record (the name string pointer) into TARGET_REC, then loops 20 times copying characters. Byte 0 of the name string is the length; characters beyond the length are replaced with spaces ($20). Returns with carry clear (success).

613 ⟨*find first pc* 612⟩+≡ (909) ◁612 614▷

```
      ORG     $0ACE
  COPY_PC_NAME_TO_BUF:
  ; ------------------------------------------------
  ; COPY_PC_NAME_TO_BUF -- Copy PC name to buffer
  ;
  ; Copies the length-prefixed name string from the
  ; mob record at MOB_DATA_PTR into the 20-byte
  ; CHAR_SELECT_NAME display buffer, padding with
  ; spaces.
  ;
  ; Input:   MOB_DATA_PTR -> mob data record
  ; Output:  C=0 (success), name in CHAR_SELECT_NAME
  ; Clobbers: A, X, Y, MOB_PTR, TARGET_REC,
  ;           MOB_DATA_PTR
  ; ------------------------------------------------
      LDA     MOB_DATA_PTR                ; \
      STA     MOB_PTR                     ;  | MOB_PTR = MOB_DATA_PTR
      LDA     MOB_DATA_PTR+1              ;  |
      STA     MOB_PTR+1                   ; /
      LDY     #MFIELD_NAME             ; byte 0: name ptr low
      LDA     (MOB_PTR),Y                 ; load name ptr low
      STA     TARGET_REC                  ; \
      INY                                 ;  | TARGET_REC = name string
      LDA     (MOB_PTR),Y                 ;  | (Y = MFIELD_NAME+1)
      STA     TARGET_REC+1                ; /
      LDA     #<CHAR_SELECT_NAME          ; \
      STA     MOB_DATA_PTR                ;  | MOB_DATA_PTR = display buf
      LDA     #>CHAR_SELECT_NAME          ;  |
      STA     MOB_DATA_PTR+1              ; /
      LDY     #$00                        ; string offset 0
      LDA     (TARGET_REC),Y              ; byte 0 = name length
      TAX                                 ; X = chars remaining
  .copy_loop:
      INY                                 ; advance to next char
      LDA     (TARGET_REC),Y              ; load name character
      DEX                                 ; decrement remaining count
      BPL     .store                      ; count >= 0 -> keep char
      LDA     #$20                        ; else pad with space
  .store:
      STA     (MOB_DATA_PTR),Y            ; store into display buffer
      CPY     #NAME_BUF_LEN               ; copied 20 chars?
      BNE     .copy_loop                  ; no -> continue
```

```
    CLC                                 ; success
    RTS
```

Defines:
COPY_PC_NAME_TO_BUF, used in chunks 612, 615, and 616.
Uses CHAR_SELECT_NAME 610a, MFIELD_NAME 561, MOB_DATA_PTR 600b, MOB_PTR 608, NAME_BUF_LEN 601, and TARGET_REC 618b.

FIND_NEXT_PC_WRAP is a trivial wrapper that calls FIND_NEXT_PC and returns. This exists as a separate entry point at FIND_NEXT_PC_WRAP so callers can JSR to it.

614 ⟨*find first pc* 612⟩+≡ (909) ◁613 615▷

```
    ORG     $0AFE
; ------------------------------------------------
; FIND_NEXT_PC_WRAP -- JSR wrapper for FIND_NEXT_PC
;
; Input:   MOB_PTR -> current mob data record
; Output:  (see FIND_NEXT_PC)
; Clobbers: (see FIND_NEXT_PC)
; ------------------------------------------------
FIND_NEXT_PC_WRAP:
    JSR     FIND_NEXT_PC                ; find next PC in list
    RTS
```

Defines:
FIND_NEXT_PC_WRAP, used in chunk 283.
Uses FIND_NEXT_PC 615 and MOB_PTR 608.

FIND_NEXT_PC follows the byte-2 link from MOB_PTR to find the next player character in the list. If byte 2 is zero (end of chain), it wraps around by calling FIND_FIRST_PC. Otherwise it resolves the link via GET_MOB_DATA, checks byte 4 for a PC (appearance < $15, bit 7 clear), and either copies the name or recurses.

615 ⟨*find first pc* 612⟩+≡ (909) ◁614 616▷

```
      ORG     $0B02
  ; ------------------------------------------------
  ; FIND_NEXT_PC -- Find next PC in linked list
  ;
  ; Follows MFIELD_NEXT link from MOB_PTR.  Wraps
  ; to list head if chain ends.  Recurses through
  ; non-PC entries.
  ;
  ; Input:   MOB_PTR -> current mob data record
  ; Output:  BROWSE_CHAR_INDEX set to next PC,
  ;          name copied to CHAR_SELECT_NAME
  ; Clobbers: A, X, Y, MOB_DATA_PTR, MOB_PTR,
  ;           TARGET_REC
  ; ------------------------------------------------
  FIND_NEXT_PC:
      LDY     #MFIELD_NEXT                ; byte 2: next link
      LDA     (MOB_PTR),Y                 ; load next mob index
      BNE     .have_link                  ; nonzero -> follow it
      JSR     FIND_FIRST_PC               ; zero -> wrap to first PC
      RTS
  .have_link:
      STA     BROWSE_CHAR_INDEX           ; save mob index
      JSR     GET_MOB_DATA                ; -> MOB_DATA_PTR = mob record
      LDA     MOB_DATA_PTR                ; \
      STA     MOB_PTR                     ;  | MOB_PTR = MOB_DATA_PTR
      LDA     MOB_DATA_PTR+1              ;  |
      STA     MOB_PTR+1                   ; /
      LDY     #MFIELD_APPEARANCE          ; byte 4: appearance
      LDA     (MOB_PTR),Y                 ; load appearance
      BMI     FIND_NEXT_PC                ; bit 7 set -> skip
      CMP     #NPC_APPEARANCE_MIN         ; < $15?
      BPL     FIND_NEXT_PC                ; no -> skip, try next
      JSR     COPY_PC_NAME_TO_BUF         ; yes -> copy name
      RTS
```

Defines:
FIND_NEXT_PC, used in chunks 614 and 616.

Uses BROWSE_CHAR_INDEX 610b, CHAR_SELECT_NAME 610a, COPY_PC_NAME_TO_BUF 613, FIND_FIRST_PC 612, GET_MOB_DATA 602, MFIELD_APPEARANCE 561, MFIELD_NEXT 561, MOB_DATA_PTR 600b, MOB_PTR 608, and TARGET_REC 618b.

FIND_PREV_PC finds the player character that precedes the current one in the circular linked list. It saves the current BROWSE_CHAR_INDEX in INPUT_BUFFER, then walks forward from the first PC, recording each PC's index in INPUT_BUFFER+1, until wrapping back to the saved index. At that point INPUT_BUFFER+1 holds the predecessor. It then resolves that index and copies the name.

616 ⟨*find first pc* 612⟩+≡ (909) ◁615 617▷

```
    ORG     $0B28
; ------------------------------------------------
; FIND_PREV_PC -- Find previous PC in linked list
;
; Walks forward from list head, recording each PC
; index, until wrapping back to the current one.
; The last recorded index is the predecessor.
;
; Input:   BROWSE_CHAR_INDEX = current PC index
; Output:  BROWSE_CHAR_INDEX = predecessor PC,
;          name copied to CHAR_SELECT_NAME
; Clobbers: A, X, Y, MOB_DATA_PTR, MOB_PTR,
;           TARGET_REC, INPUT_BUFFER (2 bytes)
; ------------------------------------------------
FIND_PREV_PC:
    LDA     BROWSE_CHAR_INDEX          ; \
    STA     INPUT_BUFFER               ; / save current index
    JSR     FIND_FIRST_PC              ; start from first PC
.scan:
    LDA     BROWSE_CHAR_INDEX          ; \
    STA     INPUT_BUFFER+1             ; / save as candidate predecessor
    JSR     FIND_NEXT_PC               ; advance to next PC
    LDA     BROWSE_CHAR_INDEX          ; wrapped back to original?
    CMP     INPUT_BUFFER               ; compare with saved index
    BNE     .scan                      ; no -> keep scanning
    LDA     INPUT_BUFFER+1             ; yes -> predecessor index
    STA     BROWSE_CHAR_INDEX          ; set as current
    JSR     GET_MOB_DATA               ; resolve to record pointer
    JSR     COPY_PC_NAME_TO_BUF        ; copy name to buffer
    RTS
```

Defines:
FIND_PREV_PC, used in chunk 283.

Uses BROWSE_CHAR_INDEX 610b, CHAR_SELECT_NAME 610a, COPY_PC_NAME_TO_BUF 613, FIND_FIRST_PC 612, FIND_NEXT_PC 615, GET_MOB_DATA 602, INPUT_BUFFER 611, MOB_DATA_PTR 600b, MOB_PTR 608, and TARGET_REC 618b.

INIT_CHAR_SELECT is a scene event handler entry point. It marks the event as handled (SCENE_EVENT_RESULT = 1), calls SCENE_SETUP to initialize the scene, then searches for the first player character. If found (carry clear), it jumps to ENTER_CHAR_SELECT. If no player characters exist (carry set), it falls back to MOVE_LOOP.

617 ⟨*find first pc* 612⟩+≡ (909) ◁616

```
      ORG     $0B4F
  ; ------------------------------------------------
  ; INIT_CHAR_SELECT -- Scene event: character select
  ;
  ; Marks event handled, sets up scene, then finds
  ; first PC.  Jumps to ENTER_CHAR_SELECT on success
  ; or MOVE_LOOP if no PCs exist.
  ;
  ; Input:   none (scene event entry point)
  ; Output:  jumps to ENTER_CHAR_SELECT or MOVE_LOOP
  ; Clobbers: A, X, Y, MOB_DATA_PTR, MOB_PTR,
  ;           TARGET_REC
  ; ------------------------------------------------
  INIT_CHAR_SELECT:
      STOB    #$01,SCENE_EVENT_RESULT    ; mark event handled
      JSR     SCENE_SETUP                ; set up scene boundaries
      JSR     FIND_FIRST_PC              ; find first player character
      BCC     .found                     ; carry clear -> found
      JMP     MOVE_LOOP                  ; carry set -> no PCs
  .found:
      JMP     ENTER_CHAR_SELECT          ; -> character selection menu
```

Defines:
INIT_CHAR_SELECT, used in chunk 281.

Uses ENTER_CHAR_SELECT 283, FIND_FIRST_PC 612, MOB_DATA_PTR 600b, MOB_PTR 608, MOVE_LOOP 232, SCENE_EVENT_RESULT 207, SCENE_SETUP 494, STOB 8c, and TARGET_REC 618b.

The scene loop calls LOAD_SCENE_FROM_DISK each iteration to load the current scene's text sector from disk. If carry is set (read failed), the loop exits. Otherwise, it displays the status bar, renders the scene text from SCENE_TEXT_DATA, plays the attract tune, and checks SCENE_TRANSITION for a scene change.

The SCENE_TEXT_DATA region is populated dynamically by LOAD_SCENE_DATA from disk. Its contents change with each scene.

SAVE_ZP_CONTEXT and RESTORE_ZP_CONTEXT save and restore four zero-page pointer pairs to/from shadow locations in the $5A page: TARGET_REC (PRINT_STRING_ADDR), ENTITY_PTR (ENTITY_PTR), DATA_PTR (ENTITY_REC), and MOB_PTR (MOB_PTR). These pointers are heavily aliased—the same zero-page bytes serve different roles in different routines—so saving and restoring them is necessary when entering nested operations like scene loading or disk I/O.

618a ⟨*zero page defines* 25a⟩+≡ (909) ◁608 618b▷

```
DATA_PTR                EQU     $BE     ; general data pointer / return value (2 bytes)
```

Defines:
DATA_PTR, used in chunks 417, 422, 423, 429, 481, 488, 537, 583b, 619, and 620.

618b ⟨*zero page defines* 25a⟩+≡ (909) ◁618a 630a▷

```
TARGET_REC              EQU     $BC     ; secondary entity record pointer in combat/reorder (2 bytes
```

Defines:
TARGET_REC, used in chunks 312, 317, 320, 322, 328, 334, 338, 349, 357, 365, 385, 420, 430, 432, 478, 481, 489a, 494, 612, 613, 615–20, and 898.

618c ⟨*var saved target rec* 618c⟩≡ (909)

```
        ORG     $5A9A
SAVED_TARGET_REC:
        DC.B    $D9,$8D                 ; initial value (runtime: saved TARGET_REC ($BC/$BD, 2 bytes))
```

Defines:
SAVED_TARGET_REC, used in chunks 619c and 620.
Uses TARGET_REC 618b.

618d ⟨*var saved entity ptr* 618d⟩≡ (909)

```
        ORG     $5A9C
SAVED_ENTITY_PTR:
        DC.B    $A0,$CC                 ; initial value (runtime: saved ENTITY_PTR ($F8/$F9, 2 bytes))
```

Defines:
SAVED_ENTITY_PTR, used in chunks 619c and 620.
Uses ENTITY_PTR 561.

619a ⟨*var saved data ptr* 619a⟩≡ (909)

```
      ORG     $5A9E
  SAVED_DATA_PTR:
      DC.B    $C4,$C1                 ; initial value (runtime: saved DATA_PTR ($BE/$BF, 2 bytes))
```

Defines:
SAVED_DATA_PTR, used in chunks 619c and 620.
Uses DATA_PTR 618a.

619b ⟨*var saved mob ptr* 619b⟩≡ (909)

```
      ORG     $5AA0
  SAVED_MOB_PTR:
      DC.B    $A0,$A8                 ; initial value (runtime: saved MOB_PTR ($F4/$F5, 2 bytes))
```

Defines:
SAVED_MOB_PTR, used in chunks 619c and 620.
Uses MOB_PTR 608.

619c ⟨*save zp context* 619c⟩≡ (909)

```
      ORG     $6D53
  SAVE_ZP_CONTEXT:
      SUBROUTINE SAVE_ZP_CONTEXT
      ; Save four ZP pointer pairs to shadow locations in the $5A page.
      ;
      ; Behavior:
      ;   Copies TARGET_REC, ENTITY_PTR, DATA_PTR, and MOB_PTR
      ;   to SAVED_TARGET_REC, SAVED_ENTITY_PTR, SAVED_DATA_PTR,
      ;   and SAVED_MOB_PTR respectively.
      ;
      ; Modifies: SAVED_TARGET_REC, SAVED_ENTITY_PTR, SAVED_DATA_PTR, SAVED_MOB_PTR
      ; Clobbers: A

      MOVW    TARGET_REC,SAVED_TARGET_REC
      MOVW    ENTITY_PTR,SAVED_ENTITY_PTR
      MOVW    DATA_PTR,SAVED_DATA_PTR
      MOVW    MOB_PTR,SAVED_MOB_PTR
      RTS
```

Defines:
SAVE_ZP_CONTEXT, used in chunk 527.
Uses DATA_PTR 618a, ENTITY_PTR 561, MOB_PTR 608, MOVW 9a, SAVED_DATA_PTR 619a, SAVED_ENTITY_PTR 618d, SAVED_MOB_PTR 619b, SAVED_TARGET_REC 618c, and TARGET_REC 618b.

RESTORE_ZP_CONTEXT is the inverse: it copies the saved values back into zero page, restoring the caller's pointer context after a scene loop or other operation that may have clobbered them.

620 ⟨*restore zp context* 620⟩≡ (909)

```
    ORG     $6D7C
RESTORE_ZP_CONTEXT:
    SUBROUTINE RESTORE_ZP_CONTEXT
    ; Restore four ZP pointer pairs from shadow locations.
    ;
    ; Behavior:
    ;   Copies SAVED_TARGET_REC, SAVED_ENTITY_PTR, SAVED_DATA_PTR,
    ;   and SAVED_MOB_PTR back to their ZP locations.
    ;
    ; Modifies: TARGET_REC, ENTITY_PTR, DATA_PTR, MOB_PTR (2 bytes each)
    ; Clobbers: A

    MOVW    SAVED_TARGET_REC,TARGET_REC
    MOVW    SAVED_ENTITY_PTR,ENTITY_PTR
    MOVW    SAVED_DATA_PTR,DATA_PTR
    MOVW    SAVED_MOB_PTR,MOB_PTR
    RTS
```

Defines:
RESTORE_ZP_CONTEXT, used in chunk 527.
Uses DATA_PTR 618a, ENTITY_PTR 561, MOB_PTR 608, MOVW 9a, SAVED_DATA_PTR 619a, SAVED_ENTITY_PTR 618d, SAVED_MOB_PTR 619b, SAVED_TARGET_REC 618c, and TARGET_REC 618b.

Chapter 17

Text and Display Primitives

The PRINT_STRING routine prints a Pascal-style string (length first), from an address stored in PRINT_STRING_ADDR, at the current cursor position.

PRINT_STRING_AT_ADDR stores the X and Y registers into PRINT_STRING_ADDR as a 16-bit string pointer, then falls through to PRINT_STRING. This provides a convenient entry point for callers that pass the string address in registers rather than pre-loading the zero-page pointer.

621 ⟨*print string* 621⟩≡ (909) 622▷

```
      ORG     $7758
  PRINT_STRING_AT_ADDR:
      SUBROUTINE PRINT_STRING_AT_ADDR
      ; Set string pointer and print a length-prefixed string.
      ;
      ; Inputs:
      ;   X | low byte of string address
      ;   Y | high byte of string address
      ;
      ; Behavior:
      ;   Stores X/Y into PRINT_STRING_ADDR, then falls through to PRINT_STRING.
      ;
      ; Clobbers: A, X, Y

      STX     PRINT_STRING_ADDR
      STY     PRINT_STRING_ADDR+1

      ; falls through to PRINT_STRING at [[PRINT_STRING]]
```

Defines:
PRINT_STRING_AT_ADDR, used in chunks 556 and 574.
Uses PRINT_STRING 622 and PRINT_STRING_ADDR 576c.

PRINT_STRING prints a length-prefixed string from \PRINT_STRING_ADDR/\PRINT_STRING_ADDR+1 via COUT. PRINT_STRING_AT_ADDR stores X/Y into \PRINT_STRING_ADDR/\PRINT_STRING_ADDR+1 first, then falls through.

622 ⟨*print string* 621⟩+≡ (909) ◁621

```
      ORG     $775C
  PRINT_STRING:
      SUBROUTINE PRINT_STRING
      ; Print a length-prefixed string via COUT1.
      ;
      ; Inputs:
      ;   PRINT_STRING_ADDR | pointer to length-prefixed string
      ;
      ; Behavior:
      ;   Reads the first byte as a length count, then outputs each
      ;   subsequent byte through ROM_COUT1.
      ;
      ; Clobbers: A, X, Y

      LDY     #$00
      LDA     (PRINT_STRING_ADDR),Y ; first byte = length
      TAX
  .loop:
      DEX
      BPL     .next
      RTS                           ; done

  .next:
      INY
      LDA     (PRINT_STRING_ADDR),Y ; read next char
      JSR     ROM_COUT1
      JMP     .loop
```

Defines:
PRINT_STRING, used in chunks 253, 466, 538, 550, 556, 578, 621, and 629.
Uses PRINT_STRING_ADDR 576c and ROM_COUT1 571a.

Sends Ctrl-A and '0' to COUT (selects character set 0 in the HRCG):

623a ⟨*print ctrl ab* 623a⟩≡ (909)

```
      ORG     $774E
  PRINT_CTRL_AB:
      SUBROUTINE PRINT_CTRL_AB
      ; Select character set 0 in the HRCG.
      ;
      ; Behavior:
      ;   Sends Ctrl-A ($81) followed by '0' ($B0) to ROM_COUT1,
      ;   which selects the standard character set.
      ;
      ; Clobbers: A

      LDA     #$81 ; Ctrl-A with high bit
      JSR     ROM_COUT1
      LDA     #$B0 ; '0' with high bit
      JMP     ROM_COUT1
```

Defines:
PRINT_CTRL_AB, used in chunks 253, 259, 550, 552, 556, 574, 625, 627, 629, and 637.
Uses ROM_COUT1 571a.

Outputs Ctrl-N (normal video mode) to the Apple II character output routine via COUT1. $8E is Ctrl-N with the high bit set. This is distinct from SET_NORMAL_VIDEO at $75F9 which uses COUT instead.

623b ⟨*print ctrl n* 623b⟩≡ (909)

```
      ORG     $7735
  PRINT_CTRL_N:
      SUBROUTINE PRINT_CTRL_N
      ; Set normal video mode via COUT1.
      ;
      ; Behavior:
      ;   Sends Ctrl-N ($8E) to ROM_COUT1 to restore normal
      ;   (non-inverse) video output.
      ;
      ; Clobbers: A

      LDA     #$8E      ; Ctrl-N with high bit
      JMP     ROM_COUT1 ; output via COUT1
```

Defines:
PRINT_CTRL_N, used in chunks 253 and 473b.
Uses ROM_COUT1 571a.

Ali Baba provides several convenience routines which translate to HRCG control codes.

624 ⟨*control routines* 624⟩≡ (909)

```
      ORG     $7730
  SET_INVERSE_VIDEO:
      SUBROUTINE SET_INVERSE_VIDEO
      ; Set inverse video mode via COUT1.
      ;
      ; Behavior:
      ;   Sends Ctrl-I ($89) to ROM_COUT1 to enable inverse video output.
      ;
      ; Clobbers: A

      LDA     #$89        ; ctrl-I
      JMP     ROM_COUT1
```

Defines:
SET_INVERSE_VIDEO, used in chunk 252.
Uses ROM_COUT1 571a.

Two more convenience wrappers set the text window behavior. SET_TEXT_WINDOW_WRAP sends ctrl-O (next-shifted) followed by ctrl-W (shifted: enable wrap mode), so that text reaching the bottom of the window wraps to the top instead of scrolling. SET_TEXT_WINDOW_SCROLL sends ctrl-O followed by ctrl-S (shifted: clear wrap flag), restoring normal scroll behavior. These are used when switching between the status area (which scrolls) and the map area (which wraps or is managed manually).

625 ⟨*text window routines* 625⟩≡ (909)

```
      ORG     $773A
  SET_TEXT_WINDOW_WRAP:
      SUBROUTINE SET_TEXT_WINDOW_WRAP
      ; Enable text window wrap mode.
      ;
      ; Behavior:
      ;   Sends Ctrl-O then Ctrl-W to ROM_COUT1, enabling wrap mode
      ;   so text wraps to the top of the window instead of scrolling.
      ;
      ; Clobbers: A

      LDA     #$8F ; ctrl-O
      JSR     ROM_COUT1
      LDA     #$97 ; ctrl-W
      JMP     ROM_COUT1

  SET_TEXT_WINDOW_SCROLL:
      SUBROUTINE SET_TEXT_WINDOW_SCROLL
      ; Enable text window scroll mode.
      ;
      ; Behavior:
      ;   Sends Ctrl-O then Ctrl-S to ROM_COUT1, clearing the wrap
      ;   flag and restoring normal scroll behavior.
      ;
      ; Clobbers: A

      LDA     #$8F ; ctrl-O
      JSR     ROM_COUT1
      LDA     #$93 ; ctrl-S
      JMP     ROM_COUT1

  ; PRINT_CTRL_AB (PRINT_CTRL_AB) follows at [[PRINT_CTRL_AB]], defined in its own chunk
```

Defines:
SET_TEXT_WINDOW_SCROLL, used in chunk 639.
SET_TEXT_WINDOW_WRAP, used in chunks 564 and 639.
Uses PRINT_CTRL_AB 623a and ROM_COUT1 571a.

The game screen is divided into an upper map area and a lower status/input area. These routines set the text window's top-left corner via the HRCG's ctrl-V command. SET_TEXT_WINDOW_UPPER_LEFT_ALL sets the window to start at row 0, giving access to the full screen. SET_TEXT_WINDOW_UPPER_LEFT_LOW sets it to row 21, confining output to the bottom three rows used for status messages and player input.

626 ⟨*set text window upper left* 626⟩≡ (909)

```
    ORG     $792D
SET_TEXT_WINDOW_UPPER_LEFT_ALL:
    SUBROUTINE SET_TEXT_WINDOW_UPPER_LEFT_ALL
    ; Set text window origin to row 0, col 0 (full screen).
    ;
    ; Behavior:
    ;   Sets TEXT_ROW to 0 and TEXT_COL to 0, then sends Ctrl-V
    ;   to ROM_COUT1 to reposition the HRCG text window.
    ;
    ; Modifies: TEXT_ROW, TEXT_COL
    ; Clobbers: A

    LDA     #0
    JMP     SET_TEXT_WINDOW_UPPER_LEFT

SET_TEXT_WINDOW_UPPER_LEFT_LOW:
    LDA     #21

    ; fall through

SET_TEXT_WINDOW_UPPER_LEFT:

    STA     TEXT_ROW
    STOB    #0,TEXT_COL
    LDA     #$96        ; ctrl-V (set text window upper left)
    JMP     ROM_COUT1
```

Uses ROM_COUT1 571a, STOB 8c, TEXT_COL 572a, and TEXT_ROW 572a.

These two routines share the same code; they differ only in the window-bottom value loaded in A. The BNE at $76C0 forms a "stretchy branch" that skips the LDA \#$14, so both entry points share all code from $76C4 onward.

627 ⟨*set text window bottom* 627⟩≡ (909)

```
    ORG     $76BE
SET_CURSOR_ROW21:
    SUBROUTINE SET_CURSOR_ROW21
    ; Position cursor at row 21 and configure status area.
    ;
    ; Behavior:
    ;   Sets TEXT_ROW to 21 and TEXT_COL to 0, then sends charset
    ;   select and Ctrl-F to ROM_COUT1 to position the cursor.
    ;
    ; Modifies: TEXT_ROW, TEXT_COL
    ; Clobbers: A

    LDA     #$15     ; WNDBTM = 21
    BNE     .shared  ; always taken (stretchy branch)

SET_TEXT_WINDOW:
SET_TEXT_WINDOW_BOTTOM:
    LDA     #$14     ; WNDBTM = 20

.shared:
    STA     TEXT_ROW
    STOB    #$00,TEXT_COL ; WNDLFT = 0
    JSR     PRINT_CTRL_AB
    LDA     #$86     ; Ctrl-F with high bit
    JMP     ROM_COUT1
```

Defines:
SET_CURSOR_ROW21, used in chunks 56, 220, 225, 283, 289, 295, 297, 305, 325, 338, 342, 346, 349, 354, 357, 360, 363, 377, 378, 385, 404, 428, 433, 436, 440, 447, 454, 456, 483b, 542, 590, and 898.
SET_TEXT_WINDOW, never used.
SET_TEXT_WINDOW_BOTTOM, used in chunks 527, 535, and 564.
Uses PRINT_CTRL_AB 623a, ROM_COUT1 571a, STOB 8c, TEXT_COL 572a, and TEXT_ROW 572a.

The PRINT_BOTTOM_CENTERED centers text on row 23, which is the bottom of the screen.

628 ⟨*print bottom centered* 628⟩≡ (909)

```
      ORG     $7705
  PRINT_BOTTOM_CENTERED:
      SUBROUTINE PRINT_BOTTOM_CENTERED
      ; Print a centered string at the bottom of the screen.
      ;
      ; Inputs:
      ;   PRINT_STRING_ADDR | pointer to length-prefixed string
      ;
      ; Behavior:
      ;   Sets cursor to row 23, col 0, then falls through to
      ;   PRINT_CENTERED_PLUS_COL.
      ;
      ; Modifies: TEXT_ROW, TEXT_COL
      ; Clobbers: A, X, Y

      STOB    #$17,TEXT_ROW
      STOB    #$00,TEXT_COL
      JMP     PRINT_CENTERED_PLUS_COL
```

Defines:
PRINT_BOTTOM_CENTERED, used in chunks 338, 342, 375, 483–85, 542, and 637.
Uses PRINT_CENTERED_PLUS_COL 629, PRINT_STRING_ADDR 576c, STOB 8c, TEXT_COL 572a, and TEXT_ROW 572a.

The PRINT_CENTERED_PLUS_COL centers text, but moves it right by the current cursor's column. If the column is zero, the routine would center the text on the screen.

629 ⟨*print centered plus col* 629⟩≡ (909)

```
      ORG     $7710
  PRINT_CENTERED_PLUS_COL:
      SUBROUTINE PRINT_CENTERED_PLUS_COL
      ; Center and print a string, offset by current column.
      ;
      ; Inputs:
      ;   PRINT_STRING_ADDR | pointer to length-prefixed string
      ;   TEXT_COL | additional column offset added to centering
      ;
      ; Behavior:
      ;   Computes (40 - string_length) / 2 + TEXT_COL as the starting
      ;   column, selects charset 0, then prints the string.
      ;
      ; Modifies: TEXT_COL
      ; Clobbers: A, X, Y

      LDA     #40
      SEC
      LDY     #$00
      SBC     (PRINT_STRING_ADDR),Y
      CLC
      LSR                             ; accumulator
      ADC     TEXT_COL
      STA     TEXT_COL
      JSR     PRINT_CTRL_AB
      JMP     PRINT_STRING
```

Defines:
PRINT_CENTERED_PLUS_COL, used in chunks 552 and 628.
Uses PRINT_CTRL_AB 623a, PRINT_STRING 622, PRINT_STRING_ADDR 576c, and TEXT_COL 572a.

Converts a 16-bit unsigned value in NUM_VALUE to up to 5 ASCII decimal digits, stored sequentially at $(DECIMAL_OUT_PTR),Y$ (with Y incrementing). Leading zeros are suppressed unless the value is exactly zero (prints "0"). The output uses Apple II high-ASCII ($$B0$–$$B9$).

The driver at $0DA1 calls the digit-extraction helper at $0DE1 five times with divisors 10000, 1000, 100, 10, and 1. The helper performs repeated subtraction of $5A1C/$5A1D from NUM_VALUE, counting iterations as the digit value.

A second entry point at $0DA4 (1 caller) skips the initialization of Y and the leading-zero flag, allowing the caller to control the starting output position and suppression behavior.

NUM_TO_DECIMAL converts a 16-bit value in NUM_VALUE (PRINT_STRING_ADDR) to a decimal ASCII string stored at DECIMAL_OUT_PTR (MOB_DATA_PTR). It divides repeatedly by powers of ten (10000, 1000, 100, 10, 1), accumulating each digit in NUM_DIGIT and writing it to the output buffer. NUM_LEADING suppresses leading zeros: it starts at 0 (suppress) and flips to 1 after the first nonzero digit. NUM_DIVISOR ($5A1C/$5A1D) holds the current power-of-ten divisor.

630a ⟨*zero page defines* 25a⟩+≡ (909) ◁618b 646▷

```
DECIMAL_OUT_PTR         EQU     $BA     ; num-to-decimal output buffer pointer (2 bytes)
NUM_VALUE               EQU     $BC     ; 16-bit numeric value for division (2 bytes)
```

Defines:
DECIMAL_OUT_PTR, used in chunks 437, 447, 556, 632, 685, and 686.
NUM_VALUE, used in chunks 56, 380, 382, 437, 447, 452, 454, 556, 632, and 686.

630b ⟨*var num digit* 630b⟩≡ (909)

```
    ORG     $5A1B
NUM_DIGIT:
    DC.B    $C1                     ; initial value (runtime: current digit accumulator (ASCII $B0-$
```

Defines:
NUM_DIGIT, used in chunk 632.

630c ⟨*var num divisor* 630c⟩≡ (909)

```
    ORG     $5A1C
NUM_DIVISOR:
    DC.B    $A0,$A8                 ; initial value (runtime: 16-bit divisor for decimal conversion)
```

Defines:
NUM_DIVISOR, used in chunk 632.

631 ⟨*var num leading* 631⟩≡ (909)

```
    ORG     $5A1E
NUM_LEADING:
    DC.B    $C1                     ; initial value (runtime: leading-zero suppress flag (0=suppress
```

Defines:
NUM_LEADING, used in chunk 632.

632 ⟨*num to decimal* 632⟩≡ (909)

```
    ORG     $0DA1
NUM_TO_DECIMAL:
    SUBROUTINE NUM_TO_DECIMAL
    ; Convert a 16-bit value to up to 5 ASCII decimal digits.
    ;
    ; Inputs:
    ;   NUM_VALUE | 16-bit unsigned value to convert
    ;   DECIMAL_OUT_PTR | pointer to output buffer
    ;
    ; Behavior:
    ;   Divides NUM_VALUE by 10000, 1000, 100, 10, and 1 via
    ;   repeated subtraction, storing high-ASCII digit characters
    ;   at DECIMAL_OUT_PTR with leading zeros suppressed.
    ;
    ; Modifies: NUM_VALUE, NUM_DIVISOR, NUM_DIGIT, NUM_LEADING
    ; Clobbers: A, X, Y

    LDY     #$00                ; output index = 0
    STY     NUM_LEADING         ; leading-zero flag = 0 (suppress)
NUM_TO_DECIMAL_CONT:
    STOW    $2710,NUM_DIVISOR ; divisor = $2710 = 10000
    JSR     .extract_digit      ; ten-thousands digit
    STOW    $03E8,NUM_DIVISOR ; divisor = $03E8 = 1000
    JSR     .extract_digit      ; thousands digit
    STOW    $0064,NUM_DIVISOR ; divisor = $0064 = 100
    JSR     .extract_digit      ; hundreds digit
    STOB    #$0A,NUM_DIVISOR ; divisor = $000A = 10
    JSR     .extract_digit      ; tens digit
    STOB    #$01,NUM_DIVISOR ; divisor = $0001 = 1
    STA     NUM_LEADING         ; /  force output (no suppress)
    JSR     .extract_digit      ; ones digit
    RTS

; --- Digit extraction helper ($0DE1) ---
; Divides NUM_VALUE/NUM_VALUE+1 by $5A1C/$5A1D via repeated subtraction.
; Stores ASCII digit at (DECIMAL_OUT_PTR),Y. Suppresses leading zeros.
.extract_digit:
    STOB    #$B0,NUM_DIGIT ; digit = '0' (high ASCII)
.cmp_loop:
    LDX     #$01                ; start comparing high byte
.cmp_x:
    LDA     NUM_VALUE,X         ; \
    CMP     NUM_DIVISOR,X       ;  | compare NUM_VALUE/NUM_VALUE+1 with divisor
    BCC     .output             ;  | if less, done (digit found)
    BNE     .subtract           ; /  if greater, subtract
    DEX                         ; \  high bytes equal: check low
    BNE     .subtract           ; /  (if X was 1, now 0 -> recompare)
    JMP     .cmp_x              ;    compare low byte (X=0)
.subtract:
```

```
    SUBW    NUM_VALUE,NUM_DIVISOR,NUM_VALUE ;  NUM_VALUE/NUM_VALUE+1 -= divisor
    INC     NUM_DIGIT           ; digit++
    JMP     .cmp_loop           ; try again (reset X=1)
.output:
    LDA     NUM_DIGIT           ; \
    BANE    #$B0,.store         ; and leading-zero suppress active
    LDA     NUM_LEADING         ;  |   skip output
    BANE    #$01,.skip
    LDA     #$B0                ; (reload '0' for final digit)
.store:
    STA     (DECIMAL_OUT_PTR),Y ; store digit at output buffer
    STOB    #$01,NUM_LEADING ; clear leading-zero suppress
.skip:
    INY                         ; advance output index
    RTS
```

Defines:
NUM_TO_DECIMAL, used in chunks 404, 437, 447, 554, 556, and 686.
NUM_TO_DECIMAL_CONT, never used.
Uses DECIMAL_OUT_PTR 630a, NUM_DIGIT 630b, NUM_DIVISOR 630c, NUM_LEADING 631, NUM_VALUE 630a, STOB 8c, STOW 8a, and SUBW 15a.

634 ⟨*defines* 25b⟩+≡ (909) ◁610a 674b▷

```
; --- Message number constants (index into game message table) ---
MSG_DOOR_OPENS      EQU     $01     ; "THE DOOR SWINGS OPEN!"
MSG_INSERT_DISK     EQU     $02     ; "INSERT ALI BABA DISK"
MSG_INSERT_GAME     EQU     $03     ; "HAH! JUST A SCRATCH!" (also insert game disk prompt)
MSG_FEELING_WEAK    EQU     $09     ; "IS FEELING RATHER WEAK."
MSG_LOTS_OF_FIGHT   EQU     $0A     ; "STILL HAS LOTS OF FIGHT LEFT."
MSG_HIT_WALL        EQU     $0B     ; "SMASHES INTO A SOLID WALL."
MSG_HIT_STATUE      EQU     $0C     ; "CHARGES INTO A STONE STATUE."
MSG_DEATH           EQU     $0D     ; "DEPARTS THE LAND OF THE LIVING."
MSG_ROCK_FALLS      EQU     $0F     ; "A LITTLE ROCK CRACKS AND FALLS."
MSG_HEAVY_ROCK      EQU     $10     ; "A HEAVY PIECE OF ROCK FALLS,"
MSG_DUST_CLOUD      EQU     $11     ; "A BILLOWING CLOUD OF DUST"
MSG_TOPPLES         EQU     $12     ; "THE SCULPTURE TOPPLES OVER"
MSG_TOUCHED         EQU     $14     ; "AS SOON AS IT IS TOUCHED,"
MSG_TOUGH_HIDE      EQU     $16     ; "WHOSE TOUGH HIDE SOFTENS THE BLOW."
MSG_BREAKS_FREE     EQU     $1B     ; "BREAKS FREE OF THE FRACAS."
MSG_ESCAPE_FAIL     EQU     $1C     ; "FAILS TO ESCAPE FROM THE ENEMY."
MSG_TACKLE_FAIL     EQU     $1D     ; "FAILS TO TACKLE THE ENEMY."
MSG_FLYING_TACKLE   EQU     $1E     ; "LEAPS INTO A FLYING TACKLE"
MSG_UNCONSCIOUS     EQU     $1F     ; "IS UNCONSCIOUS."
MSG_REST_STRONGER   EQU     $20     ; "FEELS STRONGER AFTER RESTING."
MSG_REGAIN_CONSC    EQU     $21     ; "STRUGGLES BACK TO CONSCIOUSNESS."
MSG_JOINS_PARTY     EQU     $24     ; "IN GRATITUDE I AM YOURS TO COMMAND."
MSG_THANK_YOU       EQU     $25     ; "THANK YOU."
MSG_SHUFFLES_OFF    EQU     $26     ; "SHUFFLES OFF THIS MORTAL COIL."
MSG_ARMOR_WORTHLESS EQU     $2A     ; "YOUR USED ARMOR IS WORTHLESS."
MSG_XP_GAINED       EQU     $2B     ; "ALL THE GOLD SINKS INTO SOME QUICKS"
MSG_GOLD_LOST       EQU     $2C     ; "SOME GOLD FALLS THRU HOLES..."
MSG_GOLD_DROPPED    EQU     $2D     ; gold dropped on map
MSG_NO_GOLD         EQU     $2E     ; "YOU HAVE NO GOLD!"
MSG_JOYSTICK_MODE   EQU     $2F     ; joystick confirmation
MSG_PRESS_KEY       EQU     $30     ; "PRESS ANY KEY TO CONTINUE."
MSG_TOO_POOR        EQU     $31     ; "SORRY, BUT YOU ARE TOO POOR."
MSG_INSERT_SCENARIO EQU     $33     ; "WHO, UNAWARE OF DANGER," (scenario disk prompt)
MSG_ENOUGH          EQU     $34     ; "ENOUGH IS ENOUGH!"
MSG_SAVED_ME        EQU     $35     ; "YOU SAVED ME FROM MY IMPRISONMENT,"
MSG_NO_XP           EQU     $37     ; no XP to flush
MSG_INPUT_PROMPT    EQU     $38     ; input mode prompt (keyboard/joystick)
MSG_READ_ERROR      EQU     $3B     ; disk read error
```

Defines:
MSG_ARMOR_WORTHLESS, used in chunk 454.
MSG_BREAKS_FREE, used in chunks 295 and 527.
MSG_DEATH, used in chunk 360.
MSG_DOOR_OPENS, used in chunk 354.
MSG_DUST_CLOUD, used in chunk 360.
MSG_ENOUGH, used in chunk 346.
MSG_ESCAPE_FAIL, used in chunk 456.
MSG_FEELING_WEAK, used in chunk 349.
MSG_FLYING_TACKLE, used in chunks 357 and 898.
MSG_GOLD_DROPPED, used in chunk 440.

MSG_GOLD_LOST, used in chunk 440.
MSG_HEAVY_ROCK, used in chunks 342 and 360.
MSG_HIT_STATUE, used in chunk 363.
MSG_HIT_WALL, used in chunk 349.
MSG_INPUT_PROMPT, used in chunk 590.
MSG_INSERT_DISK, used in chunks 342 and 354.
MSG_INSERT_GAME, used in chunk 297.
MSG_INSERT_SCENARIO, used in chunks 289 and 295.
MSG_JOINS_PARTY, used in chunk 385.
MSG_JOYSTICK_MODE, used in chunk 590.
MSG_LOTS_OF_FIGHT, used in chunk 349.
MSG_NO_GOLD, used in chunk 436.
MSG_NO_XP, used in chunk 433.
MSG_PRESS_KEY, used in chunk 297.
MSG_READ_ERROR, used in chunk 305.
MSG_REGAIN_CONSC, used in chunk 377.
MSG_REST_STRONGER, used in chunk 224.
MSG_ROCK_FALLS, used in chunk 542.
MSG_SAVED_ME, used in chunk 283.
MSG_SHUFFLES_OFF, never used.
MSG_TACKLE_FAIL, used in chunk 456.
MSG_THANK_YOU, used in chunk 385.
MSG_TOO_POOR, never used.
MSG_TOPPLES, used in chunks 342 and 360.
MSG_TOUCHED, used in chunk 342.
MSG_TOUGH_HIDE, used in chunks 289 and 295.
MSG_UNCONSCIOUS, used in chunks 357 and 898.
MSG_XP_GAINED, used in chunk 433.

Displays a message from the game's message table. Takes the message index in A. The message table base address is stored at $4005/$4006; each entry is a length-prefixed string. The routine walks the table to find the Xth entry, then displays up to three lines in the status window.

The display behavior depends on the line number:

- Line 1 (MSG_LINE_COUNT = 0): calls PRINT_BOTTOM_CENTERED, then scrolls.
- Line 2 (MSG_LINE_COUNT = 1): scrolls first, then prints.
- Line 3+: scrolls, prints, then calls DELAY_WITH_ANIMATION (delay) and PRINT_CTRL_AB (PRINT_CTRL_AB) before looping back.

An entry with a zero length byte terminates the message. If exactly two lines were printed, a final scroll is performed on exit.

DISPLAY_MESSAGE indexes into the message table at MSG_TABLE_PTR ($4005/$4006) to find the string for message number A. It walks the table by skipping length-prefixed entries until the desired index is reached. MSG_LINE_COUNT tracks how many lines have been printed so far, which controls whether the routine scrolls before or after printing.

636 ⟨*var msg line count* 636⟩≡ (909)

```
    ORG     $5A57
MSG_LINE_COUNT:
    DC.B    $BC                     ; initial value (runtime: lines printed so far)
```

Defines:
MSG_LINE_COUNT, used in chunks 452 and 637.

The routine below implements the message display.

637 ⟨*display message* 637⟩≡ (909)

```
    ORG     $761E
DISPLAY_MESSAGE:
    SUBROUTINE DISPLAY_MESSAGE
    ; Display a game message by index from the message table.
    ;
    ; Inputs:
    ;   A | message index (0-based)
    ;
    ; Behavior:
    ;   Skips through the message table to find the indexed entry,
    ;   then prints each sub-string centered on row 23. Multi-line
    ;   messages scroll the status area between lines and pause
    ;   with animation between groups.
    ;
    ; Modifies: PRINT_STRING_ADDR
    ; Clobbers: A, X, Y

    TAX                             ; X = message index
    MOVW    MSG_TABLE_PTR,PRINT_STRING_ADDR ; load table base into PRINT_STRING_ADDR/PRINT_STRING_AD
    STOB    #$00,MSG_LINE_COUNT ; reset line counter

    JMP     .check_index            ; skip to index check

.skip_entry:
    LDY     #$00
    LDA     (PRINT_STRING_ADDR),Y ; get string length
    TAY
    SEC                             ; +1 for length byte itself
    ADC     PRINT_STRING_ADDR       ; advance pointer past this entry
    STA     PRINT_STRING_ADDR
    LDA     PRINT_STRING_ADDR+1
    ADC     #$00
    STA     PRINT_STRING_ADDR+1
    BYNE    #$00,.skip_entry        ; no -- keep skipping within group
.next_index:
    DEX
.check_index:
    BXNE    #$00,.skip_entry        ; more entries to skip

DISPLAY_MESSAGE_LOOP:
.print_loop:
    LDY     #$00
    LDA     (PRINT_STRING_ADDR),Y ; get length of current string
    BNE     .has_text               ; nonzero -- print it

    ; length is zero -- message is done
    LDA     MSG_LINE_COUNT
```

```
    BAEQ    #$02,.final_scroll    ; printed exactly 2 lines?
    RTS                           ; otherwise just return
.final_scroll:
    JMP     SCROLL_STATUS_LINE    ; scroll once more and return

.has_text:
    LDA     MSG_LINE_COUNT
    BAMI    #$01,.first_line ; line_count < 1: first line
    BEQ     .second_line          ; line_count == 1: second line

    ; third+ line: scroll, then print
    JSR     SCROLL_STATUS_LINE    ; scroll status window
.second_line:
    JSR     PRINT_BOTTOM_CENTERED ; print centered on row 23
    JMP     .advance

.first_line:
    JSR     PRINT_BOTTOM_CENTERED ; print centered on row 23
    JSR     SCROLL_STATUS_LINE    ; scroll status window

.advance:
    INC     MSG_LINE_COUNT
    LDY     #$00
    LDA     PRINT_STRING_ADDR
    SEC
    ADC     (PRINT_STRING_ADDR),Y ; add length + 1 (SEC sets carry)
    STA     PRINT_STRING_ADDR
    LDA     PRINT_STRING_ADDR+1
    ADC     #$00
    STA     PRINT_STRING_ADDR+1
    LDA     #$01
    BALT    MSG_LINE_COUNT,.pause ; printed <= 1 lines?
    LDY     #$00
    LDA     (PRINT_STRING_ADDR),Y ; peek at next entry
    BNE     .print_loop           ; more text -- keep printing

.pause:
    JSR     DELAY_WITH_ANIMATION  ; delay
    JSR     PRINT_CTRL_AB         ; reset character set
    JMP     .print_loop           ; continue with next line
```

Defines:
DISPLAY_MESSAGE, used in chunks 56, 224, 283, 289, 295, 297, 305, 342, 346, 349, 354, 357, 360, 363, 377, 378, 385, 433, 436, 440, 454, 456, 527, 542, 590, 640, and 898.
DISPLAY_MESSAGE_LOOP, used in chunk 452.

Uses BYNE 18, DELAY_WITH_ANIMATION 603b, MOVW 9a, MSG_LINE_COUNT 636, MSG_TABLE_PTR 53a, PRINT_BOTTOM_CENTERED 628, PRINT_CTRL_AB 623a, PRINT_STRING_ADDR 576c, SCROLL_STATUS_LINE 639, and STOB 8c.

SCROLL_STATUS_LINE restores the HRCG text window to its default configuration after a status message. It sets the window to scroll mode, positions the cursor at row 23, outputs control character $83 (which triggers a scroll-up in the HRCG handler), and then resets the window boundaries. This sequence effectively clears the bottom status area while preserving the rest of the display.

639 ⟨*scroll status line* 639⟩≡ (909)

```
      ORG     $769B
  SCROLL_STATUS_LINE:
      SUBROUTINE SCROLL_STATUS_LINE
      ; Scroll the bottom status area up by one line.
      ;
      ; Behavior:
      ;   Switches to scroll mode, confines the window to rows 21-23,
      ;   positions cursor at row 23, sends $83 to trigger a scroll-up,
      ;   then restores full-screen wrap mode.
      ;
      ; Modifies: TEXT_ROW, TEXT_COL
      ; Clobbers: A

      ; Text display: init HRCG, set window bottom, output char $83, reset
      JSR     SET_TEXT_WINDOW_SCROLL
      JSR     SET_TEXT_WINDOW_UPPER_LEFT_LOW
      STOB    #$17,TEXT_ROW ; cursor to row 23
      LDA     #$83
      JSR     ROM_COUT1
      JSR     SET_TEXT_WINDOW_UPPER_LEFT_ALL
      JMP     SET_TEXT_WINDOW_WRAP
```

Defines:
SCROLL_STATUS_LINE, used in chunks 224, 338, 342, 346, 357, 360, 363, 377, 378, 385, 428, 456, 483b, 545, 556, 637, and 898.

Uses ROM_COUT1 571a, SET_TEXT_WINDOW_SCROLL 625, SET_TEXT_WINDOW_WRAP 625, STOB 8c, TEXT_COL 572a, and TEXT_ROW 572a.

17.1 Game message strings

The 61-entry message string table at GAME_MESSAGE_STRINGS holds all narrative text displayed by DISPLAY_MESSAGE. Each message is one or more consecutive Pascal strings (length byte + high-bit ASCII) terminated by a $00 separator. Multi-string messages display as consecutive lines.

640 ⟨*game message strings* 640⟩≡ (909)

```
        ORG     $9AA5
    GAME_MESSAGE_STRINGS:
        ; Game narrative messages, accessed by index via DISPLAY_MESSAGE.
        ; Each entry is a $00-terminated group of Pascal strings.
        DC.B    $04,$00              ; header
        APSTR "THE DOOR CREAKS BUT REMAINS SHUT."
        DC.B    $00                  ; --- message 0 separator ---
        APSTR "THE DOOR SWINGS OPEN!"
        DC.B    $00                  ; --- message 1 separator ---
        APSTR "INSERT ALI BABA DISK"
        DC.B    $00                  ; --- message 2 separator ---
        APSTR "HAH! JUST A SCRATCH!"
        DC.B    $00                  ; --- message 3 separator ---
        APSTR "OUCH!"
        DC.B    $00                  ; --- message 4 separator ---
        APSTR "AAARGH!"
        DC.B    $00                  ; --- message 5 separator ---
        APSTR "AIIYEEEEE!!"
        DC.B    $00                  ; --- message 6 separator ---
        APSTR "OH MA, I THINK IT'S MY TIME."
        DC.B    $00                  ; --- message 7 separator ---
        APSTR "FALLS UNCONSCIOUS TO THE FLOOR."
        DC.B    $00                  ; --- message 8 separator ---
        APSTR "IS FEELING RATHER WEAK."
        DC.B    $00                  ; --- message 9 separator ---
        APSTR "STILL HAS LOTS OF FIGHT LEFT."
        DC.B    $00                  ; --- message 10 separator ---
        APSTR "SMASHES INTO A SOLID WALL."
        DC.B    $00                  ; --- message 11 separator ---
        APSTR "CHARGES INTO A STONE STATUE."
        DC.B    $00                  ; --- message 12 separator ---
        APSTR "DEPARTS THE LAND OF THE LIVING."
        DC.B    $00                  ; --- message 13 separator ---
        APSTR "CONGRATULATIONS!"
        APSTR "YOU ARE IN THE MOUNTAIN."
        APSTR "I HAVE SHOWN YOU DOORS,"
        APSTR "WEALTH AND TRADING OUTPOSTS."
        APSTR "NO LONGER SHALL I POINT THEM OUT."
        APSTR ". . .  THE FRIENDLY MAGE"
        DC.B    $00                  ; --- message 14 separator ---
        APSTR "A LITTLE ROCK CRACKS AND FALLS."
```

```
DC.B    $00                ; --- message 15 separator ---
APSTR "A HEAVY PIECE OF ROCK FALLS,"
APSTR "KNOCKING DOWN"
DC.B    $00                ; --- message 16 separator ---
APSTR "A BILLOWING CLOUD OF DUST"
APSTR "ENVELOPS THE WALL."
DC.B    $00                ; --- message 17 separator ---
APSTR "THE SCULPTURE TOPPLES OVER"
APSTR "AND CRUMBLES INTO DUST."
DC.B    $00                ; --- message 18 separator ---
APSTR "BUT NOTHING INTERESTING HAPPENS."
DC.B    $00                ; --- message 19 separator ---
APSTR "AS SOON AS IT IS TOUCHED,"
APSTR "THE ILLUSION DISAPPEARS!"
DC.B    $00                ; --- message 20 separator ---
APSTR "THAT'S NOT A SCENARIO DISK!"
DC.B    $00                ; --- message 21 separator ---
APSTR "WHOSE TOUGH HIDE SOFTENS THE BLOW."
DC.B    $00                ; --- message 22 separator ---
APSTR "WHOSE ARMOR TAKES SOME OF THE IMPACT."
DC.B    $00                ; --- message 23 separator ---
APSTR "WHO, UNHARMED, GROWLS OMINOUSLY."
DC.B    $00                ; --- message 24 separator ---
APSTR "WHO CHORTLES MERRILY"
APSTR "AS THE ARMOR TAKES THE FULL BLOW."
DC.B    $00                ; --- message 25 separator ---
APSTR "DISK I/O ERROR!"
DC.B    $00                ; --- message 26 separator ---
APSTR "BREAKS FREE OF THE FRACAS."
DC.B    $00                ; --- message 27 separator ---
APSTR "FAILS TO ESCAPE FROM THE ENEMY."
DC.B    $00                ; --- message 28 separator ---
APSTR "FAILS TO TACKLE THE ENEMY."
DC.B    $00                ; --- message 29 separator ---
APSTR "LEAPS INTO A FLYING TACKLE"
APSTR "AND WRESTLES WITH THE FOE."
DC.B    $00                ; --- message 30 separator ---
APSTR "IS UNCONSCIOUS."
DC.B    $00                ; --- message 31 separator ---
APSTR "FEELS STRONGER AFTER RESTING."
DC.B    $00                ; --- message 32 separator ---
APSTR "STRUGGLES BACK TO CONSCIOUSNESS."
DC.B    $00                ; --- message 33 separator ---
APSTR "PLEASE ENTER A NUMBER"
APSTR "FROM 1 TO 24."
DC.B    $00                ; --- message 34 separator ---
APSTR "YOU SAVED ME FROM MY IMPRISONMENT,"
DC.B    $00                ; --- message 35 separator ---
APSTR "IN GRATITUDE I AM YOURS TO COMMAND."
DC.B    $00                ; --- message 36 separator ---
```

```
APSTR "THANK YOU."
DC.B    $00                ; --- message 37 separator ---
APSTR "SHUFFLES OFF THIS MORTAL COIL."
DC.B    $00                ; --- message 38 separator ---
APSTR "GIVES AN OBOLUS TO CHARON."
DC.B    $00                ; --- message 39 separator ---
APSTR "TURNS THE TOES UP TO THE DAISIES."
DC.B    $00                ; --- message 40 separator ---
APSTR "GOLD EATING ROCK SLIME GOBBLES THE CASH."
DC.B    $00                ; --- message 41 separator ---
APSTR "YOUR USED ARMOR IS WORTHLESS."
DC.B    $00                ; --- message 42 separator ---
APSTR "ALL THE GOLD SINKS INTO SOME QUICKSAND."
DC.B    $00                ; --- message 43 separator ---
APSTR "SOME GOLD FALLS THRU HOLES IN THE FLOOR."
DC.B    $00                ; --- message 44 separator ---
APSTR "YOU HAVE NO GOLD!"
DC.B    $00                ; --- message 45 separator ---
APSTR "MOVE WITH THE JOYSTICK OR"
APSTR "PRESS BUTTON FOR OPTIONS."
DC.B    $00                ; --- message 46 separator ---
APSTR "PRESS ANY KEY TO CONTINUE."
DC.B    $00                ; --- message 47 separator ---
APSTR "SORRY, BUT YOU ARE TOO POOR."
DC.B    $00                ; --- message 48 separator ---
DC.B    $00                ; --- message 49 separator ---
APSTR "PLEASE INSERT SCENARIO DISK."
DC.B    $00                ; --- message 50 separator ---
APSTR "WHO, UNAWARE OF DANGER,"
APSTR "ESCAPES THE BLOW UNHARMED."
DC.B    $00                ; --- message 51 separator ---
APSTR "ENOUGH IS ENOUGH!"
APSTR "NO MORE PLAYERS ARE ALLOWED."
DC.B    $00                ; --- message 52 separator ---
DC.B    $00                ; --- message 53 separator ---
APSTR "YOU HAVE NO ARMOR!"
DC.B    $00                ; --- message 54 separator ---
APSTR "PRESS SPACE BAR TO USE KEYBOARD OR"
APSTR "JOYSTICK BUTTON TO USE JOYSTICK."
DC.B    $00                ; --- message 55 separator ---
DC.B    $00                ; --- message 56 separator ---
DC.B    $00                ; --- message 57 separator ---
APSTR "CAN NOT READ SCENARIO FROM DISK"
APSTR "PROGRAM WILL NOW CRASH!!"
DC.B    $00                ; --- message 58 separator ---
DC.B    $00                ; --- message 59 separator ---
DC.B    $00                ; --- message 60 separator ---
```

Defines:
GAME_MESSAGE_STRINGS, never used.
Uses DISPLAY_MESSAGE 637.

Reads a byte stream from the pointer at TEXT_STREAM_PTR and renders characters on the hi-res screen. It maintains column/row state and supports several control codes.

TEXT_LEFT_MARGIN and TEXT_RIGHT_MARGIN define the column bounds for text wrapping. TEXT_STREAM_IDX saves the Y index into the byte stream across RENDER_FONT_CHAR calls, since rendering may clobber Y. TEXT_RENDERER initializes the margins (2 and 18) and row, then loops through bytes, advancing the column and wrapping to the next row when it passes the right margin.

643a ⟨*var text right margin* 643a⟩≡ (909)

```
    ORG     $5A96
TEXT_RIGHT_MARGIN:
    DC.B    $8D                     ; initial value (runtime: right margin column for text wrap)
```

Defines:
TEXT_RIGHT_MARGIN, used in chunk 644.

643b ⟨*var text left margin* 643b⟩≡ (909)

```
    ORG     $5A97
TEXT_LEFT_MARGIN:
    DC.B    $A0                     ; initial value (runtime: left margin column for text wrap)
```

Defines:
TEXT_LEFT_MARGIN, used in chunk 644.

643c ⟨*var text stream idx* 643c⟩≡ (909)

```
    ORG     $5A98
TEXT_STREAM_IDX:
    DC.B    $C9                     ; initial value (runtime: saved Y index into byte stream)
```

Defines:
TEXT_STREAM_IDX, used in chunk 644.

The routine below processes the byte stream.

644 ⟨*text renderer* 644⟩≡ (909)

```
    ORG     $6C8E
TEXT_RENDERER:
    SUBROUTINE TEXT_RENDERER
    ; Render a byte-stream of text and control codes to the hi-res screen.
    ;
    ; Inputs:
    ;   TEXT_STREAM_PTR  | pointer to byte stream to render
    ;
    ; Behavior:
    ;   Initializes TEXT_LEFT_MARGIN=2, TEXT_RIGHT_MARGIN=18, FONT_ROW=1,
    ;   then reads bytes in a loop.  Bytes $00-$7C are rendered as custom
    ;   font characters (FONT_CHARSET=2), advancing the column and wrapping
    ;   at TEXT_RIGHT_MARGIN.  $7D resets to full-screen layout.  $7E pauses:
    ;   plays TUNE_ATTRACT, calls DISPLAY_STATUS_BAR, then reinitializes.
    ;   $7F ends the stream and returns.  $80-$FE are position commands:
    ;   AND #$7F gives the new row, the following byte gives the column.
    ;
    ; Modifies:
    ;   FONT_COL, FONT_ROW, FONT_CHARNUM, FONT_CHARSET,
    ;   TEXT_LEFT_MARGIN, TEXT_RIGHT_MARGIN, TEXT_STREAM_IDX
    ; Clobbers: A, X, Y
    LDY     #$00
.init:
    STOB    #$02,TEXT_LEFT_MARGIN ; left margin = 2
    STA     FONT_COL              ; current column = 2
    STOB    #$12,TEXT_RIGHT_MARGIN ; right margin = $12 (18)
    STOB    #$01,FONT_ROW ; current row = 1

    ; --- Main byte-reading loop ---
.fetch:
    LDA     (TEXT_STREAM_PTR),Y ; read byte from stream
    BMI     .set_pos            ; bit 7 set -> position command
    BANE    #$7F,.not_end
    RTS                         ; $7F = end of stream

.not_end:
    BAEQ    #$7E,.pause         ; $7E = pause
    BAEQ    #$7D,.reset         ; $7D = reset to full-screen mode

    ; --- Character output ---
    STY     TEXT_STREAM_IDX     ; save stream index
    STA     FONT_CHARNUM        ; character code
    STOB    #$02,FONT_CHARSET ; character width = 2
    JSR     RENDER_FONT_CHAR    ; draw at current position
    INC     FONT_COL            ; advance column
    LDA     FONT_COL
    BALT    TEXT_RIGHT_MARGIN,.next ; past right margin?
```

```
    INC     FONT_ROW            ; yes -> next row
    MOVB    TEXT_LEFT_MARGIN,FONT_COL ;   reset column to left margin
.next:
    LDY     TEXT_STREAM_IDX     ; restore stream index
    INY                         ; advance to next byte
    BNE     .fetch              ; loop (max 256 per page)
    JMP     MOVE_LOOP           ; overflow handler

    ; --- Position command ($80-$FE) ---
.set_pos:
    AND     #$7F                ; row = byte & $7F
    STA     FONT_ROW
    INY
    LDA     (TEXT_STREAM_PTR),Y ; next byte = column
    STA     FONT_COL
    INY
    JMP     .fetch

    ; --- Pause ($7E): run tune player, show status bar ---
.pause:
    TYA                         ; \
    PHA                         ;  | save stream index
    NOP
    LDX     #TUNE_ATTRACT       ; tune 6: attract/opening
    JSR     TUNE_PLAYER
    PSHW    TEXT_STREAM_PTR     ; save stream pointer
    JSR     DISPLAY_STATUS_BAR  ; (clobbers TEXT_STREAM_PTR)
    PULW    TEXT_STREAM_PTR     ; restore stream pointer
    PLA                         ; restore stream index
    TAY
    NOP
    INY                         ; advance past $7E
    JMP     .init               ; reinit margins/position

    ; --- Reset ($7D): full-screen text mode ---
.reset:
    STOB    #$00,TEXT_LEFT_MARGIN  ; left margin = 0
    STA     FONT_COL               ; column = 0
    STA     FONT_ROW               ; row = 0
    STOB    #MAP_WIDTH,TEXT_RIGHT_MARGIN ; right margin = 20 (MAP_WIDTH)
    INY                         ; advance past $7D
    JMP     .fetch
```

Defines:
TEXT_RENDERER, used in chunks 527, 550, 571c, and 574.

Uses DISPLAY_STATUS_BAR 550, FONT_CHARNUM 575, FONT_CHARSET 576a, FONT_COL 492a, FONT_ROW 534c, MAP_WIDTH 388a, MOVB 8c, MOVE_LOOP 232, PSHW 9b, PULW 10a, RENDER_FONT_CHAR 466, STOB 8c, TEXT_LEFT_MARGIN 643b, TEXT_RIGHT_MARGIN 643a, TEXT_STREAM_IDX 643c, TEXT_STREAM_PTR 571c, TUNE_ATTRACT 526a, and TUNE_PLAYER 682.

Chapter 18

Sound

Playing a tone involves accessing the speaker address twice per audio cycle. The `tone` routine delays `10X+31` cycles between speaker accesses, and based on an average Apple II cycle time of `0.98` microseconds, this means that the frequency of the note played will be equal to `510204 / (10X+31)` Hz. See The Oft Misunderstood WAIT for a derivation of average cycle time.

`PLAY_HIGH_TONE` generates a square wave tone by toggling the Apple II speaker at `SPEAKER`. The outer loop runs 10 iterations (Y), each toggling the speaker then delaying by X cycles. After all 10 toggles, the whole sequence repeats `SOUND_DURATION` times. `SOUND_FREQ` controls the pitch (higher value = lower frequency).

646 ⟨*zero page defines* 25a⟩+≡ (909) ◁630a 647b▷

```
SOUND_FREQ          EQU     $E1     ; sound frequency / tune modified action
SOUND_DURATION      EQU     $E2     ; sound duration / tune parameter byte
```

Defines:
`SOUND_DURATION`, used in chunks 647a, 648, and 682.
`SOUND_FREQ`, used in chunks 647a, 648, and 682.

647a ⟨*play tone* 647a⟩≡ (909)

```
      ORG     $7A7F
  PLAY_HIGH_TONE:
      SUBROUTINE PLAY_HIGH_TONE
      ; Play a high-frequency tone via the speaker.
      ;
      ; Inputs:
      ;   SOUND_FREQ | pitch delay counter
      ;   SOUND_DURATION | number of full cycles to play
      ;
      ; Behavior:
      ;   Toggles the speaker with an inner delay loop. Frequency is
      ;   approximately 510204 / (5*X + 12) Hz.
      ;
      ; Clobbers: A, X, Y

  .cycle:
      LDY     #$0A           ; 10 speaker toggles per cycle
  .toggle:
      LDA     $C030          ; toggle speaker
  .delay:
      DEX
      BNE     .delay         ; inner delay loop (no toggle)
      LDX     SOUND_FREQ     ; reload pitch counter
      DEY
      BNE     .toggle        ; next toggle
      DEC     SOUND_DURATION ; repeat whole cycle
      BNE     .cycle
      RTS
```

Defines:
PLAY_HIGH_TONE, used in chunks 607b, 648, 679, 681, and 686.
Uses SOUND_DURATION 646 and SOUND_FREQ 646.

PLAY_LOW_TONE generates a low-octave tone. It has the same structure as PLAY_HIGH_TONE but with extra cycles in the inner loop (ROL/ROR $E3 plus three NOPs), producing a longer toggle period and thus a lower frequency for the same X value. The formula is 510204 / (10X+31) Hz, roughly one octave below PLAY_HIGH_TONE.

The ROL/ROR pair on $E3 cancels out (no net effect on the shift register); it exists purely for cycle padding.

647b ⟨*zero page defines* 25a⟩+≡ (909) ◁646 654a▷

```
  TONE_PADDING            EQU     $E3     ; byte used for cycle padding in PLAY_LOW_TONE
```

Defines:
TONE_PADDING, used in chunk 648.
Uses PLAY_LOW_TONE 648.

648 ⟨*play low tone* 648⟩≡ (909)

```
      ORG     $7A91
  PLAY_LOW_TONE:
      SUBROUTINE PLAY_LOW_TONE
      ; Play a low-frequency tone via the speaker.
      ;
      ; Inputs:
      ;   SOUND_FREQ | pitch delay counter
      ;   SOUND_DURATION | number of full cycles to play
      ;
      ; Behavior:
      ;   Same structure as PLAY_HIGH_TONE but with extra cycle padding,
      ;   producing a lower octave. Frequency is approximately
      ;   510204 / (10*X + 31) Hz.
      ;
      ; Clobbers: A, X, Y

  .cycle:
      LDY     #$0A           ; 10 toggles per cycle
  .toggle:
      LDA     $C030          ; toggle speaker
  .delay:
      ROL     TONE_PADDING   ; cycle padding (5 cycles)
      DEX
      BNE     .delay         ; inner delay loop (no toggle)
      LDX     SOUND_FREQ     ; reload pitch counter
      ROL     TONE_PADDING   ; \ more cycle padding
      ROR     TONE_PADDING   ; / (ROL+ROR cancels out)
      NOP                    ; \
      NOP                    ;  | timing padding
      NOP                    ; /
      DEY
      BNE     .toggle        ; next toggle
      DEC     SOUND_DURATION ; repeat whole cycle
      BNE     .cycle
      RTS
```

Defines:
PLAY_LOW_TONE, used in chunks 647b, 679, 681, and 686.
Uses PLAY_HIGH_TONE 647a, SOUND_DURATION 646, SOUND_FREQ 646, and TONE_PADDING 647b.

Note	Freq	Low Tone	Low Freq	High Tone	High Freq
A3	220.0	E5	219.8	--	--
Bb3	233.1	D8	232.9	--	--
B3	246.9	CC	246.4	--	--
C4	261.6	C0	261.5	--	--
Db4	277.2	B5	277.1	--	--
D4	293.7	AB	293.1	--	--
Eb4	311.1	A1	310.9	--	--
E4	329.6	98	329.0	--	--
F4	349.2	8F	349.2	--	--
Gb4	370.0	87	369.4	--	--
G4	392.0	7F	392.2	--	--
Ab4	415.3	78	414.5	F3	415.8
A4	440.0	71	439.5	E6	439.1
Bb4	466.2	6A	467.6	D8	467.2
B4	493.9	64	494.9	CC	494.4
C5	523.2	5E	525.4	C1	522.2
Db5	554.4	59	554.0	B6	553.4
D5	587.3	54	585.8	AB	588.5
Eb5	622.2	4F	621.4	A2	620.7
E5	659.2	4A	661.7	98	660.9
F5	698.5	46	698.0	90	697.0
Gb5	740.0	42	738.4	87	742.7
G5	784.0	3E	783.7	80	782.5
Ab5	830.6	3A	835.0	78	833.7
A5	880.0	37	878.1	72	876.6
Bb5	932.3	34	926.0	6B	932.7
B5	987.8	31	979.3	65	986.9
C6	1046.5	2E	1039.1	5F	1047.6
Db6	1108.7	2B	1106.7	5A	1104.3
D6	1174.7	28	1183.8	54	1181.0
Eb6	1244.5	26	1241.4	50	1238.4
E6	1318.5	24	1304.9	4B	1318.4
F6	1396.9	21	1413.3	47	1390.2
Gb6	1480.0	1F	1496.2	43	1470.3
G6	1568.0	1D	1589.4	3F	1560.3
Ab6	1661.2	1C	1640.5	3B	1661.9
A6	1760.0	1A	1753.3	38	1747.3
Bb6	1864.7	18	1882.7	34	1875.8
B6	1975.5	17	1954.8	31	1985.2
C7	2093.0	15	2117.0	2E	2108.3
Db7	2217.5	14	2208.7	2C	2199.2
D7	2349.3	13	2308.6	29	2351.2
Eb7	2489.0	11	2538.3	27	2464.8
E7	2637.0	10	2671.2	24	2657.3
F7	2793.8	0F	2818.8	22	2803.3
Gb7	2960.0	0E	2983.6	20	2966.3
G7	3136.0	0D	3169.0	1E	3149.4
Ab7	3322.4	0C	3378.8	1C	3356.6

Table 18.1: Notes and tone values

Byte	Opcode	Argument	Operation
1F	MARK2	none	TUNE_LOOP_ADDR <- IP+1
20	MARK1	none	TUNE_GOSUB_ADDR <- IP+1
21	GOTO1	none	IP <- TUNE_GOSUB_ADDR
22	LOW	none	TONE_ROUTINE_ADDR <- tone
23	HIGH	none	TONE_ROUTINE_ADDR <- high_tone
24	END	none	Ends the song routine
25	HIRAND	none	TONE_RANDOMIZE_MASK <- 0F
26	NORAND	none	TONE_RANDOMIZE_MASK <- 00
27-FF	NOTE	duration	Plays note

Table 18.2: Song program opcodes

Based on cycle times, these are the closest values for various notes. At the higher frequencies, the values get so low that they differ only by one, which means the notes aren't very accurate.

We run into an issue when we look at the song table for the splash screen. The very first note played is low `3F`. This is closest to `G5`, although it results in a frequency of `771.9` Hz, which is less accurate than the `783.7` Hz that would be given by `3F`. In fact, all the low tones are off by a bit, unless we assume that the programmer erred, thinking that the low tones were just one octave below the high tones. In that case, we should just use the high tone table and subtract one octave for the low tones.

The splash song bytecode is documented in `TUNE_3_ATTRACT` (§18.1).

The `TONE_RANDOMIZE_MASK` is initially set to 1 to apply a little bit of randomness to the song notes. However, this can be set to `00` via opcode `NORAND` to play exact notes, or set to `0F` via opcode `HIRAND` to provide high randomness.

18.1 Tune player

The tune player runs short bytecode programs that play sound effects and sound effects and music. It is called with a tune index in X.

`ATTRACT_FLAG` is set to 1 when `TUNE_PLAYER` is invoked with tune index 6 (the attract/demo mode) and cleared to 0 for all other indices. Downstream code checks this flag to alter behavior during the title screen sequence.

The script base addresses are stored in the table below. The X register passed to `TUNE_PLAYER` is a byte offset into this table (X = 0 for script 0, X = 2 for script 1, etc.).

The table contains 12 entries (scripts 0–11). The X register passed to TUNE_PLAYER is a byte offset (X = 0 for script 0, X = 2 for script 1, etc.).

X	Tune	Address	Purpose	Callers
$00	0	$80C6	(empty — single END byte)	—
$02	1	$80C7	combat hit effect	attack routines
$04	2	$80D7	combat damage effect	damage handler
$06	3	$A300	attract/opening	attract screen, scene loop, game init
$08	4	$A3C3	mob death effect	death handler (field 4 < $15)
$0A	5	$A3CE	NPC death effect	death handler (field 4 ≥ $15)
$0C	6	$A3DE	combat avoided effect	hit handler, avoid handler
$0E	7	$A409	entity interaction effect	movement, entity scripts
$10	8	$A432	AI turn effect	SETUP_SCENE_DISPLAY
$12	9	$A437	menu selection effect	INPUT_LOOP
$14	10	$A3FC	encounter effect	encounter handler
$16	11	$A43C	movement effect	COMMIT_MOVE

651a ⟨*tune pointer table* 651a⟩≡ (909)

```
    ORG     $80AE
TUNE_TABLE:
    DC.W    $80C6 ; tune 0  (X=$00): empty (single END byte)
    DC.W    $80C7 ; tune 1  (X=$02): combat hit effect
    DC.W    $80D7 ; tune 2  (X=$04): combat damage effect
    DC.W    $A300 ; tune 3  (X=$06): attract/opening
    DC.W    $A3C3 ; tune 4  (X=$08): mob death effect
    DC.W    $A3CE ; tune 5  (X=$0A): NPC death effect
    DC.W    $A3DE ; tune 6  (X=$0C): combat avoided effect
    DC.W    $A409 ; tune 7  (X=$0E): entity interaction effect
    DC.W    $A432 ; tune 8  (X=$10): AI turn effect
    DC.W    $A437 ; tune 9  (X=$12): menu selection effect
    DC.W    $A3FC ; script 10 (X=$14): encounter effect
    DC.W    $A43C ; script 11 (X=$16): movement effect
```

Defines:
TUNE_TABLE, used in chunks 653 and 654b.

651b ⟨*var tune loop ctr* 651b⟩≡ (909)

```
    ORG     $5A8D
TUNE_LOOP_CTR:
    DC.B    $8D                     ; initial value (runtime: tune loop iteration counter)
```

Defines:
TUNE_LOOP_CTR, used in chunks 678 and 679.

652a ⟨*var tune loop addr* 652a⟩≡ (909)

```
    ORG     $5A8E
TUNE_LOOP_ADDR:
    DC.B    $A0,$D3                 ; initial value (runtime: tune loop address (2 bytes))
```

Defines:
TUNE_LOOP_ADDR, used in chunks 679 and 681.

652b ⟨*var tune gosub addr* 652b⟩≡ (909)

```
    ORG     $5A90
TUNE_GOSUB_ADDR:
    DC.B    $D4,$C1                 ; initial value (runtime: tune gosub return address (2 bytes))
```

Defines:
TUNE_GOSUB_ADDR, used in chunk 681.

652c ⟨*var tune prng mask* 652c⟩≡ (909)

```
    ORG     $5A92
TUNE_PRNG_MASK:
    DC.B    $A0                     ; initial value (runtime: PRNG mask for action randomization)
```

Defines:
TUNE_PRNG_MASK, used in chunks 678, 681, and 682.

652d ⟨*var tune action vec* 652d⟩≡ (909)

```
    ORG     $5A93
TUNE_ACTION_VEC:
    DC.B    $C1,$D2                 ; initial value (runtime: action handler vector (2 bytes))
```

Defines:
TUNE_ACTION_VEC, used in chunks 681, 682, and 684a.

652e ⟨*var attract flag* 652e⟩≡ (909)

```
    ORG     $5AA7
ATTRACT_FLAG:
    DC.B    $A9                     ; initial value (runtime: 1 = attract/demo mode, 0 = normal)
```

Defines:
ATTRACT_FLAG, used in chunks 653 and 683.

653 ⟨*tune player* 653⟩≡ (909) 654b ▷

```
    ORG     $795B
TUNE_PLAYER:
    SUBROUTINE TUNE_PLAYER
    ; Bytecode interpreter for sound effects and music.
    ;
    ; Input:  X = tune index (byte offset into TUNE_TABLE:
    ;             $00=empty, $02=hit, $04=damage, $06=attract,
    ;             $08=mob death, $0A=NPC death, $0C=avoided,
    ;             $0E=interact, $10=AI turn, $12=menu, $14=encounter,
    ;             $16=movement)
    ;         Y = combat strength (used only for X=$02 or X=$04,
    ;             patched into script loop count)
    ; Output: sound played; ATTRACT_FLAG set for index $06
    ; Behavior:
    ;   - Clears keyboard strobe
    ;   - Sets ATTRACT_FLAG = 1 for tune index $06, else 0
    ;   - For indices $02/$04, patches loop count from Y-1
    ;   - Loads script address from TUNE_TABLE into TUNE_PC
    ;   - Runs bytecode interpreter (opcodes $00-$26 = control,
    ;     $27+ = note with PRNG-randomized frequency)
    ;   - In attract mode, polls keyboard/joystick each step
    ; Clobbers: A, X, Y

    STA     $C010             ; clear keyboard strobe
    STOB    #$00,ATTRACT_FLAG ; attract_flag = 0
    BXNE    #$06,.not_attract ; tune index 6 = attract
    INC     ATTRACT_FLAG      ; attract_flag = 1
.not_attract:

    BXEQ    #$02,.setup_2     ; special setup for index 2
    BXEQ    #$04,.setup_4     ; special setup for index 4
    JMP     .load_ptr

.setup_2:
    DEY
    STY     $80D5             ; patch byte in script 1 data
    JMP     .load_ptr
.setup_4:
    DEY
    STY     $80E3             ; patch byte in script 2 data
```

Uses ATTRACT_FLAG 652e, STOB 8c, TUNE_PC 654a, TUNE_PLAYER 682, and TUNE_TABLE 651a.

TUNE_PC is the 2-byte script instruction pointer at TUNE_PC. The script base address is loaded from the table at TUNE_TABLE into TUNE_PC, and the interpreter reads successive bytes through this pointer, advancing it after each instruction.

654a ⟨*zero page defines* 25a⟩+≡ (909) ◁647b 670▷

```
TUNE_PC             EQU     $E4     ; script instruction pointer (2 bytes, $E4/$E5)
```

Defines:
TUNE_PC, used in chunks 653, 654b, 678, 679, and 681–84.

654b ⟨*tune player* 653⟩+≡ (909) ◁653 678▷

```
        ORG     $7980

    .load_ptr:
        LDA     TUNE_TABLE,X    ; low byte
        STA     TUNE_PC
        LDA     TUNE_TABLE+1,X  ; high byte
        STA     TUNE_PC+1       ; TUNE_PC/TUNE_PC+1 = script start address
```

Uses TUNE_PC 654a and TUNE_TABLE 651a.

Script 0 — empty

A single END byte. Used as a no-op placeholder (script index 0 is never called).

654c ⟨*tune bytecode* 654c⟩≡ (909) 655a▷

```
        ORG     $80C6
    TUNE_0:
        DC.B    $24                 ; END
```

Defines:
TUNE_0, never used.

Script 1 — combat hit

Played when an attack lands. The Y register holds the combat strength on entry; the tune player patches byte $80D5 with the decremented value to control the loop iteration count.

655a ⟨*tune bytecode* 654c⟩+≡ (909) ◁654c 655b▷

```
      ORG     $80C7
  TUNE_1_COMBAT_HIT:
      DC.B    $1F                ; MARK2
      DC.B    $26                ; NORAND
      DC.B    $23                ; HIGH
      DC.B    $C1,$05             ; C5 (48ms)
      DC.B    $22                ; LOW
      DC.B    $50,$06             ; Eb5 (49ms)
      DC.B    $3F,$08             ; G5 (52ms)
      DC.B    $25                ; HIRAND
      DC.B    $23                ; HIGH
      DC.B    $60,$10             ; C6 (77ms)
      DC.B    $00                ; LOOP 0 (patched at runtime)
      DC.B    $24                ; END
```

Defines:
TUNE_1_COMBAT_HIT, never used.

Script 2 — combat damage

Played when damage is dealt. Byte $80E3 is patched with the decremented damage amount.

655b ⟨*tune bytecode* 654c⟩+≡ (909) ◁655a

```
      ORG     $80D7
  TUNE_2_COMBAT_DAMAGE:
      DC.B    $25                ; HIRAND
      DC.B    $1F                ; MARK2
      DC.B    $22                ; LOW
      DC.B    $3F,$05             ; G5 (32ms)
      DC.B    $50,$04             ; Eb5 (33ms)
      DC.B    $80,$02             ; G4 (26ms)
      DC.B    $23                ; HIGH
      DC.B    $C1,$04             ; C5 (38ms)
      DC.B    $00                ; LOOP 0 (patched at runtime)
      DC.B    $24                ; END
```

Defines:
TUNE_2_COMBAT_DAMAGE, never used.

Scripts 3–11 — resident region

Note that scripts 4 and 5 overlap: script 5 (NPC death) starts at TUNE_5_NPC_DEATH, which is partway through script 4 (mob death). Script 5 is the shorter suffix of the same descending melody.

656 ⟨*resident tune bytecode* 656⟩≡ (909) 659a ▷

```
      ORG     $A300
  TUNE_3_ATTRACT:
      ; --- intro: exact notes, set gosub return point ---
      DC.B    $26               ; NORAND
      DC.B    $20               ; MARK1 (gosub target for GOTO1 at end)
      ; --- opening phrase (low then high) ---
      DC.B    $22               ; LOW
      DC.B    $3F,$65            ; G5 (654ms)
      DC.B    $23               ; HIGH
      DC.B    $3F,$49            ; G6 (234ms)
      DC.B    $3F,$8A            ; G6 (442ms)
      DC.B    $3F,$8A            ; G6 (442ms)
      DC.B    $3F,$8A            ; G6 (442ms)
      DC.B    $3F,$5C            ; G6 (295ms)
      ; --- ascending arpeggio (loop 1x) ---
      DC.B    $1F               ; MARK2
      DC.B    $22               ; LOW
      DC.B    $C1,$03            ; C4 (58ms)
      DC.B    $80,$04            ; G4 (51ms)
      DC.B    $50,$06            ; Eb5 (49ms)
      DC.B    $3F,$08            ; G5 (52ms)
      DC.B    $23               ; HIGH
      DC.B    $60,$0A            ; C6 (48ms)
      DC.B    $50,$0C            ; Eb6 (48ms)
      ; --- melody ---
      DC.B    $3F,$2E            ; G6 (147ms)
      DC.B    $47,$1B            ; F6 (97ms)
      DC.B    $3F,$1F            ; G6 (99ms)
      DC.B    $47,$1B            ; F6 (97ms)
      DC.B    $50,$18            ; Eb6 (97ms)
      DC.B    $47,$1B            ; F6 (97ms)
      DC.B    $50,$18            ; Eb6 (97ms)
      DC.B    $55,$17            ; D6 (98ms)
      DC.B    $50,$18            ; Eb6 (97ms)
      DC.B    $55,$17            ; D6 (98ms)
      DC.B    $60,$14            ; C6 (96ms)
      DC.B    $50,$18            ; Eb6 (97ms)
      DC.B    $3F,$1F            ; G6 (99ms)
      DC.B    $35,$24            ; Bb6 (98ms)
      DC.B    $38,$22            ; A6 (97ms)
      DC.B    $3F,$1F            ; G6 (99ms)
      DC.B    $3F,$E5            ; G6 (734ms)
      DC.B    $01               ; LOOP 1 (repeat arpeggio+melody once)
```

```
; --- second section ---
DC.B    $22                ; LOW
DC.B    $C1,$03             ; C4 (58ms)
DC.B    $80,$04             ; G4 (51ms)
DC.B    $23                ; HIGH
DC.B    $D9,$05             ; Bb4 (54ms)
DC.B    $22                ; LOW
DC.B    $50,$06             ; Eb5 (49ms)
DC.B    $3F,$08             ; G5 (52ms)
DC.B    $23                ; HIGH
DC.B    $6C,$09             ; Bb5 (49ms)
DC.B    $50,$0C             ; Eb6 (48ms)
DC.B    $3F,$1F             ; G6 (99ms)
DC.B    $60,$14             ; C6 (96ms)
DC.B    $50,$18             ; Eb6 (97ms)
DC.B    $3F,$1F             ; G6 (99ms)
DC.B    $35,$24             ; Bb6 (98ms)
DC.B    $38,$22             ; A6 (97ms)
DC.B    $3F,$1F             ; G6 (99ms)
DC.B    $3F,$5C             ; G6 (295ms)
; --- bridge ---
DC.B    $C1,$0A             ; C5 (96ms)
DC.B    $22                ; LOW
DC.B    $50,$0C             ; Eb5 (98ms)
DC.B    $3F,$0F             ; G5 (97ms)
DC.B    $23                ; HIGH
DC.B    $6C,$12             ; Bb5 (97ms)
DC.B    $72,$11             ; A5 (97ms)
DC.B    $22                ; LOW
DC.B    $3F,$0F             ; G5 (97ms)
DC.B    $3F,$8A             ; G5 (894ms)
DC.B    $3F,$8A             ; G5 (894ms)
; --- slow descending phrase ---
DC.B    $C1,$0F             ; C4 (288ms)
DC.B    $A2,$12             ; Eb4 (291ms)
DC.B    $80,$17             ; G4 (295ms)
DC.B    $23                ; HIGH
DC.B    $D9,$1B             ; Bb4 (290ms)
DC.B    $22                ; LOW
DC.B    $72,$1A             ; A4 (298ms)
DC.B    $80,$17             ; G4 (295ms)
DC.B    $23                ; HIGH
DC.B    $C1,$1F             ; C5 (297ms)
DC.B    $22                ; LOW
DC.B    $50,$24             ; Eb5 (293ms)
DC.B    $3F,$2E             ; G5 (298ms)
DC.B    $23                ; HIGH
DC.B    $6C,$36             ; Bb5 (292ms)
DC.B    $72,$34             ; A5 (297ms)
DC.B    $22                ; LOW
```

```
DC.B    $3F,$2E            ; G5 (298ms)
; --- long sustain + finale ---
DC.B    $23               ; HIGH
DC.B    $60,$7A            ; C6 (588ms)
DC.B    $50,$92            ; Eb6 (589ms)
DC.B    $3F,$B8            ; G6 (590ms)
DC.B    $35,$D9            ; Bb6 (589ms)
DC.B    $38,$CE            ; A6 (589ms)
DC.B    $38,$67            ; A6 (295ms)
DC.B    $3F,$5C            ; G6 (295ms)
; --- quick ascending run ---
DC.B    $22               ; LOW
DC.B    $FF,$02            ; G3 (51ms)
DC.B    $AC,$03            ; D4 (51ms)
DC.B    $80,$04            ; G4 (51ms)
DC.B    $23               ; HIGH
DC.B    $CD,$05            ; B4 (51ms)
DC.B    $AC,$06            ; D5 (51ms)
DC.B    $22               ; LOW
DC.B    $3F,$08            ; G5 (52ms)
DC.B    $23               ; HIGH
DC.B    $65,$0A            ; B5 (51ms)
DC.B    $55,$0B            ; D6 (47ms)
; --- closing sustained notes ---
DC.B    $3F,$C7            ; G6 (638ms)
DC.B    $3F,$8A            ; G6 (442ms)
DC.B    $3F,$8A            ; G6 (442ms)
DC.B    $3F,$8A            ; G6 (442ms)
DC.B    $3F,$8A            ; G6 (442ms)
DC.B    $21               ; GOTO1 (loop back to MARK1)
```

Defines:
TUNE_3_ATTRACT, never used.

Script 4 (mob death) and script 5 (NPC death) overlap: script 5 begins at TUNE_5_NPC_DEATH, which is partway through script 4. Both end at $A3DD.

659a ⟨*resident tune bytecode* 656⟩+≡ (909) ◁656 659b▷

```
    ORG     $A3C3
TUNE_4_MOB_DEATH:
    DC.B    $23                 ; HIGH
    DC.B    $50,$3D              ; Eb6 (246ms)
    DC.B    $47,$44              ; F6 (245ms)
    DC.B    $43,$90              ; Gb6 (490ms)
    DC.B    $47,$88              ; F6 (489ms)
    DC.B    $50,$79              ; Eb6 (489ms)
    ; --- script 5 entry point (NPC death, shorter suffix) ---
TUNE_5_NPC_DEATH:
    DC.B    $23                 ; HIGH
    DC.B    $50,$3D              ; Eb6 (246ms)
    DC.B    $47,$44              ; F6 (245ms)
    DC.B    $43,$48              ; Gb6 (245ms)
    DC.B    $35,$5A              ; Bb6 (244ms)
    DC.B    $47,$44              ; F6 (245ms)
    DC.B    $43,$48              ; Gb6 (245ms)
    DC.B    $50,$79              ; Eb6 (489ms)
    DC.B    $24                 ; END
```

Defines:
TUNE_4_MOB_DEATH, never used.
TUNE_5_NPC_DEATH, never used.

659b ⟨*resident tune bytecode* 656⟩+≡ (909) ◁659a 660a▷

```
    ORG     $A3DE
TUNE_6_COMBAT_AVOIDED:
    DC.B    $25                 ; HIRAND
    DC.B    $22                 ; LOW
    DC.B    $80,$07              ; G4 (90ms)
    DC.B    $23                 ; HIGH
    DC.B    $D9,$08              ; Bb4 (86ms)
    DC.B    $AC,$0A              ; D5 (85ms)
    DC.B    $22                 ; LOW
    DC.B    $3F,$0E              ; G5 (91ms)
    DC.B    $23                 ; HIGH
    DC.B    $6C,$10              ; Bb5 (87ms)
    DC.B    $55,$15              ; D6 (90ms)
    DC.B    $3F,$1C              ; G6 (90ms)
    DC.B    $35,$21              ; Bb6 (90ms)
    DC.B    $29,$29              ; D7 (87ms)
    DC.B    $27,$2B              ; Eb7 (87ms)
    DC.B    $27,$2B              ; Eb7 (87ms)
    DC.B    $27,$50              ; Eb7 (162ms)
    DC.B    $24                 ; END
```

Defines:
TUNE_6_COMBAT_AVOIDED, never used.

660a ⟨*resident tune bytecode* 656⟩+≡ (909) ◁659b 660b▷

```
        ORG     $A3FC
    TUNE_10_ENCOUNTER:
        DC.B    $25                 ; HIRAND
        DC.B    $23                 ; HIGH
        DC.B    $FA,$07              ; Ab4 (87ms)
        DC.B    $D7,$08              ; Bb4 (85ms)
        DC.B    $BE,$0A              ; C5 (94ms)
        DC.B    $AA,$0A              ; D5 (84ms)
        DC.B    $82,$0A              ; G5 (65ms)
        DC.B    $24                 ; END
```

Defines:
TUNE_10_ENCOUNTER, never used.

660b ⟨*resident tune bytecode* 656⟩+≡ (909) ◁660a 661a▷

```
        ORG     $A409
    TUNE_7_ENTITY_INTERACTION:
        DC.B    $25                 ; HIRAND
        DC.B    $23                 ; HIGH
        DC.B    $C1,$02              ; C5 (19ms)
        DC.B    $C1,$02              ; C5 (19ms)
        DC.B    $AC,$02              ; D5 (17ms)
        DC.B    $22                 ; LOW
        DC.B    $72,$02              ; A4 (23ms)
        DC.B    $23                 ; HIGH
        DC.B    $C1,$02              ; C5 (19ms)
        DC.B    $C1,$02              ; C5 (19ms)
        DC.B    $99,$06              ; E5 (46ms)
        DC.B    $22                 ; LOW
        DC.B    $80,$04              ; G4 (51ms)
        DC.B    $23                 ; HIGH
        DC.B    $C1,$03              ; C5 (29ms)
        DC.B    $C1,$01              ; C5 (10ms)
        DC.B    $C1,$02              ; C5 (19ms)
        DC.B    $C1,$08              ; C5 (77ms)
        DC.B    $AC,$01              ; D5 (9ms)
        DC.B    $22                 ; LOW
        DC.B    $72,$01              ; A4 (11ms)
        DC.B    $23                 ; HIGH
        DC.B    $C1,$03              ; C5 (29ms)
        DC.B    $C1,$05              ; C5 (48ms)
        DC.B    $24                 ; END
```

Defines:
TUNE_7_ENTITY_INTERACTION, never used.

661a ⟨*resident tune bytecode* 656⟩+≡ (909) ◁660b 661b▷

```
      ORG     $A432
  TUNE_8_AI_TURN:
      DC.B    $26                ; NORAND
      DC.B    $22                ; LOW
      DC.B    $3F,$12             ; G5 (117ms)
      DC.B    $24                ; END
```

Defines:
TUNE_8_AI_TURN, never used.

661b ⟨*resident tune bytecode* 656⟩+≡ (909) ◁661a 661c▷

```
      ORG     $A437
  TUNE_9_MENU_SELECTION:
      DC.B    $26                ; NORAND
      DC.B    $22                ; LOW
      DC.B    $AC,$05             ; D4 (86ms)
      DC.B    $24                ; END
```

Defines:
TUNE_9_MENU_SELECTION, never used.

661c ⟨*resident tune bytecode* 656⟩+≡ (909) ◁661b

```
      ORG     $A43C
  TUNE_11_MOVEMENT:
      DC.B    $26                ; NORAND
      DC.B    $23                ; HIGH
      DC.B    $3F,$1E             ; G6 (96ms)
      ; END byte ($24) is provided by TUNE_STUB_BIT's BIT opcode at [[TUNE_END_MARKER]]
```

Defines:
TUNE_11_MOVEMENT, never used.
Uses TUNE_END_MARKER 662.

18.2 Game support routines

The following routines provide keyboard input, scene data loading, disk checkpointing, and integrity checking.

18.2.1 Tune stubs and keyboard dispatch

TUNE_STUB_BIT is a short entry point that tests bit $A2 (a ZP flag), loads A=$0A, and jumps to the tune player at SETUP_AND_READ_DISK. DISK_READ_JUMP jumps to DISK_READ. DISK_RTS is a single RTS used as a no-op call target.

662 ⟨*tune stubs* 662⟩≡ (909)

```
    ORG     $A440
TUNE_END_MARKER:
    DC.B    $24                ; tune END byte (shared with TUNE_11)

LOAD_SCENE_DISK_ENTRY:
    LDX     #$0D                ; track $0D
    LDA     #$0A                ; sector $0A
    JMP     SETUP_AND_READ_DISK

DISK_READ_JUMP:
    JMP     DISK_READ

DISK_RTS:
    RTS
```

Defines:
DISK_READ_JUMP, used in chunks 666 and 669.
DISK_RTS, used in chunk 666.
LOAD_SCENE_DISK_ENTRY, never used.
TUNE_END_MARKER, used in chunk 661c.
Uses DISK_READ 546 and SETUP_AND_READ_DISK 669.

18.2.2 Read keyboard

READ_KEYBOARD polls for a keypress while running idle processing. It ticks the animation, steps the PRNG, and periodically verifies code integrity (1-in-64 chance per call via VERIFY_INTEGRITY). The main poll loop calls CHECK_INPUT (undocumented keyboard/joystick input handler) and IDLE_ANIM_HANDLER (undocumented idle animation handler) each iteration. On keypress it calls ANIM_CHECK_FRAME (the frame-check entry of ANIM_TICK_AND_WAIT) before returning the key code.

663 ⟨*read keyboard routine* 663⟩≡ (909)

```
    ORG     $A44C
READ_KEYBOARD:
    SUBROUTINE READ_KEYBOARD
    ; Poll for keyboard input with idle animation processing.
    ;
    ; Behavior:
    ;   1. Clears keyboard strobe, ticks animation via ANIM_TICK_AND_WAIT.
    ;   2. 1-in-64 chance: calls VERIFY_INTEGRITY (PRNG AND $7E == 0).
    ;   3. Poll loop: steps PRNG, checks keyboard/joystick via $755E.
    ;      On no input, runs idle handler at [[IDLE_ANIM_HANDLER]] and loops.
    ;   4. On keypress: clears strobe, runs frame check, returns key.
    ;
    ; Outputs:
    ;   A | key code (high bit set)
    ; Clobbers: A, X, Y

    LDA     #$FF
    STA     $C010               ; clear keyboard strobe
    JSR     ANIM_TICK_AND_WAIT ; tick animation + burn time
    LDA     PRNG_OUTPUT
    AND     #$7E                ; 1-in-64 chance (bits 6-1 all zero)
    BNE     .display
    JSR     VERIFY_INTEGRITY    ; periodic integrity check

.display:
    JSR     TICK_ANIM_COUNTER   ; tick animation counter
.poll:
    JSR     STEP_PRNG
    JSR     CHECK_INPUT          ; poll keyboard/joystick for input
    BPL     .got_key            ; key available (N flag clear)
    JSR     IDLE_ANIM_HANDLER   ; run idle animation
    JMP     .poll

.got_key:
    STA     $C010               ; clear keyboard strobe
    PHA
    JSR     ANIM_CHECK_FRAME    ; check frame counter (skip tick/wait)
    PLA
    RTS
```

Defines:
READ_KEYBOARD, used in chunks 232 and 247.
Uses ANIM_CHECK_FRAME 605b, ANIM_TICK_AND_WAIT 605b, CHECK_INPUT 58, IDLE_ANIM_HANDLER 52, PRNG_OUTPUT 593a, STEP_PRNG 595, TICK_ANIM_COUNTER 604b, and VERIFY_INTEGRITY 671.

18.2.3 Load scene data from disk

LOAD_SCENE_FROM_DISK reads one 256-byte sector of scene text data from the game disk into SCENE_TEXT_DATA.

Dual-use memory at SCENE_TEXT_DATA–$5CFF. The 256-byte buffer at SCENE_TEXT_DATA overlaps with active code. Loading a scene *overwrites* the following routines:

Address	Routine	Called from
$5C00–$5C0D	TICK_BORDER_ANIM tail	GAME_ACTION_HANDLER
$5C0E	ANIM_BORDER_ICONS_A	GAME_ACTION_HANDLER
$5C1A	ANIM_BORDER_ICONS_B	GAME_ACTION_HANDLER
$5C6D	INIT_GAME_DISPLAY	INIT_GAME_STATE
$5CEC	DRAW_BORDER_CHAR	INIT_GAME_DISPLAY
$5CFD	DRAW_BORDER_LABELS	INIT_GAME_DISPLAY

Call-chain analysis shows that this is **intentional**:

- **Startup-only routines** (INIT_GAME_DISPLAY, DRAW_BORDER_CHAR, DRAW_BORDER_LABELS): called exactly once during GAME_INIT → INIT_GAME_STATE. After the initial border frame is drawn, they are never called again and can be safely overwritten.
- **Idle animation** (GAME_ACTION_HANDLER, which contains TICK_BORDER_ANIM and ANIM_BORDER_ICONS_A/B): called during the input-poll idle loop via DISPATCH_VEC → TUNE_ADVANCE → JMP (DISPATCH_VEC). The first scene load *permanently destroys* the idle border animation. After that, the animated icon cycling along the border ceases, but gameplay is unaffected.
- **Post-scene code** (ENTER_LOCATION, called after SCENE_LOOP returns): does *not* call any $5C00+ routines. It redraws the map via DRAW_MAP_COL and DRAW_MAP_ENTITIES, neither of which depends on the border animation code.

In summary, the border code is expendable after the initial display is set up. The game deliberately reuses $5C00–$5CFF as a scene text buffer, accepting that the idle border animation is a one-time effect that stops after the first scene is shown.

Disk layout. Scene sectors begin at track \$11, sector 0 (GAME_SIG_TRACK/GAME_SIG_SECTOR). Scene N maps to:

$$\text{track} = \$11 + \lfloor N/13 \rfloor, \qquad \text{sector} = N \bmod 13$$

With 13 sectors per track, the 70 known scenes span tracks \$11–\$16.

Scene data format. Each 256-byte sector is a byte stream interpreted by TEXT_RENDERER:

- \$00 = space.
- \$01–\$1A = letters A–Z.
- \$1B–\$24 = digits 0–9.
- \$25 = comma, \$26 = colon/period, \$27 = hyphen, \$28 = apostrophe, \$2C = left paren, \$2D = right paren, \$2F = asterisk/divider.
- \$7D = reset to full-screen layout.
- \$7E = page break (plays tune, refreshes status bar, then re-initializes margins).
- \$7F = end of stream.
- \$80–\$FE = position command: bits 6–0 = row, next byte = column.

Characters \$00–\$7C are rendered using the custom 14×16 font (FONT_CHARSET=2). The renderer wraps text between TEXT_LEFT_MARGIN (default 2) and TEXT_RIGHT_MARGIN (default 18).

RWTS interface. The routine sets the RWTS IOB buffer address to \$5C00 (IOB bytes 8/9), computes the target track/sector from GAME_SIG_TRACK + scene number, and calls DISK_READ_JUMP with up to 2 retries.

666 ⟨*load scene from disk* 666⟩≡ (909)

```
      ORG     $A478
  LOAD_SCENE_FROM_DISK:
      SUBROUTINE LOAD_SCENE_FROM_DISK
      ;
      ; Reads one sector of scene text data from disk.
      ;
      ; Input:   SCENE_NUMBER = scene index (0-69)
      ; Output:  C clear = success (data at [[SCENE_TEXT_DATA]]), C set = failure
      ; Clobbers: A, X, Y, $BE
      ;
```

```
    ; Disk mapping:
    ;   track  = GAME_SIG_TRACK + (GAME_SIG_SECTOR + SCENE_NUMBER) / 13
    ;   sector = (GAME_SIG_SECTOR + SCENE_NUMBER) mod 13
    ;   With GAME_SIG_TRACK=$11, GAME_SIG_SECTOR=$00:
    ;     scene 0  -> T$11 S$00
    ;     scene 12 -> T$11 S$0C
    ;     scene 13 -> T$12 S$00
    ;     scene 69 -> T$16 S$04
    ;
    ; --- Set RWTS IOB buffer to $5C00 (SCENE_TEXT_DATA) ---
    ;
    JSR     DISK_RTS            ; no-op (cracked-out hook)
    LDA     #$00
    LDY     #$08
    STA     (RWTS_IOB_PTR),Y    ; IOB+8 = buffer lo ($00)
    INY
    LDA     #$5C
    STA     (RWTS_IOB_PTR),Y    ; IOB+9 = buffer hi ($5C)
    ;
    ; --- Compute track and sector ---
    ;
    LDA     GAME_SIG_TRACK      ; track = base track ($11)
    STA     $BE                 ; $BE = working track number
    LDA     GAME_SIG_SECTOR     ; A = base sector ($00)
    CLC
    ADC     SCENE_NUMBER        ; A = base_sector + scene_number
.mod13:
    CMP     #$0D                ; A >= 13?
    BCC     .set_iob            ; no -> A = final sector
    INC     $BE                 ; yes -> track++
    SEC
    SBC     #$0D                ; A -= 13
    JMP     .mod13              ; repeat until A < 13
    ;
    ; --- Store track/sector in RWTS IOB and read ---
    ;
.set_iob:
    LDY     #$05
    STA     (RWTS_IOB_PTR),Y    ; IOB+5 = sector
    DEY
    LDA     $BE
    STA     (RWTS_IOB_PTR),Y    ; IOB+4 = track
    LDA     #$02                ; 2 retries on failure
    STA     $BE
.retry:
    JSR     DISK_READ_JUMP      ; RWTS read
    BCS     .error              ; C set = read failed
    JSR     DISK_RTS            ; no-op (cracked-out hook)
    CLC                         ; C clear = success
    RTS
```

```
.error:
    DEC     $BE                 ; retries--
    BNE     .retry              ; try again if retries remain
    JSR     DISK_RTS            ; no-op (cracked-out hook)
    SEC                         ; C set = all retries exhausted
    RTS
```

Defines:
LOAD_SCENE_FROM_DISK, used in chunks 527 and 584.
Uses DISK_READ_JUMP 662, DISK_RTS 662, GAME_SIG_SECTOR 300, GAME_SIG_TRACK 300, RWTS_IOB_PTR 589a, SCENE_NUMBER 526b, and SCENE_TEXT_DATA 526a.

18.2.4 Tune player dispatch

SETUP_AND_READ_DISK sets up the RWTS IOB with the given track (X) and sector (A), buffer at $A400, then reads from disk with retries. On persistent failure it jumps to the COLD_RESTART error handler.

669 ⟨*tune player dispatch* 669⟩≡ (909)

```
      ORG     $A4BE
  SETUP_AND_READ_DISK:
      SUBROUTINE SETUP_AND_READ_DISK
      ; Set up RWTS IOB and read one sector into $A400.
      ;
      ; Inputs:
      ;   X | track number
      ;   A | sector number
      ;
      ; Behavior:
      ;   Stores track/sector into RWTS IOB, sets buffer to $A400.
      ;   Attempts disk read with up to 2 retries. On persistent
      ;   failure, jumps to COLD_RESTART (does not return).
      ;
      ; Clobbers: A, X, Y

      LDY     #$05
      STA     (RWTS_IOB_PTR),Y   ; IOB+5 = sector (A)
      DEY
      TXA
      STA     (RWTS_IOB_PTR),Y   ; IOB+4 = track (X)
      LDA     #$00
      LDY     #$08
      STA     (RWTS_IOB_PTR),Y   ; IOB+8 = buffer lo ($00)
      INY
      LDA     #$A4
      STA     (RWTS_IOB_PTR),Y   ; IOB+9 = buffer hi ($A4 -> $A400)
      LDA     #$02               ; retry counter
      STA     $BE
  .retry:
      JSR     DISK_READ_JUMP
      BCS     .error
      RTS                        ; success
  .error:
      DEC     $BE
      BNE     .retry
      JMP     COLD_RESTART       ; fatal: disk read failure
```

Defines:
SETUP_AND_READ_DISK, used in chunk 662.
Uses COLD_RESTART 675, DISK_READ_JUMP 662, and RWTS_IOB_PTR 589a.

18.2.5 Integrity verification

670 ⟨*zero page defines* 25a⟩+≡ (909) ◁654a 674a▷

```
CKSUM_START          EQU     $BA     ; checksum range start pointer (2 bytes, aliased)
CKSUM_END            EQU     $BE     ; checksum range end pointer (2 bytes, aliased)
```

Defines:
CKSUM_END, used in chunks 671 and 677.
CKSUM_START, used in chunks 671 and 677.

INTEGRITY_DATA ($A4E2). Holds two pairs of expected checksum values (4 bytes total), stored as self-modifying operands.

VERIFY_INTEGRITY ($A4E6). Computes a checksum over DISK_IO_START–DISK_IO_END (the disk I/O routines) using COMPUTE_CHECKSUM, and compares the result with the first expected pair at INTEGRITY_DATA. On mismatch, jumps to INTEGRITY_FAIL_ADDR to reinitialize.

VERIFY_DISK_IO ($A4F8). Computes a checksum over SCENE_LOADER_START–SCENE_LOADER_END (the scene loader) and compares with the second expected pair.

COMPUTE_CHECKSUM ($A50E). Accumulates a running XOR/ADD checksum over the byte range CKSUM_START/CKSUM_START+1 through CKSUM_END/CKSUM_END+1.

- Odd-indexed bytes are ADCed into CKSUM_ACC.
- Even-indexed bytes are ADCed into CKSUM_ACC+1, then EORed into CKSUM_ACC.
- Returns the 16-bit result in A (low) and Y (high).

671 ⟨*integrity verification* 671⟩≡ (909)

```
    ORG     $A4E2
INTEGRITY_DATA:
    HEX     AE 7A 4F 87        ; expected checksums (2 pairs)

VERIFY_INTEGRITY:
    SUBROUTINE VERIFY_INTEGRITY
    ; Verify checksum of disk I/O routines.
    ;
    ; Behavior:
    ;   Computes a 16-bit XOR/ADD checksum over DISK_IO_START to
    ;   DISK_IO_END via COMPUTE_CHECKSUM.  Compares against expected
    ;   values at INTEGRITY_DATA.  On mismatch, jumps to
    ;   INTEGRITY_FAIL_ADDR (does not return).
    ;
    ; Modifies: CKSUM_START, CKSUM_END, CKSUM_ACC, CKSUM_ACC+1
    ; Clobbers: A, X, Y

    LDA     #<DISK_IO_START    ; \ range: DISK_IO_START to DISK_IO_END
    STA     CKSUM_START        ;  |
    LDA     #>DISK_IO_START    ;  |
    STA     CKSUM_START+1      ;  |
    LDA     #<DISK_IO_END      ;  |
    STA     CKSUM_END          ;  |
    LDA     #>DISK_IO_END      ;  |
```

```
    STA     CKSUM_END+1         ; /
    JSR     COMPUTE_CHECKSUM
    CMP     INTEGRITY_DATA      ; compare low byte
    BNE     .fail
    TYA
    CMP     INTEGRITY_DATA+1    ; compare high byte
    BNE     .fail
    RTS                         ; checksums match
.fail:
    JMP     INTEGRITY_FAIL_ADDR ; reinitialize on mismatch

VERIFY_DISK_IO:
    LDA     #<SCENE_LOADER_START ; \ range: SCENE_LOADER_START to SCENE_LOADER_END
    STA     CKSUM_START          ;  |
    LDA     #>SCENE_LOADER_START ;  |
    STA     CKSUM_START+1        ;  |
    LDA     #<SCENE_LOADER_END ;  |
    STA     CKSUM_END          ;  |
    LDA     #>SCENE_LOADER_END ;  |
    STA     CKSUM_END+1          ; /
    JSR     COMPUTE_CHECKSUM
    CMP     INTEGRITY_DATA+2    ; compare low byte
    BNE     .fail
    TYA
    CMP     INTEGRITY_DATA+3    ; compare high byte
    BNE     .fail
    RTS
    ; unreachable padding (overwritten at runtime as checksum accumulators)
CKSUM_ACC:
    DC.B    $B1                 ; checksum accumulator low (self-modifying)
    DC.B    $A0                 ; checksum accumulator high (+1)

COMPUTE_CHECKSUM:
    SUBROUTINE COMPUTE_CHECKSUM
    ; Compute a 16-bit checksum over a memory range.
    ;
    ; Inputs:
    ;   CKSUM_START/+1 | start address (inclusive)
    ;   CKSUM_END/+1   | end address (exclusive)
    ;
    ; Behavior:
    ;   Iterates byte-by-byte. Odd bytes are ADCed into CKSUM_ACC;
    ;   even bytes are ADCed into CKSUM_ACC+1 then EORed into
    ;   CKSUM_ACC.
    ;
    ; Outputs:
    ;   A | checksum low byte
    ;   Y | checksum high byte
    ; Modifies: CKSUM_START (advanced to end), CKSUM_ACC, CKSUM_ACC+1
    ; Clobbers: A, Y
```

```
    CLC
    LDY     #$00
    STY     CKSUM_ACC       ; checksum low = 0
    STY     CKSUM_ACC+1       ; checksum high = 0
.loop:
    LDA     (CKSUM_START),Y
    ADC     CKSUM_ACC
    STA     CKSUM_ACC
    JSR     .advance           ; advance pointer
    BCS     .done              ; reached end -> return
    LDA     (CKSUM_START),Y
    ADC     CKSUM_ACC+1
    STA     CKSUM_ACC+1
    EOR     CKSUM_ACC
    STA     CKSUM_ACC
    JSR     .advance
    BCC     .loop              ; not done -> continue

.done:
    LDA     CKSUM_ACC       ; return checksum low in A
    LDY     CKSUM_ACC+1       ; return checksum high in Y
    RTS

.advance:
    INC     CKSUM_START
    BNE     .no_carry
    INC     CKSUM_START+1
.no_carry:
    LDA     CKSUM_START
    CMP     CKSUM_END
    BNE     .not_end
    LDA     CKSUM_START+1
    CMP     CKSUM_END+1
    BNE     .not_end
    SEC                        ; reached end of range
    RTS
.not_end:
    CLC
    RTS
```

Defines:
CKSUM_ACC, never used.
COMPUTE_CHECKSUM, used in chunk 677.
INTEGRITY_DATA, never used.
VERIFY_DISK_IO, used in chunk 222.
VERIFY_INTEGRITY, used in chunk 663.

Uses CKSUM_END 670, CKSUM_START 670, DISK_IO_END 777, DISK_IO_START 771, INTEGRITY_FAIL_ADDR 529, SCENE_LOADER_END 428, and SCENE_LOADER_START 529.

18.2.6 Cold restart error handler

674a ⟨*zero page defines* 25a⟩+≡ (909) ◁670 676▷

```
CLEAR_PTR          EQU     $BE     ; memory-clear pointer (2 bytes, aliased)
```

Defines:
CLEAR_PTR, used in chunk 675.

674b ⟨*defines* 25b⟩+≡ (909) ◁634 685▷

```
DISK_BOOT_ROM      EQU     $C600   ; Disk II boot ROM entry point
WIPE_END_PAGE      EQU     $A5     ; page past last full page to clear ($A500)
WIPE_TAIL_LEN      EQU     $6F     ; bytes to clear in final partial page ($A500-$A56E)
```

Defines:
DISK_BOOT_ROM, used in chunks 675, 800a, 801, 832, and 853b.
WIPE_END_PAGE, used in chunk 675.
WIPE_TAIL_LEN, used in chunk 675.

COLD_RESTART is the fatal error handler. It clears memory from MAIN_ENTRY to COLD_RESTART, then jumps to DISK_BOOT_ROM to restart the system from disk.

675 ⟨*cold restart* 675⟩≡ (909)

```
    ORG     $A56F
COLD_RESTART:
    SUBROUTINE COLD_RESTART
    ; Wipe program memory and reboot from disk.
    ;
    ; Behavior:
    ;   Zeros all memory from MAIN_ENTRY ($0800) through COLD_RESTART
    ;   ($A56F), then jumps to DISK_BOOT_ROM. Does not return.
    ;
    ; Modifies: $0800-$A56F, CLEAR_PTR
    ; Clobbers: A, Y

    LDA     #<MAIN_ENTRY        ; \ CLEAR_PTR = $0800
    STA     CLEAR_PTR           ;  |
    LDA     #>MAIN_ENTRY        ;  |
    STA     CLEAR_PTR+1         ; /
    LDY     #$00
.clear:
    STA     (CLEAR_PTR),Y
    DEY
    BNE     .clear
    INC     CLEAR_PTR+1
    LDA     CLEAR_PTR+1
    CMP     #WIPE_END_PAGE      ; stop at page $A5
    BNE     .clear
    LDY     #WIPE_TAIL_LEN
.clear_tail:
    STA     (CLEAR_PTR),Y       ; clear $A500-$A56E
    DEY
    BNE     .clear_tail
    JMP     DISK_BOOT_ROM       ; reboot from Disk II
```

Defines:
COLD_RESTART, used in chunks 305 and 669.
Uses CLEAR_PTR 674a, DISK_BOOT_ROM 674b, MAIN_ENTRY 219, WIPE_END_PAGE 674b, and WIPE_TAIL_LEN 674b.

18.2.7 Pointer advance helpers

These are small utility routines for advancing pointers and comparing against end-of-range boundaries. They duplicate logic from COMPUTE_CHECKSUM's .advance but use different ZP pointer pairs. All three routines are unreferenced dead code — zero callers in the binary.

LOAD_END_OFFSET ($A590). Loads Y from $A563, which is the BNE operand byte inside COMPUTE_CHECKSUM. No callers exist; always returns the disk-image branch offset.

ADVANCE_BA ($A594). Increments the 16-bit pointer at CKSUM_START and compares against CKSUM_END. Returns carry set when the end is reached.

ADVANCE_1A ($A5AA). Same as ADVANCE_CKSUM_START but using the ZP pointer pair RANGE_CTR/RANGE_CTR+1 vs RANGE_END/RANGE_END+1.

676 ⟨*zero page defines* 25a⟩+≡ (909) ◁674a 695▷

```
RANGE_CTR       EQU     $1A     ; range counter pointer (2 bytes, dead code)
RANGE_END       EQU     $1E     ; range end pointer (2 bytes, dead code)
```

Defines:
RANGE_CTR, used in chunk 677.
RANGE_END, used in chunk 677.

677 ⟨*pointer helpers* 677⟩≡ (909)

```
        ORG     $A590
    LOAD_END_OFFSET:
        LDY     $A563               ; BNE operand inside COMPUTE_CHECKSUM (dead code)
        RTS

    ADVANCE_CKSUM_START:
        INC     CKSUM_START
        BNE     .no_carry
        INC     CKSUM_START+1
    .no_carry:
        LDA     CKSUM_START
        CMP     CKSUM_END
        BNE     .not_end
        LDA     CKSUM_START+1
        CMP     CKSUM_END+1
        BNE     .not_end
        SEC
        RTS
    .not_end:
        CLC
        RTS

    ADVANCE_RANGE_CTR:
        INC     RANGE_CTR+1
        LDA     RANGE_CTR
        CMP     RANGE_END
        BNE     .not_end2
        LDA     RANGE_CTR+1
        CMP     RANGE_END+1
        BNE     .not_end2
        SEC
        RTS
    .not_end2:
        CLC
        RTS
```

Defines:
ADVANCE_CKSUM_START, never used.
ADVANCE_RANGE_CTR, never used.
LOAD_END_OFFSET, never used.

Uses CKSUM_END 670, CKSUM_START 670, COMPUTE_CHECKSUM 671, RANGE_CTR 676, and RANGE_END 676.

TUNE_LOOP_CTR counts iterations of the script's built-in loop construct. TUNE_PRNG_MASK controls action randomization: the interpreter ANDs the PRNG output with this mask before dispatching. Both are initialized to 0 and 1 respectively at the start of each script invocation.

After setup, the interpreter initializes its loop counter and PRNG mask:

678 ⟨*tune player* 653⟩+≡ (909) ◁654b 679▷

```
    ORG     $798A

    STOB    #$00,TUNE_LOOP_CTR ; loop counter = 0
    STOB    #$01,TUNE_PRNG_MASK ; PRNG mask = 1
TUNE_PLAYER_FETCH:
.fetch:
    LDY     #$00
    LDA     (TUNE_PC),Y    ; read script byte
    BALT    #$27,.control ; < $27 -> control command
    JMP     .action          ; >= $27 -> action dispatch
```

Uses STOB 8c, TUNE_LOOP_CTR 651b, TUNE_PC 654a, and TUNE_PRNG_MASK 652c.

Opcode reference

Each script byte is interpreted as follows:

Byte	Mnemonic	Effect
$00–$1E	LOOP n	If loop counter $< n$, increment counter and jump to MARK2 address; else reset co
$1F	MARK2	Save next address as loop target
$20	MARK1	Save next address as gosub target
$21	GOTO1	Set PC to MARK1 address (gosub/restart)
$22	LOW	Set action vector to `PLAY_LOW_TONE`
$23	HIGH	Set action vector to `PLAY_HIGH_TONE`
$24	END	Return from tune
$25	HIRAND	Set PRNG mask to $0F (high randomness)
$26	NORAND	Set PRNG mask to $00 (exact notes)
$27–$FF	NOTE f,d	Play note: frequency byte f (XORed with masked PRNG), followed by duration b

NOTE dispatches through `TUNE_ACTION_VEC`, which is set by the LOW/HIGH commands to point at `PLAY_HIGH_TONE`. The PRNG mask (set by NORAND/HIRAND) controls how much the frequency byte is randomized before playback.

Control commands (¡ $27)

`TUNE_LOOP_ADDR` is a 2-byte address saved by the "set loop address" command ($1F). When the loop counter has not yet reached the target byte value, the interpreter copies `TUNE_LOOP_ADDR` back into `TUNE_PC` to restart the loop body.

679 ⟨*tune player* 653⟩+≡ (909) ◁678 681▷

```
    ORG     $799F

.control:
    BAEQ    #$25,.cmd_25            ; set PRNG mask = $0F
    BCS     .cmd_26                 ; $26: PRNG mask = $00

    BAEQ    #$23,.cmd_23            ; $23: HIGH (PLAY_HIGH_TONE)
    BCS     .cmd_24                 ; $24: END (return)

    BAEQ    #$21,.cmd_21            ; $21: gosub
    BCS     .cmd_22                 ; $22: LOW (PLAY_LOW_TONE)

    BAEQ    #$1F,.cmd_1F            ; set loop address
    BCS     .cmd_20                 ; set gosub address

    ; --- Loop/end-loop (byte < $1F) ---
    BAEQ    TUNE_LOOP_CTR,.end_loop ; compare with loop counter
```

```
    BCC     .end_loop

    INC     TUNE_LOOP_CTR           ; counter++
    MOVW    TUNE_LOOP_ADDR,TUNE_PC ;  TUNE_PC/TUNE_PC+1 = loop address
    JMP     .fetch

.end_loop:
    STOB    #$00,TUNE_LOOP_CTR ; reset loop counter
    JMP     TUNE_ADVANCE
```

Uses MOVW 9a, PLAY_HIGH_TONE 647a, PLAY_LOW_TONE 648, STOB 8c, TUNE_ADVANCE 683, TUNE_LOOP_ADDR 652a, TUNE_LOOP_CTR 651b, and TUNE_PC 654a.

When the byte value equals or exceeds the loop counter, the loop terminates and the counter resets. Otherwise, the counter increments and execution jumps back to the saved loop address.

Command implementations

TUNE_GOSUB_ADDR saves the return address for the script's gosub command ($21): after the subroutine finishes, TUNE_PC is restored from this 2-byte value. TUNE_ACTION_VEC is a 2-byte vector set by commands $22 and $23 to select between the low and high tone handlers.

681 ⟨*tune player* 653⟩+≡ (909) ◁679 682▷

```
    ORG     $79D6

.cmd_25:                            ; PRNG mask = $0F
    STOB    #$0F,TUNE_PRNG_MASK
    JMP     TUNE_ADVANCE

.cmd_26:                            ; PRNG mask = $00
    STOB    #$00,TUNE_PRNG_MASK
    JMP     TUNE_ADVANCE

.cmd_23:                            ; HIGH ($23): action vector -> PLAY_HIGH_TONE
    STOW    PLAY_HIGH_TONE,TUNE_ACTION_VEC
    JMP     TUNE_ADVANCE

.cmd_22:                            ; LOW ($22): action vector -> PLAY_LOW_TONE
    STOW    PLAY_LOW_TONE,TUNE_ACTION_VEC
    JMP     TUNE_ADVANCE

.cmd_24:                            ; END ($24): return from tune
    RTS

.cmd_21:                            ; GOTO1 ($21): jump to saved address
    MOVW    TUNE_GOSUB_ADDR,TUNE_PC
    JMP     .fetch

.cmd_1F:                            ; set loop address = current+1
    JSR     TUNE_INC_PC
    MOVW    TUNE_PC,TUNE_LOOP_ADDR
    JMP     .fetch

.cmd_20:                            ; set gosub address = current+1
    JSR     TUNE_INC_PC
    MOVW    TUNE_PC,TUNE_GOSUB_ADDR
    JMP     .fetch
```

Uses MOVW 9a, PLAY_HIGH_TONE 647a, PLAY_LOW_TONE 648, STOB 8c, STOW 8a, TUNE_ACTION_VEC 652d, TUNE_ADVANCE 683, TUNE_GOSUB_ADDR 652b, TUNE_INC_PC 684b, TUNE_LOOP_ADDR 652a, TUNE_PC 654a, and TUNE_PRNG_MASK 652c.

Action dispatch (≥ $27)

When the script byte is $27 or higher, the PRNG output is masked and XORed with the action byte, introducing randomness. The action vector at \DISPATCH_VEC is set by the handlers at TUNE_ACTION_VEC.

682 ⟨*tune player* 653⟩+≡ (909) ◁681

```
    ORG     $7A2E

.action:
    STA     SOUND_DURATION          ; save action byte
    JSR     STEP_PRNG
    LDA     PRNG_OUTPUT             ; PRNG output
    AND     TUNE_PRNG_MASK        ; apply mask
    EOR     SOUND_DURATION          ; XOR with action byte
    STA     SOUND_FREQ              ; modified action
    JSR     TUNE_INC_PC           ; advance tune pointer
    LDY     #$00
    LDA     (TUNE_PC),Y           ; read parameter byte
    STA     SOUND_DURATION
    JSR     TUNE_ACTION_INDIRECT ; -> JMP (TUNE_ACTION_VEC)
    JMP     (DISPATCH_VEC)          ; dispatch via action vector
```

Defines:
TUNE_PLAYER, used in chunks 232, 239, 247, 342, 346, 349, 354, 360, 371, 527, 532c, 570b, 644, and 653.

Uses DISPATCH_VEC 569b, PRNG_OUTPUT 593a, SOUND_DURATION 646, SOUND_FREQ 646, STEP_PRNG 595, TUNE_ACTION_INDIRECT 684a, TUNE_ACTION_VEC 652d, TUNE_INC_PC 684b, TUNE_PC 654a, and TUNE_PRNG_MASK 652c.

The "advance" routine is the target of all JMP TUNE_ADVANCE in the control commands. It increments the tune pointer, checks for user input (during attract mode), and loops back to .fetch.

During the attract screen (ATTRACT_FLAG = 1), TUNE_ADVANCE checks for keyboard or joystick input. If detected, it returns immediately via RTS, which exits the entire TUNE_PLAYER call.

683 ⟨*tune advance* 683⟩≡ (909)

```
    ORG     $7A4F
TUNE_ADVANCE:
    SUBROUTINE TUNE_ADVANCE
    ; Advance the tune player by one step.
    ;
    ; Behavior:
    ;   Increments the tune program counter, then checks for user
    ;   input if in attract mode. Returns early if input detected;
    ;   otherwise halves WAIT_LOOP_COUNT and jumps back to the
    ;   tune interpreter fetch loop.
    ;
    ; Modifies: TUNE_PC, WAIT_LOOP_COUNT
    ; Clobbers: A, X

    JSR     TUNE_INC_PC       ; advance tune pointer
    LDA     ATTRACT_FLAG        ; attract_flag
    BEQ     .continue           ; not attract mode -> skip input check

    ; Attract mode: check keyboard
    LDA     $C000               ; keyboard register
    BPL     .check_joy          ; no key -> check joystick
    RTS                         ; key pressed -> exit

.check_joy:
    LDA     INPUT_MODE          ; joystick flag
    BEQ     .continue           ; no joystick -> continue script
    LDX     #$00
    LDA     $C061,X             ; button 0
    BPL     .btn1               ; not pressed
    RTS                         ; pressed -> exit
.btn1:
    INX
    LDA     $C061,X             ; button 1
    BPL     .continue           ; not pressed -> continue
    RTS                         ; pressed -> exit

.continue:
    LSR     WAIT_LOOP_COUNT     ; shift timing flag
    JMP     TUNE_PLAYER_FETCH ; loop back to interpreter
```

Defines:
TUNE_ADVANCE, used in chunks 570b, 584, 679, and 681.

Uses ATTRACT_FLAG 652e, INPUT_MODE 589c, TUNE_INC_PC 684b, TUNE_PC 654a, and WAIT_LOOP_COUNT 51.

684a ⟨*tune action indirect* 684a⟩≡ (909)

```
    ORG     $7A4C
TUNE_ACTION_INDIRECT:
    JMP     (TUNE_ACTION_VEC)
```

Defines:
TUNE_ACTION_INDIRECT, used in chunk 682.
Uses TUNE_ACTION_VEC 652d.

TUNE_INC_PC increments the 16-bit script program counter at TUNE_PC/TUNE_PC+1, handling the carry from low to high byte.

684b ⟨*tune helpers* 684b⟩≡ (909)

```
    ORG     $7A77
TUNE_INC_PC:
    SUBROUTINE TUNE_INC_PC
    ; Increment the 16-bit tune program counter.
    ;
    ; Behavior:
    ;   Increments TUNE_PC low byte; if it wraps to zero,
    ;   increments TUNE_PC+1 high byte.
    ;
    ; Modifies: TUNE_PC
    ; Clobbers: none (flags only)

    INC     TUNE_PC
    BEQ     .carry
    RTS
.carry:
    INC     TUNE_PC+1
    RTS
```

Defines:
TUNE_INC_PC, used in chunks 681–83.
Uses TUNE_PC 654a.

Chapter 19

The Attract Screen

The attract screen is not special—it is an ordinary scene (scene $26) played through SCENE_LOOP, just like every other scene in the game. It animates and plays a tune until the player presses a key. The only difference is that ATTRACT_SCREEN is called from MAIN_ENTRY on first run (when TOTAL_MOB_COUNT = 0), and when the scene ends, control returns to MAIN_ENTRY where the game is initialized and started normally.

The soft-switch reads at TXTCLR–HIRES configure the Apple II display: GR, full-screen (no split text), page 1, hi-res. This puts the display into HGR mode showing the framebuffer at HIRES_PAGE–$3FFF.

19.0.1 Sound and settings routines

685 ⟨*defines* 25b⟩+≡ (909) ◁674b 705▷

```
S_SOUND_CHAR1      EQU    $7B0F    ; 'N'/'F' position in "SOUND IS ON/OFF" label
S_SOUND_CHAR2      EQU    $7B10    ; ' '/'F' position in "SOUND IS ON/OFF" label
S_DANGER_OUT       EQU    $7B22    ; DECIMAL_OUT_PTR target for danger level display
RTS_OPCODE         EQU    $60      ; RTS instruction byte (used to disable tone routines)
LDY_OPCODE         EQU    $A0      ; LDY immediate byte (original tone routine entry)
MAX_THRESHOLD      EQU    $63      ; maximum EVENT_THRESHOLD value (99 decimal)
SCENE_RETIRE       EQU    $23      ; scene number for character retirement
```

Defines:
LDY_OPCODE, used in chunk 686.
MAX_THRESHOLD, used in chunk 686.
RTS_OPCODE, used in chunk 686.
SCENE_RETIRE, used in chunk 268.
S_DANGER_OUT, used in chunk 686.
S_SOUND_CHAR1, used in chunk 686.
S_SOUND_CHAR2, used in chunk 686.
Uses DECIMAL_OUT_PTR 630a and EVENT_THRESHOLD 207.

TOGGLE_SOUND is the handler for the "SOUND IS ON" menu option. It checks whether PLAY_HIGH_TONE starts with $60 (RTS, meaning sound is off) or $A0 (LDY, meaning sound is on). To disable sound, it writes RTS ($60) to both PLAY_HIGH_TONE and PLAY_LOW_TONE, making them return immediately. To enable sound, it restores the original LDY #$0A opcode ($A0). It also patches the "SOUND IS ON" label string: "ON " ($CE/$A0) becomes "OFF" ($C6/$C6).

SETTINGS_DISPLAY_INIT initializes the game settings display by writing spaces into the "DANGER" label, formatting EVENT_THRESHOLD via NUM_TO_DECIMAL, and jumping to the settings menu.

DEC_EVENT_THRESHOLD and INC_EVENT_THRESHOLD adjust the random event probability (clamped to 0–99).

686 ⟨*sound and settings* 686⟩≡ (909)

```
    ORG     $0984
TOGGLE_SOUND:
    SUBROUTINE TOGGLE_SOUND
    ; Sound toggle, settings display, threshold adjustment, and retirement.
    ;
    ; Entry points:
    ;   TOGGLE_SOUND          | toggles sound on/off by patching tone routines
    ;   SETTINGS_DISPLAY_INIT | formats danger level display, enters settings menu
    ;   DEC_THRESHOLD_WRAP    | decrements event threshold, returns to caller
    ;   INC_THRESHOLD_WRAP    | increments event threshold, returns to caller
    ;   DEC_EVENT_THRESHOLD   | decrements EVENT_THRESHOLD (clamps to 0)
    ;   INC_EVENT_THRESHOLD   | increments EVENT_THRESHOLD (clamps to MAX_THRESHOLD)
    ;
    ; Modifies: PLAY_HIGH_TONE, PLAY_LOW_TONE (patched), EVENT_THRESHOLD,
    ;           S_SOUND_CHAR1, S_SOUND_CHAR2, S_DANGER_OUT+2..+4
    ; Clobbers: A

    ; --- Toggle sound on/off ---
    LDA     #RTS_OPCODE                 ; RTS opcode
    CMP     PLAY_HIGH_TONE              ; is sound disabled?
    BNE     .turn_off
    ; --- Enable sound ---
    LDA     #$CE                        ; 'N' (high-bit ASCII)
    STA     S_SOUND_CHAR1               ; patch "OFF" -> "ON "
    LDA     #LDY_OPCODE                 ; ' ' (space) / LDY opcode
    STA     S_SOUND_CHAR2               ; trailing space
    LDA     #LDY_OPCODE                 ; restore original entry byte
.patch_tones:
    STA     PLAY_HIGH_TONE              ; \ patch tone routine entry bytes
    STA     PLAY_LOW_TONE               ; /
    JMP     MENU_GAME_SETTINGS          ; re-enter settings menu
.turn_off:
    ; --- Disable sound ---
    LDA     #$C6                        ; 'F' (high-bit ASCII)
```

```
    STA     S_SOUND_CHAR1               ; patch "ON " -> "OFF"
    STA     S_SOUND_CHAR2               ;
    LDA     #RTS_OPCODE                 ; RTS opcode
    BNE     .patch_tones                ; always taken

    ; --- Initialize settings display ---
SETTINGS_DISPLAY_INIT:
    LDA     #$20                        ; \ clear danger digit area
    STA     S_DANGER_OUT+2              ;  |
    STA     S_DANGER_OUT+3              ;  |
    STA     S_DANGER_OUT+4              ; /
    LDA     #$00                        ; \ NUM_VALUE = EVENT_THRESHOLD
    STA     NUM_VALUE+1                 ;  | (high byte = 0)
    LDA     EVENT_THRESHOLD             ;  |
    STA     NUM_VALUE                   ; /
    LDA     #<S_DANGER_OUT              ; \ DECIMAL_OUT_PTR -> S_DANGER_OUT
    STA     DECIMAL_OUT_PTR             ;  |
    LDA     #>S_DANGER_OUT              ;  |
    STA     DECIMAL_OUT_PTR+1           ; /
    JMP     NUM_TO_DECIMAL              ; format and return

    ; --- Decrement event threshold (wrapper for menu) ---
DEC_THRESHOLD_WRAP:
    JSR     DEC_EVENT_THRESHOLD
    RTS

    ; --- Increment event threshold (wrapper for menu) ---
INC_THRESHOLD_WRAP:
    JSR     INC_EVENT_THRESHOLD
    RTS

    ; --- Decrement EVENT_THRESHOLD (clamp to 0) ---
DEC_EVENT_THRESHOLD:
    DEC     EVENT_THRESHOLD
    BPL     .dec_done                   ; >= 0 -> ok
    INC     EVENT_THRESHOLD             ; clamp: -1 -> 0
.dec_done:
    RTS

    ; --- Increment EVENT_THRESHOLD (clamp to MAX_THRESHOLD) ---
INC_EVENT_THRESHOLD:
    LDA     EVENT_THRESHOLD
    CMP     #MAX_THRESHOLD              ; 99 decimal
    BCS     .inc_done                   ; already at max
    INC     EVENT_THRESHOLD
.inc_done:
    RTS
```

Defines:
DEC_EVENT_THRESHOLD, never used.

DEC_THRESHOLD_WRAP, used in chunk 271.
INC_EVENT_THRESHOLD, never used.
INC_THRESHOLD_WRAP, used in chunk 271.
SETTINGS_DISPLAY_INIT, used in chunk 270a.
TOGGLE_SOUND, never used.

Uses DECIMAL_OUT_PTR 630a, EVENT_THRESHOLD 207, LDY_OPCODE 685, MAX_THRESHOLD 685, MENU_GAME_SETTINGS 270a, NUM_TO_DECIMAL 632, NUM_VALUE 630a, PLAY_HIGH_TONE 647a, PLAY_LOW_TONE 648, RTS_OPCODE 685, S_DANGER_OUT 685, S_SOUND_CHAR1 685, and S_SOUND_CHAR2 685.

Chapter 20

Resident Routines

EALDR copies 768 bytes from TUNE_3_ATTRACT–$A5FF to RESET_VECTOR–$07FF before entering the main program. This region contains a second bytecode interpreter, separate from the EALDR VM, that runs during gameplay and the attract screen. It uses the same XOR encryption scheme as the EALDR VM (low byte XOR $03, high byte XOR $D9).

Disk origin. The source data at TUNE_3_ATTRACT–$A5FF is part of the EALDR image that BOOT1 loads from tracks 1–2 into $A000–$BFFF. Within track 1, the three pages map to physical sectors via the custom loader interleave:

Page	Physical sector	DOS sector	File offset
$A3	6	12	$1C00
$A4	8	11	$1B00
$A5	10	10	$1A00

After GAME_INIT, the original TUNE_3_ATTRACT–$A5FF is overwritten with HRCG/font data. From that point on, only the copy at RESET_VECTOR–$07FF survives. The tune pointer table (§18.1) references the original TUNE_3_ATTRACT-based addresses, which is safe because tunes 3–7 run only during attract mode, before GAME_INIT.

The reset vector points here. Simply jumps to RES_VM_LOOP.

689 ⟨*reset vector* 689⟩≡ (909)

```
        ORG     $0500
  RESET_VECTOR:
```

```
        JMP     RES_VM_LOOP
```

Defines:
RESET_VECTOR, never used.
Uses RES_VM_LOOP 221.

READ_DATA_FIELD is a copy-protection signature check, not a normal sector read. It searches for a D5 AA AD data field prologue, then reads all 342 bytes (86 + 256, matching the standard 6&2-encoded data field size) and verifies that *every* byte equals $AF. No data is stored in memory—the bytes are compared and discarded. After the data field, the DE AA epilogue is verified. Returns with carry clear on success, carry set on error.

The entry at READ_DATA_FIELD_AD ($051E) skips the D5 AA search and checks only for the AD byte, used when the address field has already been located.

During reading, BIT \$C000 polls the keyboard—if a key is pressed, the read is aborted and the last byte is stored at RES_ABORT_NIBBLE.

The 30 bytes at $07E2--$07FF are runtime storage locations used by the resident copy-protection verifier and disk I/O routines:

- RES_ERROR_COUNT ($07E2, 2 bytes): error accumulator for the copy-protection verifier (byte 0 is the counter; byte 1 is read as the return status).
- RES_ZP_SHADOW ($07E5, 16 bytes): shadow area for ZP $50–$5F, used by CONTEXT_SWAP to save/restore the caller's zero-page data.
- RES_ABORT_NIBBLE ($07F6, 1 byte): last nibble stored when a disk read is aborted by a keypress.

The 9 bytes at RESIDUAL_E_CHECKER–$07FF are residual data from the EALDR copy, decoding as the low-ASCII fragment "e checker"—see the Dead Code chapter.

691 ⟨*resident page tail* 691⟩≡ (909)

```
    ORG     $07E2
RES_ERROR_COUNT:
    DC.B    $00                     ; error accumulator (1 byte)
    DC.B    $00                     ; error status (read via RES_ERROR_COUNT+1)
    DC.B    $00                     ; padding ($07E4)
RES_ZP_SHADOW:
    HEX     0e968c41964120d3        ; shadow for ZP $50-$57 (8 bytes)
    HEX     d3d3d3d3d3d3d3d3        ; shadow for ZP $58-$5F (8 bytes)
    DC.B    $D3                     ; padding ($07F5)
RES_ABORT_NIBBLE:
    DC.B    $00                     ; last nibble on keyboard abort ($07F6)
```

Defines:
RES_ABORT_NIBBLE, used in chunk 692.
RES_ERROR_COUNT, used in chunk 696.
RES_ZP_SHADOW, used in chunk 700.

692 ⟨*read data field* 692⟩≡ (909)

```
    ORG     $0503
READ_DATA_FIELD:
    SUBROUTINE READ_DATA_FIELD
    ;
    ; Copy-protection signature check.  Searches for the D5 AA AD
    ; data field prologue, verifies all 342 bytes equal $AF, then
    ; checks the DE AA epilogue.  No data is stored in memory.
    ;
    ; A standard Apple II 6&2-encoded data field contains 342 bytes:
    ; 86 (auxiliary buffer) + 256 (main data).  A legitimate
    ; copy-protected sector is written with all $AF bytes; this
    ; routine detects tampered or missing sectors.
    ;
    ; Entry points:
    ;   READ_DATA_FIELD     | search for D5 AA AD, then verify
    ;   READ_DATA_FIELD_AD  | assume D5 AA already found, check AD
    ;
    ; Output:  C clear = success, C set = error or abort
    ; Side effect: on keyboard abort, stores the last byte at
    ;              RES_ABORT_NIBBLE ($07F6)
    ; Clobbers: A, Y
    ;
    ; The bytes at .error ($0560: SEC) and .abort+3 ($0567: SEC) are
    ; integrity-checked by the VM bytecodes via VMLD + VMSUBI $38.
    ;
    ; --- Search for D5 AA AD data field prologue ---
    ;
    ; Reads bytes from DISK_DATA_REG until D5 AA AD appears, or
    ; Y retries are exhausted.  The EOR #$D5 / BNE pattern handles
    ; the common case where a non-D5 byte is read; if the byte after
    ; D5 is not AA, .recheck re-tests it as a potential D5 start.
    ;
    LDY     #$20              ; 32 retries (outer loop)
.search_d5:
    DEY
    BEQ     .error            ; all retries exhausted -> error
    LDA     DISK_DATA_REG     ; \ read byte from drive
    BPL     *-3               ; / (wait for data ready: bit 7 set)
.recheck:
    EOR     #$D5              ; is it D5?
    BNE     .search_d5        ; no -> decrement retry, read next
    ;                         ; yes -> found D5, check for AA
    LDA     DISK_DATA_REG     ; \ read next byte
    BPL     *-3               ; /
    CMP     #$AA              ; AA?
    BNE     .recheck          ; no -> re-test this byte as new D5
    ;
    ; --- Alternate entry: D5 AA already matched, check AD ---
    ;
```

```
READ_DATA_FIELD_AD:
    LDA     DISK_DATA_REG     ; \ read next byte
    BPL     *-3               ; /
    CMP     #$AD              ; AD? (data field marker)
    BNE     .recheck          ; no -> re-test as potential D5
    ;
    ; --- Prologue matched.  Verify 342 data bytes = $AF ---
    ;
    ; Phase 1: 86 bytes (6&2 auxiliary buffer portion)
    ;
    PHA                       ; \ timing padding (2+4 cycles) to
    PLA                       ; / stay synchronized with disk rotation
    LDY     #$56              ; 86 ($56) bytes
.read_86:
    LDA     DISK_DATA_REG     ; \ read byte
    BPL     *-3               ; /
    BIT     $C000             ; keyboard pressed? (bit 7 -> N flag)
    CMP     #$AF              ; is it the $AF signature byte?
    BNE     .abort            ; mismatch -> abort
    DEY
    BNE     .read_86
    ;
    ; Phase 2: 256 bytes (6&2 main data portion)
    ;
    LDY     #$00              ; 256 ($00 wraps) bytes
.read_256:
    LDA     DISK_DATA_REG     ; \ read byte
    BPL     *-3               ; /
    BIT     $C000             ; keyboard pressed?
    CMP     #$AF              ; signature byte?
    BNE     .abort            ; mismatch -> abort
    DEY
    BNE     .read_256
    ;
    ; --- Verify DE AA epilogue ---
    ;
    LDY     DISK_DATA_REG     ; \ read epilogue byte 1 (discarded;
    BPL     *-3               ; / LDY used for timing vs LDA)
    PHA                       ; \ timing padding
    PLA                       ; /
    LDA     DISK_DATA_REG     ; \ read epilogue byte 2
    BPL     *-3               ; /
    CMP     #$DE              ; DE?
    BNE     .error            ; no -> error
    LDA     DISK_DATA_REG     ; \ read epilogue byte 3
    BPL     *-3               ; /
    CMP     #$AA              ; AA?
    BEQ     .ok               ; yes -> success
    ;                         ; no -> fall through to .error
    ;
```

```
    ; --- Exit paths ---
    ;
RDF_VERIFY_SEC1:            ; $0560 | integrity-checked by VM (expect $38=SEC)
.error:
    SEC                     ; C=1: error
    RTS
.ok:
    CLC                     ; C=0: success
    RTS
.abort:
    STA     RES_ABORT_NIBBLE ; save the mismatched byte
RDF_VERIFY_SEC2:            ; $0567 | integrity-checked by VM (expect $38=SEC)
    SEC                     ; C=1: aborted
    RTS
```

Defines:
RDF_VERIFY_SEC1, used in chunk 696.
RDF_VERIFY_SEC2, used in chunk 696.
READ_DATA_FIELD, used in chunk 696.
READ_DATA_FIELD_AD, never used.
Uses DISK_DATA_REG 772 and RES_ABORT_NIBBLE 691.

This 321-byte region between RES_VM_LOOP and CONTEXT_SWAP contains EALDR VM bytecodes interleaved with native 6502 routines. The bytecodes drive the attract-screen display and the copy-protection verifier that runs before the game starts.

Embedded within the bytecodes are several native routines called via VM CALLO opcodes:

- **SEARCH_ADDR_FIELD** ($05FD–$0649): searches for a disk address field (D5 AA 96 prologue) and decodes 4&4 nibble data. Self-modifying code at RES_RETRY_COUNT stores the retry count.
- **DELAY_LOOP** ($0681–$068B): nested countdown delay for motor spin-up timing.
- **READ_AND_VERIFY** ($064B–$0667): calls SEARCH_ADDR_FIELD_RES, then READ_DATA_FIELD to verify the all-$AF signature, accumulating errors. Resets all stepper phases after each read.

Three short native utility routines at the end ($06A6–$06B2) provide VM-to-native exit points and bytecode decryption:

- $06A6: swap context and return carry clear (OK).
- $06AB: swap context and return carry set (error).
- $06B0: decrypt high byte (EOR #$D9).

The address at $0678 doubles as RWTS scratch storage during disk operations.

695 ⟨*zero page defines* 25a⟩+≡ (909) ◁676

```
VM_SCRATCH      EQU     $50     ; decoded nibble / scratch (base of VM ZP range $50-$5F)
VM_IP           EQU     $52     ; bytecode instruction pointer (2 bytes, lo/hi)
VM_OPERAND      EQU     $54     ; decoded address/operand (2 bytes, lo/hi)
VM_ACC          EQU     $56     ; VM accumulator
```

Defines:
VM_ACC, used in chunks 700, 707, 709, 710, 712, and 713a.
VM_IP, used in chunks 700, 702, 704, 706, 708, 709c, 711, and 712b.
VM_OPERAND, used in chunks 700, 702, 704, and 706–13.
VM_SCRATCH, used in chunks 696 and 700.

696 ⟨*resident data and bytecodes* 696⟩≡ (909)

```
    ORG     $0572
RESIDENT_BYTECODES:
    SUBROUTINE RESIDENT_BYTECODES
    ; --- Copy-protection verifier bytecodes ($0572-$05FD) ---
    ; Checks integrity of three disk addresses, then reads sectors
    ; with half-track seeks.  On failure, falls through to error display.
    ;
    ; Phase 1: verify three addresses match expected values
    VMLD    RDF_VERIFY_SEC1   ; check addr byte 1 (expect $38 = SEC)
    VMSUBI  $38
    VMBEQ   VM_RES_ERROR      ; mismatch -> error
    VMLD    RDF_VERIFY_SEC1+1 ; check addr byte 2 (expect $60 = RTS)
    VMSUBI  $60
    VMBEQ   VM_RES_ERROR
    VMLD    RDF_VERIFY_SEC2   ; check addr byte 3 (expect $38 = SEC)
    VMSUBI  $38
    VMBEQ   VM_RES_ERROR
    ;
    ; Phase 2: two half-track read attempts
    VMCALL  .vm_read_tracks
    VMBEQ   .vm_pass1_ok
    VMGOTO  .vm_done          ; read failed -> exit ok (cracked)
.vm_pass1_ok:
    VMCALL  .vm_read_tracks
    VMBEQ   .vm_pass2_ok
    VMGOTO  .vm_done
.vm_pass2_ok:
    VMSUBI  $02               ; check error count
    VMBEQ   VM_RES_ERROR      ; too many errors -> fail
    VMGOTO  RESIDENT_BYTECODES ; retry from start
.vm_done:
    VMCALL0 VM_EXIT_OK        ; exit via VM_EXIT_OK

    ; --- VM subroutine: read tracks with half-track seeks ---
    ; The Disk II stepper motor has 4 phases; each phase activation
    ; moves the head one half-track.  .vm_read_one toggles phases
    ; (PH3->PH0->PH3->PH2) to read at 4 different half-track
    ; positions, verifying the $AF signature at each.
.vm_read_tracks:
    VMLD    IWM_MOTORON       ; motor on
    VMLDI   $FF               ; \ delay for motor spin-up
    VMCALL1 ROM_WAIT          ;  | (ROM WAIT routine)
    VMLDI   $FF               ;  |
    VMCALL1 ROM_WAIT          ; /
    VMLD    IWM_READMODE      ; set drive to read mode
    VMLDI   $00
    VMST    RES_ERROR_COUNT   ; clear error counter
    VMCALL  .vm_read_one      ; \ read 4 sectors at
    VMCALL  .vm_read_one      ;  | different half-track
```

```
    VMCALL  .vm_read_one      ;  | positions
    VMCALL  .vm_read_one      ; /
    VMLD    IWM_MOTOROFF      ; motor off
    VMLD    RES_ERROR_COUNT+1 ; return error status
    VMRET

    ; --- VM subroutine: read one sector + seek half track ---
.vm_read_one:
    VMLDI   $03
    VMST    RES_RETRY_COUNT ; 3 retries
    VMCALL1 READ_AND_VERIFY_RES ; read sector
    VMLD    IWM_PH3_ON        ; \ seek: toggle phases
    VMCALL1 READ_AND_VERIFY_RES ;  | for half-track
    VMCALL1 READ_AND_VERIFY_RES ; /
    VMLD    IWM_PH0_ON        ; phase 0 on
    VMCALL  .vm_seek_delay
    VMLDI   $03
    VMST    RES_RETRY_COUNT
    VMCALL1 READ_AND_VERIFY_RES
    VMLD    IWM_PH3_ON        ; \ more phase toggling
    VMCALL1 READ_AND_VERIFY_RES ;  |
    VMCALL1 READ_AND_VERIFY_RES ; /
    VMLD    IWM_PH2_ON        ; phase 2 on
    VMCALL  .vm_seek_delay
    VMRET

    ; --- Native: search for D5 AA 96 address field ---
    ; After the address field, decodes two 4&4 encoded
    ; bytes, storing the second one in VM_SCRATCH.
SEARCH_ADDR_FIELD_RES:
    LDY     #$FF              ; outer retry counter
.search_d5:
    LDX     RES_RETRY_COUNT ; inner retry counter
.wait_d5:
    LDA     DISK_DATA_REG     ; \ wait for disk byte
    BPL     .wait_d5          ; /
    CMP     #$D5              ; D5 prologue?
    BEQ     .found_d5
    DEY
    BNE     .wait_d5
    DEX
    BNE     .wait_d5
    RTS                       ; timeout (carry undefined)
.found_d5:
    LDA     DISK_DATA_REG     ; \ wait for next byte
    BPL     *-3               ; /
    CMP     #$AA              ; AA?
    BEQ     .found_aa
    DEY
    BNE     .wait_d5          ; retry
```

```
    SEC
    RTS                         ; timeout
.found_aa:
    LDA     DISK_DATA_REG       ; \ wait for next byte
    BPL     *-3                 ; /
    CMP     #$96                ; 96? (address field marker)
    BEQ     .found_96
    DEY
    BNE     .wait_d5            ; retry
    SEC
    RTS
.found_96:
    ; --- Decode 4&4 nibble data (2 pairs -> VM_SCRATCH) ---
    LDY     #$02                ; 2 pairs to decode
.decode_pair:
    LDA     DISK_DATA_REG       ; \ wait for even nibble
    BPL     *-3                 ; /
    ROL                         ; shift into position
    STA     VM_SCRATCH          ; save
    LDA     DISK_DATA_REG       ; \ wait for odd nibble
    BPL     *-3                 ; /
    AND     VM_SCRATCH          ; combine
    STA     VM_SCRATCH          ; decoded byte
    DEY
    STX     RES_RETRY_COUNT ; save retry count
    BPL     .decode_pair        ; next pair
    CLC                         ; success
    LDX     #$01
    RTS
RES_RETRY_COUNT:
    DC.B    $03                 ; retry count (self-modified)

    ; --- Native: read and verify one sector ---
READ_AND_VERIFY_RES:
    JSR     SEARCH_ADDR_FIELD_RES
    BCS     .read_done          ; error
    JSR     READ_DATA_FIELD     ; read data field
.read_done:
    LDA     #$00
    ADC     RES_ERROR_COUNT     ; accumulate errors
    STA     RES_ERROR_COUNT
    LDA     IWM_PH0_OFF         ; \ reset drive phases
    LDA     IWM_PH1_OFF         ;  |
    LDA     IWM_PH2_OFF         ;  |
    LDA     IWM_PH3_OFF         ; /
    RTS

    ; --- VM bytecodes: seek delay subroutine ---
.vm_seek_delay:
    VMLDI   $70                 ; delay count
```

```
    VMCALL1 DELAY_RES         ; delay
    VMLD    IWM_PH0_OFF       ; \ toggle phases for
    VMLD    IWM_PH1_OFF       ;  | half-track positioning
    VMLD    IWM_PH2_OFF       ;  |
    VMLD    IWM_PH3_OFF       ; /
    VMLDI   $28               ; shorter delay
    VMCALL1 DELAY_RES
    VMRET

    ; --- Native: delay loop ---
DELAY_RES:
    LDX     #$20              ; outer counter
.delay_inner:
    DEX
    BNE     .delay_inner      ; inner loop
    SEC
    SBC     #$01              ; decrement A
    BNE     DELAY_RES         ; outer loop
    RTS

    ; --- VM bytecodes: error display loop ---
VM_RES_ERROR:
    VMST    TEXT_PAGE1        ; write char to screen ($0400)
    VMLDX   $068E             ; load from self-ref byte (VMST operand low)
    VMBEQ   VM_RES_ERROR      ; zero -> loop
    VMLD    $068F             ; load self-ref byte (VMLDX opcode)
    VMCALL1 DECRYPT_HIGH_BYTE ; decrypt high byte
    VMSUBI  $FF               ; adjust
    VMCALL1 DECRYPT_HIGH_BYTE ; decrypt again
    VMST    $068F             ; store back (advance screen pointer)
    VMGOTO  VM_RES_ERROR      ; infinite error loop
;
; $06A6-$06B2: Native utility routines
VM_EXIT_OK:
    JSR     CONTEXT_SWAP                ; swap back to main ZP
    CLC                                 ; carry clear = success
    RTS
VM_EXIT_ERR:
    JSR     CONTEXT_SWAP                ; swap back to main ZP
    SEC                                 ; carry set = error
    RTS
DECRYPT_HIGH_BYTE:
    EOR     #$D9                        ; decrypt high byte of address
    RTS
```

Defines:
DECRYPT_HIGH_BYTE, never used.
RESIDENT_BYTECODES, used in chunks 221 and 702.
VM_EXIT_ERR, never used.
VM_EXIT_OK, used in chunks 221 and 700.
Uses CONTEXT_SWAP 700, DISK_DATA_REG 772, IWM_MOTOROFF 772, IWM_MOTORON 772,

IWM_PH0_OFF 772, IWM_PH0_ON 772, IWM_PH1_OFF 772, IWM_PH2_OFF 772, IWM_PH2_ON 772, IWM_PH3_OFF 772, IWM_PH3_ON 772, IWM_READMODE 772, RDF_VERIFY_SEC1 692, RDF_VERIFY_SEC2 692, READ_DATA_FIELD 692, RES_ERROR_COUNT 691, ROM_WAIT 606b, TEXT_PAGE1 772, and VM_SCRATCH 695.

CONTEXT_SWAP saves and restores zero-page $50–$5F so the VM can use those addresses for its own registers without clobbering the caller's data. Before the VM runs, CONTEXT_SWAP saves the caller's $50–$5F into RES_ZP_SHADOW and loads the VM's saved state into ZP. When the VM exits, VM_EXIT_OK/VM_EXIT_ERR calls CONTEXT_SWAP again, saving the VM state back to the shadow area and restoring the caller's original values.

700 ⟨*context swap* 700⟩≡ (909)

```
    ORG     $06E0
CONTEXT_SWAP:
    SUBROUTINE CONTEXT_SWAP
    ;
    ; Saves the caller's ZP $50-$5F into the shadow area at
    ; RES_ZP_SHADOW ($07E5-$07F4) and loads the VM's saved state
    ; from the shadow into ZP.  This lets the VM use $50-$5F for
    ; its registers without clobbering the caller's data:
    ;
    ;   $50 (VM_SCRATCH)     | decoded nibble / general scratch
    ;   $51                  | additional VM scratch
    ;   $52/$53 (VM_IP)      | bytecode instruction pointer
    ;   $54/$55 (VM_OPERAND) | decoded address operand
    ;   $56 (VM_ACC)         | VM accumulator
    ;   $57-$5F              | additional VM scratch
    ;
    ; Called twice per VM execution:
    ;   1. Before RESIDENT_DISPATCH | saves caller's ZP, loads VM state
    ;   2. Inside VM_EXIT_OK/ERR   | saves VM state, restores caller's ZP
    ;
    ; On exit: X = $FF (loop underflows past 0), Y unchanged.
    ;
    LDX     #$0F                    ; 16 bytes ($50-$5F)
.loop:
    LDA     VM_SCRATCH,X            ; A = ZP[$50+X]
    PHA                             ; save old ZP byte on stack
    LDA     RES_ZP_SHADOW,X         ; A = shadow[$07E5+X]
    STA     VM_SCRATCH,X            ; ZP[$50+X] = shadow value
    PLA                             ; A = old ZP byte
    STA     RES_ZP_SHADOW,X         ; shadow[$07E5+X] = old ZP value
    DEX                             ; X-- ($0F -> $00 -> $FF)
    BPL     .loop                   ; loop while X >= 0
    RTS
```

Defines:
CONTEXT_SWAP, used in chunks 221 and 696.
Uses RESIDENT_DISPATCH 702, RES_ZP_SHADOW 691, VM_ACC 695, VM_EXIT_OK 696, VM_IP 695, VM_OPERAND 695, and VM_SCRATCH 695.

The 13-byte table maps opcode indices to handler addresses in page $07:

701 ⟨*resident dispatch table* 701⟩≡ (909)

```
    ORG     $06D3
RESIDENT_DISPATCH_TABLE:
    DC.B    #<RES_VM_DISPATCH ; $00: VMGOTO
    DC.B    #<RES_VM_CALL1            ; $01: VMCALL1
    DC.B    #<RES_VM_VMBEQ            ; $02: VMBEQ
    DC.B    #<RES_VM_VMLDI            ; $03: VMLDI
    DC.B    #<RES_VM_VMLD             ; $04: VMLD
    DC.B    #<RES_VM_VMCALL           ; $05: VMCALL
    DC.B    #<RES_VM_VMST             ; $06: VMST
    DC.B    #<RES_VM_VMSUBI           ; $07: VMSUBI
    DC.B    #<RES_VM_CALL0            ; $08: VMCALL0
    DC.B    #<RES_VM_VMRET            ; $09: VMRET
    DC.B    #<RES_VM_VMINC            ; $0A: VMINC
    DC.B    #<RES_VM_VMYIELD            ; $0B: VMYIELD (no-op RTS)
    DC.B    #<RES_VM_VMADD            ; $0C: VMADD
```

Defines:
RESIDENT_DISPATCH_TABLE, used in chunk 702.

Uses RES_VM_CALL0 709a, RES_VM_CALL1 707, RES_VM_DISPATCH 704, RES_VM_VMADD 710b, RES_VM_VMBEQ 709b, RES_VM_VMCALL 711a, RES_VM_VMINC 713a, RES_VM_VMLD 710a, RES_VM_VMLDI 709c, RES_VM_VMRET 711b, RES_VM_VMST 712a, RES_VM_VMSUBI 712b, and RES_VM_VMYIELD 706.

Runs the entire bytecode program starting at RESIDENT_BYTECODES. The VM IP is derived from the caller's return address via a stack trick; the opcode handlers chain to each other through RESIDENT_DISPATCH_FETCH and never return here until the program exits.

The stack trick works as follows:

1. Push X and Y onto the stack as **padding**—their values are irrelevant and are immediately discarded.
2. JSR PULL_TWO_RET_ADDRS—pulls four stack entries: its own return address → VM_OPERAND, then the X/Y padding → VM_IP (discarded). Jumps to \texttt{.resume} via JMP (VM_OPERAND).
3. At \texttt{.resume}: pulls the *caller's* return address (from the outer JSR RESIDENT_DISPATCH) into VM_IP, overwriting the discarded X/Y values.
4. LDY \#\$04 skips 4 bytes past the return address.

Since JSR RESIDENT_DISPATCH is at $056C, the return address on the stack is $056E. Adding 4 gives $0572 = RESIDENT_BYTECODES—the first VM instruction. The "JMP ATTRACT_LOOP" at $056F and the bytecodes' ORG at RESIDENT_BYTECODES are placed so that the IP lands exactly at the start of the bytecode program.

The VM always starts from $0572. Each call to RESIDENT_DISPATCH rederives the IP from the return address; there is no mechanism to resume a partially-executed program.

702 ⟨*resident dispatch* 702⟩≡ (909)

```
    ORG     $06B3
RESIDENT_DISPATCH:
    SUBROUTINE RESIDENT_DISPATCH
    ;
    ; Called from RES_VM_LOOP via JSR RESIDENT_DISPATCH ($056C).
    ; On entry, the 6502 stack contains (top shown right):
    ;
    ;   ... | $05 $6E |
    ;           ^^^^^^^
    ;   return addr from JSR RESIDENT_DISPATCH (= $056F - 1)
    ;
    TXA                             ; \  push X and Y as stack padding
    PHA                             ;  | (values are irrelevant;
    TYA                             ;  |  PULL_TWO_RET_ADDRS needs
    PHA                             ; /  4 entries to consume)
    ;
    ; Stack: ... | $05 $6E | X | Y |
```

```
    ;
    JSR     PULL_TWO_RET_ADDRS       ; pushes $06B9 (.resume - 1)
    ;
    ; Stack at entry to PULL_TWO_RET_ADDRS:
    ;   ... | $05 $6E | X | Y | $06 $B9 |
    ;
    ; PULL_TWO_RET_ADDRS pulls all 4 data bytes:
    ;   $06/$B9 -> VM_OPERAND  (incremented to $06BA = .resume)
    ;   Y/X     -> VM_IP       (discarded | overwritten below)
    ; then JMP (VM_OPERAND) jumps here:
    ;
    ; Stack after PULL_TWO_RET_ADDRS:
    ;   ... | $05 $6E |
    ;
.resume:
    PULB    VM_IP                    ; pull $6E -> VM_IP (lo)
    PULB    VM_IP+1                  ; pull $05 -> VM_IP+1 (hi)
    ;                                ; VM_IP = $056E
    ; Stack: ... |  (caller's ret addr consumed)
    ;
    LDY     #$04                     ; Y = 4
    ;                                ; first fetch: (VM_IP),Y = *($056E+4) = *$0572
    ;                                ; $0572 = RESIDENT_BYTECODES
    ;
    ; --- Fetch-decode-dispatch loop (handlers JMP back here) ---
RESIDENT_DISPATCH_FETCH:
.fetch:
    LDA     (VM_IP),Y                ; read opcode byte
    INY                              ; advance Y past opcode
    BNE     .no_carry
    INC     VM_IP+1                  ; handle page crossing
.no_carry:
    TAX                              ; opcode -> X
    LDA     RESIDENT_DISPATCH_TABLE,X ; look up handler low byte
    STA     DECODE_DISPATCH_RELAY+1   ; patch JMP target
    JMP     DECODE_DISPATCH_RELAY     ; dispatch to handler
```

Defines:
RESIDENT_DISPATCH, used in chunks 221 and 700.
Uses DECODE_DISPATCH_RELAY 713b, PULB 9c, PULL_TWO_RET_ADDRS 708, RESIDENT_BYTECODES 696, RESIDENT_DISPATCH_TABLE 701, RES_VM_LOOP 221, VM_IP 695, and VM_OPERAND 695.

Opcode $00 handler—the resident interpreter's GOTO instruction. Decodes an address and redirects execution there.

704 ⟨*resident decode dispatch* 704⟩≡ (909)

```
    ORG     $0700
RES_VM_DISPATCH:
    SUBROUTINE RES_VM_DISPATCH
    ;
    ; Opcode $00 (VMGOTO): unconditional branch.
    ; Reads a 2-byte encrypted address from the bytecode stream,
    ; sets VM_IP to the decoded address, and resumes execution there.
    ;
    ; Also used as a shared entry point: RES_VM_SET_IP is jumped to
    ; by VMCALL ($05) after pushing the return IP onto the stack.
    ;
    JSR     DECODE_ENCRYPTED_ADDR    ; decrypt 2 bytes -> VM_OPERAND
RES_VM_SET_IP:
    MOVW    VM_OPERAND,VM_IP         ; VM_IP = decoded target address
    LDY     #$00                     ; reset Y (new page, offset 0)
    JMP     RESIDENT_DISPATCH_FETCH  ; fetch next opcode from new IP
```

Defines:
RES_VM_DISPATCH, used in chunks 701 and 713b.
RES_VM_SET_IP, used in chunk 711a.
Uses DECODE_ENCRYPTED_ADDR 706, MOVW 9a, VM_IP 695, and VM_OPERAND 695.

Opcode	Mnemonic	Address	Handler label
$00	VMGOTO	$0700	RES_VM_DISPATCH
$01	VMCALL1	$0727	RES_VM_CALL1
$02	VMBEQ	$0755	RES_VM_VMBEQ
$03	VMLDI	$075F	RES_VM_VMLDI
$04	VMLD	$076D	RES_VM_VMLD
$05	VMCALL	$078A	RES_VM_VMCALL
$06	VMST	$07AD	RES_VM_VMST
$07	VMSUBI	$07B9	RES_VM_VMSUBI
$08	VMCALL0	$074F	RES_VM_CALL0
$09	VMRET	$07A2	RES_VM_VMRET
$0A	VMINC	$07CE	RES_VM_VMINC
$0B	VMYIELD	$0726	RES_VM_VMYIELD (no-op RTS, unused)
$0C	VMADD	$0779	RES_VM_VMADD

Decodes a 2-byte encrypted address from the bytecode stream. Uses the same XOR scheme as the EALDR VM (low byte XOR RES_XOR_LO, high byte XOR RES_XOR_HI). The decoded address is left in VM_OPERAND/VM_OPERAND+1.

705 ⟨*defines* 25b⟩+≡ (909) ◁685 772▷

```
RES_XOR_LO      EQU     $03     ; bytecode encryption XOR key (low byte)
RES_XOR_HI      EQU     $D9     ; bytecode encryption XOR key (high byte)
RES_XOR_IMM     EQU     $4C     ; bytecode encryption XOR key (immediate operand)
```

Defines:
RES_XOR_HI, used in chunk 706.
RES_XOR_IMM, used in chunks 709c and 712b.
RES_XOR_LO, used in chunk 706.

706 ⟨*decode encrypted addr* 706⟩≡ (909)

```
    ORG     $0710
DECODE_ENCRYPTED_ADDR:
    SUBROUTINE DECODE_ENCRYPTED_ADDR
    ;
    ; Reads 2 encrypted bytes from the bytecode stream at (VM_IP),Y
    ; and decodes them into a 16-bit address in VM_OPERAND.
    ;
    ; Encryption: low byte XOR $03, high byte XOR $D9.
    ; Y is advanced past both bytes (with page-crossing handled).
    ;
    ; On entry:  (VM_IP),Y points to the encrypted low byte
    ; On exit:   VM_OPERAND = decoded address, Y advanced by 2
    ;
    LDA     (VM_IP),Y               ; read encrypted low byte
    EOR     #RES_XOR_LO             ; XOR $03 -> plaintext low
    INY                              ; advance past low byte
    BNE     .no_carry1
    INC     VM_IP+1                  ; handle Y overflow (page cross)
.no_carry1:
    STA     VM_OPERAND              ; store decoded low byte
    LDA     (VM_IP),Y               ; read encrypted high byte
    INY                              ; advance past high byte
    BNE     .no_carry2
    INC     VM_IP+1                  ; handle Y overflow
.no_carry2:
    EOR     #RES_XOR_HI             ; XOR $D9 -> plaintext high
    STA     VM_OPERAND+1            ; store decoded high byte
RES_VM_VMYIELD:                      ; opcode $0B: VMYIELD (unused, just RTS)
    RTS
```

Defines:
DECODE_ENCRYPTED_ADDR, used in chunks 704, 707, and 709–13.
RES_VM_VMYIELD, used in chunk 701.
Uses RES_XOR_HI 705, RES_XOR_LO 705, VM_IP 695, and VM_OPERAND 695.

The resident EALDR VM opcode handlers. Each reads its arguments via DECODE_ENCRYPTED_ADDR and returns to the dispatch loop at $06C2. VM state: VM_IP/VM_IP+1=IP, VM_OPERAND/VM_OPERAND+1=decoded address, VM_ACC=accumulator.

Note: the VMBEQ handler at $075A contains a BNE +0 (no-op branch)—this is one of the Passport crack patches that disables copy-protection branching.

Opcode $0B (VMYIELD in the resident VM, VMASL in the EALDR VM) reuses the RTS at RES_VM_VMYIELD, the last byte of DECODE_ENCRYPTED_ADDR. The handler is simply RTS—a no-op that returns to the dispatch loop without modifying the accumulator. This opcode is not used by the resident bytecode program.

707 ⟨*res vm call1* 707⟩≡ (909)

```
      ORG     $0727
  RES_VM_CALL1:
      SUBROUTINE RES_VM_CALL1
      JSR     DECODE_ENCRYPTED_ADDR
      TYA                        ; \ save bytecode position
      PHA                        ; /
      LDA     VM_ACC                   ; A = VM accumulator
      JSR     RES_VM_CALL1_IND         ; call via (VM_OPERAND)
      STA     VM_ACC                   ; result -> VM acc
      PLA                        ; \ restore Y
      TAY                        ; /
      JMP     RESIDENT_DISPATCH_FETCH ; dispatch next
```

Defines:
RES_VM_CALL1, used in chunk 701.
Uses DECODE_ENCRYPTED_ADDR 706, RES_VM_CALL1_IND 709a, VM_ACC 695, and VM_OPERAND 695.

PULL_TWO_RET_ADDRS is a stack-manipulation utility used by RESIDENT_DISPATCH. It pulls four bytes from the stack:

1. Its own return address → VM_OPERAND (incremented by 1 to correct the RTS addr−1 convention).
2. The X/Y padding pushed by RESIDENT_DISPATCH → VM_IP (these values are meaningless and are immediately overwritten by \texttt{.resume}).

It then jumps through VM_OPERAND, which points to \texttt{.resume} in RESIDENT_DISPATCH. This leaves the *caller's* return address (from JSR RESIDENT_DISPATCH) as the top of the stack, which \texttt{.resume} pulls into VM_IP to derive the byte-code start address.

708 ⟨*pull two ret addrs* 708⟩≡ (909)

```
      ORG     $0738
  PULL_TWO_RET_ADDRS:
      SUBROUTINE PULL_TWO_RET_ADDRS
      ;
      ; On entry, the 6502 stack is (top shown right):
      ;   ... | $05 $6E | X | Y | $06 $B9 |
      ;          -------   ----   ---------
      ;          caller's  pad    own ret addr
      ;          ret addr        (= .resume - 1)
      ;
      PULB    VM_OPERAND              ; PLA -> $B9 -> VM_OPERAND
      PULB    VM_OPERAND+1            ; PLA -> $06 -> VM_OPERAND+1
      ;                               ; VM_OPERAND = $06B9
      ;
      ; Stack: ... | $05 $6E | X | Y |
      ;
      PULB    VM_IP                   ; PLA -> Y -> VM_IP   (padding; discarded)
      PULB    VM_IP+1                 ; PLA -> X -> VM_IP+1 (padding; discarded)
      ;
      ; Stack: ... | $05 $6E |  (caller's ret addr remains)
      ;
      INC     VM_IP                   ; (meaningless | .resume overwrites VM_IP)
      INC     VM_OPERAND              ; $06B9 + 1 = $06BA = .resume
      BNE     .no_inc
      INC     VM_OPERAND+1
  .no_inc:
      JMP     (VM_OPERAND)            ; JMP .resume ($06BA)
```

Defines:
PULL_TWO_RET_ADDRS, used in chunk 702.
Uses PULB 9c, VM_IP 695, and VM_OPERAND 695.

Opcode $08: VMCALL0 calls native code as a terminal jump. RES_VM_CALL1_IND is shared by RES_VM_CALL1 for indirect calls.

709a ⟨*res vm call0* 709a⟩≡ (909)

```
      ORG     $074F
  RES_VM_CALL0:
      SUBROUTINE RES_VM_CALL0
      JSR     DECODE_ENCRYPTED_ADDR
  RES_VM_CALL1_IND:
      JMP     (VM_OPERAND)            ; indirect jump through VM_OPERAND
```

Defines:
RES_VM_CALL0, used in chunk 701.
RES_VM_CALL1_IND, used in chunk 707.
Uses DECODE_ENCRYPTED_ADDR 706 and VM_OPERAND 695.

Opcode $02: VMBEQ branches if the VM accumulator is zero. The BNE +0 is a Passport crack patch that disables copy-protection branching.

709b ⟨*res vm beq* 709b⟩≡ (909)

```
      ORG     $0755
  RES_VM_VMBEQ:
      JSR     DECODE_ENCRYPTED_ADDR
      LDA     VM_ACC                  ; test VM accumulator
      BNE     *+2                     ; PATCHED: BNE +0 = no-op (was branch to GOTO)
      JMP     RESIDENT_DISPATCH_FETCH ; always: dispatch next (branch disabled)
```

Defines:
RES_VM_VMBEQ, used in chunk 701.
Uses DECODE_ENCRYPTED_ADDR 706 and VM_ACC 695.

Opcode $03: VMLDI loads an immediate byte (decrypted) into the VM accumulator.

709c ⟨*res vm ldi* 709c⟩≡ (909)

```
      ORG     $075F
  RES_VM_VMLDI:
      SUBROUTINE RES_VM_VMLDI
      LDA     (VM_IP),Y               ; read immediate byte
      INY
      BNE     .no_carry1
      INC     VM_IP+1
  .no_carry1:
      EOR     #RES_XOR_IMM            ; decrypt
      STA     VM_ACC                  ; -> VM acc
      JMP     RESIDENT_DISPATCH_FETCH
```

Defines:
RES_VM_VMLDI, used in chunk 701.
Uses RES_XOR_IMM 705, VM_ACC 695, and VM_IP 695.

Opcode $04: VMLD loads a byte from the decoded address into the VM accumulator.

710a ⟨*res vm ld* 710a⟩≡ (909)

```
      ORG     $076D
  RES_VM_VMLD:
      JSR     DECODE_ENCRYPTED_ADDR
      LDX     #$00
      LDA     (VM_OPERAND,X)          ; acc = *(addr)
      STA     VM_ACC
      JMP     RESIDENT_DISPATCH_FETCH
```

Defines:
RES_VM_VMLD, used in chunks 701 and 710b.
Uses DECODE_ENCRYPTED_ADDR 706, VM_ACC 695, and VM_OPERAND 695.

Opcode $0C: VMADD adds the VM accumulator to the decoded address low byte, then loads from the resulting pointer. Falls through to RES_VM_VMLD+3.

710b ⟨*res vm add* 710b⟩≡ (909)

```
      ORG     $0779
  RES_VM_VMADD:
      SUBROUTINE RES_VM_VMADD
      JSR     DECODE_ENCRYPTED_ADDR
      LDA     VM_ACC                  ; acc
      CLC
      ADC     VM_OPERAND              ; + decoded low byte
      STA     VM_OPERAND
      BCC     .no_carry2
      INC     VM_OPERAND+1
  .no_carry2:
      JMP     RES_VM_VMLD+3           ; -> LDX #$00 / LDA (VM_OPERAND,X) / ...
```

Defines:
RES_VM_VMADD, used in chunk 701.
Uses DECODE_ENCRYPTED_ADDR 706, RES_VM_VMLD 710a, VM_ACC 695, and VM_OPERAND 695.

Opcode $05: VMCALL performs a VM subroutine call. It pushes the current IP (adjusted by Y offset) and sets the IP to the decoded address.

711a ⟨*res vm call* 711a⟩≡ (909)

```
      ORG     $078A
  RES_VM_VMCALL:
      SUBROUTINE RES_VM_VMCALL
      JSR     DECODE_ENCRYPTED_ADDR
      TYA                         ; \ compute return address:
      CLC                         ;  | IP + Y offset
      ADC     VM_IP                     ;  |
      STA     VM_IP                     ;  |
      BCC     .no_carry3                ;  |
      INC     VM_IP+1                   ; /
  .no_carry3:
      LDA     VM_IP                     ; \ push return IP
      PHA                               ;  |
      LDA     VM_IP+1                   ;  |
      PHA                         ; /
      LDY     #$00                ; reset Y
      JMP     RES_VM_SET_IP             ; set IP = VM_OPERAND, dispatch
```

Defines:
RES_VM_VMCALL, used in chunk 701.
Uses DECODE_ENCRYPTED_ADDR 706, RES_VM_SET_IP 704, VM_IP 695, and VM_OPERAND 695.

Opcode $09: VMRET returns from a VM subroutine by popping the saved IP.

711b ⟨*res vm ret* 711b⟩≡ (909)

```
      ORG     $07A2
  RES_VM_VMRET:
      PLA                         ; \ pop return IP
      STA     VM_IP+1                   ;  | (high byte first)
      PLA                               ;  |
      STA     VM_IP                     ; /
      LDY     #$00
      JMP     RESIDENT_DISPATCH_FETCH ; dispatch next
```

Defines:
RES_VM_VMRET, used in chunk 701.
Uses VM_IP 695.

Opcode $06: VMST stores the VM accumulator to the decoded memory address.

712a ⟨*res vm st* 712a⟩≡ (909)

```
      ORG     $07AD
  RES_VM_VMST:
      JSR     DECODE_ENCRYPTED_ADDR
      LDA     VM_ACC                  ; VM acc
      LDX     #$00
      STA     (VM_OPERAND,X)          ; *(addr) = acc
      JMP     RESIDENT_DISPATCH_FETCH
```

Defines:
RES_VM_VMST, used in chunk 701.
Uses DECODE_ENCRYPTED_ADDR 706, VM_ACC 695, and VM_OPERAND 695.

Opcode $07: VMSUBI subtracts a decrypted immediate byte from the VM accumulator.

712b ⟨*res vm subi* 712b⟩≡ (909)

```
      ORG     $07B9
  RES_VM_VMSUBI:
      SUBROUTINE RES_VM_VMSUBI
      LDA     (VM_IP),Y               ; read immediate byte
      INY
      BNE     .no_carry4
      INC     VM_IP+1
  .no_carry4:
      EOR     #RES_XOR_IMM            ; decrypt
      STA     VM_OPERAND              ; temp
      LDA     VM_ACC                  ; acc
      SEC
      SBC     VM_OPERAND              ; acc - immediate
      STA     VM_ACC
      JMP     RESIDENT_DISPATCH_FETCH
```

Defines:
RES_VM_VMSUBI, used in chunk 701.
Uses RES_XOR_IMM 705, VM_ACC 695, VM_IP 695, and VM_OPERAND 695.

Opcode $0A: VMINC increments the byte at the decoded address and stores the result in the VM accumulator.

713a ⟨*res vm inc* 713a⟩≡ (909)

```
      ORG     $07CE
  RES_VM_VMINC:
      JSR     DECODE_ENCRYPTED_ADDR
      LDX     #$00
      LDA     (VM_OPERAND,X)          ; load byte
      CLC
      ADC     #$01                    ; increment
      STA     (VM_OPERAND,X)          ; store back
      STA     VM_ACC                  ; -> VM acc
      JMP     RESIDENT_DISPATCH_FETCH
```

Defines:
RES_VM_VMINC, used in chunk 701.
Uses DECODE_ENCRYPTED_ADDR 706, VM_ACC 695, and VM_OPERAND 695.

A 3-byte relay in the handler page. Jumps to RES_VM_DISPATCH.

713b ⟨*decode dispatch relay* 713b⟩≡ (909)

```
      ORG     $07DF
  DECODE_DISPATCH_RELAY:
      JMP     RES_VM_DISPATCH
```

Defines:
DECODE_DISPATCH_RELAY, used in chunk 702.
Uses RES_VM_DISPATCH 704.

The remaining 30 bytes of the handler page ($07E2--$07FF) are

Chapter 21

The HRCG Text Engine

The Hi-Res Character Generator (HRCG) is a library provided by the Applesoft Toolkit which allows a developer to display a custom font. Applesoft Toolkit also contains Animatrix, which is a utility used to create fonts for the HRCG.

The Applesoft Toolkit manual part 1 and manual part 2 can be found on the Apple Asimov Archive.

The HRCG defines 96 characters as a character set. These characters are the "printing" characters, which are ASCII 32 (space) through 127 (delete). Character set 0 is always the standard set of Apple text characters. The font data for this standard set is stored at `STD_FONT_DATA`, consisting of 8 bytes per character. Each byte represents 7 consecutive pixels since the high bit is used for color set selection.

The Ali Baba font data for HRCG is stored at `FONT_DATA` (3840 bytes, 120 characters × 32 bytes each). Each character is four 7×8 blocks (upper-left, upper-right, lower-left, lower-right) forming a 14×16 pixel glyph.

The 120 characters span five overlapping HRCG charsets (see Section **??** for the charset overlap model):

- Characters 0–23 (charset 1): game map entity icons.
- Characters 24–47 (charset 2): border tiles and decorations. Characters 114–119 (charset 2, positions 18–23) form the EA logo displayed on the splash screen.
- Characters 48–119 (charsets 3–5): scene text in an ornate Arabian-style typeface with colored pixels.

Charset 0 (STD_FONT_DATA) provides the standard 7×8 fallback font (characters 32–127).

Corrupted digit glyphs. In the 4am/san inc crack, the font glyphs for digits 2–8 (scene bytes $1D–$23, font characters 77–83) are zeroed out. Digits 0, 1, and 9 have partial data. As a result, numeric text in scenes (e.g. the copyright year "1981" and the phone number in scene 0) renders with missing digits. The original disk image contains intact digit glyphs.

715a ⟨*font data* 715a⟩≡ (909)

```
    ORG     $83A5
FONT_DATA:
    INCLUDE "fontdata.asm"
```

Defines:
FONT_DATA, used in chunk 736.

715b ⟨*std font data* 715b⟩≡ (909)

```
    ORG     $97A5
STD_FONT_DATA:
    INCLUDE "stdfontdata.asm"
```

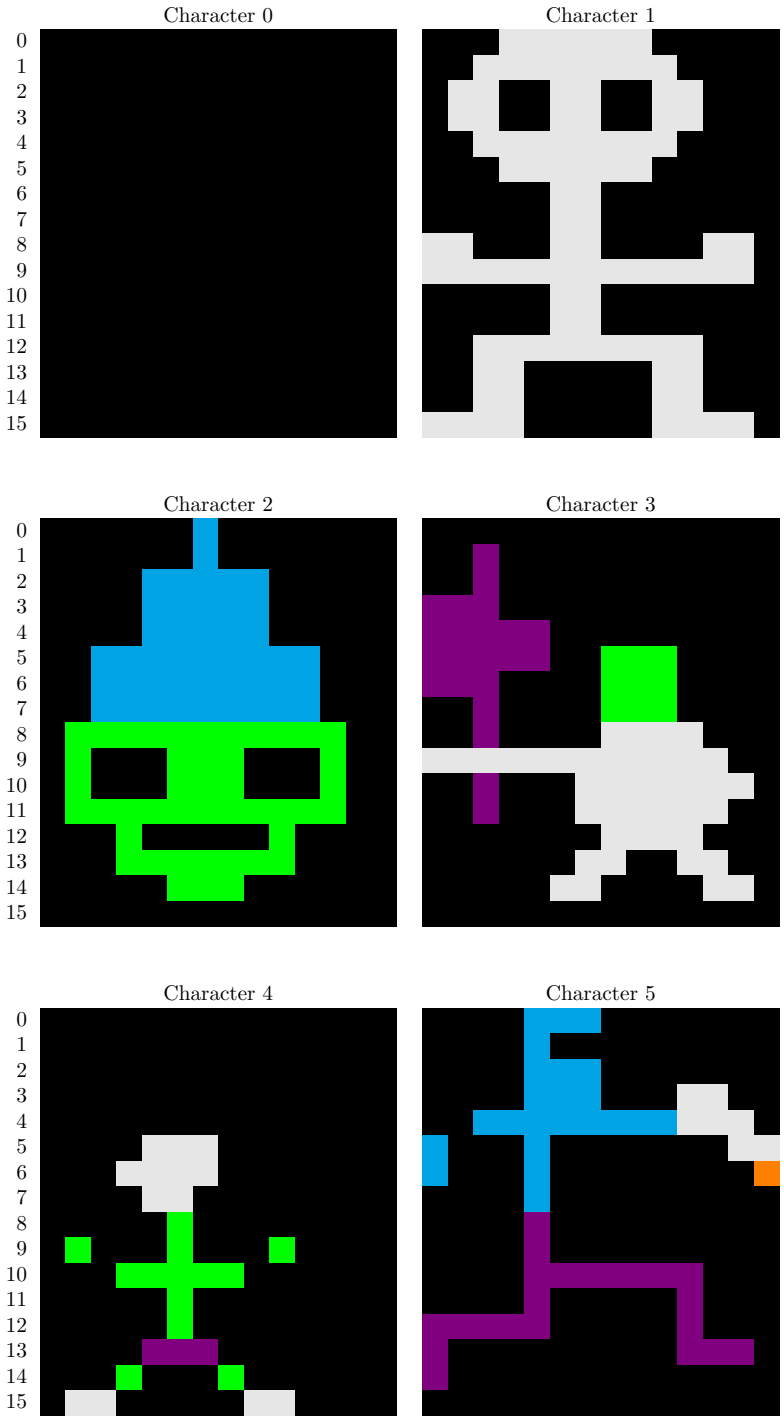
Defines:
STD_FONT_DATA, used in chunk 745.

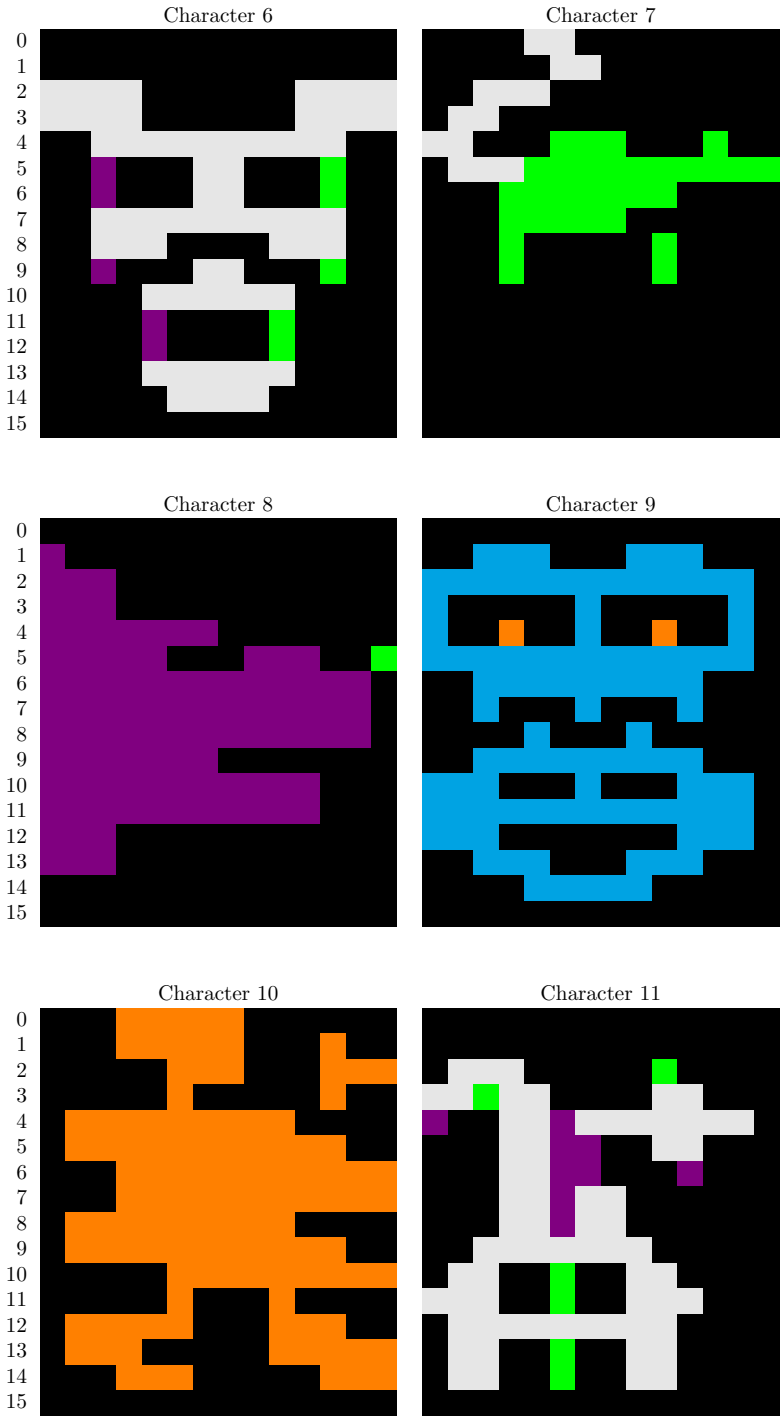
Immediately following the entry table is the HRCG version string. The string is stored in high-bit ASCII (the standard Apple II text encoding) and is prefixed with a ctrl-P character, which triggers the HRCG's "home cursor" command. A carriage return ($8D) terminates the string. HRCG_PRINT_VERSION prints this string to the screen character by character via ROM_COUT1; it is not used during normal gameplay.

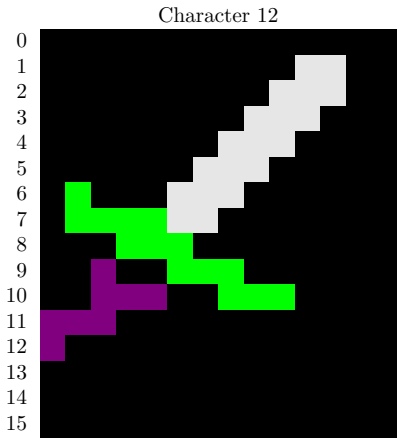
715c ⟨*hrcg version string* 715c⟩≡ (909)

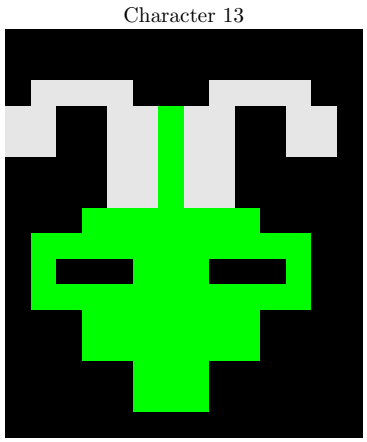
```
    ORG     $92E8
HRCG_VERSION_STR:
; "HI-RES CHAR GEN VERSION 1.0" with ctrl-P prefix and CR suffix (high-bit ASCII)
    HEX     90 C8 C9 AD D2 C5 D3 A0 C3 C8 C1 D2 A0 C7 C5 CE
    HEX     A0 D6 C5 D2 D3 C9 CF CE A0 B1 AE B0 8D
```

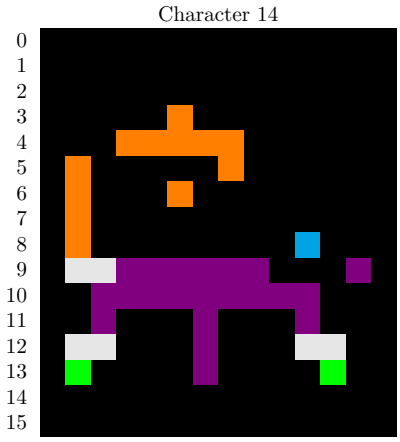
Defines:
HRCG_VERSION_STR, used in chunk 736.

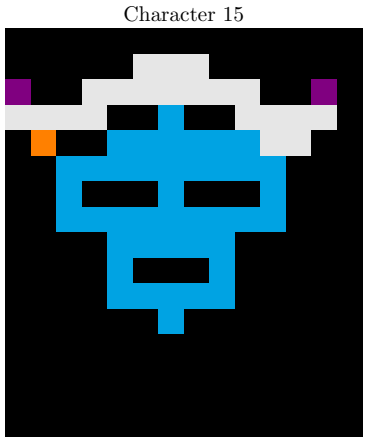

Character 0
Character 1
Character 2
Character 3
Character 4
Character 5
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

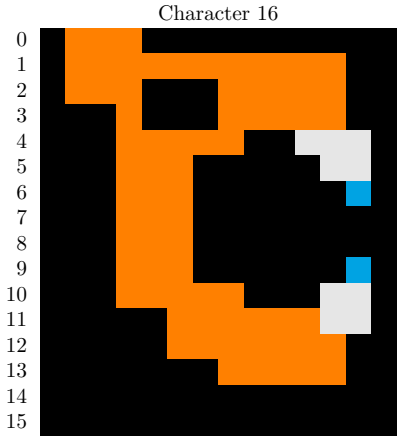

Character 6
Character 7
Character 8
Character 9
Character 10
Character 11
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

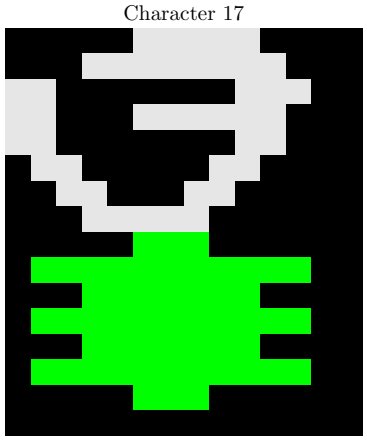

Character 12
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

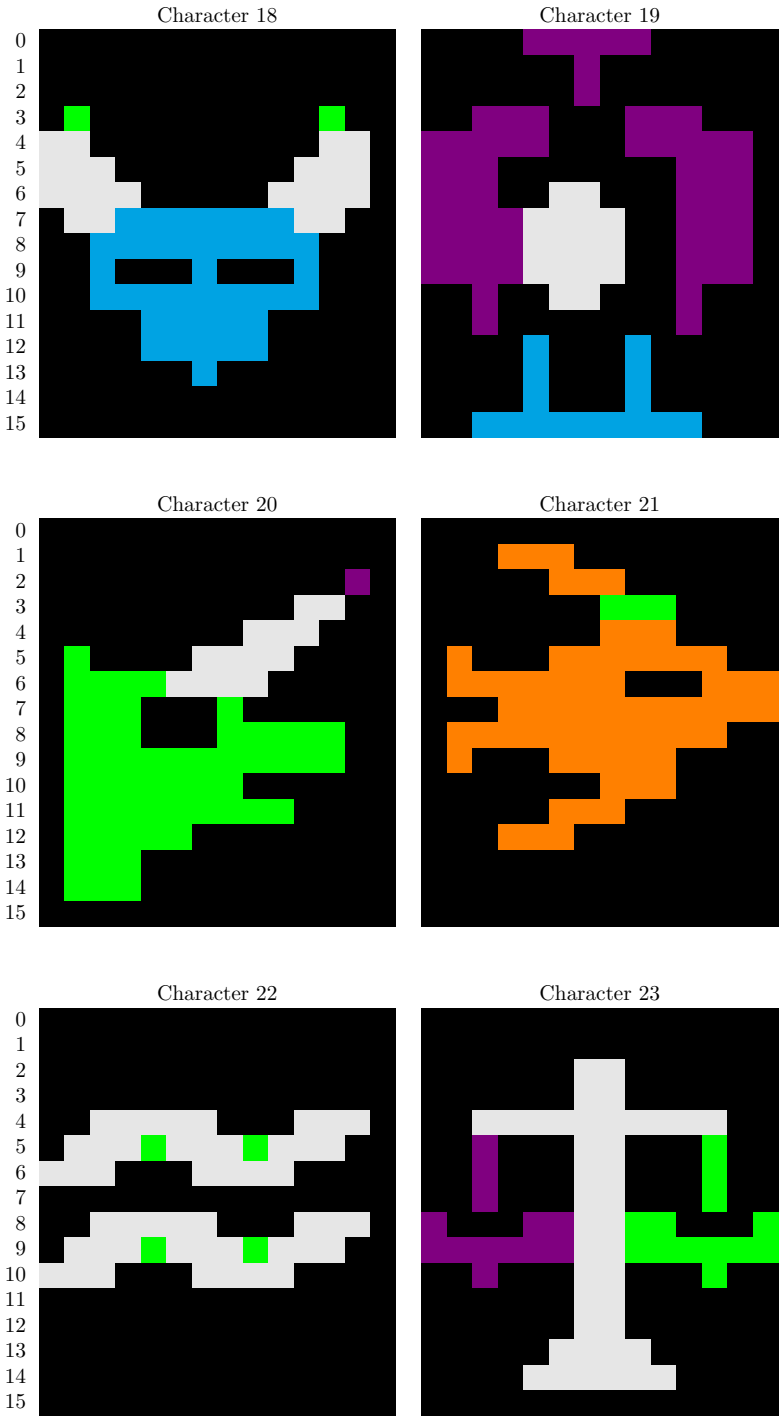

Character 13


Character 14
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

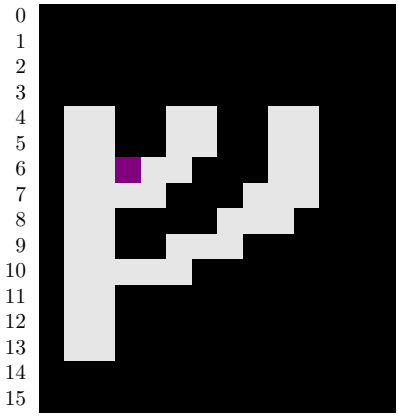

Character 15


Character 16
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

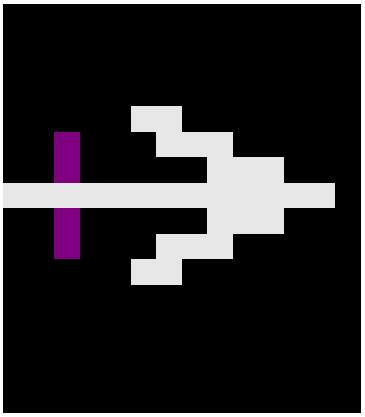

Character 17


Character 18
Character 19
Character 20
Character 21
Character 22
Character 23
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

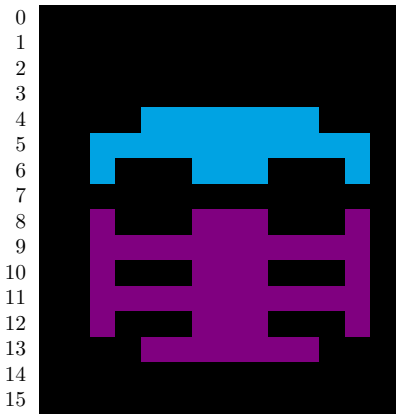
Character 24



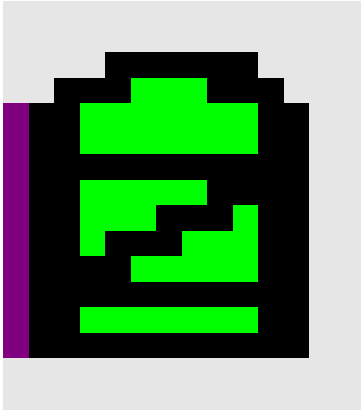
Character 25



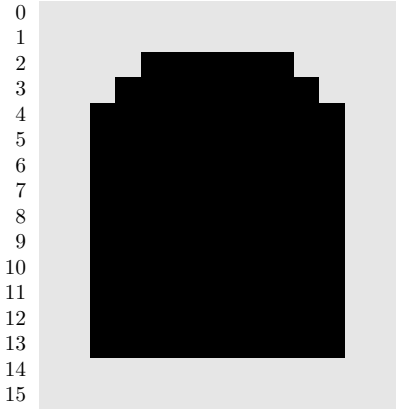
Character 26



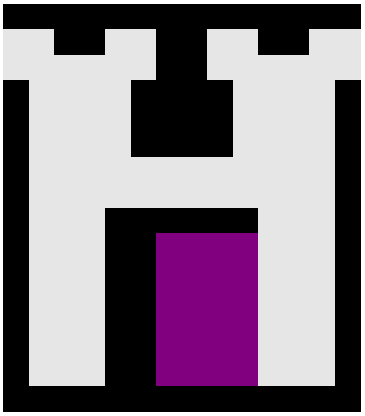
Character 27

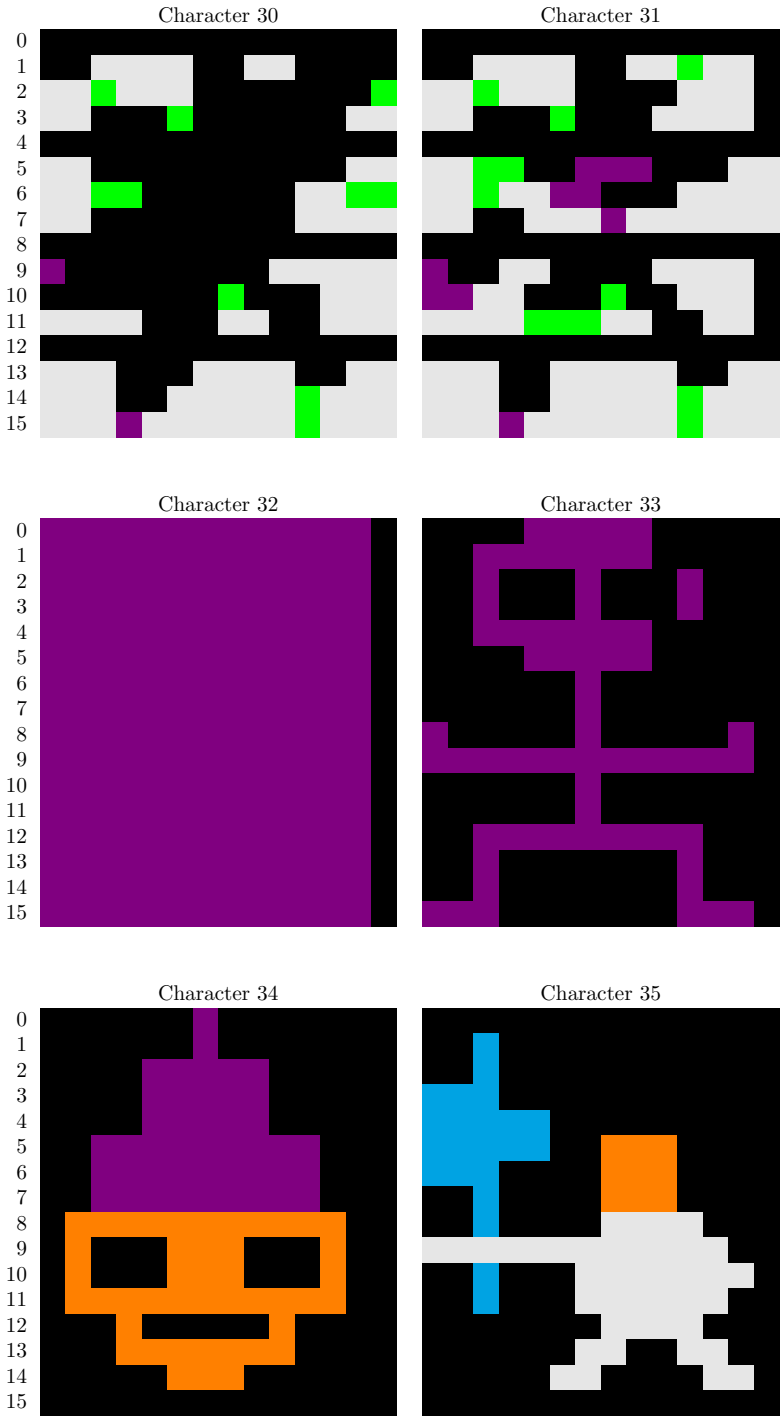


Character 28

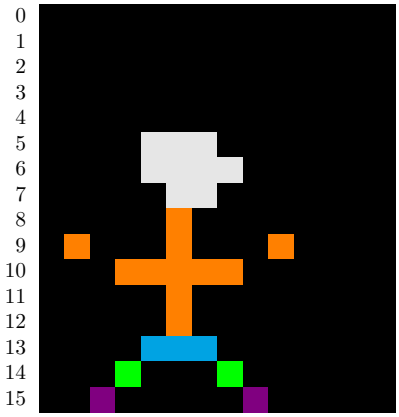


Character 29

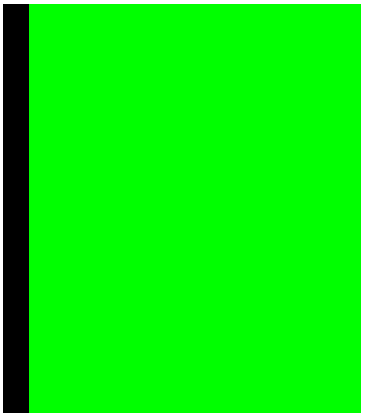



Character 30
Character 31
Character 32
Character 33
Character 34
Character 35
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

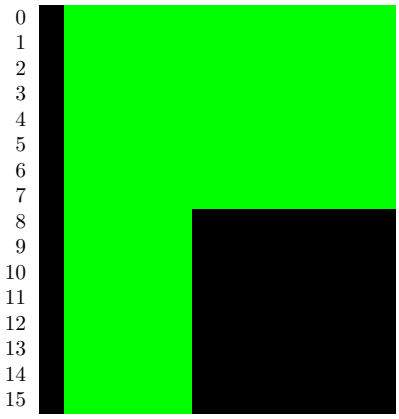
Character 36



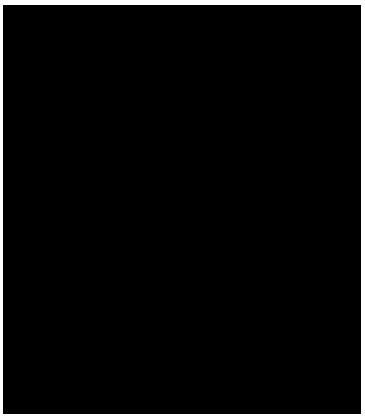
Character 37



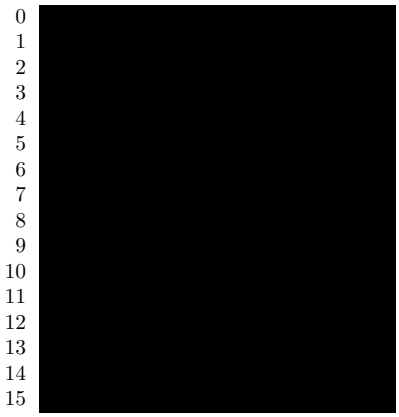
Character 38



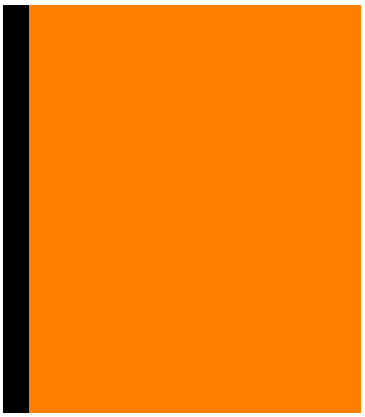
Character 39



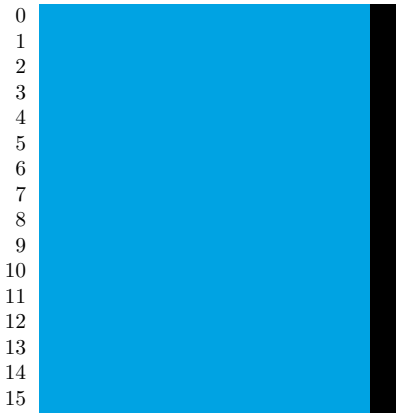
Character 40



Character 41

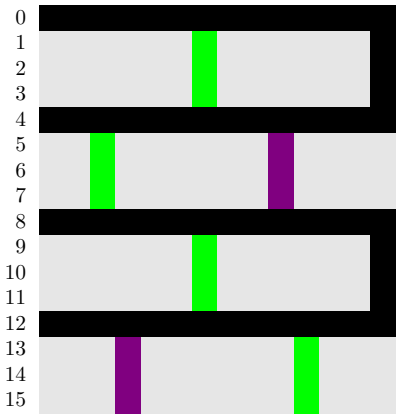


Character 42

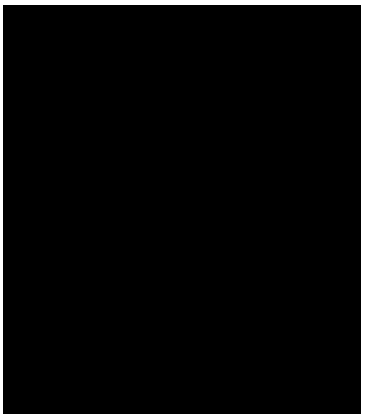


Character 43

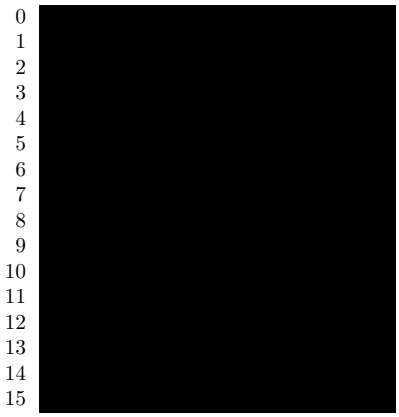
Character 44



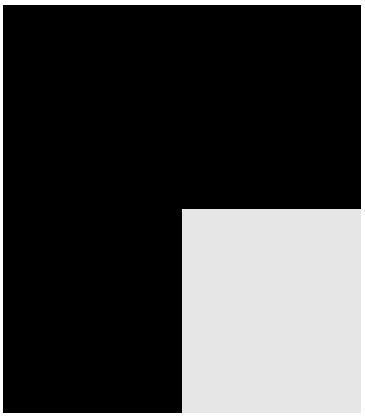
Character 45

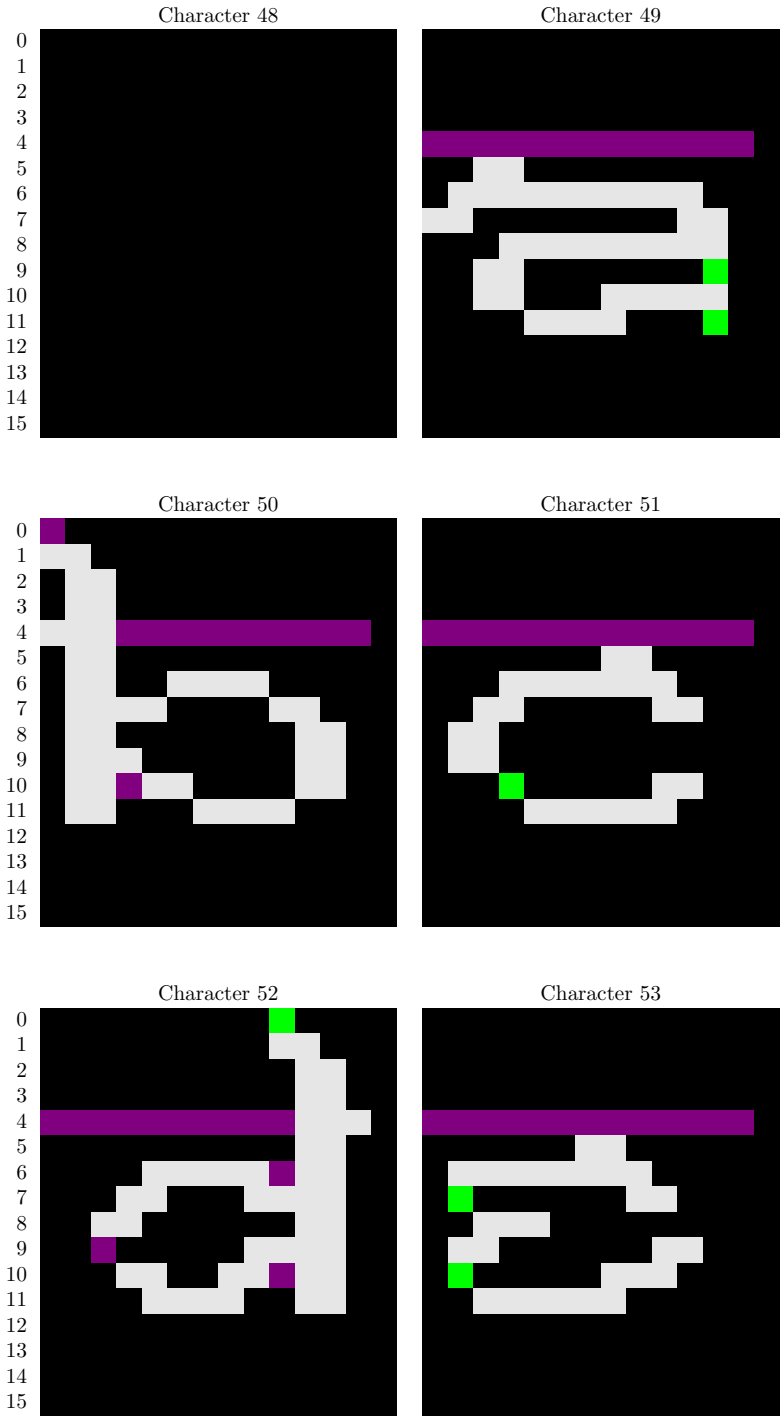


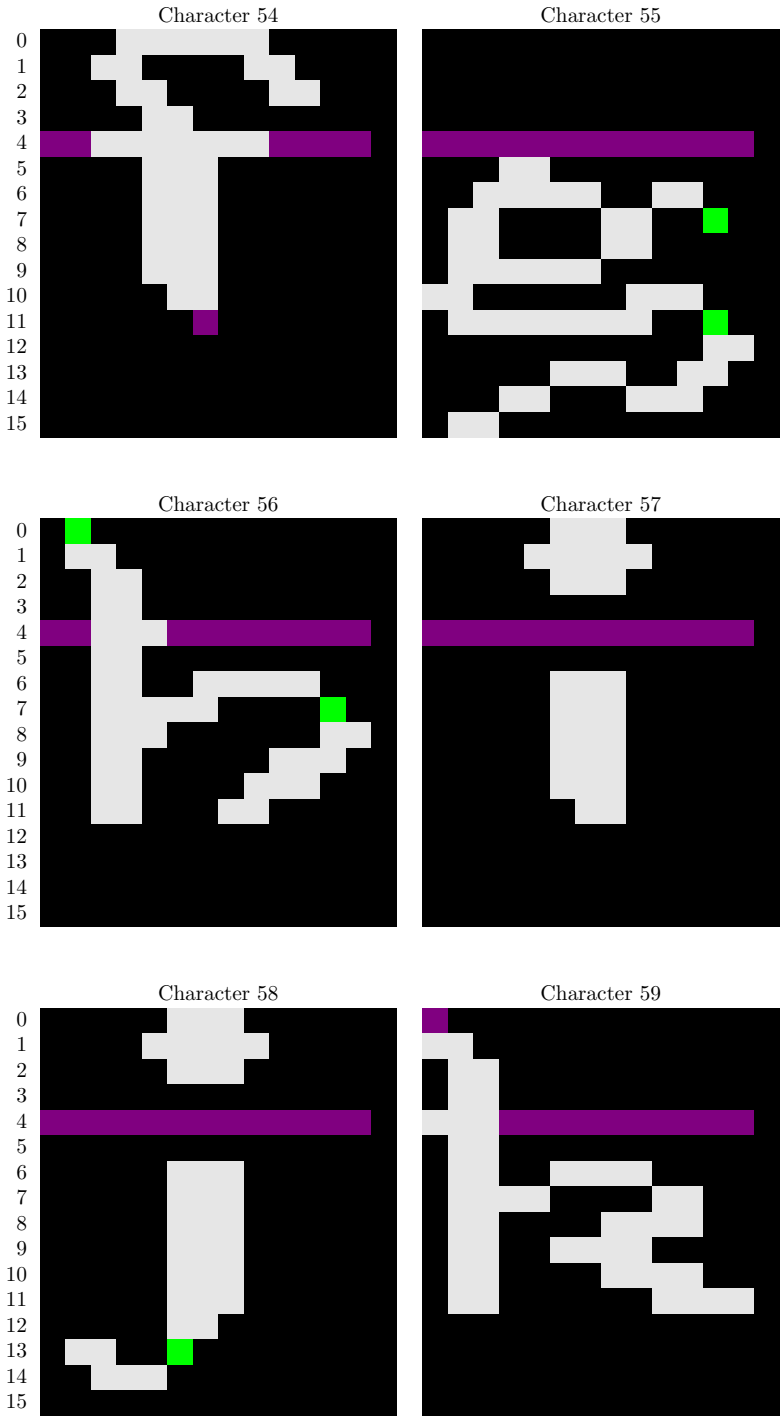
Character 46

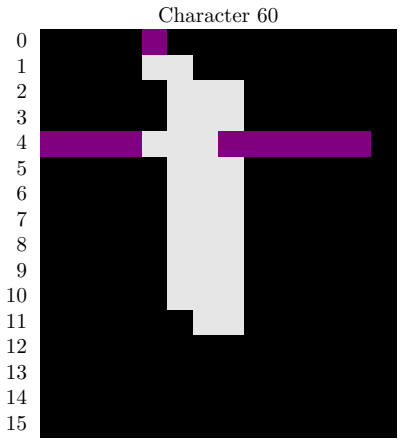


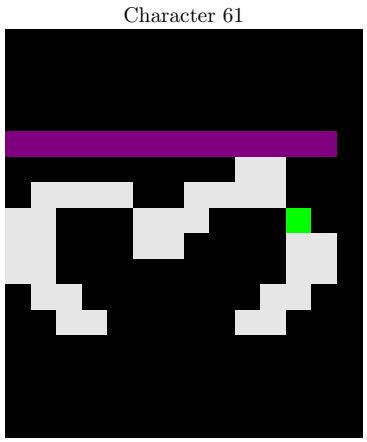
Character 47

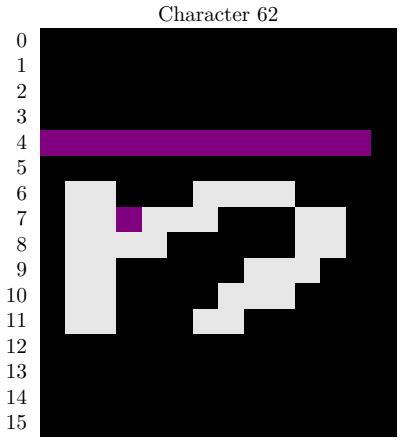


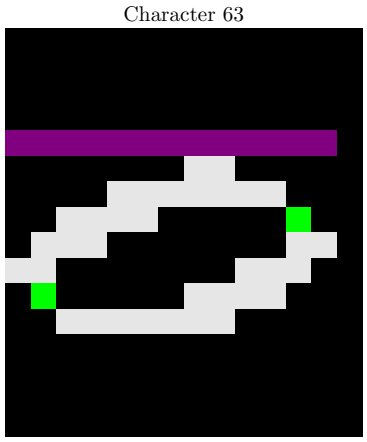

Character 48
Character 49
Character 50
Character 51
Character 52
Character 53
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

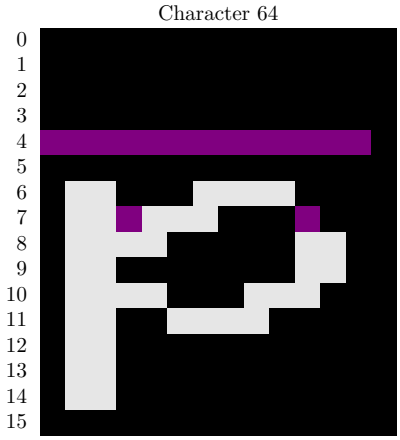

Character 54
Character 55
Character 56
Character 57
Character 58
Character 59
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

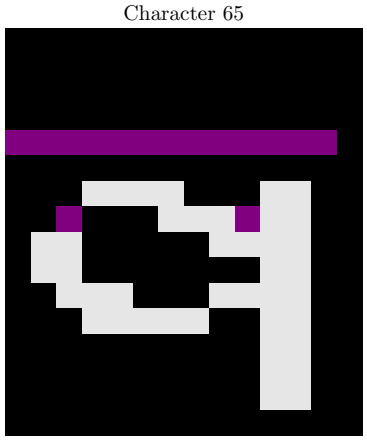

Character 60
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

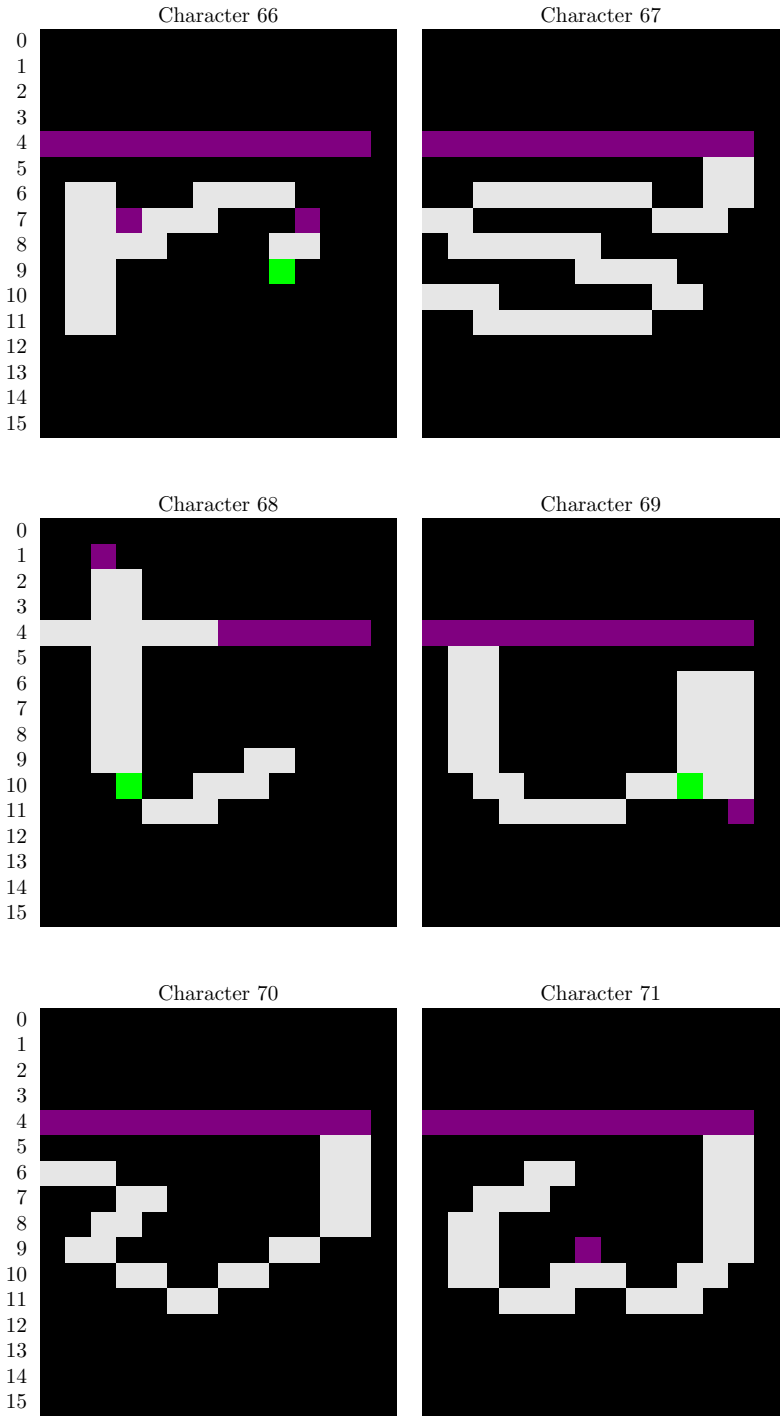

Character 61


Character 62
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

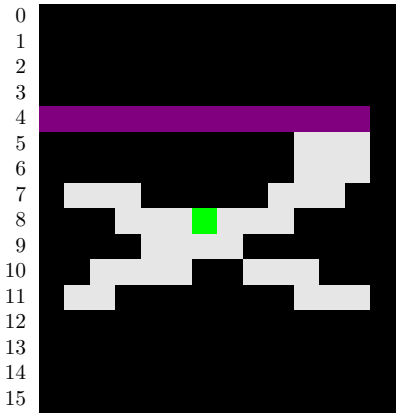

Character 63


Character 64
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

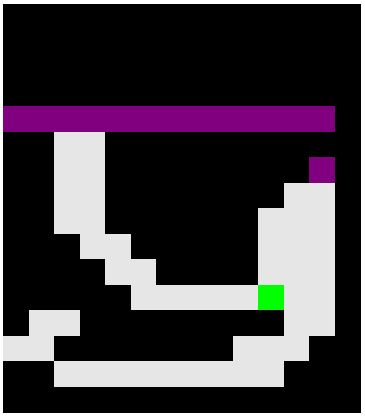

Character 65


Character 66
Character 67
Character 68
Character 69
Character 70
Character 71
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

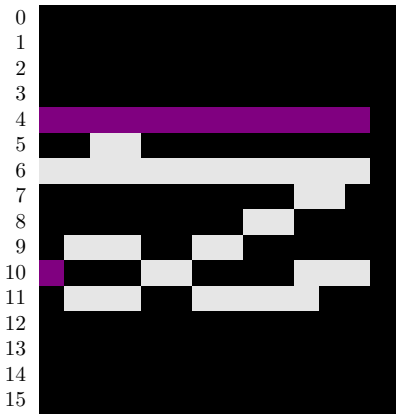
Character 72



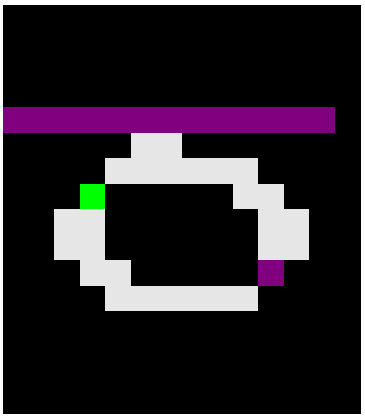
Character 73



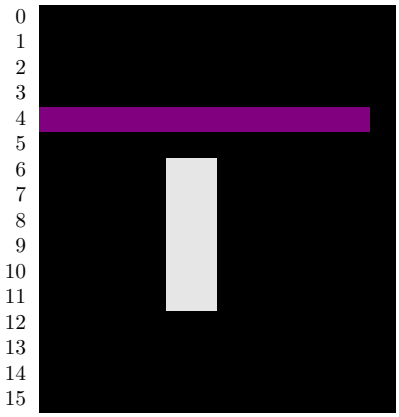
Character 74



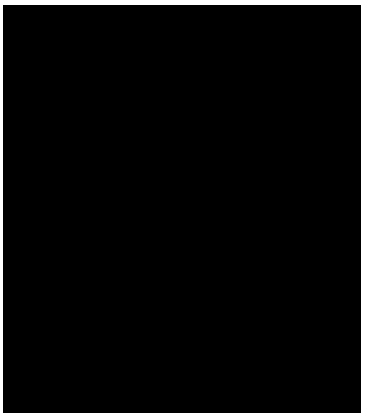
Character 75



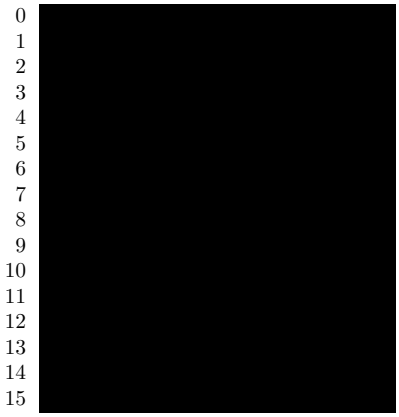
Character 76



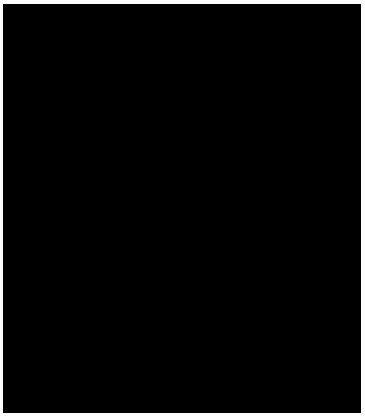
Character 77



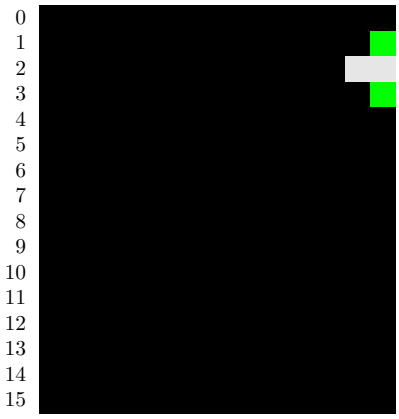
Character 78



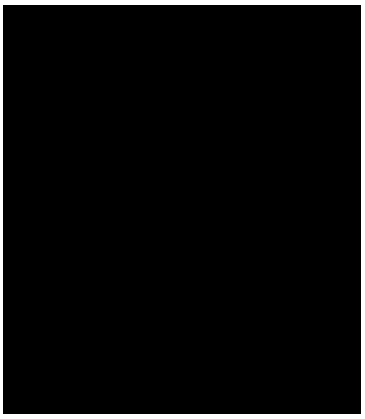
Character 79



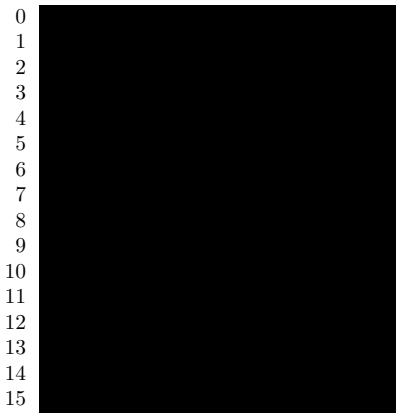
Character 80



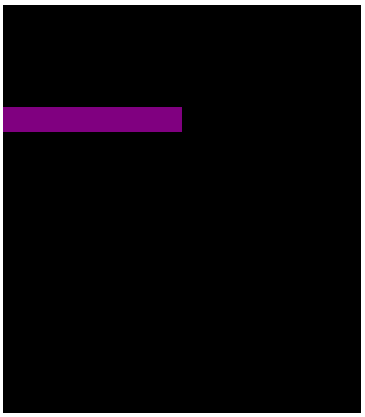
Character 81

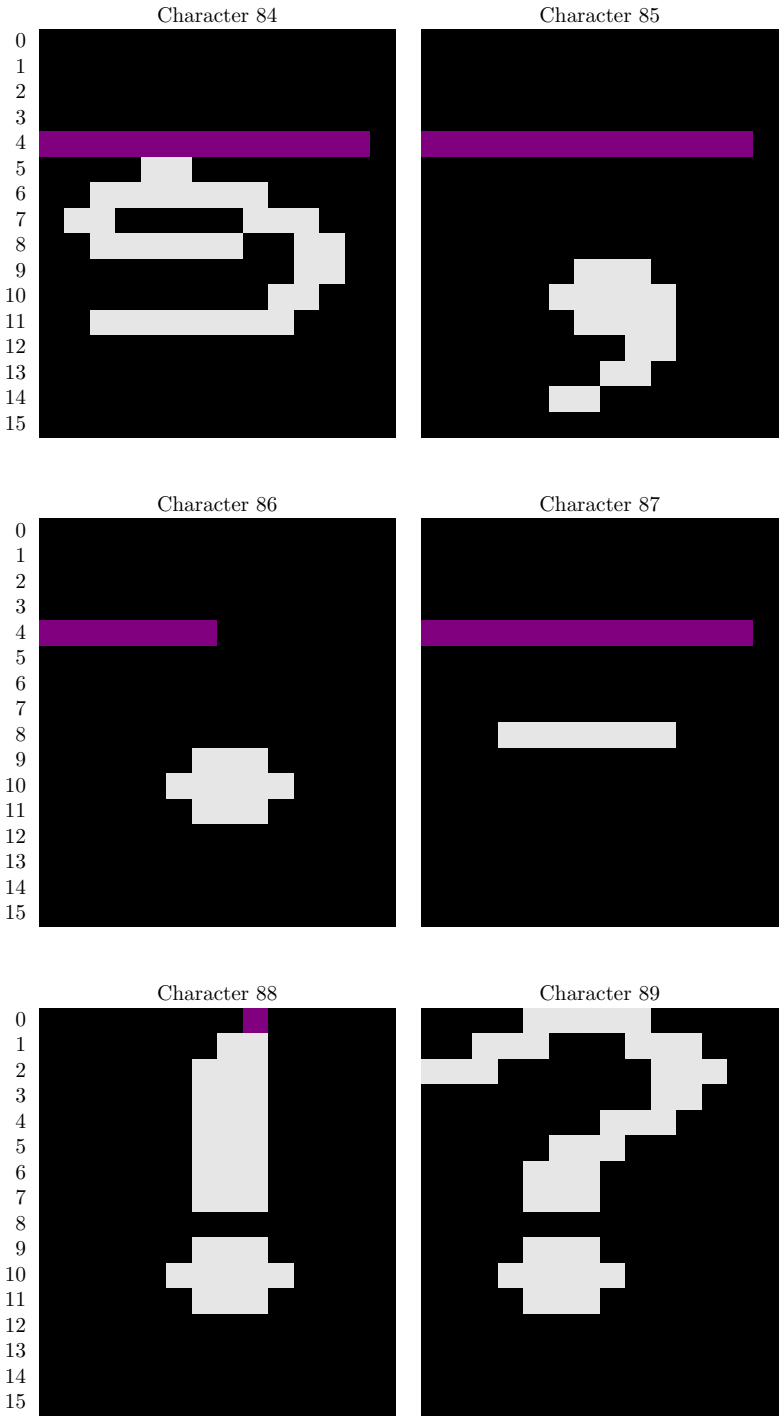


Character 82

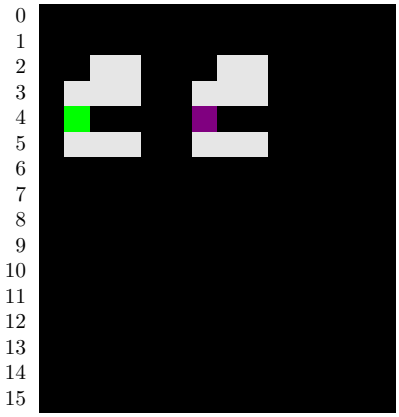


Character 83

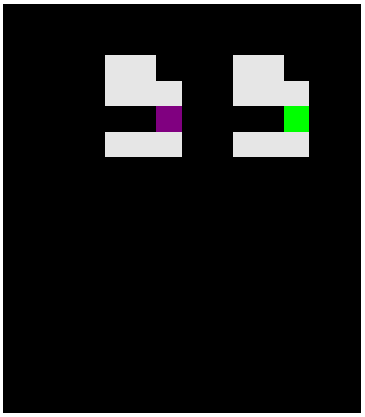



Character 84
Character 85
Character 86
Character 87
Character 88
Character 89
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

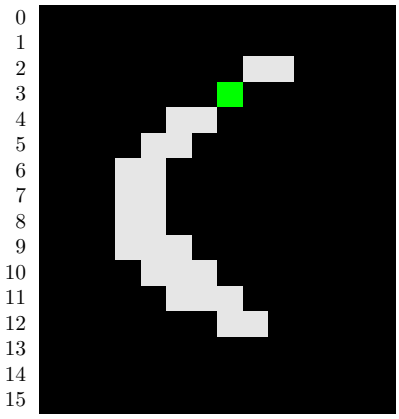
Character 90



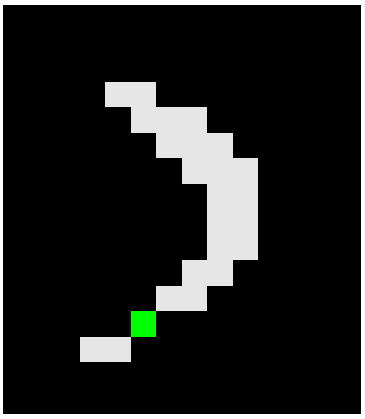
Character 91



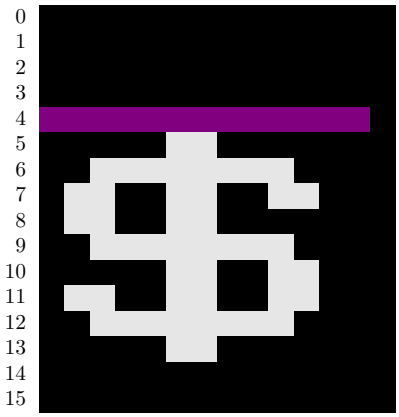
Character 92



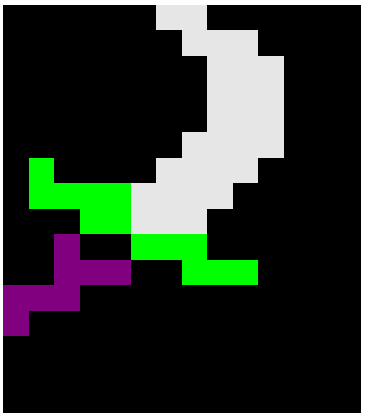
Character 93

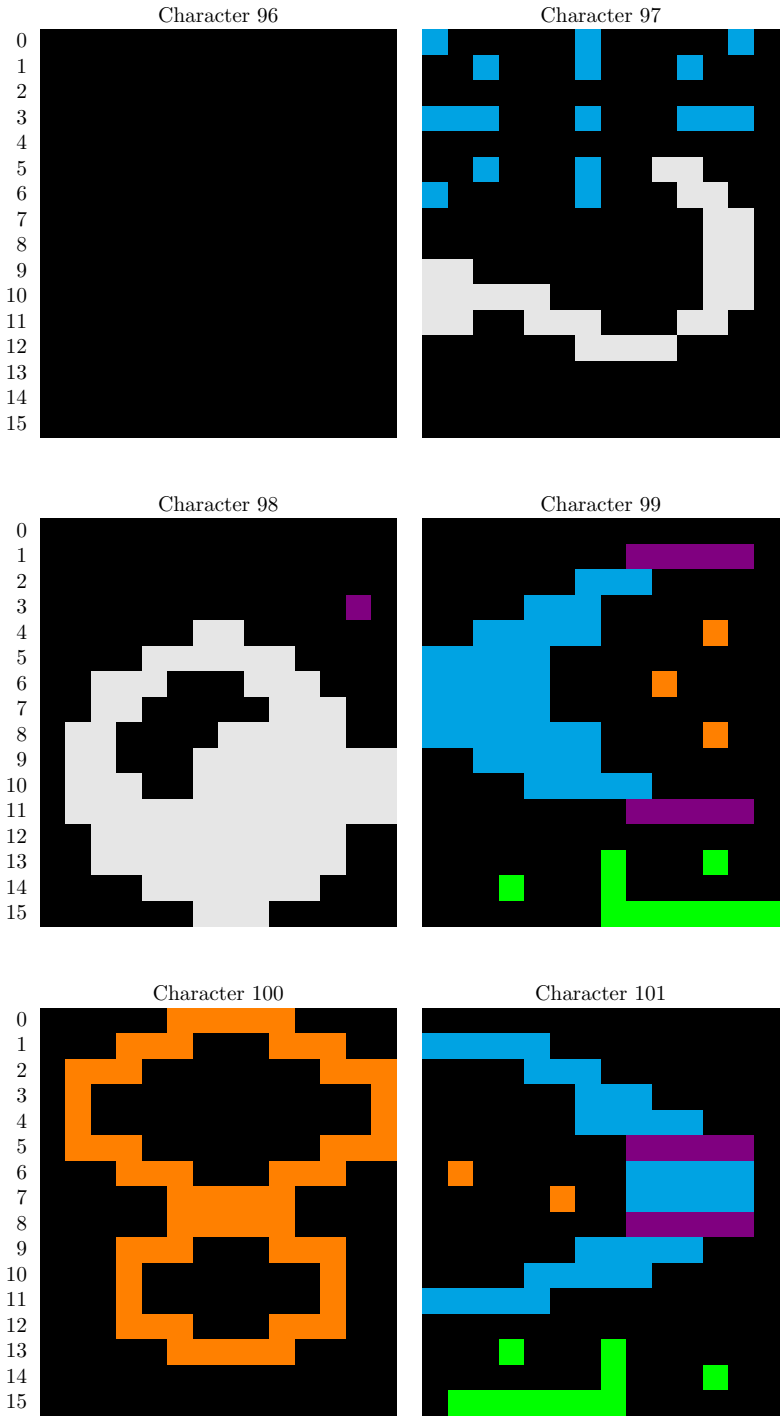


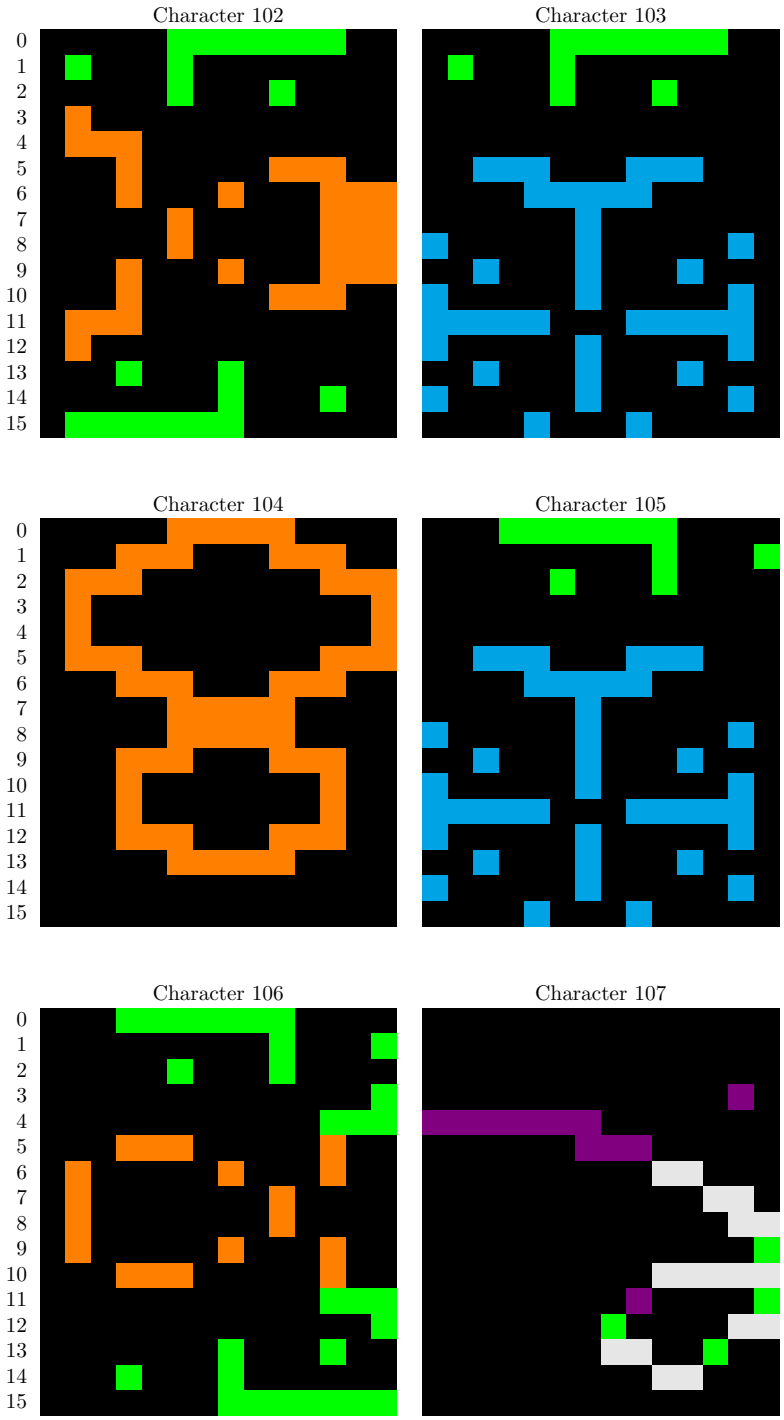
Character 94

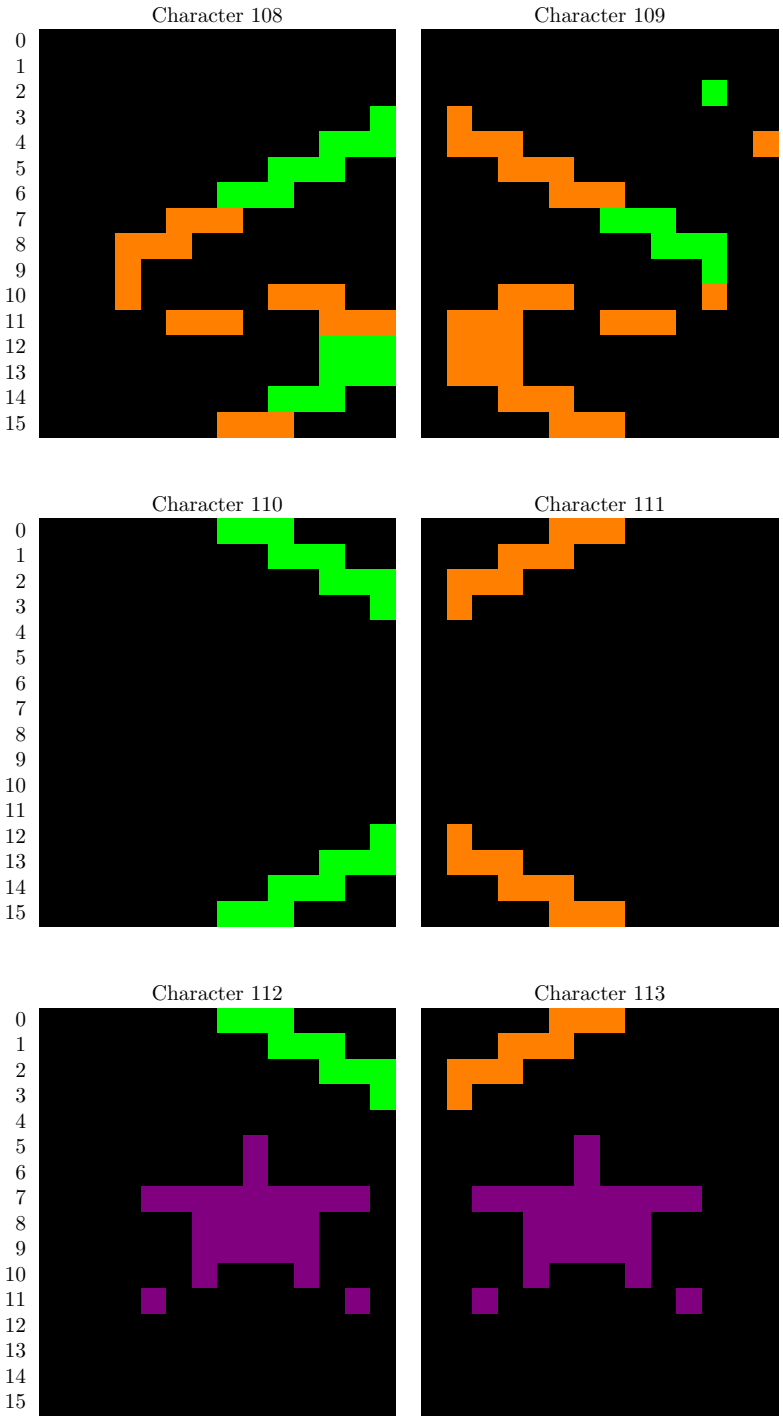


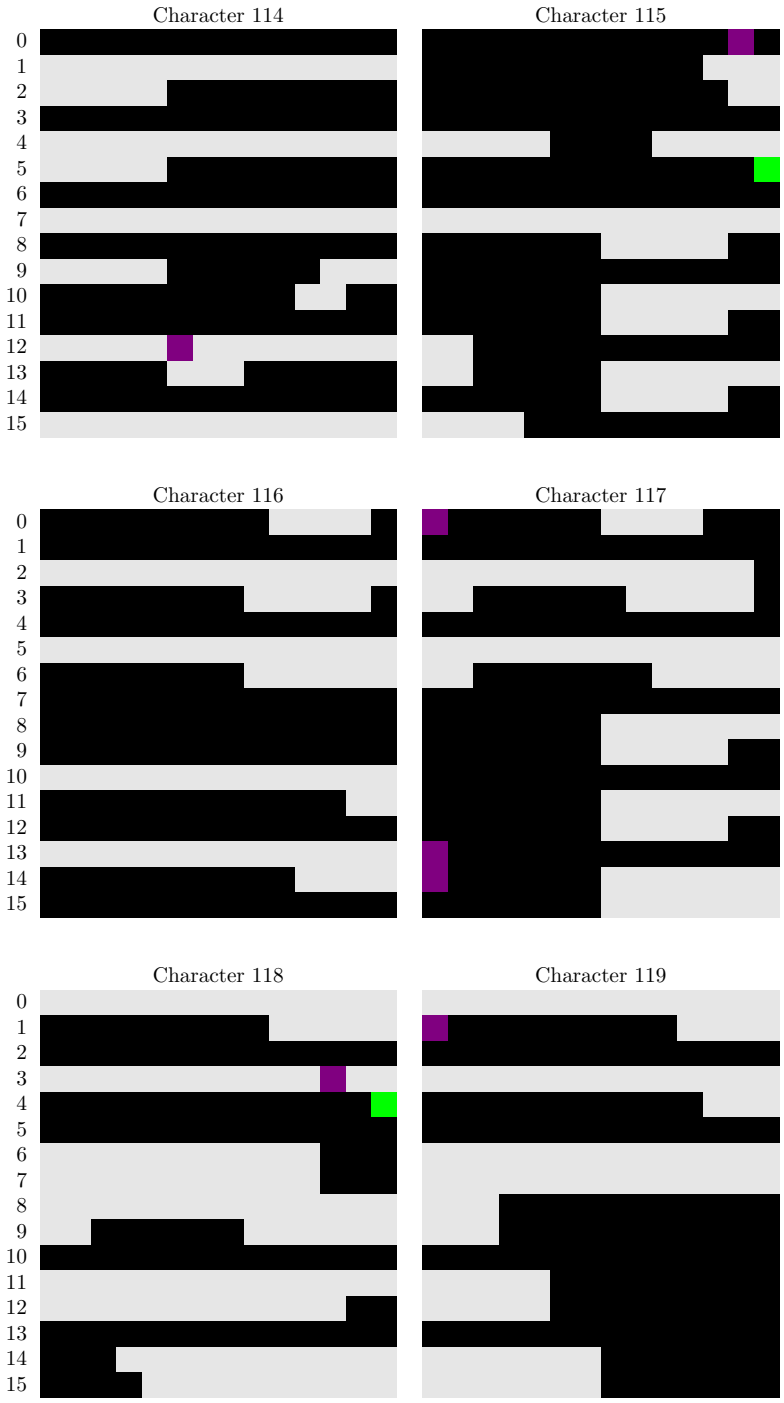
Character 95




Character 96
Character 97
Character 98
Character 99
Character 100
Character 101
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15


Character 102
Character 103
Character 104
Character 105
Character 106
Character 107
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15


Character 108
Character 109
Character 110
Character 111
Character 112
Character 113
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15



The HRCG hooks the character output (CSW) and keyboard input (KSW)

vectors to render text on the hi-res graphics screen. The library is self-contained at HRCG_INIT–$97A4 and provides:

- Character rendering in multiple fonts (up to 10 charsets of 8-byte glyphs)
- Video modes: replace, OR, AND, XOR with optional page-2 overlay
- Inverse, lowercase-invert, and shifted-character support
- A full set of control-character handlers for cursor movement, screen clearing, window management, and state save/restore
- Cursor blinking in the keyboard input handler
- Scrolling and line-wrap modes

The game's INIT_HRCG routine patches out the DOS reconnect call and injects an early RTS into HRCG_DISPLAY_PAGE before jumping to HRCG_INIT.

736 ⟨*hrcg init* 736⟩≡ (909)

```
    ORG     $92A8
HRCG_INIT:
; --- HRCG entry jump table ---
    JMP     HRCG_INIT_BODY ; main init entry
HRCG_NUM_CHARSETS:
    DC.B    $0A            ; 10 custom charsets
HRCG_FONT_BASE:
    DC.W    FONT_DATA      ; font base address ($83A5)
HRCG_ENTRY2:
    JMP     HRCG_RTS       ; secondary entry (disabled)
HRCG_ENTRY3:
    JMP     HRCG_RTS       ; secondary entry (disabled)
; --- HRCG_PRINT_VERSION ---
HRCG_PRINT_VERSION:
    SUBROUTINE HRCG_PRINT_VERSION
    JSR     HRCG_INIT_BODY
    LDY     #$00
.loop:
    LDA     HRCG_VERSION_STR,Y
    JSR     ROM_COUT1
    INY
    CPY     #$1D           ; 29 chars
    BCC     .loop
    RTS
; --- HRCG_INIT_BODY ---
HRCG_INIT_BODY:
    SUBROUTINE HRCG_INIT_BODY
    STOW    HRCG_GETKEY,$38 ; KSW low
    STOW    HRCG_COUT,$36 ; CSW low
    JSR     $03EA          ; reconnect DOS (patched to NOPs by game)
```

```
    BIT     $C052          ; full-screen mode (no mixed)
    BIT     $C057          ; hi-res mode
    JSR     HRCG_RESET
    JSR     HRCG_DISPLAY_PAGE
    BIT     $C050          ; graphics mode on
    RTS
```

Defines:
HRCG_ENTRY2, used in chunk 759.
HRCG_ENTRY3, used in chunk 759.
HRCG_FONT_BASE, used in chunk 745.
HRCG_INIT, used in chunk 568.
HRCG_INIT_BODY, never used.
HRCG_NUM_CHARSETS, never used.
HRCG_PRINT_VERSION, used in chunk 61c.
Uses FONT_DATA 715a, HRCG_COUT 743, HRCG_DISPLAY_PAGE 749, HRCG_GETKEY 738, HRCG_RESET 759, HRCG_RTS 757, HRCG_VERSION_STR 715c, ROM_COUT1 571a, and STOW 8a.

The HRCG maintains its state in a self-modifying parameter block at $9305--$9319. These 21 bytes control every aspect of text rendering: which font is active, whether characters are drawn in replace, OR, AND, or XOR mode, whether inverse video is enabled, and whether the cursor wraps or scrolls at the bottom of the text window.

The parameter block is laid out so that it can be saved and restored in bulk by the ctrl-B and ctrl-D handlers. The first 8 bytes (HRCG_VIDEO_MODE through HRCG_FONT_PTR_HI) are the saveable state; these are followed by a saved copy, a "saved" flag byte, and the saved cursor position.

737a ⟨*hrcg params* 737a⟩≡ (909)

```
    ORG     $9305
HRCG_SHIFT_FLAG:
    DC.B    $00                         ; $00=normal, $40=next-shifted, $80=charset-select
```

Defines:
HRCG_SHIFT_FLAG, used in chunk 738.

737b ⟨*hrcg video mode* 737b⟩≡ (909)

```
    ORG     $9307
HRCG_VIDEO_MODE:
    DC.B    $20                         ; $20=replace, $40=OR, $80=AND, $C0=XOR
```

Defines:
HRCG_VIDEO_MODE, used in chunks 738, 747a, 749, 759, and 768.

The HRCG replaces both the Apple II's character output vector (CSW at $36/$37) and its keyboard input vector (KSW at $38/$39). HRCG_GETKEY is the KSW handler. Its primary job is to blink the cursor while waiting for a keypress, but it also interprets ESC-key sequences for cursor movement and screen editing—a miniature line editor built into the input routine.

The blinking works by toggling the 7 pixel bits at the cursor's screen position. A countdown register (X) controls the blink rate: every time the high byte of the blink timer overflows, X decrements, and when it reaches zero the cursor pixels are inverted. The blink timer at $4E/$4F ticks continuously while the routine polls KBD_DATA for a keypress.

When a key arrives, the routine checks whether it was called from the ROM input routine at $F8xx (by peeking at the return address on the stack). If so, it processes ESC sequences—ESC followed by I/J/K/M for cursor movement, or ESC-@/E/F/C for screen editing. If the caller is not the ROM, the raw keycode is returned immediately.

738 ⟨*hrcg getkey* 738⟩≡ (909)

```
    ORG     $931A
HRCG_GETKEY:
    SUBROUTINE HRCG_GETKEY
; --- KSW handler: cursor blink + keyboard input ---
    STA     ($28),Y             ; save char at cursor pos (BASL)
    STX     $EB                 ; save X
    MOVB    $28,$2A ; BASL
    LDA     $29                 ; BASH
    ORA     #$1C                ; set hi-res page offset bits
    ORA     HRCG_VIDEO_MODE
    STA     $2B                 ; store hi byte
    LDX     #$01                ; blink counter init
.blink_loop:
    LDA     ($2A),Y             ; read screen byte at cursor
    PHA                         ; save it
.blink_wait:
    INC     $4E                 ; increment blink timer low
    BNE     .blink_check
    INC     $4F                 ; increment blink timer high
    DEX                         ; decrement blink counter
    BNE     .blink_check
    EOR     #$7F                ; toggle cursor (invert bits 0-6)
    STA     ($2A),Y             ; write toggled cursor
    LDX     #$50                ; reset blink counter
.blink_check:
    BIT     $C000               ; check keyboard
    BPL     .blink_wait         ; no key -> keep blinking
    PLA                         ; restore original screen byte
    STA     ($2A),Y             ; restore cursor position
    TSX                         ; get stack pointer
    LDA     $0104,X             ; peek at return address high byte
```

```
    CMP     #$F8                ; called from ROM input ($F8xx)?
    LDA     $C000               ; read key
    BIT     $C010               ; clear keyboard strobe
    BCC     .return_direct      ; not from ROM -> return key directly
    PHA                         ; save key
    BIT     HRCG_SHIFT_FLAG     ; check shift flag
    BMI     .shifted            ; if set -> handle shifted key
    BANE    #$9B,.check_cr      ; ESC key?
    STOB    #$80,HRCG_SHIFT_FLAG ; set shift flag
    BNE     .discard_key        ; always branch -> return
.check_cr:
    BANE    #$8D,.discard_key   ; CR key?
    JSR     HRCG_CLREOL         ; clear rest of line
.reload_ch:
    LDY     $24                 ; reload cursor X
.discard_key:
    PLA                         ; discard saved key
.return_direct:
    LDX     $EB                 ; restore X
    RTS
; --- Shifted key handling (ESC was pressed) ---
.shifted:
    BALT    #$C9,.try_other     ; 'I' ?
    BAEQ    #$CD,.scroll_down   ; 'M' ?
    BCS     .try_other          ; > 'M' -> try as normal
    BAEQ    #$CB,.cursor_right  ; 'K' ?
    BCC     .discard_key        ; < 'K' (='J') -> ignore
.try_other:
    ASL     HRCG_SHIFT_FLAG     ; clear shift flag
    BANE    #$C0,.try_e         ; '@' ?
    JSR     HRCG_HOME
    JMP     .reload_ch
.try_e:
    BANE    #$C5,.try_f         ; 'E' ?
    JSR     HRCG_CLREOL
    JMP     .reload_ch
.try_f:
    BANE    #$C6,.try_c         ; 'F' ?
    JSR     HRCG_CLRLINE
    JMP     .reload_ch
.try_c:
    BAEQ    #$C3,.scroll_down   ; 'C' ?
    BANE    #$C1,.discard_key   ; not recognized -> return
.cursor_right:
    INY                         ; cursor right: Y++
    CPY     $21                 ; past right edge (WNDWDTH)?
    BCC     .reload_ch
.scroll_down:
    LDY     $25                 ; get cursor Y (CV)
    INY
```

```
CPY     $23                 ; past bottom (WNDBTM)?
BCC     .reload_ch
JSR     HRCG_SCROLL_UP
JMP     .reload_ch
```

Defines:
HRCG_GETKEY, used in chunk 736.
Uses HRCG_CLREOL 766, HRCG_CLRLINE 766, HRCG_HOME 766, HRCG_SCROLL_UP 764, HRCG_SHIFT_FLAG 737a, HRCG_VIDEO_MODE 737b, MOVB 8c, and STOB 8c.

The dispatch table maps each control character (ctrl-@ through ctrl-Z) to a handler address, stored as address-minus-one for the RTS trick. The HRCG supports a "shifted" variant of each control character: if the previous character was ctrl-A or ctrl-O, a flag is set in HRCG_MODE_FLAGS, and when the next handler is entered, the carry flag will be set (from the ASL HRCG_MODE_FLAGS in HRCG_PROCESS_CHAR). Each handler checks carry on entry to decide between its normal and shifted behavior.

741 ⟨*hrcg dispatch table* 741⟩≡ (909)

```
      ORG     $93EE
  HRCG_DISPATCH_TBL:
  ; Control character dispatch table (address-1 for RTS trick)
  ; ctrl-@: no-op
      DC.W    HRCG_CTRL_NOP-1
  ; ctrl-A: set shift/charset flag
      DC.W    HRCG_CTRL_A-1
  ; ctrl-B: save state
      DC.W    HRCG_CTRL_B_SAVE-1
  ; ctrl-C: restore cursor / XOR mode
      DC.W    HRCG_CTRL_C-1
  ; ctrl-D: restore state / display page
      DC.W    HRCG_CTRL_D-1
  ; ctrl-E: clear EOL
      DC.W    HRCG_CTRL_E-1
  ; ctrl-F: clear to end of screen
      DC.W    HRCG_CTRL_F-1
  ; ctrl-G: bell
      DC.W    $FBDC
  ; ctrl-H: backspace
      DC.W    HRCG_CTRL_H-1
  ; ctrl-I: inverse video
      DC.W    HRCG_CTRL_I-1
  ; ctrl-J: line feed
      DC.W    HRCG_LINE_FEED-1
  ; ctrl-K: clear invert flag
      DC.W    HRCG_CTRL_K-1
  ; ctrl-L: set invert flag
      DC.W    HRCG_CTRL_L-1
  ; ctrl-M: carriage return
      DC.W    HRCG_CTRL_M-1
  ; ctrl-N: normal video
      DC.W    HRCG_CTRL_N-1
  ; ctrl-O: next-shifted / OR mode
      DC.W    HRCG_CTRL_O-1
  ; ctrl-P: home / clear draw mode
      DC.W    HRCG_CTRL_P-1
  ; ctrl-Q: goto saved cursor
      DC.W    HRCG_CTRL_Q-1
  ; ctrl-R: XOR draw mode
      DC.W    HRCG_CTRL_R-1
```

```
; ctrl-S: XOR invert / clear wrap
    DC.W    HRCG_CTRL_S-1
; ctrl-T: AND draw mode
    DC.W    HRCG_CTRL_T-1
; ctrl-U: no-op
    DC.W    HRCG_CTRL_NOP-1
; ctrl-V: set text window
    DC.W    HRCG_CTRL_V-1
; ctrl-W: set margins
    DC.W    HRCG_CTRL_W-1
; ctrl-X: no-op
    DC.W    HRCG_CTRL_NOP-1
; ctrl-Y: clear screen
    DC.W    HRCG_CTRL_Y-1
; ctrl-Z: reset
    DC.W    HRCG_CTRL_Z-1
```

Defines:
HRCG_DISPATCH_TBL, used in chunk 743.
Uses HRCG_CTRL_A 745, HRCG_CTRL_B_SAVE 747a, HRCG_CTRL_C 748, HRCG_CTRL_D 749, HRCG_CTRL_E 751, HRCG_CTRL_F 751, HRCG_CTRL_H 751, HRCG_CTRL_I 751, HRCG_CTRL_K 753, HRCG_CTRL_L 753, HRCG_CTRL_M 753, HRCG_CTRL_N 754, HRCG_CTRL_NOP 751, HRCG_CTRL_O 754, HRCG_CTRL_P 754, HRCG_CTRL_Q 754, HRCG_CTRL_R 756, HRCG_CTRL_S 756, HRCG_CTRL_T 756, HRCG_CTRL_V 757, HRCG_CTRL_W 757, HRCG_CTRL_Y 759, HRCG_CTRL_Z 759, and HRCG_LINE_FEED 763.

HRCG_COUT is the CSW handler—the replacement for the Apple II's standard character output routine. Every JSR ROM_COUT1 in the game ultimately lands here. The handler is a thin wrapper: it saves the CPU registers, calls HRCG_PROCESS_CHAR, and restores them.

HRCG_PROCESS_CHAR strips the high bit from the character (Apple II text uses high-bit ASCII) and decides what to do with it. If the "charset select" flag is set in HRCG_MODE_FLAGS, the character is treated as a charset number and routed to HRCG_CHARSET_SELECT. Otherwise, printable characters ($20 and above) go to HRCG_DRAW_CHAR for rendering, and control characters (below $20) are dispatched through a 27-entry RTS-trick jump table. The dispatch doubles the control code to form a table index, pushes the handler address (minus one) onto the stack, and falls through to RTS—the classic 6502 technique for computed jumps.

742 ⟨*hrcg mode flags* 742⟩≡ (909)

```
    ORG     $9306
HRCG_MODE_FLAGS:
    DC.B    $00                         ; bit 7/6 shifted by ASL during dispatch
```

Defines:
HRCG_MODE_FLAGS, used in chunks 743, 745, 754, and 761.

743 ⟨*hrcg cout* 743⟩≡ (909)

```
    ORG     $93BE
HRCG_COUT:
    SUBROUTINE HRCG_COUT
; --- CSW handler: save regs, process char, restore ---
    STA     $FF                     ; save A
    STX     $EB                     ; save X
    STY     $35                     ; save Y
    JSR     HRCG_PROCESS_CHAR
    LDY     $35                     ; restore Y
    LDX     $EB                     ; restore X
    LDA     $FF                     ; restore A
    RTS
; --- HRCG_PROCESS_CHAR ---
HRCG_PROCESS_CHAR:
    AND     #$7F                    ; strip high bit
    ASL     HRCG_MODE_FLAGS         ; shift mode bit 7 into carry
    BCS     HRCG_CHARSET_SELECT     ; if set -> charset select
    BALT    #$20,.ctrl_dispatch ; printable char (>= space)?
    JMP     HRCG_DRAW_CHAR
.ctrl_dispatch:
    BAGE    #$1B,.shift_and_rts ; >= ESC ($1B)?
    ASL                             ; char * 2 for table index
    TAX
    LDA     HRCG_DISPATCH_TBL+1,X ; high byte of handler-1
    PHA
    LDA     HRCG_DISPATCH_TBL,X   ; low byte of handler-1
    PHA
.shift_and_rts:
    ASL     HRCG_MODE_FLAGS         ; shift mode bit 6 into carry
    RTS                             ; RTS-trick dispatch
```

Defines:
HRCG_COUT, used in chunk 736.
HRCG_PROCESS_CHAR, used in chunk 745.
Uses HRCG_CHARSET_SELECT 745, HRCG_DISPATCH_TBL 741, HRCG_DRAW_CHAR 761, and HRCG_MODE_FLAGS 742.

Control character handlers

The control character handlers follow. Each one checks the carry flag on entry to distinguish its normal behavior from its shifted variant.

Ctrl-A is the charset selector. In normal mode, it sets HRCG_MODE_FLAGS bit 7 so that the *next* character output will be routed to HRCG_CHARSET_SELECT and interpreted as a charset number ("1" through "9") rather than printed. In shifted mode (ctrl-A, ctrl-A, *char*), it resets the video mode to replace.

When the charset-select path fires, the selector byte is converted to a font pointer. Each charset occupies 3 pages (768 bytes = 96 glyphs × 8 bytes), and the font address is computed as HRCG_FONT_BASE + (selector − 1) × $300. Selectors outside 1–9 fall through to the standard 7×8 font at STD_FONT_DATA.

The game's custom font data spans $83A5–$92A4 (3840 bytes, 120 characters at 32 bytes each). Each charset page contains 96 HRCG glyphs (768 bytes), and each custom character uses 4 consecutive glyphs, so one page holds 24 characters. Since charset pages overlap (each starts only 768 bytes after the previous), the 120 characters are distributed as:

- Charset 1 ($83A5): characters 0–23 (game map icons).
- Charset 2 ($86A5): characters 24–47 (border tiles).
- Charset 3 ($89A5): characters 48–71 (scene text, stylized).
- Charset 4 ($8CA5): characters 72–95 (scene text cont.).
- Charset 5 ($8FA5): characters 96–119 (scene text cont.).

The scene text font (charsets 3–5) uses an ornate, medieval/Arabian-themed typeface with a decorative horizontal divider row across each letter—fitting the game's Arabian Nights setting. Characters are rendered in Apple II hi-res color (not monochrome): the HRCG color artifact algorithm produces green, violet, orange, blue, and white pixels depending on bit patterns and the palette bit (bit 7). See font_data.py for the full color rendering algorithm, and font_data.tex for a complete visual reference (characters 48–119 correspond to the scene text glyphs).

744 ⟨*hrcg font ptr* 744⟩≡ (909)

```
    ORG     $930D
HRCG_FONT_PTR_LO:
    DC.B    $A5                         ; font data pointer low
HRCG_FONT_PTR_HI:
    DC.B    $97                         ; font data pointer high ($97A5 = standard font)
```

Defines:
HRCG_FONT_PTR_HI, used in chunks 745 and 761.
HRCG_FONT_PTR_LO, used in chunks 745 and 761.

745 ⟨*hrcg ctrl a* 745⟩≡ (909)

```
    ORG     $9424
HRCG_CTRL_A:
    SUBROUTINE HRCG_CTRL_A
    ;
    ; Ctrl-A handler: charset selector.
    ;
    ; Normal mode (carry clear):
    ;   Sets HRCG_MODE_FLAGS bit 7 so that the NEXT character output
    ;   will be routed to HRCG_CHARSET_SELECT instead of being printed.
    ;   The next byte is interpreted as a charset number ('1'-'9').
    ;
    ; Shifted mode (carry set, from ctrl-A ctrl-A):
    ;   Resets the video mode to replace ($20).
    ;
; --- Normal: arm charset-select for next character ---
    BCS     HRCG_SET_REPLACE  ; shifted -> set replace mode
    STOB    #$80,HRCG_MODE_FLAGS ; set bit 7: next char -> charset select
    RTS
    ;
    ; --- Charset select (entered when HRCG_MODE_FLAGS bit 7 was set) ---
    ;
    ; Input:  A = charset selector byte ('1'-'9' = $B1-$B9 with high
    ;             bit already stripped by HRCG_PROCESS_CHAR)
    ;
    ; Charset memory layout:
    ;   Each charset occupies 3 pages (768 bytes = 96 glyphs * 8 bytes).
    ;   Charset 1 starts at HRCG_FONT_BASE ($83A5).
    ;   Charset N starts at HRCG_FONT_BASE + (N-1) * $300.
    ;
    ;   Charset  Selector  Address  Chars   Contents
    ;   -------  --------  -------  -----   --------
    ;      1       $B1     $83A5    0-23    Game map icons
    ;      2       $B2     $86A5    24-47   Border tiles
    ;      3       $B3     $89A5    48-71   Scene text (ornate font)
    ;      4       $B4     $8CA5    72-95   Scene text cont.
    ;      5       $B5     $8FA5    96-119  Scene text cont.
    ;    0 or >9   ---     $97A5    ---     Standard 7x8 font
    ;
    ;   Each charset holds 96 glyphs (768 bytes).  Charsets overlap:
    ;   charset N+1 starts 24 characters (768 bytes) after charset N.
    ;   Scene text (group 2) starts at char 48 = charset 3.
    ;
    ; Output: HRCG_FONT_PTR set to the charset's base address.
    ;
HRCG_CHARSET_SELECT:
    SBC     #$31              ; A = selector - '1' (carry set from caller)
                              ; charset '1' -> 0, '2' -> 1, ... '9' -> 8
    BAGE    #$09,HRCG_SET_STD_FONT ; >= 9 -> invalid, use standard font
    ;
```

```
    ; Compute font address: HRCG_FONT_BASE + index * 3 pages
    ;   index * 3 = index * 2 + index (via ASL + ADC)
    ;   Result is the HIGH BYTE of the font address.
    ;   Low byte is always HRCG_FONT_BASE low ($A5).
    ;
    LDX     HRCG_FONT_BASE    ; X = font base low ($A5)
    STA     $EE               ; save index
    ASL                       ; A = index * 2
    ADC     $EE               ; A = index * 3 (pages)
    ADC     HRCG_FONT_BASE+1  ; A = $83 + index*3 = font high byte
    BNE     .store_font       ; always taken (result never zero)
    ;
    ; Standard font fallback (charset 0 or invalid selector)
    ;
HRCG_SET_STD_FONT:
    LDX     #<STD_FONT_DATA   ; X = $A5 (standard font low)
    LDA     #>STD_FONT_DATA   ; A = $97 (standard font high -> $97A5)
.store_font:
    STX     HRCG_FONT_PTR_LO  ; update font pointer low byte
    STA     HRCG_FONT_PTR_HI  ; update font pointer high byte
    RTS
```

Defines:
HRCG_CHARSET_SELECT, used in chunk 743.
HRCG_CTRL_A, used in chunk 741.
HRCG_SET_STD_FONT, used in chunk 759.
Uses HRCG_FONT_BASE 736, HRCG_FONT_PTR_HI 744, HRCG_FONT_PTR_LO 744, HRCG_MODE_FLAGS 742, HRCG_PROCESS_CHAR 743, HRCG_SET_REPLACE 747a, STD_FONT_DATA 715b, and STOB 8c.

Ctrl-B saves the HRCG's rendering state—the 8-byte parameter block (video mode through font pointer) plus the cursor position—so it can be restored later with ctrl-D. Only one level of save is supported; if the state has already been saved, only the cursor position is updated. The shifted variant sets OR video mode instead.

This save/restore mechanism is used by the game to temporarily switch fonts or video modes for status display, then return to the previous rendering context.

746 ⟨*hrcg saved state* 746⟩≡ (909)

```
    ORG     $930F
HRCG_SAVED_PARAMS:
    DC.B    $20,$00,$00,$00,$00,$00,$A5,$97 ; saved copy of video mode..font ptr (8 bytes)
    DC.B    $FF                             ; HRCG_SAVED_FLAG ($00=not saved, $FF=saved)
HRCG_SAVED_CH:
    DC.B    $01                             ; saved cursor X
HRCG_SAVED_CV:
    DC.B    $17                             ; saved cursor Y (23)
```

Defines:
HRCG_SAVED_CH, used in chunks 747–49, 754, and 759.
HRCG_SAVED_CV, used in chunks 747a, 749, 754, 757, and 759.
HRCG_SAVED_PARAMS, used in chunks 747a, 749, 757, and 759.

747a ⟨*hrcg ctrl b* 747a⟩≡ (909)

```
    ORG     $944A
HRCG_SET_REPLACE:
; --- ctrl-A shifted: set replace video mode ---
    STOB    #$20,HRCG_VIDEO_MODE ; video mode = replace ($20)
    RTS
; --- ctrl-B: save params + cursor ---
HRCG_CTRL_B_SAVE:
    SUBROUTINE HRCG_CTRL_B_SAVE
    BCS     .set_or              ; double-shifted -> set OR mode
    BIT     HRCG_SAVED_PARAMS+8 ; already saved?
    BMI     .save_cursor         ; yes -> just update cursor
    STOB    #$FF,HRCG_SAVED_PARAMS+8 ; mark as saved
    LDY     #$07                 ; 8 bytes to save
.save_loop:
    LDA     HRCG_VIDEO_MODE,Y    ; active params
    STA     HRCG_SAVED_PARAMS,Y ; saved copy
    DEY
    BPL     .save_loop
.save_cursor:
    MOVB    $24,HRCG_SAVED_CH ; cursor X (CH)
    MOVB    $25,HRCG_SAVED_CV ; cursor Y (CV)
    RTS
; --- ctrl-B double-shifted: OR video mode ---
.set_or:
    STOB    #$40,HRCG_VIDEO_MODE ; video mode = OR ($40)
    RTS
```

Defines:
HRCG_CTRL_B_SAVE, used in chunk 741.
HRCG_SET_REPLACE, used in chunks 745 and 759.
Uses HRCG_SAVED_CH 746, HRCG_SAVED_CV 746, HRCG_SAVED_PARAMS 746, HRCG_VIDEO_MODE 737b, MOVB 8c, and STOB 8c.

Ctrl-C restores the saved cursor position (from ctrl-B). If the saved column matches the current column, it also performs a line feed. If the saved column is to the right of the current column, it does nothing—this prevents the cursor from jumping backwards unexpectedly. The shifted variant enables XOR draw mode on page 1.

747b ⟨*hrcg draw mode* 747b⟩≡ (909)

```
    ORG     $9309
HRCG_DRAW_MODE:
    DC.B    $00                          ; video mode for drawing ($00/$80/$C0)
HRCG_PAGE_MASK:
    DC.B    $00                          ; $00=page1, $60=page2
```

Defines:
HRCG_DRAW_MODE, used in chunks 748, 754, 756, 761, 764, and 766.
HRCG_PAGE_MASK, used in chunks 748, 754, 756, 761, and 766.

748 ⟨*hrcg ctrl c* 748⟩≡ (909)

```
    ORG     $9478
HRCG_CTRL_C:
    SUBROUTINE HRCG_CTRL_C
; --- ctrl-C normal: restore cursor position ---
    BCS     .set_xor_draw  ; shifted -> XOR draw mode
    LDA     HRCG_SAVED_CH  ; saved CH
    CMP     $24            ; compare to current CH
    STA     $24            ; restore CH
    BEQ     .do_lf         ; same -> just do line feed
    BCS     HRCG_CTRL_NOP  ; saved > current -> RTS
.do_lf:
    JMP     HRCG_LINE_FEED
; --- ctrl-C shifted: XOR draw mode ---
.set_xor_draw:
    STOB    #$C0,HRCG_DRAW_MODE ; draw mode = XOR ($C0)
    STOB    #$00,HRCG_PAGE_MASK ; page mask = 0
    RTS
```

Defines:
HRCG_CTRL_C, used in chunk 741.

Uses HRCG_CTRL_NOP 751, HRCG_DRAW_MODE 747b, HRCG_LINE_FEED 763, HRCG_PAGE_MASK 747b, HRCG_SAVED_CH 746, and STOB 8c.

Ctrl-D is the companion to ctrl-B: it restores the full 8-byte parameter block that was previously saved. If nothing was saved, it does nothing. The shifted variant calls HRCG_DISPLAY_PAGE, which selects hi-res page 1 or page 2 based on the current video mode—the game patches this to an early RTS, effectively disabling page switching.

749 ⟨*hrcg ctrl d* 749⟩≡ (909)

```
    ORG     $9493
HRCG_CTRL_D:
    SUBROUTINE HRCG_CTRL_D
; --- ctrl-D normal: restore saved params ---
    BCS     HRCG_DISPLAY_PAGE   ; shifted -> display page
    BIT     HRCG_SAVED_PARAMS+8 ; params saved?
    BPL     HRCG_CTRL_NOP       ; no -> NOP return
    LDY     #$07                ; 8 bytes
.restore_loop:
    LDA     HRCG_SAVED_PARAMS,Y ; saved params
    STA     HRCG_VIDEO_MODE,Y   ; restore to active
    DEY
    BPL     .restore_loop
    STOB    #$00,HRCG_SAVED_PARAMS+8 ; clear saved flag
    STA     HRCG_SAVED_CH       ; clear saved CH
    MOVB    $22,HRCG_SAVED_CV ; WNDTOP
    RTS
; --- HRCG_DISPLAY_PAGE: page flip based on video mode ---
HRCG_DISPLAY_PAGE:
    BIT     $C054               ; select page 1
    LDA     HRCG_VIDEO_MODE     ; video mode
    BANE    #$40,HRCG_CTRL_NOP  ; no -> return (stay on page 1)
    BIT     $C055               ; yes -> select page 2
    RTS
```

Defines:
HRCG_CTRL_D, used in chunk 741.
HRCG_DISPLAY_PAGE, used in chunk 736.

Uses HRCG_CTRL_NOP 751, HRCG_SAVED_CH 746, HRCG_SAVED_CV 746, HRCG_SAVED_PARAMS 746, HRCG_VIDEO_MODE 737b, MOVB 8c, and STOB 8c.

The next group of handlers covers screen clearing, cursor movement, and inverse video. Ctrl-E clears from the cursor to the end of the current line. Ctrl-F clears from the cursor to the end of the screen. Both delegate to the HRCG's own clear routines and then call the corresponding ROM routine to update the text-page shadow.

Ctrl-H is backspace: it decrements the cursor column, and if it wraps past column 0, it moves to the end of the previous row (provided the cursor is not already at the top of the text window). Ctrl-I enables inverse video by setting `HRCG_MASK` to `$7F`, which causes every font byte to be XORed with `$7F` before rendering—flipping all 7 pixel bits.

All shifted variants in this group are no-ops.

750 ⟨*hrcg mask* 750⟩≡ (909)

```
    ORG     $9308
HRCG_MASK:
    DC.B    $00                             ; $00=normal, $7F=inverse
```

Defines:
`HRCG_MASK`, used in chunks 751, 754, 761, and 766.

751 ⟨*hrcg ctrl efhi* 751⟩≡ (909)

```
    ORG     $94C1
HRCG_CTRL_E:
; --- ctrl-E normal: clear to end of line ---
    BCS     HRCG_CTRL_NOP ; shifted -> NOP
    JSR     HRCG_CLREOL
    JMP     $FC9C         ; ROM CLREOLZ
; --- ctrl-F normal: clear to end of screen ---
HRCG_CTRL_F:
    BCS     HRCG_CTRL_NOP ; shifted -> NOP
    JSR     HRCG_CLRLINE
    JMP     $FC42         ; ROM CLREOP
; --- ctrl-H: backspace ---
HRCG_CTRL_H:
    SUBROUTINE HRCG_CTRL_H
    DEC     $24           ; CH--
    BPL     HRCG_CTRL_NOP ; still >= 0 -> done
    MOVB    $21,$24 ; WNDWDTH
    DEC     $24           ; CH = WNDWDTH - 1
    LDA     $22           ; WNDTOP
    BAGE    $25,HRCG_CTRL_NOP ; CV >= WNDTOP?
    DEC     $25           ; CV--
    JMP     $FC22         ; ROM VTAB
; --- ctrl-I: inverse video ---
HRCG_CTRL_I:
    BCS     HRCG_CTRL_NOP ; shifted -> NOP
    STOB    #$7F,HRCG_MASK ; mask = $7F (invert all 7 pixels)
HRCG_CTRL_NOP:
    RTS                   ; shared NOP return
```

Defines:
HRCG_CTRL_E, used in chunk 741.
HRCG_CTRL_F, used in chunk 741.
HRCG_CTRL_H, used in chunk 741.
HRCG_CTRL_I, used in chunk 741.
HRCG_CTRL_NOP, used in chunks 741, 748, 749, 753, and 754.
Uses HRCG_CLREOL 766, HRCG_CLRLINE 766, HRCG_MASK 750, MOVB 8c, and STOB 8c.

Ctrl-K and ctrl-L control the "lowercase invert" feature. HRCG_INVERT_FLAG has three states: $00 (normal), $80 (invert lowercase letters), and $C0 (XOR-invert, toggled by ctrl-S). When set to $80, uppercase letters A–Z are rendered using the font data for their lowercase counterparts without subtracting $20—effectively swapping case in the display. Ctrl-K clears the flag; ctrl-L sets it to $80.

Ctrl-M is carriage return. Before moving the cursor, it checks the keyboard for ctrl-S (the pause key, $93). If a pause is detected, the routine waits for another keypress before continuing. Ctrl-C ($83) cancels the pause. After the pause check, the handler sets the cursor column to zero and falls through to the line feed logic.

752 ⟨*hrcg invert flag* 752⟩≡ (909)

```
    ORG     $930C
HRCG_INVERT_FLAG:
    DC.B    $00                          ; $00=normal, $80=lowercase, $C0=XOR-invert
```

Defines:
HRCG_INVERT_FLAG, used in chunks 753, 756, and 761.

753 ⟨*hrcg ctrl klm* 753⟩≡ (909)

```
    ORG     $94EE
HRCG_CTRL_K:
; --- ctrl-K: clear invert flag ---
    BCS     HRCG_CTRL_NOP    ; shifted -> NOP
    STOB    #$00,HRCG_INVERT_FLAG ; clear invert flag
    RTS
; --- ctrl-L: set invert flag ---
HRCG_CTRL_L:
    BCS     HRCG_CTRL_NOP    ; shifted -> NOP
    STOB    #$80,HRCG_INVERT_FLAG ; invert flag = $80
    RTS
; --- ctrl-M: carriage return (with pause check) ---
HRCG_CTRL_M:
    SUBROUTINE HRCG_CTRL_M
    LDY     $C000            ; check keyboard
    BPL     .no_pause        ; no key -> skip pause
    BYNE    #$93,.no_pause   ; ctrl-S (pause)?
    BIT     $C010            ; clear strobe
.wait_key:
    LDY     $C000            ; wait for key
    BPL     .wait_key
    BYEQ    #$83,.no_pause   ; ctrl-C (cancel)?
    BIT     $C010            ; clear strobe
.no_pause:
    JMP     HRCG_CR_ENTRY    ; -> set CH=0, line feed
```

Defines:
HRCG_CTRL_K, used in chunk 741.
HRCG_CTRL_L, used in chunk 741.
HRCG_CTRL_M, used in chunk 741.
Uses BYNE 18, HRCG_CR_ENTRY 763, HRCG_CTRL_NOP 751, HRCG_INVERT_FLAG 752, and STOB 8c.

Ctrl-N returns to normal video by clearing the inverse mask. Ctrl-O in normal mode sets HRCG_MODE_FLAGS to $40, which causes the *next* control character to enter its shifted variant (the "next-shifted" flag); in shifted mode, it sets OR draw mode. Ctrl-P homes the cursor and clears the screen; its shifted variant clears the draw mode and page mask.

Ctrl-Q moves the cursor to the saved position (from ctrl-B) without restoring any other state—a "goto" for the cursor.

754 ⟨*hrcg ctrl nopq* 754⟩≡ (909)

```
    ORG     $9519
HRCG_CTRL_N:
; --- ctrl-N: normal video ---
    BCS     HRCG_CTRL_NOP   ; shifted -> NOP
    STOB    #$00,HRCG_MASK ; clear mask (normal video)
    RTS
; --- ctrl-O: set next-shifted / OR mode ---
HRCG_CTRL_O:
    SUBROUTINE HRCG_CTRL_O
    BCS     .set_or_draw    ; shifted -> OR draw mode
    STOB    #$40,HRCG_MODE_FLAGS ; next ctrl -> shifted variant
    RTS
.set_or_draw:
    STOB    #$80,HRCG_DRAW_MODE ; draw mode = OR ($80)
    STOB    #$00,HRCG_PAGE_MASK ; page mask = 0
    RTS
; --- ctrl-P: home cursor ---
HRCG_CTRL_P:
    SUBROUTINE HRCG_CTRL_P
    BCS     .clear_draw     ; shifted -> clear draw mode
    JSR     HRCG_HOME
    JMP     $FC58           ; ROM HOME (clear text screen)
.clear_draw:
    STOB    #$00,HRCG_DRAW_MODE ; draw mode = 0
    STA     HRCG_PAGE_MASK  ; page mask = 0
    RTS
; --- ctrl-Q: goto saved cursor ---
HRCG_CTRL_Q:
    BCS     HRCG_CTRL_NOP   ; shifted -> NOP
    MOVB    HRCG_SAVED_CH,$24 ; saved CH
    MOVB    HRCG_SAVED_CV,$25 ; saved CV
    JMP     $FC24           ; ROM VTAB
```

Defines:
HRCG_CTRL_N, used in chunk 741.
HRCG_CTRL_O, used in chunk 741.
HRCG_CTRL_P, used in chunk 741.
HRCG_CTRL_Q, used in chunk 741.

Uses HRCG_CTRL_NOP 751, HRCG_DRAW_MODE 747b, HRCG_HOME 766, HRCG_MASK 750, HRCG_MODE_FLAGS 742, HRCG_PAGE_MASK 747b, HRCG_SAVED_CH 746, HRCG_SAVED_CV 746, MOVB 8c, and STOB 8c.

Ctrl-R, ctrl-S, and ctrl-T configure the advanced draw modes. These modes control how new character pixels combine with existing screen content:

- **Replace** (the default): new pixels overwrite the screen.
- **OR**: new pixels are ORed with existing screen content, allowing characters to overlay graphics without erasing the background.
- **AND**: new pixels are ANDed with the screen, masking out areas.
- **XOR**: new pixels are XORed with the screen, useful for flashing cursors or toggling sprites.

Ctrl-R (shifted only) sets XOR draw mode with page-2 overlay. Ctrl-S sets the XOR-invert flag (for character-level inversion); its shifted variant clears the wrap flag, switching to scroll mode. Ctrl-T (shifted only) sets AND draw mode with page-2 overlay. The normal variants of ctrl-R and ctrl-T are no-ops.

755 ⟨*hrcg wrap flag* 755⟩≡ (909)

```
    ORG     $930B
HRCG_WRAP_FLAG:
    DC.B    $00                         ; $00=scroll, $FF=wrap
```

Defines:
HRCG_WRAP_FLAG, used in chunks 756, 757, and 763.

756 ⟨*hrcg ctrl rst* 756⟩≡ (909)

```
      ORG     $9554
  HRCG_CTRL_R:
  ; --- ctrl-R shifted: XOR draw + page2 ---
      BCC     HRCG_RTS         ; normal -> RTS (no-op)
      STOB    #$C0,HRCG_DRAW_MODE ; draw mode = XOR ($C0)
      STOB    #$60,HRCG_PAGE_MASK ; page mask = $60 (page 2)
      RTS
  ; --- ctrl-S: XOR invert / clear wrap ---
  HRCG_CTRL_S:
      SUBROUTINE HRCG_CTRL_S
      BCS     .clear_wrap      ; shifted -> clear wrap
      STOB    #$C0,HRCG_INVERT_FLAG ; invert = XOR ($C0)
      RTS
  .clear_wrap:
      STOB    #$00,HRCG_WRAP_FLAG ; wrap = 0 (scroll mode)
      RTS
  ; --- ctrl-T shifted: AND draw mode ---
  HRCG_CTRL_T:
      BCC     HRCG_RTS         ; normal -> RTS (no-op)
      STOB    #$80,HRCG_DRAW_MODE ; draw mode = AND ($80)
      STOB    #$60,HRCG_PAGE_MASK ; page mask = $60
      RTS
```

Defines:
HRCG_CTRL_R, used in chunk 741.
HRCG_CTRL_S, used in chunk 741.
HRCG_CTRL_T, used in chunk 741.

Uses HRCG_DRAW_MODE 747b, HRCG_INVERT_FLAG 752, HRCG_PAGE_MASK 747b, HRCG_RTS 757, HRCG_WRAP_FLAG 755, and STOB 8c.

Ctrl-V and ctrl-W together define the text window—a rectangular region of the screen where text output is confined. Ctrl-V sets the left edge and top row: the current cursor column (relative to WNDLFT) becomes the new left margin, and the current row becomes the new top. Ctrl-W sets the right edge and bottom row, and its shifted variant enables wrap mode (where the cursor wraps to the top of the window instead of scrolling).

Ali Baba uses these extensively to confine status messages and input prompts to the bottom portion of the screen, while the map occupies the upper area.

757 ⟨*hrcg ctrl vw* 757⟩≡ (909)

```
    ORG     $957C
HRCG_CTRL_V:
    SUBROUTINE HRCG_CTRL_V
; --- ctrl-V: set text window ---
    BCS     HRCG_RTS            ; shifted -> RTS
    LDA     $24                 ; CH
    ADC     $20                 ; + WNDLFT
    BALT    #$28,.store_lft ; >= 40?
    LDA     #$27                ; clamp to 39
.store_lft:
    STA     $20                 ; WNDLFT = CH + old WNDLFT
    SEC
    LDA     $21                 ; WNDWDTH
    SBC     $24                 ; - CH
    STA     $21                 ; new WNDWDTH
    STOB    #$00,$24 ; CH = 0
    LDA     $25                 ; CV
    BALT    #$18,.store_top ; >= 24?
    LDA     #$17                ; clamp to 23
.store_top:
    STA     $22                 ; WNDTOP = CV
    BIT     HRCG_SAVED_PARAMS+8 ; params saved?
    BMI     .do_vtab            ; yes -> skip saving CV
    STA     HRCG_SAVED_CV       ; saved CV = WNDTOP
.do_vtab:
    JMP     $FC22               ; ROM VTAB
; --- ctrl-W: set right margin + bottom ---
HRCG_CTRL_W:
    SUBROUTINE HRCG_CTRL_W
    BCS     .set_wrap           ; shifted -> set wrap
    MOVB    $24,$21 ; CH
    ADC     $20                 ; + WNDLFT
    BALT    #$28,.width_ok ; >= 40?
    LDA     #$27                ; clamp to 39
    SBC     $20                 ; - WNDLFT
    STA     $21                 ; WNDWDTH = 39 - WNDLFT
.width_ok:
    INC     $21                 ; WNDWDTH + 1
    LDA     $25                 ; CV
```

```
    BALT    #$18,.bottom_ok ; >= 24?
    LDA     #$17                ; clamp to 23
.bottom_ok:
    STA     $23                 ; WNDBTM = CV
    INC     $23                 ; WNDBTM + 1
HRCG_RTS:
    RTS                         ; shared RTS
; --- ctrl-W shifted: set wrap mode ---
.set_wrap:
    STOB    #$FF,HRCG_WRAP_FLAG ; wrap flag = $FF (wrap mode)
    RTS
```

Defines:
HRCG_CTRL_V, used in chunk 741.
HRCG_CTRL_W, used in chunk 741.
HRCG_RTS, used in chunks 736, 756, 759, and 761.
Uses HRCG_SAVED_CV 746, HRCG_SAVED_PARAMS 746, HRCG_WRAP_FLAG 755, MOVB 8c, and STOB 8c.

Ctrl-Y clears the entire screen by resetting the text window to full size (40 columns by 24 rows) and filling it. Its shifted variant jumps to HRCG_ENTRY2, one of the secondary entry points in the jump table—which the game patches to a simple RTS.

Ctrl-Z is the full reset:

1. Clears the entire parameter block (19 bytes).
2. Resets the text window.
3. Selects the standard font.
4. Sets replace video mode.
5. Patches both secondary entry points to RTS.

This brings the HRCG back to its power-on state. The shifted variant jumps to HRCG_ENTRY3 (also patched to RTS).

759 ⟨*hrcg ctrl yz* 759⟩≡ (909)

```
    ORG     $95D1
HRCG_CTRL_Y:
    SUBROUTINE HRCG_CTRL_Y
; --- ctrl-Y normal: clear screen (reset window to full) ---
    BCS     .jump_entry2        ; shifted -> JMP HRCG_ENTRY2
HRCG_RESET_WINDOW:
; --- shared: reset window to full screen ---
    LDA     $20                 ; WNDLFT
    ADC     $24                 ; + CH
    STA     $24                 ; CH = absolute column
    STOB    #$00,$20 ; WNDLFT = 0
    STA     $22                 ; WNDTOP = 0
    BIT     HRCG_SAVED_PARAMS+8 ; params saved?
    BMI     .set_dims           ; yes -> skip
    STA     HRCG_SAVED_CH       ; saved CH = 0
    STA     HRCG_SAVED_CV       ; saved CV = 0
.set_dims:
    STOB    #$28,$21 ; 40
    STOB    #$18,$23 ; 24
    JMP     $FC22               ; ROM VTAB
.jump_entry2:
    JMP     HRCG_ENTRY2         ; -> secondary entry (disabled)
; --- ctrl-Z: full reset ---
HRCG_CTRL_Z:
    SUBROUTINE HRCG_CTRL_Z
    BCS     .jump_entry3        ; shifted -> JMP HRCG_ENTRY3
HRCG_RESET:
    LDA     #$00
    LDY     #$12                ; 19 bytes (indices 0-18)
```

```
.clear_loop:
    STA     HRCG_VIDEO_MODE,Y   ; clear param block
    DEY
    BPL     .clear_loop
    JSR     HRCG_RESET_WINDOW   ; reset window
    JSR     HRCG_SET_STD_FONT   ; set standard font ($97A5)
    JSR     HRCG_SET_REPLACE    ; set replace video mode ($20)
    STOB    #<HRCG_RTS,HRCG_ENTRY2+1 ; patch entry2 low
    STA     HRCG_ENTRY3+1       ; patch entry3 low
    STOB    #>HRCG_RTS,HRCG_ENTRY2+2 ; patch entry2 high
    STA     HRCG_ENTRY3+2       ; patch entry3 high
    RTS
.jump_entry3:
    JMP     HRCG_ENTRY3         ; -> secondary entry (disabled)
```

Defines:
HRCG_CTRL_Y, used in chunk 741.
HRCG_CTRL_Z, used in chunk 741.
HRCG_RESET, used in chunk 736.
HRCG_RESET_WINDOW, never used.

Uses HRCG_ENTRY2 736, HRCG_ENTRY3 736, HRCG_RTS 757, HRCG_SAVED_CH 746, HRCG_SAVED_CV 746, HRCG_SAVED_PARAMS 746, HRCG_SET_REPLACE 747a, HRCG_SET_STD_FONT 745, HRCG_VIDEO_MODE 737b, and STOB 8c.

Character rendering

HRCG_DRAW_CHAR is the heart of the HRCG: the routine that actually puts pixels on the hi-res screen. For each printable character, it computes a pointer into the active font data, calculates the hi-res screen address for the current cursor position, and renders 8 scan lines of font data.

The font address calculation multiplies the character index (character code minus $20) by 8 and adds the font pointer. The multiplication is done by three successive ASL/ROL pairs, shifting the index left 3 bits while propagating the carry into $EF for the high byte.

The lowercase-invert feature adds a twist: when HRCG_INVERT_FLAG is set to $80, letters A–Z skip the subtraction of $20, so they index into the lowercase range of the font instead. The $C0 (XOR-invert) state toggles only once and then reverts to $80.

Once the font address is in $EE/$EF and the screen address is in $2A/$2B, the render loop processes 8 bytes. Each font byte is XORed with HRCG_MASK (for inverse video), then combined with the screen according to the draw mode: replace writes directly, OR combines with the existing screen content, AND masks it, and XOR toggles it. The screen address advances by TEXT_PAGE1 per scan line—the standard Apple II hi-res interleaving within a character cell.

761 ⟨*hrcg draw char* 761⟩≡ (909)

```
    ORG     $9621
HRCG_DRAW_CHAR:
    SUBROUTINE HRCG_DRAW_CHAR
; --- Main character rendering ---
    ASL     HRCG_MODE_FLAGS     ; shift mode flags
    BCS     HRCG_RTS            ; shifted -> RTS (ignore shifted printable)
    BIT     HRCG_INVERT_FLAG    ; check invert flag
    BPL     .normal_char        ; not set -> normal
    BVC     .check_lower        ; bit 6 clear -> check lowercase
    ASL     HRCG_INVERT_FLAG    ; shift invert flag ($C0 -> $80)
    BCS     .normal_char        ; was $C0, now $80 -> treat as normal
.check_lower:
    BALT    #$41,.normal_char ; 'A' ?
    BALT    #$5B,.calc_offset ; '[' ?
.normal_char:
    SEC
    SBC     #$20                ; convert to font index (char - $20)
.calc_offset:
    ASL                         ; \ index * 8 = byte offset in font
    ROL     $EF                 ;  |
    ASL                         ;  |
    ROL     $EF                 ;  |
    ASL                         ;  |
    ROL     $EF                 ; /
```

```
    CLC
    ADC     HRCG_FONT_PTR_LO    ; + font pointer low
    STA     $EE                 ; font address low
    LDA     $EF
    AND     #$07                ; mask to 3 bits
    ADC     HRCG_FONT_PTR_HI    ; + font pointer high
    STA     $EF                 ; font address high
    JSR     HRCG_CALC_HIRES_ADDR
    ADC     $24                 ; + cursor X
    STA     $2A                 ; screen address low
    STA     $EC                 ; save copy
    LDA     $2B                 ; screen address high
    EOR     HRCG_PAGE_MASK      ; XOR with page mask (0 or $60)
    STA     $ED                 ; alt-page address high
    LDY     $24                 ; cursor X
    LDA     $FF                 ; original character
    STA     ($28),Y             ; store char in text page
    LDX     #$00
    LDY     #$00
; --- Render loop: 8 scan lines ---
.render_loop:
    LDA     ($EE),Y             ; load font byte
    EOR     HRCG_MASK           ; apply mask (inverse)
    BIT     HRCG_DRAW_MODE      ; check draw mode
    BPL     .store              ; $00 -> replace mode
    BVS     .xor_mode           ; $C0 -> XOR mode
    BIT     HRCG_MASK           ; check mask for AND vs OR
    BVS     .and_mode           ; mask $7F + draw $80 -> AND
    ORA     ($EC,X)             ; OR mode: combine with screen
    BVC     .store              ; always -> store
.and_mode:
    AND     ($EC,X)             ; AND mode: mask with screen
    BVS     .store              ; always -> store
.xor_mode:
    EOR     ($EC,X)             ; XOR mode: toggle screen bits
.store:
    STA     ($2A,X)             ; write to screen
    INY                         ; next font byte
    CPY     #$08                ; done 8 bytes?
    BCS     HRCG_ADVANCE_CURSOR ; yes -> advance cursor
    LDA     $2B                 ; screen high byte
    ADC     #$04                ; next scan line group (+$0400)
    STA     $2B
    EOR     HRCG_PAGE_MASK      ; update alt-page address
    STA     $ED
    BCC     .render_loop        ; loop (always, carry clear)
```

Defines:
HRCG_DRAW_CHAR, used in chunk 743.
Uses HRCG_ADVANCE_CURSOR 763, HRCG_CALC_HIRES_ADDR 768, HRCG_DRAW_MODE 747b, HRCG_FONT_PTR_HI 744, HRCG_FONT_PTR_LO 744, HRCG_INVERT_FLAG 752, HRCG_MASK 750,

HRCG_MODE_FLAGS 742, HRCG_PAGE_MASK 747b, and HRCG_RTS 757.

After rendering a character, HRCG_ADVANCE_CURSOR moves the cursor one column to the right. If the cursor reaches the right edge of the text window (WNDWDTH), it wraps to column 0 and performs a line feed.

The line feed logic increments CV (cursor row). If the cursor passes the bottom of the text window (WNDBTM), the behavior depends on the wrap flag: in scroll mode, the window scrolls up one line (via HRCG_SCROLL_UP) and the cursor stays at the bottom row; in wrap mode, the cursor jumps back to the top of the window.

This chunk also contains HRCG_CR_ENTRY, which the carriage return handler (ctrl-M) uses to set the column to zero and fall through into the line feed.

763 ⟨*hrcg advance cursor* 763⟩≡ (909)

```
    ORG     $969E
HRCG_ADVANCE_CURSOR:
    SUBROUTINE HRCG_ADVANCE_CURSOR
; --- Advance cursor after character ---
    INC     $24             ; CH++
    LDA     $24
    BALT    $21,.done ; >= WNDWDTH?
HRCG_CR_ENTRY:
    STOB    #$00,$24 ; (CR entry point: set CH=0)
; --- HRCG_LINE_FEED ---
HRCG_LINE_FEED:
    INC     $25             ; CV++
    LDA     $25
    BALT    $23,.update_cursor ; >= WNDBTM?
    BIT     HRCG_WRAP_FLAG ; check wrap flag
    BVS     .wrap           ; wrap mode -> wrap to top
    DEC     $25             ; CV-- (stay at bottom)
    JSR     HRCG_SCROLL_UP
    JMP     $FC70           ; ROM SCROLL
.wrap:
    MOVB    $22,$25 ; WNDTOP
.update_cursor:
    JSR     $FC24           ; ROM VTAB
.done:
    RTS
```

Defines:
HRCG_ADVANCE_CURSOR, used in chunk 761.
HRCG_CR_ENTRY, used in chunk 753.
HRCG_LINE_FEED, used in chunks 741 and 748.
Uses HRCG_SCROLL_UP 764, HRCG_WRAP_FLAG 755, MOVB 8c, and STOB 8c.

HRCG_SCROLL_UP shifts the entire text window up by one character row. Since each text row occupies 8 hi-res scan lines, the routine must copy 8 rows of pixel data for every text row in the window. It walks from the top of the window downward, using HRCG_CALC_HIRES_ROW to compute the hi-res address for both the source (row below) and destination (current row), then copies all columns across all 8 scan lines.

After all rows have been shifted up, the bottom row is cleared using HRCG_FILL_LINE with the draw mode temporarily forced to replace. This ensures the last row is erased cleanly regardless of the current video mode.

764 ⟨*hrcg scroll up* 764⟩≡ (909)

```
    ORG     $96C7
HRCG_SCROLL_UP:
    SUBROUTINE HRCG_SCROLL_UP
    LDA     $22                 ; WNDTOP (top row)
    PHA                         ; save for later
.next_row:
    JSR     HRCG_CALC_HIRES_ROW ; calc hi-res addr for row A
.copy_row:
    MOVB    $2A,$EC ; dest address low
    LDA     $2B                 ; dest address high
    AND     #$E3                ; mask off scan line bits
    STA     $ED
    PLA                         ; restore source row
    CLC
    ADC     #$01                ; source = dest + 1
    BAGE    $23,.clear_last ; >= WNDBTM?
    PHA
    JSR     HRCG_CALC_HIRES_ROW ; calc hi-res addr for source row
    LDX     #$07                ; 8 scan lines
.copy_scanline:
    LDY     $21                 ; WNDWDTH
    DEY                         ; index = WNDWDTH - 1
.copy_byte:
    LDA     ($2A),Y             ; read source
    STA     ($EC),Y             ; write dest
    DEY
    BPL     .copy_byte          ; next column
    DEX
    BMI     .copy_row           ; all 8 lines done -> next row
    CLC
    LDA     $2B                 ; advance source
    ADC     #$04
    STA     $2B
    LDA     $ED                 ; advance dest
    ADC     #$04
    STA     $ED
    BNE     .copy_scanline      ; always -> continue
.clear_last:
```

```
LDA     HRCG_DRAW_MODE      ; save draw mode
PHA
LDA     $23                 ; WNDBTM
SBC     #$01                ; last row index
LDY     #$00
STY     HRCG_DRAW_MODE      ; draw mode = 0 (replace)
JSR     HRCG_FILL_LINE      ; clear last row
PULB    HRCG_DRAW_MODE ; restore draw mode
RTS
```

Defines:
HRCG_SCROLL_UP, used in chunks 738 and 763.
Uses HRCG_CALC_HIRES_ROW 768, HRCG_DRAW_MODE 747b, HRCG_FILL_LINE 766, MOVB 8c, and PULB 9c.

This chunk contains the screen-clearing family of routines, all built on top of HRCG_FILL_LINE. HRCG_HOME sets the cursor to the top-left corner of the text window and clears the entire window. HRCG_CLRLINE clears from the current row to the bottom of the window. HRCG_CLREOL clears from the cursor position to the end of the current line.

HRCG_FILL_LINE is the shared workhorse: it fills a single text row (8 scan lines) starting at a given column. The fill value is HRCG_MASK—normally $00 (black), but $7F when inverse video is active, producing a white bar. In OR or XOR draw modes, the fill operation reads the existing screen content and combines it with the mask, allowing selective clearing without disturbing the other hi-res page.

766 ⟨*hrcg home clr* 766⟩≡ (909)

```
    ORG     $9716
HRCG_HOME:
    SUBROUTINE HRCG_HOME
; --- Set cursor to top-left and clear screen ---
    MOVB    $22,$25 ; WNDTOP
    LDY     #$00
    STY     $24                 ; CH = 0
    BEQ     .clear_entry        ; always -> fall into clear loop
; --- HRCG_CLRLINE: clear from current row to bottom ---
HRCG_CLRLINE:
    LDY     $24                 ; preserve CH for fill start column
    LDA     $25                 ; CV = starting row
.clear_entry:
    PHA                         ; save row
    JSR     HRCG_FILL_LINE      ; clear this row
    PLA
    CLC
    ADC     #$01                ; next row
    CMP     $23                 ; >= WNDBTM?
    LDY     #$00                ; fill from column 0 for next rows
    BCC     .clear_entry        ; continue loop
    RTS
; --- HRCG_CLREOL: clear from cursor to end of line ---
HRCG_CLREOL:
    LDY     $24                 ; start column = CH
    LDA     $25                 ; current row = CV
; --- HRCG_FILL_LINE: fill one row ---
HRCG_FILL_LINE:
    STY     $EE                 ; save start column
    JSR     HRCG_CALC_HIRES_ROW ; calc hi-res addr for row A
    LDX     #$07                ; 8 scan lines
.fill_byte:
    LDA     HRCG_MASK           ; fill byte (mask value)
    BIT     HRCG_DRAW_MODE      ; check draw mode
    BPL     .write              ; replace mode -> write directly
    MOVB    $2A,$EC ; compute alt-page address
```

```
    LDA     $2B
    EOR     HRCG_PAGE_MASK
    STA     $ED
    LDA     ($EC),Y             ; read existing screen byte
    BIT     HRCG_DRAW_MODE      ; check draw mode again
    BVC     .write              ; OR mode -> skip XOR
    EOR     HRCG_MASK           ; XOR with mask
.write:
    STA     ($2A),Y             ; write to screen
    INY                         ; next column
    CPY     $21                 ; >= WNDWDTH?
    BCC     .fill_byte          ; no -> continue
    DEX                         ; next scan line
    BMI     .fill_done          ; all 8 done -> return
    LDY     $EE                 ; reset column to start
    LDA     $2B                 ; advance to next scan line
    CLC
    ADC     #$04
    STA     $2B
    BNE     .fill_byte          ; always -> continue
.fill_done:
    RTS
```

Defines:
HRCG_CLREOL, used in chunks 738 and 751.
HRCG_CLRLINE, used in chunks 738 and 751.
HRCG_FILL_LINE, used in chunk 764.
HRCG_HOME, used in chunks 738 and 754.

Uses HRCG_CALC_HIRES_ROW 768, HRCG_DRAW_MODE 747b, HRCG_MASK 750, HRCG_PAGE_MASK 747b, and MOVB 8c.

Every character rendering and screen clearing operation needs to convert a text row number into a hi-res screen address. HRCG_CALC_HIRES_ADDR performs this conversion. The Apple II hi-res screen has a notoriously non-linear address layout—consecutive text rows do not map to consecutive memory addresses. The HRCG solves this with a 12-entry lookup table (HRCG_ROW_TABLE) indexed by row/2, combined with bit manipulation to encode the row's position within the hi-res page.

The computation works as follows: the row is divided by 2 (the carry from the LSR is preserved for later), the quotient mod 4 selects which TEXT_PAGE1 group the row falls in, and the table provides the base offset ($00, $50, or $A0). The saved carry bit (odd vs. even row) is rotated back in via ROR to set bit 7 of the low byte, and WNDLFT is added to account for the left margin.

The 20 trailing bytes after the table are of unknown purpose—they may be residual data from the HRCG development environment or unused padding.

768 ⟨*hrcg calc addr* 768⟩≡ (909)

```
    ORG     $9771
HRCG_CALC_HIRES_ADDR:
; --- Compute hi-res screen address from CV ---
; Input: ZP $25 (CV)   Output: $2A/$2B
    LDA     $25              ; CV (cursor Y)
HRCG_CALC_HIRES_ROW:
; --- Alternate entry: row in A ---
    LSR                      ; A = row / 2
    TAX                      ; X = row / 2 (table index)
    AND     #$03             ; A = (row/2) % 4
    ORA     HRCG_VIDEO_MODE  ; combine with page base ($20/$40)
    STA     $2B              ; screen high byte
    LDA     HRCG_ROW_TABLE,X ; row base from table
    ROR                      ; rotate carry from LSR into bit 7
    ADC     $20              ; + WNDLFT
    STA     $2A              ; screen low byte
    RTS
; --- Row address table (12 entries) ---
HRCG_ROW_TABLE:
    HEX     00 00 00 00 50 50 50 50 A0 A0 A0 A0
; --- Trailing data (20 bytes, purpose unknown) ---
HRCG_TRAILING_DATA:
    HEX     B9 A2 A0 D2 E9 A0 F2 A0 A5 B9 C1 80 C4 CD A0 8C
    HEX     A0 AC A0 A0
```

Defines:
HRCG_CALC_HIRES_ADDR, used in chunk 761.
HRCG_CALC_HIRES_ROW, used in chunks 764 and 766.
HRCG_ROW_TABLE, never used.
HRCG_TRAILING_DATA, never used.
Uses HRCG_VIDEO_MODE 737b.

Chapter 22

Disk I/O

The disk I/O routines occupy `$B300--$BFFF` at runtime. They are not loaded directly to their final addresses by the EALDR. Instead, the EALDR loads them into the hi-res page staging area `$3300--$3FFF` (tracks `$0F` and `$10`). During `GAME_INIT`, `RELOCATE_HIRES_STAGING` copies `$3300--$3FFF` to `$B300--$BFFF`.

These routines handle all disk access for the main game, including scene loading and save/restore operations.

The area splits into two distinct regions:

- `$B300--$B5FF`: Custom game disk code—a VM-driven sector reader with its own data field verifier, context-swap mechanism, and embedded byte-code program.
- `$B600--$BFFF`: Standard Apple II DOS 3.3 RWTS (Read/Write Track/Sector), byte-identical to the system master after relocation, with only three patched bytes:
  - `$B748--$B749`: JMP target changed from `$9B03` (DOS file manager) to `$9D84` (game's handler).
  - `$B7E1`: max-drives configuration byte changed from `$02` to `$00`.

  The standard RWTS is fully documented in *Beneath Apple DOS* (Worth & Lechner, 1984) and will not be re-disassembled here.

The main program calls two entry points in this area:

Address	Label	Called from
DISK_READ_FIELD	DISK_READ_FIELD	LOAD_SCENE_DATA ($6DD8)
RWTS_ENTRY	RWTS_ENTRY	RWTS_DISPATCH

The RWTS (Read/Write Track/Sector) at $B600--$BFFF is a copy of the standard Apple II DOS 3.3 RWTS from the system master, with the standard $80 page relocation applied (disk pages $36--$3F relocated to $B6--$BF). Byte-for-byte comparison with the DOS 3.3 System Master (680-0051-00) confirms that all code is identical after relocation, except for the three patched bytes noted above. The remaining differences are runtime state in the IOB parameter block at RWTS_IOB and the contents of the nibble buffers at $BB00--$BC55.

The RWTS is fully documented in *Beneath Apple DOS* (Worth & Lechner, 1984), chapter 4, and is not disassembled here. The binary data is included verbatim from dos.asm. The only call into the DOS RWTS ($B600–$BFFF) from outside that region is JSR RWTS_ENTRY at $6C26 in DISK_READ_FIELD.

RWTS_CORE is the main dispatcher inside the standard DOS 3.3 RWTS. RWTS_ENTRY calls it after saving the processor status and disabling interrupts.

The chunk below includes the standard RWTS binary verbatim, accessed through RWTS_ENTRY and RWTS_CORE.

770 ⟨*dos33 rwts* 770⟩≡ (909)

```
        ORG     $B600
    DOS33_RWTS:
        INCLUDE "dos.asm"
```

Defines:
DOS33_RWTS, never used.
RWTS_CORE, never used.
RWTS_ENTRY, used in chunk 546.
RWTS_IOB, used in chunk 589b.

DISK_READ_FIELD is the entry point called by LOAD_SCENE_DATA to read sector data from disk. It is a trampoline into the disk VM interpreter loop at DISK_VM_LOOP, which uses an embedded bytecode program to control the multi-sector read process. The carry flag is clear on entry (set by the caller) to indicate a read operation.

771 ⟨*disk read field* 771⟩≡ (909)

```
    ORG     $B300
DISK_IO_START:
DISK_READ_FIELD:
    JMP     DISK_VM_LOOP ; enter VM interpreter
    NOP                  ; pad
```

Defines:
DISK_IO_START, used in chunk 671.
DISK_READ_FIELD, used in chunk 529.
Uses DISK_VM_LOOP 774.

Searches for the standard data field prologue (D5 AA AD) on the disk, then reads all 342 nibbles (86 secondary + 256 primary) and verifies each byte equals $AF. Finally checks the data epilogue (DE AA). Returns carry clear on success, carry set on error.

VERIFY_DATA_FIELD is called internally by the disk VM (via JSR VERIFY_DATA_FIELD).

DISK_DATA_REG is the Disk II controller's data register for the active slot. VERIFY_DATA_FIELD reads nibble bytes from this register in a tight loop while searching for the D5 AA AD prologue and verifying the data field contents.

772 ⟨*defines* 25b⟩+≡ (909) ◁705

```
IWM_MOTORON         EQU     $C0E9   ; drive motor on
IWM_PH0_OFF         EQU     $C0E0   ; stepper phase 0 off
IWM_PH0_ON          EQU     $C0E1   ; stepper phase 0 on
IWM_PH1_OFF         EQU     $C0E2   ; stepper phase 1 off
IWM_PH2_OFF         EQU     $C0E4   ; stepper phase 2 off
IWM_PH2_ON          EQU     $C0E5   ; stepper phase 2 on
IWM_PH3_OFF         EQU     $C0E6   ; stepper phase 3 off
IWM_PH3_ON          EQU     $C0E7   ; stepper phase 3 on (seek trigger)
IWM_MOTOROFF        EQU     $C0E8   ; drive motor off
IWM_READMODE        EQU     $C0EE   ; set drive to read mode
DISK_DATA_REG       EQU     $C0EC   ; disk controller data register (active slot)
TEXT_PAGE1          EQU     $0400   ; Apple II text screen page 1
```

Defines:
DISK_DATA_REG, used in chunks 692, 696, 773, and 777.
IWM_MOTOROFF, used in chunks 696 and 775.
IWM_MOTORON, used in chunks 696, 775, 800a, and 801.
IWM_PH0_OFF, used in chunks 696 and 779b.
IWM_PH0_ON, used in chunks 696 and 775.
IWM_PH1_OFF, used in chunks 696 and 779b.
IWM_PH2_OFF, used in chunks 696 and 779b.
IWM_PH2_ON, used in chunks 696 and 775.
IWM_PH3_OFF, used in chunks 696 and 779b.
IWM_PH3_ON, used in chunks 696 and 775.
IWM_READMODE, used in chunks 696 and 775.
TEXT_PAGE1, used in chunk 696.

773 ⟨*verify data field* 773⟩≡ (909)

```
    ORG     $B304
VERIFY_DATA_FIELD:
    SUBROUTINE VERIFY_DATA_FIELD

; --- Search for D5 AA AD data field prologue ---
    LDY     #$20            ; timeout counter = 32
.search_d5:
    DEY
    BEQ     .error          ; timeout
    LDA     DISK_DATA_REG   ; read disk nibble
    BPL     *-3             ; wait for data ready
.check_d5:
    EOR     #$D5            ; check for D5
    BNE     .search_d5      ; not D5, retry
    LDA     DISK_DATA_REG   ; read next nibble
    BPL     *-3
    BANE    #$AA,.check_d5  ; not AA, re-check as D5
    LDA     DISK_DATA_REG   ; read third nibble
    BPL     *-3
    BANE    #$AD,.check_d5  ; not AD, re-check as D5

; --- Found D5 AA AD: read and verify data nibbles ---
    PHA                     ; \ waste cycles (timing)
    PLA                     ; /
    LDY     #$56            ; 86 nibbles (secondary buffer)
.read_secondary:
    LDA     DISK_DATA_REG   ; read nibble
    BPL     *-3
    BIT     $C000           ; timing sync
    BANE    #$AF,.bad_nibble ; mismatch -> error
    DEY
    BNE     .read_secondary

    LDY     #$00            ; 256 nibbles (primary buffer)
.read_primary:
    LDA     DISK_DATA_REG   ; read nibble
    BPL     *-3
    BIT     $C000           ; timing sync
    BANE    #$AF,.bad_nibble ; mismatch -> error
    DEY
    BNE     .read_primary

; --- Verify DE AA data field epilogue ---
    LDY     DISK_DATA_REG   ; read (discard) checksum nibble
    BPL     *-3
    PHA                     ; \ waste cycles
    PLA                     ; /
    LDA     DISK_DATA_REG   ; read epilogue byte 1
    BPL     *-3
```

```
    BANE    #$DE,.error      ; bad epilogue
    LDA     DISK_DATA_REG    ; read epilogue byte 2
    BPL     *-3
    BAEQ    #$AA,.ok         ; good epilogue

.error:
    SEC                      ; carry set = error
    RTS

.ok:
    CLC                      ; carry clear = success
    RTS

.bad_nibble:
    STA     DISK_BAD_NIBBLE  ; save mismatched nibble for diagnostics
    SEC                      ; carry set = error
    RTS
```

Defines:
VERIFY_DATA_FIELD, used in chunks 775, 779a, 793, and 822.
Uses DISK_BAD_NIBBLE 793 and DISK_DATA_REG 772.

The disk VM interpreter loop. Each iteration swaps the VM context (zero-page $50--$5F ↔ saved state at DISK_VM_SAVED_STATE) and then dispatches the next VM opcode. The VM bytecode program follows immediately after the JMP at $B370 and controls the multi-sector disk read sequence. The loop has no explicit exit; the VM program returns to the caller by manipulating the stack.

774 ⟨*disk vm loop* 774⟩≡ (909)

```
    ORG     $B36A
DISK_VM_LOOP:
    JSR     DISK_VM_SWAP     ; swap ZP $50-$5F with saved state
    JSR     DISK_VM_DISPATCH ; execute next VM opcode
    JMP     DISK_VM_LOOP     ; repeat
; VM bytecode program follows at [[DISK_VM_BYTECODE]]
```

Defines:
DISK_VM_LOOP, used in chunk 771.
Uses DISK_VM_BYTECODE 775, DISK_VM_DISPATCH 783, and DISK_VM_SWAP 781c.

The VM bytecode program follows immediately after the VM loop. All address operands are encrypted (low byte XOR $9C, high byte XOR $AD); immediate operands are XOR'd with $7A. The program implements a disk copy-protection verifier: it checks code integrity, turns on the drive motor, reads data from multiple half-track positions, and exits with carry indicating success or failure.

Decoded program flow:

1. Three integrity checks: verifies bytes at $B361, $B362, and $B368 (the SEC, RTS, and SEC opcodes in VERIFY_DATA_FIELD). Exits with error if any byte was patched.
2. Calls the disk-read subroutine at $B3A8 (embedded below). On failure, retries once; on second failure, allows up to 2 errors before restarting.
3. The subroutine turns on the motor, waits for spin-up, sets read mode, calls the half-track verifier (READ_AND_VERIFY) from multiple head positions, then turns off the motor.
4. On success, exits via DISK_VM_EXIT_OK (carry clear).

775 ⟨*disk vm bytecode* 775⟩≡ (909)

```
    ORG     $B373
DISK_VM_BYTECODE:
    SUBROUTINE DISK_VM_BYTECODE
; --- Integrity checks ---
; Verify that key bytes in VERIFY_DATA_FIELD have not been patched.
; $B361 = SEC ($38), $B362 = RTS ($60), $B368 = SEC ($38).
    DVMLD   $B361                 ; load byte at .error
    DVMSUBI $38                   ; compare with SEC opcode
    DVMBNE  DISK_VM_SUB_EXIT_ERR  ; patched -> error exit
    DVMLD   $B362                 ; load byte at .error+1
    DVMSUBI $60                   ; compare with RTS opcode
    DVMBNE  DISK_VM_SUB_EXIT_ERR  ; patched -> error exit
    DVMLD   $B368                 ; load byte at .bad_nibble+3
    DVMSUBI $38                   ; compare with SEC opcode
    DVMBNE  DISK_VM_SUB_EXIT_ERR  ; patched -> error exit
; --- First attempt ---
    DVMCALL .disk_read_sub        ; call disk read subroutine
    DVMBNE  .retry                ; error -> retry
    DVMGOTO .exit_ok              ; success -> exit
; --- Retry (second attempt) ---
.retry:
    DVMCALL .disk_read_sub        ; retry disk read
    DVMBNE  .error_handler        ; error -> handler
    DVMGOTO .exit_ok              ; success -> exit
; --- Error handler ---
.error_handler:
```

```
    DVMSUBI $02                    ; subtract 2 from error count
    DVMBNE  DISK_VM_SUB_EXIT_ERR  ; >2 errors -> fatal exit
    DVMGOTO DISK_VM_BYTECODE       ; restart from beginning
; --- Success exit ---
.exit_ok:
    DVMCALL0 DISK_VM_EXIT_OK       ; swap context, return carry clear
; --- Disk read subroutine (called via DVMCALL) ---
.disk_read_sub:
    DVMLD   IWM_MOTORON            ; turn on drive motor
    DVMLDI  $FF                    ; \ motor spin-up delay
    DVMCALL1 ROM_WAIT              ; /  WAIT($FF)
    DVMLDI  $FF                    ; \ second spin-up delay
    DVMCALL1 ROM_WAIT              ; /  WAIT($FF)
    DVMLD   IWM_READMODE           ; set read mode (Q7L)
    DVMLDI  $00                    ; \ clear error accumulator
    DVMST   DISK_VM_ERROR_ACCUM    ; /
    DVMCALL .half_track_verify     ; \ four half-track
    DVMCALL .half_track_verify     ;  | verification
    DVMCALL .half_track_verify     ;  | passes
    DVMCALL .half_track_verify     ; /
    DVMLD   IWM_MOTOROFF           ; turn off motor
    DVMLD   DISK_VM_ERROR_ACCUM    ; load error count
    DVMRET                         ; return to caller
; --- Half-track verifier subroutine ---
.half_track_verify:
    DVMLDI  $03                    ; \ set retry counter
    DVMST   DISK_RETRY_COUNT       ; /  to 3
    DVMCALL1 READ_AND_VERIFY       ; read and verify sector
    DVMLD   IWM_PH3_ON             ; phase 3 on (half-track step)
    DVMCALL1 READ_AND_VERIFY       ; \ verify two more
    DVMCALL1 READ_AND_VERIFY       ; /  sectors
    DVMLD   IWM_PH0_ON             ; phase 0 on (half-track step)
    DVMCALL DISK_VM_SUB_PARK_STEPPER ; park stepper motor
    DVMLDI  $03                    ; \ reset retry counter
    DVMST   DISK_RETRY_COUNT       ; /  to 3
    DVMCALL1 READ_AND_VERIFY       ; read and verify sector
    DVMLD   IWM_PH3_ON             ; phase 3 on
    DVMCALL1 READ_AND_VERIFY       ; \ verify two more
    DVMCALL1 READ_AND_VERIFY       ; /  sectors
    DVMLD   IWM_PH2_ON             ; phase 2 on (half-track step)
    DVMCALL DISK_VM_SUB_PARK_STEPPER ; park stepper
    DVMRET                         ; return to caller
; $B3FF: next byte ($A0) is the LDY opcode of SEARCH_ADDR_FIELD
```

Defines:
DISK_VM_BYTECODE, used in chunk 774.

Uses DISK_RETRY_COUNT 777, DISK_VM_ERROR_ACCUM 778, DISK_VM_EXIT_OK 780c, DISK_VM_SUB_EXIT_ERR 780b, DISK_VM_SUB_PARK_STEPPER 779b, IWM_MOTOROFF 772, IWM_MOTORON 772, IWM_PH0_ON 772, IWM_PH2_ON 772, IWM_PH3_ON 772, IWM_READMODE 772, READ_AND_VERIFY 779a, ROM_WAIT 606b, SEARCH_ADDR_FIELD 777, and VERIFY_DATA_FIELD 773.

Searches for the D5 AA 96 address field prologue on disk. The entry point at SEARCH_ADDR_FIELD sets Y to $FF (256 inner retries), then falls through to the search loop. The outer retry count is loaded from and saved back to the data byte at DISK_RETRY_COUNT. On success, returns with carry clear, X = 1, and the last decoded address-field nibble (sector number) in $50. On failure, returns with carry set.

The 4-and-4 decoding loop reads three field pairs (volume, track, sector), each encoded as two disk nibbles whose even/odd bits must be combined via ROL/AND.

777 ⟨*search addr field* 777⟩≡ (909)

```
    ORG     $B3FF
SEARCH_ADDR_FIELD:
    LDY     #$FF                ; 256 inner retries
; $B400: FF is the immediate operand (part of the LDY instruction)
.search:
    LDX     DISK_RETRY_COUNT ; outer retry count
.next_nibble:
    LDA     DISK_DATA_REG       ; read nibble
    BPL     *-3                 ; wait for ready
    BAEQ    #$D5,.got_d5        ; first prologue byte?
    DEY                         ; inner countdown
    BNE     .next_nibble
    DEX                         ; outer countdown
    BNE     .next_nibble
    RTS                         ; timeout (carry unchanged)

.got_d5:
    LDA     DISK_DATA_REG
    BPL     *-3
    BAEQ    #$AA,.got_aa        ; second prologue byte?
    DEY
    BNE     .next_nibble
    SEC
    RTS                         ; timeout

.got_aa:
    LDA     DISK_DATA_REG
    BPL     *-3
    BAEQ    #$96,.got_96        ; third prologue byte (addr mark)?
    DEY
    BNE     .next_nibble
    SEC
    RTS                         ; timeout

.got_96:
; --- Decode 3 address field pairs (4-and-4 encoding) ---
    LDY     #$02                ; 3 pairs: volume, track, sector
.decode_pair:
```

```
    LDA     DISK_DATA_REG      ; read odd-bits nibble
    BPL     *-3
    ROL                        ; shift data, odd bits in even positions
    STA     $50                ; save partial
    LDA     DISK_DATA_REG      ; read even-bits nibble
    BPL     *-3
    AND     $50                ; combine: result = decoded byte
    STA     $50                ; last decoded value (sector on final iter)
DISK_IO_END:
    DEY
    STX     DISK_RETRY_COUNT ; save remaining retry count
    BPL     .decode_pair       ; Y: 2->1->0->FF (3 iterations)

    CLC                        ; success
    LDX     #$01
    RTS

DISK_RETRY_COUNT:
    DC.B    3                  ; $B44D: outer retry counter (default 3)
```

Defines:
DISK_IO_END, used in chunk 671.
DISK_RETRY_COUNT, used in chunk 775.
SEARCH_ADDR_FIELD, used in chunks 775 and 779a.
Uses DISK_DATA_REG 772.

Three bytes between the dispatch trampoline and the saved state. DISK_VM_ERROR_ACCUM is the error accumulator used by READ_AND_VERIFY to count failed sector reads across multiple attempts.

778 ⟨*disk vm error accum* 778⟩≡ (909)

```
    ORG     $B5E2
DISK_VM_ERROR_ACCUM:
    DC.B    0   ; error count (updated by READ_AND_VERIFY)
    DC.B    0,0 ; scratch
```

Defines:
DISK_VM_ERROR_ACCUM, used in chunks 775 and 779a.
Uses READ_AND_VERIFY 779a.

Native routine called by the VM via CALL1. Searches for an address field (JSR SEARCH_ADDR_FIELD), then verifies the data field (JSR VERIFY_DATA_FIELD). Accumulates errors in DISK_VM_ERROR_ACCUM at DISK_VM_ERROR_ACCUM. Afterward, turns off all four stepper motor phases (preventing motor heating).

779a ⟨*read and verify* 779a⟩≡ (909)

```
    ORG     $B44E
READ_AND_VERIFY:
    JSR     SEARCH_ADDR_FIELD   ; find D5 AA 96 address prologue
    BCS     .skip_verify        ; no address field -> skip data verify
    JSR     VERIFY_DATA_FIELD   ; verify D5 AA AD data field
.skip_verify:
    LDA     #$00
    ADC     DISK_VM_ERROR_ACCUM ; carry -> error count
    STA     DISK_VM_ERROR_ACCUM
; --- Turn off all stepper motor phases ---
    LDA     $C0E0               ; phase 0 off
    LDA     $C0E2               ; phase 1 off
    LDA     $C0E4               ; phase 2 off
    LDA     $C0E6               ; phase 3 off
    RTS
```

Defines:
READ_AND_VERIFY, used in chunks 775, 778, and 822.
Uses DISK_VM_ERROR_ACCUM 778, SEARCH_ADDR_FIELD 777, and VERIFY_DATA_FIELD 773.

VM bytecode subroutine that delays and turns off all stepper motor phases. Called via VM opcode 5 (CALL) from the half-track verifier. The delays allow the stepper motor to settle between half-track movements.

779b ⟨*disk vm sub park stepper* 779b⟩≡ (909)

```
    ORG     $B46B
DISK_VM_SUB_PARK_STEPPER:
    DVMLDI  $70                     ; \ long delay
    DVMCALL1 DELAY_LOOP             ; /  for stepper settle
    DVMLD   IWM_PH0_OFF             ; phase 0 off
    DVMLD   IWM_PH1_OFF             ; phase 1 off
    DVMLD   IWM_PH2_OFF             ; phase 2 off
    DVMLD   IWM_PH3_OFF             ; phase 3 off
    DVMLDI  $28                     ; \ short delay
    DVMCALL1 DELAY_LOOP             ; /  for settle
    DVMRET                          ; return to caller
```

Defines:
DISK_VM_SUB_PARK_STEPPER, used in chunk 775.
Uses DELAY_LOOP 780a, IWM_PH0_OFF 772, IWM_PH1_OFF 772, IWM_PH2_OFF 772, and IWM_PH3_OFF 772.

Native delay routine. Takes the delay count in A (passed from VM accumulator via CALL1). Inner loop: DEX from $20 to 0 (32 iterations). Outer loop: SBC~\#$01 until A reaches 0. Total delay ≈ 160 × A cycles.

780a ⟨*delay loop* 780a⟩≡ (909)

```
    ORG     $B482
DELAY_LOOP:
    SUBROUTINE DELAY_LOOP
    LDX     #$20
.inner:
    DEX
    BNE     .inner
    SEC
    SBC     #$01
    BNE     DELAY_LOOP
    RTS
```

Defines:
DELAY_LOOP, used in chunk 779b.

Short VM bytecode fragment that loads $20 into the accumulator and jumps to DISK_VM_EXIT_ERR, which swaps context and returns to the caller with carry set. Reached by the integrity-check and error-handler BNZ branches in the main bytecode program.

780b ⟨*disk vm sub exit err* 780b⟩≡ (909)

```
    ORG     $B48D
DISK_VM_SUB_EXIT_ERR:
    DVMLDI  $20                     ; load error code
    DVMCALL0 DISK_VM_EXIT_ERR       ; exit VM with carry set
```

Defines:
DISK_VM_SUB_EXIT_ERR, used in chunk 775.
Uses DISK_VM_EXIT_ERR 781a.

These native routines exit the disk VM. Each calls DISK_VM_SWAP to restore the caller's zero-page state, then returns with carry flag indicating success or failure.

780c ⟨*disk vm exit ok* 780c⟩≡ (909)

```
    ORG     $B492
DISK_VM_EXIT_OK:
    JSR     DISK_VM_SWAP ; restore caller's ZP
    CLC                  ; carry clear = success
    RTS
```

Defines:
DISK_VM_EXIT_OK, used in chunk 775.
Uses DISK_VM_SWAP 781c.

DISK_VM_EXIT_ERR is the error counterpart: it swaps context back and returns with carry set, signalling failure to the caller.

781a ⟨*disk vm exit err* 781a⟩≡ (909)

```
      ORG     $B497
  DISK_VM_EXIT_ERR:
      JSR     DISK_VM_SWAP ; restore caller's ZP
      SEC                  ; carry set = error
      RTS
```

Defines:
DISK_VM_EXIT_ERR, used in chunk 780b.
Uses DISK_VM_SWAP 781c.

DISK_VM_MISC_B49C is a two-byte helper that XORs A with $AD and returns. It is used by the VM opcode decoder to decrypt the high byte of address operands (the encryption key for high bytes is $AD).

781b ⟨*disk vm misc b49c* 781b⟩≡ (909)

```
      ORG     $B49C
  DISK_VM_MISC_B49C:
      EOR     #$AD
      RTS
```

Defines:
DISK_VM_MISC_B49C, never used.

Swaps 16 bytes between zero-page locations $50--$5F and the saved VM state at DISK_VM_SAVED_STATE. This preserves the caller's zero-page state while the VM executes, and restores it afterward.

781c ⟨*disk vm swap* 781c⟩≡ (909)

```
      ORG     $B4CC
  DISK_VM_SWAP:
      SUBROUTINE DISK_VM_SWAP

      LDX     #$0F    ; 16 bytes ($50-$5F)
  .loop:
      LDA     $50,X   ; \ swap ZP with saved state
      PHA             ;  |
      LDA     $B5E5,X ;  |
      STA     $50,X   ;  |
      PLA             ;  |
      STA     $B5E5,X ; /
      DEX
      BPL     .loop
      RTS
```

Defines:
DISK_VM_SWAP, used in chunks 774, 780c, and 781a.

Scratch/data area between DISK_VM_SWAP and the opcode handlers. Contains mostly zeros with bytes $4A $4C $4E at $B4EE--$B4F0, mirroring the initial saved-state values for ZP $5B--$5D.

782 ⟨*disk vm data b4de* 782⟩≡ (909)

```
    ORG     $B4DE
DISK_VM_DATA_B4DE:
    HEX     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
    HEX     4a 4c 4e
    HEX     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Defines:
DISK_VM_DATA_B4DE, never used.

Executes one VM opcode. Pulls the return address from the stack to locate the current instruction pointer, reads the next opcode byte, looks up the handler address (low byte) in the table at DISK_VM_HANDLER_TBL, and self-modifies a JMP at DISK_VM_TRAMPOLINE to dispatch to the handler in page $B5.

783 ⟨*disk vm dispatch* 783⟩≡ (909)

```
    ORG     $B49F
DISK_VM_DISPATCH:
    SUBROUTINE DISK_VM_DISPATCH

    TXA
    PHA                         ; save X
    TYA
    PHA                         ; save Y
    JSR     DISK_VM_PULL_IP     ; pull IP from stack -> $52/$53
    PULB    $52 ; \ restore as VM instruction pointer
    PULB    $53 ; / values set by DISK_VM_PULL_IP
    LDY     #$04
DISK_VM_RESUME:
    LDA     ($52),Y             ; read opcode from VM program
    INY
    BNE     .no_page
    INC     $53
.no_page:
    TAX
    LDA     DISK_VM_HANDLER_TBL,X ; look up handler low byte
    STA     $B5E0               ; self-modify JMP target at [[DISK_VM_TRAMPOLINE]]
    JMP     DISK_VM_TRAMPOLINE  ; dispatch (JMP $B5xx)

; --- VM opcode handler table ---
; Each byte is the low address of a handler in page $B5.
; Opcode index -> handler address ($B5xx).
DISK_VM_HANDLER_TBL:
    DC.B    #<DISK_VM_OP_LDA_IDX   ; $00: DVMLDX
    DC.B    #<DISK_VM_OP_CALL1     ; $01: DVMCALL1
    DC.B    #<DISK_VM_OP_BNZ       ; $02: DVMBNE
    DC.B    #<DISK_VM_OP_LDA_IMM   ; $03: DVMLDI
    DC.B    #<DISK_VM_OP_LDA_ABS   ; $04: DVMLD
    DC.B    #<DISK_VM_OP_CALL      ; $05: DVMCALL
    DC.B    #<DISK_VM_OP_STA_ABS   ; $06: DVMST
    DC.B    #<DISK_VM_OP_SUB_IMM   ; $07: DVMSUBI
    DC.B    #<DISK_VM_OP_RET       ; $08: DVMRET
    DC.B    #<DISK_VM_OP_CALL0     ; $09: DVMCALL0
    DC.B    #<DISK_VM_OP_INC_LOAD  ; $0A: DVMINC
    DC.B    #<DISK_VM_HALT         ; $0B: DVMHALT (RTS)
    DC.B    #<DISK_VM_OP_GOTO      ; $0C: DVMGOTO
```

Defines:
DISK_VM_DISPATCH, used in chunks 774 and 784.
DISK_VM_HANDLER_TBL, never used.
DISK_VM_RESUME, used in chunks 787–89, 791, and 792a.
Uses DISK_VM_HALT 785, DISK_VM_OP_BNZ 789a, DISK_VM_OP_CALL 790b, DISK_VM_OP_CALL0 788b,

DISK_VM_OP_CALL1 788a, DISK_VM_OP_GOTO 787, DISK_VM_OP_INC_LOAD 792a, DISK_VM_OP_LDA_ABS 789c, DISK_VM_OP_LDA_IDX 790a, DISK_VM_OP_LDA_IMM 789b, DISK_VM_OP_RET 791a, DISK_VM_OP_STA_ABS 791b, DISK_VM_OP_SUB_IMM 791c, DISK_VM_PULL_IP 784, DISK_VM_TRAMPOLINE 792b, and PULB 9c.

DISK_VM_PULL_IP is an RTS-trick bootstrap used during VM startup. It pulls two return addresses from the 6502 stack: the inner one (from the JSR that called it) and the outer one (from the JSR that entered DISK_VM_DISPATCH). The outer address becomes the VM instruction pointer ($52/$53), and both are incremented by 1 to compensate for the 6502's convention of pushing address−1 during JSR. Execution then resumes at the inner return address via an indirect jump, allowing the dispatcher to "capture" its caller's position as the starting bytecode address.

This is the same stack trick used by PULL_TWO_RET_ADDRS (PULL_TWO_RET_ADDRS) in the resident VM dispatcher. Both routines pull two nested return addresses to bootstrap the VM instruction pointer from the call site.

784 ⟨*disk vm pull ip* 784⟩≡ (909)

```
    ORG     $B538
DISK_VM_PULL_IP:
    SUBROUTINE DISK_VM_PULL_IP

; Pulls two return addresses from the stack.
; The outer return address (from JSR DISK_VM_DISPATCH) becomes the
; VM instruction pointer.  Resumes execution at the inner return
; address + 1 (i.e., returns to the code after JSR DISK_VM_PULL_IP).
    PULB    $54 ; inner return address
    PULB    $55
    PULB    $52 ; outer return address
    PULB    $53 ; = VM instruction pointer
    INC     $52       ; adjust IP (6502 pushes addr-1)
    INC     $54       ; \ adjust inner return address
    BNE     .no_carry ;  |
    INC     $55       ; /
.no_carry:
    JMP     ($54)     ; resume at inner return + 1
```

Defines:
DISK_VM_PULL_IP, used in chunk 783.
Uses DISK_VM_DISPATCH 783 and PULB 9c.

Shared subroutine used by most VM opcode handlers. Reads two encrypted bytes from the bytecode stream at `$52/$53 + Y`, decrypts the address operand (low byte XOR `$9C`, high byte XOR `$AD`), stores the result in `$54/$55`, and advances Y past both bytes.

785 ⟨*disk vm read addr* 785⟩≡ (909)

```
    ORG     $B510
DISK_VM_READ_ADDR:
    LDA     ($52),Y ; read encrypted low byte
    EOR     #$9C    ; decrypt
    INY
    BNE     .no_page1
    INC     $53
.no_page1:
    STA     $54     ; operand low
    LDA     ($52),Y ; read encrypted high byte
    INY
    BNE     .no_page2
    INC     $53
.no_page2:
    EOR     #$AD    ; decrypt
    STA     $55     ; operand high
DISK_VM_HALT:                   ; opcode $0B handler: HALT (just RTS)
    RTS
```

Defines:
`DISK_VM_HALT`, used in chunk 783.
`DISK_VM_READ_ADDR`, used in chunks 787–92.

The disk VM has 13 opcodes (indices 0–12). Each handler's low address is stored in the table at `DISK_VM_HANDLER_TBL`; dispatch jumps to `$B5xx`. Handlers return to the dispatch loop by jumping to `DISK_VM_RESUME` at `DISK_VM_RESUME`, which reads the next opcode and dispatches again.

Op	Addr	Mnemonic	Description
0	$B579	LDA_IDX	ACC ← *(addr + ACC)
1	$B527	CALL1	Call native; A=ACC in, A→ACC out
2	$B555	BNZ	Branch if ACC ≠ 0
3	$B55F	LDA_IMM	ACC ← immediate (XOR $7A)
4	$B56D	LDA_ABS	ACC ← *(addr)
5	$B58A	CALL	VM subroutine call (push IP)
6	$B5AD	STA_ABS	*(addr) ← ACC
7	$B5B9	SUB_IMM	ACC ← ACC − immediate
8	$B5A2	RET	VM subroutine return (pop IP)
9	$B54F	CALL0	Jump to native code (no return)
10	$B5CE	INC_LOAD	*(addr) += 1; ACC ← result
11	$B526	HALT	Exit VM (RTS)
12	$B500	GOTO	Unconditional branch

The following macros emit encrypted Disk VM bytecodes, analogous to the EALDR VM macros (Table 25.1). The Disk VM uses different XOR keys: address low byte XOR `$9C`, address high byte XOR `$AD`, immediate XOR `$7A`. The opcode numbering also differs from the EALDR VM; notably, opcode `$0B` is `HALT` (exit VM) in the disk VM, whereas in the EALDR VM it was `VMASL` (shift left).

786 ⟨*Disk VM macros* 786⟩≡ (909)

```
; --- Disk VM bytecode macros ---
; Each macro emits the opcode byte followed by XOR-encrypted arguments.
; Address args: low ^ $9C, high ^ $AD.  Immediate args: ^ $7A.
MACRO DVMLDX   ;  $00 addr -- ACC <- *(addr + ACC)
    DC.B $00, <[{1}] ^ $9C, >[{1}] ^ $AD
ENDM
MACRO DVMCALL1 ;  $01 addr -- call native, result -> ACC
    DC.B $01, <[{1}] ^ $9C, >[{1}] ^ $AD
ENDM
MACRO DVMBNE   ;  $02 addr -- branch if ACC != 0
    DC.B $02, <[{1}] ^ $9C, >[{1}] ^ $AD
ENDM
MACRO DVMLDI   ;  $03 imm  -- ACC <- immediate
    DC.B $03, [{1}] ^ $7A
ENDM
MACRO DVMLD    ;  $04 addr -- ACC <- *(addr)
```

```
    DC.B $04, <[{1}] ^ $9C, >[{1}] ^ $AD
ENDM
MACRO DVMCALL  ;  $05 addr -- VM subroutine call
    DC.B $05, <[{1}] ^ $9C, >[{1}] ^ $AD
ENDM
MACRO DVMST    ;  $06 addr -- *(addr) <- ACC
    DC.B $06, <[{1}] ^ $9C, >[{1}] ^ $AD
ENDM
MACRO DVMSUBI  ;  $07 imm  -- ACC <- ACC - immediate
    DC.B $07, [{1}] ^ $7A
ENDM
MACRO DVMRET   ;  $08      -- VM subroutine return
    DC.B $08
ENDM
MACRO DVMCALL0 ;  $09 addr -- jump to native (no return)
    DC.B $09, <[{1}] ^ $9C, >[{1}] ^ $AD
ENDM
MACRO DVMINC   ;  $0A addr -- *(addr)++, result -> ACC
    DC.B $0A, <[{1}] ^ $9C, >[{1}] ^ $AD
ENDM
MACRO DVMHALT  ;  $0B      -- exit VM (RTS)
    DC.B $0B
ENDM
MACRO DVMGOTO  ;  $0C addr -- unconditional branch
    DC.B $0C, <[{1}] ^ $9C, >[{1}] ^ $AD
ENDM
```

Unconditional branch. Reads a 2-byte encrypted address operand and sets the VM instruction pointer to that address.

787 ⟨*disk vm op goto* 787⟩≡ (909)

```
    ORG     $B500
DISK_VM_OP_GOTO:
    JSR     DISK_VM_READ_ADDR ; read target -> $54/$55
DISK_VM_SET_IP:
    MOVW    $54,$52 ; set IP = operand
    LDY     #$00
    JMP     DISK_VM_RESUME    ; dispatch next opcode
```

Defines:
DISK_VM_OP_GOTO, used in chunks 783 and 792b.
DISK_VM_SET_IP, used in chunks 789a and 790b.
Uses DISK_VM_READ_ADDR 785, DISK_VM_RESUME 783, and MOVW 9a.

Exits the VM by executing RTS, which returns to the caller of DISK_VM_DISPATCH. This handler is just the RTS at the end of DISK_VM_READ_ADDR—the dispatch table points to DISK_VM_HALT which is that instruction.

Calls a native 6502 subroutine. Passes the VM accumulator in A and captures the subroutine's return value from A back into the accumulator. The Y register (bytecode offset) is preserved across the call.

788a ⟨*disk vm op call1* 788a⟩≡ (909)

```
    ORG     $B527
DISK_VM_OP_CALL1:
    SUBROUTINE DISK_VM_OP_CALL1
    JSR     DISK_VM_READ_ADDR ; read native addr -> $54/$55
    TYA
    PHA                       ; save Y (bytecode offset)
    LDA     $56               ; A = VM accumulator
    JSR     DISK_VM_JMP_IND   ; call native routine via JMP ($54)
    STA     $56               ; capture return A -> accumulator
    PLA
    TAY                       ; restore Y
    JMP     DISK_VM_RESUME    ; dispatch next opcode
```

Defines:
DISK_VM_OP_CALL1, used in chunk 783.
Uses DISK_VM_JMP_IND 788b, DISK_VM_READ_ADDR 785, and DISK_VM_RESUME 783.

Jumps to a native 6502 routine and does not return to the VM. Used for operations like "enter the main program" or "jump to a disk routine" that take over control flow. The JMP ($54) at DISK_VM_JMP_IND also serves as a reusable trampoline for opcode 1.

788b ⟨*disk vm op call0* 788b⟩≡ (909)

```
    ORG     $B54F
DISK_VM_OP_CALL0:
    JSR     DISK_VM_READ_ADDR ; read native addr -> $54/$55
DISK_VM_JMP_IND:
    JMP     ($54)             ; indirect jump (no return to VM)
```

Defines:
DISK_VM_JMP_IND, used in chunk 788a.
DISK_VM_OP_CALL0, used in chunk 783.
Uses DISK_VM_READ_ADDR 785.

Branch if the VM accumulator is nonzero. Reads a 2-byte encrypted target address, then either branches (reuses DISK_VM_SET_IP) or falls through to the next opcode.

789a ⟨*disk vm op bnz* 789a⟩≡ (909)

```
      ORG     $B555
  DISK_VM_OP_BNZ:
      JSR     DISK_VM_READ_ADDR ; read branch target -> $54/$55
      LDA     $56               ; test accumulator
      BNE     DISK_VM_SET_IP    ; nonzero -> branch (reuse GOTO tail)
      JMP     DISK_VM_RESUME    ; zero -> continue
```

Defines:
DISK_VM_OP_BNZ, used in chunk 783.
Uses DISK_VM_READ_ADDR 785, DISK_VM_RESUME 783, and DISK_VM_SET_IP 787.

Loads an immediate value into the VM accumulator. The operand is a single encrypted byte (XOR $7A).

789b ⟨*disk vm op lda imm* 789b⟩≡ (909)

```
      ORG     $B55F
  DISK_VM_OP_LDA_IMM:
      LDA     ($52),Y ; read encrypted byte
      INY
      BNE     .no_page
      INC     $53
  .no_page:
      EOR     #$7A    ; decrypt
      STA     $56     ; accumulator = value
      JMP     DISK_VM_RESUME
```

Defines:
DISK_VM_OP_LDA_IMM, used in chunk 783.
Uses DISK_VM_RESUME 783.

Loads a byte from an absolute address into the VM accumulator.

789c ⟨*disk vm op lda abs* 789c⟩≡ (909)

```
      ORG     $B56D
  DISK_VM_OP_LDA_ABS:
      JSR     DISK_VM_READ_ADDR ; read address -> $54/$55
  DISK_VM_LOAD_IND:
      LDX     #$00
      LDA     ($54,X)           ; ACC = *(address)
      STA     $56
      JMP     DISK_VM_RESUME
```

Defines:
DISK_VM_LOAD_IND, used in chunk 790a.
DISK_VM_OP_LDA_ABS, used in chunk 783.
Uses DISK_VM_READ_ADDR 785 and DISK_VM_RESUME 783.

Indexed load. Reads a 2-byte base address, adds the VM accumulator as an offset, then loads the byte at (base + ACC) into the accumulator. Useful for table lookups. Falls into DISK_VM_LOAD_IND to perform the actual load.

790a ⟨*disk vm op lda idx* 790a⟩≡ (909)

```
    ORG     $B579
DISK_VM_OP_LDA_IDX:
    SUBROUTINE DISK_VM_OP_LDA_IDX
    JSR     DISK_VM_READ_ADDR ; read base addr -> $54/$55
    LDA     $56               ; index = old accumulator
    CLC
    ADC     $54               ; base_lo + index
    STA     $54
    BCC     .no_carry
    INC     $55               ; propagate carry
.no_carry:
    JMP     DISK_VM_LOAD_IND  ; load from ($54/$55) -> ACC
```

Defines:
DISK_VM_OP_LDA_IDX, used in chunk 783.
Uses DISK_VM_LOAD_IND 789c and DISK_VM_READ_ADDR 785.

VM-level subroutine call. Pushes the current absolute IP onto the 6502 hardware stack, then branches to the operand address.

790b ⟨*disk vm op call* 790b⟩≡ (909)

```
    ORG     $B58A
DISK_VM_OP_CALL:
    SUBROUTINE DISK_VM_OP_CALL
    JSR     DISK_VM_READ_ADDR ; read target -> $54/$55
    TYA
    CLC
    ADC     $52               ; \ resolve absolute IP
    STA     $52               ;  | (base + Y offset)
    BCC     .no_carry
    INC     $53               ; /
.no_carry:
    PSHW    $52 ; push IP for RET
    LDY     #$00
    JMP     DISK_VM_SET_IP    ; GOTO target address
```

Defines:
DISK_VM_OP_CALL, used in chunk 783.
Uses DISK_VM_READ_ADDR 785, DISK_VM_SET_IP 787, and PSHW 9b.

VM-level subroutine return. Pops the IP previously pushed by opcode 5 (CALL) and resumes execution there.

791a ⟨*disk vm op ret* 791a⟩≡ (909)

```
      ORG     $B5A2
  DISK_VM_OP_RET:
      PULW    $52 ; pop saved IP
      LDY     #$00
      JMP     DISK_VM_RESUME
```

Defines:
DISK_VM_OP_RET, used in chunk 783.
Uses DISK_VM_RESUME 783 and PULW 10a.

Stores the VM accumulator to an absolute address.

791b ⟨*disk vm op sta abs* 791b⟩≡ (909)

```
      ORG     $B5AD
  DISK_VM_OP_STA_ABS:
      JSR     DISK_VM_READ_ADDR ; read address -> $54/$55
      LDA     $56               ; A = accumulator
      LDX     #$00
      STA     ($54,X)           ; *(address) = ACC
      JMP     DISK_VM_RESUME
```

Defines:
DISK_VM_OP_STA_ABS, used in chunk 783.
Uses DISK_VM_READ_ADDR 785 and DISK_VM_RESUME 783.

Subtracts an encrypted immediate value from the VM accumulator.

791c ⟨*disk vm op sub imm* 791c⟩≡ (909)

```
      ORG     $B5B9
  DISK_VM_OP_SUB_IMM:
      LDA     ($52),Y ; read encrypted byte
      INY
      BNE     .no_page
      INC     $53
  .no_page:
      EOR     #$7A    ; decrypt
      STA     $54     ; temp
      LDA     $56     ; A = accumulator
      SEC
      SBC     $54     ; ACC - immediate
      STA     $56     ; accumulator = result
      JMP     DISK_VM_RESUME
```

Defines:
DISK_VM_OP_SUB_IMM, used in chunk 783.
Uses DISK_VM_RESUME 783.

Increments the byte at an absolute address and loads the new value into the VM accumulator. Atomic read-modify-write.

792a ⟨*disk vm op inc load* 792a⟩≡ (909)

```
      ORG     $B5CE
  DISK_VM_OP_INC_LOAD:
      JSR     DISK_VM_READ_ADDR ; read address -> $54/$55
      LDX     #$00
      LDA     ($54,X)           ; read current value
      CLC
      ADC     #$01              ; increment
      STA     ($54,X)           ; write back
      STA     $56               ; accumulator = new value
      JMP     DISK_VM_RESUME
```

Defines:
DISK_VM_OP_INC_LOAD, used in chunk 783.
Uses DISK_VM_READ_ADDR 785 and DISK_VM_RESUME 783.

The self-modifying JMP instruction whose low byte is patched by DISK_VM_DISPATCH before each handler call. The byte at $B5E0 is overwritten with the handler's low address from the opcode table.

792b ⟨*disk vm trampoline* 792b⟩≡ (909)

```
      ORG     $B5DF
  DISK_VM_TRAMPOLINE:
      JMP     DISK_VM_OP_GOTO         ; self-modified: low byte patched
```

Defines:
DISK_VM_TRAMPOLINE, used in chunk 783.
Uses DISK_VM_OP_GOTO 787.

The 16-byte region at $B5E5--$B5F4 holds the saved copy of zero-page $50--$5F when the VM is not executing. The context swap routine (DISK_VM_SWAP) exchanges these with the live zero-page values on each VM entry and exit. Bytes 11–13 ($5B–$5D) hold initial values $4A $4C $4E.

Bytes at $B5F5--$B5FF are additional VM workspace: DISK_BAD_NIBBLE stores a mismatched nibble from VERIFY_DATA_FIELD.

793 ⟨*disk vm saved state* 793⟩≡ (909)

```
    ORG     $B5E5
DISK_VM_SAVED_STATE:
;            $50                         $54                 $58
    HEX     00 00 00 00 00 00 00 00 00 4a 4c 4e 00 00 00 00
;           saved ZP $50-$5F; bytes 9-11 ($59-$5B) = $4A,$4C,$4E
DISK_VM_SCRATCH:
    HEX     00          ; $B5F5
DISK_BAD_NIBBLE:
    HEX     00          ; $B5F6: stores mismatched nibble from VERIFY_DATA_FIELD
    HEX     00 00 00    ; $B5F7-$B5F9
    HEX     a0 a0       ; $B5FA-$B5FB
    HEX     00 00 00 01 ; $B5FC-$B5FF
```

Defines:
DISK_BAD_NIBBLE, used in chunk 773.
DISK_VM_SAVED_STATE, never used.
DISK_VM_SCRATCH, never used.
Uses VERIFY_DATA_FIELD 773.

Chapter 23

The boot process

This disk image is credited to "4am and san inc." These are two separate efforts:

- **4am / Passport:** The automatic cracking tool Passport identified the Electronic Arts bootloader and applied three single-byte patches to the EALDR bytecode and resident code (documented in §24). These patches disable the copy-protection checks while leaving all other code intact.
- **san inc** (Peter Ferrie, also known as "qkumba"): A "san inc crack" is a conversion from a floppy disk image into files suitable for running from a hard disk. This involves rewriting the disk access routines entirely, removing "insert disk" prompts, and compressing content to save space and decrease load times—but removing nothing else. The code at `$0840`–`$08FF` in `BOOT1` is san inc's work: it checks for a language card, BLOADs a two-part overlay into language card RAM, and enters Applesoft.

The result is a `DOS 3.3` image with standard sector layout—no custom prologues or epilogues—while the EALDR's decryption routines and VM architecture remain intact.

The game was distributed by Electronic Arts using their custom loader (`EALDR`). The EA copy-protection system—including the VM bytecode interpreter, the DXR decryption, and the half-track verification—may have been developed by Rich Hilleman, who described writing EA's early copy protection in an oral history interview with the Computer History Museum (2013). Understanding what `EALDR` does during the boot process is essential for extracting the main program.

23.1 The booting of a disk

Suggested reading: *Beneath Apple DOS* (Don Worth, Pieter Lechner, 1982) page 5-6, "What happens during booting".

In general, booting the typical Apple disk until it runs the actual program looks like this:

```
DISKCARD -> BOOT1 -> BOOT2 -> PROGRAM
```

The disk card has a small ROM whose purpose is to load `BOOT1`, which starts from track 0 sector 0, where the first byte of sector 0 tells the card how many sectors are in `BOOT1`. Standard `DOS 3.3`'s `BOOT1` is only one sector long, but other more custom disk loaders have more sectors (up to 16). In this case, it is 5 sectors long (see §24).

The purpose of `BOOT1` is to load the next, larger stage, `BOOT2`.

In this case, `BOOT2` is a custom DOS, `EALDR` (the Electronic Arts Loader). This is loaded from tracks 1–2 into `$A000`–`$BFFF`, then `BOOT1` enters it at `$A806` via a return address pushed onto the stack.

`EALDR` is a mini virtual machine that reads game data from various tracks, decrypts a portion of it, and then jumps to the main program at `$0800`.

Since we're interested in getting at the main program, we need to find out what `EALDR` loads from disk and where. If the program itself uses the disk for any reason, then we have to look at `BOOT1`'s disk routines more closely.

23.1.1 Sector interleaving

Within a single track, sectors are not necessarily stored in consecutive order. The reason is that it takes time for the program to process the data for a sector it just read. By the time the program is ready to read the next sector, the disk has rotated some amount.

It would make loading multiple sectors faster if the program "speed" and the disk "speed" were coordinated. Thus, for example, sector 1 might be placed half the disk around from sector 0. In standard `DOS 3.3`, sector 1 is placed nearly all the way around the disk – `7/8` of the way around!

The `.dsk` file stores sectors in `DOS 3.3` logical order. The mapping between physical sectors on the disk and logical sectors in the file is the standard `DOS 3.3` 2:1 software interleave:

Physical	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
DOS/File	0	7	14	6	13	5	12	4	11	3	10	2	9	1	8	15

To find physical sector `P` of track `T` in the file: `file_offset = (T * 16 + skew_table[P]) * 256`

The disk card's boot ROM reads physical sectors sequentially (0, 1, 2, ...) and loads them into consecutive memory pages starting at `$0800`.

In addition, the game's custom loader (in `BOOT1`) uses its own interleave table for loading data from tracks beyond track 0. This table maps physical sector numbers to memory page offsets within the target region:

Physical	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Page offset	F	8	1	9	2	A	3	B	4	C	5	D	6	E	7	0

The translation tables `boot1_xlat.txt` and `boot2_xlat.txt` encode the compound mapping from memory order to file position, accounting for both the DOS 3.3 skew and (for `boot2_xlat.txt`) the custom loader interleave.

23.2 Extracting the sections for analysis

The disk image contains the following sections. Note that the disk has 16 sectors per track, and we will refer to tracks and sectors only by `16 * track + sector`.

- Sector 0-4: `BOOT1`: Target address `$0800`, entry point `$0801`.
- Sector 16-47: `EALDR` (EA Loader VM): Target address `$A000`, entry point `$A806`.
- Tracks `$0D`–`$10` (sectors 208–271): Main program code. Target address `$0800`, entry point `$0800`. Track `$0D` loads to `$0800`–`$17FF`, then track `$0E` overwrites `$1000`–`$1FFF`, so the final layout is `$0800`–`$3FFF`.

The sections can be extracted from the disk image using the following commands:

```
python -m extract --first 0 -n 5 \
  -i "Ali Baba and the Forty Thieves (4am and san inc crack).dsk" \
```

```
  -o boot1.bin --table boot1_xlat.txt
python -m extract --first 16 -n 32 \
  -i "Ali Baba and the Forty Thieves (4am and san inc crack).dsk" \
  -o ealdr.bin --table EALDR_XLAT.txt
python -m extract --first 208 -n 8 \
  -i "Ali Baba and the Forty Thieves (4am and san inc crack).dsk" \
  -o main_0800.bin --table EALDR_XLAT.txt
python -m extract --first 224 -n 48 \
  -i "Ali Baba and the Forty Thieves (4am and san inc crack).dsk" \
  -o main_1000.bin --table EALDR_XLAT.txt
cat main_0800.bin main_1000.bin > main.bin
```

The main program extraction accounts for the overlap: track `$0D` is loaded at base page `$08` (`$0800`–`$17FF`), but track `$0E` at base page `$10` overwrites `$1000`–`$17FF`. We extract only the first 8 sectors (pages `$08`–`$0F`) from track `$0D`, then all 48 sectors from tracks `$0E`–`$10`, producing a contiguous `$3800`-byte image loadable at `$0800`.

Note: the `EALDR` also loads game data from tracks `$07`–`$0C` into `$4000`–`$95FF` and decrypts `$4000`–`$67FF` (see §25). It also copies `$A300`–`$A5FF` from `EALDR` memory to `$0500`–`$07FF`.

Chapter 24

BOOT1

`BOOT1` consists of 5 sectors from track 0, loaded by the disk card's boot ROM into `$0800`–`$0CFF`. The boot ROM reads physical sectors 0–4 into consecutive pages. After loading, execution enters at `$0801`.

Address	Size	Contents
$0800	1	Sector count byte ($05)
$0801–$0803	3	JMP \$0804
$0804–$083F	60	BOOT1_ENTRY: set reset vector, load EALDR, enter it
$0840–$086D	46	san inc crack: language card check, BLOAD overlay
$086E–$0880	19	san inc crack: no-language-card path (print error, JMP $03D0)
$0881–$08DD	93	san inc crack: high-bit ASCII strings
$08DE–$08E3	6	Zero padding
$08E4–$08E7	4	BOOT1_PRINT_CR: JSR ROM_CRDO; RTS
$08E8–$08FD	22	Zero padding
$08FE–$08FF	2	Reset vector complement ($B7 \$FF)
$0900–$0901	2	BOOT1_READ_ERROR: SEC; RTS
$0902–$09C5	196	BOOT1_READ_SECTOR: nibble-decode a data field
$09C6	1	BOOT1_TRACK: current track number
$09C7	1	BOOT1_ERROR_FLAG: set on timeout
$09C8–$09F7	48	BOOT1_SEEK_TRACK: half-track arm stepping
$09F8–$0A04	13	BOOT1_PHASE_ON: activate stepper phase
$0A05–$0A12	14	BOOT1_DELAY: nested delay loop
$0A13–$0A14	2	BOOT1_ADDR_ERROR: SEC; RTS
$0A15–$0A6F	91	BOOT1_FIND_SECTOR: search for address field
$0A70–$0A97	40	Nibble translation table (secondary) + ASCII fragment
$0A98–$0AFF	104	Nibble translation table (6-and-2 primary decode)
$0B00–$0BFF	256	Nibble translation table (pair decode)
$0C00–$0C47	72	BOOT1_LOAD_TRACK: read all 16 sectors of a track
$0C48–$0C57	16	Custom interleave table
$0C58–$0C67	16	Destination page table (runtime-computed)
$0C68–$0CFF	152	Residual EA Loader documentation

After the disk card reads BOOT1, the zero-page location $3E is left as the pointer to the page just after BOOT1, so that BOOT1 knows where it ends. The location $2B is the disk card's slot index (slot×16).

BOOT1_ENTRY performs the following steps:

1. Sets the reset vector to $C600 (reboot).
2. Turns on the drive motor.
3. Loads track 2 into $B000–$BFFF (base page $B0) via BOOT1_LOAD_TRACK.
4. Pushes $A8 (high byte of return address) onto the stack.
5. Loads track 1 into $A000–$AFFF (base page $A0) via BOOT1_LOAD_TRACK.
6. Pushes $05 (low byte), completing the return address $A805.
7. Seeks back to track 0 via BOOT1_SEEK_TRACK.

8. Executes RTS to enter EALDR at $A806.

SOFTEV ($03F2) and PWREDUP ($03F4) are the Apple II reset vector and its complement byte. IWM_MOTORON ($C0E9) turns on the disk drive motor for slot 6.

800a ⟨*boot1 defines* 800a⟩≡ (921) 802 ▷

```
SOFTEV          EQU     $03F2   ; reset vector (2 bytes)
PWREDUP         EQU     $03F4   ; power-up byte (EOR of SOFTEV bytes)
IWM_MOTORON     EQU     $C0E9   ; IWM motor on (slot-indexed)
DISK_BOOT_ROM   EQU     $C600   ; Disk II boot ROM entry point
```

Defines:
PWREDUP, used in chunks 801, 832, and 837.
SOFTEV, used in chunks 801, 832, and 837.
Uses DISK_BOOT_ROM 674b and IWM_MOTORON 772.

BOOT1_ERROR_FLAG is set by BOOT1_LOAD_TRACK when a timeout occurs.

800b ⟨*boot1 error flag* 800b⟩≡ (921)

```
    ORG     $09C7
BOOT1_ERROR_FLAG:
    HEX     00              ; nonzero = error occurred
```

Defines:
BOOT1_ERROR_FLAG, used in chunks 801 and 816.

801 ⟨*boot1 entry* 801⟩≡ (921)

```
      ORG     $0800
  BOOT1_SECTOR_COUNT:
      HEX     05              ; sector count: 5 sectors in BOOT1
      JMP     BOOT1_ENTRY

      ORG     $0804
  BOOT1_ENTRY:
      SUBROUTINE BOOT1_ENTRY

      STOW    DISK_BOOT_ROM,SOFTEV ;  reset vector = $C600 (reboot)
      EOR     #$A5            ;  PWREDUP = $00 EOR $C6 EOR $A5 = $63
      STA     PWREDUP
      STOB    #$00,BOOT1_ERROR_FLAG ; clear error flag
      LDA     IWM_MOTORON     ; turn on drive motor

      ; Load track 2 -> $B000-$BFFF
      STOB    #$B0,$3E ; base page = $B0
      LDA     #$02            ; track 2
      JSR     BOOT1_LOAD_TRACK

      ; Load track 1 -> $A000-$AFFF
      STOB    #$A0,$3E ; base page = $A0

      ; Push $A8 (high byte of EALDR entry - 1)
      LDA     #$A8
      PHA

      ; Load track 1 -> $A000-$AFFF
      LDA     #$01            ; track 1
      JSR     BOOT1_LOAD_TRACK

      ; Push $05 (low byte), completing return address $A805 -> RTS enters $A806
      LDA     #$05
      PHA

      ; Seek to track 0 and wait
      LDA     #$00
      JSR     BOOT1_SEEK_TRACK

      ; Check for errors, retry or enter EALDR
      LDA     BOOT1_ERROR_FLAG
      BNE     BOOT1_ENTRY     ; retry on error
      JMP     BOOT1_DELAY     ; delay, then RTS -> $A806
```

Defines:
BOOT1_ENTRY, never used.
BOOT1_SECTOR_COUNT, never used.

Uses BOOT1_DELAY 810b, BOOT1_ERROR_FLAG 800b, BOOT1_LOAD_TRACK 816, BOOT1_SEEK_TRACK 809, DISK_BOOT_ROM 674b, IWM_MOTORON 772, PWREDUP 800a, SOFTEV 800a, STOB 8c, and STOW 8a.

The region at $0840–$08FF is part of the san inc hard-disk conversion. As described in the introduction (§24), a "san inc crack" rewrites the disk access routines to run from a hard disk rather than floppy. This code replaces whatever originally occupied $0840–$08FF in the EA bootloader (likely a portion of the copy-protection check, which would probably have called the now-unused EALDR_PROT_ENTRY routine).

The san inc code performs the following:

1. Checks for a language card by accessing $C089 twice.
2. If present: BLOADs ASM.CODE.1 and ASM.CODE.2 into language card RAM at $D000, then enters the Applesoft interpreter at $E000.
3. If absent: prints "NO LANGUAGE CARD INSTALLED" via ROM_COUT1 and jumps to the monitor warm-start at $03D0.

Separately, the Passport automatic crack (logged in the accompanying .txt file) applies three single-byte patches to the EALDR on track 1:

1. $A907: VMCALL1 ($01) → VMLD ($04). The original calls native 6502 code at $A000 as a copy-protection subroutine; the patch changes it to a harmless load-and-discard.
2. $A55B ($075B in resident code): BNE offset $A7 → $00. Disables a backward branch in a secondary protection check.
3. $A3CC ($05CC in resident code): $E1 → $E0. Modifies a byte in what appears to be a second VM bytecode program embedded in the resident code area—Passport's source calls this "p-code madness."

ROM_COUT1 ($FDED) is the standard Apple II character output routine. ROM_CRDO ($FDDA) outputs a carriage return.

802 ⟨*boot1 defines* 800a⟩+≡ (921) ◁800a

```
ROM_COUT1       EQU     $FDED   ; character output
ROM_CRDO        EQU     $FDDA   ; output carriage return
```

Uses ROM_COUT1 571a.

803 ⟨*boot1 saninc* 803⟩≡ (921)

```
    ORG     $0840
BOOT1_SANINC:
    SUBROUTINE BOOT1_SANINC
    ; --- Language card present path ---
    HEX     89 C0           ; BIT #$C0 (65C02) -- no-op on NMOS 6502
    LDA     $C089           ; read RAM bank 2 (two accesses to enable)
.print1:
    LDA     .s_bload1,Y     ; \
    BEQ     .exec1          ;  | print BLOAD command string 1
    JSR     ROM_COUT1       ;  |
    INY                     ;  |
    JMP     .print1         ; /
.exec1:
    LDA     $C083           ; \  read/write RAM bank 2
    LDA     $C083           ;  | (two accesses)
    LDA     $C083           ; /  (third for good measure)
    LDY     #$00
.print2:
    LDA     .s_bload2,Y     ; \
    BEQ     .exec2          ;  | print BLOAD command string 2
    JSR     ROM_COUT1       ;  |
    INY                     ;  |
    JMP     .print2         ; /
.exec2:
    LDA     $C080           ; read RAM bank 2 (enable lang card)
    JMP     $E000           ; enter Applesoft interpreter

    ; --- No language card path ---
BOOT1_SANINC_NOLC:
    LDX     #$00
.print_err:
    LDA     .s_no_langcard,X ; \
    BEQ     .done           ;  | print "NO LANGUAGE CARD INSTALLED"
    JSR     ROM_COUT1       ;  |
    INX                     ;  |
    BNE     .print_err      ; /
.done:
    LDA     $C081           ; read ROM, write RAM bank 2
    JMP     $03D0           ; monitor warm-start

    ; High-bit ASCII strings
.s_no_langcard:
    HEX     CE CF A0 CC C1  ; "NO LANGUAGE CARD INSTALLED" (high-bit)
    HEX     CE C7 D5 C1 C7
    HEX     C5 A0 C3 C1 D2
    HEX     C4 A0 C9 CE D3
    HEX     D4 C1 CC CC C5
    HEX     C4 8D 87 00
.s_bload1:
```

```
    HEX     8D 84 C2 CC CF  ; CR, Ctrl-D, "BLOAD ASM.CODE.1,A$D000" (high-bit)
    HEX     C1 C4 A0 C1 D3
    HEX     CD AE C3 CF C4
    HEX     C5 AE B1 AC C1
    HEX     A4 C4 B0 B0 B0
    HEX     8D 00
.s_bload2:
    HEX     8D 84 C2 CC CF  ; CR, Ctrl-D, "BLOAD ASM.CODE.2,A$D000" (high-bit)
    HEX     C1 C4 A0 C1 D3
    HEX     CD AE C3 CF C4
    HEX     C5 AE B2 AC C1
    HEX     A4 C4 B0 B0 B0
    HEX     8D
.s_mono:
    HEX     84 CD CF CE CF  ; Ctrl-D, "MONO,I,C" (high-bit)
    HEX     AC C9 AC C3 8D
    HEX     00

    ; Padding and small utility
    DS      6,$00           ; zero padding
BOOT1_PRINT_CR:
    JSR     ROM_CRDO
    RTS

    DS      22,$00          ; zero padding
    HEX     B7 FF           ; reset vector complement
```

Defines:
BOOT1_CRACK, never used.
BOOT1_PRINT_CR, never used.
BOOT1_SANINC_NOLC, never used.
Uses ROM_COUT1 571a.

BOOT1_READ_ERROR and BOOT1_ADDR_ERROR are the two error-return entry points: both set carry and return. Code branches here when a timeout or checksum mismatch occurs.

BOOT1_READ_SECTOR reads and decodes one sector's data field from the disk. It searches for the D5 AA AD data prologue, then reads the 6-and-2 encoded nibbles, decodes them through the translation tables at $0A00 and $0B00, and stores 256 bytes at the address in $44/$45. Returns with carry clear on success, carry set on error.

The routine uses self-modifying code: the STA instructions at $0960, $0987, and $099F have their high byte patched by BOOT1_READ_SECTOR itself (at $0906/$0909, $0916/$0919, $0921/$0924) to set up three different destination regions within the 256-byte output page.

805 ⟨*boot1 read error* 805⟩≡ (921)

```
        ORG     $0900
  BOOT1_READ_ERROR:
        SEC
        RTS
```

Defines:
BOOT1_READ_ERROR, used in chunk 806.

806 ⟨*boot1 read sector* 806⟩≡ (921)

```
    ORG     $0902
BOOT1_READ_SECTOR:
    SUBROUTINE BOOT1_READ_SECTOR

    ; Self-modify STA targets based on destination address ($44/$45)
    LDA     #$00
    LDY     $45             ; destination page
    STA     .sta_1+1        ; \  patch STA $xx00,Y low byte
    STY     .sta_1+2        ;  | patch STA $xx00,Y high byte
    SEC
    SBC     #$54            ; \
    BCS     .no_borrow1     ;  | subtract $54, borrow from page
    DEY                     ;  |
    SEC                     ; /
.no_borrow1:
    STA     .sta_2+1        ; \  patch second STA target
    STY     .sta_2+2        ; /
    SBC     #$57            ; \
    BCS     .no_borrow2     ;  | subtract $57, borrow from page
    DEY                     ; /
.no_borrow2:
    STA     .sta_3+1        ; \  patch third STA target
    STY     .sta_3+2        ; /

    ; Search for data prologue: D5 AA AD
    LDY     #$20            ; timeout counter
.search:
    DEY
    BEQ     BOOT1_READ_ERROR ; timeout
.wait1:
    LDA     $C0EC           ; read nibble
    BPL     .wait1
.check_d5:
    EOR     #$D5
    BNE     .search
    NOP
.wait2:
    LDA     $C0EC
    BPL     .wait2
    BANE    #$AA,.check_d5 ; not AA -> re-check for D5
    NOP
.wait3:
    LDA     $C0EC
    BPL     .wait3
    BANE    #$AD,.check_d5 ; not AD -> re-check for D5

    ; Read 86 bytes of secondary data ($AA-$FF range of page)
    LDY     #$AA
    LDA     #$00
```

```
.read_secondary:
    STA     $26             ; running checksum
.read_secondary_wait:
    LDX     $C0EC
    BPL     .read_secondary_wait
    LDA     $0A00,X         ; decode via primary nibble table
    STA     $0200,Y         ; store in buffer
    EOR     $26             ; update checksum
    INY
    BNE     .read_secondary ; loop $AA..$FF (86 bytes)

    ; Read and decode primary data (86 bytes -> page+$00..$55)
    LDY     #$AA
    BNE     .read_primary_start
.sta_3:
    STA     $FFFF,Y         ; self-modified: page+$AC..$FF
.read_primary_start:
    LDX     $C0EC
    BPL     .read_primary_start
    EOR     $0A00,X         ; decode nibble
    LDX     $0200,Y         ; secondary data
    EOR     $0B00,X         ; secondary decode table
    INY
    BNE     .sta_3          ; loop $AA..$FF (86 bytes)
    PHA                     ; save last decoded value
    AND     #$FC

    ; Read and decode middle data (86 bytes -> page+$56..$AB)
    LDY     #$AA
.read_mid:
    LDX     $C0EC
    BPL     .read_mid
    EOR     $0A00,X
    LDX     $0200,Y
    EOR     $0B01,X         ; offset +1 in decode table
.sta_2:
    STA     $FFFF,Y         ; self-modified: page+$56..$AB
    INY
    BNE     .read_mid

    ; Read and decode final data (84 bytes -> page+$00..$53)
.wait_final1:
    LDX     $C0EC
    BPL     .wait_final1
    AND     #$FC
    LDY     #$AC
.read_final:
    EOR     $0A00,X
    LDX     $01FE,Y         ; secondary data (offset -2)
    EOR     $0B02,X         ; offset +2 in decode table
```

```
.sta_1:
    STA     $FFFF,Y         ; self-modified: page+$00..$55
.wait_final2:
    LDX     $C0EC
    BPL     .wait_final2
    INY
    BNE     .read_final

    ; Verify checksum byte
    AND     #$FC
    EOR     $0A00,X         ; decode final nibble
    LDX     $2B             ; restore slot index
    TAY
    BNE     .checksum_bad   ; checksum must be zero

    ; Read epilogue byte (expect $DE)
.wait_epilogue:
    LDA     $C0EC
    BPL     .wait_epilogue
    BAEQ    #$DE,.ok
.checksum_bad:
    SEC                     ; error: carry set
    HEX     24              ; BIT $zp opcode -- hides next CLC byte
.ok:
    CLC                     ; success: carry clear
    PLA                     ; discard saved byte
    LDY     #$55
    STA     ($44),Y         ; store final byte
    RTS
```

Defines:
BOOT1_READ_SECTOR, used in chunk 816.
Uses BOOT1_READ_ERROR 805.

BOOT1_TRACK is the current track number, modified by BOOT1_SEEK_TRACK.

808 ⟨*boot1 track vars* 808⟩≡ (921)

```
    ORG     $09C6
BOOT1_TRACK:
    HEX     00              ; current track number
```

Defines:
BOOT1_TRACK, used in chunks 809 and 810a.

BOOT1_SEEK_TRACK moves the drive head to the track in A. The target track (doubled for half-track stepping) is compared against the current position. It steps inward or outward one half-track at a time, activating the stepper phases via BOOT1_PHASE_ON and waiting with BOOT1_DELAY between steps.

809 ⟨*boot1 seek track* 809⟩≡ (921)

```
    ORG     $09C8
BOOT1_SEEK_TRACK:
    SUBROUTINE BOOT1_SEEK_TRACK

    ASL                     ; double for half-track addressing
    STA     $26             ; target half-track
.compare:
    MOVB    BOOT1_TRACK,$27 ; save current position
    SEC
    SBC     $26             ; current - target
    BEQ     BOOT1_PHASE_ON_RTS ; already there -> RTS at $0A04
    BCS     .step_out
    INC     BOOT1_TRACK     ; step inward
    BCC     .do_step
.step_out:
    DEC     BOOT1_TRACK     ; step outward
.do_step:
    JSR     BOOT1_PHASE_ON  ; activate stepper phase
    JSR     BOOT1_DELAY     ; wait for head to settle
    LDA     $27             ; previous half-track
    AND     #$03
    ASL                     ; phase index x 2
    ORA     #$60            ; $C0x0 slot offset
    TAY
    LDA     $C080,Y         ; deactivate previous phase
    JSR     BOOT1_DELAY     ; wait
    BEQ     .compare        ; always taken (delay returns Z=1)
    JSR     BOOT1_DELAY     ; (unreachable)
```

Defines:
BOOT1_SEEK_TRACK, used in chunks 801 and 816.
Uses BOOT1_DELAY 810b, BOOT1_PHASE_ON 810a, BOOT1_PHASE_ON_RTS 810a, BOOT1_TRACK 808, and MOVB 8c.

BOOT1_PHASE_ON activates the stepper motor phase corresponding to the current half-track position.

810a ⟨*boot1 phase on* 810a⟩≡ (921)

```
      ORG     $09F8
  BOOT1_PHASE_ON:
      SUBROUTINE BOOT1_PHASE_ON

      LDA     BOOT1_TRACK
      AND     #$03            ; phase = track mod 4
      ASL                     ; x 2 for IWM register spacing
      ORA     #$60            ; slot offset $60 (slot 6)
      TAY
      LDA     $C081,Y         ; activate phase (odd address = on)
  BOOT1_PHASE_ON_RTS:
      RTS
```

Defines:
BOOT1_PHASE_ON, used in chunk 809.
BOOT1_PHASE_ON_RTS, used in chunk 809.
Uses BOOT1_TRACK 808.

BOOT1_DELAY is a nested delay loop providing approximately $40 \times 256 \times 5 = 51{,}200$ cycles of delay, giving the drive head time to settle after a step.

810b ⟨*boot1 delay* 810b⟩≡ (921)

```
      ORG     $0A05
  BOOT1_DELAY:
      SUBROUTINE BOOT1_DELAY

      LDA     #$28            ; outer counter = 40
      SEC
  .outer:
      PHA
  .inner:
      SBC     #$01
      BNE     .inner          ; inner loop: 256 iterations
      PLA
      SBC     #$01
      BNE     .outer          ; outer loop: 40 iterations
      RTS
```

Defines:
BOOT1_DELAY, used in chunks 801 and 809.

BOOT1_ADDR_ERROR is the address-field error return.

811 ⟨*boot1 addr error* 811⟩≡ (921)

```
      ORG     $0A13
  BOOT1_ADDR_ERROR:
      SEC
      RTS
```

Defines:
BOOT1_ADDR_ERROR, used in chunk 812.

BOOT1_FIND_SECTOR searches for an address field prologue (D5 AA 96) and reads the four two-bit-encoded header bytes: volume, track, sector, and checksum. The decoded sector number is stored in $2D. Returns with carry clear on success.

812 ⟨*boot1 find sector* 812⟩≡ (921)

```
    ORG     $0A15
BOOT1_FIND_SECTOR:
    SUBROUTINE BOOT1_FIND_SECTOR

    ; Timeout counter
    LDY     #$FC
    STY     $26             ; outer counter (high byte)
.retry:
    INY
    BNE     .wait_d5
    INC     $26
    BEQ     BOOT1_ADDR_ERROR ; timeout

    ; Search for D5 AA 96 address prologue
.wait_d5:
    LDA     $C0EC
    BPL     .wait_d5
.check_d5:
    BANE    #$D5,.retry
    NOP
.wait_aa:
    LDA     $C0EC
    BPL     .wait_aa
    BANE    #$AA,.check_d5 ; not AA -> re-check D5
    LDY     #$03            ; 4 header fields to read
.wait_96:
    LDA     $C0EC
    BPL     .wait_96
    BANE    #$96,.check_d5 ; not 96 -> re-check D5

    ; Read 4 encoded header bytes (volume, track, sector, checksum)
    LDA     #$00
.read_header:
    STA     $27             ; running checksum
.wait_odd:
    LDA     $C0EC           ; \
    BPL     .wait_odd       ;  | read odd-bits byte
    ROL                     ;  |
    STA     $26             ; /  save shifted value
.wait_even:
    LDA     $C0EC           ; \
    BPL     .wait_even      ;  | read even-bits byte
    AND     $26             ;  | combine with odd bits
    STA     $2C,Y           ;  | store at $2F,$2E,$2D,$2C (Y=3..0)
    EOR     $27             ;  | update checksum
```

```
    DEY                     ;  |
    BPL     .read_header    ; /  loop for all 4 fields

    ; Verify checksum
    TAY
    BNE     BOOT1_ADDR_ERROR ; checksum must be zero

    ; Read address epilogue: DE AA
.wait_de:
    LDA     $C0EC
    BPL     .wait_de
    BANE    #$DE,BOOT1_ADDR_ERROR
    NOP
.wait_aa2:
    LDA     $C0EC
    BPL     .wait_aa2
    BANE    #$AA,BOOT1_ADDR_ERROR
    CLC                     ; success
```

Defines:
BOOT1_FIND_SECTOR, used in chunk 816.
Uses BOOT1_ADDR_ERROR 811.

The nibble translation tables at $0A70–$0AFF and $0B00–$0BFF are used by BOOT1_READ_SECTOR to decode the 6-and-2 encoded disk nibbles back into data bytes. The primary table at $0A00 maps raw disk nibbles (high-bit set values in the range $96–$FF) to 6-bit values. The secondary table at $0B00 provides the pair-decode lookup.

814 ⟨*boot1 nibble tables* 814⟩≡ (921)

```
    ORG     $0A70
BOOT1_NIBBLE_TABLE_A:
    ; $0A70-$0A97: secondary nibble decode + ASCII text fragment (40 bytes)
    HEX     60 20 6D 6F 64 69 66 79 20 74 68 65 20 73 6F 75
    HEX     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
    HEX     00 00 00 00 00 00 00 04
    ; $0A98-$0AFF: primary 6-and-2 decode table (104 bytes)
    HEX     98 99 08 0C 9C 10 14 18
    HEX     A0 A1 A2 A3 A4 A5 1C 20
    HEX     A8 A9 AA 24 28 2C 30 34
    HEX     B0 B1 38 3C 40 44 48 4C
    HEX     B8 50 54 58 5C 60 64 68
    HEX     C0 C1 C2 C3 C4 C5 C6 C7
    HEX     C8 C9 CA 6C CC 70 74 78
    HEX     D0 D1 D2 7C D4 D5 80 84
    HEX     D8 88 8C 90 94 98 9C A0
    HEX     E0 E1 E2 E3 E4 A4 A8 AC
    HEX     E8 B0 B4 B8 BC C0 C4 C8
    HEX     F0 F1 CC D0 D4 D8 DC E0
    HEX     F8 E4 E8 EC F0 F4 F8 FC
```

Defines:
BOOT1_NIBBLE_TABLE_A, never used.

815a ⟨*boot1 nibble tables b* 815a⟩≡ (921)

```
    ORG     $0B00
BOOT1_NIBBLE_TABLE_B:
    ; 256-byte secondary decode table
    HEX     00 00 00 96 02 00 00 97 01 00 00 9A 03 00 00 9B
    HEX     00 02 00 9D 02 02 00 9E 01 02 00 9F 03 02 00 A6
    HEX     00 01 00 A7 02 01 00 AB 01 01 00 AC 03 01 00 AD
    HEX     00 03 00 AE 02 03 00 AF 01 03 00 B2 03 03 00 B3
    HEX     00 00 02 B4 02 00 02 B5 01 00 02 B6 03 00 02 B7
    HEX     00 02 02 B9 02 02 02 BA 01 02 02 BB 03 02 02 BC
    HEX     00 01 02 BD 02 01 02 BE 01 01 02 BF 03 01 02 CB
    HEX     00 03 02 CD 02 03 02 CE 01 03 02 CF 03 03 02 D3
    HEX     00 00 01 D6 02 00 01 D7 01 00 01 D9 03 00 01 DA
    HEX     00 02 01 DB 02 02 01 DC 01 02 01 DD 03 02 01 DE
    HEX     00 01 01 DF 02 01 01 E5 01 01 01 E6 03 01 01 E7
    HEX     00 03 01 E9 02 03 01 EA 01 03 01 EB 03 03 01 EC
    HEX     00 00 03 ED 02 00 03 EE 01 00 03 EF 03 00 03 F2
    HEX     00 02 03 F3 02 02 03 F4 01 02 03 F5 03 02 03 F6
    HEX     00 01 03 F7 02 01 03 F9 01 01 03 FA 03 01 03 FB
    HEX     00 03 03 FC 02 03 03 FD 01 03 03 FE 03 03 03 FF
```

Defines:
BOOT1_NIBBLE_TABLE_B, never used.

The interleave table maps physical sector numbers to page offsets within the target region. The destination page table is computed at runtime by adding the base page to each interleave entry.

815b ⟨*boot1 interleave table* 815b⟩≡ (921)

```
    ORG     $0C48
BOOT1_INTERLEAVE:
    HEX     0F 08 01 09 02 0A 03 0B 04 0C 05 0D 06 0E 07 00
```

Defines:
BOOT1_INTERLEAVE, used in chunk 816.

BOOT1_LOAD_TRACK is the sector loader. It takes a track number in A and reads all 16 sectors:

1. Seeks to the target track via BOOT1_SEEK_TRACK.
2. Builds the destination page table by adding the base page ($3E) to each entry of the interleave table at $0C48.
3. For each sector found by BOOT1_FIND_SECTOR, looks up the destination page in the table and calls BOOT1_READ_SECTOR to decode the data.
4. Zeros the table entry after a successful read so it will be skipped on subsequent passes.
5. Loops until all 16 entries are zero (all sectors read).

A retry counter at $40 prevents infinite loops on persistent errors.

816 ⟨*boot1 load track* 816⟩≡ (921)

```
    ORG     $0C00
BOOT1_LOAD_TRACK:
    SUBROUTINE BOOT1_LOAD_TRACK

    JSR     BOOT1_SEEK_TRACK ; seek to track in A

    ; Build destination page table: interleave[i] + base page
    LDY     #$0F
.build_table:
    LDA     BOOT1_INTERLEAVE,Y
    CLC
    ADC     $3E             ; add base page
    STA     BOOT1_DEST_PAGES,Y
    DEY
    BPL     .build_table

    LDX     $2B             ; slot index
    STOB    #$60,$40 ; retry counter
.retry:
    DEC     $40
    BEQ     .timeout
    JSR     BOOT1_FIND_SECTOR ; find address field -> sector in $2D
    BCS     .retry          ; error -> retry

    LDY     $2D             ; sector number
    LDA     BOOT1_DEST_PAGES,Y ; destination page for this sector
    BEQ     .retry          ; zero = already read, skip
    STA     $45             ; destination page
    STOB    #$00,$44 ; destination = $xx00
    JSR     BOOT1_READ_SECTOR ; read and decode data field
    BCS     .retry          ; error -> retry
```

```
    ; Mark sector as done
    LDY     $2D
    LDA     #$00
    STA     BOOT1_DEST_PAGES,Y ; zero = done

    ; Check if all sectors read
    LDY     #$0F
.check_done:
    LDA     BOOT1_DEST_PAGES,Y
    BNE     .retry          ; still have sectors to read
    DEY
    BPL     .check_done
    RTS                     ; all 16 sectors read

.timeout:
    INC     BOOT1_ERROR_FLAG ; signal error
    RTS
```

Defines:
BOOT1_LOAD_TRACK, used in chunks 801 and 817.
Uses BOOT1_DEST_PAGES 817, BOOT1_ERROR_FLAG 800b, BOOT1_FIND_SECTOR 812, BOOT1_INTERLEAVE 815b, BOOT1_READ_SECTOR 806, BOOT1_SEEK_TRACK 809, and STOB 8c.

817 ⟨*boot1 dest pages* 817⟩≡ (921)

```
    ORG     $0C58
BOOT1_DEST_PAGES:
    DS      16,$00          ; filled at runtime by BOOT1_LOAD_TRACK
```

Defines:
BOOT1_DEST_PAGES, used in chunk 816.
Uses BOOT1_LOAD_TRACK 816.

The remainder of page $0C contains a fragment of documentation for the EA Loader system. This is not crack-inserted text—it is residual data left over from the original EA development tools. The text describes LOADER.O, a utility program distributed to EA authors for writing the loader to disk. The fragment begins mid-sentence ("ll of the above.") and is truncated at the end ("modifyin"), indicating that the full documentation was longer and this region preserves only the portion that happened to fall within BOOT1's 5-sector footprint.

818 ⟨*boot1 loader doc* 818⟩≡ (921)

```
    ORG     $0C68
BOOT1_LOADER_DOC:
    DC      "ll of the above."
    HEX     0D 10 20 0D 10 20       ; CR, DLE, space, CR, DLE, space
    DC      "The authors all receive a program called LOADER.O, which writes the loader"
    HEX     0D 10 20                ; CR, DLE, space
    DC      "to disk.  To create a new LOADER.O after modifyin"
    HEX     0C 0C                   ; form feed, form feed
    DC      "an"
```

Defines:
BOOT1_LOADER_DOC, used in chunk 860.

Chapter 25

EALDR: The Electronic Arts Loader

`EALDR` is a mini virtual machine with 18 opcodes, whose purpose is to load the game data from disk, decrypt a portion of it, and transfer control to the main program at `$0800`. It occupies `$A000`–`$BFFF`, with the entry point at `$A806`.

25.1 Unused copy-protection VM ($A000–$A2FF)

The first 768 bytes of the EALDR contain a complete, self-contained copy-protection virtual machine that is never executed. The EALDR VM bytecodes at $A8CC checksum this region to verify it is intact, but the code itself is dead—the 4am crack neutralized the call path.

The structure is identical to the Disk VM at $B300+ and the Resident VM at $0569+: a data field verifier, an address field search routine, a VM execution loop with context swap, an RTS-trick dispatcher with a 13-entry opcode table, and opcode handlers using the same zero-page layout ($50–$5F).

Layout.

- $A000–$A002: `JMP` to VM loop.
- $A003–$A068: Data field verifier (searches for D5 AA AD, verifies 342 nibbles, checks DE AA epilogue).
- $A069–$A071: VM execution loop.

- $A072–$A0FF: VM bytecodes (copy-protection program).
- $A100–$A14C: Address field search (D5 AA 96 prologue).
- $A14D–$A169: Read-and-verify wrapper.
- $A16A–$A181: More VM bytecodes.
- $A182–$A18B: Delay loop.
- $A18C–$A1A5: VM bytecodes (continued).
- $A1A6–$A1B2: VM exit routines (OK/error) and XOR helper.
- $A1B3–$A1D2: VM dispatch (RTS-trick).
- $A1D3–$A1DF: Dispatch table (13 one-byte offsets, page $A2).
- $A1E0–$A1F1: Context swap (ZP $50–$5F ↔ $A2E5).
- $A1F2–$A1FF: High-ASCII string "DON'T BREAK TH" (truncated message, similar to `RESIDUAL_DONT_BREAK` at $06F2).
- $A200–$A2DE: VM opcode handlers.
- $A2DF–$A2E1: Dispatch trampoline (`JMP` to handler page).
- $A2E2–$A2E4: Error accumulator and scratch.
- $A2E5–$A2F5: Saved ZP state (shadow of $50–$5F + padding).
- $A2F6–$A2FF: Scratch storage.

The protection VM uses a different opcode numbering than the main EALDR VM at $A806. The following macros encode the protection VM bytecodes with the same XOR encryption ($03/$D9 for addresses, $4C for immediates).

820 ⟨*EALDR prot VM macros* 820⟩≡ (922)

```
; Protection VM bytecode macros (different opcode table from main EALDR VM)
MACRO PVMGOTO  ;  $00 addr
    DC.B $00, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO PVMCALL1 ;  $01 addr -- call native, ACC in/out
    DC.B $01, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO PVMBNE   ;  $02 addr -- branch if ACC != 0
    DC.B $02, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO PVMLDI   ;  $03 val  -- load immediate
    DC.B $03, [{1}] ^ $4C
ENDM
MACRO PVMLD    ;  $04 addr -- load absolute
```

```
    DC.B $04, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO PVMCALLS ;  $05 addr -- VM subroutine call (saves IP)
    DC.B $05, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO PVMST    ;  $06 addr -- store ACC
    DC.B $06, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO PVMSUBI  ;  $07 val  -- ACC -= immediate
    DC.B $07, [{1}] ^ $4C
ENDM
MACRO PVMCALL  ;  $08 addr -- call via indirect
    DC.B $08, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO PVMRET   ;  $09      -- return from VM subroutine
    DC.B $09
ENDM
MACRO PVMINC   ;  $0A addr -- increment and load
    DC.B $0A, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO PVMHALT  ;  $0B      -- halt (exit VM)
    DC.B $0B
ENDM
MACRO PVMLDX   ;  $0C addr -- load indexed by ACC
    DC.B $0C, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
```

822 ⟨*EALDR unused prot vm* 822⟩≡ (922)

```
    ORG     $A000
EALDR_PROT_VM:
    SUBROUTINE EALDR_PROT_VM
    ; Unused copy-protection VM ($A000-$A2FF).
    ; Checksummed by EALDR VM bytecodes but never executed.

    ; --- $A000: entry trampoline ---
    JMP     .vm_loop

    ; --- $A003: VERIFY_DATA_FIELD ---
    ; D5 AA AD prologue, verify 342 nibbles = $AF, DE AA epilogue.
.verify_data:
    LDY     #$20
.vd_retry:
    DEY
    BEQ     .vd_fail
.vd_wait_d5:
    LDA     $C0EC
    BPL     .vd_wait_d5
.vd_chk_d5:
    EOR     #$D5
    BNE     .vd_retry
.vd_wait_aa:
    LDA     $C0EC
    BPL     .vd_wait_aa
    CMP     #$AA
    BNE     .vd_chk_d5
.vd_wait_ad:
    LDA     $C0EC
    BPL     .vd_wait_ad
    CMP     #$AD
    BNE     .vd_chk_d5
    PHA
    PLA
    LDY     #$56
.vd_sec:
    LDA     $C0EC
    BPL     .vd_sec
    BIT     $C000
    CMP     #$AF
    BNE     .vd_bad
    DEY
    BNE     .vd_sec
    LDY     #$00
.vd_pri:
    LDA     $C0EC
    BPL     .vd_pri
    BIT     $C000
    CMP     #$AF
```

```
    BNE     .vd_bad
    DEY
    BNE     .vd_pri
.vd_epi:
    LDY     $C0EC
    BPL     .vd_epi
    PHA
    PLA
.vd_de:
    LDA     $C0EC
    BPL     .vd_de
    CMP     #$DE
    BNE     .vd_fail
.vd_aa2:
    LDA     $C0EC
    BPL     .vd_aa2
    CMP     #$AA
    BEQ     .vd_ok
.vd_fail:
    SEC
    RTS
.vd_ok:
    CLC
    RTS
.vd_bad:
    STA     EALDR_PROT_LAST_BAD
    SEC
    RTS

    ; --- $A069: VM execution loop ---
.vm_loop:
    JSR     .context_swap
    JSR     .vm_dispatch
    JMP     .vm_loop

    ; --- $A072: VM bytecodes (copy-protection program) ---
    ; Verifies disk integrity: checks specific bytes in the data field
    ; verifier ($A060-$A068), then reads/verifies sectors on multiple
    ; tracks using stepper phase manipulation.
    ; Last 2 bytes ($A0FE-$A0FF) = LDY #$FF entry for find_sector.
EALDR_PROT_BYTECODES_1:
    ; --- check data field verifier integrity ---
    PVMLD   $A060              ; load byte at $A060 (SEC = $38)
    PVMSUBI #$38               ; subtract expected value
    PVMBNE  EALDR_PROT_BYTECODES_3 ; mismatch -> error handler
    PVMLD   $A061              ; load byte at $A061 (RTS = $60)
    PVMSUBI #$60
    PVMBNE  EALDR_PROT_BYTECODES_3
    PVMLD   $A067              ; load byte at $A067 (SEC = $38)
    PVMSUBI #$38
```

```
    PVMBNE  EALDR_PROT_BYTECODES_3
    ; --- read and verify sectors on two tracks ---
    PVMCALLS .prot_read_track  ; read track (subroutine at $A0A7)
    PVMBNE  .prot_try2         ; nonzero errors -> try again
    PVMGOTO .prot_ok           ; zero errors -> success
.prot_try2:
    PVMCALLS .prot_read_track
    PVMBNE  .prot_chk_err
    PVMGOTO .prot_ok
.prot_chk_err:
    PVMSUBI #$02               ; allow up to 2 errors
    PVMBNE  EALDR_PROT_BYTECODES_3 ; too many -> error handler
    PVMGOTO EALDR_PROT_BYTECODES_1 ; retry from start
    ; --- success: call vm_exit_ok ---
.prot_ok:
    PVMCALL .vm_exit_ok        ; exit VM with carry clear
    ; --- subroutine: read one track ---
.prot_read_track:
    PVMLD   $C0E9              ; motor on
    PVMLDI  #$FF               ;   \  delay via ROM_WAIT
    PVMCALL1 $FCA8             ;   /
    PVMLDI  #$FF               ;   \  delay again
    PVMCALL1 $FCA8             ;   /
    PVMLD   $C0EE              ; read mode
    PVMLDI  #$00
    PVMST   EALDR_PROT_ERR_ACC ; clear error counter
    PVMCALLS .prot_verify_4    ; verify 4 sectors
    PVMCALLS .prot_verify_4    ; verify 4 more
    PVMCALLS .prot_verify_4
    PVMCALLS .prot_verify_4    ; 16 total
    PVMLD   $C0E8              ; motor off
    PVMLD   EALDR_PROT_ERR_ACC ; return error count in ACC
    PVMRET
    ; --- subroutine: verify 4 sectors and step head ---
.prot_verify_4:
    PVMLDI  #$03               ;   \  set retry count
    PVMST   .fs_retries        ;   /  (self-modify find_sector)
    PVMCALL1 .read_verify      ; read + verify sector
    PVMLD   $C0E7              ; stepper phase 3 on
    PVMCALL1 .read_verify
    PVMCALL1 .read_verify
    PVMLD   $C0E1              ; stepper phase 0 on
    PVMCALLS EALDR_PROT_BYTECODES_2 ; delay + phases off
    PVMLDI  #$03
    PVMST   .fs_retries
    PVMCALL1 .read_verify
    PVMLD   $C0E7              ; stepper phase 3 on
    PVMCALL1 .read_verify
    PVMCALL1 .read_verify
    PVMLD   $C0E5              ; stepper phase 2 on
```

```
    PVMCALLS EALDR_PROT_BYTECODES_2 ; delay + phases off
    PVMRET
    ; --- entry for find_sector with Y=#$FF preloaded ---
    LDY     #$FF                ; $A0FE: also part of bytecodes

    ; --- $A100: FIND_SECTOR (D5 AA 96 search) ---
.find_sector:
    LDX     .fs_retries
.fs_d5:
    LDA     $C0EC
    BPL     .fs_d5
    CMP     #$D5
    BEQ     .fs_got_d5
    DEY
    BNE     .fs_d5
    DEX
    BNE     .fs_d5
    RTS
.fs_got_d5:
    LDA     $C0EC
    BPL     .fs_got_d5
    CMP     #$AA
    BEQ     .fs_got_aa
    DEY
    BNE     .fs_d5
    SEC
    RTS
.fs_got_aa:
    LDA     $C0EC
    BPL     .fs_got_aa
    CMP     #$96
    BEQ     .fs_hdr
    DEY
    BNE     .fs_d5
    SEC
    RTS
.fs_hdr:
    LDY     #$02
.fs_pair:
    LDA     $C0EC
    BPL     .fs_pair
    ROL
    STA     $50
.fs_even:
    LDA     $C0EC
    BPL     .fs_even
    AND     $50
    STA     $50
    DEY
    STX     .fs_retries
```

```
    BPL     .fs_pair
    CLC
    LDX     #$01
    RTS
.fs_retries:
    DC.B    $03

    ; --- $A14D: READ_AND_VERIFY ---
.read_verify:
    JSR     EALDR_PROT_BYTECODES_1+140
    BCS     .rv_done
    JSR     .verify_data
.rv_done:
    LDA     #$00
    ADC     EALDR_PROT_ERR_ACC
    STA     EALDR_PROT_ERR_ACC
    LDA     $C0E0
    LDA     $C0E2
    LDA     $C0E4
    LDA     $C0E6
    RTS

    ; --- $A16A: subroutine: delay + turn off stepper phases ---
EALDR_PROT_BYTECODES_2:
    PVMLDI  #$70                ; delay count
    PVMCALL1 .prot_delay        ; call native delay loop
    PVMLD   $C0E0               ; phase 0 off
    PVMLD   $C0E2               ; phase 1 off
    PVMLD   $C0E4               ; phase 2 off
    PVMLD   $C0E6               ; phase 3 off
    PVMLDI  #$28                ; shorter delay
    PVMCALL1 .prot_delay
    PVMRET

    ; --- $A181: delay loop (called by VM CALL1) ---
.prot_delay:
    LDX     #$20
.pd_inner:
    DEX
    BNE     .pd_inner
    SEC
    SBC     #$01
    BNE     .prot_delay
    RTS

    ; --- $A18C: error handler ---
    ; Fills text page with encoded bytes, then halts in infinite loop.
    ; Self-modifying: increments its own ST target address to walk
    ; through the text screen.
EALDR_PROT_BYTECODES_3:
```

```
    PVMST   $0400              ; store to text page 1
    PVMINC  EALDR_PROT_BYTECODES_3+2 ; increment encrypted addr hi of ST
    PVMBNE  EALDR_PROT_BYTECODES_3   ; loop until wraps to 0
    PVMLD   EALDR_PROT_BYTECODES_3+3 ; load next byte (encrypted addr depends on layout)
    PVMCALL1 .xor_d9           ; XOR with $D9 (decode)
    PVMSUBI #$FF               ; subtract $FF
    PVMCALL1 .xor_d9           ; XOR with $D9 (re-encode)
    PVMST   EALDR_PROT_BYTECODES_3+3 ; store back
    PVMGOTO EALDR_PROT_BYTECODES_3   ; loop forever

    ; --- $A1A6: VM exit routines ---
.vm_exit_ok:
    JSR     .context_swap
    CLC
    RTS
.vm_exit_err:
    JSR     .context_swap
    SEC
    RTS
.xor_d9:
    EOR     #$D9
    RTS

    ; --- $A1B3: VM dispatch (RTS-trick) ---
.vm_dispatch:
    TXA
    PHA
    TYA
    PHA
    JSR     .pull_ip
    PLA
    STA     $52
    PLA
    STA     $53
    LDY     #$04
.vm_resume:
    LDA     ($52),Y
    INY
    BNE     .vm_np
    INC     $53
.vm_np:
    TAX
    LDA     .dispatch_tbl,X
    STA     EALDR_PROT_JMP+1
    JMP     EALDR_PROT_JMP

    ; --- $A1D3: dispatch table (13 entries, page $A2) ---
.dispatch_tbl:
    HEX     00 27 55 5F 6D 8A AD B9 4F A2 CE 26 79
```

```
    ; --- $A1E0: context swap ($50-$5F <-> saved state) ---
.context_swap:
    LDX     #$0F
.cs_loop:
    LDA     $50,X
    PHA
    LDA     EALDR_PROT_ZP_SAVE,X
    STA     $50,X
    PLA
    STA     EALDR_PROT_ZP_SAVE,X
    DEX
    BPL     .cs_loop
    RTS

    ; --- $A1F2: "DON'T BREAK TH" (high-ASCII) ---
EALDR_PROT_MESSAGE:
    HEX     C4 CF CE A7 D4 A0 C2 D2 C5 C1 CB A0 D4 C8

    ; --- $A200: VM opcode handlers ---
    ; OP $00: GOTO
EALDR_PROT_OP_GOTO:
    JSR     .read_arg16
    LDA     $54
    STA     $52
    LDA     $55
    STA     $53
    LDY     #$00
    JMP     .vm_resume
    ; read_arg16: decrypt 2-byte address -> $54/$55
.read_arg16:
    LDA     ($52),Y
    EOR     #$03
    INY
    BNE     .ra1
    INC     $53
.ra1:
    STA     $54
    LDA     ($52),Y
    INY
    BNE     .ra2
    INC     $53
.ra2:
    EOR     #$D9
    STA     $55
    RTS
    ; OP $01: CALL1
.op_call1:
    JSR     .read_arg16
    TYA
    PHA
```

```
    LDA     $56
    JSR     .call_ind
    STA     $56
    PLA
    TAY
    JMP     .vm_resume
    ; pull_ip (RTS-trick bootstrap)
.pull_ip:
    PLA
    STA     $54
    PLA
    STA     $55
    PLA
    STA     $52
    PLA
    STA     $53
    INC     $52
    INC     $54
    BNE     .pi_nc
    INC     $55
.pi_nc:
    JMP     ($0054)
    ; OP $05: CALL
.op_call:
    JSR     .read_arg16
.call_ind:
    JMP     ($0054)
    ; OP $02: BNE
.op_bne:
    JSR     .read_arg16
    LDA     $56
    BNE     EALDR_PROT_OP_GOTO+3
    JMP     .vm_resume
    ; OP $03: LDI
.op_ldi:
    LDA     ($52),Y
    INY
    BNE     .ldi_np
    INC     $53
.ldi_np:
    EOR     #$4C
    STA     $56
    JMP     .vm_resume
    ; OP $04: LD
.op_ld:
    JSR     .read_arg16
.ld_do:
    LDX     #$00
    LDA     ($54,X)
    STA     $56
```

```
    JMP     .vm_resume
    ; OP $0B: LDX (indexed)
.op_ldx:
    JSR     .read_arg16
    LDA     $56
    CLC
    ADC     $54
    STA     $54
    BCC     .ldx_nc
    INC     $55
.ldx_nc:
    JMP     .ld_do
    ; OP $05 alt: CALL with IP save
.op_call_s:
    JSR     .read_arg16
    TYA
    CLC
    ADC     $52
    STA     $52
    BCC     .ocs_nc
    INC     $53
.ocs_nc:
    LDA     $52
    PHA
    LDA     $53
    PHA
    LDY     #$00
    JMP     EALDR_PROT_OP_GOTO+3
    ; OP $06: RET
.op_ret:
    PLA
    STA     $53
    PLA
    STA     $52
    LDY     #$00
    JMP     .vm_resume
    ; OP $07: ST
.op_st:
    JSR     .read_arg16
    LDA     $56
    LDX     #$00
    STA     ($54,X)
    JMP     .vm_resume
    ; OP $08: SUBI
.op_subi:
    LDA     ($52),Y
    INY
    BNE     .sub_np
    INC     $53
.sub_np:
```

```
    EOR     #$4C
    STA     $54
    LDA     $56
    SEC
    SBC     $54
    STA     $56
    JMP     .vm_resume
    ; OP $0A: INC
.op_inc:
    JSR     .read_arg16
    LDX     #$00
    LDA     ($54,X)
    CLC
    ADC     #$01
    STA     ($54,X)
    STA     $56
    JMP     .vm_resume

    ; --- $A2DF: dispatch trampoline ---
EALDR_PROT_JMP:
    JMP     EALDR_PROT_OP_GOTO
EALDR_PROT_ERR_ACC:
    DC.B    $00,$00,$00
EALDR_PROT_ZP_SAVE:
    HEX     0E 96 8C 41 96 41 20 D3 D3 D3 D3 D3 D3 D3 D3 D3 D3
EALDR_PROT_LAST_BAD:
    DC.B    $00
    HEX     5E 24 46 46 35 39 0D 02 3B
```

Defines:
EALDR_PROT_VM, never used.
Uses READ_AND_VERIFY 779a, ROM_WAIT 606b, and VERIFY_DATA_FIELD 773.

25.1.1 Resident routines copy ($A300–$A5FF)

The region $A300–$A5FF contains the resident routines that the EALDR copies to $0500–$07FF during boot (see chapter on Resident Routines). This data is byte-identical to the assembled resident routines in `main.bin`. Rather than duplicating the assembly source, this chunk includes the generated file `resident\_copy.asm`, which is extracted from `output/main.bin` by the build pipeline.

831 ⟨*EALDR resident copy* 831⟩≡ (922)

```
    INCLUDE "resident_copy.asm"
```

Defines:
EALDR_RESIDENT_COPY, never used.

25.2 Entry point

The entry code at $A806 fills the hi-res HGR1 screen ($2000--$3FFF) and beyond ($4000--$7FFF) with $FF, copies the EA splash screen data from $B000+ to $202C+, then jumps to the VM execution loop at $A849.

EALDR_SRC ($4C/$4D) is a 2-byte zero-page pointer used by the VM as a general-purpose source address. The entry code uses it to iterate through the hi-res screen pages during the fill loop.

832 ⟨*ealdr defines* 832⟩≡ (922) 845a ▷

```
EALDR_SRC          EQU     $4C      ; VM source pointer (2 bytes)
SOFTEV             EQU     $03F2    ; Apple II reset vector (2 bytes)
PWREDUP            EQU     $03F4    ; power-up byte
DISK_BOOT_ROM      EQU     $C600    ; Disk II boot ROM entry point
```

Uses DISK_BOOT_ROM 674b, PWREDUP 800a, and SOFTEV 800a.

833 ⟨*EALDR entry* 833⟩≡ (922)

```
    ORG     $A806
EALDR_ENTRY:
    LDY     #$00            ; Fill $2000-$7FFF with $FF
    STY     EALDR_SRC       ; ($4C) = pointer low
    STOB   #$20,EALDR_SRC+1 ; ($4D) = $20 (start page)
    LDA     #$FF
.fill:
    STA     (EALDR_SRC),Y
    INY
    BNE     .fill
    INC     EALDR_SRC+1
    BPL     .fill           ; loop until page $80 (sign bit set)

.copy_lo:
    LDX     #$00            ; Copy splash screen from $B000+
.copy_next:
    JSR     EALDR_COPY_BYTE ; copy byte, advance source
    INX
    BXNE    #$20,.copy_next ; X = $00..$1F (first 32 bytes)
    LDX     #$80
.copy_hi:
    JSR     EALDR_COPY_BYTE ; X = $80..$9F (next 32 bytes)
    INX
    BXNE    #$A0,.copy_hi
    INC     EALDR_COPY_DST  ; self-modify: advance dest page
    LDA     EALDR_COPY_DST
    BANE   #$40,.copy_lo    ; done when dest page reaches $40
    JMP     EALDR_VM_EXEC   ; enter VM
```

Defines:
EALDR_ENTRY, never used.
Uses EALDR_COPY_BYTE 834a, EALDR_VM_EXEC 834b, and STOB 8c.

The copy subroutine is self-modifying: the source address at $A83B/$A83C is incremented after each byte:

834a ⟨*EALDR copy subroutine* 834a⟩≡ (922)

```
    ORG     $A83A
EALDR_COPY_BYTE:
EALDR_COPY_SRC = *+1
    LDA     $B000            ; self-modified source address
EALDR_COPY_DST = *+2
    STA     $202C,X          ; dest = HGR page + offset
    INC     EALDR_COPY_SRC   ; advance source (low byte)
    BNE     .done
    INC     EALDR_COPY_SRC+1 ; carry to high byte
.done:
    RTS
```

Defines:
EALDR_COPY_BYTE, used in chunk 833.
EALDR_COPY_SRC, never used.

834b ⟨*EALDR VM exec* 834b⟩≡ (922)

```
    ORG     $A849
EALDR_VM_EXEC:
    LDX     #$30            ; filler (skipped by Y=4)
    JSR     EALDR_VM_INTERP ; call interpreter
    DEX                     ; \  these 3 bytes are skipped;
    BPL     EALDR_VM_EXEC   ; /  the VM never returns here
                               ; VM bytecodes begin at $A851
```

Defines:
EALDR_VM_EXEC, used in chunk 833.
Uses EALDR_VM_INTERP 846.

Byte	Opcode	Arg	Handler	Operation
00	GOTO	addr	$AA72	IP ← addr
01	CALL1	addr	$AAAE	Call 6502 subroutine at addr; A ← result
02	BEQ	addr	$AAC5	If A = 0: IP ← addr, else linear
03	LDI	val	$AACF	A ← val (1-byte immediate)
04	LD	addr	$AADD	A ← (addr)
05	CALL	addr	$AAFA	Push return address; IP ← addr
06	ST	addr	$AB1D	(addr) ← A
07	SUBI	val	$AB2E	A ← A − val (1-byte immediate)
08	CALL0	addr	$AABF	Jump to 6502 code at addr (terminal, no return)
09	RET	none	$AB12	Pop return address; IP ← saved address
0A	LDX	addr	$AAE9	A ← (addr + A) (indexed load)
0B	ASL	none	$AB29	A ← A ×2
0C	INC	addr	$AB7B	A ← ++(addr) (increment memory, result to A)
0D	ADD	addr	$AB8C	A ← A + (addr)
0E	DXR	none	$AB55	(src) ← (src) ⊕ A; src += 2; A ← $\mathtt{src}_H$ ⊕ $68
0F	BNE	addr	$AB6E	If A ≠ 0: IP ← addr, else linear
10	SUB	addr	$AB9B	A ← A − (addr)
11	COPY	none	$ABAA	(dest++) ← (src++)

Table 25.1: EALDR VM opcodes. Opcodes 03 and 07 take a 1-byte XOR $4C argument; opcodes 0B, 0E, 09, and 11 take no arguments; all others take a 2-byte XOR $03/$D9 address argument.

The VM bytecode program occupies $A851–$A9F0, interleaved with the native 6502 subroutines that the VM calls. The bytecodes are XOR-encrypted (address low XOR $03, address high XOR $D9, immediate XOR $4C). The VM macros emit the encrypted bytes directly so the source reads like the decoded VM program.

Two data bytes at $A9F1–$A9F2 (EALDR_TMP0 and EALDR_TMP1) are variables used by the bytecode program, not opcodes.

25.2.1 Opcode table

835 ⟨*EALDR VM macros* 835⟩≡ (909 922) 864 ▷

```
; --- EALDR VM bytecode macros ---
; Each macro emits the opcode byte followed by XOR-encrypted arguments.
; Address args: low ^ $03, high ^ $D9.  Immediate args: ^ $4C.
MACRO VMGOTO  ;  $00 addr -- IP <- addr
    DC.B $00, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO VMCALL1 ;  $01 addr -- call 6502 subroutine, result -> acc
    DC.B $01, <[{1}] ^ $03, >[{1}] ^ $D9
```

```
ENDM
MACRO VMBEQ   ;  $02 addr -- branch if acc = 0
    DC.B $02, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO VMLDI   ;  $03 val  -- load immediate -> acc
    DC.B $03, [{1}] ^ $4C
ENDM
MACRO VMLD    ;  $04 addr -- acc <- (addr)
    DC.B $04, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO VMCALL  ;  $05 addr -- VM subroutine call
    DC.B $05, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO VMST    ;  $06 addr -- (addr) <- acc
    DC.B $06, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO VMSUBI  ;  $07 val  -- acc <- acc - val
    DC.B $07, [{1}] ^ $4C
ENDM
MACRO VMCALL0 ;  $08 addr -- terminal jump to 6502 code
    DC.B $08, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO VMRET   ;  $09      -- return from VM subroutine
    DC.B $09
ENDM
MACRO VMLDX   ;  $0A addr -- acc <- (addr + acc)
    DC.B $0A, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO VMASL   ;  $0B      -- acc <- acc << 1
    DC.B $0B
ENDM
MACRO VMINC   ;  $0C addr -- (addr)++, result -> acc
    DC.B $0C, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO VMADD   ;  $0D addr -- acc <- acc + (addr)
    DC.B $0D, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO VMDXR   ;  $0E      -- DXR decryption step
    DC.B $0E
ENDM
MACRO VMBNE   ;  $0F addr -- branch if acc != 0
    DC.B $0F, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO VMSUB   ;  $10 addr -- acc <- acc - (addr)
    DC.B $10, <[{1}] ^ $03, >[{1}] ^ $D9
ENDM
MACRO VMCOPY  ;  $11      -- (dest++) <- (src++)
    DC.B $11
ENDM
```

Bytecode part 1 begins immediately after the EALDR_VM_EXEC entry point. It performs the initial display setup:

1. Switches the Apple II to hi-res graphics mode (full screen, no text window).
2. Sets the reset vector to EALDR_REBOOT ($A948) via SOFTEV/PWREDUP.
3. Runs a splash screen delay loop (8 iterations of WAIT).
4. Optionally loads Group 1 (tracks $03, $04, $21, $22) if the flag at $A805 is nonzero.
5. Loads Group 2 (tracks $07–$0C) into $4000–$95FF via VM_READ_TRACKS.
6. Clears the text screen via EALDR_CLEAR_TEXT.

The bytecode program sets SOFTEV to $A948 (the address of EALDR_REBOOT minus one, per the RTS convention) and PWREDUP to $0C. The Apple II reset check validates PWREDUP against SOFTEV low EOR high EOR $A5; the correct value would be $48 EOR $A9 EOR $A5 = $44, but the EALDR stores $A9 EOR $A5 = $0C instead, omitting the low byte. This deliberate mismatch means Ctrl-Reset will *not* jump to EALDR_REBOOT—the monitor will detect a bad PWREDUP and cold-start instead. Compare with BOOT1_ENTRY, which sets SOFTEV = $C600 (low = $00) where the omission is harmless.

837 ⟨*EALDR VM bytecode 1* 837⟩≡ (922)

```
    ORG     $A851
EALDR_VM_BYTECODE:
    ; --- Display setup ---
    VMLD    $C050               ; TXTCLR (show graphics)
    VMLD    $C052               ; MIXCLR (full screen)
    VMLD    $C057               ; HIRES (hi-res mode)
    VMLDI   $F8                 ; \
    VMST    EALDR_SRC           ; /  EALDR_SRC = $F8 (delay counter)
    VMLDI   $48                 ; \
    VMST    SOFTEV              ; /  SOFTEV lo = $48
    VMLDI   $A9                 ; \
    VMST    SOFTEV+1            ; /  SOFTEV hi = $A9 -> reset = EALDR_REBOOT
    VMLDI   $0C                 ; \
    VMST    PWREDUP             ; /  PWREDUP = $A9 EOR $A5 = $0C

    ; --- Splash screen delay ---
VM_SPLASH_DELAY:
    VMLDI   $FF                 ; \
    VMCALL1 $FCA8               ;  | WAIT ($FF iterations)
    VMINC   EALDR_SRC           ;  | EALDR_SRC++
    VMBEQ   VM_LOAD_GROUP1      ;  | done when wraps to 0
    VMGOTO  VM_SPLASH_DELAY     ; /  loop
```

```
    ; --- Group 1: optional tracks $03,$04,$21,$22 ---
VM_LOAD_GROUP1:
    VMLD    $A805              ; flag byte (0 = skip Group 1)
    VMBEQ   VM_LOAD_GROUP2     ; skip if zero
    VMLDI   $00                ; \
    VMCALL  VM_READ_TRACKS     ;  | read_tracks(0) -> Group 1
    VMLD    $C0E8              ;  | IWM: motor off
    VMCALL1 $0000              ; /  ???

    ; --- Group 2: tracks $07-$0C -> $4000-$95FF ---
VM_LOAD_GROUP2:
    VMLDI   $05                ; \
    VMCALL  VM_READ_TRACKS     ;  | read_tracks(5) -> Group 2
    VMCALL1 EALDR_CLEAR_TEXT   ; /  clear text screen
    VMGOTO  VM_LOAD_GROUP3     ; continue at bytecode part 2
```

Defines:
EALDR_VM_BYTECODE, never used.
VM_LOAD_GROUP1, never used.
VM_LOAD_GROUP2, never used.
VM_SPLASH_DELAY, never used.

Uses EALDR_CLEAR_TEXT 853a, EALDR_REBOOT 853b, HIRES 571b, MIXCLR 571b, PWREDUP 800a, SOFTEV 800a, TXTCLR 571b, VM_LOAD_GROUP3 839, and VM_READ_TRACKS 841.

Bytecode part 2 continues after the native EALDR_CLEAR_TEXT subroutine. It performs the remaining loading, verification, and decryption steps:

1. Loads Group 3 (tracks $0D–$10) into $0800–$3FFF via VM_READ_TRACKS.
2. Checksums $A000–$A2DF to verify the EALDR's own code; if the result is not $60, branches to VM_ERROR_SPLASH.
3. Seeks to track 5 and turns off the drive motor.
4. Decrypts $4000–$67FF using the VMDXR opcode with the "funny increment" key schedule (EALDR_FUNNY_INC).
5. Copies $A300–$A5FF to $0500–$07FF (the resident routines).
6. Transfers control to the main program at $0800 via VMCALL0 (never returns).

839 ⟨*EALDR VM bytecode 2* 839⟩≡ (922)

```
    ORG     $A8B2
    ; --- Group 3: tracks $0D-$10 -> $0800-$3FFF ---
VM_LOAD_GROUP3:
    VMLD    $A800               ; flag byte (nonzero = show lo-res)
    VMBEQ   VM_DO_GROUP3        ; skip if zero
    VMLD    $C056               ; LORES (lo-res mode)
VM_DO_GROUP3:
    VMLDI   $0C                 ; \
    VMCALL  VM_READ_TRACKS      ;  | read_tracks(12) -> Group 3
    VMLDI   $00                 ; \
    VMST    EALDR_SRC           ;  | EALDR_SRC = 0
    VMST    EALDR_TMP1          ; /  checksum accumulator = 0

    ; --- Checksum verification ($A000-$A2E0) ---
VM_CHECKSUM_LOOP:
    VMLD    EALDR_SRC           ; A = index
    VMLDX   $A000               ; \
    VMSUB   EALDR_TMP1          ;  | checksum -= mem[$A000 + index]
    VMST    EALDR_TMP1          ; /
    VMLD    EALDR_SRC           ; \
    VMLDX   $A100               ;  | checksum -= mem[$A100 + index]
    VMSUB   EALDR_TMP1          ;  |
    VMST    EALDR_TMP1          ; /
    VMINC   EALDR_SRC           ; index++
    VMBNE   VM_CHECKSUM_LOOP    ; loop until index wraps
VM_CHECKSUM_A200:
    VMLD    EALDR_SRC           ; \
    VMLDX   $A200               ;  | checksum -= mem[$A200 + index]
    VMSUB   EALDR_TMP1          ;  |
    VMST    EALDR_TMP1          ; /
    VMINC   EALDR_SRC           ; index++
```

```
    VMSUBI  $E0                 ; \
    VMBNE   VM_CHECKSUM_A200    ; /  loop until index = $E0
    VMLD    EALDR_TMP1          ; checksum result
    VMSUBI  $60                 ; \
    VMBNE   VM_ERROR_SPLASH     ; /  if checksum != $60 -> error

    ; --- DXR decryption ($4000-$67FF) ---
    VMLDI   $05                 ; \
    VMCALL1 EALDR_MOVE_ARM_TRAMPOLINE     ; /  move arm to track 5
    VMLD    $A000               ; (read value, discard)
    VMLDI   $00                 ; \
    VMLD    $C0E8               ; /  IWM: motor off
    VMLDI   $00                 ; \
    VMST    EALDR_SRC           ;  | src = $4000
    VMLDI   $40                 ;  |
    VMST    EALDR_SRC+1         ; /
    VMLDI   $02                 ; \
    VMST    EALDR_BOT           ;  | funny-inc counters: bot=2, top=3
    VMLDI   $03                 ;  |
    VMST    EALDR_TOP           ; /
VM_DXR_LOOP:
    VMCALL1 EALDR_FUNNY_INC    ; advance key schedule
    VMDXR                       ; decrypt one byte
    VMBNE   VM_DXR_LOOP         ; loop until acc = 0 (src high = $68)

    ; --- Copy $A300-$A5FF -> $0500-$07FF ---
    VMLDI   $00                 ; \
    VMST    EALDR_SRC           ;  | src = $A300
    VMST    EALDR_DEST          ;  | dest = $0500
    VMLDI   $A3                 ;  |
    VMST    EALDR_SRC+1         ;  |
    VMLDI   $05                 ;  |
    VMST    EALDR_DEST+1        ; /
VM_COPY_LOOP:
    VMCOPY                      ; copy one byte
    VMLD    EALDR_DEST+1        ; check if dest page reached $08
    VMSUBI  $08                 ; \
    VMBNE   VM_COPY_LOOP        ; /  loop until done (3 pages)

    ; --- Enter main program ---
    VMCALL0 $0800               ; jump to main program (never returns)
```

Defines:
VM_LOAD_GROUP3, used in chunk 837.

Uses EALDR_BOT 854a, EALDR_DEST 848b, EALDR_FUNNY_INC 854b, EALDR_MOVE_ARM_TRAMPOLINE 872, EALDR_TMP1 842, EALDR_TOP 854a, VM_CHECKSUM_A200 842, VM_CHECKSUM_LOOP 842, VM_COPY_LOOP 842, VM_DO_GROUP3 842, VM_DXR_LOOP 842, VM_ERROR_SPLASH 842, and VM_READ_TRACKS 841.

Bytecode part 3 is the entry point of the VM_READ_TRACKS subroutine, located between the native EALDR_REBOOT and EALDR_FUNNY_INC routines:

1. Saves the table index (passed in the VM accumulator) to EALDR_TMP0.
2. Turns on the drive motor.
3. Jumps to the subroutine body in part 4.

841 ⟨*EALDR VM bytecode 3* 841⟩≡ (922)

```
    ORG     $A967
    ; --- read_tracks subroutine (entry) ---
VM_READ_TRACKS:
    VMST    EALDR_TMP0          ; save table index
    VMLD    $C0E9               ; IWM: motor on
    VMGOTO  VM_READ_LOOP        ; continue at bytecode part 4
```

Defines:
VM_READ_TRACKS, used in chunks 837 and 839.
Uses EALDR_TMP0 842 and VM_READ_LOOP 842.

Bytecode part 4 is the body of the VM_READ_TRACKS subroutine, located after the native EALDR_FUNNY_INC routine. The track-loading loop works as follows:

1. Reads EALDR_TRACK_PAGES at the current index to get the destination page; a zero byte terminates the loop (VM_READ_DONE).
2. Stores the destination page in $3E.
3. Reads EALDR_TRACKS at the same index to get the track number.
4. Calls EALDR_READ_TRACK_TRAMPOLINE to load the track; on failure, jumps to VM_ERROR_DISPLAY.
5. Increments the index and loops.

This chunk also contains two error handlers and two data bytes:

- VM_ERROR_SPLASH: displays a '?' and loops with beeping (entered on checksum failure).
- VM_ERROR_DISPLAY: shows "ERR" and halts (entered on disk read failure).
- EALDR_TMP0 and EALDR_TMP1: runtime variables (table index and checksum accumulator), not bytecodes.

842 ⟨*EALDR VM bytecode 4* 842⟩≡ (922)

```
    ORG     $A985
    ; --- read_tracks subroutine (body) ---
VM_READ_LOOP:
    VMLD    EALDR_TMP0          ; A = table index
    VMLDX   EALDR_TRACK_PAGES   ; A = track_pages[index]
    VMBEQ   VM_READ_DONE        ; zero terminator -> return
    VMST    $003E               ; base page = track_pages[index]
    VMLD    EALDR_TMP0                    ; \
    VMLDX   EALDR_TRACKS                  ;  | A = tracks[index]
    VMCALL1 EALDR_READ_TRACK_TRAMPOLINE   ; /  read track
    VMBEQ   VM_READ_NEXT                  ; success -> advance index
    VMGOTO  VM_ERROR_DISPLAY              ; error -> error handler

    ; --- advance to next track ---
VM_READ_NEXT:
    VMLD    EALDR_TMP0          ; \
    VMSUBI  $FF                 ;  | index++ (subtract $FF = add 1 mod 256)
    VMST    EALDR_TMP0          ;  |
    VMGOTO  VM_READ_LOOP        ; /  loop

    ; --- error: splash screen loop ---
VM_ERROR_SPLASH:
    VMLD    $C0E9               ; IWM: motor on
```

```
    VMCALL1 $FC58           ; HOME (clear screen)
    VMLD    $C051           ; TXTSET (show text)
    VMLDI   $BF             ; \
    VMST    $0400           ; /  '?' on screen
    VMLDI   $40             ; \
    VMCALL1 $FCA8           ; /  WAIT ($40)
    VMLD    $C0E8           ; IWM: motor off
VM_ERROR_BEEP:
    VMLDI   $60             ; \
    VMCALL1 $FCA8           ;  | WAIT ($60)
    VMLDI   $08             ;  | WAIT ($08)
    VMCALL1 $FCA8           ;  |
    VMINC   EALDR_TMP1      ;  | increment counter
    VMBNE   VM_ERROR_BEEP   ; /  loop (beep pattern)
    VMGOTO  VM_ERROR_SPLASH ; restart splash loop

    ; --- error: display "EA" and reboot ---
VM_ERROR_DISPLAY:
    VMCALL1 $FC58           ; HOME
    VMCALL1 $FBDD           ; BELL (beep)
    VMLD    $C051           ; TXTSET
    VMLDI   $C5             ; \
    VMST    $0400           ;  | 'E' on screen
    VMLDI   $D2             ;  |
    VMST    $0401           ;  | 'R' on screen (error code)
    VMST    $0402           ; /  'R' again
    VMLD    $C0E8           ; IWM: motor off
VM_ERROR_HALT:
    VMGOTO  VM_ERROR_HALT   ; infinite loop

    ; --- data ---
VM_READ_DONE:
    VMRET                   ; return from read_tracks
EALDR_TMP0:
    DC.B    $00             ; table index (runtime)
EALDR_TMP1:
    DC.B    $00             ; checksum accumulator (runtime)
```

Defines:
EALDR_TMP0, used in chunk 841.
EALDR_TMP1, used in chunk 839.
VM_CHECKSUM_A200, used in chunk 839.
VM_CHECKSUM_LOOP, used in chunk 839.
VM_COPY_LOOP, used in chunk 839.
VM_DO_GROUP3, used in chunk 839.
VM_DXR_LOOP, used in chunk 839.
VM_ERROR_BEEP, never used.
VM_ERROR_DISPLAY, used in chunk 859.
VM_ERROR_HALT, never used.
VM_ERROR_SPLASH, used in chunk 839.
VM_READ_DONE, never used.
VM_READ_LOOP, used in chunk 841.

VM_READ_NEXT, never used.
Uses EALDR_READ_TRACK_TRAMPOLINE 872, EALDR_TRACKS 885b, and EALDR_TRACK_PAGES 885b.

25.3 VM architecture

The VM has the following state:

- **Accumulator** (A): One byte, stored at zero-page $48.
- **Instruction pointer**: A two-byte pointer in $46/$47 plus a Y register offset. The current bytecode address is $46/$47 + Y.
- **Source pointer** (src): $4C/$4D, used by DXR and COPY.
- **Destination pointer** (dest): $4E/$4F, used by COPY.
- The 6502 hardware stack is used for VM subroutine return addresses.

The VM interpreter at $AA35 uses an RTS trick to obtain the bytecode address: it pulls the JSR return address ($A84D) from the stack into $46/$47, then starts reading opcodes at offset Y=4, which lands on $A851—the first bytecode, just past the DEX/BPL bytes. The DEX/BPL loop is never executed; it could serve as a retry mechanism, if X were actually saved and restored, but would then just crash if retries run out, by trying to execute VM bytecode as 6502 code.

25.3.1 Opcode dispatch

The interpreter reads one byte from the bytecode stream as an opcode index. It looks up the index in a dispatch table at $AA60 and computes the handler address as $AA72 + table[opcode]. The JMP instruction at $AA5D is self-modified to jump to the handler.

After a handler executes, control flow takes one of two paths:

- **Linear**: The handler jumps to $AA45, which reads the next opcode from the current stream position (Y was already advanced past the handler's arguments). Used by most opcodes.
- **Goto**: The handler falls through to $AA72, which reads a 2-byte encoded address from the stream, loads it into $46/$47, resets Y=0, and loops back to the dispatcher. Used by GOTO, BEQ, BNE, CALL, and RET.

25.3.2 Bytecode encryption

Arguments in the bytecode stream are XOR-encrypted:

- Address arguments (2 bytes): low byte XOR $03, high byte XOR $D9. The subroutine at $AA82 reads and decodes these into $42/$43.
- Immediate values (1 byte): XOR $4C. Decoded inline by the handler.

25.3.3 VM interpreter

The interpreter at $AA35 saves the accumulator and Y, uses the RTS trick to obtain the bytecode pointer from the caller's return address, then enters the dispatch loop.

EALDR_PTR ($46/$47) is the VM instruction pointer—the current position in the bytecode stream. EALDR_PSUEDOACC ($48) is the VM's single-byte accumulator register.

845a ⟨*ealdr defines* 832⟩+≡ (922) ◁832 848b▷

```
EALDR_PTR          EQU     $46     ; VM instruction pointer (2 bytes)
EALDR_PSUEDOACC    EQU     $48     ; VM accumulator
```

Defines:
EALDR_PSUEDOACC, used in chunks 846, 849, and 855.
EALDR_PTR, used in chunks 846–49.

25.3.4 Dispatch table

The dispatch table at $AA60 holds 18 one-byte offsets. The handler address is $AA72 + offset. Opcodes 0C–11 point to three-byte JMP trampolines at $AB43–$AB54 because their handlers lie beyond the one-byte offset range.

845b ⟨*EALDR dispatch table* 845b⟩≡ (922)

```
    ORG     $AA60
EALDR_DISPATCH_TABLE:
    HEX  00 3C 53 5D 6B 88 AB BC 4D A0 77 B7
    ;    GOTO CALL1 BEQ LDI LD CALL ST SUBI CALL0 RET LDX ASL
    HEX  D1 D4 D7 DA DD E0
    ;    INC  ADD  DXR  BNE  SUB  COPY (trampolines)
```

Defines:
EALDR_DISPATCH_TABLE, used in chunk 846.

846 ⟨*EALDR VM interpreter* 846⟩≡ (922)

```
    ORG     $AA35
    SUBROUTINE EALDR_VM_INTERP
EALDR_VM_INTERP:
    LDA     EALDR_PSUEDOACC         ; save VM accumulator
    PHA
    TYA                             ; save Y
    PHA
    JSR     EALDR_VM_INIT           ; RTS trick: pull return addr -> $46/$47

    ; --- After RTS trick, execution resumes here ---
    PULB    EALDR_PTR ; pull saved Y -> $46 (low byte of return addr)
    PULB    EALDR_PTR+1 ; pull saved acc -> $47 (high byte)

    ; Dispatch loop
    LDY     #$04                    ; skip 4 bytes (DEX/BPL/offset + JSR addr byte)
EALDR_DISPATCH:                 ; $AA45
    LDA     (EALDR_PTR),Y           ; read opcode byte
    INY
    BNE     .no_carry
    INC     EALDR_PTR+1
.no_carry:
    TAX                             ; opcode -> X
    LDA     EALDR_DISPATCH_TABLE,X ; handler offset
    CLC
    ADC     #<EALDR_VM_LOOP         ; handler = $AA72 + offset
    STA     EALDR_VM_JMP+1          ; self-modify JMP low byte
    LDA     #>EALDR_VM_LOOP
    ADC     #$00                    ; carry
    STA     EALDR_VM_JMP+2          ; self-modify JMP high byte
EALDR_VM_JMP:
    JMP     EALDR_VM_LOOP           ; self-modified: jump to handler
```

Defines:
EALDR_DISPATCH, used in chunk 848a.
EALDR_VM_INTERP, used in chunks 834b and 859.
Uses EALDR_DISPATCH_TABLE 845b, EALDR_PSUEDOACC 845a, EALDR_PTR 845a, EALDR_VM_INIT 847b, EALDR_VM_LOOP 848a, and PULB 9c.

The argument reader decodes a 2-byte address from the bytecode stream:

847a ⟨*EALDR VM arg reader* 847a⟩≡ (922)

```
      ORG     $AA82
  EALDR_READ_ARG16:
      LDA      (EALDR_PTR),Y ; read low byte
      EOR      #$03          ; decrypt
      INY
      BNE      .no_carry1
      INC      EALDR_PTR+1
  .no_carry1:
      STA      $42           ; decoded low byte
      LDA      (EALDR_PTR),Y ; read high byte
      INY
      BNE      .no_carry2
      INC      EALDR_PTR+1
  .no_carry2:
      EOR      #$D9          ; decrypt
      STA      $43           ; decoded high byte
      RTS
```

Defines:
EALDR_READ_ARG16, used in chunks 848a and 849.
Uses EALDR_PTR 845a.

The RTS trick subroutine: pulls 4 values from the stack (JSR return address, then the saved Y and accumulator), increments the return address, and jumps back to the instruction after JSR EALDR_VM_INIT:

847b ⟨*EALDR VM init* 847b⟩≡ (922)

```
      ORG     $AA99
      SUBROUTINE EALDR_VM_INIT
  EALDR_VM_INIT:
      PULB    $42 ; return addr low
      PULB    $43 ; return addr high
      PULB    EALDR_PTR ; saved Y -> $46 (overwritten later)
      PULB    EALDR_PTR ; saved acc -> $46 (overwrites Y)
      INC      $42       ; return addr + 1
      BNE      .no_carry
      INC      $43
  .no_carry:
      JMP      ($0042)   ; jump to instruction after JSR EALDR_VM_INIT
```

Defines:
EALDR_VM_INIT, used in chunk 846.
Uses EALDR_PTR 845a and PULB 9c.

The goto path at $AA72 is entered when an opcode handler needs to set the instruction pointer from a decoded address in $42/$43:

848a ⟨*EALDR VM goto path* 848a⟩≡ (922)

```
      ORG     $AA72
  EALDR_VM_LOOP:
      JSR      EALDR_READ_ARG16 ; read 2-byte encoded address -> $42/$43
  EALDR_VM_GOTO:                  ; $AA75 - entry when addr already in $42/$43
      MOVW    $42,EALDR_PTR ; update bytecode pointer
      LDY      #$00
      JMP      EALDR_DISPATCH   ; read next opcode
```

Defines:
EALDR_VM_GOTO, used in chunk 849.
EALDR_VM_LOOP, used in chunk 846.
Uses EALDR_DISPATCH 846, EALDR_PTR 845a, EALDR_READ_ARG16 847a, and MOVW 9a.

25.3.5 Opcode handlers

Each handler is reached via the self-modifying JMP at $AA5D. After execution, handlers jump to either $AA45 (linear: next opcode from current stream position) or $AA75 (goto: set IP from $42/$43 and dispatch).

EALDR_VM_DISPATCH ($AA45) is the dispatch loop entry point. Opcode handlers jump here to continue with the next bytecode instruction. EALDR_DEST ($4E/$4F) is the VM's destination pointer, used by the COPY opcode to write bytes.

848b ⟨*ealdr defines* 832⟩+≡ (922) ◁845a 854a▷

```
  EALDR_VM_DISPATCH   EQU     $AA45   ; dispatch loop entry (linear continuation)
  EALDR_DEST          EQU     $4E     ; VM dest pointer (2 bytes)
```

Defines:
EALDR_DEST, used in chunks 839 and 849.
EALDR_VM_DISPATCH, used in chunks 849 and 855.

849 ⟨*EALDR opcode handlers* 849⟩≡ (922)

```
    ORG     $AAAE
    SUBROUTINE EALDR_OP_CALL1
; --- Opcode 01: CALL1 addr ---
; Call 6502 subroutine; result -> VM acc.  Handler at $AAAE.
EALDR_OP_CALL1:
    JSR     EALDR_READ_ARG16  ; decode addr -> $42/$43
    TYA
    PHA                       ; save Y (subroutine may clobber it)
    LDA     EALDR_PSUEDOACC
    JSR     EALDR_JMP_IND     ; JMP ($0042) -- enters subroutine
    STA     EALDR_PSUEDOACC   ; return value -> VM acc
    PLA
    TAY                       ; restore Y
    JMP     EALDR_VM_DISPATCH ; linear

; --- Opcode 08: CALL0 addr ---
; Terminal jump to 6502 code (no return).  Handler at $AABF.
; Falls through into EALDR_JMP_IND.
EALDR_OP_CALL0:
    JSR     EALDR_READ_ARG16
; Helper: indirect jump through $42/$43.  Used by CALL1 and CALL0.
EALDR_JMP_IND:                 ; $AAC2
    JMP     ($0042)

; --- Opcode 02: BEQ addr ---
; Branch if VM acc = 0.  Handler at $AAC5.
EALDR_OP_BEQ:
    JSR     EALDR_READ_ARG16
    LDA     EALDR_PSUEDOACC
    BEQ     EALDR_VM_GOTO     ; zero -> goto path (IP = $42/$43)
    JMP     EALDR_VM_DISPATCH ; nonzero -> linear

    SUBROUTINE EALDR_OP_LDI
; --- Opcode 03: LDI val ---
; Load 1-byte immediate (XOR $4C) into VM acc.  Handler at $AACF.
EALDR_OP_LDI:
    LDA     (EALDR_PTR),Y     ; read encoded byte
    INY
    BNE     .nc
    INC     EALDR_PTR+1
.nc:
    EOR     #$4C              ; decrypt
    STA     EALDR_PSUEDOACC
    JMP     EALDR_VM_DISPATCH

; --- Opcode 04: LD addr ---
; Load byte at addr into VM acc.  Handler at $AADD.
EALDR_OP_LD:
    JSR     EALDR_READ_ARG16
```

```
EALDR_OP_LD_DO:                 ; $AAE0 (shared by LDX)
    LDX      #$00
    LDA      ($42,X)           ; load byte at decoded address
    STA      EALDR_PSUEDOACC
    JMP      EALDR_VM_DISPATCH

    SUBROUTINE EALDR_OP_LDX
; --- Opcode 0A: LDX addr ---
; Indexed load: A <- (addr + A).  Handler at $AAE9.
EALDR_OP_LDX:
    JSR      EALDR_READ_ARG16
    LDA      EALDR_PSUEDOACC   ; add VM acc to address
    CLC
    ADC      $42
    STA      $42
    BCC      .nc
    INC      $43
.nc:
    JMP      EALDR_OP_LD_DO    ; fall into LD handler

    SUBROUTINE EALDR_OP_CALL
; --- Opcode 05: CALL addr ---
; VM subroutine call: push return IP, goto addr.  Handler at $AAFA.
EALDR_OP_CALL:
    JSR      EALDR_READ_ARG16  ; target -> $42/$43
    TYA                        ; compute return IP = $46 + Y
    CLC
    ADC      EALDR_PTR
    STA      EALDR_PTR
    BCC      .nc
    INC      EALDR_PTR+1
.nc:
    PSHW     EALDR_PTR         ; push return addr (lo first, hi second)
    LDY      #$00
    JMP      EALDR_VM_GOTO     ; goto path (IP = target)

; --- Opcode 09: RET ---
; Return from VM subroutine.  Handler at $AB12.
EALDR_OP_RET:
    PULW     EALDR_PTR         ; pop return addr (hi first, lo second)
    LDY      #$00
    JMP      EALDR_VM_DISPATCH

; --- Opcode 06: ST addr ---
; Store VM acc at addr.  Handler at $AB1D.
EALDR_OP_ST:
    JSR      EALDR_READ_ARG16
    LDA      EALDR_PSUEDOACC
    LDX      #$00
    STA      ($42,X)
```

```
    JMP      EALDR_VM_DISPATCH

; --- Opcode 0B: ASL ---
; Shift VM acc left.  Handler at $AB29.
EALDR_OP_ASL:
    ASL      EALDR_PSUEDOACC
    JMP      EALDR_VM_DISPATCH

    SUBROUTINE EALDR_OP_SUBI
; --- Opcode 07: SUBI val ---
; Subtract 1-byte immediate (XOR $4C) from VM acc.  Handler at $AB2E.
EALDR_OP_SUBI:
    LDA      (EALDR_PTR),Y     ; read encoded byte
    INY
    BNE      .nc
    INC      EALDR_PTR+1
.nc:
    EOR      #$4C              ; decrypt
    STA      $42               ; temp
    LDA      EALDR_PSUEDOACC
    SEC
    SBC      $42               ; acc - immediate
    STA      EALDR_PSUEDOACC
    JMP      EALDR_VM_DISPATCH

; --- Trampolines for opcodes 0C-11 ---
; These JMPs bridge the 1-byte dispatch offset limit.
EALDR_TRAMP_INC:                ; $AB43
    JMP      EALDR_OP_INC
EALDR_TRAMP_ADD:
    JMP      EALDR_OP_ADD
EALDR_TRAMP_DXR:
    JMP      EALDR_DXR_STEP
EALDR_TRAMP_BNE:
    JMP      EALDR_OP_BNE
EALDR_TRAMP_SUB:
    JMP      EALDR_OP_SUB
EALDR_TRAMP_COPY:
    JMP      EALDR_OP_COPY

; --- Opcode 0E: DXR ---
⟨EALDR DXR step 855⟩

; --- Opcode 0F: BNE addr ---
; Branch if VM acc != 0.  Handler at $AB6E.
EALDR_OP_BNE:
    JSR      EALDR_READ_ARG16
    LDA      EALDR_PSUEDOACC
    BEQ      .skip
    JMP      EALDR_VM_GOTO     ; nonzero -> goto path
```

```
.skip:
    JMP      EALDR_VM_DISPATCH ; zero -> linear

; --- Opcode 0C: INC addr ---
; Increment byte at addr; result also -> VM acc.  Handler at $AB7B.
EALDR_OP_INC:
    JSR      EALDR_READ_ARG16
    LDX      #$00
    LDA      ($42,X)
    CLC
    ADC      #$01
    STA      ($42,X)           ; store incremented value
    STA      EALDR_PSUEDOACC   ; also update VM acc
    JMP      EALDR_VM_DISPATCH

; --- Opcode 0D: ADD addr ---
; A <- A + (addr).  Handler at $AB8C.
EALDR_OP_ADD:
    JSR      EALDR_READ_ARG16
    LDX      #$00
    LDA      ($42,X)
    CLC
    ADC      EALDR_PSUEDOACC
    STA      EALDR_PSUEDOACC
    JMP      EALDR_VM_DISPATCH

; --- Opcode 10: SUB addr ---
; A <- A - (addr).  Handler at $AB9B.
EALDR_OP_SUB:
    JSR      EALDR_READ_ARG16
    LDX      #$00
    LDA      EALDR_PSUEDOACC
    SEC
    SBC      ($42,X)
    STA      EALDR_PSUEDOACC
    JMP      EALDR_VM_DISPATCH

; --- Opcode 11: COPY ---
; Copy one byte: (dest++) <- (src++).  Handler at $ABAA.
EALDR_OP_COPY:
    LDX      #$00
    LDA      (EALDR_SRC,X)     ; read from src
    STA      (EALDR_DEST,X)    ; write to dest
    INCW    EALDR_SRC ; src++
    INCW    EALDR_DEST ; dest++
    JMP      EALDR_VM_DISPATCH
```

Defines:
EALDR_OP_BEQ, never used.
EALDR_OP_CALL0, never used.
EALDR_OP_CALL1, never used.

EALDR_OP_LDI, never used.
Uses EALDR_DEST 848b, EALDR_PSUEDOACC 845a, EALDR_PTR 845a, EALDR_READ_ARG16 847a, EALDR_VM_DISPATCH 848b, EALDR_VM_GOTO 848a, INCW 10b, PSHW 9b, and PULW 10a.

25.3.6 Machine language subroutines

These are 6502 routines called by CALL1 or CALL0 from the VM:

853a ⟨*EALDR clear text screen* 853a⟩≡ (922)

```
      ORG     $A898
  EALDR_CLEAR_TEXT:
      STOW    $0400,EALDR_SRC ; $4C = 0
      LDY      #$00
  .loop:
      LDA      #$00
      STA      (EALDR_SRC),Y ; zero out
      INY
      BNE      .loop
      INC      EALDR_SRC+1
      LDA      EALDR_SRC+1
      BANE    #$08,.loop     ; done at $0800
      RTS
```

Defines:
EALDR_CLEAR_TEXT, used in chunk 837.
Uses STOW 8a.

853b ⟨*EALDR reboot handler* 853b⟩≡ (922)

```
      ORG     $A948
  EALDR_REBOOT:
      JSR      $FC58 ; HOME (clear screen)
      LDA      #$FF
  .delay:
      LDX      #$A0
  .inner:
      DEX
      BNE      .inner
      BIT      $C030 ; click speaker
      SEC
      SBC      #$01
      BNE      .delay
      STOW    $C1C5,$0400 ; 'E'
      JMP      DISK_BOOT_ROM ; reboot from Disk II
```

Defines:
EALDR_REBOOT, used in chunk 837.
Uses DISK_BOOT_ROM 674b and STOW 8a.

The "funny increment" routine manages the DXR key schedule. EALDR_BOT ($AC60) holds the low counter and EALDR_TOP ($AC61) holds the high counter. Bot increments each call; when it reaches top, it resets to 1 and top increments:

854a ⟨*ealdr defines* 832⟩+≡ (922) ◁848b

```
EALDR_BOT          EQU     $AC60   ; funny-increment low counter
EALDR_TOP          EQU     $AC61   ; funny-increment high counter
```

Defines:
EALDR_BOT, used in chunks 839 and 854b.
EALDR_TOP, used in chunks 839 and 854b.

854b ⟨*EALDR funny increment* 854b⟩≡ (922)

```
    ORG     $A970
EALDR_FUNNY_INC:
    INC      EALDR_BOT       ; $AC60
    LDA      EALDR_BOT
    BAEQ    EALDR_TOP,.wrap ; $AC61
    RTS
.wrap:
    STOB    #$01,EALDR_BOT
    INC      EALDR_TOP
    RTS
```

Defines:
EALDR_FUNNY_INC, used in chunks 839 and 890c.
Uses EALDR_BOT 854a, EALDR_TOP 854a, and STOB 8c.

25.3.7 DXR decryption step

The DXR opcode (0E) XORs a single byte at the source pointer with the accumulator, stores the result back, advances the pointer by 2 (processing every other byte), and derives the new accumulator from the pointer's high byte:

855 ⟨*EALDR DXR step* 855⟩≡ (849)

```
      ORG     $AB55
      SUBROUTINE EALDR_DXR_STEP
  EALDR_DXR_STEP:
      LDX      #$00
      LDA      (EALDR_SRC,X)     ; load byte at (src)
      EOR      EALDR_PSUEDOACC   ; XOR with accumulator
      STA      (EALDR_SRC,X)     ; store back
      INC      EALDR_SRC         ; src += 2 (skip every other byte)
      INC      EALDR_SRC
      BNE      .no_carry
      INC      EALDR_SRC+1
  .no_carry:
      LDA      EALDR_SRC+1       ; new acc = high byte of src XOR $68
      EOR      #$68
      STA      EALDR_PSUEDOACC
      JMP      EALDR_VM_DISPATCH ; linear continuation
```

Uses EALDR_PSUEDOACC 845a and EALDR_VM_DISPATCH 848b.

The region $ABBF–$BBFF contains three distinct areas:

1. $ABBF–$AC67: Disk I/O code used by the EALDR to read sectors during boot.
2. $AC68–$B7FF: Splash screen bitmap data (the EA logo displayed during loading), interspersed with residual EA loader documentation and checksum/protection tables.
3. $B800–$BBFF: Residual EA development source code—assembly comments documenting the VM opcode set, left over from the EALDR build tools.

25.3.8 Copy protection (unused)

EALDR_DISK_IO is apparently part of the copy-protection system, although it is bypassed by the crack. It performs half-track reads across multiple tracks, scanning for address fields and recording the range of sector numbers found. If the range (max−min) is at least $22 (34), the disk passes the check; otherwise it aborts to the caller's caller. This exploits the fact that a legitimately-pressed disk has slight track alignment variations that produce a wider range of sector numbers when reading between tracks, whereas a copied disk has uniform alignment.

The routine uses zero-page variables:

- $00: sector search timeout counter.
- $01: flag (set by caller; zero = abort).
- $02: temporary for address field decode.
- $03: maximum sector number seen.
- $04: minimum sector number seen.
- $06: current half-track number.

856 ⟨*EALDR disk io* 856⟩≡ (922)

```
    ORG     $ABBF
EALDR_DISK_IO:
    SUBROUTINE EALDR_DISK_IO

    LDA     $C0E9               ; IWM: motor on
    LDX     #$FF                ; \
    STX     $04                 ;  | min = $FF
    INX                         ;  | X = 0
    STX     $01                 ;  | flag = 0
```

```
    STX     $03                 ; /  max = 0
    STOB    #$18,$06 ; start at half-track $18
    JSR     EALDR_MOVE_ARM_TRAMPOLINE     ; seek to half-track $18
.next_track:
    INC     $06                 ; advance to next half-track
.search_sector:
    JSR     EALDR_SEARCH_ADDR_TRAMPOLINE  ; search for address field
    BCS     .search_sector      ; error -> retry
    LDA     $2D                 ; sector number from address field
    BNE     .search_sector      ; not sector 0 -> keep searching
    ; Found sector 0: read a data field as verification
    LDA     $06                 ; \
    JSR     EALDR_MOVE_ARM_TRAMPOLINE     ;  | seek to current half-track
    INC     $06                 ;  | advance half-track
    LDA     #$19                ;  |
    JSR     EALDR_READ_DATA_TRAMPOLINE    ; /  read data field (page $19)
    ; Scan for address field headers on this half-track
    STOB    #$00,$00 ; timeout counter = 0
.scan_loop:
    INC     $00                 ; increment timeout
    BEQ     .scan_done          ; wrapped to 0 -> done scanning
    ; Search for D5 AA 96 prologue
.wait_d5:
    LDA     $C0EC               ; \
    BPL     .wait_d5            ;  | wait for nibble
.check_d5:
    BANE    #$D5,.scan_loop ; not D5 -> restart
    PHA                         ; \
    PLA                         ; /  timing delay
.wait_aa:
    LDA     $C0EC               ; \
    BPL     .wait_aa            ;  |
    BANE    #$AA,.check_d5 ; not AA -> re-check D5
    NOP                         ; timing
    LDY     #$02                ; read 3 header fields (track, sector, ?)
.wait_96:
    LDA     $C0EC               ; \
    BPL     .wait_96            ;  |
    BANE    #$96,.check_d5 ; not 96 -> re-check D5
    ; Decode header fields (odd/even encoding)
.read_field:
    LDA     $C0EC               ; \
    BPL     .read_field         ;  | read odd-bits byte
    ROL                         ;  |
    STA     $02                 ; /  save shifted
.wait_even:
    LDA     $C0EC               ; \
    BPL     .wait_even          ;  | read even-bits byte
    AND     $02                 ; /  combine
    DEY                         ; \
```

```
    BPL     .read_field        ; /  loop for 3 fields
    ; A now holds the last decoded field (sector number from this half-track)
    BAEQ    #$02,.update_range ;  sector 2 -> update range
.scan_done:
    RTS                        ; /  otherwise return (also timeout exit)
.update_range:
    LDA     $00                ; \  update max
    BALT    $03,.check_min
    STA     $03                ; /  max = counter if counter > max
.check_min:
    BAGE    $04,.check_done ; update min
    STA     $04                ; /  min = counter if counter < min
.check_done:
    LDA     $06                ; \
    BAEQ    #$1F,.verify_range ;  scanned all half-tracks up to $1F?
    JMP     .search_sector     ; /  no -> continue scanning
.verify_range:
    LDA     $03                ; \
    SEC                        ;  | range = max - min
    SBC     $04                ;  |
    BALT    #$22,EALDR_CHECK_FLAG ;  range >= $22 (34)?
    RTS                        ; /  yes -> pass (return normally)
EALDR_ABORT:                   ;     range too small -> fail
    PLA                        ; \
    PLA                        ;  | discard return address
    RTS                        ; /  abort to caller's caller
```

Defines:
EALDR_DISK_IO, used in chunk 859.
Uses EALDR_ABORT 859, EALDR_CHECK_FLAG 858, EALDR_MOVE_ARM_TRAMPOLINE 872, EALDR_READ_DATA_TRAMPOLINE 872, EALDR_SEARCH_ADDR_TRAMPOLINE 872, and STOB 8c.

EALDR_CHECK_FLAG checks the flag at $01. If zero, it aborts via PLA; PLA; RTS. Called by the VM bytecode program's read_tracks subroutine.

858 ⟨*EALDR check flag* 858⟩≡ (922)

```
    ORG     $AC4C
EALDR_CHECK_FLAG:
    LDA     $01                ; \
    BNE     .ok                ;  | if flag is zero,
    JMP     EALDR_ABORT        ;  | abort (PLA; PLA; RTS at $AC49)
.ok:
    RTS
```

Defines:
EALDR_CHECK_FLAG, used in chunk 856.
Uses EALDR_ABORT 859.

EALDR_PROT_ENTRY calls EALDR_DISK_IO then enters the VM interpreter. The VM bytecodes following the DEX/BPL loop are a VMGOTO to VM_ERROR_DISPLAY, executed if the protection check fails and the VM takes over.

859 ⟨*EALDR prot entry* 859⟩≡ (922)

```
      ORG     $AC54
  EALDR_PROT_ENTRY:
      JSR     EALDR_DISK_IO      ; run half-track protection check
      JSR     EALDR_VM_INTERP    ; enter VM interpreter
      DEX                        ; \  these bytes are skipped;
      BPL     EALDR_PROT_ENTRY+3 ; /  the VM never returns here
      ; VM bytecodes: goto error display if reached
      VMGOTO  VM_ERROR_DISPLAY
      DS      8,$00              ; padding
```

Defines:
EALDR_ABORT, used in chunks 856 and 858.
EALDR_PROT_ENTRY, never used.
Uses EALDR_DISK_IO 856, EALDR_VM_INTERP 846, and VM_ERROR_DISPLAY 842.

25.4 EA splash screen

The splash screen data occupies $AC68–$B7FF in three parts:

1. $AC68–$ACFF (152 bytes): Residual EA loader documentation, the same text fragment that appears in BOOT1_LOADER_DOC.
2. $AD00–$AFFF (768 bytes): Unused residual data from the EA build process. Page $AD is almost entirely $FF (with two $0D bytes at $ADFE--$ADFF that may be trailing CRs from the loader documentation). Page $AE is mostly zero with a few scattered non-zero bytes. Page $AF is mostly $FF with five non-$FF bytes at the boundaries. None of these pages are referenced by any code in the EALDR.
3. $B000–$B7FF (2048 bytes): The hi-res bitmap of the EA splash screen logo. The EALDR_ENTRY code copies this data to $202C+ via EALDR_COPY_BYTE, painting the logo over the $FF-filled hi-res screen.

860 ⟨*EALDR residual docs* 860⟩≡ (922)

```
    ORG     $AC68
EALDR_RESIDUAL_DOCS:
    ; Residual EA loader documentation: $AC68-$ACFF (152 bytes)
    ; Same text as BOOT1_LOADER_DOC
    DC      "ll of the above."
    HEX     0D 10 20 0D 10 20
    DC      "The authors all receive a program called LOADER.O, which writes the loader"
    HEX     0D 10 20
    DC      "to disk.  To create a new LOADER.O after modifyin"
    HEX     0C 0C
    DC      "an"
```

Defines:
EALDR_RESIDUAL_DOCS, never used.
Uses BOOT1_LOADER_DOC 818.

Pages $AD–$AF are unused residual data, not referenced by any EALDR code.

861 ⟨*EALDR padding* 861⟩≡ (922)

```
      ORG     $AD00
  EALDR_PADDING:
      ; Unused residual data: $AD00-$AFFF (768 bytes)
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF 0D 0D FF
      HEX     D3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
      HEX     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
      HEX     00 00 00 00 00 00 00 00 FF 00 00 00 00 00 00 00
      HEX     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
      HEX     BF 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
      HEX     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
      HEX     00 00 00 00 00 00 00 00 FF 00 00 00 00 00 00 00
      HEX     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
      HEX     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
      HEX     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
      HEX     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
      HEX     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
      HEX     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
      HEX     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
      HEX     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
      HEX     00 00 00 00 00 00 00 00 00 00 00 00 00 00 0E 0E
      HEX     0F FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
      HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
```

```
    HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
    HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
    HEX     FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
    HEX     FF FF FF FF FF FF FF FF FF FF FF 35 35 5E 56 45
```

Defines:
EALDR_PADDING, never used.

The splash screen bitmap at $B000–$B7FF is included from ea_splash_screen.asm.

862 ⟨*EALDR splash bitmap* 862⟩≡ (922)

```
    ORG     $B000
EALDR_SPLASH_BITMAP:
    INCLUDE "ea_splash_screen.asm"
```

Defines:
EALDR_SPLASH_BITMAP, never used.

The region $B800–$BBFF contains residual assembler source code from the EA development environment, preserved because it fell within the EALDR's 2-track footprint. This is a remarkable piece of digital archaeology: the original EA source for the VM opcode definitions.

The data is structured as length-prefixed lines (each preceded by $0D and a length byte). Each opcode entry consists of a description line (a semicolon comment) followed by a definition line containing the 8-character mnemonic, a $D9 field separator, and the opcode number in decimal. The fragment begins mid-entry at opcode 5 (opcodes 0–4 were in memory before $B800 and have been overwritten by the splash screen data).

The $D9 field separator is notable: it is the same byte used as the XOR key for encrypting the high byte of 16-bit address arguments in the VM bytecode (see §25). This may not be a coincidence—the encryption key may have been chosen from the source code format.

The EA mnemonics reveal the original naming conventions:

Op	EA mnemonic	Our name	EA description
0	JMP	VMGOTO	?
1	JSRA	VMCALL1	?
2	BEQ	VMBEQ	?
3	LDI	VMLDI	?
4	LDA	VMLD	?
5	JSR	VMCALL	?
6	STA	VMST	Store acc. absolute
7	SUB	VMSUBI	Subtract acc. immediate
8	JMPA	VMCALL0	Jump to assembly
9	RTS	VMRET	M-code return from subroutine
10	LDX	VMLDX	Load acc. a99. indexed by Acc.
11	ASL	VMASL	Shift acc. left
12	INC	VMINC	Increment absolute
13	ADD	VMADD	Add immediate
14	EOR	VMDXR	EOR encryption bytes
15	BNE	VMBNE	Jump if acc. non-zero
16	SBA	VMSUB	Subtract absolute
17	COP	VMCOPY	Copy byte from (PTR) to (PTR2)

Following the opcode definitions is a fragment of documentation, `EALDR_SOURCE_COMMENTS`, that maps opcodes (starting from `5`) to their handler labels (e.g., `IJMP`, `IJSRA`, `IBEQ`). The `$E5` bytes (high-bit `'e'`) are likely an assembler pseudo-op (equate or entry directive). This could be a leftover table for a VM code disassembler.

`EACR` emits a `$0D` line separator followed by a length byte. `EASEP` emits the `$D9` record separator.

864 ⟨*EALDR VM macros* 835⟩+≡ (909 922) ◁835

```
MACRO EACR    ; line separator: $0D
    DC.B $0D
ENDM
MACRO EASEP   ; record separator: $D9 (high-bit 'Y')
    DC.B $D9
ENDM
MACRO SLEN    ; string length byte
    DC.B {1}
ENDM
```

865 ⟨*EALDR source comments* 865⟩≡ (922)

```
      ORG     $B800
  EALDR_SOURCE_COMMENTS:
      ; --- EA VM opcode documentation (opcodes 5-17) ---
      DC      "R     "
      EASEP
      DC      " 5"
      EACR
      SLEN    $16
      DC      "; Store acc. absolute"
      EACR
      SLEN    $0C
      DC      "STA   88"
      EASEP
      DC      " 6"
      EACR
      SLEN    $1A
      DC      "; Subtract acc. immediate"
      EACR
      SLEN    $0C
      DC      "SUB     "
      EASEP
      DC      " 7"
      EACR
      SLEN    $22
      DC      "; Jump to assembly (used for exit"
      EACR
      SLEN    $1F
      DC      "; from interpreter, 6502 stack"
      EACR
      SLEN    $1E
      DC      "; should be balanced properly"
      EACR
      SLEN    $20
      DC      "; unless M-code did JSR without"
      EACR
      SLEN    $06
      DC      "; RTS"
      EACR
      SLEN    $0C
      DC      "JMPA    "
      EASEP
      DC      " 8"
      EACR
      SLEN    $20
      DC      "; M-code return from subroutine"
      EACR
      SLEN    $0C
      DC      "RTS     "
      EASEP
```

```
DC      " 9"
EACR
SLEN    $21
DC      "; Load acc. a99. indexed by Acc."
EACR
SLEN    $0D
DC      "LDX     "
EASEP
DC      " 10"
EACR
SLEN    $12
DC      "; Shift acc. left"
EACR
SLEN    $0D
DC      "ASL     "
EASEP
DC      " 11"
EACR
SLEN    $15
DC      "; Increment absolute"
EACR
SLEN    $0D
DC      "INC     "
EASEP
DC      " 12"
EACR
SLEN    $10
DC      "; Add immediate"
EACR
SLEN    $0D
DC      "ADD     "
EASEP
DC      " 13"
EACR
SLEN    $20
DC      "; EOR encryption bytes: see the"
EACR
SLEN    $22
DC      "; code below for IEOR1 to get the"
EACR
SLEN    $1E
DC      "; algorithm.  Acc. holds 0 if"
EACR
SLEN    $1F
DC      "; hi order pointer gets::o $68"
EACR
SLEN    $1E
DC      "; (end of encrypted RAM area)"
EACR
SLEN    $0D
```

```
DC      "EOR     "
EASEP
DC      " 14"
EACR
SLEN    $18
DC      "; Jump if acc. non-zero"
EACR
SLEN    $0D
DC      "BNE     "
EASEP
DC      " 15"
EACR
SLEN    $14
DC      "; Subtract absolute"
EACR
SLEN    $0D
DC      "SBA     "
EASEP
DC      " 16"
EACR
SLEN    $22
DC      "; Copy a byte from (PTR) - (PTR2)"
EACR
SLEN    $1F
DC      "; then increment both pointers"
EACR
SLEN    $0D
DC      "COP     "
EASEP
DC      " 17"
EACR
SLEN    $14
DC      "ICODE   "
HEX     E5                  ; high-bit 'e'
DC      " IJMP-IJMP"
EACR
SLEN    $0D
HEX     E5                  ; high-bit 'e'
DC      " IJSRA-IJMP"
EACR
SLEN    $0C
HEX     E5                  ; high-bit 'e'
DC      " IBEQ-IJMP"
EACR
SLEN    $0C
DC      ";;ILDI-IJMP"
EACR
SLEN    $0C
HEX     E5                  ; high-bit 'e'
DC      " ILDA-IJMP"
```

```
EACR
SLEN    $0C
HEX     E5                  ; high-bit 'e'
DC      " IJSR-IJMP"
EACR
SLEN    $0C
HEX     E5                  ; high-bit 'e'
DC      " ISTA-IJMP"
EACR
SLEN    $0C
HEX     E5                  ; high-bit 'e'
DC      " ISUB-IJMP"
EACR
SLEN    $0D
HEX     E5                  ; high-bit 'e'
DC      " IJMPA-IJMP"
EACR
SLEN    $0C
HEX     E5                  ; high-bit 'e'
DC      " IRTS-IJMP"
EACR
SLEN    $0C
HEX     E5                  ; high-bit 'e'
DC      " ILDX-IJMP"
EACR
SLEN    $0C
HEX     E5                  ; high-bit 'e'
DC      " IASL-IJMP"
EACR
SLEN    $0C
HEX     E5                  ; high-bit 'e'
DC      " IINC-IJMP"
EACR
SLEN    $0C
HEX     E5                  ; high-bit 'e'
DC      " IADD-IJMP"
EACR
SLEN    $0C
HEX     E5                  ; high-bit 'e'
DC      " IEOR-IJMP"
EACR
SLEN    $03
HEX     E2                  ; high-bit 'b'
DC      " "
EACR
SLEN    $0C
HEX     E5                  ; high-bit 'e'
DC      " IBNE-IJMP"
EACR
SLEN    $0C
```

```
HEX     E5                  ; high-bit 'e'
DC      " ISBA-IJMP"
EACR
SLEN    $0C
HEX     E5                  ; high-bit 'e'
DC      " ICOP-IJMP"
EACR
SLEN    $03
HEX     E3                  ; high-bit 'c'
DC      " "
EACR
SLEN    $11
DC      "IJMP    "
HEX     CB                  ; high-bit 'K'
```

Defines:
EALDR_SOURCE_COMMENTS, never used.

25.4.1 Checksum source listing

The region $A600–$A805 contains embedded EA VM source code that documents the checksum verification bytecode program at $A072–$A0FF. It uses the same formatting conventions as `EALDR_SOURCE_COMMENTS` at $B800: `$0D` = line separator, `$E5` = opcode marker, `$D8` = operand marker, and printable ASCII for mnemonics, labels, and comments. The listing describes the checksum loop that sums memory $A000–$A2E0 using the VM's LDI, LDA, LDX, SBA, STA, INC, BNE, and SUB opcodes.

The last 6 bytes ($A800–$A805) are not part of the source listing; they are residual data between the listing and the EALDR entry point at $A806.

870 ⟨*EALDR checksum source listing* 870⟩≡ (922)

```
    ORG     $A600
EALDR_CHECKSUM_LISTING:
    ; VM source listing for the checksum verification program.
    ; Format: $0D = line break, $E5 = opcode marker, $D8 = operand,
    ;         $0C/$06/$0E/$10/$11/$0A/$1F/$21 = indent/length codes.
    ;
    ; Line 1: LDI,$4C^0          (load immediate $4C XOR 0)
    HEX     0D 0C E5 20 4C 44 49 2C 24 34 43 5E 30
    ; Line 2: STA &&55555^PTR    (store to PTR)
    HEX     0D 06 E5 20 53 54 41 0D 0C 26 26 35 35 35 35 35 5E 50 54 52
    ; Line 3: STA 55555^SUM      (store to SUM)
    HEX     0D 06 E5 20 53 54 41 0D 0C D8 20 35 35 35 35 35 5E 53 55 4D
    ; Line 4: CKSLOOP LDA 55555^PTR
    HEX     0D 0E 43 4B 53 4C 4F 4F 50 20 E5 20 4C 44 41 0D 0C D8 20 35 35 35 35 35 5E 50 54 52
    ; Comment: "; LDX: LOAD pseudo accumulator"
    HEX     0D 1F 3B 20 4C 44 58 3A 20 4C 4F 41 44 20 70 73 65 75 64 6F 20 61 63 63 75 6D 75 6C 61 7
    ; Comment: "!; indexed by current pseudo acc."
    HEX     0D 21 3B 20 69 6E 64 65 78 65 64 20 62 79 20 63 75 72 72 65 6E 74 20 70 73 65 75 64 6F 2
    ; Line 5: LDX 55555^CHECKER
    HEX     0D 06 E5 20 4C 44 58 0D 10 D8 20 35 35 35 35 35 5E 43 48 45 43 4B 45 52
    ; Line 6: SBA 55555^SUM
    HEX     0D 06 E5 20 53 42 41 0D 0C D8 20 35 35 35 35 35 5E 53 55 4D
    ; Line 7: STA 55555^SUM
    HEX     0D 06 E5 20 53 54 41 0D 0C D8 20 35 35 35 35 35 5E 53 55 4D
    ; Line 8: LDA 55555^PTR
    HEX     0D 06 E5 20 4C 44 41 0D 0C D8 20 35 35 35 35 35 5E 50 54 52
    ; Line 9: LDX 55555^CHECKER1
    HEX     0D 06 E5 20 4C 44 58 0D 11 D8 20 35 35 35 35 35 5E 43 48 45 43 4B 45 52 31
    ; Line 10: SBA 55555^SUM
    HEX     0D 06 E5 20 53 42 41 0D 0C D8 20 35 35 35 35 35 5E 53 55 4D
    ; Line 11: ''TA 55555^SUM    (STA with corrupted mnemonic)
    HEX     0D 06 E5 27 27 54 41 0D 0C D8 20 35 35 35 35 35 5E 53 55 4D
    ; Line 12: INC 55555^PTR
    HEX     0D 06 E5 20 49 4E 43 0D 0C D8 20 35 35 35 35 35 5E 50 54 52
    ; Line 13: BNE 55555^CKSLOOP
```

```
HEX     0D 06 E5 20 42 4E 45 0D 10 D8 20 35 35 35 35 35 5E 43 4B 53 4C 4F 4F 50
; Line 14: CKSLOO2 LDA 55555^PTR
HEX     0D 0E 43 4B 53 4C 4F 4F 32 20 E5 20 4C 44 41 0D 0C D8 20 35 35 35 35 35 5E 50 54 52
; Line 15: LDX 55555^CHECKER2
HEX     0D 06 E5 20 4C 44 58 0D 11 D8 20 35 35 35 35 35 5E 43 48 45 43 4B 45 52 32
; Line 16: SBA 55555^SUM
HEX     0D 06 E5 20 53 42 41 0D 0C D8 20 35 35 35 35 35 5E 53 55 4D
; Line 17: STA 55555^SUM
HEX     0D 06 E5 20 53 54 41 0D 0C D8 20 35 35 35 35 35 5E 53 55 4D
; Line 18: INC 55555^PTR
HEX     0D 06 E5 20 49 4E 43 0D 0C D8 20 35 35 35 35 35 5E 50 54 52
; Line 19: SUB $4C^$E0       (subtract: length = $2E0)
HEX     0D 06 E5 20 53 55 42 0D 0A E5 20 24 34 43 5E 24 45 30
; Comment: ";LENGTH=$2E0"
HEX     0D 0D 3B 4C 45 4E 47 54 48 3D 24 32 45 30
; Line 20: BNE 55555^CKSLOO2
HEX     0D 06 E5 20 42 4E 45 0D 10 D8 20 35 35 35 35 35 5E 43 4B 53 4C 4F 4F 32
; Trailing data (last opcode marker + residual bytes before $A806 entry)
HEX     0D 06 E5 01 17 05 DA 04 00
```

Defines:
EALDR_CHECKSUM_LISTING, never used.

25.5 Disk I/O routines

The EALDR RWTS (Read/Write Track/Sector) occupies $BC00–$BFFF and provides the disk routines used by the EALDR VM to load game data. These routines are structurally identical to the BOOT1 disk routines at $0900–$0CFF (see §24), relocated to different addresses and using different zero-page locations:

Purpose	BOOT1	EALDR RWTS
Scratch variables	$26/$27	$4A/$4B
Address header fields	$2C–$2F	$38–$3B
Slot index	$2B (variable)	$60 (hardcoded)
Current track	$09C6	$BC0C
Error flag	$09C7	$BC0D

Four entry points at $BC00 are JMP trampolines used by the EALDR VM:

Entry	Target	Function
$BC00	$BCD4	Move disk arm to half-track in A
$BC03	$BF00	Read one track (16 sectors) to memory at page $3E
$BC06	$BD21	Search for next address field
$BC09	$BD14	Delay (enters EALDR_RWTS_DELAY inner loop)

The sector loader at $BF1F builds a destination page table from the interleave table at $BF64 and reads all 16 sectors using EALDR_RWTS_SEARCH_ADDR and EALDR_RWTS_READ_SECTOR.

872 ⟨*EALDR disk entry points* 872⟩≡ (922)

```
    ORG     $BC00
EALDR_MOVE_ARM_TRAMPOLINE:
    JMP     EALDR_RWTS_MOVE_ARM
EALDR_READ_TRACK_TRAMPOLINE:                ; $BC03
    JMP     EALDR_RWTS_READ_TRACK
EALDR_SEARCH_ADDR_TRAMPOLINE:               ; $BC06
    JMP     EALDR_RWTS_SEARCH_ADDR
EALDR_READ_DATA_TRAMPOLINE:                 ; $BC09
    JMP     EALDR_RWTS_DELAY_INNER
```

Defines:
EALDR_MOVE_ARM_TRAMPOLINE, used in chunks 839 and 856.
EALDR_READ_DATA_TRAMPOLINE, used in chunk 856.
EALDR_READ_TRACK_TRAMPOLINE, used in chunk 842.
EALDR_SEARCH_ADDR_TRAMPOLINE, used in chunk 856.

Uses EALDR_RWTS_DELAY_INNER 878b, EALDR_RWTS_MOVE_ARM 877, EALDR_RWTS_READ_TRACK 883, and EALDR_RWTS_SEARCH_ADDR 880.

EALDR_RWTS_TRACK at $BC0C holds the current track number (modified by EALDR_RWTS_MOVE_ARM). EALDR_RWTS_ERROR_FLAG at $BC0D is set by EALDR_RWTS_READ_TRACK when a timeout occurs. EALDR_RWTS_READ_ERROR at $BC0E is the error-return entry point: it sets carry and returns.

873a ⟨*EALDR RWTS track vars* 873a⟩≡ (922)

```
    ORG     $BC0C
EALDR_RWTS_TRACK:
    HEX     00              ; current track number
EALDR_RWTS_ERROR_FLAG:
    HEX     00              ; nonzero = error occurred
EALDR_RWTS_READ_ERROR:
    SEC
    RTS
```

Defines:
EALDR_RWTS_ERROR_FLAG, used in chunk 883.
EALDR_RWTS_READ_ERROR, used in chunk 874.
EALDR_RWTS_TRACK, used in chunks 877 and 878a.

873b ⟨*EALDR RWTS nibble table B* 873b⟩≡ (922)

```
    ORG     $BE00
EALDR_RWTS_NIBBLE_B:
    ; 256-byte secondary decode table
    HEX     00 00 00 96 02 00 00 97 01 00 00 9A 03 00 00 9B
    HEX     00 02 00 9D 02 02 00 9E 01 02 00 9F 03 02 00 A6
    HEX     00 01 00 A7 02 01 00 AB 01 01 00 AC 03 01 00 AD
    HEX     00 03 00 AE 02 03 00 AF 01 03 00 B2 03 03 00 B3
    HEX     00 00 02 B4 02 00 02 B5 01 00 02 B6 03 00 02 B7
    HEX     00 02 02 B9 02 02 02 BA 01 02 02 BB 03 02 02 BC
    HEX     00 01 02 BD 02 01 02 BE 01 01 02 BF 03 01 02 CB
    HEX     00 03 02 CD 02 03 02 CE 01 03 02 CF 03 03 02 D3
    HEX     00 00 01 D6 02 00 01 D7 01 00 01 D9 03 00 01 DA
    HEX     00 02 01 DB 02 02 01 DC 01 02 01 DD 03 02 01 DE
    HEX     00 01 01 DF 02 01 01 E5 01 01 01 E6 03 01 01 E7
    HEX     00 03 01 E9 02 03 01 EA 01 03 01 EB 03 03 01 EC
    HEX     00 00 03 ED 02 00 03 EE 01 00 03 EF 03 00 03 F2
    HEX     00 02 03 F3 02 02 03 F4 01 02 03 F5 03 02 03 F6
    HEX     00 01 03 F7 02 01 03 F9 01 01 03 FA 03 01 03 FB
    HEX     00 03 03 FC 02 03 03 FD 01 03 03 FE 03 03 03 FF
```

Defines:
EALDR_RWTS_NIBBLE_B, used in chunk 874.

EALDR_RWTS_READ_SECTOR reads and decodes one sector's data field from the disk. It searches for the D5 AA AD data prologue, then reads the 6-and-2 encoded nibbles, decodes them through the translation tables at $BD98 and $BE00, and stores 256 bytes at the address in $44/$45. Returns with carry clear on success, carry set on error.

The routine uses self-modifying code: the STA instructions at $BC6E, $BC95, and $BCAD have their high byte patched by EALDR_RWTS_READ_SECTOR itself (at $BC14/$BC17, $BC21/$BC24, and $BC2C/$BC2F) to set up three different destination regions within the 256-byte output page.

This is structurally identical to BOOT1_READ_SECTOR at $0902, with zero-page $4A used instead of $26 for the running checksum, $44/$45 for the destination pointer (same as BOOT1), and the slot index hardcoded to $60 instead of read from $2B.

874 ⟨*EALDR RWTS read sector* 874⟩≡ (922)

```
    ORG     $BC10
EALDR_RWTS_READ_SECTOR:
    SUBROUTINE EALDR_RWTS_READ_SECTOR

    ; Self-modify STA targets based on destination address ($44/$45)
    LDA     #$00
    LDY     $45             ; destination page
    STA     .sta_1+1        ; \  patch STA $xx00,Y low byte
    STY     .sta_1+2        ;  | patch STA $xx00,Y high byte
    SEC
    SBC     #$54            ; \
    BCS     .no_borrow1     ;  | subtract $54, borrow from page
    DEY                     ;  |
    SEC                     ; /
.no_borrow1:
    STA     .sta_2+1        ; \  patch second STA target
    STY     .sta_2+2        ; /
    SBC     #$57            ; \
    BCS     .no_borrow2     ;  | subtract $57, borrow from page
    DEY                     ; /
.no_borrow2:
    STA     .sta_3+1        ; \  patch third STA target
    STY     .sta_3+2        ; /

    ; Search for data prologue: D5 AA AD
    LDY     #$20            ; timeout counter
.search:
    DEY
    BEQ     EALDR_RWTS_READ_ERROR ; timeout
.wait1:
    LDA     $C0EC           ; read nibble
    BPL     .wait1
.check_d5:
```

```
    EOR     #$D5
    BNE     .search
    NOP
.wait2:
    LDA     $C0EC
    BPL     .wait2
    BANE    #$AA,.check_d5 ; not AA -> re-check for D5
    NOP
.wait3:
    LDA     $C0EC
    BPL     .wait3
    BANE    #$AD,.check_d5 ; not AD -> re-check for D5

    ; Read 86 bytes of secondary data ($AA-$FF range of page)
    LDY     #$AA
    LDA     #$00
.read_secondary:
    STA     $4A             ; running checksum
.read_secondary_wait:
    LDX     $C0EC
    BPL     .read_secondary_wait
    LDA     $BD00,X         ; decode via primary nibble table
    STA     $0200,Y         ; store in buffer
    EOR     $4A             ; update checksum
    INY
    BNE     .read_secondary ; loop $AA..$FF (86 bytes)

    ; Read and decode primary data (86 bytes -> page+$00..$55)
    LDY     #$AA
    BNE     .read_primary_start
.sta_3:
    STA     $FFFF,Y         ; self-modified: page+$AC..$FF
.read_primary_start:
    LDX     $C0EC
    BPL     .read_primary_start
    EOR     $BD00,X         ; decode nibble
    LDX     $0200,Y         ; secondary data
    EOR     EALDR_RWTS_NIBBLE_B,X ; pair-decode table
    INY
    BNE     .sta_3          ; loop $AA..$FF (86 bytes)
    PHA                     ; save last decoded value
    AND     #$FC

    ; Read and decode middle data (86 bytes -> page+$56..$AB)
    LDY     #$AA
.read_mid:
    LDX     $C0EC
    BPL     .read_mid
    EOR     $BD00,X
    LDX     $0200,Y
```

```
    EOR     EALDR_RWTS_NIBBLE_B+1,X ; offset +1 in pair-decode table
.sta_2:
    STA     $FFFF,Y         ; self-modified: page+$56..$AB
    INY
    BNE     .read_mid

    ; Read and decode final data (84 bytes -> page+$00..$53)
.wait_final1:
    LDX     $C0EC
    BPL     .wait_final1
    AND     #$FC
    LDY     #$AC
.read_final:
    EOR     $BD00,X
    LDX     $01FE,Y         ; secondary data (offset -2)
    EOR     EALDR_RWTS_NIBBLE_B+2,X ; offset +2 in pair-decode table
.sta_1:
    STA     $FFFF,Y         ; self-modified: page+$00..$55
.wait_final2:
    LDX     $C0EC
    BPL     .wait_final2
    INY
    BNE     .read_final

    ; Verify checksum byte
    AND     #$FC
    EOR     $BD00,X         ; decode final nibble
    LDX     #$60            ; slot index (hardcoded slot 6)
    TAY
    BNE     .checksum_bad   ; checksum must be zero

    ; Read epilogue byte (expect $DE)
.wait_epilogue:
    LDA     $C0EC
    BPL     .wait_epilogue
    BAEQ    #$DE,.ok
.checksum_bad:
    SEC                     ; error: carry set
    HEX     24              ; BIT $zp opcode -- hides next CLC byte
.ok:
    CLC                     ; success: carry clear
    PLA                     ; discard saved byte
    LDY     #$55
    STA     ($44),Y         ; store final byte
    RTS
```

Defines:
EALDR_RWTS_READ_SECTOR, used in chunk 883.
Uses EALDR_RWTS_NIBBLE_B 873b and EALDR_RWTS_READ_ERROR 873a.

EALDR_RWTS_MOVE_ARM moves the drive head to the half-track in A. The target track (doubled for half-track stepping) is compared against the current position in EALDR_RWTS_TRACK. It steps inward or outward one half-track at a time, activating the stepper phases via EALDR_RWTS_PHASE_ON and waiting with EALDR_RWTS_DELAY between steps. Zero-page $4A holds the target half-track, $4B saves the previous position.

This is structurally identical to BOOT1_SEEK_TRACK at $09C8, using $4A/$4B instead of $26/$27 and the slot index hardcoded to $60.

877 ⟨*EALDR RWTS move arm* 877⟩≡ (922)

```
    ORG     $BCD4
EALDR_RWTS_MOVE_ARM:
    SUBROUTINE EALDR_RWTS_MOVE_ARM

    ASL                     ; double for half-track addressing
    STA     $4A             ; target half-track
.compare:
    MOVB    EALDR_RWTS_TRACK,$4B ; save current position
    SEC
    SBC     $4A             ; current - target
    BEQ     EALDR_RWTS_PHASE_ON_RTS ; already there -> RTS at $BD10
    BCS     .step_out
    INC     EALDR_RWTS_TRACK ; step inward
    BCC     .do_step
.step_out:
    DEC     EALDR_RWTS_TRACK ; step outward
.do_step:
    JSR     EALDR_RWTS_PHASE_ON ; activate stepper phase
    JSR     EALDR_RWTS_DELAY ; wait for head to settle
    LDA     $4B             ; previous half-track
    AND     #$03
    ASL                     ; phase index x 2
    ORA     #$60            ; $C0x0 slot offset
    TAY
    LDA     $C080,Y         ; deactivate previous phase
    JSR     EALDR_RWTS_DELAY ; wait
    BEQ     .compare        ; always taken (delay returns Z=1)
    JSR     EALDR_RWTS_DELAY ; (unreachable)
```

Defines:
EALDR_RWTS_MOVE_ARM, used in chunks 872 and 883.
Uses EALDR_RWTS_DELAY 878b, EALDR_RWTS_PHASE_ON 878a, EALDR_RWTS_PHASE_ON_RTS 878a, EALDR_RWTS_TRACK 873a, and MOVB 8c.

EALDR_RWTS_PHASE_ON activates the stepper motor phase corresponding to the current half-track position. The slot index is hardcoded to $60 (slot 6).

878a ⟨*EALDR RWTS phase on* 878a⟩≡ (922)

```
    ORG     $BD04
EALDR_RWTS_PHASE_ON:
    SUBROUTINE EALDR_RWTS_PHASE_ON

    LDA     EALDR_RWTS_TRACK
    AND     #$03            ; phase = track mod 4
    ASL                     ; x 2 for IWM register spacing
    ORA     #$60            ; slot offset $60 (slot 6)
    TAY
    LDA     $C081,Y         ; activate phase (odd address = on)
EALDR_RWTS_PHASE_ON_RTS:
    RTS
```

Defines:
EALDR_RWTS_PHASE_ON, used in chunk 877.
EALDR_RWTS_PHASE_ON_RTS, used in chunk 877.
Uses EALDR_RWTS_TRACK 873a.

EALDR_RWTS_DELAY is a nested delay loop providing approximately 40×256×5 = 51,200 cycles, giving the drive head time to settle after a step. EALDR_RWTS_DELAY_INNER at $BD14 is the inner entry point used by EALDR_READ_DATA_TRAMPOLINE: entering here skips the initial LDA #\$28 and SEC, running the delay with whatever count and carry the caller has set in A.

878b ⟨*EALDR RWTS delay* 878b⟩≡ (922)

```
    ORG     $BD11
EALDR_RWTS_DELAY:
    SUBROUTINE EALDR_RWTS_DELAY

    LDA     #$28            ; outer counter = 40
    SEC
EALDR_RWTS_DELAY_INNER:                     ; $BD14
.outer:
    PHA
.inner:
    SBC     #$01
    BNE     .inner          ; inner loop: 256 iterations
    PLA
    SBC     #$01
    BNE     .outer          ; outer loop: 40 iterations
    RTS
```

Defines:
EALDR_RWTS_DELAY, used in chunk 877.
EALDR_RWTS_DELAY_INNER, used in chunk 872.

EALDR_RWTS_ADDR_ERROR is the address-field error return.

879 ⟨*EALDR RWTS addr error* 879⟩≡ (922)

```
      ORG     $BD1F
  EALDR_RWTS_ADDR_ERROR:
      SEC
      RTS
```

Defines:
EALDR_RWTS_ADDR_ERROR, used in chunk 880.

EALDR_RWTS_SEARCH_ADDR searches for an address field prologue (D5 AA 96) and reads the four two-bit-encoded header bytes: volume, track, sector, and checksum. The decoded sector number is stored in $39. Returns with carry clear on success.

This is structurally identical to BOOT1_FIND_SECTOR at $0A15, using $4A/$4B instead of $26/$27 for the timeout counter and scratch, and $38–$3B instead of $2C–$2F for the header fields.

880 ⟨*EALDR RWTS search addr* 880⟩≡ (922)

```
    ORG     $BD21
EALDR_RWTS_SEARCH_ADDR:
    SUBROUTINE EALDR_RWTS_SEARCH_ADDR

    ; Timeout counter
    LDY     #$FC
    STY     $4A             ; outer counter (high byte)
.retry:
    INY
    BNE     .wait_d5
    INC     $4A
    BEQ     EALDR_RWTS_ADDR_ERROR ; timeout

    ; Search for D5 AA 96 address prologue
.wait_d5:
    LDA     $C0EC
    BPL     .wait_d5
.check_d5:
    BANE    #$D5,.retry
    NOP
.wait_aa:
    LDA     $C0EC
    BPL     .wait_aa
    BANE    #$AA,.check_d5 ; not AA -> re-check D5
    LDY     #$03            ; 4 header fields to read
.wait_96:
    LDA     $C0EC
    BPL     .wait_96
    BANE    #$96,.check_d5 ; not 96 -> re-check D5

    ; Read 4 encoded header bytes (volume, track, sector, checksum)
    LDA     #$00
.read_header:
    STA     $4B             ; running checksum
.wait_odd:
    LDA     $C0EC           ; \
    BPL     .wait_odd       ;  | read odd-bits byte
    ROL                     ;  |
    STA     $4A             ; /  save shifted value
.wait_even:
```

```
    LDA     $C0EC           ; \
    BPL     .wait_even      ;  | read even-bits byte
    AND     $4A             ;  | combine with odd bits
    STA     $38,Y           ;  | store at $3B,$3A,$39,$38 (Y=3..0)
    EOR     $4B             ;  | update checksum
    DEY                     ;  |
    BPL     .read_header    ; /  loop for all 4 fields

    ; Verify checksum
    TAY
    BNE     EALDR_RWTS_ADDR_ERROR ; checksum must be zero

    ; Read address epilogue: DE AA
.wait_de:
    LDA     $C0EC
    BPL     .wait_de
    BANE    #$DE,EALDR_RWTS_ADDR_ERROR
    NOP
.wait_aa2:
    LDA     $C0EC
    BPL     .wait_aa2
    BANE    #$AA,EALDR_RWTS_ADDR_ERROR
    CLC                     ; success
    RTS
```

Defines:
EALDR_RWTS_SEARCH_ADDR, used in chunks 872 and 883.
Uses EALDR_RWTS_ADDR_ERROR 879.

The region at $BD7D–$BD97 is unused padding between EALDR_RWTS_SEARCH_ADDR and the nibble translation table. It contains a residual CLC; RTS pair at $BD7D (dead code) and a $04 byte at $BD97 that is the last entry of the secondary nibble decode region (matching the $04 at $0A97 in BOOT1).

881 ⟨*EALDR RWTS padding* 881⟩≡ (922)

```
    ORG     $BD7D
EALDR_RWTS_PADDING:
    HEX     18 60           ; residual CLC; RTS (dead code)
    DS      24,$00          ; zero padding
    HEX     04              ; secondary nibble table tail byte
```

Defines:
EALDR_RWTS_PADDING, never used.

The nibble translation tables at $BD98–$BDFF and $BE00–$BEFF are used by EALDR_RWTS_READ_SECTOR to decode the 6-and-2 encoded disk nibbles back into data bytes. These are byte-identical to the BOOT1 tables at $0A98–$0AFF and $0B00–$0BFF. The primary table at $BD98 maps raw disk nibbles (high-bit set values in the range $96–$FF) to 6-bit values. The secondary table at $BE00 provides the pair-decode lookup.

882a ⟨*EALDR RWTS nibble tables* 882a⟩≡ (922)

```
    ORG     $BD98
EALDR_RWTS_NIBBLE_A:
    ; $BD98-$BDFF: primary 6-and-2 decode table (104 bytes)
    HEX     98 99 08 0C 9C 10 14 18
    HEX     A0 A1 A2 A3 A4 A5 1C 20
    HEX     A8 A9 AA 24 28 2C 30 34
    HEX     B0 B1 38 3C 40 44 48 4C
    HEX     B8 50 54 58 5C 60 64 68
    HEX     C0 C1 C2 C3 C4 C5 C6 C7
    HEX     C8 C9 CA 6C CC 70 74 78
    HEX     D0 D1 D2 7C D4 D5 80 84
    HEX     D8 88 8C 90 94 98 9C A0
    HEX     E0 E1 E2 E3 E4 A4 A8 AC
    HEX     E8 B0 B4 B8 BC C0 C4 C8
    HEX     F0 F1 CC D0 D4 D8 DC E0
    HEX     F8 E4 E8 EC F0 F4 F8 FC
```

Defines:
EALDR_RWTS_NIBBLE_A, never used.

The interleave table maps physical sector numbers to page offsets within the target region. This is identical to BOOT1_INTERLEAVE at $0C48. The destination page table at $BF74 is computed at runtime by adding the base page to each interleave entry.

882b ⟨*EALDR RWTS interleave table* 882b⟩≡ (922)

```
    ORG     $BF64
EALDR_RWTS_INTERLEAVE:
    HEX     0F 08 01 09 02 0A 03 0B 04 0C 05 0D 06 0E 07 00
```

Defines:
EALDR_RWTS_INTERLEAVE, used in chunk 883.

EALDR_RWTS_READ_TRACK reads all 16 sectors of a track into memory. It takes a track number in A, seeks to it, and loops until every sector has been read or a timeout occurs. On entry, the track number is saved and EALDR_RWTS_ERROR_FLAG is cleared. If the first attempt encounters errors, the error flag is cleared and a second attempt is made.

The inner routine at $BF1F builds the destination page table by adding the base page ($3E) to each entry of the interleave table at $BF64, then iterates: EALDR_RWTS_SEARCH_ADDR finds each sector's address field, the sector number in $39 indexes the destination page table, and EALDR_RWTS_READ_SECTOR decodes the data. Table entries are zeroed after successful reads; the loop ends when all 16 are zero. A retry counter at $40 prevents infinite loops.

This is structurally identical to BOOT1_LOAD_TRACK at $0C00, using $39 instead of $2D for the sector number and $BF64 instead of $0C48 for the interleave table.

883 ⟨*EALDR RWTS read track* 883⟩≡ (922)

```
    ORG     $BF00
EALDR_RWTS_READ_TRACK:
    SUBROUTINE EALDR_RWTS_READ_TRACK

    ; First attempt: clear error flag, seek, load
    PHA
    STOB    #$00,EALDR_RWTS_ERROR_FLAG
    PLA
    JSR     .seek_and_load  ; seek + load all sectors
    LDA     EALDR_RWTS_ERROR_FLAG
    BNE     .retry
    RTS                     ; success

.retry:
    ; Second attempt on error
    STOB    #$00,EALDR_RWTS_ERROR_FLAG
    JSR     .load_sectors   ; try again (already on track)
    LDA     EALDR_RWTS_ERROR_FLAG
    RTS                     ; return with error flag in A

.seek_and_load:
    JSR     EALDR_RWTS_MOVE_ARM ; seek to track in A
    ; Fall through to .load_sectors

.load_sectors:
    ; Build destination page table: interleave[i] + base page
    LDY     #$0F
.build_table:
    LDA     EALDR_RWTS_INTERLEAVE,Y
    CLC
    ADC     $3E             ; add base page
    STA     EALDR_RWTS_DEST_PAGES,Y
```

```
    DEY
    BPL     .build_table

    LDX     #$60            ; slot index (hardcoded slot 6)
    STOB    #$60,$40 ; retry counter
.retry_sector:
    DEC     $40
    BEQ     .timeout
    JSR     EALDR_RWTS_SEARCH_ADDR ; find address field -> sector in $39
    BCS     .retry_sector   ; error -> retry

    LDY     $39             ; sector number
    LDA     EALDR_RWTS_DEST_PAGES,Y ; destination page for this sector
    BEQ     .retry_sector   ; zero = already read, skip
    STA     $45             ; destination page
    STOB    #$00,$44 ; destination = $xx00
    JSR     EALDR_RWTS_READ_SECTOR ; read and decode data field
    BCS     .retry_sector   ; error -> retry

    ; Mark sector as done
    LDY     $39
    LDA     #$00
    STA     EALDR_RWTS_DEST_PAGES,Y ; zero = done

    ; Check if all sectors read
    LDY     #$0F
.check_done:
    LDA     EALDR_RWTS_DEST_PAGES,Y
    BNE     .retry_sector   ; still have sectors to read
    DEY
    BPL     .check_done
    RTS                     ; all 16 sectors read

.timeout:
    INC     EALDR_RWTS_ERROR_FLAG ; signal error
    RTS
```

Defines:
EALDR_RWTS_READ_TRACK, used in chunks 872 and 884.
Uses EALDR_RWTS_DEST_PAGES 884, EALDR_RWTS_ERROR_FLAG 873a, EALDR_RWTS_INTERLEAVE 882b, EALDR_RWTS_MOVE_ARM 877, EALDR_RWTS_READ_SECTOR 874, EALDR_RWTS_SEARCH_ADDR 880, and STOB 8c.

884 ⟨*EALDR RWTS dest pages* 884⟩≡ (922)

```
    ORG     $BF74
EALDR_RWTS_DEST_PAGES:
    DS      16,$00          ; filled at runtime by EALDR_RWTS_READ_TRACK
```

Defines:
EALDR_RWTS_DEST_PAGES, used in chunk 883.
Uses EALDR_RWTS_READ_TRACK 883.

The remainder of the EALDR RWTS area ($BF84–$BFFF) is unused padding filled with an alternating $FF $00 pattern, except for the final byte at $BFFF which is $0F.

885a ⟨*EALDR RWTS tail padding* 885a⟩≡ (922)

```
    ORG     $BF84
EALDR_RWTS_TAIL_PADDING:
    HEX     FF 00 FF 00 FF 00 FF 00 FF 00 FF 00 FF 00 FF 00
    HEX     FF 00 FF 00 FF 00 FF 00 FF 00 FF 00 FF 00 FF 00
    HEX     FF 00 FF 00 FF 00 FF 00 FF 00 FF 00 FF 00 FF 00
    HEX     FF 00 FF 00 FF 00 FF 00 FF 00 FF 00 FF 00 FF 00
    HEX     FF 00 FF 00 FF 00 FF 00 FF 00 FF 00 FF 00 FF 00
    HEX     FF 00 FF 00 FF 00 FF 00 FF 00 FF 00 FF 00 FF 00
    HEX     FF 00 FF 00 FF 00 FF 00 FF 00 FF 00 FF 00 FF 00
    HEX     FF 00 FF 00 FF 00 FF 00 FF 00 FF 0F
```

Defines:
EALDR_RWTS_TAIL_PADDING, never used.

885b ⟨*EALDR track pages* 885b⟩≡ (922)

```
    ORG     $A9F3
EALDR_TRACK_PAGES:
    HEX     08 10 20 30 00
    HEX     40 50 60 70 80 86 00
    HEX     08 10 20 30 00

EALDR_TRACKS:
    HEX     03 04 21 22 00
    HEX     07 08 09 0A 0B 0C 00
    HEX     0D 0E 0F 10 00
```

Defines:
EALDR_TRACKS, used in chunk 842.
EALDR_TRACK_PAGES, used in chunk 842.

The address field search routine EALDR_FIND_SECTOR at $AA15 scans the disk for a D5 AA prologue, used by the sector loader to locate sectors.

885c ⟨*EALDR sector search* 885c⟩≡ (922)

```
    ORG     $AA15
EALDR_FIND_SECTOR:
    ; Sector search routine: $AA15-$AA34 (32 bytes)
    HEX     A0 FC 84 26 C8 D0 04 E6 26 F0 F3 AD EC C0 10 FB
    HEX     C9 D5 D0 F0 EA AD EC C0 10 FB C9 AA D0 F2 A0 03
```

Defines:
EALDR_FIND_SECTOR, never used.

Chapter 26

Startup

When EALDR finishes, it transfers control to `$0800` via `CALL0 0800`.

26.1 Memory map at entry

The VM program loads three groups of tracks using the track tables at `EALDR_TRACK_PAGES`/`EALDR_TRACKS`. Each track is 16 sectors of 256 bytes, placed into memory by the sector loader using the custom interleave table (see §24). The base page `XX` for each track determines the starting address, `$XX00`.

26.1.1 Group 1 (conditional, skipped in this image)

Loaded only when the flag byte at `$A805` is nonzero (it is 0 in this disk image, so Group 1 is skipped). Group 3 overwrites the same region anyway.

Track	Base page	Memory range
$03	$08	$0800–$17FF
$04	$10	$1000–$1FFF
$21	$20	$2000–$2FFF
$22	$30	$3000–$3FFF

26.1.2 Group 2: game data

Track	Base page	Memory range	Notes
$07	$40	$4000–$4FFF	
$08	$50	$5000–$5FFF	
$09	$60	$6000–$6FFF	
$0A	$70	$7000–$7FFF	
$0B	$80	$8000–$8FFF	
$0C	$86	$8600–$95FF	Overwrites $8600–$8FFF from track $0B

26.1.3 Group 3: main program

Track	Base page	Memory range	Notes
$0D	$08	$0800–$17FF	
$0E	$10	$1000–$1FFF	Overwrites $1000–$17FF from track $0D
$0F	$20	$2000–$2FFF	
$10	$30	$3000–$3FFF	

26.1.4 DXR decryption

After loading, the VM decrypts every other byte from `$4000` to `$67FE` (5120 iterations, stepping by 2). The XOR key for each byte comes from `EALDR_FUNNY_INC`, which generates the sequence

$$1,\ 2,\ 3,\ 1,\ 2,\ 3,\ 4,\ 1,\ 2,\ 3,\ 4,\ 5,\ \ldots$$

starting with `EALDR_BOT`=2, `EALDR_TOP`=3. Each "round" counts $1 \ldots N-1$, then N increments: the round length grows by one each time.

The loop terminates when the source pointer's high byte equals `$68` (i.e. src reaches `$6800`), giving `src_hi XOR $68 = 0`.

26.1.5 Final copy

The VM copies `$A300`–`$A5FF` (768 bytes from the EALDR image) to `$0500`–`$07FF`. This provides a small stub of routines and data below the main entry point.

26.1.6 Summary

At the `CALL0 $0800` instruction, memory is laid out as follows:

Address range	Source	Notes
$0500–$07FF	EALDR $A300–$A5FF	Copied last
$0800–$0FFF	Track $0D	
$1000–$1FFF	Track $0E	
$2000–$2FFF	Track $0F	
$3000–$3FFF	Track $10	
$4000–$67FF	Tracks $07–$09	DXR-decrypted (even bytes)
$6800–$6FFF	Track $09	Not decrypted
$7000–$7FFF	Track $0A	Not decrypted
$8000–$85FF	Track $0B	Not decrypted
$8600–$95FF	Track $0C	Not decrypted

Entry point: `$0800`.

26.2 Runtime memory map

During `GAME_INIT`, `RELOCATE_HIRES_STAGING` copies data from the hi-res page area into its runtime addresses: $2000–$32FF → $9600–$A8FF and $3300–$3FFF → $B300–$BFFF. The source area ($2000–$3FFF) is then cleared, freeing the hi-res page for graphics. After this relocation the full memory map is:

Address range	Size	Contents
$0500–$07FF	768	Resident routines (from EALDR $A300–$A5FF)
$0800–$1FFF	6 144	Main program code
$2000–$3FFF	8 192	Hi-res graphics page 1 (cleared by game_init)
$4000–$4008	9	Character record header (record 0)
$4009–$423F	567	Location / character records (62 × 9 bytes + 9-byte sentinel)
$4240–$451D	734	Location name strings
$451E–$4C7F	1 890	Location event lists (3-byte entries per location)
$4C80–$547F	2 048	Mob data records (128 × 16 bytes)
$5480–$59C8	1 353	Character / mob name strings
$59C9–$5AFF	311	Residual debug text (unused)
$5B00–$5FFF	1 280	Game state, display setup, UI tables
$6000–$7FFF	8 192	Main program code (continued)
$80A4–$83A4	769	Tune table and tune bytecode
$83A5–$92A4	3 840	Custom font data (120 chars × 32 bytes)
$92A8–$97A4	1 277	HRCG library (Hi-Res Character Generator)
$97A5–$9AA4	768	Standard font data (96 chars × 8 bytes)
$9AA5–$B2FF	6 235	*Unused / not mapped*
$B300–$B5FF	768	Disk I/O: VM interpreter and bytecode
$B600–$BFFF	2 560	DOS 3.3 RWTS (unmodified, relocated)

The main program code occupies two non-contiguous regions ($0800–$1FFF and $6000–$7FFF) with game data tables and the hi-res framebuffer in between.

26.3 Extracting `main.bin`

The following script reads the disk image, applies the EALDR loading sequence, and writes a flat binary covering `$0500`–`$95FF` (37 120 bytes).

889a ⟨*extract_main.py* 889a⟩≡
  ⟨*extract main imports* 889b⟩
  ⟨*extract main constants* 890a⟩
  ⟨*extract main read track* 890b⟩
  ⟨*extract main funny inc* 890c⟩
  ⟨*extract main dxr* 891a⟩
  ⟨*extract main entry* 891b⟩

889b ⟨*extract main imports* 889b⟩≡ (889a)
```
import sys
```

890a ⟨*extract main constants* 890a⟩≡ (889a)

```
DSK_FILE = "Ali Baba and the Forty Thieves (4am and san inc crack).dsk"
EALDR_FILE = "ealdr.bin"
OUTPUT_FILE = "main.bin"

BASE_ADDR = 0x0500
END_ADDR = 0x9600
SIZE = END_ADDR - BASE_ADDR  # $9100 = 37120

# DOS 3.3 physical-to-logical skew (physical sector -> file sector)
DOS_SKEW = [0, 7, 14, 6, 13, 5, 12, 4, 11, 3, 10, 2, 9, 1, 8, 15]

# Custom game loader interleave: physical sector -> page offset from base
LOADER_INTERLEAVE = [0xF, 0x8, 0x1, 0x9, 0x2, 0xA, 0x3, 0xB,
                     0x4, 0xC, 0x5, 0xD, 0x6, 0xE, 0x7, 0x0]

# Track loading tables (from EALDR $A9F3 and $AA04)
GROUP2 = [(0x07, 0x40), (0x08, 0x50), (0x09, 0x60),
          (0x0A, 0x70), (0x0B, 0x80), (0x0C, 0x86)]
GROUP3 = [(0x0D, 0x08), (0x0E, 0x10), (0x0F, 0x20), (0x10, 0x30)]
```

890b ⟨*extract main read track* 890b⟩≡ (889a)

```
def read_track(dsk, track, base_page, mem):
    """Read 16 sectors of a track into mem using the custom interleave.

    Each physical sector on disk maps to a page offset via
    LOADER_INTERLEAVE, so the data lands at (base_page + offset) * 256.
    """
    for phys in range(16):
        page_offset = LOADER_INTERLEAVE[phys]
        dos_logical = DOS_SKEW[phys]
        file_pos = (track * 16 + dos_logical) * 256
        dest = (base_page + page_offset) * 256 - BASE_ADDR
        if 0 <= dest and dest + 256 <= SIZE:
            mem[dest:dest + 256] = dsk[file_pos:file_pos + 256]
```

890c ⟨*extract main funny inc* 890c⟩≡ (889a)

```
def funny_inc_gen(bot=2, top=3):
    """Yield the EALDR_FUNNY_INC key sequence: 1,2,3, 1,2,3,4, ..."""
    while True:
        bot += 1
        if bot == top:
            bot = 1
            top += 1
            yield 1
        else:
            yield bot
```

Uses EALDR_FUNNY_INC 854b.

891a ⟨*extract main dxr* 891a⟩≡ (889a)

```
def dxr_decrypt(mem):
    """XOR every other byte from $4000 to $67FE with the funny_inc key."""
    keys = funny_inc_gen()
    src = 0x4000
    while True:
        key = next(keys)
        offset = src - BASE_ADDR
        if 0 <= offset < SIZE:
            mem[offset] ^= key
        src += 2
        if (src >> 8) == 0x68:  # src_hi ^ $68 == 0 -> done
            break
```

891b ⟨*extract main entry* 891b⟩≡ (889a)

```
def main():
    with open(DSK_FILE, "rb") as f:
        dsk = f.read()
    with open(EALDR_FILE, "rb") as f:
        ealdr = f.read()

    mem = bytearray(SIZE)

    # Group 2: tracks $07-$0C -> $4000-$95FF
    for track, base_page in GROUP2:
        read_track(dsk, track, base_page, mem)

    # Group 3: tracks $0D-$10 -> $0800-$3FFF
    for track, base_page in GROUP3:
        read_track(dsk, track, base_page, mem)

    # DXR decrypt $4000-$67FE
    dxr_decrypt(mem)

    # Copy EALDR $A300-$A5FF -> $0500-$07FF
    EALDR_SRC = 0xA300 - 0xA000  # offset in ealdr.bin
    mem_dest = 0x0500 - BASE_ADDR
    mem[mem_dest:mem_dest + 0x300] = ealdr[EALDR_SRC:EALDR_SRC + 0x300]

    with open(OUTPUT_FILE, "wb") as f:
        f.write(mem)
    print(f"Wrote {len(mem)} bytes to {OUTPUT_FILE}")
    print(f"Load in Ghidra at $0500, entry point $0800")

if __name__ == "__main__":
    main()
```

Chapter 27

Dead Code

The following chunks contain confirmed dead code: unreachable routines, residual strings, and unused data left from earlier builds. They are kept in the assembly output to preserve byte-for-byte binary fidelity, but serve no runtime purpose.

27.1 Residual development string

The 14 bytes at `$06F2--$06FF` between `CONTEXT_SWAP` and `RES_VM_DISPATCH` are unreferenced and decode as the high-bit ASCII fragment "`DON'T BREAK TH`"—likely a truncated development message.

The padding is done so that the first VM opcode starts on a page boundary. This makes it so that the opcode offset table only needs one byte per opcode.

892 ⟨*context swap padding* 892⟩≡ (909)

```
      ORG     $06F2
  RESIDUAL_DONT_BREAK:
  ; Residual string: "DON'T BREAK TH" (high-bit ASCII, truncated)
      HEX     c4cfcea7d4a0c2d2c5c1cba0d4c8
```

Defines:
`RESIDUAL_DONT_BREAK`, never used.

The 9 bytes at $07F7--$07FF are also residual data from the EALDR copy, decoding as the low-ASCII fragment "e checker" (likely the tail of "type checker" or similar). Related to the "DON'T BREAK TH" fragment above—both are truncated development messages that survived in the resident page.

893a ⟨*resident page residual* 893a⟩≡ (909)

```
    ORG     $07F7
RESIDUAL_E_CHECKER:
; Residual string: "e checker" (low ASCII)
    HEX     6520636865636b6572
```

Defines:
RESIDUAL_E_CHECKER, never used.

27.2 Compressed cursor routine

SET_CURSOR_COMPRESSED packs column (bits 5–0) and row offset (bits 7–6, added to 20) into a single byte. No callers exist in the binary (Ghidra confirms zero xrefs). Likely an unused utility from development.

893b ⟨*set cursor compressed* 893b⟩≡ (909)

```
    ORG     $776E
SET_CURSOR_COMPRESSED:
    SUBROUTINE SET_CURSOR_COMPRESSED
    ; Dead code: no callers found in binary.
    ; Input:  A = packed cursor (bits 5-0 = col, bits 7-6 = row offset)
    ; Output: TEXT_COL, TEXT_ROW set
    ; Clobbers: A

    PHA
    AND     #$3F
    STA     TEXT_COL
    PLA
    LSR
    LSR
    LSR
    LSR
    LSR
    LSR
    CLC
    ADC     #20
    STA     TEXT_ROW
    RTS
```

Defines:
SET_CURSOR_COMPRESSED, never used.
Uses TEXT_COL 572a and TEXT_ROW 572a.

27.3 Leftover DOS code

HRCG library tail ($A264–$A2FF): code and data from the HRCG library. Contains subroutines with internal references to $2Cxx addresses (the library's original load address).

The only confirmed code references are from the DOS 3.3 RWTS at $B600+, for the verify function, but the verify function is never used. Thus, this is dead code.

894 ⟨*hrcg library tail* 894⟩≡ (909)

```
    ORG     $A264
HRCG_LIBRARY_TAIL:
    HEX     B1 8B 20 D6 2C C8 B1 8B 20 D6 2C C8 B1 8B 20 D6
    HEX     2C 4C 7A 2C C8 C8 C8 98 18 65 8B 85 8B A5 8C 69
    HEX     00 85 8C A5 8B C5 65 A5 8C E5 66 90 B4 A9 00 20
    HEX     D6 2C A2 00 A9 00 A0 02 20 FA 2C A5 82 48 A5 81
    HEX     20 D6 2C 68 20 D6 2C 24 A3 10 0A A5 7F 20 D6 2C
    HEX     A5 80 20 D6 2C 20 B8 2B 46 A6 A0 1D B9 A6 2E C9
    HEX     A0 D0 03 88 10 F6 18 69 01 99 A6 2E 68 AA 68 A8
    HEX     68 60 8D 94 2E 48 98 48 8A 48 A2 00 20 C5 2D 20
    HEX     D2 2D EE 8E 2E D0 03 EE 8F 2E E6 81 D0 02 E6 82
    HEX     68 AA 68 A8 68 60 84 AA 85 AB 48 98
```

Defines:
HRCG_LIBRARY_TAIL, never used.

27.4 Residual assembler source text

The bytes at `$59C9--$5B0F` are high-ASCII text containing a fragment of 6502 assembly source code from the game's development. Since the data starting at `$5A00` is uninitialized runtime memory, the text bleeds over into that area. The text is part of a diagnostic routine that reads mob record fields via an `(ACTOR),Y` pointer and calls `SHOWNM` to display each value. The fragment begins mid-routine (the accumulator already holds a value) and is truncated at the end (`LDA SLFDUC+`).

This is assembler source left in memory from the development process, overwritten into the game data area but never cleared.

The label names reveal the original developer's naming conventions:

Developer label	Our label	Notes
`ACTOR`	`MOB_PTR` / `ENTITY_PTR`	entity record pointer
`ARG`	(unknown)	routine argument, likely zero page
`TEMP`	(unknown)	temporary 16-bit pointer
`SHOWNM`	(unknown)	"show name/number" display routine
`SLFSPD`	—	"self speed": mob byte 5 bits 4–0 (we call this "strength")
`SLFDX`	—	"self DX": mob byte 11 bits 4–0 (we call this "level")
`SLFDUC`	—	"self ducats": mob bytes 13–14 (16-bit) (entity's gold)

The "SLF" prefix is short for "self," referring to the entity's own record fields. "SPD" for what we call strength and "DX" (dexterity) for what we call level suggest the developer thought of these attributes differently than their final in-game labeling. "DUC" is short for "ducats" — the entity's gold, stored as a 16-bit value across mob data bytes 13–14.

895 ⟨*mob residual data* 895⟩≡ (909)

```
      ORG     $59C9
RESIDUAL_DEBUG_TEXT:
; Residual 6502 assembly source (high-ASCII text, $8D = newline)
; Decoded text:
;   STA ARG
;   LDA #>SLFSPD
;   STA TEMP
;   LDA #<SLFSPD
;   STA TEMP+1
;   JSR SHOWNM
;   LDY #11
;   LDA (ACTOR),Y
;   AND #$1F
```

```
;   STA ARG
;   LDA #>SLFDX
;   STA TEMP
;   LDA #<SLFDX
;   STA TEMP+1
;   JSR SHOWNM
;   LDY #13
;   LDA (ACTOR),Y
;   STA ARG
;   INY
;   LDA (ACTOR),Y
;   STA ARG+1
;   LDA #>SLFDUC
;   STA TEMP
;   LDA #<SLFDUC
;   STA TEMP+1
;   JSR SHOWNM
;   LDA SLFDUC+
    HEX     D3 D4 C1 A0 C1 D2 C7   ; "STA ARG"
    HEX     8D                      ; CR
    HEX     A0 CC C4 C1 A0 A3 BE   ; " LDA #>SLFS"
    HEX     D3 CC C6 D3 D0 C4      ; "LFSPD"
    HEX     8D                      ; CR
    HEX     A0 D3 D4 C1 A0 D4 C5 CD D0 ; " STA TEMP"
    HEX     8D                      ; CR
    HEX     A0 CC C4 C1 A0 A3 BC   ; " LDA #<SLFS"
    HEX     D3 CC C6 D3 D0 C4      ; "LFSPD"
    HEX     8D                      ; CR
    HEX     A0 D3 D4 C1 A0 D4 C5 CD D0 ; " STA TEMP" (truncated at $59FF)
```

Defines:
RESIDUAL_DEBUG_TEXT, never used.

Residual data gaps in the $5A variable page.

896a ⟨*var page gap 5a2d-5a50* 896a⟩≡ (909)

```
    ORG     $5A2D
RESIDUAL_5A_5A2D:
    HEX     A4 B1 C6 8D A0 D3 D4 C1 A0 C1 D2 C7 8D A0 CC C4
    HEX     C1 A0 A3 BE D3 CC C6 C4 D8 8D A0 D3 D4 C1 A0 D4
    HEX     C5 CD D0 8D
```

Defines:
RESIDUAL_5A_5A2D, never used.

896b ⟨*var page gap 5a5b-5a5c* 896b⟩≡ (909)

```
    ORG     $5A5B
RESIDUAL_5A_5A5B:
    HEX     C4 D8
```

Defines:
RESIDUAL_5A_5A5B, never used.

897a ⟨*var page gap 5a60-5a60* 897a⟩≡ (909)

```
      ORG     $5A60
  RESIDUAL_5A_5A60:
      HEX     D4
```

Defines:
RESIDUAL_5A_5A60, never used.

897b ⟨*var page gap 5a95-5a95* 897b⟩≡ (909)

```
      ORG     $5A95
  RESIDUAL_5A_5A95:
      HEX     C7
```

Defines:
RESIDUAL_5A_5A95, never used.

897c ⟨*var page gap 5aa5-5aa6* 897c⟩≡ (909)

```
      ORG     $5AA5
  RESIDUAL_5A_5AA5:
      HEX     CF D2
```

Defines:
RESIDUAL_5A_5AA5, never used.

897d ⟨*var page gap 5aa8-5aa8* 897d⟩≡ (909)

```
      ORG     $5AA8
  RESIDUAL_5A_5AA8:
      HEX     AC
```

Defines:
RESIDUAL_5A_5AA8, never used.

27.5 Orphaned encounter handler

The 231 bytes from $1F19 to $1FFF contain an orphaned earlier version of HANDLE_MOB_ENCOUNTER. No external JSR, JMP, branch, pointer, or RTS-trick reference reaches any address in $1F19--$1FFD; the code is unreachable during gameplay.

The logic mirrors the current HANDLE_MOB_ENCOUNTER almost exactly: threat assessment (strength + 16 − threats, clamped 4…28, randomised), flee with message #$1E, encounter dispatch, and a combat flag loop with message #$1F. However, all JSR/JMP targets point to addresses from an earlier build where every routine had shifted by a small offset (e.g. STEP_PRNG was at $764A instead of $78A8). The first five bytes are garbled—an ORA \$CE12 into expansion-ROM space followed by two undocumented NMOS 6502 $5A NOPs—suggesting the entry point was partially overwritten.

The only live byte is the RTS at $1FFE, which serves as a no-op stub for external callers at $4A1F and $4BE4.

898 ⟨*orphaned encounter handler* 898⟩≡ (909)

```
    ORG     $1F19
ORPHANED_ENCOUNTER_HANDLER:
    SUBROUTINE ORPHANED_ENCOUNTER_HANDLER
    ; Dead code: unreachable encounter handler with stale addresses.
    ; Clobbers: A, X, Y

; Garbled entry: ORA $CE12 + two undocumented $5A NOPs
    HEX     0d12ce5a5a

; Event record update (earlier build's entry path?)
    LDA     EVENT_SLOT_BYTE
    AND     #$E0
    ORA     EVENT_SUB_VALUE
    LDY     #$02
    STA     (EVENT_PTR),Y
    JMP     .save_pos

; Alternate entry: mark record as deleted
    LDY     #$00
    LDA     #$FE
    STA     (EVENT_PTR),Y

.save_pos:
    MOVB    CURRENT_COL,TURN_START_COL
    MOVB    CURRENT_ROW,TURN_START_ROW
    JSR     $1C6B
    STOB    #$14,WAIT_LOOP_COUNT
    JSR     $0DA4
    JMP     $1EC5
```

```
; Threat assessment
    STOB    #$00,ADJACENT_THREAT
    MOVW    ENTITY_PTR,TARGET_REC
    LDA     CURRENT_COL
    LDY     CURRENT_ROW
    JSR     $0DDF           ; -> COLROW_TO_POS (stale)
    JSR     $0F90           ; -> SUM_HOSTILE_AT_POS (stale)
    LDA     ADJACENT_THREAT
    BEQ     .no_flee

    LDY     #MFIELD_STRENGTH
    LDA     (ENTITY_PTR),Y
    AND     #STRENGTH_MASK
    CLC
    ADC     #$10
    SEC
    SBC     ADJACENT_THREAT
    BMI     .clamp_low
    BAMI    #$1D,.in_range
    LDA     #$1C
    BNE     .in_range
.clamp_low:
    LDA     #$04
.in_range:
    ASL
    ASL
    ORA     #$03
    TAX
    JSR     $764A           ; -> STEP_PRNG (stale)
    TXA
    BAPL    PRNG_OUTPUT,.no_flee

    JSR     $7444           ; -> SET_CURSOR_ROW21 (stale)
    JSR     $0DD1           ; -> PRINT_MOB_NAME (stale)
    JSR     $7421           ; -> SCROLL_STATUS_LINE (stale)
    LDA     #MSG_FLYING_TACKLE
    JSR     $739F           ; -> DISPLAY_MESSAGE (stale)
    MOVB    TURN_START_COL,CURRENT_COL
    MOVB    TURN_START_ROW,CURRENT_ROW
    JMP     $1C91           ; -> END_TURN (stale)

.no_flee:
    JSR     $1C3C           ; -> COMMIT_MOVE (stale)
    LDY     #MFIELD_POSITION
    LDA     (ENTITY_PTR),Y
    JSR     $0E6C           ; -> CHECK_ENCOUNTER (stale)
    LDA     ENCOUNTER_RESULT
    BAPL    #$02,.hostile
    JSR     $5F19           ; -> CHECK_THREATS_HERE (stale)
```

```
    BAEQ    #$01,.one_threat
    JMP     $1C91           ; -> END_TURN (stale)
.one_threat:
    JMP     $1AD8           ; -> MOVE_LOOP (stale)

.hostile:
    JSR     $1341           ; -> FIRST_GROUP_MEMBER (stale)
.walk_group:
    LDA     MOB_PTR+1
    BEQ     .group_done
    LDY     #MFIELD_POSITION
    LDA     (ENTITY_PTR),Y
    CMP     (MOB_PTR),Y
    BNE     .next_member
    JSR     .set_combat_flag
.next_member:
    JSR     $1347           ; -> NEXT_GROUP_MEMBER (stale)
    JMP     .walk_group

.group_done:
    JSR     $7444           ; -> SET_CURSOR_ROW21 (stale)
    JSR     $0DD1           ; -> PRINT_MOB_NAME (stale)
    JSR     $7421           ; -> SCROLL_STATUS_LINE (stale)
    LDA     #MSG_UNCONSCIOUS
    JSR     $739F           ; -> DISPLAY_MESSAGE (stale)
    JMP     $1C91           ; -> END_TURN (stale)

.set_combat_flag:
    LDY     #MFIELD_FLAGS
    LDA     (MOB_PTR),Y
    ORA     #$04
    STA     (MOB_PTR),Y
    RTS

; Trailing orphan byte
    HEX     61
```

Defines:
ORPHANED_ENCOUNTER_HANDLER, never used.

Uses ADJACENT_THREAT 226b, CHECK_ENCOUNTER 312, CHECK_THREATS_HERE 323, COLROW_TO_POS 600a, COMMIT_MOVE 239, CURRENT_COL 472b, CURRENT_ROW 473a, DISPLAY_MESSAGE 637, ENCOUNTER_RESULT 226a, END_TURN 241, ENTITY_PTR 561, EVENT_PTR 353a, EVENT_SLOT_BYTE 359a, EVENT_SUB_VALUE 359b, FIRST_GROUP_MEMBER 482, MFIELD_POSITION 561, MFIELD_STRENGTH 561, MOB_PTR 608, MOVB 8c, MOVE_LOOP 232, MOVW 9a, MSG_FLYING_TACKLE 634, MSG_UNCONSCIOUS 634, NEXT_GROUP_MEMBER 482, PRINT_MOB_NAME 484, PRNG_OUTPUT 593a, SCROLL_STATUS_LINE 639, SET_CURSOR_ROW21 627, STEP_PRNG 595, STOB 8c, SUM_HOSTILE_AT_POS 322, TARGET_REC 618b, TURN_START_COL 230a, TURN_START_ROW 230b, and WAIT_LOOP_COUNT 51.

27.6 Apple 6502 Assembler/Editor residual

Residual dead data from the Apple 6502 Assembler/Editor (DOS Toolkit).[1] This region ($A5BC–$A8C6) was occupied by the assembler during development and was never overwritten. The interpretation of this data is merely speculation.

This data section contains the assembler's editor display template, error/status messages, the complete opcode and directive lookup table (83 entries), and trailing addressing-mode lookup data.

The opcode table entries have variable length:

- Mnemonic: 2–5 ASCII bytes, last byte OR'd with $80
- Mode byte: $00=multi-mode, $01=JMP, $02=X/Y swap, $04=shift/rotate, $08=branch, $10=implied, $80=directive, $81=directive with expression
- Data: 1–2 more bytes (mode mask and/or base opcode, or handler address for directives)

For multi-mode opcodes, the base opcode plus an entry from the mode offset table (+04 through +20) yields the opcode for each addressing mode. Two-character directives (DW, DS, DO) use a high-ASCII space ($A0) as their third byte.

901 ⟨*apple asm residual* 901⟩≡ (909)

```
      ORG     $A5BC
  APPLE_ASM_RESIDUAL:
  ;
  ; --- Editor display template (122 bytes) ---
  ; High-ASCII text: "T"=Tab, "F"=Format, "S"=Set, "-"=dash
  ; Rows 0-3: tab/format ruler display
  ;   "  T     T     T "
      HEX     A0 A0 D4 A0 A0 A0 A0 A0 D4 A0 A0 A0 A0 A0 D4 A0
  ;   "  F T   F T   F "
      HEX     A0 A0 C6 A0 D4 A0 A0 A0 C6 A0 D4 A0 A0 A0 C6 A0
  ;   "T   F T - S   - "
      HEX     D4 A0 A0 A0 C6 A0 D4 A0 AD A0 D3 A0 A0 A0 AD A0
  ;   "S   - S-  - S-  "
      HEX     D3 A0 A0 A0 AD A0 D3 AD A0 A0 AD A0 D3 AD A0 A0
  ; Rows 4-7: editor settings/parameters
      HEX     E2 9F E0 88             ; control chars (BS, etc.)
      HEX     B7 A0 B3 B0 B0          ; "7 300"
      HEX     A0 B7 A0 B3 B0 B0       ; " 7 300"
      HEX     A0 B7 A0 B9 B0 B0       ; " 7 900"
      HEX     A0 B7 A0 B9 B0 B0       ; " 7 900"
```

[1] https://archive.org/details/apple-6502-assembler-editor

```
    HEX     A0 B7 A0 B9 B0          ; " 7 90"
    HEX     B0 A0 B7 A0 B9 B0       ; "0 7 90"
    HEX     A0 A0 B5 A0 C4 B0       ; "  5 D0"
    HEX     A0 A0 B5 A0             ; "  5 "
    HEX     C4 B0 A0               ; "D0 "
    HEX     00                      ; null terminator
    HEX     8C                      ; form feed (clear screen)
    HEX     2E 76 2E A6 2E C4 2E   ; ".v.&.D." (version tag)
;
; --- Error/status messages ---
; "-----"
    HEX     AD AD AD AD AD          ; "-----"
; "- NEXT OBJECT FILE NAME IS " CR
    HEX     A0 CE C5 D8 D4 A0 CF C2 CA C5 C3 D4
    HEX     A0 C6 C9 CC C5 A0 CE C1 CD C5 A0 C9 D3 A0
    HEX     8D                      ; carriage return
; "SOURCE FILE: ** "
    HEX     D3 CF D5 D2 C3 C5 A0 C6 C9 CC C5 BA A0 AA AA A0
; "OOPS! DOS ERROR! CODE="
    HEX     CF CF D0 D3 A1 A0 C4 CF D3 A0 C5 D2 D2 CF D2 A1
    HEX     A0 C3 CF C4 C5 BD
    HEX     87 87 87                ; three BELLs
;
; --- Assembler internal data (30 bytes) ---
; Expression evaluator dispatch or tokenizer data
    HEX     A8 01 AC D8 A9 02 A3 01 06 A3 03
    HEX     06 A8 01 A9 AC D9 0A C1 16 01 AC D8 0C 03 AC D9
    HEX     0E 03 AC D8 10 A8 03 A9 12 01 AC D9 14 01 04 03
    HEX     08 00
;
; ================================================================
; Opcode/directive lookup table (83 entries, 504 bytes)
; ================================================================
; Entry format: name (last char|$80) + mode + [mask +] base/opcode
;
; --- Multi-mode ALU opcodes ($00 = multi-mode) ---
; Base opcode + mode offset table entry = final opcode
    HEX     41 44 C3 00 FF 5D       ; ADC   mask=$FF base=$5D
    HEX     41 4E C4 00 FF 1D       ; AND   mask=$FF base=$1D
; --- Shift/rotate ($04 = shift, includes accumulator mode) ---
    HEX     41 53 CC 04 AA FE       ; ASL   mask=$AA base=$FE
; --- Branch instructions ($08 = relative) ---
    HEX     42 43 C3 08 00 90       ; BCC   opcode=$90
    HEX     42 4C D4 08 00 90       ; BLT   opcode=$90 (=BCC)
    HEX     42 47 C5 08 00 B0       ; BGE   opcode=$B0 (=BCS)
    HEX     42 43 D3 08 00 B0       ; BCS   opcode=$B0
    HEX     42 45 D1 08 00 F0       ; BEQ   opcode=$F0
    HEX     42 4D C9 08 00 30       ; BMI   opcode=$30
    HEX     42 4E C5 08 00 D0       ; BNE   opcode=$D0
    HEX     42 50 CC 08 00 10       ; BPL   opcode=$10
```

```
    HEX     42 56 C3 08 00 50       ; BVC   opcode=$50
    HEX     42 56 D3 08 00 70       ; BVS   opcode=$70
; --- Multi-mode / implied ---
    HEX     42 49 D4 00 0A 1C       ; BIT   mask=$0A base=$1C
    HEX     42 52 CB 10 00 00       ; BRK   implied  opcode=$00
; --- Implied ($10 = no operand) ---
    HEX     43 4C C3 10 00 18       ; CLC   implied  opcode=$18
    HEX     43 4C C4 10 00 D8       ; CLD   implied  opcode=$D8
    HEX     43 4C C9 10 00 58       ; CLI   implied  opcode=$58
    HEX     43 4C D6 10 00 B8       ; CLV   implied  opcode=$B8
; --- Multi-mode ---
    HEX     43 4D D0 00 FF BD       ; CMP   mask=$FF base=$BD
    HEX     43 50 D8 00 0E DC       ; CPX   mask=$0E base=$DC
    HEX     43 50 D9 00 0E BC       ; CPY   mask=$0E base=$BC
    HEX     44 45 C3 00 AA BE       ; DEC   mask=$AA base=$BE
; --- Implied ---
    HEX     44 45 D8 10 00 CA       ; DEX   implied  opcode=$CA
    HEX     44 45 D9 10 00 88       ; DEY   implied  opcode=$88
; --- Multi-mode ---
    HEX     45 4F D2 00 FF 3D       ; EOR   mask=$FF base=$3D
    HEX     49 4E C3 00 AA DE       ; INC   mask=$AA base=$DE
; --- Implied ---
    HEX     49 4E D8 10 00 E8       ; INX   implied  opcode=$E8
    HEX     49 4E D9 10 00 C8       ; INY   implied  opcode=$C8
; --- JMP ($01 = abs + indirect) ---
    HEX     4A 4D D0 01 08 3C       ; JMP   mode=$01 base=$3C
; --- Multi-mode ---
    HEX     4A 53 D2 00 08 10       ; JSR   mask=$08 base=$10
    HEX     4C 44 C1 00 FF 9D       ; LDA   mask=$FF base=$9D
; --- X/Y swap ($02 = uses ZP,Y / abs,Y) ---
    HEX     4C 44 D8 02 4E 9E       ; LDX   mask=$4E base=$9E
; --- Multi-mode ---
    HEX     4C 44 D9 00 AE 9C       ; LDY   mask=$AE base=$9C
; --- Shift ---
    HEX     4C 53 D2 04 AA 3E       ; LSR   mask=$AA base=$3E
; --- Implied ---
    HEX     4E 4F D0 10 00 EA       ; NOP   implied  opcode=$EA
; --- Multi-mode ---
    HEX     4F 52 C1 00 FF FD       ; ORA   mask=$FF base=$FD
; --- Implied ---
    HEX     50 48 C1 10 00 48       ; PHA   implied  opcode=$48
    HEX     50 48 D0 10 00 08       ; PHP   implied  opcode=$08
    HEX     50 4C C1 10 00 68       ; PLA   implied  opcode=$68
    HEX     50 4C D0 10 00 28       ; PLP   implied  opcode=$28
; --- Shift ---
    HEX     52 4F CC 04 AA 1E       ; ROL   mask=$AA base=$1E
    HEX     52 4F D2 04 AA 5E       ; ROR   mask=$AA base=$5E
; --- Implied ---
    HEX     52 54 C9 10 00 40       ; RTI   implied  opcode=$40
    HEX     52 54 D3 10 00 60       ; RTS   implied  opcode=$60
```

```
; --- Multi-mode ---
    HEX     53 42 C3 00 FF DD       ; SBC   mask=$FF base=$DD
; --- Implied ---
    HEX     53 45 C3 10 00 38       ; SEC   implied  opcode=$38
    HEX     53 45 C4 10 00 F8       ; SED   implied  opcode=$F8
    HEX     53 45 C9 10 00 78       ; SEI   implied  opcode=$78
; --- Multi-mode ---
    HEX     53 54 C1 00 FB 7D       ; STA   mask=$FB base=$7D
; --- X/Y swap ---
    HEX     53 54 D8 02 0A 7E       ; STX   mask=$0A base=$7E
; --- Multi-mode ---
    HEX     53 54 D9 00 2A 7C       ; STY   mask=$2A base=$7C
; --- Implied ---
    HEX     54 41 D8 10 00 AA       ; TAX   implied  opcode=$AA
    HEX     54 41 D9 10 00 A8       ; TAY   implied  opcode=$A8
    HEX     54 59 C1 10 00 98       ; TYA   implied  opcode=$98
    HEX     54 53 D8 10 00 BA       ; TSX   implied  opcode=$BA
    HEX     54 58 C1 10 00 8A       ; TXA   implied  opcode=$8A
    HEX     54 58 D3 10 00 9A       ; TXS   implied  opcode=$9A
;
; --- Assembler directives ---
; $81 = directive with expression operand, $80 = directive (no expression)
; Data bytes after mode = handler address (hi, lo) in assembler code
    HEX     45 51 D5 81 15 55       ; EQU   $81 handler=$1555
    HEX     4F 52 C7 81 15 92       ; ORG   $81 handler=$1592
    HEX     4F 42 CA 81 16 79       ; OBJ   $81 handler=$1679
    HEX     4C 53 D4 80 17 07       ; LST   $80 handler=$1707
    HEX     44 46 C2 80 17 14       ; DFB   $80 handler=$1714
    HEX     50 41 47 C5 80 17 DD    ; PAGE  $80 handler=$17DD
    HEX     41 53 C3 80 18 2B       ; ASC   $80 handler=$182B
    HEX     44 43 C9 80 18 86       ; DCI   $80 handler=$1886
    HEX     44 57 A0 80 18 B4       ; DW    $80 handler=$18B4
    HEX     4D 53 C2 80 18 A0       ; MSB   $80 handler=$18A0
    HEX     44 44 C2 80 18 DD       ; DDB   $80 handler=$18DD
    HEX     44 53 A0 81 16 B3       ; DS    $81 handler=$16B3
    HEX     52 45 D0 80 19 08       ; REP   $80 handler=$1908
    HEX     43 48 D2 80 19 2B       ; CHR   $80 handler=$192B
    HEX     53 4B D0 80 19 36       ; SKP   $80 handler=$1936
    HEX     44 53 45 43 D4 80 19 5C ; DSECT $80 handler=$195C
    HEX     44 45 4E C4 80 19 80    ; DEND  $80 handler=$1980
    HEX     44 4F A0 81 19 AB       ; DO    $81 handler=$19AB
    HEX     45 4C 53 C5 80 19 BD    ; ELSE  $80 handler=$19BD
    HEX     46 49 CE 80 19 C9       ; FIN   $80 handler=$19C9
    HEX     43 48 CE 80 19 D0       ; CHN   $80 handler=$19D0
    HEX     52 45 CC 80 1A B4       ; REL   $80 handler=$1AB4
    HEX     45 58 54 52 CE 81 1A F6 ; EXTRN $81 handler=$1AF6
    HEX     45 4E 54 52 D9 81 1A BF ; ENTRY $81 handler=$1ABF
    HEX     53 42 54 CC 80 17 F2    ; SBTL  $80 handler=$17F2
;
; --- End of table + trailing data ---
```

```
    HEX     00 00 00 00 00          ; end-of-table marker (5 null bytes)
; The remaining 27 bytes are assembler internal data whose exact
; purpose is unknown.  The first 8 bytes ($04..$20 = $04*1..$04*8)
; resemble addressing mode offsets added to base opcodes, but this
; is unconfirmed.
    HEX     04 08 0C 10 14 18 1C 20
    HEX     30 18 0C 00 00 00 00
    HEX     35 21 54 01 69 00
    HEX     01 03 00 13 3C 00
    HEX     14 19 5D
    HEX     FF                      ; gap byte ($A8CD)
; --- Pre-initialization data ($A8CE-$A8F0) ---
; Likely more assembler/editor residual data, not used at runtime.
    HEX     00 00 01 00 03 33 33 32 33 13 A5 13 02 15 02 00
    HEX     A3 1C 32 00 4C 14 00 BD C9 00 C0 04 00 3C 80 00
    HEX     FF 14 00
```

Defines:
APPLE_ASM_RESIDUAL, never used.

Chapter 28

Apple II Hi-Res Graphics

Hi-res graphics on the Apple II is odd. Graphics are memory-mapped, not exactly consecutively, and bits don't always correspond to pixels. Color especially is odd, compared to today's luxurious 32-bit per pixel RGBA.

The Apple II has two hi-res graphics pages, and maps the area from `$2000-$3FFF` to high-res graphics page 1 (HGR1), and `$4000-$5FFF` to page 2 (HGR2).

First we'll talk about pixels. Nominally, the resolution of the hi-res graphics screen is 280 pixels wide by 192 pixels tall. In the memory map, each row is represented by 40 bytes. The high bit of each byte is not used for pixel data, but is used to control color.

Here are some rules for how these bytes are turned into pixels:

- Pixels are drawn to the screen from byte data least significant bit first. This means that for the first byte bit 0 is column 0, bit 1 is column 1, and so on.
- A pattern of `11` results in two white pixels at the `1` positions.
- A pattern of `010` results at least in a colored pixel at the `1` position.
- A pattern of `101` results at least in a colored pixel at the `0` position.
- So, a pattern of `01010` results in at least three consecutive colored pixels starting from the first `1` to the last `1`. The last `0` bit would also be colored if followed by a `1`.
- Likewise, a pattern of `11011` results in two white pixels, a colored pixel, and then two more white pixels.

- The color of a `010` pixel depends on the column that the `1` falls on, and also whether the high bit of its byte was set or not.
- The color of a `11011` pixel depends on the column that the `0` falls on, and also whether the high bit of its byte was set or not.

	Odd	Even
High bit clear	Green	Violet
High bit set	Orange	Blue

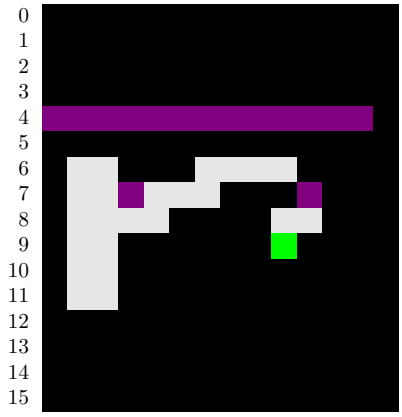
The implication is that you can only select one pair of colors per byte.

An example would probably be good here. We will take one of the font characters from the game, the lower-case `r`:

Bytes	Bits	Colorset	Pixel Data
00 00	00000000 00000000	0	00000000000000
00 00	00000000 00000000	0	00000000000000
00 00	00000000 00000000	0	00000000000000
00 00	00000000 00000000	0	00000000000000
55 2A	01010101 00101010	0	10101010101010
00 00	00000000 00000000	0	00000000000000
46 07	01000110 00000111	0	01100011110000
76 08	01110110 00001000	0	01101110001000
00 1E	00000000 00011110	0	01111000011000
00 06	00000000 00000110	0	01100000010000
00 06	00000000 00000110	0	01100000000000
00 06	00000000 00000110	0	01100000000000
2A 00	00101010 00000000	0	00000000000000
00 00	00000000 00000000	0	00000000000000
07 00	00000111 00000000	0	00000000000000
08 00	00001000 00000000	0	00000000000000

Table 28.1: Font character example

Assuming that the following bits are all zero, and we place the character starting at column 0, we should see this:



Here is a screenshot of that character, cut from the splash screen:

The violet line goes all the way across the character in that image because it is followed by another font character with that line, so the lines connect according to the pixel color rules.

Chapter 29

Assembly output

The following root chunks produce the assembly files when tangled.

909 ⟨*main.asm* 909⟩≡

```
    PROCESSOR 6502
⟨Macros 8a⟩
⟨EALDR VM macros 835⟩
⟨Disk VM macros 786⟩
⟨zero page defines 25a⟩
⟨defines 25b⟩

; The EA "checker" -- initial copy protection and verification
⟨reset vector 689⟩
⟨read data field 692⟩
⟨res vm loop 221⟩
⟨resident data and bytecodes 696⟩
⟨resident dispatch 702⟩
⟨resident dispatch table 701⟩
⟨context swap 700⟩
⟨context swap padding 892⟩
⟨resident decode dispatch 704⟩
⟨decode encrypted addr 706⟩
⟨res vm call1 707⟩
⟨pull two ret addrs 708⟩
⟨res vm call0 709a⟩
⟨res vm beq 709b⟩
⟨res vm ldi 709c⟩
⟨res vm ld 710a⟩
⟨res vm add 710b⟩
⟨res vm call 711a⟩
⟨res vm ret 711b⟩
⟨res vm st 712a⟩
⟨res vm subi 712b⟩
```

⟨*res vm inc* 713a⟩
⟨*decode dispatch relay* 713b⟩
⟨*resident page tail* 691⟩
⟨*resident page residual* 893a⟩

```
; Game code main
```
⟨*main entry* 219⟩
⟨*select input mode* 590⟩
⟨*process mob groups* 222⟩
⟨*show idle status* 224⟩
⟨*start mob turn* 225⟩
⟨*setup scene display* 532c⟩
⟨*load char name ptr* 486a⟩
⟨*setup char sprite* 487⟩
⟨*check mob needs turn* 227⟩
⟨*sound and settings* 686⟩
⟨*retire character* 268⟩
⟨*attract screen* 220⟩
⟨*scene setup* 494⟩
⟨*clamp char field* 609⟩
⟨*find first pc* 612⟩
⟨*find entity at pos* 469⟩
⟨*check at boundary* 537⟩
⟨*pos to colrow* 599⟩
⟨*get mob data* 602⟩
⟨*init mob count* 592⟩
⟨*get entity fontchar* 462⟩
⟨*appearance to fontchar* 461⟩
⟨*delay with animation* 603b⟩
⟨*tick anim counter* 604b⟩
⟨*anim tick and wait* 605b⟩
⟨*timed wait* 606a⟩
⟨*call rom wait* 607a⟩
⟨*idle anim handler* 52⟩
⟨*print mob name* 484⟩
⟨*colrow to pos* 600a⟩
⟨*num to decimal* 632⟩
⟨*check encounter* 312⟩
⟨*find mob at same pos* 315⟩
⟨*check hostile* 317⟩
⟨*get hostility* 318⟩
⟨*check adjacent threats* 320⟩
⟨*sum hostile at pos* 322⟩
⟨*reorder char* 478⟩
⟨*adjust char byte8* 481⟩
⟨*get char record* 26⟩
⟨*apply damage* 349⟩
⟨*print entity name* 483b⟩
⟨*draw char at pos* 471⟩
⟨*classify entity type* 459⟩

⟨*map value to string* 375⟩
⟨*check can leave* 456⟩
⟨*update event list* 454⟩
⟨*pick random member* 373⟩
⟨*advance group link* 482⟩
⟨*random in range* 597⟩
⟨*find nearest event* 398⟩
⟨*manhattan distance* 400⟩
⟨*print member name* 485⟩
⟨*reorder current char* 489a⟩
⟨*apply damage to current* 352⟩
⟨*apply damage to mob* 372a⟩
⟨*auto heal* 377⟩
⟨*increment hp* 378⟩
⟨*turn dispatch* 237⟩

```
; Combat
```
⟨*enter combat state* 327⟩
⟨*start combat* 328⟩
⟨*attack direction handlers* 329⟩
⟨*find attack target* 330⟩
⟨*init attack display* 332⟩
⟨*attack eligibility* 334⟩
⟨*resolve attack* 338⟩
⟨*add willingness* 372b⟩
⟨*set blink target* 475⟩
⟨*calc combat strength* 325⟩
⟨*player attack* 342⟩
⟨*process entity script* 371⟩
⟨*mob attack* 346⟩
⟨*select combat target* 368⟩
⟨*get entity threat* 365⟩
⟨*load target ptr* 370a⟩

```
; Game loop
```
⟨*game turn loop* 232⟩
⟨*commit move* 239⟩
⟨*end turn* 241⟩
⟨*set state fight* 243⟩
⟨*check and restrict* 244⟩
⟨*set char state* 242⟩
⟨*clear restricted* 245⟩
⟨*set restricted* 246⟩
⟨*handle encounter* 354⟩
⟨*show locked msg* 363⟩
⟨*trigger scene event* 444⟩
⟨*handle event slot* 360⟩
⟨*handle mob encounter* 357⟩
⟨*set target restricted* 364⟩
⟨*dispatch on mode* 607b⟩

⟨*orphaned encounter handler* 898⟩

```
; Graphics page
```
⟨*zeroed hires page* 567⟩

```
; Data tables
```
⟨*char record header* 53a⟩
⟨*Location data* 207⟩
⟨*Location names* 27⟩
⟨*Location events* 32⟩
⟨*mob data records* 68⟩
⟨*mob name strings* 197⟩
⟨*mob residual data* 895⟩

```
; Runtime data
```
⟨*var input mode* 589c⟩
⟨*var total mob count* 591b⟩
⟨*var current player* 562a⟩
⟨*var current room* 476⟩
⟨*var char sprite* 486b⟩
⟨*var char ai mode* 531b⟩
⟨*var ai wait flag* 532a⟩
⟨*var browse char index* 610b⟩
⟨*var view max col* 533b⟩
⟨*var view max row* 533c⟩
⟨*var view min col* 534a⟩
⟨*var view min row* 534b⟩
⟨*var font col* 492a⟩
⟨*var font row* 534c⟩
⟨*var font charnum* 575⟩
⟨*var font charset* 576a⟩
⟨*var damage amount* 348⟩
⟨*var rng limit* 596a⟩
⟨*var rng mask* 596b⟩
⟨*var nearest pos* 397b⟩
⟨*var nearest dist* 397c⟩
⟨*var candidate pos* 534d⟩
⟨*var heal notified* 376⟩
⟨*var location id* 536⟩
⟨*var prng output* 593a⟩
⟨*var prng state* 593b⟩
⟨*var prng temp* 594⟩
⟨*var num digit* 630b⟩
⟨*var num divisor* 630c⟩
⟨*var num leading* 631⟩
⟨*var encounter result* 226a⟩
⟨*var colocate link* 314a⟩
⟨*var colocate pos* 314b⟩
⟨*var colocate wrapped* 314c⟩
⟨*var hostile app* 316a⟩

⟨*var hostile a* 316b⟩
⟨*var hostile y* 316c⟩
⟨*var adjacent threat* 226b⟩
⟨*var adjacent pos* 319⟩
⟨*var source char* 492b⟩
⟨*var active char* 492c⟩
⟨*var reorder link* 477a⟩
⟨*var reorder sort key* 477b⟩
⟨*var group count delta* 480⟩
⟨*var page gap 5a2d-5a50* 896a⟩
⟨*var bounds min col* 492d⟩
⟨*var bounds max col* 492e⟩
⟨*var bounds min row* 492f⟩
⟨*var bounds max row* 493⟩
⟨*var location pos* 562b⟩
⟨*var location style* 563a⟩
⟨*var msg line count* 636⟩
⟨*var trap type* 446⟩
⟨*var event slot byte* 359a⟩
⟨*var event sub value* 359b⟩
⟨*var page gap 5a5b-5a5c* 896b⟩
⟨*var ai prev dir* 384a⟩
⟨*var event pos* 532b⟩
⟨*var event dest pos* 353c⟩
⟨*var page gap 5a60-5a60* 897a⟩
⟨*var turn start col* 230a⟩
⟨*var turn start row* 230b⟩
⟨*var current col* 472b⟩
⟨*var current row* 473a⟩
⟨*var move points* 230c⟩
⟨*var steps taken* 231a⟩
⟨*var input dir* 231b⟩
⟨*var ai best dist* 389a⟩
⟨*var ai saved pos* 393⟩
⟨*var dir scores* 389b⟩
⟨*var ai direction* 389c⟩
⟨*var ai test pos* 389d⟩
⟨*var ai target pos* 384b⟩
⟨*var ai target col* 384c⟩
⟨*var ai target row* 384d⟩
⟨*var is player turn* 474⟩
⟨*var saved mob link* 333⟩
⟨*var combat willingness* 337a⟩
⟨*var target defense* 337b⟩
⟨*var combat strength* 324⟩
⟨*var mob defense* 345⟩
⟨*var drop amount* 434b⟩
⟨*var drop step* 435⟩
⟨*var stat scale* 379b⟩
⟨*var stat level bits* 379c⟩

⟨*var shop item mask* 403a⟩
⟨*var shop price* 54c⟩
⟨*var shop valid mask* 403b⟩
⟨*var location style bits* 403c⟩
⟨*var shop item offset* 55⟩
⟨*var target ptr* 366⟩
⟨*var best threat* 367⟩
⟨*var scene found flag* 427a⟩
⟨*var scene hostile flag* 427b⟩
⟨*var scene group idx* 427c⟩
⟨*var wait loop count* 51⟩
⟨*var tune loop ctr* 651b⟩
⟨*var tune loop addr* 652a⟩
⟨*var tune gosub addr* 652b⟩
⟨*var tune prng mask* 652c⟩
⟨*var tune action vec* 652d⟩
⟨*var page gap 5a95-5a95* 897b⟩
⟨*var text right margin* 643a⟩
⟨*var text left margin* 643b⟩
⟨*var text stream idx* 643c⟩
⟨*var scene number* 526b⟩
⟨*var saved target rec* 618c⟩
⟨*var saved entity ptr* 618d⟩
⟨*var saved data ptr* 619a⟩
⟨*var saved mob ptr* 619b⟩
⟨*var blink col* 563b⟩
⟨*var blink row* 544b⟩
⟨*var blink char* 544c⟩
⟨*var page gap 5aa5-5aa6* 897c⟩
⟨*var attract flag* 652e⟩
⟨*var page gap 5aa8-5aa8* 897d⟩
⟨*var turn tune guard* 238⟩
⟨*var input option count* 249b⟩
⟨*var current selection* 54a⟩
⟨*var input buffer* 611⟩

```
; Display
```
⟨*init game state* 570b⟩
⟨*border animation* 584⟩
⟨*init game display* 574⟩
⟨*draw border char* 576b⟩
⟨*draw border labels* 578⟩
⟨*draw border icons* 580⟩
⟨*plot char* 49⟩
⟨*border animation data* 581⟩
⟨*ui text stream* 573⟩
⟨*border label positions* 577a⟩
⟨*border frame pattern* 577b⟩
⟨*copyright string* 570a⟩
⟨*boot display strings* 572b⟩

```
; Input
⟨input loop 247⟩
⟨input jump vector 53b⟩
⟨setup input 249a⟩
⟨draw input options 250⟩
⟨calc record ptr 251⟩
⟨setup cursor ext 252⟩
⟨setup cursor 253⟩
⟨set cursor pos 254⟩
⟨validate input 255⟩
⟨read input char 256⟩
⟨call input init 257⟩
⟨clear input buffer 258⟩
⟨render input buffer 259⟩
⟨copy record data 260⟩
⟨store selection to record 54b⟩
⟨game input table 261⟩
⟨teardown input 262⟩
⟨get input table ptr 263⟩
⟨configure input 264⟩
⟨menu handlers 265⟩
⟨attack direction table 266⟩
⟨other options table 267a⟩
⟨other options entry 267b⟩
⟨game settings table 269⟩
⟨game settings entry 270a⟩
⟨settings handlers 271⟩
⟨attack target table 272⟩
⟨drop gold table 273⟩

; Save and load
⟨save load table 274⟩
⟨init world 588a⟩
⟨save load init 275c⟩
⟨slot number routines 277⟩
⟨save game slot table 278⟩
⟨save game slot entry 279⟩
⟨scenario disk table 280a⟩
⟨scenario disk entry 280b⟩
⟨char select table 281⟩
⟨char select handlers 283⟩
⟨alt input table 285⟩
⟨start game input 286⟩

; More game code
⟨check threats here 323⟩
⟨pickup treasure 447⟩
⟨inc strength 450⟩
⟨clear treasure digits 451b⟩
```

⟨*clear and display* 452⟩
⟨*random events* 415⟩
⟨*event pick mob* 417a⟩
⟨*event pick char* 417b⟩
⟨*event faction scene* 419⟩
⟨*event spawn mob* 420⟩
⟨*pick random mob* 422⟩
⟨*pick random char* 423⟩
⟨*process char events* 424⟩
⟨*find mob by faction* 426⟩
⟨*check location match* 429⟩
⟨*npc move ai* 390⟩
⟨*evaluate direction* 394⟩
⟨*inc mob counters* 396⟩
⟨*ai choose target* 385⟩
⟨*continue game* 289⟩
⟨*shuffle entities* 291⟩

```
; Game save
```
⟨*save game* 293⟩
⟨*scenario disk handlers* 295⟩
⟨*ensure game disk* 297⟩
⟨*format and signature* 298⟩
⟨*setup data iob* 301⟩
⟨*write game data* 303⟩
⟨*init stub2* 588b⟩
⟨*read game data* 305⟩
⟨*disk read write* 546⟩
⟨*init disk iob* 589b⟩

```
; More game code
```
⟨*scene loop* 527⟩
⟨*text renderer* 644⟩
⟨*display status bar* 550⟩
⟨*save zp context* 619c⟩
⟨*restore zp context* 620⟩
⟨*relocate hires staging* 566⟩
⟨*load scene data* 529⟩
⟨*check scene events* 428⟩
⟨*scan for hostile* 430⟩
⟨*process scene groups* 431⟩
⟨*load char to fa* 488⟩
⟨*scan and reorder* 432⟩
⟨*flush pending xp* 433⟩
⟨*init drop gold* 436⟩
⟨*init drop display* 437⟩
⟨*adj drop amount* 438⟩
⟨*drop gold exec* 440⟩
⟨*arithmetic helpers* 442⟩
⟨*modify char stats* 380⟩

⟨*modify current stats* 382⟩
⟨*display shop* 404⟩
⟨*shop buy handler* 56⟩
⟨*advance shop ptr* 407b⟩
⟨*add price component* 408⟩
⟨*shop buy effects* 409⟩
⟨*xp award* 412⟩
⟨*enter location* 535⟩

```
; Map display
```
⟨*draw map col* 540⟩
⟨*draw map entities* 542⟩
⟨*draw map icon b* 545⟩
⟨*fill map* 538⟩
⟨*load map tiles* 531a⟩
⟨*render font char* 466⟩
⟨*draw blink alt* 472a⟩
⟨*draw blink char* 473b⟩
⟨*check combat eligibility* 308⟩
⟨*check input* 58⟩
⟨*display message* 637⟩
⟨*scroll status line* 639⟩
⟨*clear map* 539⟩
⟨*set text window bottom* 627⟩
⟨*setup text pos* 552⟩
⟨*print bottom centered* 628⟩
⟨*print centered plus col* 629⟩
⟨*font pos* 467⟩
⟨*control routines* 624⟩
⟨*print ctrl n* 623b⟩
⟨*text window routines* 625⟩
⟨*print ctrl ab* 623a⟩
⟨*print string* 621⟩
⟨*set cursor compressed* 893b⟩
⟨*display char stats* 556⟩
⟨*step prng* 595⟩

```
; Initialization
```
⟨*game init* 564⟩
⟨*init stub* 569a⟩
⟨*init hrcg* 568⟩
⟨*set text window upper left* 626⟩
⟨*clear strength flags* 61a⟩

```
; Audio
```
⟨*tune player* 653⟩
⟨*tune action indirect* 684a⟩
⟨*tune advance* 683⟩
⟨*tune helpers* 684b⟩
⟨*play tone* 647a⟩

⟨*play low tone* 648⟩
⟨*blink delay* 544a⟩
⟨*wait duration* 606c⟩
⟨*joy settle delay* 57⟩
⟨*anim frame ctr* 604a⟩
⟨*anim frame table* 605a⟩
⟨*max game speed* 270b⟩
⟨*anim index* 583a⟩
⟨*wait loop init* 353b⟩

```
; Constants
```
⟨*delay count* 603a⟩
⟨*slot number* 275b⟩
⟨*map char defaults* 530b⟩
⟨*map redraw flag* 541c⟩
⟨*game disk slots* 287⟩
⟨*max slot number* 276⟩
⟨*save track base* 304⟩
⟨*save sector count* 302⟩
⟨*game sig params* 300⟩
⟨*disk signature* 288⟩
⟨*shuffle count* 290⟩
⟨*tile style tables* 530c⟩
⟨*menu strings* 310⟩
⟨*stats template data* 554⟩
⟨*status border data* 490⟩
⟨*press space string* 572c⟩
⟨*press button string* 489b⟩
⟨*shop table* 402⟩
⟨*item name strings* 62⟩
⟨*map object names* 63⟩
⟨*attack verb strings* 374⟩
⟨*print font char data* 465⟩
⟨*tune pointer table* 651a⟩
⟨*tune bytecode* 654c⟩
⟨*tune bytecode data tail* 61b⟩
⟨*hrcg control prefix* 64⟩
⟨*duplicate item strings* 65⟩
⟨*hrcg control tail* 66a⟩
⟨*hrcg control continuation* 66b⟩
⟨*tune table copy* 66c⟩
⟨*tune bytecodes copy* 67a⟩
⟨*border bitmap data* 67b⟩
⟨*font data* 715a⟩

```
; High Resolution Character Generator
```
⟨*hrcg original entry* 61c⟩
⟨*hrcg init* 736⟩
⟨*hrcg version string* 715c⟩
⟨*hrcg params* 737a⟩

⟨*hrcg mode flags* 742⟩
⟨*hrcg video mode* 737b⟩
⟨*hrcg mask* 750⟩
⟨*hrcg draw mode* 747b⟩
⟨*hrcg wrap flag* 755⟩
⟨*hrcg invert flag* 752⟩
⟨*hrcg font ptr* 744⟩
⟨*hrcg saved state* 746⟩
⟨*hrcg getkey* 738⟩
⟨*hrcg cout* 743⟩
⟨*hrcg dispatch table* 741⟩
⟨*hrcg ctrl a* 745⟩
⟨*hrcg ctrl b* 747a⟩
⟨*hrcg ctrl c* 748⟩
⟨*hrcg ctrl d* 749⟩
⟨*hrcg ctrl efhi* 751⟩
⟨*hrcg ctrl klm* 753⟩
⟨*hrcg ctrl nopq* 754⟩
⟨*hrcg ctrl rst* 756⟩
⟨*hrcg ctrl vw* 757⟩
⟨*hrcg ctrl yz* 759⟩
⟨*hrcg draw char* 761⟩
⟨*hrcg advance cursor* 763⟩
⟨*hrcg scroll up* 764⟩
⟨*hrcg home clr* 766⟩
⟨*hrcg calc addr* 768⟩
⟨*std font data* 715b⟩

```
; Game messages
```
⟨*game message strings* 640⟩
⟨*hrcg library tail* 894⟩
⟨*resident tune bytecode* 656⟩
⟨*tune stubs* 662⟩
⟨*read keyboard routine* 663⟩
⟨*load scene from disk* 666⟩
⟨*tune player dispatch* 669⟩
⟨*integrity verification* 671⟩
⟨*cold restart* 675⟩
⟨*pointer helpers* 677⟩

```
; Unused padding
```
⟨*apple asm residual* 901⟩

```
; Uninitialized data
```
⟨*runtime data area* 50⟩

```
; Another EA checker
```
⟨*disk read field* 771⟩
⟨*verify data field* 773⟩
⟨*disk vm loop* 774⟩

⟨*disk vm bytecode* 775⟩
⟨*search addr field* 777⟩
⟨*read and verify* 779a⟩
⟨*disk vm sub park stepper* 779b⟩
⟨*delay loop* 780a⟩
⟨*disk vm sub exit err* 780b⟩
⟨*disk vm exit ok* 780c⟩
⟨*disk vm exit err* 781a⟩
⟨*disk vm misc b49c* 781b⟩
⟨*disk vm dispatch* 783⟩
⟨*disk vm swap* 781c⟩
⟨*disk vm data b4de* 782⟩
⟨*disk vm op goto* 787⟩
⟨*disk vm read addr* 785⟩
⟨*disk vm op call1* 788a⟩
⟨*disk vm pull ip* 784⟩
⟨*disk vm op call0* 788b⟩
⟨*disk vm op bnz* 789a⟩
⟨*disk vm op lda imm* 789b⟩
⟨*disk vm op lda abs* 789c⟩
⟨*disk vm op lda idx* 790a⟩
⟨*disk vm op call* 790b⟩
⟨*disk vm op ret* 791a⟩
⟨*disk vm op sta abs* 791b⟩
⟨*disk vm op sub imm* 791c⟩
⟨*disk vm op inc load* 792a⟩
⟨*disk vm trampoline* 792b⟩
⟨*disk vm error accum* 778⟩
⟨*disk vm saved state* 793⟩

```
; Disk I/O
```
⟨*dos33 rwts* 770⟩

921 ⟨*boot1.asm* 921⟩≡

```
        PROCESSOR 6502
```

⟨*Macros* 8a⟩
⟨*boot1 defines* 800a⟩
⟨*boot1 entry* 801⟩
⟨*boot1 saninc* 803⟩
⟨*boot1 read error* 805⟩
⟨*boot1 read sector* 806⟩
⟨*boot1 track vars* 808⟩
⟨*boot1 error flag* 800b⟩
⟨*boot1 seek track* 809⟩
⟨*boot1 phase on* 810a⟩
⟨*boot1 delay* 810b⟩
⟨*boot1 addr error* 811⟩
⟨*boot1 find sector* 812⟩
⟨*boot1 nibble tables* 814⟩
⟨*boot1 nibble tables b* 815a⟩
⟨*boot1 load track* 816⟩
⟨*boot1 interleave table* 815b⟩
⟨*boot1 dest pages* 817⟩
⟨*boot1 loader doc* 818⟩

922 ⟨*ealdr.asm* 922⟩≡

```
    PROCESSOR 6502
```

⟨*Macros* 8a⟩
⟨*EALDR VM macros* 835⟩
⟨*ealdr defines* 832⟩
⟨*EALDR prot VM macros* 820⟩
⟨*EALDR unused prot vm* 822⟩
⟨*EALDR resident copy* 831⟩
⟨*EALDR checksum source listing* 870⟩
⟨*EALDR entry* 833⟩
⟨*EALDR copy subroutine* 834a⟩
⟨*EALDR VM exec* 834b⟩
⟨*EALDR VM bytecode 1* 837⟩
⟨*EALDR clear text screen* 853a⟩
⟨*EALDR VM bytecode 2* 839⟩
⟨*EALDR reboot handler* 853b⟩
⟨*EALDR VM bytecode 3* 841⟩
⟨*EALDR funny increment* 854b⟩
⟨*EALDR VM bytecode 4* 842⟩
⟨*EALDR track pages* 885b⟩
⟨*EALDR sector search* 885c⟩
⟨*EALDR VM interpreter* 846⟩
⟨*EALDR dispatch table* 845b⟩
⟨*EALDR VM goto path* 848a⟩
⟨*EALDR VM arg reader* 847a⟩
⟨*EALDR VM init* 847b⟩
⟨*EALDR opcode handlers* 849⟩
⟨*EALDR disk io* 856⟩
⟨*EALDR check flag* 858⟩
⟨*EALDR prot entry* 859⟩
⟨*EALDR residual docs* 860⟩
⟨*EALDR padding* 861⟩
⟨*EALDR splash bitmap* 862⟩
⟨*EALDR source comments* 865⟩
⟨*EALDR disk entry points* 872⟩
⟨*EALDR RWTS track vars* 873a⟩
⟨*EALDR RWTS read sector* 874⟩
⟨*EALDR RWTS move arm* 877⟩
⟨*EALDR RWTS phase on* 878a⟩
⟨*EALDR RWTS delay* 878b⟩
⟨*EALDR RWTS addr error* 879⟩
⟨*EALDR RWTS search addr* 880⟩
⟨*EALDR RWTS padding* 881⟩
⟨*EALDR RWTS nibble tables* 882a⟩
⟨*EALDR RWTS nibble table B* 873b⟩
⟨*EALDR RWTS read track* 883⟩
⟨*EALDR RWTS interleave table* 882b⟩
⟨*EALDR RWTS dest pages* 884⟩
⟨*EALDR RWTS tail padding* 885a⟩

Chapter 30

Index

ACTIVE_CHAR: 220, 268, 283, 349, 354, 419, 420, 424, 429, 431, 478, 492c, 494
ADDA: 11a
ADDAC: 11b
ADDB: 12a
ADDB2: 12b
ADDW: 13a
ADDWC: 13b
ADD_DROP: 438, 442
ADD_PRICE_COMPONENT: 404, 408
ADD_WILLINGNESS: 338, 372b
ADJACENT_POS: 319, 320
ADJACENT_THREAT: 226b, 227, 308, 320, 322, 357, 898
ADJ_DROP_DEC: 273, 438
ADJ_DROP_INC: 273, 438
ADVANCE_CKSUM_START: 677
ADVANCE_RANGE_CTR: 677
ADVANCE_SECTOR: 303, 305
ADVANCE_SHOP_PTR: 404, 407b
AI_BEST_DIR: 388b, 390
AI_BEST_DIST: 389a, 390, 394
AI_BEST_POS: 383, 385
AI_BEST_STR: 383, 385
AI_CHOOSE_TARGET: 232, 383, 384b, 385
AI_DIRECTION: 389c, 390, 394, 396
AI_MIN_SCORE: 388b, 390
AI_PREV_DIR: 384a, 385, 390
AI_RANDOM_NORMAL: 388a, 390
AI_RANDOM_PHASE4: 388a, 390
AI_SAVED_POS: 390, 393, 394

AI_SCRATCH: 383, 385
AI_TARGET_COL: 384c, 385
AI_TARGET_POS: 384b, 385, 390, 394
AI_TARGET_ROW: 384d, 385
AI_TEST_POS: 389d, 390, 394
AI_WAIT_FLAG: 532a, 532c
ALT_INPUT_TABLE: 285, 286
ANIM_BORDER_ICONS_A: 584
ANIM_BORDER_ICONS_B: 584
ANIM_CHECK_FRAME: 52, 605b, 663
ANIM_FRAME_CTR: 604a, 604b, 605b
ANIM_FRAME_TABLE: 605a, 605b
ANIM_INDEX: 270a, 271, 583a, 604b, 605b
ANIM_TICK_AND_WAIT: 360, 390, 535, 545, 556, 603b, 605b, 607b, 663
ANIM_WAIT_TABLE: 604a, 604b
APPEARANCE_TO_FONTCHAR: 225, 317, 461, 462, 475, 542
APPLE_ASM_RESIDUAL: 901
APPLY_DAMAGE: 349, 352, 372a
APPLY_DAMAGE_TO_CURRENT: 352, 354, 360, 444, 447
APPLY_DAMAGE_TO_MOB: 346, 372a
APPLY_SHOP_EFFECT: 409
ATTACK_DIR_TABLE: 266
ATTACK_DOWN: 266, 329
ATTACK_LEFT: 266, 329
ATTACK_NAME_BUF: 331, 332
ATTACK_RIGHT: 266, 329
ATTACK_TARGET_TABLE: 272
ATTACK_UP: 266, 329
ATTACK_VERB_STRINGS: 374, 375
ATTRACT_FLAG: 652e, 653, 683
ATTRACT_SCREEN: 219, 220
AUTO_HEAL: 222, 232, 237, 377
AWARD_XP: 412
BAEQ,: 18
BAGE,: 18
BALT,: 18
BAMI,: 18
BANE,: 18
BAPL,: 18
BEST_THREAT: 367, 368
BLINK_ALT_CHAR: 462, 472a, 530b, 531a
BLINK_CHAR: 239, 354, 472a, 473b, 475, 487, 544c, 545
BLINK_COL: 222, 239, 354, 473b, 475, 487, 527, 545, 563b, 564
BLINK_DELAY: 544a, 545
BLINK_ROW: 239, 354, 473b, 475, 487, 544b, 545
BOOT1_ADDR_ERROR: 811, 812

BOOT1_CRACK: 803
BOOT1_DELAY: 801, 809, 810b
BOOT1_DEST_PAGES: 816, 817
BOOT1_ENTRY: 801
BOOT1_ERROR_FLAG: 800b, 801, 816
BOOT1_FIND_SECTOR: 812, 816
BOOT1_INTERLEAVE: 815b, 816
BOOT1_LOADER_DOC: 818, 860
BOOT1_LOAD_TRACK: 801, 816, 817
BOOT1_NIBBLE_TABLE_A: 814
BOOT1_NIBBLE_TABLE_B: 815a
BOOT1_PHASE_ON: 809, 810a
BOOT1_PHASE_ON_RTS: 809, 810a
BOOT1_PRINT_CR: 803
BOOT1_READ_ERROR: 805, 806
BOOT1_READ_SECTOR: 806, 816
BOOT1_SANINC_NOLC: 803
BOOT1_SECTOR_COUNT: 801
BOOT1_SEEK_TRACK: 801, 809, 816
BOOT1_TRACK: 808, 809, 810a
BORDER_ANIM_INDEX: 581, 584
BORDER_ANIM_SAVE: 581, 584
BORDER_BITMAP_DATA: 67b
BORDER_HANDLER_A: 581, 584
BORDER_HANDLER_B: 581, 584
BORDER_HANDLER_C: 581, 584
BORDER_HANDLER_TABLE: 581, 584
BORDER_ICON_CHARS: 581, 583b
BORDER_ICON_OFFSET: 583b, 584
BORDER_INITIAL_CHAR: 581, 584
BORDER_LOOP_CTR: 576d, 578, 584
BORDER_POS_HORIZ: 577a, 578
BORDER_POS_TABLE: 581, 584
BORDER_POS_VERT_L: 577a, 578
BORDER_POS_VERT_R: 577a, 578
BORDER_STR_HORIZ: 577b, 578
BORDER_STR_VERT_L: 577b, 578
BORDER_STR_VERT_R: 577b, 578
BORDER_SWAP_FLAG: 583b, 584
BOUNDS_MAX_COL: 492e, 494
BOUNDS_MAX_ROW: 493, 494
BOUNDS_MIN_COL: 492d, 494
BOUNDS_MIN_ROW: 492f, 494
BROWSE_CHAR_INDEX: 610b, 612, 615, 616
BROWSE_HANDLER: 56, 285, 404, 409, 412
BXEQ,: 18

BXNE,: 18
BYEQ,: 18
BYNE: 18, 259, 317, 469, 637, 753
CALC_COMBAT_STRENGTH: 324, 325, 342, 346
CALC_RECORD_PTR: 247, 250, 251, 255
CALL_INPUT_INIT: 249a, 257
CALL_ROM_WAIT: 52, 606a, 607a
CANCEL_SCENARIO: 280a, 295
CANDIDATE_POS: 398, 534d, 535
CFIELD_APPEARANCE: 478, 481, 561
CFIELD_DRAWN_FLAG: 541a, 542
CFIELD_EVENTS: 398, 424, 454, 469, 561
CFIELD_FACTION_MASK: 419, 420, 561
CFIELD_NEXT: 222, 315, 426, 478, 482, 542, 561
CFIELD_POSITION: 561
CFIELD_STYLE: 419, 420, 533a, 535, 541a, 542, 561
CFIELD_XP: 373, 385, 420, 422, 561
CHARS_PER_FONT_PAGE: 464, 466
CHAR_AI_MODE: 225, 227, 232, 237, 325, 368, 385, 390, 404, 531b, 532c
CHAR_CHARSET: 465, 466, 570b
CHAR_INDEX: 415, 417a, 417b, 418, 419, 420, 422, 423, 424
CHAR_LIST_HEAD: 53a, 612
CHAR_LOWER_LEFT: 465, 466
CHAR_LOWER_RIGHT: 465, 466
CHAR_PTR: 219, 222, 315, 373, 385, 398, 415, 417a, 417b, 419, 420, 422, 423, 424, 426, 430, 454, 469, 482, 488, 535, 539, 542, 561, 591a, 592, 612
CHAR_REC: 25a, 26, 61a, 478, 488, 494
CHAR_RECORD_HEADER: 53a
CHAR_RECORD_SIZE: 25b, 26, 219
CHAR_SELECT_HERE: 281, 283
CHAR_SELECT_HOME: 281, 283
CHAR_SELECT_INIT: 267a, 283
CHAR_SELECT_NAME: 281, 610a, 612, 613, 615, 616
CHAR_SELECT_NEXT: 281, 283
CHAR_SELECT_PREV: 281, 283
CHAR_SELECT_TABLE: 281, 283
CHAR_SPRITE: 225, 239, 486b, 487
CHAR_TABLE_BASE: 25b, 26
CHAR_UPPER_LEFT: 465, 466
CHAR_UPPER_RIGHT: 465, 466
CHECK_ADJACENT_THREATS: 227, 308, 320, 323
CHECK_AND_RESTRICT: 232, 241, 244
CHECK_ATTACK_ELIGIBLE: 265, 334
CHECK_AT_BOUNDARY: 469, 537
CHECK_CAN_LEAVE: 232, 456
CHECK_COMBAT_ELIGIBILITY: 264, 308

CHECK_ENCOUNTER: 226a, 227, 232, 244, 308, 312, 327, 357, 368, 898
CHECK_HOSTILE: 312, 317, 322, 334, 365, 385, 430
CHECK_INPUT: 58, 663
CHECK_LOCATION_MATCH: 419, 424, 429
CHECK_MOB_NEEDS_TURN: 225, 227
CHECK_SCENE_EVENTS: 219, 428
CHECK_THREATS_HERE: 232, 323, 357, 360, 368, 898
CKSUM_ACC: 671
CKSUM_END: 670, 671, 677
CKSUM_START: 670, 671, 677
CLAMP_CHAR_FIELD: 420, 494, 609
CLASSIFY_ENTITY_TYPE: 342, 459, 542, 561
CLEAR_AND_DISPLAY: 447, 452
CLEAR_INPUT_BUFFER: 249a, 258
CLEAR_MAP: 222, 535, 539, 556
CLEAR_PTR: 674a, 675
CLEAR_RESTRICTED: 227, 232, 244, 245, 327
CLEAR_STRENGTH_FLAGS: 61a
CLEAR_TREASURE_DIGITS: 447, 451b
COLD_RESTART: 305, 669, 675
COLOCATE_LINK: 314a, 315
COLOCATE_POS: 314b, 315
COLOCATE_WRAPPED: 314c, 315
COLROW_TO_POS: 232, 239, 323, 330, 357, 471, 494, 542, 600a, 898
COMBAT_STRENGTH: 324, 325, 342, 346
COMBAT_WILLINGNESS: 337a, 338, 372b
COMMIT_MOVE: 232, 239, 354, 357, 360, 444, 447, 898
COMPUTE_CHECKSUM: 671, 677
CONFIGURE_INPUT: 232, 261, 264
CONTEXT_SWAP: 221, 696, 700
CONTINUE_GAME: 274, 289
COPY_BE_TO_FA: 422, 423
COPY_DEST: 565, 566
COPY_PC_NAME_TO_BUF: 612, 613, 615, 616
COPY_RECORD_DATA: 250, 260
COPY_SRC: 565, 566
CURRENT_COL: 232, 239, 323, 329, 330, 354, 357, 360, 472b, 475, 898
CURRENT_PLAYER: 61a, 219, 283, 349, 354, 417b, 420, 423, 428, 431, 489a, 494, 562a, 564
CURRENT_ROOM: 222, 291, 447, 476, 478
CURRENT_ROW: 232, 239, 323, 329, 330, 354, 357, 360, 473a, 475, 898
CURRENT_SELECTION: 54a, 54b, 247, 255
DAMAGE_AMOUNT: 348, 349
DATA_PTR: 417a, 417b, 422, 423, 429, 481, 488, 537, 583b, 618a, 619a, 619c, 620
DECIMAL_OUT_PTR: 437, 447, 556, 630a, 632, 685, 686
DECODE_DISPATCH_RELAY: 702, 713b

DECODE_ENCRYPTED_ADDR: 704, 706, 707, 709a, 709b, 710a, 710b, 711a, 712a, 713a
DECRYPT_HIGH_BYTE: 696
DECR_GROUP_COUNT: 478, 481
DEC_EVENT_THRESHOLD: 686
DEC_THRESHOLD_WRAP: 271, 686
DEC_WAIT_IF_NONZERO: 52
DEFAULT_CHAR: 462, 530b, 531a
DEFENSE_HALF_MASK: 320, 325, 336, 338, 346
DELAY_COUNT: 603a, 603b
DELAY_LOOP: 779b, 780a
DELAY_WITH_ANIMATION: 338, 360, 385, 603b, 607b, 637
DIR_KEYS_CTRL: 58
DIR_KEYS_LOWER: 58
DIR_KEYS_UPPER: 58
DIR_SCORES: 389b, 390, 394, 396
DISK_BAD_NIBBLE: 773, 793
DISK_BOOT_ROM: 674b, 675, 800a, 801, 832, 853b
DISK_DATA_REG: 692, 696, 772, 773, 777
DISK_IO_END: 671, 777
DISK_IO_START: 671, 771
DISK_READ: 298, 301, 305, 546, 662
DISK_READ_FIELD: 529, 771
DISK_READ_JUMP: 662, 666, 669
DISK_RETRY_COUNT: 775, 777
DISK_RTS: 662, 666
DISK_SIGNATURE: 288, 298
DISK_VM_BYTECODE: 774, 775
DISK_VM_DATA_B4DE: 782
DISK_VM_DISPATCH: 774, 783, 784
DISK_VM_ERROR_ACCUM: 775, 778, 779a
DISK_VM_EXIT_ERR: 780b, 781a
DISK_VM_EXIT_OK: 775, 780c
DISK_VM_HALT: 783, 785
DISK_VM_HANDLER_TBL: 783
DISK_VM_JMP_IND: 788a, 788b
DISK_VM_LOAD_IND: 789c, 790a
DISK_VM_LOOP: 771, 774
DISK_VM_MISC_B49C: 781b
DISK_VM_OP_BNZ: 783, 789a
DISK_VM_OP_CALL: 783, 790b
DISK_VM_OP_CALL0: 783, 788b
DISK_VM_OP_CALL1: 783, 788a
DISK_VM_OP_GOTO: 783, 787, 792b
DISK_VM_OP_INC_LOAD: 783, 792a
DISK_VM_OP_LDA_ABS: 783, 789c

DISK_VM_OP_LDA_IDX: 783, 790a
DISK_VM_OP_LDA_IMM: 783, 789b
DISK_VM_OP_RET: 783, 791a
DISK_VM_OP_STA_ABS: 783, 791b
DISK_VM_OP_SUB_IMM: 783, 791c
DISK_VM_PULL_IP: 783, 784
DISK_VM_READ_ADDR: 785, 787, 788a, 788b, 789a, 789c, 790a, 790b, 791b, 792a
DISK_VM_RESUME: 783, 787, 788a, 789a, 789b, 789c, 791a, 791b, 791c, 792a
DISK_VM_SAVED_STATE: 793
DISK_VM_SCRATCH: 793
DISK_VM_SET_IP: 787, 789a, 790b
DISK_VM_SUB_EXIT_ERR: 775, 780b
DISK_VM_SUB_PARK_STEPPER: 775, 779b
DISK_VM_SWAP: 774, 780c, 781a, 781c
DISK_VM_TRAMPOLINE: 783, 792b
DISK_WRITE: 298, 303, 546
DISPATCH_ON_MODE: 342, 346, 371, 607b
DISPATCH_VEC: 564, 569b, 570b, 584, 682
DISPLAY_CHAR_STATS: 265, 556
DISPLAY_INSERT_MSG: 295
DISPLAY_MESSAGE: 56, 224, 283, 289, 295, 297, 305, 342, 346, 349, 354, 357, 360, 363, 377, 378, 385, 433, 436, 440, 454, 456, 527, 542, 590, 637, 640, 898
DISPLAY_MESSAGE_LOOP: 452, 637
DISPLAY_SHOP: 232, 310, 404
DISPLAY_STATUS_BAR: 527, 550, 644
DIST_COL: 399, 400
DOS33_RWTS: 770
DRAW_BLINK_ALT: 472a, 545, 604b
DRAW_BLINK_CHAR: 472a, 473b
DRAW_BLINK_NORMAL: 473b, 545, 605b
DRAW_BORDER_CHAR: 574, 576b
DRAW_BORDER_CHAR_AT: 584
DRAW_BORDER_ICONS: 574, 580
DRAW_BORDER_ICON_ENTRY: 584
DRAW_BORDER_ICON_LIST: 584
DRAW_BORDER_LABELS: 574, 578, 584
DRAW_CHAR_AT_POS: 239, 342, 349, 371, 471, 494
DRAW_INPUT_OPTIONS: 249a, 250
DRAW_MAP_COL: 535, 540
DRAW_MAP_ENTITIES: 535, 542
DRAW_MAP_ICON_B: 542, 545
DROP_AMOUNT: 434b, 436, 437, 438, 440, 442
DROP_DISPLAY_BUF: 434a, 437
DROP_GOLD_EXEC: 273, 440
DROP_GOLD_TABLE: 273
DROP_STEP: 435, 436, 438, 440, 442

EALDR_ABORT: 856, 858, 859
EALDR_BOT: 839, 854a, 854b
EALDR_CHECKSUM_LISTING: 870
EALDR_CHECK_FLAG: 856, 858
EALDR_CLEAR_TEXT: 837, 853a
EALDR_COPY_BYTE: 833, 834a
EALDR_COPY_SRC: 834a
EALDR_DEST: 839, 848b, 849
EALDR_DISK_IO: 856, 859
EALDR_DISPATCH: 846, 848a
EALDR_DISPATCH_TABLE: 845b, 846
EALDR_ENTRY: 833
EALDR_FIND_SECTOR: 885c
EALDR_FUNNY_INC: 839, 854b, 890c
EALDR_MOVE_ARM_TRAMPOLINE: 839, 856, 872
EALDR_OP_BEQ: 849
EALDR_OP_CALL0: 849
EALDR_OP_CALL1: 849
EALDR_OP_LDI: 849
EALDR_PADDING: 861
EALDR_PROT_ENTRY: 859
EALDR_PROT_VM: 822
EALDR_PSUEDOACC: 845a, 846, 849, 855
EALDR_PTR: 845a, 846, 847a, 847b, 848a, 849
EALDR_READ_ARG16: 847a, 848a, 849
EALDR_READ_DATA_TRAMPOLINE: 856, 872
EALDR_READ_TRACK_TRAMPOLINE: 842, 872
EALDR_REBOOT: 837, 853b
EALDR_RESIDENT_COPY: 831
EALDR_RESIDUAL_DOCS: 860
EALDR_RWTS_ADDR_ERROR: 879, 880
EALDR_RWTS_DELAY: 877, 878b
EALDR_RWTS_DELAY_INNER: 872, 878b
EALDR_RWTS_DEST_PAGES: 883, 884
EALDR_RWTS_ERROR_FLAG: 873a, 883
EALDR_RWTS_INTERLEAVE: 882b, 883
EALDR_RWTS_MOVE_ARM: 872, 877, 883
EALDR_RWTS_NIBBLE_A: 882a
EALDR_RWTS_NIBBLE_B: 873b, 874
EALDR_RWTS_PADDING: 881
EALDR_RWTS_PHASE_ON: 877, 878a
EALDR_RWTS_PHASE_ON_RTS: 877, 878a
EALDR_RWTS_READ_ERROR: 873a, 874
EALDR_RWTS_READ_SECTOR: 874, 883
EALDR_RWTS_READ_TRACK: 872, 883, 884
EALDR_RWTS_SEARCH_ADDR: 872, 880, 883

EALDR_RWTS_TAIL_PADDING: 885a
EALDR_RWTS_TRACK: 873a, 877, 878a
EALDR_SEARCH_ADDR_TRAMPOLINE: 856, 872
EALDR_SOURCE_COMMENTS: 865
EALDR_SPLASH_BITMAP: 862
EALDR_TMP0: 841, 842
EALDR_TMP1: 839, 842
EALDR_TOP: 839, 854a, 854b
EALDR_TRACKS: 842, 885b
EALDR_TRACK_PAGES: 842, 885b
EALDR_VM_BYTECODE: 837
EALDR_VM_DISPATCH: 848b, 849, 855
EALDR_VM_EXEC: 833, 834b
EALDR_VM_GOTO: 848a, 849
EALDR_VM_INIT: 846, 847b
EALDR_VM_INTERP: 834b, 846, 859
EALDR_VM_LOOP: 846, 848a
EFFECT_REC: 349, 352, 372a, 380, 382, 483a, 483b
ENCOUNTER_RESULT: 226a, 227, 232, 244, 308, 312, 327, 357, 368, 898
END_TURN: 232, 241, 357, 898
ENSURE_GAME_DISK: 289, 293, 295, 297
ENTER_ATTACK_MENU: 265, 266
ENTER_CHAR_SELECT: 283, 617
ENTER_COMBAT_STATE: 232, 237, 266, 327
ENTER_LOCATION: 219, 429, 527, 535
ENTER_SAVE_GAME_SLOT: 269, 279
ENTER_SCENARIO_DISK: 280b, 293, 295
ENTITY_PTR: 56, 222, 224, 225, 227, 232, 239, 241, 242, 244, 245, 246, 308, 320, 323, 325, 327, 328, 334, 338, 342, 346, 352, 354, 357, 360, 365, 368, 377, 378, 382, 385, 390, 394, 398, 400, 409, 412, 433, 436, 438, 440, 444, 447, 450, 484, 486a, 487, 489a, 535, 556, 561, 564, 609, 618d, 619c, 620, 898
ENTITY_REC: 232, 312, 322, 342, 394, 404, 459, 462, 468, 469, 471, 475, 541b, 542
ENTITY_SEARCH_POS: 469, 536
EV1SCN: 21, 22, 32
EVALUATE_DIRECTION: 390, 394
EVDATA: 19, 22, 32
EVENCO: 20a, 22, 32
EVENT_ACCUM_MAX: 397a, 454
EVENT_ACTIVATED: 232, 397a, 398, 454, 542
EVENT_DEST_POS: 353c, 354
EVENT_EMPTY: 232, 342, 371, 397a, 398, 454, 459, 462, 542
EVENT_END: 397a, 398, 454, 469
EVENT_FACTION_SCENE: 415, 417b, 419
EVENT_FACTION_SCENE_CONT: 415, 419
EVENT_PICK_CHAR: 415, 417a, 417b

EVENT_PICK_MOB: 415, 417a
EVENT_POS: 354, 385, 532b, 535
EVENT_PRNG_CHECK: 342, 360
EVENT_PTR: 353a, 354, 360, 398, 424, 444, 447, 454, 898
EVENT_RECORD_SIZE: 397a, 398, 454, 469
EVENT_SAVED_POS: 453, 454
EVENT_SLOT_BYTE: 359a, 360, 898
EVENT_SPAWN_MOB: 415, 419, 420
EVENT_SUB_VALUE: 359b, 360, 898
EVENT_THRESHOLD: 207, 415, 685, 686
EVENT_VALUE_HI_MASK: 397a, 454
EVFIELD_POSITION: 354, 397a, 398, 454
EVFIELD_TYPE: 232, 354, 397a, 398, 454, 542
EVFIELD_VALUE: 397a, 454
EVGOLD: 20b, 22, 32
EVISCN: 21, 22, 32
EVPSCN: 21, 22, 32
EVRAND: 20a, 22, 32
EVSHOP: 22, 32
EVSLOT: 22, 32
EVSTOP: 19, 22, 32
EXTRACT_PENDING_XP: 412
FACTION_MASK: 415, 418, 419, 420, 426
FILL_MAP: 535, 538, 564
FIND_ATTACK_TARGET: 265, 329, 330
FIND_ENTITY_AT_POS: 232, 312, 322, 330, 394, 469, 471, 540, 541b, 542
FIND_FIRST_PC: 283, 612, 615, 616, 617
FIND_MOB_AT_SAME_POS: 312, 315, 322, 330, 334
FIND_MOB_BY_FACTION: 418, 419, 420, 426
FIND_MSB: 436, 442
FIND_NEAREST_EVENT: 385, 398, 532b, 535
FIND_NEXT_PC: 614, 615, 616
FIND_NEXT_PC_WRAP: 283, 614
FIND_PREV_PC: 283, 616
FIRST_GROUP_MEMBER: 357, 368, 373, 385, 482, 542, 898
FLAGS_ENGAGED_BIT: 245, 246, 322, 336, 338, 349
FLAGS_FACTION_MASK: 425, 426
FLAGS_STATE_MASK: 242, 336, 338
FLUSH_PENDING_XP: 433
FONT_CHARNUM: 49, 239, 354, 462, 466, 473b, 475, 487, 538, 542, 545, 574, 575, 576b, 584, 644
FONT_CHARSET: 49, 239, 462, 466, 473b, 487, 538, 542, 574, 576a, 576b, 644
FONT_COL: 239, 342, 400, 466, 467, 471, 473b, 487, 492a, 494, 535, 538, 540, 542, 545, 644
FONT_DATA: 715a, 736
FONT_POS_TO_TEXT_POS: 466, 467, 538

FONT_ROW: 239, 342, 400, 466, 467, 471, 473b, 487, 534c, 535, 538, 540, 542, 545, 644
FORMAT_DISK_TRACK: 295, 298
FORMAT_SCENARIO_DISK: 280a, 295
FORMAT_SLOT_NUMBER: 275c, 277, 278
GAME_ACTION_HANDLER: 570b, 584
GAME_DISK_SLOTS: 287, 289, 293, 305
GAME_INIT: 219, 564
GAME_INPUT_TABLE: 232, 261, 262, 263, 264
GAME_MESSAGE_STRINGS: 640
GAME_SETTINGS_TABLE: 269, 270a
GAME_SIG_SECTOR: 300, 301, 666
GAME_SIG_TRACK: 300, 301, 666
GAME_TURN_LOOP: 222, 225, 232
GET_CHAR_RECORD: 25a, 26, 61a, 422, 423, 429, 478, 488, 494
GET_ENTITY_FONTCHAR: 342, 462, 471, 475, 540, 542
GET_ENTITY_THREAT: 365, 368
GET_HOSTILITY: 316b, 316c, 317, 318
GET_INPUT_TABLE_PTR: 232, 262, 263
GET_MOB_DATA: 220, 222, 291, 315, 426, 430, 469, 478, 482, 602, 612, 615, 616
GLYPH_BASE: 464, 466
GROUP_COUNT_DELTA: 480, 481
HALVE_STEP: 436, 438, 442
HANDLER_PTR: 458, 459
HANDLE_ENCOUNTER: 232, 354
HANDLE_EVENT_SLOT: 232, 360
HANDLE_MOB_ENCOUNTER: 232, 357
HEAL_NOTIFIED: 376, 377, 378
HIGH_NIBBLE_MASK: 601, 602
HIRES: 571b, 837
HIRES_PAGE: 567
HOSTILE_A: 316b, 317
HOSTILE_APP: 316a, 317
HOSTILE_Y: 316c, 317
HP_FLAGS_MASK: 561, 609
HRCG_ADVANCE_CURSOR: 761, 763
HRCG_CALC_HIRES_ADDR: 761, 768
HRCG_CALC_HIRES_ROW: 764, 766, 768
HRCG_CHARSET1_SEL: 464, 466
HRCG_CHARSET_SELECT: 743, 745
HRCG_CLREOL: 738, 751, 766
HRCG_CLRLINE: 738, 751, 766
HRCG_COUT: 736, 743
HRCG_CR_ENTRY: 753, 763
HRCG_CTRL_A: 741, 745
HRCG_CTRL_B_SAVE: 741, 747a

HRCG_CTRL_C: 741, 748
HRCG_CTRL_CONT: 66b
HRCG_CTRL_D: 741, 749
HRCG_CTRL_E: 741, 751
HRCG_CTRL_F: 741, 751
HRCG_CTRL_H: 741, 751
HRCG_CTRL_I: 741, 751
HRCG_CTRL_K: 741, 753
HRCG_CTRL_L: 741, 753
HRCG_CTRL_M: 741, 753
HRCG_CTRL_N: 741, 754
HRCG_CTRL_NOP: 741, 748, 749, 751, 753, 754
HRCG_CTRL_O: 741, 754
HRCG_CTRL_P: 741, 754
HRCG_CTRL_PREFIX: 64
HRCG_CTRL_Q: 741, 754
HRCG_CTRL_R: 741, 756
HRCG_CTRL_S: 741, 756
HRCG_CTRL_T: 741, 756
HRCG_CTRL_TAIL: 66a
HRCG_CTRL_V: 741, 757
HRCG_CTRL_W: 741, 757
HRCG_CTRL_Y: 741, 759
HRCG_CTRL_Z: 741, 759
HRCG_DISPATCH_TBL: 741, 743
HRCG_DISPLAY_PAGE: 736, 749
HRCG_DRAW_CHAR: 743, 761
HRCG_DRAW_MODE: 747b, 748, 754, 756, 761, 764, 766
HRCG_ENTRY2: 736, 759
HRCG_ENTRY3: 736, 759
HRCG_FILL_LINE: 764, 766
HRCG_FONT_BASE: 736, 745
HRCG_FONT_PTR_HI: 744, 745, 761
HRCG_FONT_PTR_LO: 744, 745, 761
HRCG_GETKEY: 736, 738
HRCG_HOME: 738, 754, 766
HRCG_INIT: 568, 736
HRCG_INIT_BODY: 736
HRCG_INVERT_FLAG: 752, 753, 756, 761
HRCG_LIBRARY_TAIL: 894
HRCG_LINE_FEED: 741, 748, 763
HRCG_MASK: 750, 751, 754, 761, 766
HRCG_MODE_FLAGS: 742, 743, 745, 754, 761
HRCG_NUM_CHARSETS: 736
HRCG_ORIGINAL_ENTRY: 61c
HRCG_PAGE_MASK: 747b, 748, 754, 756, 761, 766

HRCG_PRINT_VERSION: 61c, 736
HRCG_PROCESS_CHAR: 743, 745
HRCG_RESET: 736, 759
HRCG_RESET_WINDOW: 759
HRCG_ROW_TABLE: 768
HRCG_RTS: 736, 756, 757, 759, 761
HRCG_SAVED_CH: 746, 747a, 748, 749, 754, 759
HRCG_SAVED_CV: 746, 747a, 749, 754, 757, 759
HRCG_SAVED_PARAMS: 746, 747a, 749, 757, 759
HRCG_SCROLL_UP: 738, 763, 764
HRCG_SET_REPLACE: 745, 747a, 759
HRCG_SET_STD_FONT: 745, 759
HRCG_SHIFT_FLAG: 737a, 738
HRCG_TRAILING_DATA: 768
HRCG_VERSION_STR: 715c, 736
HRCG_VIDEO_MODE: 737b, 738, 747a, 749, 759, 768
HRCG_WRAP_FLAG: 755, 756, 757, 763
ICON_TABLE_PTR: 583b, 584
IDLE_ANIM_HANDLER: 52, 663
INCREMENT_HP: 377, 378
INCR_GROUP_COUNT: 478, 481
INCW: 10b, 849
INC_EVENT_THRESHOLD: 686
INC_LEVEL: 447, 450
INC_MOB_COUNTERS: 394, 396
INC_STRENGTH: 447, 450
INC_THRESHOLD_WRAP: 271, 686
INIT_ATTACK_DISPLAY: 272, 332
INIT_CHAR_SELECT: 281, 617
INIT_DISK_IOB: 564, 589b
INIT_DROP_DISPLAY: 273, 437
INIT_DROP_GOLD: 436, 438
INIT_GAME_DISPLAY: 570b, 574
INIT_GAME_STATE: 564, 570b
INIT_HRCG: 564, 568
INIT_MOB_COUNT: 564, 592
INIT_STUB: 564, 569a
INIT_STUB2: 564, 588b
INPUT_BUFFER: 250, 258, 259, 260, 611, 616
INPUT_DIR: 231b, 232
INPUT_JMP_VEC: 53b, 247, 257
INPUT_LOOP: 247, 262, 266, 267b, 270a, 272, 273, 279, 280b, 283, 286, 588a
INPUT_MODE: 52, 58, 550, 556, 570b, 589c, 590, 683
INPUT_OPTION_COUNT: 249b, 250
INPUT_SCAN_OFFSET: 255, 256, 570b
INTEGRITY_DATA: 671

INTEGRITY_FAIL_ADDR: 529, 671
IS_PLAYER_TURN: 325, 330, 338, 370a, 474, 475
ITEM_STRINGS_COPY: 65
IWM_MOTOROFF: 696, 772, 775
IWM_MOTORON: 696, 772, 775, 800a, 801
IWM_PH0_OFF: 696, 772, 779b
IWM_PH0_ON: 696, 772, 775
IWM_PH1_OFF: 696, 772, 779b
IWM_PH2_OFF: 696, 772, 779b
IWM_PH2_ON: 696, 772, 775
IWM_PH3_OFF: 696, 772, 779b
IWM_PH3_ON: 696, 772, 775
IWM_READMODE: 696, 772, 775
JOY_AXIS_X: 58
JOY_AXIS_Y: 58
JOY_DELAY: 58
JOY_SETTLE_DELAY: 57, 58
JOY_THRESH_HI: 58
JOY_THRESH_LO: 58
JOY_TO_ZONE: 58
KBD_DATA: 571b
KBD_STROBE: 571b
LDWIY: 17, 247, 253, 257, 260, 349, 398, 424, 447, 454, 483b, 484, 485, 486a, 539
LDY_OPCODE: 685, 686
LEVEL_BIT5_MASK: 336, 338
LEVEL_CLEAR_BIT4: 336, 338
LEVEL_FLAG_MASK: 411a, 412
LEVEL_MASK: 338, 411a, 412, 447, 450, 478, 556
LEVEL_THRESHOLD: 379a, 380
LFIELD_MAX_POS: 491, 494, 535
LFIELD_MIN_POS: 491, 494, 535
LOAD_CHAR_NAME_PTR: 404, 486a, 532c
LOAD_CHAR_TO_FA: 431, 488
LOAD_END_OFFSET: 677
LOAD_MAP_TILES: 531a, 535
LOAD_SCENE_DATA: 527, 529
LOAD_SCENE_DISK_ENTRY: 662
LOAD_SCENE_FROM_DISK: 527, 584, 666
LOAD_TARGET_PTR: 328, 370a
LOCATION_DATA: 207, 219
LOCATION_EVENTS: 32
LOCATION_ID: 536, 537
LOCATION_NAMES: 27
LOCATION_POS: 220, 429, 527, 535, 562b, 564
LOCATION_STYLE: 404, 527, 531a, 535, 541a, 542, 563a, 564

LOCATION_STYLE_BITS: 403c, 404, 409, 412
LOW_NIBBLE_MASK: 227, 232, 542, 553, 556, 602
MAIN_ENTRY: 219, 428, 675
MANHATTAN_DISTANCE: 365, 385, 390, 394, 398, 399, 400
MAP_FILL_CHAR: 530b, 531a, 538
MAP_OBJECT_NAMES: 63
MAP_REDRAW_FLAG: 541c, 542
MAP_VALUE_TO_STRING: 342, 346, 375
MAP_WIDTH: 222, 320, 388a, 390, 473b, 527, 540, 545, 552, 599, 600a, 644
MAX_GAME_SPEED: 270b, 271
MAX_SLOT_NUMBER: 276, 277
MAX_THRESHOLD: 685, 686
MENU_ATTACK_INIT: 272
MENU_ATTACK_TARGET: 272, 330, 334
MENU_CHECK_ATTACK: 265, 266
MENU_DESC_PTR: 53c, 54b
MENU_DROP_DEC: 273
MENU_DROP_GOLD: 273, 436, 438
MENU_DROP_GOLD_INIT: 273
MENU_DROP_INC: 273
MENU_GAME_SETTINGS: 270a, 686
MENU_OPTION_PTR: 53c
MENU_OPTION_STRINGS: 310
MENU_OTHER_OPTIONS: 267b, 436
MENU_SEE_YOUR_LOT: 265
MENU_SETTINGS_INIT: 269, 270a
MFIELD_APPEARANCE: 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 225, 317, 334, 346, 349, 354, 373, 377, 385, 447, 462, 475, 542, 561, 612, 615
MFIELD_ATTACK: 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 325, 409, 556, 561
MFIELD_DEFENSE: 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119,

120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 325, 409, 556, 561

MFIELD_GOLD: 56, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 349, 380, 447, 494, 556, 561

MFIELD_HP: 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 222, 232, 312, 338, 346, 349, 377, 378, 385, 556, 561, 609

MFIELD_LEVEL: 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 336, 338, 411a, 412, 447, 450, 478, 556, 561

MFIELD_NAME: 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 561, 613

MFIELD_NEXT: 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152,

153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 222, 315, 334, 342, 394, 426, 447, 459, 462, 469, 478, 482, 561, 612, 615
MFIELD_POSITION: 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 227, 232, 239, 244, 315, 325, 338, 349, 354, 357, 365, 368, 385, 390, 394, 400, 469, 475, 487, 494, 542, 561, 898
MFIELD_STRENGTH: 61a, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 320, 322, 338, 354, 357, 368, 380, 385, 409, 450, 556, 561, 609, 898
MFIELD_XP: 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 219, 346, 411a, 412, 481, 556, 561
MIXCLR: 571b, 837
MOB_ATTACK: 338, 346
MOB_BASE_HI: 53a, 602
MOB_BASE_LO: 53a, 291, 305, 419, 420, 602
MOB_DATA_PTR: 220, 222, 315, 426, 430, 462, 469, 478, 482, 600b, 602, 609, 612, 613, 615, 616, 617
MOB_DATA_TABLE: 53a, 68, 196
MOB_DEFENSE: 345, 346
MOB_PTR: 220, 312, 315, 317, 322, 325, 330, 332, 334, 338, 342, 346, 357, 364, 365, 368, 370a, 371, 372a, 373, 385, 415, 417a, 419, 420, 422, 426, 430, 432, 447, 456, 475, 482, 485, 494, 542, 561, 608, 609, 612, 613, 614, 615, 616, 617, 619b, 619c, 620, 898
MOB_PTR_FLAG: 232, 468, 469
MODIFY_CHAR_STATS: 349, 380, 382

MODIFY_CURRENT_STATS: 56, 382, 440, 447
MOVB: 8c, 225, 239, 264, 312, 322, 349, 354, 357, 360, 368, 370a, 385, 398, 404, 419, 420, 424, 462, 472a, 473b, 475, 487, 489a, 535, 538, 542, 545, 644, 738, 747a, 749, 751, 754, 757, 763, 764, 766, 809, 877, 898
MOVE_LOOP: 232, 273, 278, 281, 283, 293, 295, 357, 360, 433, 440, 617, 644, 898
MOVE_POINTS: 230c, 232, 261, 264, 360
MOVW: 9a, 220, 222, 312, 315, 320, 342, 349, 352, 357, 372a, 382, 385, 420, 422, 426, 430, 432, 444, 447, 475, 478, 482, 488, 489a, 494, 619c, 620, 637, 679, 681, 704, 787, 848a, 898
MSG_ARMOR_WORTHLESS: 454, 634
MSG_BREAKS_FREE: 295, 527, 634
MSG_DEATH: 360, 634
MSG_DOOR_OPENS: 354, 634
MSG_DUST_CLOUD: 360, 634
MSG_ENOUGH: 346, 634
MSG_ESCAPE_FAIL: 456, 634
MSG_FEELING_WEAK: 349, 634
MSG_FLYING_TACKLE: 357, 634, 898
MSG_GOLD_DROPPED: 440, 634
MSG_GOLD_LOST: 440, 634
MSG_HEAVY_ROCK: 342, 360, 634
MSG_HIT_STATUE: 363, 634
MSG_HIT_WALL: 349, 634
MSG_INPUT_PROMPT: 590, 634
MSG_INSERT_DISK: 342, 354, 634
MSG_INSERT_GAME: 297, 634
MSG_INSERT_SCENARIO: 289, 295, 634
MSG_JOINS_PARTY: 385, 634
MSG_JOYSTICK_MODE: 590, 634
MSG_LINE_COUNT: 452, 636, 637
MSG_LOTS_OF_FIGHT: 349, 634
MSG_NO_GOLD: 436, 634
MSG_NO_XP: 433, 634
MSG_PRESS_KEY: 297, 634
MSG_READ_ERROR: 305, 634
MSG_REGAIN_CONSC: 377, 634
MSG_REST_STRONGER: 224, 634
MSG_ROCK_FALLS: 542, 634
MSG_SAVED_ME: 283, 634
MSG_SHUFFLES_OFF: 634
MSG_TABLE_PTR: 53a, 637
MSG_TACKLE_FAIL: 456, 634
MSG_THANK_YOU: 385, 634
MSG_TOO_POOR: 634
MSG_TOPPLES: 342, 360, 634
MSG_TOUCHED: 342, 634

MSG_TOUGH_HIDE: 289, 295, 634
MSG_UNCONSCIOUS: 357, 634, 898
MSG_XP_GAINED: 433, 634
NAME_BUF_LEN: 601, 613
NEAREST_DIST: 397c, 398
NEAREST_POS: 397b, 398
NEXT_COLOCATED_MOB: 330, 334
NEXT_GROUP_MEMBER: 357, 368, 373, 385, 482, 542, 898
NPC_MOVE_AI: 232, 385, 390
NUM_DIGIT: 630b, 632
NUM_DIVISOR: 630c, 632
NUM_LEADING: 631, 632
NUM_TO_DECIMAL: 404, 437, 447, 554, 556, 632, 686
NUM_TO_DECIMAL_CONT: 632
NUM_VALUE: 56, 380, 382, 437, 447, 452, 454, 556, 630a, 632, 686
ORPHANED_ENCOUNTER_HANDLER: 898
OTHER_OPTIONS_TABLE: 267a, 267b
PADDL0: 571b
PAGE1: 571b
PENDING_XP_UNIT: 411a, 412
PICKUP_TREASURE: 232, 447
PICK_RANDOM_CHAR: 415, 417b, 420, 422, 423
PICK_RANDOM_MEMBER: 373, 385
PICK_RANDOM_MOB: 417a, 419, 422
PLAYER_ATTACK: 338, 342
PLAY_HIGH_TONE: 607b, 647a, 648, 679, 681, 686
PLAY_LOW_TONE: 647b, 648, 679, 681, 686
PLOT_CHAR: 49, 580
POS_TO_COLROW: 232, 342, 354, 384d, 385, 400, 471, 475, 487, 494, 535, 537, 542, 599
PRINT_BOTTOM_CENTERED: 338, 342, 375, 483b, 484, 485, 542, 628, 637
PRINT_CENTERED_PLUS_COL: 552, 628, 629
PRINT_CTRL_AB: 253, 259, 550, 552, 556, 574, 623a, 625, 627, 629, 637
PRINT_CTRL_N: 253, 473b, 623b
PRINT_ENTITY_NAME: 349, 483b
PRINT_FONTCHAR_AT_TEXT_POS: 49, 466, 576b
PRINT_MEMBER_NAME: 338, 346, 385, 485, 542
PRINT_MOB_NAME: 224, 338, 342, 346, 357, 360, 363, 377, 378, 456, 484, 898
PRINT_STRING: 253, 466, 538, 550, 556, 578, 621, 622, 629
PRINT_STRING_ADDR: 253, 260, 375, 404, 466, 473b, 483b, 484, 485, 486a, 539, 550, 556, 576c, 578, 621, 622, 628, 629, 637
PRINT_STRING_AT_ADDR: 556, 574, 621
PRNG_OUTPUT: 222, 354, 357, 360, 377, 415, 456, 478, 527, 529, 593a, 595, 597, 663, 682, 898
PRNG_SCENE_SKIP_MASK: 526a, 527
PRNG_STATE: 593b, 595

PRNG_TEMP: 594, 595
PROCESS_CHAR_EVENTS: 417a, 417b, 424
PROCESS_ENTITY_SCRIPT: 342, 371
PROCESS_MOB_GROUPS: 219, 222
PROCESS_SCENE_GROUPS: 428, 431
PROMPT_INSERT_DISK: 289, 295, 297
PSHW: 9b, 232, 255, 473b, 644, 790b, 849
PULB: 9c, 472a, 702, 708, 764, 783, 784, 846, 847b
PULL_TWO_RET_ADDRS: 702, 708
PULW: 10a, 232, 255, 473b, 644, 791a, 849
PWREDUP: 800a, 801, 832, 837
RANDOM_EVENTS: 219, 415, 419
RANDOM_IN_RANGE: 232, 291, 325, 338, 342, 349, 354, 360, 373, 377, 390, 422, 423, 440, 444, 447, 494, 584, 597
RANGE_CTR: 676, 677
RANGE_END: 676, 677
RDF_VERIFY_SEC1: 692, 696
RDF_VERIFY_SEC2: 692, 696
READ_AND_VERIFY: 775, 778, 779a, 822
READ_DATA_FIELD: 692, 696
READ_DATA_FIELD_AD: 692
READ_GAME_DATA: 289, 305
READ_INPUT_CHAR: 255, 256
READ_KEYBOARD: 232, 247, 663
RELOCATE_HIRES_STAGING: 564, 566
RENDER_FONT_CHAR: 239, 342, 465, 466, 471, 473b, 487, 538, 540, 542, 545, 644
RENDER_FONT_CHAR_DIRECT: 466, 584
RENDER_INPUT_BUFFER: 249a, 259
REORDER_CHAR: 349, 420, 432, 478, 489a, 494
REORDER_CURRENT_CHAR: 268, 354, 489a
REORDER_LINK: 477a, 478
REORDER_SORT_KEY: 477b, 478
RESET_TURN_POS: 239, 354, 360
RESET_VECTOR: 689
RESIDENT_BYTECODES: 221, 696, 702
RESIDENT_DISPATCH: 221, 700, 702
RESIDENT_DISPATCH_TABLE: 701, 702
RESIDUAL_5A_5A2D: 896a
RESIDUAL_5A_5A5B: 896b
RESIDUAL_5A_5A60: 897a
RESIDUAL_5A_5A95: 897b
RESIDUAL_5A_5AA5: 897c
RESIDUAL_5A_5AA8: 897d
RESIDUAL_DEBUG_TEXT: 895
RESIDUAL_DONT_BREAK: 892
RESIDUAL_E_CHECKER: 893a

RESOLVE_ATTACK: 272, 328, 330, 338
RESTORE_ZP_CONTEXT: 527, 620
RES_ABORT_NIBBLE: 691, 692
RES_ERROR_COUNT: 691, 696
RES_VM_CALL0: 701, 709a
RES_VM_CALL1: 701, 707
RES_VM_CALL1_IND: 707, 709a
RES_VM_DISPATCH: 701, 704, 713b
RES_VM_LOOP: 221, 689, 702
RES_VM_SET_IP: 704, 711a
RES_VM_VMADD: 701, 710b
RES_VM_VMBEQ: 701, 709b
RES_VM_VMCALL: 701, 711a
RES_VM_VMINC: 701, 713a
RES_VM_VMLD: 701, 710a, 710b
RES_VM_VMLDI: 701, 709c
RES_VM_VMRET: 701, 711b
RES_VM_VMST: 701, 712a
RES_VM_VMSUBI: 701, 712b
RES_VM_VMYIELD: 701, 706
RES_XOR_HI: 705, 706
RES_XOR_IMM: 705, 709c, 712b
RES_XOR_LO: 705, 706
RES_ZP_SHADOW: 691, 700
RETIRE_CHARACTER: 268
RNG_LIMIT: 596a, 597
RNG_MASK: 596b, 597
ROL2: 16c, 360, 380, 390, 404, 531a, 602
ROLW: 16a
ROM_COUT1: 259, 550, 552, 571a, 574, 578, 622, 623a, 623b, 624, 625, 626, 627, 639, 736, 802, 803
ROM_PREAD: 571a
ROM_SETRES: 571a
ROM_WAIT: 58, 606b, 607a, 696, 775, 822
RORW: 16b
RTS_OPCODE: 685, 686
RUNTIME_DATA_AREA: 50
RUNTIME_DATA_INIT: 50
RWTS_CORE: 770
RWTS_DISPATCH: 298, 529, 546
RWTS_ENTRY: 546, 770
RWTS_IOB: 589b, 770
RWTS_IOB_PTR: 301, 305, 589a, 589b, 666, 669
SAFE_TO_REST: 261, 264, 308, 570b
SAVED_CHAR: 370b, 371
SAVED_DATA_PTR: 619a, 619c, 620

SAVED_ENTITY_PTR: 618d, 619c, 620
SAVED_MOB_LINK: 333, 334
SAVED_MOB_PTR: 619b, 619c, 620
SAVED_TARGET_REC: 618c, 619c, 620
SAVE_GAME: 278, 293
SAVE_GAME_SLOT_TABLE: 278, 279
SAVE_LOAD_INIT: 274, 275c
SAVE_LOAD_TABLE: 274
SAVE_SECTOR_COUNT: 302, 303, 305
SAVE_TRACK_BASE: 304, 305
SAVE_ZP_CONTEXT: 527, 619c
SCAN_AND_REORDER: 431, 432
SCAN_FOR_HOSTILE: 428, 430, 432
SCENARIO_DISK_TABLE: 280a, 280b
SCENARIO_INIT_FLAG: 293
SCENARIO_INSERT_PROMPT: 295
SCENE_COMBAT_FLAG: 207, 346, 428
SCENE_EVENT_RESULT: 207, 220, 428, 617
SCENE_FOUND_FLAG: 427a, 428, 430, 432
SCENE_GROUP_IDX: 427c, 428, 430, 431, 432
SCENE_HOSTILE_DONE: 207, 428
SCENE_HOSTILE_FLAG: 427b, 428, 430
SCENE_LOADER_END: 428, 671
SCENE_LOADER_START: 529, 671
SCENE_LOOP: 220, 268, 293, 325, 342, 354, 360, 404, 428, 444, 447, 527
SCENE_NUMBER: 526b, 527, 666
SCENE_P1_DONE: 207, 428
SCENE_P3_DONE: 207, 428
SCENE_RETIRE: 268, 685
SCENE_SETUP: 220, 419, 494, 617
SCENE_TEXT_DATA: 298, 301, 526a, 527, 584, 666
SCENE_TRANSITION: 526a, 527
SCROLL_STATUS_LINE: 224, 338, 342, 346, 357, 360, 363, 377, 378, 385, 428, 456, 483b, 545, 556, 637, 639, 898
SEARCH_ADDR_FIELD: 775, 777, 779a
SELECT_COMBAT_TARGET: 237, 368, 370a
SELECT_INPUT_MODE: 564, 590
SELECT_SOMEONE_ELSE: 272, 334
SETTINGS_DANGER_DOWN: 271
SETTINGS_DANGER_UP: 271
SETTINGS_DISPLAY_INIT: 270a, 686
SETTINGS_SPEED_DOWN: 271
SETTINGS_SPEED_UP: 271
SETUP_AND_READ_DISK: 662, 669
SETUP_CHAR_SPRITE: 404, 487, 532c
SETUP_CURSOR: 247, 252, 253

SETUP_CURSOR_EXT: 247, 252
SETUP_DATA_IOB: 298, 301
SETUP_INPUT: 232, 247, 249a
SETUP_SCENE_DISPLAY: 225, 532c
SETUP_SLOT_IOB: 303, 305
SETUP_TEXT_POS: 539, 552
SETUP_TEXT_POS_COL20: 486a, 552
SET_BLINK_TARGET: 338, 475
SET_CHAR_STATE: 232, 242, 243, 327, 328, 354, 360, 363, 444, 447
SET_CURSOR_COMPRESSED: 893b
SET_CURSOR_POS: 253, 254
SET_CURSOR_ROW21: 56, 220, 225, 283, 289, 295, 297, 305, 325, 338, 342, 346, 349, 354, 357, 360, 363, 377, 378, 385, 404, 428, 433, 436, 440, 447, 454, 456, 483b, 542, 590, 627, 898
SET_INVERSE_VIDEO: 252, 624
SET_RESTRICTED: 232, 244, 246, 327, 360
SET_STATE_FIGHT: 232, 243
SET_TARGET_RESTRICTED: 357, 364
SET_TEXT_WINDOW: 627
SET_TEXT_WINDOW_BOTTOM: 527, 535, 564, 627
SET_TEXT_WINDOW_SCROLL: 625, 639
SET_TEXT_WINDOW_WRAP: 564, 625, 639
SHOP_AVAIL_FIELD: 404, 407a
SHOP_BUY_EFFECT_0: 402, 409
SHOP_BUY_EFFECT_1: 402, 409
SHOP_BUY_EFFECT_2: 402, 409
SHOP_BUY_EFFECT_3: 402, 409
SHOP_BUY_EFFECT_4: 402, 409
SHOP_BUY_HANDLER: 56, 285
SHOP_DATA_PTR: 56, 401, 404, 407b, 408
SHOP_EFFECT_TABLE: 62, 409
SHOP_ITEM_MASK: 403a, 404
SHOP_ITEM_NAME: 404, 407a
SHOP_ITEM_OFFSET: 55, 56, 404
SHOP_PRICE: 54c, 56, 404, 408
SHOP_PRICE_DIGITS: 404, 407a
SHOP_RECORD_SIZE: 407a, 407b
SHOP_SCENE_OFFSET: 404, 407a
SHOP_TABLE: 402, 404
SHOP_TYPE_FIELD: 404, 407a
SHOP_VALID_MASK: 403b, 404
SHOW_IDLE_STATUS: 222, 224
SHOW_LOCKED_MSG: 232, 360, 363
SHUFFLE_COUNT: 290, 291
SHUFFLE_ENTITIES: 291
SLOT_DEC: 277, 279

SLOT_DEC_ENTER: 274, 277
SLOT_DEC_REENTER: 278, 279
SLOT_INC: 277, 279
SLOT_INC_ENTER: 274, 277
SLOT_INC_REENTER: 278, 279
SLOT_NUMBER: 275b, 277, 289, 293, 305
SLOT_UPDATE: 277
SOFTEV: 800a, 801, 832, 837
SOUND_DURATION: 646, 647a, 648, 682
SOUND_FREQ: 646, 647a, 648, 682
SOURCE_CHAR: 349, 419, 420, 431, 478, 489a, 492b, 494
SPEAKER: 571b
START_COMBAT: 232, 237, 328
START_GAME_INPUT: 286, 404
START_MOB_TURN: 222, 225
STATS_ARMOR: 554
STATS_DAGGR: 554, 555a, 556
STATS_DAGGR_OUT: 555a, 556
STATS_DEXT: 554, 555a, 556
STATS_DEXT_OUT: 555a, 556
STATS_DUCATS: 554, 555a, 556
STATS_DUCATS_OUT: 555a, 556
STATS_LIFE: 554, 555a, 556
STATS_LIFE_OUT: 555a, 556
STATS_PROMPT_JOYSTICK: 555b, 556
STATS_PROMPT_KEYBOARD: 555b, 556
STATS_SPEED: 554, 555a, 556
STATS_SPEED_OUT: 555a, 556
STATS_STR: 554, 555a, 556
STATS_STR_OUT: 555a, 556
STATS_SWORD: 554, 555a, 556
STATS_SWORD_OUT: 555a, 556
STATS_TEMPLATE: 554, 556
STATS_WEAP: 554, 555a
STATS_WEAP_OUT: 555a, 556
STATUS_BORDER_DATA: 490, 550, 555b
STAT_LEVEL_BITS: 379c, 380
STAT_SCALE: 379b, 380
STD_FONT_DATA: 715b, 745
STEPS_TAKEN: 231a, 232, 241, 308, 368
STEP_PRNG: 222, 354, 357, 360, 377, 415, 456, 478, 527, 529, 595, 597, 663, 682, 898
STOB: 8c, 49, 219, 220, 222, 232, 239, 241, 308, 312, 320, 338, 346, 349, 354, 357, 368, 370a, 377, 378, 385, 390, 398, 404, 419, 420, 422, 428, 429, 430, 431, 452, 462, 473b, 487, 494, 527, 532c, 537, 538, 542, 545, 546, 550, 552, 564, 566, 568, 570b, 574, 576b, 590, 592, 617, 626, 627, 628, 632, 637, 639,

644, 653, 678, 679, 681, 738, 745, 747a, 748, 749, 751, 753, 754, 756, 757, 759, 763, 801, 816, 833, 854b, 856, 883, 898
STORE_SELECTION_TO_RECORD: 54b, 271, 273, 277, 283
STOW: 8a, 26, 219, 338, 375, 404, 419, 447, 459, 466, 527, 535, 550, 556, 570b, 592, 632, 681, 736, 801, 853a, 853b
STOW2: 8b, 259, 537
STYLE_ID_MASK: 533a, 535
STYLE_INDEX_MASK: 530a, 531a
STYLE_SCROLL_MASK: 541a, 542
SUBB: 14a
SUBB2: 14b
SUBW: 15a, 632
SUBWL: 15b
SUB_DROP: 438, 440, 442
SUM_HOSTILE_AT_POS: 320, 322, 357, 898
S_CHAIN_MAIL: 62, 402
S_CHEST_OPENS: 310, 447
S_CONTINUE_ONES: 275a, 277
S_CONTINUE_TENS: 275a, 277
S_COPYRIGHT: 570a, 574
S_DANGER_OUT: 685, 686
S_DECOR_ROW1: 572b, 574
S_DECOR_ROW2: 572b, 574
S_DECOR_ROW3: 572b, 574
S_IRON_SWORD: 62, 402
S_LEATHER_ARMOR: 62, 402
S_MISSES: 310, 338
S_NOW_HAVE_DIGITS: 447, 451a, 451b
S_NOW_HAVE_GOLD: 310, 451a
S_OAK_CLUB: 62, 402
S_PAY_FOR: 285, 310, 407a
S_PLATE_ARMOR: 62, 402
S_PRESS_BUTTON: 489b, 550
S_PRESS_SPACE: 550, 572c
S_PRESS_SPACE_BOOT: 572b, 574
S_REVEAL_DIGITS: 447, 451a, 451b
S_REVEAL_GOLD: 310, 451a
S_RITHMIL_CHAIN: 62, 402
S_RITHMIL_PLATE: 62, 402
S_SAVE_SLOT_ONES: 275a, 277
S_SAVE_SLOT_TENS: 275a, 277
S_SHIV: 62, 402
S_SOUND_CHAR1: 685, 686
S_SOUND_CHAR2: 685, 686
S_STEEL_DAGGER: 62, 402
S_STEEL_SWORD: 62, 402

TARGET_BIT6_MASK: 336, 338
TARGET_BIT7_MASK: 325, 336, 338
TARGET_DEFENSE: 337b, 338
TARGET_PTR: 328, 366, 368, 370a
TARGET_REC: 312, 317, 320, 322, 328, 334, 338, 349, 357, 365, 385, 420, 430, 432, 478, 481, 489a, 494, 612, 613, 615, 616, 617, 618b, 618c, 619c, 620, 898
TEARDOWN_INPUT: 232, 237, 262, 265
TEXT_COL: 49, 253, 254, 259, 467, 550, 552, 572a, 574, 576b, 578, 584, 626, 627, 628, 629, 639, 893b
TEXT_LEFT_MARGIN: 643b, 644
TEXT_PAGE1: 696, 772
TEXT_RENDERER: 527, 550, 571c, 574, 644
TEXT_RIGHT_MARGIN: 643a, 644
TEXT_ROW: 49, 253, 254, 259, 467, 550, 552, 572a, 574, 576b, 578, 584, 626, 627, 628, 639, 893b
TEXT_STREAM_IDX: 643c, 644
TEXT_STREAM_PTR: 527, 550, 571c, 574, 644
TICK_ANIM_COUNTER: 52, 604b, 605b, 663
TICK_BORDER_ANIM: 584
TILE_BLINK_TAB: 530c, 531a
TILE_DEFAULT_TAB: 530c, 531a
TILE_FILL_TAB: 530c, 531a
TIMED_WAIT: 354, 532c, 545, 605b, 606a
TOGGLE_SOUND: 686
TONE_PADDING: 647b, 648
TOTAL_MOB_COUNT: 219, 385, 417a, 419, 422, 478, 591b, 592
TRAP_TYPE: 446, 447
TRIGGER_SCENE_EVENT: 232, 444
TUNE_0: 654c
TUNE_10_ENCOUNTER: 660a
TUNE_11_MOVEMENT: 661c
TUNE_1_COMBAT_HIT: 655a
TUNE_2_COMBAT_DAMAGE: 655b
TUNE_3_ATTRACT: 656
TUNE_4_MOB_DEATH: 659a
TUNE_5_NPC_DEATH: 659a
TUNE_6_COMBAT_AVOIDED: 659b
TUNE_7_ENTITY_INTERACTION: 660b
TUNE_8_AI_TURN: 661a
TUNE_9_MENU_SELECTION: 661b
TUNE_ACTION_INDIRECT: 682, 684a
TUNE_ACTION_VEC: 652d, 681, 682, 684a
TUNE_ADVANCE: 570b, 584, 679, 681, 683
TUNE_ATTRACT: 526a, 527, 644
TUNE_BYTECODES_COPY: 67a
TUNE_END_MARKER: 661c, 662

TUNE_GOSUB_ADDR: 652b, 681
TUNE_INC_PC: 681, 682, 683, 684b
TUNE_LOOP_ADDR: 652a, 679, 681
TUNE_LOOP_CTR: 651b, 678, 679
TUNE_PC: 653, 654a, 654b, 678, 679, 681, 682, 683, 684b
TUNE_PLAYER: 232, 239, 247, 342, 346, 349, 354, 360, 371, 527, 532c, 570b, 644, 653, 682
TUNE_PRNG_MASK: 652c, 678, 681, 682
TUNE_TABLE: 651a, 653, 654b
TUNE_TABLE_COPY: 66c
TURN_ACTIVE: 232, 241, 264, 570b
TURN_DISPATCH: 237, 241, 262
TURN_START_COL: 230a, 232, 239, 354, 357, 360, 898
TURN_START_ROW: 230b, 232, 239, 354, 357, 360, 898
TURN_TUNE_GUARD: 238, 239
TXTCLR: 571b, 837
UI_TEXT_STREAM: 573, 574
UNDER_LEVEL: 261, 264, 308, 570b
UPDATE_EVENT_LIST: 349, 440, 454
USE_SCENARIO_DISK: 280a, 295
VALIDATE_INPUT: 247, 255
VERIFY_DATA_FIELD: 773, 775, 779a, 793, 822
VERIFY_DISK_IO: 222, 671
VERIFY_DISK_SIGNATURE: 289, 295, 298
VERIFY_GAME_DISK: 295, 297, 301
VERIFY_INTEGRITY: 663, 671
VIEW_MAX_COL: 533b, 535, 537, 542
VIEW_MAX_ROW: 533c, 535, 537, 542
VIEW_MIN_COL: 534a, 535, 537, 542
VIEW_MIN_ROW: 534b, 535, 537, 542
VM_ACC: 695, 700, 707, 709b, 709c, 710a, 710b, 712a, 712b, 713a
VM_CHECKSUM_A200: 839, 842
VM_CHECKSUM_LOOP: 839, 842
VM_COPY_LOOP: 839, 842
VM_DO_GROUP3: 839, 842
VM_DXR_LOOP: 839, 842
VM_ERROR_BEEP: 842
VM_ERROR_DISPLAY: 842, 859
VM_ERROR_HALT: 842
VM_ERROR_SPLASH: 839, 842
VM_EXIT_ERR: 696
VM_EXIT_OK: 221, 696, 700
VM_IP: 695, 700, 702, 704, 706, 708, 709c, 711a, 711b, 712b
VM_LOAD_GROUP1: 837
VM_LOAD_GROUP2: 837
VM_LOAD_GROUP3: 837, 839

VM_OPERAND: 695, 700, 702, 704, 706, 707, 708, 709a, 710a, 710b, 711a, 712a, 712b, 713a
VM_READ_DONE: 842
VM_READ_LOOP: 841, 842
VM_READ_NEXT: 842
VM_READ_TRACKS: 837, 839, 841
VM_SCRATCH: 695, 696, 700
VM_SPLASH_DELAY: 837
WAIT_DURATION: 606c, 607a
WAIT_KEYPRESS: 289, 297
WAIT_LOOP_COUNT: 51, 52, 354, 532c, 544a, 545, 604b, 606a, 683, 898
WAIT_LOOP_INIT: 353b, 354
WIPE_END_PAGE: 674b, 675
WIPE_TAIL_LEN: 674b, 675
WORLD_INIT_DATA: 274, 588a
WRITE_DISK_SIGNATURE: 295, 298
WRITE_GAME_DATA: 293, 295, 303
XP_ACCUM: 411b, 412
XP_AWARD_0: 402, 412
XP_AWARD_1: 402, 412
XP_AWARD_2: 402, 412
XP_AWARD_3: 402, 412
XP_AWARD_4: 402, 412
XP_AWARD_TABLE: 62, 412
s_ABDALLA: 127, 197
s_ABD_US_SAMAD: 100, 197
s_ABOU_HASSAN: 73, 197
s_ABSAL: 93, 197
s_ABUL_FATH: 89, 197
s_AHMED: 122, 197
s_ALADDIN: 184, 197
s_ALI: 103, 197
s_ALI_BABA: 69, 197
s_ALP_ARSLAN: 96, 197
s_ARIES_THE_RAM: 146, 197
s_ATTAR: 109, 197
s_BABY_OWL: 179, 197
s_BAHRAM: 117, 197
s_BAIRAM_MEDIO: 72, 197
s_BANE_OF_THIEVES: 183, 197
s_BARN_OWL: 178, 197
s_BIG_ZOMBIE: 138, 197
s_BLACK_BEAR: 186, 197
s_BLACK_RAT: 152, 197
s_BROWN_BEAR: 187, 197
s_BROWN_RAT: 151, 197

s_BULL: 196, 197
s_CALICO_RAT: 139, 197
s_CANCER_THE_CRAB: 147, 197
s_CAVEBEAR: 149, 197
s_CAVE_RAT: 150, 197
s_CELEGORM: 76, 197
s_CINDER: 81, 197
s_CODADAD: 125, 197
s_COGIA_HOUSSAIN: 86, 197
s_CURUFIN: 77, 197
s_DAUNTLESS_MINNIE: 195, 197
s_DINARZADE: 118, 197
s_DOCTOR_WHO: 182, 197
s_DOUBAN: 120, 197
s_EARTH_WANDERER: 169, 197
s_ELDO: 78, 197
s_ENCHANTED_STATUE: 128, 197
s_ENCHANTED_SWORD: 166, 197
s_EQUUS: 191, 197
s_FEROZE: 121, 197
s_GANADRINIL: 180, 197
s_GEMINI_I: 153, 197
s_GEMINI_II: 154, 197
s_GIANT_SCORPION: 160, 197
s_GRAY_RAT: 130, 197
s_GREEN_DRAGON: 163, 197
s_GRIZZLY_BEAR: 162, 197
s_HAFIZ: 106, 197
s_HAKIM: 101, 197
s_HAROUD_EL_LARGE: 71, 197
s_HAROUN_AL_RASCHID: 124, 197
s_HASAN_BEN_SABBAH: 102, 197
s_HATIM_TAI: 116, 197
s_HOOT_OWL: 140, 197
s_HUAN: 75, 197
s_IBRAHIM: 90, 197
s_IRAM: 112, 197
s_JALAL_UD_DIN: 91, 197
s_JAMI: 94, 197
s_JAMSHYD: 111, 197
s_JATTE: 82, 197
s_JELALUDDIN: 107, 197
s_JUNGLE_RAT: 159, 197
s_KAIKHOSRU: 114, 197
s_KAIKOBAD: 113, 197
s_KHORASSAN: 104, 197

s_KHWAJAH_NIZAMI: 105, 197
s_KWADRACH: 108, 197
s_LISSOM: 181, 197
s_LOST_BARN_OWL: 171, 197
s_LUTHIEN: 74, 197
s_MAHMUD: 88, 197
s_MALIK: 97, 197
s_MECHANICAL_SWORD: 164, 197
s_MERRYWATER: 79, 197
s_MINOTAUR: 177, 197
s_MORGIANA: 126, 197
s_MOUNTAIN_SCORPION: 141, 197
s_MOWAFFAK: 99, 197
s_NELL: 190, 197
s_NIZAM_UL_MULK: 95, 197
s_OG: 84, 197
s_OMAR: 87, 197
s_OOZING_STENCHBEAST: 175, 197
s_PEHLEVI: 110, 197
s_PERIE_BANOU: 123, 197
s_PRINCESS_BUDDIR: 132, 197
s_PRINT_FONT_CHAR: 465, 466
s_RING_EYED_OWL: 185, 197
s_RUSTUM: 115, 197
s_SALAMAN: 92, 197
s_SCHEHERAZADE: 70, 197
s_SEWER_RAT: 158, 197
s_SHAHZENAN: 119, 197
s_SKY_WANDERER: 170, 197
s_SLIME: 172, 197
s_SMALL_ZOMBIE: 137, 197
s_SONNY_BOY_MINOTAUR: 194, 197
s_SON_OF_MINOTAUR: 176, 197
s_STATUE_OF_MINOTAUR: 174, 197
s_STATUE_OF_TAURUS: 145, 197
s_STILBO: 80, 197
s_SUCCUBUS: 173, 197
s_SULTANS_LEFT_GUARD: 134, 197
s_SULTANS_RIGHT_GUARD: 133, 197
s_SULTAN_SHAHRIAR: 131, 197
s_SUN_WANDERER: 168, 197
s_SWORD_OF_DAMOCLES: 155, 197
s_TAN_RAT: 129, 197
s_THE_BLOB: 167, 197
s_THORA: 83, 197
s_TIGER: 135, 197

s_TOGHRUL_BEG: 98, 197
s_TROG: 189, 197
s_TWO_EDGED_SWORD: 156, 197
s_ULVA: 85, 197
s_UNICORN: 142, 197
s_VERY_YOUNG_DRAGON: 165, 197
s_WAUKA: 188, 197
s_WEREJACKAL: 148, 197
s_WERERAT: 157, 197
s_WEREWOLF: 161, 197
s_WHITE_DRAGON: 193, 197
s_WHITE_OWL: 192, 197
s_WITLESS_ZOMBIE: 136, 197
sm_LEO: 143, 197
sm_SCORPIUS: 144, 197